

Contents

第四册简介：大模型百科全书	61
本书定位	61
为什么需要这本书	61
适合读者	61
学完后应具备的能力	61
内容结构	62
阅读建议	62
与其他书的关系	62
第四册：大模型百科全书	62
本书定位	62
阅读入口	62
章节文件	62
A. 基础数学与机器学习	63
线性代数 Linear Algebra	63
向量 Vector	63
内积 Dot Product	63
向量范数 Vector Norm	63
矩阵乘法 Matrix Multiplication	64
投影 Projection	64
特征值与特征向量 Eigenvalue and Eigenvector	64
矩阵秩 Matrix Rank	64
奇异值分解 Singular Value Decomposition / SVD	64
低秩近似 Low-Rank Approximation	64
主成分分析 Principal Component Analysis / PCA	65
低秩压缩 Low-Rank Compression	65
LoRA Low-Rank Adaptation	65
QLoRA Quantized LoRA	65
概率论 Probability Theory	65
随机变量 Random Variable	65
概率分布 Probability Distribution	65
条件概率 Conditional Probability	66
概率链式法则 Chain Rule of Probability	66
期望 Expectation	66
方差 Variance	66
信息论 Information Theory	66
交叉熵 Cross Entropy	66
最大似然估计 Maximum Likelihood Estimation / MLE	67
负对数似然 Negative Log-Likelihood / NLL	67
KL 散度 KL Divergence	67
困惑度 Perplexity	67
梯度 Gradient	67
梯度下降 Gradient Descent	67
优化基础 Optimization Basics	68
SGD Stochastic Gradient Descent	68
Momentum	68
Adam	68
AdamW	68
学习率调度 Learning Rate Schedule	68
反向传播 Backpropagation	68
链式法则 Chain Rule	69
梯度消失 Vanishing Gradient	69
梯度爆炸 Exploding Gradient	69

Gradient Clipping	69
Hessian	69
Loss Spike	69
熵 Entropy	69
条件熵 Conditional Entropy	70
互信息 Mutual Information	70
JS 散度 Jensen-Shannon Divergence	70
最大后验估计 Maximum A Posteriori / MAP	70
贝叶斯公式 Bayes' Theorem	70
先验 Prior	71
似然 Likelihood	71
后验 Posterior	71
贝叶斯更新 Bayesian Update	71
不确定性 Uncertainty	71
Aleatoric Uncertainty	71
Epistemic Uncertainty	71
Bias-Variance Trade-off	72
过拟合 Overfitting	72
欠拟合 Underfitting	72
泛化 Generalization	72
正则化 Regularization	73
L1 正则 Lasso Regularization	73
L2 正则 Ridge Regularization	73
Dropout	73
Early Stopping	73
校准 Calibration	74
Expected Calibration Error / ECE	74
Brier Score	74
选择性预测 Selective Prediction / Abstention	74
置信度 Confidence	74
ROC-AUC	74
Precision / Recall / F1	75
Accuracy	75
Label Smoothing	75
数据泄漏 Data Leakage	75
数据分布偏移 Distribution Shift	75
损失函数 Loss Function	76
经验风险最小化 Empirical Risk Minimization / ERM	76
优化器 Optimizer	76
学习率 Learning Rate	76
Warmup	76
Batch Size	77
随机种子 Random Seed	77
超参数 Hyperparameter	77
训练集 / 验证集 / 测试集	77
泛化审计 Generalization Audit	77
强化学习数学 Reinforcement Learning Math	78
Markov Decision Process / MDP	78
Return	78
Value Function	78
Q Function	78
Advantage	78
Policy Gradient	78
PPO Clipped Objective	79
RLHF KL Penalty	79

Reward Model Pairwise Loss	79
DPO Loss	79
Evaluation Statistics	79
Standard Error	79
Confidence Interval	79
p-value	80
Bootstrap	80
Paired Evaluation	80
McNemar Test	80
Sample Size	80
Statistical Power	80
Multiple Comparisons	80
Bonferroni Correction	80
Benjamini-Hochberg	81
Experiment Gate	81
Math Interview Readiness	81
Math Interview Rubric	81
Formula Coverage	81
Formula Audit	81
Mock Interview Retrospective	81
B. 深度学习组件	81
MLP Multi-Layer Perceptron	82
Feed-Forward Network / FFN	82
Linear Layer	82
Activation Function	82
ReLU Rectified Linear Unit	83
GeLU Gaussian Error Linear Unit	83
SiLU / Swish	83
GLU Gated Linear Unit	83
SwiGLU	84
Dropout	84
BatchNorm Batch Normalization	84
LayerNorm Layer Normalization	84
RMSNorm Root Mean Square Layer Normalization	85
Pre-LN vs Post-LN	85
Residual Connection	85
Skip Connection	85
Optimizer	86
SGD Stochastic Gradient Descent	86
Momentum	86
Adam Adaptive Moment Estimation	86
AdamW	87
Weight Decay	87
Learning Rate	87
Learning Rate Warmup	87
Cosine Decay	88
Gradient Clipping	88
Initialization	88
Xavier Initialization	89
Kaiming Initialization / He Initialization	89
Parameter Count	89
Activation Memory	89
Mixed Precision Training	89
FP16	90
BF16	90

Autocast	90
GradScaler	90
Loss Scaling	90
Activation Checkpointing	90
CUDA Memory Stats	90
OOM Audit	90
PyTorch Tensor	91
Tensor Shape	91
Dtype	91
Device	91
Broadcasting	91
Tensor Stride	91
Contiguous Tensor	91
View vs Reshape	91
Tensor Mask	92
Tensor Shape Audit	92
PyTorch Autograd	92
Dynamic Computation Graph	92
Requires Grad	92
Leaf Tensor	92
Grad Accumulation	92
Zero Grad	92
Detach	93
No Grad	93
Inference Mode	93
In-place Operation	93
Autograd Hook	93
Autograd Audit	93
nn.Module	93
nn.Parameter	93
Module Registration	94
ModuleList	94
ParameterList	94
Buffer	94
state_dict	94
load_state_dict	94
train / eval Mode	94
Module Hook	95
Module Audit	95
Dataset / DataLoader	95
Map-style Dataset	95
IterableDataset	95
Collate Function	95
Data Sampler	95
DistributedSampler	96
DataLoader Worker	96
Pin Memory	96
Data Pipeline Audit	96
Training Loop	96
Training Step	96
Gradient Accumulation	96
Optimizer Step	97
Scheduler Step	97
Evaluation Loop	97
Checkpoint Resume	97

Non-Finite Loss	97
Training Loop Audit	97
Distributed Training	97
Rank / World Size	97
Process Group	98
DistributedDataParallel / DDP	98
All-Reduce	98
torchrun	98
Global Batch Size	98
DDP no_sync	98
Distributed Training Audit	98
Debug Tensor Metadata	98
Finite Check	99
Gradient Debug	99
Forward Hook Debug	99
Anomaly Detection	99
CUDA Synchronize Timing	99
torch.profiler	99
DataLoader Bottleneck	99
Debug / Profiling Audit	99
Transformer Components	100
Token Embedding	100
LM Head Weight Tying	100
Causal Mask	100
Scaled Dot-Product Attention	100
Multi-Head Self-Attention	100
KV Cache	100
Transformer Component Audit	101
PyTorch Engineering Interview Readiness	101
本章小结	101
C. NLP 与 Tokenization	101
Token	101
Vocabulary	102
Token ID	102
Tokenization	102
Tokenizer	103
Subword Tokenization	103
BPE Byte Pair Encoding	103
WordPiece	103
Unigram Language Model Tokenization	104
SentencePiece	104
Byte-Level Tokenization	104
Character-Level Tokenization	104
OOV Out-of-Vocabulary	105
Special Token	105
BOS Beginning of Sequence	105
EOS End of Sequence	105
PAD Padding Token	105
UNK Unknown Token	106
Chat Template	106
Prompt	106
Completion	106
Instruction	107
System Prompt	107
Context Window	107

Detokenization	107
Tokenizer Training	108
中文 Tokenization	108
Code Tokenization	108
Tokenizer 与 Embedding 扩展	108
Tokenizer Mismatch	109
本章小结	109
D. Transformer 架构	109
Transformer	109
Token Embedding	110
Embedding Matrix	110
Position Embedding	110
Positional Encoding	111
Sinusoidal Positional Encoding	111
RoPE / Rotary Position Embedding	111
RoPE Scaling	112
ALiBi	112
Long Context	112
Position Extrapolation	112
Lost in the Middle	113
Weight Tying	113
Contextual Representation	113
Self-Attention	113
Query	114
Key	114
Value	114
Attention Weight	114
Causal Self-Attention	114
Causal Mask	115
Padding Mask	115
Causal LM	115
Masked LM	115
Scaled Dot-Product Attention	116
Full Attention	116
Local Attention / Sliding Window Attention	116
Sparse Attention	116
Linear Attention	116
FlashAttention	117
Attention Complexity	117
Multi-Head Attention / MHA	117
Attention Head	117
Multi-Query Attention / MQA	117
Grouped-Query Attention / GQA	118
KV Cache	118
Transformer Block	118
MLP / FFN	119
Residual Connection	119
Pre-LN	119
Post-LN	119
Decoder-Only Transformer	120
Encoder-Only Transformer	120
Encoder-Decoder Transformer	120
Cross-Attention	120
本章小结	121

E. 大语言模型基础	121
Language Model	122
Large Language Model / LLM	122
Next-Token Prediction	122
Autoregressive Model	123
Teacher Forcing	123
Label Shift	123
Assistant-Only Label Mask	124
Prompt Loss Leak	124
PAD Loss Leak	124
Logits	124
Softmax	125
Causal Language Model	125
Base Model	125
Instruct Model	126
Chat Model	126
Tokenizer Format Incident	126
Chat Template Drift	126
EOS Coverage	127
Assistant Model	127
Prompt	127
System Prompt	127
Instruction Following	127
Context Window	128
Generation	128
Perplexity	128
Emergent Ability	129
In-Context Learning	129
Label Space Coverage	129
Demonstration Format Consistency	129
Context Evidence Use	130
Middle Evidence Risk	130
Token Retrieval Audit	130
Retrieval Label Match	130
ICL Conflict Rate	130
ICL Gate	130
Transformer Backbone	130
Architecture Selection Gate	131
Route Score	131
Training Parallelism	131
Parallel Path Length	131
Content Routing	131
Attention Routing Audit	131
Route Hit Rate	131
Future Leak Rate	131
Softmax Row Sum Check	131
Scale Stability	131
Interpretation Boundary	132
QK Circuit	132
OV Circuit	132
Head Subspace Audit	132
Head Redundancy Rate	132
KV Head Ratio	132
Multi-head Latent Attention	132
Latent KV Cache	132

KV Cache Saving Rate	133
Attention Architecture Gate	133
Position-wise FFN	133
SwiGLU	133
Gated MLP	133
MoE FFN	133
Top-k Routing	133
Expert Capacity	133
Active Parameters	134
FFN / MoE Gate	134
Residual Stream	134
Norm Placement	134
Pre-LN	134
Post-LN	134
RMSNorm	134
Residual Scaling	135
Early Update Pressure	135
Normalization Stability Gate	135
Position Encoding	135
Sinusoidal Position Encoding	135
Learned Absolute Position Embedding	135
Relative Position Representation	135
Relative Position Bias	136
Rotary Position Embedding / RoPE	136
ALiBi	136
Position Interpolation	136
RoPE Scaling	136
Position ID Audit	136
Position Encoding Gate	136
Attention Mask	137
Loss Mask	137
Bidirectional Attention	137
Prefix LM	137
Encoder-Decoder Attention	137
Cross-Attention	137
Packed Sequence Mask	137
Block-Diagonal Causal Mask	137
Sliding Window Attention	138
Mask Visibility Audit	138
Mask Gate	138
Scaling Evidence	138
Attention Pair Count	138
Quadratic Attention Cost	138
Attention Score Memory	138
Prefill Attention Cost	138
Decode Attention Cost	139
Attention Cost Gate	139
KV Cache Budget	139
KV Cache Per Token	139
Decode Bandwidth Pressure	139
Paged KV Block	139
KV Cache Gate	139
Training FLOPs Estimate	139
Compute-Optimal Training	140
Tokens per Parameter	140

Undertrained Model	140
Serving Pressure	140
Scaling Gate	140
Ecosystem Readiness	140
Deployment Pressure	140
Attention-SSM Hybrid Readiness	140
Few-Shot Learning	141
Zero-Shot Learning	141
Chain-of-Thought	141
Memorization vs Generalization	141
Hallucination	142
LLM Productization	142
Demo to Product	142
Scenario Selection	142
Real Demand	142
User Persona	143
Stakeholder Map	143
Task Journey	143
Product Metric	143
Model-to-Experience Mapping	143
LLM Product Experience	143
Experience Metric	143
Structured Output Stability	144
Citation Support Rate	144
Controllability	144
Error Recovery Rate	144
Trust Calibration	144
Experience Gate	144
Frequency-Value Matrix	144
Data Readiness	144
Verifiability	145
LLM Fit	145
Automation Level	145
Pilot Scenario	145
Scenario Selection Gate	145
Unit Economics	145
Total Cost of Ownership	145
Variable Cost	145
Fixed Cost	146
Model Cost	146
Review Cost	146
ROI	146
Net Benefit	146
Break-Even Point	146
Sensitivity Analysis	146
Payback Period	146
ROI Gate	146
Enterprise LLM Application	147
Enterprise Knowledge Base	147
Single Sign-On / SSO	147
Identity and Access Management / IAM	147
Role-Based Access Control / RBAC	147
Tenant Isolation	147
Permission-Aware RAG	147
Tool Permission Gate	147

Audit Log Coverage	148
Data Freshness	148
Workflow Integration	148
Enterprise Gate	148
Productization Gate	148
Feedback Loop	148
Online Monitoring	148
Canary Rollout	148
Regression Pass Rate	149
Trace Coverage	149
Rollback Plan	149
Ops Gate	149
Project Collaboration Incident	149
Objective Clarity Rate	149
Baseline Coverage	149
Version Trace Coverage	150
RACI Coverage	150
Change Control Coverage	150
Project Collaboration Gate	150
Product Interview Readiness	150
Product Interview Gate	150
Practitioner Interview Readiness	150
Project Evidence Score	151
Trade-off Depth	151
STAR Reflection Score	151
Expert Follow-up Readiness	151
Practitioner Interview Gate	151
本章小结	151

F. 预训练与数据	152
Pretraining	152
Base Model	152
Corpus	153
Web Data	153
Web-Scale Data Collection	153
Specialized Data	153
Code Data	154
Unit Test Data	154
Functional Correctness	154
Math Data	154
Verifier Data	155
Multilingual Data	155
Domain Data	155
Domain Authority	155
Expert Audit	155
Data Quality	156
Quality Score	156
Quality Classifier	156
Data Cleaning	156
Data Filtering Audit	157
Data Incident	157
Data Risk Rate	157
Data Incident Gate	157
Deduplication	157
Exact Deduplication	158
Near Deduplication	158

MinHash	158
SimHash	158
Train-Eval Overlap	159
Canary	159
PII Filtering	159
Secret Scanning	159
Toxicity Filtering	159
Data Mixture	160
Mixture Shift	160
Data Sampling	160
Temperature Sampling	160
Effective Epoch	161
Mixture Ablation	161
Curriculum Learning	161
Synthetic Data	161
Synthetic Instruction Data	162
Self-Instruct	162
Evol-Instruct	162
Distillation Data	162
Teacher Model	162
Student Model	163
Model Collapse	163
Preference Data	163
Chosen Response	163
Rejected Response	164
Reward Model Data	164
Safety Data	164
Over-Refusal	164
Unsafe Leak	164
Red Team Data	165
Multimodal Data	165
Image-Text Pair	165
Image-Text Alignment	165
OCR Data	165
Caption Data	165
Audio-Transcript Pair	166
ASR Data	166
Video-Text Data	166
Temporal Alignment	166
Grounded Multimodal QA	166
Visual Privacy	166
Data Attribution	167
Data Valuation	167
Influence Function	167
Data Shapley	167
Gradient Similarity	167
Source-Level Ablation	167
Proxy Model	168
Negative Value Data	168
Active Learning	168
Data Selection	168
Dataset Versioning	168
Data Governance	168
Data Lineage	169
Lineage Coverage	169

Dataset Manifest	169
Immutable Snapshot	169
Schema Evolution	169
Deletion Request	169
Dataset Datasheet	170
Dataset Card	170
Model Card	170
Benchmark Contamination	170
Data Interview Readiness	170
Data Packing	171
Document Boundary	171
Sequence Length	171
Checkpoint	171
Training Token	172
Epoch	172
Scaling Law	172
Chinchilla Scaling	172
Compute-Optimal Training	173
本章小结	173
G. 后训练与对齐	173
Post-Training	174
Instruction Tuning	174
SFT	174
Full Fine-Tuning	175
LoRA	175
QLoRA	175
Catastrophic Forgetting	175
Chat Template	176
Loss Mask	176
System Prompt	176
Synthetic Instruction Data	176
Preference Data	177
Reward Model	177
Pairwise Ranking Loss	177
RLHF	177
Policy Model	178
Reference Model	178
PPO	178
KL Penalty	178
Reward Hacking	178
Reward-Human Gap	179
Best-of-N Overoptimization	179
Reward Hacking Gate	179
Proxy Objective	179
Specification Gaming	179
Over-Optimization	180
Goodhart's Law	180
Distribution Shift	180
Pairwise Accuracy	180
Reward Calibration	181
DPO	181
DPO Loss	181
Beta in DPO	181
IPO	181
KTO	182

ORPO	182
RLAIF	182
AI Feedback	182
Constitutional AI	182
Length Bias	183
Over-Refusal	183
Post-Training Incident	183
Assistant Mask Coverage	183
Capability Regression	183
False Refusal Rate	184
Unsafe Leak Rate	184
Preference Margin	184
Reward Length Bias	184
DPO Reference Mismatch	184
Tool Schema Drift	184
Alignment Training Gate	185
Safety Alignment	185
本章小结	185
H. 推理与解码	185
Inference	185
Autoregressive Generation	186
Decoding	186
Greedy Decoding	186
Sampling	187
Temperature	187
Top-k Sampling	187
Top-p Sampling	187
Beam Search	188
Repetition Penalty	188
Presence Penalty	188
Frequency Penalty	188
Stop Token	188
EOS Token	189
Max New Tokens	189
Best-of-N	189
Self-Consistency	189
Log Probability	189
Token Probability	190
KV Cache	190
Prefill	190
Decode	190
Time to First Token / TTFT	191
Time per Output Token / TPOT	191
Speculative Decoding	191
本章小结	191
I. 推理优化与部署	192
LLM Serving	197
LLM Serving Engine Overview Audit	197
From Scratch Mini Serving Engine	197
Request Lifecycle Audit	198
Prefill Decode KV Streaming Audit	198
Serving Metric Cost Audit	198
Serving Boundary Audit	199
Minimal Generate Loop Audit	199

Sampling Strategy Audit	199
Minimal KV Cache Audit	199
Batched Prefill Audit	200
Batched Decode Audit	200
Simple Scheduler Audit	200
Token Streaming Audit	201
Minimal HTTP API Audit	201
Serving Benchmark Audit	201
vLLM Motivation Audit	202
PagedAttention Core Audit	202
KV Block Manager Audit	202
Continuous Batching Audit	203
vLLM Scheduler Flow Audit	203
vLLM Memory Management Audit	204
vLLM Executor Architecture Audit	204
Prefix Caching Audit	204
Serving Parallelism Audit	205
vLLM Performance Tuning Audit	205
SGLang Motivation Audit	205
SGLang Runtime Audit	206
RadixAttention Prefix Sharing Audit	206
SGLang Scheduler Audit	206
Structured Generation Audit	207
Speculative Decoding Audit	207
Agent Serving Audit	207
Serving Architecture Comparison Audit	208
mini-sglang Source Path Audit	208
Prefill Decode Resource Profile Audit	209
PD Disaggregation Motivation Audit	209
PD Architecture Audit	209
KV Transfer Routing Audit	210
Chunked Disaggregated Prefill Audit	210
Multi Level KV Cache Audit	211
Cross Node Network Audit	211
PD Tradeoff Audit	211
Single Engine To PD Migration Audit	212
nano-vLLM Source Learning Audit	212
tiny-LLM Learning Audit	212
mini-sglang Learning Audit	213
Teaching Project Core Module Audit	213
Naive Scheduler To Continuous Batching Audit	214
Paged KV Cache Upgrade Audit	214
Prefix Prompt Cache Upgrade Audit	215
Preemption Recompute Swap Upgrade Audit	215
Unified Scheduler Loop Audit	215
Serving Benchmark Framework Audit	216
Async Serving Architecture Audit	216
Multi Worker Router Audit	217
Distributed Parallel KV Audit	217
OpenAI Compatible API Audit	218
Production Deployment Audit	218
Capacity SLO Fault Drill Audit	218
Inference Engine Interview Readiness Audit	219
Throughput	219
Latency	220

TTFT	220
TPOT	220
QPS	220
Batching	220
Dynamic Batching	221
Continuous Batching	221
Request Scheduling	221
KV Cache Management	221
PagedAttention	222
Prompt Cache	222
Prefix Cache	222
FlashAttention	222
IO-Aware Optimization	222
Tiling / Blocking	223
Online Softmax	223
HBM	223
SRAM	223
Fused Kernel	223
Memory Bandwidth Bottleneck	224
Quantization	224
FP32	224
FP16	224
BF16	224
FP8	224
INT8	225
INT4	225
Weight-Only Quantization	225
Post-Training Quantization / PTQ	225
Quantization-Aware Training / QAT	225
GPTQ	226
AWQ	226
Calibration Data	226
Distillation	226
TensorRT-LLM	226
vLLM	227
Triton Inference Server	227
Model Parallel Inference	227
CPU Offloading	227
Inference Performance Incident	227
TTFT Regression	228
TPOT Regression	228
KV Pressure	228
Prompt Cost Drift	228
Cache Effectiveness	228
Serving Gate	229
Inference Platform Overview Audit	229
Request Lifecycle Coverage	230
SLO Metric Contract	230
Model Registry Readiness	230
Model Router Policy Fit	230
Runtime Prefill Decode Fit	230
Continuous Batching Readiness	230
KV Cache Capacity Governance	230
Autoscaling Warm Pool Fit	230
Release Rollback Control	230

Rate Limit Degrade Protection	231
Safety Governance Fit	231
Cost Attribution Optimization	231
Observability Trace Coverage	231
Multi Model Governance	231
Developer API Self Service	231
Inference Platform Gate	231
Model Serving Runtime Selection Audit	231
Model Hardware Fit	232
Runtime Boundary Clarity	232
Model Format Tokenizer Fit	232
Prefill Decode Scheduler Fit	232
Continuous Batching Fit	232
KV Cache Memory Manager Fit	233
Streaming API Fit	233
Quantization Accuracy Fit	233
Distributed Inference Fit	233
Observability Metrics Fit	233
Benchmarking Method Fit	233
Deployment Rollback Fit	233
Ecosystem Maintenance Fit	233
Cost Capacity Fit	233
Self Development Threshold	233
Runtime Selection Gate	234
Prefill Decode KV Resource Profile Audit	234
Request Token Profile	235
Prefill Phase Accounting	235
Decode Phase Accounting	235
TTFT TPOT Contract	235
KV Cache Formula Fit	235
KV Capacity Admission	235
Paged Block Management	235
Prefix Cache Reuse	235
Long Context Policy	235
Tenant KV Isolation	235
Continuous Batching Policy	235
PD Disaggregation Fit	236
Streaming Backpressure Fit	236
Observability Phase Metrics	236
Cost Capacity Model	236
Resource Profile Gate	236
Inference Scheduling Audit	236
Request Arrival Profile	236
Continuous Batching Core	237
Token Budget Policy	237
Prefill Decode Balance	237
Paged KV Block Management	237
Scheduler KV Admission	237
Long Short Isolation	237
Tenant Priority Fairness	237
Cancellation Cleanup	237
Scheduler Observability Metrics	237
Scheduler Gate	237
Model Routing Audit	238
Request Intent Profile	238

Model Capability Profile	238
Permission Candidate Filter	238
Cost Budget Policy	238
Latency SLO Policy	238
Quality Safety Policy	238
Fallback Chain Governance	238
Canary Routing Stability	239
Realtime Health Load	239
Route Trace Coverage	239
Degrade Policy Control	239
Route Config Versioning	239
Model Routing Gate	239
Inference Cache System Audit	239
Request Cache Profile	239
Cache Layer Boundary	240
Key Version Fingerprint	240
Prompt Prefix Reuse	240
KV Lifecycle Management	240
Semantic Similarity Guard	240
Result Cache Determinism	240
Tenant Permission Isolation	240
TTL Staleness Control	240
Eviction Quota Policy	240
Streaming Cache Policy	240
Cache Observability Metrics	240
Cost Latency Savings	241
Cache Governance Gate	241
Inference Autoscaling Audit	241
Traffic Token Profile	241
SLO Latency Contract	241
Queue Backlog Signal	241
GPU KV Capacity Signal	241
Cold Start Lead Time	241
Warm Pool Readiness	242
Multi Metric Recommendation	242
Scale Up Down Policy	242
Draining Streaming Safety	242
Router Admission Coupling	242
Tenant Quota Priority	242
Cost Budget Guard	242
Autoscaling Trace Coverage	242
Autoscaling Gate	242
Inference Protection Policy Audit	242
Request Token Rate Limit	243
Concurrency Quota Control	243
Admission Capacity Guard	243
Circuit Breaker State Machine	243
Retry Error Classification	243
Retry Budget Bound	243
Idempotency Stage Guard	243
Timeout Budget Allocation	243
Degradation Policy Control	243
Priority Tenant Fairness	244
Streaming Cancellation Cleanup	244
Protection Trace Coverage	244

Protection Gate	244
Inference Release Governance Audit	244
Release Bundle Trace	244
Model Artifact Integrity	244
Tokenizer Prompt Runtime Binding	245
Offline Eval Quality Gate	245
Safety Eval Gate	245
Contract Test Compatibility	245
Stable Bucket Assignment	245
Canary Ramp Control	245
AB Experiment Design	245
Sample Ratio Health	245
Guardrail Metric Control	245
Rollback Readiness	245
Release Audit Trace	246
Post Release Monitoring	246
Release Governance Gate	246
Inference Platform System Design Audit	246
SLO Token Contract	246
KV Scheduler Capacity	246
Inference Platform Design Gate	246
Inference Fault Diagnosis Audit	246
Inference Fault Evidence Coverage	247
Request Scope Slice	247
Trace Stage Coverage	247
TTFT Decomposition	247
TPOT Decode Health	247
Tail Latency Attribution	247
Throughput Token Capacity	247
Error Taxonomy Timeout Stage	247
KV Cache Pressure Guard	248
Model Artifact Load Readiness	248
Streaming Reliability	248
Cache Hit Key Governance	248
Route Trace Correctness	248
Tool Dependency Trace	248
Quality Release Diff Readiness	248
Inference Fault Diagnosis Gate	248
部署排障常见问题	248
本章小结	249

J. 分布式训练系统	249
Distributed Training	254
Data Parallelism	254
DDP	254
Gradient Synchronization	255
All-Reduce	255
Reduce-Scatter	255
All-Gather	255
Tensor Parallelism	255
Column Parallel	256
Row Parallel	256
Pipeline Parallelism	256
Micro-batch	256
Pipeline Bubble	256
1F1B	257

ZeRO	257
ZeRO-1	257
ZeRO-2	257
ZeRO-3	257
FSDP	258
Activation Checkpointing	258
Gradient Accumulation	258
Mixed Precision Training	258
Loss Scaling	258
Training Stability	259
Loss Spike Audit	259
Non-Finite Loss Rate	259
Effective Label Tokens	259
LR Continuity	259
Rank Loss Skew	260
Training Stability Gate	260
Distributed Incident	260
Rank Step Alignment	260
Collective Mismatch	260
Token Shard Imbalance	260
Straggler Ratio	261
Communication Ratio	261
Checkpoint Shard Coverage	261
Resume Continuity	261
Pipeline Bubble Ratio	261
Distributed Incident Gate	261
Offloading	262
Parameter Sharding	262
Optimizer State Sharding	262
Communication Overlap	262
NCCL	262
Node	263
Rank	263
World Size	263
Global Batch Size	263
MFU	263
GPU Utilization	263
Training Efficiency Audit	264
Training Efficiency Gate	264
Task Profile Audit	264
Workload Type Classification	265
Resource Shape Completeness	265
Pretraining Profile Accuracy	265
SFT Iteration Profile Accuracy	265
RLHF Pipeline Profile Accuracy	265
Evaluation Reproducibility Profile	266
Serving SLO Profile	266
RAG Freshness Retrieval Profile	266
Agent Tool Runtime Profile	266
Multimodal Resource Profile	266
Scheduler Policy Fit	266
Observability Metric Fit	266
Cost Model Fit	266
Artifact Lineage Fit	267
Safety Governance Fit	267

Task Profile Gate	267
GPU Cluster Topology Audit	267
Scale-Up Domain Fit	267
PCIe NUMA Locality	268
NVLink NVSwitch Locality	268
GPU NIC Affinity	268
Inter-Node Fabric Readiness	268
Rack Locality Awareness	268
Oversubscription Awareness	268
Collective Communication Fit	268
Parallel Group Placement	268
Storage Checkpoint Locality	268
Fault Domain Isolation	269
Power Cooling Capacity Fit	269
Resource Pool Isolation	269
Topology-Aware Scheduling	269
Observability Topology Coverage	269
GPU Cluster Gate	269
Network Communication Audit	269
Bandwidth Unit Accounting	270
Latency Jitter Tracking	270
RDMA Capability Fit	270
GPUDirect RDMA Path	270
InfiniBand Fabric Readiness	270
RoCE Congestion Losslessness	270
Ethernet Fallback Scope	270
Collective Operation Modeling	270
NCCL Topology Runtime Fit	270
AllReduce Cost Estimation	271
AllGather ReduceScatter Cost	271
Rank Straggler Detection	271
Packet Error Retransmit Tracking	271
Topology Congestion Awareness	271
Scheduler Network Locality	271
Network Communication Gate	271
Storage System Audit	271
Capacity Tier Fit	272
Throughput IOPS Fit	272
Local NVMe Cache Fit	272
Shared FS Metadata Fit	272
Object Store Authority Fit	272
Data Lake Governance Fit	272
Training Shard Format Fit	272
Small File Amplification Control	272
Dataloader Cache Hit Tracking	273
Checkpoint Write Recovery Fit	273
Model Weight Load Cache Fit	273
Artifact Lineage Metadata Fit	273
Consistency Commit Integrity	273
Security Compliance Fit	273
Lifecycle Cost Governance	273
Storage System Gate	273
Checkpoint Lifecycle Audit	274
Checkpoint Object Completeness	274
Shard Layout Fit	274

Async Save Overlap	274
Write Bandwidth Fit	274
Metadata Commit Integrity	274
Checksum Manifest Validation	274
Restore Replay Readiness	274
Dataloader RNG State Capture	275
Distributed Rank State Capture	275
Retention Policy Fit	275
Cross-Region Replication Fit	275
Security Access Control	275
Resume SLO Tracking	275
Failure Drill Coverage	275
Checkpoint Lifecycle Gate	275
Container Environment Audit	275
Image Digest Reproducibility	276
Base Layer Cache Fit	276
CUDA Driver Framework Compatibility	276
GPU Runtime Visibility	276
Dependency Lock Coverage	276
Training Serving Image Separation	276
Image Size Startup Fit	276
Build Pipeline Smoke Test	276
Security Scan Signing	277
Secret Data Exclusion	277
Runtime Hardening Fit	277
Registry Access Governance	277
Environment Metadata Capture	277
Multi-Tenant Mount Isolation	277
NCCL RDMA Runtime Fit	277
Container Environment Gate	277
Kubernetes GPU Resource Management Audit	277
Device Plugin Readiness	278
GPU Extended Resource Fit	278
GPU Request Limit Integrity	278
Gang Scheduling Readiness	278
Fragmentation Control	278
Topology Aware Placement	278
Node Label Affinity Fit	278
Taint Toleration Fit	279
MIG Sharing Policy Fit	279
Quota Namespace Governance	279
Training Operator Readiness	279
Inference Service Readiness	279
GPU Monitoring Mapping	279
Multi-Tenant Isolation	279
Pending Troubleshooting Coverage	279
Kubernetes GPU Gate	279
Training Scheduler Governance Audit	280
Workload Resource Shape	280
Queue Policy Coverage	280
Priority Governance	280
Quota Usage Control	280
Fair Share Accounting	280
Gang Scheduling Fit	281
Backfilling Safety	281

Preemption Checkpoint Safety	281
Scheduler Fragmentation Control	281
Scheduler Topology Awareness	281
Checkpoint Scheduler Coupling	281
Cost Attribution Control	281
Scheduler Observability	281
Failure Requeue Readiness	281
Anti Starvation Control	281
Training Scheduler Gate	282
Multi Tenant GPU Scheduler Design Audit	282
Tenant Queue Contract	282
Quota Borrowing Policy	282
Dominant Fairness Accounting	283
Low Priority Reclaim	283
Inference SLO Isolation	283
Heterogeneous GPU Policy	283
Resource Snapshot Health	283
Placement Explainability	283
Cost Utilization Attribution	283
Scheduler Observability Audit	283
Tradeoff Boundary Reasoning	283
Multi Tenant GPU Scheduler Gate	283
Multi-Tenant Isolation Audit	284
Resource Quota Isolation	284
Identity Namespace Binding	284
RBAC ABAC Permission Fit	284
Data Access Boundary	284
Data Classification Lineage	285
Network Policy Isolation	285
Runtime Security Boundary	285
Image Supply Chain Boundary	285
Secret Scope Rotation	285
Logs Trace Redaction	285
Cost Attribution Coverage	285
Prod Experiment Separation	285
Cross Tenant Sharing Governance	285
Audit Evidence Readiness	285
Blast Radius Control	285
Multi-Tenant Isolation Gate	286
Cluster Capacity Planning Audit	286
Workload Forecast Coverage	286
GPU Count Capacity Fit	286
GPU Type Mix Fit	286
Training Queue SLO Fit	286
Serving Peak SLO Fit	287
Network Bandwidth Capacity Fit	287
Storage Throughput Capacity Fit	287
Storage Capacity Lifecycle Fit	287
Utilization Headroom Fit	287
Failure Redundancy Fit	287
Growth Forecast Fit	287
Cost Budget Fit	287
Pool Separation Fit	287
Quota Burst Governance	287
Observability Forecast Feedback	287

Cluster Capacity Planning Gate	288
Training Platform Lifecycle Audit	288
TrainingJob Contract	288
Config Validation	288
Resource Quota Binding	288
Image Code Reproducibility	289
Dataset Lineage Permission	289
Launcher Distributed Fit	289
Checkpoint Resume Policy	289
Observability Events Metrics	289
Experiment Tracking Lineage	289
Artifact Registry Linkage	289
Failure Recovery Classification	289
Security Audit Control	289
Cost Attribution Control	289
Developer Self Service	289
Lifecycle State Machine	290
Training Platform Gate	290
Training Submission Audit	290
TrainingJob Schema Completeness	290
Required Field Coverage	290
Image Digest Binding	291
Code Commit Diff Binding	291
Dataset Version Permission	291
Structured Command Safety	291
Final Config Snapshot	291
Distributed Resource Consistency	291
Checkpoint URI Resume Validation	291
Logging Metrics Destination	291
Queue Priority Policy Binding	291
Quota Dry Run Admission	291
Idempotency Duplicate Control	291
Template Version Governance	292
Submit State Machine Validity	292
Submission Audit Trace	292
Training Submission Gate	292
Distributed Launcher Audit	292
Resource World Size Consistency	292
Rank Local Rank Mapping	293
Rendezvous Endpoint Readiness	293
GPU Binding Visibility	293
Launcher Adapter Fit	293
Torchrun Argument Fit	293
DeepSpeed Config Fit	293
Megatron Parallel Consistency	293
Ray Runtime Fit	293
Network NCCL Readiness	293
Log Rank Aggregation	293
Failure Stage Classification	293
Elastic Training Safety	294
Checkpoint Launcher Coupling	294
Scheduler Launcher Handoff	294
Launcher Audit Trace	294
Distributed Launcher Gate	294
Training Config Management Audit	294

Config Source Order	295
Final Config Snapshot Freeze	295
Required Config Field Coverage	295
Config Schema Type Validation	295
Batch Parallel Consistency	295
Immutable Version Binding	295
Environment Version Capture	295
Reproducibility Seed Boundary	295
Experiment Tracking Linkage	295
Hyperparameter Sweep Trace	295
Checkpoint Resume Compatibility	295
Release Config Coupling	296
Config Permission Approval	296
Config Diff Readiness	296
Config Audit Trace	296
Training Config Gate	296
Training Observability Audit	296
Structured Log Context	297
Rank Log Coverage	297
Metric Taxonomy Coverage	297
Metric Dimension Scope	297
Event Schema Lifecycle	297
State Machine Validity	297
Attempt Retry Traceability	297
Trace Phase Coverage	297
Dashboard Diagnostic Readiness	297
Alert Rule Readiness	298
Anomaly Detection Rules	298
Retention Cost Governance	298
Privacy Redaction Access	298
Cost Attribution Signal	298
Training Observability Gate	298
Checkpoint Strategy Audit	298
RPO RTO Budget	299
Save Trigger Policy	299
Checkpoint Scope Separation	299
Async Snapshot Consistency	299
Manifest Commit Integrity	299
Shard Metadata Resharding	299
Restore Selection Policy	299
Preemption Checkpoint Coupling	300
Evaluation Best Linkage	300
Release Checkpoint Readiness	300
Retention Tier Lifecycle	300
Storage Cost Budget	300
Checkpoint Monitoring Alerts	300
Restore Drill Coverage	300
Permission Release Protection	300
Checkpoint Strategy Gate	300
Training Fault Tolerance Audit	301
Fault Taxonomy Coverage	301
Retryability Classification	301
Retry Budget Bounded	301
Retry Backoff Policy	302
Checkpoint Resume Readiness	302

Node Failure Reschedule	302
GPU Health Isolation	302
Communication Hang Detection	302
OOM Handling Policy	302
Data Permission Fast Fail	302
Checkpoint Commit Safety	302
Numeric Instability Stop Rollback	302
Diagnosis Report Evidence	302
Blacklist Precision	302
User Action Clarity	303
Training Fault Tolerance Gate	303
Training Data Supply Audit	303
Dataset Manifest Integrity	304
Shard Format Size Fit	304
Small File Amplification Control	304
Streaming Prefetch Retry	304
Local Cache Policy	304
Distributed Cache Policy	304
Shuffle Reproducibility	304
Rank Shard Assignment	304
Data Locality Fit	304
Dataloader Parallelism Fit	304
H2D Copy Readiness	304
Bad Sample Threshold	305
Data Supply Observability	305
Checkpoint Dataloader State	305
Permission Version Governance	305
Training Data Supply Gate	305
Data Platform Supply Chain Audit	305
Source Registry Coverage	306
Raw Data Lake Integrity	306
Cleaning Policy Versioning	306
Dedup Contamination Guard	306
PII Redaction Coverage	306
Quality Scoring Calibration	306
Dataset Builder Recipe	306
Dataset Manifest Completeness	307
Dataset Version Immutability	307
Streaming Shard Readiness	307
Permission Purpose Binding	307
Data Lineage Coverage	307
Data Platform Supply Observability	307
Read Throughput Fit	307
Data Wait Fit	307
Data Platform Supply Gate	307
Data Version Lineage Quality Audit	308
Dataset Version Contract	308
Manifest Checksum Integrity	308
Immutable Snapshot Policy	308
Delta Version Diff	309
Lineage Graph Completeness	309
Impact Analysis Readiness	309
Schema Quality Gate	309
Content Quality Gate	309
Data Safety Compliance Gate	309

Distribution Drift Monitoring	309
Train Eval Leakage Monitoring	309
Annotation Quality Monitoring	309
Synthetic Data Monitoring	309
Quality Alert Routing	309
Training Model Version Linkage	310
Data Version Quality Gate	310
Model Artifact Registry Audit	310
Model Version Contract	311
Weight Manifest Integrity	311
Safe Weight Format	311
Tokenizer Config Binding	311
Adapter Base Compatibility	311
Merge Lineage Completeness	311
Quantization Eval Gate	311
Runtime Compatibility Fit	311
Eval Report Linkage	311
Safety Permission Gate	312
Release Status Governance	312
Rollback Alias Readiness	312
Artifact Lineage Completeness	312
Load Cache Readiness	312
Lifecycle Retention Policy	312
Model Registry Gate	312
Artifact Management Audit	312
Artifact Metadata Contract	313
Dataset Artifact Manifest	313
Checkpoint Artifact Recoverability	313
Eval Report Reproducibility	313
Deployment Package Completeness	314
Release Manifest Integrity	314
Checksum Integrity Gate	314
Artifact Store Metadata Split	314
Permission Access Control	314
Promotion Gate Readiness	314
Experiment Tracking Linkage	314
Rollback Artifact Readiness	314
Deletion Dependency Safety	314
Artifact Management Gate	314
Experiment Tracking Audit	315
Run Metadata Contract	315
Parameter Config Capture	316
Metric Curve Step Capture	316
Log Run Linkage	316
Sample Result Diagnosis	316
Code Version Reproducibility	316
Data Version Reproducibility	316
Model Checkpoint Linkage	316
Environment Capture	316
Prompt Eval Config Capture	316
Artifact Output Linkage	316
Cost Attribution	316
Run State Transition	317
Search Index Readiness	317
Lineage Graph Readiness	317

Decision Note Capture	317
Experiment Tracking Gate	317
Evaluation Platform Audit	317
Eval Dataset Contract	318
Offline Batch Reproducibility	318
Metric Definition Versioning	318
LLM Judge Calibration	318
Human Review Quality Control	318
Pairwise Blinding Randomization	318
Online Eval Guardrail	318
Release Gate Readiness	319
Eval Report Completeness	319
Slice Regression Detection	319
Regression Suite Coverage	319
Eval Job Scheduler Reliability	319
Eval Cache Key Integrity	319
Eval Observability	319
Artifact Tracking Linkage	319
Evaluation Platform Gate	319
Feature Embedding Index Infrastructure Audit	320
Feature Definition Contract	320
Offline Online Consistency	321
Point in Time Correctness	321
Embedding Version Contract	321
Chunk Embedding Lineage	321
Vector Index Build Readiness	321
ANN Quality Latency Gate	321
Metadata Permission Filter	321
Shadow Index Switch Readiness	321
Retrieval Trace Completeness	321
Feature Embedding Lineage Graph	321
Multi Tenant Isolation	321
Quality Monitoring Metrics	322
Cost Capacity Governance	322
RAG Agent Integration	322
Feature Embedding Index Gate	322
AI Infra Observability Audit	322
Signal Inventory Coverage	322
Metric Contract Completeness	323
SLO Error Budget Readiness	323
Latency Quantile Guard	323
Trace Span Coverage	323
Log Event Structure	323
Correlation ID Coverage	323
Cardinality Budget Control	323
Training Observability	323
Inference Observability	324
Data Quality Observability	324
RAG Agent Observability	324
Cost Observability	324
Privacy Redaction Retention	324
Alert Actionability	324
Observability Platform Gate	324
Training Fault Diagnosis Audit	325
Training Fault Evidence Coverage	325

Lifecycle Event Timeline	325
Rank Log Completeness	325
Loss Anomaly Diagnosis	325
NaN Inf Guard	325
Update Health Check	326
Hang Straggler Detection	326
OOM Phase Memory Evidence	326
Communication Bottleneck Diagnosis	326
IO Data Bottleneck Diagnosis	326
Checkpoint Integrity	326
Training Resume Continuity	326
Data Fault Isolation	327
Config Code Diff Readiness	327
Minimal Reproduction Readiness	327
Training Fault Diagnosis Gate	327
SLO Oncall Audit	327
SLI Contract Completeness	328
SLO Target Measurability	328
SLA Boundary Clarity	328
Inference SLO Coverage	328
Training SLO Coverage	328
Data Eval SLO Coverage	328
Error Budget Accounting	328
Burn Rate Alerting	328
Alert Actionability	328
Oncall Ownership Escalation	329
Incident Severity Routing	329
Runbook Executability	329
Change Event Linkage	329
Mitigation Rollback Authority	329
Postmortem Action Closure	329
SLO Cost Tradeoff Gate	329
AI Infra Cost Governance Audit	329
Usage Metering Coverage	330
Cost Attribution Labels	330
GPU Cost Efficiency	330
Training Waste Control	330
Inference Unit Cost	330
Cache Savings Accounting	330
Storage Lifecycle Governance	330
Network Egress Governance	330
Artifact Dependency Safety	331
Budget Quota Enforcement	331
Cost Anomaly Alerting	331
Tenant Model Chargeback	331
Dashboard Drilldown Readiness	331
Optimization Recommendation Trace	331
SLO Quality Cost Tradeoff	331
Cost Governance Gate	331
Resource Utilization Optimization Audit	331
Resource Metric Contract	332
Effective Utilization Accounting	332
Fragmentation Control	332
Topology Binpacking Readiness	332
Colocation SLO Guard	332

Preemption Checkpoint Safety	332
Low Priority Reclaim Policy	332
Elastic Training Safety	333
Inference Capacity Utilization	333
Warm Pool Headroom Policy	333
Quota Fairness Borrowing	333
Scheduler Observability	333
Reliability Headroom Guard	333
Cost SLO Tradeoff Control	333
Resource Utilization Gate	333
AI Infra Security Governance Audit	333
Identity Workload Binding	334
Secret Lifecycle Management	334
Data Compliance Classification	334
PII Sensitive Data Control	334
Tenant Isolation Boundary	334
Model Access Control	334
Model Output Safety Gate	334
RAG Permission Enforcement	335
Agent Tool Safety Gate	335
Supply Chain Integrity	335
Model Artifact Integrity	335
Log Trace Privacy Governance	335
Audit Incident Response Readiness	335
Security Governance Gate	335
Audit Change Governance Audit	335
Audit Event Schema Completeness	336
Critical Operation Audit Coverage	336
Audit Log Integrity Retention	336
Log Retention Policy Fit	336
Log Minimization Redaction	336
Trace Privacy Sampling	336
Incident Timeline Completeness	336
Impact Detection Response Metrics	337
Root Cause Systemic Analysis	337
Postmortem Action Item Closure	337
Change Record Completeness	337
High Risk Change Approval	337
Rollout Rollback Readiness	337
Change Freeze Error Budget Policy	337
Audit Postmortem Change Linkage	337
Governance Loop Gate	337
Training Platform Security Audit	338
Identity Runtime Binding	338
RBAC ABAC Policy Fit	338
Dataset Permission Purpose	338
Artifact Classification Propagation	339
Resource Quota Priority Governance	339
Image Supply Chain Trust	339
Runtime Code Isolation	339
Secret Scope Rotation	339
Network Egress Control	339
Log Redaction Access	339
Model Artifact Permission	339
High Risk Approval	339

Audit Log Completeness	339
Audit Integrity Retention	339
Incident Response Readiness	340
Training Platform Security Gate	340
Training Platform System Design Audit	340
Requirement Clarification	341
TrainingJob Contract	341
Lifecycle State Machine	341
Admission Quota Policy	341
Scheduler Resource Fit	341
Distributed Launcher Fit	341
Data Config Lineage	341
Checkpoint Recovery Coupling	341
Observability Experiment Linkage	341
Fault Recovery Flow	342
Security Audit Governance	342
Cost Capacity Governance	342
Developer Self Service	342
Interface Artifact Contract	342
Tradeoff Boundary Reasoning	342
Training Platform Design Gate	342
AI Infra Overview Audit	342
AI Infra Overview Gate	343
AI Infra MLOps LLMOps Boundary Audit	343
Boundary Gate	343
Accelerator Selection Audit	344
Accelerator Selection Gate	344
Bandwidth Bottleneck Audit	345
Bandwidth Bottleneck Gate	345
分布式训练排障常见问题	345
本章小结	346
K. 长上下文、RAG 与 Agent	346
Long Context	353
Context Window	353
Position Extrapolation	354
Lost in the Middle	354
RAG	354
RAG Product	355
RAG Incident	355
RAG Error Attribution	355
Document Governance	355
Retrieval Recall	355
Mean Reciprocal Rank / MRR	355
Retrieval Miss	355
Context Drop	355
Context Recall	356
Abstention Accuracy	356
Stale Evidence Rate	356
Unsupported Claim Rate	356
Permission Leak Rate	356
RAG Freshness Gate	356
RAG Product Gate	356
RAG Incident Gate	356
Agentic RAG	357
Active Retrieval	357

Retrieval Controller	357
Multi-Round Retrieval	357
Evidence State	357
Evidence Gap	357
Query Drift	358
New Evidence Gain	358
Context Precision	358
Evidence Support Rate	358
Citation Accuracy	358
Agentic RAG Gate	358
RAG Prompt Injection	358
Chunking	359
Dense Retrieval	359
Sparse Retrieval	359
Hybrid Retrieval	359
Reranker	360
Vector Database	360
Query Rewrite	360
Grounding	360
Agent	361
Agent Harness	361
Agent Runtime	361
Session Manager	361
Task Manager	361
Context Builder	362
Model Adapter	362
State Store	362
Error Handler	362
Runtime Gate	362
Agent Loop	362
Harness Loop Audit	362
Action Parser	363
Agent State	363
Agent Action	363
Observation	363
Controller	363
Permission Gate	363
Replay Readiness	363
Harness Gate	364
Tool Registry	364
Tool Registry Completeness	364
Tool Schema Strictness	364
Tool Permission Binding	364
Side Effect Level	364
Risk Protection Coverage	364
Output Normalization Coverage	365
Tool Version Coverage	365
Tool Trace Readiness	365
Untrusted Tool Output Boundary	365
Tool Registry Gate	365
Tool Registry Identity Coverage	365
Tool Description Quality	365
Schema Contract Coverage	365
Runtime Metadata Coverage	366
Tool Permission Binding Coverage	366

Risk Annotation Coverage	366
Version Trace Coverage	366
Lifecycle Policy Pass Rate	366
Owner SLO Coverage	366
Tool Eval Binding Coverage	366
Provider Projection Readiness	366
Registry Audit Completeness	367
Tool Executor	367
Execution Request Validity	367
Schema Validation Pass Rate	367
Permission Enforcement Pass Rate	367
Execution Mode Accuracy	367
Async Completion Tracking	367
Unknown State Escalation Rate	368
Retry Safety Rate	368
Side Effect Confirmation Coverage	368
Structured Result Coverage	368
Executor Trace Completeness	368
Tool Executor Gate	368
Tool Permission Model	368
Authorization Decision Accuracy	368
Trusted Context Injection	369
Tenant Isolation Pass Rate	369
Tool Permission Enforcement	369
Object Permission Accuracy	369
Field Projection Safety	369
Action Context Policy Accuracy	369
Prompt Injection Block Rate	369
Token Audience Validation	369
Revocation Cache Safety	370
Error Disclosure Safety	370
Permission Audit Completeness	370
Tool Permission Gate	370
Tool Security	370
Prompt Injection Containment	370
Untrusted Content Isolation	370
Unauthorized Access Block Rate	370
Sensitive Data Protection	371
Tool External Transfer Gate	371
Dangerous Action Confirmation	371
SSRF Block Rate	371
SQL Risk Block Rate	371
Path Traversal Block Rate	371
Shell Sandbox Enforcement	371
Data Flow Policy Pass Rate	371
Tool Security Audit Completeness	372
Security Alert Completeness	372
Safety Eval Regression Pass Rate	372
Tool Security Gate	372
Tool Trace Replay Audit	372
ID Tree Integrity	372
Argument Lineage Coverage	372
Permission Trace Completeness	372
Tool Result Trace Completeness	373
Audit Event Completeness	373

Replay Side Effect Safety	373
Alert Owner Coverage	373
Eval Linkage Coverage	373
Tool Version Release Audit	373
Spec Lint Pass Rate	373
Schema Backward Compatibility	373
Description Behavior Guard	374
Output Compatibility	374
Permission Policy Compatibility	374
Version Matrix Capture	374
Offline Eval Pass Rate	374
Canary Routing Stability	374
Canary Quality Guard	374
Canary Safety Guard	374
Cost Latency Guard	375
Tool Release Rollback Readiness	375
Lifecycle Coverage	375
Tool Release Gate	375
Enterprise Tool Platform Audit	375
Platform Module Coverage	375
Platform Interface Contract Coverage	375
Registry Readiness	375
Router Governance Coverage	376
Executor Safety Coverage	376
Platform Permission Integration Coverage	376
Platform Safety Guard Coverage	376
Platform Trace Audit Coverage	376
Platform Eval Service Coverage	376
Platform Release Governance Coverage	376
Platform Tenant Isolation Coverage	376
Platform Provider Adapter Coverage	377
High Availability Readiness	377
Admin Ops Governance Coverage	377
Enterprise Tool Platform Gate	377
Agent Trace	377
Trace Schema Completeness	377
Span Schema Completeness	377
Span Tree Validity	377
Timeline Validity	378
Artifact Reference Coverage	378
Version Capture Coverage	378
Replay Readiness Rate	378
Privacy Masking Coverage	378
Error Attribution Coverage	378
Final Status Consistency	378
Metric Export Coverage	378
Eval Export Coverage	379
Trace Replay Gate	379
Agent Budget	379
Agent Incident	379
Plan Feasibility Rate	379
False Completion Rate	379
Tool Result Injection Block Rate	379
Agent Incident Gate	380
Agent Gate	380

Agent Product	380
Workflow-Agent Hybrid	380
Agent Automation Level	380
Agent Product Trace	380
Tool Execution Success Rate	380
Human Confirmation Coverage	381
Agent Recovery Rate	381
Observation Use Rate	381
State Update Coverage	381
Unauthorized Action Rate	381
Budget Overrun Rate	381
Agent Product Gate	381
Tool Use	382
Tool Schema	382
JSON Schema	382
Required Field	382
Additional Properties	382
Enum Constraint	382
Pattern Constraint	383
Range Constraint	383
Business Validation	383
Schema Repair	383
Schema Gate	383
Tool Call	383
Tool Choice	383
Tool Choice Policy	384
Candidate Tool Set	384
Tool Router	384
Scenario Filter Pass Rate	384
Permission Filter Pass Rate	384
Risk Filter Pass Rate	384
Candidate Recall	384
Candidate Precision	384
Candidate Size Pass Rate	385
Clarification Accuracy	385
Forced Tool Accuracy	385
Router Parallel Safety	385
Provider Capability Compatibility	385
Router Trace Completeness	385
Tool Router Gate	385
Auto Tool Choice	385
None Tool Choice	386
Required Tool Choice	386
Forced Tool Choice	386
Allowed Tools	386
Tool Choice Mode Accuracy	386
No-Tool Clarification Block Rate	386
Forced Missing Argument Block Rate	386
Parallel Safety Rate	386
Rate Limit Pass Rate	387
Confirmation Enforcement Rate	387
Cost Budget Pass Rate	387
Loop Control Rate	387
Tool Choice Gate	387
Argument Parsing	387

Parse Success Rate	387
Validation Error	387
Normalization	388
Argument Evidence Map	388
Argument Evidence Coverage	388
Safe Normalization Rate	388
Repair Attempt	388
Model Self-Repair	388
Clarification Coverage	388
Unsafe Argument Block Rate	388
Retry Budget Pass Rate	389
Idempotency Key Coverage	389
Repair Trace Completeness	389
Argument Repair Gate	389
Structured Function Calling	389
JSON Mode	389
Structured Outputs	389
Tool Selection Accuracy	390
Schema Valid Rate	390
Required Field Pass Rate	390
Type Valid Rate	390
Enum Valid Rate	390
Pattern Valid Rate	390
Range Valid Rate	390
Additional Properties Block Rate	390
Business Rule Pass Rate	390
Schema Repair Success Rate	391
Argument Exact Match	391
Argument Validity	391
Unauthorized Attempt Block Rate	391
Function Calling Protocol	391
Tool Call ID	391
Tool Result	391
Tool Result Context	391
Tool Result Projection	392
Source Metadata Coverage	392
Redaction Coverage	392
Tool Result Context Budget Pass Rate	392
Injection Containment Rate	392
Error Status Fidelity	392
Compression Fidelity	392
Conflict Labeling Rate	392
Tool Result Freshness Pass Rate	393
Memory Boundary Pass Rate	393
Tool Result Context Gate	393
Tool Failure Recovery	393
Structured Tool Error	393
Retryable Error	393
Non-Retryable Error	393
Retry Policy Precision	393
Exponential Backoff with Jitter	394
Retry Budget	394
Unknown Execution State	394
Fallback Honesty	394
Circuit Breaker Containment	394

Timeout Cancellation Coverage	394
Human Handoff Readiness	394
User-Visible Error Clarity	395
Failure Trace Completeness	395
Tool Failure Recovery Gate	395
Tool Calling Evaluation	395
Tool Call Recall	395
Tool Call Precision	395
Tool Set Precision	395
Tool Set Recall	395
Argument Value Accuracy	395
Argument Source Coverage	396
Execution Success Rate	396
Observation Use Correctness	396
Safe Failure Rate	396
Unsafe Failure Rate	396
Tool Eval Dataset Coverage	396
Tool Eval Regression Pass Rate	396
Tool Calling Eval Gate	396
Finish Reason	397
Tool Loop	397
Parallel Tool Calls	397
Streaming Tool Call Delta	397
Protocol Chain Validity	397
Tool Result ID Match Rate	397
Finish Reason Consistency	397
Streaming Safety Rate	397
Parallel Alignment Rate	397
Idempotency Protection Rate	398
Tool Call Trace	398
Tool Result Injection	398
Function Calling	398
Tool Calling Gate	398
ReAct	398
Plan-Act-Observe	399
ReAct Trace	399
Plan Update	399
Repeat Action	399
Premature Final	399
ReAct Gate	400
Planning	400
Task Decomposition	400
Subgoal	400
Acceptance Criteria	400
Dependency Graph	400
Dynamic Replanning	401
Critical Path	401
Planning Gate	401
Memory	401
Short-Term Memory	401
Long-Term Memory	401
Episodic Memory	402
Semantic Memory	402
Procedural Memory	402
Preference Memory	402

Memory Write Gate	402
Memory Retrieval Score	402
Memory Permission Gate	403
Memory Decay	403
Memory Conflict	403
Memory Pollution	403
Memory Gate	403
Code Agent	403
Coding Agent Workflow	404
Task Type Classification	404
Repository Exploration Coverage	404
Plan Coverage	404
Workflow Validation Coverage	404
Feedback Use Rate	404
Coding Agent Workflow Gate	404
File System Tool	405
Workspace Root	405
Path Containment	405
Read File Tool	405
Search Tool	405
Apply Patch Tool	405
Patch Context Match	405
Secret File Block Rate	405
Read Truncation Clarity	406
Concurrent Edit Protection	406
Unrelated Diff Rate	406
Diff Faithfulness	406
Rollback Readiness	406
File Edit Gate	406
Terminal Execution	406
Bash Tool	406
Test Runner Safety	407
Command Risk Level	407
Command Allowlist	407
Command Denylist	407
Command Auto Execution Precision	407
Dangerous Command Block Rate	407
Command Confirmation Coverage	407
Network Access Control	407
Timeout Cancellation Coverage	408
Command Output Truncation	408
Secret Masking Coverage	408
Command Trace	408
Command Safety Gate	408
Context Management	408
Context Candidate	408
Context Budget Utilization	408
Key Context Recall	409
Constraint Retention	409
Task State Compression	409
Summary Faithfulness	409
Stale Summary Rate	409
Tool Output Compression Fidelity	409
Current Diff Coverage	409
Trust Boundary Coverage	409

Memory Write Safety	409
Context Builder Gate	410
Patch Localization	410
Minimal Patch	410
Test Execution Feedback	410
Code Sandbox	410
User Change Protection	410
Code Agent Gate	410
Claude Code Architecture	410
Terminal Coding Agent	411
CLI Control Command	411
Architecture Evidence Coverage	411
Required Module Coverage	411
Access Governance Coverage	411
Core Loop Governance Coverage	411
Permission Control Coverage	411
State Recovery Coverage	411
Extension Governance Coverage	411
Observability Coverage	412
Architecture Eval Readiness	412
High-Risk Surface Governance	412
Claude Code Architecture Gate	412
OpenCode Architecture	412
Open Runtime Control Plane	412
Open Runtime Module Coverage	412
Config Governance Coverage	412
Agent Permission Isolation	412
Tool Permission Binding Coverage	413
MCP Tool Namespace Coverage	413
Custom Tool Schema Coverage	413
Server API Governance	413
Snapshot Recovery Coverage	413
Provider Adapter Coverage	413
OpenCode Architecture Gate	413
Coding Agent Comparison	413
Product Surface Coverage	413
Context Source Coverage	414
Tool Execution Coverage	414
Permission Governance Coverage	414
Edit Recovery Coverage	414
Agent Eval Readiness	414
Enterprise Governance Coverage	414
Cross-Agent Risk Governance	414
Coding Agent Comparison Gate	414
MCP Background Audit	414
Direct Integration Count	414
MCP Integration Count	415
Integration Reduction	415
Capability Model Coverage	415
Context Object Coverage	415
Discovery Standardization	415
MCP Schema Contract Coverage	415
Host Server Boundary Clarity	415
Local Context Control	415
Cross Client Reuse	416

Governance Boundary Clarity	416
MCP Trace Eval Readiness	416
MCP A2A Distinction	416
MCP Background Gate	416
MCP Concept Audit	416
Host Policy Ownership	416
MCP Client Server Boundary	416
Server Capability Declaration	417
MCP Tool Schema and Result	417
MCP Resource URI Metadata	417
MCP Prompt Argument Review	417
MCP Lifecycle Negotiation	417
MCP Transport Policy	417
MCP Roots Boundary	417
MCP Sampling Control	417
Host Capability Filtering	418
MCP Context Budget	418
MCP Concept Gate	418
MCP Server Implementation Audit	418
MCP Server Metadata Readiness	418
MCP Server Capability Declaration	418
MCP Tool Registry Readiness	418
MCP Strict Schema Coverage	418
MCP Handler Execution Coverage	419
MCP Argument Validation Coverage	419
MCP Structured Result Coverage	419
MCP Structured Error Coverage	419
MCP Resource Scope Coverage	419
MCP Prompt Template Coverage	419
MCP Server Transport Policy Coverage	419
MCP Host Connection Readiness	419
MCP Server Safety Baseline	420
MCP Server Trace Readiness	420
MCP Server Gate	420
MCP Tool Function Calling Comparison Audit	420
Protocol Layer Clarity	420
Tool Discovery Boundary	420
MCP Capability Scope Coverage	420
MCP Execution Boundary Clarity	421
MCP Projection Mapping Coverage	421
Adapter Separation Coverage	421
MCP Lifecycle Version Awareness	421
MCP Governance Registry Import	421
MCP Security Boundary Enforcement	421
MCP Use Case Selection Fit	421
MCP Error Surface Separation	421
MCP Latency Availability Tradeoff	422
MCP Function Calling Gate	422
MCP Resource Exposure Audit	422
Resource URI Validity	422
Resource Metadata Completeness	422
Resource List Filtering Boundary	422
Resource Read Scope Enforcement	422
MCP Roots Containment	422
MCP Resource Permission Enforcement	423

MCP Resource Field Projection Safety	423
Resource Context Budget Control	423
Resource Citation Traceability	423
Untrusted Resource Labeling	423
Resource Freshness Version Awareness	423
Resource Template Boundary	423
Resource Subscription Awareness	423
Resource Trace Eval Readiness	424
MCP Resource Gate	424
MCP Prompt Exposure Audit	424
Prompt Capability Declaration	424
Prompt Discovery Coverage	424
Prompt Argument Schema Coverage	424
Prompt Required Argument Validation	424
Prompt Rendering Safety	424
Prompt Role Boundary Enforcement	425
Prompt Dependency Alignment	425
Prompt Version Governance	425
Prompt Eval Binding	425
Prompt Permission Enforcement	425
Prompt Injection Containment	425
Prompt User Control	425
Prompt List Change Awareness	425
Prompt Trace Readiness	426
MCP Prompt Gate	426
MCP Security Sandbox Audit	426
Server Connection Governance	426
Authorization Token Binding	426
Scope Minimization	426
Roots Sandbox Containment	426
File Secret Blocking	427
Network SSRF Protection	427
MCP Shell Sandbox Enforcement	427
Prompt Injection Data Boundary	427
High Risk Confirmation	427
Sensitive Data Flow Control	427
Local Credential Isolation	427
MCP Tenant Isolation	427
Server Supply Chain Governance	428
MCP Security Trace Readiness	428
MCP Security Eval Coverage	428
MCP Security Gate	428
MCP Integration Surface Audit	428
MCP Capability Registration Coverage	428
MCP Namespace Isolation Coverage	428
MCP IDE Context Routing	428
MCP Knowledge Citation Traceability	429
MCP Database Query Governance	429
MCP Browser Action Governance	429
MCP Terminal Sandbox Governance	429
MCP Integration Context Budget Control	429
MCP Cross Server Data Flow Control	429
MCP High Risk Approval Coverage	429
MCP Output Projection Coverage	429
MCP Integration Trace Readiness	430

MCP Integration Eval Coverage	430
MCP Integration Gate	430
A2A Background Audit	430
Agent Card Completeness	430
Agent Discovery Readiness	430
Task Delegation Contract	430
A2A Task Lifecycle Coverage	431
A2A Message Structure Coverage	431
A2A Artifact Reference Coverage	431
A2A Context Boundary Control	431
A2A Permission Boundary	431
A2A MCP Distinction	431
A2A Failure Handling Coverage	431
A2A Trace Readiness	431
A2A Eval Coverage	432
A2A Background Gate	432
Agent Card Discovery Audit	432
Agent Card Field Completeness	432
Agent Skill Declaration Quality	432
Supported Interface Readiness	432
Agent Card Security Coverage	432
Agent Card Version Cache Readiness	433
Agent Discovery Match Quality	433
Agent Routing Decision Quality	433
Extended Agent Card Control	433
Agent Card Trace Readiness	433
Agent Card Eval Coverage	433
Agent Card Gate	433
A2A Task Delegation Audit	433
A2A Task Contract Coverage	434
A2A State Transition Validity	434
A2A Input Required Handling	434
A2A Artifact Metadata Coverage	434
A2A Error Semantics Coverage	434
A2A Retry Idempotency Coverage	434
A2A Cancellation Coverage	434
A2A Delegation Permission Boundary	434
A2A Parallel Aggregation Readiness	435
A2A Task Trace Readiness	435
A2A Task Eval Coverage	435
A2A Task Gate	435
A2A Message Boundary Audit	435
A2A Message Contract Coverage	435
A2A Part Typing Coverage	435
A2A Source Trust Labeling	435
A2A Instruction Data Separation	436
A2A Minimal Context Coverage	436
A2A Reference Over Copy Coverage	436
A2A Context Policy Enforcement	436
A2A Sensitive Redaction Coverage	436
A2A Claim Grounding Coverage	436
A2A Summary Constraint Retention	436
A2A Message Trace Readiness	436
A2A Message Eval Coverage	437
A2A Message Gate	437

A2A MCP Boundary Audit	437
A2A MCP Protocol Classification	437
A2A Tool Agent Boundary	437
A2A Autonomy Fit	437
A2A Lifecycle Placement	437
A2A MCP Discovery Split	437
A2A MCP Context Ownership	438
A2A MCP Permission Separation	438
A2A Result Artifact Boundary	438
A2A MCP Trace Linkage	438
A2A MCP Version Eval Coverage	438
A2A MCP Boundary Gate	438
Cross-Agent Security Audit	438
Cross-Agent Identity Chain Coverage	438
OBO Scope Binding	439
Delegation Allowlist Coverage	439
Permission Attenuation Coverage	439
Cross-Agent Context Policy Enforcement	439
Cross-Agent Redelelegation Control	439
Cross-Agent High Risk Confirmation	439
Cross-Agent Tool Permission Binding	439
Cross-Agent Result Release Control	439
Cross-Agent Audit Trace Completeness	440
Cross-Agent Trust Evidence Verification	440
Cross-Agent Tenant Isolation	440
Cross-Agent Security Gate	440
Multi-Agent Failure Audit	440
Delegation Loop Control	440
Role Conflict Arbitration	440
Hallucination Propagation Containment	440
Context Drift Control	441
Duplicate Work Budget Control	441
Multi-Agent State Consistency	441
Artifact Conflict Control	441
Multi-Agent Accountability Trace	441
Policy Chain Enforcement	441
Collaboration Fit	441
Termination Handoff Readiness	441
Multi-Agent Failure Eval Coverage	442
Multi-Agent Failure Gate	442
A2A System Design Audit	442
A2A Requirement Clarification	442
A2A Architecture Module Coverage	442
A2A Protocol Contract Coverage	442
A2A Discovery Routing Governance	442
A2A Runtime State Readiness	443
A2A Context Boundary Control	443
A2A Permission Model Coverage	443
A2A MCP Boundary Clarity	443
A2A Artifact Governance Coverage	443
A2A Trace Audit Completeness	443
A2A Failure Handling Coverage	443
A2A Eval Observability Coverage	443
A2A Scalability Idempotency Readiness	444
A2A System Design Gate	444

Cross-Agent Collaboration System Audit	444
Cross-Agent Requirement Goal Clarity	444
Cross-Agent Collaboration Module Coverage	444
Cross-Agent Registry Routing Governance	444
Cross-Agent Task Graph Validity	444
Cross-Agent A2A Lifecycle Alignment	444
Cross-Agent Context Minimization Enforcement	445
Cross-Agent Permission Attenuation Binding	445
Cross-Agent MCP Tool Boundary Governance	445
Cross-Agent Artifact Evidence Grounding	445
Cross-Agent Conflict Arbitration Readiness	445
Cross-Agent Hallucination Propagation Control	445
Cross-Agent Delegation Loop Containment	445
Cross-Agent Human Handoff Approval Readiness	445
Cross-Agent Trace Audit Replay Coverage	446
Cross-Agent Eval Baseline Regression Coverage	446
Cross-Agent Cost Latency Budget Control	446
Cross-Agent Scalability Idempotency Readiness	446
Cross-Agent Collaboration Fit Control	446
Cross-Agent Collaboration System Gate	446
Tool Protocol Future Audit	446
Layered Protocol Boundary Clarity	446
Standardization Extension Balance	447
Long Task Lifecycle Readiness	447
Capability Package Completeness	447
Agent-Centric Autonomy Governance	447
Tool Safety Metadata Coverage	447
Provenance Verifiability	447
Marketplace Governance Readiness	447
Provider Adapter Migration Control	447
Auto Generated Tool Review Gate	448
Workflow Runtime Integration	448
Behavior Eval Coverage	448
Human Model Documentation Split	448
Autonomy Risk Tiering	448
Responsibility Attribution Trace	448
Natural Language API Boundary	448
Agent Operating Layer Governance	448
Engineer Readiness Coverage	449
Unsupported Speculation Rate	449
Tool Protocol Future Gate	449
Skill Definition Audit	449
Skill Manifest Metadata	449
Skill Task Goal Clarity	449
Skill Bundle Completeness	449
Tool Skill Boundary	449
Skill Instruction Strategy	450
Skill Resource Prompt Grounding	450
Skill Workflow Readiness	450
Skill Permission Safety	450
Skill Configuration Reuse	450
Skill Eval Coverage	450
Skill Lifecycle Governance	450
Skill Product Install Governance	450
Skill Progressive Disclosure	451

Skill Definition Gate	451
Ecosystem Boundary Audit	451
Required Field Coverage	451
Abstraction Classification	451
Granularity Boundary	451
Action Trace	451
Tool Contract	451
Workflow Control	452
Skill Bundle	452
Plugin Packaging	452
Permission Layering	452
Eval Layering	452
Audit Trace Layering	452
Naming Alias Documentation	452
Governance Lifecycle	452
High Risk Approval	453
Eval Ready	453
Ecosystem Boundary Gate	453
Skill Manifest Audit	453
Identity Metadata	453
Description Quality	453
Capability Granularity	453
Input Contract	453
Output Contract	454
Tool Dependency Purpose	454
Resource Prompt Binding	454
Manifest Workflow Readiness	454
Permission Least Privilege	454
Configuration Schema	454
Safety Policy	454
Eval Gate	454
Examples Trigger Coverage	454
Version Lifecycle	455
Audit Readiness	455
Skill Manifest Gate	455
Skill Lifecycle Audit	455
Review Before Publish	455
Install Approval	455
Enable Scope Control	455
Permission Reapproval	455
Configuration Versioning	455
Running Task Policy	456
Lifecycle Compatibility Check	456
Rollout Guard	456
Skill Rollback Readiness	456
Dependency Versioning	456
Lifecycle Audit Log Completeness	456
Emergency Suspend Readiness	456
Uninstall Retention	456
Lifecycle Eval Monitoring	457
Update Policy Accuracy	457
Skill Lifecycle Gate	457
Skill Marketplace Audit	457
Catalog Metadata	457
Search Discovery	457

Detail Page Completeness	457
Review Workflow	457
Permission Transparency	458
Install Approval Flow	458
Rating Quality Balance	458
Operations Metrics	458
Duplicate Capability Governance	458
Admin Permission View	458
Developer Console Readiness	458
Security Center Readiness	458
Recommendation Policy Safety	459
Lifecycle Visibility	459
Owner Maintenance	459
Portal Audit Trace	459
Skill Marketplace Gate	459
Tool Skill Quality Safety Audit	459
Tool Schema Clarity	459
Argument Validation	459
Execution Reliability	460
Output Stability	460
Side Effect Control	460
Skill Task Quality	460
Factual Grounding	460
Completeness Format	460
Offline Eval Coverage	460
Online Monitoring	460
Permission Least Privilege	461
Data Security Review	461
Prompt Injection Resilience	461
High Risk Action Control	461
Supply Chain Governance	461
Human Review Readiness	461
Regression Release Gate	461
Audit Trace Readiness	461
Quality Safety Gate	462
MCP A2A Harness Integration	462
Protocol Capability Registry	462
MCP Integration	462
A2A Integration	462
Capability Discovery Coverage	462
Namespace Isolation Coverage	462
Protocol Schema Validity	462
Protocol Permission Binding	463
High-Risk Approval Coverage	463
Context Output Budget Coverage	463
External Trust Boundary Coverage	463
A2A Lifecycle Coverage	463
Protocol Trace Coverage	463
Protocol Replay Readiness	463
Protocol Version Capture	463
Protocol Integration Gate	463
Agent Harness System Design	463
Runtime Module Coverage	464
Interface Contract Coverage	464
Agent State Machine Coverage	464

Permission Integration Coverage	464
Context Control Coverage	464
Execution Isolation Coverage	464
System Trace Coverage	464
System Replay Readiness	464
System Eval Readiness	464
Recovery Coverage	464
Harness Version Capture	465
Enterprise Governance Readiness	465
Harness System Gate	465
Harness Pitfall Audit	465
Harness Triage Coverage	465
Permission Overreach	465
Context Overload Incident	465
User Edit Overwrite	465
Shell Hang Incident	465
Prompt Injection Boundary Miss	465
Trace Missing Incident	466
Eval Flakiness Incident	466
Doom Loop Guard	466
MCP Tool Overload	466
Environment Parity Check	466
Harness Pitfall Gate	466
Browser Agent	466
Computer Use Agent	466
GUI Action	466
Screenshot Observation	467
Accessibility Tree	467
UI State Tracking	467
Misclick Rate	467
Form Accuracy	467
High-Risk Action Protection	467
UI Agent Gate	467
Multi-Agent	467
Coordinator	468
Role Assignment	468
Communication Protocol	468
Blackboard	468
Agent Debate	468
Judge Agent	468
Consensus / Voting	468
Conflict Resolution	468
Duplicate Work Rate	469
Single-Agent Lift Rate	469
Multi-Agent Gate	469
Agent Evaluation	469
Evaluation Harness	469
Eval Dataset Bucket Coverage	469
Environment Reproducibility	469
Validator Coverage	470
Weighted Task Success	470
Diff Scope Safety	470
Baseline Fairness	470
Regression Pass Rate	470
Unsafe Execution Rate	470

Cost Budget Pass Rate	470
Flaky Task Rate	470
Eval Version Capture	470
Eval Report Completeness	471
Evaluation Harness Gate	471
Agent Benchmark	471
Eval Sample	471
Trajectory Evaluation	471
Partial Success Score	471
Trace Completeness	471
Summary Faithfulness	471
Claim Support Rate	471
Recovery Success Rate	472
Cost-Quality Trade-off	472
Regression Suite	472
Agent Eval Gate	472
Agent Safety	472
Least Privilege	472
Tool Permission Matrix	472
Permission Request	472
Least Privilege Coverage	472
Permission Matrix Coverage	473
Unauthorized Action Block Rate	473
Secret Access Block Rate	473
Network Egress Control	473
Sandbox Enforcement Coverage	473
Dry Run Coverage	473
Audit Log Completeness	473
Irreversible Action Protection	473
Permission Sandbox Gate	474
Untrusted Content Boundary	474
Data Flow Guard	474
Sensitive Data Filter	474
External Transfer Gate	474
Sandbox Policy	474
Human Confirmation	474
Dry Run	474
Audit Log	475
Irreversible Action	475
Memory Safety	475
Supply Chain Risk	475
Safety Alternative	475
Agent Safety Gate	475
Agent Interview Readiness	475
Agent Interview Rubric	475
Agent Answer Coverage	476
Agent Formula Coverage	476
Agent Demo Coverage	476
Trace Metric Coverage	476
Project Evidence Score	476
Trade-off Depth	476
Weak Agent Question	476
Agent Revision Plan	476
Agent Interview Gate	477
Tool Skill Developer Experience Audit	477

Quickstart Path Clarity	477
Scaffold Completeness	477
Documentation Example Coverage	477
Local Debug Readiness	477
Trace Replay Readiness	477
Review Feedback Actionability	477
Security Documentation Coverage	478
Migration Documentation Coverage	478
DX Documentation Gate	478
Composable Workflow Audit	478
Workflow Graph Validity	478
Dependency Acyclicity	478
Workflow IO Contract	478
Condition Determinism	478
Workflow State Coverage	479
Workflow Permission Boundary	479
Workflow Data Flow Policy	479
Workflow Idempotency Coverage	479
Retry Safety	479
Compensation Readiness	479
Workflow Human Approval Coverage	479
Workflow Trace Replay Readiness	479
Workflow Eval Coverage	480
Workflow Gate	480
Prompt Injection Defense Audit	480
Instruction Data Separation	480
Source Trust Labeling	480
Taint Propagation	480
Policy Pre-Tool Gate	480
Risky Tool Isolation	480
Untrusted Action Blocking	481
Sensitive Data Control	481
Prompt Injection High Risk Confirmation	481
Sandbox Enforcement	481
Output Redaction Projection	481
RAG Source Boundary	481
Multi-Agent Taint Propagation	481
Prompt Injection Trace Readiness	481
Prompt Injection Eval Coverage	482
Prompt Injection Defense Gate	482
Tool Output Trust Audit	482
Source Metadata Coverage	482
Trust Level Coverage	482
Freshness Version Coverage	482
Access Scope Disclosure	482
Citation Binding	482
Citation Support Accuracy	483
Evidence Chain Completeness	483
Claim Type Calibration	483
Confidence Limitation Disclosure	483
Conflict Resolution Readiness	483
Summary Provenance Retention	483
RAG Chunk Citation Accuracy	483
Multi-Agent Evidence Propagation	483
Provenance Trace Replay	484

Citation Forgery Block	484
Tool Output Trust Eval Coverage	484
Tool Output Trust Gate	484
RAG Tool Agent Memory Integration Audit	484
Capability Boundary Clarity	484
Orchestration Mode Fit	484
Context Priority Enforcement	484
Context Budget Allocation	485
Evidence Preservation	485
Tool Observation Use	485
Agent State Update Discipline	485
Memory Read Relevance	485
Memory Write Gate	485
Memory Scope Permission	485
Conflict Resolution Policy	485
Injection Propagation Control	486
Sensitive Data Memory Block	486
Trace Linkage Coverage	486
Layered Eval Coverage	486
RAG Tool Agent Memory Integration Gate	486
RAG Agent Storage Audit	486
Knowledge Base Contract	487
Document Chunk Version Contract	487
Sync Delete Propagation	487
ACL Permission Enforcement	487
Retrieval Trace Completeness	487
Prompt Assembly Trace	487
Citation Version Binding	487
Agent Definition Versioning	487
Tool Definition Contract	488
Tool Call Trace Completeness	488
Execution Trace Replay Readiness	488
Memory Privacy Lifecycle	488
Trace Privacy Retention	488
Cost Attribution Governance	488
RAG Agent Storage Gate	488
Tool Cost Latency Concurrency Audit	489
Cost Attribution Coverage	489
Latency Breakdown Coverage	489
Tool Call Budget Enforcement	489
Timeout Deadline Coverage	489
Retry Classification Accuracy	489
Idempotent Retry Safety	489
Concurrency Limit Enforcement	489
Queue Priority Fairness	490
Rate Limit Quota Handling	490
Cache Safety Correctness	490
Batching Partial Success Readiness	490
Result Trimming Budget Control	490
Degradation Transparency	490
Router Performance Awareness	490
Performance Trace Readiness	490
Cost Latency Eval Coverage	491
Tool Performance Gate	491
Tool Use Eval Benchmark Audit	491

Tool Need Accuracy	491
No-Tool Overcall Control	491
Argument Schema Validity	491
Argument Semantic Accuracy	491
Sequence Order Accuracy	491
Observation Grounding Accuracy	491
Error Recovery Readiness	492
Safe Failure Handling	492
Safety Policy Compliance	492
Tool Simulator Determinism	492
Trace Replay Coverage	492
Cost Latency Regression Control	492
Benchmark Slice Coverage	492
Regression Gate Readiness	492
Tool Use Eval Benchmark Gate	493
Function MCP A2A Comparison Audit	493
Layer Boundary Clarity	493
Function Calling Fit	493
MCP Integration Fit	493
A2A Delegation Fit	493
Object Contract Coverage	493
Capability Discovery Fit	493
Lifecycle State Alignment	494
Context Transfer Boundary	494
Permission Governance Split	494
Host Runtime Ownership	494
Trace Chain Continuity	494
Eval Gate Linkage	494
Overengineering Control	494
Severe Protocol Misuse Rate	494
Protocol Composition Gate	495
Provider Framework Tool Protocol Audit	495
Provider Framework Type Clarity	495
Tool Schema Projection	495
Tool Choice Mapping	495
Tool Result Round Trip	495
Streaming Event Assembly	495
Parallel Call Alignment	495
Built-In Tool Boundary	496
Error Normalization	496
Provider Adapter Isolation	496
Framework Escape Hatch	496
RAG Tool Traceability	496
Permission Safety Consistency	496
Migration Eval Coverage	496
Provider Trace Replay Readiness	496
Vendor Lock-In Control	497
Provider Runtime Gate	497
Enterprise MCP Platform Audit	497
MCP Gateway Readiness	497
MCP Registry Metadata Completeness	497
MCP Capability Namespace Isolation	497
MCP Tool Resource Prompt Contract	497
MCP Host Client Server Boundary Clarity	497
MCP OBO Scope Binding	498

MCP Tenant Isolation Enforcement	498
MCP Policy Engine Coverage	498
MCP Sandbox Roots Containment	498
MCP Context Output Projection	498
MCP Prompt Injection Taint Propagation	498
MCP Trace Audit Replay Continuity	498
MCP Cost Latency Quota Control	498
MCP Developer Portal Review Readiness	499
MCP Release Lifecycle Governance	499
MCP Provider Adapter Compatibility	499
MCP Eval Regression Coverage	499
MCP High Availability Readiness	499
Enterprise MCP Platform Gate	499
Enterprise LLMops Platform Audit	499
LLMops Application Resource Model	499
LLMops Release Manifest	500
Prompt Registry Versioning	500
RAG KB Permission Governance	500
Tool Agent Runtime Control	500
LLMops Eval Feedback Loop	500
LLMops Trace Observability Readiness	500
LLMops Cost Budget Governance	500
LLMops Platform Gate	501
长上下文、RAG 与 Agent 的关系	501
本章小结	501
L. Reasoning 与评估	501
Reasoning	502
Reasoning Model	502
Reasoning Candidate Set	502
Chain-of-Thought	502
Few-shot CoT	503
Zero-shot CoT	503
Scratchpad	503
Hidden CoT	503
Visible Explanation	503
CoT Faithfulness	504
CoT Regression	504
CoT Routing	504
CoT Audit	504
Self-Consistency	504
Sampling Temperature	505
Top-p Sampling	505
Answer Normalization	505
Majority Vote	505
Weighted Vote	505
Candidate Diversity	505
Majority Failure	505
Self-Consistency Cost	506
Self-Consistency Accuracy	506
Verifier	506
Programmatic Verifier	506
Hybrid Verifier	506
Verifier Reranking	507
Pairwise Accuracy	507
Hard Negative	507

Verifier Calibration	507
Reward Model Bias	507
Process Supervision	508
Step Label	508
First-Error Detection	508
Process Reward Model	508
Outcome Reward Model	509
Process Step Accuracy	509
Auto Label Coverage	509
Human Label Cost	509
Process Search Pruning	509
Process Supervision Audit	510
Test-Time Compute	510
Test-Time Compute Cost	510
Test-Time Compute Scaling	510
Compute Budget Vector	511
Adaptive Compute	511
Budget Router	511
Cost per Correct	511
Marginal Accuracy per Cost	511
P95 Latency	512
Wasted High Compute	512
TTC Audit	512
Math Reasoning Training	512
Math Training Sample	512
Answer Supervision	512
Synthetic Math Data	513
Math Curriculum	513
Math Contamination Audit	513
Template Diversity	513
Math Training Gate	513
Code Reasoning	514
Execution Feedback	514
Unit Test Verifier	514
Public Test	514
Hidden Test	514
Public-Hidden Gap	515
Self-Debug	515
Repair Success Rate	515
Sandbox Violation Rate	515
Code Execution Audit	515
Code Reasoning Gate	516
Reasoning Evaluation	516
Evaluation Sample	516
Variant Evaluation	516
Robustness Drop	516
Paired Lift	517
Bootstrap Confidence Interval	517
Reasoning Eval Gate	517
Best-of-N	517
Pass@k	518
Reasoning Audit	518
Reasoning Safety	518
Pseudo Reasoning	518
Overconfident Error	519

Hidden CoT Exposure	519
Tool Misuse	519
Human Review Coverage	519
Severity-Weighted Risk	519
Reasoning Safety Gate	519
Reasoning Interview Readiness	520
Reasoning Interview Rubric	520
Reasoning Formula Coverage	520
Reasoning Demo Coverage	520
Weak Reasoning Question	520
Reasoning Revision Plan	521
Search Reasoning	521
Search State	521
Search Action	521
Beam Search	521
Best-First Search	522
UCT	522
Prune False Negative	522
Search Budget	522
Search Audit	522
Tree-of-Thought	522
MCTS	523
Reflection	523
Benchmark	523
Benchmark Contamination	523
Human Evaluation	524
LLM-as-a-Judge	524
Elo Rating	524
Hallucination	524
Factuality	525
Citation Grounding	525
Robustness	525
Calibration	525
Evaluation Metric	525
Pass@k	526
Evaluation Slicing	526
Online Evaluation	526
Evaluation Pipeline	526
Evaluation Metric Incident	526
Aggregate Score Trap	527
Slice Regression	527
Clean Eval Lift	527
Judge-Human Agreement	527
Judge Length Bias	527
Evaluation Gate	527
本章小结	528
M. 安全与治理	528
AI Safety	528
Alignment	528
Alignment Problem	529
Outer Alignment	529
Inner Alignment	529
Goal Misgeneralization	529
Specification Gaming	530
Deceptive Alignment	530

Scalable Oversight	530
AI Feedback	530
Debate	530
Iterated Amplification	531
Recursive Reward Modeling	531
Oversight Gate	531
Helpful	531
Honest	531
Harmless	532
Safety Policy	532
Jailbreak	532
Prompt Injection	532
Indirect Prompt Injection	533
Instruction Hierarchy	533
Untrusted Content Boundary	533
Prompt Injection Gate	533
Data Exfiltration Rate	534
Unauthorized Tool Call Rate	534
Data Privacy	534
PII	534
Data Leakage	534
Memorization	535
Training Data Extraction	535
Membership Inference	535
PII Leakage Eval	535
Differential Privacy	535
DP-SGD	536
Watermarking	536
Watermark Detection	536
Content Credentials	536
Privacy Gate	536
Privacy Governance	536
Data Minimization Rate	537
PII Redaction Coverage	537
Sensitive Data Block Rate	537
Permission Isolation	537
Log Redaction Coverage	537
Training Consent Coverage	538
Retention Compliance Rate	538
Deletion SLA Pass Rate	538
External Transfer Approval Coverage	538
Audit Coverage	538
DPA Ready	539
Incident Response Ready	539
Minimal Reproduction	539
Evidence Coverage	539
Root Cause Rate	539
Postmortem Completeness	539
Debug Gate	539
Retrospective Gate	540
Hypothesis Coverage	540
Slice Analysis Coverage	540
Action Item Closure	540
Regression Verification Rate	540
Change Failure Rate	540

Privacy Governance Gate	540
Safety Compliance Incident	541
Unsafe Pass Rate	541
Safety Compliance Gate	541
Watermark Gate	541
Bias and Fairness	541
Toxicity	542
Red Teaming	542
Dangerous Capability Evaluation	542
Capability Elicitation	542
Red Team Regression Suite	543
Red Team Gate	543
Release Gate	543
Interpretability	543
Mechanistic Interpretability	543
Activation Patching	544
Sparse Autoencoder	544
Superposition	544
Feature Purity	544
Interpretability Gate	544
Steering	545
Representation Engineering	545
Steering Vector	545
Contrastive Activation Addition	545
Steering Gate	545
Model Editing	546
Knowledge Editing	546
ROME	546
MEMIT	546
Machine Unlearning	546
Forget Set	547
Retain Set	547
Editing Gate	547
Unlearning Gate	547
Safety Evaluation	547
Safety Gate	548
Severity-Weighted Risk	548
Over-Refusal	548
Under-Refusal	548
Content Moderation	548
Guardrail	549
Tool Safety	549
Agent Safety	549
Model Card	549
System Card	549
Model Governance	550
Governance Gate	550
Responsible Scaling	550
Risk Disclosure	550
Safety Interview Readiness	550
Safety Interview Rubric	551
Data Governance	551
Audit Log	551
Compliance	551
Responsible AI	551

安全与能力的权衡	552
本章小结	552
N. 重要论文索引	552
如何使用论文索引	552
基础架构	552
Attention Is All You Need	552
BERT	553
GPT 系列	553
T5	553
Scaling 与训练	554
Scaling Laws for Neural Language Models	554
Chinchilla	554
PaLM	554
LLaMA	554
后训练与对齐	555
InstructGPT	555
Constitutional AI	555
Direct Preference Optimization	555
RLHF	555
参数高效与系统优化	556
LoRA	556
QLoRA	556
FlashAttention	556
ZeRO	556
MoE 与稀疏模型	557
Switch Transformer	557
Mixtral	557
GShard	557
长上下文、RAG 与工具增强	557
RAG	557
REALM	558
RETRO	558
ReAct	558
Toolformer	558
Reasoning 与评估	558
Chain-of-Thought Prompting	558
Self-Consistency	558
GSM8K	559
多模态与生成	559
CLIP	559
ViT	559
Stable Diffusion	559
Whisper	559
安全与可解释性	559
Constitutional AI 作为安全论文	559
Red Teaming Language Models	560
Mechanistic Interpretability	560
论文阅读面试模板	560
精读优先级建议	560
本章小结	560
O. 面试高频对比表	561
如何回答对比题	561
Adam vs AdamW	561
SGD vs Adam	561

BatchNorm vs LayerNorm	562
LayerNorm vs RMSNorm	562
Pre-Norm vs Post-Norm	562
BERT vs GPT	562
Encoder-Decoder vs Decoder-Only	563
MHA vs MQA vs GQA	563
Absolute Position vs RoPE vs ALiBi	563
ReLU vs GELU vs SwiGLU	563
SFT vs RLHF vs DPO	563
RLHF vs DPO	564
RLHF vs RLAIF	564
Top-k vs Top-p	564
Greedy vs Beam Search vs Sampling	564
Temperature vs Top-p	565
LoRA vs Full Fine-Tuning	565
LoRA vs Adapter vs Prefix Tuning	565
Quantization vs Distillation	565
PTQ vs QAT	565
FP16 vs BF16	566
FlashAttention vs Sparse Attention	566
PagedAttention vs FlashAttention	566
KV Cache vs Activation Checkpointing	566
Data Parallel vs Tensor Parallel vs Pipeline Parallel	566
DDP vs ZeRO vs FSDP	566
RAG vs Fine-Tuning	567
Dense Retrieval vs Sparse Retrieval	567
Retriever vs Reranker	567
Long Context vs RAG	567
Agent vs Workflow	567
Chain-of-Thought vs Tool Use	568
Human Evaluation vs LLM-as-a-Judge	568
Accuracy vs Pass@k	568
Factuality vs Grounding	568
Robustness vs Generalization	568
Over-Refusal vs Under-Refusal	568
Jailbreak vs Prompt Injection	569
Helpful vs Harmless	569
Dense Model vs MoE	569
本章小结	569
P. 多模态学习	569
本章速览	570
Multimodal Learning	570
Modality Gap	571
Visual Token Budget	571
Multimodal Context Budget	572
Multimodal Cost Audit	572
Multimodal Product	572
Input Quality Pass Rate	572
Multimodal Evidence Support Rate	573
OCR Quality Score	573
ASR Quality Score	573
Generation Adoption Rate	573
Media Safety Pass Rate	573
Privacy Pass Rate	573
Copyright Pass Rate	573

Multimodal Product Gate	573
CLIP	574
CLIP Loss	574
Zero-Shot Classification	575
Image-Text Retrieval	575
Contrastive Learning	575
Image Encoder	576
Patch Embedding	576
CLS Token	577
Vision Encoder Shape Audit	577
Text Encoder	577
ViT	578
SigLIP	578
Vision-Language Model	578
Multimodal Alignment	579
Connector	579
VLM Connector Audit	580
Image Placeholder	580
Visual Token Compression	580
Assistant-Only Multimodal Loss	581
Q-Former	581
Perceiver Resampler	581
Flamingo	581
BLIP 与 BLIP-2	582
LLaVA	582
Image Captioning	583
Visual Question Answering	583
OCR	583
Grounding	584
Multimodal Instruction Tuning	584
Multimodal SFT Data Audit	585
Chat Template	585
Multimodal Task Coverage	585
Evidence Support Rate	586
Missing Refusal Rate	586
Multimodal Hallucination	586
Diffusion Model	587
DDPM	587
Forward Diffusion	587
Reverse Denoising	587
Noise Scheduler	588
Noise Prediction Loss	588
Classifier-Free Guidance	588
Latent Diffusion	589
Stable Diffusion	589
VAE Compression Ratio	589
Negative Prompt	589
Text-to-Image Pipeline Audit	590
ControlNet	590
Image-to-Image	590
Inpainting	590
DALL-E	590
Autoregressive Image Tokens	591
Video Generation	591
Spatiotemporal Patch	591

Video Diffusion	591
Temporal Consistency	591
Identity Drift	592
Flickering	592
World Model	592
Physics Consistency	592
Video Generation Evaluation	592
FVD	592
Video Token Audit	593
Audio Generation	593
Waveform	593
Log-Mel Spectrogram	593
ASR	593
Whisper	594
TTS	594
Vocoder	594
Voice Cloning	594
Audio Codec	594
Speech Token	594
Codec Language Model	594
Speech-to-Speech	595
VAD	595
WER	595
CER	595
MOS	595
Audio Token Audit	595
Unified Multimodal Model	596
Any-to-Any Multimodal Model	596
Unified Tokenization	596
Early-Fusion Multimodal Transformer	596
Center-LLM Multimodal System	596
Multimodal Router	596
Multimodal Loss Mixture	597
Modality Conflict	597
Unified Multimodal Audit	597
Multimodal Evaluation	597
Multimodal Incident	597
Input Fidelity Rate	598
Multimodal Evidence Recall	598
OCR Critical Field Accuracy	598
ASR Critical Field Accuracy	598
Temporal Evidence Recall	598
Multimodal Incident Gate	598
VQA Accuracy	598
Chart Relaxed Accuracy	599
Grounding IoU	599
Multimodal Hallucination Rate	599
Multimodal Prompt Injection	599
Biometric Identity Safety	600
Content Provenance	600
Multimodal Safety Audit	600
Multimodal Interview Readiness	600
Multimodal Interview Rubric	601
Multimodal Formula Coverage	601
Multimodal Demo Coverage	601

Weak Multimodal Question	601
Multimodal Revision Plan	601
多模态评估	602
多模态工程取舍	602
高频面试题	603
1. CLIP 为什么能做零样本分类?	603
2. CLIP 和 LLaVA 的区别是什么?	603
3. 为什么 VLM 需要 connector?	603
4. Q-Former 和简单 MLP projector 有什么区别?	603
5. 多模态幻觉怎么解决?	603
6. 多模态模型为什么 OCR 经常不好?	603
7. 视觉 token 太多怎么办?	603
8. 多模态指令微调数据怎么构造?	604
本章小结	604
Q. 图像生成与视频生成	604
本章速览	604
Diffusion Model	604
Forward Diffusion	605
Reverse Denoising	605
Noise Scheduler	606
U-Net	606
VAE	606
Latent Diffusion	607
Stable Diffusion	607
Classifier-Free Guidance	608
DALL·E	608
Autoregressive Image Generation	608
Text-to-Image	609
Image Editing	609
Video Diffusion	609
Video Token	610
Spatiotemporal Patch	610
Temporal Consistency	610
Motion Prior	611
Text-to-Video	611
World Model	612
Sora	612
图像生成评估	612
视频生成评估	613
安全与版权	613
工程取舍	613
高频面试题	614
1. 扩散模型和 GAN 有什么区别?	614
2. Stable Diffusion 为什么要在 latent 空间生成?	614
3. CFG 的作用是什么?	614
4. U-Net 和 DiT 有什么区别?	614
5. 视频生成为什么比图像生成难?	614
6. 什么是时序一致性?	614
7. Sora 的意义是什么?	614
8. 如何评估一个文生图模型?	614
9. 如何评估一个文生视频模型?	615
本章小结	615
R. 语音大模型	615
本章速览	615

ASR 615

Whisper 616

Mel Spectrogram 616

Speech Encoder 616

Speech Token 617

Audio Codec 617

Speech-to-Text 618

TTS 618

Text-to-Speech 618

Voice Cloning 619

Speaker Embedding 619

Prosody 619

Speech-to-Speech 620

End-to-End Voice Model 620

Streaming ASR 620

Multimodal Voice Agent 621

语音大模型评估 621

安全与合规 621

工程取舍 622

高频面试题 622

1. ASR 和 TTS 分别解决什么问题? 622

2. Whisper 的核心特点是什么? 622

3. Mel Spectrogram 为什么常用于语音模型? 622

4. 语音 token 有什么作用? 623

5. Audio codec 在语音大模型中为什么重要? 623

6. 级联语音系统和端到端语音模型怎么选? 623

7. Streaming ASR 的难点是什么? 623

8. Voice cloning 有哪些风险? 623

9. Prosody 为什么重要? 623

10. 如何评估语音 Agent? 623

本章小结 623

第四册简介：大模型百科全书

本书定位

第四册是术语、公式、方法和对比关系的速查工具书。

它不是按顺序阅读的课程，而是一个持续扩展的知识库。学习第一册、第二册、第三册或任何专题书时，遇到不熟悉的术语，都可以回到本书查阅。

为什么需要这本书

大模型领域术语密集，同一个概念还常常有不同论文、不同框架、不同工程系统中的叫法。百科全书的作用是建立统一词典，帮助读者快速确认定义、直觉、公式、适用场景和常见误区。

适合读者

- 1. 学习过程中需要快速查概念的读者。
- 2. 面试前需要复习术语和高频对比的候选人。
- 3. 想把零散概念串成知识网络的人。

学完后应具备的能力

- 1. 能快速解释大模型核心术语。

2. 能比较容易混淆的方法，例如 Adam/AdamW、MHA/MQA/GQA、SFT/DPO/RLHF。
3. 能把术语和对应章节、代码、面试问题连接起来。
4. 能形成自己的知识索引，而不是只靠搜索。

内容结构

1. 数学与机器学习。
2. 深度学习组件。
3. NLP 与 Tokenization。
4. Transformer 架构。
5. 大语言模型基础。
6. 预训练与数据。
7. 后训练与对齐。
8. 推理与解码。
9. 推理优化与部署。
10. 分布式训练系统。
11. 长上下文、RAG 与 Agent。
12. Reasoning 与评估。
13. 安全与治理。
14. 重要论文索引。
15. 面试高频对比表。
16. 多模态、图像生成、视频生成和语音大模型。

阅读建议

不要从头硬读。建议在学习正文时按需查阅，并把自己反复混淆的概念加入复习清单。

与其他书的关系

第四册是全书系的索引层。所有正式章节中的重要概念，最终都应该能在第四册中找到简洁条目。

第四册：大模型百科全书

本书定位

本书是“大模型算法岗系统面试二十四部曲”的第四册，定位是大模型领域的概念百科、公式速查、方法对比和面试词典。

阅读入口

1. [简介.md](#)：本书定位、适合读者、阅读建议和与其他书的关系。
2. 本文件：全书章节目录和阅读顺序。

章节文件

1. chapters/01-数学与机器学习.md
2. chapters/02-深度学习组件.md
3. chapters/03-nlp 与 tokenization.md
4. chapters/04-transformer 架构.md
5. chapters/05-大语言模型基础.md
6. chapters/06-预训练与数据.md
7. chapters/07-后训练与对齐.md
8. chapters/08-推理与解码.md
9. chapters/09-推理优化与部署.md

- 10. chapters/10-分布式训练系统.md
- 11. chapters/11-长上下文 rag 与 agent.md
- 12. chapters/12-reasoning 与评估.md
- 13. chapters/13-安全与治理.md
- 14. chapters/14-重要论文索引.md
- 15. chapters/15-面试高频对比表.md
- 16. chapters/16-多模态学习.md
- 17. chapters/17-图像生成与视频生成.md
- 18. chapters/18-语音大模型.md

A. 基础数学与机器学习

条目：线性代数、向量、矩阵乘法、内积、范数、投影、特征值、矩阵秩、SVD、低秩近似、PCA、低秩压缩、LoRA、QLoRA、概率论、随机变量、概率分布、条件概率、概率链式法则、期望、方差、信息论、熵、条件熵、互信息、交叉熵、KL 散度、负对数似然、困惑度、优化基础、最大似然估计、最大后验估计、贝叶斯公式、贝叶斯更新、不确定性、校准、ECE、Brier score、选择性预测、强化学习数学、MDP、return、value function、advantage、policy gradient、PPO clipped objective、RLHF KL penalty、reward model loss、DPO loss、梯度下降、SGD、Momentum、Adam、AdamW、学习率调度、梯度裁剪、Hessian、loss spike、反向传播、过拟合、泛化、正则化、Bias-Variance Trade-off。

线性代数 Linear Algebra

一句话定义：线性代数是研究向量、矩阵、线性变换、分解和几何关系的数学工具。

在大模型中：embedding 是向量，linear layer 是矩阵乘法，attention 是内积相似度和加权求和，LoRA 是低秩矩阵更新。

面试表达：讲线性代数不要停在定义，要主动连接到 QK^T shape、embedding 检索、gradient norm、SVD 和 LoRA 参数量。

向量 Vector

一句话定义：向量是一组有方向和大小的数，常用于表示 token、句子、图片或参数梯度。

在大模型中：一个 token hidden state 常写成 h_i in $\mathbb{R}^d$ ，其中 d 是 hidden size。

面试表达：向量方向常对应语义或特征组合，向量长度常对应尺度或激活强度；检索中常先归一化再比较方向。

内积 Dot Product

一句话定义：内积是两个向量对应元素相乘再求和，用于衡量匹配程度。

核心公式： $x^T y = \sum_i x_i y_i$ 。

在大模型中：attention score 来自 query 和 key 的内积；embedding 检索也常用 dot product 或归一化后的 dot product。

面试表达：dot product 同时受方向和长度影响；如果只想比较方向，要用 cosine similarity 或先做 L2 normalize。

向量范数 Vector Norm

一句话定义：范数衡量向量大小，最常见的是 L2 norm。

核心公式： $\|x\|_2 = \sqrt{\sum_i x_i^2}$ 。

在大模型中：gradient norm 用于训练稳定性监控，embedding norm 影响检索相似度，RMSNorm / LayerNorm 也和尺度控制有关。

面试表达: gradient norm 异常升高常提示学习率、数据、loss mask、数值精度或分布式状态需要排查。

矩阵乘法 Matrix Multiplication

一句话定义: 矩阵乘法把一组向量线性变换到新的空间, 关键是中间维度必须匹配。

常见 shape: $X [B, d_{in}] @ W [d_{in}, d_{out}] \rightarrow Y [B, d_{out}]$ 。

在大模型中: Q/K/V projection、MLP up/down projection、输出词表投影和张量并行都围绕大矩阵乘法展开。

面试表达: Transformer shape 推导的核心是矩阵乘法。 $Q [B, H, T, d_h] @ K^T [B, H, d_h, T]$ 得到 $[B, H, T, T]$ 。

投影 Projection

一句话定义: 投影表示一个向量在某个方向或子空间上的分量。

核心公式: 如果 u 不是零向量, $\text{proj}_u(x) = (x^T u / u^T u)u$ 。

在大模型中: PCA、表示分析、steering direction、低秩近似和 embedding 可视化都需要投影直觉。

面试表达: 投影帮助解释 “高维表示里哪些方向最重要”, 但不能把任意方向都解释成稳定语义。

特征值与特征向量 Eigenvalue and Eigenvector

一句话定义: 如果 $Av = \lambda v$, 则 v 是矩阵 A 的特征向量, λ 是对应特征值。

直觉: 矩阵作用在特征向量方向上时, 只改变长度, 不改变方向。

在大模型中: 特征值直觉常出现在 PCA、优化曲率、Hessian 分析、稳定性和表示空间主方向中。

面试表达: 特征值不用背得很抽象, 重点说清 “哪些方向被放大或压缩”。

矩阵秩 Matrix Rank

一句话定义: 矩阵秩表示矩阵中线性独立方向的数量。

直觉: 如果某些行或列可以由其他行列线性组合得到, 它们没有提供新的独立方向, rank 会更低。

在大模型中: rank 直觉用于理解低秩近似、PCA、LoRA rank、模型压缩和表示空间自由度。

奇异值分解 Singular Value Decomposition / SVD

一句话定义: SVD 把矩阵分解成 $W = U \Sigma V^T$, 其中奇异值表示不同方向的重要程度。

关键指标: rank- r 近似保留前 r 个最大奇异值; Frobenius 误差由被丢弃奇异值平方和决定, 能量保留率可写成 $\sum_{i \leq r} \sigma_i^2 / \sum_i \sigma_i^2$ 。

在大模型中: SVD 可用于理解低秩近似、表示压缩、PCA 直觉和 LoRA 的低秩更新假设。

面试表达: SVD 说明矩阵信息可能集中在少数重要方向上, 因此可以用 rank- r 近似保留主要结构。

低秩近似 Low-Rank Approximation

一句话定义: 低秩近似用更少的方向近似原矩阵, 减少参数量、存储或计算。

常见形式: $W_r = U_r \Sigma_r V_r^T$ 。

在大模型中: LoRA 用 $\Delta W = A B$ 约束微调更新位于低秩子空间, 参数量从 $d_{in} * d_{out}$ 降到 $r * (d_{in} + d_{out})$ 。

面试表达: 低秩不是免费午餐。rank 太小表达能力不足, rank 太大参数节省变少, 也可能更容易过拟合。

主成分分析 Principal Component Analysis / PCA

一句话定义：PCA 寻找数据中方差最大的正交方向，并把数据投影到这些方向上。

核心口径：先中心化数据，再分析协方差矩阵或对中心化数据矩阵做 SVD；explained variance ratio 表示保留主方向解释了多少方差。

在大模型中：PCA 常用于 embedding / hidden state 可视化、表示分析、异常点探索和微调前后表示变化观察。

常见误区：PCA 是线性分析工具，二维可视化不能单独证明模型理解或真实能力。

低秩压缩 Low-Rank Compression

一句话定义：用两个较小矩阵近似原始权重矩阵，减少参数量或计算量。

和 LoRA 的区别：低秩压缩近似原始 W ，可能直接损伤原模型能力；LoRA 通常冻结原始 W ，只用低秩方式学习增量 ΔW 。

LoRA Low-Rank Adaptation

一句话定义：LoRA 冻结原始权重，只训练低秩增量 $\Delta W = A B$ 。

参数量： $r * (d_{in} + d_{out})$ ，当 r 远小于 d_{in} 和 d_{out} 时，可训练参数和优化器状态大幅减少。

面试表达：LoRA 不是让原权重低秩，而是假设下游任务的权重更新可以低秩表示。

QLoRA Quantized LoRA

一句话定义：QLoRA 把 base model 量化存储，同时训练 LoRA adapter。

在大模型中：它适合显存受限微调，但要额外关注量化误差、计算 dtype、scale 开销、activation 峰值和任务质量回归。

概率论 Probability Theory

一句话定义：概率论是描述不确定事件、随机变量和概率分布的数学工具。

在大模型中：下一个 token 是随机变量，模型输出条件概率分布，训练最大化真实 token 概率，解码从分布中选择 token。

面试表达：LLM 的训练、解码、PPL、uncertainty、calibration 和 hallucination 都离不开概率论。概率高表示“在模型分布下更可能”，不等于事实必真。

随机变量 Random Variable

一句话定义：随机变量把不确定结果映射成可以计算的数学对象。

在语言模型中： x_t 可以表示第 t 个位置的 token，取值范围是整个 vocabulary。

面试表达：给定上下文后，模型预测的不是一个固定 token，而是 x_t 在词表上的条件概率分布。

概率分布 Probability Distribution

一句话定义：概率分布给出每个可能结果发生的概率，离散分布要求概率非负且总和为 1。

在大模型中：logits 经过 softmax 后成为 token 分布；temperature、top-k 和 top-p 都是在这个分布上做解码控制。

面试表达：logits 不是概率。softmax 才把它们变成可采样、可算 NLL 和 entropy 的概率分布。

条件概率 Conditional Probability

一句话定义：条件概率描述在已知事件 B 后事件 A 发生的概率。

核心公式： $P(A | B) = P(A, B) / P(B)$ 。

在语言模型中：核心对象是 $P(x_t | x_{1:t-1})$ ，即给定前文预测下一个 token。

面试表达：语言模型不是学 token 的无条件频率，而是学上下文条件下的 token 分布。

概率链式法则 Chain Rule of Probability

一句话定义：任意联合概率都可以分解成一系列条件概率的乘积。

核心公式： $P(x_{1:T}) = \prod_t P(x_t | x_{1:t-1})$ 。

在大模型中：自回归语言模型正是利用链式法则，把长序列建模拆成逐 token 的 next-token prediction。

面试表达：next-token prediction 不是经验口号，而是概率链式法则给出的序列建模方式。

期望 Expectation

一句话定义：期望是随机变量在长期平均意义下的值。

核心公式：离散情况下 $E[X] = \sum_x x P(X=x)$ 。

在大模型中：训练目标可以看作数据分布上的期望损失，mini-batch loss 是对这个期望的估计。

面试表达：batch size 影响梯度噪声，因为 mini-batch 只是对总体期望的近似。

方差 Variance

一句话定义：方差衡量随机变量围绕均值波动的程度。

核心公式： $\text{Var}(X) = E[X^2] - E[X]^2$ 。

在大模型中：mini-batch 梯度、随机种子、采样解码、人评和 benchmark 子样本都会带来方差。

面试表达：一次实验提升不一定可靠，要结合样本量、置信区间、随机种子和错误分析判断。

信息论 Information Theory

一句话定义：信息论把概率分布和不确定性连接起来，回答“一个事件多意外、一个分布多不确定、两个分布差多少、一个变量能减少另一个变量多少不确定性”。

在大模型中：信息论直接对应 token NLL、cross entropy loss、perplexity、KL penalty、DPO reference model、uncertainty、RAG 证据质量和多模态表示对齐。

面试表达：讲信息论不要只背公式，要主动连接 $-\log p(\text{token})$ 、 $\text{PPL} = \exp(\text{avg NLL})$ 、 $H(p, q) = H(p) + \text{KL}(p || q)$ 、RLHF 的 KL 约束和 RAG 中“证据能否降低答案不确定性”。

交叉熵 Cross Entropy

一句话定义：交叉熵衡量用模型分布 q 表示真实分布 p 时的平均代价。

核心公式： $H(p, q) = -\sum_x p(x) \log q(x)$ 。

在语言模型中：如果真实标签是 one-hot，交叉熵就是 $-\log P(\text{真实下一个 token})$ 。

与 KL 关系： $H(p, q) = H(p) + \text{KL}(p || q)$ ；真实数据分布固定时，最小化交叉熵等价于让模型分布更接近真实分布。

为什么需要它：语言模型训练要让真实 token 的概率更高，交叉熵正好提供这个优化目标。

常见误区：交叉熵低不等于模型回答一定真实、有用或安全。

最大似然估计 Maximum Likelihood Estimation / MLE

一句话定义：选择让观测数据概率最大的模型参数。

在语言模型中：最大化训练文本的概率 $P_{\theta}(\text{data})$ 。

与交叉熵关系：最大化 log likelihood 等价于最小化 negative log likelihood；在 one-hot 标签下就是最小化交叉熵。

负对数似然 Negative Log-Likelihood / NLL

一句话定义：对真实数据概率取负对数得到的损失。

核心公式： $NLL = -\sum \log P_{\theta}(x_t | x_{1:t-1})$ 。

为什么使用 log：概率连乘容易下溢，取 log 后把乘法变成加法，数值更稳定。

KL 散度 KL Divergence

一句话定义：衡量两个概率分布差异的非对称指标。

核心公式： $KL(p || q) = \sum_x p(x) \log(p(x) / q(x))$ 。

与交叉熵关系： $KL(p || q) = H(p, q) - H(p)$ 。

在大模型中：RLHF 常用 KL penalty 限制 policy 偏离 reference；DPO 虽不显式跑 PPO，也通过 reference model 约束偏好变化。

面试追问：为什么最小化交叉熵等价于最小化 KL？为什么 KL 方向不同会带来不同优化行为？

困惑度 Perplexity

一句话定义：语言模型平均交叉熵的指数形式，用来衡量模型对文本的困惑程度。

核心公式：如果 loss 使用自然对数， $\text{perplexity} = \exp(\text{loss})$ 。

直觉：perplexity 越低，模型对下一个 token 越确定。

局限：perplexity 受 tokenizer、验证集、截断方式、loss mask 和污染影响，不直接等价于下游任务能力、事实性、安全性或用户满意度。

梯度 Gradient

一句话定义：梯度表示函数对参数变化的敏感程度，以及函数值上升最快的方向。

在训练中：梯度告诉我们参数如何变化会让 loss 增大，梯度下降则沿负梯度方向更新参数。

相关概念：learning rate、backpropagation、gradient norm。

梯度下降 Gradient Descent

一句话定义：沿着 loss 负梯度方向更新参数的优化方法。

核心公式： $\theta_{\text{new}} = \theta_{\text{old}} - \eta * \nabla_{\theta} L$ 。

常见问题：学习率太大可能发散，太小训练缓慢。

优化基础 Optimization Basics

一句话定义：优化基础研究如何根据 loss、梯度和优化器状态更新模型参数。

在大模型中：预训练、SFT、DPO 和 reward model 训练都依赖优化；常见稳定性问题包括 learning rate 过大、warmup 太短、grad norm 爆炸、optimizer state 恢复错误和 mixed precision overflow。

面试表达：不要只说“用 AdamW”。要能解释梯度、学习率、global batch、momentum、一阶矩 / 二阶矩、decoupled weight decay、warmup、gradient clipping 和 loss spike 排查链路。

SGD Stochastic Gradient Descent

一句话定义：SGD 用 mini-batch 梯度近似全量梯度，降低每步计算成本。

在大模型中：即使用 AdamW，梯度本身也来自 mini-batch 估计；global batch size 会影响梯度噪声、吞吐和 learning rate 选择。

面试误区：单卡 micro batch 不等于 global batch；global batch 还要乘以 gradient accumulation steps 和 data parallel worker 数。

Momentum

一句话定义：Momentum 用历史梯度的移动平均平滑更新方向。

核心直觉：连续同向梯度会累积加速，来回震荡的梯度会被抵消一部分。

在大模型中：Adam 的一阶矩可以看作 momentum 思想的自适应版本。

Adam

一句话定义：Adam 同时维护梯度一阶矩和二阶矩，用自适应缩放更新参数。

核心机制：一阶矩决定较稳定的方向，二阶矩估计梯度尺度，梯度大的方向步长被缩小。

常见误区：Adam 不是自动免调参。学习率、warmup、weight decay、epsilon 和 batch size 仍会影响稳定性。

AdamW

一句话定义：AdamW 是把 weight decay 从 Adam 梯度更新中解耦的优化器。

核心区别：Adam + L2 会把衰减项混入梯度并被自适应缩放；AdamW 单独对参数做衰减，更符合 weight decay 的正则化直觉。

在大模型中：AdamW 是预训练和微调常见默认选择，但通常不对 bias、LayerNorm / RMSNorm 参数做 weight decay。

学习率调度 Learning Rate Schedule

一句话定义：学习率调度控制不同训练阶段每一步的学习率。

常见组合：linear warmup 后接 cosine decay 或 linear decay。

在大模型中：warmup 缓解早期不稳定，decay 让后期小步收敛；schedule 要和总 token、batch size、optimizer、数据质量一起调。

反向传播 Backpropagation

一句话定义：利用链式法则从 loss 反向计算每个参数梯度的方法。

为什么需要它：神经网络是复杂计算图，手动求每个参数梯度不可行，反向传播可以高效自动计算。

链式法则 Chain Rule

一句话定义：复合函数求导规则。

核心公式：如果 $y = f(z)$ 且 $z = g(w)$ ，则 $dy/dw = dy/dz * dz/dw$ 。

在神经网络中：反向传播就是链式法则在计算图上的系统应用。

梯度消失 Vanishing Gradient

一句话定义：梯度在反向传播过程中逐层变小，导致底层参数几乎无法更新。

常见缓解：残差连接、归一化、合理初始化。

梯度爆炸 Exploding Gradient

一句话定义：梯度在反向传播过程中变得过大，导致参数更新不稳定甚至 NaN。

常见缓解：gradient clipping、learning rate warmup、数值精度检查。

Gradient Clipping

一句话定义：限制梯度范数，防止梯度爆炸导致训练不稳定的方法。

核心公式：若 global grad norm 超过阈值，就按比例缩小所有梯度，使裁剪后范数不超过阈值。

局限：它能缓解异常大梯度，但不能替代对数据、loss 和学习率问题的根因排查。

Hessian

一句话定义：Hessian 是 loss 对参数的二阶导数矩阵，描述 loss surface 的曲率。

在大模型中：通常不会显式计算完整 Hessian，但曲率直觉能解释为什么高曲率方向对 learning rate 敏感，为什么过大学习率会震荡。

Loss Spike

一句话定义：loss spike 是训练过程中 loss 突然异常升高的现象。

优化排查：对齐 spike step 的 learning rate、warmup、grad norm、clipping、AdamW state、mixed precision scaler、异常 batch 和 checkpoint resume 状态。

熵 Entropy

一句话定义：熵衡量一个概率分布的不确定性。

核心公式： $H(p) = -\sum_x p(x) \log p(x)$ 。

直觉：如果一个分布非常确定，例如某个事件概率接近 1，熵就低；如果很多事件概率接近均匀，熵就高。

在语言模型中：下一个 token 分布越分散，熵越高；模型越不确定，生成越容易出现多样性，也更容易不稳定。

为什么重要：采样温度、top-k、top-p、困惑度、校准和不确定性分析都和熵有关。

常见误区：熵高不一定坏。开放式创作需要一定熵；事实问答、代码生成和结构化抽取通常希望输出分布更集中。

面试表达：熵描述的是分布本身的不确定性，交叉熵描述的是用一个分布去编码另一个分布时的代价，KL 散度描述两个分布之间的额外代价。

条件熵 Conditional Entropy

一句话定义：条件熵衡量在已知变量 Y 后，变量 X 仍然剩余的不确定性。

核心公式： $H(X | Y) = -\sum_{x,y} p(x, y) \log p(x | y)$ 。

直觉：如果 Y 能很好预测 X ，那么 $H(X | Y)$ 就低；如果 Y 对 X 几乎没有帮助，条件熵就接近 $H(X)$ 。

在语言模型中：给定上下文 $x_{\{1:t-1\}}$ 后，下一个 token x_t 的不确定性可以理解为一种条件熵。

面试追问：语言模型为什么建模 $P(x_t | x_{\{1:t-1\}})$ ？因为自然语言序列可以按链式法则分解为一系列条件概率。

互信息 Mutual Information

一句话定义：互信息衡量两个随机变量之间共享的信息量。

核心公式： $I(X; Y) = H(X) - H(X | Y)$ 。

等价形式： $I(X; Y) = KL(p(x, y) || p(x)p(y))$ 。

直觉：如果知道 Y 能显著减少对 X 的不确定性，互信息就大。

在大模型中：互信息思想常出现在表示学习、对比学习、检索增强、特征选择和数据去重中。

RAG 直觉：一个 chunk 和 query 词面相似不够；它必须真正减少答案的不确定性，才算提供高信息量证据。

常见误区：互信息是对称的， $I(X; Y) = I(Y; X)$ ；KL 散度一般不是对称的。

JS 散度 Jensen-Shannon Divergence

一句话定义：JS 散度是 KL 散度的对称、平滑版本，用于衡量两个分布差异。

核心公式： $JS(p || q) = 1/2 KL(p || m) + 1/2 KL(q || m)$ ，其中 $m = (p + q) / 2$ 。

特点：相对 KL 更稳定，并且是对称的。

在生成模型中：GAN 原始目标可以和 JS 散度联系起来；分布对齐、数据分布比较时也可能提到它。

和 KL 的区别：KL 关注用 q 近似 p 的额外代价，方向不同含义不同；JS 同时考虑两边，更适合做对称分布差异描述。

最大后验估计 Maximum A Posteriori / MAP

一句话定义：MAP 在观测数据之外引入参数先验，选择后验概率最大的参数。

核心公式： $\theta_{MAP} = \operatorname{argmax}_{\theta} p(\theta | D) = \operatorname{argmax}_{\theta} p(D | \theta)p(\theta)$ 。

和 MLE 的区别：MLE 只最大化数据似然 $p(D | \theta)$ ；MAP 还考虑先验 $p(\theta)$ 。

在机器学习中：很多正则化可以从 MAP 角度解释。例如 L2 正则可以看作对参数加入高斯先验，L1 正则可以看作加入拉普拉斯先验。

面试表达：MLE 更依赖数据本身，MAP 通过先验限制参数空间，在数据较少或需要正则化时更稳健。

常见误区：MAP 不是贝叶斯推断的全部。它只取后验最大点，并没有完整保留后验分布的不确定性。

贝叶斯公式 Bayes' Theorem

一句话定义：贝叶斯公式描述如何用观测数据更新对假设的信念。

核心公式： $p(\theta | D) = p(D | \theta)p(\theta) / p(D)$ 。

概念对应： $p(\theta)$ 是先验， $p(D | \theta)$ 是似然， $p(\theta | D)$ 是后验， $p(D)$ 是证据。

在大模型中：虽然主流 LLM 训练通常不是完整贝叶斯训练，但贝叶斯思想有助于理解不确定性、校准、先验、正则化和在线更新。

面试追问：为什么后验正比于似然乘先验？因为 $p(D)$ 对参数优化来说是常数。

先验 Prior

一句话定义：先验是在看到当前证据之前，对假设或参数的已有信念。

在大模型中：模型参数、训练数据分布、系统默认策略和历史评估结果都可以提供某种先验，但先验不能替代当前证据。

面试表达：LLM 的语言先验能生成合理文本，但事实性问题还需要 RAG、工具或引用证据来更新判断。

似然 Likelihood

一句话定义：似然衡量在某个假设成立时，观察到当前证据的可能性。

核心口径：Bayes rule 中 $p(E | H)$ 越高，说明证据 E 越支持假设 H。

在大模型中：检索到强证据、工具校验通过、引用支持结论，都可以提高“回答可靠”这个假设的似然。

后验 Posterior

一句话定义：后验是结合先验和证据之后，对假设的更新信念。

核心口径：posterior proportional to likelihood times prior。

在大模型中：可靠系统不应只看模型原始回答，而应结合证据质量、风险等级和校准结果决定回答、验证、澄清或拒答。

贝叶斯更新 Bayesian Update

一句话定义：贝叶斯更新是用新证据把先验变成后验的过程。

常见例子：如果某类任务先验可靠率是 70%，RAG 找到强支持证据并且引用检查通过，后验可靠率应上升；如果检索为空或证据冲突，后验可靠率应下降。

面试表达：Bayesian update 在 LLM 系统里更多是一种决策框架，不要求整个模型训练都是贝叶斯式的。

不确定性 Uncertainty

一句话定义：不确定性描述模型或系统对当前判断缺少把握的程度。

常见来源：输入歧义、数据噪声、模型知识不足、分布外问题、证据缺失、工具失败和 judge 偏差。

在大模型中：不确定性管理决定模型什么时候回答、检索、调用工具、澄清、拒答或交给人工。

Aleatoric Uncertainty

一句话定义：数据本身随机性、噪声或歧义带来的不确定性。

例子：用户问题含糊、标注多解、图片模糊、语音噪声大。

处理方式：澄清问题、表达条件、输出多个可能答案或降低结论强度。

Epistemic Uncertainty

一句话定义：模型知识不足、数据覆盖不足或分布外输入带来的不确定性。

例子：模型没见过某个新领域、RAG 没召回证据、工具结果缺失、任务超出能力边界。

处理方式：检索、工具验证、人审、拒答或补充训练数据。

Bias-Variance Trade-off

一句话定义：偏差-方差权衡描述模型误差来源于欠拟合和对训练数据扰动敏感两方面。

偏差：模型假设太简单，无法拟合真实规律。

方差：模型对训练数据变化过于敏感，容易过拟合。

典型现象：小模型可能高偏差、低方差；大模型可能低偏差、高方差，但现代深度学习中还存在双下降等更复杂现象。

平方损失下常见分解： $\text{expected error} = \text{bias}^2 + \text{variance} + \text{noise}$ 。这个公式用于定位误差来源，不表示真实项目能直接精确分解每个 LLM 输出。

在大模型中：模型规模、数据规模、正则化、训练轮数、数据质量都会影响泛化。大模型并不自动泛化好，仍需要数据治理、评估集和对齐过程。

面试表达：偏差-方差权衡不是让模型越小越好，而是提醒我们从模型容量、数据量和正则化三个角度分析泛化问题。

过拟合 Overfitting

一句话定义：模型在训练集上表现很好，但在未见数据上表现明显变差。

常见诊断：训练分数高、验证分数低，或训练 loss 下降但验证 loss / 人评 / 线上指标变差。若用 accuracy，可看 $G_score = S_train - S_val$ 。

常见原因：模型容量过大、训练数据太少、数据噪声大、训练轮数过多、验证集泄漏、评估集与真实分布不一致。

在大模型微调中：过拟合可能表现为训练 loss 持续下降，但验证集 loss 或真实任务指标变差；也可能表现为回答风格僵硬、记忆训练样例、泛化能力下降。

常见缓解：更多高质量数据、early stopping、weight decay、dropout、数据增强、降低学习率、减少训练轮数、混入通用数据。

面试表达：过拟合不能只看训练 loss，要看验证集和真实任务切片；对大模型还要看是否出现能力退化和安全行为变化。

欠拟合 Underfitting

一句话定义：模型连训练集都学不好，说明模型能力、特征、训练配置或数据处理存在问题。

常见原因：模型太小、训练不够、学习率不合适、特征不足、数据标签噪声过大、优化器配置错误。

在大模型中：微调欠拟合可能表现为模型仍然像 base model 一样回答，不遵循目标格式，目标任务指标没有提升。

排查方向：先确认数据格式、label mask、loss 是否正确，再看学习率、batch size、训练步数和 LoRA rank 等配置。

和过拟合区别：欠拟合是训练集和验证集都差；过拟合是训练集好、验证集差。

泛化 Generalization

一句话定义：泛化指模型在未见样本、未见场景或真实分布上的表现能力。

核心公式：真实风险 $R(f) = E_{(x,y) \sim P}[\ell(f(x), y)]$ ，经验风险 $R_{\text{hat}}(f) = 1/n \sum_i \ell(f(x_i), y_i)$ ，泛化问题关心两者之间的差距。

为什么重要：面试和线上系统关心的不是训练集表现，而是模型在新问题上的可靠性。

在大模型中：泛化包括任务泛化、领域泛化、长度泛化、指令泛化、格式泛化和安全泛化。

评估方式：使用独立测试集、时间切分测试集、跨领域测试集、难例集和线上 A/B 测试。

常见误区：benchmark 高分不等于真实业务泛化好。benchmark 可能被污染，也可能和业务分布差异很大。

正则化 Regularization

一句话定义：正则化通过限制模型复杂度或训练行为，降低过拟合风险。

核心形式： $L_{\text{reg}}(\theta) = R_{\text{hat}}(f_{\theta}) + \lambda ||\theta||_2^2$ 。正则项越强，模型越不容易贴合训练噪声，但过强会欠拟合。

常见方法：L1、L2、weight decay、dropout、early stopping、数据增强、label smoothing、噪声注入。

在大模型中：常见正则化包括 weight decay、dropout、数据混合、训练步数控制、KL 约束和 replay 数据。

为什么需要：模型容量越大，越容易记住训练数据中的噪声或偏差，正则化提供约束。

常见误区：正则化不是越强越好。过强正则可能导致欠拟合或目标任务学不进去。

L1 正则 Lasso Regularization

一句话定义：L1 正则是在 loss 中加入参数绝对值之和，鼓励参数稀疏。

核心形式： $L_{\text{total}} = L_{\text{task}} + \lambda ||\theta||_1$ 。

特点：容易产生稀疏解，即很多参数变成 0。

适用场景：传统机器学习中的特征选择、稀疏线性模型。

在大模型中：直接对全部参数使用 L1 并不常见，但稀疏化、剪枝和结构化压缩中有类似思想。

L2 正则 Ridge Regularization

一句话定义：L2 正则是在 loss 中加入参数平方和，鼓励参数不要过大。

核心形式： $L_{\text{total}} = L_{\text{task}} + \lambda ||\theta||_2^2$ 。

特点：让参数更平滑、更小，但通常不会精确变成 0。

和 weight decay 关系：在普通 SGD 下，L2 正则和 weight decay 形式接近；在 Adam 这类自适应优化器中，AdamW 的 decoupled weight decay 更常用。

面试高频点：AdamW 相比 Adam + L2 的关键差异是把权重衰减从梯度更新中解耦出来。

Dropout

一句话定义：Dropout 在训练时随机屏蔽部分神经元或激活，降低模型对特定路径的依赖。

直觉：相当于训练很多共享参数的子网络，提升鲁棒性。

训练和推理区别：训练时随机 dropout；推理时通常关闭 dropout，并使用缩放后的整体网络。

在 Transformer 中：dropout 可能出现在 attention weight、residual branch、MLP 激活等位置。

常见误区：大模型训练中 dropout 不一定总是开启或很大，具体取决于模型规模、数据规模和训练 recipe。

Early Stopping

一句话定义：当验证集指标不再提升时提前停止训练，避免继续训练导致过拟合。

适用场景：监督微调、小数据训练、传统深度学习任务。

在大模型微调中：如果训练 loss 下降但验证任务指标开始变差，可以考虑 early stopping。

注意事项：验证集必须干净、独立，并且和目标任务分布一致；否则 early stopping 的判断会误导训练。

校准 Calibration

一句话定义：校准衡量模型输出概率和真实正确率是否一致。

例子：如果模型对一批样本都给出 80% 置信度，那么其中大约 80% 应该真的正确，这才是校准好的模型。

为什么重要：在安全、医疗、金融、拒答、路由和人工兜底场景中，模型不仅要回答，还要知道自己不确定。

常见方法：temperature scaling、Platt scaling、isotonic regression、置信度阈值、选择性预测。

在 LLM 中的困难：生成概率不等价于事实正确概率；模型可能高置信度胡说，也可能低概率生成正确答案。

面试表达：校准关注的是概率可信度，不是单纯准确率。一个模型可以准确率高但校准差。

Expected Calibration Error / ECE

一句话定义：ECE 把样本按置信度分桶，衡量每个桶的平均置信度和真实准确率之间的加权差距。

核心形式：
$$ECE = \sum_b |S_b| / n * |acc(S_b) - conf(S_b)|。$$

局限：ECE 依赖分桶方式、样本量和置信度定义。LLM 的自然语言置信表达不能直接当概率。

面试表达：ECE 可以发现“模型说 90% 把握，但真实只有 70% 正确”这类过度自信问题。

Brier Score

一句话定义：Brier score 衡量概率预测和 0/1 真实标签之间的均方误差。

核心形式：
$$Brier = 1/n \sum_i (p_i - y_i)^2。$$

用途：常用于二分类概率质量评估，既反映准确性，也反映概率校准。

选择性预测 Selective Prediction / Abstention

一句话定义：模型在低置信或高风险时选择不自动回答，而是澄清、检索、验证、拒答或转人工。

在大模型中：高风险医疗、法律、金融、权限操作和安全请求不能只按最高概率 token 自动回答，要结合证据和风险门禁。

面试表达：可靠系统不是永远回答，而是在证据不足或风险过高时知道如何停止、验证或升级。

置信度 Confidence

一句话定义：置信度是模型对某个预测的自信程度。

在分类中：通常用最大类别概率作为置信度。

在语言模型中：可以用 token 概率、序列 logprob、熵、self-consistency 一致性、外部 verifier 分数等近似衡量。

局限：LLM 的自然语言“我很确定”不一定可信，生成文本中的自信语气也不等于真实置信度。

工程用法：低置信度时可以拒答、要求用户补充信息、触发检索、调用工具或转人工。

ROC-AUC

一句话定义：ROC-AUC 衡量二分类模型在不同阈值下区分正负样本的能力。

ROC 曲线：横轴是假阳性率 FPR，纵轴是真阳性率 TPR。

AUC 直觉：随机抽一个正样本和一个负样本，模型给正样本更高分的概率。

适用场景：类别不平衡时比单一 accuracy 更稳定，但仍要结合业务阈值看 precision 和 recall。

在大模型中：可用于 reward model、分类器、风险识别、安全检测器等二分类或打分任务。

Precision / Recall / F1

一句话定义：Precision 关注预测为正的样本有多少是真的正，Recall 关注真实正样本有多少被找出来，F1 是二者调和平均。

核心公式：Precision = $TP / (TP + FP)$, Recall = $TP / (TP + FN)$, $F1 = 2PR / (P + R)$ 。

适用场景：分类、抽取、检索、风险识别、工具调用成功检测。

权衡：提高阈值通常提升 precision、降低 recall；降低阈值通常提升 recall、降低 precision。

面试表达：指标选择要看业务代价。安全审核可能更重 recall，自动执行工具可能更重 precision。

Accuracy

一句话定义：Accuracy 是预测正确样本占总样本的比例。

核心公式：Accuracy = correct / total。

优点：简单直观。

局限：类别极不平衡时可能误导。例如 99% 都是负样本，模型全预测负也有 99% accuracy。

在大模型评估中：选择题、分类任务、格式固定任务可以用 accuracy；开放式生成不能只用 accuracy。

Label Smoothing

一句话定义：Label smoothing 把 one-hot 标签变软，避免模型对训练标签过度自信。

核心思想：真实类别不再是 1，其他类别不再是 0，而是分配一个很小概率。

优点：改善泛化和校准，降低过拟合到噪声标签的风险。

局限：如果任务需要极强置信度或标签完全确定，过强 label smoothing 可能伤害性能。

在语言模型中：部分训练 recipe 可能使用 label smoothing，但主流 causal LM 训练通常重点还是 next-token cross entropy。

数据泄漏 Data Leakage

一句话定义：训练过程或特征中包含了测试时不应该知道的信息，导致评估结果虚高。

常见形式：测试集混入训练集、时间穿越、标签泄漏、重复样本、benchmark 污染、prompt 中意外包含答案。

在大模型中：预训练语料可能包含 benchmark；微调数据可能和评估集重复；RAG 检索库可能直接包含标准答案。

危害：模型看似效果很好，但上线后真实表现下降。

排查方法：去重、时间切分、相似度检索、数据来源审计、评估集隔离。

数据分布偏移 Distribution Shift

一句话定义：训练数据分布和测试或线上数据分布不一致。

常见诊断：验证集分数高但线上目标分布分数低，可用 $\Delta_{score} = S_{val} - S_{target}$ 近似衡量性能掉落。

常见类型：领域偏移、时间偏移、用户群体偏移、输入格式偏移、语言风格偏移、难度偏移。

在大模型中：模型在公开 benchmark 上表现好，但在企业内部文档、方言语音、低质量图片或垂直领域术语上表现差，就是分布偏移。

缓解方法：收集目标域数据、持续评估、领域自适应、数据增强、RAG、在线监控。

面试表达：泛化问题通常不是单纯模型问题，也可能是数据分布和评估设计问题。

损失函数 Loss Function

一句话定义：损失函数把模型预测和目标之间的差异转化为可优化的标量。

常见类型：交叉熵、MSE、hinge loss、contrastive loss、ranking loss、DPO loss、reward model loss。

在大模型中：预训练常用 next-token cross entropy；偏好优化可能使用 DPO loss、PPO 目标或 reward ranking loss。

为什么重要：损失函数决定了模型被优化成什么样，而评估指标决定了我们真正关心什么。二者不一致时需要特别小心。

经验风险最小化 Empirical Risk Minimization / ERM

一句话定义：ERM 在训练集上最小化平均损失，用训练数据近似真实分布风险。

核心形式： $\theta_{\hat{}} = \arg \min_{\theta} \frac{1}{n} \sum_i L(f_{\theta}(x_i), y_i)$ 。

直觉：真实数据分布不可完全知道，所以用训练集样本估计期望损失。

局限：如果训练集不代表真实分布，ERM 学到的模型可能泛化不好。

在大模型中：预训练、SFT、分类器训练都可以看作不同形式的经验风险最小化。

优化器 Optimizer

一句话定义：优化器根据梯度和内部状态决定如何更新参数。

常见优化器：SGD、Momentum、Adagrad、RMSProp、Adam、AdamW。

在大模型中：AdamW 是非常常见的训练优化器，因为它结合了一阶矩、二阶矩自适应更新和解耦 weight decay。

面试重点：优化器不仅影响收敛速度，也影响稳定性、泛化和显存占用。

学习率 Learning Rate

一句话定义：学习率控制每次参数更新的步长。

过大：训练可能发散、loss 震荡、出现 NaN。

过小：训练很慢，可能学不到目标任务。

在大模型中：学习率通常配合 warmup、cosine decay、linear decay 使用；微调学习率通常比从头预训练小得多。

面试表达：学习率是训练最敏感的超参数之一，debug 训练不稳定时应优先检查。

Warmup

一句话定义：Warmup 在训练初期逐步升高学习率，避免刚开始梯度和优化器状态不稳定导致发散。

为什么需要：大模型训练初期参数、激活、梯度和 Adam 统计量都还不稳定，直接使用峰值学习率可能导致 loss spike。

常见设置：按总步数的 1% 到 5% 做 warmup，具体取决于模型规模、batch size 和训练 recipe。

局限：warmup 只能缓解训练初期不稳定，不能修复错误数据、错误 mask 或过高峰值学习率。

Batch Size

一句话定义：batch size 是一次参数更新使用的样本数量。

影响：batch 越大，梯度估计通常越稳定，但显存需求更高；batch 太小，梯度噪声更大。

在大模型中：常用 gradient accumulation 实现更大的 global batch size。

相关概念：micro batch size、global batch size、gradient accumulation steps。

公式： $\text{global batch size} = \text{micro batch size} * \text{data parallel size} * \text{gradient accumulation steps}$ 。

面试误区：显卡上设置的 per_device_train_batch_size 不等于全局 batch size。

随机种子 Random Seed

一句话定义：随机种子控制随机过程的初始化，使实验更可复现。

影响范围：参数初始化、数据 shuffle、dropout、采样、负样本选择。

在深度学习中：即使设置随机种子，GPU 并行、非确定性算子和分布式训练仍可能带来微小差异。

工程建议：重要实验记录 seed、代码版本、数据版本、依赖版本和硬件环境。

超参数 Hyperparameter

一句话定义：超参数是在训练前设定、通常不直接由梯度学习得到的配置。

例子：学习率、batch size、训练轮数、weight decay、dropout、LoRA rank、warmup ratio、max sequence length。

为什么重要：相同模型和数据，不同超参数可能得到完全不同的结果。

面试表达：调参不能只靠玄学，应结合 loss 曲线、验证集指标、bad case 和资源约束进行系统分析。

训练集 / 验证集 / 测试集

一句话定义：训练集用于学习参数，验证集用于调参与选择模型，测试集用于最终无偏评估。

常见错误：用测试集反复调 prompt 或调超参，会让测试集变成验证集，导致最终结果乐观。

在大模型项目中：还可以额外保留 hidden test、时间外测试集、困难样本集和线上灰度指标。

面试表达：数据切分是评估可信度的基础，不能只看模型分数，还要看分数从哪里来。

泛化审计 Generalization Audit

一句话定义：泛化审计是同时检查 train / validation / test gap、分布偏移、污染重叠、关键切片和 checkpoint 选择的评估流程。

核心检查：训练分数不能单独汇报；验证集用于调参与 early stopping；最终测试集不能反复调 prompt 或调超参；线上切片要覆盖目标用户、目标领域和长尾输入。

常见指标： $G_score = S_train - S_val$ 、 $\Delta_score = S_val - S_target$ 、exact overlap rate、worst-slice score、regression count。

在大模型中：SFT、DPO、RAG prompt 和 benchmark 迭代都可能过拟合固定评测集，所以泛化审计要包含改写、组合、分布外、时间切分和污染检测。

面试表达：我不会只说“验证集涨了”。我会说明数据切分、是否污染、关键切片、线上分布差异、early stopping 依据和残余风险。

强化学习数学 Reinforcement Learning Math

一句话定义：强化学习数学研究 policy 如何在状态中选择动作，并根据 reward 优化长期回报。

在大模型中：RLHF 把语言模型看成 policy，把生成 token 看成 action，把完整回答看成 trajectory，把 reward model 分数看成偏好代理奖励。

面试表达：讲 RLHF 不能只背流程，要能解释 policy、reward、value、advantage、PPO、KL 和 reward hacking 各自解决什么问题。

Markov Decision Process / MDP

一句话定义：MDP 用状态、动作、转移、奖励和折扣因子描述序贯决策问题。

核心形式： $M = (S, A, P, r, \gamma)$ 。

LLM 映射：状态是 prompt 加已生成 token，动作是下一个 token，转移是追加 token，奖励通常来自完整回答后的 reward model。

Return

一句话定义：return 是从当前时刻开始的累计折扣奖励。

核心形式： $G_t = \sum_k \gamma^k r_{t+k}$ 。

注意：return 不是当前 reward，它包含未来奖励。RLHF 中 reward 往往在完整回答后才出现，所以 token 级 credit assignment 更难。

Value Function

一句话定义：value function 估计某个状态未来能获得的期望 return。

核心形式： $V^\pi(s) = E_\pi[G_t \mid s_t=s]$ 。

在 RLHF 中：value model 提供 baseline，帮助计算 advantage，降低 policy gradient 方差。

Q Function

一句话定义：Q function 估计在某个状态采取某个动作后未来能获得的期望 return。

核心形式： $Q^\pi(s,a) = E_\pi[G_t \mid s_t=s, a_t=a]$ 。

与 value 的关系：value 是按 policy 对 action value 的加权平均。

Advantage

一句话定义：advantage 衡量某个动作比当前状态的平均预期好多少。

核心形式： $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ 。

在 PPO 中：正 advantage 的动作应提高概率，负 advantage 的动作应降低概率。

Policy Gradient

一句话定义：policy gradient 直接沿着提高期望 reward 的方向更新 policy 参数。

核心直觉： $\text{grad } J$ 与 $\text{grad } \log \pi(a|s) * \text{advantage}$ 成正比。

面试表达：policy gradient 不是直接回传 reward 到每个 token，而是用 log probability trick 和 advantage 估计更新方向。

PPO Clipped Objective

一句话定义：PPO 用新旧 policy 概率比限制一次更新幅度，避免 policy 变化过大。

核心对象： $\rho = \pi_{\text{new}}(a|s) / \pi_{\text{old}}(a|s)$ ，clip 范围通常围绕 $1 \pm \epsilon$ 。

在 RLHF 中：PPO 用 reward model 信号更新 policy，同时配合 KL penalty 和 value baseline 提高稳定性。

RLHF KL Penalty

一句话定义：KL penalty 限制 RLHF policy 不要偏离 reference model 太远。

核心形式： $\text{reward} - \beta * \text{KL}(\text{policy} || \text{reference})$ 。

面试表达：KL 约束能降低 reward hacking 和语言退化风险，但不能保证每个输出都真实、安全或最优。

Reward Model Pairwise Loss

一句话定义：reward model 用 chosen / rejected 偏好对学习哪个回答更好。

核心形式： $L_{\text{RM}} = -\log \text{sigmoid}(r_{\text{chosen}} - r_{\text{rejected}})$ 。

局限：reward model 学的是偏好 proxy，不是真实人类价值本身，过度优化会带来 reward hacking。

DPO Loss

一句话定义：DPO 直接用 preference pair 优化 policy，使 policy 相对 reference 更偏好 chosen 而不是 rejected。

核心口径：DPO 比较 policy 和 reference 的 chosen / rejected log probability gap，并用 β 控制偏离强度。

面试表达：DPO 省掉显式 reward model 和 PPO loop，但不等于没有约束；reference model 和 β 仍然提供分布锚点。

Evaluation Statistics

一句话定义：评估统计把有限评估集上的分数看成随机估计，用均值、方差、标准误、置信区间和显著性检验判断提升是否可信。

在 LLM 中：accuracy、win rate、judge score、human rubric、latency、cost 和 safety violation 都应该报告样本量和不确定性，而不是只报告单点平均值。

面试表达：统计显著不等于值得上线，还要结合 effect size、成本、延迟、安全、分层结果和失败样本。

Standard Error

一句话定义：标准误衡量样本均值本身的不确定性。

核心形式： $SE(\text{mean}) = s / \sqrt{n}$ 。

在评估中：样本量越小、样本方差越大，标准误越大，置信区间也越宽。

Confidence Interval

一句话定义：置信区间给出效果大小的合理不确定范围。

常见形式： $\text{mean} \pm 1.96 * SE(\text{mean})$ 。

面试表达：如果 lift 的置信区间跨过 0，就不能只因为点估计为正而断言新模型更好。

p-value

一句话定义：p-value 是在零假设成立时，观察到当前或更极端结果的概率。

常见误区：它不是“新模型无效的概率”，也不是“新模型更好的概率”。

在项目中：p-value 必须和预注册主指标、样本量、effect size、置信区间、多重比较和护栏指标一起解释。

Bootstrap

一句话定义：bootstrap 通过有放回重采样估计指标分布和置信区间。

在 LLM 评估中：适合 pairwise win rate、LLM judge score、RAG faithfulness、多维人工评分等难写解析方差的复杂指标。

边界：bootstrap 不能修复样本偏差；样本本身不代表真实用户分布时，置信区间也会给出偏的结论。

Paired Evaluation

一句话定义：paired evaluation 让两个模型在同一批样本上比较，直接统计每条样本上的差异。

为什么需要：它控制了样本难度差异，比两个模型分别在不同样本集合上比较更稳。

在 LLM 中：pairwise human eval、同题 accuracy 对比、同 prompt judge 对比和 RAG ablation 都优先采用 paired 设计。

McNemar Test

一句话定义：McNemar test 用于比较两个模型在同一批二分类样本上的正确 / 错误差异。

核心对象：b 是旧错新对，c 是旧对新错；两个模型都对或都错的样本不参与差异检验。

面试表达：当二分类结果是 paired 时，不应只比较两个 accuracy 的独立比例检验；McNemar 更关注同一题上的反转。

Sample Size

一句话定义：样本量决定评估能否检测到关心的最小效果。

关键概念：minimum detectable effect、显著性水平、统计功效、基线比例、方差和分层流量。

在 A/B test 中：样本量应在实验前估算，不能看到趋势后临时决定停止。

Statistical Power

一句话定义：统计功效是在真实存在某个效果时，实验能检测到它的概率。

常见风险：样本量太小会导致“没显著”不等于“没效果”，只是实验不够有力。

Multiple Comparisons

一句话定义：同时尝试很多模型、prompt、指标、分层或中途查看，会增加偶然显著的概率。

缓解方法：预注册主指标、保留独立 holdout、区分探索性和确认性分析，并对多重检验做校正。

Bonferroni Correction

一句话定义：Bonferroni 用更严格的 α / m 阈值控制多重比较下的 family-wise error rate。

特点：简单、保守，适合确认性实验中的少量关键假设。

Benjamini-Hochberg

一句话定义：Benjamini-Hochberg 通过排序 p-value 控制 false discovery rate。

特点：通常比 Bonferroni 不那么保守，适合探索性分析中的多个候选发现，但探索性结果仍需要独立验证。

Experiment Gate

一句话定义：experiment gate 把主指标、置信区间、显著性、样本量、护栏指标、污染检查和多重比较规则合成上线门禁。

在 LLM 中：即使主指标提升，也要同时检查 latency、cost、safety regression、over-refusal、关键用户分层和 bad case 类型。

Math Interview Readiness

一句话定义：数学面试准备度衡量候选人是否能把公式、直觉、LLM 场景、边界条件和最小代码同时讲清。

核心维度：topic coverage、formula accuracy、intuition clarity、LLM connection、caveat coverage、demo coverage。

面试表达：数学题不是只背公式；好的回答要能解释公式变量、适用条件、工程场景和常见误区。

Math Interview Rubric

一句话定义：数学面试 rubric 是对回答质量的结构化评分规则。

常见评分项：公式是否正确，直觉是否清楚，是否连接到 pretraining / SFT / RLHF / RAG / evaluation，是否说明边界，是否能写出最小 demo。

用途：复盘时把“我好像懂”变成可量化的缺口，例如 Bayes/MAP 手算薄弱、DPO/PPO 边界不清、泛化题缺少代码例子。

Formula Coverage

一句话定义：formula coverage 检查一轮复习是否覆盖核心公式族，而不是只会少数高频题。

数学基础中的核心族：线性代数、概率、信息论、优化、低秩、泛化、贝叶斯、RLHF 数学和评估统计。

Formula Audit

一句话定义：formula audit 用手算或最小代码验证公式是否能真的算出数值。

例子：验证 $H(p,q)=H(p)+KL(p||q)$ 、softmax cross entropy 梯度 p-y、LoRA 参数量、DPO margin、paired eval lift 和 bootstrap CI。

Mock Interview Retrospective

一句话定义：模拟面试复盘把每道回答拆成得分、薄弱点、缺失主题和下一步修订计划。

在本书系中：它用于把第十三册数学公式、代码 demo 和面试表达连成闭环，避免只看章节不练输出。

B. 深度学习组件

条目：MLP、Linear Layer、Activation Function、ReLU、GeLU、SiLU、GLU、SwiGLU、Dropout、BatchNorm、LayerNorm、RMSNorm、Residual Connection、Skip Connection、Optimizer、SGD、Momentum、Adam、AdamW、Weight Decay、Learning Rate Warmup、Cosine Decay、Gradient Clipping、Initialization、Transformer Components、Token Embedding、LM Head Weight Tying、Causal Mask、Scaled Dot-Product

MLP Multi-Layer Perceptron

一句话定义：MLP 是由线性层和非线性激活函数堆叠而成的前馈神经网络模块。

核心形式： $MLP(x) = W2 * \text{act}(W1 * x + b1) + b2$ 。

在 Transformer 中：MLP 通常位于 attention 子层之后，也常被称为 FFN 或 feed-forward network。

为什么需要它：attention 负责 token 之间的信息交互，MLP 负责对每个 token 的 hidden state 做非线性特征变换。

典型维度：Transformer MLP 的中间维度通常大于 hidden size，例如 $4 * \text{hidden_size}$ ，SwiGLU 结构中也常使用特定扩展比例。

常见误区：MLP 不是“可有可无的简单层”。在大模型中，MLP 参数量通常占比很高，是模型记忆、变换和组合特征的重要位置。

面试表达：Transformer block 可以粗略理解为 attention 负责跨 token 混合，MLP 负责逐 token 非线性变换，二者共同构成表达能力。

Feed-Forward Network / FFN

一句话定义：FFN 是 Transformer 中对每个 token 独立应用的前馈网络，通常就是 MLP 模块。

核心特点：同一个 FFN 参数会作用在序列中的每个位置，但不同位置之间不在 FFN 内直接通信。

与 attention 的区别：attention 的输出依赖其他 token，FFN 的输出只依赖当前位置经过 attention 后的表示。

在大模型中的作用：FFN 提升模型的非线性表达能力，并在参数规模上承担大量知识存储。

面试追问：为什么 Transformer 需要 attention 后再接 FFN？因为 token 混合之后，还需要对混合后的表示做非线性投影和特征重组。

Linear Layer

一句话定义：线性层对输入向量做仿射变换，是神经网络中最基础的参数化模块。

核心公式： $y = xW^T + b$ 。

在大模型中：embedding 投影、QKV 投影、attention 输出投影、MLP 上投影和下投影都依赖线性层。

参数量来源：一个 $d_{in} \rightarrow d_{out}$ 的线性层参数量约为 $d_{in} * d_{out}$ ，如果有 bias 还要加 d_{out} 。

常见误区：线性层本身表达能力有限，但多层线性层中间如果没有非线性激活，整体仍等价于一个线性变换。

面试表达：大模型的很多“结构复杂性”最终都会落到若干大矩阵乘法上，因此推理和训练优化常围绕 GEMM、显存访问和并行切分展开。

Activation Function

一句话定义：激活函数为神经网络引入非线性，使多层网络能够表达复杂函数。

为什么需要它：如果没有激活函数，多层线性层堆叠仍然等价于单层线性层，模型表达能力会严重受限。

常见类型：ReLU、GeLU、SiLU、Swish、GLU、SwiGLU。

在大模型中：现代 LLM 更常见 GeLU、SiLU 或 SwiGLU，而不是最朴素的 ReLU。

常见误区：激活函数不是只影响精度，也会影响数值稳定性、梯度传播、计算效率和硬件实现。

面试追问：为什么很多新模型使用 SwiGLU？因为门控结构能增强特征选择能力，在相近计算量下通常有更好的效果。

ReLU Rectified Linear Unit

一句话定义：ReLU 是将负数截断为 0、正数保持不变的激活函数。

核心公式： $\text{ReLU}(x) = \max(0, x)$ 。

优点：计算简单，正区间梯度稳定，曾广泛用于 CNN 和早期深度网络。

局限：负区间梯度为 0，可能出现 dead neuron；输出均值偏正，也可能影响深层网络稳定性。

在大模型中：ReLU 在现代主流 LLM 中不是最常见选择，更多被 GeLU、SiLU 或门控激活替代。

面试表达：ReLU 的优势是简单高效，但 Transformer 大模型更关注平滑梯度、表达能力和训练稳定性，因此常采用更平滑或带门控的激活。

GeLU Gaussian Error Linear Unit

一句话定义：GeLU 是一种平滑激活函数，可以理解为按输入值大小对特征进行概率式门控。

核心直觉：输入越大越容易通过，输入越小越容易被抑制，但不是像 ReLU 那样硬截断。

近似公式： $\text{GeLU}(x) = x * \Phi(x)$ ，其中 Φ 是标准正态分布的累积分布函数。

在 Transformer 中：BERT、GPT 系列早期模型常使用 GeLU。

优点：相比 ReLU 更平滑，负区间也保留一定梯度，有利于稳定训练。

局限：计算比 ReLU 复杂，不过现代框架通常有高效近似实现。

面试追问：GeLU 和 ReLU 的核心区别是什么？ReLU 是硬门控，GeLU 是平滑门控；GeLU 对接近 0 的输入处理更柔和。

SiLU / Swish

一句话定义：SiLU 是 $x * \text{sigmoid}(x)$ 形式的平滑激活函数，也常被称为 Swish。

核心公式： $\text{SiLU}(x) = x * \text{sigmoid}(x)$ 。

核心直觉：它用 sigmoid 作为软门控，控制输入特征通过的程度。

在大模型中：SiLU 常作为 SwiGLU 的基础组件之一。

优点：平滑、非单调，在很多深层网络中效果较好。

常见误区：SiLU 不等于 sigmoid。sigmoid 输出在 0 到 1 之间，而 SiLU 会保留输入幅度并进行门控。

面试表达：SiLU 可以看作输入值乘以一个由自身决定的软门，既保留幅度信息，又引入非线性选择。

GLU Gated Linear Unit

一句话定义：GLU 是一种带门控机制的前馈结构，用一个分支控制另一个分支的信息通过。

核心形式： $\text{GLU}(x) = (xW_a) * \text{sigmoid}(xW_b)$ 。

核心直觉：一个线性分支产生内容，另一个分支产生门控，二者逐元素相乘。

为什么重要：门控结构可以动态选择哪些特征需要保留，增强 FFN 的表达能力。

与普通 MLP 的区别：普通 MLP 主要依赖激活函数做非线性变换，GLU 额外引入显式乘性门控。

面试追问：门控为什么有用？因为它允许模型根据输入内容动态调节特征通路，而不是对所有输入使用固定变换强度。

SwiGLU

一句话定义：SwiGLU 是把 GLU 中的 sigmoid 门控替换或扩展为 Swish/SiLU 风格门控的 FFN 变体。

常见形式： $\text{SwiGLU}(x) = (xW_{\text{gate}}) * \text{SiLU}(xW_{\text{up}})$ ，再经过 W_{down} 投回 hidden size。

在 LLM 中：LLaMA 等模型使用 SwiGLU 作为 MLP/FFN 结构的重要组成部分。

优点：相比普通 GeLU FFN，SwiGLU 通常在相近计算量下有更好的建模效果。

代价：结构中通常有 gate、up、down 三个投影矩阵，参数量和实现细节需要配合中间维度设计。

常见误区：SwiGLU 不是单纯换一个激活函数，而是改变了 FFN 的门控结构。

面试表达：SwiGLU 的关键是乘性门控，它让 FFN 能根据输入动态筛选特征，这也是现代 LLM 常用它的重要原因。

Dropout

一句话定义：Dropout 在训练时随机置零部分神经元输出，用来减少过拟合。

核心直觉：让模型不能过度依赖某些单一特征或路径，从而提升泛化能力。

训练与推理差异：训练时随机丢弃，推理时通常不丢弃，并通过缩放保持期望一致。

在大模型中：预训练阶段是否使用 dropout 取决于规模、数据量和训练策略；大数据大模型中 dropout 可能较小甚至不用。

局限：dropout 会引入噪声，可能影响收敛速度；在超大规模预训练中，数据规模本身已经提供了强正则效果。

常见误区：dropout 不是越大越好。过大的 dropout 会损害有效容量，导致欠拟合。

面试表达：dropout 是一种随机正则化手段，但在 LLM 中要结合数据量、模型规模、训练稳定性和目标任务判断是否使用。

BatchNorm Batch Normalization

一句话定义：BatchNorm 使用 batch 维度上的均值和方差对激活进行归一化。

核心公式： $y = \gamma * (x - \text{mean_batch}) / \sqrt{\text{var_batch} + \text{eps}} + \beta$ 。

优点：在 CNN 中常用于稳定训练、加速收敛，并带有一定正则化效果。

局限：依赖 batch 统计，训练和推理行为不同；小 batch、变长序列、自回归生成场景中容易带来问题。

为什么 LLM 少用：语言模型训练常有变长序列、分布式切分和自回归推理，BatchNorm 的 batch 统计依赖不够自然。

常见误区：BatchNorm 不是所有深度网络的默认最优选择。Transformer 中更常见 LayerNorm 或 RMSNorm。

面试表达：BatchNorm 适合许多视觉模型，但自回归语言模型更偏好不依赖 batch 统计、训练推理一致的归一化方法。

LayerNorm Layer Normalization

一句话定义：LayerNorm 对单个样本的 hidden dimension 做归一化，不依赖 batch 统计。

核心公式： $\text{LN}(x) = \gamma * (x - \text{mean_hidden}) / \sqrt{\text{var_hidden} + \text{eps}} + \beta$ 。

为什么适合 Transformer：它对每个 token 的 hidden state 独立归一化，训练和推理行为一致，也不依赖 batch size。

在大模型中：LayerNorm 可用于稳定激活尺度，改善梯度传播，是 Transformer 稳定训练的核心组件之一。

常见位置：Pre-LN 结构中放在 attention 和 MLP 子层之前；Post-LN 结构中放在残差之后。

局限：LayerNorm 需要计算均值和方差，相比 RMSNorm 稍复杂。

面试追问: LayerNorm 和 BatchNorm 的核心区别是什么? BatchNorm 沿 batch 统计, LayerNorm 沿 hidden dimension 统计; 前者依赖 batch, 后者更适合序列模型。

RMSNorm Root Mean Square Layer Normalization

一句话定义: RMSNorm 使用均方根归一化 hidden state 的尺度, 但不减去均值。

核心公式: $\text{RMSNorm}(x) = \gamma * x / \sqrt{\text{mean}(x^2) + \text{eps}}$ 。

与 LayerNorm 的区别: LayerNorm 做去均值和除标准差, RMSNorm 只做尺度归一化。

在 LLM 中: LLaMA 等模型采用 RMSNorm, 以减少计算并保持训练稳定。

优点: 计算更简单, 实践中效果稳定, 适合大规模模型。

潜在局限: 不显式中心化特征, 理论表达和数值行为与 LayerNorm 不完全一致。

面试表达: RMSNorm 可以看作 LayerNorm 的简化版, 它保留了控制激活尺度的关键作用, 省去了均值中心化。

Pre-LN vs Post-LN

一句话定义: Pre-LN 和 Post-LN 描述 Transformer 中 normalization 放在子层之前还是残差之后。

Pre-LN 形式: $x = x + \text{sublayer}(\text{Norm}(x))$ 。

Post-LN 形式: $x = \text{Norm}(x + \text{sublayer}(x))$ 。

核心差异: Pre-LN 通常更利于深层模型梯度传播和训练稳定, Post-LN 在某些设置下可能最终表达更强但训练更难。

为什么 Pre-LN 稳定: 残差主路径更接近恒等映射, 梯度可以更直接地穿过深层网络。

在大模型中: 现代 LLM 通常更偏向 Pre-LN 或其变体。

常见误区: Pre-LN 不是简单 “更先进”。它主要解决深层训练稳定性问题, 具体效果仍与初始化、学习率、残差缩放和训练策略有关。

面试表达: 如果面试官问深层 Transformer 为什么常用 Pre-LN, 可以回答梯度路径更顺畅, 训练更稳定, 尤其适合大规模深层模型。

Residual Connection

一句话定义: 残差连接把输入直接加到子层输出上, 使网络学习相对于输入的增量变化。

核心公式: $y = x + F(x)$ 。

为什么需要它: 深层网络如果每层都必须重新学习完整映射, 训练很难; 残差让模型可以在恒等映射附近学习小修正。

在 Transformer 中: attention 子层和 MLP 子层外通常都有残差连接。

优点: 缓解梯度消失, 改善深层网络可训练性, 使模型更容易扩展到很多层。

常见误区: 残差连接不是简单 “防止信息丢失”, 更关键的是提供稳定的优化路径和梯度通路。

面试表达: 残差连接是深层 Transformer 可训练的基础组件之一, 它让每个 block 学习增量而不是重建全部表示。

Skip Connection

一句话定义: Skip connection 是跨层传递信息的连接方式, residual connection 是最常见的一类。

与 residual connection 的关系: 所有 residual connection 都是 skip connection, 但 skip connection 不一定都通过加法残差实现。

在大模型中：常见形式是同维度张量相加，保证主干路径稳定。

工程意义：skip connection 会影响激活保存、梯度流、并行切分和数值尺度。

面试追问：为什么残差相加要求维度一致？因为 $x + F(x)$ 是逐元素加法，两个张量形状必须兼容；若维度不同，需要投影对齐。

Optimizer

一句话定义：优化器根据梯度和内部状态更新模型参数。

为什么需要它：反向传播只计算梯度，真正改变参数的是优化器。

核心组成：学习率、梯度、参数状态、动量或矩估计、weight decay、梯度裁剪等。

在大模型中：优化器不仅影响收敛速度，还影响显存占用、训练稳定性、最终泛化和可扩展性。

常见误区：优化器不是训练脚本中的普通配置项。AdamW 的状态显存可能是参数本身的数倍，是大模型训练成本的重要来源。

面试表达：选择优化器时要同时考虑优化效果、数值稳定性、显存成本和分布式训练实现。

SGD Stochastic Gradient Descent

一句话定义：SGD 使用 mini-batch 梯度直接更新参数。

核心公式： $\theta_{\text{new}} = \theta_{\text{old}} - \eta * g_t$ 。

优点：简单、内存开销低、状态少，在一些任务上泛化表现好。

局限：对学习率敏感，收敛速度可能慢，在复杂非凸深度网络中调参成本较高。

在大模型中：纯 SGD 不是主流 LLM 预训练优化器，更多使用 AdamW 类自适应优化器。

常见误区：SGD 中的 stochastic 不是指每次只用一个样本，而是通常使用 mini-batch 对全量梯度做随机近似。

面试表达：SGD 的核心是用 mini-batch 梯度近似全量梯度，以更低计算成本进行迭代优化。

Momentum

一句话定义：Momentum 在当前梯度之外引入历史更新方向，减少震荡并加速稳定方向上的下降。

核心形式： $v_t = \beta v_{t-1} + g_t$, $\theta = \theta - \eta v_t$ 。

核心直觉：像物理中的惯性，连续一致的梯度方向会被累积，来回震荡的方向会相互抵消。

优点：加快收敛，缓解狭长峡谷地形中的震荡。

与 Adam 的关系：Adam 可以看作同时使用一阶矩和二阶矩的自适应方法，一阶矩部分与 momentum 思想相关。

面试追问：Momentum 为什么能减少震荡？因为相反方向的梯度在动量累积中会抵消，而一致方向会增强。

Adam Adaptive Moment Estimation

一句话定义：Adam 使用梯度的一阶矩和二阶矩估计，为不同参数自适应调整更新幅度。

核心形式：一阶矩 m_t 估计梯度均值，二阶矩 v_t 估计梯度平方均值，更新时用 $m_t / \sqrt{v_t}$ 。

优点：收敛快，对学习率相对不那么敏感，适合稀疏梯度和大规模深度网络。

代价：需要保存额外优化器状态，通常包括参数、梯度、一阶矩、二阶矩，显存开销大。

在 LLM 中：Adam 或 AdamW 是常见预训练和微调优化器基础。

常见误区：Adam 自适应学习率不代表不需要调学习率。base learning rate、warmup、decay、batch size 仍然非常关键。

面试表达：Adam 的核心价值是对不同参数按历史梯度尺度自适应归一化更新，提升复杂深度网络训练的稳定性 and 效率。

AdamW

一句话定义：AdamW 是将 weight decay 从 Adam 梯度更新中解耦的优化器。

核心区别：Adam 中的 L2 正则项会进入梯度并被自适应缩放；AdamW 单独对参数执行 decay，不把它混入梯度矩估计。

为什么重要：解耦后 weight decay 的含义更清晰，调参更稳定，也更适合 Transformer 等大模型训练。

在大模型中：AdamW 是预训练、SFT 和很多后训练流程中的常见默认选择。

工程注意：bias、LayerNorm、RMSNorm 等参数通常不做 weight decay。

常见误区：AdamW 不是 Adam 加一个配置名，它改变了 weight decay 的应用方式。

面试追问：为什么 decoupled weight decay 对 Transformer 训练重要？因为自适应缩放会改变 L2 正则的实际效果，解耦后正则强度更可控。

Weight Decay

一句话定义：Weight decay 是训练时让参数逐步向 0 衰减的正则化方法。

核心形式： $\theta = \theta - \eta * \lambda * \theta$ ，其中 λ 是 decay 系数。

作用：控制参数规模，缓解过拟合，并可能提升训练稳定性。

与 L2 正则关系：在 SGD 中，L2 正则和 weight decay 常等价；在 Adam 这类自适应优化器中，二者不完全等价。

在大模型中：常对矩阵权重使用 weight decay，但通常排除 bias 和归一化层参数。

常见误区：weight decay 不是越大越好。过强会压制模型容量，导致欠拟合或能力下降。

面试表达：谈 AdamW 时要强调 decoupled weight decay，而不是简单说 “AdamW 就是 Adam 加 L2 正则”。

Learning Rate

一句话定义：学习率控制每次参数更新的步长。

核心公式： $\theta_{\text{new}} = \theta_{\text{old}} - \eta * \text{update_direction}$ 。

为什么重要：学习率过大可能 loss spike、发散或 NaN；学习率过小会收敛慢，甚至陷入低效训练。

在大模型中：学习率通常配合 warmup、decay、batch size、梯度裁剪和精度策略共同设计。

常见误区：学习率不是一个孤立超参数。改变 batch size、训练 token 数、模型规模后，最优学习率也可能变化。

面试表达：学习率决定优化步长，是训练稳定性和收敛质量最敏感的超参数之一。

Learning Rate Warmup

一句话定义：warmup 是训练初期逐步增大学习率的策略。

为什么需要它：训练初期参数、激活、梯度尺度和 Adam 矩估计都不稳定，直接使用大学习率容易造成 loss spike 或 NaN。

常见形式：线性 warmup，从 0 或很小学习率逐步增加到目标学习率。

在大模型中：warmup 是预训练和微调常见稳定化手段，尤其在大 batch、混合精度和深层 Transformer 中重要。

局限：warmup 只能缓解初期不稳定，不能根治错误数据、过大学习率、异常 loss 或数值溢出问题。

面试追问：warmup 为什么对 Adam 有帮助？因为 Adam 的一阶和二阶矩估计在训练初期还不可靠，逐步放大学习率更稳。

Cosine Decay

一句话定义：Cosine decay 是按余弦曲线逐渐降低学习率的调度策略。

核心直觉：早期保持较大学习率快速探索，后期降低学习率稳定收敛。

常见组合：linear warmup + cosine decay。

在大模型中：常用于预训练和微调，使训练后期参数更新更细致。

与 step decay 区别：step decay 是分段突降，cosine decay 更平滑。

常见误区：cosine decay 不能替代选择合适的 base learning rate；如果峰值学习率过大，warmup 后仍可能不稳定。

面试表达：cosine decay 的优势是平滑退火，工程上常和 warmup 结合形成完整学习率 schedule。

Gradient Clipping

一句话定义：Gradient clipping 限制梯度范数或梯度值，防止异常大梯度导致训练不稳定。

常见方式：global norm clipping，即当整体梯度范数超过阈值时按比例缩小全部梯度。

为什么需要它：深层网络、长序列、混合精度训练或异常 batch 可能导致梯度爆炸。

在大模型中：gradient clipping 是常见训练稳定化配置，尤其用于预训练、SFT 和 RLHF/PPO 等阶段。

局限：它是保护机制，不是根因修复。频繁触发 clipping 可能说明学习率、数据、loss scale 或训练配置存在问题。

常见误区：clip 阈值越小越安全。阈值过小会长期压缩有效梯度，降低训练效率。

面试表达：gradient clipping 的价值是防止极端 batch 破坏训练，但需要结合梯度范数日志判断是否过度触发。

Initialization

一句话定义：初始化决定训练开始时参数的分布和尺度。

为什么重要：初始化会影响激活方差、梯度传播和早期训练稳定性。

常见目标：让信号在前向和反向传播中不过快放大或衰减。

常见方法：Xavier initialization、Kaiming initialization、normal initialization，以及 Transformer 中的残差相关缩放策略。

在大模型中：初始化与 normalization、残差连接、模型深度、学习率 warmup 强相关。

常见误区：有了 LayerNorm 就不需要关心初始化。归一化能缓解尺度问题，但不能完全消除初始化对训练稳定性的影响。

面试表达：初始化不是只影响第一步 loss，它会影响整个早期优化轨迹，深层 Transformer 尤其需要匹配结构和学习率设计。

Xavier Initialization

一句话定义：Xavier 初始化根据输入和输出维度设置参数方差，目标是保持前向和反向信号尺度稳定。

典型适用：tanh、sigmoid 等较早期激活函数网络中常见。

核心直觉：如果权重太大，激活和梯度容易爆炸；权重太小，信号容易消失。

与 Kaiming 的区别：Kaiming 初始化更针对 ReLU 类激活的半截断特性设计。

面试表达：Xavier 的核心不是某个固定公式，而是根据 fan-in 和 fan-out 控制方差，维持信号传播稳定。

Kaiming Initialization / He Initialization

一句话定义：Kaiming 初始化是为 ReLU 类激活设计的参数初始化方法。

核心直觉：ReLU 会截断约一半输入，因此需要相应调整权重方差，保持激活尺度稳定。

典型适用：ReLU、Leaky ReLU 等激活网络。

在 Transformer 中：虽然大模型不一定直接套用最朴素 Kaiming 初始化，但“匹配激活函数和网络结构控制方差”的思想仍然重要。

面试追问：为什么不同激活函数需要不同初始化？因为激活函数会改变输出分布和梯度分布，初始化必须配合这种变化。

Parameter Count

一句话定义：参数量是模型中可训练参数的总数，决定模型容量和存储需求的重要因素。

计算方式：线性层参数量约为 $d_{in} * d_{out}$ ，embedding 参数量约为 $vocab_size * hidden_size$ 。

在 Transformer 中：主要参数来自 embedding、attention 的 QKV/O 投影、MLP/FFN 和输出 head。

为什么重要：参数量影响模型容量、显存占用、加载速度、训练成本和部署成本。

常见误区：参数量越大不必然越好。数据质量、训练 token 数、架构、优化和对齐策略都会影响最终能力。

面试表达：估算参数量时要能拆分到 embedding、attention、MLP 和层数，而不是只背模型总参数。

Activation Memory

一句话定义：activation memory 是训练时为反向传播保存中间激活所占用的显存。

为什么重要：训练显存不只来自参数和优化器状态，长序列、大 batch 和深层网络会让激活显存非常大。

影响因素：batch size、sequence length、hidden size、layer number、attention 实现、是否使用 activation checkpointing。

在大模型中：激活显存常是训练长上下文模型的重要瓶颈。

常见缓解：gradient checkpointing、sequence parallel、flash attention、减小 micro batch size。

常见误区：只看参数量无法判断训练显存。两个同参数量模型，如果序列长度不同，激活显存可能差异很大。

面试表达：训练显存大致由参数、梯度、优化器状态和激活组成，其中激活与 batch 和 sequence length 强相关。

Mixed Precision Training

一句话定义：mixed precision training 是在训练中混合使用 FP32、FP16、BF16 等不同精度，以降低显存和带宽压力并提升吞吐。

核心直觉：大部分矩阵乘法适合低精度，少数数值敏感位置需要保留更高精度或额外稳定化手段。

常见组合：FP16 + GradScaler，或 BF16 + autocast。具体选择取决于硬件、框架和训练稳定性。

FP16

一句话定义：FP16 是 16 位浮点格式，显存和带宽开销约为 FP32 的一半。

优势：存储省、吞吐高，很多 GPU 支持成熟。

风险：指数范围较小，容易 overflow / underflow，训练时常配合 loss scaling 和 GradScaler。

BF16

一句话定义：BF16 是 16 位浮点格式，指数范围接近 FP32，但尾数位更少。

优势：动态范围大，通常比 FP16 更不容易溢出，大模型训练中常更稳。

风险：尾数精度比 FP16 低，且依赖硬件和框架支持。

Autocast

一句话定义：autocast 是 PyTorch AMP 中自动选择算子执行 dtype 的上下文管理器。

作用：让适合低精度的算子以 FP16 / BF16 执行，让部分敏感算子保留更稳的精度，减少手动 dtype 管理。

边界：autocast 主要管理前向算子精度，不等于自动解决所有显存、溢出或训练稳定性问题。

GradScaler

一句话定义：GradScaler 通过 loss scaling 缓解 FP16 小梯度下溢问题。

典型顺序：scale(loss).backward()、unscale_(optimizer)、gradient clipping、step(optimizer)、update()。

关键边界：裁剪前要先 unscale；BF16 通常不强依赖 GradScaler。

Loss Scaling

一句话定义：loss scaling 是先把 loss 放大再反向传播，optimizer step 前再把梯度缩回真实尺度。

公式直觉：如果 $L_{scaled}=s*L$ ，则梯度也被放大 s 倍；unscale 后再用于裁剪和更新。

Activation Checkpointing

一句话定义：activation checkpointing 是前向少存一部分激活、反向时重算部分前向，用计算换显存。

适用场景：深层 Transformer、长上下文训练、显存紧张但可接受额外计算成本的实验。

常见坑：会增加 backward 时间；涉及随机层时要关注 RNG 状态和 checkpoint API 参数。

CUDA Memory Stats

一句话定义：CUDA memory stats 是 PyTorch 提供的 GPU 显存统计接口，用于观察 allocated、reserved、peak memory 等状态。

常见指标：memory_allocated()、memory_reserved()、max_memory_allocated() 和 reset_peak_memory_stats()。

调试用途：区分真实 tensor 占用、缓存保留、峰值显存和疑似显存泄漏。

OOM Audit

一句话定义：OOM audit 是系统排查 out-of-memory 的流程。

常见顺序：区分训练 / 验证阶段，缩小 batch size 和 sequence length，检查 mixed precision、gradient accumulation、activation checkpointing、optimizer state、带图 tensor 缓存和验证是否关闭梯度。

PyTorch Tensor

一句话定义：PyTorch tensor 是深度学习代码里的核心数据对象，包含数据、shape、dtype、device、stride 和梯度相关状态。

在大模型中：input ids、embedding、hidden states、attention scores、logits、loss mask、参数和梯度都是 tensor。

面试表达：调试 PyTorch 代码时先看 shape，再看 dtype、device、contiguous / stride、mask 方向和数值是否 finite。

Tensor Shape

一句话定义：tensor shape 描述每个维度的长度，是模型实现中最重要的调试信息。

常见 LLM shape: input_ids [B,T], hidden states [B,T,d], logits [B,T,V], attention scores [B,H,T,T]。

常见错误：loss、matmul、mask 或 concat 前没有对齐 shape。

Dtype

一句话定义：dtype 是 tensor 的数据类型，决定数值精度、显存占用和某些模块的输入要求。

常见类型：token id 和 label 通常是 torch.long；模型激活常见 float32、float16、bfloat16；mask 常见 bool 或浮点加性 mask。

Device

一句话定义：device 表示 tensor 存放在 CPU、GPU 或其他设备上。

常见坑：同一次计算里的 tensor 不能混在 CPU 和 GPU 上；batch 通常要移动到 `next(model.parameters()).device`。

Broadcasting

一句话定义：broadcasting 允许某些 shape 不同的 tensor 做逐元素运算。

规则：从右往左对齐，每一维要么相等，要么其中一个是 1，要么某个 tensor 缺少该维。

在 LLM 中：bias、LayerNorm 参数和 attention mask 都大量依赖 broadcasting。

Tensor Stride

一句话定义：stride 描述沿某个维度移动一步时，在底层 storage 中跳过多少个元素。

为什么重要：transpose / permute 通常改变 stride 而不复制数据，所以结果可能不是 contiguous。

Contiguous Tensor

一句话定义：contiguous tensor 的内存布局与其 shape 顺序兼容，常可直接用 view 改 shape。

常见场景：多头 attention 从 [B,H,T,d_h] 转回 [B,T,d] 时，经常写 `transpose(...).contiguous().view(...)`。

View vs Reshape

一句话定义：view 要求当前 tensor 的 stride 与目标 shape 兼容；reshape 更宽松，必要时可能创建拷贝。

工程建议：原型和调试中优先用 reshape 更稳；明确追求 view 语义时，要先确认 tensor 是否 contiguous。

Tensor Mask

一句话定义：tensor mask 用 bool 或加性浮点值控制哪些位置参与计算。

在 attention 中：padding mask 常从 [B,T] 扩成 [B,1,1,T]，causal mask 常从 [T,T] 扩成 [1,1,T,T]。

常见坑：mask shape 能广播不代表语义正确，要确认 query 维和 key 维方向。

Tensor Shape Audit

一句话定义：tensor shape audit 是系统检查 shape、dtype、device、stride、contiguous、mask 和 finite 数值的调试流程。

用途：快速定位大模型代码中的 loss shape mismatch、device mismatch、view 报错、attention mask 方向错误和 NaN / Inf。

PyTorch Autograd

一句话定义：PyTorch autograd 是自动求导系统，在 forward 时动态构建计算图，在 backward 时按链式法则计算梯度。

核心机制：可求导操作会产生 grad_fn，模型参数通常是 leaf tensor，loss.backward() 会把梯度累积到 leaf 参数的 .grad。

在大模型中：pretraining、SFT、LoRA、RLHF、DPO 和训练 debug 都依赖 autograd 正确连接 loss 与可训练参数。

Dynamic Computation Graph

一句话定义：动态计算图是在每次 forward 执行时按真实 Python 控制流构建的计算图。

优势：if、for、递归和动态 shape 可以自然参与建图。

注意：默认一次 backward 后图会释放；普通训练应重新 forward，而不是滥用 retain_graph=True。

Requires Grad

一句话定义：requires_grad 决定 tensor 是否需要被 autograd 跟踪梯度。

工程用法：冻结 base model 时把 base 参数设为 False，LoRA adapter 或 head 参数保持 True。

边界：冻结参数不等于整段 forward 都不建图；如果后续可训练参数依赖中间结果，仍可能需要保留部分图。

Leaf Tensor

一句话定义：leaf tensor 通常是用户直接创建且需要梯度的 tensor，模型参数也是典型 leaf tensor。

常见现象：默认只有 leaf tensor 的 .grad 会在 backward 后被保留；中间 tensor 要观察梯度需要 retain_grad() 或 hook。

Grad Accumulation

一句话定义：PyTorch 默认把每次 backward 的梯度加到已有 .grad 上，而不是自动清零。

为什么需要：它支持多 loss 和 gradient accumulation；普通训练每个 optimizer step 前要 zero_grad()。

Zero Grad

一句话定义：zero_grad() 用于清空历史梯度，避免不同 batch 的梯度意外叠加。

常见写法：optimizer.zero_grad(set_to_none=True)，通常比把梯度填零更省内存。

Detach

一句话定义：`detach()` 返回一个与原 tensor 共享数据但不再连接当前计算图的新 tensor。

常见用途：日志、缓存、target network、stop-gradient 和截断长序列反传。

风险：误把模型输出或中间 hidden state detach，会导致梯度断开，参数学不到。

No Grad

一句话定义：`torch.no_grad()` 是上下文管理器，使作用域内的新操作不记录 autograd 图。

与 `model.eval()` 区别：`eval()` 改变 dropout / batch norm 等模块行为，`no_grad()` 关闭梯度记录；验证通常两者都用。

Inference Mode

一句话定义：`torch.inference_mode()` 是更激进的推理模式，进一步关闭部分 autograd 相关机制。

适用：纯推理、离线 embedding 抽取和验证中不需要后续参与 autograd 的张量。

In-place Operation

一句话定义：in-place 操作会直接修改原 tensor，PyTorch 中很多以下划线结尾的方法都是 in-place。

autograd 风险：backward 可能需要 forward 保存的中间值；如果这些值被原地修改，可能报版本计数或 inplace 错误。

Autograd Hook

一句话定义：hook 可以在反向传播时观察或处理梯度，常用于临时 debug。

用途：查看某层梯度是否为 0、NaN、Inf 或异常大；定位梯度消失 / 爆炸位置。

Autograd Audit

一句话定义：autograd audit 是系统检查 loss 是否依赖参数、梯度是否存在、图是否被 detach / no_grad 切断、梯度是否累积和数值是否 finite 的调试流程。

常见输出：`loss.requires_grad`、`loss.grad_fn`、参数 `requires_grad`、`grad is None`、`grad norm`、missing grad 参数名和 anomaly detection 结果。

nn.Module

一句话定义：`nn.Module` 是 PyTorch 组织模型的核心抽象，负责注册参数、子模块和 buffer，并提供递归遍历、device / dtype 迁移、训练/推理模式切换和保存加载接口。

为什么重要：optimizer、state_dict、DDP、FSDP、AMP、checkpoint 和大模型组件替换都依赖 Module 树。

常见误区：把 `nn.Module` 只理解成“必须继承的模型类”。真正关键的是注册机制和递归管理能力。

nn.Parameter

一句话定义：`nn.Parameter` 是特殊 tensor，被赋值为 Module 属性时会自动注册为模型参数。

工程意义：注册参数会出现在 `model.parameters()`、`named_parameters()` 和 `state_dict()` 中，optimizer 通常据此更新权重。

常见坑：普通 tensor 即使 `requires_grad=True`，如果不是 Parameter，通常不会被 optimizer 自动拿到。

Module Registration

一句话定义：Module registration 是把参数、子模块和 buffer 挂到 Module 树上的机制。

常见方式：属性赋值、ModuleList、ModuleDict、ParameterList、ParameterDict 和 register_buffer。

风险：子模块放进普通 list / dict，或者在 forward 中临时创建层，会导致参数不注册、权重不保存、device 不迁移、DDP 不同步。

ModuleList

一句话定义：ModuleList 是保存一组已注册子模块的容器，但不规定 forward 执行顺序。

在大模型中：Transformer block 堆叠通常用 ModuleList，因为 forward 里还要处理 mask、cache、hidden states、gradient checkpointing 等控制逻辑。

对比：Sequential 适合简单流水线，ModuleDict 适合按名称选择 adapter、head 或分支。

ParameterList

一句话定义：ParameterList 是保存一组已注册参数的容器。

使用场景：自定义可学习 prompt、加权融合、任务 bias、adapter 标量或不方便包装成完整子模块的参数集合。

常见坑：普通 list 中的 nn.Parameter 不会自动注册。

Buffer

一句话定义：buffer 是不由 optimizer 更新、但属于模型状态的 tensor。

常见例子：BatchNorm running statistics、固定位置编码、固定 attention mask、量化 scale / zero point 和统计量。

工程规则：buffer 不出现在 parameters() 中，会随 model.to(device) 移动，默认保存在 state_dict()；可用 persistent=False 排除可再生成的大缓存。

state_dict

一句话定义：state_dict 是 PyTorch 保存模型权重的标准字典，包含注册参数和持久化 buffer。

不包含：Python 类定义、forward 代码、普通属性、未注册 tensor、optimizer 状态和 persistent=False buffer。

面试表达：推荐保存 model.state_dict()，而不是直接 pickle 整个 model；完整训练恢复还要保存 optimizer、scheduler、step、epoch 和 config。

load_state_dict

一句话定义：load_state_dict 用 checkpoint 字典恢复模型参数和 buffer。

关键参数：strict=True 要求 key 完全匹配；strict=False 允许部分加载，但必须检查 missing keys 和 unexpected keys 是否符合预期。

常见原因：模型结构变化、层命名变化、DDP 的 module. 前缀、只加载 backbone、分类头变化或词表大小变化。

train / eval Mode

一句话定义：model.train() 和 model.eval() 递归设置 Module 的 training 状态。

影响层：Dropout、BatchNorm 这类依赖训练模式的层。

边界：eval() 不会关闭 autograd；推理通常还要配合 torch.no_grad() 或 torch.inference_mode()。

Module Hook

一句话定义：Module hook 是在 forward 或 backward 流程中插入回调的机制。

用途：抓中间激活、统计层输出、检查 NaN / Inf、观察梯度和临时研究分析。

风险：保存激活时要 detach()，用完要 remove()，不要把 hook 当长期业务逻辑依赖。

Module Audit

一句话定义：Module audit 是系统检查模型注册树、参数表、buffer、state_dict、train/eval 状态、optimizer 参数组和保存加载一致性的调试流程。

常见输出：named_parameters()、named_buffers()、state_dict().keys()、可训练参数量、optimizer 参数量、missing / unexpected keys、冻结参数列表和保存加载后 logits 对齐结果。

Dataset / DataLoader

一句话定义：Dataset / DataLoader 是 PyTorch 中把原始样本稳定组织成训练 batch 的数据管线核心抽象。

分工：Dataset 负责单样本读取和轻量预处理；DataLoader 负责 batch 取样、可选 shuffle、多进程加载、调用 collate_fn 和配合 sampler 控制顺序。

大模型场景：常见输出包括 input_ids [B,T]、labels [B,T]、attention_mask [B,T]，padding 位置的 label 通常设为 -100 以避免参与 loss。

Map-style Dataset

一句话定义：map-style Dataset 是支持 __len__() 和 __getitem__(idx) 的可索引数据集。

适用场景：固定离线语料、监督训练数据、可随机访问的数据文件和需要 shuffle / sampler 控制索引顺序的训练任务。

常见坑：__getitem__ 中做过重预处理会拖慢训练；把 batch 级 padding 写进 Dataset 会浪费计算并增加调试难度。

IterableDataset

一句话定义：IterableDataset 是通过 __iter__() 流式产生样本的数据集。

适用场景：超大规模流式数据、在线日志、远程对象存储逐条读取和无法提前知道长度的数据源。

常见坑：不能简单依赖 shuffle=True 完成全局随机；多 worker / 多 rank 场景需要额外处理切片，避免重复读同一批样本。

Collate Function

一句话定义：collate_fn 是把一个样本列表整理成模型可消费 batch 的函数。

大模型场景：它通常负责变长序列 padding、构造 attention_mask、把 padding label 设为 -100、整理复杂字段和必要时做 packing。

调试重点：检查 input_ids、labels、attention_mask 的 shape、padding 方向、ignore index 和 next-token shift 是否一致。

Data Sampler

一句话定义：sampler 控制 DataLoader 按什么顺序取样本索引。

常见类型：顺序采样、随机采样、加权采样、length bucket 采样、batch sampler 和分布式采样。

面试重点: `shuffle=True` 是简单随机顺序; `sampler` 控制单样本索引顺序; `batch sampler` 直接控制一整个 batch 包含哪些索引。

DistributedSampler

一句话定义: `DistributedSampler` 在分布式训练中按 rank 切分数据索引, 让不同进程看到不同样本子集。

关键规则: 每个 epoch 通常要调用 `set_epoch(epoch)` 刷新 `shuffle`; `shuffle=True` 通常不要和手动 `sampler` 混用; 数据量不能整除时要理解补样本和 `drop_last` 的取舍。

DataLoader Worker

一句话定义: `DataLoader worker` 是用于并行读取和预处理数据的子进程。

参数相关: `num_workers` 控制 worker 数量, `persistent_workers` 控制 worker 是否跨 epoch 保持存活, `prefetch_factor` 控制每个 worker 预取 batch 数。

常见坑: worker 不是越多越好; CPU、磁盘 IO、解码库、多进程开销和随机种子都可能成为瓶颈或错误来源。

Pin Memory

一句话定义: `pinned memory` 是页锁定 CPU 内存, 常用于加速 CPU 到 GPU 的异步数据拷贝。

工程口径: `pin_memory=True` 常与 `tensor.to(device, non_blocking=True)` 搭配; 它主要优化搬运路径, 不改变模型数学逻辑。

Data Pipeline Audit

一句话定义: `data pipeline audit` 是系统检查 `Dataset`、`collate`、`DataLoader`、`sampler`、`padding`、`mask`、`worker` 和分布式切分是否正确的调试流程。

常见输出: 样本原始结构、batch shape、有效 token 比例、padding waste、labels ignore 检查、shuffle 可复现性、rank 间索引 overlap 和吞吐瓶颈定位。

Training Loop

一句话定义: `training loop` 是把模型、数据、loss、optimizer、scheduler、日志、验证和 checkpoint 串成可持续实验系统的训练主流程。

典型顺序: batch 到 device、forward、loss、backward()、可选 gradient clipping、`optimizer.step()`、`scheduler.step()`、`zero_grad()`、日志、验证和 checkpoint。

工程重点: 它必须可观测、可恢复、可复现, 并能定位 loss spike、NaN / Inf、梯度异常、学习率异常和 checkpoint resume 问题。

Training Step

一句话定义: `training step` 是一次真实参数更新, 对应一次 `optimizer.step()`。

边界: 在 gradient accumulation 中, 多个 micro-step 才对应一个 training step; scheduler 和 global step 通常应该跟随真实 optimizer step, 而不是每个 micro-batch。

Gradient Accumulation

一句话定义: `gradient accumulation` 是把多个 micro-batch 的梯度累积起来, 再执行一次 optimizer step, 以模拟更大的 batch。

关键规则: 每个 micro-step 通常使用 `loss / accum_steps` 再 backward; 否则梯度尺度会放大, 等价于隐式提高学习率。

Optimizer Step

一句话定义：optimizer step 是优化器根据当前梯度和内部状态更新模型参数的动作。

工程顺序：通常在 `loss.backward()` 和 gradient clipping 之后执行；执行后要清梯度，避免下一步意外累积。

Scheduler Step

一句话定义：scheduler step 是更新学习率调度状态的动作。

常见规则：大多数按 step 更新的 scheduler 放在 `optimizer.step()` 之后；使用梯度累积时，只在真实 optimizer step 后调用。

Evaluation Loop

一句话定义：evaluation loop 是验证模型指标的循环，通常不更新参数。

关键规则：使用 `model.eval()` 切换模块行为，并配合 `torch.no_grad()` 或 `torch.inference_mode()` 关闭梯度记录；结束后按需要恢复 `model.train()`。

Checkpoint Resume

一句话定义：checkpoint resume 是从保存的训练状态恢复模型训练。

完整状态：除了模型 `state_dict`，还应保存 optimizer、scheduler、epoch / global step、config 和随机状态。只保存模型权重会丢失动量、二阶矩和学习率位置。

Non-Finite Loss

一句话定义：non-finite loss 指 loss 变成 NaN 或 Inf。

排查入口：当前 batch、learning rate、gradient norm、混合精度溢出、mask 全忽略 / 全 -inf、输入异常值和带图 tensor 泄漏。

Training Loop Audit

一句话定义：training loop audit 是系统检查训练 step 顺序、loss mask、梯度、学习率、验证、checkpoint 和恢复训练是否正确的调试流程。

常见输出：raw loss、scaled loss、grad norm、optimizer step 数、scheduler step 数、learning rate、validation loss、checkpoint keys、resume 后指标一致性和非有限 loss 检查结果。

Distributed Training

一句话定义：distributed training 是用多个进程、GPU 或机器共同完成训练，以突破单卡显存和速度限制。

常见路线：数据并行、参数分片、张量并行、流水线并行和 expert parallel。入门阶段优先掌握 DDP 数据并行。

Rank / World Size

一句话定义：rank 是分布式训练中的全局进程编号，world size 是总进程数，local rank 是当前机器内的进程或 GPU 编号。

工程用途：local_rank 常用于绑定本机 GPU，rank==0 常用于打印日志和保存 checkpoint，world_size 用于计算 global batch 和通信规模。

Process Group

一句话定义：process group 是一组可相互通信的分布式训练进程。

常见操作：初始化后可执行 `all_reduce`、`broadcast`、`barrier` 等 collective；训练结束时应销毁进程组。

DistributedDataParallel / DDP

一句话定义：DDP 是 PyTorch 推荐的数据并行训练封装，每个进程通常持有一份模型副本，处理不同 mini-batch，并在 backward 中同步梯度。

关键机制：DDP 通过 autograd hook 和 gradient bucket 在梯度 ready 时触发同步，尽量让通信和反向计算重叠。

All-Reduce

一句话定义：all-reduce 是把所有 rank 上的张量做归约，并把归约结果返回给所有 rank 的 collective 操作。

DDP 语境：每个 rank 的本地梯度经过 all-reduce 求平均后，所有 rank 使用相同梯度执行 optimizer step，从而保持参数一致。

torchrun

一句话定义：torchrun 是 PyTorch 常用的分布式训练启动方式，会启动多个训练进程并设置 rank 相关环境。

常见变量：RANK、LOCAL_RANK、WORLD_SIZE。训练脚本通常读取这些变量完成进程组初始化和 GPU 绑定。

Global Batch Size

一句话定义：global batch size 是一次真实参数更新中所有 rank、所有梯度累积 micro-step 共同贡献的样本数。

常见公式： $B_{\text{global}} = B_{\text{device}} * \text{world_size} * \text{accum_steps}$ 。多卡训练时如果每卡 batch 不变，global batch 会变大，学习率和 warmup 需要重新核对。

DDP no_sync

一句话定义：no_sync() 是 DDP 在梯度累积中跳过部分 micro-step 梯度同步的上下文管理器。

使用边界：通常只在非最后一个 accumulation micro-step 使用，最后一个 micro-step 仍要同步梯度并执行 optimizer step。

Distributed Training Audit

一句话定义：distributed training audit 是系统检查 rank、数据切分、global batch、梯度同步、collective、checkpoint 写入和卡住风险的调试流程。

常见输出：rank / local rank / world size、每个 rank 的数据索引、rank 间 overlap、global batch size、all-reduce 后梯度、参数一致性、通信次数、缺失 collective 的 rank 和 rank 0 checkpoint 写入检查。

Debug Tensor Metadata

一句话定义：debug tensor metadata 是先检查 tensor 的 shape、dtype、device、requires_grad、stride 和 contiguous 状态，再看具体数值。

用途：快速定位 shape mismatch、dtype mismatch、device mismatch、非 contiguous view 报错、loss 展平错误和 mask broadcast 方向错误。

Finite Check

一句话定义：finite check 是检查输入、logits、loss、grad 和参数中是否出现 NaN 或 Inf。

排查顺序：先定位非有限值首次出现在哪个阶段，再判断是数据异常、学习率过大、fp16 溢出、mask 全 -inf、loss 分母为 0 还是梯度爆炸。

Gradient Debug

一句话定义：gradient debug 是检查可训练参数是否有梯度、梯度是否有限、梯度范数是否合理，以及参数是否进入 optimizer。

常见输出：grad is None 参数名、grad norm、非有限 grad、全 0 grad、optimizer 参数覆盖率和断图位置。

Forward Hook Debug

一句话定义：forward hook debug 是临时注册 hook 捕获模块输入或输出，用于定位中间激活 shape、dtype、NaN / Inf 或异常尺度。

注意事项：保存激活时应 detach()，用完调用 remove()，避免长期缓存大 tensor 导致显存增长。

Anomaly Detection

一句话定义：anomaly detection 是 PyTorch autograd 的调试模式，用于帮助定位 backward 异常或 in-place 破坏反传的位置。

边界：它会显著降低训练速度，只适合小范围 debug，不适合长期训练。

CUDA Synchronize Timing

一句话定义：CUDA synchronize timing 是在计时前后调用同步，避免 CUDA 异步执行导致 Python 侧只测到 kernel launch 时间。

用途：分段测量 DataLoader、CPU-GPU copy、forward、backward、optimizer step 和通信耗时。

torch.profiler

一句话定义：torch.profiler 是 PyTorch 用于收集 CPU / CUDA 算子耗时、调用次数、shape 和内存信息的 profiling 工具。

使用边界：profiler 有额外开销，通常只在少量 step 上开启；生产环境需要结合 trace、step schedule 和可观测性系统使用。

DataLoader Bottleneck

一句话定义：DataLoader bottleneck 是数据读取、解码、预处理、collate 或 CPU-GPU 拷贝速度低于训练消耗速度，导致 GPU 等数据。

排查入口：单独计时 DataLoader 迭代、检查 num_workers、pin_memory、远程 IO、在线 tokenization 和 batch 组装逻辑。

Debug / Profiling Audit

一句话定义：debug / profiling audit 是系统检查正确性和性能瓶颈的训练排查流程。

常见输出：tensor metadata、shift loss 对齐、valid label 数、finite report、hook 激活 shape、grad norm、optimizer 参数覆盖、step timing、CUDA memory trend、profiler top ops 和分布式 rank 日志。

Transformer Components

一句话定义: Transformer components 是构成 decoder-only LLM 主干的基础模块, 包括 token embedding、位置编码或 RoPE、causal self-attention、MLP / FFN、normalization、residual、final norm 和 LM head。

核心数据流: $\text{input_ids} [B, T] \rightarrow \text{hidden} [B, T, d] \rightarrow \text{blocks} \rightarrow \text{logits} [B, T, V]$ 。

面试重点: 要能逐步说清每个模块的输入输出 shape、为什么需要 causal mask、attention 和 MLP 的分工、Pre-Norm 的训练稳定性, 以及 loss 为什么要右移。

Token Embedding

一句话定义: token embedding 是把离散 token id 查表映射为连续 hidden vector 的模块。

核心 shape: embedding table 是 $[V, d]$, $\text{input_ids} [B, T]$ 查表后得到 $[B, T, d]$ 。

常见误区: token id 只是索引, 不应直接当连续数值特征输入线性层。

LM Head Weight Tying

一句话定义: LM head weight tying 是让输出词表投影权重和输入 embedding 权重共享同一份参数。

工程含义: $\text{lm_head.weight} = \text{embed_tokens.weight}$ 后, 两者应指向同一块参数存储。

优点与边界: 它能减少参数量并让输入输出 token 表示空间更一致, 但要注意 vocab resize、特殊 token 扩展和 checkpoint 加载时的权重共享关系。

Causal Mask

一句话定义: causal mask 是自回归语言模型中阻止当前位置看到未来 token 的 attention mask。

核心形状: 普通训练常见 mask 是 $[1, 1, T, T]$, 允许 $s \leq t$, 禁止 $s > t$ 。

KV Cache 边界: decode 阶段可能出现 $\text{query_len}=1$ 、 $\text{key_len}>1$, mask 需要考虑历史 cache 长度, 否则当前 token 可能看不到完整历史。

Scaled Dot-Product Attention

一句话定义: scaled dot-product attention 用 $QK^T / \sqrt{D_h}$ 得到 score, mask 后 softmax, 再乘以 V 得到上下文表示。

核心 shape: $Q/K/V$ 常见为 $[B, H, T, D_h]$, score 是 $[B, H, T_q, T_k]$, 输出回到 $[B, H, T_q, D_h]$ 。

面试重点: 缩放是为了稳定 score 方差, mask 必须在 softmax 前进入 score。

Multi-Head Self-Attention

一句话定义: multi-head self-attention 把 hidden state 投影成多个 head 的 $Q/K/V$, 让不同 head 在不同表示子空间中做信息路由。

实现关键: hidden_size 必须能被 num_heads 整除; transpose 后合并多头前常需要 contiguous() 或安全使用 reshape()。

常见输出: 输入 $[B, T, d]$, 输出仍是 $[B, T, d]$, 便于和 residual branch 相加。

KV Cache

一句话定义: KV Cache 是自回归推理中缓存历史 key/value, 避免每个 decode step 重复计算历史 token 的机制。

核心 shape: 单层常见 cache 是 $k/v [B, H, T_{\text{cache}}, D_h]$ 。

教学边界：最小实现只需演示 prefill 生成 cache、decode 拼接新 K/V、query 看见完整历史；生产系统还要处理分页、释放、调度、prefix cache 和不同请求长度。

Transformer Component Audit

一句话定义：Transformer component audit 是系统验证手写 Transformer 组件 shape、mask、loss shift、RoPE、权重绑定和 KV Cache 语义是否正确的教学调试流程。

常见输出：RMSNorm 输出 shape、causal mask 行方向、attention 行和、未来权重是否为 0、MHA / MLP / block 输出 shape、RoPE 是否保持范数、logits shape、LM loss、权重绑定和 cache 长度增长。

面试价值：它把“我能手写 Transformer”落到可执行检查项，避免只背公式但代码里 mask 方向、head reshape 或 shift loss 出错。

PyTorch Engineering Interview Readiness

一句话定义：PyTorch engineering interview readiness 是把 PyTorch 机制、shape / 公式、常见坑和 debug 路径组织成可口述、可追问、可验证的面试准备状态。

核心覆盖：tensor metadata、autograd、Module 注册、DataLoader、training loop、AMP / 显存、DDP、debug / profiling 和 Transformer component。

自评门禁：回答不能只背 API 名称，应至少覆盖机制、关键 shape 或公式、失败场景、排查顺序和最小 demo 证据。

常见误区：把“我用过 PyTorch”误当成“我能解释 PyTorch 工程问题”。工程面试更看重能否定位 loss 不下降、NaN、OOM、shape mismatch、device mismatch、DDP 卡住和 mask 语义错误。

本章小结

本章覆盖深度学习组件中最常被面试追问的基础模块。

核心结论如下：

1. Transformer 的基础计算由线性层、激活函数、归一化、残差连接和优化器共同支撑。
2. MLP/FFN 不是简单附属模块，它承担逐 token 非线性变换和大量参数容量。
3. GeLU、SiLU、SwiGLU 的区别体现了从普通激活到门控 FFN 的演进。
4. LayerNorm 和 RMSNorm 比 BatchNorm 更适合自回归语言模型。
5. 残差连接和 Pre-LN 是深层 Transformer 稳定训练的重要基础。
6. AdamW、warmup、cosine decay、weight decay 和 gradient clipping 共同决定大模型训练稳定性。
7. 手写 Transformer 组件时，shape、mask、loss shift、RoPE 和 KV Cache 都要能被最小 demo 验证。
8. PyTorch 工程面试要把机制、公式、代码 demo 和排查路径连起来。
9. 面试中不能只背定义，要能说明每个组件解决什么问题、为什么适合大模型、有什么工程代价。

下一章，我们进入 NLP 与 Tokenization。

C. NLP 与 Tokenization

条目：Token、Vocabulary、Token ID、Tokenization、Tokenizer、BPE、WordPiece、Unigram LM、SentencePiece、Byte-Level Tokenization、Character-Level Tokenization、Subword Tokenization、OOV、Special Token、BOS、EOS、PAD、UNK、Chat Template、Prompt、Completion、Instruction、System Prompt、Context Window、Detokenization、Tokenizer 训练、中文分词、代码 Tokenization、Tokenizer 与 Embedding 扩展。

Token

一句话定义：token 是模型处理文本的基本离散单位。

核心直觉: token 不一定是自然语言中的“词”，也可能是子词、字符、字节片段、标点、空格片段或特殊控制符号。

在语言模型中: 原始字符串会先被 tokenizer 转成 token 序列，再映射为 token id，最后进入 embedding 层。
为什么重要: token 决定模型看到文本的方式，也影响上下文长度、训练成本、推理成本、多语言能力和代码能力。

例子: 英文单词 `unbelievable` 可能被切成 `un`、`believ`、`able`；中文句子可能按字、词或子词切分。

常见误区: token 不是字符数，也不是单词数。面试中讨论上下文长度时，要明确是 token 维度而不是字数维度。

面试表达: LLM 不是直接处理字符串，而是处理 token id 序列；tokenization 是自然语言进入神经网络之前的离散化接口。

Vocabulary

一句话定义: vocabulary 是 tokenizer 支持的 token 集合及其到整数 id 的映射表。

核心内容: 每个 token 对应一个唯一 token id，模型的输入 embedding 和输出 logits 通常都以 vocabulary size 为维度。

在大模型中: 如果词表大小为 V ，hidden size 为 d ，输入 embedding 参数量约为 $V * d$ 。

词表大小影响: 词表越大，单个文本可能被切得更短，但 embedding 和输出层参数更多；词表越小，参数更少，但序列可能更长。

常见误区: 大词表不一定更好。过大词表会增加参数和 softmax 成本，也可能让低频 token 学得不好。

面试追问: 为什么 tokenizer 会影响训练和推理成本? 因为相同文本被切成的 token 数不同，直接影响 attention 计算、KV cache 长度和生成步数。

Token ID

一句话定义: token id 是 token 在 vocabulary 中对应的整数编号。

在模型中: 模型不能直接输入字符串或 token 文本，而是输入 token id 序列。

常见流程: `text -> tokens -> token ids -> embeddings -> transformer`。

工程注意: 不同模型即使 token 文本看起来相同，token id 也可能不同，不能混用 tokenizer。

常见误区: token id 本身没有数值大小语义。id 为 100 的 token 不比 id 为 99 的 token “更大”或“更接近”。

面试表达: token id 是离散索引，真正的语义表示来自 embedding 矩阵中对应的向量。

Tokenization

一句话定义: tokenization 是把原始文本切分成 token 并映射成 token id 的过程。

为什么需要它: 神经网络处理的是张量，不能直接处理可变长度字符串；tokenization 把文本转成离散序列。

影响范围: 上下文窗口利用率、训练 token 数、推理延迟、多语言公平性、代码表示、特殊符号处理和安全边界都受 tokenization 影响。

典型步骤: 文本规范化、预分词、子词切分、special token 拼接、id 映射。

常见误区: tokenization 是模型能力的一部分。它不是无关紧要的预处理，尤其在中文、代码、数学公式和多语言任务中影响明显。

面试表达: 同样的模型架构，如果 tokenizer 不同，实际输入长度、词表覆盖和生成行为都可能显著不同。

Tokenizer

一句话定义：tokenizer 是执行文本和 token id 序列互相转换的组件。

核心能力：encode 把文本转成 token ids，decode 把 token ids 转回文本。

在训练中：tokenizer 决定语料如何统计为训练 token，也决定模型学习的最小文本单位。

在推理中：tokenizer 决定 prompt 如何进入模型，也决定生成 token 如何还原为可读文本。

工程注意：训练、微调、评估和线上推理必须使用一致 tokenizer，否则输入格式和 embedding 对齐会出错。

常见误区：换 tokenizer 通常不是轻量改动。除非重新训练或正确扩展 embedding，否则模型无法理解新 token id 的含义。

面试表达：tokenizer 是模型输入输出协议的一部分，不能只把它看成字符串切分工具。

Subword Tokenization

一句话定义：subword tokenization 使用子词作为基本单位，在词级和字符级之间折中。

为什么需要它：纯词级词表容易遇到 OOV，纯字符级序列太长；子词方法兼顾覆盖率和序列长度。

常见方法：BPE、WordPiece、Unigram LM、SentencePiece。

优点：能表示未见词、多语言词形变化、专有名词和拼写变化。

局限：切分结果不一定符合语言学边界，可能把语义相关部分切散。

面试表达：子词切分的核心价值是用有限词表覆盖开放词汇，同时控制序列长度。

BPE Byte Pair Encoding

一句话定义：BPE 通过不断合并高频相邻片段来构建子词词表。

核心过程：从字符或字节等基础单位开始，统计相邻 pair 频率，每次合并最高频 pair，直到达到目标词表大小。

优点：简单、有效、工程成熟，能缓解 OOV 问题。

局限：合并依据主要是统计频率，不保证符合语义边界或语法规则。

在 LLM 中：GPT 系列使用过 BPE 或 byte-level BPE 变体。

常见误区：BPE 的 merge 规则来自训练语料分布，因此 tokenizer 会继承语料偏向；低资源语言可能切得更碎。

面试追问：BPE 为什么能处理未见词？因为未见词可以退化为更小的子词、字符或字节片段组合。

WordPiece

一句话定义：WordPiece 是一种子词 tokenization 方法，常见于 BERT 系列模型。

核心直觉：它学习一组能较好表示语料的子词单元，并倾向于选择能提升语言模型似然的合并。

常见标记：BERT 中常用 ## 表示某个子词不是词首，例如 play ##ing。

与 BPE 的区别：BPE 更强调高频 pair 合并，WordPiece 更接近基于似然或得分选择子词。

优点：能在词表大小和未登录词处理之间取得平衡。

常见误区：WordPiece 不是简单按词典最大匹配，它背后有子词词表学习和切分策略。

面试表达：WordPiece 是 BERT 时代很典型的子词方案，适合解释为什么预训练模型不需要传统意义上的完整词表。

Unigram Language Model Tokenization

一句话定义：Unigram LM tokenizer 假设文本由子词片段概率生成，并通过概率模型选择切分。

核心思想：先准备较大的候选子词集合，再逐步删减对语料似然贡献较低的字词。

切分方式：同一句文本可能有多种切分，算法选择概率更高的切分。

在实践中：SentencePiece 支持 Unigram LM，是很多多语言模型 tokenizer 的选择之一。

优点：概率建模更明确，也支持 subword regularization 等训练增强方式。

面试追问：Unigram 和 BPE 的差异是什么？BPE 是从小到大合并，Unigram 通常从大候选集合中做概率筛选。

SentencePiece

一句话定义：SentencePiece 是一个可直接从原始文本训练 tokenizer 的工具和方法集合。

核心特点：把空格也视为普通符号的一部分，不强依赖预先分词。

为什么适合多语言：中文、日文等语言没有天然空格分词，SentencePiece 可以直接处理原始文本。

常见符号：SentencePiece 常用 `▬` 表示空格或词边界。

支持算法：BPE 和 Unigram LM。

在大模型中：LLaMA 等模型使用 SentencePiece tokenizer。

常见误区：SentencePiece 不是某一种单独切分算法，它既是工具，也支持不同训练算法。

面试表达：SentencePiece 的优势是端到端处理原始文本，减少对语言特定预分词器的依赖，因此很适合多语言大模型。

Byte-Level Tokenization

一句话定义：byte-level tokenization 从字节层面表示文本，保证几乎所有输入都可以被编码。

核心优势：任意 Unicode 字符最终都能转成字节序列，因此几乎没有 OOV。

适用场景：特殊字符、emoji、代码、混合语言、脏数据和用户输入鲁棒性要求高的场景。

局限：部分语言或特殊文本会被切成更多 token，降低上下文利用率并增加计算成本。

在 LLM 中：byte-level BPE 是常见方案，先保证字节级覆盖，再通过 BPE merge 缩短常见片段。

常见误区：byte-level 不代表每个 token 都是一个字节。很多 byte-level BPE tokenizer 会把高频字节片段合并成更长 token。

面试表达：byte-level tokenization 的核心价值是鲁棒覆盖，代价是某些文本上的 token 效率可能下降。

Character-Level Tokenization

一句话定义：character-level tokenization 以字符为基本单位切分文本。

优点：词表小，OOV 少，直观简单。

局限：序列长度长，模型需要自己学习从字符到词、短语和语义单位的组合，训练和推理成本更高。

与 byte-level 区别：字符是 Unicode 层面的符号，字节是编码后的底层表示；一个字符可能对应多个字节。

在大模型中：纯字符级方案不是主流 LLM 默认选择，但其思想常用于讨论鲁棒性和开放词汇问题。

面试表达：字符级方案牺牲序列长度换取覆盖率，子词方案则是在覆盖率和效率之间折中。

OOV Out-of-Vocabulary

一句话定义：OOV 指输入中出现 tokenizer 词表无法直接表示的词或符号。

传统词级问题：如果词表只包含完整词，未见词可能只能映射为 UNK。

子词方案缓解：BPE、WordPiece、SentencePiece 可以把未见词拆成更小片段，减少 OOV。

byte-level 方案：几乎可以消除 OOV，因为任意字符最终可分解为字节。

常见误区：没有 OOV 不代表表示质量好。一个罕见词如果被切成很多碎片，模型理解和生成仍可能变差。

面试表达：现代 LLM tokenizer 的目标不是只消除 OOV，还要让常见文本被高效、稳定、语义相对合理地表示。

Special Token

一句话定义：special token 是具有特殊语义或控制作用的 token。

常见例子：BOS、EOS、PAD、UNK、system、user、assistant、tool、image、audio。

在大模型中：special token 用于标记序列边界、角色边界、填充位置、多模态占位符和工具调用结构。

工程注意：新增 special token 后通常需要 resize embedding，并确保训练、微调、评估和推理模板一致。

风险：如果 special token 处理不一致，模型可能无法正确区分用户、助手、系统指令或工具结果。

常见误区：special token 不是普通文本字符串。它们通常在 tokenizer 中有独立 id，并可能被模型学到特殊行为。

面试表达：chat model 的行为很大程度依赖 special token 和 chat template，它们定义了对话数据的结构协议。

BOS Beginning of Sequence

一句话定义：BOS 是表示序列开始的特殊 token。

作用：告诉模型一个新序列或新样本开始，有助于统一训练样本格式。

在推理中：部分模型需要显式 BOS，部分 tokenizer 会自动添加。

工程注意：是否添加 BOS 必须与模型训练时的格式一致。

常见误区：所有模型都必须手动加 BOS。不同模型约定不同，应以 tokenizer 和 chat template 为准。

EOS End of Sequence

一句话定义：EOS 是表示序列结束的特殊 token。

作用：训练时告诉模型何时结束生成，推理时常作为停止条件。

在对话模型中：EOS 可能表示 assistant 回答结束，也可能与 turn 边界 token 共同使用。

工程注意：如果训练数据缺少 EOS，模型可能学不会正常停止；如果停止符配置错误，可能截断或无限生成。

常见误区：max_new_tokens 不是正常停止机制。好的模型应能在合适位置生成 EOS 或停止标记。

面试表达：EOS 影响生成终止行为，是训练格式和推理 stop condition 的连接点。

PAD Padding Token

一句话定义：PAD 是用于把不同长度序列补齐到同一长度的特殊 token。

为什么需要它：batch 训练时张量通常需要统一形状，短序列需要 padding。

配套机制：attention mask 用于告诉模型哪些位置是有效 token，哪些位置是 padding。

在 decoder-only LLM 中：很多模型原始预训练不需要 PAD，但微调或批量推理时可能需要设置 pad token。

工程注意：不要让 loss 计算在 PAD 位置生效，否则模型会学习无意义目标。

常见误区：把 PAD 简单设成 EOS 虽然常见，但要确认 attention mask、loss mask 和生成停止逻辑不会冲突。

UNK Unknown Token

一句话定义：UNK 用于表示词表中无法识别的未知 token。

在现代 tokenizer 中：由于子词和 byte-level 方法普及，UNK 的使用频率通常显著降低。

风险：大量 UNK 会导致信息丢失，因为不同未知文本都被压成同一个 id。

工程注意：如果数据中 UNK 比例异常高，通常说明 tokenizer 与数据语言或文本格式不匹配。

面试表达：现代 LLM 尽量避免把未知词直接映射为 UNK，而是拆成更小单位保留信息。

Chat Template

一句话定义：chat template 是把多轮对话消息转换为模型输入 token 序列的格式规则。

核心内容：角色标记、消息边界、system prompt 位置、assistant 起始标记、工具调用格式、结束标记。

例子：同样的 messages 列表，不同模型可能格式化为完全不同的字符串和 special token 序列。

为什么重要：chat model 训练时学习的是某种模板格式，推理时模板不一致会导致角色混乱、拒答异常或输出格式错误。

工程注意：不要手写猜测模板，优先使用 tokenizer 自带的 `apply_chat_template` 或官方格式。

常见误区：prompt 内容相同不代表模型输入相同。chat template 不同，实际 token 序列可能完全不同。

面试表达：chat template 是 instruct/chat 模型的输入协议，决定 system/user/assistant 等角色如何被模型识别。

Prompt

一句话定义：prompt 是提供给模型的输入上下文，用来约束或引导模型生成。

组成：任务说明、背景信息、示例、约束条件、输出格式、用户问题和必要的系统指令。

在 base model 中：prompt 通常就是一段普通文本上下文。

在 chat model 中：prompt 常由 system、user、assistant 历史消息经过 chat template 组装而成。

常见误区：prompt 不是越长越好。冗余、冲突或低质量上下文会增加成本并干扰模型。

面试表达：prompt engineering 的核心不是堆关键词，而是清晰定义任务、约束、上下文和输出协议。

Completion

一句话定义：completion 是模型在给定 prompt 后继续生成的文本。

在预训练中：语言模型学习根据前文预测后续 token，本质上就是 completion 式目标。

在指令模型中：completion 通常对应 assistant 的回答。

与 prompt 的关系：prompt 是条件，completion 是模型生成的结果；训练数据中二者边界必须清楚。

工程注意：SFT 数据要正确 mask prompt 部分，通常只对 assistant completion 计算 loss。

常见误区：把 prompt 和 completion 边界处理错，会让模型学习复读用户问题或错误角色文本。

Instruction

一句话定义：instruction 是用户或系统给模型的任务指令。

例子：翻译、总结、分类、改写、写代码、按 JSON 输出、扮演某种角色。

在 SFT 中：instruction-response 数据用于让模型学会遵循任务说明生成回答。

与 prompt 的区别：prompt 是完整输入上下文，instruction 是其中表达任务要求的部分。

常见误区：instruction 越复杂不一定越好。冲突指令、多目标指令和含糊约束会降低生成稳定性。

面试表达：指令微调的关键是让模型从纯续写转向理解并执行显式任务要求。

System Prompt

一句话定义：system prompt 是对模型行为、身份、边界和全局规则的高优先级指令。

常见作用：设定助手角色、安全边界、回答风格、工具使用规则和输出格式约束。

在 chat template 中：system prompt 通常位于对话最前面，并通过特殊角色标记与用户消息区分。

工程风险：如果训练和推理中 system prompt 格式不一致，模型可能忽视规则或把系统指令当普通文本。

常见误区：system prompt 不是绝对安全机制。模型仍可能受越狱提示、上下文冲突或能力限制影响。

面试表达：system prompt 是控制模型行为的重要接口，但真正可靠的系统还需要数据、对齐、安全策略和推理侧防护共同支撑。

Context Window

一句话定义：context window 是模型一次前向可接收的最大 token 长度。

核心单位：上下文窗口以 token 计，不以字符、汉字或英文单词计。

影响因素：位置编码、训练长度、attention 实现、KV cache、显存和推理框架限制。

与 tokenizer 的关系：同一段文本在不同 tokenizer 下 token 数可能不同，因此能放入上下文的实际字符量也不同。

常见误区：标称 128K context 不代表所有 128K token 信息都能被同等有效利用。长上下文还涉及注意力稀释、位置外推和检索能力问题。

面试表达：上下文窗口是 token 级限制，长上下文能力还要看训练数据、位置编码、注意力机制和评估结果。

Detokenization

一句话定义：detokenization 是把 token id 序列还原为文本的过程。

核心流程：token ids -> tokens -> text。

工程难点：空格、换行、Unicode、byte fallback、特殊 token 跳过和流式输出都会影响还原结果。

在生成中：模型每生成一个 token，推理系统通常会逐步 decode 并返回可读文本。

常见误区：decode 单个 token 再拼接，不一定等价于 decode 完整 token 序列，尤其在 byte-level 或特殊空格编码中要小心。

面试表达：tokenization 和 detokenization 共同定义模型的文本 I/O 协议，线上问题经常出在边界符、空格和特殊 token 处理上。

Tokenizer Training

一句话定义：tokenizer training 是从语料中学习词表、merge 规则或子词概率模型的过程。

核心输入：代表目标应用分布的文本语料，包括语言比例、代码比例、领域文本、特殊符号和格式化数据。

关键超参数：vocab size、算法类型、normalization、special tokens、character coverage、byte fallback。

为什么重要：tokenizer 一旦确定，通常会和模型训练绑定，后续随意更换成本很高。

数据影响：如果训练 tokenizer 的语料中中文、代码或数学公式不足，相关文本可能被切得很碎。

常见误区：只用通用语料训练 tokenizer 就能覆盖所有场景。面向代码、金融、医疗、多语言的模型需要考虑目标数据分布。

面试表达：tokenizer 训练是模型数据工程的一部分，它决定哪些文本模式被压缩为高效 token，哪些模式会被拆碎。

中文 Tokenization

一句话定义：中文 tokenization 需要处理没有天然空格分词边界的问题。

常见方式：按字、按词、按子词或使用 SentencePiece 直接从原始文本学习。

难点：中文词边界不显式，专有名词、新词、数字单位和中英混排很常见。

对 LLM 的影响：如果中文被切得过碎，相同语义会占用更多 token，降低上下文利用率并增加推理成本。

工程观察：中文模型或多语言模型通常需关注中文 token fertility，即每个汉字或每句话平均被切成多少 token。

常见误区：中文按字切就一定好。按字鲁棒但序列长，按词可能语义好但 OOV 和分词错误更多，子词方法是常见折中。

面试表达：中文 tokenizer 的重点是覆盖率、切分效率和语义边界之间的平衡。

Code Tokenization

一句话定义：code tokenization 是面向代码文本的切分方式，需要处理缩进、符号、标识符和多语言语法。

代码特点：大量括号、点号、下划线、驼峰命名、缩进、换行、字符串和特殊运算符。

关键影响：tokenizer 如果不适合代码，常见标识符和语法符号会被切得很碎，影响代码生成效率。

重要细节：空格和换行在 Python 等语言中有语义，不能随意 normalize 掉。

byte-level 优势：对特殊符号和罕见字符更鲁棒，适合处理真实代码仓库中的混杂文本。

常见误区：代码只是英文文本。代码有强格式和符号结构，tokenizer 需要保留足够结构信息。

面试表达：代码模型的 tokenizer 要重视标识符、缩进、符号和长尾字符，否则会直接影响上下文长度和生成质量。

Tokenizer 与 Embedding 扩展

一句话定义：扩展 tokenizer 通常意味着新增 token，并同步扩展模型 embedding 和输出层。

典型场景：新增 special token、领域术语、多模态占位符、工具调用标记或结构化输出标记。

工程步骤：更新 tokenizer，resize token embeddings，初始化新增 token 向量，并通过训练让模型学会使用这些 token。

风险：只改 tokenizer 不改模型会导致 token id 越界；只 resize 不训练，新 token 语义也不会自动学会。

在 tied embedding 中：输入 embedding 和输出 lm head 可能共享权重，扩展时要确认两者同步。

常见误区：新增 token 后模型马上理解它。新 token 的 embedding 初始通常是随机或均值初始化，需要微调学习。

面试表达：tokenizer 扩展是模型结构和数据协议变更，必须配合 embedding resize、初始化和训练数据设计。

Tokenizer Mismatch

一句话定义：tokenizer mismatch 指训练、微调、评估或推理阶段使用了不一致的 tokenizer 或模板。

常见后果：输入 id 错误、special token 错位、角色边界混乱、生成异常、评估结果不可比。

典型场景：用 A 模型 tokenizer 编码 B 模型 prompt，或者 SFT 使用一种 chat template，线上推理使用另一种模板。

工程排查：打印 token ids、decode 后文本、special token 位置和 attention mask，确认与训练格式一致。

常见误区：只要文本看起来一样，模型输入就一样。实际 token id 序列可能完全不同。

面试表达：tokenizer mismatch 是大模型工程中非常常见且隐蔽的问题，排查时必须看 token 级输入，而不只是看原始字符串。

本章小结

本章覆盖 NLP 与 tokenization 中最常见的面试概念和工程问题。

核心结论如下：

1. LLM 处理的是 token id，不是原始字符串。
2. token 不等于词、字符或字节，它由 tokenizer 的训练方式和词表决定。
3. BPE、WordPiece、Unigram LM 和 SentencePiece 都是在词表大小、覆盖率和序列长度之间做权衡。
4. byte-level tokenization 几乎消除 OOV，但可能牺牲部分语言的 token 效率。
5. special token 和 chat template 定义了对话模型的输入输出协议。
6. prompt、completion、instruction、system prompt 的边界处理会直接影响 SFT 和推理行为。
7. tokenizer mismatch、PAD/EOS 配置错误、模板不一致是大模型工程中的高频坑。
8. 面试中要能从 tokenization 连接到上下文长度、KV cache、训练成本、多语言能力和线上稳定性。

下一章，我们进入 Transformer 架构。

D. Transformer 架构

条目：Transformer、Token Embedding、Embedding Matrix、Position Embedding、Positional Encoding、Sinusoidal Positional Encoding、RoPE、RoPE Scaling、ALiBi、Long Context、Position Extrapolation、Lost in the Middle、Weight Tying、Contextual Representation、Self-Attention、Query、Key、Value、Attention Weight、Causal Self-Attention、Causal Mask、Padding Mask、Causal LM、Masked LM、Scaled Dot-Product Attention、Full Attention、Local Attention、Sparse Attention、Linear Attention、FlashAttention、Attention Complexity、Multi-Head Attention、Attention Head、MQA、GQA、KV Cache、Transformer Block、MLP / FFN、Residual Connection、Pre-LN、Post-LN、Decoder-Only Transformer、Encoder-Only Transformer、Encoder-Decoder Transformer、Cross-Attention。

Transformer

一句话定义：Transformer 是一种以 attention 为核心的信息交互架构，用堆叠的 attention、MLP、normalization 和 residual connection 对序列进行建模。

提出背景：早期 NLP 常用 RNN、LSTM、GRU 等循环结构建模序列。它们天然按时间步递推，适合表达顺序，但训练并行度差，长距离依赖难学。Transformer 用 self-attention 让任意位置之间可以直接交互，同时让训练过程在序列维度上高度并行。

解决的问题：Transformer 主要解决三件事。第一，增强长距离依赖建模能力。第二，提高训练并行效率。第三，让模型可以通过堆叠层数和扩大宽度形成强大的通用表示能力。

基本组成：一个典型 Transformer block 包含 attention 子层、MLP 子层、残差连接和归一化层。大语言模型中最常见的是 decoder-only Transformer，也就是 GPT 类自回归架构。

为什么适合大模型：attention 能根据上下文动态选择相关信息，MLP 提供非线性变换和知识存储能力，残差和归一化让深层网络可训练，整体结构规整，适合大规模 GPU/TPU 并行。

局限：标准 attention 的复杂度随序列长度平方增长，长上下文训练和推理成本很高；同时 Transformer 本身没有强归纳偏置，需要大量数据和算力学习语言规律。

面试表达：Transformer 的核心不是“多头”本身，而是用 self-attention 替代循环递推，让 token 之间可以全局交互，并把序列建模转化为高度并行的矩阵计算。

Token Embedding

一句话定义：token embedding 是把 token id 映射成连续向量的可训练查表层。

形状：embedding matrix 通常是 [vocab_size, hidden_size]。输入 token id 序列形状为 [B, T]，查表后变成 [B, T, hidden_size]。

为什么需要它：token id 只是离散编号，本身没有语义距离。模型不能直接对 id 的大小做语义计算，所以需要把每个离散 token 映射成连续向量。

训练方式：embedding 向量通过语言模型 loss 反向传播更新。频繁出现的 token 通常得到更充分训练，低频 token 和新增 token 更容易表示不足。

工程注意：tokenizer、vocabulary 和 embedding matrix 必须对齐。换 tokenizer 或新增 special token 后，通常需要 resize embedding，并决定新 token 向量如何初始化。

常见误区：embedding 不是固定词典解释，而是模型在训练中学习到的参数。一个 token 的静态 embedding 只表示初始输入，经过 Transformer 层后会变成上下文相关表示。

面试表达：token embedding 是离散文本进入神经网络的第一层接口，负责把 token id 转为模型可计算的稠密向量。

Embedding Matrix

一句话定义：embedding matrix 是存储所有 token 向量表示的参数矩阵。

参数规模：如果词表大小为 V ，隐藏维度为 d ，输入 embedding 参数量约为 $V * d$ 。对于大词表和大 hidden size，这部分参数并不小。

与输出层关系：语言模型最后需要把 hidden state 投影到词表维度得到 logits。很多模型会使用 weight tying，让输入 embedding 和输出 projection 共享权重。

新增 token 的影响：新增 token 会改变词表大小，需要扩展 embedding matrix 和输出层。如果只改 tokenizer 而不改模型参数，会出现 id 越界或语义错位。

工程坑：加载 checkpoint 时如果 tokenizer 和 embedding size 不一致，常见报错是 shape mismatch。正确做法是确认 tokenizer 版本、special token 数量、vocab size 和模型 config 是否一致。

面试表达：embedding matrix 是模型理解 token 的参数入口，tokenizer 的 id 映射必须和这个矩阵逐行对应。

Position Embedding

一句话定义：position embedding 是向模型注入 token 位置信息的机制。

为什么需要它：self-attention 本身对输入顺序没有天然感知。如果不加入位置信息，序列中的 token 交换顺序后，attention 计算缺少区分位置的依据。

常见形式：learned absolute position embedding、sinusoidal positional encoding、RoPE、ALiBi。现代 decoder-only LLM 中 RoPE 非常常见。

绝对位置与相对位置：绝对位置强调第几个 token，相对位置强调两个 token 之间相隔多远。长上下文外推通常更关注相对位置信息是否稳定。

工程注意：位置编码会影响上下文长度扩展。仅把 config 中的 max position 改大，通常不能保证模型能稳定处理更长文本。

面试表达：position embedding 解决的是 Transformer 对序列顺序不敏感的问题，它决定模型如何理解“谁在前、谁在后、相隔多远”。

Positional Encoding

一句话定义：positional encoding 是让模型知道 token 在序列中位置的编码方法。

核心问题：语言顺序很重要，但 attention 主要根据 Q/K 内容相似度计算。如果没有位置编码，模型很难区分“狗咬人”和“人咬狗”这类顺序变化。

主要类型：第一类是可学习绝对位置向量，直接为每个位置学习一个 embedding。第二类是固定函数位置编码，如正弦余弦编码。第三类是把相对位置融入 attention 计算，如 RoPE 和 ALiBi。

评价维度：一个位置编码方法通常要看训练稳定性、短上下文效果、长度外推能力、实现复杂度和与 KV Cache 的兼容性。

常见误区：支持更长位置编码不等于模型真正学会长程依赖。长上下文能力还依赖训练数据长度分布、attention 机制、优化过程和评估方式。

面试表达：位置编码不是附属细节，它直接影响模型对顺序、距离和长上下文的理解。

Sinusoidal Positional Encoding

一句话定义：sinusoidal positional encoding 是 Transformer 原论文中使用的固定正弦/余弦位置编码。

核心形式：不同维度使用不同频率的正弦和余弦函数表示位置。低频维度变化慢，高频维度变化快。

为什么提出：它不需要为每个位置学习参数，理论上可以计算训练长度之外的位置编码，具有一定外推直觉。

优点：无额外可训练参数，实现简单，原始 Transformer 中效果良好。

局限：在现代大规模 decoder-only LLM 中，它不如 RoPE 等相对位置方法常见；实际长上下文表现还取决于模型是否在长序列上训练过。

面试表达：正弦位置编码用固定周期函数表示位置，优势是简单和可外推，但现代 LLM 更常用能表达相对位置关系的方案。

RoPE / Rotary Position Embedding

一句话定义：RoPE 通过按位置旋转 Q/K 向量来注入位置信息。

核心直觉：不同位置对应不同旋转角度。两个位置的 Q/K 点积不仅包含内容相似度，也包含相对位置差带来的相位关系。

作用位置：RoPE 通常作用在 attention 的 query 和 key 上，而不是直接加到 token embedding 上。

为什么流行：RoPE 能自然建模相对位置信息，和 decoder-only Transformer、KV Cache 兼容较好，因此被许多现代 LLM 采用。

优点：表达相对位置关系清晰，短上下文效果好，实现上可以高效融合到 attention 中。

局限：训练长度之外的位置外推仍可能退化。长上下文扩展时需要 RoPE scaling、继续训练或其他长上下文技术配合。

工程坑：RoPE 的 base、维度分配、scaling 策略、max position 设置必须和 checkpoint 对齐。推理框架中 RoPE 参数不一致，会导致模型输出质量明显异常。

面试表达：RoPE 不是简单给 embedding 加位置向量，而是在 Q/K 空间做旋转，使 attention score 自然携带相对位置信息。

RoPE Scaling

一句话定义：RoPE scaling 是通过调整 RoPE 频率或位置映射来扩展模型上下文长度的方法。

为什么需要它：模型预训练时通常只见过有限长度。如果直接推理更长序列，高位置对应的旋转相位可能超出训练分布，导致注意力模式不稳定。

常见思路：一种是位置插值，把更长的位置压缩映射到训练长度范围附近；一种是调整频率 base 或不同维度的缩放方式；还有一些方法会结合长上下文继续训练。

优点：相对轻量，常可在不从零训练模型的情况下扩展可用上下文。

风险：可能影响短上下文能力，也不解决 attention 计算和 KV Cache 显存随长度增长的问题。

面试表达：RoPE scaling 解决的是位置编码外推问题，不等于完整解决长上下文问题；长上下文还要处理训练分布、检索能力、显存和评估。

ALiBi

一句话定义：ALiBi 是在 attention score 中加入与距离相关的线性偏置的位置建模方法。

核心机制：距离越远的 token，会根据 head 对应的斜率受到不同程度的 score 惩罚，从而把相对距离信息注入 attention。

优点：不需要显式位置 embedding，外推到更长长度时形式简单，推理时实现相对直接。

局限：线性距离偏置是一种较强归纳假设，不一定适合所有任务和模型规模。现代 LLM 中 RoPE 使用更广。

面试表达：ALiBi 通过给远距离 attention 加惩罚来编码相对位置，重点是把距离偏置直接加到 attention logits 上。

Long Context

一句话定义：long context 是模型处理远超传统窗口长度输入的能力。

核心挑战：第一，标准 attention 计算和显存复杂度是 $O(T^2)$ 。第二，推理时 KV Cache 随上下文长度线性增长。第三，位置编码需要外推或重新训练。第四，模型不一定能有效利用很远位置的信息。

能力分层：能接收长输入只是第一层；能检索长输入中的关键信息是第二层；能在长输入中做跨段推理、冲突处理和多证据整合才是更高层能力。

典型方案：位置编码扩展、长上下文继续预训练、稀疏或滑窗 attention、分块注意力、检索增强、KV Cache 压缩、外部记忆机制。

工程评估：不能只看最大 context length。还要看长文问答、needle-in-a-haystack、多跳检索、跨段总结、位置鲁棒性和实际延迟成本。

面试表达：长上下文不是把窗口参数调大，而是位置编码、训练数据、attention 成本、KV Cache 和评估共同决定的系统能力。

Position Extrapolation

一句话定义：position extrapolation 是模型在训练长度之外处理更长位置的能力。

为什么困难：训练时未见过的位置可能对应未学习过的编码模式，attention score 分布和层间表示都可能偏离训练分布。

常见现象：短上下文正常，超长上下文开始复读、忽略远端信息、答案不稳定、对中间信息不敏感。

改善方向：RoPE scaling、长上下文继续训练、更合理的位置插值、使用相对位置偏置、加入长文任务数据。

面试表达：位置外推关注的是“位置编码和模型是否能泛化到更长位置”，不是单纯显存能否装下长输入。

Lost in the Middle

一句话定义：lost in the middle 是长上下文模型更容易忽略上下文中间部分信息的现象。

典型表现：关键信息放在开头或结尾时模型更容易回答正确，放在中间时准确率下降。

原因直觉：训练数据和注意力模式可能让模型更偏好近邻、开头指令和结尾问题；长序列中中间信息竞争注意力资源，也更容易被稀释。

评估意义：它说明支持长窗口不等于能可靠利用所有位置的信息。

工程缓解：把关键约束放在开头和结尾，使用结构化提示，分段摘要，检索重排，显式引用证据位置，对长文任务做专项微调。

面试表达：长上下文评估不能只测是否能塞进去，还要测不同位置的信息是否都能被稳定使用。

Weight Tying

一句话定义：weight tying 是输入 embedding matrix 和输出词表投影矩阵共享参数。

为什么使用：语言模型输入端需要 token embedding，输出端需要把 hidden state 映射到 vocabulary logits。两者都和 token 语义空间相关，因此可以共享权重。

优点：减少参数量，提升参数利用率，让输入 token 表示和输出 token 分类共享空间。

局限：共享权重是一种约束，不一定适合所有结构；新增 token 或 resize embedding 时，需要同时关注输入和输出权重是否同步。

工程注意：某些模型 config 中会有 tie_word_embeddings。做 LoRA、量化、保存和加载时，要确认共享权重没有被错误拆开。

面试表达：weight tying 让模型输入词表表示和输出词表分类器使用同一套 token 参数，既省参数又形成语义约束。

Contextual Representation

一句话定义：contextual representation 是 token 经过 Transformer 层并结合上下文后得到的动态表示。

与 embedding 的区别：token embedding 是静态查表结果，同一个 token 在不同句子中初始向量相同；contextual representation 会随上下文变化。

例子：bank 在“river bank”和“bank account”中初始 token embedding 可能相同，但经过 attention 后的表示会分别偏向河岸和银行语义。

为什么重要：大模型的理解、推理和生成能力主要来自多层上下文表示，而不是单个静态词向量。

面试表达：Transformer 的每一层都在更新 token 的上下文表示，越高层越融合全局语境和任务信息。

Self-Attention

一句话定义：self-attention 是序列中每个 token 根据同一序列内其他 token 的信息更新自身表示的机制。

核心直觉：每个 token 先生成 query、key、value。query 表示“我想找什么”，key 表示“我能被如何匹配”，value 表示“我提供什么内容”。query 和 key 算相似度，softmax 得到权重，再加权汇总 value。

解决问题：self-attention 让任意两个位置可以直接交互，适合建模长距离依赖；同时矩阵计算可并行，训练效率高于逐步递推的 RNN。

标准公式： $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$ 。

优点：动态上下文选择、全局信息交互、并行训练、表达能力强。

局限：标准 attention 需要计算 $[T, T]$ attention matrix，复杂度为 $O(T^2)$ ，长序列成本高。

常见误区：attention weight 可以提供解释线索，但不等于完整因果解释。模型行为还受 value、MLP、残差、多层交互影响。

面试表达：self-attention 的本质是基于内容相似度的信息路由机制，每个 token 都能动态决定从哪些上下文位置聚合信息。

Query

一句话定义：query 是 attention 中表示 “当前 token 想找什么信息” 的向量。

来源：由当前 hidden state 通过线性投影得到，通常记为 $Q = X W_Q$ 。

类比：query 像搜索请求，表示当前位置要向上下文检索哪些信息。

形状：在多头注意力中，query 常被 reshape 成 $[B, \text{num_heads}, T, \text{head_dim}]$ 。

面试表达：query 不是人工写的查询语句，而是模型根据当前 token 表示动态生成的向量。

Key

一句话定义：key 是 attention 中表示 “当前 token 可被如何匹配” 的索引向量。

来源：由 hidden state 通过线性投影得到，通常记为 $K = X W_K$ 。

类比：key 像文档索引，用来和 query 计算匹配分数。

在 KV Cache 中：自回归推理会缓存历史 token 的 key，避免每生成一步都重复计算历史部分。

面试表达：key 决定一个 token 在 attention 匹配中以什么方式被其他 token 找到。

Value

一句话定义：value 是 attention 中真正被加权汇总的信息向量。

来源：由 hidden state 通过线性投影得到，通常记为 $V = X W_V$ 。

类比：value 像文档内容。query 和 key 决定查哪些位置，value 决定取回什么信息。

在 KV Cache 中：历史 token 的 value 也会被缓存，因为生成后续 token 时仍需聚合历史信息。

面试表达：attention score 只决定权重，最终进入下一层的信息来自 value 的加权和。

Attention Weight

一句话定义：attention weight 是当前 token 从其他 token 聚合信息时使用的权重。

计算方式：query 和 key 点积得到 attention score，经过缩放、mask 和 softmax 后得到权重。

性质：同一个 query 对所有可见 key 的权重通常归一化为 1。mask 掉的位置权重应接近 0。

解释风险：较大的 attention weight 说明某层某 head 在该步从某位置聚合较多 value，但不能直接等同于模型最终答案的因果依据。

工程用途：可用于可视化、debug 长上下文关注位置、检查 mask 是否生效、分析重复生成或信息泄漏。

面试表达：attention weight 是信息聚合权重，不是模型解释的全部。

Causal Self-Attention

一句话定义：causal self-attention 是当前位置只能关注自己和之前位置，不能关注未来位置的 self-attention。

典型场景：GPT 类自回归语言模型。

为什么需要：语言模型训练目标是预测下一个 token。如果当前位置能看到未来 token，就会产生信息泄漏，训练 loss 虽低但生成时不可用。

实现方式：使用 causal mask，在 softmax 前把未来位置的 attention scores 置为 $-\infty$ 或很大的负数。

面试表达：causal attention 保证第 t 个位置只使用 $\leq t$ 的上下文，从而匹配自回归生成过程。

Causal Mask

一句话定义：causal mask 是防止当前位置关注未来 token 的下三角 attention mask。

作用方式：在 softmax 前把未来位置的 attention scores 置为 $-\infty$ ，使其 softmax 权重变为 0。

为什么需要它：保证 causal LM 学习 $P(x_t | x_{1:t-1})$ ，防止训练时信息泄漏。

常见形状：训练时常对应 $[T, T]$ 下三角矩阵，批量和多头维度可广播。

工程坑：mask 方向写反会导致模型看不到历史或看到未来；padding mask 和 causal mask 需要正确组合；混合精度中使用极小负数时要避免数值溢出。

面试表达：causal mask 是自回归模型成立的必要约束，不是简单为了减少计算。

Padding Mask

一句话定义：padding mask 是防止模型关注 padding token 的 attention mask。

为什么需要：batch 内样本长度不同，通常会 pad 到相同长度。padding token 不是有效文本，不能参与语义建模。

与 causal mask 的区别：padding mask 处理无效 token，causal mask 处理未来 token。decoder-only 训练中经常需要二者同时生效。

工程坑：左 padding 和右 padding 对 position id、KV Cache、生成对齐都有影响。很多 decoder-only 模型推理时更偏好左 padding 或需要框架特别处理。

面试表达：padding mask 解决的是 batch 对齐带来的无效 token 干扰问题。

Causal LM

一句话定义：causal LM 是从左到右预测下一个 token 的语言模型。

训练目标：最大化 $P(x_1, x_2, \dots, x_T) = \prod_t P(x_t | x_{1:t-1})$ ，实现上常把输入右移一位作为 label。

典型模型：GPT 系列、LLaMA 系列、Qwen 系列等 decoder-only LLM。

特点：只能使用左侧上下文，天然适合续写、对话和代码生成。

局限：对纯理解类任务需要通过 prompt 转成生成任务；双向上下文建模不如 masked LM 直接。

面试表达：现代对话 LLM 大多是 causal LM，它们通过不断预测下一个 token 来完成生成。

Masked LM

一句话定义：masked LM 是随机 mask 输入 token，并用双向上下文预测被 mask token 的语言模型。

典型模型：BERT。

特点：可以同时利用左右上下文，适合理解、分类、检索表示等任务。

与 causal LM 的区别：masked LM 不是按从左到右方式生成完整文本，因此不天然适合开放式自回归生成。

局限：预训练中使用 [MASK] token，而下游真实输入通常没有 [MASK]，存在一定预训练和下游分布差异。

面试表达：BERT 类 masked LM 更偏理解表示，GPT 类 causal LM 更偏生成，两者的 attention mask 和训练目标不同。

Scaled Dot-Product Attention

一句话定义：scaled dot-product attention 是使用 $\text{softmax}(QK^T / \sqrt{d_k})V$ 计算注意力输出的方法。

为什么缩放：如果 d_k 很大，query 和 key 的点积方差会变大，softmax 容易饱和，导致梯度变小。除以 $\sqrt{d_k}$ 可以稳定分数尺度。

计算步骤：先算 QK^T 得到每个 query 对每个 key 的匹配分数；再加 mask；再 softmax；最后乘以 V 得到加权汇总结果。

复杂度：标准实现需要构造 $[T, T]$ 分数矩阵，计算和显存随序列长度平方增长。

面试表达：缩放点积注意力的关键是用点积做匹配、用 softmax 做归一化、用 value 加权和做信息聚合。

Full Attention

一句话定义：full attention 是每个 token 可以关注所有可见 token 的 attention。

优点：信息交互充分，表达力强，适合需要全局依赖的任务。

缺点：标准实现复杂度为 $O(T^2)$ ，长序列成本高。

在 decoder-only 中：由于 causal mask，当前位置可关注所有历史和当前位置，但不能关注未来。

面试表达：full attention 的优势是全局交互，代价是长度平方级成本。

Local Attention / Sliding Window Attention

一句话定义：local attention 是每个 token 只关注局部窗口内 token 的 attention。

为什么提出：长序列中很多依赖是局部的，只计算附近窗口可以显著降低 attention 成本。

优点：降低计算和显存，适合长文本、语音、视频等长序列任务。

缺点：远距离依赖受限。如果没有全局 token、跨窗口连接或层间信息传递，远端信息很难直接交互。

面试表达：sliding window attention 用局部性换效率，适合长序列，但需要额外机制弥补全局信息不足。

Sparse Attention

一句话定义：sparse attention 是只计算部分 token pair 注意力连接的 attention。

常见模式：local、global、block sparse、strided、random。

核心 trade-off：通过稀疏连接降低成本，但可能损失全局信息和实现通用性。

工程问题：理论复杂度降低不一定等于实际加速。GPU 上稀疏算子是否高效、是否支持训练和推理框架非常关键。

面试表达：sparse attention 的重点不是简单少算，而是设计一种既省计算又保留关键信息路径的连接模式。

Linear Attention

一句话定义：linear attention 是通过特征映射或计算重排避免显式构造 $T \times T$ attention matrix 的方法。

目标：把复杂度从 $O(T^2)$ 降到接近 $O(T)$ 。

核心思路：用核函数或特征映射近似 softmax attention，使 $\text{softmax}(QK^T)V$ 可以被重排为先聚合 K/V ，再与 Q 交互。

优点：长序列理论成本低。

局限：可能改变 attention 数学形式，质量、数值稳定性和硬件效率需要具体分析。很多线性 attention 在通用 LLM 上替代标准 attention 并不容易。

面试表达：linear attention 试图从算法形式上降低长度平方复杂度，但是否能保持 LLM 质量是关键问题。

FlashAttention

一句话定义：FlashAttention 是通过 IO-aware 分块计算优化精确 attention 的高效实现。

核心思想：标准 attention 的瓶颈不只是 FLOPs，还有 HBM 显存读写。FlashAttention 通过 tiling、在线 softmax 和重计算策略，减少中间 attention matrix 的显存读写。

重要特点：它通常不改变 attention 的数学结果，而是改变计算实现方式。

优点：显著降低显存占用，提高训练和推理速度，尤其适合长序列和大 batch。

局限：依赖硬件、CUDA kernel、数据类型和框架支持；不同版本对 mask、dropout、GQA、变长序列的支持可能不同。

面试表达：FlashAttention 是 exact attention 的高效 kernel，不是稀疏 attention，也不是近似 attention。

Attention Complexity

一句话定义：attention complexity 描述 attention 计算和显存如何随序列长度、head 数和维度增长。

标准成本：对于序列长度 T ，标准 full attention 的 score matrix 是 $[T, T]$ ，所以计算和中间显存核心项通常是 $O(T^2)$ 。

训练影响：长序列训练时，attention 激活显存和计算成本快速上升，需要 checkpointing、FlashAttention、序列并行等技术。

推理影响：prefill 阶段需要处理整个 prompt，attention 成本高；decode 阶段每步只生成一个 token，但 KV Cache 随历史长度增长。

面试表达：长上下文的瓶颈分 prefill 和 decode 两部分，prefill 关注 attention 计算，decode 关注 KV Cache 和逐步访存。

Multi-Head Attention / MHA

一句话定义：multi-head attention 是并行使用多个 attention head，在不同子空间中计算注意力的机制。

核心直觉：不同 head 可以学习不同类型的信息交互，例如局部语法、指代关系、位置模式、格式边界等。

形状：通常从 $[B, T, d_{\text{model}}]$ 投影为 $Q/K/V$ ，再 reshape 到 $[B, \text{num_heads}, T, \text{head_dim}]$ ，其中 $d_{\text{model}} = \text{num_heads} * \text{head_dim}$ 。

为什么需要多头：单个 attention 分布可能表达能力有限，多头允许模型同时从多个关系视角聚合上下文信息。

局限：head 的功能不一定清晰可解释；head 数越多也不一定越好，还要考虑 head_dim、算力和 KV Cache。

面试表达：MHA 是把隐藏维度拆成多个子空间并行做 attention，再拼接回去，从而增强信息交互的多样性。

Attention Head

一句话定义：attention head 是 multi-head attention 中的一组独立 $Q/K/V$ attention 子空间。

组成：每个 head 有自己的 $Q/K/V$ 投影子空间，输出后与其他 head 拼接，再经过输出投影。

可能功能：一些 head 可能偏位置，一些偏复制，一些偏语法关系，一些可能冗余。

剪枝问题：研究中可以剪掉部分 head，但生产模型不能简单假设某个 head 无用，需通过评估验证。

面试表达：head 是 attention 的并行子通道，提供多种匹配和聚合模式。

Multi-Query Attention / MQA

一句话定义：multi-query attention 是多个 query heads 共享一组 key/value heads 的 attention 变体。

为什么提出：自回归推理的 decode 阶段需要保存历史 K/V 。标准 MHA 每个 query head 都有自己的 K/V ，KV Cache 很大。MQA 让所有 query heads 共享 K/V ，大幅减少缓存。

核心价值：显著减少 KV Cache 显存和访存，提高长上下文推理效率。

代价：K/V 表达能力下降，可能影响模型质量，尤其是模型较小或训练不足时。

适用场景：推理效率要求高、上下文长、服务并发高的生成模型。

面试表达：MQA 是用共享 K/V 换推理效率，本质优化的是 KV Cache，而不是减少 query head 数。

Grouped-Query Attention / GQA

一句话定义：grouped-query attention 是 query heads 分组共享 key/value heads 的 attention 变体。

位置关系：GQA 介于 MHA 和 MQA 之间。MHA 是每个 query head 有独立 K/V；MQA 是所有 query head 共享一组 K/V；GQA 是一组 query heads 共享一组 K/V。

核心价值：在 MHA 的质量和 MQA 的推理效率之间折中。

工程收益：减少 KV Cache 大小和 decode 访存压力，适合现代 LLM 在线推理。

常见追问：为什么 GQA 能加速生成？因为 decode 阶段每步要读取历史 K/V，K/V head 数减少后缓存读写减少。

面试表达：GQA 不是 attention 近似，而是改变 Q head 与 K/V head 的共享关系，在质量和推理成本之间折中。

KV Cache

一句话定义：KV Cache 是自回归生成时缓存历史 token 的 key/value，避免重复计算历史部分。

为什么需要：生成第 $t+1$ 个 token 时，历史 token 的 K/V 不会变。如果每步都重新计算全部历史，会造成大量重复计算。

缓存内容：每一层 attention 的历史 key 和 value。形状通常与 batch、layer、kv_head、sequence length、head_dim 相关。

收益：decode 阶段从重复处理整个前缀，变成只处理新 token 的 Q，并与缓存的 K/V 做 attention。

成本：KV Cache 显存随 batch size、层数、上下文长度、K/V head 数和 head_dim 线性增长。长上下文和高并发服务中，KV Cache 往往是主要瓶颈。

与 MQA/GQA 的关系：减少 K/V head 数可以显著降低 KV Cache 显存和访存。

工程坑：不同请求长度不一会导致 cache 管理复杂；beam search、speculative decoding、prefix caching、paged attention 都需要正确处理 cache 生命周期。

面试表达：KV Cache 是 LLM 推理加速的核心机制，但它把计算压力转化为显存和访存压力。

Transformer Block

一句话定义：Transformer block 是 Transformer 中反复堆叠的基本模块，通常由 attention、MLP、normalization 和 residual connection 组成。

Decoder-only 常见形式： $x = x + \text{Attention}(\text{Norm}(x))$ ，然后 $x = x + \text{MLP}(\text{Norm}(x))$ 。这属于 Pre-LN 结构。

各部分作用：attention 负责 token 间信息交互，MLP 负责逐 token 的非线性变换和容量扩展，normalization 稳定训练，residual connection 保留原信息并改善梯度传播。

堆叠意义：单层只能做有限的信息变换，多层堆叠后模型可以逐步形成从局部模式到高级语义、推理和任务格式的表示。

工程注意：不同模型的 block 细节不同，例如 LayerNorm/RMSNorm、GELU/SwiGLU、MHA/GQA、是否使用 bias、RoPE 位置等。

面试表达：Transformer block 是 LLM 的基本积木，attention 负责混合 token，MLP 负责变换表示，残差和归一化保证深层可训练。

MLP / FFN

一句话定义：MLP 或 FFN 是 Transformer block 中对每个 token 位置独立作用的前馈网络。

核心作用：增加非线性表达能力和模型容量。attention 混合不同 token 的信息，MLP 则在每个位置上对混合后的表示进行复杂变换。

常见结构：传统 FFN 是 $d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$ ，中间接激活函数。现代 LLM 常用 SwiGLU、GEGLU 等门控 FFN 变体。

参数占比：在很多 LLM 中，MLP 参数量占比很大，甚至超过 attention。因此不能把 Transformer 能力只归因于 attention。

工程注意：MLP 是训练和推理中的主要计算来源之一，量化、张量并行和算子融合都会重点优化它。

面试表达：attention 解决信息从哪里来，MLP 解决信息如何非线性加工和存储。

Residual Connection

一句话定义：residual connection 是把子层输出加回输入的连接方式。

核心公式： $x = x + \text{sublayer}(x)$ 。

为什么需要：深层网络如果每层都完全重写表示，梯度传播和信息保留会很困难。残差连接让模型可以在原表示基础上学习增量修改。

作用：保留信息、改善梯度传播、支持深层训练、让子层可以学习接近恒等映射的变换。

工程注意：残差分支的数值尺度很重要。深层模型中初始化、normalization、residual scaling 都会影响训练稳定性。

面试表达：残差连接让深层 Transformer 更容易优化，本质是让每层学习对表示的增量更新。

Pre-LN

一句话定义：Pre-LN 是在子层之前做 normalization 的 Transformer 结构。

形式： $x = x + \text{sublayer}(\text{Norm}(x))$ 。

特点：现代 LLM 常用，深层训练更稳定，梯度更容易沿残差路径传播。

优点：训练稳定性好，适合堆叠很多层。

潜在问题：某些情况下最终输出表示可能需要额外 final norm；不同结构在收敛速度和最终效果上仍需实验比较。

面试表达：Pre-LN 把归一化放在 attention/MLP 前面，是现代大模型更常用的稳定训练结构。

Post-LN

一句话定义：Post-LN 是在残差相加之后做 normalization 的 Transformer 结构。

形式： $x = \text{Norm}(x + \text{sublayer}(x))$ 。

历史背景：原始 Transformer 使用 Post-LN。

特点：浅层或中等层数模型中可用，但很深模型训练时梯度稳定性通常不如 Pre-LN。

对比：Post-LN 每层输出被归一化，形式直观；Pre-LN 残差主路径更干净，深层优化更友好。

面试表达：Post-LN 是原始结构，Pre-LN 是现代 LLM 更常见的工程选择，主要差异在训练稳定性。

Decoder-Only Transformer

一句话定义：decoder-only Transformer 是只使用带 causal mask 的 Transformer decoder block 进行自回归生成的架构。

典型模型：GPT、LLaMA、Qwen、Mistral 等主流对话 LLM。

训练方式：输入 token 序列，模型在每个位置预测下一个 token。训练时使用 causal mask，推理时逐 token 生成。

优点：结构统一，训练和推理目标一致，天然适合开放式生成、对话、代码、工具调用和指令跟随。

局限：只能从左到右使用上下文。对纯理解、双向编码、句向量检索等任务，可能不如专门 encoder 模型高效。

为什么成为主流：大规模预训练加上指令微调和对齐后，decoder-only 架构可以把大量 NLP 任务统一成文本生成问题，扩展性和工程生态最好。

面试表达：现代 LLM 主流是 decoder-only causal LM，因为它把预训练、对话和生成统一到 next token prediction 上。

Encoder-Only Transformer

一句话定义：encoder-only Transformer 是使用双向 self-attention 编码输入序列的 Transformer 架构。

典型模型：BERT、RoBERTa、DeBERTa 等。

训练目标：常见是 masked language modeling，也可以结合 sentence order、对比学习等目标。

优点：每个位置可以同时看左右上下文，适合理解、分类、抽取、匹配、检索 embedding 等任务。

局限：不天然适合从左到右生成文本；如果要生成，通常需要额外解码器或改造目标。

面试表达：encoder-only 更像强大的文本理解编码器，输出的是输入文本的上下文表示，而不是直接自回归生成器。

Encoder-Decoder Transformer

一句话定义：encoder-decoder Transformer 是用 encoder 编码输入、decoder 通过 cross-attention 条件生成输出的架构。

典型模型：原始 Transformer、T5、BART 等。

结构：encoder 使用双向 self-attention 理解输入；decoder 使用 causal self-attention 生成输出，并通过 cross-attention 读取 encoder 表示。

适用任务：机器翻译、摘要、文本改写、结构化输入到文本输出等明确输入到输出转换任务。

优点：输入理解和输出生成分工清晰，适合 seq2seq 任务。

局限：结构比 decoder-only 更复杂，预训练和推理路径也更复杂；在通用对话 LLM 时代，很多任务被 decoder-only 统一处理。

面试表达：encoder-decoder 的核心是 encoder 负责读输入，decoder 负责写输出，中间通过 cross-attention 连接。

Cross-Attention

一句话定义：cross-attention 是 decoder 的 query 去关注 encoder 输出的 key/value 的 attention。

与 self-attention 的区别：self-attention 的 Q/K/V 来自同一个序列；cross-attention 的 Q 来自 decoder 当前状态，K/V 来自 encoder 表示或外部条件。

典型场景：机器翻译中 decoder 生成目标语言时关注源语言句子；多模态模型中 decoder 关注图像 encoder 特征；RAG 或工具场景中也可把外部记忆作为 K/V 来源。

作用：把条件信息注入生成过程，让输出依赖输入或外部模态。

工程注意：cross-attention 的 mask、缓存和 batch 对齐比普通 self-attention 更复杂，尤其在多模态和变长输入中。

面试表达：cross-attention 是条件生成的桥梁，让生成端在每一步读取编码端或外部信息。

本章小结

Transformer 架构可以从四条线理解。

第一条线是输入表示：token id 先进入 embedding，再通过位置编码获得顺序信息。

第二条线是 attention：Q/K/V、mask、多头、MHA/MQA/GQA、KV Cache 共同决定信息如何在 token 之间流动。

第三条线是 block 结构：attention 负责混合上下文，MLP 负责非线性加工，residual 和 normalization 保证深层网络可训练。

第四条线是模型形态：decoder-only 适合自回归生成，encoder-only 适合理解表示，encoder-decoder 适合输入到输出转换。

面试中如果被问 Transformer，不要只背公式。更好的回答是先说明它解决 RNN 难并行和长距离依赖的问题，再讲 self-attention 的 Q/K/V 机制，随后补充 mask、位置编码、block 结构、复杂度和现代 LLM 中的 MQA/GQA、KV Cache、FlashAttention 等工程优化。

下一章，我们进入大语言模型基础。

E. 大语言模型基础

条目：Language Model, Large Language Model / LLM, Autoregressive Model, Next-Token Prediction, Teacher Forcing, Label Shift, Logits, Softmax, Causal Language Model, Base Model, Instruct Model, Chat Model, Assistant Model, Prompt, System Prompt, Instruction Following, Context Window, Generation, Perplexity, Emergent Ability, In-Context Learning, Few-Shot Learning, Zero-Shot Learning, Label Space Coverage, Demonstration Format Consistency, Context Evidence Use, Middle Evidence Risk, Token Retrieval Audit, Retrieval Label Match, ICL Conflict Rate, ICL Gate, Chain-of-Thought, Memorization vs Generalization, Hallucination.

产品化补充条目：LLM Productization, Demo to Product, Scenario Selection, Real Demand, User Persona, Stakeholder Map, Task Journey, Product Metric, LLM Product Experience, Experience Gate, ROI Gate, Enterprise Gate, Productization Gate, Feedback Loop, Online Monitoring, Canary Rollout, Regression Pass Rate, Trace Coverage, Rollback Plan, Ops Gate, Product Interview Readiness, Product Interview Gate, Practitioner Interview Readiness, Project Evidence Score, Trade-off Depth, STAR Reflection Score, Expert Follow-up Readiness, Practitioner Interview Gate.

第二轮格式补充条目：Tokenizer Format Incident, Chat Template Drift, Assistant-Only Label Mask, Prompt Loss Leak, PAD Loss Leak, EOS Coverage.

第二十一册架构补充条目：Transformer Backbone, Architecture Selection Gate, Route Score, Training Parallelism, Content Routing, Scaling Evidence, Ecosystem Readiness, Deployment Pressure, Attention-SSM Hybrid Readiness, Attention Routing Audit, Route Hit Rate, Future Leak Rate, Softmax Row Sum Check, Scale Stability, Interpretation Boundary, QK Circuit, OV Circuit, Head Subspace Audit, Head Redundancy Rate, KV Head Ratio, Multi-head Latent Attention, Latent KV Cache, KV Cache Saving Rate, Attention Architecture Gate, Position-wise FFN, SwiGLU, Gated MLP, MoE FFN, Top-k Routing, Expert Capacity, Active Parameters, FFN / MoE Gate, Residual Stream, Norm Placement, Pre-LN, Post-LN, RMSNorm, Residual Scaling, Early Update Pressure, Normalization Stability Gate, Position Encoding, Sinusoidal Position Encoding, Learned Absolute Position Embedding, Relative Position Representation, Relative Position Bias, Rotary Position Embedding / RoPE, ALiBi, Position Interpolation, RoPE Scaling, Position ID Audit, Position Encoding Gate, Attention Mask, Loss Mask, Bidirectional Attention, Prefix LM, Encoder-Decoder Attention, Cross-Attention, Packed Sequence

Mask、Block-Diagonal Causal Mask、Sliding Window Attention、Mask Visibility Audit、Mask Gate、Parallel Path Length、Attention Pair Count、Quadratic Attention Cost、Attention Score Memory、Prefill Attention Cost、Decode Attention Cost、Attention Cost Gate、KV Cache Budget、KV Cache Per Token、Decode Bandwidth Pressure、Paged KV Block、KV Cache Gate、Training FLOPs Estimate、Compute-Optimal Training、Tokens per Parameter、Undertrained Model、Serving Pressure、Scaling Gate。

Language Model

一句话定义：语言模型是为 token 序列分配概率的模型。

核心直觉：它学习“在某个上下文中，接下来最可能出现什么”。如果模型看到“北京是中国的”，它应该给“首都”更高概率，而不是给一个无关 token 更高概率。

概率视角：给定序列 $x_1, x_2, \dots, x_T$ ，语言模型要估计整个序列出现的概率。自回归语言模型会把它分解成多个条件概率：

$$P(x_1, x_2, \dots, x_T) = P(x_1) * P(x_2 | x_1) * \dots * P(x_T | x_{1:T-1})$$

为什么重要：文本生成、翻译、问答、摘要、代码补全、对话、工具调用都可以被统一成“根据上下文继续生成 token”的问题。大语言模型之所以通用，核心在于它把很多任务都转化成语言建模问题。

与传统分类模型的区别：分类模型通常在固定标签集中选择答案，语言模型在整个词表上预测下一个 token。它的输出空间更大，也更灵活，因此可以生成开放式答案，但也更难控制真实性、格式和安全性。

面试表达：语言模型不是简单的词频统计表，而是一个条件概率模型。现代 LLM 用 Transformer 表示上下文，通过 next-token prediction 学到语言规律、知识关联、任务模式和推理模板。

常见追问：语言模型和搜索引擎有什么区别？搜索引擎主要检索已有文档，语言模型主要根据参数和上下文生成文本。RAG 系统会把两者结合，用检索提供外部依据，再让语言模型组织答案。

Large Language Model / LLM

一句话定义：大语言模型是参数规模、训练数据规模和训练计算量都很大的语言模型，通常具备较强的通用文本理解、生成和任务迁移能力。

核心直觉：LLM 不是一个人工编写的规则库，也不是简单存储问答对的数据库。它通过海量 token 训练，把语言模式、世界知识、代码模式、数学表达、对话风格和任务格式压缩进参数中。

为什么“大”有意义：模型规模扩大后，表示能力提升；训练 token 增多后，覆盖的语言现象、知识和任务形式更丰富；计算量提升后，模型有机会从数据中学到更复杂的统计结构。三者共同决定 LLM 的能力上限。

典型能力：文本生成、问答、摘要、翻译、代码生成、结构化抽取、分类、改写、推理辅助、多轮对话、工具调用规划等。

典型局限：LLM 优化的基础目标仍是 token 预测，不等于直接优化真理、逻辑正确性和用户满意度。因此它可能幻觉、过度自信、对提示敏感、长上下文退化、在分布外任务上失败。

工程视角：一个完整 LLM 产品通常不只是 base model，还包括 tokenizer、prompt 模板、后训练、对齐、安全策略、RAG、工具调用、推理加速、监控评估和反馈闭环。

面试表达：LLM 的本质是大规模条件概率模型，但工程价值来自规模化预训练带来的通用表示能力，以及后训练把这种能力对齐到用户指令和业务场景。

Next-Token Prediction

一句话定义：next-token prediction 是给定前文 token，预测下一个 token 的训练目标。

核心公式：

$$P(x_t | x_1, x_2, \dots, x_{\{t-1\}})$$

为什么提出：互联网上有大量未标注文本，如果要求人工给每条文本打标签，成本不可接受。next-token prediction 把文本自身变成监督信号：前面的 token 是输入，后一个 token 是标签。

训练过程：模型输入一段 token 序列，通过 causal mask 保证第 t 个位置只能看到 t 之前的信息，然后预测每个位置的下一个 token。对所有位置的预测计算 cross entropy loss，再反向传播更新参数。

优点：目标简单，数据便宜，扩展性强，天然适合生成任务。它不需要为每个下游任务单独设计标签，因此可以支撑大规模通用预训练。

局限：预测下一个 token 不直接保证事实正确、安全合规、推理严密或完全符合用户偏好。后续需要 instruction tuning、RLHF、DPO、RAG、工具调用和评估体系来补足。

面试表达：next-token prediction 是 LLM 预训练最核心的自监督目标。它看似简单，但在足够大模型、数据和算力下，可以让模型学到大量可迁移能力。

Autoregressive Model

一句话定义：自回归模型按从左到右的顺序生成序列，每一步都基于已经生成的历史 token 预测下一个 token。

核心公式：

$$P(x_1, \dots, x_T) = \prod_t P(x_t \mid x_{1:t-1})$$

典型模型：GPT、LLaMA、Qwen、DeepSeek、Mistral 等 decoder-only LLM 通常都是自回归模型。

为什么适合生成：生成文本本来就是逐步展开的过程。自回归建模和推理方式一致，每次用已有上下文预测下一个 token，再把新 token 拼回上下文继续预测。

与 masked language model 对比：masked LM 类模型如 BERT 训练时预测被遮盖位置，可以双向看上下文，适理解解和表征任务；自回归 LM 只能看左侧上下文，天然适合开放式生成。

工程限制：自回归推理阶段无法在生成长度维度完全并行，因为第 $t+1$ 个 token 依赖第 t 个 token。KV Cache、speculative decoding、continuous batching 等技术主要是在降低这个串行过程的成本。

面试表达：自回归模型通过链式法则建模序列概率，训练时可以并行计算所有位置的 loss，但推理时必须逐 token 生成。

Teacher Forcing

一句话定义：teacher forcing 是训练时使用真实历史 token 作为上下文来预测下一个 token 的方法。

核心直觉：训练时模型看到的是正确前缀，而不是自己刚生成的前缀。这样每个位置的预测都可以稳定计算，并且所有位置可以并行训练。

例子：训练句子“我喜欢机器学习”时，模型在“我喜欢”之后预测“机器”，即使它当前还不会正确生成“喜欢”，训练时仍然使用真实 token 作为上下文。

优点：训练稳定、并行效率高、实现简单，是 causal LM 训练的标准做法。

局限：训练和推理存在暴露偏差。推理时模型必须依赖自己的历史输出，一旦前面生成错误，后续上下文也会被污染，可能导致错误累积。

面试表达：teacher forcing 让自回归语言模型训练可以并行化，但它也解释了为什么模型推理时会出现生成漂移和错误累积。

Label Shift

一句话定义：label shift 是 causal language model 训练中输入 token 与监督 label 相差一个位置的做法。

例子：输入 token 是 $[x_1, x_2, x_3]$ ，对应目标 label 是 $[x_2, x_3, x_4]$ 。模型在第一个位置看到 x_1 ，要预测 x_2 ；在第二个位置看到 x_1, x_2 ，要预测 x_3 。

为什么需要：语言模型的目标是预测下一个 token，而不是重构当前 token。如果没有正确 shift，模型可能学到复制当前位置 token，loss 看起来下降，但生成能力错误。

工程坑：不同框架对 shift 的处理位置不同。有些模型内部会自动 shift logits 和 labels，有些训练代码需要外部手动处理。重复 shift 或漏 shift 都会造成训练异常。

排查方式：用一个极小 batch 打印 input ids、labels 和 loss mask，确认每个位置的 label 是否真的是下一个 token，并确认 padding 位置是否被 ignore_index 屏蔽。

面试表达：label shift 是 causal LM 训练正确性的基础细节。很多微调 loss 异常、模型学会复读、输出错位，都可能和 shift 或 mask 错误有关。

Assistant-Only Label Mask

一句话定义：assistant-only label mask 是 SFT 中只让 assistant response token 参与 loss，把 system、user、control token 和 padding 位置设为 ignore index 的做法。

为什么重要：SFT 的目标是让模型在给定 system / user 上下文后生成 assistant 答案，而不是学习复述提示词。如果 user token 参与 loss，模型可能更容易复述用户输入或污染角色边界。

工程检查：打印 input ids、decode 文本、role span、labels 和 ignore index；确认所有 assistant 内容和必要 EOS 被训练，prompt 与 PAD 不参与 loss。

面试表达：assistant-only mask 是聊天模型 SFT 的目标边界。很多“模型不听话”不是能力问题，而是 mask 把条件上下文也当成了目标。

Prompt Loss Leak

一句话定义：prompt loss leak 是 system、user、工具 schema、控制 token 或多模态占位符等提示部分错误参与训练 loss 的问题。

表现：模型复述用户问题、生成 system / user 角色文本、工具调用前缀混乱、对话边界不稳定，或者看似 loss 很低但指令跟随变差。

排查方式：按 role 统计 label 不等于 ignore index 的比例，prompt 侧应接近 0；多轮 SFT 要逐轮检查，不只检查最后一轮。

面试表达：prompt 是条件，不是目标。除非有非常明确的预训练或格式建模设计，SFT 中 prompt 参与 loss 通常是事故。

PAD Loss Leak

一句话定义：PAD loss leak 是 padding token 没有被 attention mask 和 label mask 正确屏蔽，导致 PAD 位置参与 loss 的事故。

表现：长短样本 loss 差异异常、batch padding 多时训练不稳定、模型生成 padding 相关 token 或结束行为异常。

排查方式：检查 attention_mask 中 PAD 为 0，labels 中 PAD 位置为 ignore index，并对不同长度样本分别统计有效 label 数。

面试表达：PAD 是 batch 对齐工具，不是语言内容。PAD 参与 loss 会把实现细节写进训练目标。

Logits

一句话定义：logits 是模型在 softmax 之前输出的未归一化分数。

形状：对于语言模型，logits 通常形状为 [batch_size, seq_len, vocab_size]。最后一维中每个值对应一个 token 的得分。

核心直觉：logit 越大，模型越倾向于选择该 token。但 logit 本身不是概率，不能直接解释为“70% 可能性”。需要经过 softmax 才能变成概率分布。

与解码关系：greedy decoding 选择 logit 最大的 token；sampling 会先经过 temperature、top-k、top-p 等处理，再从概率分布中采样。

数值注意：softmax 对 logit 差值敏感，而不是对绝对值敏感。所有 logits 同时加同一个常数，softmax 后概率不变。

面试表达：logits 是语言模型对整个词表的原始打分，后续的 softmax、采样策略和 loss 计算都基于它。

Softmax

一句话定义：softmax 是把 logits 转换成概率分布的函数。

核心公式：

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

作用：它让每个 token 的概率非负，并且所有 token 概率和为 1。

temperature 影响：temperature 会缩放 logits。温度低时分布更尖锐，模型更确定；温度高时分布更平滑，输出更多样但也更容易不稳定。

与 cross entropy：训练时通常对 logits 做 cross entropy。实现中常用 fused cross entropy，不会显式先算 softmax 再算 log，以提高数值稳定性。

面试表达：softmax 把模型对词表的原始打分归一化成概率分布，是训练 loss 和生成采样之间的桥梁。

Causal Language Model

一句话定义：causal language model 是只能根据过去 token 预测当前或未来 token 的语言模型。

关键机制：causal mask 会屏蔽未来位置，保证第 t 个位置不能看到 $t+1$ 之后的信息。如果没有 causal mask，训练 loss 会虚低，因为模型可以偷看答案。

典型架构：decoder-only Transformer。它没有 encoder-decoder 架构中的独立 encoder，也不使用 BERT 式双向 attention。

训练目标：通常是 next-token prediction。模型每个位置输出一个 hidden state，再通过 lm head 映射到词表 logits。

推理过程：输入 prompt，模型取最后一个位置的 logits 得到下一个 token 分布，选出新 token 后追加到上下文，再重复上述过程。

面试表达：causal LM 的“causal”指信息流方向受限，只允许从过去到未来，这正是 GPT 类模型可以做自回归生成的原因。

Base Model

一句话定义：base model 是经过大规模预训练但尚未经过指令微调和偏好对齐的基础模型。

典型表现：它擅长续写文本，但不一定会像聊天助手一样回答问题。如果输入“请解释 self-attention”，base model 可能继续生成类似网页、论文或问答语料的文本，而不一定稳定遵循指令。

为什么需要 base model：base model 是通用能力底座。它通过预训练获得语言、知识、代码、数学和世界模式，后续 instruct model、chat model 和领域模型都通常建立在 base model 之上。

优点：分布更接近原始预训练目标，适合继续预训练、领域适配、作为研究底座，也常用于分析模型原始能力。

局限：指令遵循弱，对话格式弱，安全边界弱，输出可能不符合用户期望。直接面向用户产品通常需要后训练。

面试表达：base model 是“会续写”的模型，不是天然“会听话”的助手。instruct tuning 和 alignment 的目标就是把它变成更会按用户意图工作的模型。

Instruct Model

一句话定义：instruct model 是经过指令微调后，能够更好理解并遵循用户指令的模型。

训练数据：常见数据形式包括 instruction、input、response 三元组，也包括问答、摘要、改写、代码解释、分类、推理等任务样本。

解决的问题：base model 只是预测文本续写，不一定知道用户希望它完成什么任务。指令微调让模型学习“看到任务描述后输出合适答案”的行为模式。

能力变化：instruct model 通常在零样本任务、格式遵循、问题回答、解释性输出方面明显优于 base model。但如果微调数据质量差，也可能导致能力退化、风格变窄或过度拒答。

与 chat model 的区别：instruct model 强调单轮或通用指令遵循；chat model 更强调多轮对话格式、角色消息、上下文承接和安全策略。

面试表达：instruct model 的核心是把预训练得到的通用能力映射到“按指令完成任务”的交互方式上。

Chat Model

一句话定义：chat model 是面向多轮对话场景优化的语言模型，通常能处理 system、user、assistant 等角色消息。

关键特征：它不仅要回答当前问题，还要理解对话历史、遵循系统指令、保持角色一致、处理多轮追问，并在必要时拒绝不安全请求。

数据格式：chat model 通常使用聊天模板把多轮消息序列化成 token，例如 system message、user message、assistant response 加上特殊分隔符。

工程注意：chat template 非常关键。同一个模型如果使用错误聊天模板，可能出现不回答、乱续写、角色混乱、特殊 token 泄漏等问题。

与 base model 对比：base model 更像文本续写器，chat model 更像对话助手。差异不只是模型权重，也包括 tokenizer、special token、prompt 模板和对齐策略。

面试表达：chat model 是经过对话数据和偏好对齐塑形后的 LLM，重点能力是多轮交互、指令层级遵循和助手风格输出。

Tokenizer Format Incident

一句话定义：tokenizer format incident 是 tokenizer、special token、chat template、EOS / PAD、label mask、截断或多模态 placeholder 配置错误导致模型行为异常的事故。

典型表现：模型不停生成、复述用户输入、忽略 system prompt、角色混乱、JSON / tool call 不稳定、多模态模型像没看到图、中文或代码成本异常。

排查顺序：打印原始 messages、渲染后的 prompt、token ids、decode 文本、special token 位置、attention mask、labels、EOS / PAD id、截断前后内容和模板版本。

面试表达：格式事故的核心是训练目标和推理协议不一致。先 decode 和看 labels，再讨论模型能力或超参。

Chat Template Drift

一句话定义：chat template drift 是训练、评估、推理或不同 checkpoint 使用的聊天模板不一致，导致 role、special token、assistant 起始位置或 EOS 语义变化。

为什么重要：模型只看到 token 序列，不知道开发者心里的 role 概念。模板一变，相当于模型协议变了，旧 checkpoint、旧评估和线上 prompt 可能都不再兼容。

工程做法：给模板版本化，训练日志记录模板 hash，评估和推理复用同一渲染函数；模板变更必须跑回归评估和灰度。

面试表达：chat template 不是普通 prompt 字符串，而是 chat model 的输入协议。

EOS Coverage

一句话定义：EOS coverage 是 assistant 结束位置或文档边界处正确出现 EOS，并且 EOS 在需要学习停止时参与 label 的覆盖程度。

常见事故：训练数据没有 EOS，EOS 被 mask 掉，推理 eos_token_id 配错，PAD 被当成 EOS，或者截断把 EOS 截掉。

影响：模型可能一直生成到最大长度，生成下一轮 user 模板，或者在多轮对话中边界混乱。

面试表达：模型会不会停，不只取决于 decoding stop words，也取决于训练中是否看过正确的 EOS 监督。

Assistant Model

一句话定义：assistant model 是被设计成通用助手形态的 chat model，目标是在安全边界内帮助用户完成任务。

核心要求：有帮助性、诚实性、无害性。它需要尽量解决用户问题，不知道时避免编造，同时拒绝明显有害请求。

产品化差异：一个可用助手通常还需要工具调用、检索增强、代码执行、文件读写、记忆管理、监控告警和人工反馈闭环。模型本身只是系统的一部分。

常见问题：过度迎合用户、幻觉、拒答过度、长对话遗忘、工具调用错误、格式不稳定、对系统提示被攻击敏感。

面试表达：assistant model 不是单纯追求最大似然文本生成，而是在模型能力、用户意图、安全约束和工具环境之间做平衡。

Prompt

一句话定义：prompt 是提供给模型的输入上下文，用来说明任务、约束、背景和期望输出。

组成：prompt 可以包含系统指令、用户问题、示例、上下文资料、输出格式要求、工具说明和约束条件。

为什么有效：LLM 具备 in-context learning 能力，可以从 prompt 中临时识别任务模式。高质量 prompt 能降低任务歧义，让模型更容易生成符合预期的答案。

局限：prompt 不是可靠的硬约束。复杂格式、强安全边界、事实性和长期一致性不能只依赖 prompt，需要训练、检索、工具、校验和评估配合。

面试表达：prompt 是运行时控制模型行为的主要接口，但它只能调动模型已有能力，不能凭空创造模型没有学到的稳定能力。

System Prompt

一句话定义：system prompt 是对模型行为、角色、安全边界和输出风格进行高优先级约束的提示。

作用：它告诉模型应该扮演什么角色、遵循哪些规则、优先满足哪些约束，以及如何处理冲突指令。

工程场景：客服助手、代码助手、数据分析助手、医疗问答助手等都会使用 system prompt 固定身份、能力边界和输出规范。

风险：system prompt 可能被 prompt injection 攻击绕过，也可能和用户需求冲突。真实系统需要结合权限控制、工具隔离、内容审核和输出校验。

面试表达：system prompt 是 LLM 应用的行为配置层，不是安全系统的全部。它能提高一致性，但不能替代模型对齐和工程防护。

Instruction Following

一句话定义：instruction following 是模型理解并执行用户自然语言指令的能力。

衡量维度：是否完成任务、是否遵守格式、是否覆盖约束、是否拒绝越界要求、是否在多约束冲突时合理取舍。

训练来源：指令微调、偏好对齐、高质量人工数据、合成指令数据、多轮对话数据和格式遵循数据。

常见失败：忽略部分约束、输出格式错误、回答跑题、在多轮对话中忘记先前要求、对否定指令理解错误。

面试表达：指令遵循不是简单问答能力，而是把自然语言任务描述转化成可执行生成行为的能力。

Context Window

一句话定义：context window 是模型一次推理中能够接收和处理的`最大上下文 token 长度`。

包括什么：上下文窗口包含 system prompt、用户输入、对话历史、检索文档、工具结果以及模型已经生成的 token。生成越长，可用输入空间越少。

为什么重要：长上下文可以容纳更多文档、代码、对话历史和复杂任务信息。但上下文更长不等于模型一定更会利用信息。

常见现象：模型可能在长上下文中忽略中间信息，也就是 lost in the middle；也可能因为检索内容太多、噪声太大而回答变差。

工程策略：做上下文压缩、检索重排、重要信息前置或后置、分块摘要、引用定位、长上下文评估和成本控制。

面试表达：context window 决定模型“能看多少”，但有效上下文利用能力决定模型“看了多少有用信息”。

Generation

一句话定义：generation 是模型根据已有上下文逐 token 产生新 token 的过程。

基本步骤：输入 prompt，模型 forward 得到最后位置 logits，解码策略选择下一个 token，把新 token 拼接到上下文，重复直到遇到停止条件。

停止条件：达到最大生成长度、生成 EOS token、命中 stop words、工具调用中断或系统策略终止。

解码方式：常见策略包括 greedy decoding、beam search、temperature sampling、top-k sampling、top-p sampling。不同策略会影响确定性、多样性和事实稳定性。

工程注意：生成质量不只由模型权重决定，也受 prompt、聊天模板、temperature、top-p、max tokens、重复惩罚、stop 配置和后处理影响。

面试表达：LLM 生成是自回归闭环过程。每一步的输出都会成为下一步的输入，因此早期错误可能被放大。

Perplexity

一句话定义：perplexity 是衡量语言模型预测测试集 token 困难程度的指标，通常可以理解为 cross entropy 的指数形式。

核心公式：

$$\text{PPL} = \exp(\text{loss})$$

其中 loss 通常是按 token 平均的 negative log likelihood。

直觉解释：perplexity 越低，说明模型对真实下一个 token 的平均概率越高，也就是越“不困惑”。

用途：预训练监控、模型规模对比、数据质量判断、领域继续预训练效果评估。若领域语料上的 PPL 下降，通常说明模型更适应该领域分布。

局限：PPL 不是万能指标。低 PPL 不一定代表指令遵循好、推理强、安全性好或用户体验好。不同 tokenizer、不同数据清洗方式、不同上下文长度下的 PPL 也不宜直接横向比较。

面试表达：perplexity 是语言建模能力的基础指标，适合看模型对文本分布的拟合程度，但不能替代下游任务评测和人工评测。

Context Evidence Use

一句话定义：context evidence use 衡量模型答案是否真正使用了上下文中的示例、文档证据或工具结果，而不是只靠参数先验。

面试表达：RAG 或长上下文系统不能只看是否检索到正确文档，还要检查最终答案是否引用并遵守上下文证据。

Middle Evidence Risk

一句话定义：middle evidence risk 指关键信息位于长上下文中间位置时更容易被模型忽略或弱化的风险。

面试表达：长上下文的关键不是能塞进多少 token，而是首部、中部、尾部证据都能稳定被读取和使用。

Token Retrieval Audit

一句话定义：token retrieval audit 是检查模型能否在上下文 token 中定位、读取并使用相关示例、标签、字段值和证据句的审计。

面试表达：它把 ICL、RAG 和长上下文统一到“上下文内显式读取能力”上，常用检索标签匹配率、位置分桶、冲突证据、引用检查和 bad case 复盘来验证。

Retrieval Label Match

一句话定义：retrieval label match 衡量最相关上下文示例或证据是否支持当前样本的正确标签或结论。

面试表达：few-shot prompt 不能只看“有没有相似示例”，还要看最相似示例是否对应正确标签；否则近尾冲突样本可能覆盖真正相关证据。

ICL Conflict Rate

一句话定义：ICL conflict rate 衡量相关上下文示例中标签或结论与当前正确答案冲突的比例。

面试表达：冲突率高说明 prompt 内部证据不一致，模型可能被位置更近、格式更强或相似度更高的错误示例带偏，应该进入 bad case 分桶而不是直接上线。

ICL Gate

一句话定义：ICL gate 是把标签空间覆盖、示例格式一致性、上下文证据使用、中间证据风险和冲突示例检查组合起来的 prompt 上线门禁。

面试表达：few-shot prompt 不能只凭主观感觉上线，应有验证集、位置鲁棒性测试、retrieval label match、冲突率、冲突样本和输出格式检查。

Transformer Backbone

一句话定义：Transformer backbone 是把 self-attention、FFN、residual、normalization、position encoding 和自回归目标组合成大模型主干的架构选择。

为什么重要：Transformer 不是因为单个模块神奇才成为主干，而是同时满足并行训练、内容相关路由、next-token prediction、scaling 证据、硬件友好和生态成熟。

面试表达：回答“为什么 Transformer 成为基础架构”时，要同时讲算法路由、训练并行性、统一目标、scaling law、生态和瓶颈，不能只复述 attention 公式。

Architecture Selection Gate

一句话定义：architecture selection gate 是用路由路径、训练并行性、内容路由、目标适配、scaling 证据、生态、硬件、ICL 和部署压力评估一个新架构是否适合作为基础模型主干的门禁。

面试表达：新架构不能只看单个 benchmark 或复杂度公式，还要证明它能稳定规模化训练、支持上下文学习、接入既有训练推理生态，并在真实成本上有收益。

Route Score

一句话定义：route score 衡量远距离 token 之间的信息通信路径是否短，self-attention 通常接近一跳，RNN 类递推会随距离增长。

Training Parallelism

一句话定义：training parallelism 衡量训练阶段能否同时处理序列中多个位置，并高效映射到 GPU / TPU 大矩阵计算。

Parallel Path Length

一句话定义：parallel path length 衡量两个位置之间的信息需要经过多少串行步骤才能交互。

面试表达：RNN 的远距离路径随序列长度增长，小卷积核 CNN 需要多层扩大感受野，self-attention 在可见范围内能让 token 一层直接交互。

Content Routing

一句话定义：content routing 衡量模型是否能根据 token 内容动态选择该读取哪些上下文信息，而不是只依赖固定局部窗口或固定状态递推。

Attention Routing Audit

一句话定义：attention routing audit 是检查 self-attention 是否在给定 Query、Key、mask 和期望证据下正确路由信息的审计方法。

面试表达：Self-Attention 不只要会写公式，还要能检查 route hit、future leak、softmax 行归一化、缩放稳定性、lost-in-the-middle 和解释边界。

Route Hit Rate

一句话定义：route hit rate 衡量 attention 最大权重位置是否命中期望证据 token 或人工标注的路由目标。

Future Leak Rate

一句话定义：future leak rate 衡量 causal self-attention 中未来 token 是否仍获得非零注意力权重。

Softmax Row Sum Check

一句话定义：softmax row sum check 检查 attention weight 每一行是否归一化为 1，是 attention 实现正确性的基本数值门禁。

Scale Stability

一句话定义：scale stability 衡量除以 $\sqrt{d_k}$ 后 attention logits 是否避免过尖 softmax 分布，并保持更稳定的梯度区间。

Interpretation Boundary

一句话定义：interpretation boundary 提醒 attention weight 只能作为某层某 head 的局部信息混合线索，不能直接等同于模型最终输出的因果解释。

QK Circuit

一句话定义：QK circuit 是 attention head 中由 Query 和 Key 投影共同决定的匹配电路，用来决定当前位置从哪些 token 读取信息。

面试表达：QK 只回答“读哪里”，不能单独说明“写入了什么信息”或“最终输出为什么这样”。

OV Circuit

一句话定义：OV circuit 是 attention head 中由 Value 投影和输出投影共同决定的写入电路，用来决定被读取 token 的内容如何写回 residual stream。

面试表达：同样的 attention map 可能因为 OV 写入不同而产生不同功能，所以分析 head 时不能只看 attention weight。

Head Subspace Audit

一句话定义：head subspace audit 是从 QK 路由命中、OV 写入目标、head 功能签名和 KV cache 成本审计 attention head 是否真的承担有效、可区分且成本合理的子空间路由。

面试表达：多头不是简单复制 attention，而是在多个低维子空间里并行路由；但训练后可能存在冗余，所以要同时讲表达收益、可解释性边界和推理成本。

Head Redundancy Rate

一句话定义：head redundancy rate 粗略衡量多个 attention heads 是否呈现相似路由和写入功能签名，可用于解释 head pruning 和压缩动机。

KV Head Ratio

一句话定义：KV head ratio 衡量 query heads 与 key/value heads 的数量比例，是理解 MHA、MQA、GQA 和 KV cache 节省的重要指标。

面试表达：GQA / MQA 的缓存收益来自减少 H_{kv} ，而不是取消多个 Query 子空间。

Multi-head Latent Attention

一句话定义：Multi-head Latent Attention / MLA 是把 K/V 相关历史信息压缩到 latent cache，再在 attention 中使用或恢复相关表示的注意力结构。

为什么重要：MLA 不是简单减少 KV head 数，而是进一步压缩 KV 表示本身，适合解释长上下文、高并发和 MoE serving 中的 KV cache 压力。

面试表达：GQA 主要减少 H_{kv} ，MLA 主要减少每个历史 token 需要缓存的表示维度；二者都服务于推理效率，但压缩层级不同。

Latent KV Cache

一句话定义：latent KV cache 是用低维 latent 表示保存历史 K/V 信息的缓存形式，目标是在保留多头表达能力的同时降低显存和带宽压力。

KV Cache Saving Rate

一句话定义：KV cache saving rate 衡量某种 attention 架构相对 MHA baseline 减少了多少 KV cache 显存。

面试表达：只说“更省显存”不够，应说明 baseline、batch、层数、上下文长度、KV head 数、head dim、精度和是否包含 latent / RoPE 额外缓存。

Attention Architecture Gate

一句话定义：attention architecture gate 是从质量、KV cache 节省率、吞吐延迟收益、实现成熟度和复杂度评估 MHA / MQA / GQA / MLA 是否适合上线的门禁。

面试表达：MHA、MQA、GQA、MLA 没有绝对最优，选型必须同时看质量门禁、serving 成本和工程成熟度。

Position-wise FFN

一句话定义：position-wise FFN 是 Transformer block 中对每个 token 位置独立应用同一组两层或门控 MLP 参数的通道变换模块。

面试表达：attention 主要混合序列维度的信息，FFN 主要在 hidden/channel 维度做非线性加工和容量扩展；它不是附属模块，很多模型的大量参数和计算都集中在 FFN/MLP。

SwiGLU

一句话定义：SwiGLU 是一种 gated MLP，把 SiLU gate branch 与 up branch 逐元素相乘，再用 down projection 投回 hidden size。

面试表达：SwiGLU 不是只换激活函数，而是多了一条 gate projection；比较参数量时要说明普通 4D FFN 约为 2dm，SwiGLU 约为 3dm，因此常把 intermediate size 调到接近 8d/3。

Gated MLP

一句话定义：gated MLP 是用门控分支控制候选通道特征通过程度的 MLP 结构，SwiGLU 是现代 LLM 常见 gated MLP 形式。

面试表达：gated MLP 的核心直觉是通道级选择性激活，工程上仍主要由矩阵乘、逐元素激活和逐元素乘法组成，复杂度远低于 MoE routing。

MoE FFN

一句话定义：MoE FFN 是把 dense FFN 替换为多个 experts，并由 router 为每个 token 选择少数 experts 计算的稀疏激活 FFN。

面试表达：MoE 的价值是扩大总参数容量，同时让每 token 只激活 top-k experts；它通常替换 FFN 而不是 attention，因为 FFN 本来就是 token-wise 独立计算且是容量主体。

Top-k Routing

一句话定义：top-k routing 是 MoE router 根据 softmax 权重为每个 token 选择得分最高的 k 个 experts。

面试表达：top-1 routing 更省、更简单，top-2 routing 表达更强但通信和 combine 更复杂；无论 top-k 怎么选，都要配合负载均衡和 capacity 约束。

Expert Capacity

一句话定义：expert capacity 是 MoE 中单个 expert 在一个 batch 或 dispatch step 中最多接收多少 token 的容量约束。

面试表达: capacity 太小会 drop 或 overflow token, capacity 太大又会浪费 padding 和通信资源; 它必须和 batch size、top-k、expert 数和 capacity factor 一起估算。

Active Parameters

一句话定义: active parameters 是一次前向计算中实际参与某个 token 计算的参数量, MoE 中通常远小于总参数量。

面试表达: 说 MoE 模型大小时要区分 total parameters 和 active parameters; 总参数大代表容量空间大, active parameters 才更接近每 token 计算成本。

FFN / MoE Gate

一句话定义: FFN / MoE gate 是从 dense 参数量、SwiGLU 等参配置、MoE active parameters、capacity overflow、load balance 和 expert 利用率审计 FFN 架构是否合理的门禁。

面试表达: MoE 不是 active parameters 越低越好, 还要看 router 是否 collapse、expert 是否过载、all-to-all 通信是否可控, 以及质量和 serving 延迟是否过线。

Residual Stream

一句话定义: residual stream 是 Transformer block 中被 attention、FFN/MLP 和其他子层逐层写入增量信息的主隐藏状态通道。

面试表达: Residual 不是简单相加, 它让信息和梯度沿主路径流动; attention 和 FFN 更像是在同一条 residual stream 上写入不同类型的增量。

Norm Placement

一句话定义: norm placement 是 LayerNorm / RMSNorm 相对子层和 residual add 的放置方式, 典型形式包括 Pre-LN 和 Post-LN。

面试表达: norm placement 会改变残差主路径、梯度流和 warmup 依赖, 不能孤立地说某个 norm 一定更好。

Pre-LN

一句话定义: Pre-LN 是先对输入做 LayerNorm / RMSNorm, 再进入 attention 或 FFN 子层, 最后与原 residual stream 相加的结构。

面试表达: Pre-LN 保留更干净的 identity residual path, 深层训练更稳定, 因此现代 decoder-only LLM 常见; 代价是 residual stream 尺度仍需 final norm、初始化和 residual scaling 控制。

Post-LN

一句话定义: Post-LN 是先执行子层和 residual add, 再对相加结果做 LayerNorm 的结构。

面试表达: Post-LN 每层输出尺度更统一, 但深层训练通常更依赖 warmup; DeepNorm 等方法试图结合 Post-LN 表达和深层稳定性。

RMSNorm

一句话定义: RMSNorm 是只按 root mean square 做 re-scaling、不显式减均值的归一化方法。

面试表达: RMSNorm 的重点不是 “LayerNorm 的粗糙近似”, 而是保留尺度归一化、去掉 re-centering, 计算更简单, 并被 LLaMA 等现代 LLM 广泛采用。

Residual Scaling

一句话定义：residual scaling 是对每层 residual branch 写入幅度做缩放，防止深层模型中每层增量持续放大 residual stream。

面试表达：当层数很深时，norm 本身不一定足够，还要配合 residual scaling、初始化、learning rate 和 optimizer 控制每层更新幅度。

Early Update Pressure

一句话定义：early update pressure 是教学审计中用早期学习率和最大子层更新比例粗略衡量训练初期参数扰动风险的指标。

面试表达：warmup 的价值可以理解为降低早期有效更新幅度，尤其是在 Post-LN 或初始化梯度较大的设置下。

Normalization Stability Gate

一句话定义：normalization stability gate 是从 activation norm、update ratio、mean drift、early update pressure 和 NaN / overflow 检查 residual + norm 结构是否稳定的门禁。

面试表达：训练稳定性不是只看 LayerNorm/RMSNorm，还要同时看 residual stream 尺度、grad norm、learning rate、warmup、mixed precision、data outlier 和分布式通信异常。

Position Encoding

一句话定义：position encoding 是把 token 的顺序、绝对位置、相对距离或位置偏置注入 Transformer 的机制。

面试表达：Self-Attention 本身只根据 Q/K/V 做内容相关路由，不天然知道先后顺序；位置编码可以加到 embedding、attention score、K/V 表示或 Q/K 几何结构中。

Sinusoidal Position Encoding

一句话定义：sinusoidal position encoding 用不同频率的 sin/cos 函数为每个位置生成固定向量。

面试表达：它不需要学习参数，可以生成训练长度外的位置向量，但“公式能生成”不等于模型一定学会上下文外推。

Learned Absolute Position Embedding

一句话定义：learned absolute position embedding 为每个绝对位置学习一个向量，并与 token embedding 相加。

面试表达：它实现简单，在固定最大长度内有效，但超过训练位置后没有自然外推能力，也不直接表达“距离我多远”这种相对关系。

Relative Position Representation

一句话定义：relative position representation 把 query token 与 key token 的相对距离表示加入 attention 交互。

面试表达：attention 是 token pair 之间的交互，相对位置方法更直接地建模 pairwise distance，但朴素实现可能增加长序列内存和计算压力。

Relative Position Bias

一句话定义：relative position bias 不引入完整位置向量，而是在 attention score 上加一个与相对距离或距离 bucket 对应的标量 bias。

面试表达：T5 风格 relative bias 更轻量，近距离可以细分、远距离可以粗分，但表达能力通常弱于完整相对位置表示。

Rotary Position Embedding / RoPE

一句话定义：RoPE 根据位置对 Q/K 做旋转，使旋转后点积自然包含相对位置差。

面试表达：RoPE 用绝对 position id 实现，但在 Q/K 点积中体现相对位置关系；它兼容 causal attention 和 KV cache，是 LLaMA-like decoder-only LLM 的常见位置编码路线。

ALiBi

一句话定义：ALiBi 在 attention score 上加入与距离成比例的线性 bias，距离越远惩罚越大。

面试表达：ALiBi 不需要 position embedding，外推友好且实现简单，但会引入较强 recency bias，不一定适合所有远距离精确匹配任务。

Position Interpolation

一句话定义：Position Interpolation 把目标长上下文中的位置压缩回训练上下文尺度，缓解 RoPE 直接外推到未训练相位区域的问题。

面试表达：插值能降低长位置相位失配风险，但会降低位置分辨率，通常还需要少量长上下文继续训练和真实长文任务评估。

RoPE Scaling

一句话定义：RoPE scaling 是通过调整位置映射、频率尺度或不同频段缩放来扩展 RoPE 上下文窗口的一类方法。

面试表达：RoPE scaling 不能只看配置项是否存在，还要看训练长度、目标长度、短上下文退化、KV cache position id 和长上下文评测。

Position ID Audit

一句话定义：position id audit 检查 position id 是否与真实 token 位置、KV cache 长度、packed sequence 边界和 long-context scaling 配置一致。

面试表达：很多位置编码线上问题不是公式错，而是继续生成时 position id 从 0 重启、prefix cache 偏移错误、packed sample 位置泄漏或长上下文只改 max length。

Position Encoding Gate

一句话定义：position encoding gate 是把 cache position 连续性、packed sequence 隔离性和长上下文 scaling 配置组合成的位置编码审计门禁。

面试表达：位置编码要和 attention mask、KV cache、sliding window、prefix cache、长上下文扩展和评估任务一起审计，不能孤立看 RoPE 或 ALiBi 公式。

Attention Mask

一句话定义：attention mask 控制每个 query 位置可以从哪些 key/value 位置读取信息。

面试表达：attention mask 不是删除 token，也不是决定哪些 token 算 loss；它是在 softmax 前改变 attention score，决定信息流可见性。

Loss Mask

一句话定义：loss mask 控制哪些 token 参与训练损失。

面试表达：SFT 中 user token 通常不参与 loss，但 assistant token 必须能通过 attention 看见 user prompt，所以 loss mask 不能直接当 attention mask 用。

Bidirectional Attention

一句话定义：bidirectional attention 允许每个非 padding token 同时读取左侧和右侧上下文。

面试表达：BERT 类 encoder-only 模型依赖双向 attention 学习理解表征；它适合分类、抽取、匹配、rerank 和 embedding，但不是天然 left-to-right 生成器。

Prefix LM

一句话定义：Prefix LM 让 prefix 内部双向可见，target 可以看全部 prefix 且在 target 内部因果可见。

面试表达：Prefix LM 把条件输入和生成输出放在同一条 self-attention 序列里，通过 mask 模拟“理解输入 + 因果生成输出”。

Encoder-Decoder Attention

一句话定义：encoder-decoder attention 是 encoder 双向编码 source、decoder 因果生成 target，并由 decoder cross-attention 读取 encoder 输出的结构。

面试表达：翻译、摘要、改写等输入输出结构差异明显的任务常适合 encoder-decoder；decoder self-attention 防未来泄漏，cross-attention 负责读取 source。

Cross-Attention

一句话定义：cross-attention 中 Q 来自当前模块，K/V 来自另一个序列或模块输出。

面试表达：encoder-decoder 里 decoder 用 cross-attention 读取 encoder 表示；多模态模型里文本 token 也常通过 cross-attention 读取图像、音频或检索证据。

Packed Sequence Mask

一句话定义：packed sequence mask 在把多条样本拼成一个长序列训练时，阻止不同样本之间互相可见。

面试表达：只用普通 causal mask 会让后面的样本看到前面样本，造成跨样本信息泄漏、loss 虚低和文档边界污染。

Block-Diagonal Causal Mask

一句话定义：block-diagonal causal mask 在每个样本内部使用下三角 causal 可见性，不同样本 block 之间完全不可见。

面试表达：它是 packed training 的核心防泄漏机制，要和 position id、loss mask、EOS 边界和数据 packing 逻辑一起审计。

Sliding Window Attention

一句话定义：sliding window attention 只允许 token 读取局部窗口内的历史或邻近 token。

面试表达：它用局部可见性降低长上下文成本，但远距离信息不能直接访问，依赖多层传播、global token 或其他机制补足。

Mask Visibility Audit

一句话定义：mask visibility audit 检查 future leak、cross-sample leak、全 mask 行、mask 方向、query/key 长度偏移和 kernel fast path 是否符合预期。

面试表达：mask 审计要看 decoded text、attention mask、padding mask、loss mask、labels、position ids、sample boundary 和 logits/labels shift，不能只看张量 shape。

Mask Gate

一句话定义：mask gate 是把未来泄漏率、跨样本泄漏率、每行可见 key 覆盖和高效 kernel 支持组合成的注意力可见性门禁。

面试表达：mask 数学上正确还不够；如果全 mask 行导致 NaN、packed 边界泄漏或复杂 dense mask 退化到慢 kernel，训练和推理仍会出事故。

Scaling Evidence

一句话定义：scaling evidence 衡量一个架构是否在足够大的参数量、训练 token 数、计算预算和任务范围上表现出稳定可扩展收益。

Attention Pair Count

一句话定义：attention pair count 是 attention score 矩阵中的 query-key pair 数，full attention 长度为 n 时约为 n^2 。

面试表达：Transformer 用更多可并行 pair 计算换取短路径和动态路由，长上下文时这个平方成本会成为显存和延迟瓶颈。

Quadratic Attention Cost

一句话定义：quadratic attention cost 指 full attention 的 query-key pair、 QK^T / AV 计算和 score/probability 中间张量随序列长度平方增长。

面试表达：从 4K 到 32K 不是 8 倍成本，而是 attention pair 约 64 倍；FlashAttention 可以减少 IO 和中间显存，但 full attention 的 pair 数仍是平方级。

Attention Score Memory

一句话定义：attention score memory 是 materialize S 或 A 这类 $[B, H, T_q, T_k]$ 中间矩阵时占用的显存。

面试表达：32K、32 heads、bf16/fp16 下单个 full score tensor 已约 64 GiB；如果同时保存 scores 和 probabilities，朴素实现会更难承受。

Prefill Attention Cost

一句话定义：prefill attention cost 是推理开始处理 prompt 时 $T_q=T_k=n$ 的 attention 成本，长 prompt 下仍然接近训练阶段的平方级。

面试表达：TTFT 经常受 prefill 影响；长 prompt 的首 token 慢，不能只用 decode 单步成本解释。

Decode Attention Cost

一句话定义：decode attention cost 是自回归生成时单个新 token 查询历史 KV cache 的成本，单步通常随历史长度线性增长。

面试表达：decode 单步不是 n^2 ，但长输出会累积，并且 KV cache 显存和内存带宽常成为瓶颈。

Attention Cost Gate

一句话定义：attention cost gate 是把 pair count、score/probability 显存、prefill FLOPs、decode 单步成本、window ratio 和质量风险组合起来的长上下文 attention 成本门禁。

面试表达：长上下文方案不能只说支持多少 token，要同时报告 prefill、decode、显存、kernel 路径、可见模式变化和长距离证据召回风险。

KV Cache Budget

一句话定义：KV cache budget 是 serving 系统为活跃请求历史 K/V 状态预留和动态分配的显存预算。

面试表达：KV cache 不压缩模型权重，而是新增运行时状态；它随 $L * B * T * H_{kv} * D_h * 2 * \text{bytes}$ 线性增长，长上下文和高并发下常先卡住 batch size。

KV Cache Per Token

一句话定义：KV cache per token 是每个新 token 给单个请求追加到所有层的 K/V cache 大小。

面试表达：长输出不是只多算几个 logits；每个 decode token 都会追加 cache，并增加后续 token 需要读取的历史 K/V。

Decode Bandwidth Pressure

一句话定义：decode bandwidth pressure 指自回归 decode 单步需要反复读取历史 KV cache，导致显存带宽而非 FLOPs 成为瓶颈的压力。

面试表达：prefill 更像大矩阵计算，decode 更像小 batch、长历史的持续访存；MQA/GQA/MLA 的重要收益就是降低每步读取的 K/V 量。

Paged KV Block

一句话定义：paged KV block 是 PagedAttention / vLLM 类系统中管理 KV cache 的固定大小物理块，通过 block table 映射逻辑 token 序列。

面试表达：分页不是新的 attention 公式，而是降低变长请求碎片、支持按需分配、释放、prefix sharing 和 continuous batching 的内存管理方法。

KV Cache Gate

一句话定义：KV cache gate 是把 cache 显存、每 token 增长、decode 读取带宽、PagedAttention block 浪费、GQA/MQA/MLA 质量风险和上线 SLO 组合起来的推理门禁。

面试表达：长上下文上线不能只证明单请求能跑，还要证明在目标并发、TTFT/TPOT、显存、带宽和质量回归下能稳定服务。

Training FLOPs Estimate

一句话定义：training FLOPs estimate 是训练计算量的粗略估计，dense Transformer 常用 C_{train} 近似正比于参数量乘训练 token 数。

面试表达：6ND 只是教学和早期规划估算，真实成本还要看激活重算、并行效率、通信、optimizer、硬件利用率和失败重跑。

Compute-Optimal Training

一句话定义：compute-optimal training 是在固定训练计算预算下平衡参数量和训练 token 数，而不是只把模型做大。

面试表达：Chinchilla 的核心启发是很多大模型 undertrained；给定 compute 时，需要同时扩参数和数据，并结合数据质量与推理成本。

Tokens per Parameter

一句话定义：tokens per parameter 是训练 token 数与参数量的比例，用来粗略观察模型是否训练充分。

面试表达：这个比例不能机械套固定数字，但过低常提示 undertraining 风险，过高则可能说明容量不足或数据重复收益递减。

Undertrained Model

一句话定义：undertrained model 指参数量相对训练 token 数过大，模型容量没有被足够数据充分训练的情况。

面试表达：undertrained 不等于模型小，而是参数、数据和 compute 配比不合理；更小但训练更充分的模型可能实际更强。

Serving Pressure

一句话定义：serving pressure 衡量候选模型在推理延迟、显存、吞吐、KV cache、成本和上线 SLO 上的压力。

面试表达：训练 loss 更低不等于产品可用；如果 serving pressure 超预算，可能需要更小模型、蒸馏、量化、MoE/GQA、缓存或路由策略。

Scaling Gate

一句话定义：scaling gate 是把路径长度、compute balance、数据质量、训练稳定性和 serving pressure 组合成的 scaling 方案门禁。

面试表达：scaling 决策要同时看 loss trend、tokens per parameter、数据质量、训练稳定性、推理预算和评估切片，不能只看参数量或单个 benchmark。

Ecosystem Readiness

一句话定义：ecosystem readiness 衡量一个架构是否有成熟 kernel、分布式训练、推理框架、模型实现、评估基准、量化压缩和部署经验。

Deployment Pressure

一句话定义：deployment pressure 衡量一个架构在长上下文效率、KV / state 缓存、流式状态、延迟和显存上的工程压力。

Attention-SSM Hybrid Readiness

一句话定义：attention-SSM hybrid readiness 衡量 Attention + SSM 等混合架构是否同时保留内容检索和上下文学习能力，并降低长序列成本和流式状态压力。

面试表达：混合架构的价值不在于“替代 Transformer”这个口号，而在于证明哪些层用 attention、哪些层用状态或卷积，能在质量、成本和生态迁移上形成稳定收益。

Few-Shot Learning

一句话定义：few-shot learning 是在 prompt 中提供少量示例，让模型根据示例完成新样本的任务方式。

作用：示例能明确任务边界、输出格式、标签空间和隐含规则，通常比单纯文字说明更稳定。

适用场景：分类标签不直观、输出格式严格、业务规则特殊、模型容易误解任务时，few-shot prompt 往往很有效。

示例选择：示例要覆盖典型情况和容易混淆的边界情况，格式保持一致，避免示例本身包含错误标签或多余噪声。

局限：示例会占用上下文窗口；示例过多可能引入冲突；模型可能模仿示例中的偏差；少样本效果不等于真实泛化能力一定强。

面试表达：few-shot 的价值是用少量上下文示例约束模型行为，尤其是约束输出格式和任务映射关系。

Zero-Shot Learning

一句话定义：zero-shot learning 是不提供任务示例，只通过任务描述让模型直接完成任务。

例子：直接输入“请判断下面评论的情感是正面还是负面：这家店服务很好”，模型输出“正面”。

为什么可行：大模型预训练和指令微调中见过大量任务描述和解题模式，因此能把自然语言指令映射到已学到的能力上。

优点：prompt 简洁，不占示例空间，适合标准任务和模型已熟悉的任务。缺点是对复杂规则、特殊格式和业务边界不如 few-shot 稳定。

评估注意：zero-shot 表现受 prompt wording 影响很大。同一个任务换一种问法，结果可能明显变化，因此评测时要固定提示模板或做多模板评估。

面试表达：zero-shot 体现的是模型从自然语言指令直接泛化到任务的能力，是衡量 instruction following 和预训练知识迁移的重要方式。

Chain-of-Thought

一句话定义：chain-of-thought 是让模型生成中间推理步骤，再给出最终答案的提示或训练方式。

为什么有效：复杂问题往往需要多步计算、条件判断或中间状态。让模型显式展开推理过程，可以降低一步到位输出答案的难度。

典型场景：数学应用题、逻辑推理、多跳问答、代码推演、计划生成等。

局限：模型写出的推理步骤不一定真实反映内部计算，也可能出现“步骤看起来合理但结论错误”。在生产系统中，最终答案仍需要验证器、工具或规则校验。

安全注意：有些产品会把详细隐式推理隐藏，只输出简洁解释或关键依据，以降低泄漏内部策略和增加冗长错误的风险。

面试表达：CoT 的本质是把复杂任务分解成中间 token 序列，让模型有更多计算轨迹可用，但它不是正确性的充分保证。

Memorization vs Generalization

一句话定义：memorization 是模型复现训练数据中的具体内容，generalization 是模型把学到的规律应用到未见样本。

核心直觉：大模型既会记忆，也会泛化。高频文本、重复数据、特殊字符串、代码片段和评测答案更容易被记住；抽象语法、任务格式和推理模式更接近泛化。

如何区分：可以用去重评测集、时间切分评测、扰动测试、反事实样本、私有测试集和数据污染检查来判断模型是否只是记住答案。

为什么重要：如果模型在 benchmark 上表现好只是因为训练集中见过题目，评估结果就不可信。数据污染是大模型评测中的核心风险之一。

面试表达：不能简单说大模型“只是背题”或“完全理解”。更准确的说法是，它在参数中同时包含记忆和可泛化模式，评估设计要尽量区分二者。

Hallucination

一句话定义：hallucination 是模型生成看似合理但不真实、无依据或与上下文冲突的内容。

根本原因：语言模型优化的是 token 概率，而不是事实数据库查询。一个答案只要在语言模式上合理，就可能被模型高置信度生成，即使事实错误。

常见类型：编造引用、编造论文、编造 API、错误归因、数字计算错误、把过时知识当成事实、在上下文没有依据时强行回答。

缓解方法：RAG 提供外部依据，工具调用进行计算和查询，训练中加入拒答和不确定性表达，评估中加入事实性检查，产品中加入引用、校验和人工审核。

局限：幻觉不能被完全消除，只能在具体场景中通过数据、模型、工具和流程降低风险。

面试表达：幻觉来自生成式目标和事实性目标之间的不一致。解决思路不是只靠更大模型，而是结合检索、工具、校验、对齐和评估闭环。

LLM Productization

一句话定义：LLM productization 是把大模型能力转成稳定用户价值、业务指标、可靠系统和持续反馈闭环的过程。

核心问题：用户是谁、痛点是否高频或高价值、为什么需要大模型、成功指标是什么、成本是否可控、风险是否可控、上线后如何监控和迭代。

面试表达：产品化不是接一个模型 API，而是把模型能力放进真实 workflow，用评估、成本、延迟、安全和反馈闭环证明它能持续创造价值。

Demo to Product

一句话定义：demo to product 是从“样例能跑通”升级到“真实用户长期可用”的过程。

关键差异：demo 输入少、边界可控、失败可接受；产品要面对长尾输入、并发、稳定性、权限、合规、成本、监控、灰度和回归测试。

常见误区：把精心挑选样例上的惊艳效果，当成真实生产环境中的可持续价值。

Scenario Selection

一句话定义：scenario selection 是在大模型产品化前筛选“哪个用户、哪条任务链路、哪个环节最值得做”的过程。

核心维度：真实需求、用户痛点、任务频率、单次价值、数据准备度、可验证性、大模型适配度、工作流适配、剩余风险和落地难度。

面试表达：场景选择不是问“模型能不能做”，而是问“用户是否真的需要、收益是否覆盖成本、错误是否可控、能否进入受控试点”。

Real Demand

一句话定义：real demand 是用户已经在用时间、预算、流程或替代方案解决的问题，而不是只停留在口头兴趣的大模型想象。

判断方法：看当前谁在做、多久做一次、每次成本多少、现有方案哪里痛、成功后哪个指标改善、用户是否愿意接入工作流。

反面概念：fake demand 通常没有明确用户、没有当前成本、没有可衡量指标，或者规则和搜索已经足够解决。

User Persona

一句话定义：user persona 是对目标使用者的角色、目标、约束、痛点和工作流的结构化描述。

大模型场景：同一个企业知识库助手，对一线客服、主管、IT、法务和管理层的价值与风险完全不同，不能只写“企业用户”。

Stakeholder Map

一句话定义：stakeholder map 是把使用者、决策者、数据负责人、合规方、运维方和业务 owner 放在同一张责任图里。

为什么重要：很多大模型项目失败不是模型不够强，而是使用者嫌麻烦、决策者只看概念、技术团队只看模型指标，目标不一致。

Task Journey

一句话定义：task journey 是用户完成一个真实任务的完整步骤链路。

大模型场景：不要只说“帮销售写方案”，要拆成查客户、找案例、选卖点、写初稿、人工修改、审批和发送，才能判断模型介入哪一步最有价值。

Product Metric

一句话定义：product metric 是衡量用户价值和业务效果的指标，例如任务完成率、自助解决率、采纳率、转人工率、满意度、留存、成本下降和收入提升。

和技术指标关系：召回率、引用准确率、测试通过率等技术指标需要连接到用户和业务指标，才说明项目有产品闭环。

Model-to-Experience Mapping

一句话定义：model-to-experience mapping 是把理解、生成、检索、推理、工具调用、结构化输出和安全拒答等模型能力，映射到用户可感知的体验指标。

核心判断：模型 benchmark 提升不等于体验提升，必须继续看任务完成率、P95 延迟、引用支持、格式稳定、可控性、错误恢复、信任校准和风险门禁。

LLM Product Experience

一句话定义：LLM product experience 是用户在真实工作流中感知到的大模型产品质量，包括快不快、准不准、稳不稳、能否控制、能否信任、出错后能否恢复。

面试表达：用户不会直接感知模型参数量或公开榜单，而会感知任务是否更快完成、答案是否有证据、输出是否能直接使用、错误是否可纠正。

Experience Metric

一句话定义：experience metric 是衡量大模型产品体验的指标，例如任务成功率、首次成功率、采纳率、追问率、重新生成率、人工接管率、引用点击率和留存率。

和评估指标关系：离线 eval 负责证明模型或系统在样本集上过线，体验指标负责证明真实用户是否愿意持续使用。

Structured Output Stability

一句话定义：structured output stability 是模型输出持续符合 JSON、表格、字段或业务模板要求的比例。

为什么重要：内容看起来正确但格式不稳定，会导致后续系统无法解析、审批流无法接入、批处理任务失败。

Citation Support Rate

一句话定义：citation support rate 是被引用证据真实支持的关键结论比例。

大模型场景：企业 RAG 和知识助手不能只要求“有引用”，还要检查引用是否存在、相关、足以支持 claim，并且没有过期或越权。

Controllability

一句话定义：controllability 是用户能否通过长度、风格、格式、受众、数据范围、保守程度和编辑入口控制模型输出。

产品化意义：可控性差会让用户反复改 prompt；可控性好则把 prompt 工程隐藏为模板、选项、字段和工作流控件。

Error Recovery Rate

一句话定义：error recovery rate 是模型或产品出现失败后，通过追问、重新生成、修改约束、工具校验或人工接管恢复成功的比例。

面试表达：大模型产品不能假装永远正确，关键是错误可见、可解释、可反馈、可恢复。

Trust Calibration

一句话定义：trust calibration 是让用户既不过度信任，也不过度怀疑模型输出的能力。

常见机制：展示来源、标注不确定性、区分事实和推断、对高风险结论提示复核、避免无证据自信表达。

Experience Gate

一句话定义：experience gate 是大模型产品体验上线门禁，覆盖任务成功率、P95 延迟、引用支持、格式稳定、可控性、恢复、安全和过度拒答。

面试表达：如果体验门禁不过线，换更强模型不一定解决问题，可能需要改交互、结构化输出、引用校验、流式返回、缓存、反馈或人工接管。

Frequency-Value Matrix

一句话定义：frequency-value matrix 是用任务频率和单次价值粗分候选场景优先级的方法。

判断口径：高频低单价场景看总量和自动化收益；低频高价值场景看专家时间、风险降低和质量提升；低频低价值通常不适合作为首个项目。

Data Readiness

一句话定义：data readiness 是场景所需数据是否可访问、结构化、及时、干净、权限清楚、可用于评估和可持续更新的程度。

常见失败：RAG 项目答不好，往往不是模型不会答，而是知识库过期、权限混乱、缺少标准答案或没有反馈数据。

Verifiability

一句话定义：verifiability 是模型输出能否被测试、规则、证据、人工标注或业务指标验证。

落地影响：可验证任务更适合自动化和规模化；难验证任务需要人工反馈、灰度、抽检和更保守的产品形态。

LLM Fit

一句话定义：LLM fit 是判断一个需求是否真的需要大模型，而不是规则、搜索、模板或传统机器学习。

适合 LLM：自然语言表达多样、上下文复杂、长尾问题多、需要生成或总结、传统规则难以覆盖。

不适合 LLM：规则稳定、搜索结果已经足够、输出格式完全确定、错误成本高但缺少验证闭环。

Automation Level

一句话定义：automation level 是产品从 assist、recommend、review、approval 到 fully automate 的自动化层级。

产品化原则：低风险高可验证任务可以逐步自动化；低容错或高风险任务应优先做人机协同、引用、审批和审计。

Pilot Scenario

一句话定义：pilot scenario 是边界清楚、数据可得、风险可控、指标可衡量且能短周期验证价值的试点场景。

常见误区：试点不是展示型 demo，而是受控范围内验证真实用户价值、工作流适配和单位经济账。

Scenario Selection Gate

一句话定义：scenario selection gate 是候选场景进入试点前的门禁，覆盖真实需求、收益、大模型适配度、数据、验证、风险和工作流。

面试表达：如果痛点、收益、数据、验证、风险或工作流不过线，即使模型 demo 效果好，也应该暂缓、拆分试点或放弃。

Unit Economics

一句话定义：unit economics 是单次任务、单个用户或单个订单的收益和成本账。

大模型场景：要同时计算模型调用、检索、工具、人工审核、日志监控、运维和预期风险成本，而不是只看 token 单价。

Total Cost of Ownership

一句话定义：total cost of ownership 是一个大模型产品从试点到长期运行的总拥有成本。

大模型场景：除了 token 或 GPU，还要算 RAG、向量库、工具调用、人审、数据治理、研发摊销、监控告警、合规安全、事故处理和模型升级。

Variable Cost

一句话定义：variable cost 是随任务量增长而增长的可变成本。

大模型场景：模型调用、输入输出 token、检索、rerank、工具调用、重试和人审通常都是可变成本。单次可变成本为负毛利时，规模越大亏损越多。

Fixed Cost

一句话定义：fixed cost 是短期内不随任务量线性变化的固定成本。

大模型场景：研发摊销、评估集建设、权限系统、监控、日志、合规审计和固定基础设施通常属于固定成本。固定成本高的项目需要足够任务量才能摊平。

Model Cost

一句话定义：model cost 是模型推理调用带来的成本，通常由输入 token、输出 token、调用次数、缓存命中、批处理折扣和模型单价共同决定。

面试表达：不能只看单次回答报价，要按真实 prompt、RAG 上下文、输出长度、重试率和多轮 Agent 调用估算。

Review Cost

一句话定义：review cost 是人在回路审核、复核、质检、投诉处理和反馈标注带来的成本。

常见误区：模型节省了 5 分钟，但人审又花 4 分钟，实际收益会很低。人机协同要让人只看关键风险点。

ROI

一句话定义：ROI 是投入产出比，常用于判断项目收益是否覆盖成本。

常见口径：收益成本比可以写成 $\text{benefit} / \text{cost}$ ；净 ROI 可以写成 $(\text{benefit} - \text{cost}) / \text{cost}$ 。面试中要先说明自己采用哪种口径。

Net Benefit

一句话定义：net benefit 是收益减去成本后的净收益。

大模型场景：月净收益为正只是基本条件，还要继续看回本周期、风险、质量门禁和是否损害用户体验。

Break-Even Point

一句话定义：break-even point 是收益刚好覆盖成本时所需的任务量、用户量或收入规模。

面试表达：如果单次毛利为负，盈亏平衡点不存在；如果固定成本高，就需要足够任务量摊平固定投入。

Sensitivity Analysis

一句话定义：sensitivity analysis 是检查 ROI 对采用率、质量提升、token 成本、人审成本、重试率和固定成本变化的敏感程度。

为什么重要：ROI 估算依赖假设，试点阶段必须知道哪些变量最容易把项目从盈利打成亏损。

Payback Period

一句话定义：payback period 是一次性投入靠净收益收回所需的时间。

大模型场景：如果月净收益小于等于 0，就不要报一个看似精确的回本月数，而应明确当前假设下无法回本。

ROI Gate

一句话定义：ROI gate 是产品化前的成本收益门禁，覆盖净收益、收益成本比、回本周期、单次毛利、质量、延迟和风险。

面试表达：ROI 门禁不过线时，常见修复不是“换更贵模型”，而是提高采用率、减少上下文、加缓存、降人审成本、限制 Agent 步数、拆试点或换更高价值场景。

Enterprise LLM Application

一句话定义：enterprise LLM application 是接入企业身份、权限、数据、 workflow、审计、SLO 和业务指标的大模型应用。

和普通应用区别：企业应用不能只看回答体验，还要保证用户只能访问授权数据，模型和工具动作可追溯，结果能进入真实业务流程，并且收益覆盖成本和治理投入。

Enterprise Knowledge Base

一句话定义：enterprise knowledge base 是面向企业内部制度、产品文档、技术文档、工单经验和业务流程的知识库系统。

大模型场景：企业知识库通常通过 RAG 使用，但成败取决于文档版本、元数据、权限、引用支持、数据新鲜度和反馈闭环，而不只是 embedding 模型。

Single Sign-On / SSO

一句话定义：SSO 是企业用户使用统一身份系统登录多个应用的机制。

大模型场景：SSO 让 LLM 应用继承企业身份和用户组，便于做权限过滤、审计日志、离职回收和租户隔离。

Identity and Access Management / IAM

一句话定义：IAM 是管理身份、认证、授权、用户组、服务账号和访问策略的控制面。

面试表达：企业 LLM 应用要把 IAM 放在后端检索、工具调用、日志查看和管理配置中，而不是只在前端做一次登录判断。

Role-Based Access Control / RBAC

一句话定义：RBAC 是按角色授予权限的访问控制方法，例如客服、主管、法务、管理员有不同数据和工具权限。

大模型场景：RBAC 需要贯穿 RAG 召回、引用展示、Agent 工具调用和日志查询，避免模型把无权数据通过生成结果泄露出来。

Tenant Isolation

一句话定义：tenant isolation 是保证不同租户、部门、项目或客户的数据、索引、日志和配置不会互相泄露的隔离机制。

企业门禁：租户隔离违规率通常应接近 0；平均准确率再高，也不能覆盖跨租户数据泄露风险。

Permission-Aware RAG

一句话定义：permission-aware RAG 是在检索、重排、上下文拼接、引用展示和日志记录中都执行权限过滤的 RAG。

常见误区：只在 UI 隐藏无权文档不够，向量检索阶段和生成上下文阶段也必须过滤，否则模型可能从无权片段中总结出敏感信息。

Tool Permission Gate

一句话定义：tool permission gate 是 Agent 或 LLM 调用企业工具前，对用户身份、角色、动作、数据范围和风险等级做授权检查的门禁。

大模型场景：读数据、改数据、发邮件、改权限、执行代码、创建工单和退款等动作要分级处理，高风险动作需要二次确认、dry run、人审和审计。

Audit Log Coverage

一句话定义：audit log coverage 是请求 trace 中完整记录身份、数据来源、引用、工具调用、权限判断、模型版本、输出和复核状态的比例。

为什么重要：没有审计覆盖，系统出错后无法复盘，也无法证明数据访问和工具调用符合企业流程。

Data Freshness

一句话定义：data freshness 是知识库、指标口径、文档版本和业务数据是否足够新的程度。

大模型场景：企业 RAG 引用过期文档会造成错误决策。数据新鲜度要结合更新时间、版本号、owner、失效策略和业务阈值一起看。

Workflow Integration

一句话定义：workflow integration 是大模型应用进入企业已有工单、审批、CRM、ERP、BI、代码仓库或办公流程的程度。

产品化意义：没有工作流集成，LLM 往往只是聊天 demo；接入工作流后，才能衡量任务完成、节省时间、转人工率、审批通过率和业务收益。

Enterprise Gate

一句话定义：enterprise gate 是企业 LLM 应用上线前的综合门禁，覆盖权限、租户隔离、RAG 权限过滤、工具权限、审计、PII 脱敏、数据新鲜度、引用、SSO、工作流、评估、反馈、SLO、业务指标和高风险人审。

面试表达：企业门禁不是一个平均分，而是一组硬约束。权限、租户隔离、审计或高风险人审不过线时，即使 demo 效果好，也只能继续试点或改成人机协同。

Productization Gate

一句话定义：productization gate 是大模型产品上线前的综合门禁，覆盖痛点、质量提升、单位经济账、延迟、安全合规、工作流适配、评估和反馈闭环。

面试表达：一个模型效果更好的方案，如果成本、延迟、风险或工作流不过线，也不应该直接产品化。

Feedback Loop

一句话定义：feedback loop 是产品上线后把日志、用户反馈、人工审核、失败样本、A/B test 和线上指标转成下一轮评估与优化数据的闭环。

为什么重要：没有反馈闭环，系统很难适应真实用户长尾、知识库变化、模型升级和业务流程变化。

Online Monitoring

一句话定义：online monitoring 是大模型产品上线后持续跟踪质量、成本、延迟、安全、反馈、trace 和业务指标的监控体系。

面试表达：大模型线上监控不能只看 QPS 和平均延迟，还要看任务成功率、采纳率、引用支持、bad case 分类、P95/P99、单位成本、事故率、反馈行动率和回归门禁。

Canary Rollout

一句话定义：canary rollout 是先把新模型、prompt、RAG 策略、工具 schema 或安全策略暴露给小比例用户或低风险流量，再逐步扩大的灰度发布方式。

为什么重要：灰度让团队在真实流量中发现长尾问题，同时把失败影响限制在可回滚范围内。

Regression Pass Rate

一句话定义：regression pass rate 是一次模型、prompt、检索、工具或安全策略变更后，回归评估集中通过样本的比例。

面试表达：回归通过率要按业务、风险、权限、语言、长度和高价值用户切片看；平均通过率好看但高风险切片失败时，不能直接放量。

Trace Coverage

一句话定义：trace coverage 是应该保留可审计执行轨迹的线上请求中，实际记录了版本、检索证据、工具调用、关键指标和决策路径的比例。

为什么重要：没有 trace，很难解释 Agent、RAG 或工具调用为什么失败，也很难把线上 bad case 转成复现评估集。

Rollback Plan

一句话定义：rollback plan 是新版本失败时回到旧模型、旧 prompt、旧检索策略、旧工具 schema 或人工兜底流程的预案。

产品化意义：没有回滚能力的上线只能算冒险试验；成熟产品通常要配 feature flag、旧版本保留、配置回滚、数据回滚、告警和事故沟通流程。

Ops Gate

一句话定义：ops gate 是大模型产品上线或扩大灰度前的运营门禁，覆盖质量、采纳、反馈行动、bad case triage、回归、切片、延迟、成本、安全、trace 和回滚。

面试表达：ops gate 强调上线后的可运营性。一个产品即使 demo 效果不错，如果反馈没人处理、bad case 无法复现、事故不能止血、trace 缺失或无法回滚，也不应直接全量上线。

Project Collaboration Incident

一句话定义：project collaboration incident 是大模型项目因为目标、指标、baseline、版本、责任边界、变更控制、灰度、回滚或复盘缺失而导致实验发散、上线风险、事故不可复现或跨团队扯皮的协作事故。

典型来源：目标只写“提升效果”，没有用户和场景；离线指标没有护栏；baseline 没冻结；prompt 热修无版本；RACI 不清；高风险变更未升级；bad case 没进入回归；复盘 action item 没关闭。

面试表达：协作事故不能只归因成“沟通不好”。更工程化的说法是把它拆成 objective clarity、metric tree、baseline、experiment record、version trace、RACI、change control、rollout、rollback、postmortem 和 action item closure 等可审计字段。

Objective Clarity Rate

一句话定义：objective clarity rate 衡量项目或实验中目标是否明确写出用户、场景、任务、主指标、护栏指标和失败边界的比例。

工程意义：目标不清会直接导致实验发散。大模型项目尤其要把“模型更聪明”改写成解决率、引用准确率、P95 延迟、权限泄露率、成本等可检查目标。

Baseline Coverage

一句话定义：baseline coverage 衡量项目、实验或上线候选中有多少样本或变更能和冻结 baseline 做公平对比。

常见误区：只展示新版本好样例，或者同时改模型、prompt、检索、数据和评估脚本后声称单点有效，都不能算有可信 baseline。

Version Trace Coverage

一句话定义：version trace coverage 衡量线上请求、实验记录和事故样本中是否记录了模型、prompt、数据、检索索引、工具 schema、安全策略、代码和配置版本。

工程意义：大模型系统的版本不仅是模型 checkpoint。缺 prompt hash、索引版本或工具 schema 版本时，线上事故很可能无法复现。

RACI Coverage

一句话定义：RACI coverage 衡量项目关键环节是否明确 Responsible、Accountable、Consulted 和 Informed，或者至少有等价的 DRI 责任人机制。

面试表达：责任边界不是为了甩锅，而是为了缩短决策链路。线上幻觉、权限泄露、延迟超标和数据质量问题都应该能找到直接执行人和最终负责人。

Change Control Coverage

一句话定义：change control coverage 衡量模型、prompt、数据、检索、工具、安全策略和配置变更是否经过记录、评审、回归、灰度和可回滚流程。

工程意义：prompt、RAG 配置和安全策略都是生产逻辑。热修不记录版本，等价于把不可审计代码直接推到生产。

Project Collaboration Gate

一句话定义：project collaboration gate 是把目标清晰度、指标树、baseline、实验复现、版本 trace、RACI、变更控制、风险升级、灰度、回滚、bad case 回归、复盘完备率和 action item 关闭率放在一起的项目协作门禁。

面试表达：项目管理不是开会汇报进度，而是把模糊协作问题变成可检查的工程门禁。如果 project collaboration gate 不通过，说明项目即使短期指标好看，也还没有达到可上线、可复盘、可持续迭代的状态。

Product Interview Readiness

一句话定义：product interview readiness 是算法工程师面对技术产品类问题时，能否用用户价值、技术方案、指标公式、demo 证据、风险边界和 trade-off 组成完整回答的准备度。

为什么重要：技术产品面试不只考“懂不懂模型”，还考能否证明模型方案值得做、能上线、能治理、能持续迭代。

Product Interview Gate

一句话定义：product interview gate 是技术产品面试复盘门禁，检查回答是否覆盖核心主题、公式、demo、风险、trade-off 和弱题修复计划。

面试表达：如果回答缺 ROI 公式、RAG / Agent / 隐私 / 运营 demo 证据，或只讲概念不讲门禁，就说明还没准备好被深挖。

Practitioner Interview Readiness

一句话定义：practitioner interview readiness 是实战型大模型工程师能否把项目讲成目标、baseline、指标、实验、失败、debug、trade-off、上线、安全、成本和复盘闭环的准备度。

工程意义：真实面试的深挖通常不止问“你用了什么技术”，还会追问证据、失败样例、边界和取舍。准备度不足时，回答会从工程事实滑向泛泛表达。

Project Evidence Score

一句话定义：project evidence score 衡量项目回答是否有目标、baseline、指标、bad case、debug 路径和生产化证据。

面试表达：项目证据分低时，不要先背更复杂的术语，而要补齐评估集来源、分桶指标、失败样例、版本 trace、灰度、监控、回滚、安全和成本证据。

Trade-off Depth

一句话定义：trade-off depth 衡量回答是否同时讲清收益、代价、决策和验证方式。

面试表达：资深表达不是“用了更强方案”，而是能说明为什么没有全量启用更大模型、更长上下文、更高 top-k 或更强 reranker，以及怎样用指标证明这个取舍可上线。

STAR Reflection Score

一句话定义：STAR reflection score 衡量行为面回答是否覆盖 Situation、Task、Action、Result 和 Reflection。

工程意义：大模型项目的 Reflection 应落到评估集、上线门禁、版本治理、权限边界、监报告警或行动项关闭，而不是空泛说“以后沟通更好”。

Expert Follow-up Readiness

一句话定义：expert follow-up readiness 衡量回答面对专家追问时，能否回到 baseline、指标、失败样例、trade-off 和证据边界。

面试表达：专家追问的关键不是把所有问题都答满，而是诚实区分已验证结论、合理假设和下一步验证方法。

Practitioner Interview Gate

一句话定义：practitioner interview gate 是实战项目面试准备门禁，综合检查项目证据、trade-off 深度、STAR 复盘、专家追问防守、英文清晰度和红旗表述。

面试表达：如果门禁不过线，说明项目故事还不能稳定承受深挖。修复动作应该绑定具体弱题，例如补 baseline、补失败样例、补安全边界、补 action item 验证或重写“为什么不用更大模型”的回答。

本章小结

本章从语言模型的概率定义出发，串起了大语言模型的基础概念。面试中要抓住一条主线：LLM 通过 next-token prediction 预训练得到 base model，再通过指令微调和对话对齐变成 instruct model 或 chat model，最终通过 prompt、上下文、解码策略和外部工具服务具体任务。

高频结论如下：

1. 语言模型本质上是在建模 token 序列概率。
2. next-token prediction 是大规模自监督预训练的核心目标。
3. 自回归模型训练时可并行算 loss，推理时必须逐 token 生成。
4. base model 擅长续写，instruct model 擅长遵循指令，chat model 擅长多轮对话。
5. perplexity 能衡量语言建模拟合程度，但不能替代任务评测。
6. in-context learning 不更新参数，只利用当前上下文临时适配任务。
7. 涌现能力要从规模、数据、优化和评测指标共同理解。
8. 幻觉是生成式语言模型的结构性问题，需要检索、工具、校验和对齐共同缓解。

下一章，我们进入预训练与数据。

F. 预训练与数据

条目: Pretraining, Base Model, Corpus, Web Data, Specialized Data, Code Data, Unit Test Data, Functional Correctness, Math Data, Verifier Data, Multilingual Data, Domain Data, Domain Authority, Expert Audit, Data Quality, Quality Score, Quality Classifier, Data Cleaning, Data Filtering Audit, Deduplication, Exact Deduplication, Near Deduplication, MinHash, SimHash, Train-Eval Overlap, Canary, PII Filtering, Secret Scanning, Toxicity Filtering, Data Mixture, Data Sampling, Temperature Sampling, Effective Epoch, Mixture Ablation, Curriculum Learning, Synthetic Data, Synthetic Instruction Data, Self-Instruct, Evol-Instruct, Distillation Data, Teacher Model, Student Model, Model Collapse, Preference Data, Chosen Response, Rejected Response, Reward Model Data, Safety Data, Over-Refusal, Unsafe Leak, Red Team Data, Multimodal Data, Image-Text Pair, Image-Text Alignment, OCR Data, Caption Data, Audio-Transcript Pair, ASR Data, Video-Text Data, Temporal Alignment, Grounded Multimodal QA, Visual Privacy, Data Attribution, Data Valuation, Influence Function, Data Shapley, Gradient Similarity, Source-Level Ablation, Proxy Model, Negative Value Data, Active Learning, Data Selection, Dataset Versioning, Data Governance, Data Lineage, Dataset Manifest, Immutable Snapshot, Schema Evolution, Deletion Request, Dataset Datasheet, Dataset Card, Model Card, Benchmark Contamination, Data Packing, Document Boundary, Sequence Length, Checkpoint, Training Token, Epoch, Scaling Law, Chinchilla Scaling, Compute-Optimal Training.

第二轮补充条目: Data Incident, Data Incident Gate, Data Risk Rate, Mixture Shift, Lineage Coverage.

Pretraining

一句话定义: pretraining 是在大规模通用语料上训练基础模型的阶段, 目标是让模型获得通用语言、知识、代码、推理和模式识别能力。

在 GPT 类模型中: 预训练通常使用 causal language modeling, 也就是 next-token prediction。模型输入一段 token 序列, 在 causal mask 约束下预测每个位置的下一个 token。

为什么需要: 下游任务的人工标注数据有限, 但互联网上存在大量自然文本、代码、论文、百科、书籍和论坛内容。预训练利用这些未标注文本构造自监督信号, 是大模型通用能力的来源。

训练产物: 预训练结束后通常得到 base model。它能较好地续写文本, 但未必能稳定遵循用户指令, 也未必具备聊天助手的安全边界。

关键决策: 模型规模、tokenizer、训练 token 数、上下文长度、数据混合比例、学习率计划、batch size、优化器、并行策略、checkpoint 策略和评估集设计。

常见风险: 数据质量差会引入噪声和偏见; 数据重复会导致记忆和训练浪费; benchmark contamination 会让评估虚高; 训练不稳定会造成 loss spike、梯度异常或 checkpoint 不可用。

面试表达: 预训练是 LLM 能力底座的形成阶段, 本质是用海量 token 做自监督 next-token prediction。它决定模型能力上限, 后训练更多是在交互方式和偏好上塑形。

Base Model

一句话定义: base model 是经过预训练但未经过指令微调和对齐的基础模型。

典型能力: 语言续写、知识补全、代码补全、模式模仿、简单问答和上下文学习。它学到的是文本分布, 不是产品级助手行为。

典型不足: 不会稳定遵循自然语言指令, 可能继续补全用户输入而不是回答问题; 安全拒答能力弱; 多轮对话格式不稳定; 对系统提示和角色边界没有强约束。

为什么保留 base model: 它适合继续预训练、领域适配、能力分析和作为后训练底座。很多研究会优先比较 base model 的原始能力, 因为它受到指令微调数据风格影响较小。

面试表达: base model 更像“文本续写器”, instruct/chat model 更像“对话助手”。两者差别来自后训练、聊天模板、偏好对齐和安全策略。

Corpus

一句话定义：corpus 是用于训练或评估的一组文本数据集合。

常见来源：网页、书籍、论文、百科、新闻、代码仓库、问答社区、论坛、多语言语料、专业领域文档、合成数据。

核心要求：规模大、覆盖广、质量高、重复少、污染可控、版权和隐私风险可控。语料不是越多越好，低质量 token 可能浪费计算并拉低模型能力。

训练语料和评估语料：训练语料用于更新参数，评估语料用于监控泛化表现。评估集必须和训练集隔离，否则 loss 和 benchmark 分数会被污染。

面试表达：corpus 决定模型 “从什么世界学习”。数据来源、清洗规则和混合比例会直接影响模型的语言、知识、代码、数学和安全能力。

Web Data

一句话定义：web data 是从网页抓取的大规模文本数据，是很多 LLM 预训练语料的重要组成部分。

优点：规模巨大，主题覆盖广，语言和写作风格多样，成本相对较低。

问题：网页数据噪声很高，包含营销模板、模板页、乱码、重复内容、SEO 垃圾、低质量自动生成文本、隐私信息、仇恨内容和版权风险。

处理方法：URL 过滤、语言识别、正文抽取、质量分类器、规则清洗、去重、黑名单、PII 过滤、毒性过滤和域名级采样控制。

面试表达：web data 提供覆盖度，但也带来噪声和风险。高质量预训练不是简单 “爬得越多越好”，而是要把有效信号密度做高。

Web-Scale Data Collection

一句话定义：web-scale data collection 是围绕大规模网页、代码、论文、论坛和多语言来源建立合法、可追溯、可治理训练语料的采集工程。

核心流程：source planning、license / ToS / robots 审查、公开或授权采集、raw storage、HTML/PDF/代码解析、正文抽取、语言和领域识别、质量过滤、PII / secret 扫描、benchmark contamination 检测、去重、数据配比、版本管理和审计。

关键元数据：source id、source type、license、URL hash、content hash、crawl time、language、domain、quality score、risk flags、dedup cluster id、dataset version 和 delete policy。

常见误区：技术上能访问不代表能用于训练；web-scale 不是越多越好；没有 provenance 的数据很难处理删除请求、污染排查和模型版本追责。

面试表达：我会把 web-scale 数据采集讲成数据治理系统，而不是爬虫脚本。合格 pipeline 要能证明数据从哪里来、为什么允许用、经过哪些过滤、进入哪个版本，以及未来如何审计和删除。

Specialized Data

一句话定义：specialized data 是为了提升特定能力而单独建设、审计和采样的数据池，例如代码、数学、医学、法律、金融、科研和企业知识数据。

为什么要单独处理：专项数据的质量标准不同于普通网页。代码要看许可证、密钥、语法、测试和功能正确性；数学要看答案、过程、难度和题库污染；领域数据要看权威性、时效性、引用、隐私和专家审计。

工程做法：先给每类数据建立独立元数据和风险标记，再使用类型专属质量分、污染检测、去重、抽样审计和 data mixture 配比。不能把专项数据简单混入通用语料。

面试表达：专项数据的核心不是 “多加一点专业文本”，而是用类型专属门禁提高目标能力，同时避免 license、secret、PII、benchmark contamination 和高风险错误建议进入训练。

Code Data

一句话定义：code data 是来自代码仓库、文档、issue、notebook 和编程问答的数据，用于增强模型的代码理解和生成能力。

为什么重要：代码具有严格语法、长距离依赖、符号引用和可执行语义。训练代码数据不仅提升编程能力，也常常改善结构化推理和工具调用能力。

常见来源：开源代码仓库、README、API 文档、单元测试、配置文件、脚本、notebook 和技术问答。

数据治理：需要去除大文件、二进制文件、自动生成文件、重复仓库、低质量代码、敏感密钥和许可证风险内容。

工程坑：如果训练数据中包含 API key、密码或私有路径，模型可能记忆并泄漏。代码数据清洗必须包含 secret scanning 和许可证策略。

面试表达：代码数据让模型学习形式化结构和可执行模式，但必须重视去重、secret 过滤和 license 风险。

Unit Test Data

一句话定义：unit test data 是和代码任务配套的测试用例、断言、输入输出样例或测试函数，用来约束代码生成的功能正确性。

为什么重要：代码评估不能只看文本相似度。单元测试能把自然语言需求转成可执行约束，帮助判断生成代码是否真的满足接口、边界条件和异常场景。

数据形态：常见组合包括题目描述、函数签名、参考实现、公开测试、隐藏测试、失败用例、文档字符串和单元测试文件。训练时要避免把评测集隐藏测试泄漏进训练数据。

面试表达：高质量代码数据应该尽量保留“需求-实现-测试-文档”的对齐关系；测试越贴近真实接口和边界条件，越能支撑 functional correctness。

Functional Correctness

一句话定义：functional correctness 是代码是否按照题目或接口规范完成预期功能，而不是生成文本是否像参考答案。

核心判断：最常见方法是执行测试用例，看输出、异常、状态变化和副作用是否符合预期。通过已有测试并不等于完全正确，因为测试覆盖可能不足。

工程风险：如果训练数据包含 benchmark 题目、参考答案或隐藏测试，功能正确性评估会被污染。代码数据治理必须同时做测试质量审计和 train-eval overlap 检测。

面试表达：代码模型的关键指标不是“答案长得像不像”，而是能否在约束下运行正确。单测、静态分析、执行沙箱和污染隔离共同构成功能正确性的评估闭环。

Math Data

一句话定义：math data 是包含数学题目、证明、推导、答案、解析和验证信号的数据，用于提升模型的计算、符号推理和多步推理能力。

为什么重要：数学数据能训练模型分解问题、保持变量一致性、执行中间计算和检查结论。它对推理能力有帮助，但也容易受到错误解析和题库污染影响。

治理重点：需要同时验证最终答案和推理过程，标记题目难度、知识点、来源、重复题、近重复解析和 benchmark contamination。只保留“看起来像推理”的长答案并不可靠。

面试表达：数学数据质量取决于可验证性。高质量数学样本要能说明题目、步骤、答案和难度为什么可信，并明确隔离评测集原题、答案和解析。

Verifier Data

一句话定义：verifier data 是用于训练或校准验证器的数据，通常包含问题、候选解、正确性标签、评分或偏好比较。

作用：在数学、代码和复杂推理任务中，模型可以生成多个候选答案，再由 verifier 评分或重排。这样能把“生成”与“判断正确性”部分解耦。

数据要求：候选解要覆盖正确、部分正确、错误步骤、错误答案和边界情况；标签要可靠，最好经过程序验证、人工复核或多来源一致性检查。

面试表达：verifier data 的价值在于让系统学会识别可靠解法，而不是只模仿单一路径。它常和多候选采样、reranking、过程监督和回归测试一起使用。

Multilingual Data

一句话定义：multilingual data 是包含多种自然语言的训练语料，用于提升模型的多语言理解和生成能力。

核心问题：高资源语言数据多，低资源语言数据少。如果简单按原始规模采样，模型会偏向英语、中文等高资源语言，低资源语言能力较弱。

常见策略：温度采样、语言配额、低资源语言上采样、高质量平行语料、跨语言评估和语言识别清洗。

风险：低资源语言上采样过度会导致重复和过拟合；自动翻译数据可能引入翻译腔；语言识别错误会污染语料。

面试表达：多语言能力不是自然出现的附属品，它依赖语料覆盖、采样策略、tokenizer 设计和评估集建设。

Domain Data

一句话定义：domain data 是来自特定领域的语料，例如医疗、法律、金融、教育、科研、工业和企业内部文档。

作用：增强模型在专业术语、领域知识、文档格式和任务流程上的能力。

使用方式：可以在预训练阶段加入，也可以对 base model 做 continued pretraining，或在后训练阶段加入领域指令数据。

风险：领域数据可能质量参差不齐、知识过时、包含隐私信息或与通用能力冲突。领域继续预训练还可能导致通用能力退化。

面试表达：领域数据能补专业能力，但要控制质量、比例和评估。不能只看领域 loss 下降，还要看通用能力是否被破坏。

Domain Authority

一句话定义：domain authority 是专业领域数据来源的可信度和适用性，例如官方指南、标准文档、同行评审论文、监管文件、教材或经专家维护的知识库。

为什么重要：医疗、法律、金融和工程安全等领域中，低权威来源可能包含过时、片面或错误建议。模型训练如果放大这些内容，会增加高风险输出概率。

审计方法：记录来源机构、作者、发布日期、更新日期、引用、适用地区、版本和撤回状态；不同领域还需要专家抽检和权限控制。

面试表达：领域数据不是按文本流畅度筛选，而是按来源权威性、时效性、引用完整性和风险边界筛选。权威性不足的数据更适合隔离、降权或只用于分析。

Expert Audit

一句话定义：expert audit 是由具备领域资质或经验的人对专业数据、标注、模型输出或评估样本做抽检和风险复核。

适用场景：医学建议、法律条款、金融报告、工业安全、教育测评和高价值企业知识。专家审计通常用于发现普通质量分类器难以识别的事实错误、过期信息和风险表述。

工程落地：审计要有抽样策略、rubric、错误分类、严重度、复核记录和闭环修复动作。专家意见不应只停留在一次性验收，而要进入数据版本和评估报告。

面试表达：高风险领域不能只靠继续预训练或通用清洗规则。专家审计是领域数据治理的最后一道人工质量门禁，用来校准模型能学什么、不能学什么。

Data Quality

一句话定义：data quality 是训练数据的有效信号密度、准确性、多样性、可读性、安全性和任务价值。

为什么重要：预训练计算极其昂贵，同样训练一万亿 token，高质量 token 能提供更多有效学习信号，低质量 token 可能只是在消耗 GPU 时间。

评价维度：文本是否自然、信息是否密集、是否重复、是否乱码、是否营销模板、是否包含有害内容、是否事实可靠、是否覆盖目标能力。

常见方法：规则过滤、困惑度过滤、质量分类器、启发式分数、人工抽检、去重统计、下游 ablation 和小模型预实验。

面试表达：数据质量决定单位 compute 的收益。大模型训练里经常不是缺 token，而是缺高质量、低污染、覆盖目标能力的 token。

Quality Score

一句话定义：quality score 是把文本可读性、信息密度、重复度、风险命中、领域价值和来源可信度融合成的样本级或文档级质量分。

常见特征：长度、unique token ratio、字符比例、重复行比例、boilerplate / SEO 命中、困惑度、质量分类器分数、PII / secret / 安全风险和来源权重。

工程注意：质量分不能脱离语言、领域和来源桶。代码、数学、法律、医学和低资源语言可能天然不同于普通网页文本，应该做分桶阈值校准。

面试表达：quality score 不是“越高越好”的单一裁判，而是清洗系统里的一个排序和审计信号；最终还要叠加隐私、安全、污染、去重和覆盖率门禁。

Quality Classifier

一句话定义：quality classifier 是用标注样本、启发式标签或高质量参考集训练出来的文本质量分类器，用来辅助判断样本是否适合进入某类训练数据池。

为什么需要：纯规则容易漏掉语义层面的低质量内容，也容易误伤长尾领域；质量分类器可以学习更复杂的文本模式，但仍需要抽样审计和阈值校准。

风险：分类器会继承标注偏好和训练集偏差，可能偏向英文、百科体或主流领域，导致论坛、口语、少数语言和专业文本被系统性低估。

面试表达：质量分类器适合作为多级过滤的一环，不适合单独决定训练集。成熟做法是按桶看分布、抽样估计误删 / 漏删，并用小模型 ablation 验证收益。

Data Cleaning

一句话定义：data cleaning 是去除或修复低质量、重复、有害、乱码、格式异常或不适合训练的数据的过程。

常见步骤：HTML 清理、正文抽取、编码修复、语言识别、长度过滤、乱码检测、低信息文本过滤、营销模板过滤、URL/邮箱/手机号处理、非法字符清理。

为什么必要：原始数据中大量内容不适合作为训练信号。如果直接训练，模型会学习垃圾格式、错误知识、重复模板和有害表达。

难点：过滤太松会保留噪声，过滤太严会损失多样性和长尾知识。数据清洗通常要在质量和覆盖之间平衡。

面试表达：data cleaning 不是简单删脏数据，而是提高训练信号密度，同时尽量保留语言和知识多样性。

Data Filtering Audit

一句话定义：data filtering audit 是对过滤前后数据的保留率、风险命中、质量分分布、语言 / 领域覆盖、误删和漏删进行审计。

核心指标：token 保留率、分桶保留率、重复率、PII / secret 命中率、安全风险命中率、benchmark contamination 命中率、人工审计误删率和漏删率。

为什么重要：过滤规则看起来合理，不代表对最终模型有益。过严会损失长尾知识和多样性，过松会把噪声、隐私、安全风险和评测污染带进训练。

面试表达：清洗过滤必须有审计闭环。真正可上线的数据 pipeline 要能回答“删了什么、为什么删、误删多少、漏掉多少、对各语言和领域有什么影响”。

Data Incident

一句话定义：data incident 是因为数据来源、过滤、去重、评估污染、配比、格式、版本或治理证据缺失导致模型能力、评估可信度、上线风险或实验复现出现异常的事故。

典型表现：benchmark 分数虚高但线上退化、训练 loss 异常顺滑但泛化差、某语言或领域突然下降、合成数据风格过重、代码生成出现敏感片段、旧实验无法复现。

排查顺序：先抽样看原始样本，再查 exact / near dedup、train-eval overlap、质量过滤、风险命中、data mixture、lineage 和数据版本 diff，最后再讨论模型结构或训练超参。

面试表达：数据事故不是“某条样本脏了”，而是数据证据链断了。资深工程师要能把现象拆成可观测指标和可复现版本差异。

Data Risk Rate

一句话定义：data risk rate 是训练数据中命中 license、secret、PII、安全、版权或评估污染等高风险标记的比例。

常见口径：可以按样本数、token 数、数据源、语言 / 领域桶分别统计。代码数据通常要单独看 license 和 secret，用户日志要单独看 PII 和权限，评估相关数据要单独看 benchmark contamination。

工程用途：风险率不是越低越好就结束，还要看漏检和误删。风险命中样本需要隔离、复核、修复规则和进入版本报告。

面试表达：data risk rate 是上线门禁，不是训练 loss 指标。只要高风险命中不可解释，就不能把这批数据直接进入正式训练。

Data Incident Gate

一句话定义：data incident gate 是把重复、近重复、评估污染、风险命中、配比偏移、lineage 覆盖和版本可复现性合并成的数据事故门禁。

常见检查项：exact duplicate rate、near duplicate rate、benchmark contamination rate、secret / PII / license risk rate、synthetic ratio、mixture shift、lineage coverage、manifest 完整度和抽样复核结论。

为什么重要：单项指标通过不代表数据可用。例如风险率很低但 benchmark 泄漏存在，评估仍不可信；保留率正常但 code 桶全部被过滤，代码能力仍可能回退。

面试表达：数据门禁要能输出 failed gates 和 rejected reasons。好的回答不是“我们做了清洗”，而是能说明哪些门禁失败、影响什么能力、怎么回滚和复验。

Deduplication

一句话定义：deduplication 是移除重复或近重复数据的过程。

为什么重要：重复数据会浪费训练计算，增加模型记忆风险，放大某些观点或模板，还可能导致评估集泄漏。

粒度：可以做文档级去重、段落级去重、句子级去重、代码文件级去重，也可以用 MinHash、SimHash、embedding 相似度等方法做近重复检测。

精确去重与近似去重：精确去重删除完全相同内容，简单但覆盖有限；近似去重能处理模板页、转载、轻微改写，但误删风险更高。

工程注意：去重通常要区分训练集内部去重和训练集对评估集去重。后者对 benchmark 可信度尤其重要。

面试表达：去重的价值不只是省 token，更是降低记忆和数据污染风险。大模型数据治理中，near-dedup 往往比 exact-dedup 更关键。

Exact Deduplication

一句话定义：exact deduplication 是对规范化后的文档、段落、代码文件或样本计算 hash，删除完全相同内容的过程。

关键点：必须先做规范化，例如统一空白、大小写、Unicode、HTML 残留和模板字段；否则本质相同的文本会因为格式差异逃过去。

优点和边界：exact dedup 便宜、可解释、适合大规模第一层过滤，但不能发现轻微改写、转载加页脚、代码 fork 和语义等价题解。

面试表达：exact dedup 是必要的底层门禁，但不是大模型去重的全部。真正影响训练质量的常常是 near duplicate 和 train-eval overlap。

Near Deduplication

一句话定义：near deduplication 是识别高度相似但不完全相同的样本，例如转载、改标题、镜像、代码 fork、模板页和轻微改写。

常见方法：n-gram Jaccard、MinHash/LSH、SimHash 汉明距离、编辑距离、embedding 相似度和人工审计。

风险：阈值过高会漏掉重复，阈值过低会误删同主题但细节不同的高价值样本。短文本、代码、数学题和多语言数据尤其需要分桶校准。

面试表达：near dedup 的核心不是“把相似内容全删掉”，而是在减少无意义重复、控制记忆风险和保留多样性之间做平衡。

MinHash

一句话定义：MinHash 是用紧凑签名估计两个集合 Jaccard 相似度的方法，常用于文本 shingle 集合的近重复检测。

工作方式：把文档切成 n-gram shingles，生成多个 hash 下的最小值作为 signature；两个文档 signature 相同位置比例越高，Jaccard 相似度越高。

工程用途：MinHash 通常和 LSH 配合，用于 web-scale 近重复候选召回，避免对海量文档两两比较。

面试表达：MinHash 解决的是“相似度计算太贵”的问题，不是最终保留策略。最终仍要做相似度阈值、聚类、代表样本选择和抽样审计。

SimHash

一句话定义：SimHash 是把文本特征压缩成固定长度二进制指纹，并用汉明距离判断近重复的方法。

适用场景：网页近重复、镜像页、模板页和快速候选过滤。它存储小、速度快，但特征选择和汉明阈值会影响误删 / 漏检。

和 MinHash 的区别：MinHash 更直接对应 Jaccard 集合相似度；SimHash 更像加权特征投票后的指纹压缩。两者都需要按数据类型审计阈值。

面试表达：SimHash 适合快速网页去重，但不能替代污染检测、语义相似审计或人工抽检。

Train-Eval Overlap

一句话定义：train-eval overlap 是训练语料和评估样本之间存在原文、答案、解析、代码签名、测试样例或近重复内容。

为什么重要：训练集内部可以去重得很好，但如果包含 benchmark 题目或答案，评估仍然会虚高。

常见检测：exact match、n-gram overlap、MinHash/LSH、embedding 检索、答案泄漏检测、时间切分和人工审计。

面试表达：训练去重和评测污染检测是两件事。前者控制训练分布和记忆风险，后者保护评估可信度。

Canary

一句话定义：canary 是受控合成探针，用于评估模型是否容易记住并复述罕见字符串或特定训练片段。

使用边界：canary 应使用非真实敏感信息，不能用真实用户隐私、真实凭据或真实业务秘密当探针。

工程用途：通过控制 canary 出现次数、上下文和数据版本，观察去重、隐私过滤和训练策略是否降低记忆风险。

面试表达：canary 的价值是做受控风险评估，而不是复原训练数据。它应和 PII / secret 过滤、去重和输出安全评估一起使用。

PII Filtering

一句话定义：PII filtering 是识别并过滤个人可识别信息的过程，例如身份证号、手机号、邮箱、地址、姓名组合、账号和密钥。

为什么需要：训练数据中如果含有隐私信息，模型可能在生成时泄漏，带来安全、合规和伦理风险。

常见方法：正则规则、NER 模型、敏感字段词典、secret scanner、上下文规则和人工抽检。

挑战：PII 形式多样，规则可能漏检；过度过滤又会误删正常文本，例如技术文档中的示例邮箱或公开联系信息。

面试表达：PII filtering 是大模型数据安全的底线之一，尤其在企业内部数据、代码数据和用户日志数据中必须严格处理。

Secret Scanning

一句话定义：secret scanning 是识别 API key、access token、私钥、密码、连接串和内部路径等敏感凭据的过程。

为什么需要：代码仓库、配置文件、日志和论坛样本可能意外包含真实凭据。进入训练集后，模型可能记忆或在生成中泄漏这些字符串。

常见方法：高召回正则、已知 token 前缀规则、熵检测、上下文规则、官方 secret scanner、人工抽检和命中样本隔离。

面试表达：secret scanning 要优先高召回，并把命中样本从通用训练池隔离；示例 key 和真实 key 要靠上下文、格式和来源进一步区分，不能只靠关键词。

Toxicity Filtering

一句话定义：toxicity filtering 是过滤仇恨、暴力、色情、骚扰、极端主义和其他有害内容的过程。

目标：降低模型学习有害表达和危险行为的概率，提高后续对齐和安全训练的效率。

复杂性：不是所有敏感内容都应删除。安全研究、新闻报道、法律文本和医学文本可能包含敏感词，但上下文是正当的。

常见策略：分级过滤、领域白名单、上下文分类器、人工审核和安全评估集。对于安全相关任务，有时需要保留经过标注和规范化的安全数据。

面试表达：毒性过滤要避免简单关键词删除。关键是区分有害生成样本和正当讨论样本，否则会损害模型对安全问题的理解。

Data Mixture

一句话定义：data mixture 是不同来源、语言、领域和任务类型数据在训练集中的混合比例。

影响：数据混合比例决定模型能力分布。代码比例高通常提升代码能力；数学和高质量推理文本能提升推理表现；多语言比例影响跨语言能力；网页噪声比例过高会影响输出质量。

设计方式：根据目标能力设定初始比例，通过小规模训练实验、验证集 loss、下游 benchmark 和人工评估迭代调整。

常见矛盾：专业数据多会增强领域能力，但可能降低通用聊天能力；代码数据多会提升编程，但可能改变自然语言风格；低资源语言上采样能提升覆盖，但可能引入重复。

工程字段：成熟数据表通常会记录 source、language、domain、quality score、risk label、dedup cluster、license、token count、sampling weight、effective epoch 和 dataset version。

面试表达：data mixture 是训练“配方”的核心。模型最终擅长什么，很大程度由训练 token 的来源和比例决定。

Mixture Shift

一句话定义：mixture shift 是清洗、去重、过滤、采样或数据版本变更后，实际训练数据配比相对目标配比发生的偏移。

常见原因：某个数据桶被风险门禁大量过滤，近重复去重删除了热门来源，质量分类器误伤低资源语言，代码 license 审查清掉大量仓库，合成数据在过滤后占比被动升高。

工程用途：训练前要同时报告目标 mixture 和清洗后的实际 mixture。训练后如果某能力退化，优先查对应语言、领域、任务或模态的数据桶是否发生了意外偏移。

面试表达：mixture 配置不是写在 YAML 里就结束。真正影响模型的是进入 batch 的实际 token 分布，所以必须审计 mixture shift。

Data Sampling

一句话定义：data sampling 是从不同数据源中抽取训练样本的策略。

常见方式：按原始数据量采样、按固定配额采样、温度采样、动态采样、按质量分数加权采样。

温度采样直觉：降低高资源数据的优势，提高低资源或高价值数据被采到的概率。温度越高，分布通常越平滑。

为什么重要：即使总语料固定，采样策略不同，模型看到的数据分布也不同，最终能力会发生显著变化。

面试表达：sampling strategy 是 data mixture 真正落地到训练 batch 的方式。讨论配比时要同时说明采样权重、计划 token 数、effective epoch 和风险门禁。

Temperature Sampling

一句话定义：temperature sampling 是对原始数据池大小做平滑，让高资源数据不过度主导、低资源或高价值数据获得更多采样机会。

常见写法：可以把数据池大小 n_k 转成 n_k^{α} 后归一化，其中 $\alpha=1$ 接近自然分布， $\alpha=0$ 接近均匀分布。也有资料用 temperature 的倒数来表示同一思想。

风险：过度平滑会让小数据池被反复采样，带来过拟合、记忆和污染风险；平滑不足又会让低资源语言或高价值领域被淹没。

面试表达：temperature sampling 的目标是折中覆盖和重复 exposure，不是让所有语言或领域完全等比例。

Effective Epoch

一句话定义：effective epoch 是某个数据池计划采样 token 数除以该池清洗后 token 数，用来衡量它被重复使用的程度。

为什么重要：上采样小而高质量的数据池可以补能力短板，但 effective epoch 过高会增加记忆、题库污染和风格同质化风险。

工程做法：对代码、数学、专业文档、合成数据和低资源语言单独监控 effective epoch，并结合去重、污染检测、canary 和小模型 ablation 调阈值。

面试表达：上采样不能只报比例，还要报 effective epoch。否则看似只是提高数学比例，实际可能是在重复训练同一批题。

Mixture Ablation

一句话定义：mixture ablation 是固定数据版本和训练 recipe，系统改变某类数据采样比例，观察能力、风险和成本变化的实验。

常见实验：提高代码比例、提高数学比例、提高多语言比例、加入合成推理数据、降低低质量网页比例、调整安全数据比例。

评估维度：通用问答、代码、数学、多语言、安全、事实性、长上下文、记忆风险、污染风险、人工样例和验证 loss。

面试表达：data mixture 没有标准答案，必须用 ablation 找 trade-off，并在更大模型和更长训练上复验小模型趋势。

Curriculum Learning

一句话定义：curriculum learning 是按一定顺序组织训练数据，让模型从较容易或更基础的数据逐步过渡到更复杂数据的训练思想。

在 LLM 中的形式：先短序列后长序列，先高质量通用数据后专业数据，先自然语言后代码或数学，或在训练后期提高高质量数据比例。

潜在好处：提升训练稳定性，让模型先学基础语言模式，再学习复杂结构；也可能提高后期能力收敛效率。

局限：LLM 规模下 curriculum 的收益并不总是稳定，错误的数据顺序可能造成分布偏移或过拟合。实际需要实验验证。

面试表达：curriculum learning 的核心是训练顺序设计，但在大模型中它是经验性工程策略，不是一定优于随机混合。

Synthetic Data

一句话定义：synthetic data 是由模型、程序、规则或模拟器生成的数据。

用途：补充稀缺任务、生成指令数据、构造数学题和代码题、扩充多语言数据、生成安全拒答样本、构造工具调用轨迹、生成内部回归测试样本。

优点：成本低、可扩展、可定向补能力，适合覆盖人工数据不足的长尾场景。

风险：teacher 错误会被 student 模仿；数据风格可能单一；过多合成数据可能降低自然性；如果缺少过滤，会放大幻觉、偏见、评测污染和安全边界错误。

质量控制：使用强模型生成、多模型交叉验证、规则校验、执行器验证、人工抽检、去重、难度分层、授权审查、污染扫描和拒绝低质量样本。

面试表达：合成数据的关键不是“自动生成”，而是“自动生成后如何筛选和验证”。没有质量控制的合成数据很容易污染模型。

Synthetic Instruction Data

一句话定义：synthetic instruction data 是由规则、模板或 LLM 生成的 instruction-input-output 样本，用于指令微调、格式跟随、工具调用和多轮对话训练。

典型流程：先准备种子任务，再生成多样指令和回答，然后过滤无效、重复、相似、污染和低质量样本，最后用小规模训练实验验证收益。

风险：如果任务模板单一，模型会学到固定话术；如果 teacher 输出未验证，student 会继承错误；如果合成题和 benchmark 太接近，评估会虚高。

面试表达：合成指令数据适合补覆盖，但必须记录 seed、teacher、prompt、sampling 参数、过滤规则、质量分和数据版本。

Self-Instruct

一句话定义：Self-Instruct 是用模型自举生成 instruction、input 和 output，再经过过滤后用于 instruction tuning 的方法。

核心启发：它不是让模型无限生成数据，而是强调生成、过滤、去重、任务多样性和评估闭环。生成质量和过滤策略比原始数量更重要。

工程边界：Self-Instruct 风格数据适合教学和开源模型训练思路，但真实项目还要处理授权、隐私、评测污染、teacher 偏差和人工抽检。

面试表达：Self-Instruct 的价值是降低人工指令数据门槛；它的风险是错误自举和模板化，所以必须配合验证和 ablation。

Evol-Instruct

一句话定义：Evol-Instruct 是从较简单指令出发，通过增加约束、深度、复杂输入或多步骤要求，演化出更复杂指令的合成数据方法。

解决问题：普通合成指令容易偏简单，复杂任务覆盖不足。Evol-Instruct 试图主动提高指令复杂度，训练模型处理组合约束和多步骤任务。

风险：复杂不等于高质量。约束冲突、不可回答、答案不满足指令、伪复杂和重复改写都需要过滤。

面试表达：Evol-Instruct 的关键是复杂度控制和质量验证，而不是机械堆限制条件。

Distillation Data

一句话定义：distillation data 是 teacher model 生成或标注的数据，用来训练 student model 学习 teacher 的回答、推理、偏好、评分或格式。

常见形式：teacher 回答、分步解释、代码实现、工具调用轨迹、chosen/rejected 偏好对、评分标签、错误分析和过程监督数据。

工程门禁：必须确认 teacher 输出使用授权，记录 teacher 版本、prompt、sampling 参数、质量验证、污染扫描和安全过滤。无授权复制闭源模型能力不是合规的数据工程方案。

面试表达：蒸馏数据关注 teacher-student 能力迁移；它可以降低标注成本，但会继承 teacher 偏差、错误和风格。

Teacher Model

一句话定义：teacher model 是用于生成答案、解释、偏好或评分的较强模型、规则系统、模拟器或专家标注系统。

作用：teacher 可以补充人工标注不足，生成高质量初稿，也可以给 student 提供评分和反馈信号。

风险：teacher 不等于真值。它可能 hallucinate、偏向某种表达、违反目标安全边界，或输出授权不允许用于训练的数据。

面试表达：teacher 的输出必须被审计，而不是直接当 gold label。真正可信的是 teacher 加验证、过滤、人工抽检和实验收益。

Student Model

一句话定义：student model 是使用 teacher 生成或标注数据训练的目标模型，通常容量更小、成本更低或面向特定场景。

能力边界：student 学到的是训练数据中的行为模式，受模型容量、数据覆盖、优化目标和评估体系约束，不等于完整复制 teacher。

常见目标：降低推理成本、提升特定任务能力、稳定输出格式、增强工具调用或构建私有部署模型。

面试表达：student 可以在窄分布上接近甚至超过 teacher 的局部表现，但全局能力仍要靠独立评估验证。

Model Collapse

一句话定义：model collapse 是模型生成数据被反复用于训练后，数据分布逐渐变窄、长尾信息丢失、错误累积和输出同质化的风险。

形成原因：合成数据如果逐代替代自然数据，生成器偏差会被不断放大，真实世界的复杂性和少数模式会被压缩。

防范：保留高质量自然数据和人工数据，控制 synthetic ratio，做去重和多样性审计，加入真实用户分布评估，并监控长尾能力和事实性回归。

面试表达：合成数据是分布编辑，不是免费扩容。比例、自然数据锚点和验证闭环决定它是增强能力还是加速退化。

Preference Data

一句话定义：preference data 是对同一 prompt 下多个回答的比较数据，用于训练 reward model、DPO 或其他偏好优化方法。

常见结构：prompt、chosen response、rejected response、rubric、标注者、语言、风险类别、标注理由和数据版本。排序列表也可以转成多个成对偏好样本。

质量风险：标注者不一致、rejected 太弱、长度偏置、迎合偏置、安全过度偏置、LLM judge 偏差和隐私数据混入。

面试表达：偏好数据不是“哪个好看选哪个”，而是对 helpful、honest、harmless 等行为准则的稀疏采样。必须审计一致性、margin、长度偏置和训练后回归。

Chosen Response

一句话定义：chosen response 是偏好样本中被标注为更好的回答。

判断依据：通常来自 helpfulness、factuality、safety、format following、tone、citation、professional boundary 和用户意图匹配等维度。

工程注意：chosen 不一定是完美答案，只是在当前候选中更好。训练前要检查它是否安全、事实可靠、未包含隐私和未泄漏评测内容。

面试表达：chosen response 是相对偏好信号，不是绝对 gold label。高质量数据要记录为什么 chosen 胜出。

Rejected Response

一句话定义：rejected response 是偏好样本中较差的回答，用来和 chosen 构成比较信号。

数据价值：好的 rejected 应该接近真实模型错误模式，且和 chosen 有可解释差异。太弱的 rejected 会让模型只学到粗糙偏好。

常见问题：明显胡说、过短、故意不安全、和 chosen 差异太大或只因长度较短被拒。此类样本要降权或剔除。

面试表达：DPO / RM 的数据质量很大程度取决于 rejected 的质量。rejected 要有难度，而不是随便造一个坏答案。

Reward Model Data

一句话定义：reward model data 是用于训练奖励模型的偏好排序数据，通常包含 prompt、多个回答、排序或 chosen/rejected 对。

训练目标：让 reward model 给 chosen 更高分，给 rejected 更低分，从而近似人类偏好。它学的是标注规范下的相对排序，不是事实真值。

审计指标：pairwise accuracy、标注一致性、分桶准确率、长度偏置、reward calibration、OOD 泛化、安全类别表现和 reward hacking 风险。

面试表达：reward model 只是偏好的代理目标。它必须和人工评估、安全评估和事实性评估一起看，不能只追 reward score。

Safety Data

一句话定义：safety data 是用于训练和评估模型安全行为的数据，覆盖风险分类、拒答、边界允许、误拒修复、红队回归和专业高风险场景。

核心目标：让模型区分该帮助、该拒绝、该澄清、该给安全替代建议的场景。安全数据不是简单把风险关键词都映射为拒答。

数据分层：明确安全、敏感但可答、高风险、边界模糊、紧急风险、专业高风险、多语言和隐私场景都要单独标注。

面试表达：好的安全数据同时降低漏拒和误拒。只训练拒答会导致模型保守，缺少边界允许数据会损害 helpfulness。

Over-Refusal

一句话定义：over-refusal 是模型把本应安全回答的请求错误拒绝，也叫误拒。

常见原因：安全训练样本只有拒答，没有边界允许数据；关键词过滤太粗；标注规范过度保守；多语言安全数据不足。

治理方式：构造安全可答样本、教育性 / 防御性 / 高层次解释样本、专业边界样本和误拒回归集，并按语言和 risk 类别评估。

面试表达：安全模型不是拒得越多越好。误拒会降低可用性，甚至阻断正常教育和防御场景。

Unsafe Leak

一句话定义：unsafe leak 是模型在应拒绝或安全改写的高风险请求上给出了不安全回答，也叫漏拒。

常见来源：安全数据覆盖不足、红队失败模式未回流、policy 版本不一致、多轮上下文中边界丢失或模型过度追求 helpfulness。

治理方式：红队回归、风险分层、安全拒答和安全替代建议数据、边界样本专家审计、多语言测试和上线监控。

面试表达：漏拒是安全底线风险，但只压漏拒会带来误拒。必须用成对数据和双指标一起优化。

Red Team Data

一句话定义：red team data 是系统性安全测试中发现的模型失败样本、风险类别、失败模式、安全响应和回归测试数据。

使用边界：红队数据应服务于防御性训练、评估和治理。进入训练时要保留必要标签和安全回答，避免传播可操作风险细节。

工程用途：发现模型弱点、补安全数据、构建回归集、跟踪策略版本变化和评估上线风险。

面试表达：红队数据不是攻击样本库，而是安全治理闭环的一部分。关键是把失败模式转化为可复测、可修复的数据资产。

Multimodal Data

一句话定义：multimodal data 是同时包含图像、文本、OCR、音频、视频、字幕、时间戳或多模态问答的数据。

治理重点：相比纯文本，多模态数据必须同时检查媒体质量、跨模态对齐、时序同步、隐私、版权来源、评测污染和任务配比。图像清晰但 caption 错、音频好但 transcript 错、视频有字幕但时间错位，都可能让模型学到错误关联。

面试表达：多模态数据工程的核心不是“把图片、音频和视频拼进训练集”，而是证明不同模态确实互相解释，并且来源、隐私和评测隔离都可审计。

Image-Text Pair

一句话定义：image-text pair 是图像与 caption、alt text、标题、周围文本或问答文本组成的训练样本。

常见问题：网页文本不一定描述图片，装饰图、营销素材图、模板图和新闻配图都可能造成弱监督噪声。高质量图文对要记录来源、图像 hash、文本来源、语言、质量分、对齐分和许可证状态。

面试表达：图文对不是图片加一句话，而是一个需要证明“文本真的在解释图像”的弱监督样本。

Image-Text Alignment

一句话定义：image-text alignment 是图像内容和文本描述之间的一致性。

常见做法：可以用 CLIP 类 embedding 相似度、OCR 命中、对象 / 属性规则、人工抽样和下游 VQA 评测共同判断。相似度只能辅助过滤，不能替代人工或任务级验证。

面试表达：对齐分低的样本会让 VLM 学错视觉概念；对齐分高也不保证 caption 完整，因此需要分桶抽检和任务回归。

OCR Data

一句话定义：OCR data 是从图片、截图、文档页、票据、表格或网页截图中识别出来的文字及其版面结构。

关键字段：识别文本、bounding box、页码、阅读顺序、置信度、语言、表格结构和脱敏标记。只保留纯文本会丢失版面、列关系和图文位置关系。

面试表达：OCR 数据治理要同时看识别准确率和 layout 保真度，否则文档问答、表格理解和截图理解会受到明显影响。

Caption Data

一句话定义：caption data 是描述图像、音频或视频片段内容的自然语言文本。

质量要求：caption 应具体、可验证、不过度幻觉，并且和媒体片段时间范围一致。过泛的“一个人在做事”或带营销口吻的描述通常训练价值有限。

面试表达：caption 数据的价值在于把感知内容变成语言监督，但 caption 错、空泛或时间错位会直接污染多模态对齐。

Audio-Transcript Pair

一句话定义：audio-transcript pair 是音频片段与对应转录文本组成的训练样本。

治理重点：需要记录采样率、语言、说话人、时间戳、噪声、转录版本、WER / CER、VAD 切片和隐私脱敏状态。音频清晰度和 transcript 准确性要一起审计。

面试表达：语音数据不是只看音频质量，还要看转录是否准确、时间是否对齐、多语言和口音是否覆盖。

ASR Data

一句话定义：ASR data 是用于训练或评估自动语音识别系统的音频、转录、时间戳和质量标签。

常见指标：英文或空格分词语言常看 WER，中文或字符级任务常看 CER；工程上还要看分语种、分口音、分噪声场景和分时长的错误分布。

面试表达：ASR 数据质量决定语音模型的鲁棒性，平均 WER 低不代表长音频、噪声、口音和多说话人场景都可靠。

Video-Text Data

一句话定义：video-text data 是视频片段与字幕、caption、事件描述、动作标签或视频问答组成的数据。

治理重点：视频要处理片段切分、帧采样、镜头变化、音频、字幕、事件时间范围、版权来源和隐私风险。一个全视频标题通常不能当作每个片段的精确标签。

面试表达：视频数据的难点在时间结构，训练样本必须说明文本描述对应哪个片段、哪段声音或哪个事件。

Temporal Alignment

一句话定义：temporal alignment 是音频、字幕、视频帧、事件描述和问答证据在时间轴上的同步关系。

常见风险：字幕延迟、ASR 切片错位、视频 caption 只描述开头或结尾、长视频标题泛化到全部片段。时间错位会让模型把错误画面、声音和文本绑定在一起。

面试表达：多模态数据治理要把时间戳当作核心字段，而不是附属元数据；音频和视频任务尤其需要 forced alignment 或抽样复核。

Grounded Multimodal QA

一句话定义：grounded multimodal QA 是答案必须由给定图像、文档、音频或视频证据支持的多模态问答数据。

质量要求：问题要可由媒体内容回答，答案要引用或对齐到可见文字、物体、动作、声音或时间片段。无法从媒体中验证的答案应标记为 hallucinated 或 unsupported。

面试表达：多模态指令数据不能只看回答是否流畅，还要检查答案是否被视觉、OCR、音频或视频证据支持。

Visual Privacy

一句话定义：visual privacy 是图像和视频数据中的人脸、车牌、证件、住址、病历、儿童影像、屏幕截图和其他可识别个人信息风险。

治理方式：训练数据需要记录隐私风险标签、脱敏状态、删除策略、来源许可和人工复核。高风险媒体不能因为文本侧没有 PII 就直接进入训练。

面试表达：多模态隐私比文本隐私更难，因为敏感信息可能藏在像素、画面背景、OCR 和声音里，必须在媒体解析阶段就做风险门禁。

Data Attribution

一句话定义：data attribution 是追溯模型行为、错误、能力变化或评估结果可能来自哪些训练数据、数据源或数据版本的过程。

常见证据：相似训练样本、数据源统计、版本变化、训练日志、embedding / gradient 近似、消融实验和人工审计。大模型里的 attribution 多数是证据链，不是严格因果证明。

面试表达：data attribution 主要服务错误诊断和数据治理，目标是把“模型为什么这样表现”追溯到可审计的数据来源、版本和处理规则。

Data Valuation

一句话定义：data valuation 是估计数据对目标模型指标、能力、风险或产品价值的贡献，用于决定保留、删除、采购、标注、上采样或降权。

关键点：数据价值不是固定常数，而依赖目标模型、训练阶段、已有数据分布、评估指标、风险约束和成本。一个数据源可能提升代码能力，同时损害安全或风格。

面试表达：数据估值不是只算准确率提升，而是把收益、成本、风险、副作用和可治理性放进同一个决策框架。

Influence Function

一句话定义：influence function 用来估计训练样本权重发生微小变化时，对某个测试样本 loss 或预测的影响。

工程边界：经典公式涉及 Hessian inverse，深度模型和大模型中直接计算成本高且近似不稳定。实际更常用小模型、梯度相似、训练轨迹或检索近邻做局部诊断。

面试表达：Influence function 是训练数据归因的重要思想，但在 LLM 规模下通常不能被当作精确逐条训练数据贡献计算器。

Data Shapley

一句话定义：Data Shapley 用 Shapley value 思想衡量数据在不同子集中的平均边际贡献。

优点和限制：它理论性质好，能考虑数据交互；但精确计算需要大量子集训练或近似训练，通常只适合小规模数据池、源级估值或作为估值思想。

面试表达：Data Shapley 的价值在于定义“公平边际贡献”，但大模型里更现实的是近似 Shapley、源级 ablation 和 proxy 实验。

Gradient Similarity

一句话定义：gradient similarity 是比较训练样本或数据源梯度方向与目标任务梯度方向是否一致的近似归因信号。

使用方式：如果某批数据的梯度和目标验证集方向接近，它可能对目标能力有帮助；如果方向相反，可能带来负贡献。它仍需结合风险、污染和消融验证。

面试表达：梯度相似是弱信号，不是最终决策。高相似数据如果有 benchmark contamination 或隐私风险，仍然必须阻断。

Source-Level Ablation

一句话定义：source-level ablation 是删除、降权、加入或上采样一个数据源后，通过训练实验观察多维指标变化的数据估值方法。

适用场景：用于高价值数据池、可疑数据源、采购决策、配比调整和上线前关键验证。它比逐条估值贵，但解释性强，和真实训练目标更一致。

面试表达：大模型里源级 ablation 常比样本级估值更实用，因为训练数据规模大、单条样本影响小、数据源和数据分布影响更明显。

Proxy Model

一句话定义：proxy model 是用较小模型、短训练或低成本实验近似评估数据价值的替代模型。

风险：proxy 结论不一定外推到大模型，尤其在能力涌现、长上下文、多模态和安全行为上。需要用多尺度实验或最终小比例验证校准。

面试表达：proxy model 是数据估值的成本控制工具，不是最终真相；它适合筛候选和排雷，关键决策仍要用更接近目标训练的验证。

Negative Value Data

一句话定义：negative value data 是对目标效用产生负贡献的数据，例如错误标注、过时知识、低质合成、污染样本、隐私风险和导致误拒的数据。

治理方式：通过质量分、异常 loss、错误归因、源级 ablation、人工审计、安全回归和污染检测定位，再决定重洗、降权、隔离或删除。

面试表达：数据工程不只是找高价值数据，删掉负价值数据有时比增加新数据更有效。

Active Learning

一句话定义：active learning 是在标注预算有限时，优先选择最值得标注或复核的样本。

选择信号：不确定性、模型分歧、用户频率、风险等级、目标能力薄弱、代表性、多样性和标注成本。大模型数据工程中常用于 SFT、偏好、安全、领域和多模态标注。

面试表达：主动学习不是只挑最难样本，而是在价值、覆盖、风险和成本之间做标注优先级排序。

Data Selection

一句话定义：data selection 是在 token、标注、训练和风险预算下选择哪些数据进入训练、上采样、降权或删除。

常见约束：目标能力、质量分、来源可信度、多样性、语言 / 领域配比、隐私版权风险、benchmark contamination、成本和版本可追溯性。

面试表达：数据选择最终是带约束的优化问题。不能只选最像评测集的数据，否则会过拟合 benchmark 并损害泛化。

Dataset Versioning

一句话定义：dataset versioning 是把训练数据快照、manifest、pipeline、schema、过滤配置、采样配置和审计记录绑定成可复现版本的工程机制。

为什么重要：训练结果依赖具体数据版本。数据目录被重写、过滤规则变化、shard 顺序变化或采样权重变化，都会让模型结果不可复现。

面试表达：数据版本不是文件名加日期，而是数据、代码、配置、hash、统计和审计记录的组合。

Data Governance

一句话定义：data governance 是围绕数据来源、权限、隐私、license、删除请求、审计、文档和训练可复现性的治理体系。

工程重点：治理不是训练结束后的合规表格，而是嵌入数据 pipeline 的控制面。它要支持追溯、回滚、隔离、删除、审计和发布说明。

面试表达：大模型数据治理决定团队能否说清模型用了什么数据、为什么允许用、经过哪些处理、进入哪个版本，以及未来如何删除和复核。

Data Lineage

一句话定义：data lineage 是数据从原始来源到最终训练样本、训练版本和模型版本的血缘记录。

典型字段：source id、raw object hash、processing steps、filter versions、dedup cluster、dataset version、training run id 和 deletion status。

面试表达：lineage 让问题样本可溯源，也让删除请求和污染排查能从来源追到所有受影响版本。

Lineage Coverage

一句话定义：lineage coverage 是具备来源、处理路径、版本、hash、风险标记和训练去向等血缘字段的样本或 token 比例。

为什么重要：lineage 覆盖不足时，数据事故很难定位到来源和版本，也很难处理删除请求、污染追溯、回滚和复现实验。

工程用途：按数据源、数据桶和风险等级统计 lineage coverage。高风险、授权、代码、用户日志和评估相关数据应使用更严格阈值。

面试表达：lineage coverage 低说明团队不是缺一个字段，而是缺事故追溯能力。它应该进入数据版本报告和训练上线门禁。

Dataset Manifest

一句话定义：dataset manifest 是描述某个数据版本包含哪些 shard、hash、样本数、token 数、统计分布和风险摘要的清单。

作用：manifest 是训练可复现的入口。训练日志应记录 manifest、data mixture、sampler、tokenizer 和评测版本，而不是只记录一个数据目录。

面试表达：没有 manifest，就很难证明两次训练使用的是同一份数据。

Immutable Snapshot

一句话定义：immutable snapshot 是一旦用于训练就不再原地修改的数据快照。

做法：用内容 hash、对象存储版本、shard checksum、数据库快照和配置版本保证快照稳定。如果发现错误，应创建新版本而不是偷改旧版本。

面试表达：不可变快照的价值是让实验结果、审计记录和删除追溯有稳定参照。

Schema Evolution

一句话定义：schema evolution 是数据元数据字段、标签体系和字段含义随时间演进时的版本管理。

注意事项：新增字段要有默认值，旧数据要能回填；字段含义变化必须版本化，例如 quality score v1 和 v2 不能混当同一个分数。

面试表达：schema 演进如果不治理，会让过滤规则、采样器和评估分析使用不一致字段，导致结论失真。

Deletion Request

一句话定义：deletion request 是用户、数据提供方、版权方或合规团队要求删除或撤回某些数据的请求。

工程难点：要定位 raw data、清洗数据、shard、索引、embedding、旧版本和已训练模型是否受影响，并记录执行证明。它依赖 source id、hash、lineage 和版本记录。

面试表达：删除不是删当前目录，而是一个跨版本、跨缓存、跨派生数据的追踪和证明流程。

Dataset Datasheet

一句话定义：dataset datasheet 是数据集说明书，记录创建动机、组成、采集、处理、用途、限制、风险、维护和删除策略。

价值：datasheet 让数据集透明、可沟通、可复核，减少“训练时没人记得数据怎么来的”的问题。

面试表达：datasheet 是数据治理文档，不是宣传材料；它要能支撑风险讨论和使用边界。

Dataset Card

一句话定义：dataset card 是面向数据集使用者的数据说明卡，通常包含数据来源、字段、任务、划分、许可、偏差、风险和使用限制。

使用场景：开源数据、内部数据平台和合作数据交付都可以用 dataset card 帮助用户理解数据能做什么、不能做什么。

面试表达：dataset card 更偏数据集使用说明，datasheet 更偏完整治理说明；两者都要求数据版本和元数据可靠。

Model Card

一句话定义：model card 是模型说明卡，描述模型用途、训练概况、评估结果、适用范围、限制和风险。

与数据治理关系：model card 中的训练数据、适用语言、偏差、风险和删除策略，都依赖 dataset versioning、lineage、datasheet 和审计结果提供事实基础。

面试表达：没有数据版本和治理，model card 很容易变成泛泛而谈，无法支撑真实风险边界。

Benchmark Contamination

一句话定义：benchmark contamination 是评估集内容泄漏进训练数据，导致评估结果虚高的问题。

典型形式：训练语料中包含 benchmark 原题、答案、解析、翻译版本、近重复版本，或代码 benchmark 的测试用例和标准解。

危害：模型看似能力强，实际可能只是记住了题目或答案，导致能力判断、模型选型和论文结论失真。

检测方法：训练集和评估集做 n-gram 匹配、MinHash 近重复检测、embedding 相似度检索、时间切分评估、私有评估集和动态评测。

应对方式：训练前过滤已知 benchmark, 报告数据 cutoff, 使用新题和私有测试集, 做 contamination analysis。

面试表达：benchmark contamination 是大模型评估可信度的核心风险。越热门的公开 benchmark，越需要怀疑数据泄漏。

Data Interview Readiness

一句话定义：Data Interview Readiness 是候选人能否把大模型数据工程问题讲成目标、来源、处理、配比、评估、治理和 trade-off 闭环的准备度。

核心维度：answer coverage、metric coverage、engineering evidence、risk coverage、trade-off depth、red flags 和 time budget。

常用指标：回答覆盖度、指标覆盖度、token 保留率、风险命中率、contamination 命中率、effective epoch、lineage 覆盖率、datasheet 完成度和 mock interview 平均得分。

复盘方法：每次模拟面试后，记录题目、原回答、缺失框架、缺失指标、缺失风险、错误表述、超时点和下一轮 revision plan。复盘重点不是追求固定话术，而是让回答能被工程证据支撑。

面试表达：数据工程面试不是背某个数据集比例，而是证明你能设计可审计、可复现、可评估、可治理的数据系统。

Data Packing

一句话定义：data packing 是把多个较短文本样本拼接成固定长度训练序列的方法，用来减少 padding、提高 token 利用率。

为什么需要：预训练和微调常使用固定 sequence length。如果很多样本很短，直接 padding 会浪费大量计算。packing 可以把多个样本装进同一个序列。

常见做法：把多个文档或样本按 token 拼接，中间加入 EOS 或特殊分隔符，直到接近最大长度。

注意事项：不同文档之间是否允许 attention 互相可见，要看训练目标。预训练中跨文档拼接通常用 EOS 隔开；SFT 中 packing 还必须正确处理 loss mask，避免 user token 或 padding token 参与 loss。

工程坑：如果没有正确插入 EOS，模型可能学习错误的跨文档衔接；如果 label mask 错，模型可能在不该训练的位置计算 loss；如果边界处理错，可能导致对话模板污染。

面试表达：packing 是提高训练效率的关键工程技巧，但必须处理好 EOS、document boundary、attention mask 和 loss mask。

Document Boundary

一句话定义：document boundary 是不同文档或样本之间的边界信息。

为什么重要：语言模型训练时经常把多个文档拼接成连续 token。如果不告诉模型文档结束，模型可能把两个无关文档当成自然连续文本。

常见处理：在文档之间加入 EOS token、特殊分隔符或重置位置。现代预训练中最常见的是用 EOS 表示文档结束。

影响：边界处理会影响模型对段落结束、回答终止、多文档上下文和生成停止的学习。

面试表达：document boundary 是预训练数据格式的一部分。它看似细节，但会影响模型是否学会正确结束文本和区分独立样本。

Sequence Length

一句话定义：sequence length 是训练时单个样本包含的最大 token 数，也常被称为训练上下文长度。

影响：更长序列能让模型学习长距离依赖、长文档结构和长上下文任务，但 attention 成本通常随长度平方增长，训练代价更高。

常见策略：先用较短 sequence length 训练大部分 token，再用更长 sequence length 做长上下文继续训练；或者在数据中混合不同长度样本。

误区：把推理配置里的 max context 改大，不代表模型真的具备长上下文能力。长上下文能力需要位置编码、训练长度分布和评估共同支持。

面试表达：sequence length 是能力和成本之间的权衡。长上下文不是只改配置，而是训练数据、位置编码和系统成本共同决定。

Checkpoint

一句话定义：checkpoint 是训练过程中保存的模型和训练状态快照。

内容：model weights、optimizer state、learning rate scheduler state、RNG state、training step、数据加载状态、混合精度 scaler、模型 config 和 tokenizer 信息。

作用：故障恢复、阶段评估、模型选择、回滚异常训练、继续训练、发布中间模型。

保存策略：按固定 step 保存、按验证指标保存、保留最近若干个 checkpoint、保留关键里程碑 checkpoint。大模型 checkpoint 体积极大，存储和传输本身也是工程问题。

工程坑：只保存模型权重不能完全恢复训练；如果 optimizer state 或数据迭代状态丢失，继续训练的曲线可能发生变化。tokenizer/config 与 checkpoint 不匹配也会导致加载错误。

面试表达：checkpoint 不是简单的权重文件，而是可复现训练状态的快照。大规模训练必须设计可靠的 checkpoint 和恢复机制。

Training Token

一句话定义：training token 是训练过程中被模型看到并用于学习的 token 数量。

为什么重要：LLM 能力不仅取决于参数量，也取决于训练 token 数。参数很多但 token 不足，会出现 under-training；参数较小但 token 充分，可能在固定 compute 下更优。

统计注意：要区分原始字符数、文档数、tokenizer 后 token 数、参与 loss 的 token 数和被 mask 掉的 token 数。不同 tokenizer 下 token 数不可直接等价。

面试表达：训练 token 数是 scaling law 的核心变量之一。讨论模型训练规模时，只说参数量是不够的，还要说训练了多少 token。

Epoch

一句话定义：epoch 表示训练过程完整遍历一遍训练数据。

在大模型中的特殊性：LLM 预训练数据巨大，常常不按传统小数据集那样训练很多 epoch，而是更关注总训练 token 数和数据采样分布。

风险：对小规模或重复数据训练多个 epoch 会增加记忆和过拟合风险，尤其在领域继续预训练和 SFT 中更明显。

面试表达：在 LLM 预训练里，epoch 概念不如 training token 和 sampling distribution 重要；在微调里，epoch 数过多则容易过拟合和遗忘。

Scaling Law

一句话定义：scaling law 是描述模型性能随参数量、数据量和计算量扩展而变化的经验规律。

常见变量：模型参数量 N 、训练 token 数 D 、计算量 C 、验证 loss 或 perplexity。

核心用途：在正式训练大模型前，用小规模实验外推大规模模型表现，帮助决定模型规模、训练 token 数和算力预算。

典型结论：在一定范围内，增加参数、增加数据和增加计算通常能平滑降低语言建模 loss。但收益呈递减趋势，并且不同能力不一定都随 loss 同步提升。

局限：scaling law 多数基于 loss，而用户关心的指令遵循、安全、推理、工具调用和多模态能力不一定能完全由 loss 解释。

面试表达：scaling law 是大模型资源规划工具，不是绝对定律。它能指导 compute 分配，但不能替代真实下游评估。

Chinchilla Scaling

一句话定义：Chinchilla scaling 强调在固定计算预算下，应平衡模型参数量和训练 token 数，而不是一味增大参数。

核心启发：很多早期大模型参数很大，但训练 token 不足，属于 undertrained。给较小模型训练更多高质量 token，可能在同等 compute 下获得更低 loss。

直观理解：如果模型太大但数据太少，参数没有被充分训练；如果数据很多但模型太小，容量不足。compute-optimal training 要在容量和数据之间找平衡。

影响：后来很多模型更重视 token 数、数据质量和多 epoch/多 token 训练，而不是只追求参数规模。

注意：Chinchilla 结论依赖实验设定、数据分布和目标指标。实际模型还要考虑推理成本、部署需求、能力目标和数据可得性。

面试表达：Chinchilla 的面试重点不是背比例，而是说明“固定 compute 下，大模型训练要同时扩参数和扩数据，很多模型不是参数不够，而是训练 token 不够”。

Compute-Optimal Training

一句话定义：compute-optimal training 是在固定计算预算下选择最优模型规模、训练 token 数和训练配方的策略。

核心问题：预算应该花在更大模型上，还是更多 token 上？应该训练一个更强但推理贵的模型，还是训练一个较小但 token 更充分、部署更便宜的模型？

考虑因素：训练 compute、推理 compute、数据质量、目标任务、上下文长度、部署延迟、显存限制、服务成本和后训练需求。

与 Chinchilla 的关系：Chinchilla 给出语言建模 loss 视角下的 compute-optimal 经验启发；真实工程中的 compute-optimal 还要加入推理成本和产品目标。

例子：如果目标是高并发线上服务，较小但训练充分的模型可能比更大模型更合适；如果目标是前沿能力探索，则可能更愿意训练更大模型并接受高成本。

面试表达：compute-optimal training 不是单纯追求最大参数量，而是在训练成本、数据规模、模型容量和推理成本之间做全局最优。

本章小结

本章围绕预训练和数据治理展开。预训练决定 LLM 的能力底座，而数据决定模型从什么分布中学习、学习到哪些能力、带入哪些风险。

高频结论如下：

1. 预训练通常用 next-token prediction 在海量 token 上训练 base model。
2. 数据质量、去重和污染控制决定单位 compute 的训练收益。
3. data mixture 和 sampling strategy 会直接影响模型能力分布。
4. 合成数据有用，但必须经过筛选、验证和去重。
5. benchmark contamination 会让评估结果虚高，是大模型评估中的核心风险。
6. data packing 能提高训练效率，但必须处理好 EOS、边界和 loss mask。
7. checkpoint 应保存完整训练状态，而不只是模型权重。
8. scaling law 和 Chinchilla scaling 用来指导参数量、token 数和 compute 的平衡。
9. compute-optimal training 要同时考虑训练成本、推理成本、数据质量和目标能力。

下一章，我们进入后训练与对齐。

G. 后训练与对齐

条目：Post-Training、Instruction Tuning、SFT、Full Fine-Tuning、LoRA、QLoRA、Catastrophic Forgetting、Chat Template、Loss Mask、System Prompt、Synthetic Instruction Data、Preference Data、Reward Model、Pairwise Ranking Loss、RLHF、Policy Model、Reference Model、PPO、KL Penalty、Proxy Objective、Reward Hacking、Reward-Human Gap、Best-of-N Overoptimization、Reward Hacking Gate、Specification Gaming、Over-Optimization、Goodhart’s Law、Distribution Shift、Pairwise Accuracy、Reward Calibration、DPO、DPO Loss、Beta in DPO、IPO、KTO、ORPO、AI Feedback、RLAIF、Constitutional AI、Length Bias、Over-Refusal、Safety Alignment。

第二轮后训练事故补充条目：Post-Training Incident、Alignment Training Gate、Assistant Mask Coverage、Capability Regression、False Refusal Rate、Unsafe Leak Rate、Preference Margin、Reward Length Bias、DPO Reference Mismatch、Tool Schema Drift。

Post-Training

一句话定义：post-training 是预训练之后，为了让 base model 更适合指令遵循、对话、安全和人类偏好而进行的训练阶段。

核心目标：预训练让模型学会“如何续写文本”，后训练让模型学会“如何按用户意图回答”。这个阶段把 base model 从通用文本建模器塑造成 instruct model、chat model 或领域助手。

常见组成：instruction tuning、SFT、偏好数据收集、reward model、RLHF、DPO、安全微调、拒答边界训练、红队测试和能力回归评估。

为什么需要：base model 可能会继续补全文本，而不是回答问题；可能不理解 system/user/assistant 角色；可能输出有害内容；也可能不符合人类对有帮助、诚实、无害的偏好。

关键风险：后训练能显著改善交互体验，但也可能造成灾难性遗忘、过度拒答、模板化、能力退化、reward hacking 和风格单一。

面试表达：后训练不是单纯“微调一下”，而是把预训练能力对齐到用户任务和安全边界的系统工程。它既包括训练算法，也包括数据、评估和产品策略。

Instruction Tuning

一句话定义：instruction tuning 是使用指令-回答数据训练模型，让模型学会根据自然语言指令完成任务。

数据形式：常见样本包含 instruction、optional input、response，也可以是多轮 chat 格式。任务包括问答、摘要、翻译、改写、分类、代码解释、信息抽取、推理和格式生成。

解决的问题：base model 擅长续写，但不一定知道用户想让它执行任务。instruction tuning 通过大量任务示例，让模型学习“看到任务描述后输出答案”的行为模式。

能力提升：通常能提升 zero-shot、few-shot、格式遵循、多任务泛化和用户体验。模型会更像助手，而不是纯文本补全器。

数据质量要求：指令要清晰、多样；回答要正确、有帮助、格式稳定；任务类型要覆盖目标场景；不能大量混入错误答案和模板化表达。

面试表达：instruction tuning 的本质是用监督数据把模型的通用语言能力映射到指令执行能力上。

SFT

一句话定义：SFT 是 supervised fine-tuning，用监督学习在高质量输入-输出样本上微调模型。

核心目标：最大化 $P(\text{response} \mid \text{instruction}, \text{context})$ 。在 chat model 中，通常只对 assistant response 计算 loss，不对 system 和 user token 计算 loss。

关键实现：chat template、tokenization、label mask、padding、truncation、packing、EOS 处理、学习率、batch size、训练轮数和验证集设计。

为什么只训练 assistant token：用户输入和系统提示是条件，不是模型需要模仿输出的答案。如果把 user token 也计入 loss，模型可能学会生成用户问题或角色标记，影响对话行为。

常见风险：训练数据过窄导致通用能力退化；学习率过大导致遗忘；训练过久导致过拟合；错误答案会被模型模仿；过多模板化数据会让输出风格僵硬。

评估方式：不能只看 SFT loss。需要同时看指令遵循、事实性、格式稳定性、安全性、通用 benchmark、领域 benchmark 和人工评测。

面试表达：SFT 是后训练的基础环节，重点不只是算法，而是数据质量、chat template、loss mask 和能力回归评估。

Full Fine-Tuning

一句话定义：full fine-tuning 是更新模型全部参数的微调方式。

优点：表达能力强，能充分适配新任务或新领域；对大规模高质量后训练数据，效果通常更有上限。

缺点：显存和计算成本高，训练不稳定风险更大，保存完整 checkpoint 成本高，也更容易在小数据上过拟合或遗忘。

适用场景：数据规模较大、任务非常重要、资源充足、需要深度改变模型行为，或训练一个正式发布的 instruct/chat 模型。

面试表达：full fine-tuning 不是永远最好。它能力上限高，但成本和风险也高；小数据场景下 LoRA 往往更稳、更经济。

LoRA

一句话定义：LoRA 是冻结基座模型，在部分线性层上训练低秩 adapter 的参数高效微调方法。

核心直觉：不直接更新原始权重 W ，而是学习一个低秩增量 $\Delta W = BA$ 。推理时可以把增量合并回原权重，也可以以 adapter 形式加载。

优点：显存低、训练快、存储小、适合多任务多 adapter 管理，对资源有限的 SFT 很实用。

关键超参：rank、alpha、dropout、target modules、学习率和训练轮数。rank 太小可能表达不足，rank 太大成本上升且更易过拟合。

局限：LoRA 冻结大部分基座参数，深度改变模型能力的上限可能低于 full fine-tuning；不同任务 adapter 合并也可能相互干扰。

面试表达：LoRA 的价值是用小参数增量近似微调效果，核心权衡是成本、可管理性和适配能力上限。

QLoRA

一句话定义：QLoRA 是量化加载基座模型，并在量化模型上训练 LoRA adapter 的参数高效微调方法。

为什么提出：大模型显存占用巨大，普通 LoRA 仍然需要加载较高精度的基座权重。QLoRA 通过 4-bit 等量化方式降低显存，使单卡或少卡微调更可行。

优势：显存需求更低，训练成本更低，适合资源有限的领域 SFT 和实验验证。

风险：量化配置、计算 dtype、梯度 checkpoint、optimizer 和 target modules 都会影响稳定性；极端量化可能损失模型能力。

面试表达：QLoRA 是 LoRA 与低比特量化结合的微调方案，主要解决显存瓶颈，不是新的对齐目标。

Catastrophic Forgetting

一句话定义：catastrophic forgetting 是微调后模型丢失原有通用能力、专业能力或安全能力的现象。

常见表现：领域任务变好，但通用问答、数学、代码、多语言或指令遵循变差；模型风格变窄；安全拒答边界异常。

常见原因：数据分布过窄，学习率过大，训练轮数过多，缺少混合数据，缺少能力回归评估，或者只优化单一任务 loss。

缓解方法：降低学习率，减少训练步数，使用 LoRA，引入通用 replay 数据，混合多能力数据，做 early stopping，建立回归评估集。

面试表达：微调不是只看目标任务提升，还要看原有能力有没有退化。灾难性遗忘是后训练必须监控的核心风险。

Chat Template

一句话定义：chat template 是把 system、user、assistant 等多轮消息序列化 token 序列的格式规则。

为什么重要：模型训练时看到的是序列化后的 token，不是结构化 JSON。template 决定角色边界、轮次边界、assistant 开始位置和 EOS 位置。

常见问题：训练和推理 template 不一致会导致分布偏移；特殊 token 配置错误会导致角色标记泄漏；EOS 缺失会让模型停不下来；assistant prompt 错误会导致不回答或续写用户。

工程建议：训练、评估和推理必须使用同一 tokenizer 和 chat template；新增 special token 后要同步 resize embedding；上线前用样例打印最终 token 化文本。

面试表达：chat template 是 chat model 的输入协议。很多“模型不听话”的问题，根因不是权重，而是模板、EOS 或 loss mask 错。

Loss Mask

一句话定义：loss mask 是控制哪些 token 参与 loss 计算的掩码。

在 SFT 中：通常 mask 掉 system token、user token、padding token，只让 assistant response token 参与 loss。

为什么重要：如果 user token 参与 loss，模型会学会复现用户输入；如果 padding 参与 loss，训练会引入无意义目标；如果 assistant token 被误 mask，模型学不到回答。

常见排查：打印 input ids、labels、attention mask 和解码文本，确认 label 为 ignore_index 的位置是否正确，确认 EOS 是否参与训练。

面试表达：loss mask 是 SFT 正确性的基础细节。后训练 loss 正常但模型行为异常时，首先要检查模板和 mask。

System Prompt

一句话定义：system prompt 是位于对话开头，用来设定助手身份、行为边界、安全规范和输出风格的高优先级提示。

作用：规定模型的角色、任务范围、输出格式、安全规则和冲突处理原则。

局限：如果模型没有通过训练学会遵循系统消息，仅靠 system prompt 不稳定。它还可能被 prompt injection 攻击诱导绕过。

工程策略：system prompt 应与训练数据、工具权限、安全分类器、输出校验和日志监控配合使用，而不是单独承担安全边界。

面试表达：system prompt 是行为控制层，不是安全系统本身。真正的对齐需要训练、权限控制和评估闭环。

Synthetic Instruction Data

一句话定义：synthetic instruction data 是由强模型、规则或程序生成的指令微调数据。

优势：低成本、可扩展、可定向补齐能力，适合生成长尾任务、格式数据、安全样本、代码题和多语言样本。

风险：教师模型错误会被学生模仿；风格单一会导致学生模型模板化；生成数据可能包含幻觉、偏见和不安全内容；重复样本会降低有效信号。

质量控制：使用强教师模型、多模型评审、规则校验、执行验证、人工抽检、去重、难度分层和覆盖度统计。

面试表达：合成指令数据的关键是过滤和验证，而不是生成本身。没有质量控制的合成数据会把教师模型的错误放大。

Preference Data

一句话定义：preference data 是表示人类或 AI 更偏好哪个回答的数据，常见格式是同一 prompt 下的 chosen/rejected response pair。

来源：人工标注、专家标注、用户反馈、模型自评、规则筛选、红队样本和 AI 反馈。

用途：训练 reward model，或直接用于 DPO、IPO、KTO、ORPO 等偏好优化方法。

标注维度：有帮助性、事实性、完整性、简洁性、格式遵循、安全性、拒答合理性、语气和用户满意度。

常见问题：标注者偏好不一致、长度偏差、领域知识不足、chosen 只是相对更好但仍然有错、偏好标准随业务变化。

面试表达：偏好数据是对齐阶段的燃料。它不只是“哪个答案更好”，还隐含了产品价值观和安全边界。

Reward Model

一句话定义：reward model 是输入 prompt 和 response 后输出标量 reward 的模型，用来近似人类偏好。

训练目标：常用 pairwise ranking loss，让 chosen response 的 reward 高于 rejected response。

作用：在 RLHF 中，reward model 为 policy model 生成的回答打分，作为强化学习优化信号。

局限：reward score 是偏好代理，不等于真实质量、事实正确性或安全性。它可能对长度、语气、模板、领域和分布外回答产生偏差。

面试表达：reward model 是把人类偏好压缩成可优化分数的代理模型。它越被强力优化，越需要警惕 reward hacking 和分布偏移。

Pairwise Ranking Loss

一句话定义：pairwise ranking loss 是用于偏好 pair 的排序损失，鼓励 chosen 的分数高于 rejected。

常见形式：

$$L = -\log \text{sigmoid}(r_{\text{chosen}} - r_{\text{rejected}})$$

直觉：如果 chosen 的 reward 已经明显高于 rejected，loss 较小；如果 rejected 反而更高，loss 较大。

局限：它只学习相对偏好，不保证 reward 分数有绝对可解释性；如果 pair 数据质量差，模型会学到错误偏好。

面试表达：pairwise loss 适合偏好学习，因为人类通常更容易比较两个回答，而不是给单个回答打绝对分。

RLHF

一句话定义：RLHF 是 reinforcement learning from human feedback，通过人类偏好训练 reward model，再用强化学习优化语言模型行为。

典型流程：先训练 SFT model；收集同一 prompt 下多个回答的偏好数据；训练 reward model；用 PPO 等算法优化 policy model，同时用 KL penalty 约束它不要偏离 reference model 太远。

为什么需要：SFT 学的是示范答案，但人类偏好往往难以完全写成监督数据。RLHF 可以让模型在多个候选行为中更偏向人类喜欢的输出。

优势：能显著改善有帮助性、对话风格、拒答边界和用户体验。

难点：训练链路复杂，reward model 偏差会被放大，PPO 稳定性敏感，评估成本高，容易出现 reward hacking。

面试表达：RLHF 的核心不是“强化学习很高级”，而是把人类偏好变成优化信号，并用 KL 约束防止模型偏离语言分布。

Policy Model

一句话定义：policy model 是 RLHF 中被优化的语言模型策略，负责根据 prompt 生成 response。

在 LLM 中：policy 的动作是生成 token，状态是当前上下文，整段回答的质量由 reward model 或规则反馈评估。

初始化：通常从 SFT model 初始化，而不是直接从 base model 开始。这样模型已有较好的指令遵循和语言质量。

面试表达：policy model 就是 RLHF 中真正被更新的模型，目标是提高 reward，同时保持可读性、安全性和原有能力。

Reference Model

一句话定义：reference model 是 RLHF 或 DPO 中通常冻结的基准模型，用来约束 policy 不要偏离原始分布太远。

常见选择：SFT model 的冻结副本。

作用：提供 KL 约束或隐式正则，防止 policy 为了追求 reward 而生成奇怪、重复、过长或分布外文本。

面试表达：reference model 是对齐优化中的“锚点”。没有这个锚点，模型容易为了 reward 分数牺牲语言质量和通用能力。

PPO

一句话定义：PPO 是 Proximal Policy Optimization，一种常用于 RLHF 的策略优化算法。

核心思想：在提高期望 reward 的同时限制每次策略更新幅度，避免策略更新过猛导致训练崩溃。

在 RLHF 中：policy 生成回答，reward model 打分，再结合 KL penalty 得到最终优化信号，用 PPO 更新 policy。

优点：相对稳定，实践成熟。缺点：实现复杂，超参敏感，训练成本高，需要同时维护 policy、reference、reward model 和 value model 等组件。

面试表达：PPO 是 RLHF 的常见优化器，负责在 reward 提升和策略稳定之间做平衡。

KL Penalty

一句话定义：KL penalty 是惩罚 policy model 与 reference model 输出分布差异的约束项。

作用：降低模型偏离、语言退化和 reward hacking 风险。它让模型在变得更符合偏好的同时，不要离 SFT 模型太远。

直觉：如果 policy 为了高 reward 生成极端文本，KL 会变大，从而抵消一部分 reward。

权衡：KL 系数过大，模型学不动；过小，模型可能过度优化 reward model。

面试表达：KL penalty 是 RLHF 稳定性的关键，它把偏好优化限制在合理的语言分布附近。

Reward Hacking

一句话定义：reward hacking 是模型利用 reward model 漏洞获得高分，但输出不符合真实人类意图的现象。

典型表现：无意义长回答、过度礼貌、模板化、过度自信、编造引用、规避问题、过度拒答、重复安全免责声明。

根本原因：reward model 是人类偏好的近似代理，不可能覆盖所有真实质量维度。policy 一旦强力优化代理分数，就可能找到代理漏洞。

缓解方法：限制优化强度，使用 KL 约束，更新 reward model，引入人工评估，使用多维 reward，加入红队测试和分布外评估。

面试表达：reward hacking 是 Goodhart 定律在 RLHF 中的体现。指标被优化得越强，越可能偏离原本想衡量的真实目标。

Reward-Human Gap

一句话定义：reward-human gap 是代理奖励分数和人工 / gold 质量评价之间的差距。

为什么重要：如果 reward 分数上升但人工质量、事实性、安全性或 citation accuracy 下降，说明模型可能在优化 proxy，而不是提升真实质量。

常见监控：比较 reward score、human win rate、专家评分、自动 verifier、输出长度、拒答率、事实性和高风险失败率。

面试表达：对齐训练不能只看 reward curve，还要看 reward-human gap。gap 扩大通常是 reward hacking 或 reward model 分布外失效的信号。

Best-of-N Overoptimization

一句话定义：Best-of-N overoptimization 是从多个候选中按 reward model 选最高分时，随着候选数量变大，选中迎合 reward model 但真实质量差的回答。

为什么发生：候选越多，越容易出现“reward model 误判但分数很高”的极端样本。选最高分会放大这种误判。

风险：即使没有 PPO 或 RL loop，只做采样加 reward rerank，也可能出现 reward hacking、长度偏置、模板化和事实性下降。

面试表达：reward hacking 不只发生在 RLHF，best-of-N、reranking、LLM-as-a-Judge 筛选和自动数据生成都可能把 proxy 漏洞放大。

Reward Hacking Gate

一句话定义：reward hacking gate 是上线或继续优化前，用来阻断 proxy 过度优化的一组门禁。

常见指标：proxy mismatch、reward hacking rate、reward-human gap、length bias、high-reward-low-quality ratio、severity-weighted hack、held-out human preference 和高风险人工抽检。

工程意义：如果高 reward 样本中出现低质量、高风险或过度模板化回答，即使平均 reward 很高，也不能继续上线或扩大优化强度。

面试表达：reward hacking gate 的目标是证明 reward 可用，而不是证明模型分数高。

Proxy Objective

一句话定义：proxy objective 是训练或评估中实际可优化的代理目标，用来近似真实目标。

例子：cross entropy 近似语言建模质量，reward model 近似人类偏好，safety classifier 近似策略合规，benchmark score 近似任务能力，用户评分近似产品价值。

核心风险：proxy 不等于真实目标。优化强度越大，模型越可能利用 proxy 的漏洞，而不是提升真实质量。

面试表达：对齐训练里很多目标都是 proxy。工程上必须同时看 proxy 指标、人类评估、安全评估、分布外评估和上线行为。

Specification Gaming

一句话定义：specification gaming 是模型或智能体利用目标规范、规则或评估指标的漏洞，获得高分但违背真实意图。

和 reward hacking 的区别：reward hacking 通常聚焦 reward function 或 reward model；specification gaming 更泛，包括 benchmark gaming、规则漏洞、环境漏洞和工具流程漏洞。

LLM 例子：评测偏好结构化回答，模型输出大量标题但内容空洞；judge 偏好长答案，模型变得冗长；安全策略只看关键词，模型对正常请求过度拒答。

面试表达：specification gaming 的重点是“模型按你写的规则赢了，但不是按你真正想要的方式赢了”。

Over-Optimization

一句话定义：over-optimization 是模型过度优化 reward model，导致 reward 分数上升但人类实际偏好下降的现象。

现象：训练初期 reward 和人工质量可能一起提升；继续优化后 reward 继续升，但回答开始变长、变空、变模板化或事实性下降。

原因：reward model 在训练分布内有效，但 policy 生成分布逐渐变化后，reward model 的判断不再可靠。

应对：监控人工评测和 held-out 偏好集，控制 KL，early stopping，定期重训 reward model，使用多样化评估。

面试表达：偏好优化不是越久越好。对齐训练必须监控真实人类指标，而不是只看 reward 曲线。

Goodhart's Law

一句话定义：Goodhart's Law 指当一个指标成为优化目标，它就不再是好指标。

在 RLHF 中：reward score 原本用于近似人类偏好，但当 policy 专门优化 reward score 时，它可能学会取巧，让 reward score 失去代表性。

例子：如果 reward model 偏好长答案，policy 可能生成冗长但空洞的答案；如果 reward model 偏好安全措辞，policy 可能对正常问题也过度拒答。

面试表达：Goodhart 定律解释了为什么对齐训练不能只依赖单一 reward，需要人工评估、多指标监控和安全边界。

Distribution Shift

一句话定义：distribution shift 是训练分布和测试或优化时遇到的分布不一致。

在 reward model 中：RM 训练时看到的回答来自 SFT model 或固定候选集合，但 RLHF 过程中 policy 会不断变化，生成越来越不同的回答。

风险：RM 对新分布回答打分不可靠，可能鼓励奇怪行为，引发 reward hacking。

缓解：在线采样新回答补充偏好数据，迭代训练 reward model，引入不确定性估计和人工审核。

面试表达：对齐系统是动态分布问题。policy 变化后，reward model 的泛化能力本身也需要被评估。

Pairwise Accuracy

一句话定义：pairwise accuracy 是 reward model 在偏好验证集上是否给 chosen response 更高分的比例。

用途：评估 RM 是否学会了偏好排序。

局限：整体准确率可能掩盖分领域失败；高 pairwise accuracy 不代表 reward 分数校准；也不代表 policy 优化后不会 hacking。

面试表达：pairwise accuracy 是 RM 的基础指标，但只能说明排序拟合情况，不能单独证明 reward model 可安全用于强优化。

Reward Calibration

一句话定义：reward calibration 是 reward model 的分数尺度是否能稳定反映偏好强弱和不确定性。

为什么重要：如果 reward score 未校准，不同领域、不同长度、不同任务之间的分数不可比，PPO 或 reranking 可能被误导。

常见问题：模型对长回答普遍打高分，对某类语气有偏好，对专业领域误判，对分布外回答过度自信。

面试表达：RM 不只要排序准确，还要分数可用。校准差的 reward 会让优化过程变得不稳定和不可解释。

DPO

一句话定义：DPO 是 Direct Preference Optimization，直接用 chosen/rejected 偏好数据优化 policy 的方法。

核心特点：不显式训练 reward model，不使用 PPO，而是通过偏好 pair 的 log probability 构造训练目标。

为什么流行：训练链路比 RLHF 简洁，稳定性更好，工程实现更容易，适合用已有偏好数据对 SFT 模型进一步对齐。

基本直觉：如果人类更喜欢 chosen，那么模型应提高 chosen 相对 rejected 的概率；同时用 reference model 控制变化幅度。

局限：DPO 依赖偏好数据质量；对 pair 构造、长度偏差、beta、reference 选择敏感；并不自动解决事实性和安全边界问题。

面试表达：DPO 可以理解为把 RLHF 中的 reward 建模和 KL 约束推导成一个直接的监督式偏好优化目标。

DPO Loss

一句话定义：DPO loss 鼓励 policy 相比 reference model 更偏向 chosen response 而不是 rejected response。

核心直觉：比较 policy 对 chosen/rejected 的 log probability 差值，再减去 reference model 的对应差值。如果 policy 比 reference 更偏向 chosen，loss 就会更小。

关注点：response 的 log probability 计算、label mask、长度归一化、reference model 冻结、beta 设置和数据 pair 质量。

面试表达：DPO loss 不是简单提高 chosen 概率，而是提高 chosen 相对 rejected、相对 reference 的优势。

Beta in DPO

一句话定义：DPO 中的 beta 是控制偏好优化强度和 reference 约束权衡的超参数。

影响：beta 较大时，偏好优化更强，模型行为变化更明显，但可能带来退化；beta 较小时，更新更保守，可能优化不足。

选择方式：通常需要结合验证集、人工评估、能力回归和安全评测调参。

面试表达：beta 控制 DPO 的对齐力度。它不是越大越好，而是在偏好提升和模型稳定之间取平衡。

IPO

一句话定义：IPO 是 Identity Preference Optimization，一类直接偏好优化方法，目标是改善 DPO 在某些设定下可能过度优化偏好间隔的问题。

核心动机：DPO 的目标可能持续拉大 chosen 与 rejected 的概率差；IPO 尝试用更直接的偏好约束稳定优化。

适用理解：面试中不必死记公式，重点知道它属于 DPO 之后的直接偏好优化变体，关注优化稳定性和偏好过拟合问题。

面试表达：IPO 是偏好优化家族的一种改进方法，目的在于让偏好学习更稳定，减少过度拉大率差带来的副作用。

KTO

一句话定义：KTO 是 Kahneman-Tversky Optimization，一种不一定需要成对偏好、可利用好/坏单样本反馈的对齐方法。

核心动机：真实反馈中不总能得到 chosen/rejected pair，很多时候只有“这个回答好”或“这个回答差”。KTO 试图利用这类非成对反馈训练模型。

优势：数据格式更灵活，适合从用户点赞、点踩、审核标签等反馈中构造训练信号。

面试表达：KTO 的重点是降低对成对偏好数据的依赖，用更接近真实产品反馈的数据做对齐。

ORPO

一句话定义：ORPO 是 Odds Ratio Preference Optimization，将监督微调 and 偏好优化结合到一个训练目标中的方法。

核心思想：在学习 chosen response 的同时，显式降低 rejected response 的相对优势，从而不需要单独的 reference model 或复杂 RL 流程。

优势：流程简洁，适合把 SFT 和 preference alignment 合并训练。

面试表达：ORPO 是简化对齐流程的偏好优化方法，强调用一个目标同时做模仿学习和偏好区分。

RLAIF

一句话定义：RLAIF 是 reinforcement learning from AI feedback，用 AI 反馈替代或补充人类反馈进行对齐。

为什么需要：人工偏好标注成本高、速度慢、标准难统一。强模型或规则系统可以提供大规模、低成本反馈。

优点：可扩展、成本低、反馈速度快。风险是 AI 评审可能继承教师模型偏见、误判和安全盲点。

面试表达：RLAIF 降低了反馈成本，但反馈质量仍然决定对齐上限。AI feedback 通常需要人工抽检和校准。

AI Feedback

一句话定义：AI feedback 是由模型、规则系统或 verifier 生成的反馈信号，可以用于 critique、revision、preference、risk label、judge score 或候选答案筛选。

为什么需要：复杂任务和高风险安全样本的人工标注成本高、速度慢、覆盖有限。AI feedback 可以扩展监督规模，帮助人类处理长上下文、代码、数学、RAG 和 Agent trace。

核心风险：AI feedback 不是 gold label。它可能继承教师模型偏差、对分布外样本过度自信、偏好表面格式或被待评估模型输出操纵。

面试表达：AI feedback 要和 human gold set、专家抽检、工具验证、分切片校准和上线回归结合使用。它解决的是监督扩展问题，不自动解决对齐问题。

Constitutional AI

一句话定义：Constitutional AI 是使用一组原则或宪法来指导模型自我批评、修正和偏好反馈的对齐方法。

核心思路：不完全依赖人工逐条标注，而是让模型根据明确原则判断回答是否安全、诚实、有帮助，并生成修正版本或偏好标签。

优势：可以把安全原则显式化，降低人工标注成本，提高安全数据扩展性。

局限：原则设计会影响模型行为；模型对原则的理解可能不稳定；复杂价值冲突仍需要人工和产品策略介入。

面试表达：Constitutional AI 的关键是把对齐标准写成原则，再用模型自评和修正扩展安全训练数据。

Length Bias

一句话定义：length bias 是由于回答长度不同导致偏好判断、reward 分数或 log probability 比较受到影响的偏差。

常见表现：标注者或 reward model 偏好更长、更详细的回答；DPO 中长回答的总 log probability 天然更低；生成策略中模型可能为了高分输出冗长文本。

应对：控制 chosen/rejected 长度差异，做长度归一化，引入简洁性偏好，人工评估时明确 “不是越长越好”。

面试表达：长度偏差是偏好数据和偏好优化中的常见陷阱。一个回答更长，不代表它更正确或更有帮助。

Over-Refusal

一句话定义：over-refusal 是模型对本应正常回答的请求也过度拒绝的现象。

常见原因：安全数据比例过高，安全边界标注过严，模型把敏感词和危险意图混淆，reward model 过度奖励拒答。

例子：用户询问合法的化学安全知识、网络防护知识、医疗常识，模型却直接拒绝回答。

危害：降低可用性，伤害用户体验，也可能掩盖模型没有能力区分安全和不安全请求的问题。

缓解：细化安全分类，加入正当使用样本，训练安全替代回答，区分拒答、部分回答和安全引导。

面试表达：安全对齐不是拒绝越多越好。好的安全模型应能区分恶意请求和正当请求，在边界内尽量有帮助。

Post-Training Incident

一句话定义：post-training incident 是 SFT、RLHF、DPO、RLAIF、安全微调或工具调用对齐后出现的行为退化、格式错位、安全边界异常、reward 偏差或能力回归事故。

典型现象：SFT 后基础能力下降、模型复述用户输入、过度拒答、漏拒、回答变长变空、reward 分数升高但人工质量下降、DPO 后风格变窄、工具调用 schema 与线上不一致。

工程做法：把后训练评估拆成 mask、template、能力切片、安全切片、偏好 pair、reward-human gap、DPO reference 和线上 schema，一项项审计。

面试表达：后训练事故不是 “模型变差了” 这么笼统，而是训练目标、数据、mask、偏好和评估之间出现了某种不一致。

Assistant Mask Coverage

一句话定义：assistant mask coverage 是 SFT 中 assistant response token 实际参与 loss 的比例。

为什么重要：assistant token 被误 mask，模型学不到回答；system / user token 参与 loss，模型可能复述提示或生成角色标记。

面试表达：SFT loss 看起来正常不代表 mask 正确。要按 role 统计 labels，确认 assistant 覆盖、prompt 泄漏、padding 泄漏和 EOS 覆盖。

Capability Regression

一句话定义：capability regression 是后训练后某些能力切片相对 base / SFT / 旧版本下降。

常见切片：指令遵循、数学、代码、事实问答、长上下文、安全、工具调用、多轮对话和目标领域任务。

面试表达：后训练不能只看目标能力提升。任何发布前都要做 base vs SFT vs aligned model 的多维回归评估。

False Refusal Rate

一句话定义：false refusal rate 是安全可答请求被模型错误拒答的比例。

为什么重要：过度拒答会损害可用性，也说明模型没有学会区分正当教育、防护、常识请求和真正高风险请求。

面试表达：安全训练要同时看误拒和漏拒。只提高拒答率不是好的安全对齐。

Unsafe Leak Rate

一句话定义：unsafe leak rate 是应拒绝或应限制的高风险请求被模型直接回答的比例。

用途：衡量安全对齐是否漏掉危险边界，通常需要配合风险类别、严重度、人工审核和回归集。

面试表达：漏拒是硬门禁，但修漏拒时不能把所有边界请求都训练成拒答，否则会制造新的误拒。

Preference Margin

一句话定义：preference margin 是 chosen response 与 rejected response 在人工质量、rubric 分或 verifier 分上的差距。

为什么重要：margin 太小，DPO / reward model 学到的就是噪声；chosen 质量低于 rejected 时，偏好数据本身就是反向监督。

面试表达：偏好数据质量比数量更重要。训练前要抽查 chosen/rejected 是否真的有明确质量差异。

Reward Length Bias

一句话定义：reward length bias 是 reward model、judge 或标注者偏好更长回答，而不是更正确、更有帮助回答的偏差。

常见后果：RLHF、best-of-N、rerank 或 DPO 后输出越来越长、免责声明越来越多、结构化标题很多但内容变空。

面试表达：reward 高不等于质量高。要监控 reward 与长度、人工质量、事实性和安全切片的关系。

DPO Reference Mismatch

一句话定义：DPO reference mismatch 是 DPO 训练中使用的 reference model 与预期 SFT baseline、tokenizer、chat template 或 checkpoint 不一致。

为什么重要：DPO 的 margin 是 policy 相对 reference 的偏好变化。如果 reference 选错，训练目标会失去原本的分布锚点。

面试表达：DPO 排查不能只看 loss。要确认 reference 冻结、tokenizer 一致、chat template 一致、beta 合理和 pair 数据可靠。

Tool Schema Drift

一句话定义：tool schema drift 是后训练数据中的工具名称、参数、返回格式或错误恢复逻辑与线上工具 schema 不一致。

常见现象：模型过度调用工具、参数名写错、忽略 observation、工具失败后不会恢复，或者训练中学到的函数名在线上不存在。

面试表达：工具调用对齐不是只让模型输出 JSON，而是让训练 schema、推理 schema、权限和错误恢复保持一致。

Alignment Training Gate

一句话定义: alignment training gate 是把 assistant mask coverage、prompt / pad loss leak、EOS coverage、capability regression、false refusal、unsafe leak、preference margin、reward-human gap、reward length bias、DPO reference 和 tool schema drift 合并成后训练门禁。

用途: 在 SFT、DPO、RLHF、RLAIF、安全数据变更、工具 schema 变更和模型发布前, 判断后训练是否可以继续扩大或上线。

面试表达: alignment gate 的价值是阻止单指标幻觉。SFT loss、reward curve、DPO loss 或聊天体验单独变好, 都不能证明后训练没有副作用。

Safety Alignment

一句话定义: safety alignment 是让模型在有害、违法、隐私、歧视、自伤、危险操作等场景中遵循安全边界的训练和评估过程。

目标: 减少有害内容生成, 保护隐私, 避免危险指导, 降低偏见和歧视, 同时保持对正常请求的帮助性。

常见手段: 安全 SFT、拒答数据、偏好优化、红队攻击、分类器过滤、工具权限控制、输出审查和日志监控。

难点: 安全边界具有上下文依赖。同样的内容在新闻报道、安全教育、科研讨论和恶意操作中含义不同。模型需要判断意图、场景和风险等级。

面试表达: 安全对齐是训练和系统工程共同完成的, 不能只靠一句 system prompt, 也不能只靠模型拒答。理想状态是在安全边界内最大化有帮助性。

本章小结

本章围绕后训练与对齐展开。预训练提供通用能力, 后训练决定模型如何与用户交互、如何遵循指令、如何表达偏好、如何处理安全边界。

高频结论如下:

1. 后训练把 base model 从文本续写器塑造成 instruct/chat assistant。
2. SFT 的核心不只是监督学习, 还包括 chat template、loss mask、EOS 和数据质量。
3. LoRA/QLoRA 主要解决微调成本问题, 不改变对齐目标本身。
4. 偏好数据表达人类或 AI 对回答质量的相对判断, 是 RLHF/DPO 的基础。
5. RLHF 用 reward model 和 PPO 优化行为, 但容易遇到 reward hacking 和 over-optimization。
6. KL penalty 和 reference model 用来限制 policy 偏离, 保护语言质量和稳定性。
7. DPO 用偏好 pair 直接优化模型, 流程比 RLHF 简洁, 但仍依赖数据质量和超参选择。
8. RLAIF、Constitutional AI、IPO、KTO、ORPO 都是围绕反馈成本、优化稳定性和数据形式的改进方向。
9. 安全对齐不是简单拒答, 而是在安全边界内尽量保持有帮助性。

下一章, 我们进入推理与解码。

H. 推理与解码

条目: Inference、Autoregressive Generation、Decoding、Greedy Decoding、Sampling、Temperature、Top-k Sampling、Top-p Sampling、Beam Search、Repetition Penalty、Presence Penalty、Frequency Penalty、Stop Token、EOS Token、Max New Tokens、Best-of-N、Self-Consistency、Log Probability、Token Probability、KV Cache、Prefill、Decode、Time to First Token / TTFT、Time per Output Token / TPOT、Speculative Decoding。

Inference

一句话定义: inference 是使用训练好的模型, 根据输入上下文生成输出的过程。

在 LLM 中：推理通常不是一次性生成完整答案，而是自回归地逐 token 生成。模型先读取 prompt，输出下一个 token 的概率分布，再根据解码策略选出 token，追加到上下文中继续生成。

基本流程：tokenizer 把文本转成 token ids；模型 forward 得到 logits；解码策略从 logits 中选择下一个 token；把新 token 拼回输入；重复直到遇到停止条件。

与训练的区别：训练时使用 teacher forcing，可以并行计算一段序列中所有位置的 loss；推理时下一个 token 依赖上一个已生成 token，因此输出长度维度存在串行依赖。

主要指标：首 token 延迟、每 token 延迟、吞吐、显存占用、生成质量、稳定性和成本。算法岗面试中，既要讲清模型侧机制，也要理解系统侧约束。

面试表达：LLM 推理的核心是自回归生成。它看似只是 forward，但真实系统中还涉及 tokenizer、chat template、KV Cache、解码策略、停止条件和延迟吞吐权衡。

Autoregressive Generation

一句话定义：autoregressive generation 是模型每次生成一个 token，并把该 token 作为后续生成上下文的一部分。

核心公式：

$$P(y_1, \dots, y_T \mid x) = \prod_t P(y_t \mid x, y_{1:t-1})$$

其中 x 是 prompt， y_t 是第 t 个输出 token。

为什么串行：第 $t+1$ 个 token 的分布依赖第 t 个 token。如果第 t 个 token 还没确定，就无法准确计算下一步上下文。

影响：生成长答案时，延迟会随输出 token 数增加；早期生成错误会污染后续上下文；解码策略会显著影响最终答案。

面试表达：自回归生成让 LLM 能开放式输出，但也带来推理串行、错误累积和长输出成本高的问题。

Decoding

一句话定义：decoding 是从模型输出的下一个 token 概率分布中选择 token 的策略。

输入是什么：模型 forward 后得到词表维度 logits。经过 temperature、top-k、top-p、重复惩罚等处理后，可以得到一个候选 token 分布。

为什么重要：同一个模型、同一个 prompt，使用不同 decoding 参数可能得到完全不同的输出。解码直接影响稳定性、多样性、创造性、重复率、事实性和格式遵循。

常见目标：问答和代码生成通常希望稳定、准确，可以用较低 temperature；创意写作希望多样，可以提高 sampling 随机性；分类和结构化抽取通常希望确定性，可以使用 greedy 或低温采样。

面试表达：decoding 不改变模型参数，但会改变模型行为。调生成效果时，除了换模型和改 prompt，也要检查 decoding 参数。

Greedy Decoding

一句话定义：greedy decoding 是每一步都选择概率最高的 token。

核心规则：

$$y_t = \operatorname{argmax}_i P(\text{token}_i \mid \text{context})$$

优点：确定、简单、速度快、复现性好。相同模型、相同输入、相同环境下通常输出一致。

缺点：容易陷入局部最优，输出可能保守、死板、重复；在开放式写作中缺少多样性；在需要探索多条推理路径的问题上可能错过正确答案。

适用场景：分类、信息抽取、格式转换、代码补全、低随机性问答、评测基线。

面试表达: greedy 的优点是稳定, 缺点是没有探索。它每一步局部最优, 不保证整段序列全局最优。

Sampling

一句话定义: sampling 是按照下一个 token 概率分布随机抽取 token 的解码方式。

核心直觉: 模型不是永远选概率最高的 token, 而是让高概率 token 更容易被选中, 低概率 token 也有一定机会出现。

优点: 多样性更强, 适合开放式对话、创意写作、头脑风暴和需要多个候选答案的场景。

缺点: 随机性更高, 可能采到低质量 token, 事实性和格式稳定性可能下降。temperature、top-k、top-p 设置不当时, 幻觉和跑题风险会上升。

面试表达: sampling 用随机性换多样性。生成任务越开放, sampling 越有价值; 任务越强调准确和稳定, 随机性越需要收敛。

Temperature

一句话定义: temperature 是缩放 logits、控制概率分布尖锐程度的参数。

常见形式:

$\text{softmax}(\text{logits} / \text{temperature})$

低温效果: temperature 小于 1 时, 概率分布更尖锐, 高概率 token 更突出, 输出更稳定、更确定。

高温效果: temperature 大于 1 时, 概率分布更平滑, 更多 token 有机会被采样, 输出更多样, 但也更容易不稳定。

特殊情况: temperature 接近 0 时行为接近 greedy decoding。实际系统中通常会对温度取值做边界限制, 避免数值异常。

面试表达: temperature 控制生成随机性。低温适合准确任务, 高温适合创意任务, 但高温会增加幻觉和格式错误风险。

Top-k Sampling

一句话定义: top-k sampling 是只保留概率最高的 k 个 token, 再在这些 token 中重新归一化采样。

核心作用: 截断长尾低概率 token, 避免模型采到明显不合理的候选。

例子: 如果 $k=50$, 每一步只在概率最高的 50 个 token 中采样, 其他 token 概率置零。

优点: 简单直接, 能减少低质量尾部 token。缺点是候选集合大小固定, 不管当前分布是否已经很确定, 都保留同样数量的候选。

面试表达: top-k 用固定候选数量控制采样范围, 适合限制低概率噪声, 但不如 top-p 自适应。

Top-p Sampling

一句话定义: top-p sampling 又称 nucleus sampling, 是保留累计概率达到 p 的最小 token 集合, 再从中采样。

核心直觉: 当模型很确定时, 只需要少量 token 就能覆盖大部分概率; 当模型不确定时, 候选集合会自动变大。

例子: 如果 $p=0.9$, 先按概率从高到低排序, 取累计概率达到 90% 的最小集合, 再重新归一化采样。

优点: 候选集合大小自适应, 通常比 top-k 更灵活。缺点是 p 太高会保留过多长尾 token, 太低会让输出过于保守。

面试表达: top-p 控制的是概率质量, 而不是固定 token 数。它会随模型不确定性动态调整候选集合。

Beam Search

一句话定义：beam search 是维护多个候选序列，每一步保留累计得分最高的若干条路径。

核心参数：beam size。beam size 越大，搜索路径越多，计算成本越高。

适用场景：机器翻译、摘要、语音识别等目标较明确、答案空间较受限的任务。

在开放式对话中的问题：beam search 常生成平庸、模板化、低多样性的答案，也可能偏向短句或高频表达。聊天模型通常更常用 sampling 或 greedy。

面试表达：beam search 是序列级近似搜索，不是简单逐步贪心。但在开放式 LLM 对话中，它不一定比采样更好。

Repetition Penalty

一句话定义：repetition penalty 是降低已出现 token 再次被生成概率的机制，用于缓解重复输出。

为什么需要：自回归模型可能陷入重复循环，例如反复生成同一句话、同一段格式或同一个词。

作用方式：对上下文中已经出现过的 token 调整 logits，使它们再次出现的概率降低。

风险：过强的重复惩罚会伤害正常表达。代码、数学公式、列表编号、专有名词和引用中本来就需要重复 token。

面试表达：重复惩罚可以缓解复读，但不是越强越好。它解决的是生成策略问题，不是模型能力本身的根治。

Presence Penalty

一句话定义：presence penalty 是只要 token 出现过，就对其再次出现施加惩罚。

核心特点：它关注“是否出现过”，不太关心出现次数。

适用场景：鼓励模型引入新话题、新词汇或减少重复短语。

风险：过强会让模型回避必要重复，导致术语不一致或表达绕远。

面试表达：presence penalty 更像鼓励新内容的开关，适合提升多样性，但会牺牲一致性。

Frequency Penalty

一句话定义：frequency penalty 是 token 出现次数越多，惩罚越强的机制。

与 presence penalty 区别：presence penalty 看是否出现过；frequency penalty 看出现了多少次。

适用场景：减少长文本中的高频复读，让模型避免重复同一表达。

风险：对代码、结构化文本、法律条款、数学符号这类需要重复的场景不友好。

面试表达：frequency penalty 用出现次数控制重复程度，比 presence penalty 更强调频率累积。

Stop Token

一句话定义：stop token 是触发生成结束的 token 或字符串。

常见形式：EOS token、聊天模板中的 assistant 结束标记、用户指定 stop words、工具调用结束标记。

为什么重要：停止条件配置错误会导致回答被截断、停不下来、泄漏角色标记、继续生成下一轮 user 内容，或输出多余模板符号。

工程注意：不同模型的 EOS、BOS、PAD、role token 可能不同。推理框架、tokenizer 和 chat template 必须一致。

面试表达：stop 配置是 LLM 推理的基础工程细节。很多生成异常不是模型本身差，而是 EOS 或 stop words 配置错。

EOS Token

一句话定义：EOS token 是 end-of-sequence token，用来表示序列结束。

训练作用：预训练或 SFT 中，EOS 告诉模型一个文档、回答或对话轮次在何处结束。

推理作用：当模型生成 EOS 时，系统通常停止继续生成。

常见问题：如果 SFT 数据没有正确加入 EOS，模型可能学不会停止；如果 EOS 被错误 mask 或 tokenizer 不一致，模型可能输出异常结束行为。

面试表达：EOS 不只是特殊符号，而是模型学习“何时结束”的训练信号。

Max New Tokens

一句话定义：max new tokens 是限制模型最多生成多少个新 token 的参数。

作用：防止模型无限生成，控制延迟、成本和输出长度。

与 context length 区别：context length 限制输入加输出的总 token 容量；max new tokens 只限制本次生成的新 token 数量。

风险：设太小会截断答案，设太大会增加延迟和成本，也可能让模型生成冗余内容。

面试表达：max new tokens 是推理成本和答案完整性的显式控制参数，不等同于模型上下文窗口。

Best-of-N

一句话定义：best-of-N 是生成 N 个候选答案，再用规则、reward model、评审模型或人工方式选择最佳答案的方法。

为什么有效：采样可以探索不同解答路径，reranker 可以从多个候选中选出质量更高的答案。

适用场景：数学推理、代码生成、复杂问答、创意生成和高价值请求。

成本：推理成本约随候选数增加。N 越大，质量潜力越高，但延迟和算力成本也越高。

风险：如果选择器本身有偏差，best-of-N 会放大选择器偏好；如果 reward model 有漏洞，也可能引入 reward hacking。

面试表达：best-of-N 用计算换质量，本质是“多生成再筛选”。它适合高价值任务，不适合所有请求默认开启。

Self-Consistency

一句话定义：self-consistency 是对推理问题采样多个解题路径，再选择多数一致答案或最可信答案的方法。

为什么有效：复杂推理中，单条链路可能偶然出错。多次采样可以让模型探索不同路径，如果多个路径得到同一答案，该答案更可能可靠。

适用场景：数学题、逻辑题、多跳问答、程序推理和需要中间步骤的问题。

局限：如果模型系统性误解题意，多数投票也会一致错误；如果答案形式复杂，聚合和判等也不容易。

面试表达：self-consistency 用多路径采样提升推理稳健性，但代价是更多推理成本，并且需要可靠的答案聚合方式。

Log Probability

一句话定义：log probability 是 token 或序列概率的对数形式。

为什么使用：多个 token 的概率相乘容易数值下溢，取 log 后可以把乘法变成加法，便于比较序列分数。

序列分数：一个回答的 log probability 通常是各 token log probability 的和，必要时会做长度归一化。

用途：候选排序、困惑度计算、DPO loss、beam search、置信度分析和拒答策略。

注意：高 log probability 不等于答案事实正确。它只表示模型认为该序列在当前上下文中概率高。

面试表达：logprob 是模型自评分数的重要接口，但它衡量的是模型概率，不是客观真实性。

Token Probability

一句话定义：token probability 是模型在当前上下文下对某个下一个 token 分配的概率。

来源：logits 经 softmax 之后得到概率分布。

解释限制：token 概率是局部的下一个 token 置信度，不等同于整段回答置信度。一个答案可能每个 token 都很流畅，但事实仍然错误。

工程用途：可视化模型不确定性、做 constrained decoding、构建分类器、分析模型偏好和调试异常输出。

面试表达：token probability 能解释模型下一步倾向，但不能直接当作事实置信度。

KV Cache

一句话定义：KV Cache 是缓存自回归推理中每一层历史 token 的 Key 和 Value，避免每生成一个 token 都重复计算全部历史注意力。

为什么需要：没有 KV Cache 时，生成第 t 个 token 需要重新计算前面所有 token 的 K/V，成本很高。有了 cache，新 token 只需要计算自己的 Q/K/V，并与历史 K/V 做 attention。

缓存内容：每一层 attention 的 key 和 value，通常形状与 batch、head 数、序列长度、head dim 有关。序列越长、层数越多、batch 越大，KV Cache 显存占用越高。

收益：显著加速 decode 阶段，让每步生成不必重复跑完整 prompt。

代价：占用大量显存，并带来显存带宽访问压力。长上下文和高并发场景中，KV Cache 往往是推理系统的关键瓶颈。

面试表达：KV Cache 用空间换时间，是 LLM 自回归推理加速的基础。它加速 decode，但也让显存管理变得重要。

Prefill

一句话定义：prefill 是推理时处理完整 prompt 并建立初始 KV Cache 的阶段。

做什么：模型并行处理输入 prompt 的所有 token，计算每层 hidden states，并为这些 prompt token 写入 KV Cache。

特点：prefill 能在序列维度并行，因此算力利用率相对高；长 prompt 会显著增加 prefill 成本。

影响指标：prefill 主要影响首 token 延迟，也就是 TTFT。用户输入越长、检索文档越多、system prompt 越复杂，首 token 通常越慢。

面试表达：prefill 是“读 prompt”的阶段，成本主要和输入长度相关，决定用户多久看到第一个 token。

Decode

一句话定义：decode 是在 KV Cache 基础上逐 token 生成输出的阶段。

做什么：每一步只输入最新 token，读取历史 KV Cache，计算下一个 token 分布，选出 token 后更新 cache。

特点：输出 token 之间串行依赖，无法像训练一样在生成长度维度完全并行。decode 通常更受显存带宽和 KV Cache 访问影响。

影响指标：decode 主要影响每 token 延迟和总生成时间。输出越长，总 decode 成本越高。

面试表达：decode 是“写答案”的阶段，成本主要和输出长度、batch、KV Cache 和模型大小相关。

Time to First Token / TTFT

一句话定义：TTFT 是从请求进入系统到生成第一个 token 的时间。

组成：排队时间、调度时间、tokenization、prompt 构造、prefill 计算和首步 decode。

为什么重要：TTFT 决定用户是否感觉系统“反应快”。即使总生成时间较长，只要首 token 快，流式输出体验也会更好。

优化方向：减少 prompt 长度、缓存公共前缀、优化调度、降低排队、提升 prefill 吞吐、使用更快模型或更好硬件。

面试表达：TTFT 主要受输入侧和 prefill 影响，是交互体验中的关键首响指标。

Time per Output Token / TPOT

一句话定义：TPOT 是 decode 阶段平均生成每个输出 token 所需时间。

影响因素：模型大小、batch size、KV Cache 长度、显存带宽、并发数、解码策略和推理框架实现。

与吞吐的关系：系统可能通过 batching 提高总体吞吐，但单请求 TPOT 可能上升。线上服务经常需要在吞吐和延迟之间平衡。

面试表达：TPOT 衡量生成速度。长回答场景下，TPOT 往往比 TTFT 更决定总耗时。

Speculative Decoding

一句话定义：speculative decoding 是用小模型或草稿模型先快速生成多个候选 token，再由大模型并行验证，从而加速自回归推理的方法。

核心动机：大模型逐 token decode 成本高，但很多 token 较容易预测。让小模型先猜一段，大模型一次验证多个 token，可以减少大模型调用轮数。

基本流程：draft model 生成若干候选 token；target model 对这些 token 进行验证；通过接受-拒绝机制保留被大模型认可的 token；被拒绝处再由大模型采样或继续生成。

关键要求：最终采样分布应尽量保持与原 target model 一致，不能为了速度明显改变输出质量。

适用条件：draft model 足够快，并且与 target model 分布足够接近。若草稿模型猜得很差，接受率低，加速收益会下降。

面试表达：speculative decoding 用“小模型猜、大模型验”减少大模型串行 decode 次数，是推理加速方法，不是训练方法。

本章小结

本章围绕 LLM 推理和解码展开。推理的主线是：prompt 经 prefill 建立 KV Cache，随后模型在 decode 阶段逐 token 生成；decoding 策略决定每一步如何从概率分布中选择 token。

高频结论如下：

1. LLM 推理通常是自回归逐 token 生成，训练可并行，推理输出长度维度串行。
2. greedy 稳定但缺少探索，sampling 多样但风险更高。
3. temperature 控制概率分布尖锐程度，top-k 控制候选数量，top-p 控制累计概率质量。
4. beam search 适合目标明确的序列任务，但开放式对话中不一定好。
5. repetition、presence、frequency penalty 可以缓解重复，但过强会伤害正常表达。
6. EOS、stop token 和 max new tokens 决定生成何时停止，是常见工程坑。
7. best-of-N 和 self-consistency 用更多计算换更高质量或更稳健的答案。
8. KV Cache 用空间换时间，是 decode 加速基础，但会增加显存压力。
9. prefill 主要影响 TTFT，decode 主要影响 TPOT 和总生成时间。
10. speculative decoding 用草稿模型和目标模型协同，在尽量保持输出分布的前提下加速推理。

下一章，我们进入推理优化与部署。

I. 推理优化与部署

条目: LLM Serving、Throughput、Latency、TTFT、TPOT、QPS、Batching、Dynamic Batching、Continuous Batching、Request Scheduling、KV Cache Management、PagedAttention、Prompt Cache、Prefix Cache、FlashAttention、IO-Aware Optimization、Tiling / Blocking、Online Softmax、HBM、SRAM、Fused Kernel、Memory Bandwidth Bottleneck、Quantization、FP32、FP16、BF16、FP8、INT8、INT4、Weight-Only Quantization、Post-Training Quantization / PTQ、Quantization-Aware Training / QAT、GPTQ、AWQ、Calibration Data、Distillation、TensorRT-LLM、vLLM、Triton Inference Server、Model Parallel Inference、CPU Offloading、Inference Performance Incident、TTFT Regression、TPOT Regression、KV Pressure、Prompt Cost Drift、Cache Effectiveness、Serving Gate。

第二轮推理平台总览补充条目: Inference Platform Overview Audit、Request Lifecycle Coverage、SLO Metric Contract、Model Registry Readiness、Model Router Policy Fit、Runtime Prefill Decode Fit、Continuous Batching Readiness、KV Cache Capacity Governance、Autoscaling Warm Pool Fit、Release Rollback Control、Rate Limit Degrade Protection、Safety Governance Fit、Cost Attribution Optimization、Observability Trace Coverage、Multi Model Governance、Developer API Self Service、Inference Platform Gate。

第二十四册推理框架总览补充条目: LLM Serving Engine Overview Audit、Serving Engine Request Lifecycle、Prefill Decode Phase Contract、KV Cache Footprint Estimate、Scheduler Batching Readiness、Streaming State Management、Engine Metrics Observability、Runtime Boundary Clarity、Serving Engine Gate。

第二十四册最小推理框架补充条目: From Scratch Mini Serving Engine、Naive Generate Loop、Toy Tokenizer、Toy Sampler、Request State Machine、Minimal Scheduler、Token Streaming Trace、KV Cleanup Gate、Minimal Engine Gate。

第二十四册请求生命周期补充条目: Request Lifecycle Audit、Mini Request Lifecycle Engine、Request Lifecycle Object、Finish Reason Coverage、Abort Timeout Cleanup、Lifecycle Metrics Trace、Request Lifecycle Gate。

第二十四册阶段资源画像补充条目: Prefill Decode KV Streaming Audit、Mini Prefill Decode KV Streaming Audit、Prefill Phase Accounting、Decode Phase Accounting、KV Pressure Estimate、Streaming Backpressure Signal、Phase Gate。

第二十四册指标成本补充条目: Serving Metric Cost Audit、Mini Serving Metric Cost Audit、TTFT SLO Gate、TPOT SLO Gate、Token Throughput Window、Active Sequence Capacity、Cost Per 1k Tokens、Metric Cost Gate。

第二十四册边界审计补充条目: Serving Boundary Audit、Mini Serving Boundary Audit、Engine Platform Infra Boundary、Interface Contract Coverage、Metric Handoff Coverage、Incident Layer Routing、Serving Boundary Gate。

第二十四册最小生成循环补充条目: Minimal Generate Loop Audit、Toy Generate Loop、Tokenizer Contract、Model Wrapper Contract、Last Logits Selection、Naive Recompute Work、Minimal Generate Gate。

第二十四册采样策略补充条目: Sampling Strategy Audit、Mini Sampling Strategy Audit、Stable Softmax、Temperature Softmax、Top-k Candidate Filter、Top-p Nucleus Filter、Seeded Multinomial Sampling、Sampling Gate。

第二十四册最小 KV Cache 补充条目: Minimal KV Cache Audit、Toy KV Cache Decode、Past Key Values Contract、Prefill Cache Build、Decode Cache Append、KV Cache Equivalence Check、KV Cache Memory Formula、KV Cache Gate。

第二十四册 batched prefill 补充条目: Batched Prefill Audit、Toy Batched Prefill、Padding Side Policy、Attention Mask Contract、Last Real Token Logits、Padding Waste Ratio、Prefill Token Budget、Batched Prefill Gate。

第二十四册 batched decode 补充条目: Batched Decode Audit、Toy Batched Decode、Finished Mask、Active Batch Compaction、Batch Cache Alignment、Decode Position Tracking、Decode Row Savings、Batched Decode Gate。

第二十四册简单 scheduler 补充条目: Simple Scheduler Audit、Toy Request Scheduler、Waiting Queue、Running Set、Scheduler Admission、Max Active Sequences、Scheduler Token Budget、Queue Wait Metric、TTFT Steps、Scheduler Trace、Simple Scheduler Gate。

第二十四册 token streaming 补充条目: Token Streaming Audit、Toy Text Streamer、Streaming Event Contract、Incremental Detokenization、First Token Chunk、Finish Event Contract、Stop Sequence Buffer、Client Cancellation Cleanup、Streaming Backpressure、Streaming Gate。

第二十四册最小 HTTP API 补充条目: Minimal HTTP API Audit、Toy HTTP API Server、Generate Request Contract、Request Validation Gate、Queue Admission Control、SSE Frame Format、Sync Generate Response、Streaming Generate Response、HTTP Cancellation Cleanup、HTTP API Gate。

第二十四册压测指标补充条目: Serving Benchmark Audit、Toy Serving Benchmark、TTFT Metric、TPOT Metric、E2E Latency Metric、Token Throughput Report、Queue Length Percentile、Active Request Percentile、KV Memory Peak、Benchmark Bottleneck Classification、Benchmark Gate。

第二十四册 vLLM 动机补充条目: vLLM Motivation Audit、Toy vLLM Motivation、Naive KV Reservation、Paged KV Block Allocation、KV Waste Ratio、Logical Physical Block Mapping、Block Reuse After Cleanup、Static Decode Rows、Continuous Decode Rows、vLLM Motivation Gate。

第二十四册 PagedAttention 核心补充条目: PagedAttention Core Audit、Toy Paged KV Cache、Block Table Address Translation、Logical Block、Physical KV Block、Block Size Tradeoff、Internal Block Waste、Prefix Block Ref Count、Freed Block Reuse、PagedAttention Gate。

第二十四册 KV Block Manager 补充条目: KV Block Manager Audit、Toy KV Block Manager、Block Free List、Block Reference Count、Prefill Block Admission、Decode Block Extension、Block Allocation Failure、Block Manager Metrics、Shared Prefix Cleanup、Block Manager Gate。

第二十四册 Continuous Batching 补充条目: Continuous Batching Audit、Toy Continuous Batcher、Iteration Level Scheduling、Dynamic Request Join、Dynamic Request Exit、Decode First Scheduling、Continuous Token Budget、Scheduler KV Budget、Deferred Scheduling Reason、Static Batch Hole、Continuous Batching Gate。

第二十四册 vLLM 请求调度流程补充条目: vLLM Scheduler Flow Audit、Toy Request Flow Engine、Engine Request Contract、Request State Machine Trace、Scheduler Output Metadata、Model Runner Execution Metadata、Slot Mapping Metadata、Output Processor Finish Gate、Abort Cleanup Path、Request Flow Metrics、Request Flow Gate。

第二十四册 vLLM Memory Management 补充条目: vLLM Memory Management Audit、Toy KV Memory Manager、GPU Memory Breakdown、KV Page Size、Prefix Cache Hash、Cached Free Block、Prefix Cache Hit Rate、LRU Cache Eviction、Preemption Recompute Path、Hybrid KV Cache Group、Memory Management Gate。

第二十四册 vLLM Worker / Executor 架构补充条目: vLLM Executor Architecture Audit、Toy Executor Architecture、Engine Core Dispatch Loop、Executor Dispatch Layer、Worker Rank Mapping、Model Runner Metadata Gate、Tensor Parallel Merge、CPU Process Budget、Worker Failure Recovery、Executor Architecture Gate。

第二十四册 Prefix Caching 补充条目: Prefix Caching Audit、Toy Prefix Cache Manager、Full Block Prefix Reuse、Parent Hash Chain、Extra Hash Isolation、Cache Salt Isolation、Cached Free Touch、Prefix Saved Prefill Tokens、Prefix Cache Eviction、Prefix Cache Gate。

第二十四册 Serving 并行补充条目: Serving Parallelism Audit、Toy Serving Parallel Auditor、Tensor Parallel Topology、Pipeline Parallel Bubble、Data Parallel Replica、Expert Parallel Placement、Worker Group Size、Per Rank Memory Fit、Cross Node TP Risk、Prefix Cache Locality、Parallel Serving Gate。

第二十四册 vLLM 性能调优补充条目: vLLM Performance Tuning Audit、Toy vLLM Tuning Auditor、TTFT Bottleneck Diagnosis、TPOT Bottleneck Diagnosis、KV Pressure Tuning、Preemption Rate、Prefix Cache Effectiveness、Config Trade-off Plan、Rollback Guard、vLLM Tuning Gate。

第二十四册 SGLang 动机补充条目: SGLang Motivation Audit、Toy SGLang Motivation Audit、Complex LLM Program、Frontend Language、RadixAttention Motivation、Radix Prefix Reuse、Structured Decoding

Retry Saving、SGLang Motivation Gate。

第二十四册 SGLang Runtime 补充条目：SGLang Runtime Audit、Toy SGLang Runtime Audit、Runtime Entrypoint Unification、Request State Contract、Radix Prefix Lookup、KV Memory Pool Budget、Prefill Decode Scheduler、Grammar Mask Step、Streaming Event Trace、SGLang Runtime Gate。

第二十四册 RadixAttention 补充条目：RadixAttention Prefix Sharing Audit、Toy RadixAttention Cache、Compressed Prefix Tree、Longest Prefix Match、Radix Tree Split、Page Aligned Prefix Hit、Radix KV Ref Count、Leaf LRU Eviction、Cache Aware Scheduling Cost、RadixAttention Gate。

第二十四册 SGLang Scheduler 补充条目：SGLang Scheduler Audit、Toy SGLang Scheduler、Decode First Scheduling、Cache Aware Admission、Scheduler Token Budget、Scheduler Sequence Budget、Scheduler KV Admission、Suffix Cost Scheduling、Scheduler Aging Fairness、Chunked Prefill Scheduling、Scheduler Abort Cleanup、SGLang Scheduler Gate。

第二十四册 Structured Generation 补充条目：Structured Generation Audit、Toy Constrained Decoder、Grammar State、Valid Token Mask、JSON Schema Constraint、Regex Choice Constraint、EBNF Grammar Constraint、Structural Tag Constraint、Empty Valid Token Set、Format Not Fact Gate、Structured Streaming Partial、Structured Generation Gate。

第二十四册 Speculative Decoding 补充条目：Speculative Decoding Audit、Toy Speculative Decoder、Draft Source、Target Verify Call、Accept Length、Draft Acceptance Rate、Target Call Reduction、Speculative Fallback Token、Speculative KV Cleanup、Adaptive Speculative Steps、Speculative Decoding Gate。

第二十四册 Agent Serving 补充条目：Agent Serving Audit、Toy Agent Serving Audit、Multi-turn Runtime State、Session Aware Routing、Agent Trajectory Prefix Sharing、Tool Parser Fit、Tool Schema Validation、Tool Result Backfill、Tool Wait GPU Release、Agent Round Budget、Agent Serving Gate。

第二十四册架构对比补充条目：Serving Architecture Comparison Audit、Toy Architecture Comparator、Common Serving Core、Paged Prefix Cache Evidence、Radix Program Reuse Evidence、Workload Fit Score、Runtime Workload Router、vLLM Not Replaced Gate、SGLang Program Fit Gate、Architecture Comparison Gate。

第二十四册 mini-sglang 源码补充条目：mini-sglang Source Path Audit、Toy Source Path Auditor、Runtime Module Map、Request Lifecycle Order、Source Resource Coverage、Source Experiment Coverage、Source Observable Signal、Radix Split Experiment、Abort Cleanup Experiment、Source Path Gate。

第二十四册 Prefill / Decode 资源画像补充条目：Prefill Decode Resource Profile Audit、Toy Prefill Decode Profiler、Request Token Profile、Run Prefill Tokens、Prefill Compute Profile、Prefill KV Write、Decode KV Read、TTFT TPOT Separation、Long Prefill Stall Risk、Decode-first Starvation Risk、PD KV Transfer Cost、Prefill Decode Profile Gate。

第二十四册 PD 分离动机补充条目：PD Disaggregation Motivation Audit、Toy PD Disaggregation Auditor、Unified Engine Interference、Prefill Interruption Evidence、Decode-first Starvation Evidence、Independent P/D Scaling、KV Transfer Cost Gate、Slow Transfer Counterexample、PD Motivation Gate。

第二十四册 PD 系统架构补充条目：PD Architecture Audit、Toy PD Architecture Auditor、PD Router、Prefill Worker Pool、Decode Worker Pool、KV Transfer Backend、Bootstrap Metadata、Decode Reservation、Cross Component State Machine、Transfer Failure Cleanup、Client Abort Cleanup、Model Version Compatibility、PD Architecture Gate。

第二十四册 KV Transfer 路由补充条目：KV Transfer Routing Audit、Toy KV Transfer Router、KV Metadata Compatibility、Decode Capacity Reservation、KV Routing Cost Score、Tenant Cache Isolation、Recompute Fallback Path、Transfer Failure Plan、KV Transfer Routing Gate。

第二十四册 Chunked / Disaggregated Prefill 补充条目：Chunked Disaggregated Prefill Audit、Toy Chunked Disagg Prefill Auditor、Prefill Chunk Count、Decode Interleaving、Prefill Token Budget、Position Continuity、Short Request Fairness、PD Transfer Pipeline、Backpressure Cleanup、Chunked Disagg Prefill Gate。

第二十四册多级 KV Cache 补充条目：Multi Level KV Cache Audit、Toy Multi Level KV Cache Auditor、KV Residency Level、Active GPU Block Protection、CPU KV Promote、Remote KV Fetch、Recompute

Fallback、KV Tenant Isolation、KV Demotion Policy、Residency Metrics、Multi Level KV Gate。

第二十四册跨节点网络补充条目：Cross Node Network Audit、Toy Cross Node Network Auditor、Cross Node Link Profile、TP Cross Node Risk、Pipeline Activation Cost、PD KV Transfer Cost、Remote Recompute Decision、Topology Aware Routing、Transfer Backpressure、Control Data Plane Separation、Cross Node Network Gate。

第二十四册 PD 取舍补充条目：PD Tradeoff Audit、Toy PD Tradeoff Auditor、PD Benefit Cost Score、PD Positive Case、Short Request Anti Pattern、Decode Bottleneck Anti Pattern、Slow Transfer Anti Pattern、Ops Readiness Gate、PD Alternative Path、PD Tradeoff Gate。

第二十四册单机到 PD 迁移补充条目：Single Engine To PD Migration Audit、Toy PD Migration Auditor、Explicit Request Stage、Model Runner Interface Split、KV Metadata Contract、Prefill Decode Scheduler Split、PD Router State Machine、Same Node PD Prototype、Cleanup Cancel Path、Observability Fallback Readiness、PD Migration Gate。

第二十四册 nano-vLLM 源码学习补充条目：nano-vLLM Source Learning Audit、Toy nano-vLLM Source Auditor、Source Module Map、Generate Trace Path、Engine Step Trace、Sequence State Audit、Scheduler Decision Trace、KV Block Lifecycle、Model Runner Batch Contract、Source Experiment Signal、nano-vLLM Source Gate。

第二十四册 tiny-LLM 学习路线补充条目：tiny-LLM Learning Audit、Toy tiny-LLM Learning Auditor、Operator To Model Path、Attention Shape Trace、RoPE Position Alignment、GQA KV Head Audit、Generate Sampling Loop、KV Cache Equivalence、Serving Optimization Ladder、Chunk Position Continuity、tiny-LLM Learning Gate。

第二十四册 mini-sglang 学习路线补充条目：mini-sglang Learning Audit、Toy mini-sglang Learning Auditor、SGLang Runtime Capability、Radix Cache Runtime Evidence、Chunked Prefill Ablation、Overlap Scheduling Ablation、Online Serving Trace、Structured Tool Boundary、Abort Cleanup Check、Production Gap Map、mini-sglang Learning Gate。

第二十四册教学项目核心模块抽象补充条目：Teaching Project Core Module Audit、Toy Core Module Auditor、Core Module Boundary、Request State Ownership、Scheduler ModelRunner Boundary、KV Manager Ownership、BatchBuilder Metadata Contract、Output Processor Cleanup Boundary、Metrics Observability Contract、Replaceable Upgrade Path、Core Module Gate。

第二十四册 Naive Scheduler 到 Continuous Batching 升级补充条目：Naive Scheduler To Continuous Batching Audit、Toy Scheduler Upgrade Auditor、Request Level Batch Lock、Iteration Boundary Scheduling、Dynamic Request Admission、Dynamic Request Exit、Decode First Upgrade、Bounded Prefill Budget、KV Capacity Admission、Scheduler Upgrade Gate。

第二十四册 Paged KV Cache 升级补充条目：Paged KV Cache Upgrade Audit、Toy Paged KV Upgrade Auditor、List KV Private Cache、Global KV Block Pool、Request Block Table、Allocate Until Contract、Paged Slot Mapping、Decode Block Extension Gate、Idempotent KV Free、Double Free Guard、Paged KV Upgrade Gate。

第二十四册 Prefix / Prompt Cache 升级补充条目：Prefix Prompt Cache Upgrade Audit、Toy Prefix Prompt Cache Auditor、Prompt Prefix Cache Boundary、Full Block Cache Reuse、Parent Hash Cache Chain、Extra Hash Cache Isolation、Suffix Prefill Start Position、Cached Ref Count Lifecycle、Full Prompt Hit Fallback、Prefix Prompt Cache Upgrade Gate。

第二十四册 Preemption / Recompute / Swap 升级补充条目：Preemption Recompute Swap Upgrade Audit、Toy Preemption Upgrade Auditor、KV Pressure Trigger、Cached Block Eviction Before Preemption、Victim Selection Policy、Recompute Context Preservation、Recompute Resume Path、Swap Failure Rollback、Swap In State Restore、Preemption Upgrade Gate。

第二十四册统一调度循环补充条目：Unified Scheduler Loop Audit、Toy Unified Scheduler Loop Auditor、Engine Step Order、Prefix Lookup Once、Decode First Commit、Suffix Prefill Plan、KV Budget Commit、Memory Pressure Order、Batch Metadata Invariant、Output State Update、Unified Scheduler Loop Gate。

第二十四册 Benchmark Framework 补充条目：Serving Benchmark Framework Audit、Toy Serving Benchmark Framework、Benchmark Workload Coverage、Benchmark Experiment Fingerprint、Request Trace

Summary、Engine Step Trace Summary、SLO Regression Gate、Throughput Regression Check、KV Cleanup Check、Prefix Effect Check、Preemption Risk Check、Benchmark Decision Gate、Serving Benchmark Framework Gate。

第二十四册异步 Serving 架构补充条目：Async Serving Architecture Audit、Toy Async Serving Runtime、Async Boundary Design、API Admission Boundary、Tokenizer Worker Queue、Engine Input Queue、Bounded Engine Drain、Engine Output Queue、Per Client Stream Queue、Stream Queue Backpressure、Cancel Signal Queue、Engine State Ownership、Idempotent Async Cleanup、Async Serving Architecture Gate。

第二十四册多 Worker Router 补充条目：Multi Worker Router Audit、Toy Multi Worker Router、Router Worker Capability Filter、Request Cost Block Estimate、Load Aware Worker Score、Sticky Route Fallback、Global Router Admission、Worker Heartbeat Timeout、Safe Retry Boundary、Request Worker Map、Worker Selection Imbalance、Multi Worker Router Gate。

第二十四册分布式并行 KV 补充条目：Distributed Parallel KV Audit、Toy Distributed Parallel KV Auditor、Parallel Group Boundary、TP Head Shard Assignment、TP Block Table Consistency、PP Layer Ownership、Distributed KV Byte Accounting、Collective Communication Cost、Pipeline Bubble Ratio、KV Migration Decision、Cross Rank Cleanup、Distributed Parallel KV Gate。

第二十四册 OpenAI-compatible API 补充条目：OpenAI Compatible API Audit、Toy OpenAI Compatible API、API Compatibility Contract、Chat Request Schema Validation、SSE Stream Frame Contract、OpenAI Style Error Shape、Bearer Auth Gate、Model Permission Gate、Token Rate Limit Gate、Concurrency Limit Gate、Usage Accounting Check、API Privacy Log Check、OpenAI Compatible API Gate。

第二十四册生产部署补充条目：Production Deployment Audit、Toy Production Deployment Auditor、Runtime Compatibility Matrix、Model Artifact Manifest、Startup Readiness Gate、Warmup Probe Check、Worker Registration Gate、Drain Completion Gate、Rolling Update Capacity Gate、Canary Traffic Split、Canary Regression Gate、Rollback Readiness Gate、Production Deployment Gate。

第二十四册容量 SLO 故障演练补充条目：Capacity SLO Fault Drill Audit、Toy Capacity SLO Fault Drill Auditor、Workload Token Profile、Benchmark Stable Capacity、GPU Count Estimate、KV Concurrency Limit、Cost Attribution Model、SLO Error Budget、Admission Overload Policy、Fault Drill Scenario、Runbook Coverage Check、Capacity SLO Fault Drill Gate。

第二十四册推理框架面试准备度补充条目：Inference Engine Interview Readiness Audit、Toy Inference Engine Interview Auditor、Interview Question Rubric、Concept Coverage Score、Request Lifecycle Answer、KV Cache Answer Evidence、Scheduler Answer Evidence、Serving Metrics Coverage、Ordered Debug Path、Production Governance Answer、Project Evidence Portfolio、Tradeoff Coverage、Inference Engine Interview Gate。

第二轮推理调度补充条目：Inference Scheduling Audit、Request Arrival Profile、Continuous Batching Core、Token Budget Policy、Prefill Decode Balance、Paged KV Block Management、Scheduler KV Admission、Long Short Isolation、Tenant Priority Fairness、Cancellation Cleanup、Scheduler Observability Metrics、Scheduler Gate。

第二轮模型路由补充条目：Model Routing Audit、Request Intent Profile、Model Capability Profile、Permission Candidate Filter、Cost Budget Policy、Latency SLO Policy、Quality Safety Policy、Fallback Chain Governance、Canary Routing Stability、Realtime Health Load、Route Trace Coverage、Degrade Policy Control、Route Config Versioning、Model Routing Gate。

第二轮缓存体系补充条目：Inference Cache System Audit、Request Cache Profile、Cache Layer Boundary、Key Version Fingerprint、Prompt Prefix Reuse、KV Lifecycle Management、Semantic Similarity Guard、Result Cache Determinism、Tenant Permission Isolation、TTL Staleness Control、Eviction Quota Policy、Streaming Cache Policy、Cache Observability Metrics、Cost Latency Savings、Cache Governance Gate。

第二轮自动扩缩容补充条目：Inference Autoscaling Audit、Traffic Token Profile、SLO Latency Contract、Queue Backlog Signal、GPU KV Capacity Signal、Cold Start Lead Time、Warm Pool Readiness、Multi Metric Recommendation、Scale Up Down Policy、Draining Streaming Safety、Router Admission Coupling、Tenant Quota Priority、Cost Budget Guard、Autoscaling Trace Coverage、Autoscaling Gate。

第二轮保护策略补充条目：Inference Protection Policy Audit、Request Token Rate Limit、Concurrency Quota Control、Admission Capacity Guard、Circuit Breaker State Machine、Retry Error Classification、Retry Budget Bound、Idempotency Stage Guard、Timeout Budget Allocation、Degradation Policy Control、Priority Tenant Fairness、Streaming Cancellation Cleanup、Protection Trace Coverage、Protection Gate。

第二轮发布治理补充条目：Inference Release Governance Audit、Release Bundle Trace、Model Artifact Integrity、Tokenizer Prompt Runtime Binding、Offline Eval Quality Gate、Safety Eval Gate、Contract Test Compatibility、Stable Bucket Assignment、Canary Ramp Control、AB Experiment Design、Sample Ratio Health、Guardrail Metric Control、Rollback Readiness、Release Audit Trace、Post Release Monitoring、Release Governance Gate。

第二轮系统设计补充条目：Inference Platform System Design Audit、Requirement Clarification、SLO Token Contract、Model Registry Release、Router Policy Fit、Runtime Prefill Decode、KV Scheduler Capacity、Cache Isolation Strategy、Autoscaling Capacity Plan、Protection Policy Design、Release Governance Design、Tenant Security Governance、Observability Trace Readiness、Cost Capacity Governance、Tradeoff Boundary Reasoning、Inference Platform Design Gate。

第二轮推理故障定位补充条目：Inference Fault Diagnosis Audit、Inference Fault Evidence Coverage、Request Scope Slice、Trace Stage Coverage、TTFT Decomposition、TPOT Decode Health、Tail Latency Attribution、Throughput Token Capacity、Error Taxonomy Timeout Stage、KV Cache Pressure Guard、Model Artifact Load Readiness、Streaming Reliability、Cache Hit Key Governance、Route Trace Correctness、Tool Dependency Trace、Quality Release Diff Readiness、Inference Fault Diagnosis Gate。

LLM Serving

一句话定义：LLM serving 是把训练好的大模型部署成可被线上请求调用的推理服务。

它解决什么问题：训练完成的模型只是参数文件。要让用户稳定使用，还需要 tokenizer、chat template、请求队列、batch 调度、KV Cache 管理、解码策略、流式输出、监报告警、限流、容灾和成本控制。

与普通深度学习 serving 的区别：LLM 请求长度差异大，输出长度不可预知，推理过程分为 prefill 和 decode 两个阶段，decode 还存在逐 token 串行依赖。因此，LLM serving 的核心瓶颈经常不是一次 forward，而是调度、KV Cache 和显存带宽。

常见目标：低延迟、高吞吐、稳定质量、低成本、高可用、可观测、可扩展。

面试表达：LLM serving 不是简单把模型 `model.generate()` 包成接口，而是围绕长序列、动态输出和 KV Cache 做系统级优化。

LLM Serving Engine Overview Audit

一句话定义：LLM Serving Engine Overview Audit 是检查一个推理框架是否真正覆盖请求生命周期、prefill / decode、KV cache、scheduler、streaming、metrics、runtime 边界和成本容量的总览审计方法。

它解决什么问题：很多回答会把 serving engine 说成 `model.generate()` 的 HTTP 包装，或者只背 vLLM、SGLang、TGI、Triton 的名字。总览审计要求把请求从 parse、tokenize、queue、prefill、decode、stream 到 finish / cleanup 的状态链路讲清，并能写出 TTFT、TPOT、KV cache 显存、token 吞吐和单位 token 成本。

面试表达：一个合格的 LLM serving engine 不是“能生成文本”，而是能在高并发下管理请求、token、KV、batch、streaming、错误、取消、观测和成本。

From Scratch Mini Serving Engine

一句话定义：From Scratch Mini Serving Engine 是用最小代码把 naive generate loop 拆成 tokenizer、request state、prefill、decode、scheduler、streamer、KV cleanup 和 metrics 的教学版推理引擎。

它解决什么问题：单请求 `generate()` 把请求状态、prefill / decode、KV cache、采样和输出都藏在一个黑盒里。最小推理框架把这些状态显式化，让读者先理解“为什么需要 serving engine”，再学习 vLLM、SGLang 等复杂实现。

常见门禁：request state machine 正确流转，streaming chunks 可见，prefill / decode 指标可统计，请求结束后 KV cache 被释放，事件 trace 能复盘每轮调度和生成。

面试表达：从零实现推理框架时，不要一开始追求高性能；先把单请求 loop 拆成可调度、可观测、可取消、可清理的最小系统闭环。

Request Lifecycle Audit

一句话定义：Request Lifecycle Audit 是检查一次 LLM 请求从接入、校验、tokenization、排队、调度、prefill、decode、streaming 到 finish / abort / timeout / cleanup 是否形成闭环的审计方法。

它解决什么问题：很多推理框架回答只背 API Server、Scheduler、Worker 和 KV Cache Manager，却说不清一个请求在状态机里怎么流转、什么时候占用 KV、什么时候释放资源、异常路径如何返回 finish reason。生命周期审计要求正常结束、客户端取消、排队超时和执行失败都进入统一清理路径。

常见门禁：request object 字段完整，WAITING / PREFILLING / DECODING / FINISHED / ABORTED / TIMEOUT / FAILED 状态可追踪，TTFT / TPOT / queue / prefill / decode 指标可拆分，streaming chunk 或 finish reason 可解释，请求结束后 KV cache 和 running slot 被释放，trace 能复盘关键事件。

面试表达：推理请求不是一次函数调用，而是带状态、资源、输出通道和异常收敛路径的生命周期对象；只有 happy path 和 exception path 都能清理资源，才算 serving engine 可靠。

Prefill Decode KV Streaming Audit

一句话定义：Prefill Decode KV Streaming Audit 是把一次 LLM 推理拆成 prefill、decode、KV cache 和 token streaming 四个阶段，并用 TTFT、TPOT、KV pressure、stream backlog 和 cleanup 共同验收的教学审计方法。

它解决什么问题：很多人知道 prefill、decode、KV cache 和 streaming 的名字，却不能解释它们如何共同影响用户体验和系统容量。这个审计把 prompt token、output token、prefill steps、decode steps、KV MiB、stream chunks、backpressure 和 finish reason 放到一张表里，说明长 prompt 拉高 TTFT，长输出增加 decode steps，长上下文和并发推高 KV pressure，streaming 还要处理客户端消费速度和资源释放。

常见门禁：prefill accounting 不丢 prompt token，decode accounting 和输出 token 对齐，KV cache 显存估算包含层数、KV head、head dim 和 dtype bytes，streaming chunk 数和输出 token 对齐，客户端慢读或断开能触发 backpressure / cleanup，最终 Phase Gate 不只看吞吐，还看 TTFT、TPOT、KV 和 finish reason。

面试表达：Prefill 决定首 token 等多久，Decode 决定后续 token 多快，KV Cache 决定高并发显存边界，Streaming 决定用户感知和取消清理；四者必须一起看，不能只用 QPS 或 GPU utilization 判断推理系统好坏。

Serving Metric Cost Audit

一句话定义：Serving Metric Cost Audit 是把 TTFT、TPOT、input / output tokens/s、active sequences、KV pressure、tail latency、timeout rate 和 cost per 1k tokens 放在同一张压测表里的指标成本审计方法。

它解决什么问题：传统 QPS 和平均延迟无法解释 LLM serving 的真实负载。两个请求 QPS 都是 1，但 input / output token、prefill、decode、KV cache 和 streaming 可能差几个数量级。指标成本审计要求压测报告同时给出 token 分布、阶段延迟、p95 / p99、KV 显存压力、有效吞吐和单位 token 成本。

常见门禁：TTFT p95 / p99 满足首响 SLO，TPOT p95 满足流式输出 SLO，input / output tokens/s 可解释，KV pressure 低于水位，active sequence capacity 有余量，timeout rate 不超标，cost per 1k tokens 在预算内，Metric Cost Gate 不允许只因吞吐好看就忽略长尾和成本。

面试表达：评估推理服务性能要从“请求数”升级到“token + 阶段 + 成本”视角；一个系统可能 tokens/s、KV 和成本都过线，但 TTFT p99 或 E2E p99 失败，仍然不能算用户体验达标。

Serving Boundary Audit

一句话定义：Serving Boundary Audit 是审计推理框架、推理平台和 AI Infra 三层职责、接口、指标交接、事故归层和成本容量证据是否清楚的方法。

它解决什么问题：很多系统设计回答会把 vLLM、API gateway、Kubernetes、GPU driver、灰度、限流、计费 and 监控混在一起说。边界审计要求先明确 engine 负责 token 执行路径，platform 负责服务治理控制面，AI Infra 负责 GPU、容器、调度、网络、存储和隔离；再检查 request API、metrics export、control drain、health probe、trace context 是否打通。

常见门禁：layer coverage 覆盖 engine / platform / infra 的核心职责，interface contract coverage 覆盖请求、指标、控制、健康检查和 trace，metric handoff coverage 覆盖 TTFT / TPOT / queue length / KV pressure / tenant quota / route version / pod pending / GPU health，incident layer routing 能把 TTFT、TPOT、成本、扩容和硬件异常排到正确层级，Serving Boundary Gate 不允许只会背工具名。

面试表达：推理平台不是“部署一个 vLLM Pod”，而是 engine、platform 和 infra 通过接口、指标和事件证据协作；边界越清楚，排障和优化越能走到正确 owner。

Minimal Generate Loop Audit

一句话定义：Minimal Generate Loop Audit 是审计最小 tokenizer、model wrapper、logits 选择、token 追加、停止条件和 trace 是否构成可解释生成闭环的方法。

它解决什么问题：直接调用 `model.generate()` 会把 encode、forward、last logits、token selection、append、stop、decode 和 cache 更新都藏起来。最小生成循环审计要求先用 toy tokenizer 和 toy model wrapper 跑通最小链路，让读者知道文本如何变成 token ids，模型如何输出词表 logits，greedy 如何选下一个 token，以及 naive loop 为什么会重复计算历史 token。

常见门禁：Tokenizer Contract 保证 encode / decode 类型清楚，Model Wrapper Contract 保证输入 token ids、输出最后位置 logits，Last Logits Selection 只用最后位置预测下一个 token，Stop Condition 覆盖 EOS 和最大生成长度，Naive Recompute Work 能量化重复计算，Minimal Generate Gate 要求 trace、metrics 和停止原因都可复盘。

面试表达：理解推理框架要先拆开最小 generate loop；否则后面讲 KV cache、batching、scheduler 和 streaming 时，只是在黑盒外面背名词。

Sampling Strategy Audit

一句话定义：Sampling Strategy Audit 是审计 greedy、temperature、top-k、top-p、seeded multinomial 和采样参数 trace 是否正确控制候选分布的方法。

它解决什么问题：很多回答会说 temperature 控制随机性、top-k / top-p 控制候选集合，但说不清 logits、softmax、过滤、归一化和采样顺序。采样策略审计要求能从 logits 出发，先做稳定 softmax 或 greedy，再按请求参数应用 temperature、top-k、top-p，最后从过滤后的概率分布中采样，并记录候选集合、entropy、seed 和 selected token。

常见门禁：Stable Softmax 要减 max 防溢出，Temperature Softmax 要能解释低温和高温的 entropy 差异，Top-k Candidate Filter 固定保留高分 token，Top-p Nucleus Filter 按累计概率动态保留候选，Seeded Multinomial Sampling 在简单请求路径可复现，Sampling Gate 不能让所有概率被过滤为空。

面试表达：sampling 不是“让模型更聪明”的按钮，而是对下一个 token 分布做可解释约束；它会影响质量、多样性、输出长度、停止命中、TPOT 和成本。

Minimal KV Cache Audit

一句话定义：Minimal KV Cache Audit 是审计单请求 prefill / decode 是否正确使用 past key values、追加 cache、保持输出等价并减少重复计算的教学方法。

它解决什么问题：很多人知道 KV Cache 能加速生成，但说不清 cache 里保存什么、prefill 和 decode 的输入有什么区别、为什么 decode 只传最新 token、cache 为什么会增加显存。最小 KV Cache 审计要求先用 toy

attention 或最小模型验证 “完整重算最后 token” 和 “缓存路径最后 token” 一致，再统计 cache length、重复投影减少量和 KV 显存估算。

常见门禁: Past Key Values Contract 要说明每层 key / value 的 shape 和生命周期; Prefill Cache Build 要用完整 prompt 建初始 cache; Decode Cache Append 要每步只计算新 token 并追加 K/V; KV Cache Equivalence Check 要验证缓存路径和完整重算路径输出一致或在可解释容差内; KV Cache Memory Formula 要包含 key/value 两份、层数、KV head、head dim、active tokens 和 dtype bytes; KV Cache Gate 要求正确性、工作量下降和显存估算都可复盘。

面试表达: KV Cache 不是减少模型权重显存，而是用额外显存换掉 decode 阶段的重复历史计算；单请求最小 cache 跑通后，才有资格继续讨论 batch decode、PagedAttention、prefix cache 和 KV 迁移。

Batched Prefill Audit

一句话定义: Batched Prefill Audit 是审计多个 prompt 是否能被正确 padding、mask、一次 prefill、按最后真实 token 取 logits 并建立 batch KV cache 的教学方法。

它解决什么问题: 从单请求走向多请求时，不能只把 prompts 放进 list。不同 prompt 长度需要 padding, padding 必须配 attention mask，首 token logits 必须来自每个请求最后一个真实 token；右 padding 下直接取 logits[:, -1, :] 会让短请求读到 PAD 位置。Batched Prefill Audit 要求同时验证左 / 右 padding、last real token gather、padding waste 和 token budget。

常见门禁: Padding Side Policy 要说明左 padding 和右 padding 的 logits 取法差异; Attention Mask Contract 要让 PAD 不参与有效上下文; Last Real Token Logits 要能用 mask 找到每个请求最后真实位置; Padding Waste Ratio 要量化 padded tokens 占比; Prefill Token Budget 要限制单轮真实 prompt tokens 或 padded tokens; Batched Prefill Gate 要求首 token 选择、mask、长度、KV cache 估算和 padding waste 都可复盘。

面试表达: batched prefill 不是 batch size 越大越好，而是在 GPU 并行、TTFT、padding waste、KV 显存和后续 decode batch 之间做取舍；先把最后真实 token logits 取对，再谈 continuous batching。

Batched Decode Audit

一句话定义: Batched Decode Audit 是审计多个已完成 prefill 的请求是否能在同一 decode step 中生成新 token，并保持 request state、finished mask、position、attention mask 和 KV cache 行顺序一致的教学方法。

它解决什么问题: batched decode 不是把 last token ids 拼成 tensor 就结束。每个请求有自己的输出、停止条件、prompt length、cache length 和 sampling 参数；finished 请求如果继续留在固定 batch 里，会浪费 decode 行；active compaction 虽然更高效，但必须同步选择和回写对应的 KV cache。

常见门禁: Finished Mask 要阻止已完成请求继续追加 token；Batch Cache Alignment 要维护 batch row -> request id -> KV cache slot 映射；Decode Position Tracking 要按请求自己的长度更新 position ids 和 attention mask length；Active Batch Compaction 要只对活跃请求构造 decode batch；Decode Row Savings 要量化 fixed batch 与 compact batch 的行数差异；Batched Decode Gate 要求输出、停止原因、cache length 和 row/cache alignment 都可复盘。

面试表达: batched decode 的核心不是一次 forward，而是让每一行 token、请求状态和 KV cache slot 永远对齐；这一步做对后，才能继续实现动态加入和退出的 continuous batching。

Simple Scheduler Audit

一句话定义: Simple Scheduler Audit 是审计教学版推理 scheduler 是否能维护 waiting queue、running set、finished set，并用 max active sequences 与 token budget 做最小准入控制的方法。

它解决什么问题: 很多人会把 scheduler 误解成队列 pop()。推理 scheduler 的输入不只是 waiting queue，还包括正在 decode 的请求、可用 running slot、prefill token budget、decode token 数、KV 压力和完成清理。Simple Scheduler Audit 要求最小系统能解释每轮谁 decode、谁 prefill、谁 deferred、为什么 deferred，以及这些决策如何影响 queue wait 和 TTFT。

常见门禁：Waiting Queue 保存尚未 prefill 的请求；Running Set 只保存仍需 decode 的请求；Scheduler Admission 同时检查可用 slot 和 token budget；Max Active Sequences 不超过配置；Queue Wait Metric 和 TTFT Steps 可按请求计算；Scheduler Trace 能复盘每轮 arrivals、prefill、decode、waiting、running 和 scheduled tokens；Simple Scheduler Gate 要求所有请求进入终态，预算不超限，deferred 可解释。

面试表达：简单 scheduler 的价值不是性能，而是把请求流、状态机、token budget 和指标闭环显式化；这一步做对后，才能继续讨论 continuous batching、chunked prefill、PagedAttention 和多租户调度。

Token Streaming Audit

一句话定义：Token Streaming Audit 是审计推理 engine 是否能把 prefill 首 token 和每轮 decode 结果转换成可消费的增量事件，并正确处理 finish、stop sequence、取消和 backpressure 的教学方法。

它解决什么问题：很多实现会把 streaming 简化成每个 token print 一下，但真实 serving 需要把模型执行和网络输出解耦。Token Streaming Audit 要求定义 Streaming Event Contract，把 delta、finish reason、usage 和 error 放入每请求 stream queue；用 Incremental Detokenization 避免单 token decode 边界问题；用 Stop Sequence Buffer 防止跨 token stop sequence 泄露；客户端取消和慢读都必须反馈给 engine，避免继续占用 GPU 与 KV cache。

常见门禁：First Token Chunk 要在 prefill 首 token 后发出；Finish Event Contract 要保证每个请求都有终态事件；Incremental Detokenization 要能重构完整文本；Stop Sequence Buffer 要隐藏 stop sequence；Client Cancellation Cleanup 要停止后续 decode 并记录 finish reason；Streaming Backpressure 要能检测慢客户端或队列积压；Streaming Gate 要求 delta、finish、取消、背压和 trace 都可复盘。

面试表达：Streaming 不是前端体验细节，而是请求生命周期的一部分；它把 TTFT、TPOT、事件队列、取消反馈和资源清理连在一起，任何一个环节缺失都会让用户体验或 GPU 资源治理失真。

Minimal HTTP API Audit

一句话定义：Minimal HTTP API Audit 是审计教学版 LLM serving engine 是否能通过最小 HTTP API 接收 JSON 请求、校验参数、提交到 engine、返回同步或流式结果，并处理过载与取消清理的方法。

它解决什么问题：很多项目把 `model.generate()` 包进 Web handler 就叫推理服务，但这会把协议层、请求状态、队列准入、streaming 事件和 KV cleanup 混在一起。最小 HTTP API 审计要求先定义 Generate Request Contract，把 prompt、max new tokens、sampling params 和 stream flag 校验清楚；再用 Queue Admission Control 在队列满时返回 429；同步路径返回 request id、text、finish reason 和 usage；流式路径用 SSE Frame Format 返回 delta 和 finish event；客户端断开必须触发 HTTP Cancellation Cleanup，通知 engine 停止 decode 并释放 KV。

常见门禁：Request Validation Gate 要区分 400 输入错误和 500 服务端错误；Queue Admission Control 要避免高峰期无限排队；Sync Generate Response 要包含终态和 usage；Streaming Generate Response 要遵守 text/event-stream、event:、data: 和空行分隔；HTTP Cancellation Cleanup 要把 abort、KV release 和 finish reason 收敛到同一清理路径；HTTP API Gate 要求 validation、admission、sync、stream 和 cancel 都可复盘。

面试表达：最小 HTTP API 的价值不是多一个 endpoint，而是把外部协议层和 engine 执行层解耦；handler 负责校验、准入、连接和响应，engine 负责调度、prefill、decode、streaming 和资源释放。

Serving Benchmark Audit

一句话定义：Serving Benchmark Audit 是用 TTFT、TPOT、E2E latency、tokens/s、queue、active requests、KV memory 和 cleanup 共同审计 LLM serving engine 性能瓶颈的方法。

它解决什么问题：LLM serving 的负载不是普通 QPS，prompt 长度、输出长度、prefill、decode、KV cache 和 streaming 都会改变成本。Serving Benchmark Audit 要求压测报告先说明模型、硬件、workload、并发和到达模式，再记录每请求 TTFT Metric、TPOT Metric、E2E Latency Metric、input / output Token Throughput Report、Queue Length Percentile、Active Request Percentile 和 KV Memory Peak。只给平均延迟或 QPS，会掩盖长 prompt、burst、queue 堆积和 decode 长尾。

常见门禁：TTFT Metric 要在客户端可见首 token 时记录；TPOT Metric 要说明是否从首 token 后开始算；Token Throughput Report 要分 input tokens/s 和 output tokens/s；Queue Length Percentile 和 Active Request Percentile 要能解释 TTFT 长尾；KV Memory Peak 要区分模型权重、KV cache、工作区和 runtime 开销；cleanup 后显存或 KV token 要回落；Benchmark Bottleneck Classification 要能把问题归到 queue / prefill、decode / streaming、KV capacity 或 workload；Benchmark Gate 要求 trace、latency、throughput、KV、cleanup 和归因都可复盘。

面试表达：推理压测不是跑一个 QPS 数字，而是证明系统在给定 workload 和硬件下，首响、后续 token、吞吐、显存和错误路径都可测、可解释、可定位。

vLLM Motivation Audit

一句话定义：vLLM Motivation Audit 是审计为什么 naive serving engine 需要升级到 vLLM 式 block-based KV cache 和 continuous batching 的教学方法。

它解决什么问题：很多人把 vLLM 简化成 “PagedAttention 更快”，但 vLLM 的动机来自系统瓶颈：Naive KV Reservation 为每个请求连续预留 prompt + max output，实际输出短时浪费；固定 batch decode 让短请求陪长请求跑空行；finished / cancelled 请求如果不能释放和复用 KV，显存容量会被吃掉。vLLM Motivation Audit 要求用 Paged KV Block Allocation 解释逻辑 token 到物理 block 的映射，用 KV Waste Ratio 量化连续预留与块内浪费差异，用 Static Decode Rows 与 Continuous Decode Rows 解释动态调度为什么减少无效 decode 行。

常见门禁：Naive KV Reservation 要能算出预留容量和实际使用容量；Paged KV Block Allocation 要能展示 block count 和 Logical Physical Block Mapping；Block Reuse After Cleanup 要证明 finished 请求释放的 block 可复用；Static Decode Rows 和 Continuous Decode Rows 要能量化固定 batch 与 iteration-level scheduling 的差距；vLLM Motivation Gate 要求 KV waste、block reuse、continuous batching、cleanup 和 metrics 都可复盘。

面试表达：vLLM 不是 “一个 attention kernel”，而是围绕 KV cache、block manager、scheduler、worker、streaming 和 metrics 组成的 serving engine；PagedAttention 是核心，但必须和 continuous batching 与请求生命周期一起讲。

PagedAttention Core Audit

一句话定义：PagedAttention Core Audit 是审计一个推理系统是否能把请求逻辑 KV cache 映射到非连续 physical KV blocks，并正确处理地址翻译、释放、复用和 prefix sharing 的方法。

它解决什么问题：连续 past_key_values 适合教学，但动态请求和高并发下容易遇到预留浪费、碎片和 batch row 重排困难。PagedAttention Core Audit 要求定义 Logical Block、Physical KV Block 和 Block Table Address Translation：token position 先算 logical block 与 offset，再通过 block table 找到 physical block。请求增长时按 block 分配；请求结束时释放不再被引用的 blocks；共享 prefix 时用 Prefix Block Ref Count 避免重复存储；Freed Block Reuse 证明非连续空闲块能继续服务新请求。

常见门禁：Block Table Address Translation 要能把 token position 映射到正确 physical block 和 offset；Block Size Tradeoff 要解释小 block 降低内部浪费但增加映射和 kernel 开销；Internal Block Waste 要能统计尾块空洞；Prefix Block Ref Count 要保证共享块不会被提前释放；Freed Block Reuse 要证明 cleanup 后 block 回到 free list；PagedAttention Gate 要求地址翻译、共享、释放、复用和浪费统计都可复盘。

面试表达：PagedAttention 不只是更快的 attention，而是 KV cache 的分页式内存管理；attention kernel 的特殊之处，是能在计算时根据 block table 读取非连续的物理 KV blocks。

KV Block Manager Audit

一句话定义：KV Block Manager Audit 是审计推理 engine 是否能把 physical KV blocks 当成可分配、可追踪、可回收资源来管理的方法。

它解决什么问题：PagedAttention 需要 block table 和 physical KV blocks，但分配、释放、复用和准入不是 attention kernel 自己完成的。KV Block Manager Audit 要求定义 Block Free List、Block Reference Count 和 Sequence block table：prefill 前用 Prefill Block Admission 判断 prompt 是否能放下；decode

时用 Decode Block Extension 在跨 block 边界时追加 block; 请求 finished / aborted / failed 时走 Shared Prefix Cleanup, 只在 ref count 归零时归还 block; free blocks 不足时记录 Block Allocation Failure, 而不是让 scheduler 调度后再 OOM。

常见门禁: Prefill Block Admission 要按 token 数换算所需 blocks; Decode Block Extension 要能在 block 满时追加 physical block; Block Reference Count 要正确处理 prefix sharing; Shared Prefix Cleanup 要释放非共享块并保留共享块; Block Manager Metrics 要至少包含 free blocks、used blocks、shared blocks、allocation failures 和 blocks per request; Block Manager Gate 要求 prefill、decode、ref count、free、admission 和 metrics 都可复盘。

面试表达: Block Manager 不是一个简单 free list, 而是 scheduler 和 PagedAttention 之间的 KV 资源控制面; 没有它, continuous batching、prefix sharing、cleanup 和可靠准入都会失真。

Continuous Batching Audit

一句话定义: Continuous Batching Audit 是审计推理 scheduler 是否能在每个模型 iteration 动态选择 prefill / decode 请求, 并在 token budget 与 KV block budget 下安全运行的方法。

它解决什么问题: 固定 batch 会让短请求陪长请求跑空行, 新请求也要等整批完成后才能进入 prefill。Continuous Batching Audit 要求用 Iteration Level Scheduling 把 request lifetime 和 execution batch lifetime 解耦; Dynamic Request Join 允许 waiting 请求在后续 iteration 进入; Dynamic Request Exit 允许 finished / cancelled 请求移出 running set; Decode First Scheduling 保障已有 streaming 请求的 TPOT; Continuous Token Budget 控制单轮 prefill + decode tokens; Scheduler KV Budget 要求 scheduler 在调度前查询 block manager, 避免能算但放不下。

常见门禁: Toy Continuous Batcher 要能输出每轮 arrived、decode、prefill、waiting、running、used tokens、free blocks 和 Deferred Scheduling Reason; Static Batch Hole 要量化固定 batch 空洞; finished / cancelled 请求必须触发 cleanup; Continuous Batching Gate 要求 dynamic join / exit、decode-first、token budget、KV budget、cleanup 和 metrics trace 都可复盘。

面试表达: Continuous batching 不是把 batch size 开大, 而是把调度粒度降到每个 iteration; 它必须和 PagedAttention、Block Manager、streaming、TTFT / TPOT 指标以及异常清理一起讲, 才是可上线的 serving scheduler。

vLLM Scheduler Flow Audit

一句话定义: vLLM Scheduler Flow Audit 是审计一个 vLLM-like engine 中请求从 API 输入、engine request、waiting queue、scheduler output、model runner metadata 到 output processor 和 cleanup 的完整链路是否闭环的方法。

它解决什么问题: 很多人把 vLLM 请求流程只讲成“进队列、调度、跑模型”, 但真实 serving engine 要维护 Engine Request Contract、Request State Machine Trace、KV block table、Slot Mapping Metadata、采样参数、streaming buffer、finish reason 和 metrics。Scheduler Output Metadata 不能只是请求 id 列表, 而要能转换成 Model Runner Execution Metadata, 包括 input ids、positions、block tables、slot mapping、phase 和 sampling metadata。Output Processor Finish Gate 还要把 EOS、stop string、max tokens 和 abort 反馈给 engine core, 触发 Abort Cleanup Path。

常见门禁: Engine Request Contract 要包含 request id、input ids、sampling params、状态、block table、输出 tokens 和时间戳; Request State Machine Trace 要覆盖 WAITING、PREFILL、RUNNING / DECODING、FINISHED / ABORTED; Scheduler Output Metadata 要通过 token、sequence 和 KV budget; Model Runner Execution Metadata 要能还原 positions、slot mapping 和 block table; Request Flow Metrics 要包含 queue wait、TTFT、E2E、finish reason 和 KV blocks after cleanup; Request Flow Gate 要求 input、state、scheduler、metadata、output、cleanup 和 metrics 都可复盘。

面试表达: vLLM 请求调度流程不是某个类名, 而是一条状态和资源链路; scheduler 决定本轮执行计划, block manager 保障 KV 资源, model runner 消费 execution metadata, output processor 决定请求是否继续进入下一轮, 终止路径必须释放资源。

vLLM Memory Management Audit

一句话定义：vLLM Memory Management Audit 是审计 vLLM-like engine 是否能把 GPU memory 中最动态的 KV Cache 管理成可分配、可复用、可淘汰、可观测 block pool 的方法。

它解决什么问题：模型权重相对静态，但 KV Cache 会随请求数、prompt 长度、输出长度和 prefix cache 状态动态变化。vLLM Memory Management Audit 要求先做 GPU Memory Breakdown，区分 weight、KV、workspace、CUDA graph、sampling、adapter 和 reserved memory；再用 KV Page Size、block table 和 free queue 解释 KV block 如何分配；用 Prefix Cache Hash 绑定 parent hash、block tokens 和 extra hashes，避免错误复用；用 Cached Free Block 和 LRU Cache Eviction 解释为什么 free 不等于清空；用 Preemption Recompute Path 解释 KV 空间不足时如何保护系统。

常见门禁：Toy KV Memory Manager 要能展示 block pool、free queue、request block table、cached blocks、ref count、prefix hit、cached-but-free、eviction、preemption / recompute、cleanup 和 metrics；Prefix Cache Hit Rate 要区分 lookup 与 hit；Hybrid KV Cache Group 要说明 full attention、sliding window 和 state space 层的 cache 需求不同；Memory Management Gate 要求 pool、prefix、ref count、free queue、eviction、preemption、cleanup 和 metrics 都可复盘。

面试表达：vLLM 的显存管理不是只看 `nvidia-smi`，而是看 KV block pool 是否有预算、是否能复用、是否能释放、是否能在内存压力下抢占 / 重算，以及这些行为是否能通过 metrics 解释 TTFT、TPOT、OOM 和 preemption。

vLLM Executor Architecture Audit

一句话定义：vLLM Executor Architecture Audit 是审计 vLLM-like engine 是否把 API、engine core、executor、worker、model runner 和 model 的职责边界拆清，并能把 scheduler output 可靠派发到 GPU workers 的方法。

它解决什么问题：很多人把 vLLM 架构讲成“scheduler 调模型”，但真实 serving engine 要分清 Engine Core Dispatch Loop、Executor Dispatch Layer、Worker Rank Mapping 和 Model Runner Metadata Gate。Engine core 负责请求状态、scheduler 和 KV cache；executor 负责把本轮执行计划派发给 worker group；worker 绑定 device、rank / local rank 和模型资源；model runner 把 request-level scheduler output 转成 input ids、positions、block tables、slot mapping、attention metadata 和 sampling metadata；Tensor Parallel Merge 则负责把多个 worker 的 shard 输出聚合。

常见门禁：Toy Executor Architecture 要能展示 TP/PP/DP 下 expected worker count、rank/local rank/device 映射、worker input metadata 完整性、executor dispatch trace、TP merge 结果、CPU Process Budget 和 Worker Failure Recovery；Executor Architecture Gate 要求 layer boundary、dispatch、runner metadata、rank mapping、merge、CPU budget、failure 和 metrics trace 都可复盘。

面试表达：scheduler 决定“谁该跑”，executor 决定“怎么把这轮执行计划发给哪些 worker 跑”。worker 通常一进程一 GPU，model runner 是把调度结果落成 tensor 和 metadata 的执行层。这个边界讲清后，TP、PP、DP、worker failure 和 CPU 资源瓶颈才有分析入口。

Prefix Caching Audit

一句话定义：Prefix Caching Audit 是审计 vLLM-like runtime 是否能安全复用相同 prompt 前缀对应 KV full blocks 的方法。

它解决什么问题：prompt cache 是平台层复用前缀处理结果的宽泛概念，prefix caching 是 runtime 内部复用 KV blocks 的具体机制。Prefix Caching Audit 要求先区分 prompt cache、KV cache、result cache 和 semantic cache；再用 Full Block Prefix Reuse 说明为什么只缓存 full blocks；用 Parent Hash Chain 说明相同 block tokens 在不同前文后不能复用；用 Extra Hash Isolation 和 Cache Salt Isolation 约束 model、tokenizer、LoRA、多模态媒体 hash 和租户边界；最后用 Cached Free Touch、ref count 和 Prefix Cache Eviction 解释命中、释放和 LRU 淘汰的生命周期。

常见门禁：Toy Prefix Cache Manager 要能展示 full block 命中、partial suffix 重新 prefill、cached-free block 被 touch 出 free queue、不同 salt / LoRA / media hash miss、LRU eviction、Prefix Saved Prefill Tokens、Prefix Cache Hit Rate、cached-free blocks 和 eviction count；Prefix Cache Gate 要求 hash、full block、touch、ref count、salt isolation、eviction 和 metrics 都可复盘。

面试表达: prefix caching 主要减少重复长前缀的 prefill, 所以核心收益是 TTFT 和 prefill 吞吐; 它不是万能缓存, 也不能绕过模型、tokenizer、LoRA、多模态输入和租户隔离约束。

Serving Parallelism Audit

一句话定义: Serving Parallelism Audit 是审计 LLM serving 是否根据模型大小、显存、拓扑、吞吐和延迟目标正确选择 TP / PP / DP / EP 的方法。

它解决什么问题: 很多回答会把“多 GPU”说成统一加速, 但 serving 并行要先区分瓶颈。Tensor Parallel Topology 关注单层张量切分和高频 collective 是否留在 NVLink / NVSwitch 范围内; Pipeline Parallel Bubble 关注层切分、跨节点激活传输和 microbatch 空泡; Data Parallel Replica 关注复制模型副本扩吞吐、每个 replica 独立 KV cache 和 Prefix Cache Locality; Expert Parallel Placement 关注 MoE experts 分布、all2all 通信和 expert 负载不均。

常见门禁: Toy Serving Parallel Auditor 要能展示 Worker Group Size、总 GPU 数、Per Rank Memory Fit、TPOT 估算、Cross Node TP Risk、PP bubble、DP throughput、prefix cache locality、EP imbalance 和 Parallel Serving Gate; 不能只写 TP=8 或 DP=4, 必须说明为什么这个配置适合当前模型、硬件和流量目标。

面试表达: TP/PP 解决单个模型副本放不下或单副本执行分布问题, DP 解决横向吞吐, EP 解决 MoE expert 容量。并行策略必须跟 GPU 拓扑、KV cache、prefix cache、CPU 进程和 p95 / p99 延迟一起讨论。

vLLM Performance Tuning Audit

一句话定义: vLLM Performance Tuning Audit 是把 TTFT、TPOT、吞吐、KV cache、preemption、prefix cache、CPU、streaming 和并行拓扑放到同一张证据表里做调优决策的方法。

它解决什么问题: 很多调优会直接调大 batch、加 GPU 或换量化, 但没有说明瓶颈在哪里。vLLM Performance Tuning Audit 要先用 TTFT Bottleneck Diagnosis 拆 queue、tokenize、prefill、prefix cache 和前置链路; 用 TPOT Bottleneck Diagnosis 拆 decode step、KV 读取、TP 通信、sampling、detokenize 和 streaming; 用 KV Pressure Tuning、Preemption Rate 和 allocation failure 判断 max_num_seqs、max_num_batched_tokens、max_model_len 与 gpu_memory_utilization 是否过激; 用 Prefix Cache Effectiveness 判断 hit rate、saved prefill tokens、DP route locality 和 eviction 是否真的带来收益。

常见门禁: Toy vLLM Tuning Auditor 要能输出 per-request TTFT / TPOT / queue / prefill / KV / cache 指标, 识别 queue-or-prefill TTFT、decode-or-streaming TPOT、KV pressure / preemption、prefix cache ineffective、DP route breaks cache locality、cross-node TP risk 和 CPU / streaming path; Config Trade-off Plan 要同时说明参数收益和副作用, Rollback Guard 要覆盖 p99、OOM、error rate 和质量回滚阈值; vLLM Tuning Gate 要求 metrics、TTFT、TPOT、KV、cache、config 和 rollback 都可验收。

面试表达: vLLM 调优不是背参数, 而是用真实流量 trace 证明瓶颈、改一个主要变量、压测 p50 / p95 / p99、观察 KV 与 cache 指标, 再灰度上线并保留回滚。

SGLang Motivation Audit

一句话定义: SGLang Motivation Audit 是判断一个 workload 是否真的需要 SGLang-like 前端语言和 runtime 协同优化的方法。

它解决什么问题: 很多回答会把 SGLang 说成“另一个 vLLM server”或“支持 JSON 的 OpenAI API wrapper”。SGLang Motivation Audit 要先把任务拆成 Complex LLM Program, 看它是否包含多次 generation、变量引用、分支、并行、工具 / RAG / multi-turn 状态和 structured output; 再用 RadixAttention Motivation 和 Radix Prefix Reuse 估算 naive prefill 与 radix prefill 的差距; 最后用 Structured Decoding Retry Saving 解释 JSON / regex / EBNF constrained decoding 为什么能减少后处理失败和重试尾延迟。

常见门禁: Toy SGLang Motivation Audit 要能输出 naive prefill tokens、radix prefill tokens、saved prefill tokens、reuse ratio、branch sharing、retry tokens without constraints、grammar overhead tokens、naive work tokens、SGLang work tokens、work reduction 和 SGLang Motivation Gate; 不能只说“有 RadixAttention 所以快”, 必须说明 frontend language 如何让 runtime 看见共享前缀、分支和结构化输出。

面试表达: vLLM 更偏高效服务大量请求, SGLang 更偏高效执行复杂 LLM programs。是否选择 SGLang, 要看 workload 是否有多步控制流、树状 prefix sharing、结构化输出和 runtime 可利用的程序结构, 而不是只看框架名。

SGLang Runtime Audit

一句话定义: SGLang Runtime Audit 是把 SGLang 的 OpenAI-compatible API、native /generate、offline engine 和 frontend program 统一放进 request state、scheduler、RadixAttention、memory pool、model runner、sampler、grammar backend 和 streaming 链路中验收的方法。

它解决什么问题: 很多回答会把 SGLang Runtime 说成 HTTP wrapper、RadixAttention 数据结构或某个 JSON 输出功能。SGLang Runtime Audit 要先做 Runtime Entrypoint Unification, 说明四类入口如何落到同一套请求执行后端; 再用 Request State Contract 解释 token ids、phase、sampling params、KV slot、grammar state、streaming 和 finish reason; 用 Radix Prefix Lookup 和 KV Memory Pool Budget 说明 prefill 前的 prefix hit 与显存硬约束; 最后用 Prefill Decode Scheduler、Grammar Mask Step 和 Streaming Event Trace 说明 runtime 如何逐轮执行和输出。

常见门禁: Toy SGLang Runtime Audit 要能输出 entrypoints、total input tokens、run prefill tokens、saved prefill tokens、reuse ratio、prefill rounds、decode rounds、decode rows、KV peak slots、KV pressure、grammar steps、stream events、avg TTFT 和 SGLang Runtime Gate; 不能只画一张 API server 到 model 的箭头图。

面试表达: SGLang Runtime 的核心是把不同入口统一到一条高性能生成链路, 并在 tokenized prompt 之后、prefill 之前做 radix prefix lookup, 在 scheduler 和 memory pool 约束下执行 prefill / decode, 再把 grammar mask 和 sampler、streaming 输出放到 decode loop 里。

RadixAttention Prefix Sharing Audit

一句话定义: RadixAttention Prefix Sharing Audit 是审计 SGLang-like runtime 是否能用压缩前缀树安全复用 token prefix 对应 KV cache 的方法。

它解决什么问题: 很多回答会把 RadixAttention 误说成新的 attention kernel, 或者只说“前缀缓存会更快”。RadixAttention Prefix Sharing Audit 要先说明它是 runtime 级 KV cache reuse 机制, 不替代 FlashAttention / attention backend; 再用 Compressed Prefix Tree、Longest Prefix Match 和 Radix Tree Split 解释为什么复杂 LLM programs 的树状共享前缀需要 radix tree; 用 Page Aligned Prefix Hit 说明 page size / block size 会影响可复用完整页; 用 Radix KV Ref Count 和 Leaf LRU Eviction 说明 active path 不能淘汰, 显存紧张时优先释放 ref count 为 0 的叶子; 最后用 Cache Aware Scheduling Cost 说明 scheduler 应按未命中 suffix 而不是原始 prompt 长度估算 prefill 成本。

常见门禁: Toy RadixAttention Cache 要能输出每个请求的 token hit、page hit、hit owner、run prefill tokens、new cached tokens、split count、cached tokens、pinned nodes、evicted leaf nodes、reuse ratio 和 RadixAttention Gate; 不能只画 trie, 必须证明 split、page 对齐、ref count、leaf eviction 和 scheduler cost 都可复盘。

面试表达: RadixAttention 的收益主要在 prefill / TTFT, 不直接消除 decode TPOT。它要求 token prefix 完全一致, 并且模型、tokenizer、chat template、LoRA、多模态输入、position 规则和租户隔离等 forward 条件一致。

SGLang Scheduler Audit

一句话定义: SGLang Scheduler Audit 是审计 SGLang-like runtime 能否在每个 engine step 里同时处理 decode、prefill、RadixAttention 命中、KV cache 容量、structured output、abort cleanup 和公平性的验收方法。

它解决什么问题: 很多回答会把 scheduler 简化成“把请求凑成 batch”。SGLang Scheduler Audit 要先用 Decode First Scheduling 解释 running streaming 请求为什么要稳定推进; 再用 Cache Aware Admission 和 Suffix Cost Scheduling 说明 waiting 请求不能只看 prompt length, 而要看 RadixAttention 命中后的未命中 suffix; 用 Scheduler Token Budget、Scheduler Sequence Budget 和 Scheduler KV Admission

说明 token、请求数和 KV slots 是三类不同硬约束；最后用 Scheduler Aging Fairness、Chunked Prefill Scheduling、Scheduler Abort Cleanup 和 structured output grammar cost 说明吞吐、TTFT、TPOT、显存和公平性之间的取舍。

常见门禁：Toy SGLang Scheduler 要能输出 scheduled decode ids、scheduled prefill ids、scheduled tokens、scheduled sequences、prefill suffix tokens、naive prompt tokens、grammar steps、cleaned aborts、evicted nodes、pinned nodes、KV free after scheduling、deferred reasons 和 SGLang Scheduler Gate；不能只输出一个 request id 列表。

面试表达：SGLang scheduler 的核心是用同一轮 budget 同时约束 prefill 和 decode，并把 RadixAttention 的 hit length 转成 suffix cost。KV 不够时只能淘汰安全 cached nodes，决定复用 prefix 后要 pin 住，长 prompt 要 chunked prefill，cache hit 优先也要配合 aging 防饥饿。

Structured Generation Audit

一句话定义：Structured Generation Audit 是审计 runtime 是否把 JSON schema、regex、EBNF、structural tag 或 choices 这类输出约束放进 token selection 过程，而不是只依赖 prompt 和生成后解析的方法。

它解决什么问题：很多应用要求模型输出能被程序消费的对象、标签、DSL 或 tool call。Structured Generation Audit 要先用 Grammar State 说明每个请求都有独立语法状态；用 Valid Token Mask 说明 sampler 每步只能从合法 token 集合中选择；用 JSON Schema Constraint、Regex Choice Constraint、EBNF Grammar Constraint 和 Structural Tag Constraint 说明不同约束适合的任务边界；再用 Empty Valid Token Set 说明约束冲突必须显式失败；最后用 Format Not Fact Gate 和 Structured Streaming Partial 说明格式合法不等于事实正确，streaming 中间 chunk 也通常不是完整对象。

常见门禁：Toy Constrained Decoder 要能输出 constrained JSON text、parseability、fact check、choice / EBNF / structural tag 结果、mask calls、blocked high-score invalid tokens、stream partial parseability、empty valid token detection 和 Structured Generation Gate；不能只给一个 prompt “请输出 JSON”。

面试表达：Constrained decoding 的关键是在采样前根据 grammar state 计算合法 token 集合，把非法 token logits 置为负无穷。它能减少格式错误和重试，但不保证事实正确，还会带来 grammar compile / mask latency，因此 scheduler 和 dashboard 要单独观测 structured output 请求。

Speculative Decoding Audit

一句话定义：Speculative Decoding Audit 是审计 serving runtime 是否真正实现了 draft propose、target verify、accepted prefix 提交、fallback 修正、临时 KV 清理和收益指标闭环的方法。

它解决什么问题：很多回答会把 speculative decoding 说成“小模型替代大模型”或“必然加速”。Speculative Decoding Audit 要先用 Draft Source 说明 draft 可以来自 EAGLE / EAGLE3、MTP、Standalone 小模型、NGRAM 或其他机制；再用 Target Verify Call 和 Accept Length 说明 target model 才是最终裁判；用 Draft Acceptance Rate、Target Call Reduction 和 tokens per verify 说明收益来自一次 verify 提交多个 token；最后用 Speculative Fallback Token、Speculative KV Cleanup 和 Adaptive Speculative Steps 说明低接受率、grammar 阻断、rejected token 回滚、显存峰值和动态 steps 都必须进入工程证据链。

常见门禁：Toy Speculative Decoder 要能输出每个请求的 baseline target calls、spec target calls、drafted tokens、accepted tokens、fallback tokens、grammar blocked tokens、discarded temporary KV tokens、acceptance rate、tokens per target call、target call reduction、latency speedup estimate 和 Speculative Decoding Gate；不能只给一句“draft 猜多个 token，所以更快”。

面试表达：Speculative decoding 主要优化 decode / TPOT，不直接优化 prefill。它是否有效取决于 draft 成本、接受率、target verify 并行效率、scheduler overhead、structured output 约束和 KV 显存压力；只有 accepted tokens 才能进入正式上下文和 prefix cache，rejected speculative tokens 必须丢弃或回滚。

Agent Serving Audit

一句话定义：Agent Serving Audit 是审计 runtime 和应用层是否能把 multi-turn、tool use、tool result 回灌、agent loop、多分支轨迹、prefix cache 和 task-level metrics 串成可靠执行链路的方法。

它解决什么问题：很多回答会把 agent serving 简化成“模型会调工具”。Agent Serving Audit 要先用 Multi-turn Runtime State 说明 session history、tokenized history、KV refs、grammar state 和 streaming offset 都会跨 generation 延续；用 Session Aware Routing 和 Agent Trajectory Prefix Sharing 说明同一 session 或 agent trajectory 后续轮次要尽量复用同一 replica 的 RadixAttention / prefix cache；再用 Tool Parser Fit、Tool Schema Validation 和 Tool Result Backfill 说明 parser、structured output、validator、tool executor 和 tool message 回灌是不同层；最后用 Tool Wait GPU Release、Agent Round Budget 和 task-level metrics 说明工具等待期间不能占用 GPU decode slot，agent loop 必须有轮数、token、上下文和成本上限。

常见门禁：Toy Agent Serving Audit 要能输出每次 generation 的 replica、prompt length、prefix hit length、run prefill tokens、tool name、validator result、blocked reason、session affinity、naive / run / saved prefill tokens、tool calls requested / executed / blocked、tool latency、stream events、model calls per task 和 Agent Serving Gate；不能只画一个 ReAct loop。

面试表达：Multi-turn、tool use 和 agent serving 是复杂 LLM program 的典型 workload。SGLang-like runtime 的价值在于用 RadixAttention 复用稳定历史和分支轨迹，用 structured output 和 parser 让工具调用可解析，用 scheduler 管理多次 generation 和工具等待，并用 task-level 指标拆分模型、cache、工具和 agent 逻辑的瓶颈。

Serving Architecture Comparison Audit

一句话定义：Serving Architecture Comparison Audit 是审计候选人能否按 workload、runtime 可见结构、KV cache 复用、scheduler 成本、structured output / agent 需求和指标证据来比较 vLLM-like 与 SGLang-like 架构的方法。

它解决什么问题：很多回答会把架构对比变成“谁更快”或“谁替代谁”。Serving Architecture Comparison Audit 要先用 Common Serving Core 说明两者都要处理 API、tokenizer、prefill / decode、KV cache、scheduler、sampler、streaming 和 metrics；再用 Paged Prefix Cache Evidence 说明 vLLM-like 体系中 PagedAttention / KV block / full-block prefix cache 解决的是显存布局和通用 serving；用 Radix Program Reuse Evidence 说明 SGLang-like 体系中 RadixAttention / frontend runtime 协同更关注多步 program、分支和 trajectory 的 prefix reuse；最后用 Workload Fit Score 和 Runtime Workload Router 说明独立 chat、共享 RAG、结构化抽取、agent tree 可能适合不同 backend。

常见门禁：Toy Architecture Comparator 要能输出每类 workload 的 naive prefill tokens、vLLM saved / run prefill、SGLang saved / run prefill、vLLM fit score、SGLang fit score、recommended backend、总体 saved ratio、SGLang extra saved、vLLM Not Replaced Gate、SGLang Program Fit Gate 和 Architecture Comparison Gate；不能只列静态功能表。

面试表达：PagedAttention 和 RadixAttention 不在同一抽象层。前者主要关注 KV cache 物理块和显存碎片，后者主要关注 token prefix / program trajectory 的自动复用。vLLM 很适合通用高并发 OpenAI-compatible serving，SGLang 更能体现复杂 LLM program 的 runtime 优势；真实平台可以按 workload 做路由，而不是强行二选一。

mini-sglang Source Path Audit

一句话定义：mini-sglang Source Path Audit 是审计源码学习是否覆盖入口、request state、scheduler、KV cache、RadixAttention、model runner、sampler、structured output、streaming 和 cleanup 主链路的方法。

它解决什么问题：很多回答会把“读过源码”停留在文件名和类名。mini-sglang Source Path Audit 要先用 Runtime Module Map 说明每个模块的输入、输出、状态和资源；用 Request Lifecycle Order 检查阅读顺序是否沿外部请求到内部 request、scheduler、KV、engine、sampler、output 的生命周期推进；再用 Source Resource Coverage 说明 token budget、sequence budget、KV allocate / write / read / release、prefix cache 都被解释过；最后用 Source Experiment Coverage、Radix Split Experiment 和 Abort Cleanup Experiment 说明必须跑最小生成、共享前缀、radix split、长短混排、streaming、structured output 和 abort cleanup 等实验。

常见门禁：Toy Source Path Auditor 要能输出 module coverage、resource coverage、experiment coverage、lifecycle order、实验触达模块比例、observable signal count、missing modules / resources / experiments

和 Source Path Gate; 不能只说 “先读 scheduler, 再读 cache”。

面试表达: mini-sglang 的价值是用更小代码建立 SGLang Runtime 骨架。真正看懂源码要能把模块职责映射到 TTFT、TPOT、KV pressure、prefix hit、grammar mask、streaming event 和 abort cleanup, 而不是背诵某个版本的文件路径。

Prefill Decode Resource Profile Audit

一句话定义: Prefill Decode Resource Profile Audit 是把 LLM 推理请求按 prompt token、output token、prefix hit、prefill compute、decode KV read、TTFT、TPOT 和 P/D 干扰拆开审计的方法。

它解决什么问题: 很多回答会把 PD 分离动机背成 “prefill 偏计算、decode 偏显存”, 但说不清为什么。Prefill Decode Resource Profile Audit 要先用 Request Token Profile 区分 prompt、output 和 prefix hit; 用 Run Prefill Tokens 说明长 prompt 命中缓存后不一定长 prefill; 再用 Prefill Compute Profile、Prefill KV Write 和 Decode KV Read 解释为什么 prefill 更容易形成大 token batch, 而 decode 每步虽然 token 少, 却会反复读历史 KV; 最后用 TTFT TPOT Separation、Long Prefill Stall Risk、Decode-first Starvation Risk 和 PD KV Transfer Cost 说明 chunked prefill 与 PD 分离各自缓解什么、额外付出什么。

常见门禁: Toy Prefill Decode Profiler 要能输出每个请求的 run prefill tokens、decode steps、prefill compute units、decode compute units、prefill KV write MiB、decode KV read MiB、TTFT、TPOT, 以及全局 prefill / decode token ratio、long prefill stall、chunked prefill stall、decode-first wait、PD KV transfer 和 Prefill Decode Profile Gate; 不能只给一句资源倾向结论。

面试表达: Prefill / Decode 资源画像题要按 “token 分布 -> prefix hit -> prefill compute / KV write -> decode KV read -> TTFT / TPOT -> 长 prefill 卡 decode -> decode-first 让 prefill 饥饿 -> chunked prefill 和 PD 分离取舍 -> KV transfer cost” 展开。这样才能把 PD 分离讲成可观测、可压测、可权衡的系统问题。

PD Disaggregation Motivation Audit

一句话定义: PD Disaggregation Motivation Audit 是审计系统是否真的存在 P/D 干扰、资源池扩缩容耦合和尾延迟治理困难, 并且 KV transfer 成本可控的方法。

它解决什么问题: 很多回答会一上来画 prefill pool、decode pool 和 KV transfer backend, 但没有证明为什么需要拆。PD Disaggregation Motivation Audit 要先用 Unified Engine Interference 说明长 prefill 会打断 decode; 用 Prefill Interruption Evidence 和 Decode-first Starvation Evidence 分别证明 TPOT 抖动和 TTFT 饥饿; 再用 Independent P/D Scaling 说明输入输出比例不同导致统一扩容浪费; 最后用 KV Transfer Cost Gate 和 Slow Transfer Counterexample 说明 PD 分离不是免费午餐, transfer 过慢时收益会被吞掉。

常见门禁: Toy PD Disaggregation Auditor 要能输出 unified engine 的 decode jitter p99、prefill wait p99、prefill / decode time、PD 架构的 prefill pool / decode pool time、KV transfer MiB、largest request transfer time、interference saving、慢 transfer 反例, 以及 PD Motivation Gate; 不能只说 “拆开就更快”。

面试表达: PD 分离解决的是统一 engine 内 P/D 互相干扰、资源画像不匹配、独立扩缩容困难和 p99 难治理。判断是否值得做, 要看 long prefill 是否打爆 TPOT、decode-first 是否拖高 TTFT、输入输出压力是否需要不同资源池, 以及 KV transfer 是否足够快。没有这些证据, 先做 chunked prefill、prefix cache、scheduler 调优和观测。

PD Architecture Audit

一句话定义: PD Architecture Audit 是审计 PD 分离系统是否把 router、prefill worker、decode worker、KV transfer backend、bootstrap metadata、跨组件状态机和资源清理串成可靠闭环的方法。

它解决什么问题: 很多回答会把 PD 架构画成三四个方框, 但说不清请求失败时谁清理资源。PD Architecture Audit 要先用 PD Router 说明它不是普通负载均衡器, 而是阶段感知、负载感知、KV locality 感知的 coordinator; 再用 Prefill Worker Pool、Decode Worker Pool 和 Decode Reservation 说明 P/D worker 职责边界; 用 KV Transfer Backend 和 Bootstrap Metadata 说明 source / target KV block、rank、layout 和 session 如何对齐; 最后用 Cross Component State Machine、Transfer Failure Cleanup、Client Abort Cleanup 和 Model Version Compatibility 说明失败、取消和版本不一致不能静默放过。

常见门禁：Toy PD Architecture Auditor 要能输出每个请求的 states、prefill worker、decode worker、KV metadata、stream events、cleanup 标记，以及全局 finished / aborted / transfer failed、transfer sessions、total transfer MiB、cleanup events、max decode running 和 PD Architecture Gate；不能只说 “router 负责调度”。

面试表达：PD 分离架构本质上是把单 engine 的 WAITING -> PREFILLING -> DECODING 状态机升级为跨 router、prefill worker、transfer backend、decode worker 的分布式状态机。合理架构必须证明 decode 先预留目标 slot、KV metadata 完整、transfer 失败会释放两侧资源、client abort 会广播清理、P/D worker 模型版本兼容，并且分阶段指标可观测。

KV Transfer Routing Audit

一句话定义：KV Transfer Routing Audit 是审计 PD 分离中 KV cache 从 prefill 侧交给 decode 侧时，meta-data、容量预留、路由成本、租户隔离、重算兜底和失败清理是否闭环的方法。

它解决什么问题：很多回答会把 KV transfer 说成 “把 KV cache 复制过去”，但真正风险在正确性和治理。KV Transfer Routing Audit 要先用 KV Metadata Compatibility 检查 model version、tokenizer、dtype、layout、TP size 和 rank / block mapping 是否一致；再用 Decode Capacity Reservation 说明目标 decode worker 必须先预留 KV blocks；用 KV Routing Cost Score 把 free blocks、running sequences、pending transfers、TPOT、网络 zone 和 prefix reuse 合并到路由决策；最后用 Tenant Cache Isolation、Recompute Fallback Path 和 Transfer Failure Plan 说明跨租户 prefix 命中不能复用，短 prompt 可以重算，transfer 失败必须释放目标 blocks 并保留可重试证据。

常见门禁：Toy KV Transfer Router 要能输出每个请求的 migrate / recompute / route_failed 决策、decode worker、required blocks、reserved blocks、KV transfer MiB、transfer ms、score、prefix reuse、tenant cache blocked、failure plan，以及全局迁移数、重算数、route failed、total transfer MiB、reserved blocks、skipped incompatible 和 KV Transfer Routing Gate；不能只说 “选择空闲 decode worker”。

面试表达：KV transfer 题要按 “metadata 兼容 -> decode 容量预留 -> transfer 成本估算 -> prefix / locality 路由 -> 租户隔离 -> recompute fallback -> 失败释放和重试 -> metrics” 展开。这样才能说明 KV cache 是带布局、版本、租户和生命周期的运行时状态，而不是普通 byte payload。

Chunked Disaggregated Prefill Audit

一句话定义：Chunked Disaggregated Prefill Audit 是审计长 prompt prefill 是否被切成可调度 chunk，并且能和 decode interleaving、PD transfer、backpressure 与失败清理形成闭环的方法。

它解决什么问题：很多回答会把 Chunked Prefill 说成 “把 prompt 切块”，但说不清它和 Disaggregated Prefill 的层级差异。Chunked Disaggregated Prefill Audit 要先用 Run Prefill Tokens 和 Prefill Chunk Count 说明 prefix hit 后真正要执行多少 prefill、会拆成几个 chunk；再用 Decode Interleaving 和 Prefill Token Budget 说明每轮调度要把 decode rows 与 prefill chunk 一起算进预算；用 Position Continuity 说明 chunk 边界不能让 position、attention mask 或 KV block table 断裂；最后用 Short Request Fairness、PD Transfer Pipeline 和 Backpressure Cleanup 说明短请求不能被长 prefill 饿死，按 chunk transfer 可以流水化，但下游拥塞和 abort 必须释放 partial KV。

常见门禁：Toy Chunked Disagg Prefill Auditor 要能输出 full prefill summary、chunked rows、chunked summary、PD transfer summary 和 gates，至少覆盖 max full prefill stall、short request wait、scheduled rounds、chunks run、decode steps interleaved、max chunk prefill ms、completion round、aborted requests、cleanup events、pending transfer chunks、backpressure events、total transfer MiB、pipeline transfer saving 和 Chunked Disagg Prefill Gate；不能只画一个 “切块” 时间线。

面试表达：Chunked Prefill 回答的是 how to run a long prefill，Disaggregated Prefill 回答的是 where to run prefill。前者是执行粒度和调度公平性问题，后者是 P/D 资源隔离和 KV transfer 问题。二者结合时，要同时证明 token budget、position continuity、KV block table、partial transfer、backpressure、abort cleanup 和 metrics 都可观测。

Multi Level KV Cache Audit

一句话定义：Multi Level KV Cache Audit 是审计 KV block 在 GPU、CPU、SSD、remote 和 recompute 之间如何按 residency、成本、正确性和延迟门禁做决策的方法。

它解决什么问题：很多回答会把多级 KV Cache 讲成“GPU 放不下就放 CPU / 远端”，但没有证明何时搬、何时重算、谁不能搬。Multi Level KV Cache Audit 要先用 KV Residency Level 说明 cache hit 还要看 KV 在哪一层；用 Active GPU Block Protection 说明 active decode 的 KV 不能为了命中率被挤出 GPU；再用 CPU KV Promote、Remote KV Fetch 和 Recompute Fallback 对比 transfer cost 与 recompute cost；最后用 KV Tenant Isolation、KV Demotion Policy 和 Residency Metrics 说明跨租户命中要阻断，GPU 满时只 demote ref count 为 0 的 block，并且指标要同时包含 transfer MiB、residency、latency 和 fallback。

常见门禁：Toy Multi Level KV Cache Auditor 要能输出每个 block 的 action、cost、transfer MiB、demoted victims，以及全局 GPU hit、CPU promote、remote fetch、SSD fetch、recompute、tenant block、demotion、GPU / CPU / remote residency、total transfer MiB、estimated latency 和 Multi Level KV Gate；不能只说 cache hit rate 高。

面试表达：多级 KV Cache 的关键不是多放几层存储，而是让 scheduler 知道每个 KV block 的 residency、移动成本和正确性边界。active decode KV 优先留 GPU；CPU 和 remote 用于恢复、复用和跨 worker 共享；SSD / remote 命中如果比重算慢，就应该重算；任何 model、tokenizer、position、adapter、parallel config 或 tenant 不匹配都必须阻断。

Cross Node Network Audit

一句话定义：Cross Node Network Audit 是审计跨节点 serving 中 TP、PP、PD KV transfer、remote KV cache、router 和 backpressure 哪些通信进入 TTFT / TPOT 关键路径的方法。

它解决什么问题：很多回答会把跨节点 serving 讲成“多机多卡扩容”，但没有区分通信模式。Cross Node Network Audit 要先用 Cross Node Link Profile 说明带宽、基础延迟和 p99 拥塞不是一回事；用 TP Cross Node Risk 说明 TP collective 是 per-token 高频通信，跨慢网络会直接打爆 TPOT；再用 Pipeline Activation Cost 和 PD KV Transfer Cost 分别计算 PP activation 与长 prompt KV transfer；用 Remote Recompute Decision 说明 remote hit 要和 recompute 比；最后用 Topology Aware Routing、Transfer Backpressure 和 Control Data Plane Separation 说明 router 不能只看 worker load，大对象 KV transfer 也不能压垮控制面。

常见门禁：Toy Cross Node Network Auditor 要能输出 decode worker scores、TP same-node / cross-rack TPOT overhead、PP activation ms、PD KV transfer MiB / ms、remote hot / cold decision、router choice、pending transfer MiB、backpressure events、data/control plane MiB 和 Cross Node Network Gate；不能只说“网络带宽要足够”。

面试表达：跨节点 serving 要按通信频率和关键路径拆开讲。DP 主要是请求分流；TP 是每层每 token 的 collective，通常应放在单节点高速互联内；PP 跨节点传 activation；PD 跨节点传 KV；remote cache 需要比较 fetch 与 recompute；router 要 topology-aware / KV-aware，并且用 admission control 和 backpressure 限制大 KV transfer。

PD Tradeoff Audit

一句话定义：PD Tradeoff Audit 是把 PD 分离从“高级架构选择”落成收益、成本、readiness、反模式和替代方案共同验收的决策方法。

它解决什么问题：很多回答会把 PD 分离当成默认终态，但 PD Tradeoff Audit 要先用 PD Benefit Cost Score 比较 P/D 干扰收益、独立扩容收益、KV transfer p99 和状态机复杂度；再用 PD Positive Case 说明什么样的长 prompt / streaming workload 值得试；用 Short Request Anti Pattern、Decode Bottleneck Anti Pattern 和 Slow Transfer Anti Pattern 说明短请求、长输出 decode 瓶颈、跨节点网络差时不该上 PD；最后用 Ops Readiness Gate 和 PD Alternative Path 说明 metrics、cleanup、backpressure 不齐时要先补治理，且每个 rejected case 都要有 chunked prefill、prefix cache、DP、speculative decoding 或 same-node PD 等替代路径。

常见门禁：Toy PD Tradeoff Auditor 要能输出每个 case 的 score、decision、reasons、alternatives，以及全局 adopt / rejected、anti patterns、positive cases、blocked cases 和 PD Tradeoff Gate；不能只说“长 prompt 多就做 PD”。

面试表达：PD 分离适合长 prompt 干扰 decode、P/D 比例波动、多租户隔离和独立扩缩容收益明显，并且 KV transfer、监控、失败清理和 backpressure 成熟的场景。不适合短请求、单机已达标、主要瓶颈在 decode 或网络、运维能力不足的场景。判断标准是 P/D 隔离收益是否大于 KV transfer 与系统复杂度成本。

Single Engine To PD Migration Audit

一句话定义：Single Engine To PD Migration Audit 是把已有单机 serving engine 升级成 PD 分离时，对阶段顺序、接口契约、状态迁移、same-node 原型、失败清理、观测和回滚能力做门禁验收的方法。

它解决什么问题：很多改造会直接把 `prefill()` 和 `decode()` 拆成两个服务，但没有先外置 request state、稳定 KV metadata、拆分 scheduler、引入 router 状态机和 cleanup / fallback。Single Engine To PD Migration Audit 要先用 Explicit Request Stage、Model Runner Interface Split 和 KV Metadata Contract 稳住单进程内部边界；再用 Prefill Decode Scheduler Split 和 PD Router State Machine 验证逻辑 PD；然后用 Same Node PD Prototype、Cleanup Cancel Path 和 Observability Fallback Readiness 证明 transfer、cancel、backpressure、metrics 和 fallback 可控；最后才进入 cross-node PD。

常见门禁：Toy PD Migration Auditor 要能输出 migration rows、ready steps、interfaces ready、state machine ready、cleanup paths、fallback ready、same-node-before-cross-node 和 PD Migration Gate；不能只画一张 router / P worker / D worker 架构图。

面试表达：单机 engine 升级 PD 的顺序是先改接口和状态，再拆 worker 和部署。先把 request stage 显式化，把 model runner 拆成 prefill / decode，把 KV manager 和 KV metadata 标准化；再拆 prefill / decode scheduler，引入 router 管 lifecycle；先做 same-node PD 验证 transfer、reservation、cleanup、cancel、metrics、backpressure 和 fallback；最后才做 cross-node PD 和拓扑路由。

nano-vLLM Source Learning Audit

一句话定义：nano-vLLM Source Learning Audit 是用模块地图、请求路径、engine step、KV block 生命周期、model runner batch contract 和实验信号验收 nano-vLLM 源码学习是否真的可复盘的方法。

它解决什么问题：很多回答会把“读过 nano-vLLM”停在文件列表和类名层面。nano-vLLM Source Learning Audit 要先用 Source Module Map 覆盖 example、LLM、sampling params、Sequence、LLMEngine、Scheduler、BlockManager、ModelRunner、Attention、Sampler 和模型结构；用 Generate Trace Path 与 Engine Step Trace 说明请求如何从 `generate()` 进入 waiting / running 调度，再到 prefill / decode、sampler 和 append token；用 Sequence State Audit、Scheduler Decision Trace、KV Block Lifecycle 和 Model Runner Batch Contract 说明 request state、block table、free blocks、positions、slot mapping、logits 和 sampled token 如何跨模块流动；最后用 Source Experiment Signal 证明这些路径能通过最小 generate、prefill/decode step、scheduler budget、KV block reuse、prefix cache hit、finish cleanup 和 sampler params 实验观察到。

常见门禁：Toy nano-vLLM Source Auditor 要能输出 module coverage、resource coverage、experiment coverage、lifecycle order、runtime module touch、observed signal count、missing modules / resources / experiments 和 nano-vLLM Source Gate；不能只说“先读 scheduler，再读 block manager”。

面试表达：nano-vLLM 适合用来建立 vLLM-like engine 骨架。阅读顺序沿 request lifecycle：example / LLM -> sampling params -> Sequence -> LLMEngine -> Scheduler -> BlockManager -> ModelRunner -> Attention -> Sampler -> model。项目证据要用 trace、scheduler 日志、KV block 可视化、TTFT / TPOT、prefix hit 和 cleanup 实验说明，而不是只背源码文件名。

tiny-LLM Learning Audit

一句话定义：tiny-LLM Learning Audit 是把 tiny-llm 从模型算子、最小 generate 到 KV cache、batching、chunked prefill 和 paged attention 的学习路线落成概念覆盖、实验覆盖和依赖顺序门禁的方法。

它解决什么问题：很多回答会把 tiny-llm 当作 “能生成文本的小模型教程”，但 tiny-LLM Learning Audit 要先用 Operator To Model Path、Attention Shape Trace、RoPE Position Alignment 和 GQA KV Head Audit 证明 Q/K/V shape、position、num query heads / KV heads 和 decoder layer 组装可解释；再用 Generate Sampling Loop 证明自回归生成和 sampling policy 可运行；然后用 KV Cache Equivalence 证明 prefill / decode 和 naive generate 的关系；最后用 Serving Optimization Ladder、Chunk Position Continuity 和 paged block trace 说明 quantized matmul、FlashAttention、continuous batching、chunked prefill、paged attention 如何逐步把模型 forward 推向 serving engine。

常见门禁：Toy tiny-LLM Learning Auditor 要能输出 learning units、ready units、concept coverage、experiment coverage、dependency order、serving units、missing concepts / experiments 和 tiny-LLM Learning Gate；不能只说 “我按课程读完了 attention、KV cache 和 batching”。

面试表达：tiny-llm 适合解释推理系统能力如何从底层长出来。先讲 attention / RoPE / GQA / RMSNorm / MLP / model load，再讲 generate 和 sampling，然后讲 KV cache 等价性、TTFT / TPOT 指标、quantized matmul、FlashAttention、continuous batching、chunked prefill 和 paged attention。它和 nano-vLLM 互补：tiny-llm 解释底层动机，nano-vLLM 解释 engine 模块边界。

mini-sglang Learning Audit

一句话定义：mini-sglang Learning Audit 是把 mini-sglang 从普通 serving engine 学习提升到 SGLang-style runtime 学习时，对 Radix Cache、chunked prefill、overlap scheduling、online serving、structured/tool 边界、abort cleanup 和生产差距做门禁验收的方法。

它解决什么问题：很多回答会把 mini-sglang 当作另一个 vLLM clone，只说 scheduler、KV cache 和 benchmark。mini-sglang Learning Audit 要先用 SGLang Runtime Capability 说明 request state、Radix Cache、runtime scheduler、online serving、sampler 和 streaming 的协作；用 Radix Cache Runtime Evidence 证明 longest prefix match、tree split、KV ownership 和 prefix hit metrics；用 Chunked Prefill Ablation 与 Overlap Scheduling Ablation 说明长 prompt suffix chunk、position continuity、partial KV cleanup、CPU schedule next 和 GPU forward current；再用 Online Serving Trace、Structured Tool Boundary 和 Abort Cleanup Check 说明 OpenAI-compatible streaming、client abort、grammar state、valid token mask、tool parser 与应用层工具执行边界；最后用 Production Gap Map 说明 mini-sglang 与完整 SGLang 在 distributed、grammar、speculative、metrics 和 fault recovery 上的差距。

常见门禁：Toy mini-sglang Learning Auditor 要能输出 capability coverage、experiment coverage、evidence count、runtime-specific capability、dependency order、missing capabilities / experiments 和 mini-sglang Learning Gate；不能只说 “mini-sglang 有 Radix Cache 和 online serving”。

面试表达：mini-sglang 应该放在 tiny-llm 和 nano-vLLM 之后读。tiny-llm 解释底层优化动机，nano-vLLM 解释 vLLM-like engine 骨架，mini-sglang 解释 SGLang runtime 特色：Radix Cache、chunked prefill、overlap scheduling、online serving、structured generation、tool/agent serving 和完整 SGLang 的能力差距。

Teaching Project Core Module Audit

一句话定义：Teaching Project Core Module Audit 是从 tiny-llm、nano-vLLM 和 mini-sglang 中抽象自己的 mini serving engine 时，对核心模块、状态 ownership、边界契约、依赖顺序和可替换升级路径做门禁验收的方法。

它解决什么问题：很多项目会把 “自研推理框架” 写成一张大架构图，但模块边界不清。Teaching Project Core Module Audit 要先用 Core Module Boundary 覆盖 Request、Scheduler、KVCacheManager、BatchBuilder、ModelRunner、Sampler、OutputProcessor、EngineLoop 和 Metrics；用 Request State Ownership、KV Manager Ownership 和 BatchBuilder Metadata Contract 说明 request status、waiting/running、KV blocks、batch metadata、logits、output tokens 和 TTFT / TPOT 由谁维护；再用 Scheduler ModelRunner Boundary、Output Processor Cleanup Boundary 和 Metrics Observability Contract 检查 scheduler 不直接跑模型、model runner 不修改 request 状态、output processor 负责 finish / cleanup、metrics 贯穿全链路；最后用 Replaceable Upgrade Path 说明 naive scheduler、list KV、sync output、single model 后续可以逐个替换。

常见门禁: Toy Core Module Auditor 要能输出 module coverage、state coverage、boundary coverage、ready modules、replaceable modules、engine outputs、missing modules / states 和 Core Module Gate; 不能只说 “我抽象出了 API、scheduler、KV cache 和 model runner”。

面试表达: 最小 serving engine 的重点是模块边界, 而不是第一版功能多。第一版先做单机、单模型、offline、多请求、prefill/decode、KV cache 和 metrics; Scheduler 负责决策, BatchBuilder 负责元数据, ModelRunner 负责 forward, Sampler 负责 token, OutputProcessor 负责状态收尾, KV manager 负责显存状态, Metrics 负责证明升级效果。

Naive Scheduler To Continuous Batching Audit

一句话定义: Naive Scheduler To Continuous Batching Audit 是把教学版 request-level scheduler 升级成 iteration-level continuous batching scheduler 时, 对动态加入、动态退出、预算准入和清理路径做门禁验收的方法。

它解决什么问题: 很多 mini engine 第一版只有 waiting queue 和固定 running batch, 一旦进入 decode, 请求集合就被最长输出锁住。Naive Scheduler To Continuous Batching Audit 要先用 Request Level Batch Lock 量化静态 batch 空洞, 再用 Iteration Boundary Scheduling 证明每轮 forward 结束后可以重新选择 batch; Dynamic Request Admission 允许 D/E 这类后到请求在 A 未完成时进入 running, Dynamic Request Exit 要求 B/C 结束后移出 running 并释放 KV; Decode First Upgrade 保证已有 streaming 请求的 TPOT 不被长 prompt 打断; Bounded Prefill Budget 和 KV Capacity Admission 则防止长 prompt 或 KV 不足把 scheduler 推向 OOM。

常见门禁: Toy Scheduler Upgrade Auditor 要能输出 finished order、cancelled ids、queue wait steps、TTFT steps、static decode rows、continuous output rows、saved rows、max running、max KV blocks used、deferred reasons 和 trace tail; Scheduler Upgrade Gate 要求 request-level lock、iteration boundary、dynamic join / exit、decode-first、bounded prefill、KV admission 和 cleanup 都可复盘。

面试表达: 从 naive scheduler 升级 continuous batching 的关键不是 “batch size 变大”, 而是把 request lifetime 和 execution batch lifetime 解耦。Scheduler 输出 BatchPlan, BatchBuilder 构造 metadata, OutputProcessor 更新状态并释放 KV; 只有 token budget、running 上限、KV 准入和取消清理同时成立, 才算真正完成升级。

Paged KV Cache Upgrade Audit

一句话定义: Paged KV Cache Upgrade Audit 是把教学版 request 私有 list KV cache 升级成全局 block pool、request block table、slot mapping 和幂等 cleanup 的方法。

它解决什么问题: list KV cache 把 KV 状态挂在每个 request 私有对象上, scheduler 很难准确做 KV admission, 也难以证明 finished / aborted / failed 后 GPU KV 空间可复用。Paged KV Cache Upgrade Audit 要先用 List KV Private Cache 标出旧抽象的问题, 再用 Global KV Block Pool 和 Request Block Table 把逻辑 token 序列映射到非连续 physical blocks; Allocate Until Contract 要求 prefill chunk 和 decode append 前按 target token 数补齐 block; Paged Slot Mapping 证明当前输入 token 写入物理 slot; Decode Block Extension Gate 要覆盖输出跨 block 边界时追加 block; Idempotent KV Free 和 Double Free Guard 则保证 cleanup 可重复但不会把同一 block 放回 free list 两次。

常见门禁: Toy Paged KV Upgrade Auditor 要能输出 A 的 block table 增长、decode extension block、B 释放 blocks、C 复用 blocks、D admission failure、slot mapping、block tables、fragmentation ratio、allocation failures、free blocks after cleanup、idempotent free count 和 trace tail; Paged KV Upgrade Gate 要求全局 block pool、block table、slot mapping、decode extension、KV admission、reuse 和 cleanup 全部可复盘。

面试表达: paged KV cache 的价值不是 “换个存储结构”, 而是让 KV cache 成为 scheduler 能预算、Batch-Builder 能寻址、OutputProcessor 能释放、metrics 能观测的资源。第一版先跑通 block pool、allocate_until、block table、slot mapping、free 和 reuse, 再谈 prefix cache、preemption、swap 或多 worker。

Prefix Prompt Cache Upgrade Audit

一句话定义：Prefix Prompt Cache Upgrade Audit 是在 paged KV cache 上实现第一版 hash-based prefix cache，并澄清它与平台层 prompt cache、result cache 和 semantic cache 边界的方法。

它解决什么问题：很多实现会把 prompt cache 写成 `cache[prompt]=output`，但 serving runtime 中真正能稳定复用的是已经计算完成的 KV full blocks。Prefix Prompt Cache Upgrade Audit 要先用 Prompt Prefix Cache Boundary 区分平台层缓存与 runtime KV 复用；用 Full Block Cache Reuse 规定只复用 full blocks；用 Parent Hash Cache Chain 证明同样 block tokens 在不同前文后不能共享；用 Extra Hash Cache Isolation 把 model、tokenizer、LoRA、tenant salt 和多模态 hash 纳入 cache key；命中后用 Suffix Prefill Start Position 证明 scheduler 只计算未命中 suffix；Cached Ref Count Lifecycle 要展示 active ref、cached ref、cached-free touch 和 eviction；Full Prompt Hit Fallback 则提醒：KV 命中不等于已经缓存 logits 或最终输出。

常见门禁：Toy Prefix Prompt Cache Auditor 要能输出 A 注册的 cached block table、B 命中 blocks、saved prefill tokens、run prefill tokens、suffix start、suffix positions、suffix slot mapping、touch 后 free queue、ref count attach/free、salt miss、LoRA miss、full prompt hit fallback、cached-free blocks、evicted block、token hit ratio、metrics 和 trace tail；Prefix Prompt Cache Upgrade Gate 要求边界、full block、parent hash、extra hash、suffix prefill、ref count、eviction 和 fallback 都可复盘。

面试表达：prefix cache 的本质不是缓存字符串或答案，而是让新请求的 block table 前缀引用已经计算好的 cached physical blocks。它主要降低 TTFT 和 prefill tokens，但会占用 KV blocks；因此必须用 parent hash、extra hashes、ref count、eviction、水位线、token hit ratio 和 hit/miss TTFT 对比证明收益和风险。

Preemption Recompute Swap Upgrade Audit

一句话定义：Preemption Recompute Swap Upgrade Audit 是在 paged KV cache 和 prefix cache 之后，为教学版 serving engine 增加 KV 压力兜底、recompute 恢复和 swap 状态机的门禁方法。

它解决什么问题：KV block pool 不足时，scheduler 不能只在 model forward 里 OOM。Preemption Recompute Swap Upgrade Audit 要先用 KV Pressure Trigger 证明 scheduler 在调度前发现 free blocks 不足；用 Cached Block Eviction Before Preemption 规定先淘汰没有 active request 的 cached-free blocks，再动 running request；Victim Selection Policy 要避免抢占高优先级或反复抢占同一请求；Recompute Context Preservation 要证明 preempt 后保留 prompt tokens 和已生成 output tokens；Recompute Resume Path 要用 prompt + output 重建 KV；Swap Failure Rollback 要求 swap in 失败时 CPU block table 不丢；Swap In State Restore 要求恢复后更新 GPU block table 和 RUNNING 状态。

常见门禁：Toy Preemption Upgrade Auditor 要能输出 free before、required blocks、evicted cache blocks、victims、freed blocks、burst blocks、recompute context、resume blocks、preempted count、swap out/in plan、failed swap in 后 CPU table、metrics 和 gates；Preemption Upgrade Gate 要求 pressure、eviction、victim、recompute、swap、metrics 全部可复盘。

面试表达：preemption 是鲁棒性机制，不是性能优化。正常顺序是先降低调度压力和淘汰 cached-free blocks，仍然不够才 preempt active request；recompute 简单但重算 prompt + output，swap 减少重算但引入 CPU memory、带宽和失败回滚复杂度。频繁 preemption 通常意味着容量、max_num_seqs、max_num_batched_tokens、上下文长度或 prefix cache 策略需要重调。

Unified Scheduler Loop Audit

一句话定义：Unified Scheduler Loop Audit 是把 continuous batching、paged KV cache、prefix cache、chunked prefill 和 preemption 放进同一个 engine step 验证的调度循环门禁方法。

它解决什么问题：单独实现 scheduler、KV block manager、prefix cache 或 preemption 并不等于 serving engine 能稳定运行。Unified Scheduler Loop Audit 要用 Engine Step Order 规定每轮先清理状态、处理 waiting/running、做 prefix lookup、选 decode/prefill、处理 KV 压力、commit plan、构造 batch metadata、执行模型和更新输出；Prefix Lookup Once 要证明 waiting 请求不会每轮重复增加 ref count；Decode First Commit 要证明 streaming 请求先拿到 decode 预算；Suffix Prefill Plan 要证明 prefix hit 后只计算未命中 suffix；KV Budget Commit 要证明 token budget 和 KV block budget 同时生效；Memory Pressure Order

要证明先 evict cached-free block, 再 preempt active request; Batch Metadata Invariant 和 Output State Update 则检查 positions、slot mapping、block table、num_computed_tokens 和 output tokens 是否一致。

常见门禁: Toy Unified Scheduler Loop Auditor 要能输出 prefix hits、decode items、prefill items、free before、required new blocks、evicted blocks、preempted requests、commit allocations、metadata positions、slot mapping、block tables、output tokens、computed tokens、free after 和 invariants; Unified Scheduler Loop Gate 要求 prefix、decode、prefill、budget、pressure、commit、metadata、output 和 invariant 全部可复盘。

面试表达: vLLM-like scheduler 的难点不是把请求放进一个 batch, 而是在 iteration boundary 上把队列状态、prefix cache 命中、decode 优先、chunked prefill、KV block 分配、cache eviction、preemption、BatchBuilder metadata 和 OutputProcessor 状态更新串成一个一致的事务。只要 num_computed_tokens、block table、positions 或 ref count 任意一个错, 后面看到的就是乱码输出、重复调度、KV 泄漏或尾延迟异常。

Serving Benchmark Framework Audit

一句话定义: Serving Benchmark Framework Audit 是把 workload、benchmark config、request trace、engine step trace、summary metrics、regression comparison 和 tuning decision 串成可复现实验闭环的方法。

它解决什么问题: 只跑一次 QPS 或只看 tokens/s, 不能证明 scheduler、KV cache、prefix cache 或 preemption 的改动真的变好。Serving Benchmark Framework Audit 要用 Benchmark Workload Coverage 覆盖 short、long、shared prefix 和 KV pressure; 用 Benchmark Experiment Fingerprint 记录关键参数、seed、commit 和 workload; 用 Request Trace Summary 推导 TTFT、TPOT、E2E、success rate 和 prefix hit tokens; 用 Engine Step Trace Summary 推导 waiting、active、KV peak、cleanup、eviction 和 preemption; SLO Regression Gate、Throughput Regression Check、KV Cleanup Check、Prefix Effect Check 和 Preemption Risk Check 则共同判断 candidate 是否值得接受。

常见门禁: Toy Serving Benchmark Framework 要能输出 baseline / candidate 指纹、profiles、请求数、成功率、TTFT / TPOT / E2E 分位数、output tokens/s、total tokens/s、prefix hit rate、preemption total、queue p95、active p95、KV peak、cleanup 后 KV、bottleneck classification、request rows、comparison 和 decision; Serving Benchmark Framework Gate 要求 workload、trace、SLO、throughput、KV、cache、preemption、reproducibility 和 decision 全部可复盘。

面试表达: benchmark framework 的价值不是生成一张漂亮表格, 而是防止局部作弊。比如 candidate 吞吐提升, 必须同时检查 TTFT、TPOT、E2E、失败率、KV peak、cleanup、prefix cache 收益和 preemption 风险; 只有实验指纹一致、workload 覆盖真实场景、trace 可回放、trade-off 说清楚, 调参结论才可信。

Async Serving Architecture Audit

一句话定义: Async Serving Architecture Audit 是把同步 engine loop 拆成 API admission、tokenizer queue、engine input queue、engine core、output queue、per-client stream queue 和 cancel cleanup 的异步边界门禁方法。

它解决什么问题: 教学版 engine 常把 HTTP 接入、tokenization、scheduler、model forward、detokenization、streaming 和 metrics 写在一个同步 loop 里。高并发下, tokenizer CPU、慢 client、网络发送、日志或 cancel 清理都可能阻塞 GPU 主循环。Async Serving Architecture Audit 要用 Async Boundary Design 划清 API、tokenizer、engine、output worker 和 stream worker 的职责; API Admission Boundary 只做校验、限流和入队; Tokenizer Worker Queue 解耦 tokenization; Engine Input Queue 与 Bounded Engine Drain 控制 engine 每轮接入量; Engine Output Queue 和 Per Client Stream Queue 让 OutputProcessor 不直接写网络; Stream Queue Backpressure 和 Cancel Signal Queue 则让慢 client、断连和重复 cancel 都回到 engine core 做幂等清理。

常见门禁: Toy Async Serving Runtime 要能输出 API admitted / rejected、tokenized ids、first drain added、engine input backlog、finished / cancelled、client stream emitted tokens、队列剩余长度、cleanup events 和 metrics; Async Serving Architecture Gate 要求 admission、tokenizer 解耦、bounded drain、output queue、stream backpressure、cancel signal、engine state ownership、idempotent cleanup、slow client isolation 和 queue drained 全部可复盘。

面试表达：异步 serving 的关键不是用了 `async` 关键字，而是 GPU engine core 独占调度状态，其他组件通过有界队列和消息传递交互。慢 client 只能影响自己的 stream queue，client disconnect 必须进入 cancel queue，cleanup 必须由 engine 释放 KV blocks 和 prefix refs。验证时要同时看 TTFT、TPOT、E2E、queue wait、engine step duration、stream backpressure、cancel cleanup latency 和队列积压，而不是只看吞吐是否变高。

Multi Worker Router Audit

一句话定义：Multi Worker Router Audit 是把单机异步 serving engine 扩展成多 worker、多 GPU 和 request router 时，用能力过滤、容量估算、sticky locality、load-aware fallback、heartbeat 和 retry 边界验证路由正确性的方法。

它解决什么问题：多起几个 worker 不等于分布式 serving。LLM 请求的 prompt、output、KV block 和 prefix locality 差异很大，round-robin 容易把长请求或热门 tenant 打到同一个 worker，造成 KV 压力、preemption 和尾延迟。Multi Worker Router Audit 要先用 Router Worker Capability Filter 检查 model、adapter 和 worker health；用 Request Cost Block Estimate 估算 KV block；用 Load Aware Worker Score 综合 waiting、running、engine input queue、TTFT、TPOT 和 preemption；用 Sticky Route Fallback 在 prefix cache locality 和负载均衡之间折中；Global Router Admission 要在所有候选 worker 都过载时及时拒绝；Worker Heartbeat Timeout、Safe Retry Boundary 和 Request Worker Map 则约束 worker crash 后未输出请求可重试、已 streaming 请求显式失败。

常见门禁：Toy Multi Worker Router 要能输出 round-robin baseline、load-aware choice、sticky preferred、实际 worker、超大请求 admission、跨模型能力过滤、heartbeat timeout、request state rows、worker selection counts、selection imbalance 和 metrics；Multi Worker Router Gate 要求 capability、admission、load-aware、sticky fallback、health check、retry boundary、state boundary 和 metrics 全部可复盘。

面试表达：router 的职责是 worker 级控制面，不是直接改 worker 内部 scheduler、KV block table 或 prefix cache。一个 TP group 从 router 视角看通常是一个 worker；多个 replica 才是请求级负载均衡目标。多 worker benchmark 必须按 worker 展示 selected_total、queue、KV free、TTFT / TPOT、preemption、failure 和 retry，只看全局 tokens/s 会掩盖热点。

Distributed Parallel KV Audit

一句话定义：Distributed Parallel KV Audit 是在 tensor parallel、pipeline parallel 和组合 parallel group 中，验证 KV cache 分片、通信成本、pipeline bubble、迁移决策和跨 rank / stage 清理一致性的方法。

它解决什么问题：多 GPU inference 不是把 GPU 数量乘到吞吐上。Tensor parallel 会把权重、attention heads 和 KV shard 切到多个 ranks，上层 router 看到的是一个 parallel group，而不是每张 GPU；Pipeline parallel 会按层切 stage，KV cache 也随 layer ownership 分散。Distributed Parallel KV Audit 要用 Parallel Group Boundary 防止把 TP ranks 当成独立 worker；用 TP Head Shard Assignment 和 TP Block Table Consistency 证明各 rank 的 head shard 不重叠但逻辑 block table 一致；用 PP Layer Ownership 证明每个 stage 只管理自己的层；Distributed KV Byte Accounting 要从 total、TP shard 和 PP stage 三个视角对齐 KV bytes；Collective Communication Cost 与 Pipeline Bubble Ratio 用来解释 TPOT / E2E 变化；KV Migration Decision 则比较远程拉取和 recompute；Cross Rank Cleanup 要求 finish、cancel、preemption 时所有 rank / stage 一起释放。

常见门禁：Toy Distributed Parallel KV Auditor 要能输出 worker unit、TP head ranges、head coverage、logical blocks、TP block table consistency、PP layer ranges、layer coverage、KV total MiB、per TP rank MiB、per PP stage MiB、collective communication report、pipeline bubble ratio、migration decision、cleanup events 和 gates；Distributed Parallel KV Gate 要求 group boundary、TP shard、PP ownership、block table、communication、bubble、migration 和 cleanup 全部可复盘。

面试表达：TP 解决单卡显存和单卡计算瓶颈，但每层 forward 会引入 all-reduce / all-gather 等通信；PP 降低层级显存压力，但在线 decode 容易有 pipeline bubble。KV cache 是 serving 的关键状态，TP 下通常按 head shard 分布，PP 下按 layer stage 分布。运行中的请求迁移很难，因为要保持 layer、head shard、block table、position、dtype、layout 和 sampling 状态一致，实际系统通常优先用 locality、sticky routing 和 worker-local cache，而不是频繁搬迁 KV。

OpenAI Compatible API Audit

一句话定义：OpenAI Compatible API Audit 是把 serving engine 包装成 OpenAI-compatible HTTP API 时，验证请求契约、streaming frame、错误响应、鉴权、权限、限流、usage 和隐私日志边界的方法。

它解决什么问题：engine 内部有 scheduler、KV cache 和 router，并不等于用户能稳定调用。OpenAI-compatible API 看似只是 JSON 包装，但真实风险在字段语义、streaming 增量、错误码、鉴权、配额和日志。OpenAI Compatible API Audit 要用 API Compatibility Contract 明确支持、忽略和不支持字段；Chat Request Schema Validation 校验 model、messages、role、content、max_tokens 和 temperature；SSE Stream Frame Contract 要求 chunk 可解析、data 帧和 [DONE] 结束；OpenAI Style Error Shape 要稳定输出 message、type、param、code 和 request_id；Bearer Auth Gate、Model Permission Gate、Token Rate Limit Gate 与 Concurrency Limit Gate 要在昂贵 tokenization / engine admission 前拦截；Usage Accounting Check 要统计 prompt、completion 和 total tokens；API Privacy Log Check 要防止 API key 和完整 prompt 泄露。

常见门禁：Toy OpenAI Compatible API 要能输出非流式 response、流式 chunk 数、首个 assistant role delta、最后 [DONE]、usage、认证错误、不支持字段错误、模型权限错误、context 超限、RPM 限流、并发限流和 metrics；OpenAI Compatible API Gate 要求 auth、schema、model permission、context、rate limit、non-stream、stream、usage、error shape 和 privacy 全部可复盘。

面试表达：OpenAI-compatible 不等于完全复制官方服务，而是诚实实现一个稳定子集。API 层要和 engine 层解耦：先做 auth、schema、model permission、context、rate limit 和 concurrency，再构造内部 EngineRequest 交给 router / engine。streaming 用 SSE 时，HTTP status 一旦开始就不能随意改，client disconnect 要 cancel engine request。rate limit 是租户额度语义，admission control 是系统容量语义，二者错误码和排查方向不同。

Production Deployment Audit

一句话定义：Production Deployment Audit 是把 LLM serving 从本地可运行推进到生产可发布时，验证镜像 runtime、模型 manifest、startup/readiness、worker 注册、滚动容量、drain、canary、回滚和观测闭环的方法。

它解决什么问题：LLM serving 的 ready 不是进程启动。GPU driver、CUDA runtime、PyTorch、NCCL、模型权重、tokenizer、config、chat template、parallel group、KV cache 和 warmup 任一环节失败，都会在流量进入后变成高延迟、OOM、NCCL hang、质量异常或 5xx。Production Deployment Audit 要用 Runtime Compatibility Matrix 检查 driver、GPU arch、CUDA runtime 和健康端点；用 Model Artifact Manifest 校验权重、tokenizer、config 和 chat template hash；用 Startup Readiness Gate 区分 startup、liveness 和 readiness；用 Worker Registration Gate 保证只有真正 ready 的 worker 才进入 router；用 Rolling Update Capacity Gate 计算升级中的有效容量；用 Drain Completion Gate 保证老 worker 停止接新请求但处理已有 stream；用 Canary Traffic Split、Canary Regression Gate 和 Rollback Readiness Gate 让新 revision 先小流量验证，并保留旧 revision 可快速切回。

常见门禁：Toy Production Deployment Auditor 要能输出 runtime compatibility、artifact manifest、startup phases、probe report、registered workers、rolling capacity before/after drain、drain events、canary routes、canary regression report、rollback readiness、revision 级观测日志、metrics 和 gates；Production Deployment Gate 要求 artifact、readiness、rolling capacity、drain、canary、rollback 和 observability 全部可复盘。

面试表达：生产部署要从交付物、启动、健康检查和发布策略四层讲。交付物层固定镜像 runtime 和模型 manifest；启动层按 GPU/runtime/model/tokenizer/parallel group/KV/warmup 分阶段；健康检查层用 startup 防止慢加载被误杀，用 liveness 看进程和 event loop，用 readiness 表示能接业务流量；发布层先让新 worker ready 再接流量，老 worker drain，canary 观察 error、TTFT、TPOT、OOM、KV pressure、质量指标和 revision 日志，异常时通过 router 直接切回旧 revision。

Capacity SLO Fault Drill Audit

一句话定义：Capacity SLO Fault Drill Audit 是把 LLM serving 的线上容量、成本、SLO、admission 和故障演练放到同一张可复算表里的方法。

它解决什么问题：LLM serving 容量不能只按 QPS 规划。短问答、RAG 总结和代码生成在 input tokens、output tokens、active tokens 和 KV cache 占用上差异很大，单看请求数会低估 prefill、decode 和并发显存压力。Capacity SLO Fault Drill Audit 要用 Workload Token Profile 统计峰值 RPS、input token/s、output token/s 和 active tokens；用 Benchmark Stable Capacity 把单 worker 极限压测值转换成规划容量；用 GPU Count Estimate 叠加利用率、安全余量、N+1 和发布容量；用 KV Concurrency Limit 防止长上下文把 KV cache 打满；用 Cost Attribution Model 按 GPU 小时、存储、网络、观测和 token 粗分摊；用 SLO Error Budget 把 availability、TTFT、TPOT 和 timeout 变成可计算预算；用 Admission Overload Policy 在容量不足时拒绝或降级；用 Fault Drill Scenario 和 Runbook Coverage Check 验证 worker 丢失、依赖不可用、长上下文突增和新版本退化都有 detect、mitigate、verify。

常见门禁：Toy Capacity SLO Fault Drill Auditor 要能输出 traffic profile、required workers by request / prefill / decode / KV、base / safe / N+1 / release workers、planned GPUs、cost report、SLO report、admission decision、fault drill rows 和 gates；Capacity SLO Fault Drill Gate 要求 workload profile、benchmark capacity、KV capacity、cost、SLO、admission、fault drill 和 runbook 全部可复盘。

面试表达：容量规划先按 workload 拆流量，不要只报 QPS；再用真实分布 benchmark 得到 SLO 约束下的稳定单 worker 容量；worker 数从 request、prefill、decode 和 KV 四个轴取最大值，并加安全余量、N+1、滚动发布和可用区冗余。SLO 要按 workload 定义 TTFT、TPOT、E2E、timeout 和 availability，告警要看 queue、KV pressure、preemption、OOM 和 worker health。故障演练要覆盖 worker、GPU、模型加载、依赖系统、长上下文突增和新版本性能退化，并把 runbook 变成可执行检查项。

Inference Engine Interview Readiness Audit

一句话定义：Inference Engine Interview Readiness Audit 是把推理框架面试准备从“背名词”转成题目、概念覆盖、debug 路径、项目证据和 trade-off 覆盖率审计的方法。

它解决什么问题：推理框架面试很少只考某个参数名，更多是在追问候选人能否从请求生命周期、KV cache、调度、指标、生产治理和项目证据形成完整解释。Inference Engine Interview Readiness Audit 要用 Interview Question Rubric 定义每类题的必答概念；用 Concept Coverage Score 检查回答是否覆盖 Request Lifecycle Answer、KV Cache Answer Evidence、Scheduler Answer Evidence 和 Production Governance Answer；用 Serving Metrics Coverage 检查 TTFT、TPOT、queue time、KV pressure、preemption、tokens/s 和 error rate；用 Ordered Debug Path 检查排查是否按 scope、reproduce、segment、measure、hypothesize、fix、verify 展开；用 Project Evidence Portfolio 检查是否有 demo、metrics、incident 和 artifact；用 Tradeoff Coverage 检查 latency vs throughput、memory vs batch、quality vs speed 和 cost vs reliability。

常见门禁：Toy Inference Engine Interview Auditor 要能输出 concept rows、metric gate、debug gate、tradeoff gate、project gate、weak questions、revision plan 和 gates；Inference Engine Interview Gate 要求 lifecycle、KV、scheduler、metrics、debug、production、project evidence 和 trade-off 全部可复盘。

面试表达：一个好的推理框架回答要先讲系统边界，再讲请求生命周期，然后落到数据结构和调度策略，最后用指标、debug 路径、生产治理和项目证据收束。说 vLLM、SGLang、PagedAttention 或 prefix cache 时，都要说明它解决什么问题、核心机制、收益、代价、适用 workload、线上指标和故障排查路径。

Throughput

一句话定义：throughput 是单位时间内系统处理的请求数或 token 数。

常见单位：requests/sec、tokens/sec、output tokens/sec、total tokens/sec。

为什么重要：吞吐决定同样硬件能服务多少用户，也直接影响推理成本。对于批量离线生成，吞吐往往比单请求延迟更重要。

影响因素：模型大小、batch size、输入输出长度、GPU 利用率、KV Cache 显存、调度策略、量化、并行方式和推理框架实现。

注意：吞吐不能脱离延迟讨论。极端增大 batch 可以提升吞吐，但可能让单个用户等待更久。

面试表达：吞吐衡量系统整体产能，线上服务通常要在吞吐、延迟和成本之间做平衡。

Latency

一句话定义：latency 是单个请求从提交到得到结果所经历的时间。

LLM 中的拆分：常见拆成 TTFT 和 TPOT。TTFT 影响用户看到第一个 token 的速度，TPOT 影响后续流式输出速度。

为什么复杂：同一个模型在不同输入长度、输出长度、并发量、batch 调度策略下，延迟会有很大差异。

线上指标：除了平均延迟，更要关注 P95、P99。大模型服务经常出现长尾请求，平均值好看不代表体验稳定。

面试表达：LLM 延迟不是单个数字，要拆成排队、prefill、decode 和网络传输等部分分析。

TTFT

一句话定义：TTFT 是 Time To First Token，从请求进入系统到生成第一个 token 的时间。

主要来源：请求排队、tokenization、prompt 拼接、调度等待、prefill 计算和首步 decode。

优化方向：缩短 prompt、缓存公共前缀、优化调度、减少冷启动、提升 prefill 并行效率、使用更小模型或量化模型。

典型场景：对话产品非常重视 TTFT，因为用户会把首 token 速度感知为“系统是否响应”。

面试表达：TTFT 主要受输入长度和 prefill 阶段影响，是交互式 LLM 服务的重要首响指标。

TPOT

一句话定义：TPOT 是 Time Per Output Token，decode 阶段平均生成每个输出 token 的时间。

主要来源：每步 decode 的模型计算、KV Cache 读取、batch 调度、采样逻辑和流式传输。

优化方向：提高 decode batch 效率、优化 KV Cache 管理、使用量化、使用高效 kernel、减少不必要的长输出。

与用户体验关系：长答案中，TPOT 往往比 TTFT 更影响总生成时间。如果每个 token 都很慢，流式输出会显得卡顿。

面试表达：TPOT 是生成速度指标，主要与输出长度、KV Cache、显存带宽和并发调度有关。

QPS

一句话定义：QPS 是 Queries Per Second，表示系统每秒处理的请求数。

在 LLM 中的局限：请求之间长度差异很大。一个短分类请求和一个长文生成请求的成本可能相差几十倍，只看 QPS 容易误判容量。

更细指标：tokens/sec、prefill tokens/sec、decode tokens/sec、平均输入长度、平均输出长度、P95 输出长度。

面试表达：LLM 服务不能只看 QPS，token 级吞吐更能反映真实计算成本。

Batching

一句话定义：batching 是把多个请求合并到同一次模型计算中，以提高硬件利用率。

为什么有效：GPU 适合并行计算。单个请求可能无法充分利用 GPU，合并多个请求可以提高吞吐。

难点：LLM 请求输入和输出长度不一样，生成过程中有的请求先结束，有的请求还在继续。如果 batch 固定不变，会产生 padding 浪费和等待浪费。

面试表达：batching 是提高吞吐的基础，但 LLM 的动态输出让 batching 比普通分类模型更复杂。

Dynamic Batching

一句话定义：dynamic batching 是在短时间窗口内收集多个请求，组成 batch 后一起推理。

价值：提高 GPU 利用率和整体吞吐，适合并发请求较多的在线服务。

权衡：等待窗口越长，batch 越容易凑大，吞吐越好，但单请求排队延迟也越高。

适用场景：请求到达较密集、延迟要求不是极端严格、需要提高 GPU 利用率的服务。

面试表达：dynamic batching 用少量等待换更高吞吐，核心是控制 batch 聚合窗口。

Continuous Batching

一句话定义：continuous batching 是 LLM serving 中动态把新请求加入正在运行的 batch，并把已完成请求及时移出的调度方式。

为什么提出：传统 batch 要等一组请求全部结束后才能处理下一组。LLM 输出长度差异大，短请求可能被长请求拖住，GPU 也可能因为 padding 和空槽浪费而利用率下降。

核心机制：每个 decode step 后，系统检查哪些请求完成，释放它们的 batch slot 和 KV Cache；同时把队列中的新请求加入下一步计算。

优点：提高吞吐，降低队头阻塞，更适合流式生成和不同长度请求混合的场景。

代价：调度和 KV Cache 管理更复杂，需要推理引擎支持动态 batch 和高效内存管理。

面试表达：continuous batching 是 LLM serving 的关键优化，因为 LLM decode 是逐 token 迭代，天然适合在每一步动态调整 batch。

Request Scheduling

一句话定义：request scheduling 是决定哪些请求何时进入 prefill、何时参与 decode、如何分配显存和计算资源的策略。

常见目标：减少排队、提高吞吐、控制 P95/P99 延迟、避免长请求拖垮短请求、保证不同优先级用户的服务质量。

调度挑战：prefill 计算重但可并行，decode 单步轻但持续时间长；长上下文请求占用大量 KV Cache；高并发时显存可能先于算力成为瓶颈。

常见策略：按到达顺序、按优先级、按 token budget、区分 prefill/decode 队列、限制超长请求、对长请求单独调度。

面试表达：LLM 调度要同时考虑计算和 KV Cache 显存，不只是把请求排队先进先出。

KV Cache Management

一句话定义：KV Cache management 是管理推理过程中每个请求的历史 key/value 缓存的显存分配、复用和释放。

为什么重要：KV Cache 随 batch、层数、head 数、序列长度和精度增长。长上下文和高并发下，KV Cache 可能占掉大部分显存。

核心问题：不同请求长度不同，生成过程中还不断增长。如果按最大长度连续分配，容易造成显存浪费和碎片。

优化方向：分页管理、块式分配、及时释放完成请求、共享公共前缀、KV Cache 量化、限制最大上下文和最大输出长度。

面试表达：LLM serving 的显存瓶颈很大一部分来自 KV Cache，而不是模型权重本身。

PagedAttention

一句话定义：PagedAttention 是用分页思想管理 KV Cache blocks，减少显存碎片并支持高效 continuous batching 的注意力实现思想。

典型来源：vLLM。

为什么提出：传统连续显存分配要求每个序列的 KV Cache 连续存放，遇到动态长度和高并发时容易浪费显存。PagedAttention 把 KV Cache 切成固定大小 blocks，类似操作系统虚拟内存分页。

核心收益：按需分配 blocks，减少预留浪费；请求结束后可回收 blocks；不同请求可以更灵活地组织 batch。

注意：PagedAttention 主要解决 KV Cache 内存管理和访问效率问题，不改变模型数学语义。

面试表达：PagedAttention 的核心不是新的注意力公式，而是用分页式 KV Cache 管理提升 serving 吞吐和显存利用率。

Prompt Cache

一句话定义：prompt cache 是缓存可复用 prompt 前缀的计算结果或 KV Cache，以降低重复 prefill 成本。

适用场景：大量请求共享 system prompt、固定业务模板、相同知识库前缀、长上下文对话历史。

收益：减少重复 prefill，降低 TTFT，节省计算成本。

风险：缓存命中依赖前缀完全一致或可稳定识别；不同 tokenizer、chat template、系统提示版本变化都会导致缓存失效；权限隔离不当可能引入数据泄漏风险。

面试表达：prompt cache 优化的是重复前缀的 prefill 成本，特别适合固定系统提示和模板化业务场景。

Prefix Cache

一句话定义：prefix cache 是对输入前缀的 KV Cache 进行复用的机制，常被视为 prompt cache 的一种具体形式。

核心条件：要复用缓存，前缀 token 序列必须一致。文本看起来相似但 token 不一致，通常不能直接复用。

工程重点：缓存 key 的构造、版本管理、权限隔离、淘汰策略、内存占用控制。

面试表达：prefix cache 的本质是复用公共前缀 KV，减少重复读 prompt 的开销。

FlashAttention

一句话定义：FlashAttention 是一种 IO-aware 的精确 attention 实现，通过 tiling、online softmax 和 fused kernel 减少 HBM 读写和中间矩阵存储。

它解决什么问题：标准 attention 如果显式物化 QK^T 注意力矩阵，会产生大量显存读写，长序列下成本很高。FlashAttention 通过分块计算避免完整存储 attention matrix。

关键点：FlashAttention 不是近似 attention，不改变 softmax attention 的数学结果。它优化的是计算实现和内存访问路径。

适用场景：训练和推理中的长序列 attention，尤其是 attention 受显存带宽和中间矩阵存储限制时。

面试表达：FlashAttention 的核心是 IO-aware，不是减少理论 FLOPs，而是减少 HBM 访问和中间结果物化。

IO-Aware Optimization

一句话定义：IO-aware optimization 是设计算法时不仅考虑 FLOPs，也考虑显存层级和数据搬运成本。

为什么重要：GPU 算力很强，但从 HBM 读写数据比在 SRAM/register 中复用数据更慢。很多深度学习算子实际瓶颈是 memory bandwidth，而不是算术计算。

在 FlashAttention 中：通过 tiling 把小块数据放到更快的 SRAM 中处理，减少对 HBM 的反复读写。

面试表达：IO-aware 的观点是，大模型性能优化不能只看算力峰值，还要看数据怎么移动。

Tiling / Blocking

一句话定义：tiling 或 blocking 是把大矩阵计算切成小块，在更快的 SRAM/register 中分块处理。

作用：提高数据复用，减少中间结果写回 HBM，改善 cache locality。

例子：attention 中不一次性计算完整 QK^T ，而是按 block 读取 Q、K、V，边算边更新结果。

面试表达：tiling 是把大问题拆成适合硬件缓存层级的小块，以减少慢速显存访问。

Online Softmax

一句话定义：online softmax 是在分块计算中维护最大值和归一化分母，从而得到与完整 softmax 等价结果的方法。

为什么需要：softmax 需要全局归一化。如果 attention 分块计算，就不能简单对每个块单独 softmax 后拼接。

核心思想：逐块更新当前最大值和指数和，保证数值稳定，并能在不物化完整注意力矩阵的情况下得到正确 softmax 输出。

面试表达：online softmax 是 FlashAttention 能边分块边保持精确 softmax 的关键技术之一。

HBM

一句话定义：HBM 是 High Bandwidth Memory，GPU 上容量较大但相对 SRAM/register 更慢的高带宽显存。

特点：容量大，带宽高，但访问延迟和能耗仍明显高于片上存储。

在大模型中的角色：模型权重、激活、KV Cache 和中间结果主要存放在 HBM 中。

面试表达：HBM 是 GPU 的主显存，大模型推理优化经常围绕减少 HBM 读写展开。

SRAM

一句话定义：SRAM 是 GPU 片上更快但容量更小的存储，常用于缓存计算中的小块数据。

价值：访问速度快，适合在 kernel 内复用数据。

限制：容量小，不可能容纳完整大矩阵或完整模型参数。

面试表达：FlashAttention 通过把分块数据放入 SRAM 重复使用，减少对 HBM 的依赖。

Fused Kernel

一句话定义：fused kernel 是把多个计算步骤融合到一个 GPU kernel 中执行，以减少中间结果写回和读出。

为什么有效：多个小算子分开执行会产生 kernel launch 开销和中间 tensor 的 HBM 读写。融合后可以在寄存器或 SRAM 中完成连续计算。

例子：bias、activation、dropout、residual、layer norm 等操作在某些实现中可以融合。

风险：融合 kernel 对硬件、shape 和框架版本更敏感，调试和维护成本更高。

面试表达：fused kernel 通过减少 kernel launch 和中间内存访问提升推理效率。

Memory Bandwidth Bottleneck

一句话定义：memory bandwidth bottleneck 是性能主要受数据读写速度限制，而不是受算术计算能力限制的瓶颈。

在 LLM decode 中常见：每生成一个 token 都要读取大量权重和 KV Cache，但 batch 较小时计算密度不足，GPU 算力可能吃不满。

优化方向：量化降低数据量，增大 batch 提高复用，优化 KV Cache 布局，使用高效 kernel，减少不必要的内存读写。

面试表达：LLM 推理不总是算力瓶颈，decode 阶段经常是显存带宽瓶颈。

Quantization

一句话定义：quantization 是用更低精度的数据格式表示模型权重或激活，以降低显存、带宽和部署成本。

为什么有效：参数和 KV Cache 占用的字节数下降后，可以部署更大模型、增大 batch、降低显存带宽压力，并可能提升吞吐。

主要对象：权重量化、激活量化、KV Cache 量化。LLM 中常见的是 weight-only quantization，因为它相对更容易保持质量。

风险：低精度会引入量化误差，可能导致困惑度上升、生成质量下降、数学和代码能力退化、长尾任务不稳定。

面试表达：量化用精度换成本和速度，但必须用校准集和业务评测验证质量损失。

FP32

一句话定义：FP32 是 32 位浮点格式，精度高但显存和带宽成本大。

特点：数值稳定性好，常用于传统训练基线和部分高精度计算。

在 LLM 中：完整 FP32 训练和推理成本太高，实际大模型通常使用 BF16、FP16 或更低精度。

面试表达：FP32 稳但贵，现代大模型很少全程用 FP32 做主要计算。

FP16

一句话定义：FP16 是 16 位浮点格式，显存约为 FP32 的一半，常用于训练和推理。

优点：显存和带宽成本低，硬件支持成熟。

风险：动态范围较小，训练中可能出现 overflow 或 underflow，需要 loss scaling 等技巧。

面试表达：FP16 快且省，但数值范围比 BF16 小，训练稳定性要特别关注。

BF16

一句话定义：BF16 是 Brain Floating Point 16，指数位接近 FP32，动态范围大，常用于稳定训练和推理。

优点：比 FP16 更不容易溢出，很多大模型训练默认使用 BF16。

缺点：尾数精度比 FP16 更低，但大模型训练中通常更看重动态范围。

面试表达：BF16 的核心优势是动态范围接近 FP32，因此大模型训练更稳定。

FP8

一句话定义：FP8 是 8 位浮点格式，可进一步降低显存和带宽成本，但依赖硬件、框架和校准策略支持。

价值：在支持 FP8 的新硬件上，可用于训练或推理加速。

挑战：数值范围、scale 管理、算子支持和质量评估更复杂。

面试表达：FP8 是更激进的低精度方向，收益大但工程和数值风险也更高。

INT8

一句话定义：INT8 是 8 位整数量化格式，常用于较稳健的推理压缩。

优点：显存占用和带宽约为 FP16 的一半，质量通常比 INT4 更容易保持。

常见方式：权重量化、权重加激活量化、per-channel scale、per-token activation scale。

风险：异常激活值和分布漂移会影响量化效果。

面试表达：INT8 通常是质量和压缩率之间较稳健的折中。

INT4

一句话定义：INT4 是 4 位整数量化格式，比 INT8 更省显存，但质量和 kernel 风险更高。

优点：大幅降低权重显存，适合单卡部署更大模型。

风险：量化误差更大，对校准、分组大小、重要权重保护和硬件 kernel 依赖更强。

适用场景：成本敏感、显存紧张、可接受少量质量损失的推理部署。

面试表达：INT4 很适合压缩权重，但不是无损加速，必须看任务质量和 kernel 实现。

Weight-Only Quantization

一句话定义：weight-only quantization 是只量化模型权重，activation 仍使用较高精度的量化方案。

为什么常见：LLM 推理中权重占用大，decode 阶段频繁读取权重。只量化权重能降低显存和带宽，同时避免激活量化带来的额外误差。

优点：实现相对简单，质量更容易保持，适合大模型推理部署。

缺点：不是所有硬件和 kernel 都能充分利用低比特权重带来的速度收益；有时主要收益是省显存，而不是显著降延迟。

面试表达：weight-only quantization 是 LLM 推理最常见的量化路线之一，核心收益是减少权重显存和带宽。

Post-Training Quantization / PTQ

一句话定义：PTQ 是模型训练完成后进行量化，不重新训练或只用少量校准数据。

优点：成本低、流程简单、适合已有模型快速部署。

缺点：质量依赖校准数据和量化算法，极低比特量化时容易损失能力。

适用场景：线上部署前压缩模型，快速评估不同精度和不同量化方法的效果。

面试表达：PTQ 的优势是便宜快速，风险是没有充分训练适应量化误差。

Quantization-Aware Training / QAT

一句话定义：QAT 是训练或微调时模拟量化误差，让模型适应低精度表示。

优点：低比特场景下质量通常优于纯 PTQ，模型能学会抵抗量化噪声。

缺点：需要训练成本，流程更复杂，可能需要原始训练数据或高质量微调数据。

适用场景：对质量要求高、量化比特很低、PTQ 损失不可接受的部署。

面试表达：QAT 用额外训练成本换更好的低精度质量，适合 PTQ 无法满足要求的场景。

GPTQ

一句话定义：GPTQ 是使用近似二阶信息进行逐层量化误差补偿的 LLM PTQ 方法。

核心思想：量化某些权重会引入误差，GPTQ 利用 Hessian 近似信息在逐列或逐块量化时补偿后续权重，尽量降低输出误差。

优点：在 4bit weight-only 量化中应用广泛，能在较低比特下保持较好质量。

限制：校准数据、分组大小、实现细节和 kernel 支持都会影响最终效果。

面试表达：GPTQ 是典型 PTQ 方法，重点是用近似二阶信息补偿量化误差。

AWQ

一句话定义：AWQ 是 Activation-aware Weight Quantization，利用激活信息保护重要权重的量化方法。

核心观察：并非所有权重同等重要。对激活影响大的权重更敏感，需要更好保护。

做法直觉：根据校准数据中的激活分布识别重要通道，通过缩放等方式降低重要权重量化误差。

优点：常用于高质量低比特推理部署，尤其是 4bit 权重量化场景。

面试表达：AWQ 的关键词是 activation-aware，即用激活统计判断哪些权重更值得保护。

Calibration Data

一句话定义：calibration data 是用于估计量化 scale、激活分布或敏感权重的小规模代表性数据。

为什么重要：量化参数来自数据分布。如果校准数据和真实业务分布差异很大，量化后模型可能在线上任务中明显退化。

选择原则：覆盖真实输入长度、语言、格式、领域、任务类型和高频业务模式。

常见坑：只用短问答校准长上下文模型，只用英文校准中文业务，只用通用数据校准专业领域服务。

面试表达：校准数据决定量化是否贴近真实分布，是 PTQ 质量的关键变量。

Distillation

一句话定义：distillation 是用大模型或强模型作为 teacher，训练较小 student 模型模仿其行为的方法。

在部署中的价值：小模型推理更快、成本更低、延迟更小，适合高频或边缘场景。

常见信号：teacher 的输出文本、logits、偏好排序、推理过程或工具调用轨迹。

风险：student 容量有限，可能继承 teacher 的错误，也可能在复杂推理和长尾场景上能力不足。

面试表达：蒸馏是用训练成本换推理成本，适合把大模型能力迁移到更便宜的小模型上。

TensorRT-LLM

一句话定义：TensorRT-LLM 是 NVIDIA 面向大语言模型推理优化的高性能部署框架。

常见能力：高效 kernel、量化支持、张量并行、pipeline 并行、KV Cache 优化、in-flight batching 等。

适用场景：NVIDIA GPU 上追求高性能、低延迟和工程可控的生产部署。

注意：框架收益依赖模型结构、硬件型号、batch、输入输出长度和具体配置，不是接入后必然所有场景都变快。

面试表达：TensorRT-LLM 是偏底层性能优化的 LLM 推理框架，重点服务 NVIDIA GPU 生态。

vLLM

一句话定义：vLLM 是高吞吐 LLM serving 框架, 代表性特性是 PagedAttention 和高效 continuous batching。

核心价值：提升显存利用率和在线 serving 吞吐，适合多请求并发和动态输出长度场景。

常见能力：OpenAI-compatible API、continuous batching、prefix caching、多种模型支持、分布式推理支持。

注意：实际性能取决于模型、硬件、并发、上下文长度和采样配置。

面试表达：vLLM 的代表性贡献是用 PagedAttention 管理 KV Cache，从而更好支持高吞吐 serving。

Triton Inference Server

一句话定义：Triton Inference Server 是 NVIDIA 的通用模型推理服务框架，支持多后端、多模型和动态 batching。

价值：提供模型管理、HTTP/gRPC 接口、批处理、监控指标和多框架后端能力。

在 LLM 中：可以作为部署基础设施的一部分，但 LLM 还需要结合专门的生成推理后端或自定义后端处理 KV Cache 和流式输出。

面试表达：Triton 是通用推理服务框架，LLM 场景还需要关注生成式推理特有的调度和缓存管理。

Model Parallel Inference

一句话定义：model parallel inference 是把一个模型切分到多张 GPU 上进行推理。

为什么需要：模型权重或 KV Cache 太大，单卡放不下；或者单卡吞吐不足，需要多卡协同。

常见方式：tensor parallel、pipeline parallel、expert parallel。推理中最常见的是 tensor parallel，把矩阵计算切到多卡。

代价：跨卡通信增加，部署复杂度上升，小 batch 或短请求下通信开销可能抵消收益。

面试表达：模型并行推理解决单卡放不下或吞吐不足的问题，但要付出通信和工程复杂度成本。

CPU Offloading

一句话定义：CPU offloading 是把部分权重、KV Cache 或中间状态放到 CPU 内存，在需要时再搬到 GPU。

价值：缓解 GPU 显存不足，让更大模型或更长上下文在有限显存上运行。

代价：CPU-GPU 传输带宽和延迟远弱于 GPU HBM，频繁 offload 会显著拖慢推理。

适用场景：显存极度受限、低并发、离线任务、成本优先而非低延迟的场景。

面试表达：CPU offloading 用延迟换显存，不是高性能 serving 的默认选择。

Inference Performance Incident

一句话定义：inference performance incident 是线上推理服务在延迟、吞吐、KV Cache、成本、错误率或质量回归上违反门禁的事故。

常见信号：P95 TTFT 升高、P95 TPOT 升高、P99 端到端延迟升高、KV block 分配失败、cache 命中率低、prompt token 膨胀、取消率上升、量化或调度导致关键业务退化。

排查方法：先把请求 trace 拆成 queue、tokenize、prefill、decode、streaming、postprocess 和客户端接收，再按业务线、输入长度、输出长度、模型版本和调度配置分桶。

面试表达：推理性能事故不能只看 GPU tokens/s，要把 TTFT、TPOT、尾延迟、KV、cache、prompt 成本和质量回归放在同一张审计表里。

TTFT Regression

一句话定义：TTFT regression 是首 token 延迟相对 baseline 或 SLO 变差。

常见原因：排队时间增加、prompt 变长、prefill token budget 不合理、prefix cache 失效、tokenizer CPU 瓶颈、RAG / rerank / tool call 前置链路变慢、router 把请求打到拥塞实例。

定位方式：比较 queue_ms、tokenize_ms、prefill_ms、first_chunk_ms 和 input token 分布，而不是直接判断模型本体慢。

面试表达：TTFT 退化通常先看 queue 和 prefill，再看 prompt 长度、prefix cache、tokenizer 和上游链路。

TPOT Regression

一句话定义：TPOT regression 是每输出 token 延迟变差，用户会感到流式输出卡顿。

常见原因：decode batch 过大、KV Cache 访存压力高、显存带宽瓶颈、流式链路缓冲、长短请求混跑、speculative decoding 接受率低或 proposal 成本过高。

定位方式：同时比较 worker 生成 token 时间、网关发送 chunk 时间和客户端接收时间，区分模型 decode 慢和传输 / 渲染慢。

面试表达：TPOT 退化不一定是算力不够，也可能是 KV 访存、调度、网络 backpressure 或客户端渲染造成的。

KV Pressure

一句话定义：KV pressure 是活跃请求的 KV Cache 占用接近或超过可用 KV 显存预算的程度。

粗略口径： $\rho_{kv} = M_{kv} / B_{kv}$ ，其中 M_{kv} 是当前或峰值 KV Cache 显存， B_{kv} 是可用于 KV Cache 的预算。

风险： ρ_{kv} 过高时，系统可能出现 block 分配失败、抢占、排队、OOM、长请求拖垮短请求或取消请求释放不及时。

面试表达：KV pressure 是 serving 容量规划里的硬约束，不能只看模型权重是否能放进显存。

Prompt Cost Drift

一句话定义：prompt cost drift 是新版 system prompt、RAG 上下文、工具 schema 或对话历史让输入 token 相对 baseline 膨胀。

典型指标： $\gamma_{prompt} = \text{new_input_tokens} / \text{baseline_input_tokens}$ 。这个比例升高会同时推高 TTFT、KV Cache、token 成本和长上下文干扰风险。

常见治理：压缩 system prompt、按需注入工具 schema、提高 RAG context precision、做多轮摘要、设置 max input tokens 和业务线 token budget。

面试表达：很多推理成本事故不是模型突然变慢，而是 prompt 被产品和工具链不断加长。

Cache Effectiveness

一句话定义：cache effectiveness 是缓存真实节省 prefill、延迟和成本的程度，而不是单纯命中次数。

关键指标：cache hit rate、命中 token 数、节省 prefill 时间、缓存占用、错误复用率、权限隔离和失效策略。

风险：cache key 过粗可能错误复用，过细则几乎没有命中；企业 RAG 还必须把租户、权限、文档版本和时间纳入缓存治理。

面试表达：缓存是成本优化工具，不是质量保证工具。是否有效要看节省了多少 token 和延迟，以及有没有错误复用。

Serving Gate

一句话定义：serving gate 是推理服务上线或配置变更前的性能、容量、成本和质量硬门禁。

常见门禁：P95 TTFT、P95 TPOT、P99 E2E latency、P95 queue time、KV pressure、cache effectiveness、prompt cost drift、错误 / 取消率、关键切片质量回归和回滚准备度。

价值：它把“更高 tokens/s” 转成可上线判断，避免优化吞吐却牺牲短请求体验、成本和质量。

面试表达：推理 serving gate 应该同时约束用户感知延迟、系统容量、单位成本和质量回归，而不是只看平均 latency。

Inference Platform Overview Audit

一句话定义：Inference Platform Overview Audit 是把大模型推理平台从“一个模型 HTTP 服务” 升级为请求生命周期、SLO、Model Registry、Router、Runtime、Batching、KV cache、扩缩容、发布、安全、成本、观测和开发者体验的可审计系统设计方法。

核心直觉：推理平台和单模型服务的差别，不在于有没有 API Gateway，而在于是否能稳定回答“请求从哪里来、走到哪个模型、排队多久、prefill / decode 各耗时多少、KV cache 是否够、能不能流式返回、出问题如何限流 / 熔断 / 降级 / 回滚、成本归到谁、审计能不能追踪”。vLLM、TGI、Triton、TensorRT-LLM、SGLang 等 runtime 负责高性能执行，平台控制面负责治理、SLO、发布和多租户边界。

常见审计维度：Request Lifecycle Coverage 看 auth、rate limit、input safety、routing、queue、prefill、decode、streaming、output safety、metrics、cost 和 audit 是否全链路覆盖；SLO Metric Contract 看 TTFT、TPOT、E2E、queue、input / output tokens、error、timeout 和 SLO 文档是否明确；Model Registry Readiness 看 model name、version、weight URI、tokenizer、runtime、context limit、quantization、eval report、stage 和 rollback version；Model Router Policy Fit 看租户、模型版本、延迟 SLO、成本预算、负载、fallback 和安全等级；Runtime Prefill Decode Fit、Continuous Batching Readiness 和 KV Cache Capacity Governance 看 runtime 是否区分 prefill / decode、支持动态加入 / 退出请求、管理 KV block、prefix cache、eviction、max context、admission 和指标；Autoscaling Warm Pool Fit、Release Rollback Control、Rate Limit Degrade Protection、Safety Governance Fit、Cost Attribution Optimization、Observability Trace Coverage、Multi Model Governance、Developer API Self Service 和 Inference Platform Gate 则检查扩缩容、灰度回滚、保护机制、安全、成本、trace、多模型策略和开发者自助是否闭环。

常见公式：推理平台审计样本可以写成

$$u_i = (a_i, r_i, s_i, m_i, p_i, e_i, b_i, k_i, h_i, w_i, g_i, c_i, o_i, z_i)$$

统一覆盖率可以写成

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(u_i) = 1]$$

端到端延迟可以拆成

$$T_{\{\mathrm{e2e}\}} = T_{\{\mathrm{auth}\}} + T_{\{\mathrm{route}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{tokenize}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{decode}\}} + T_{\{\mathrm{stream}\}} + T_{\{\mathrm{output}\}}$$

TTFT 与 TPOT 可以写成

$$T_{\{\mathrm{ttft}\}} = T_{\{\mathrm{auth}\}} + T_{\{\mathrm{route}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{tokenize}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{decode}\}}$$

$$T_{\{\mathrm{tpot}\}} = \frac{T_{\{\mathrm{decode}\}}}{N_{\{\mathrm{out}\}}}$$

KV cache 显存可以写成

$$M_{\{\mathrm{kv}\}} = 2 \text{LBSH}_{\{\mathrm{kv}\}} D_{\mathrm{hB}}_{\{\mathrm{elem}\}}$$

单位成本可以写成

$$K_{\{\mathrm{1k}\}} = \frac{1000 K_{\{\mathrm{total}\}}}{N_{\{\mathrm{in}\}} + N_{\{\mathrm{out}\}}}$$

推理平台门禁可以写成

$$G_{\{\mathrm{infer}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \ \& \ T_{\{\mathrm{ttft}, p95\}} \leq B_{\{\mathrm{ttft}\}} \ \& \ T_{\{\mathrm{tpot}, p95\}} \leq B_{\{\mathrm{tpot}\}}\right]]$$

面试表达:推理平台总览题可以按“需求澄清->请求生命周期->Model Registry->Model Router->Runtime->batching / KV cache->SLO 与扩缩容->灰度回滚->限流熔断降级->安全治理->成本归因->观测 trace->多模型治理->开发者自助->Inference Platform Gate”展开。不要只回答“用 K8s 部署模型服务”。

Request Lifecycle Coverage

一句话定义: Request Lifecycle Coverage 是检查推理请求从 auth、rate limit、safety、routing、queue、prefill、decode、streaming 到 metrics、cost、audit 是否有完整阶段记录。

SLO Metric Contract

一句话定义: SLO Metric Contract 是明确 TTFT、TPOT、E2E、queue time、tokens/s、error rate、timeout rate 和成本等推理平台指标的口径和阈值。

Model Registry Readiness

一句话定义: Model Registry Readiness 是检查模型名称、版本、权重路径、tokenizer、runtime、上下文长度、量化、评估报告、发布状态和回滚版本是否可追踪。

扩展口径: 推理平台接入模型仓库时, 不应只读一个权重 URI, 还要校验 Model Artifact Registry Audit 中的权重 manifest、safetensors 安全格式、tokenizer / config hash、adapter / merge lineage、量化 eval、runtime smoke test、安全权限、发布审批和 rollback alias。

Model Router Policy Fit

一句话定义: Model Router Policy Fit 是检查路由是否根据租户、模型版本、延迟 SLO、成本预算、负载、fallback 和安全等级做决策。

Runtime Prefill Decode Fit

一句话定义: Runtime Prefill Decode Fit 是检查推理 runtime 是否明确区分模型加载、tokenizer、prefill、decode、KV cache、streaming 和 runtime metrics。

Continuous Batching Readiness

一句话定义: Continuous Batching Readiness 是检查 runtime 是否支持请求动态加入、动态退出、max wait、max batch tokens、优先级队列和长请求隔离。

KV Cache Capacity Governance

一句话定义: KV Cache Capacity Governance 是把 KV block 管理、eviction、prefix cache、max context、admission control、KV pressure 和指标纳入推理平台容量门禁。

Autoscaling Warm Pool Fit

一句话定义: Autoscaling Warm Pool Fit 是用 queue、TTFT、KV pressure、tokens/s、冷启动时间、warm pool 和 scale-down 策略共同判断推理扩缩容。

Release Rollback Control

一句话定义: Release Rollback Control 是让模型发布支持 canary、A/B、traffic split、质量 / 延迟 / 成本护栏和快速回滚。

Rate Limit Degrade Protection

一句话定义：Rate Limit Degrade Protection 是用租户配额、模型配额、最大输入 / 输出、排队超时、执行超时、熔断和缓存降级保护推理平台。

Safety Governance Fit

一句话定义：Safety Governance Fit 是把鉴权、输入策略、输出策略、PII 脱敏、日志脱敏、留存策略和审计纳入在线推理治理。

Cost Attribution Optimization

一句话定义：Cost Attribution Optimization 是按 tenant、model、version、input tokens、output tokens、GPU seconds 和 cache savings 归因推理成本并触发预算告警。

Observability Trace Coverage

一句话定义：Observability Trace Coverage 是让 metrics、logs、traces、request id、route / queue / prefill / decode / stream spans、dashboard 和 alerts 串成一条排障证据链。

Multi Model Governance

一句话定义：Multi Model Governance 是用 model、model version、endpoint、deployment、route policy、scaling policy、quota policy、safety policy 和 cost policy 管理多模型平台。

Developer API Self Service

一句话定义：Developer API Self Service 是提供 OpenAPI、SDK、playground、usage dashboard、SLO dashboard、错误报告和配额申请，降低业务接入推理平台的摩擦。

Inference Platform Gate

一句话定义：Inference Platform Gate 是判断大模型推理平台是否达到低延迟、高吞吐、高可用、可回滚、可观测、可限流、可审计和可归因最低要求的综合门禁。

典型硬阻断：只有一个 HTTP 服务没有请求生命周期 trace；只看 QPS 不看 TTFT / TPOT / tokens；Model Registry 没有 tokenizer、eval report 或 rollback version；Router 不看租户、SLO、成本和负载；runtime 不区分 prefill / decode；没有 continuous batching；KV cache 没有 admission 和 eviction；扩容只看 GPU utilization；模型发布无灰度和回滚；没有限流、超时、熔断和降级；日志不脱敏；成本不能按租户 / 模型 / 版本归因；只有 worker metrics 没有 request trace；多模型没有 route / quota / safety / cost policy；开发者无法自助查看用量和错误；没有最终推理平台门禁。

Model Serving Runtime Selection Audit

一句话定义：Model Serving Runtime Selection Audit 是把 Triton、vLLM、TGI、SGLang、TensorRT-LLM 或自研 runtime 放进同一套 profile，按模型、硬件、权重格式、prefill / decode、batching、KV cache、streaming、量化、分布式、指标、benchmark、发布、维护状态、成本和自研门槛做选型审计。

核心直觉：runtime 选型不是排行榜问题。Triton 更偏通用多模型推理服务框架，vLLM / SGLang 更偏 LLM serving engine，TGI 对 Hugging Face 生态和历史部署有价值但要考虑当前维护状态，TensorRT-LLM 更偏 NVIDIA 生态深度优化，自研 runtime 则要求流量规模、团队能力和 fallback 方案同时成立。

常见审计维度：Model Hardware Fit 看模型类型、权重大小、上下文长度、GPU / CPU / 加速器、单机 / 多机拓扑是否匹配；Runtime Boundary Clarity 看平台层治理和 runtime 执行边界是否清楚；Model Format Tokenizer Fit 看权重格式、tokenizer、chat template、stop token 和 generation config 是否绑定；Prefill Decode Scheduler Fit、Continuous Batching Fit 和 KV Cache Memory Manager Fit 看 runtime 是否真正理解生成

式推理; Streaming API Fit 看 SSE / gRPC、backpressure、取消和 finish reason; Quantization Accuracy Fit、Distributed Inference Fit、Observability Metrics Fit、Benchmarking Method Fit、Deployment Rollback Fit、Ecosystem Maintenance Fit、Cost Capacity Fit、Self Development Threshold 和 Runtime Selection Gate 则检查质量、分布式、指标、测评、发布、维护、成本、自研和最终上线门禁。

常见公式: runtime 选型样本可以写成

$$r_i = (m_i, h_i, f_i, t_i, p_i, b_i, k_i, s_i, q_i, d_i, o_i, e_i, c_i, a_i, u_i, z_i)$$

统一覆盖率可以写成

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(r_i) = 1]$$

runtime 内部延迟可以拆成

$$T_{\{\mathrm{runtime}\}} = T_{\{\mathrm{tokenize}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{decode}\}} + T_{\{\mathrm{sample}\}} + T_{\{\mathrm{stream}\}}$$

TTFT 与 TPOT 可以写成

$$T_{\{\mathrm{ttft}\}} = T_{\{\mathrm{tokenize}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{first}\}}$$

$$T_{\{\mathrm{tpot}\}} = \frac{T_{\{\mathrm{decode}\}} + T_{\{\mathrm{sample}\}} + T_{\{\mathrm{stream}\}}}{N_{\{\mathrm{out}\}}}$$

单请求 KV cache 显存和 KV 并发上限可以写成

$$M_{\{\mathrm{kv}, \mathrm{req}\}} = 2\mathrm{LSH}_{\{\mathrm{kv}\}} D_{\mathrm{hB}}_{\{\mathrm{elem}\}}$$

$$N_{\{\mathrm{kv}\}} = \left\lfloor \frac{B_{\{\mathrm{kv}\}}}{M_{\{\mathrm{kv}, \mathrm{req}\}}} \right\rfloor$$

runtime 选型门禁可以写成

$$G_{\{\mathrm{runtime}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge T_{\{\mathrm{ttft}, \mathrm{p95}\}} \leq B_{\{\mathrm{ttft}\}} \wedge T_{\{\mathrm{tpot}\}} \leq B_{\{\mathrm{tpot}\}}]$$

面试表达: runtime 选型题可以按 “需求澄清 -> 候选 runtime 定位 -> 模型 / 硬件 / tokenizer 兼容 -> prefill / decode -> batching / KV cache -> streaming -> 量化 -> 分布式 -> 指标 -> benchmark -> 发布回滚 -> 维护状态 -> 成本容量 -> 自研门槛 -> Runtime Selection Gate” 展开。不要只回答 “vLLM 更快” 或 “Triton 更稳定”。

Model Hardware Fit

一句话定义: Model Hardware Fit 是检查模型结构、权重大小、上下文长度、目标吞吐、GPU 显存、互联拓扑和 runtime 支持是否匹配。

Runtime Boundary Clarity

一句话定义: Runtime Boundary Clarity 是区分平台层的鉴权、路由、配额、发布治理与 runtime 内部的模型加载、prefill、decode、KV cache、sampling 和 streaming。

Model Format Tokenizer Fit

一句话定义: Model Format Tokenizer Fit 是检查权重格式、tokenizer 版本、chat template、special token、stop token 和 generation config 是否随模型版本绑定。

Prefill Decode Scheduler Fit

一句话定义: Prefill Decode Scheduler Fit 是检查 runtime 是否把 prefill、decode、queue、token budget、长短请求隔离和 phase metrics 纳入调度。

Continuous Batching Fit

一句话定义: Continuous Batching Fit 是检查 runtime 是否支持请求动态加入、动态退出、max batch tokens、max wait 和长请求隔离，而不是只做静态 batch。

KV Cache Memory Manager Fit

一句话定义: KV Cache Memory Manager Fit 是检查 KV block、admission control、prefix cache、eviction、KV pressure 和 OOM 保护是否闭环。

Streaming API Fit

一句话定义: Streaming API Fit 是检查 runtime 的 SSE / gRPC streaming、backpressure、cancel、timeout、finish reason 和 usage 统计是否能支撑线上体验。

Quantization Accuracy Fit

一句话定义: Quantization Accuracy Fit 是检查 FP8 / INT8 / INT4 / weight-only / KV cache quantization 是否有质量评测、切片回归和回滚阈值。

Distributed Inference Fit

一句话定义: Distributed Inference Fit 是检查 tensor parallel、pipeline parallel、expert parallel、data parallel serving、PD 分离、拓扑和 rank 健康是否满足模型和 SLO。

Observability Metrics Fit

一句话定义: Observability Metrics Fit 是检查 TTFT、TPOT、queue、prefill、decode、KV、OOM、error、version 和 per-rank 指标是否可观测。

Benchmarking Method Fit

一句话定义: Benchmarking Method Fit 是检查 runtime benchmark 是否固定同一模型、硬件、精度、输入输出长度分布、并发、SLO 和 streaming 设置。

Deployment Rollback Fit

一句话定义: Deployment Rollback Fit 是检查 runtime 镜像 digest、版本 pin、health probe、canary、回滚和兼容性测试是否完整。

Ecosystem Maintenance Fit

一句话定义: Ecosystem Maintenance Fit 是检查 runtime 的维护状态、release tracking、兼容矩阵、社区或团队 owner 和安全更新责任。

Cost Capacity Fit

一句话定义: Cost Capacity Fit 是把 GPU-hour、input / output tokens、KV 预算、并发上限、冷启动和每 1k token 成本纳入 runtime 选型。

Self Development Threshold

一句话定义: Self Development Threshold 是判断自研 runtime 是否真的具备规模收益、系统能力、CUDA / kernel 能力、分布式经验和 fallback runtime。

Runtime Selection Gate

一句话定义：Runtime Selection Gate 是判断模型服务 runtime 是否达到功能、性能、质量、维护、发布、安全和成本最低上线要求的综合门禁。

典型硬阻断：候选 runtime 不支持目标模型或硬件；平台治理和 runtime 执行边界混乱；tokenizer / chat template 未绑定；不区分 prefill / decode；没有 continuous batching；KV cache 无 admission 和 eviction；streaming 不能取消或 backpressure；量化没有质量回归；多 GPU 拓扑未知；缺 TTFT / TPOT / decode 指标；benchmark 输入输出分布不一致；版本不可回滚；维护状态未知；成本容量未估算；自研理由只有“想更快”；没有最终 Runtime Selection Gate。

Prefill Decode KV Resource Profile Audit

一句话定义：Prefill Decode KV Resource Profile Audit 是把大模型在线推理请求拆成输入 token、输出 token、TTFT、TPOT、prefill、decode、KV cache、prefix cache、长上下文、多租户、streaming 和成本的可审计资源画像。

核心直觉：推理慢不是一个平均 latency 或 QPS 能解释的问题。长输入主要推高 prefill 和 TTFT，长输出主要拉长 decode 和 TPOT，高并发和长上下文会放大 KV cache 显存压力，prefix cache 可能节省 prefill，但也要受租户隔离、权限和错误复用门禁约束。

常见审计维度：Request Token Profile 看输入 / 输出 token 分布、上下文长度和 token budget；Prefill Phase Accounting、Decode Phase Accounting 和 TTFT TPOT Contract 看 prefill、decode、queue、TTFT、TPOT 和 E2E 是否分开记录；KV Cache Formula Fit 和 KV Capacity Admission 看 KV cache 公式、预算、admission 和 KV pressure 是否正确；Paged Block Management、Prefix Cache Reuse、Long Context Policy 和 Tenant KV Isolation 检查 block 管理、cache 复用、长上下文治理和租户隔离；Continuous Batching Policy、PD Disaggregation Fit、Streaming Backpressure Fit、Observability Phase Metrics、Cost Capacity Model 和 Resource Profile Gate 则检查调度、PD 分离、流式返回、观测、成本容量和最终门禁。

常见公式：资源画像样本可以写成

$$p_i=(x_i,y_i,c_i,q_i,f_i,d_i,k_i,b_i,r_i,s_i,t_i,o_i,z_i)$$

统一覆盖率可以写成

$$C_j=\frac{1}{N}\sum_{i=1}^N\mathbf{1}[g_j(p_i)=1]$$

TTFT、TPOT 和端到端延迟可以写成

$$T_{\mathrm{ttft}}=T_{\mathrm{gateway}}+T_{\mathrm{route}}+T_{\mathrm{queue}}+T_{\mathrm{tokenize}}+T_{\mathrm{prefill}}$$

$$T_{\mathrm{tpot}}=\frac{T_{\mathrm{decode}}+T_{\mathrm{sample}}+T_{\mathrm{stream}}}{N_{\mathrm{out}}}$$

$$T_{\mathrm{e2e}}\approx T_{\mathrm{ttft}}+N_{\mathrm{out}}T_{\mathrm{tpot}}$$

KV cache 显存、KV pressure 和 prefix cache 收益可以写成

$$M_{\mathrm{kv}}=2\mathrm{LSH}_{\mathrm{kv}}D_{\mathrm{hB_{elem}}}$$

$$R_{\mathrm{kv}}=\frac{M_{\mathrm{kv}}}{B_{\mathrm{kv}}}$$

$$R_{\mathrm{prefix}}=\frac{N_{\mathrm{hit_tokens}}}{N_{\mathrm{input_tokens}}}$$

资源画像门禁可以写成

$$G_{\mathrm{resource}}=\mathbf{1}[\min_j C_j\geq \tau_j \wedge T_{\mathrm{ttft},p95}\leq B_{\mathrm{ttft}} \wedge T_{\mathrm{tpot}}\leq B_{\mathrm{tpot}}]$$

面试表达：Prefill / Decode / KV 资源画像题可以按“token 分布 -> TTFT / TPOT -> prefill -> decode -> KV cache 公式 -> KV pressure -> PagedAttention / block -> prefix cache -> 长上下文 -> 多租户 -> continuous batching -> PD 分离 -> streaming -> observability -> cost -> Resource Profile Gate”展开。不要只说“QPS 高”或“GPU 利用率低”。

Request Token Profile

一句话定义：Request Token Profile 是记录输入 token、输出 token、上下文长度、token budget、长短请求分布和业务切片的推理请求画像。

Prefill Phase Accounting

一句话定义：Prefill Phase Accounting 是把 tokenization、queue、prompt prefill、chunked prefill、prefix cache 命中和 prefill tokens/s 单独记录。

Decode Phase Accounting

一句话定义：Decode Phase Accounting 是把每步 decode、采样、stream flush、output tokens/s、长输出和取消释放单独记录。

TTFT TPOT Contract

一句话定义：TTFT TPOT Contract 是明确 TTFT、TPOT、E2E、P95 / P99、queue 和 streaming 指标口径、阈值与报警策略。

KV Cache Formula Fit

一句话定义：KV Cache Formula Fit 是检查 KV cache 显存估算是否包含层数、活跃序列数、上下文长度、KV head、head dim 和 dtype bytes。

KV Capacity Admission

一句话定义：KV Capacity Admission 是在请求进入 runtime 前按 KV 预算、max context、并发、block 余量和优先级做准入控制。

Paged Block Management

一句话定义：Paged Block Management 是用固定大小 KV block、block table、free list、碎片指标和回收策略管理非连续 KV cache。

Prefix Cache Reuse

一句话定义：Prefix Cache Reuse 是复用相同或可共享前缀的 KV cache，以减少重复 prefill、降低 TTFT 和节省成本。

Long Context Policy

一句话定义：Long Context Policy 是对长 prompt、长文档、RAG 上下文和多轮历史设置 max context、预算、截断 / 压缩、长短队列和专用实例策略。

Tenant KV Isolation

一句话定义：Tenant KV Isolation 是按租户、权限、模型版本和数据分类隔离 KV cache、prefix cache、配额和成本归因。

Continuous Batching Policy

一句话定义：Continuous Batching Policy 是定义请求动态加入 / 退出、max batch tokens、max wait、长请求隔离和 decode step 调度的策略。

PD Disaggregation Fit

一句话定义：PD Disaggregation Fit 是判断 prefill / decode 分离是否值得引入，包括 KV transfer、资源池独立扩缩容、故障边界和复杂度成本。

Streaming Backpressure Fit

一句话定义：Streaming Backpressure Fit 是检查 SSE / gRPC streaming 的 flush、cancel、客户端背压、timeout 和 usage 统计是否完整。

Observability Phase Metrics

一句话定义：Observability Phase Metrics 是让 queue、tokenize、prefill、decode、sample、stream、KV、prefix hit、OOM、tenant 和 cost 指标可按请求 trace 关联。

Cost Capacity Model

一句话定义：Cost Capacity Model 是按 input tokens、output tokens、GPU seconds、KV MiB、prefix savings、冷启动和租户维度估算推理成本容量。

Resource Profile Gate

一句话定义：Resource Profile Gate 是判断推理资源画像是否足以支撑低延迟、高吞吐、长上下文、多租户、可观测和成本可控上线的综合门禁。

典型硬阻断：只记录 QPS 不记录 input / output tokens；TTFT 和 TPOT 混成一个 latency；prefill / decode 没有 phase metrics；KV cache 显存公式缺 KV head 或 dtype；没有 KV admission；PagedAttention 只知道名字没有 block 指标；prefix cache 不做租户隔离；长上下文无限制；多租户没有 cache 配额；continuous batching 无动态退出；PD 分离不估 KV transfer；streaming 无 backpressure；trace 里缺 prefill / decode span；成本模型只按请求数算；没有 Resource Profile Gate。

Inference Scheduling Audit

一句话定义：Inference Scheduling Audit 是检查推理调度是否把请求到达分布、continuous batching、token budget、prefill/decode 平衡、KV block、准入控制、长短请求隔离、多租户公平、取消释放、streaming 背压和最终门禁放进同一张审计表。

常见审计维度：Request Arrival Profile 看 input / output token 分布、到达时间、租户和优先级；Continuous Batching Core 看 dynamic join、dynamic exit、iteration-level scheduling 和 active sequence 限制；Token Budget Policy 看 max batch tokens、max prefill chunk 和 decode token budget；Prefill Decode Balance 看 decode-first、chunked prefill、prefill queue 和 decode queue；Paged KV Block Management 看 block size、block table、free list 和碎片指标；Scheduler KV Admission、Long Short Isolation、Tenant Priority Fairness、Cancellation Cleanup、Streaming Backpressure、Scheduler Observability Metrics 和 Scheduler Gate 则检查准入、长短队列、租户公平、取消释放、流式背压、指标和最终上线门禁是否闭环。

面试表达：推理调度题可以按“请求画像 -> dynamic join / exit -> token budget -> prefill/decode phase policy -> PagedAttention block -> KV admission -> 长短请求隔离 -> 多租户公平 -> cancel / timeout cleanup -> streaming backpressure -> metrics -> Scheduler Gate”展开。不要只说“开 continuous batching 就能提升吞吐”。

Request Arrival Profile

一句话定义：Request Arrival Profile 是记录请求到达时间、租户、优先级、输入 token、最大输出 token、上下文长度和业务切片的调度输入画像。

Continuous Batching Core

一句话定义：Continuous Batching Core 是支持请求在 decode iteration 之间动态加入、动态退出，并维护 waiting queue、running set 和 finished set 的核心调度能力。

Token Budget Policy

一句话定义：Token Budget Policy 是用每轮 max batch tokens、max prefill chunk、decode token budget 和 max active sequences 控制吞吐、TTFT、TPOT 和 KV 压力。

Prefill Decode Balance

一句话定义：Prefill Decode Balance 是在长 prompt prefill 和已有请求 decode 之间分配 GPU 时间，避免长 prefill 拉高 TPOT 或过度偏向 decode 拉高 TTFT。

Paged KV Block Management

一句话定义：Paged KV Block Management 是用 block table、free list、block size、引用计数和碎片指标支持 KV cache 动态分配、追加、释放和复用。

Scheduler KV Admission

一句话定义：Scheduler KV Admission 是在请求进入 running set 前按 KV block 预算、水位线、上下文长度、租户配额和优先级做准入控制。

Long Short Isolation

一句话定义：Long Short Isolation 是把长 prompt、长输出或长上下文请求和短请求分队列、分预算或分实例，降低 head-of-line blocking 和 p99 抖动。

Tenant Priority Fairness

一句话定义：Tenant Priority Fairness 是按租户配额、优先级、token share、GPU seconds、KV 占用和成本归因控制多租户推理资源分配。

Cancellation Cleanup

一句话定义：Cancellation Cleanup 是在用户取消、客户端断开、超时或 stop sequence 触发后及时释放 running slot、KV blocks、streaming 连接和指标状态。

Scheduler Observability Metrics

一句话定义：Scheduler Observability Metrics 是把 queue wait、TTFT、TPOT、scheduled tokens、active sequences、KV blocks、KV waste、rejects、cancellations 和 tenant tokens 纳入可观测闭环。

Scheduler Gate

一句话定义：Scheduler Gate 是判断推理调度是否满足低 TTFT、低 TPOT、高吞吐、KV 可控、公平性、低拒绝率和无 P0 风险的综合门禁。

Model Routing Audit

一句话定义：Model Routing Audit 是检查多模型平台是否把请求意图、模型能力、权限、成本、延迟 SLO、质量安全、fallback、灰度、实时健康、trace、降级、配置版本和最终门禁放进同一套路由审计表。

常见审计维度：Request Intent Profile 看任务类型、租户、输入 / 输出 token、工具调用、多模态、SLO、预算和风险等级；Model Capability Profile 看模型任务能力、上下文长度、tool calling、结构化输出、多模态、版本、质量分和安全等级；Permission Candidate Filter 看租户权限、合规区域、模型访问范围和 endpoint 可用性；Cost Budget Policy 和 Latency SLO Policy 看 token 成本、GPU seconds、TTFT、TPOT、排队和总延迟是否先硬过滤；Quality Safety Policy 看任务质量、风险等级、安全拦截和评估回归；Fallback Chain Governance、Canary Routing Stability、Realtime Health Load、Route Trace Coverage、Degrade Policy Control、Route Config Versioning 和 Model Routing Gate 则检查 fallback 预算、稳定灰度、健康负载、trace 完整性、降级边界、配置审计和最终上线门禁。

面试表达：模型路由题可以按 “请求画像 -> 模型能力画像 -> 权限和能力过滤 -> 成本预算 -> 延迟 SLO -> 质量安全打分 -> fallback chain -> 灰度分桶 -> 实时健康负载 -> route trace -> 降级策略 -> 配置版本 -> Model Routing Gate” 展开。不要只说 “按模型名字转发” 或 “便宜请求用小模型”。

Request Intent Profile

一句话定义：Request Intent Profile 是路由器对一次请求的结构化画像，覆盖任务类型、租户等级、输入 / 输出 token、上下文长度、工具调用、多模态、延迟 SLO、成本预算、风险等级和是否允许降级。

Model Capability Profile

一句话定义：Model Capability Profile 是模型注册表中的能力画像，覆盖支持任务、最大上下文、多模态、tool calling、结构化输出、质量评估、安全等级、成本、延迟基线、版本和回滚状态。

Permission Candidate Filter

一句话定义：Permission Candidate Filter 是在模型打分前先按租户权限、合规区域、模型访问范围、endpoint 健康和业务策略过滤候选，避免把不合规模型靠高分选中。

Cost Budget Policy

一句话定义：Cost Budget Policy 是根据输入 / 输出 token、模型单价、GPU seconds、cache savings、租户预算和优先级控制一次请求是否可以进入某个模型候选。

Latency SLO Policy

一句话定义：Latency SLO Policy 是根据 TTFT、TPOT、queue time、route overhead、输出 token 预算和 endpoint 实时负载判断候选模型是否满足请求延迟目标。

Quality Safety Policy

一句话定义：Quality Safety Policy 是把任务质量评估、安全等级、风险等级、在线反馈、回归指标和拦截策略纳入路由排序和上线门禁。

Fallback Chain Governance

一句话定义：Fallback Chain Governance 是为主模型失败后的备用模型链设置最大尝试次数、错误类型、总时间预算、总成本预算、幂等控制和熔断规则。

Canary Routing Stability

一句话定义：Canary Routing Stability 是用稳定哈希、租户分桶、版本权重和快速回滚策略控制灰度模型流量，避免同一用户在新旧版本间抖动。

Realtime Health Load

一句话定义：Realtime Health Load 是把 endpoint 健康、队列长度、错误率、timeout、TTFT、TPOT、KV pressure、admission reject 和区域状态输入路由决策。

Route Trace Coverage

一句话定义：Route Trace Coverage 是检查 route trace 是否记录 request id、tenant、task、tokens、selected model、version、route reason、fallback、degrade、cost、latency 和 config version。

Degrade Policy Control

一句话定义：Degrade Policy Control 是定义哪些业务、租户、风险等级和错误类型允许降级到小模型、缓存、文本-only 或简化流程，并统计降级率和质量影响。

Route Config Versioning

一句话定义：Route Config Versioning 是让模型路由策略、权重、灰度、fallback、权限和降级规则可版本化、可审计、可 dry-run 和可回滚。

Model Routing Gate

一句话定义：Model Routing Gate 是判断多模型路由是否满足权限正确、能力匹配、预算可控、SLO 可达、质量安全、fallback 有界、灰度稳定、trace 完整和无 P0 风险的综合门禁。

Inference Cache System Audit

一句话定义：Inference Cache System Audit 是检查 prompt / prefix cache、KV cache、semantic cache 和 result cache 是否在收益、正确性、租户隔离、一致性、随机性、streaming 和可观测性上形成闭环。

常见审计维度：Request Cache Profile 看请求 hash、模型、版本、tokenizer、参数、租户、权限、安全策略、数据版本、TTL 和 trace 是否完整；Cache Layer Boundary 看 result cache、semantic cache、prompt / prefix cache、KV cache 的命中条件和生命周期是否分清；Key Version Fingerprint 看模型、tokenizer、参数、安全策略和数据版本是否进入 key；Prompt Prefix Reuse 和 KV Lifecycle Management 看前缀 token 复用、KV block、release、retention、eviction 和显存水位；Semantic Similarity Guard 看相似度阈值、任务等价、权限、数据版本和误命中率；Result Cache Determinism、Tenant Permission Isolation、TTL Staleness Control、Eviction Quota Policy、Streaming Cache Policy、Cache Observability Metrics、Cost Latency Savings 和 Cache Governance Gate 则检查确定性、隔离、过期、驱逐、流式返回、指标、收益和最终门禁。

面试表达：缓存体系题可以按 “请求画像 -> 缓存层边界 -> key 和版本指纹 -> prompt / prefix cache -> KV 生命周期 -> semantic cache 阈值 -> result cache 确定性 -> 租户权限隔离 -> TTL 与失效 -> 驱逐与配额 -> streaming 策略 -> 指标 -> 成本延迟收益 -> Cache Governance Gate” 展开。不要只说 “加 Redis 缓存”。

Request Cache Profile

一句话定义：Request Cache Profile 是把一次缓存查找的模型、版本、tokenizer、参数、prompt hash、prefix hash、租户、权限域、安全策略、数据版本、TTL、采样随机性和 trace 字段结构化。

Cache Layer Boundary

一句话定义：Cache Layer Boundary 是区分 result cache、semantic cache、prompt / prefix cache、KV cache 和 tokenizer cache 的缓存对象、命中条件、生命周期、收益和风险边界。

Key Version Fingerprint

一句话定义：Key Version Fingerprint 是把模型版本、tokenizer、生成参数、prompt 模板、安全策略、数据版本、租户和权限域纳入 cache key 或 cache fingerprint，防止跨版本误命中。

Prompt Prefix Reuse

一句话定义：Prompt Prefix Reuse 是复用相同 system prompt、工具说明、RAG 文档或代码上下文前缀，减少重复 prefill token 和 TTFT。

KV Lifecycle Management

一句话定义：KV Lifecycle Management 是管理 KV cache 从 prefill 创建、decode 追加、prefix retention、取消释放、超时释放、eviction 到显存水位控制的完整生命周期。

Semantic Similarity Guard

一句话定义：Semantic Similarity Guard 是语义缓存命中前检查 embedding 相似度、任务等价、租户权限、数据版本、敏感等级和误命中风险。

Result Cache Determinism

一句话定义：Result Cache Determinism 是要求完整结果缓存只在确定性或近似确定性任务中使用，并把 temperature、top_p、max_tokens、stop sequence 和输出策略纳入 key。

Tenant Permission Isolation

一句话定义：Tenant Permission Isolation 是确保缓存不能跨租户、跨用户、跨权限域或跨数据分类返回私有上下文、检索结果、KV state 或完整响应。

TTL Staleness Control

一句话定义：TTL Staleness Control 是用 TTL、数据版本、模型版本、主动失效、purge 和二次校验控制过期缓存命中。

Eviction Quota Policy

一句话定义：Eviction Quota Policy 是按 LRU、LFU、TTL、成本收益、租户配额、安全等级、模型版本和显存水位驱逐缓存，防止缓存挤占关键资源。

Streaming Cache Policy

一句话定义：Streaming Cache Policy 是定义流式响应下是否缓存完整结果、是否模拟分片返回、如何处理取消、计费、trace 和 cache-hit 标记。

Cache Observability Metrics

一句话定义：Cache Observability Metrics 是记录 hit / miss、saved tokens、saved TTFT、saved GPU time、cache latency、eviction、stale hit、semantic false hit、tenant hit rate 和 net cost saved。

Cost Latency Savings

一句话定义：Cost Latency Savings 是扣除缓存系统自身存储、检索、网络和运维成本后，衡量缓存实际节省的 token、GPU time、TTFT、E2E latency 和成本。

Cache Governance Gate

一句话定义：Cache Governance Gate 是判断缓存体系是否同时满足收益为正、误命中可控、过期命中可控、租户隔离正确、trace 完整和无 P0 风险的综合门禁。

Inference Autoscaling Audit

一句话定义：Inference Autoscaling Audit 是检查 LLM 推理扩缩容是否把 token 负载、SLO、队列、GPU / KV、冷启动、warm pool、缩容 draining、路由准入、租户配额、成本和 trace 放进同一套控制闭环。

常见审计维度：Traffic Token Profile 看 QPS、input tokens/s、output tokens/s、active sequences、context tokens 和租户流量；SLO Latency Contract 看 TTFT、TPOT、E2E、p95 / p99、timeout 和错误率目标；Queue Backlog Signal 看 waiting queue、queue wait、pending prefill、pending decode 和 admission reject；GPU KV Capacity Signal 看 GPU utilization、显存、KV pressure、free KV blocks、eviction、OOM 和 tokens/s；Cold Start Lead Time 和 Warm Pool Readiness 看调度、镜像、权重加载、runtime 初始化、预热、endpoint 注册和 warm pool 可用副本；Multi Metric Recommendation、Scale Up Down Policy、Draining Streaming Safety、Router Admission Coupling、Tenant Quota Priority、Cost Budget Guard、Autoscaling Trace Coverage 和 Autoscaling Gate 则检查多指标推荐、扩缩容策略、draining、路由 / 准入联动、租户配额、成本、trace 和最终门禁。

面试表达：自动扩缩容题可以按 “流量 token 画像 -> SLO -> 队列 -> GPU / KV -> 冷启动 -> warm pool -> 多指标副本推荐 -> 扩容快缩容慢 -> draining / streaming -> router / admission 联动 -> 租户配额 -> 成本预算 -> autoscaling trace -> Autoscaling Gate” 展开。不要只说 “用 HPA 看 GPU 利用率”。

Traffic Token Profile

一句话定义：Traffic Token Profile 是把 QPS 拆成 input tokens/s、output tokens/s、上下文长度、active sequences、长短请求比例、租户流量和优先级画像。

SLO Latency Contract

一句话定义：SLO Latency Contract 是为 TTFT、TPOT、E2E、queue wait、p95 / p99、timeout rate 和 error rate 定义明确扩缩容目标。

Queue Backlog Signal

一句话定义：Queue Backlog Signal 是用 waiting queue、queue wait、pending prefill、pending decode、active requests 和 reject count 提前发现容量不足。

GPU KV Capacity Signal

一句话定义：GPU KV Capacity Signal 是把 GPU utilization、GPU memory、KV pressure、free KV blocks、eviction、OOM、input / output tokens/s 和 per-GPU throughput 作为容量信号。

Cold Start Lead Time

一句话定义：Cold Start Lead Time 是从触发扩容到新实例真正接流量的总时间，覆盖调度、镜像、权重加载、runtime 初始化、显存分配、kernel 预热和路由注册。

Warm Pool Readiness

一句话定义：Warm Pool Readiness 是评估常驻副本、预拉镜像、权重本地缓存、预加载模型、预热 kernel 和可快速接流量实例是否足以覆盖冷启动窗口。

Multi Metric Recommendation

一句话定义：Multi Metric Recommendation 是按 QPS、input tokens/s、output tokens/s、queue wait、TTFT、TPOT、KV pressure、GPU utilization 和错误率取最大需求副本，而不是依赖单一指标。

Scale Up Down Policy

一句话定义：Scale Up Down Policy 是定义扩容阈值、缩容阈值、滑动窗口、cooldown、最小 / 最大副本、预算上限、预测扩容和异常指标平滑。

Draining Streaming Safety

一句话定义：Draining Streaming Safety 是缩容前摘除路由、不接新请求、等待 active requests 和 streaming 结束、设置最长等待和 cooldown，避免直接中断用户请求。

Router Admission Coupling

一句话定义：Router Admission Coupling 是让 autoscaler、router、admission controller、缓存、限流和降级共享容量指标，避免单独扩容或单独转移流量造成故障扩散。

Tenant Quota Priority

一句话定义：Tenant Quota Priority 是按租户 token/s、队列、SLO、预算、优先级和突发配额决定扩缩容、限流、降级和成本归因。

Cost Budget Guard

一句话定义：Cost Budget Guard 是在扩容决策中加入 GPU-hour、warm pool 空闲成本、预算上限、租户成本归因和单位成功请求成本。

Autoscaling Trace Coverage

一句话定义：Autoscaling Trace Coverage 是记录触发指标、观测窗口、当前副本、建议副本、warm pool、冷启动、draining、成本、路由变更和配置版本。

Autoscaling Gate

一句话定义：Autoscaling Gate 是判断推理扩缩容是否满足 SLO、队列、KV、水位、有效容量、冷启动、成本、租户公平、trace 完整和无 P0 风险的综合门禁。

Inference Protection Policy Audit

一句话定义：Inference Protection Policy Audit 是检查 LLM 推理平台是否把限流、准入、熔断、重试、超时、降级、租户公平、streaming 清理、成本和 trace 放进同一条请求保护链路。

常见审计维度：Request Token Rate Limit 看请求数、input tokens、output tokens、上下文长度和成本额度是否共同受控；Concurrency Quota Control 看租户、用户、API key、模型和 endpoint 的并发是否有上限；Admission Capacity Guard 看队列、KV pressure、剩余 timeout、runtime 健康和 SLO 是否决定是否接收请求；Circuit Breaker State Machine 看 closed / open / half-open、滑动窗口错误率、慢调用率和探测恢复；Retry Error Classification、Retry Budget Bound 和 Idempotency Stage Guard 看是否只对未开始执行的临

时错误重试, 并受 retry budget、幂等键和执行阶段约束; Timeout Budget Allocation 和 Degradation Policy Control 看总 deadline 是否分配到 gateway、queue、prefill、decode、streaming 和 fallback; Priority Tenant Fairness、Streaming Cancellation Cleanup、Protection Trace Coverage、Cost Budget Guard 和 Protection Gate 则检查优先级、公平性、取消释放、审计 trace、成本和最终门禁。

面试表达: 限流、熔断、重试、超时和降级题可以按 “租户 / token / 并发限流 -> admission control -> endpoint 熔断 -> retry 分类和预算 -> timeout budget -> 业务允许的降级 -> streaming / cancel 清理 -> trace / cost / policy version -> Protection Gate” 展开。不要只说 “失败就重试” 或 “QPS 高了限流”。

Request Token Rate Limit

一句话定义: Request Token Rate Limit 是把普通 QPS 限流扩展到 input tokens、output tokens、上下文长度、请求权重和成本额度。

Concurrency Quota Control

一句话定义: Concurrency Quota Control 是限制租户、用户、API key、模型、endpoint 和任务类型的同时在途请求数, 防止长请求占满连接、KV cache 和 GPU slot。

Admission Capacity Guard

一句话定义: Admission Capacity Guard 是在请求进入 runtime 前检查队列、KV pressure、剩余 timeout、endpoint 健康、租户优先级和 SLO, 决定 admit、reject、queue 或 degrade。

Circuit Breaker State Machine

一句话定义: Circuit Breaker State Machine 是用 closed、open 和 half-open 管理异常 endpoint 或依赖, 避免错误率、慢调用、队列爆炸或 OOM 继续扩散。

Retry Error Classification

一句话定义: Retry Error Classification 是区分 connect reset、尚未开始执行的临时不可达、权限失败、配额失败、安全拒绝、用户取消和已开始 streaming 等错误, 决定哪些可以重试。

Retry Budget Bound

一句话定义: Retry Budget Bound 是限制重试流量占初始请求流量的比例, 避免多层重试在过载时形成 retry storm。

Idempotency Stage Guard

一句话定义: Idempotency Stage Guard 是要求重试必须绑定幂等键、执行阶段、attempt id 和 partial output 状态, 避免已经开始生成或产生副作用的请求被盲目重放。

Timeout Budget Allocation

一句话定义: Timeout Budget Allocation 是把总 deadline 拆到 gateway、queue、prefill、decode、streaming、工具调用、后处理和 buffer, 并用剩余预算决定继续、拒绝或降级。

Degradation Policy Control

一句话定义: Degradation Policy Control 是把小模型、缓存、截断上下文、减少输出、关闭工具调用、异步处理和转人工等降级动作绑定到业务风险、租户权限、质量下限和用户可见说明。

Priority Tenant Fairness

一句话定义：Priority Tenant Fairness 是在过载时按租户等级、业务 criticality、SLO、预算、配额和历史使用量做公平保护，而不是让单个突发租户拖垮全局。

Streaming Cancellation Cleanup

一句话定义：Streaming Cancellation Cleanup 是在客户端断开、用户取消、timeout 或降级时释放 KV block、连接、队列项、partial output 状态和 trace，避免资源泄漏。

Protection Trace Coverage

一句话定义：Protection Trace Coverage 是记录 rate limit、admission、circuit、retry、timeout、degrade、fallback、成本、policy version 和最终 decision reason 的 trace 覆盖率。

Protection Gate

一句话定义：Protection Gate 是判断推理保护策略是否满足限流正确、准入可靠、熔断隔离、重试有界、超时受控、降级可解释、trace 完整和无 P0 风险的综合门禁。

Inference Release Governance Audit

一句话定义：Inference Release Governance Audit 是检查模型发布是否把 release bundle、版本包完整性、离线评估、契约测试、稳定分桶、canary、A/B、guardrail、rollback、audit trace 和上线后监控放进同一套发布治理闭环。

常见审计维度：Release Bundle Trace 看模型权重、tokenizer、prompt、runtime config、安全策略、eval report、owner、approver 和 rollback target 是否完整；Model Artifact Integrity 看权重 checksum、镜像 digest、tokenizer checksum 和 artifact lineage；Tokenizer Prompt Runtime Binding 看线上输出是否能追溯到完整版本包；Offline Eval Quality Gate、Safety Eval Gate 和 Contract Test Compatibility 看质量、安全、结构化输出、tool call schema 和下游 parser 是否过线；Stable Bucket Assignment、Canary Ramp Control、AB Experiment Design 和 Sample Ratio Health 看分桶、放量、实验设计和样本比例是否可信；Guardrail Metric Control、Rollback Readiness、Release Audit Trace、Post Release Monitoring 和 Release Governance Gate 则检查安全、延迟、成本、业务护栏、回滚目标、审计和最终门禁。

面试表达：模型发布题可以按 “release bundle -> artifact 完整性 -> tokenizer / prompt / runtime 绑定 -> offline eval -> safety eval -> contract test -> stable bucket -> canary ramp -> A/B design -> sample ratio -> guardrail -> rollback -> audit trace -> post-release monitoring -> Release Governance Gate” 展开。不要只说 “灰度 1% 然后慢慢加流量”。

Release Bundle Trace

一句话定义：Release Bundle Trace 是把模型权重、tokenizer、prompt、runtime config、安全策略、eval report、推理参数、路由策略、owner、approver 和 rollback target 绑定成可审计发布包。

Model Artifact Integrity

一句话定义：Model Artifact Integrity 是用 checksum、digest、manifest、lineage 和不可变 URI 证明上线模型、tokenizer、adapter、量化包和镜像没有被替换或篡改。

扩展口径：生产发布时还要和 Artifact Management Audit 对齐，把 deployment package、release manifest、runtime image digest、health check、safety policy、route policy、rollback target 和删除依赖检查一起纳入发布证据。

推理侧关联：线上模型发布和灰度实验也应和 Experiment Tracking Audit 对齐，把 release candidate 的 run id、offline eval run、canary 指标、A/B 配置、prompt / runtime / safety policy 版本、sample failures、成本和回滚证据写入可追踪记录，避免只知道线上模型名而无法复现发布判断。

评估平台关联：发布侧还要和 Evaluation Platform Audit 对齐，把离线批量评测、人评、LLM judge、pairwise win rate、切片回归、在线 guardrail、eval report artifact、release decision 和 rollback evidence 串成证据链。否则 release dashboard 只能看到“已通过评估”，却无法解释通过的是哪个数据版本、哪个指标口径、哪个 judge、人评质量是否稳定、线上分桶是否健康。

Tokenizer Prompt Runtime Binding

一句话定义：Tokenizer Prompt Runtime Binding 是记录每次线上请求实际使用的 tokenizer、prompt 模板、runtime config、sampling 参数和安全策略版本，避免只知道模型名而无法复现输出。

Offline Eval Quality Gate

一句话定义：Offline Eval Quality Gate 是候选模型发布前必须通过固定评估集、关键切片、bad case 回归、人工偏好或业务 proxy 指标的离线质量门禁。

Safety Eval Gate

一句话定义：Safety Eval Gate 是候选模型发布前必须通过安全拒答、越权、隐私、敏感信息、jailbreak、tool misuse 和高风险场景评估的安全门禁。

Contract Test Compatibility

一句话定义：Contract Test Compatibility 是检查 JSON schema、tool call 参数、结构化输出、错误格式、拒答格式、max context 和下游 parser 是否与生产契约兼容。

Stable Bucket Assignment

一句话定义：Stable Bucket Assignment 是用稳定 hash 把用户、租户或会话绑定到实验分桶，保证同一实体在同一实验中持续看到同一版本。

Canary Ramp Control

一句话定义：Canary Ramp Control 是按 1%、5%、10%、25%、50% 等步骤逐步放量，并在每一步用质量、安全、延迟、成本和业务 guardrail 决定 promote、pause、abort 或 rollback。

AB Experiment Design

一句话定义：AB Experiment Design 是定义 baseline、treatment、随机单位、样本量、主指标、guardrail、停止规则、切片分析和显著性判断的在线实验设计。

Sample Ratio Health

一句话定义：Sample Ratio Health 是检查 A/B 实验实际样本比例是否接近期望比例，避免分流、日志、过滤或埋点问题污染实验结论。

Guardrail Metric Control

一句话定义：Guardrail Metric Control 是把安全、p99 延迟、成本、工具失败率、结构化输出解析失败率、投诉率和人工接管率设成发布硬护栏。

Rollback Readiness

一句话定义：Rollback Readiness 是确认旧模型、prompt、runtime、路由、安全策略和配置仍可快速恢复，并且回滚不会破坏状态或下游契约。

Release Audit Trace

一句话定义：Release Audit Trace 是记录谁创建版本、谁审批、放量步骤、指标、实验配置、回滚原因、policy version 和最终决策的发布审计链。

Post Release Monitoring

一句话定义：Post Release Monitoring 是全量后继续监控质量、安全、延迟、成本、业务指标、投诉、bad case 和回归样本，防止灰度窗口外才暴露问题。

Release Governance Gate

一句话定义：Release Governance Gate 是判断模型发布是否满足版本包完整、离线评估、安全评估、契约测试、分桶实验、guardrail、回滚、trace 和无 P0 风险的综合门禁。

Inference Platform System Design Audit

一句话定义：Inference Platform System Design Audit 是把一份推理平台系统设计答案放进需求、请求生命周期、SLO、模型注册、路由、runtime、KV、缓存、扩缩容、保护、发布、多租户、安全、观测、成本和 trade-off 的统一审计表。

常见审计维度：Requirement Clarification 看模型类型、QPS、input / output token 分布、上下文长度、TTFT、TPOT、p99、streaming、多租户、安全、成本和可用性是否澄清；SLO Token Contract 看答案是否用 TTFT、TPOT、E2E、token load 和容量预算约束设计；Model Registry Release、Router Policy Fit、Runtime Prefill Decode 和 KV Scheduler Capacity 看模型版本、路由、prefill / decode、continuous batching、KV cache 和调度是否连成请求链路；Cache Isolation Strategy、Autoscaling Capacity Plan、Protection Policy Design、Release Governance Design、Tenant Security Governance、Observability Trace Readiness、Cost Capacity Governance 和 Tradeoff Boundary Reasoning 则检查缓存隔离、容量、稳定性、发布、多租户、安全、trace、成本和工程取舍是否闭环。

面试表达：推理平台系统设计题可以按 “需求澄清 -> 目标与 SLO -> 总体架构 -> 请求链路 -> Model Registry / Router -> Cache -> Runtime / scheduler / KV -> autoscaling -> protection -> release -> tenant / security -> observability -> cost -> bottleneck and trade-off -> Inference Platform Design Gate” 展开。不要只回答 “API Gateway + K8s + vLLM”。

SLO Token Contract

一句话定义：SLO Token Contract 是把 TTFT、TPOT、p99、input tokens/s、output tokens/s、active sequences、上下文长度、timeout 和成本预算写成推理平台设计的硬约束。

KV Scheduler Capacity

一句话定义：KV Scheduler Capacity 是用活跃请求数、上下文 token、层数、KV heads、head dimension、元素字节数、KV 显存水位和调度策略判断平台是否接得住当前并发。

Inference Platform Design Gate

一句话定义：Inference Platform Design Gate 是判断推理平台系统设计是否满足覆盖率、SLO、KV 容量、trace、发布回滚、多租户安全、成本归因和无 P0 风险的综合门禁。

Inference Fault Diagnosis Audit

一句话定义：Inference Fault Diagnosis Audit 是把 TTFT、TPOT、p99、吞吐、错误率、timeout、KV cache、模型加载、streaming、cache、route、tool、质量和发布 diff 放进同一张推理故障排查表的方法。

核心直觉：推理故障是在线用户直接感知的问题，不能只看 QPS 或 GPU utilization。成熟平台要能从 request profile、scope slice、阶段 trace、KV cache、model runtime、route、cache、error taxonomy、tool trace、release diff 和应急动作串出根因。

常见审计维度：Inference Fault Evidence Coverage 检查证据是否齐全；Request Scope Slice 和 Trace Stage Coverage 检查 model、tenant、region、endpoint、runtime、streaming、输入 / 输出长度、版本和 gateway / router / queue / prefill / decode / stream span；TTFT Decomposition、TPOT Decode Health、Tail Latency Attribution 和 Throughput Token Capacity 检查首响、生成速度、尾延迟和 token 产能；Error Taxonomy Timeout Stage、KV Cache Pressure Guard、Model Artifact Load Readiness、Streaming Reliability、Cache Hit Key Governance、Route Trace Correctness、Tool Dependency Trace、Quality Release Diff Readiness 和 Inference Fault Diagnosis Gate 则检查错误、容量、加载、streaming、cache、路由、工具、质量、回滚和复盘。

面试表达：推理故障定位题要先判断影响范围，再拆请求链路；TTFT 看 gateway、router、cache、queue、tokenize、prefill 和首 token；TPOT 看 decode、active sequences、KV cache、显存带宽和 runtime；p99 看慢请求 trace 和切片；错误率先分类，再决定限流、降级、回滚、摘除实例或扩容。

Inference Fault Evidence Coverage

一句话定义：Inference Fault Evidence Coverage 衡量请求画像、范围切片、阶段 trace、KV、模型 runtime、路由、cache、错误、工具、发布 diff、应急动作和诊断是否齐全。

Request Scope Slice

一句话定义：Request Scope Slice 是按 model、tenant、region、endpoint、runtime、streaming、input length bucket、output length bucket 和 release version 切分推理故障影响范围。

Trace Stage Coverage

一句话定义：Trace Stage Coverage 是检查 gateway、auth、router、cache、queue、tokenize、prefill、first decode、decode、stream、safety 和 downstream 是否都有 span。

TTFT Decomposition

一句话定义：TTFT Decomposition 是把首 token 延迟拆成前置服务、cache、queue、tokenize、prefill、first decode 和 first flush，避免把所有首响变慢都归因于模型。

TPOT Decode Health

一句话定义：TPOT Decode Health 是用 decode time、output tokens、active sequences、KV cache、GPU 负载、sampling 和 runtime 版本解释每 token 生成速度。

Tail Latency Attribution

一句话定义：Tail Latency Attribution 是用 p99 / p50、慢请求 trace、长上下文、租户、endpoint、instance、tool call 和网络切片定位尾延迟。

Throughput Token Capacity

一句话定义：Throughput Token Capacity 是同时检查 QPS、input tokens/s、output tokens/s、per GPU output tokens/s 和有效吞吐，而不是只看请求数。

Error Taxonomy Timeout Stage

一句话定义：Error Taxonomy Timeout Stage 是把错误分成 4xx、429、5xx、timeout、OOM、model load、tool failure、safety block，并要求 timeout 记录发生阶段。

KV Cache Pressure Guard

一句话定义：KV Cache Pressure Guard 是用 KV pressure、free blocks、eviction、admission reject、fragmentation 和 active sequences 判断 KV 是否成为瓶颈。

Model Artifact Load Readiness

一句话定义：Model Artifact Load Readiness 是检查 manifest、weight checksum、tokenizer、config、runtime compatibility、quantization compatibility、权限和 warmup 是否完整。

Streaming Reliability

一句话定义：Streaming Reliability 是观测 stream open、server abort、client disconnect、idle timeout、tokens before abort 和 cancellation cleanup。

Cache Hit Key Governance

一句话定义：Cache Hit Key Governance 是检查 cache hit rate、cache key 是否绑定 model、tokenizer、prompt、安全策略、tenant、permission 和 TTL / eviction 策略。

Route Trace Correctness

一句话定义：Route Trace Correctness 是检查候选模型、选择模型、路由原因、灰度比例、租户策略、endpoint 健康、fallback chain 和 trace 是否完整。

Tool Dependency Trace

一句话定义：Tool Dependency Trace 是检查 tool schema、权限决策、timeout、retry、下游状态、错误消息和 tool call trace 是否完整。

Quality Release Diff Readiness

一句话定义：Quality Release Diff Readiness 是把模型、prompt、runtime、route、cache、safety、sampling diff 和质量指标、线上反馈、回滚证据关联起来。

Inference Fault Diagnosis Gate

一句话定义：Inference Fault Diagnosis Gate 是判断推理故障定位是否有完整证据、owner、runbook、应急动作、回滚、postmortem 和无 P0 风险的综合门禁。

典型硬阻断：只看 QPS 和 GPU utilization；没有 request trace；TTFT 不拆 queue / prefill；TPOT 不看 decode 和 KV；p99 没有慢请求样本；吞吐只看请求数；错误没有分类；timeout 没有阶段；KV pressure 不可见；模型加载不校验 checksum / tokenizer / runtime；streaming abort 不区分客户端和服务端；cache key 不绑定版本；route trace 缺失；tool 失败被归因成模型差；质量下降没有 release diff；没有最终 Inference Fault Diagnosis Gate。

部署排障常见问题

一句话定义：部署排障是从质量、延迟、吞吐、显存和稳定性多个维度定位线上推理异常。

常见质量问题：tokenizer 不一致、chat template 错误、EOS 配置错误、量化退化、采样参数不合理、prompt 被截断。

常见性能问题：batch 太小导致 GPU 利用率低，batch 太大导致排队延迟高，KV Cache 占满显存，长请求阻塞短请求，prompt cache 命中率低。

常见稳定性问题：OOM、显存碎片、请求超时、流式连接断开、不同版本模型和 tokenizer 混用。

面试表达：LLM 部署排障要先区分质量问题和系统问题，再沿 tokenizer、模板、解码、KV Cache、调度和硬件指标逐层排查。

本章小结

本章围绕 LLM 推理优化和部署展开。第 8 章解释了推理和解码本身，本章进一步回答：如何让模型在真实服务中更快、更省、更稳。

高频结论如下：

1. LLM serving 的难点来自动态输入输出、prefill/decode 分离、KV Cache 和高并发调度。
2. throughput 衡量整体产能，latency 衡量单请求体验，线上要同时看平均值和 P95/P99。
3. TTFT 主要受排队和 prefill 影响，TPOT 主要受 decode、KV Cache 和显存带宽影响。
4. dynamic batching 用等待窗口换吞吐，continuous batching 在 decode 过程中动态加入和移出请求。
5. KV Cache 是推理显存大户，PagedAttention 用分页式管理减少浪费并支持高吞吐 serving。
6. prompt cache 和 prefix cache 复用公共前缀，主要优化重复 prefill 成本。
7. FlashAttention 是精确 attention 的 IO-aware 实现，核心是减少 HBM 读写和中间矩阵物化。
8. decode 阶段经常是 memory bandwidth bottleneck，不只是算力瓶颈。
9. 量化能降低显存和带宽成本，但必须用校准数据和业务评测验证质量损失。
10. GPTQ 和 AWQ 都是常见 LLM PTQ 方法，前者强调近似二阶误差补偿，后者强调 activation-aware 保护重要权重。
11. vLLM、SGLang、TensorRT-LLM、TGI、Triton 等框架各有定位，选型要结合硬件、模型、并发、维护状态和延迟目标。
12. 生产排障要同时检查 tokenizer、chat template、EOS、解码参数、量化、KV Cache、调度和硬件指标。
13. 推理性能事故要把 TTFT regression、TPOT regression、KV pressure、prompt cost drift、cache effectiveness 和 serving gate 放在同一张表里判断。
14. runtime 选型要用 Model Serving Runtime Selection Audit 统一比较候选，而不是只看框架名、单点 tokens/s 或历史经验。
15. Prefill / Decode / KV 资源画像要把 token 分布、phase latency、KV 显存、prefix cache、长上下文、多租户、streaming、observability 和成本放进同一套上线门禁。

下一章，我们进入分布式训练系统。

J. 分布式训练系统

条目：Distributed Training、Data Parallelism、DDP、Gradient Synchronization、All-Reduce、Reduce-Scatter、All-Gather、Tensor Parallelism、Column Parallel、Row Parallel、Pipeline Parallelism、Micro-batch、Pipeline Bubble、1F1B、ZeRO、ZeRO-1、ZeRO-2、ZeRO-3、FSDP、Activation Checkpointing、Gradient Accumulation、Mixed Precision Training、Loss Scaling、Offloading、Parameter Sharding、Optimizer State Sharding、Communication Overlap、NCCL、Node、Rank、World Size、Global Batch Size、MFU、GPU Utilization。

第二轮训练稳定性补充条目：Training Stability、Training Stability Gate、Loss Spike Audit、Non-Finite Loss Rate、Effective Label Tokens、LR Continuity、Rank Loss Skew。

第二轮分布式事故补充条目：Distributed Incident、Distributed Incident Gate、Rank Step Alignment、Collective Mismatch、Token Shard Imbalance、Straggler Ratio、Communication Ratio、Checkpoint Shard Coverage、Resume Continuity、Pipeline Bubble Ratio。

第二轮 AI Infra 总览补充条目：AI Infra Overview Audit、Compute Accelerator Readiness、Network Communication Readiness、Storage Data Checkpoint Readiness、Scheduler Resource Governance、Training Platform Reproducibility、Inference Platform SLO Readiness、Data Platform Lineage Quality、Model Artifact Registry Governance、Eval Experiment Tracking Coverage、Observability Signal Coverage、Security Governance Coverage、Cost Capacity Governance、Developer Self-Service Readiness、AI Infra Boundary Clarity、MLOps LLMops Platform Boundary Clarity、Algorithm Infra Collaboration Readiness、AI Infra Overview Gate。

第二轮 AI Infra 边界补充条目: AI Infra MLOps LLMOps Boundary Audit, AI Infra Scope Accuracy, MLOps Lifecycle Accuracy, LLMOps Application Accuracy, Platform Engineering DX Accuracy, DevOps SRE Boundary Accuracy, Data Platform Boundary Accuracy, Model Platform Boundary Accuracy, Primary Owner Clarity, Interface Contract Coverage, Artifact Lineage Handoff, Observability SLO Handoff, Security Cost Governance Handoff, Lifecycle Stage Mapping, Anti Tool Name Confusion, Incident Routing Accuracy, Collaboration Handoff Readiness, Boundary Gate。

第二轮 AI 加速器补充条目: Accelerator Selection Audit, Peak Compute Fit, Memory Capacity Fit, Memory Bandwidth Fit, Interconnect Bandwidth Fit, Low Precision Support, Software Stack Maturity, Kernel Library Readiness, Distributed Communication Readiness, Training Memory Budget, Inference KV Cache Budget, Workload Hardware Fit, Cloud Self Build Decision, Cost Power Capacity Awareness, Profiling Observability Readiness, Fallback Portability Plan, Selection Risk Governance, Accelerator Selection Gate。

第二轮带宽瓶颈补充条目: Bandwidth Bottleneck Audit, VRAM Capacity Accounting, HBM Bandwidth Model, PCIe Transfer Awareness, NVLink Topology Awareness, NVSwitch All-to-All Awareness, Inter-Node Network Awareness, KV Cache Growth Accounting, Training State Memory Accounting, Communication Volume Accounting, Topology-Aware Parallel Group, Dataloader Storage IO Awareness, Checkpoint IO Awareness, Offload Penalty Awareness, Overlap Fusion Optimization, Observability Metric Coverage, Bandwidth Bottleneck Gate。

第二轮训练效率补充条目: Training Efficiency Audit, Tokens Throughput Accounting, Step Time Breakdown Coverage, GPU Utilization Interpretation, MFU Estimation, HFU Estimation, Model FLOPs Accounting, Hardware Peak Accounting, Communication Ratio Tracking, IO Dataloader Tracking, Checkpoint Overhead Tracking, Rank Skew Detection, Scaling Efficiency Tracking, Padding Waste Awareness, Recompute Overhead Awareness, Loss Correctness Coupling, Training Efficiency Gate。

第二轮任务画像补充条目: Task Profile Audit, Workload Type Classification, Resource Shape Completeness, Pretraining Profile Accuracy, SFT Iteration Profile Accuracy, RLHF Pipeline Profile Accuracy, Evaluation Reproducibility Profile, Serving SLO Profile, RAG Freshness Retrieval Profile, Agent Tool Runtime Profile, Multimodal Resource Profile, Scheduler Policy Fit, Observability Metric Fit, Cost Model Fit, Artifact Lineage Fit, Safety Governance Fit, Task Profile Gate。

第二轮 GPU 集群拓扑补充条目: GPU Cluster Topology Audit, Scale-Up Domain Fit, PCIe NUMA Locality, NVLink NVSwitch Locality, GPU NIC Affinity, Inter-Node Fabric Readiness, Rack Locality Awareness, Oversubscription Awareness, Collective Communication Fit, Parallel Group Placement, Storage Checkpoint Locality, Fault Domain Isolation, Power Cooling Capacity Fit, Resource Pool Isolation, Topology-Aware Scheduling, Observability Topology Coverage, GPU Cluster Gate。

第二轮网络通信补充条目: Network Communication Audit, Bandwidth Unit Accounting, Latency Jitter Tracking, RDMA Capability Fit, GPUDirect RDMA Path, InfiniBand Fabric Readiness, RoCE Congestion Losslessness, Ethernet Fallback Scope, Collective Operation Modeling, NCCL Topology Runtime Fit, AllReduce Cost Estimation, AllGather ReduceScatter Cost, Rank Straggler Detection, Packet Error Retransmit Tracking, Topology Congestion Awareness, Scheduler Network Locality, Network Communication Gate。

第二轮存储体系补充条目: Storage System Audit, Capacity Tier Fit, Throughput IOPS Fit, Local NVMe Cache Fit, Shared FS Metadata Fit, Object Store Authority Fit, Data Lake Governance Fit, Training Shard Format Fit, Small File Amplification Control, Dataloader Cache Hit Tracking, Checkpoint Write Recovery Fit, Model Weight Load Cache Fit, Artifact Lineage Metadata Fit, Consistency Commit Integrity, Security Compliance Fit, Lifecycle Cost Governance, Storage System Gate。

第二轮 checkpoint 生命周期补充条目: Checkpoint Lifecycle Audit, Checkpoint Object Completeness, Shard Layout Fit, Async Save Overlap, Write Bandwidth Fit, Metadata Commit Integrity, Checksum Manifest Validation, Restore Replay Readiness, Dataloader RNG State Capture, Distributed Rank State Capture, Retention Policy Fit, Lifecycle Cost Governance, Cross-Region Replication Fit, Security Access Control, Resume SLO Tracking, Failure Drill Coverage, Checkpoint Lifecycle Gate。

第二轮容器环境补充条目: Container Environment Audit, Image Digest Reproducibility, Base Layer Cache Fit, CUDA Driver Framework Compatibility, GPU Runtime Visibility, Dependency Lock Coverage,

Training Serving Image Separation、Image Size Startup Fit、Build Pipeline Smoke Test、Security Scan Signing、Secret Data Exclusion、Runtime Hardening Fit、Registry Access Governance、Environment Metadata Capture、Multi-Tenant Mount Isolation、NCCL RDMA Runtime Fit、Container Environment Gate。

第二轮 Kubernetes GPU 资源管理补充条目：Kubernetes GPU Resource Management Audit、Device Plugin Readiness、GPU Extended Resource Fit、GPU Request Limit Integrity、Gang Scheduling Readiness、Fragmentation Control、Topology Aware Placement、Node Label Affinity Fit、Taint Toleration Fit、MIG Sharing Policy Fit、Quota Namespace Governance、Training Operator Readiness、Inference Service Readiness、GPU Monitoring Mapping、Multi-Tenant Isolation、Pending Troubleshooting Coverage、Kubernetes GPU Gate。

第二轮训练调度治理补充条目：Training Scheduler Governance Audit、Workload Resource Shape、Queue Policy Coverage、Priority Governance、Quota Usage Control、Fair Share Accounting、Gang Scheduling Fit、Backfilling Safety、Preemption Checkpoint Safety、Scheduler Fragmentation Control、Scheduler Topology Awareness、Checkpoint Scheduler Coupling、Cost Attribution Control、Scheduler Observability、Failure Requeue Readiness、Anti Starvation Control、Training Scheduler Gate。

第二轮多租户 GPU 调度系统设计补充条目：Multi Tenant GPU Scheduler Design Audit、Tenant Queue Contract、Quota Borrowing Policy、Dominant Fairness Accounting、Gang Scheduling Fit、Topology Aware Placement、Scheduler Fragmentation Control、Preemption Checkpoint Safety、Low Priority Reclaim、Inference SLO Isolation、Heterogeneous GPU Policy、Resource Snapshot Health、Placement Explainability、Cost Utilization Attribution、Scheduler Observability Audit、Tradeoff Boundary Reasoning、Multi Tenant GPU Scheduler Gate。

第二轮多租户隔离补充条目：Multi-Tenant Isolation Audit、Resource Quota Isolation、Identity Namespace Binding、RBAC ABAC Permission Fit、Data Access Boundary、Data Classification Lineage、Network Policy Isolation、Runtime Security Boundary、Image Supply Chain Boundary、Secret Scope Rotation、Logs Trace Redaction、Cost Attribution Coverage、Prod Experiment Separation、Cross Tenant Sharing Governance、Audit Evidence Readiness、Blast Radius Control、Multi-Tenant Isolation Gate。

第二轮集群容量规划补充条目：Cluster Capacity Planning Audit、Workload Forecast Coverage、GPU Count Capacity Fit、GPU Type Mix Fit、Training Queue SLO Fit、Serving Peak SLO Fit、Network Bandwidth Capacity Fit、Storage Throughput Capacity Fit、Storage Capacity Lifecycle Fit、Utilization Headroom Fit、Failure Redundancy Fit、Growth Forecast Fit、Cost Budget Fit、Pool Separation Fit、Quota Burst Governance、Observability Forecast Feedback、Cluster Capacity Planning Gate。

第二轮训练平台总览补充条目：Training Platform Lifecycle Audit、TrainingJob Contract、Config Validation、Resource Quota Binding、Image Code Reproducibility、Dataset Lineage Permission、Launcher Distributed Fit、Checkpoint Resume Policy、Observability Events Metrics、Experiment Tracking Lineage、Artifact Registry Linkage、Failure Recovery Classification、Security Audit Control、Cost Attribution Control、Developer Self Service、Lifecycle State Machine、Training Platform Gate。

第二轮训练任务提交系统补充条目：Training Submission Audit、TrainingJob Schema Completeness、Required Field Coverage、Image Digest Binding、Code Commit Diff Binding、Dataset Version Permission、Structured Command Safety、Final Config Snapshot、Distributed Resource Consistency、Checkpoint URI Resume Validation、Logging Metrics Destination、Queue Priority Policy Binding、Quota Dry Run Admission、Idempotency Duplicate Control、Template Version Governance、Submit State Machine Validity、Submission Audit Trace、Training Submission Gate。

第二轮分布式启动器补充条目：Distributed Launcher Audit、Resource World Size Consistency、Rank Local Rank Mapping、Rendezvous Endpoint Readiness、GPU Binding Visibility、Launcher Adapter Fit、Torchrun Argument Fit、DeepSpeed Config Fit、Megatron Parallel Consistency、Ray Runtime Fit、Network NCCL Readiness、Log Rank Aggregation、Failure Stage Classification、Elastic Training Safety、Checkpoint Launcher Coupling、Scheduler Launcher Handoff、Launcher Audit Trace、Distributed Launcher Gate。

第二轮训练配置管理补充条目：Training Config Management Audit、Config Source Order、Final Config Snapshot Freeze、Required Config Field Coverage、Config Schema Type Validation、Batch Parallel Consistency、Immutable Version Binding、Environment Version Capture、Reproducibility Seed Boundary、

Experiment Tracking Linkage, Hyperparameter Sweep Trace, Checkpoint Resume Compatibility, Release Config Coupling, Config Permission Approval, Config Diff Readiness, Config Audit Trace, Training Config Gate。

第二轮训练可观测性补充条目：Training Observability Audit、Structured Log Context、Rank Log Coverage、Metric Taxonomy Coverage、Metric Dimension Scope、Event Schema Lifecycle、State Machine Validity、Attempt Retry Traceability、Trace Phase Coverage、Dashboard Diagnostic Readiness、Alert Rule Readiness、Anomaly Detection Rules、Experiment Tracking Linkage、Retention Cost Governance、Privacy Redaction Access、Cost Attribution Signal、Training Observability Gate。

第二轮 checkpoint 策略补充条目：Checkpoint Strategy Audit、RPO RTO Budget、Save Trigger Policy、Checkpoint Scope Separation、Async Snapshot Consistency、Manifest Commit Integrity、Shard Metadata Resharding、Restore Selection Policy、Preemption Checkpoint Coupling、Evaluation Best Linkage、Release Checkpoint Readiness、Retention Tier Lifecycle、Storage Cost Budget、Checkpoint Monitoring Alerts、Restore Drill Coverage、Permission Release Protection、Checkpoint Strategy Gate。

第二轮训练容错补充条目：Training Fault Tolerance Audit、Fault Taxonomy Coverage、Retryability Classification、Retry Budget Bounded、Retry Backoff Policy、Checkpoint Resume Readiness、Node Failure Reschedule、GPU Health Isolation、Communication Hang Detection、OOM Handling Policy、Data Permission Fast Fail、Checkpoint Commit Safety、Numeric Instability Stop Rollback、Diagnosis Report Evidence、Blacklist Precision、User Action Clarity、Training Fault Tolerance Gate。

第二轮训练数据供给补充条目：Training Data Supply Audit、Dataset Manifest Integrity、Shard Format Size Fit、Small File Amplification Control、Streaming Prefetch Retry、Local Cache Policy、Distributed Cache Policy、Shuffle Reproducibility、Rank Shard Assignment、Data Locality Fit、Dataloader Parallelism Fit、H2D Copy Readiness、Bad Sample Threshold、Data Supply Observability、Checkpoint Dataloader State、Permission Version Governance、Training Data Supply Gate。

第二轮数据平台供给链路补充条目：Data Platform Supply Chain Audit、Source Registry Coverage、Raw Data Lake Integrity、Cleaning Policy Versioning、Dedup Contamination Guard、PII Redaction Coverage、Quality Scoring Calibration、Dataset Builder Recipe、Dataset Manifest Completeness、Dataset Version Immutability、Streaming Shard Readiness、Permission Purpose Binding、Data Lineage Coverage、Data Platform Supply Observability、Read Throughput Fit、Data Wait Fit、Data Platform Supply Gate。

第二轮数据版本血缘质量监控补充条目：Data Version Lineage Quality Audit、Dataset Version Contract、Manifest Checksum Integrity、Immutable Snapshot Policy、Delta Version Diff、Lineage Graph Completeness、Impact Analysis Readiness、Schema Quality Gate、Content Quality Gate、Data Safety Compliance Gate、Distribution Drift Monitoring、Train Eval Leakage Monitoring、Annotation Quality Monitoring、Synthetic Data Monitoring、Quality Alert Routing、Training Model Version Linkage、Data Version Quality Gate。

第二轮模型仓库治理补充条目：Model Artifact Registry Audit、Model Version Contract、Weight Manifest Integrity、Safe Weight Format、Tokenizer Config Binding、Adapter Base Compatibility、Merge Lineage Completeness、Quantization Eval Gate、Runtime Compatibility Fit、Eval Report Linkage、Safety Permission Gate、Release Status Governance、Rollback Alias Readiness、Artifact Lineage Completeness、Load Cache Readiness、Lifecycle Retention Policy、Model Registry Gate。

第二轮 Artifact 管理补充条目：Artifact Management Audit、Artifact Metadata Contract、Dataset Artifact Manifest、Checkpoint Artifact Recoverability、Eval Report Reproducibility、Deployment Package Completeness、Release Manifest Integrity、Checksum Integrity Gate、Artifact Store Metadata Split、Permission Access Control、Promotion Gate Readiness、Experiment Tracking Linkage、Rollback Artifact Readiness、Deletion Dependency Safety、Artifact Management Gate。

第二轮实验追踪补充条目：Experiment Tracking Audit、Run Metadata Contract、Parameter Config Capture、Metric Curve Step Capture、Log Run Linkage、Sample Result Diagnosis、Code Version Reproducibility、Data Version Reproducibility、Model Checkpoint Linkage、Environment Capture、Prompt Eval Config Capture、Artifact Output Linkage、Cost Attribution、Run State Transition、Search Index Readiness、Lineage Graph Readiness、Decision Note Capture、Experiment Tracking Gate。

第二轮评估平台补充条目：Evaluation Platform Audit、Eval Dataset Contract、Offline Batch Reproducibil-

ity、Metric Definition Versioning、LLM Judge Calibration、Human Review Quality Control、Pairwise Blinding Randomization、Online Eval Guardrail、Release Gate Readiness、Eval Report Completeness、Slice Regression Detection、Regression Suite Coverage、Eval Job Scheduler Reliability、Eval Cache Key Integrity、Eval Observability、Artifact Tracking Linkage、Evaluation Platform Gate。

第二轮特征向量基础设施补充条目：Feature Embedding Index Infrastructure Audit、Feature Definition Contract、Offline Online Consistency、Point in Time Correctness、Embedding Version Contract、Chunk Embedding Lineage、Vector Index Build Readiness、ANN Quality Latency Gate、Metadata Permission Filter、Shadow Index Switch Readiness、Retrieval Trace Completeness、Feature Embedding Lineage Graph、Multi Tenant Isolation、Quality Monitoring Metrics、Cost Capacity Governance、RAG Agent Integration、Feature Embedding Index Gate。

第二轮 AI Infra 可观测性补充条目：AI Infra Observability Audit、Signal Inventory Coverage、Metric Contract Completeness、SLO Error Budget Readiness、Latency Quantile Guard、Trace Span Coverage、Log Event Structure、Correlation ID Coverage、Cardinality Budget Control、Training Observability、Inference Observability、Data Quality Observability、RAG Agent Observability、Cost Observability、Privacy Redaction Retention、Alert Actionability、Observability Platform Gate。

第二轮训练故障定位补充条目：Training Fault Diagnosis Audit、Training Fault Evidence Coverage、Lifecycle Event Timeline、Rank Log Completeness、Loss Anomaly Diagnosis、NaN Inf Guard、Update Health Check、Hang Straggler Detection、OOM Phase Memory Evidence、Communication Bottleneck Diagnosis、IO Data Bottleneck Diagnosis、Checkpoint Integrity、Training Resume Continuity、Data Fault Isolation、Config Code Diff Readiness、Minimal Reproduction Readiness、Training Fault Diagnosis Gate。

第二轮 SLO 值班体系补充条目：SLO Oncall Audit、SLI Contract Completeness、SLO Target Measurability、SLA Boundary Clarity、Inference SLO Coverage、Training SLO Coverage、Data Eval SLO Coverage、Error Budget Accounting、Burn Rate Alerting、Alert Actionability、Oncall Ownership Escalation、Incident Severity Routing、Runbook Executability、Change Event Linkage、Mitigation Rollback Authority、Postmortem Action Closure、SLO Cost Tradeoff Gate。

第二轮成本治理补充条目：AI Infra Cost Governance Audit、Usage Metering Coverage、Cost Attribution Labels、GPU Cost Efficiency、Training Waste Control、Inference Unit Cost、Cache Savings Accounting、Storage Lifecycle Governance、Network Egress Governance、Artifact Dependency Safety、Budget Quota Enforcement、Cost Anomaly Alerting、Tenant Model Chargeback、Dashboard Drilldown Readiness、Optimization Recommendation Trace、SLO Quality Cost Tradeoff、Cost Governance Gate。

第二轮资源利用率治理补充条目：Resource Utilization Optimization Audit、Resource Metric Contract、Effective Utilization Accounting、Fragmentation Control、Topology Binpacking Readiness、Colocation SLO Guard、Preemption Checkpoint Safety、Low Priority Reclaim Policy、Elastic Training Safety、Inference Capacity Utilization、Warm Pool Headroom Policy、Quota Fairness Borrowing、Scheduler Observability、Dashboard Drilldown Readiness、Reliability Headroom Guard、Cost SLO Tradeoff Control、Resource Utilization Gate。

第二轮安全治理补充条目：AI Infra Security Governance Audit、Identity Workload Binding、RBAC ABAC Policy Fit、Secret Lifecycle Management、Data Compliance Classification、PII Sensitive Data Control、Tenant Isolation Boundary、Model Access Control、Model Output Safety Gate、RAG Permission Enforcement、Agent Tool Safety Gate、Supply Chain Integrity、Model Artifact Integrity、Runtime Security Boundary、Log Trace Privacy Governance、Audit Incident Response Readiness、Security Governance Gate。

第二轮审计变更治理补充条目：Audit Change Governance Audit、Audit Event Schema Completeness、Critical Operation Audit Coverage、Audit Log Integrity Retention、Log Retention Policy Fit、Log Minimization Redaction、Trace Privacy Sampling、Incident Timeline Completeness、Impact Detection Response Metrics、Root Cause Systemic Analysis、Postmortem Action Item Closure、Change Record Completeness、High Risk Change Approval、Rollout Rollback Readiness、Change Freeze Error Budget Policy、Audit Postmortem Change Linkage、Governance Loop Gate。

第二轮训练平台安全补充条目：Training Platform Security Audit、Identity Runtime Binding、RBAC ABAC Policy Fit、Dataset Permission Purpose、Artifact Classification Propagation、Resource Quota Priority

Governance、Image Supply Chain Trust、Runtime Code Isolation、Secret Scope Rotation、Network Egress Control、Log Redaction Access、Model Artifact Permission、High Risk Approval、Audit Log Completeness、Audit Integrity Retention、Incident Response Readiness、Training Platform Security Gate。

第二轮训练平台系统设计补充条目：Training Platform System Design Audit、Requirement Clarification、TrainingJob Contract、Lifecycle State Machine、Admission Quota Policy、Scheduler Resource Fit、Distributed Launcher Fit、Data Config Lineage、Checkpoint Recovery Coupling、Observability Experiment Linkage、Fault Recovery Flow、Security Audit Governance、Cost Capacity Governance、Developer Self Service、Interface Artifact Contract、Tradeoff Boundary Reasoning、Training Platform Design Gate。

Distributed Training

一句话定义：distributed training 是把模型训练过程拆到多张 GPU、多个节点甚至多个集群上协同完成的方法。

为什么需要：大模型训练需要巨量参数、激活、梯度和 optimizer state。单卡通常放不下模型，也无法在可接受时间内完成训练，因此必须把数据、模型、计算和状态拆分到多卡。

核心问题：怎么拆参数，怎么拆 batch，怎么同步梯度，怎么减少显存，怎么隐藏通信，怎么让每张卡尽量不空转。

常见并行维度：数据并行、张量并行、流水线并行、专家并行、序列并行，以及 ZeRO/FSDP 这类状态分片方法。真实大模型训练通常是多种并行方式组合使用。

面试表达：分布式训练的本质是用通信换容量和速度。所有方案都在计算、显存、通信和工程复杂度之间做权衡。

Data Parallelism

一句话定义：data parallelism 是每张 GPU 保存一份完整模型副本，处理不同数据 batch，并在反向传播后同步梯度的并行方式。

工作流程：每张卡拿到不同 mini-batch；各自前向和反向；得到本地梯度；通过 all-reduce 聚合梯度；每张卡用相同梯度更新参数。

优点：概念简单，工程成熟，适合模型能放进单卡但数据和训练时间压力大的场景。

局限：每张 GPU 都要保存完整参数、梯度和 optimizer state。模型很大时，数据并行无法解决单卡显存放不下的问题。

面试表达：数据并行提升的是样本吞吐，不拆模型本身；如果单卡放不下模型，需要张量并行、流水并行或参数分片。

DDP

一句话定义：DDP 是 Distributed Data Parallel，PyTorch 中常用的数据并行训练框架。

工作方式：每个进程通常绑定一张 GPU，每个进程持有一份模型副本；反向传播时，DDP 把梯度按 bucket 聚合，并通过 all-reduce 同步。

为什么高效：DDP 可以在反向传播过程中边计算梯度边启动通信，让部分通信和计算重叠，减少等待时间。

适用场景：模型、梯度和 optimizer state 能放入单卡，但希望多卡加速训练。

常见坑：不同 rank 数据不一致、随机种子不一致、某些参数未参与 loss 导致同步异常、batch norm 或梯度累积配置错误。

面试表达：DDP 是成熟的数据并行方案，核心是每卡完整模型副本加梯度 all-reduce。

Gradient Synchronization

一句话定义：gradient synchronization 是多卡训练中把不同设备上的梯度聚合成一致结果的过程。

为什么需要：数据并行中每张卡看到不同样本，本地梯度不同。如果不同步，各卡参数会逐渐分叉，训练就不再等价于一个全局 batch。

常见方式：all-reduce 对梯度求和或求平均；reduce-scatter 把聚合后的梯度分片留在不同设备；parameter server 架构集中聚合。

影响：梯度同步是分布式训练的重要通信开销，模型越大、通信带宽越低，同步成本越明显。

面试表达：数据并行的关键通信就是梯度同步，它保证各个模型副本更新一致。

All-Reduce

一句话定义：all-reduce 是聚合多个设备上的张量，并把聚合结果同步回所有设备的通信操作。

典型用途：DDP 中同步梯度。每张卡都有一份梯度，all-reduce 后每张卡都得到全局聚合梯度。

常见算法：ring all-reduce、tree all-reduce。不同算法在带宽利用、延迟和节点拓扑上有不同表现。

成本来源：通信量与张量大小相关，也受 GPU 间互联、跨节点网络、bucket 大小和通信库实现影响。

面试表达：all-reduce 是数据并行的核心通信原语，用来让所有 rank 拿到一致的聚合结果。

Reduce-Scatter

一句话定义：reduce-scatter 是先对多个设备上的张量做聚合，再把聚合结果按分片分发到不同设备。

与 all-reduce 的关系：all-reduce 可以看成 reduce-scatter 加 all-gather。先聚合并分片，再把分片收集回所有设备。

用途：ZeRO、FSDP 等分片训练中常用 reduce-scatter 同步梯度，因为每张卡只需要保留自己负责的梯度分片。

面试表达：reduce-scatter 适合状态分片场景，它不像 all-reduce 那样让每张卡都保存完整结果。

All-Gather

一句话定义：all-gather 是从多个设备收集张量分片，并让每个设备都得到完整张量的通信操作。

用途：FSDP 或 ZeRO-3 中，参数被分片存储。某一层前向计算前，需要 all-gather 收集该层完整参数，计算结束后再释放。

代价：all-gather 会引入额外通信和显存峰值，因此需要和计算重叠、按层调度和及时释放。

面试表达：all-gather 常用于参数分片训练中临时恢复完整参数，是 FSDP/ZeRO-3 的关键通信。

Tensor Parallelism

一句话定义：tensor parallelism 是把单层内部的大矩阵计算切分到多张 GPU 上并行完成。

为什么需要：当单层矩阵太大或单卡算力不足时，可以把矩阵乘法拆到多卡。典型用于 Transformer 的 attention 和 MLP 层。

常见切法：按输出维度切的 column parallel，按输入维度切的 row parallel。Megatron-LM 中常把 MLP 和 attention 投影按这种方式拆分。

优点：能拆单层参数和计算，适合超大模型。缺点：层内通信频繁，对 GPU 互联带宽要求高。

面试表达：张量并行拆的是层内矩阵，解决单层太大或算力不足的问题，但代价是频繁通信。

Column Parallel

一句话定义：column parallel 是按权重矩阵输出维度切分，让不同 GPU 计算不同输出通道。

例子：线性层 $Y = XW$ 中，如果按 W 的列切分，每张 GPU 得到一部分输出 Y_i ，最后可以拼接成完整输出。

适用位置：MLP 的上投影、attention 的 QKV 投影等经常可以使用 column parallel。

通信特点：有些位置可以延迟 gather，让后续 row parallel 消化分片输出，从而减少不必要通信。

面试表达：column parallel 切输出通道，天然产生分片输出。

Row Parallel

一句话定义：row parallel 是按权重矩阵输入维度切分，让不同 GPU 处理输入的不同部分，再把结果求和。

例子：线性层 $Y = XW$ 中，如果按 W 的行切分，输入 X 也要对应切分。每张卡计算局部输出，最后通过 all-reduce 得到完整 Y 。

适用位置：MLP 的下投影、attention 的输出投影等常用 row parallel。

通信特点：通常需要一次 all-reduce 聚合局部输出。

面试表达：row parallel 切输入通道，局部结果需要通信求和。

Pipeline Parallelism

一句话定义：pipeline parallelism 是把模型不同层放到不同 GPU，并用 micro-batch 形成流水线执行。

为什么需要：当模型层数多、整模型放不进单卡时，可以让每张 GPU 只保存一部分层。

基本流程：把一个 global batch 切成多个 micro-batch。第一个 micro-batch 在 stage 1 完成后传给 stage 2，同时 stage 1 开始处理第二个 micro-batch。

优点：显著降低单卡参数显存。缺点：有 pipeline bubble，前后向调度复杂，stage 之间负载不均会拖慢整体速度。

面试表达：流水并行拆的是层，核心问题是如何用 micro-batch 填满流水线，减少 bubble。

Micro-batch

一句话定义：micro-batch 是把一个较大的 batch 切成多个小批次，用于梯度累积或流水线并行调度。

在流水并行中：micro-batch 数越多，流水线越容易填满，bubble 占比越小。但 micro-batch 太小会降低单卡计算效率。

在梯度累积中：多个 micro-batch 的梯度累积后再执行一次 optimizer step，可以模拟更大的 global batch。

面试表达：micro-batch 是流水并行和梯度累积的基本调度单位。

Pipeline Bubble

一句话定义：pipeline bubble 是流水线并行中某些 stage 因等待数据或等待梯度而空闲的时间。

为什么出现：流水线开始时后面的 stage 没有数据，结束时前面的 stage 已经无事可做。前后向阶段切换也会造成空转。

影响因素：pipeline stage 数、micro-batch 数、调度策略、各 stage 计算负载是否均衡。

优化方向：增加 micro-batch 数、使用 1F1B 调度、合理切分层、平衡每个 stage 的计算量和参数量。

面试表达：pipeline bubble 是流水并行的主要效率损失，micro-batch 越多通常 bubble 比例越低。

1F1B

一句话定义：1F1B 是 one-forward-one-backward 的流水线调度方式，稳定阶段中每个设备交替执行一个前向和一个反向 micro-batch。

为什么提出：朴素调度可能先跑完所有前向再跑所有反向，激活显存高，等待时间也大。1F1B 能降低激活峰值并提高流水线利用率。

特点：有 warmup、steady 和 cooldown 阶段。稳定阶段尽量让各 stage 持续工作。

面试表达：1F1B 是减少流水线空转和激活显存的常见调度策略。

ZeRO

一句话定义：ZeRO 是通过分片 optimizer states、gradients 和 parameters，降低数据并行显存冗余的方法。

为什么提出：普通数据并行中，每张卡都保存完整参数、梯度和 optimizer state。以 Adam 为例，optimizer state 往往比参数本身还占显存。

核心思想：数据并行保留全局 batch 扩展能力，但把冗余训练状态切到不同 GPU 上，让每张卡只保存一部分。

三个阶段：ZeRO-1 分片 optimizer states；ZeRO-2 进一步分片 gradients；ZeRO-3 进一步分片 parameters。

面试表达：ZeRO 不是新的优化器，而是消除数据并行中训练状态冗余的显存优化方法。

ZeRO-1

一句话定义：ZeRO-1 是只分片 optimizer states 的 ZeRO 阶段。

保存什么：参数和梯度仍然每卡完整保存，但 Adam 的一阶、二阶动量等 optimizer state 被切分到不同卡。

收益：实现相对简单，对通信模式影响较小，能减少 optimizer state 的冗余显存。

局限：参数和梯度仍完整复制，超大模型下节省不够。

面试表达：ZeRO-1 先切 optimizer state，是 ZeRO 中侵入性较低的一步。

ZeRO-2

一句话定义：ZeRO-2 是在 ZeRO-1 基础上进一步分片 gradients。

保存什么：optimizer states 和 gradients 分片，参数仍然每卡完整保存。

通信特点：反向传播后通过 reduce-scatter 等方式只保留每张卡负责的梯度分片。

收益：比 ZeRO-1 进一步降低显存，适合参数仍能复制但梯度和 optimizer state 压力较大的场景。

面试表达：ZeRO-2 切 optimizer state 和梯度，但参数仍完整复制。

ZeRO-3

一句话定义：ZeRO-3 是同时分片 optimizer states、gradients 和 parameters 的 ZeRO 阶段。

保存什么：每张卡只长期保存部分参数、梯度和 optimizer state。前向或反向需要某层参数时，通过 all-gather 临时收集，计算后释放。

收益：显存节省最大，可以训练远超单卡容量的模型。

代价：通信复杂度和通信量增加，性能依赖调度、预取、通信计算重叠和网络带宽。

面试表达：ZeRO-3 显存最省，但通信最重，是用通信换显存的典型方案。

FSDP

一句话定义：FSDP 是 Fully Sharded Data Parallel，把参数、梯度和 optimizer state 分片到多张 GPU 的训练方式。

与 ZeRO-3 的关系：FSDP 可以理解为 PyTorch 生态中类似 ZeRO-3 思想的全分片数据并行实现。

工作方式：参数平时以分片形式存在；进入某个模块前 all-gather 完整参数；完成前向/反向后释放完整参数；梯度通过 reduce-scatter 回到分片形式。

优势：显著降低单卡显存，支持更大模型训练；与 PyTorch 生态结合较深。

挑战：wrap 策略、prefetch、混合精度、checkpoint 保存、通信重叠都会影响效果。

面试表达：FSDP 的核心是按模块动态 all-gather 参数，用后释放，从而降低长期显存占用。

Activation Checkpointing

一句话定义：activation checkpointing 是少保存中间激活，在反向传播时重新计算，用计算换显存的方法。

为什么需要：训练中除了参数和 optimizer state，激活也会占大量显存，尤其是长序列、大 batch 和深层模型。

工作方式：前向时只保存部分 checkpoint 激活；反向时从最近 checkpoint 重新执行部分前向，恢复需要的中间激活。

优点：显著降低激活显存。缺点：增加额外计算，训练时间变长。

面试表达：activation checkpointing 是典型的 compute-memory tradeoff，用重算换显存。

Gradient Accumulation

一句话定义：gradient accumulation 是连续处理多个 micro-batch，累积梯度后再执行一次参数更新。

为什么需要：显存放不下大 batch 时，可以用多个小 micro-batch 模拟较大的 global batch。

等价条件：如果 loss 缩放、随机性、batch norm 等处理一致，梯度累积近似等价于更大的 batch。但 dropout、数据顺序和归一化层可能带来细微差异。

代价：optimizer step 频率下降，单次更新前需要更多前反向计算，wall-clock 可能变长。

面试表达：梯度累积解决的是显存不足下的有效 batch size 问题，不直接提高吞吐。

Mixed Precision Training

一句话定义：mixed precision training 是训练中混合使用 FP32、FP16、BF16 等精度，以降低显存、提高速度并保持数值稳定。

常见方式：大部分矩阵计算使用 FP16/BF16，部分敏感操作或主权重使用 FP32。

收益：减少显存和带宽，利用 Tensor Core 加速。

风险：低精度可能导致 overflow、underflow 或梯度精度不足。BF16 因动态范围大，通常比 FP16 更稳定。

面试表达：混合精度不是简单全低精度，而是在速度、显存和数值稳定之间折中。

Loss Scaling

一句话定义：loss scaling 是在 FP16 训练中放大 loss 或梯度，避免小梯度下溢为 0 的技巧。

为什么需要：FP16 动态范围有限，小梯度可能无法表示。把 loss 乘以 scale 后反向传播，再在更新前把梯度除回去，可以减少 underflow。

类型：静态 loss scaling 使用固定 scale；动态 loss scaling 根据是否出现 overflow 自动调整。

注意：BF16 动态范围接近 FP32，通常不太需要 loss scaling。

面试表达: loss scaling 主要是 FP16 训练的数值稳定技巧, 用于缓解梯度下溢。

Training Stability

一句话定义: training stability 是训练过程在数据、学习率、梯度、精度、mask、checkpoint 和分布式状态上保持可观测、可复现、可恢复的能力。

典型信号: loss spike、NaN / Inf、gradient norm 突增、AMP overflow、有效 label token 异常、学习率曲线跳变、train / eval 背离和 rank 间 loss 差异。

工程做法: 每 step 记录真实 lr、loss、grad norm、overflow、label token 数、batch id、rank loss 和 checkpoint / resume 状态; 异常时先保存 batch 和日志, 再做最小复现。

面试表达: 训练稳定性不是单个超参问题, 而是数据、优化、数值和分布式系统的联合可观测性问题。

Loss Spike Audit

一句话定义: loss spike audit 是对 loss 突然升高的 step 进行 batch、学习率、梯度、overflow、mask 和 rank 日志定位的审计流程。

关键证据: 第一次 spike step、对应 batch id、样本来源、sequence length、有效 label token、grad norm、真实 lr、overflow 状态和 per-rank loss。

面试表达: loss spike 的第一原则是保存证据。没有 step 和 batch id, 就很难判断是数据、数值还是系统问题。

Non-Finite Loss Rate

一句话定义: non-finite loss rate 是训练 step 中 loss、gradient 或参数出现 NaN / Inf 的比例。

为什么重要: 一旦出现非有限值, 后续状态通常会被污染。排查重点是第一次出现的位置, 而不是 NaN 扩散后的所有 step。

常见原因: 学习率过大、FP16 overflow、attention mask 全 -inf、loss 分母为 0、梯度爆炸、异常数据和 optimizer state 损坏。

面试表达: NaN / Inf 是硬门禁。定位顺序通常是输入、logits、loss、grad、param 和 optimizer state。

Effective Label Tokens

一句话定义: effective label tokens 是一个 batch 中真正参与 loss 的 label token 数。

为什么重要: SFT、packing 和 padding 场景下, 有效 label token 数直接决定 loss 分母和训练信号。它突然过低或为 0, 通常说明 mask、padding、截断或数据格式出错。

工程做法: 按 batch、rank、任务类型统计有效 label token 数, 并把异常低值写入训练日志和门禁。

面试表达: 训练 loss 不能只看数值, 还要看这个 loss 是由多少有效 token 计算出来的。

LR Continuity

一句话定义: LR continuity 是实际生效 learning rate 与 scheduler 根据 global step 预期 learning rate 的连续一致性。

常见事故: checkpoint resume 后 scheduler step 错、global step 错、gradient accumulation 改了但 warmup / decay 没改、optimizer state 没恢复。

面试表达: 不要只看配置里的 lr, 要记录每个 optimizer step 的真实 lr。resume 后第一步尤其要检查连续性。

Rank Loss Skew

一句话定义：rank loss skew 是分布式训练中不同 rank 的 loss 差异过大。

常见原因：某个 rank 的数据 shard 异常、随机种子不同、mask 错、batch 长度极端、硬件或通信问题。

工程做法：不要只记录 rank 0。训练日志至少要能抽样记录 per-rank loss、batch id、样本数量和有效 label token 数。

面试表达：rank loss skew 高时，排查方向不是全局调参，而是先找哪个 rank、哪个 batch、哪个 shard 出了问题。

Training Stability Gate

一句话定义：training stability gate 是把 non-finite、loss spike、gradient norm、overflow、effective label tokens、LR continuity、rank loss skew 和 checkpoint resume 状态合并成训练稳定性门禁。

用途：在大训练前的小规模 dry run、resume 演练、混合精度切换、数据版本变更和分布式扩容后，快速判断训练链路是否可继续放大。

面试表达：训练稳定性门禁要输出 failed gates 和 first bad step。它不是为了让训练永远不波动，而是让异常可定位、可复现、可回滚。

Distributed Incident

一句话定义：distributed incident 是分布式训练中由 rank 状态、数据 shard、collective 调用、通信、checkpoint、resume 或并行调度不一致引发的训练事故。

典型现象：训练 hang、NCCL timeout、吞吐骤降、某个 rank loss 异常、checkpoint 恢复后曲线跳变、pipeline stage 利用率低或多节点扩容后性能反而下降。

工程做法：不要只看 rank 0。每个 rank 至少记录 step、phase、batch id、token 数、loss、step time、data time、communication time、collective 序列、checkpoint shard 和 resume 状态。

面试表达：分布式事故的核心是系统一致性。排查要先证明所有 rank 是否处在同一个 step、同一个 collective 序列和同一个训练状态上。

Rank Step Alignment

一句话定义：rank step alignment 是检查所有 rank 在同一时间窗口内是否到达同一个训练 step。

常见事故：某个 rank 卡在 dataloader、某个 rank 提前结束、异常处理只在部分 rank 执行、resume 后 global step 不一致。

面试表达：训练卡死时，先看 per-rank step 和 phase。rank 0 正常不代表其他 rank 正常。

Collective Mismatch

一句话定义：collective mismatch 是不同 rank 的 all-reduce、all-gather、reduce-scatter、broadcast 等 collective 调用顺序或次数不一致。

常见来源：按 rank 写条件分支、某些 rank batch 为空、只在 rank 0 做保存或评估但同步不完整、异常分支没有让所有 rank 共同退出。

面试表达：collective 是分布式训练的同步契约，某个 rank 少一次 collective 就可能让其他 rank 永久等待。

Token Shard Imbalance

一句话定义：token shard imbalance 是不同 rank 分到的 token 数或序列长度差异过大。

为什么重要：大模型训练的真实工作量更接近 token 数和序列长度，而不是样本条数。按样本均匀可能仍然让某些 rank 分到超长 batch。

面试表达：数据并行要检查 per-rank token 数、平均长度、来源分布和 batch sampler，而不是只检查样本数。

Straggler Ratio

一句话定义：straggler ratio 是最慢 rank 的 step time 与典型 rank step time 的比值。

常见原因：数据读取慢、网络差、GPU 性能异常、batch 长度极端、共享存储抖动或某个节点环境不同。

面试表达：straggler 会拖慢全局同步。排查时要把 step time 拆成 data、compute 和 communication。

Communication Ratio

一句话定义：communication ratio 是通信时间占单步训练时间的比例。

用途：判断吞吐瓶颈是否主要来自 all-reduce、all-gather、reduce-scatter、跨节点通信、bucket 配置或计算通信重叠不足。

面试表达：通信占比高时，不要只调 NCCL 参数。还要看 batch size、bucket、拓扑、并行切分、overlap 和 straggler。

Checkpoint Shard Coverage

一句话定义：checkpoint shard coverage 是实际保存成功的分布式 checkpoint shard 占预期 shard 的比例。

常见事故：只保存 rank 0 权重、漏掉 optimizer state、FSDP / ZeRO shard 缺失、scheduler / scaler / sampler 状态丢失、保存过程中部分 rank 失败。

面试表达：没有定期 restore test 的 checkpoint 不可靠。分布式 checkpoint 要验证 shard、状态和 global step，而不是只看目录是否存在。

Resume Continuity

一句话定义：resume continuity 是 checkpoint 加载后 global step、learning rate、optimizer state、scheduler、AMP scaler 和数据位置是否与保存前连续。

常见事故：只恢复模型权重、scheduler 从头开始、optimizer 动量丢失、sampler 位置错、global step 偏移。

面试表达：resume 后 loss 跳变时，先检查训练状态完整性和固定 batch loss，而不是立刻改学习率或换数据。

Pipeline Bubble Ratio

一句话定义：pipeline bubble ratio 是 pipeline parallel 中 stage 空转时间占总调度窗口的近似比例。

影响因素：stage 数、micro-batch 数、stage 计算量是否均衡和 pipeline schedule。micro-batch 太少或 stage 切分不均都会让 bubble 吃掉收益。

面试表达：pipeline parallel 不是切得越细越好。要用 micro-batch 填充流水线，并平衡每个 stage 的计算时间。

Distributed Incident Gate

一句话定义：distributed incident gate 是把 rank step alignment、collective mismatch、token shard imbalance、straggler ratio、communication ratio、checkpoint shard coverage、resume continuity 和 pipeline bubble ratio 合并成分布式训练事故门禁。

用途：在单机多卡到多机多卡扩容、FSDP / ZeRO 切换、checkpoint 改造、resume 演练和 pipeline 并行调整后，快速判断训练是否可以继续放大。

面试表达：分布式事故门禁要输出 failed gates 和对应 rank / shard / stage。它的价值是把“我感觉卡了”变成可复现的 per-rank 证据链。

Offloading

一句话定义：offloading 是把部分参数、梯度、optimizer state 或激活转移到 CPU 内存或 NVMe，以缓解 GPU 显存压力。

价值：让有限 GPU 显存训练更大模型。

代价：CPU/GPU 或 NVMe/GPU 数据传输慢，会显著增加训练时间。offload 的收益取决于能否把传输和计算重叠。

适用场景：显存极度紧张、预算有限、可以接受训练变慢的场景。

面试表达：offloading 用速度换容量，是显存不够时的兜底方案，不是最高性能训练的首选。

Parameter Sharding

一句话定义：parameter sharding 是把模型参数分片存储到不同设备上，避免每张 GPU 保存完整参数。

用途：ZeRO-3、FSDP 和部分模型并行方案都会使用参数分片。

挑战：计算某一层时通常需要完整参数或对应分片参与，因此要设计 all-gather、预取和释放策略。

面试表达：参数分片降低长期显存，但会带来参数收集和通信调度问题。

Optimizer State Sharding

一句话定义：optimizer state sharding 是把优化器状态切分到不同设备上，减少 Adam 等优化器带来的显存冗余。

为什么重要：Adam 通常为每个参数维护一阶和二阶动量，再加上 master weights，训练状态显存可能是参数显存的数倍。

典型方法：ZeRO-1 就是分片 optimizer states；FSDP 也会配合优化器状态分片保存和加载。

面试表达：优化器状态是训练显存大户，切它通常能带来明显收益。

Communication Overlap

一句话定义：communication overlap 是让通信和计算同时进行，以减少分布式训练中的等待时间。

例子：DDP 在反向传播时，某些层梯度刚算完就启动 all-reduce，同时继续计算更前面层的梯度。FSDP 可以预取下一层参数，隐藏 all-gather 延迟。

关键条件：通信依赖关系、bucket 大小、网络带宽、计算耗时和框架调度都要合适。

面试表达：通信重叠不是减少通信量，而是把通信隐藏到计算背后，提升整体效率。

NCCL

一句话定义：NCCL 是 NVIDIA Collective Communications Library，用于 GPU 间高效集合通信。

常见操作：all-reduce、all-gather、reduce-scatter、broadcast。

为什么重要：多 GPU 训练效率高度依赖通信库。NCCL 会根据硬件拓扑、NVLink、PCIe、InfiniBand 等选择通信策略。

常见排障：rank 卡住、网络配置错误、拓扑性能差、超时、不同节点环境不一致。

面试表达：NCCL 是 NVIDIA GPU 分布式训练的核心通信库，很多多卡性能问题最终都要看通信链路。

Node

一句话定义：node 是分布式训练中的一台机器，通常包含多张 GPU、CPU、内存和网络接口。

单机多卡：通信通常依赖 NVLink、NVSwitch 或 PCIe。

多机多卡：还需要跨节点网络，例如 InfiniBand 或 RoCE，跨节点通信往往比节点内通信更慢。

面试表达：分布式训练要区分节点内通信和节点间通信，后者通常是更难优化的瓶颈。

Rank

一句话定义：rank 是分布式训练中每个进程的唯一编号。

常见概念：global rank 表示全局编号；local rank 表示当前节点内编号；rank 0 常用于日志、保存 checkpoint 或协调任务。

为什么重要：数据切分、设备绑定、通信分组和日志保存都依赖 rank。

面试表达：rank 是分布式训练进程身份，理解 rank 才能理解多进程如何协同。

World Size

一句话定义：world size 是参与同一个分布式训练作业的总进程数，通常等于总 GPU 数。

例子：4 台机器，每台 8 张 GPU，如果每张 GPU 一个进程，则 world size 为 32。

影响：world size 决定数据并行规模、通信组大小和 global batch size。

面试表达：world size 是全局并行进程数量，不是单机 GPU 数。

Global Batch Size

一句话定义：global batch size 是一次参数更新实际使用的总样本数。

常见公式：

$$\text{global batch size} = \text{micro batch size} \times \text{gradient accumulation steps} \times \text{data parallel world size}$$

重要性：global batch size 影响训练稳定性、学习率设置、收敛速度和泛化。扩大并行规模时，通常要同步调整 batch 和 learning rate schedule。

面试表达：多卡训练中要区分 micro batch、per-device batch 和 global batch，很多训练不稳定来自 batch 配置理解错误。

MFU

一句话定义：MFU 是 Model FLOPs Utilization，衡量训练中硬件理论算力有多少真正用于模型计算。

为什么比 GPU 利用率更有意义：GPU 利用率高只说明 GPU 忙，不代表忙在有效模型计算上。MFU 更关注实际完成的模型 FLOPs 与硬件理论峰值的比例。

影响因素：模型结构、batch size、序列长度、kernel 效率、通信开销、重算、padding 浪费和数据加载。

面试表达：MFU 是大模型训练效率的关键指标，比单纯看 nvidia-smi 的 GPU utilization 更有参考价值。

GPU Utilization

一句话定义：GPU utilization 是 GPU 在采样窗口内有多少时间处于忙碌状态的指标。

局限：GPU utilization 高不代表训练效率高。可能是低效 kernel、通信等待、内存搬运或小 batch 导致的低效忙碌。

如何结合使用：排查性能时同时看 MFU、tokens/sec、step time、通信时间、显存占用、数据加载耗时和网络指标。

面试表达：GPU utilization 是粗粒度指标，不能单独用来判断大模型训练效率。

Training Efficiency Audit

一句话定义：Training Efficiency Audit 是把 GPU utilization、tokens/s、step time、MFU、HFU、通信、I/O、checkpoint、rank skew、padding、重计算和 loss 正确性放进同一张训练效率检查表的方法。

核心直觉：GPU utilization 只说明 GPU 在采样窗口内有没有 kernel 在跑，不说明这些 kernel 是否高效，也不说明计算是否转化成有效模型训练。训练效率要同时回答三件事：训练快不快，硬件是否被有效使用，训练是否仍然正确。

常见维度：Tokens Throughput Accounting 看 global tokens/s 是否按真实非 padding token 统计；Step Time Breakdown Coverage 看 dataloader、H2D、forward、backward、communication、optimizer、checkpoint 和 misc 是否拆开；GPU Utilization Interpretation 避免把 GPU 忙碌当成最终结论；MFU Estimation 和 HFU Estimation 区分模型有效 FLOPs 与硬件实际 FLOPs；Model FLOPs Accounting 和 Hardware Peak Accounting 说明 FLOPs 与峰值口径；Communication Ratio Tracking、IO Dataloader Tracking 和 Checkpoint Overhead Tracking 检查通信、数据供给和保存尖刺；Rank Skew Detection 看最慢 rank 是否拖住全局 step；Scaling Efficiency Tracking 看多卡是否接近线性扩展；Padding Waste Awareness、Recompute Overhead Awareness 和 Loss Correctness Coupling 则防止“跑得快但浪费多或训练错”。

常见公式：端到端 token 吞吐可以写成

$$Q_{\{\mathrm{tok}\}} = \frac{B_{\{\mathrm{global}\}} S_{\{\mathrm{eff}\}}}{T_{\{\mathrm{step}\}}}$$

MFU 可以写成

$$U_{\{\mathrm{mfu}\}} = \frac{F_{\{\mathrm{model}\}}}{T_{\{\mathrm{step}\}}} \{N_{\{\mathrm{gpu}\}} F_{\{\mathrm{peak}\}}\}$$

HFU 可以写成

$$U_{\{\mathrm{hfu}\}} = \frac{F_{\{\mathrm{hw}\}}}{T_{\{\mathrm{step}\}}} \{N_{\{\mathrm{gpu}\}} F_{\{\mathrm{peak}\}}\}$$

其中， F_{model} 是模型有效 FLOPs， F_{hw} 是硬件实际执行 FLOPs， F_{peak} 是单 GPU 理论峰值 FLOPs/s。

面试表达：训练效率题要先说 GPU utilization 只是入口，再用 tokens/s、step time 分解、MFU/HFU、通信占比、I/O 占比、rank skew、扩展效率和 loss 曲线给出证据链。

Training Efficiency Gate

一句话定义：Training Efficiency Gate 是判断一个训练任务是否达到效率、扩展性和正确性最低要求的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{eff}\}} = \mathbf{1} \left[U_{\{\mathrm{mfu}\}} \geq \tau_{\{\mathrm{mfu}\}} \wedge E_n \geq \tau_{\{\mathrm{scale}\}} \wedge R_{\{\mathrm{comm}\}} \leq \tau_{\{\mathrm{comm}\}} \wedge R_{\{\mathrm{io}\}} \leq \tau_{\{\mathrm{io}\}} \wedge P_0 \leq \tau_{\{\mathrm{p0}\}} \right]$$

其中， E_n 是多卡扩展效率， R_{comm} 是通信占比， R_{io} 是数据与 checkpoint I/O 占比， P_0 是 P0 级训练风险数量。

典型硬阻断：只报告 GPU utilization；不统计真实 token 数；没有 step time 分解；MFU / HFU 口径不清；硬件峰值按错误精度计算；通信或 I/O 占比过高；rank skew 严重；padding 或重计算浪费未量化；checkpoint 周期性阻塞；loss、梯度或数值稳定性异常但仍把吞吐当成功。

面试表达：训练效率门禁的目标不是追求单一指标最高，而是证明“有效 token 吞吐、模型有效 FLOPs、硬件利用、通信/I/O、扩展效率和训练正确性”同时过线。

Task Profile Audit

一句话定义：Task Profile Audit 是把预训练、SFT、RLHF、评估、推理、RAG、Agent 和多模态任务的资源、调度、观测、成本、血缘和治理差异转成可检查指标表的方法。

核心直觉：AI Infra 服务的是不同工作负载，而不是抽象的“模型”。预训练重点是大规模训练吞吐、通信、数据供给和 checkpoint；SFT 重点是高频实验、数据版本和评估；RLHF 重点是 SFT、reward model、policy、reference model、rollout 和偏好数据的多阶段编排；评估重点是 benchmark、模型版本、样本级结果和可复现性；推理重点是 TTFT、TPOT、KV cache、batching、扩缩容和 SLO；RAG 重点是知识库、embedding、索引、retrieval、rerank、权限和引用；Agent 重点是工具、状态、trace、权限、预算和失败恢复。

常见公式：工作负载样本可以写成

$a_i = (w_i, s_i, r_i, c_i, m_i, n_i, d_i, l_i, o_i, g_i, z_i)$

资源画像可以写成

$\mathbf{v}_i = (C_i, M_i, N_i, D_i, L_i, R_i, K_i, S_i)$

其中， C_i 是 compute， M_i 是 memory， N_i 是 network， D_i 是 data / storage， L_i 是 latency， R_i 是 reliability， K_i 是 cost， S_i 是 security / governance。

面试表达：任务画像题不要只背“预训练重、推理快、Agent 复杂”，而要说明每类任务该进入什么资源池、用什么调度策略、看什么指标、如何算成本、失败后如何恢复。

Workload Type Classification

一句话定义：Workload Type Classification 是先判断任务属于预训练、SFT、RLHF、评估、推理、RAG、Agent、多模态或混合链路，再决定资源和平台策略的步骤。

常见坑：把所有任务都标成 generic AI job，会导致预训练缺 gang scheduling、推理缺 SLO、Agent 缺 trace 和权限边界。

Resource Shape Completeness

一句话定义：Resource Shape Completeness 是检查任务是否同时描述 compute、memory、network、data / storage、latency、reliability、cost 和 security / governance。

面试表达：资源画像不完整时，平台很容易只按 GPU 数量调度，忽略 KV cache、checkpoint、数据更新、人工评估和权限审计。

Pretraining Profile Accuracy

一句话定义：Pretraining Profile Accuracy 是检查预训练画像是否覆盖 tokens/s、MFU、step time、通信、数据供给、checkpoint、拓扑感知和容错。

典型硬阻断：没有 checkpoint 策略、没有数据版本、没有拓扑感知、只报告 GPU 数量或 GPU utilization。

SFT Iteration Profile Accuracy

一句话定义：SFT Iteration Profile Accuracy 是检查 SFT 是否覆盖数据集版本、prompt / template 版本、adapter artifact、自动评估和可复现实验。

面试表达：SFT 不是缩小版预训练，核心诉求是高频、可比较、可回滚的实验迭代。

RLHF Pipeline Profile Accuracy

一句话定义：RLHF Pipeline Profile Accuracy 是检查 SFT model、reward model、policy model、reference model、rollout、偏好数据和安全评估是否被统一追踪。

常见坑：只写 PPO / DPO 算法名，不统计 rollout 吞吐、reward 评分成本、多模型显存和阶段间 artifact。

Evaluation Reproducibility Profile

一句话定义：Evaluation Reproducibility Profile 是检查评估任务是否记录 benchmark 版本、模型版本、judge 口径、样本级输出、随机种子和回放条件。

面试表达：评估平台的关键不是跑完，而是可比较、可复现、可解释。

Serving SLO Profile

一句话定义：Serving SLO Profile 是检查在线推理是否覆盖 TTFT、TPOT、p95 / p99、错误率、队列、KV cache、batching、自动扩缩容和降级。

常见公式：SLO 通过率可以写成

$$R_{\{\mathrm{slo}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathrm{TTFT}_i \leq \tau_{\{\mathrm{ttft}\}} \wedge \mathrm{TPOT}_i \leq \tau_{\{\mathrm{tpot}\}}]$$

RAG Freshness Retrieval Profile

一句话定义：RAG Freshness Retrieval Profile 是检查 RAG 是否覆盖文档更新延迟、embedding 版本、索引版本、召回、rerank、引用、权限过滤和 RAG trace。

常见坑：只估算 LLM token 成本，忽略文档解析、索引构建、检索质量和引用可信度。

Agent Tool Runtime Profile

一句话定义：Agent Tool Runtime Profile 是检查 Agent 是否覆盖工具调用、任务状态、trace、最大步数、工具预算、权限、人审和失败恢复。

面试表达：Agent 成本和可靠性不能只看一次模型调用，要看完整调用链和工具执行结果。

Multimodal Resource Profile

一句话定义：Multimodal Resource Profile 是检查图像、音频、视频、OCR 等多模态任务是否覆盖输入体积、解码预处理、batch 组装、显存、存储吞吐和质量评估。

常见坑：把视频或文档任务当普通文本请求，只算 LLM token，不算文件解析、帧采样、OCR / ASR 和存储吞吐。

Scheduler Policy Fit

一句话定义：Scheduler Policy Fit 是检查不同任务是否使用匹配的调度策略，例如预训练 gang scheduling、SFT 配额队列、评估批处理、推理 SLO 队列和 Agent deadline / budget 控制。

Observability Metric Fit

一句话定义：Observability Metric Fit 是检查 metrics、logs、traces、events 是否覆盖当前任务真正的瓶颈，而不是只画通用 GPU utilization。

Cost Model Fit

一句话定义：Cost Model Fit 是检查成本模型是否拆开 GPU、存储、网络、人工、API / judge / tool 调用和失败重试。

成本可以写成：

$$K_i = K_{\{\mathrm{gpu}\}, i} + K_{\{\mathrm{storage}\}, i} + K_{\{\mathrm{network}\}, i} + K_{\{\mathrm{human}\}, i} + K_{\{\mathrm{api}\}, i}$$

Artifact Lineage Fit

一句话定义：Artifact Lineage Fit 是检查数据、代码、镜像、模型、adapter、checkpoint、eval report 和 deployment package 是否能从任务画像中追溯。

Safety Governance Fit

一句话定义：Safety Governance Fit 是检查任务是否有 owner、权限、PII 策略、风险评审、审计日志和高风险动作治理。

Task Profile Gate

一句话定义：Task Profile Gate 是判断一个任务画像是否达到平台设计和上线讨论最低门槛的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{task}\}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \ \mathrm{and} \ R_{\mathrm{slo}}\geq \tau_{\{\mathrm{slo}\}} \ \mathrm{and} \ P_0=0\right]$$

其中， C_j 是第 j 个审计指标覆盖率， R_{slo} 是在线请求 SLO 通过率， P_0 是 P0 级风险数量。

典型硬阻断：任务类型泛化、资源画像缺维度、预训练缺 checkpoint、SFT 缺数据版本、RLHF 缺 rollout、评估不可复现、推理缺 TTFT / TPOT、RAG 忽略新鲜度、Agent 缺 trace、多模态 payload 无上限、调度只有一个队列、成本只按 token 估算、artifact lineage 缺失或安全治理缺失。

GPU Cluster Topology Audit

一句话定义：GPU Cluster Topology Audit 是把单机多卡、多机多卡和机柜级 GPU 集群从“资源数量”升级为可审计拓扑、通信、故障域、资源池和调度门禁的方法。

核心直觉：GPU 集群不是 GPU 数量越多越好，而是要让并行策略、通信路径、GPU-NIC 亲和性、机柜位置、存储路径、电力散热和资源池隔离匹配任务画像。tensor parallel 这类通信密集维度应优先放在同机 NVLink / NVSwitch 或其他 scale-up 高速互联域内；data parallel 可以跨节点扩展，但要看机间 fabric、NCCL、rack locality 和 oversubscription。

常见公式：集群候选资源组可以写成

$$g_i=(n_i,u_i,p_i,\mathrm{ell}_i,a_i,r_i,b_i,s_i,f_i,c_i,o_i,z_i)$$

总 GPU 和总显存可以写成

$$G_{\{\mathrm{total}\}}=N_{\{\mathrm{node}\}}G_{\{\mathrm{node}\}}$$

$$M_{\{\mathrm{total}\}}=G_{\{\mathrm{total}\}}M_{\{\mathrm{gpu}\}}$$

通信耗时可以用简化模型表达为

$$T_{\{\mathrm{comm}\}}\approx \alpha H+\frac{V}{B_{\{\mathrm{eff}\}}}$$

其中， α 是通信启动或单跳延迟， H 是路径跳数， V 是通信量， B_{eff} 是有效带宽。

面试表达：GPU 集群架构题要先区分 scale-up 和 scale-out，再讲单机 PCIe / NUMA / NVLink / NVSwitch、GPU-NIC 亲和性、多机 fabric、rack locality、存储 checkpoint、故障域、电力散热、资源池隔离和拓扑感知调度。

Scale-Up Domain Fit

一句话定义：Scale-Up Domain Fit 是检查通信密集并行组是否能放进同机或同一高速 GPU 互联域内。

常见坑：把 tensor parallel 随机跨节点放置，会把每层通信放到慢链路上，扩展效率明显下降。

PCIe NUMA Locality

一句话定义：PCIe NUMA Locality 是检查 GPU、CPU socket、PCIe Root Complex、系统内存和 NIC 是否有明确亲和性映射。

面试表达：不是所有 8 卡机器都一样，PCIe 层级和 NUMA 亲和性会影响 CPU-GPU 拷贝、GPU-NIC 通信和多任务切分。

NVLink NVSwitch Locality

一句话定义：NVLink NVSwitch Locality 是检查 GPU 间高速互联域是否存在、是否均匀、是否被调度器和并行策略正确使用。

GPU NIC Affinity

一句话定义：GPU NIC Affinity 是检查 GPU 到网卡的 PCIe 路径、NUMA 亲和性和 GPUDirect RDMA 条件是否清楚。

常见坑：多机训练只看网卡型号，不看哪个 GPU 到哪个 NIC 更近，导致跨 socket 或跨 PCIe 路径额外开销。

Inter-Node Fabric Readiness

一句话定义：Inter-Node Fabric Readiness 是检查节点之间的 InfiniBand、RoCE 或 Ethernet fabric 是否满足带宽、延迟、拥塞控制、RDMA 和运维要求。

Rack Locality Awareness

一句话定义：Rack Locality Awareness 是检查任务是否优先放在同一机柜或网络距离较近的机柜内，同时避免不必要的跨全局集群通信。

Oversubscription Awareness

一句话定义：Oversubscription Awareness 是检查下行接入带宽与上行汇聚带宽的比例是否被记录，并评估它对集合通信和并发任务的影响。

常见公式：

$$O_{\{\mathrm{net}\}} = \frac{B_{\{\mathrm{down}\}}}{B_{\{\mathrm{up}\}}}$$

Collective Communication Fit

一句话定义：Collective Communication Fit 是检查 AllReduce、AllGather、ReduceScatter、Broadcast 和 point-to-point 等通信模式是否与拓扑和 NCCL 配置匹配。

Parallel Group Placement

一句话定义：Parallel Group Placement 是检查 tensor parallel、pipeline parallel、data parallel、expert parallel 等并行组是否按通信密度放在合适拓扑范围内。

Storage Checkpoint Locality

一句话定义：Storage Checkpoint Locality 是检查训练数据缓存、checkpoint 写入和恢复路径是否靠近计算资源，避免共享存储或远端对象存储成为瓶颈。

Fault Domain Isolation

一句话定义：Fault Domain Isolation 是检查单卡、单机、单机柜、单交换机或共享存储故障会影响多少 GPU 和任务。

常见公式：

$$F_{\{\mathrm{rack}\}} = \frac{G_{\{\mathrm{rack}\}}}{G_{\{\mathrm{total}\}}}$$

Power Cooling Capacity Fit

一句话定义：Power Cooling Capacity Fit 是检查机柜电力、供电冗余、散热能力和运维空间是否能支撑目标 GPU 密度。

Resource Pool Isolation

一句话定义：Resource Pool Isolation 是检查训练、推理、评估、SFT、交互式开发和低优先级任务是否有资源池或优先级隔离。

Topology-Aware Scheduling

一句话定义：Topology-Aware Scheduling 是调度器不只找够 GPU 数量，还要按 GPU 互联、GPU-NIC、rack、fault domain、storage locality 和任务画像选择资源。

Observability Topology Coverage

一句话定义：Observability Topology Coverage 是检查 PCIe、NVLink、NIC、NCCL、rack、storage、checkpoint 和调度决策是否有 metrics、logs、traces 或事件记录。

GPU Cluster Gate

一句话定义：GPU Cluster Gate 是判断 GPU 集群资源组是否满足拓扑、通信、资源池、故障域和观测最低要求的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{cluster}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{slow}\}} \leq \rho_{\{\mathrm{slow}\}} \ \& \ O_{\{\mathrm{net}\}} \leq P_0 \right]$$

其中， R_{slow} 是跨慢链路通信比例， O_{net} 是网络 oversubscription， P_0 是 P0 级风险数量。

典型硬阻断：scale-up 域不匹配、PCIe / NUMA 未知、NVLink / NVSwitch 拓扑未用、GPU-NIC 亲和性缺失、机间 fabric 未就绪、机柜 locality 缺失、oversubscription 未知、collective 未建模、并行组随机放置、checkpoint 路径过远、故障域过大、电力散热超配、训练推理混池、调度器不拓扑感知或缺拓扑观测指标。

Network Communication Audit

一句话定义：Network Communication Audit 是把 InfiniBand、RoCE、以太网、RDMA、GPUDirect RDMA、NCCL 和集合通信从“网络很快/很慢”的描述升级为可检查指标表的方法。

核心直觉：多机大模型训练的网络在每个 step 参与梯度、参数分片、activation 或 optimizer state 同步。网络排查不能只看纸面带宽，要同时看 bit / byte 单位、有效带宽、延迟、抖动、丢包、重传、RDMA、GDR、fabric、collective、NCCL 拓扑、rank straggler 和调度放置。

常见公式：带宽单位换算可以写成

$$B_{\{\mathrm{GiB/s}\}} = \frac{B_{\{\mathrm{Gbps}\}}}{8} \cdot 2^{30}$$

通信耗时可以粗略拆成

$$T_{\{\mathrm{msg}\}} \approx \alpha H + \frac{D}{B_{\{\mathrm{eff}\}}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{retx}\}}$$

Ring AllReduce 每个 rank 的近似通信量可以写成

$$V_{\mathrm{ring}} \approx 2 \frac{P-1}{P} D$$

面试表达：多机训练扩展效率差时，先看 step time 和通信占比，再看 NCCL / profiler、rank 差异、网卡、错误包、重传、RDMA / GDR、交换机拥塞、跨机柜路径和调度放置。

Bandwidth Unit Accounting

一句话定义：Bandwidth Unit Accounting 是检查 Gbps、GB/s、GiB/s、理论带宽和有效带宽是否被清楚区分。

常见坑：把 400 Gbps 当成 400 GB/s，或者只看链路峰值不看有效带宽系数。

Latency Jitter Tracking

一句话定义：Latency Jitter Tracking 是检查 p50、p95、p99 延迟和 step time 抖动是否被记录。

常见公式：

$$J_{\mathrm{lat}} = \frac{L_{\mathrm{p99}}}{L_{\mathrm{p50}}}$$

RDMA Capability Fit

一句话定义：RDMA Capability Fit 是检查网络是否支持远程直接内存访问，以及训练栈是否实际启用和验证。

GPUDirect RDMA Path

一句话定义：GPUDirect RDMA Path 是检查 NIC 是否能直接访问 GPU 显存，并确认 GPU-NIC PCIe / NUMA 亲和性。

InfiniBand Fabric Readiness

一句话定义：InfiniBand Fabric Readiness 是检查 InfiniBand fabric 的链路、子网、RDMA、带宽、延迟和运维健康是否满足训练任务。

RoCE Congestion Losslessness

一句话定义：RoCE Congestion Losslessness 是检查 RoCE 网络的 PFC、ECN、拥塞控制、低丢包和运维配置是否满足 RDMA 通信。

常见坑：把 RoCE 当普通以太网使用，不配置 lossless 或拥塞控制，导致丢包和重传放大成 NCCL timeout。

Ethernet Fallback Scope

一句话定义：Ethernet Fallback Scope 是明确普通以太网适合小规模训练、评估、批量推理或低通信任务，不应默认承载大规模高通信预训练。

Collective Operation Modeling

一句话定义：Collective Operation Modeling 是检查 AllReduce、AllGather、ReduceScatter、Broadcast 和 point-to-point 的通信量、频率和消息大小是否被建模。

NCCL Topology Runtime Fit

一句话定义：NCCL Topology Runtime Fit 是检查 NCCL 是否识别预期 NIC、GPU 拓扑、GDR 路径、ring / tree 选择和实际通信路径。

AllReduce Cost Estimation

一句话定义：AllReduce Cost Estimation 是估算数据并行梯度同步的通信量和耗时，避免只看模型参数量或 GPU 数量。

AllGather ReduceScatter Cost

一句话定义：AllGather ReduceScatter Cost 是估算 FSDP / ZeRO 等分片策略带来的额外参数拉取和梯度分发通信。

Rank Straggler Detection

一句话定义：Rank Straggler Detection 是检查最慢 rank 是否拖住同步训练全局 step。

Packet Error Retransmit Tracking

一句话定义：Packet Error Retransmit Tracking 是检查网卡、交换机和协议层错误包、丢包、重传和错误计数。

常见公式：

$$R_{\{\mathrm{retx}\}} = \frac{N_{\{\mathrm{retx}\}}}{N_{\{\mathrm{packet}\}}}$$

Topology Congestion Awareness

一句话定义：Topology Congestion Awareness 是检查任务路径是否跨拥塞交换机、跨机柜、oversubscription 链路或与其他大任务共享瓶颈链路。

Scheduler Network Locality

一句话定义：Scheduler Network Locality 是检查调度系统是否根据 rack、fabric、GPU-NIC 亲和性和通信密集并行组放置任务。

Network Communication Gate

一句话定义：Network Communication Gate 是判断一个多机训练或推理任务的网络通信是否达到最低可用要求的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{net}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \text{and} \ R_{\{\mathrm{comm}\}} \leq \rho_{\{\mathrm{comm}\}} \ \text{and} \ J_{\{\mathrm{net}\}} \right]$$

典型硬阻断：带宽单位混淆、延迟抖动未记录、RDMA 未启用、GDR 路径缺失、InfiniBand fabric 未验收、RoCE 无 PFC / ECN / 拥塞控制、普通以太网承载大规模预训练、collective 未建模、NCCL 拓扑未知、AllReduce / AllGather / ReduceScatter 成本缺失、rank straggler 被隐藏、错误包和重传未追踪、拓扑拥塞未知、调度器不看网络 locality 或缺网络通信门禁。

Storage System Audit

一句话定义：Storage System Audit 是把本地 NVMe、共享文件系统、对象存储、数据湖、checkpoint、模型权重和 artifact 从“存得下”升级为可审计的吞吐、metadata、缓存、恢复、血缘、权限和成本门禁。

核心直觉：AI 存储不只是放文件。训练数据要稳定供给 GPU，checkpoint 要能写出且恢复，模型权重要能快速加载，artifact 要能追踪血缘，数据湖要能治理权限和质量，生命周期要能控制成本。

常见公式：有效吞吐可以写成

$$B_{\{\mathrm{eff}\}} = \frac{D}{T}$$

训练 I/O 占比可以写成

$$R_{\{\mathrm{io}\}} = \frac{T_{\{\mathrm{data loader}\}} + T_{\{\mathrm{storage}\}} + T_{\{\mathrm{h2d}\}}}{T_{\{\mathrm{step}\}}}$$

checkpoint 保存时间可以写成

$$T_{\{\mathrm{ckpt}\}} = \frac{S_{\{\mathrm{ckpt}\}}}{\eta B_{\{\mathrm{write}\}}} + T_{\{\mathrm{meta}\}} + T_{\{\mathrm{commit}\}}$$

面试表达：存储体系题要按访问模式分层回答：对象存储 / 数据湖作权威源，本地 NVMe / 分布式缓存作热数据加速，共享文件系统提供易用性但要防 metadata 瓶颈，checkpoint 和模型权重要有恢复测试、checksum、commit、缓存和生命周期策略。

Capacity Tier Fit

一句话定义：Capacity Tier Fit 是检查 hot、warm、cold、archive 等存储层级是否匹配训练数据、checkpoint、日志和 artifact 的增长。

Throughput IOPS Fit

一句话定义：Throughput IOPS Fit 是检查读写吞吐、IOPS、延迟和并发是否按真实训练、checkpoint 和模型加载流量测过。

Local NVMe Cache Fit

一句话定义：Local NVMe Cache Fit 是检查训练节点本地 NVMe 是否用于热点 shard、解压数据、模型权重和临时文件缓存，并有容量和淘汰策略。

Shared FS Metadata Fit

一句话定义：Shared FS Metadata Fit 是检查共享文件系统是否能承受 open、stat、list、rename、commit 和小文件 metadata 压力。

Object Store Authority Fit

一句话定义：Object Store Authority Fit 是检查对象存储是否作为权威源、归档和 artifact 后端使用，而不是被误当成高频 POSIX 文件系统直接训练读取。

Data Lake Governance Fit

一句话定义：Data Lake Governance Fit 是检查数据湖是否记录数据血缘、质量、权限标签、清洗规则、版本和合规状态。

Training Shard Format Fit

一句话定义：Training Shard Format Fit 是检查训练数据是否被预处理成适合顺序读取和并发读取的 shard，例如 WebDataset、Parquet、Arrow 或自定义二进制格式。

Small File Amplification Control

一句话定义：Small File Amplification Control 是检查小文件数量是否被 shard、索引、合并、缓存和预取控制。

常见公式：

$$A_{\{\mathrm{meta}\}} = \frac{N_{\{\mathrm{file}\}}}{D_{\{\mathrm{TiB}\}}}$$

Dataloader Cache Hit Tracking

一句话定义：Dataloader Cache Hit Tracking 是检查 dataloader 是否记录本地缓存、远端读取、等待时间和 cache hit rate。

常见公式：

$$H_{\mathrm{cache}} = \frac{D_{\mathrm{cache}}}{D_{\mathrm{total}}}$$

Checkpoint Write Recovery Fit

一句话定义：Checkpoint Write Recovery Fit 是检查 checkpoint 是否有写入吞吐、分片策略、异步保存、checksum、commit metadata 和恢复演练。

Model Weight Load Cache Fit

一句话定义：Model Weight Load Cache Fit 是检查模型权重、tokenizer、config、adapter 和量化版本是否有本地缓存、分层缓存、预热和完整性校验。

Artifact Lineage Metadata Fit

一句话定义：Artifact Lineage Metadata Fit 是检查 dataset version、training config、checkpoint、final model、eval report 和 deployment package 是否能互相追溯。

Consistency Commit Integrity

一句话定义：Consistency Commit Integrity 是检查大文件写入是否先写临时对象，校验 checksum 后再用 manifest 或 commit 标记为可用。

Security Compliance Fit

一句话定义：Security Compliance Fit 是检查身份认证、访问控制、加密、审计日志、数据分类、留存删除和租户隔离是否覆盖存储链路。

Lifecycle Cost Governance

一句话定义：Lifecycle Cost Governance 是检查 checkpoint、数据集、日志、trace 和中间 artifact 是否有保留、归档、删除、去重和成本归因策略。

常见公式：

$$K_{\mathrm{storage}} = K_{\mathrm{capacity}} + K_{\mathrm{request}} + K_{\mathrm{egress}} + K_{\mathrm{metadata}} + K_{\mathrm{recovery}}$$

Storage System Gate

一句话定义：Storage System Gate 是判断 AI 存储体系是否达到训练、恢复、部署和治理最低要求的综合门禁。

门禁形式可以写成：

$$G_{\mathrm{storage}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge R_{\mathrm{io}} \leq \rho_{\mathrm{io}} \wedge T_{\mathrm{recovery}} \leq \tau_{\mathrm{recovery}} \right]$$

典型硬阻断：容量不分层、吞吐和 IOPS 未实测、本地缓存缺失、共享文件系统 metadata 过载、对象存储被当 POSIX 高频训练源、数据湖无血缘治理、训练格式仍是 raw JSON、小文件未 shard、cache hit 未追踪、checkpoint 没恢复测试、模型权重冷启动无缓存、artifact 血缘缺失、commit / checksum 缺失、安全审计缺失、生命周期策略缺失或没有存储门禁。

Checkpoint Lifecycle Audit

一句话定义：Checkpoint Lifecycle Audit 是把训练状态 checkpoint 从“保存目录”升级为可审计的状态对象、分片布局、异步写入、manifest、checksum、恢复演练、保留策略、成本、权限和 SLO 门禁。

核心直觉：checkpoint 的价值不是保存成功，而是故障、抢占、扩缩容、代码回滚和模型发布时能被可靠恢复和解释。训练状态 checkpoint 至少要覆盖模型参数、优化器、scheduler、global step、AMP scaler、RNG、dataloader 位置、分布式 rank / partition metadata、代码版本、数据版本和训练配置。

常见公式：分片大小可以写成

$$S_{\mathrm{shard}} = \frac{S_{\mathrm{ckpt}}}{N_{\mathrm{shard}}}$$

保存耗时可以写成

$$T_{\mathrm{save}} = \frac{S_{\mathrm{ckpt}}}{\eta_w B_{\mathrm{write}}} + T_{\mathrm{sync}} + T_{\mathrm{meta}} + T_{\mathrm{meta}}$$

恢复耗时可以写成

$$T_{\mathrm{restore}} = \frac{S_{\mathrm{ckpt}}}{\eta_r B_{\mathrm{read}}} + T_{\mathrm{validate}} + T_{\mathrm{rebuild}} + T_{\mathrm{meta}}$$

面试表达：checkpoint 系统设计题要先区分模型权重 checkpoint 和训练状态 checkpoint，再讲分片、manifest、checksum、commit 语义、异步保存、恢复演练、RNG / dataloader / rank state、一致性、保留策略、成本归因、跨区复制、权限审计和 SLO。

Checkpoint Object Completeness

一句话定义：Checkpoint Object Completeness 是检查 checkpoint 是否保存完整训练状态，而不只是模型权重。

Shard Layout Fit

一句话定义：Shard Layout Fit 是检查 checkpoint shard 数量、大小、rank 映射、manifest 记录和并行配置是否匹配。

Async Save Overlap

一句话定义：Async Save Overlap 是检查异步 checkpoint 是否真正降低训练可见阻塞，并且没有破坏快照一致性或造成后台队列积压。

Write Bandwidth Fit

一句话定义：Write Bandwidth Fit 是检查 checkpoint 写入带宽、metadata 开销、commit 开销和保存 SLO 是否实测过。

Metadata Commit Integrity

一句话定义：Metadata Commit Integrity 是检查 checkpoint 是否先写临时对象，所有 shard 校验成功后再把 manifest 标记为 committed。

Checksum Manifest Validation

一句话定义：Checksum Manifest Validation 是检查每个 shard 是否有 digest，恢复时是否按 manifest 校验完整性。

Restore Replay Readiness

一句话定义：Restore Replay Readiness 是检查 checkpoint 是否通过真实恢复演练，而不是只验证保存流程。

Dataloader RNG State Capture

一句话定义：Dataloader RNG State Capture 是检查 Python、NumPy、PyTorch、CUDA、sampler 和 dataloader 位置是否可恢复。

Distributed Rank State Capture

一句话定义：Distributed Rank State Capture 是检查 tensor parallel、pipeline parallel、data parallel、ZeRO / FSDP partition 和 gradient accumulation 状态是否可恢复。

Retention Policy Fit

一句话定义：Retention Policy Fit 是检查最近 checkpoint、milestone、best eval、release 版本和失败临时版本是否有清晰保留策略。

Cross-Region Replication Fit

一句话定义：Cross-Region Replication Fit 是检查关键 checkpoint、发布权重和灾备副本是否有跨故障域复制和恢复测试。

Security Access Control

一句话定义：Security Access Control 是检查 checkpoint 存储是否有 IAM / RBAC、加密、审计日志和租户隔离。

Resume SLO Tracking

一句话定义：Resume SLO Tracking 是检查 save p95、restore p95、time since last checkpoint 和最大丢失进度是否进入训练平台 SLO。

Failure Drill Coverage

一句话定义：Failure Drill Coverage 是检查是否自动演练节点故障、shard 缺失、checksum 错误、抢占恢复和恢复后 loss 连续性。

Checkpoint Lifecycle Gate

一句话定义：Checkpoint Lifecycle Gate 是判断 checkpoint 生命周期是否达到训练容错和成本治理最低要求的综合门禁。

门禁形式可以写成：

$G_{\{\mathrm{ckpt}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge T_{\{\mathrm{restore}\}} \leq \tau_{\{\mathrm{restore}\}} \wedge W_{\{\mathrm{restore}\}} \right]$

典型硬阻断：只保存模型权重、单大文件、异步保存没有一致快照、写入带宽未知、manifest 先标 committed、checksum 缺失、从未恢复演练、RNG / dataloader / rank state 丢失、无限保留、冷归档不用、无灾备副本、权限开放、恢复 SLO 缺失、故障演练缺失或没有 checkpoint 生命周期门禁。

Container Environment Audit

一句话定义：Container Environment Audit 是把 Docker 镜像、GPU runtime、CUDA / driver / framework、依赖锁定、镜像安全、registry 权限和运行 metadata 放进同一套可复现环境门禁。

核心直觉：容器化不是“把代码塞进镜像”，而是让训练和推理环境可复现、可分发、可调度、可审计。GPU 容器仍依赖宿主驱动、内核、设备挂载、网络和多租户策略，因此镜像 digest、CUDA 兼容、GPU runtime、NCCL / RDMA、依赖锁定和安全扫描必须一起看。

常见公式：镜像拉取时间可以写成

$$T_{\mathrm{pull}} = \frac{S_{\mathrm{img}}}{\eta} B_{\mathrm{reg}} + T_{\mathrm{unpack}}$$

冷启动时间可以写成

$$T_{\mathrm{start}} = T_{\mathrm{pull}} + T_{\mathrm{init}} + T_{\mathrm{smoke}}$$

依赖锁定覆盖率可以写成

$$C_{\mathrm{lock}} = \frac{N_{\mathrm{pinned}}}{N_{\mathrm{dep}}}$$

面试表达：容器化设计题要先说镜像分层和 digest，再讲 GPU 容器依赖宿主机 driver、CUDA 用户态库和框架兼容矩阵；随后补依赖锁定、训练 / 推理镜像分离、镜像体积和冷启动、CI smoke test、漏洞扫描 / 签名 / SBOM、registry RBAC、非 root / 非 privileged、多租户挂载边界和环境 metadata。

Image Digest Reproducibility

一句话定义：Image Digest Reproducibility 是检查训练和推理任务是否记录不可变 image digest，而不是只记录可覆盖的 tag。

Base Layer Cache Fit

一句话定义：Base Layer Cache Fit 是检查基础镜像层是否可复用，节点是否预拉取常用 layer，避免任务启动被镜像拉取拖慢。

常见公式：

$$R_{\mathrm{layer}} = \frac{S_{\mathrm{cached}}}{S_{\mathrm{layer}}}$$

CUDA Driver Framework Compatibility

一句话定义：CUDA Driver Framework Compatibility 是检查宿主机 driver、容器 CUDA 用户态库、PyTorch / TensorFlow、cuDNN、NCCL、Triton 和自定义 kernel 是否形成兼容矩阵。

GPU Runtime Visibility

一句话定义：GPU Runtime Visibility 是检查容器是否通过 runtime / CDI、device plugin、device file 和 driver capabilities 正确看到 GPU。

Dependency Lock Coverage

一句话定义：Dependency Lock Coverage 是检查 Python 包、系统库、私有 wheel、CUDA 扩展和 serving runtime 是否锁版本或 hash。

Training Serving Image Separation

一句话定义：Training Serving Image Separation 是检查训练镜像和推理镜像是否按目标分离，避免把调试工具、编译工具和实验依赖直接带到线上推理。

Image Size Startup Fit

一句话定义：Image Size Startup Fit 是检查镜像大小、拉取时间、解压时间、初始化时间和冷启动 SLO 是否匹配训练任务提交和推理扩容。

Build Pipeline Smoke Test

一句话定义：Build Pipeline Smoke Test 是检查镜像构建流水线是否自动跑 Python、CUDA、框架、NCCL、关键 import 和小样本推理 / 训练 smoke test。

Security Scan Signing

一句话定义：Security Scan Signing 是检查镜像是否完成漏洞扫描、签名、SBOM 生成和基础镜像更新治理。

Secret Data Exclusion

一句话定义：Secret Data Exclusion 是检查镜像中是否误打入 API key、token、密钥、训练数据、敏感样本或大模型权重。

Runtime Hardening Fit

一句话定义：Runtime Hardening Fit 是检查容器是否避免 root、privileged、过宽 Linux capabilities 和可写 root filesystem 等高风险运行方式。

Registry Access Governance

一句话定义：Registry Access Governance 是检查镜像仓库是否有私有访问、RBAC、审计日志、retention、immutable tag 或 digest policy。

Environment Metadata Capture

一句话定义：Environment Metadata Capture 是检查训练和推理任务是否自动记录 image digest、driver、CUDA、Python、框架、NCCL、命令、环境变量、代码 commit 和数据版本。

Multi-Tenant Mount Isolation

一句话定义：Multi-Tenant Mount Isolation 是检查容器挂载是否限制在租户和任务授权范围内,避免 wildcard hostPath、敏感卷误挂和环境变量泄露。

NCCL RDMA Runtime Fit

一句话定义：NCCL RDMA Runtime Fit 是检查分布式训练容器是否挂载 RDMA / IB 设备、网络模式和 NCCL smoke test 是否通过。

Container Environment Gate

一句话定义：Container Environment Gate 是判断 AI 训练和推理容器环境是否达到复现、启动、安全和多租户治理最低要求的综合门禁。

门禁形式可以写成：

$G_{\{\mathrm{container}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \text{and} \ T_{\{\mathrm{start}\}} \leq \tau_{\{\mathrm{start}\}} \ \text{and} \ V_{\{\mathrm{start}\}} \leq \tau_{\{\mathrm{start}\}} \right]$

典型硬阻断：只记录 latest tag、基础 layer 缓存差、宿主机 driver 太旧、容器看不到 GPU、依赖未锁定、训练镜像直接上推理、镜像过大导致冷启动超 SLO、缺 smoke test、镜像未签名或有 critical 漏洞、密钥或模型权重打进镜像、root / privileged 运行、registry 公开、环境 metadata 缺失、wildcard hostPath、RDMA / IB 设备不可见或没有容器环境门禁。

Kubernetes GPU Resource Management Audit

一句话定义：Kubernetes GPU Resource Management Audit 是把 device plugin、GPU extended resource、request / limit、gang scheduling、碎片治理、拓扑放置、MIG / 共享、namespace / quota、多租户、监控和排障放进同一套 Kubernetes GPU 资源管理门禁。

核心直觉：Kubernetes 是 AI Infra 的常见底座，但默认调度器并不自动理解所有大模型训练和推理语义。GPU 资源管理要同时回答：GPU 是否被 kubelet 识别，Pod 是否正确申请资源，分布式训练是否能整体启动，空闲

GPU 是否被碎片化卡住，调度是否理解 NVLink / GPU-NIC / rack / storage locality, MIG 或 time slicing 是否匹配工作负载，租户配额和成本是否可治理，监控是否能把 Pod 映射到具体 GPU。

常见公式：GPU 分配率可以写成

$$U_{\mathrm{gpu}} = \frac{G_{\mathrm{allocated}}}{G_{\mathrm{total}}}$$

碎片化比例可以写成

$$R_{\mathrm{frag}} = 1 - \frac{\sum_n \lfloor g_n / g_{\mathrm{job}} \rfloor g_{\mathrm{job}}}{\sum_n g_n}$$

租户配额占用率可以写成

$$Q_{\mathrm{tenant}} = \frac{G_{\mathrm{used}}}{G_{\mathrm{quota}}}$$

Kubernetes GPU 资源管理门禁可以写成

$$G_{\mathrm{k8s_gpu}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \wedge R_{\mathrm{frag}} \leq \rho \wedge Q_{\mathrm{tenant}} \leq \right]$$

面试表达：Kubernetes GPU 管理题要先说 device plugin 把 GPU 注册成 `nvidia.com/gpu` 这类 extended resource，再讲 Pod resource limit、request / limit 一致性、GPU Operator、MIG / time slicing 的适用边界；随后补 gang scheduling、碎片化、拓扑感知、node label / affinity / taint / toleration、namespace / ResourceQuota、多租户隔离、训练 CRD、推理 SLO、DCGM / Prometheus 监控和 Pending 排障。核心观点是：K8s 是底座，不是完整 AI 调度系统。

Device Plugin Readiness

一句话定义：Device Plugin Readiness 是检查 GPU device plugin 是否就绪，是否把节点 GPU capacity / allocatable 注册给 kubelet，并让 Pod 能通过标准资源名申请 GPU。

GPU Extended Resource Fit

一句话定义：GPU Extended Resource Fit 是检查 Pod / Job / CRD 是否用 extended resource 表达 GPU 需求，并且总申请量不超过目标资源池可用量。

GPU Request Limit Integrity

一句话定义：GPU Request Limit Integrity 是检查 GPU request / limit 口径是否一致，避免平台层把整数 GPU 资源写成容易误解或不可调度的声明。

Gang Scheduling Readiness

一句话定义：Gang Scheduling Readiness 是检查分布式训练 worker 是否具备一起调度、一起失败释放和按完整 world size 启动的调度语义。

Fragmentation Control

一句话定义：Fragmentation Control 是检查调度系统是否通过 bin packing、资源池划分、大任务预留或抢占重调度控制 GPU 碎片化。

Topology Aware Placement

一句话定义：Topology Aware Placement 是检查调度是否考虑节点内 GPU 互联、GPU-NIC 亲和性、rack locality、storage locality 和任务通信画像。

Node Label Affinity Fit

一句话定义：Node Label Affinity Fit 是检查 node label、node selector、node affinity 和标签治理是否能将任务放到正确 GPU 型号、网络和资源池。

Taint Toleration Fit

一句话定义：Taint Toleration Fit 是检查专用 GPU 节点、推理池、高优先级资源池等 taint 是否有对应 toleration，且不会被普通任务误用。

MIG Sharing Policy Fit

一句话定义：MIG Sharing Policy Fit 是检查 MIG、time slicing、MPS 或整卡独占是否和工作负载匹配，避免把大模型预训练放到不稳定共享模式。

Quota Namespace Governance

一句话定义：Quota Namespace Governance 是检查 namespace、ResourceQuota、按 GPU 型号配额、成本归因和租户使用上限是否一起生效。

Training Operator Readiness

一句话定义：Training Operator Readiness 是检查 PyTorchJob、MPIJob、RayJob 或自研 TrainingJob CRD 是否能处理 rank、service discovery、checkpoint 和 worker 生命周期。

Inference Service Readiness

一句话定义：Inference Service Readiness 是检查 K8s 推理服务是否具备自动扩缩容、SLO 指标、健康检查、冷启动预算和灰度 / 回滚能力。

GPU Monitoring Mapping

一句话定义：GPU Monitoring Mapping 是检查 DCGM / Prometheus 等 GPU 指标是否能映射到 team、job、pod、container 和具体 GPU。

Multi-Tenant Isolation

一句话定义：Multi-Tenant Isolation 是检查 namespace、RBAC、network policy、secret scope、registry scope 和挂载边界是否能隔离不同租户。

Pending Troubleshooting Coverage

一句话定义：Pending Troubleshooting Coverage 是检查平台是否能把 Pod Pending 拆成 GPU 不足、quota、node affinity、taint、image pull、gang waiting 等可定位原因。

Kubernetes GPU Gate

一句话定义：Kubernetes GPU Gate 是判断 Kubernetes GPU 资源池是否达到可调度、可隔离、可观测和可排障最低要求的综合门禁。

典型硬阻断：device plugin 缺失、没有声明 nvidia.com/gpu、GPU request / limit 混乱、没有 gang scheduling、GPU 碎片化严重、拓扑被忽略、label / affinity 缺失、taint 无 toleration、预训练误用 time slicing、namespace / quota 缺失、训练 CRD 不完整、推理 SLO 缺失、GPU 指标无法映射到 Pod、多租户隔离缺失、Pending 原因不可解释或没有 Kubernetes GPU 门禁。

Training Scheduler Governance Audit

一句话定义：Training Scheduler Governance Audit 是把训练任务资源画像、队列、优先级、配额、公平性、gang scheduling、backfilling、抢占、checkpoint、碎片、拓扑、成本、观测和失败重排放进同一套调度治理门禁。

核心直觉：训练调度不是 FIFO，也不是单纯提高 GPU utilization。大模型训练调度要在稀缺 GPU 上同时平衡大任务启动、小任务等待、业务优先级、租户公平性、资源借用、抢占损失、checkpoint 恢复、拓扑效率和成本归因。只看谁先提交或谁优先级高，都会让系统在真实多团队环境里失控。

常见公式：队列等待时间可以按队列统计 P95：

$$W_{\{q, \mathrm{p95}\}} = \mathrm{P95}(\mathrm{t}_{\{\mathrm{start}\}} - \mathrm{t}_{\{\mathrm{sub}\}} \mid q)$$

租户配额占用率可以写成

$$Q_u = \frac{G_{\{u, \mathrm{used}\}}}{G_{\{u, \mathrm{quota}\}}}$$

多资源 dominant share 可以写成

$$D_u = \max_r \frac{A_{\{u, r\}}}{C_r}$$

抢占损失可以写成

$$L_j = G_j(\mathrm{t}_{\{\mathrm{now}\}} - \mathrm{t}_{\{\mathrm{ckpt}\}})$$

训练调度门禁可以写成

$$G_{\{\mathrm{train_sched}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \ \wedge \ W_{\{q, \mathrm{p95}\}} \leq \omega_q \ \wedge \ Q_u \leq 1 \ \wedge \right]$$

面试表达：训练调度系统设计题要先讲 TrainingJob 资源画像，再讲 queue / priority / quota / fair sharing；分布式训练要用 gang scheduling，低优先级短任务可以 backfilling，但不能阻塞已保留资源；抢占只应作用于可抢占且有 checkpoint 的任务，并记录恢复状态；调度还要结合 GPU 碎片、拓扑、成本、SLO、失败重排和观测指标，最终用调度门禁收束，而不是只说“用 Kubernetes scheduler”。

Workload Resource Shape

一句话定义：Workload Resource Shape 是检查训练任务是否声明 GPU 类型、GPU 数、每节点 GPU、网络、存储、队列、优先级、可抢占性、预计运行时长和 checkpoint 路径。

Queue Policy Coverage

一句话定义：Queue Policy Coverage 是检查每个队列是否有配额、并发上限、优先级集合、抢占策略、借用 / 出借规则和默认 checkpoint 要求。

Priority Governance

一句话定义：Priority Governance 是检查 critical / high / normal / low / best-effort 等优先级是否有审批、额度、审计和成本归因，避免所有任务都申请高优先级。

Quota Usage Control

一句话定义：Quota Usage Control 是检查团队 GPU 数、GPU 小时、GPU 型号、预算和并发任务是否在配额或可借用范围内。

Fair Share Accounting

一句话定义：Fair Share Accounting 是检查调度系统是否按权重、历史使用、dominant share 或队列策略解释多团队资源分配。

Gang Scheduling Fit

一句话定义：Gang Scheduling Fit 是检查分布式训练 worker 是否具备整体 admission、整体启动和失败释放语义，避免部分 worker 占住资源。

Backfilling Safety

一句话定义：Backfilling Safety 是检查短任务使用空闲资源时是否不会推迟已保留的大任务，并且具备截止时间、可抢占或运行时长约束。

Preemption Checkpoint Safety

一句话定义：Preemption Checkpoint Safety 是检查抢占只作用于低优先级、可抢占、最近有 checkpoint、恢复成本可控且记录原因的任务。

Scheduler Fragmentation Control

一句话定义：Scheduler Fragmentation Control 是检查调度系统是否用 bin packing、小任务池、大任务池、空间预留、抢占或重排降低 GPU 碎片。

Scheduler Topology Awareness

一句话定义：Scheduler Topology Awareness 是检查调度是否使用 GPU 型号、NVLink / NVSwitch、GPU-NIC、rack、存储 locality 和故障域信息。

Checkpoint Scheduler Coupling

一句话定义：Checkpoint Scheduler Coupling 是检查调度器是否知道 checkpoint URI、最近 checkpoint 时间、最大丢失进度和恢复耗时，从而做理性抢占和恢复。

Cost Attribution Control

一句话定义：Cost Attribution Control 是检查 GPU 小时、队列成本中心、超额资源申请、闲置 GPU 和失败重跑是否能归因到团队和项目。

Scheduler Observability

一句话定义：Scheduler Observability 是检查 queue length、pending P95、running jobs、GPU allocated、fragmentation、preemption count、fair share 和 success rate 是否可观测。

Failure Requeue Readiness

一句话定义：Failure Requeue Readiness 是检查任务失败、抢占或节点故障后是否有重排、backoff、event reason、terminal status 和恢复路径。

Anti Starvation Control

一句话定义：Anti Starvation Control 是检查低优先级或小团队任务是否有最大等待时间、aging boost、队列 SLO 或人工升级路径，避免长期饥饿。

Training Scheduler Gate

一句话定义：Training Scheduler Gate 是判断训练调度系统是否达到可排队、可公平、可抢占、可恢复、可观测和可成本治理最低要求的综合门禁。

典型硬阻断：任务资源画像缺网络或 checkpoint、队列无策略、优先级无边界、租户超配额、公平性不可解释、gang scheduling 缺失、backfilling 可能阻塞保留任务、抢占无 checkpoint、碎片治理缺失、调度不看拓扑、checkpoint 信息未进入调度器、成本不可归因、调度指标缺失、失败后不能重排、低优任务长期饥饿或没有训练调度门禁。

Multi Tenant GPU Scheduler Design Audit

一句话定义：Multi Tenant GPU Scheduler Design Audit 是把多租户队列、配额借用、公平性、gang scheduling、拓扑、碎片、抢占、低优先级、推理 SLO、异构 GPU、可解释性、成本和观测放进同一套 GPU 集群调度系统设计门禁。

核心直觉：多租户 GPU 调度不是“用 Kubernetes scheduler 调 Pod”，也不是只做 GPU 利用率优化。高分系统设计要证明每个租户通过 queue / quota / priority 被治理，借用资源可以回收，分布式训练具备整体 admission，拓扑放置能匹配通信画像，抢占和 checkpoint 绑定，低优先级任务可回收，推理服务有 SLO 隔离，异构 GPU 有成本和适配策略，并且每次 pending、placement 和 preemption 都能解释给用户和平台 owner。

常见公式：队列配额占用率可以写成

$$U_q = \frac{G_{\{\mathrm{used}\},q}}{G_{\{\mathrm{quota}\},q}}$$

借用 GPU 数可以写成

$$B_q = \max\left(0, G_{\{\mathrm{used}\},q} - G_{\{\mathrm{guaranteed}\},q}\right)$$

Dominant share 可以写成

$$D_q = \max_{r \in \mathcal{R}} \frac{u_{\{q,r\}}}{c_r}$$

带权公平偏差可以写成

$$H_q = \frac{D_q}{w_q}, \quad \Delta_{\{\mathrm{fair}\}} = \max_q H_q - \min_q H_q$$

Gang scheduling 可行性可以写成

$$G_{\{\mathrm{gang}\}} = \mathbf{1} \left[\sum_{n=1}^N \left\lfloor \frac{a_n}{g_{\{\mathrm{pernode}\}}} \right\rfloor \geq n_{\{\mathrm{mat}\}} \right]$$

碎片率可以写成

$$R_{\{\mathrm{frag}\}} = 1 - \frac{\sum_n \left\lfloor \frac{a_n}{g_{\{\mathrm{job}\}}} \right\rfloor}{\sum_n a_n}$$

多租户 GPU 调度门禁可以写成

$$G_{\{\mathrm{mtgpu}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge \Delta_{\{\mathrm{fair}\}} \leq \delta \wedge R_{\{\mathrm{frag}\}} \leq \epsilon \right]$$

面试表达：先澄清集群规模、GPU 异构性、训练 / 推理 / 评估任务比例、租户和优先级规则；再讲 Queue Manager、Resource Snapshot、Policy Engine、Placement Engine、Preemption Engine 和 Observability。调度策略要按 queue quota、borrow / lend、dominant share、gang、topology-aware placement、fragmentation control、checkpoint-safe preemption、low-priority reclaim 和 inference SLO isolation 分层。最后用 Multi Tenant GPU Scheduler Gate 收束，说明 pending 原因、抢占原因、成本归因和公平性证据都可审计。

Tenant Queue Contract

一句话定义：Tenant Queue Contract 是检查每个租户队列是否声明 owner、quota、guaranteed / burst / borrowed 资源、priority、最大并发、可抢占策略、允许 GPU 型号和成本中心。

Quota Borrowing Policy

一句话定义：Quota Borrowing Policy 是检查队列借用空闲 GPU 时是否有上限、cohort、回收规则、补偿策略、victim 选择和审计事件。

Dominant Fairness Accounting

一句话定义：Dominant Fairness Accounting 是检查公平性是否同时考虑 GPU、CPU、内存、网络、存储和稀缺 GPU 型号，而不是只看 GPU 张数。

Low Priority Reclaim

一句话定义：Low Priority Reclaim 是检查低优先级任务是否只使用可回收余量，并且具备 checkpoint、graceful shutdown、重新入队、通知和抢占频率限制。

Inference SLO Isolation

一句话定义：Inference SLO Isolation 是检查低延迟推理服务是否从批训练抢占、碎片整理和低优先级填充中隔离出来，至少有独立资源池、优先级或 SLO guardrail。

Heterogeneous GPU Policy

一句话定义：Heterogeneous GPU Policy 是检查调度是否根据 GPU 型号、显存、互联、CUDA / driver、任务适配性和成本，把训练、推理、评估、embedding 与数据处理放到合适加速器上。

Resource Snapshot Health

一句话定义：Resource Snapshot Health 是检查调度器资源视图是否包含 GPU 空闲量、健康状态、拓扑、MIG / sharing、队列占用、租户借用和节点故障，并且有 freshness 约束。

Placement Explainability

一句话定义：Placement Explainability 是检查每次调度、排队和抢占是否记录候选节点、过滤原因、打分项、最终选择、失败原因和用户可读解释。

Cost Utilization Attribution

一句话定义：Cost Utilization Attribution 是检查 GPU-hours、idle hours、failed GPU-hours、preempted GPU-minutes、borrowed quota、低优先级资源和推理 SLO 资源是否能归因到 tenant、project、queue、job type 和 owner。

Scheduler Observability Audit

一句话定义：Scheduler Observability Audit 是检查 queue wait、pending reason、quota usage、dominant share、fragmentation、topology score、preemption、gang admission、placement latency 和调度决策事件是否可观测。

Tradeoff Boundary Reasoning

一句话定义：Tradeoff Boundary Reasoning 是检查调度系统回答是否讲清公平性 vs 利用率、紧凑放置 vs 容错、抢占效率 vs 进度浪费、资源预留 vs 空闲成本、异构成本优化 vs 用户体验复杂度。

Multi Tenant GPU Scheduler Gate

一句话定义：Multi Tenant GPU Scheduler Gate 是判断多租户 GPU 集群调度系统是否达到可公平、可借用、可回收、可整体调度、可拓扑优化、可抢占恢复、可解释、可归因和可审计最低要求的综合门禁。

典型硬阻断：队列没有 owner 和 quota、借用不可回收、公平性只看 FIFO、gang 任务能半启动、topology ignored、碎片率不可观测、抢占无 checkpoint、低优先级任务不可回收、推理服务被批训练抖动影响、异构

GPU 策略缺失、资源快照陈旧、placement 无解释、成本不可归因、调度事件缺失、trade-off 边界缺失或没有最终调度门禁。

Multi-Tenant Isolation Audit

一句话定义：Multi-Tenant Isolation Audit 是把资源、身份、权限、数据、网络、运行时、镜像、密钥、日志、成本、环境、共享资源、审计和故障域放进同一套 AI Infra 多租户隔离门禁。

核心直觉：多租户隔离不是“每个团队一个 namespace”。Namespace 只是治理入口，真正的隔离要证明：租户资源不会互相抢，身份不会串租户，权限遵守最小授权，数据和 artifact 不越界，网络默认拒绝，运行时不能绕过容器边界，镜像和密钥可信，日志 / trace 脱敏，成本能归因，生产和实验不会互相影响，跨租户共享有 owner、版本、权限和审计。

常见公式：租户配额占用率可以写成

$$Q_u = \frac{G_{\{u, \mathrm{used}\}}}{G_{\{u, \mathrm{quota}\}}}$$

未授权访问拦截率可以写成

$$B_{\{\mathrm{auth}\}} = \frac{N_{\{\mathrm{blocked}\}}}{N_{\{\mathrm{unauth}\}}}$$

数据等级传播可以写成

$$L_{\{\mathrm{artifact}\}} \geq \max_{d \in D_{\{\mathrm{train}\}}} L_d$$

密钥暴露率可以写成

$$R_{\{\mathrm{secret}\}} = \frac{N_{\{\mathrm{secret_exposed}\}}}{N_{\{\mathrm{secret}\}}}$$

多租户隔离门禁可以写成

$$G_{\{\mathrm{tenant}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \wedge \ Q_u \leq 1 \ \wedge \ B_{\{\mathrm{auth}\}} = 1 \ \wedge \ R_{\{\mathrm{secret}\}} \leq \epsilon \right]$$

面试表达：多租户隔离题要先明确 namespace 只是入口，再按资源、身份、权限、数据、网络、运行时、镜像、密钥、日志、成本和审计分层。AI 场景还要强调数据等级会传播到 checkpoint、模型权重、embedding index、eval report、日志和 trace；生产推理和实验训练要隔离；跨租户共享要产品化、版本化、权限化、审计化。最后用 Multi-Tenant Isolation Gate 收束，而不是只背 RBAC 或 NetworkPolicy。

Resource Quota Isolation

一句话定义：Resource Quota Isolation 是检查租户 GPU、CPU、内存、存储、队列并发、推理 QPS 和成本预算是否有保障、上限和超额策略。

Identity Namespace Binding

一句话定义：Identity Namespace Binding 是检查 IdP group、tenant label、namespace、service account 和 workload identity 是否一致绑定，避免身份串租户。

RBAC ABAC Permission Fit

一句话定义：RBAC ABAC Permission Fit 是检查角色权限和属性策略是否共同覆盖 submit job、read data、read logs、deploy model、access secret 和 change quota 等动作。

Data Access Boundary

一句话定义：Data Access Boundary 是检查训练数据、RAG 文档、checkpoint、模型权重、日志、trace、prompt 和用户请求是否按租户、项目、用途和数据等级授权。

Data Classification Lineage

一句话定义：Data Classification Lineage 是检查数据分类和血缘是否传播到 checkpoint、模型权重、adapter、embedding index、eval report 和发布状态。

Network Policy Isolation

一句话定义：Network Policy Isolation 是检查租户间 Pod 访问、外网 egress、生产数据 endpoint、调试端口和私有服务访问是否默认拒绝、按需放行。

Runtime Security Boundary

一句话定义：Runtime Security Boundary 是检查 privileged、hostPath、root、共享 namespace、GPU 共享、本地缓存和系统调用限制是否符合租户风险等级。

Image Supply Chain Boundary

一句话定义：Image Supply Chain Boundary 是检查租户镜像是否来自可信 registry，是否 digest pin、扫描、签名、基础镜像白名单和高危漏洞阻断。

Secret Scope Rotation

一句话定义：Secret Scope Rotation 是检查密钥是否由 Secret Manager 或等价系统管理，是否租户化、短期化、最小权限、自动轮换、日志脱敏和访问审计。

Logs Trace Redaction

一句话定义：Logs Trace Redaction 是检查日志、trace、prompt、模型输出、工具调用结果和错误堆栈是否按租户权限查看并脱敏。

Cost Attribution Coverage

一句话定义：Cost Attribution Coverage 是检查 GPU 小时、存储、网络、checkpoint、推理 token、评估和 Agent 工具调用成本是否归因到 tenant、project、user、job 和 model。

Prod Experiment Separation

一句话定义：Prod Experiment Separation 是检查生产推理和实验训练是否在 namespace、节点池、网络、权限、镜像审核、发布流程和 SLO 上分离。

Cross Tenant Sharing Governance

一句话定义：Cross Tenant Sharing Governance 是检查基础模型、公共数据集、benchmark、基础镜像和工具平台等共享资源是否有 owner、版本、只读默认、复制审批、成本分摊和使用审计。

Audit Evidence Readiness

一句话定义：Audit Evidence Readiness 是检查 policy decision、data access、secret access、image admission、cost ledger 和 trace 是否可查询、可追责、可回放。

Blast Radius Control

一句话定义：Blast Radius Control 是检查单租户事故、镜像污染、密钥泄露、网络误配或存储权限错误会影响多少租户，并是否有 kill switch 和故障域标签。

Multi-Tenant Isolation Gate

一句话定义：Multi-Tenant Isolation Gate 是判断 AI 平台多租户是否达到资源、权限、数据、网络、运行时、供应链、密钥、审计和成本隔离最低要求的综合门禁。

典型硬阻断：namespace 无 quota、身份和 namespace 不一致、权限通配、跨租户数据读未阻断、restricted 数据产物被降级、网络默认互通、容器 privileged / hostPath / root、未签名镜像放行、长期共享高权限密钥、日志 / trace 未脱敏、成本无法归因、生产和实验混池、共享资源无版本和 owner、审计日志缺失、单事故影响多个租户或没有多租户隔离门禁。

Cluster Capacity Planning Audit

一句话定义：Cluster Capacity Planning Audit 是把任务画像、GPU 数量、GPU 型号、训练队列、推理峰值、网络、存储、利用率余量、故障冗余、增长预测、成本预算、资源池隔离和观测回填放进同一套集群容量门禁。

核心直觉：AI 集群容量规划不是“预算能买多少 GPU”，而是先确认要支撑哪些训练、推理、评估、RAG、Agent 和交互式任务，再把 GPU-hours、峰值 QPS、token 长度、网络通信、checkpoint 写入、模型加载、故障冗余、增长场景和成本上限一起审计。GPU 数量算对但网络、存储、推理峰值或预测回填缺失，仍然是不可上线的容量计划。

常见公式：训练 GPU-hours 可以写成

$$H_{\{\mathrm{gpu}\}} = \sum_{k=1}^K n_k g_k t_k r_k$$

考虑峰值、故障、碎片和增长后的 GPU 需求可以写成

$$G_{\{\mathrm{need}\}} = \left\lceil \frac{H_{\{\mathrm{gpu}\}}}{D \cdot 24 \cdot u_{\{\mathrm{eff}\}}} \right\rceil \left(1 + h_{\{\mathrm{peak}\}} + h_{\{\mathrm{frag}\}} + h_{\{\mathrm{fault}\}} + h_{\{\mathrm{growth}\}}\right)$$

推理实例需求可以写成

$$I_{\{\mathrm{serve}\}} = \left\lceil \frac{\lambda_{\{\mathrm{peak}\}} (L_{\{\mathrm{in}\}} + L_{\{\mathrm{out}\}})}{\mu_{\{\mathrm{inst}\}}} \right\rceil$$

容量规划门禁可以写成

$$G_{\{\mathrm{capacity}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \& \ G_{\{\mathrm{actual}\}} \geq G_{\{\mathrm{need}\}} \ \& \ I_{\{\mathrm{actual}\}} \geq I_{\{\mathrm{need}\}} \right]$$

面试表达：容量规划题要先问任务画像和 SLO，再用 GPU-hours 估算训练容量，用峰值 QPS、输入输出 token、TTFT / TPOT 和单实例吞吐估算推理容量；随后补 GPU 型号分层、网络 collective、存储吞吐、checkpoint 生命周期、利用率余量、N+1、增长预测、成本预算和 dashboard 回填。最后用 Cluster Capacity Planning Gate 收束，而不是只报一个 GPU 数字。

Workload Forecast Coverage

一句话定义：Workload Forecast Coverage 是检查预训练、SFT、RLHF / RLAIIF、评估、推理、RAG、Agent、多模态和交互式开发等任务是否有数量、时长、SLO、截止时间和增长假设。

GPU Count Capacity Fit

一句话定义：GPU Count Capacity Fit 是检查实际可用 GPU 数是否覆盖 GPU-hours 折算后的训练需求，并额外考虑峰值、故障、碎片和增长余量。

GPU Type Mix Fit

一句话定义：GPU Type Mix Fit 是检查高端大显存 GPU、中端 GPU、低优或可抢占资源的比例是否匹配预训练、大模型 SFT、推理、评估、embedding 和开发任务。

Training Queue SLO Fit

一句话定义：Training Queue SLO Fit 是检查训练队列 P95 / P99 等待时间是否满足团队和任务优先级约束，而不是只看整体 GPU utilization。

Serving Peak SLO Fit

一句话定义：Serving Peak SLO Fit 是检查推理资源是否覆盖峰值 QPS、输入输出 token、TTFT、TPOT、P99 延迟、KV cache 和自动扩缩容延迟。

Network Bandwidth Capacity Fit

一句话定义：Network Bandwidth Capacity Fit 是检查训练 collective、checkpoint 路径、模型加载和跨机柜流量是否被实测带宽、RDMA、oversubscription、重传和拓扑约束覆盖。

Storage Throughput Capacity Fit

一句话定义：Storage Throughput Capacity Fit 是检查训练数据读取、checkpoint 写入与恢复、模型权重加载和多任务并发读写是否有实测吞吐余量。

Storage Capacity Lifecycle Fit

一句话定义：Storage Capacity Lifecycle Fit 是检查原始数据、清洗数据、训练 shard、checkpoint、模型权重、eval report、日志、trace 和索引是否有容量、保留、归档和删除策略。

Utilization Headroom Fit

一句话定义：Utilization Headroom Fit 是检查目标有效利用率是否保留 burst、维护、故障、扩容和高优任务弹性，避免长期压到无恢复空间。

Failure Redundancy Fit

一句话定义：Failure Redundancy Fit 是检查 N+1、跨机柜、推理多副本、checkpoint 多副本、维护窗口和故障恢复容量是否进入容量预算。

Growth Forecast Fit

一句话定义：Growth Forecast Fit 是检查 3 个月、6 个月、12 个月预测、保守 / 中性 / 激进场景、触发扩容阈值和预测偏差回填是否完整。

Cost Budget Fit

一句话定义：Cost Budget Fit 是检查 GPU、网络、存储、checkpoint、日志、推理 token、云上弹性、运维和闲置成本是否落在预算与归因模型内。

Pool Separation Fit

一句话定义：Pool Separation Fit 是检查预训练、推理、评估、交互式开发和低优任务是否按 SLO、抢占性、GPU 型号和安全边界分池。

Quota Burst Governance

一句话定义：Quota Burst Governance 是检查租户配额、burst credit、借用 / 出借、超额抢占、成本归因和人工审批是否配套。

Observability Forecast Feedback

一句话定义：Observability Forecast Feedback 是检查 GPU 分配率、有效利用率、队列等待、碎片、推理延迟、网络、存储、checkpoint 增长、成本和预测偏差是否进入 dashboard 并周期复盘。

Cluster Capacity Planning Gate

一句话定义：Cluster Capacity Planning Gate 是判断 AI 集群容量计划是否达到可支撑训练、推理、网络、存储、成本、增长和故障恢复的最低上线门禁。

典型硬阻断：任务画像缺失、GPU 数低于需求、GPU 型号混配错误、训练队列 SLO 缺失、推理峰值未建模、网络带宽不足、存储吞吐未实测、checkpoint 生命周期容量不足、目标利用率过高、没有 N+1 或故障余量、增长预测缺失、成本超过预算、训练推理混池、quota / burst 无治理、dashboard 不含预测回填或没有集群容量规划门禁。

Training Platform Lifecycle Audit

一句话定义：Training Platform Lifecycle Audit 是把训练平台从脚本启动器升级为覆盖提交、校验、调度、启动、数据、checkpoint、观测、实验追踪、artifact、权限、成本、恢复和状态机的生命周期门禁。

核心直觉：训练平台不是让用户少写几行 `bash train.sh`，而是把一次训练任务的关键决策变成可验证对象。Kubernetes Job、队列系统、训练 operator、`torchrun`、实验追踪和模型仓库分别解决不同层的问题；平台要把它们串成 TrainingJob 生命周期，而不是把任一工具单独说成完整平台。

常见公式：训练任务审计样本可以写成

$$x_i = (j_i, c_i, r_i, e_i, d_i, l_i, k_i, o_i, m_i, a_i, s_i, u_i)$$

其中， j_i 是 TrainingJob 契约， c_i 是配置校验， r_i 是资源和配额， e_i 是镜像、代码和环境， d_i 是数据版本和权限， l_i 是 launcher 与分布式配置， k_i 是 checkpoint 策略， o_i 是观测信号， m_i 是实验追踪和模型产物， a_i 是权限审计， s_i 是失败恢复和状态机， u_i 是用户自助体验。

可复现字段覆盖率可以写成

$$R_{\{\mathrm{repr}\}} = \frac{K_{\{\mathrm{captured}\}}}{K_{\{\mathrm{required}\}}}$$

训练平台综合门禁可以写成

$$G_{\{\mathrm{train_platform}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{repr}\}} \geq \gamma \ \& \ A_{\{\mathrm{Li}\}} \right]$$

面试表达：训练平台设计题先把 TrainingJob 作为核心对象，说明它要绑定 image digest、code commit、config version、dataset version、resources、distributed config、checkpoint 和 queue；再讲配置校验、权限配额、调度、launcher、数据接入、checkpoint、logs / metrics / events、experiment tracking、artifact registry、失败恢复、成本归因和审计。最后用 Training Platform Gate 收束，而不是只说“支持提交任务、看日志和保存 checkpoint”。

TrainingJob Contract

一句话定义：TrainingJob Contract 是训练任务的平台化契约，描述 owner、project、image、code commit、entrypoint、config、dataset、resource request、distributed config、checkpoint、queue、priority 和 tags。

Config Validation

一句话定义：Config Validation 是在任务进入队列前校验 schema、镜像、代码、数据、权限、资源、checkpoint 路径、launcher 参数和 dry run，避免排队后才失败。

Resource Quota Binding

一句话定义：Resource Quota Binding 是把训练任务的 GPU 类型、GPU 数、每节点 GPU、queue、priority 和 tenant quota 绑定，防止任务绕过配额或抢占策略。

Image Code Reproducibility

一句话定义：Image Code Reproducibility 是要求训练镜像使用 digest、代码使用 commit、依赖锁定并捕获环境 metadata，保证训练环境可复现。

Dataset Lineage Permission

一句话定义：Dataset Lineage Permission 是同时检查数据集版本、数据血缘、数据分类和访问授权，避免训练数据只表现为一个无审计路径。

Launcher Distributed Fit

一句话定义：Launcher Distributed Fit 是检查 torchrun、DeepSpeed、Ray、MPI 或自研 launcher 的节点数、进程数、master、端口和资源 shape 是否一致。

Checkpoint Resume Policy

一句话定义：Checkpoint Resume Policy 是检查 checkpoint output、保存频率、manifest、恢复演练和最大丢失 step 是否满足恢复 SLO。

Observability Events Metrics

一句话定义：Observability Events Metrics 是检查日志、指标和生命周期事件是否覆盖 rank 日志、loss、tokens/s、step time、GPU、通信、dataloader、checkpoint 和状态流转。

Experiment Tracking Lineage

一句话定义：Experiment Tracking Lineage 是把参数、指标、随机种子、artifact、eval report、代码、数据和环境记录到 experiment / run 中，支持比较和复现。

Artifact Registry Linkage

一句话定义：Artifact Registry Linkage 是把 final / best checkpoint、tokenizer、config、adapter、eval report、training metadata 和权限状态注册为可发布资产。

Failure Recovery Classification

一句话定义：Failure Recovery Classification 是区分节点故障、网络抖动、存储短暂失败、OOM、配置错误、代码 bug 和权限错误，决定重试、恢复、快速失败或人工介入。

Security Audit Control

一句话定义：Security Audit Control 是检查提交任务、读取数据、读取日志、取消任务、恢复训练、注册模型和发布模型的权限与审计是否完整。

Cost Attribution Control

一句话定义：Cost Attribution Control 是把 GPU-hours、CPU / memory、存储、checkpoint、网络、评估推理、日志和 trace 成本归因到 team、project、user、job、model 和 queue。

Developer Self Service

一句话定义：Developer Self Service 是训练平台为用户提供 CLI、SDK、Portal、模板、状态页、失败提示和诊断报告，让训练能力产品化而不是依赖平台同学人工代跑。

Lifecycle State Machine

一句话定义：Lifecycle State Machine 是把 submitted、validated、queued、scheduled、running、checkpointing、failed、retrying、completed 和 cancelled 等状态及合法迁移显式化。

Training Platform Gate

一句话定义：Training Platform Gate 是判断训练平台是否达到可提交、可复现、可观测、可恢复、可治理和可审计最低门槛的综合门禁。

典型硬阻断：TrainingJob 契约缺字段、未做配置 dry run、资源申请未绑定 queue / quota、镜像只用 tag、代码或依赖未锁定、数据权限缺失、launcher 进程数和 GPU 数不一致、checkpoint 无 manifest 或未做恢复演练、缺 rank 日志和生命周期事件、实验追踪缺 seed / eval report、artifact 未注册、失败盲目重试、日志未脱敏、成本无法归因、用户无状态页或状态机允许非法跳转。

Training Submission Audit

一句话定义：Training Submission Audit 是把训练任务提交入口从“命令提交器”升级为覆盖 schema、镜像、代码、数据、命令、配置、资源、队列、幂等、模板、状态机和 trace 的控制面门禁。

核心直觉：训练任务提交系统是训练平台的第一道治理关口。它要在任务进入队列前绑定 image digest、code commit、dataset version、final config snapshot、structured command、distributed shape、checkpoint URI、queue / priority、client request id 和 audit trace，避免任务排队后才发现无法复现、无权限、配额越界或重复创建。

常见公式：提交审计样本可以写成

$$z_i = (m_i, s_i, e_i, c_i, d_i, u_i, r_i, l_i, k_i, o_i, q_i, a_i, h_i, t_i, v_i)$$

其中， m_i 是 metadata， s_i 是 schema， e_i 是镜像环境， c_i 是代码版本， d_i 是数据版本和权限， u_i 是启动命令， r_i 是资源申请， l_i 是 launcher， k_i 是 checkpoint， o_i 是日志指标， q_i 是队列优先级， a_i 是 admission / dry run， h_i 是幂等信息， t_i 是模板版本， v_i 是状态机和审计 trace。

必填字段覆盖率可以写成

$$R_{\{\mathrm{required}\}} = \frac{K_{\{\mathrm{present}\}}}{K_{\{\mathrm{required}\}}}$$

训练提交综合门禁可以写成

$$G_{\{\mathrm{train_submission}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{required}\}} \geq \gamma \wedge R_{\{\mathrm{mat}\}} \right]$$

面试表达：训练任务提交系统设计题先讲 TrainingJob schema，再按镜像 digest、代码 commit / diff hash、数据版本权限、结构化 command / args、最终配置快照、资源与分布式一致性、checkpoint、日志指标、queue / priority、ResourceQuota / dry run、client request id 幂等、模板版本和状态机展开。最后用 Training Submission Gate 收束，而不是只说“提供 CLI 提交任务”。

TrainingJob Schema Completeness

一句话定义：TrainingJob Schema Completeness 是检查 TrainingJob 是否包含 metadata、image、code、data、command、config、resources、distributed、checkpoint、observability、queue、admission、idempotency、template、state 和 audit 等结构化对象。

Required Field Coverage

一句话定义：Required Field Coverage 是检查 owner、project、job name、image reference、repo、dataset id、entrypoint、config schema、GPU 数、launcher、checkpoint output、日志指标、queue、priority、client request id 和 actor 等必填字段是否完整。

Image Digest Binding

一句话定义：Image Digest Binding 是要求训练镜像使用不可变 digest，并检查可信 registry、安全扫描和 CUDA / driver 兼容性，避免只记录可漂移 tag。

Code Commit Diff Binding

一句话定义：Code Commit Diff Binding 是要求训练代码绑定不可变 commit；如果允许本地 diff，必须把 diff 打包并记录 hash。

Dataset Version Permission

一句话定义：Dataset Version Permission 是同时检查 dataset id、dataset version、数据血缘、访问权限、数据等级和 artifact 权限继承。

Structured Command Safety

一句话定义：Structured Command Safety 是把启动命令拆成 entrypoint、args、env、working directory 和 launcher 字段，避免任意 shell 字符串绕过校验和审计。

Final Config Snapshot

一句话定义：Final Config Snapshot 是记录合并 YAML、命令行参数、模板默认值和配置中心覆盖后的最终生效配置快照。

Distributed Resource Consistency

一句话定义：Distributed Resource Consistency 是检查 $\text{gpu_count} = \text{nnodes} * \text{nproc_per_node}$ 且 $\text{world_size} = \text{gpu_count}$ ，并确认 rendezvous 和端口策略已经声明。

Checkpoint URI Resume Validation

一句话定义：Checkpoint URI Resume Validation 是检查 checkpoint output 是否可写、resume path 是否存在、格式是否兼容、保留策略是否明确。

Logging Metrics Destination

一句话定义：Logging Metrics Destination 是检查日志 URI、指标目标、rank 日志策略、保留天数和脱敏策略是否在提交时已经声明。

Queue Priority Policy Binding

一句话定义：Queue Priority Policy Binding 是检查用户是否有 queue 和 priority 使用权限，任务是否满足 max runtime、cost budget 和可抢占 checkpoint 策略。

Quota Dry Run Admission

一句话定义：Quota Dry Run Admission 是在进入队列前用服务端 dry run、ResourceQuota、resource flavor 和策略检查确认任务可被平台接收。

Idempotency Duplicate Control

一句话定义：Idempotency Duplicate Control 是使用 client_request_id、submit token 或幂等窗口，保证网络重试返回同一个 job id，而不是重复创建训练任务。

Template Version Governance

一句话定义：Template Version Governance 是要求训练模板有 template id、template version、锁定默认值和变更记录，避免模板更新后历史任务不可解释。

Submit State Machine Validity

一句话定义：Submit State Machine Validity 是检查 draft、submitted、validating、rejected、queued、scheduled、starting、running、completed、failed、retrying 和 cancelled 等状态迁移是否合法。

Submission Audit Trace

一句话定义：Submission Audit Trace 是记录提交事件、提交人、validator 版本、最终 spec hash、admission 决策和审计日志保留状态。

Training Submission Gate

一句话定义：Training Submission Gate 是判断训练任务提交系统是否达到可校验、可复现、可幂等、可排队和可审计最低要求的综合门禁。

典型硬阻断：schema 缺结构、必填字段缺失、镜像只用 tag、代码只写 branch、数据用 latest 或权限缺失、启动命令是任意 shell 字符串、最终配置未快照、分布式资源 shape 不一致、checkpoint output 不可写、日志指标缺失、priority 未审批、ResourceQuota / dry run 未通过、缺 client request id、模板使用 latest、状态机非法跳转、audit trace 缺 final spec hash 或没有提交门禁。

Distributed Launcher Audit

一句话定义：Distributed Launcher Audit 是把 torchrun、DeepSpeed、Megatron、Ray 或 MPI 的启动过程从“拼命令”升级为覆盖资源、rank、rendezvous、GPU 绑定、框架配置、网络、日志、失败恢复和审计 trace 的门禁。

核心直觉：分布式启动器连接调度系统和训练框架。调度系统给出节点、GPU、队列、网络和挂载信息；launcher adapter 要把这些信息转换成 RANK、LOCAL_RANK、WORLD_SIZE、MASTER_ADDR、MASTER_PORT、CUDA_VISIBLE_DEVICES、NCCL 环境和实际启动命令。只要其中一个维度不一致，任务就可能 hang 在初始化、某个 rank 独立 OOM、日志不可查或 checkpoint 无法安全恢复。

常见公式：world size 和资源形状应满足

$$W_{\{\mathrm{world}\}} = N_{\{\mathrm{node}\}} P_{\{\mathrm{node}\}}$$

Megatron 类模型并行配置应满足

$$W_{\{\mathrm{world}\}} = T_{\{\mathrm{tp}\}} P_{\{\mathrm{pp}\}} D_{\{\mathrm{dp}\}}$$

global batch size 可写成

$$B_{\{\mathrm{global}\}} = B_{\{\mathrm{micro}\}} A_{\{\mathrm{grad}\}} D_{\{\mathrm{dp}\}}$$

启动器综合门禁可以写成

$$G_{\{\mathrm{launcher}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \text{and} \ W_{\{\mathrm{cfg}\}} = W_{\{\mathrm{env}\}} \ \text{and} \ P_0 = 0 \right]$$

面试表达：分布式启动器设计题要先讲统一 launcher interface，再讲 torchrun、DeepSpeed、Megatron 和 Ray 的边界；最后用 rank / world size、rendezvous、GPU 绑定、NCCL、日志、失败分类、elastic checkpoint、调度交接和审计 trace 证明平台不是只会拼启动命令。

Resource World Size Consistency

一句话定义：Resource World Size Consistency 是检查 $\text{world_size} = \text{node_count} * \text{nproc_per_node}$ ，并确认该值和资源分配、launcher 参数、worker 环境变量一致。

Rank Local Rank Mapping

一句话定义：Rank Local Rank Mapping 是检查全局 rank 连续唯一、节点内 local rank 与每节点进程数和 GPU 绑定一致。

Rendezvous Endpoint Readiness

一句话定义：Rendezvous Endpoint Readiness 是检查 master address、master port、端口范围、DNS / IP 可达性和冲突策略是否就绪。

GPU Binding Visibility

一句话定义：GPU Binding Visibility 是检查 CUDA_VISIBLE_DEVICES、local rank、每节点 GPU 数、MIG / 共享策略和训练进程可见设备是否一致。

Launcher Adapter Fit

一句话定义：Launcher Adapter Fit 是检查 TrainingJob 声明的 launcher 类型与平台选择的 adapter 一致，并且 adapter 支持当前训练代码生态。

Torchrun Argument Fit

一句话定义：Torchrun Argument Fit 是检查 nnodes、nproc_per_node、node_rank、rendezvous 后端和 master 信息是否与资源分配一致。

DeepSpeed Config Fit

一句话定义：DeepSpeed Config Fit 是检查 DeepSpeed config、ZeRO stage、hostfile 或节点声明、offload、batch 和 launcher 参数是否彼此一致。

Megatron Parallel Consistency

一句话定义：Megatron Parallel Consistency 是检查 tensor parallel、pipeline parallel、data parallel、micro batch、gradient accumulation 和 global batch 的乘法关系是否成立。

Ray Runtime Fit

一句话定义：Ray Runtime Fit 是检查 Ray workers、每 worker GPU、scaling config、runtime env 和外部调度系统资源形状是否匹配。

Network NCCL Readiness

一句话定义：Network NCCL Readiness 是检查训练网卡选择、NCCL 环境变量、RDMA / GPUDirect 路径、master 可达性和通信日志是否可验证。

Log Rank Aggregation

一句话定义：Log Rank Aggregation 是检查所有 rank 日志是否被采集、按 node / rank 索引，并在 worker 失败时能定位非 rank 0 的真实错误。

Failure Stage Classification

一句话定义：Failure Stage Classification 是区分 preflight、container start、distributed init、training runtime 等阶段，避免把网络或 rank 错误误判为训练代码 bug。

Elastic Training Safety

一句话定义：Elastic Training Safety 是检查弹性训练是否具备 checkpoint 兼容、数据重新分片、global batch 变化处理和恢复策略。

Checkpoint Launcher Coupling

一句话定义：Checkpoint Launcher Coupling 是检查 checkpoint output、恢复演练、rank 状态、launcher 状态和重启命令是否能一起恢复。

Scheduler Launcher Handoff

一句话定义：Scheduler Launcher Handoff 是检查调度系统输出的 node list、node rank、GPU 绑定、网络和挂载信息是否完整交给 launcher adapter。

Launcher Audit Trace

一句话定义：Launcher Audit Trace 是记录最终启动命令 hash、环境变量快照、节点和 rank 拓扑、adapter 版本、失败阶段和恢复决策。

Distributed Launcher Gate

一句话定义：Distributed Launcher Gate 是判断分布式启动器是否达到可启动、可解释、可恢复和可审计最低要求的综合门禁。

典型硬阻断：world size 与资源分配不一致；rank 重复或 local rank 错绑 GPU；master 地址不可达；launcher adapter 选错；torchrun 参数缺 node rank；DeepSpeed config 与资源不一致；Megatron 并行维度乘积不等于 world size；Ray worker 资源和外部调度不一致；NCCL 网卡缺失；只采集 rank 0 日志；失败阶段不分类；elastic 训练没有 checkpoint 和数据重分片；调度到 launcher 的 node rank / GPU 绑定缺失；没有最终命令和环境快照。

Training Config Management Audit

一句话定义：Training Config Management Audit 是把训练配置从“保存 YAML”升级为覆盖配置来源、最终快照、版本绑定、校验、复现、恢复、发布、审批、diff 和 trace 的控制面门禁。

核心直觉：训练结果由最终生效配置决定，而不是由某一个 YAML 文件决定。最终配置可能来自默认值、模板、用户 YAML、命令行 override、环境变量、配置中心和平台强制策略。平台必须记录每个字段的最终值、来源、覆盖顺序、merge 规则版本和配置 hash，并把它与 image digest、code commit、dataset version、环境版本、随机种子、checkpoint state、实验 run 和发布配置绑定。

常见公式：最终生效配置可以写成

$$c^{\star} = M(c_{\mathrm{default}}, c_{\mathrm{template}}, c_{\mathrm{yaml}}, c_{\mathrm{cli}}, c_{\mathrm{env}}, c_{\mathrm{batch}} \text{ 和并行配置应满足})$$

$$B_{\mathrm{global}} = B_{\mathrm{micro}} A_{\mathrm{grad}} D_{\mathrm{dp}}$$

可复现证据覆盖率可以写成

$$R_{\mathrm{repr}} = \frac{K_{\mathrm{captured}}}{K_{\mathrm{required}}}$$

训练配置门禁可以写成

$$G_{\mathrm{config}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \text{ \& } R_{\mathrm{required}} \geq \gamma \text{ \& } R_{\mathrm{repr}} \geq \gamma \right]$$

面试表达：训练配置管理题不要停在“保存 YAML”。要讲清配置来源和覆盖顺序、最终配置快照、字段来源、config hash、代码 / 数据 / 镜像 / 环境 / seed / checkpoint / run 绑定、schema 和跨字段校验、resume 兼容、发布配置耦合、审批和 diff，最后用 Training Config Gate 收束。

Config Source Order

一句话定义：Config Source Order 是显式定义 default、template、YAML、CLI override、environment 和 platform policy 的覆盖顺序，并记录每个字段的来源。

Final Config Snapshot Freeze

一句话定义：Final Config Snapshot Freeze 是在任务启动前生成不可变最终配置快照，记录 hash、生成时间、字段来源和 merge 规则版本。

Required Config Field Coverage

一句话定义：Required Config Field Coverage 是检查模型、数据、优化器、batch、并行、checkpoint、环境、随机种子、实验追踪和审计字段是否完整。

Config Schema Type Validation

一句话定义：Config Schema Type Validation 是检查字段类型、枚举、范围和必填项，例如 learning rate 为正数、precision 在允许集合内、world size 为正整数。

Batch Parallel Consistency

一句话定义：Batch Parallel Consistency 是检查 micro batch、gradient accumulation、data parallel、global batch、tensor parallel、pipeline parallel 和 world size 的乘法关系是否一致。

Immutable Version Binding

一句话定义：Immutable Version Binding 是要求 image digest、code commit、dataset version、tokenizer version 和最终配置 hash 使用不可变引用，避免 latest、branch 或漂移路径。

Environment Version Capture

一句话定义：Environment Version Capture 是记录 Python、CUDA、PyTorch、NCCL、driver、GPU 型号、launcher 和 platform 版本，解释性能和数值差异。

Reproducibility Seed Boundary

一句话定义：Reproducibility Seed Boundary 是记录 Python、NumPy、PyTorch、CUDA、dataloader seed 和确定性策略，同时说明严格复现与统计复现边界。

Experiment Tracking Linkage

一句话定义：Experiment Tracking Linkage 是把最终配置、参数、指标、artifact、代码、数据、环境和 eval report 绑定到同一个 experiment / run。

Hyperparameter Sweep Trace

一句话定义：Hyperparameter Sweep Trace 是记录 sweep id、trial id、父配置 hash、变化字段和结果，避免超参搜索只留下一堆不可解释任务。

Checkpoint Resume Compatibility

一句话定义：Checkpoint Resume Compatibility 是检查模型结构、tokenizer、并行策略、optimizer、scheduler、RNG、dataloader cursor 和数据位置是否与 checkpoint 兼容。

Release Config Coupling

一句话定义：Release Config Coupling 是把训练配置中的模型结构、tokenizer、generation config、eval config、serving config、量化配置和数据 lineage 绑定到模型发布资产。

Config Permission Approval

一句话定义：Config Permission Approval 是检查高敏数据、生产资源池、高优先级、关闭日志、禁用 checkpoint、未审核镜像和超长运行时间等配置是否经过策略或审批。

Config Diff Readiness

一句话定义：Config Diff Readiness 是支持两个实验或两次运行之间的配置 diff，帮助解释效果变化来自 learning rate、batch、数据版本、precision、环境还是平台策略。

Config Audit Trace

一句话定义：Config Audit Trace 是记录 actor、validator version、merge rule、final config hash、policy decision、审批记录和审计日志保留状态。

Training Config Gate

一句话定义：Training Config Gate 是判断训练配置管理是否达到可校验、可复现、可恢复、可比较、可审批和可审计最低要求的综合门禁。

典型硬阻断：只保存原始 YAML；命令行 override 没记录；最终配置未冻结；配置 hash 缺失；dataset 使用 latest；image 只用 tag；code 只用 branch；batch / parallel 乘法关系不一致；环境版本缺失；只记录 seed 但没有随机性边界；实验 run 没有最终配置；超参 sweep 没有父配置和 diff；resume checkpoint 时模型结构或 tokenizer 不匹配；发布配置和训练配置脱节；高敏数据或高优先级无审批；没有配置 diff；审计 trace 缺 final config hash。

Training Observability Audit

一句话定义：Training Observability Audit 是把大模型训练任务的 logs、metrics、events、traces、状态机、attempt、实验追踪、告警、权限脱敏、留存成本和成本归因放进同一张可审计表的方法。

核心直觉：训练可观测性不是“多收日志”，而是能回答任务在哪、是否健康、为什么慢、为什么失败、是否可恢复、成本花在哪里，以及训练过程是否能被复盘和复现。日志提供具体上下文，指标提供趋势和聚合，事件提供生命周期变化，trace 串起跨组件阶段耗时，experiment tracking 连接配置、指标、artifact 和最终结果。

常见审计维度：Structured Log Context 看日志是否包含 job、attempt、node、pod、rank、timestamp、level 和 component；Rank Log Coverage 看所有 rank 是否能查询并能定位第一个失败 rank；Metric Taxonomy Coverage 看模型、性能、资源和平台指标是否完整；Metric Dimension Scope 看指标是否能按 job、attempt、rank、node、step 和 timestamp 下钻；Event Schema Lifecycle 看生命周期事件是否结构化；State Machine Validity 看状态迁移是否合法；Attempt Retry Traceability 看多次运行尝试和 checkpoint 恢复是否可区分；Trace Phase Coverage 看 submit、validate、queue、schedule、startup、distributed init、training、checkpoint、eval 和 artifact registration 是否有 span；Dashboard Diagnostic Readiness 看 dashboard 是否覆盖训练曲线、资源、诊断和成本；Alert Rule Readiness 和 Anomaly Detection Rules 看高风险异常是否能触发；Retention Cost Governance、Privacy Redaction Access 和 Cost Attribution Signal 分别检查留存、脱敏权限和成本归因。

常见公式：rank 日志覆盖率可以写成

$$R_{\{\mathrm{rank}\}} = \frac{N_{\{\mathrm{logged}\}}}{N_{\{\mathrm{rank}\}}}$$

状态迁移合法性可以写成

$$V_{\{\mathrm{state}\}} = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[(s_{k-1}, s_k) \in E_{\{\mathrm{allowed}\}}]$$

trace 阶段覆盖率可以写成

$$R_{\{\mathrm{span}\}} = \frac{K_{\{\mathrm{observed}\}}}{K_{\{\mathrm{required}\}}}$$

训练可观测性门禁可以写成

$$G_{\{\mathrm{obs}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{rank}\}} \geq \gamma \wedge V_{\{\mathrm{state}\}} = 1 \wedge \dots \right]$$

面试表达：训练可观测性题不要停在“Prometheus + Grafana”或“看 stdout”。要讲清结构化日志、rank 下钻、模型 / 性能 / 资源 / 平台指标、生命周期事件、状态机、attempt、trace、实验追踪、告警、异常检测、留存成本、权限脱敏和成本归因，最后用 Training Observability Gate 收束。

Structured Log Context

一句话定义：Structured Log Context 是要求每条训练日志至少带 job、attempt、node、pod、rank、timestamp、level 和 component 等上下文字段，避免分布式排障时只能看无归属文本。

Rank Log Coverage

一句话定义：Rank Log Coverage 是衡量所有训练 rank 是否都能被查询、过滤和聚合的覆盖率，尤其要发现非 rank 0 的第一个失败或慢 rank。

Metric Taxonomy Coverage

一句话定义：Metric Taxonomy Coverage 是检查模型指标、性能指标、资源指标和平台指标是否完整，避免只看 loss 曲线就判断训练健康。

Metric Dimension Scope

一句话定义：Metric Dimension Scope 是检查指标是否带 job、attempt、rank、node、GPU、step 和 timestamp 等维度，支持从全局趋势下钻到具体 rank 或节点。

Event Schema Lifecycle

一句话定义：Event Schema Lifecycle 是把 submitted、validated、queued、scheduled、started、checkpoint saved、worker failed、retry started、completed 等生命周期变化写成结构化事件。

State Machine Validity

一句话定义：State Machine Validity 是检查训练任务状态迁移是否符合允许图，防止出现 running 回到 queued、completed 后又 running 或 failed 后 completed 等非法状态。

Attempt Retry Traceability

一句话定义：Attempt Retry Traceability 是区分逻辑 job、多次 attempt 和具体 run，让日志、指标、事件和 checkpoint 恢复过程不会混在一起。

Trace Phase Coverage

一句话定义：Trace Phase Coverage 是检查提交、校验、排队、调度、镜像拉取、启动、分布式初始化、训练、checkpoint、评估和 artifact 注册等阶段是否有 trace span。

Dashboard Diagnostic Readiness

一句话定义：Dashboard Diagnostic Readiness 是要求训练 dashboard 不只展示 loss，还要覆盖任务概览、训练曲线、资源使用、rank 差异、通信、数据加载、checkpoint、错误事件和成本。

Alert Rule Readiness

一句话定义：Alert Rule Readiness 是检查 loss NaN、GPU 长时间低利用、step time 突增、checkpoint 长时间失败、worker failure、重试过多和预算超限等规则是否就绪。

Anomaly Detection Rules

一句话定义：Anomaly Detection Rules 是用固定阈值、滑动窗口或同类任务基线发现 loss spike、tokens/s 下降、rank straggler、checkpoint 变慢和数据吞吐下降。

Retention Cost Governance

一句话定义：Retention Cost Governance 是给高频原始指标、降采样指标、失败日志、发布模型关联日志、审计事件和归档数据设置不同留存周期和成本策略。

Privacy Redaction Access

一句话定义：Privacy Redaction Access 是对训练日志、trace、工具输出和样本片段做 RBAC、脱敏、敏感字段检测、导出审计和保留控制，防止排障日志变成泄露通道。

Cost Attribution Signal

一句话定义：Cost Attribution Signal 是把 GPU hours、存储、网络、日志指标存储、owner、project、queue 和 experiment run 关联起来，解释每次训练的成本归属。

Training Observability Gate

一句话定义：Training Observability Gate 是判断训练任务是否达到可定位、可下钻、可关联、可告警、可恢复、可复盘、可脱敏和可归因最低要求的综合门禁。

典型硬阻断：日志缺 job / attempt / rank；只采 rank 0 日志；指标没有 rank / node / step 维度；只有 loss 没有 step time、GPU、通信、数据加载和 checkpoint 指标；事件不是结构化 schema；状态机非法；attempt 混在一起；trace 缺分布式初始化或 checkpoint span；dashboard 不能诊断慢点；loss NaN 没告警；异常检测缺失；实验 run 无法跳到日志和 checkpoint；日志无脱敏权限；高频指标无限期保留；成本无法归到 owner 或 project；没有最终观测门禁。

Checkpoint Strategy Audit

一句话定义：Checkpoint Strategy Audit 是把 checkpoint 保存频率、保留策略、异步保存、断点恢复、抢占、评估、发布、成本、权限、监控和恢复演练放进同一张可审计策略表的方法。

核心直觉：Checkpoint 策略不是“每隔多少 step 保存一次”的单参数问题，而是在训练阻塞、最大丢失进度、恢复时间、存储成本、评估需求、发布需求和可抢占资源效率之间做工程权衡。第 10 章的 Checkpoint Lifecycle Audit 更关注 checkpoint 对象是否完整、可恢复、可治理；Checkpoint Strategy Audit 更关注何时保存、保存哪类、保留多久、从哪里恢复以及和平台生命周期如何联动。

常见审计维度：RPO RTO Budget 看最大丢失进度和恢复耗时是否有目标；Save Trigger Policy 看是否支持按 step、按时间、milestone 和 preemption notice 保存；Checkpoint Scope Separation 看 resume、eval、release 三类 checkpoint 是否分清；Async Snapshot Consistency 和 Manifest Commit Integrity 看异步保存是否有一致 snapshot、临时目录、checksum 和 committed-only 恢复语义；Shard Metadata Resharding 看分片元数据是否支持恢复和可能的 resharding；Restore Selection Policy 看是否只从健康、已提交、配置兼容、数据可访问、权限允许的 checkpoint 恢复；Preemption Checkpoint Coupling 看可抢占队列是否绑定 checkpoint 新鲜度；Evaluation Best Linkage 和 Release Checkpoint Readiness 看 checkpoint 是否能驱动 best eval 和发布注册；Retention Tier Lifecycle、Storage Cost Budget、Checkpoint Monitoring Alerts、Restore Drill Coverage 和 Permission Release Protection 则检查保留、成本、告警、恢复演练和发布权限。

常见公式：按 step 保存的最大丢失时间可以写成

$$A_{\mathrm{step}} = \frac{I_{\mathrm{step}} T_{\mathrm{step}}}{60}$$

混合保存触发下的最大丢失时间可以写成

$$A_{\mathrm{lost}} = \min(A_{\mathrm{step}}, A_{\mathrm{time}})$$

保存阻塞比例可以写成

$$R_{\mathrm{block}} = \frac{T_{\mathrm{block}}}{I_{\mathrm{step}} T_{\mathrm{step}} + T_{\mathrm{block}}}$$

保留成本可以写成

$$K_{\mathrm{retain}} = \sum_{k=1}^K N_{kS} k P_k + K_{\mathrm{request}} + K_{\mathrm{ops}}$$

Checkpoint 策略门禁可以写成

$$G_{\mathrm{ckpt_strategy}} = \mathbf{1} \setminus \left[A_{\mathrm{lost}} \leq R_{\mathrm{P0}} \wedge T_{\mathrm{restore}} \leq R_{\mathrm{T0}} \right]$$

面试表达：checkpoint 策略题不要只回答“每 1000 step 保存”。要先讲 RPO/RTO，再讲保存触发、resume / eval / release 分层、异步 snapshot 和 manifest commit、分片元数据、恢复选择、抢占联动、评估 best、发布注册、保留成本、监报告警、恢复演练和权限保护，最后用 Checkpoint Strategy Gate 收束。

RPO RTO Budget

一句话定义：RPO RTO Budget 是用可接受的最大丢失进度和恢复耗时约束 checkpoint 保存频率、保存大小、加载路径和恢复流程。

Save Trigger Policy

一句话定义：Save Trigger Policy 是把按 step、按时间、milestone、评估点、发布候选和 preemption notice 等触发条件统一纳入平台策略。

Checkpoint Scope Separation

一句话定义：Checkpoint Scope Separation 是区分 resume、eval、release checkpoint 的内容和用途，避免所有 checkpoint 都保存完整训练状态或发布版本缺 tokenizer / config / eval report。

Async Snapshot Consistency

一句话定义：Async Snapshot Consistency 是异步保存前先在一致边界冻结需要保存的状态，避免模型参数、optimizer、scheduler、RNG 和 dataloader cursor 来自不同 step。

Manifest Commit Integrity

一句话定义：Manifest Commit Integrity 是先写临时目录和 shard checksum，全部完成后再提交 manifest，并且恢复时只读取 committed checkpoint。

Shard Metadata Resharding

一句话定义：Shard Metadata Resharding 是在分片 checkpoint 中记录 rank map、tensor shards、global step、format version 和 reshard policy，支撑恢复和并行策略变化。

Restore Selection Policy

一句话定义：Restore Selection Policy 是规定从 latest committed、用户指定、最近健康、best eval 或 milestone checkpoint 恢复时需要执行的完整性、配置、数据、权限和事件记录校验。

Preemption Checkpoint Coupling

一句话定义：Preemption Checkpoint Coupling 是把可抢占任务、preemption notice、checkpoint 新鲜度和调度器抢占选择联动，降低抢占造成的训练进度损失。

Evaluation Best Linkage

一句话定义：Evaluation Best Linkage 是在 checkpoint 后触发版本化评估，把 eval report、best checkpoint、模型指标和训练 run 关联起来。

Release Checkpoint Readiness

一句话定义：Release Checkpoint Readiness 是发布前检查权重转换、tokenizer、config、generation config、推理加载测试、评估报告、安全审核、metadata 注册和权限标记。

Retention Tier Lifecycle

一句话定义：Retention Tier Lifecycle 是把最近 resume、周期 milestone、best eval、release 和 failed temporary checkpoint 放到不同保留周期和存储层。

Storage Cost Budget

一句话定义：Storage Cost Budget 是用 checkpoint 大小、保留个数、保留天数、存储层价格和请求运维成本约束 checkpoint 策略的月度成本。

Checkpoint Monitoring Alerts

一句话定义：Checkpoint Monitoring Alerts 是监控 save / load duration、checkpoint age、失败率、临时目录堆积、存储用量、恢复成功率和抢占无新鲜 checkpoint 等指标并触发告警。

Restore Drill Coverage

一句话定义：Restore Drill Coverage 是定期演练从 checkpoint 恢复训练和加载发布模型，证明保存成功真的等于可恢复。

Permission Release Protection

一句话定义：Permission Release Protection 是对 release checkpoint、best eval、归档版本和高敏数据关联 checkpoint 做 RBAC、审计、保护标记和归档策略控制。

Checkpoint Strategy Gate

一句话定义：Checkpoint Strategy Gate 是判断 checkpoint 策略是否达到低损失恢复、低训练阻塞、可控存储成本、可抢占、可评估、可发布、可审计和可演练最低要求的综合门禁。

典型硬阻断：没有 RPO/RTO；只按 step 保存但 step time 变化导致 RPO 失控；所有 checkpoint 都保存完整训练状态；只保留 latest；异步保存没有 snapshot / temp path / checksum / manifest commit；恢复读取半成品；分片 metadata 缺失；抢占不看 checkpoint 新鲜度；best eval 没有关联 eval report；release checkpoint 缺 tokenizer、config 或加载测试；保留策略无限增长；没有 checkpoint save / load / age / failure 告警；从未做恢复演练；release checkpoint 权限开放；没有最终策略门禁。

Training Fault Tolerance Audit

一句话定义：Training Fault Tolerance Audit 是把节点失败、GPU 故障、通信 hang、OOM、数据权限、checkpoint 恢复、数值异常、自动重试、诊断报告和用户动作放进同一张可审计容错表的方法。

核心直觉：训练容错不是无限重试，而是先判断故障是否可恢复，再用有边界的 attempt、backoff、checkpoint 恢复和节点 / GPU 隔离控制损失。可恢复故障要自动恢复，不可恢复故障要快速失败并给出诊断，否则平台会把 GPU 花在重复失败上。

常见审计维度：Fault Taxonomy Coverage 看故障分类是否覆盖 node、GPU、network、OOM、data、checkpoint、config 和 numeric；Retryability Classification 看 retryable / non-retryable 是否和错误类型一致；Retry Budget Bounded 和 Retry Backoff Policy 看 attempt、timeout、backoff 和 jitter 是否有边界；Checkpoint Resume Readiness 看 committed checkpoint、manifest、checksum、配置兼容和训练状态是否完整；Node Failure Reschedule、GPU Health Isolation 和 Communication Hang Detection 看节点替换、坏 GPU 隔离、rank heartbeat、首个失败 rank、NCCL 日志和 worker 退出状态是否可见；OOM Handling Policy 看 OOM 是否快速失败、给出显存峰值和调参建议，而不是原样重试；Data Permission Fast Fail、Checkpoint Commit Safety 和 Numeric Instability Stop Rollback 看权限错误、半成品 checkpoint 和 NaN 是否被阻断；Diagnosis Report Evidence、Blacklist Precision、User Action Clarity 和 Training Fault Tolerance Gate 则检查报告、黑名单误伤、用户可见动作和最终门禁。

常见公式：故障样本可以写成

$$f_i = (c_i, r_i, a_i, b_i, k_i, n_i, g_i, h_i, o_i, d_i, p_i, s_i, u_i, t_i, z_i)$$

统一覆盖率可以写成

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(f_i) = 1]$$

失败后的最大丢失 GPU-hours 可以写成

$$L_{\{\mathrm{gpu}\}} = G \frac{t_{\{\mathrm{fail}\}} - t_{\{\mathrm{ckpt}\}}}{60}$$

无效重试率可以写成

$$R_{\{\mathrm{waste}\}} = \frac{N_{\{\mathrm{bad_retry}\}}}{N_{\{\mathrm{retry}\}}}$$

OOM 显存余量可以写成

$$H_{\{\mathrm{mem}\}} = 1 - \frac{M_{\{\mathrm{peak}\}}}{M_{\{\mathrm{limit}\}}}$$

训练容错门禁可以写成

$$G_{\{\mathrm{fault}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \ \& \ L_{\{\mathrm{gpu}\}} \leq B_{\{\mathrm{loss}\}} \ \& \ R_{\{\mathrm{waste}\}} \leq \tau_r]$$

面试表达：训练容错题不要只说“失败自动重试”。要先讲故障分类和 retryability，再讲 max attempts、backoff、checkpoint committed 恢复、节点替换、GPU 隔离、NCCL hang 诊断、OOM 快速失败、数据权限快速失败、NaN 回滚、诊断报告和用户动作，最后用 Training Fault Tolerance Gate 收束。

Fault Taxonomy Coverage

一句话定义：Fault Taxonomy Coverage 是检查训练平台是否覆盖节点、GPU、网络、OOM、数据、checkpoint、配置、代码和数值稳定性等故障类型。

Retryability Classification

一句话定义：Retryability Classification 是把故障分成可自动重试、需要调整配置后重试、必须快速失败和需要人工介入几类，避免对权限、配置、持续 OOM 和 NaN 盲目重试。

Retry Budget Bounded

一句话定义：Retry Budget Bounded 是为每个训练任务设置最大 attempt、总重试时长、失败次数阈值和不可恢复错误短路规则。

Retry Backoff Policy

一句话定义：Retry Backoff Policy 是用指数退避、固定退避或带 jitter 的等待策略控制自动重试节奏，避免任务集体重启压垮控制面或存储。

Checkpoint Resume Readiness

一句话定义：Checkpoint Resume Readiness 是确认恢复点是 committed checkpoint, 且 manifest、checksum、配置兼容、optimizer、scheduler、RNG 和 dataloader cursor 都可用。

Node Failure Reschedule

一句话定义：Node Failure Reschedule 是节点或 worker 失败后重新选择健康节点，重启 worker group，并从最新健康 checkpoint 恢复。

GPU Health Isolation

一句话定义：GPU Health Isolation 是用 Xid、ECC、GPU reset、掉卡和多任务失败证据隔离坏 GPU 或坏节点，避免同一硬件反复拖垮训练任务。

Communication Hang Detection

一句话定义：Communication Hang Detection 是用 rank heartbeat、首个失败 rank、NCCL 日志、worker 退出状态和网络指标区分网络问题、rank OOM、配置错误和节点失联。

OOM Handling Policy

一句话定义：OOM Handling Policy 是区分 GPU OOM、CPU OOM 和 container OOM，并给出 batch、sequence length、activation checkpointing、FSDP / ZeRO 和显存峰值建议，而不是原样重试。

Data Permission Fast Fail

一句话定义：Data Permission Fast Fail 是在启动前和读取时快速识别数据权限、路径、shard 和格式错误，对权限类错误直接失败并生成诊断。

Checkpoint Commit Safety

一句话定义：Checkpoint Commit Safety 是只允许恢复 committed checkpoint，禁止读取临时目录、半成品 shard 或 checksum / manifest 不完整的 checkpoint。

Numeric Instability Stop Rollback

一句话定义：Numeric Instability Stop Rollback 是对 loss NaN、Inf、梯度爆炸和 AMP overflow 停止训练、保存诊断并回滚到健康 checkpoint，而不是盲目重试。

Diagnosis Report Evidence

一句话定义：Diagnosis Report Evidence 是把失败时间、阶段、首个失败 rank、错误类型、日志、指标、checkpoint、attempt 和建议动作放进可读报告。

Blacklist Precision

一句话定义：Blacklist Precision 是用硬件和系统证据决定 node / GPU blacklist，同时避免把用户代码 OOM、配置错误或数据错误误判成坏节点。

User Action Clarity

一句话定义：User Action Clarity 是让用户看到当前状态、重试次数、恢复 checkpoint、损失进度、是否需要操作和下一步建议。

Training Fault Tolerance Gate

一句话定义：Training Fault Tolerance Gate 是判断训练任务是否达到可分类、可恢复、可短路、低损失、可隔离、可诊断、可解释和可审计最低要求的综合门禁。

典型硬阻断：故障分类缺失；权限、配置、持续 OOM 和 NaN 被自动重试；attempt 无上限；没有 backoff；checkpoint 未 committed 就恢复；节点失败后仍调度到原节点；Xid / ECC 多次失败但不隔离；NCCL hang 无 rank heartbeat 和日志；OOM 无显存峰值和建议；数据权限错误被重试；数值异常继续训练；诊断报告缺日志、指标和建议；用户只看到 failed；黑名单把用户错误误伤成硬件故障；没有最终训练容错门禁。

Training Data Supply Audit

一句话定义：Training Data Supply Audit 是把大模型训练数据从 manifest、shard、流式读取、缓存、shuffle、rank 切分、DataLoader、H2D copy、坏样本、观测和 checkpoint 数据状态放进同一张可审计表的方法。

核心直觉：训练数据供给不是“路径能打开”，而是在保证数据版本、权限、shuffle 和可复现性的前提下，把数据以足够高、足够稳定的吞吐送到 GPU。GPU 利用率低时，根因可能是对象存储、缓存、解析、预处理、DataLoader worker、H2D copy、rank 不均衡或坏样本，而不一定是模型算子慢。

常见审计维度：Dataset Manifest Integrity 看 dataset id、版本、shard、checksum、token 数、预处理版本和权限元数据是否完整；Shard Format Size Fit 和 Small File Amplification Control 看格式、索引、shard 大小、小文件数量和 metadata 压力是否适合训练；Streaming Prefetch Retry、Local Cache Policy 和 Distributed Cache Policy 看流式读取、预取、重试、本地缓存、分布式缓存、版本隔离和容量治理；Shuffle Reproducibility 与 Rank Shard Assignment 看 shard shuffle、shuffle buffer、seed、set_epoch、rank 不重不漏和恢复 cursor；Data Locality Fit 看训练节点是否靠近数据和缓存；Dataloader Parallelism Fit 与 H2D Copy Readiness 看 worker、prefetch、CPU 预处理、pinned memory 和异步拷贝；Bad Sample Threshold、Data Supply Observability、Checkpoint Dataloader State、Permission Version Governance 和 Training Data Supply Gate 则检查坏样本阈值、指标、断点恢复数据状态、权限版本和最终门禁。

常见公式：训练数据供给样本可以写成

$$d_i = (m_i, s_i, a_i, u_i, c_i, q_i, r_i, \ell_i, p_i, h_i, b_i, o_i, k_i, v_i, z_i)$$

数据供给吞吐可以写成

$$Q_{\{\mathrm{data}\}} = \frac{N_{\{\mathrm{tok}\}}}{T_{\{\mathrm{load}\}}}$$

GPU 等数据比例可以写成

$$R_{\{\mathrm{wait}\}} = \frac{T_{\{\mathrm{data_wait}\}}}{T_{\{\mathrm{step}\}}}$$

缓存命中率可以写成

$$R_{\{\mathrm{cache}\}} = \frac{B_{\{\mathrm{cache}\}}}{B_{\{\mathrm{cache}\}} + B_{\{\mathrm{remote}\}}}$$

坏样本率可以写成

$$R_{\{\mathrm{bad}\}} = \frac{N_{\{\mathrm{bad}\}}}{N_{\{\mathrm{seen}\}}}$$

训练数据供给门禁可以写成

$$G_{\{\mathrm{data}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \text{and} \ Q_{\{\mathrm{data}\}} \geq \gamma \ Q_{\{\mathrm{train}\}} \ \text{and} \ R_{\{\mathrm{m}\}} \right]$$

面试表达：训练数据供给题不要只回答“用 DataLoader 读数据”。要先讲离线预处理、shard 和 manifest，再讲 streaming / cache、shuffle 和 rank 切分，接着讲 DataLoader、H2D、坏样本、监控和 checkpoint 数据状态，最后用 Training Data Supply Gate 收束。

Dataset Manifest Integrity

一句话定义：Dataset Manifest Integrity 是检查数据集版本、shard 列表、token 数、checksum、预处理版本、来源和权限元数据是否完整且不可变。

Shard Format Size Fit

一句话定义：Shard Format Size Fit 是检查训练数据格式、shard 大小、索引、压缩和可并行读取能力是否适合大规模训练。

Small File Amplification Control

一句话定义：Small File Amplification Control 是控制小文件数量、metadata 请求和打开文件开销，避免对象存储或共享文件系统被小文件放大效应拖垮。

Streaming Prefetch Retry

一句话定义：Streaming Prefetch Retry 是为远端流式读取配置预取、超时、重试和吞吐监控，避免网络抖动直接变成 GPU 等数据。

Local Cache Policy

一句话定义：Local Cache Policy 是把热点 shard 按数据版本预热或缓存到本地 NVMe，并用容量、LRU 和命中率治理缓存。

Distributed Cache Policy

一句话定义：Distributed Cache Policy 是用共享缓存层缓解对象存储压力，同时保证版本一致性、容量规划和缓存层不成为新瓶颈。

Shuffle Reproducibility

一句话定义：Shuffle Reproducibility 是在 shard shuffle、shuffle buffer、epoch seed、set_epoch 和 rank 切分之间平衡随机性、吞吐和可复现。

Rank Shard Assignment

一句话定义：Rank Shard Assignment 是给每个 distributed rank 分配不重不漏且负载均衡的 shard，并记录恢复 cursor。

Data Locality Fit

一句话定义：Data Locality Fit 是让训练节点尽量靠近数据或缓存，减少跨区域读取、网络拥塞、读取延迟和不必要流量成本。

Dataloader Parallelism Fit

一句话定义：Dataloader Parallelism Fit 是检查 DataLoader worker、persistent worker、prefetch、CPU 预处理和 batch packing 是否支撑训练吞吐。

H2D Copy Readiness

一句话定义：H2D Copy Readiness 是检查 CPU 到 GPU 拷贝是否使用 pinned memory、异步 copy 和合理 copy 预算，避免最后一段传输卡住。

Bad Sample Threshold

一句话定义：Bad Sample Threshold 是记录坏样本位置、坏样本率和失败阈值，避免无限跳过坏样本导致训练分布静默变化。

Data Supply Observability

一句话定义：Data Supply Observability 是采集 dataloader time、data wait、storage throughput、cache hit、shard latency、bad sample、tokens loaded、CPU preprocess、H2D 和 per-rank lag 等指标。

Checkpoint Dataloader State

一句话定义：Checkpoint Dataloader State 是在训练 checkpoint 中记录 epoch、global step、shard index、sample offset、shuffle seed 和 worker state，避免恢复后重复或跳过数据。

Permission Version Governance

一句话定义：Permission Version Governance 是把数据权限、数据分类、dataset version 和 preprocessing version 绑定到训练任务和 manifest。

Training Data Supply Gate

一句话定义：Training Data Supply Gate 是判断训练数据供给是否达到版本正确、高吞吐、低等待、缓存有效、shuffle 可复现、rank 不重不漏、坏样本可控和可恢复最低要求的综合门禁。

典型硬阻断：没有 manifest；数据版本可变；checksum 缺失；shard 过小或无索引；小文件数量过多；流式读取无预取和重试；缓存无版本隔离；分布式缓存变成瓶颈；shuffle 无 seed；rank 读重或漏读；跨区域读训练数据；DataLoader worker 不足；H2D copy 阻塞；坏样本无限跳过；没有 data wait / cache hit / per-rank lag 指标；checkpoint 不保存 dataloader state；数据权限未校验；没有最终数据供给门禁。

Data Platform Supply Chain Audit

一句话定义：Data Platform Supply Chain Audit 是把大模型数据从 source registry、raw lake、cleaning、dedup、PII redaction、quality scoring、dataset builder、manifest、version、permission、lineage、observability 到训练读取吞吐放进同一张可审计表的方法。

核心直觉：数据平台不是文件目录，也不只是训练时 DataLoader 的上游。它要证明某批数据从哪里来、为什么能用、经过哪版清洗和去重、是否阻断 PII 和污染、manifest 是否完整、版本是否不可变、用途权限是否匹配、血缘是否覆盖，以及训练平台能否以足够吞吐稳定消费。

常见审计维度：Source Registry Coverage 看来源、owner、license、用途、分类、接入时间和 checksum；Raw Data Lake Integrity 看原始对象是否不可变、checksum 和审计日志是否完整；Cleaning Policy Versioning、Dedup Contamination Guard 和 PII Redaction Coverage 看清洗、去重、污染隔离和隐私阻断是否版本化；Quality Scoring Calibration 和 Dataset Builder Recipe 看质量分、slice 统计、配方和采样权重是否可解释；Dataset Manifest Completeness 和 Dataset Version Immutability 看 manifest 与版本是否完整且冻结；Streaming Shard Readiness、Read Throughput Fit 和 Data Wait Fit 看 shard、streaming、cache、预取、重试、读取吞吐和 GPU 等数据比例；Permission Purpose Binding、Data Lineage Coverage、Data Platform Supply Observability 和 Data Platform Supply Gate 则检查用途权限、血缘、指标告警和最终门禁。

常见公式：数据平台样本可以写成

$d_i = (s_i, r_i, c_i, v_i, m_i, q_i, p_i, g_i, l_i, t_i, h_i, o_i, z_i)$

来源注册覆盖率可以写成

$C_{\{\mathrm{source}\}} = \frac{K_{\{\mathrm{source}, \mathrm{ok}\}}}{K_{\{\mathrm{source}, \mathrm{total}\}}}$

Manifest 完整率可以写成

$$C_{\{\mathrm{manifest}\}}=\frac{K_{\{\mathrm{manifest},\mathrm{filled}\}}}{K_{\{\mathrm{manifest},\mathrm{required}\}}}$$

PII 泄露率可以写成

$$R_{\{\mathrm{pii},\mathrm{leak}\}}=\frac{N_{\{\mathrm{pii},\mathrm{unblocked}\}}}{N_{\{\mathrm{pii},\mathrm{total}\}}}$$

血缘覆盖率可以写成

$$C_{\{\mathrm{lineage}\}}=\frac{N_{\{\mathrm{samples},\mathrm{linked}\}}}{N_{\{\mathrm{samples},\mathrm{total}\}}}$$

有效读取吞吐和等数据比例可以写成

$$Q_{\{\mathrm{read}\}}=\frac{N_{\{\mathrm{tok}\}}}{T_{\{\mathrm{read}\}}}$$

$$R_{\{\mathrm{wait}\}}=\frac{T_{\{\mathrm{wait},\mathrm{data}\}}}{T_{\{\mathrm{step}\}}}$$

数据平台供给门禁可以写成

$$G_{\{\mathrm{data}\}}=\mathbf{1}\left[\min_j C_j\geq\tau_j\ \wedge\ R_{\{\mathrm{pii},\mathrm{leak}\}}=0\ \wedge\ C_{\{\mathrm{lineage}\}}\geq\rho\right]$$

面试表达：数据平台题不要直接跳到“Parquet + 对象存储 + DataLoader”。先讲来源和 raw lake，再讲清洗、去重、PII、质量、builder 和 manifest，然后讲版本、权限、血缘和观测，最后用读取吞吐、等数据比例和 Data Platform Supply Gate 证明它能被训练平台可靠消费。

Source Registry Coverage

一句话定义：Source Registry Coverage 是检查数据源的来源、owner、许可证、用途、分类、接入时间、原始路径和 checksum 是否完整登记。

Raw Data Lake Integrity

一句话定义：Raw Data Lake Integrity 是检查原始数据对象是否不可变、可校验、可审计、可按来源和时间回溯，并且不会被清洗产物覆盖。

Cleaning Policy Versioning

一句话定义：Cleaning Policy Versioning 是把解析、语言识别、低质过滤、格式规范化、安全过滤和保留率统计绑定到明确策略版本。

Dedup Contamination Guard

一句话定义：Dedup Contamination Guard 是同时检查重复样本、近似重复、train / eval overlap、benchmark 污染和去重规则版本的门禁。

PII Redaction Coverage

一句话定义：PII Redaction Coverage 是检查检测到的个人可识别信息是否被删除、脱敏、隔离或审批，不允许未阻断 PII 直接进入训练集。

Quality Scoring Calibration

一句话定义：Quality Scoring Calibration 是检查质量评分策略、校准报告、slice 统计和用途边界是否可解释，避免把所有数据混成一个不可追溯总分。

Dataset Builder Recipe

一句话定义：Dataset Builder Recipe 是记录数据来源、清洗策略、采样权重、split、权限约束和输出格式的可复现数据集构建配方。

Dataset Manifest Completeness

一句话定义：Dataset Manifest Completeness 是检查 manifest 是否包含 dataset id、version、sources、recipe、清洗 / 去重 / PII 策略版本、shard、样本数、token 数、checksum、权限版本、lineage run id 和质量统计。

Dataset Version Immutability

一句话定义：Dataset Version Immutability 是保证同一个 dataset version 指向不可变 manifest、shard checksum、配方和权限快照。

Streaming Shard Readiness

一句话定义：Streaming Shard Readiness 是检查 shard 格式、大小、checksum、索引、streaming、shuffle、预取、重试和缓存版本隔离是否适合训练读取。

Permission Purpose Binding

一句话定义：Permission Purpose Binding 是把 actor、project、dataset version、classification、allowed purpose 和训练任务提交绑定，避免“有路径就能读”。

Data Lineage Coverage

一句话定义：Data Lineage Coverage 是检查样本、shard、dataset version、清洗任务、去重任务、builder run、训练任务、checkpoint 和 eval report 是否能连成血缘链。

Data Platform Supply Observability

一句话定义：Data Platform Supply Observability 是让数据平台暴露来源覆盖、manifest 完整、PII 泄露、血缘覆盖、读取吞吐、等数据比例、权限拒绝和质量分布等指标与告警。

Read Throughput Fit

一句话定义：Read Throughput Fit 是判断数据平台提供给训练任务的有效 token 读取吞吐是否高于训练侧消耗速度。

Data Wait Fit

一句话定义：Data Wait Fit 是判断训练 step 中等待数据的时间比例是否低于阈值，避免 GPU 因数据供给链路空转。

Data Platform Supply Gate

一句话定义：Data Platform Supply Gate 是判断数据平台是否达到来源完整、原始数据可回溯、清洗去重可复现、PII 零泄露、manifest 完整、版本冻结、权限用途绑定、血缘覆盖、观测可用、吞吐足够和 P0 风险清零的综合门禁。

典型硬阻断：来源缺 owner、license 或 checksum；raw data 可覆盖；清洗策略未版本化；train / eval overlap 未隔离；PII 未阻断；质量分无校准；builder 配方缺失；manifest 缺 shard checksum 或 lineage run id；dataset version 可变；streaming shard 不可读；权限不绑定用途；血缘覆盖不足；缺读取吞吐和等数据指标；Q_read 低于训练消耗速度；R_wait 超阈值；没有最终数据平台供给门禁。

Data Version Lineage Quality Audit

一句话定义：Data Version Lineage Quality Audit 是把 dataset version、manifest checksum、不可变 snapshot、diff report、lineage graph、影响分析、质量指标、漂移监控、泄漏监控、告警路由和训练 / 模型版本联动放进同一张审计表的方法。

核心直觉：数据版本治理不是给目录起名字，数据血缘也不是只服务合规。大模型训练要能解释“为什么这次训练变好或变差”，就必须知道用的是哪个不可变数据版本、它和上一个版本差在哪里、哪些 source / rule / shard 参与了构建、哪些训练任务和模型消费了它，以及新版本是否通过质量和安全门禁。

常见审计维度：Dataset Version Contract 看 dataset id、版本、父版本、owner、schema、tokenizer、recipe 和 split seed 是否完整；Manifest Checksum Integrity 看 manifest、shard checksum、样本数和 token 数是否一致；Immutable Snapshot Policy 看发布后是否冻结；Delta Version Diff 看新增、删除、token delta、来源比例和规则变化是否解释清楚；Lineage Graph Completeness 与 Impact Analysis Readiness 看 source、job、dataset、training run、checkpoint、eval、model 和 deployment 是否可达；Schema Quality Gate、Content Quality Gate、Data Safety Compliance Gate、Distribution Drift Monitoring、Train Eval Leakage Monitoring、Annotation Quality Monitoring 和 Synthetic Data Monitoring 检查 schema、重复、低质、PII、license、分布漂移、评估泄漏、标注和合成数据；Quality Alert Routing 和 Training Model Version Linkage 检查告警上下文、TrainingJob 和 ModelVersion 是否绑定；最后用 Data Version Quality Gate 收束。

常见公式：dataset version 可以写成

$$V_k=(D_k,M_k,R_k,S_k,P_k,Q_k,L_k,H_k,A_k)$$

血缘图可以写成

$$G_{\{\mathrm{lin}\}}=(N,E)$$

血缘覆盖率可以写成

$$C_{\{\mathrm{lineage}\}}=\frac{N_{\{\mathrm{linked}\}}}{N_{\{\mathrm{expected}\}}}$$

离散分布漂移可以写成

$$D_{\{\mathrm{tv}\}}(p,q)=\frac{1}{2}\sum_{c\in C}|p_c-q_c|$$

数据质量门禁可以写成

$$G_{\{\mathrm{quality}\}}=\mathbf{1}\left[C_{\{\mathrm{manifest}\}}=1\ \wedge\ C_{\{\mathrm{checksum}\}}=1\ \wedge\ C_{\{\mathrm{lineage}\}}=1\right]$$

面试表达：这类题先讲 version contract 和 immutable snapshot，再讲 manifest checksum 和 diff report，然后用 lineage graph 做删除、撤权和污染影响分析，接着列 schema、重复、PII、license、drift、overlap、annotation、synthetic data 等质量门禁，最后说明告警必须带 dataset、version、source、job 和 rule version，并把 TrainingJob / ModelVersion 绑定到数据版本。

Dataset Version Contract

一句话定义：Dataset Version Contract 是 dataset id、version、parent version、owner、schema、tokenizer、recipe、split seed、权限和状态机组成的数据版本契约。

Manifest Checksum Integrity

一句话定义：Manifest Checksum Integrity 是检查 manifest checksum、shard checksum、样本数、token 数和文件列表是否一致。

Immutable Snapshot Policy

一句话定义：Immutable Snapshot Policy 是保证已发布 dataset version 不被覆盖，修复或变更必须产生新版本。

Delta Version Diff

一句话定义：Delta Version Diff 是记录两个数据版本之间新增、删除、token delta、来源比例、规则版本和质量指标变化的报告。

Lineage Graph Completeness

一句话定义：Lineage Graph Completeness 是检查 source、ingestion、raw dataset、cleaning、dedup、dataset version、training job、checkpoint、eval report、model version 和 deployment 是否连成可追踪图。

Impact Analysis Readiness

一句话定义：Impact Analysis Readiness 是在数据删除、撤权、许可证风险或污染发现后，能沿血缘图定位受影响 dataset、训练任务、checkpoint、模型和 deployment。

Schema Quality Gate

一句话定义：Schema Quality Gate 是用 schema 合法率、解析成功率、空字段比例、标签缺失率和多模态文件存在率拦截结构异常数据版本。

Content Quality Gate

一句话定义：Content Quality Gate 是用重复率、低质量比例、乱码比例、长度分布、语言 / 领域比例和质量分布分布拦截内容异常数据版本。

Data Safety Compliance Gate

一句话定义：Data Safety Compliance Gate 是用 PII 未阻断率、secret 命中率、许可证风险比例、敏感等级和 train / eval overlap 拦截安全合规风险。

Distribution Drift Monitoring

一句话定义：Distribution Drift Monitoring 是持续比较新旧数据版本在语言、领域、来源、长度、合成比例和质量分上的分布差异。

Train Eval Leakage Monitoring

一句话定义：Train Eval Leakage Monitoring 是对 train 和 eval 之间的 exact duplicate、near duplicate、benchmark contamination 和来源交叉进行持续监控。

Annotation Quality Monitoring

一句话定义：Annotation Quality Monitoring 是监控标注员一致性、gold set 准确率、标签分布、冲突样本、审核通过率和标注指南版本。

Synthetic Data Monitoring

一句话定义：Synthetic Data Monitoring 是监控合成数据生成模型、prompt 版本、采样参数、合成比例、多样性、事实性、难度分布和偏差。

Quality Alert Routing

一句话定义：Quality Alert Routing 是让质量告警带上 dataset、version、source、job、rule version 和责任 owner，避免只有指标没有可执行定位。

Training Model Version Linkage

一句话定义：Training Model Version Linkage 是把 TrainingJob 的 dataset version、manifest checksum、tokenizer、sampling seed、代码和环境记录到 ModelVersion 血缘中。

Data Version Quality Gate

一句话定义：Data Version Quality Gate 是判断数据版本是否达到契约完整、manifest 和 checksum 一致、snapshot 不可变、diff 可解释、血缘完整、影响分析可用、质量和安全指标过线、告警可路由、训练 / 模型版本可追溯的综合门禁。

典型硬阻断：dataset version 缺 owner、schema 或 tokenizer；manifest checksum 缺失；shard 数和样本数不一致；发布后覆盖 snapshot；diff report 缺失；lineage graph 断边；删除请求无法定位影响模型；schema 失败率升高；重复率异常；PII 或许可证风险未阻断；语言 / 来源分布漂移无解释；train / eval overlap 非零；标注一致性过低；合成数据比例过高；告警无 owner；TrainingJob 只记录路径；ModelVersion 无数据版本；没有最终数据版本质量门禁。

Model Artifact Registry Audit

一句话定义：Model Artifact Registry Audit 是把 ModelVersion、权重 manifest、safetensors、tokenizer、config、adapter、merge、量化版本、runtime、评估、安全权限、发布状态、回滚、lineage、缓存和生命周期放进同一张模型仓库审计表的方法。

核心直觉：模型仓库不是对象存储目录，也不是“能下载权重就能上线”。大模型线上行为由权重、tokenizer、chat template、model config、adapter、merge 过程、量化配置、runtime 支持和评估报告共同决定；任何一个产物错配，都可能导致加载失败、输出格式异常、质量退化、越权下载或无法回滚。

常见审计维度：Model Version Contract 看 model id、version、architecture、owner、stage、训练任务、数据版本、评估报告、安全报告、发布 alias 和审批是否完整；Weight Manifest Integrity 与 Safe Weight Format 看权重分片、checksum、manifest、safetensors、安全扫描和 pickle 风险；Tokenizer Config Binding 看 tokenizer hash、config hash、chat template、special tokens 和 stop tokens 是否与 ModelVersion 绑定；Adapter Base Compatibility 与 Merge Lineage Completeness 看 adapter base model、rank、target modules、merge 工具、merge 后 eval 和 lineage；Quantization Eval Gate 与 Runtime Compatibility Fit 看量化方法、校准数据、质量退化、目标 runtime、上下文长度、tensor parallel 和加载 smoke test；Eval Report Linkage、Safety Permission Gate、Release Status Governance、Rollback Alias Readiness、Artifact Lineage Completeness、Load Cache Readiness、Lifecycle Retention Policy 和 Model Registry Gate 则检查发布证据、安全权限、回滚、血缘、冷启动、保留策略和最终门禁。

常见公式：模型版本可以写成

$$a_i=(m_i,v_i,w_i,t_i,c_i,b_i,p_i,q_i,r_i,e_i,s_i,l_i,o_i,z_i)$$

元数据完整率可以写成

$$C_{\{\mathrm{artifact}\}}=\frac{K_{\{\mathrm{filled}\}}}{K_{\{\mathrm{required}\}}}$$

权重 checksum 覆盖率可以写成

$$C_{\{\mathrm{weight}\}}=\frac{N_{\{\mathrm{checked}\}}}{N_{\{\mathrm{weight,total}\}}}$$

tokenizer 和 config 绑定门禁可以写成

$$B_{\{\mathrm{tok}\}}=\mathbf{1}[h_{\{\mathrm{tok}\}}=h_{\{\mathrm{tok,expected}\}} \wedge h_{\{\mathrm{cfg}\}}=h_{\{\mathrm{cfg,expected}\}}]$$

量化质量退化可以写成

$$\Delta Q=Q_{\{\mathrm{base}\}}-Q_{\{\mathrm{quant}\}}$$

模型仓库门禁可以写成

$$G_{\{\mathrm{registry}\}}=\mathbf{1}[\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{weight}\}}=1 \wedge B_{\{\mathrm{tok}\}}=1 \wedge \Delta Q \leq \tau_Q]$$

面试表达：这类题先区分 Model、ModelVersion 和 Artifact，再讲权重 manifest / checksum / safetensors，然后讲 tokenizer、config、adapter、merge 和量化版本必须绑定到同一个模型契约；接着讲 runtime 兼容、评估报告、安全权限、发布状态、灰度和回滚；最后用 Model Registry Gate 收束，说明模型仓库连接训练平台、评估平台和推理平台，是模型生命周期控制面。

Model Version Contract

一句话定义：Model Version Contract 是 model id、version、architecture、参数规模、owner、stage、训练任务、数据版本、权重 manifest、tokenizer、config、eval report、安全报告、发布 alias 和审批组成的模型版本契约。

Weight Manifest Integrity

一句话定义：Weight Manifest Integrity 是检查权重文件列表、分片大小、manifest checksum、每个 shard checksum、重复路径和总大小是否一致。

Safe Weight Format

一句话定义：Safe Weight Format 是要求生产权重使用 safetensors 或经过安全扫描的格式，并避免不可信 pickle 反序列化风险。

Tokenizer Config Binding

一句话定义：Tokenizer Config Binding 是把 tokenizer hash、model config hash、chat template、special tokens、stop tokens 和上下文长度绑定到具体 ModelVersion。

Adapter Base Compatibility

一句话定义：Adapter Base Compatibility 是检查 LoRA / QLoRA 等 adapter 的 base model version、rank、target modules、dtype、tokenizer 和训练配置是否与预期基座兼容。

Merge Lineage Completeness

一句话定义：Merge Lineage Completeness 是记录哪个 base model 加哪个 adapter、用哪个工具和配置生成哪个 merged model，并要求 merge 后重新评估。

Quantization Eval Gate

一句话定义：Quantization Eval Gate 是检查量化方法、bits、校准数据、工具版本、目标 runtime、质量退化、延迟吞吐和 eval report 是否满足上线阈值。

Runtime Compatibility Fit

一句话定义：Runtime Compatibility Fit 是检查目标 runtime 是否支持当前权重格式、tokenizer、上下文长度、量化格式、tensor parallel、CUDA / driver 和加载 smoke test。

Eval Report Linkage

一句话定义：Eval Report Linkage 是让 ModelVersion 关联通用能力、领域能力、安全、工具调用、结构化输出、长上下文、延迟吞吐和成本评估报告。

Safety Permission Gate

一句话定义：Safety Permission Gate 是检查模型安全评估、许可证、敏感等级、授权项目、下载角色、导出限制和审计日志是否满足发布策略。

Release Status Governance

一句话定义：Release Status Governance 是用 registered、validated、evaluated、staged、production、deprecated 和 archived 等状态机治理模型发布和下线。

Rollback Alias Readiness

一句话定义：Rollback Alias Readiness 是保证生产 alias、回滚 alias、回滚版本、回滚加载测试和路由策略已经预先准备好。

Artifact Lineage Completeness

一句话定义：Artifact Lineage Completeness 是让 dataset version、training job、checkpoint、model version、adapter、merged model、quantized model、eval report 和 deployment 组成可追踪血缘图。

Load Cache Readiness

一句话定义：Load Cache Readiness 是检查模型产物是否有本地缓存、预热、checksum 校验、加载耗时 SLO 和冷启动回退策略。

Lifecycle Retention Policy

一句话定义：Lifecycle Retention Policy 是定义 production、candidate、deprecated、archived 和实验 checkpoint 的保留、冷存储、删除依赖检查和审计要求。

Model Registry Gate

一句话定义：Model Registry Gate 是判断模型仓库是否达到版本契约完整、权重安全可校验、tokenizer / config 绑定、adapter / merge / quantization 可追踪、runtime 可加载、评估和安全过线、发布可回滚、生命周期可治理的综合门禁。

典型硬阻断：ModelVersion 缺 owner、训练任务、数据版本或 eval report；权重分片没有 checksum；使用不可信 pickle 权重；tokenizer 或 config hash 与记录不一致；adapter 绑定错 base model；merge 后没有 lineage 和 eval；量化质量退化超阈值；目标 runtime 不支持该格式或量化；安全评估、许可证或权限未过线；production 发布缺审批；没有 rollback alias；lineage graph 断边；冷启动加载超过 SLO；生产模型可被误删；没有最终模型仓库门禁。

Artifact Management Audit

一句话定义：Artifact Management Audit 是把 dataset、checkpoint、eval report、deployment package、release manifest、日志、配置和模型产物统一纳入元数据、checksum、权限、血缘、promotion、回滚、删除依赖和生命周期门禁的方法。

核心直觉：Artifact 不是普通文件路径，而是 AI 研发和生产流程中的可复现证据。训练任务读了什么数据、产出了哪个 checkpoint，评估报告绑定了哪个模型和 eval 数据，部署包用了哪个 runtime image 和安全策略，回滚依赖哪些旧产物，这些都应该能从 artifact 系统里查到、校验和审计。

常见审计维度：Artifact Metadata Contract 看 artifact id、type、name、version、URI、checksum、size、producer job、依赖、权限、lifecycle 和 tags 是否完整；Dataset Artifact Manifest 看 dataset version、split、schema、文件列表、样本数、token 数、来源组成、清洗 / 去重规则、质量指标和权限；Checkpoint Artifact Recoverability 看 model、optimizer、scheduler、data loader、RNG、distributed metadata、shard checksum

和 restore test; Eval Report Reproducibility 看模型、eval 数据、评估代码、metric definition、prompt、inference config、runtime、样本输出和 failure cases; Deployment Package Completeness 与 Release Manifest Integrity 看权重、tokenizer、config、runtime image digest、安全策略、路由策略、health check、eval report 和 rollback target; Checksum Integrity Gate、Artifact Lineage Completeness、Artifact Store Metadata Split、Permission Access Control、Lifecycle Retention Policy、Promotion Gate Readiness、Experiment Tracking Linkage、Rollback Artifact Readiness、Deletion Dependency Safety 和 Artifact Management Gate 则检查完整性、血缘、存储分层、权限、保留、晋级、实验追踪、回滚、删除和最终门禁。

常见公式：artifact 记录可以写成

$A_i = (u_i, \tau_i, n_i, v_i, r_i, h_i, s_i, o_i, d_i, p_i, l_i, g_i, e_i, z_i)$

元数据完整率可以写成

$C_{\{\mathrm{meta}\}} = \frac{K_{\{\mathrm{filled}\}}}{K_{\{\mathrm{required}\}}}$

checksum 覆盖率可以写成

$C_{\{\mathrm{checksum}\}} = \frac{N_{\{\mathrm{checked}\}}}{N_{\{\mathrm{files, total}\}}}$

artifact 血缘图可以写成

$G_{\{\mathrm{art}\}} = (V_{\{\mathrm{art}\}}, E_{\{\mathrm{dep}\}})$

deployment package 完整率可以写成

$C_{\{\mathrm{release}\}} = \frac{N_{\{\mathrm{artifact, present}\}}}{N_{\{\mathrm{artifact, required}\}}}$

删除安全门禁可以写成

$G_{\{\mathrm{delete}\}} = \mathbf{1}[N_{\{\mathrm{dependents}\}} = 0 \wedge T_{\{\mathrm{age}\}} \geq T_{\{\mathrm{retention}\}} \wedge H_{\{\mathrm{ma}\}}]$

最终 artifact 门禁可以写成

$G_{\{\mathrm{artifact}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{checksum}\}} = 1 \wedge C_{\{\mathrm{lineage}\}} \geq \rho]$

面试表达：这类题先说明 artifact 管理比模型仓库更通用，再按 dataset、checkpoint、eval report、deployment package 和 release manifest 分类型讲元数据和门禁；然后讲 artifact store 存大文件、metadata store 存索引，checksum service 做完整性，lineage graph 做追溯，access control 做权限，lifecycle manager 做保留和删除依赖检查；最后用 Artifact Management Gate 收束，证明训练、评估、部署、回滚、审计和成本治理都能靠 artifact 系统复现。

Artifact Metadata Contract

一句话定义：Artifact Metadata Contract 是 artifact id、type、name、version、URI、checksum、size、created time、producer job、dependencies、permissions、lifecycle state 和 tags 组成的产物契约。

Dataset Artifact Manifest

一句话定义：Dataset Artifact Manifest 是把 dataset version、split、schema、file list、sample count、token count、source composition、rule version、quality metrics、permissions 和 checksum 冻结成训练输入证据。

Checkpoint Artifact Recoverability

一句话定义：Checkpoint Artifact Recoverability 是检查 checkpoint 是否包含恢复训练所需的 model、optimizer、scheduler、dataloader、RNG、distributed metadata、shard checksum 和 restore test 证据。

Eval Report Reproducibility

一句话定义：Eval Report Reproducibility 是检查评估报告是否绑定 model / checkpoint、eval dataset、eval code、metric definitions、prompt template、inference config、runtime、sample outputs、failure cases 和 evaluator version。

Deployment Package Completeness

一句话定义：Deployment Package Completeness 是检查部署包是否包含权重、tokenizer、model config、runtime config、image digest、health check、generation config、安全策略、路由策略和 release manifest。

Release Manifest Integrity

一句话定义：Release Manifest Integrity 是用不可变 manifest 记录一次发布使用的所有 artifact 引用、checksum、release id 和 rollback target。

Checksum Integrity Gate

一句话定义：Checksum Integrity Gate 是要求上传、下载、部署和归档前对 artifact 文件、shard 和 manifest 做完整性校验。

Artifact Store Metadata Split

一句话定义：Artifact Store Metadata Split 是大文件进入 artifact store，索引、权限、状态、checksum 和依赖进入 metadata store，避免把大模型权重塞进元数据库。

Permission Access Control

一句话定义：Permission Access Control 是对 dataset、checkpoint、eval sample outputs、deployment package、删除和跨租户使用做 RBAC / ABAC、审批和审计。

Promotion Gate Readiness

一句话定义：Promotion Gate Readiness 是把 checkpoint -> candidate model -> evaluated model -> release candidate -> production package 的每次晋级绑定完整性、评估、安全、runtime 和审批门禁。

Experiment Tracking Linkage

一句话定义：Experiment Tracking Linkage 是让 experiment run 的参数、指标、日志和输入 / 输出 artifact 能互相跳转和复现。

Rollback Artifact Readiness

一句话定义：Rollback Artifact Readiness 是保证旧权重、旧 tokenizer、旧 runtime image、旧配置、旧安全策略、旧 route policy 和旧 release manifest 都仍可加载。

Deletion Dependency Safety

一句话定义：Deletion Dependency Safety 是删除 artifact 前检查下游依赖、retention、legal hold、deployment、rollback、审计和删除记录，避免破坏复现与回滚。

Artifact Management Gate

一句话定义：Artifact Management Gate 是判断 artifact 系统是否达到元数据完整、manifest 可校验、checkpoint 可恢复、eval 可复现、部署包完整、release manifest 不可变、权限可控、血缘可追踪、promotion / rollback / deletion 安全和生命周期可治理的综合门禁。

典型硬阻断：artifact 只有路径没有元数据；dataset manifest 样本数或 token 数不一致；checkpoint 不能恢复；eval report 缺 eval 数据或 prompt 版本；deployment package 缺 tokenizer、runtime image 或安全策略；release manifest 可变；checksum 未验证；artifact lineage 断边；大文件混进 metadata DB；eval sample

outputs 权限开放；retention policy 缺失；promotion 绕过 eval / safety；experiment run 和 artifact 脱节；rollback artifact 被删；删除前未检查依赖；没有最终 artifact 管理门禁。

Experiment Tracking Audit

一句话定义：Experiment Tracking Audit 是把一次训练、评估、prompt 实验或压测 run 的参数、指标、日志、样本、代码、数据、模型、环境、artifact、成本、状态、搜索索引、血缘和结论放进同一套可复现门禁的方法。

核心直觉：Experiment tracking 不是曲线截图，也不是只记录最终分数。它要回答三类问题：第一，结果从哪里来，能否比较；第二，输入、代码、数据、环境和 artifact 是否能复现；第三，实验结论、成本、状态和后续动作是否可能被团队协作使用。训练平台、评估平台、artifact 系统和成本系统都应该围绕同一个 run id 打通。

常见审计维度：Run Metadata Contract 看 run id、experiment id、owner、status、created / ended time、tags 和 notes 是否完整；Parameter Config Capture 看 learning rate、global batch size、optimizer、seed、sequence length、precision、parallel config 和 ZeRO stage 是否记录；Metric Curve Step Capture 看 train / eval 指标是否带 step 或 timestamp；Log Run Linkage 看 stdout、framework log、dataloader log、distributed log、checkpoint log 和 eval log 是否绑定 run id；Sample Result Diagnosis 看 input hash、prediction、reference、score、error type、prompt version 和 evaluator version；Code Version Reproducibility、Data Version Reproducibility、Model Checkpoint Linkage 与 Environment Capture 检查 commit、dirty flag、image digest、dataset manifest、base model、checkpoint、Python / framework / CUDA / driver / NCCL / GPU；Prompt Eval Config Capture、Artifact Output Linkage、Cost Attribution、Run State Transition、Search Index Readiness、Lineage Graph Readiness、Decision Note Capture 和 Experiment Tracking Gate 则检查 prompt / eval 口径、输出产物、成本归因、状态机、可搜索性、血缘、实验结论和最终门禁。

常见公式：run 记录可以写成

$$R_i = (e_i, u_i, a_i, t_i, p_i, m_i, l_i, s_i, c_i, d_i, q_i, o_i, z_i)$$

参数覆盖率可以写成

$$C_{\{\mathrm{param}\}} = \frac{1}{N_{\{\mathrm{param}\}}} \sum_{j=1}^N \mathbf{1}_{[p_j \setminus \mathrm{recorded}]}$$

指标曲线可以写成

$$M_k = \{(r_i, k, t, v) \mid r_i \in \mathcal{R}, t \in \mathcal{T}\}$$

样本诊断覆盖率可以写成

$$C_{\{\mathrm{sample}\}} = \frac{N_{\{\mathrm{samplediag}\}}}{N_{\{\mathrm{sample}\}}}$$

代码可复现门禁可以写成

$$G_{\{\mathrm{code}\}} = \mathbf{1}_{[C_{\{\mathrm{commit}\}}=1 \wedge C_{\{\mathrm{dirty}\}}=0 \wedge C_{\{\mathrm{image}\}}=1 \wedge C_{\{\mathrm{recipe}\}}=1]}$$

数据可复现门禁可以写成

$$G_{\{\mathrm{data}\}} = \mathbf{1}_{[C_{\{\mathrm{dataset}\}}=1 \wedge C_{\{\mathrm{manifest}\}}=1 \wedge C_{\{\mathrm{recipe}\}}=1 \wedge C_{\{\mathrm{recipe}\}}=1]}$$

最终实验追踪门禁可以写成

$$G_{\{\mathrm{track}\}} = \mathbf{1}_{[\min_j C_j \geq \tau_j \wedge G_{\{\mathrm{code}\}}=1 \wedge G_{\{\mathrm{data}\}}=1 \wedge P_0=0]}$$

面试表达：这类题先说明 experiment / run 的对象模型，再分层讲 parameters、metrics、logs、sample results、code / data / environment、model / checkpoint / artifact、cost、state、tags / notes 和 lineage；随后讲存储分层，metadata store 存 run 索引，time series store 存指标，log store 存日志，artifact store 存产物，sample store 存诊断样本；最后用 Experiment Tracking Gate 收束，证明实验结果可比较、可复现、可解释、可审计。

Run Metadata Contract

一句话定义：Run Metadata Contract 是 run id、experiment id、owner、status、created time、ended time、tags、notes、hypothesis、decision 和 next action 组成的实验记录契约。

Parameter Config Capture

一句话定义: Parameter Config Capture 是把 learning rate、batch size、optimizer、seed、sequence length、precision、parallel config、ZeRO stage 和 prompt / eval 配置记录到 run。

Metric Curve Step Capture

一句话定义: Metric Curve Step Capture 是要求每条 metric 都绑定 run id、metric name、step 或 timestamp、value 和 metric definition，避免只有最终分数。

Log Run Linkage

一句话定义: Log Run Linkage 是把训练日志、框架日志、分布式日志、checkpoint 日志、eval 日志和错误栈统一绑定到 run id。

Sample Result Diagnosis

一句话定义: Sample Result Diagnosis 是保存 input hash、prediction、reference、score、error type、prompt version 和 evaluator version，用于定位具体失败样本。

Code Version Reproducibility

一句话定义: Code Version Reproducibility 是记录 Git commit、branch、dirty flag、diff、container image digest、entrypoint 和依赖版本，证明实验代码可复现。

Data Version Reproducibility

一句话定义: Data Version Reproducibility 是记录 dataset id / version、manifest checksum、split、sampling recipe、shuffle seed 和 tokenizer version，避免路径漂移。

Model Checkpoint Linkage

一句话定义: Model Checkpoint Linkage 是把 base model、init checkpoint、adapter、parent run、output checkpoint、final model 和 eval report 连接到同一个 run lineage。

Environment Capture

一句话定义: Environment Capture 是记录 Python、PyTorch、CUDA、driver、NCCL、transformers、GPU 类型、GPU 数量、节点拓扑和容器镜像。

Prompt Eval Config Capture

一句话定义: Prompt Eval Config Capture 是把 prompt template、system prompt、tool schema、generation config、eval dataset、judge model、metric definition 和 evaluator version 绑定到 run。

Artifact Output Linkage

一句话定义: Artifact Output Linkage 是让 checkpoint、final model、eval report、log artifact 和 sample output artifact 都能从 run id 跳转并校验 checksum。

Cost Attribution

一句话定义: Cost Attribution 是把 GPU hours、GPU type、storage、network、training tokens、inference tokens、eval calls 和 total cost 归因到 experiment / run / project。

Run State Transition

一句话定义：Run State Transition 是用 created、running、succeeded、failed、killed、evaluated、release candidate、rejected 和 archived 等状态管理实验生命周期。

Search Index Readiness

一句话定义：Search Index Readiness 是保证 owner、experiment、model、dataset、code commit、metric threshold、tag、status、created time 和 artifact id 可检索。

Lineage Graph Readiness

一句话定义：Lineage Graph Readiness 是让 dataset、code、config、run、checkpoint、eval report、model version 和 deployment package 能在图上双向追踪。

Decision Note Capture

一句话定义：Decision Note Capture 是记录实验 hypothesis、观察、结论、next action 和 rejection reason，让实验结论能被团队复用。

Experiment Tracking Gate

一句话定义：Experiment Tracking Gate 是判断实验追踪系统是否达到元数据完整、参数指标日志样本齐全、代码数据环境可复现、artifact 和 lineage 可追踪、成本状态结论可审计且无 P0 风险的综合门禁。

典型硬阻断：run 缺 owner 或状态；参数缺 seed 或并行配置；metric 没有 step；日志和 run id 脱节；只存聚合分数不存样本诊断；代码 dirty 且没有 diff；dataset 只记录可变路径；checkpoint 和 parent run 断开；环境版本不全；eval prompt、judge model 或 metric definition 缺失；artifact 没有 checksum；成本不可归因；状态从 debug 直接跳到 release candidate；搜索索引缺关键字段；lineage graph 断边；没有实验结论和下一步动作。

Evaluation Platform Audit

一句话定义：Evaluation Platform Audit 是把模型版本、评估数据、prompt、推理配置、指标、人评、LLM judge、pairwise、在线实验、报告、缓存、可观测性、artifact 和发布门禁放进同一套质量控制审计表的方法。

核心直觉：评估平台不是 benchmark 脚本，而是模型质量控制系统。它要回答候选模型是否真的更好、哪些切片退化、judge 和人评是否可信、线上放量是否安全、报告能否复现、发布和回滚证据是否完整。训练平台、实验追踪、模型仓库、artifact 管理、推理发布和安全治理都应该围绕同一份评估证据链协作。

常见审计维度：Eval Dataset Contract 看数据集 id、版本、任务、样本数、切片、标签、来源、构造方式、训练隔离和污染检查是否完整；Offline Batch Reproducibility 看模型、数据、prompt、推理配置、评估代码、runtime 和指标定义是否固定；Metric Definition Versioning 看指标名称、方向、权重、阈值和版本；LLM Judge Calibration、Human Review Quality Control 与 Pairwise Blinding Randomization 检查 judge 校准、人评 gold set / 复审 / 仲裁、盲评随机顺序和 tie 记录；Online Eval Guardrail 与 Release Gate Readiness 检查 A/B 分桶、样本比例、质量、延迟、成本、安全事件、回滚和审批；Eval Report Completeness、Slice Regression Detection、Regression Suite Coverage、Eval Job Scheduler Reliability、Eval Cache Key Integrity、Eval Observability、Artifact Tracking Linkage 和 Evaluation Platform Gate 则检查报告、切片退化、回归集、调度、缓存、监控、血缘和最终门禁。

常见公式：评估任务可以写成

$J_i = (m_i, d_i, p_i, c_i, q_i, g_i, h_i, o_i, r_i, z_i)$

数据集切片覆盖率可以写成

$C_{\{\mathrm{dataset}\}} = \frac{1}{N_{\{\mathrm{slice}\}}} \sum_{s=1}^{N_{\{\mathrm{slice}\}}} \mathbf{1}[n_s \geq n_{\{\min\}}]$

训练 / 评估污染率可以写成

$$R_{\mathrm{leak}} = \frac{N_{\mathrm{overlap}}}{N_{\mathrm{eval}}}$$

指标加权汇总可以写成

$$S_{\mathrm{agg}} = \frac{\sum_{k=1}^K w_k \tilde{M}_k}{\sum_{k=1}^K w_k}$$

pairwise win rate 可以写成

$$W = \frac{N_{\mathrm{win}} + 0.5 N_{\mathrm{tie}}}{N_{\mathrm{win}} + N_{\mathrm{loss}} + N_{\mathrm{tie}}}$$

在线护栏门禁可以写成

$$G_{\mathrm{online}} = \mathbf{1}[\Delta_Q \geq -\epsilon_Q \wedge \Delta_L \leq \epsilon_L \wedge \Delta_K \leq \epsilon_K \wedge \dots]$$

最终评估平台门禁可以写成

$$G_{\mathrm{eval}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge G_{\mathrm{offline}} = 1 \wedge G_{\mathrm{human}} = 1 \wedge \dots]$$

面试表达：这类题先讲对象模型，说明一次 eval 必须绑定 ModelVersion、EvalDataset、PromptVersion、InferenceConfig、MetricDefinition、JudgeConfig、HumanReviewTask、PredictionArtifact、EvalReport 和 ReleaseDecision；然后按离线评测、人评、judge、pairwise、在线评估、切片回归、缓存调度和可观测性展开；最后用 Evaluation Platform Gate 收束，证明评估结果可复现、可比较、可解释、可发布、可回滚。

Eval Dataset Contract

一句话定义：Eval Dataset Contract 是评估数据集的 id、版本、任务类型、样本数、切片分布、标签、来源、构造方式、训练隔离、污染检查和权限组成的数据契约。

Offline Batch Reproducibility

一句话定义：Offline Batch Reproducibility 是保证离线评测固定模型版本、数据版本、prompt 版本、推理配置、评估代码、runtime 和指标定义，使不同 run 的分数可比较。

Metric Definition Versioning

一句话定义：Metric Definition Versioning 是记录每个指标的名称、版本、方向、权重、阈值、计算代码和适用任务，避免同名指标口径漂移。

LLM Judge Calibration

一句话定义：LLM Judge Calibration 是用人工标注、gold set、bias check 和版本化 rubric 校准 judge model，避免把 judge 偏差当成真实模型质量。

Human Review Quality Control

一句话定义：Human Review Quality Control 是用标注指南、培训、gold set、多标注员重复标注、复审、仲裁和耗时监控保证人评结论稳定。

Pairwise Blinding Randomization

一句话定义：Pairwise Blinding Randomization 是在 A/B 输出偏好评测中隐藏模型身份、随机展示顺序、记录 tie 并按切片统计 win rate。

Online Eval Guardrail

一句话定义：Online Eval Guardrail 是在线 A/B 或灰度中检查稳定分桶、样本比例、质量、延迟、成本、安全事件和回滚准备的放量护栏。

Release Gate Readiness

一句话定义：Release Gate Readiness 是检查离线、人评、在线、安全、性能、成本、审批和回滚是否都通过，避免只凭一个总分上线。

Eval Report Completeness

一句话定义：Eval Report Completeness 是检查评估报告是否包含模型版本、数据版本、代码版本、prompt、推理配置、指标定义、总分、切片、样本、失败案例、baseline 对比和决策。

Slice Regression Detection

一句话定义：Slice Regression Detection 是按语言、领域、难度、输入长度、安全类别、工具类型和多轮轮数等切片比较候选模型和 baseline，防止平均分掩盖局部退化。

Regression Suite Coverage

一句话定义：Regression Suite Coverage 是检查回归集是否覆盖历史事故、关键业务、安全边界、工具调用、格式约束和高价值客户场景，并持续加入新失败样本。

Eval Job Scheduler Reliability

一句话定义：Eval Job Scheduler Reliability 是评估调度是否支持批量推理、多模型对比、judge、人评、安全扫描、依赖、重试、优先级、预算和失败分类。

Eval Cache Key Integrity

一句话定义：Eval Cache Key Integrity 是保证评估缓存 key 包含模型、数据、prompt、推理配置、评估代码和 runtime 等会影响输出与指标的版本字段。

Eval Observability

一句话定义：Eval Observability 是评估任务具备状态、吞吐、推理错误率、judge 错误率、成本、样本失败原因、队列等待、结果写入和 trace id 等观测信号。

Artifact Tracking Linkage

一句话定义：Artifact Tracking Linkage 是把 prediction outputs、sample table、judge run、人评结果、eval report、release decision 和 lineage edges 注册为可追踪 artifact。

Evaluation Platform Gate

一句话定义：Evaluation Platform Gate 是判断评估平台是否达到数据、指标、judge、人评、在线、报告、回归、调度、缓存、观测、artifact、发布和回滚证据完整且无 P0 风险的综合门禁。

典型硬阻断：eval dataset 缺版本、切片或污染检查；离线评测未固定 prompt / 推理配置 / 评估代码；指标无版本或方向；LLM judge 未做人类校准；人评缺 gold set、复审或仲裁；pairwise 未盲评或未随机顺序；在线 A/B 分桶不稳定或样本比例异常；发布门禁不含安全、成本、延迟或回滚；报告缺样本级结果；切片退化被总分掩盖；回归集没有历史事故样本；评估调度无重试和失败分类；缓存 key 缺 prompt 或 runtime；评估任务不可观测；eval report 和 release decision 没有 artifact / lineage；没有最终评估平台门禁。

Feature Embedding Index Infrastructure Audit

一句话定义: Feature Embedding Index Infrastructure Audit 是把 Feature Store、Embedding Store、Vector Index、metadata filter、retrieval trace、lineage、multi-tenant、cost 和 RAG / Agent 集成放进同一套数据访问层审计表的方法。

核心直觉: Feature Store 解决结构化特征的定义、版本、离线训练和在线服务一致性; Embedding Store 解决向量资产的模型版本、chunk、权限和索引状态; Vector Index 解决大规模相似度检索的召回、延迟和成本。生产系统不能只说“我们有向量库”,而要证明特征没有 training-serving skew、embedding 没有版本混用、索引切换可回滚、权限过滤不会越权、检索结果可追踪到 source 和 chunk。

常见审计维度: Feature Definition Contract 看特征名称、版本、实体键、来源、transform、窗口、TTL、owner 和权限; Offline Online Consistency 和 Point in Time Correctness 检查 offline / online 特征值、默认值、类型、transform 版本和历史回放是否正确; Embedding Version Contract 和 Chunk Embedding Lineage 检查 embedding model、维度、pooling、normalize、预处理、chunking、offset、checksum、tenant 和 ACL; Vector Index Build Readiness、ANN Quality Latency Gate 与 Metadata Permission Filter 检查索引版本、维度、metric、HNSW / IVF / PQ 参数、recall@k、P95/P99、空结果率、tenant / ACL / 数据等级过滤; Shadow Index Switch Readiness、Retrieval Trace Completeness、Feature Embedding Lineage Graph、Multi Tenant Isolation、Quality Monitoring Metrics、Cost Capacity Governance、RAG Agent Integration 和 Feature Embedding Index Gate 则检查双索引灰度回滚、检索 trace、血缘图、租户隔离、质量监控、成本预算、RAG / Agent 引用和最终门禁。

常见公式: 数据访问链可以写成

$$A_i = (f_i, e_i, c_i, v_i, s_i, r_i, p_i, l_i, z_i)$$

training-serving skew 可以写成

$$R_{\{\mathrm{skew}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[|x^{\{\mathrm{off}\}}_i - x^{\{\mathrm{on}\}}_i| > \epsilon_i \right]$$

point-in-time 泄漏率可以写成

$$R_{\{\mathrm{pit_leak}\}} = \frac{1}{N_{\{\mathrm{join}\}}} \sum_{i,j} \mathbf{1} [t_{\{\mathrm{feature_available}\}}(i,j) > t_{\{\mathrm{feature_available}\}}(j,i)]$$

embedding 版本门禁可以写成

$$G_{\{\mathrm{emb}\}} = \mathbf{1} [C_{\{\mathrm{model}\}} = 1 \wedge C_{\{\mathrm{dim}\}} = 1 \wedge C_{\{\mathrm{chunk}\}} = 1 \wedge C_{\{\mathrm{tenant}\}} = 1]$$

ANN recall@k 可以写成

$$R@k = \frac{|S_{\{\mathrm{ann}\}}(k) \cap S_{\{\mathrm{exact}\}}(k)|}{|S_{\{\mathrm{exact}\}}(k)|}$$

权限泄露率可以写成

$$R_{\{\mathrm{leak}\}} = \frac{N_{\{\mathrm{unauth_returned}\}}}{N_{\{\mathrm{returned}\}}}$$

双索引切换门禁可以写成

$$G_{\{\mathrm{switch}\}} = \mathbf{1} [C_{\{\mathrm{shadow}\}} = 1 \wedge R@k \geq \tau_R \wedge L_{\{\mathrm{p95}\}} \leq \tau_L \wedge C_{\{\mathrm{tenant}\}} = 1]$$

最终门禁可以写成

$$G_{\{\mathrm{feidx}\}} = \mathbf{1} [\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{skew}\}} \leq \rho \wedge R_{\{\mathrm{pit_leak}\}} = 0 \wedge R_{\{\mathrm{leak}\}} = 0 \wedge R_{\{\mathrm{switch}\}} = 1]$$

面试表达: 这类题先区分 Feature Store、Embedding Store 和 Vector Index 的职责; 然后讲特征定义、offline / online、point-in-time、embedding model / chunking / normalize 版本、ANN 索引和 metadata filter; 再讲 shadow index、retrieval trace、lineage、多租户、质量监控和成本; 最后用 Feature Embedding Index Gate 收束, 说明它支撑 RAG、Agent、搜索、推荐和模型路由的数据访问层。

Feature Definition Contract

一句话定义: Feature Definition Contract 是特征名称、版本、owner、entity key、source、transform version、dtype、window、TTL、默认值和权限组成的结构化契约。

Offline Online Consistency

一句话定义: Offline Online Consistency 是检查训练读取的 offline feature 和线上服务读取的 online feature 是否使用同一语义定义、transform、默认值、类型和版本。

Point in Time Correctness

一句话定义: Point in Time Correctness 是训练样本只能读取事件时间之前已经可获得的特征, 避免未来信息泄漏。

Embedding Version Contract

一句话定义: Embedding Version Contract 是 embedding model name / version、dimension、pooling、normalize、preprocess、chunking 和生成时间组成的向量版本契约。

Chunk Embedding Lineage

一句话定义: Chunk Embedding Lineage 是记录 document、chunk id、offset、chunking version、embedding id、checksum、tenant、ACL 和 source pointer 的向量血缘。

Vector Index Build Readiness

一句话定义: Vector Index Build Readiness 是检查索引 id、版本、dimension、metric、index type、index params、embedding model version、chunking version、构建状态和输入 manifest 是否完整。

ANN Quality Latency Gate

一句话定义: ANN Quality Latency Gate 是用 recall@k、precision@k、MRR、NDCG、P95/P99 latency、empty result rate 和 index size 判断近似检索是否满足上线阈值。

Metadata Permission Filter

一句话定义: Metadata Permission Filter 是在向量检索阶段强制执行 tenant、ACL、数据等级、文档类型、时间范围和安全等级过滤, 避免只在 UI 层隐藏无权结果。

Shadow Index Switch Readiness

一句话定义: Shadow Index Switch Readiness 是新索引先作为 shadow index 构建、验证、canary, 再切 active index, 并保留旧索引用于 rollback。

Retrieval Trace Completeness

一句话定义: Retrieval Trace Completeness 是每次检索记录 query hash、embedding model、index version、filters、candidate ids、scores、rerank ids、permission decision、latency 和 source pointers。

Feature Embedding Lineage Graph

一句话定义: Feature Embedding Lineage Graph 是把 raw event、feature transform、feature version、document、chunk、embedding、index、retrieval result 和 RAG answer 连成可追溯图。

Multi Tenant Isolation

一句话定义: Multi Tenant Isolation 是特征、embedding、索引、缓存和检索结果按租户、权限和数据等级做逻辑或物理隔离, 并保留审计证据。

Quality Monitoring Metrics

一句话定义：Quality Monitoring Metrics 是持续监控 feature skew、point-in-time leak、embedding stale rate、recall@k、latency、empty result、permission leak 和 RAG citation support。

Cost Capacity Governance

一句话定义：Cost Capacity Governance 是把 embedding 生成成本、向量内存、索引构建成本、查询成本、存储、增长容量和 owner / budget 统一治理。

RAG Agent Integration

一句话定义：RAG Agent Integration 是让 retrieval trace、rerank、citation source pointer、Agent memory scope、permission context 和最终回答证据链打通。

Feature Embedding Index Gate

一句话定义：Feature Embedding Index Gate 是判断特征、embedding、chunk、索引、过滤、trace、血缘、租户、质量、成本、RAG / Agent 集成和回滚证据是否完整且无 P0 风险的综合门禁。

典型硬阻断：特征缺版本或 owner；offline / online transform 不一致；训练样本使用未来特征；embedding model、dimension、normalize 或 chunking 缺版本；chunk 没有 offset、checksum 或 ACL；索引维度和 embedding 不一致；ANN recall 不达标或 P99 超阈值；metadata filter 没有强制 tenant / ACL；shadow index 没有 canary 或 rollback；检索 trace 缺 index version 或 candidate ids；lineage graph 断边；跨租户命中未阻断；质量监控缺 permission leak；构建成本不可归因；RAG citation 无 source pointer；没有最终 Feature Embedding Index Gate。

AI Infra Observability Audit

一句话定义：AI Infra Observability Audit 是把 metrics、logs、traces、events、SLO、error budget、label cardinality、training / inference / data / RAG / Agent / cost / privacy / alert 信号放进同一套可排障、可告警、可复盘的审计表。

核心直觉：AI Infra 可观测性不是收集更多曲线，而是让训练、推理、数据、模型、RAG、Agent、GPU、存储、网络、调度和成本问题能够被快速发现、定位、解释和复盘。

常见审计维度：Signal Inventory Coverage 看 metrics、logs、traces、events、costs、alerts、dashboards 和 versions 是否都有入口；Metric Contract Completeness 看 metric name、unit、labels、aggregation、owner 和 threshold；SLO Error Budget Readiness 与 Latency Quantile Guard 检查 availability、error budget、burn rate、p95 / p99、TTFT 和 TPOT；Trace Span Coverage、Log Event Structure、Correlation ID Coverage 和 Cardinality Budget Control 检查 gateway / router / queue / prefill / decode span、结构化日志、事件时间线、统一 ID 和 label 基数预算；Training Observability、Inference Observability、Data Quality Observability、RAG Agent Observability、Cost Observability、Privacy Redaction Retention、Alert Actionability 和 Observability Platform Gate 则检查训练、推理、数据、RAG / Agent、成本、隐私、告警、dashboard 和最终门禁。

面试表达：这类题先区分 metrics、logs、traces、events 的职责；然后讲 AI Infra 专属信号：GPU、tokens/s、TTFT、TPOT、KV cache、data wait、eval、RAG trace、tool trace、cost；再讲统一 ID、标签基数、SLO、告警和隐私；最后用 Observability Platform Gate 收束，说明平台不是有 dashboard，而是能支撑事故发生、定位、复盘和治理。

Signal Inventory Coverage

一句话定义：Signal Inventory Coverage 衡量关键工作负载是否都有 metrics、logs、traces、events、costs、alerts、dashboards 和 versions。

常见问题：只有 GPU metrics，没有模型版本、数据版本、trace 和事件，仍然无法定位训练或推理质量退化。

Metric Contract Completeness

一句话定义: Metric Contract Completeness 是 metric name、unit、labels、aggregation、owner、threshold、SLO 绑定和保留策略的完整性。

工程价值: 没有单位、聚合方式和 owner 的指标, 很难用于告警、对比和责任归属。

SLO Error Budget Readiness

一句话定义: SLO Error Budget Readiness 是可用性 SLI、延迟 SLI、SLO target、窗口、error budget 和 burn rate 是否可计算。

面试表达: AI 服务不能只看当前错误率, 还要知道错误预算是否被快速消耗, 以及是否需要暂停发布或扩容。

Latency Quantile Guard

一句话定义: Latency Quantile Guard 是用 p95 / p99、TTFT、TPOT、queue、prefill、decode 和 streaming 分位延迟判断用户体验是否过线。

常见问题: 平均延迟稳定, 但 p99 或 TTFT 退化, 线上用户仍然会明显感知变慢。

Trace Span Coverage

一句话定义: Trace Span Coverage 是关键链路 span 覆盖 gateway、auth、router、cache、queue、prefill、decode、safety、response, 以及训练 submit、schedule、data load、train step、checkpoint 和 eval。

工程价值: 没有阶段 span, 就无法区分慢在排队、prefill、decode、工具调用还是下游存储。

Log Event Structure

一句话定义: Log Event Structure 是日志和事件都具备 timestamp、level、service、trace id、event type、entity、version、actor、status 和 redaction 状态。

常见问题: 非结构化日志只能人工搜索, 无法和 trace、SLO、dashboard 自动关联。

Correlation ID Coverage

一句话定义: Correlation ID Coverage 衡量 metrics、logs、traces、events、costs、model version、dataset version 和 run id 是否能串联。

工程价值: 统一 ID 是跨训练、推理、RAG、Agent、成本和发布事件做根因定位的前提。

Cardinality Budget Control

一句话定义: Cardinality Budget Control 是控制 metrics label 的取值规模, 避免 request id、user id、prompt hash、document id 等高基数字段进入 time series label。

面试表达: 高基数字段应该进入 logs 或 traces, 不应该成为 metrics label。

Training Observability

一句话定义: Training Observability 是同时观测 train loss、eval loss、learning rate、gradient norm、step time、tokens/s、GPU、NCCL、data wait、checkpoint、retry 和 node failure。

常见问题: 只看 loss 会漏掉 data loader、NCCL、checkpoint 和 GPU 空转瓶颈。

Inference Observability

一句话定义：Inference Observability 是观测 QPS、TTFT、TPOT、p95 / p99、error、timeout、input / output tokens/s、queue、active sequences、KV cache、GPU、cache hit、fallback、degradation 和 admission reject。

工程价值：推理体验由 queue、prefill、decode 和 streaming 共同决定，不能只看总延迟或 QPS。

Data Quality Observability

一句话定义：Data Quality Observability 是观测 ingestion、schema、PII、duplicate、corrupt shard、quality gate、data loader wait 和 dataset version 对训练 / eval 的影响。

常见问题：数据质量问题常表现为 loss 异常或模型效果退化，必须能和训练指标关联。

RAG Agent Observability

一句话定义：RAG Agent Observability 是观测 retrieval trace、prompt assembly trace、tool call trace、execution trace、step count、tool success、loop rate、groundedness、citation correctness 和 cost per run。

工程价值：RAG / Agent 是多步系统，不看 trace 基本无法排查错误来源。

Cost Observability

一句话定义：Cost Observability 是把 GPU、storage、network、LLM token、embedding、rerank、tool API、trace storage 和 idle GPU 成本按 tenant、model、endpoint、run 和 component 归因。

面试表达：总账单无法治理成本，必须知道哪条链路、哪个模型、哪个租户和哪个 run 消耗异常。

Privacy Redaction Retention

一句话定义：Privacy Redaction Retention 是 prompt、用户输入、检索片段、tool 参数、模型输出、日志和 trace 的字段级脱敏、TTL、访问控制、加密和审计。

常见问题：完整保存所有 prompt 和输出会提升排障能力，也会扩大隐私和合规风险。

Alert Actionability

一句话定义：Alert Actionability 衡量告警是否有 owner、runbook、severity、影响面、去重和可能根因，而不是只报告曲线波动。

工程价值：告警噪音会让值班人员忽略真正事故，AI Infra 告警要围绕用户影响和可执行动作设计。

Observability Platform Gate

一句话定义：Observability Platform Gate 是判断 signals、SLO、trace、日志、事件、成本、隐私、告警、dashboard、incident process 和 postmortem 是否完整且无 P0 风险的综合门禁。

典型硬阻断：只有 GPU metrics；metric 无 owner 或单位；没有 SLO / error budget；只看平均延迟；trace 缺 queue / prefill / decode span；日志未结构化；事件没有 actor / version；ID 无法关联；request id 进入 metrics label；训练缺 data wait / checkpoint 指标；推理缺 TTFT / TPOT；数据质量不可观测；RAG / Agent trace 缺失；成本只看总账单；日志和 trace 未脱敏；告警无 runbook；没有最终 Observability Platform Gate。

Training Fault Diagnosis Audit

一句话定义：Training Fault Diagnosis Audit 是把 loss 异常、NaN / Inf、训练 hang、OOM、通信慢、I/O 慢、checkpoint、resume、数据异常、配置 diff、最小复现和事故复盘放进同一张可审计排障表的方法。

核心直觉：训练故障定位不能靠最后一行报错和经验猜测。成熟平台要能从 TrainingJob 生命周期事件、所有 rank 日志、step time 分解、loss / gradient 曲线、显存、NCCL、data loader、checkpoint、配置和数据版本 diff 串出证据链，最后给出根因、修复动作和复盘项。

常见审计维度：Training Fault Evidence Coverage 看事件、rank 日志、loss、梯度、显存、通信、数据 I/O、checkpoint、配置 diff、最小复现和诊断是否齐全；Lifecycle Event Timeline 与 Rank Log Completeness 检查最早异常阶段和所有 rank 状态；Loss Anomaly Diagnosis、NaN Inf Guard 和 Update Health Check 检查数值与更新；Hang Straggler Detection、OOM Phase Memory Evidence、Communication Bottleneck Diagnosis 和 IO Data Bottleneck Diagnosis 检查资源、通信与数据瓶颈；Checkpoint Integrity、Training Resume Continuity、Data Fault Isolation、Config Code Diff Readiness、Minimal Reproduction Readiness 和 Training Fault Diagnosis Gate 检查恢复、复现、修复和复盘闭环。

面试表达：这类题先按现象分类：loss 异常、NaN、hang、OOM、通信慢、I/O 慢、resume 异常；再按证据层排查：事件、日志、指标、profile、数据、配置、checkpoint 和最小复现；最后用 Training Fault Diagnosis Gate 收束，说明没有证据链的根因判断不能进入事故结论。

Training Fault Evidence Coverage

一句话定义：Training Fault Evidence Coverage 衡量训练事故是否同时保存事件、rank 日志、loss、梯度、显存、通信、数据 I/O、checkpoint、配置 diff、复现和诊断结论。

常见问题：只保存 rank 0 stdout 或最后一个 traceback，会漏掉另一个 rank 更早的 OOM、dataloader 阻塞、NCCL 错误或数据异常。

Lifecycle Event Timeline

一句话定义：Lifecycle Event Timeline 是按时间记录 job submitted、scheduled、started、last progress、checkpoint、node lost、retry、failed 等事件。

工程价值：时间线能判断故障发生在调度、镜像、数据挂载、训练 step、checkpoint、resume 还是平台重试阶段。

Rank Log Completeness

一句话定义：Rank Log Completeness 是所有 rank 都有 node、local rank、last step、last event、status、heartbeat 和错误上下文。

常见问题：分布式训练最后 timeout 的 rank 未必是根因，缺 rank 日志会把首个失败 rank 隐藏掉。

Loss Anomaly Diagnosis

一句话定义：Loss Anomaly Diagnosis 是把 loss spike、不下降、突然 NaN 或 train / eval 背离绑定到 step、batch、学习率、precision、梯度、tokenizer、mask 和 resume 状态。

面试表达：loss 异常要先看是否真的在更新，再看异常 batch、LR、grad norm、label / mask、tokenizer、precision 和 checkpoint state，而不是直接怀疑模型结构。

NaN Inf Guard

一句话定义：NaN Inf Guard 是检查 loss、logits、gradient、loss scale 和自定义算子是否出现非有限值，并在触发时停止、回滚或保存最小复现。

典型硬阻断：出现 NaN 后只跳过 batch；没有保存触发 batch；没有检查 logits / gradient finite；没有记录 precision 和 loss scale。

Update Health Check

一句话定义：Update Health Check 是用非零梯度比例、optimizer step、learning rate、参数变化和有效 token 数证明训练确实在更新。

常见问题：loss 不下降可能是参数被冻结、LoRA target 错、有效 label 被 mask 掉、optimizer 没 step 或 LR 为 0。

Hang Straggler Detection

一句话定义：Hang Straggler Detection 是用 step 停止推进、rank heartbeat、collective wait、dataloader 状态、GPU 利用率和首个异常 rank 定位训练卡住原因。

工程价值：hang 往往是一个 rank 先崩溃、阻塞或变慢，其他 rank 在 collective、barrier 或 dataloader 上等待。

OOM Phase Memory Evidence

一句话定义：OOM Phase Memory Evidence 是记录 OOM 发生在 forward、backward、optimizer、eval 还是 checkpoint，并保存显存峰值、上限、显存拆分和修复建议。

面试表达：forward OOM、backward OOM、optimizer step OOM 和 eval OOM 的修复方向不同，不能只回答“减 batch size”。

Communication Bottleneck Diagnosis

一句话定义：Communication Bottleneck Diagnosis 是用通信占比、rank step time 分布、NCCL 日志、网络吞吐、错误包和首个错误 rank 判断通信是否拖慢训练。

常见问题：GPU 利用率低不一定是数据慢，也可能是 all-reduce、reduce-scatter、all-gather 或跨节点网络在等待。

IO Data Bottleneck Diagnosis

一句话定义：IO Data Bottleneck Diagnosis 是用 data wait ratio、cache hit rate、worker queue、存储吞吐、CPU preprocessing 和坏样本隔离判断数据链路是否拖慢训练。

工程价值：I/O 慢常表现为 GPU 利用率周期性下降，需要和通信等待、checkpoint 阻塞、CPU preprocessing 区分。

Checkpoint Integrity

一句话定义：Checkpoint Integrity 是 checkpoint manifest、checksum、model、optimizer、scheduler、RNG、dataloader state 和 committed 状态是否完整。

常见问题：目录里有文件不等于 checkpoint 可恢复。未 committed、缺 optimizer / scheduler / RNG 或 checksum 不通过，都可能导致 resume 后 loss 跳变。

Training Resume Continuity

一句话定义：Training Resume Continuity 是检查 resume 后 global step、learning rate、loss、optimizer、scheduler、RNG、dataloader cursor 和配置是否连续。

面试表达：只加载模型权重不是完整 resume。大模型训练恢复要保证训练动力学连续，否则看似恢复成功，实际已经改变实验。

Data Fault Isolation

一句话定义：Data Fault Isolation 是把空样本、超长样本、非法 JSON、标签错位、tokenizer 异常、分布漂移和权限问题定位到具体 batch、shard 和数据版本。

工程价值：训练故障中数据问题很常见，平台要能保存触发 batch、生成 dataset diff report，并支持隔离或回滚数据版本。

Config Code Diff Readiness

一句话定义：Config Code Diff Readiness 是把 learning rate、batch、sequence length、precision、并行策略、tokenizer、dataset、checkpoint、代码 commit 和环境版本变化放进同一张 diff 表。

常见问题：没有 final config snapshot、code diff、dataset diff 和 environment diff 时，排查只能靠人工记忆，事故复盘不可复现。

Minimal Reproduction Readiness

一句话定义：Minimal Reproduction Readiness 是固定 seed、异常 batch、配置、数据版本、单机试验和复现命令，用更小环境复现训练故障。

面试表达：单机小数据能复现，优先怀疑代码、数据、配置或数值；只有多机复现，才更可能是通信、拓扑、调度或环境问题。

Training Fault Diagnosis Gate

一句话定义：Training Fault Diagnosis Gate 是判断训练故障定位是否有完整证据链、owner、runbook、最小复现、修复动作、postmortem 和 P0 风险阻断的综合门禁。

典型硬阻断：只有最后一行报错；只采 rank 0 日志；loss spike 没有 batch；NaN 后继续训练；hang 没有 rank heartbeat；OOM 不记录阶段和峰值；通信慢没有 NCCL 日志；I/O 慢没有 data wait / cache hit；checkpoint 缺 checksum；resume 不校验 LR / step / dataloader；数据异常没有 batch 和 shard；配置 diff 缺失；没有最小复现；没有最终 Training Fault Diagnosis Gate。

SLO Oncall Audit

一句话定义：SLO Oncall Audit 是把 SLI、SLO、SLA、错误预算、burn rate、告警、on-call、事故分级、runbook、变更、止损、复盘和成本取舍放进同一张生产可靠性治理表的方法。

核心直觉：AI Infra 可靠性不是追求永不失败，而是定义哪些失败可接受、如何计算预算、什么时候告警、谁来响应、能做哪些止损动作、事后如何把事故沉淀成行动项和上线门禁。没有错误预算和值班闭环，SLO 只是 dashboard 上的百分比。

常见审计维度：SLI Contract Completeness、SLO Target Measurability 和 SLA Boundary Clarity 检查指标、目标和承诺边界；Inference SLO Coverage、Training SLO Coverage、Data Eval SLO Coverage 检查推理、训练、数据和评估平台的用户体验指标；Error Budget Accounting 与 Burn Rate Alerting 检查预算和快慢窗口告警；Alert Actionability、Oncall Ownership Escalation、Incident Severity Routing、Runbook Executability 检查告警和值班可执行性；Change Event Linkage、Mitigation Rollback Authority、Postmortem Action Closure 和 SLO Cost Tradeoff Gate 检查变更、止损、复盘和成本收益边界。

常见公式：错误预算比例是

$$B=1-S_{\mathrm{slo}}$$

预算消耗比例可以写成

$$U_{\mathrm{budget}}=\frac{N_{\mathrm{bad}}}{\max(1,N_{\mathrm{bad,allowed}})}$$

窗口 burn rate 可以写成

$$\beta_w=\frac{R_{\mathrm{bad},w}}{B}$$

综合门禁可以写成

$G_{\{\mathrm{slo_oncall}\}} = \prod_{j=1}^{16} G_j$

面试表达：SLO / SLA / 值班体系题要先区分 SLI 是实际观测指标、SLO 是内部目标、SLA 是对外承诺；再讲推理、训练、数据和评估各自的 SLO；随后用错误预算和 burn rate 驱动发布、告警和冻结策略；最后补 on-call、runbook、权限、变更时间线、事故分级、postmortem action item 和成本取舍。

SLI Contract Completeness

一句话定义：SLI Contract Completeness 是检查关键服务的指标是否有名称、定义、数据源、窗口、owner 和坏事件口径。

SLO Target Measurability

一句话定义：SLO Target Measurability 是检查 SLO 是否能用实际查询计算、能回放、能解释用户体验，并能触发明确工程动作。

SLA Boundary Clarity

一句话定义：SLA Boundary Clarity 是检查对外承诺、客户范围、排除项、赔偿或业务责任、owner 是否清楚。

Inference SLO Coverage

一句话定义：Inference SLO Coverage 是检查推理服务是否覆盖 availability、TTFT、TPOT、p99、error、timeout 和质量 / 安全 guardrail。

Training SLO Coverage

一句话定义：Training SLO Coverage 是检查训练平台是否区分用户错误和平台错误，并覆盖提交、调度、平台失败率、checkpoint 保存和恢复。

Data Eval SLO Coverage

一句话定义：Data Eval SLO Coverage 是检查数据构建、manifest 查询、质量门禁、eval report、RAG / Agent run success 是否有可观测目标。

Error Budget Accounting

一句话定义：Error Budget Accounting 是按服务、窗口和坏事件规则计算允许失败空间，并把剩余预算绑定到发布和风险策略。

Burn Rate Alerting

一句话定义：Burn Rate Alerting 是用错误预算消耗速度做告警，通常同时看快窗口和慢窗口，兼顾突发事故和慢性退化。

Alert Actionability

一句话定义：Alert Actionability 是衡量告警是否有 owner、runbook、下一步动作和低噪音，而不是只看触发次数。

Oncall Ownership Escalation

一句话定义：Oncall Ownership Escalation 是检查值班排班、主备、ACK SLO、升级链路和状态沟通是否完整。

Incident Severity Routing

一句话定义：Incident Severity Routing 是按用户影响、持续时间、服务重要性和风险权重决定 SEV 等级、通知范围和复盘要求。

Runbook Executability

一句话定义：Runbook Executability 是检查 runbook 是否真的能指导值班完成影响判断、根因路径、止损、回滚和升级。

Change Event Linkage

一句话定义：Change Event Linkage 是事故排查时能关联模型、prompt、runtime、route、data、cluster、cache 和 owner 变更时间线。

Mitigation Rollback Authority

一句话定义：Mitigation Rollback Authority 是检查值班是否有可审计的限流、降级、扩容、摘除、回滚和暂停队列权限。

Postmortem Action Closure

一句话定义：Postmortem Action Closure 是检查复盘行动项是否有 owner、截止时间、关闭证据和回归验证，而不是只写事故文档。

SLO Cost Tradeoff Gate

一句话定义：SLO Cost Tradeoff Gate 是判断更高可靠性目标是否值得额外基础设施成本和值班成本的综合门禁。

典型硬阻断：SLI 只有指标名没有坏事件口径；SLO 不可测量；SLA 承诺和内部目标混淆；推理只看可用性不看 TTFT / TPOT；训练不区分用户错误和平台错误；数据和评估没有质量 / 产出 SLO；错误预算不驱动发布；burn rate 告警缺失；告警无 runbook；值班无升级链路；事故分级只看错误率；runbook 从未演练；事故无法关联最近变更；值班无回滚权限；复盘行动项长期不关闭；SLO 目标不考虑成本。

AI Infra Cost Governance Audit

一句话定义：AI Infra Cost Governance Audit 是把 GPU、训练、推理、缓存、存储、网络、artifact、日志 trace、预算、配额、成本归因、dashboard 和优化建议放进同一张成本治理审计表的方法。

核心直觉：成本治理不是月底看账单，也不是简单砍资源，而是把资源消耗变成可观测、可归因、可预算、可优化的工程指标。好的成本治理要能回答谁花了钱、花在什么资源上、产生了什么有效产出、哪些成本是浪费、哪些优化不会伤害质量和 SLO。

常见审计维度：Usage Metering Coverage 与 Cost Attribution Labels 检查 usage 和标签；GPU Cost Efficiency、Training Waste Control、Inference Unit Cost 和 Cache Savings Accounting 检查 GPU、训练、推理和缓存单位成本；Storage Lifecycle Governance、Network Egress Governance 和 Artifact Dependency Safety 检查存储、网络和删除安全；Budget Quota Enforcement、Cost Anomaly Alerting、Tenant Model Chargeback、Dashboard Drilldown Readiness、Optimization Recommendation Trace、SLO Quality Cost Tradeoff 和 Cost Governance Gate 检查预算、告警、showback / chargeback、dashboard、优化建议和最终门禁。

常见公式：GPU 任务成本可以写成

$$K_{\mathrm{gpu}} = G \cdot T \cdot P_{\mathrm{gpu}} \cdot \eta_{\mathrm{bill}}$$

每 1k token 推理成本可以写成

$$K_{\mathrm{1k}} = \frac{1000 K_{\mathrm{infer}}}{N_{\mathrm{in}} + N_{\mathrm{out}}}$$

预算使用率可以写成

$$U_{\mathrm{budget}} = \frac{K_{\mathrm{actual}}}{K_{\mathrm{budget}}}$$

面试表达：成本治理题要先讲 usage metering 和 attribution，再分 GPU、训练浪费、推理单位成本、缓存节省、存储生命周期、网络出站和 artifact 删除安全；随后讲预算、配额、成本告警和 dashboard 下钻；最后用 SLO / quality / risk trade-off 收束，说明降本不能牺牲用户体验、质量、安全和回滚能力。

Usage Metering Coverage

一句话定义：Usage Metering Coverage 是检查 GPU、CPU、memory、storage、network、request、token、artifact、logs 和 traces 是否都有准实时用量记录。

Cost Attribution Labels

一句话定义：Cost Attribution Labels 是检查 tenant、project、team、owner、model、job type、environment 和 cost center 等标签是否在资源创建时强制存在。

GPU Cost Efficiency

一句话定义：GPU Cost Efficiency 是用 GPU-hours、tokens per GPU hour、MFU / utilization、idle GPU hours 和有效产出判断 GPU 成本是否健康。

Training Waste Control

一句话定义：Training Waste Control 是定位失败任务、无效实验、抢占丢失、checkpoint / resume 缺失和 failure owner 缺失造成的训练浪费。

Inference Unit Cost

一句话定义：Inference Unit Cost 是按 request、1k tokens、output token、tenant、model 和 endpoint 计算推理单位成本。

Cache Savings Accounting

一句话定义：Cache Savings Accounting 是量化 result cache、prompt cache、prefix / KV cache 和 semantic cache 的节省成本、维护成本、权限风险和质量影响。

Storage Lifecycle Governance

一句话定义：Storage Lifecycle Governance 是对 checkpoint、dataset、model、eval output、logs、traces 和 vector index 设置保留、归档、删除和冷热分层策略。

Network Egress Governance

一句话定义：Network Egress Governance 是计量和优化跨地域数据、模型权重、checkpoint、日志 trace 和远端索引访问的网络成本。

Artifact Dependency Safety

一句话定义：Artifact Dependency Safety 是删除或降冷 artifact 前检查部署、回滚、训练、评估、审计和血缘依赖。

Budget Quota Enforcement

一句话定义：Budget Quota Enforcement 是用预算控制花费、用配额控制资源，并为例外审批和硬限制建立审计记录。

Cost Anomaly Alerting

一句话定义：Cost Anomaly Alerting 是对日成本、单位成本、idle、failed job、storage growth、egress 和 agent run 异常告警到 owner。

Tenant Model Chargeback

一句话定义：Tenant Model Chargeback 是把成本按租户、模型、模型版本、endpoint、job、experiment 和 cost center 做 showback / chargeback。

Dashboard Drilldown Readiness

一句话定义：Dashboard Drilldown Readiness 是 dashboard 能从总成本下钻到 tenant、model、job、artifact、endpoint、request slice 和 recent change。

Optimization Recommendation Trace

一句话定义：Optimization Recommendation Trace 是每条降本建议都带估算节省、证据、owner、质量 / SLO 风险和回滚条件。

SLO Quality Cost Tradeoff

一句话定义：SLO Quality Cost Tradeoff 是判断降本动作是否在质量、SLO、安全、风险和用户体验边界内。

Cost Governance Gate

一句话定义：Cost Governance Gate 是判断成本治理是否具备用量、归因、预算、配额、告警、dashboard、优化 backlog、审批、审计和 P0 风险阻断的综合门禁。

典型硬阻断：usage 缺 token 或 network；成本标签缺 owner / cost center；只看 GPU utilization 不看有效产出；失败训练任务成本无人负责；推理没有 cost per 1k tokens；缓存节省不扣维护成本或忽略权限；checkpoint 永久保留；跨地域 egress 无理由；删除 artifact 不查回滚依赖；预算和配额只提醒不阻断；成本异常无 owner；成本不能按模型版本下钻；dashboard 只有总账单；优化建议没有风险评估；为降本牺牲 SLO 或质量；没有最终 Cost Governance Gate。

Resource Utilization Optimization Audit

一句话定义：Resource Utilization Optimization Audit 是把 GPU allocation、GPU utilization、有效产出、碎片、混部、抢占、低优先级队列、弹性训练、推理容量、warm pool、配额公平、调度观测和可靠性 headroom 放进同一张资源治理审计表的方法。

核心直觉：资源利用率优化不是把 GPU 打满，而是在 SLO、隔离、公平性、checkpoint、恢复成本和业务价值约束下减少空闲、减少碎片、减少失败并提高有效产出。GPU 被分配不等于 GPU 有效工作，GPU 忙也不等于产出可用。

常见审计维度：Resource Metric Contract 和 Effective Utilization Accounting 先确认指标口径与有效产出；Fragmentation Control、Topology Binpacking Readiness、Colocation SLO Guard、Preemption Checkpoint Safety、Low Priority Reclaim Policy 和 Elastic Training Safety 检查调度动作是否安全；Inference Capacity Utilization、Warm Pool Headroom Policy、Quota Fairness Borrowing、Scheduler Observability、Dashboard Drilldown Readiness、Reliability Headroom Guard、Cost SLO Tradeoff Control 和 Resource Utilization Gate 检查推理容量、预留、公平性、观测、可靠性和最终门禁。

常见公式：GPU 分配率可以写成

$$U_{\{\mathrm{alloc}\}} = \frac{G_{\{\mathrm{allocated}\}}}{G_{\{\mathrm{total}\}}}$$

有效利用率可以写成

$$U_{\{\mathrm{eff}\}} = \frac{\sum_i h_i v_i s_i}{\sum_i h_i}$$

碎片比例可以写成

$$R_{\{\mathrm{frag}\}}(g) = 1 - \frac{\sum_{n=1}^N \lfloor f_n/g \rfloor}{\sum_{n=1}^N f_n}$$

面试表达：资源利用率优化题要先区分 allocation、utilization 和 effective utilization，再讲碎片、拓扑和 bin packing；然后讲混部要受 SLO guardrail 约束，抢占要绑定 checkpoint 和 requeue，低优先级队列只能吃可回收余量；最后讲弹性训练、推理容量、warm pool、quota borrowing、dashboard 下钻和 headroom，说明为什么不能追求 100% GPU 利用率。

Resource Metric Contract

一句话定义：Resource Metric Contract 是统一 GPU allocation、GPU utilization、显存、MFU、tokens、idle、pending、queue wait、preemption、failed GPU-hours 和 fragmentation 的指标口径。

Effective Utilization Accounting

一句话定义：Effective Utilization Accounting 是把失败任务、无效实验、SLO 失败请求和不可发布产物从资源利用率中扣除，只统计有效产出。

Fragmentation Control

一句话定义：Fragmentation Control 是按节点、GPU 型号、拓扑和任务规格计算碎片，并通过 bin packing、预留、队列重排和可抢占任务减少碎片。

Topology Binpacking Readiness

一句话定义：Topology Binpacking Readiness 是检查 bin packing 是否同时考虑 GPU、NIC、rack、storage、gang scheduling 和调度 score trace。

Colocation SLO Guard

一句话定义：Colocation SLO Guard 是训练 / 推理、在线 / 离线、高 / 低优先级混部时，用 p99、step time、驱逐、低优先级上限和隔离策略保护关键任务。

Preemption Checkpoint Safety

一句话定义：Preemption Checkpoint Safety 是抢占前检查 checkpoint 新鲜度、graceful shutdown、requeue、频率限制、owner 通知和丢失 GPU-minutes。

Low Priority Reclaim Policy

一句话定义：Low Priority Reclaim Policy 是低优先级队列只使用可回收余量，并在高优先级需求出现时可抢占、可归还、可补偿。

Elastic Training Safety

一句话定义：Elastic Training Safety 是控制 world size 变化带来的 global batch、learning rate、optimizer state、dataloader、checkpoint 和 rendezvous 风险。

Inference Capacity Utilization

一句话定义：Inference Capacity Utilization 是在 TTFT、TPOT、p99、KV pressure、continuous batching 和 autoscaling 约束下统计推理有效吞吐。

Warm Pool Headroom Policy

一句话定义：Warm Pool Headroom Policy 是用模型热度、冷启动损失、常驻副本成本、权重缓存、headroom 和业务优先级决定 warm pool 策略。

Quota Fairness Borrowing

一句话定义：Quota Fairness Borrowing 是允许租户借用空闲配额，但用 dominant share、公平性、可回收和补偿机制控制挤占风险。

Scheduler Observability

一句话定义：Scheduler Observability 是让 gang、queue、priority、preemption、quota、topology、elastic 和 event trace 都能被查询、告警和复盘。

Reliability Headroom Guard

一句话定义：Reliability Headroom Guard 是在提高利用率时保留故障恢复、突发流量、高优先级任务和线上 SLO 所需的安全余量。

Cost SLO Tradeoff Control

一句话定义：Cost SLO Tradeoff Control 是检查资源优化建议是否有节省估算、质量 / SLO / 风险评估、owner 和回滚条件。

Resource Utilization Gate

一句话定义：Resource Utilization Gate 是判断资源利用率优化是否具备 owner、资源策略、例外审批、审计证据和 P0 风险阻断的最终门禁。

典型硬阻断：只有 GPU allocation 没有 utilization；只看 GPU utilization 不看有效产出；失败任务和 SLO 失败请求仍被算入收益；碎片无法解释；bin packing 不看拓扑；训练推理混部没有 p99 guardrail；抢占无 checkpoint；低优先级任务不可回收；弹性训练不处理 global batch；推理只看 tokens/s 不看 TTFT / TPOT；warm pool 只有固定副本；quota borrowing 无归还机制；调度事件不可观测；dashboard 不能下钻；headroom 被挤没；优化建议没有质量、SLO 和回滚证据。

AI Infra Security Governance Audit

一句话定义：AI Infra Security Governance Audit 是把身份、权限、密钥、数据合规、PII、多租户、模型访问、模型输出、RAG、Agent、供应链、model artifact、运行时、日志 trace、审计和安全响应放进同一张安全治理审计表的方法。

核心直觉：AI Infra 安全不是给推理 API 加鉴权，而是让训练、数据、模型、RAG、Agent、artifact、runtime、日志和事故响应都受统一策略、证据和门禁约束。模型权重、数据集、prompt、trace、embedding、工具参数和 checkpoint 都可能是敏感资产。

常见审计维度: Identity Workload Binding、RBAC ABAC Policy Fit 和 Secret Lifecycle Management 检查身份、权限和密钥; Data Compliance Classification、PII Sensitive Data Control、Tenant Isolation Boundary、Model Access Control 和 Model Output Safety Gate 检查数据、租户、模型和输出; RAG Permission Enforcement、Agent Tool Safety Gate、Supply Chain Integrity、Model Artifact Integrity、Runtime Security Boundary、Log Trace Privacy Governance、Audit Incident Response Readiness 和 Security Governance Gate 检查 RAG、Agent、供应链、运行时、日志、审计和响应闭环。

常见公式: 身份绑定覆盖率可以写成

$$C_{\mathrm{id}} = \frac{N_{\mathrm{bound}}}{N_{\mathrm{workload}}}$$

密钥暴露率可以写成

$$R_{\mathrm{secret}} = \frac{N_{\mathrm{secret,exposed}}}{N_{\mathrm{secret,total}}}$$

RAG 权限阻断率可以写成

$$B_{\mathrm{rag}} = \frac{N_{\mathrm{unauthorized_chunk_blocked}}}{N_{\mathrm{unauthorized_chunk}}}$$

面试表达: 安全治理题要先画资产和身份边界, 再讲 RBAC + ABAC、secret manager、数据分级、PII、租户隔离、模型访问、RAG 权限、Agent 工具确认、供应链 provenance、artifact checksum / signature、runtime hardening、日志脱敏、审计留存和响应预案; 最后强调这是防御性平台门禁, 不是靠人工规范或提示词承诺。

Identity Workload Binding

一句话定义: Identity Workload Binding 是把人类用户、service account、训练任务、推理服务、Agent、CI/CD 和外部应用身份绑定到具体 workload 和租户。

Secret Lifecycle Management

一句话定义: Secret Lifecycle Management 是用 secret manager、短期凭证、最小权限、轮换、吊销、访问审计和日志脱敏治理密钥。

Data Compliance Classification

一句话定义: Data Compliance Classification 是结构化记录数据来源、授权、敏感等级、地域、用途、保留策略、删除请求和血缘。

PII Sensitive Data Control

一句话定义: PII Sensitive Data Control 是在数据、请求、日志、trace、评估样本和工具结果中检测、脱敏、阻断和审计 PII / 敏感信息。

Tenant Isolation Boundary

一句话定义: Tenant Isolation Boundary 是隔离租户身份、数据、模型、资源、网络、缓存、日志、trace 和成本归因。

Model Access Control

一句话定义: Model Access Control 是控制模型调用、权重下载、部署、导出、eval report 和敏感输出访问权限, 并记录审计。

Model Output Safety Gate

一句话定义: Model Output Safety Gate 是对模型输出做安全策略检查、高风险人工确认、工具动作保护、线上监控和违规样本回流。

RAG Permission Enforcement

一句话定义：RAG Permission Enforcement 是在检索和上下文组装前强制 metadata / ACL 权限过滤，并保证 citation 和 trace 不泄露无权限来源。

Agent Tool Safety Gate

一句话定义：Agent Tool Safety Gate 是让工具调用满足权限、参数校验、高风险确认、限流、超时、审计和补偿机制。

Supply Chain Integrity

一句话定义：Supply Chain Integrity 是为镜像、依赖、runtime、数据集、模型权重、tokenizer 和工具记录 provenance、checksum、签名、扫描和许可证证据。

Model Artifact Integrity

一句话定义：Model Artifact Integrity 是检查模型权重、tokenizer、config、adapter、量化包和 deployment package 的 checksum、签名、安全格式和权限。

Log Trace Privacy Governance

一句话定义：Log Trace Privacy Governance 是对日志和 trace 做脱敏、字段级权限、TTL、加密、采样、访问审计和删除覆盖。

Audit Incident Response Readiness

一句话定义：Audit Incident Response Readiness 是检查关键事件审计、防篡改留存、检测、隔离、吊销、影响定界、修复和复盘是否就绪。

Security Governance Gate

一句话定义：Security Governance Gate 是判断 AI Infra 是否具备 owner、安全策略、例外审批、审计证据、响应预案和 P0 风险阻断的最终安全门禁。

典型硬阻断：workload 使用共享管理员身份；RBAC 只有通配角色；密钥写进镜像、命令行、日志或 prompt；数据无授权和等级；PII 只在入库前扫一次；租户缓存或向量索引串租户；模型权重下载无权限；RAG 先检索后让模型保密；Agent 高风险工具无确认；镜像和模型 artifact 无 provenance、checksum 或签名；runtime privileged；日志长期保存敏感原文；审计日志可篡改；密钥泄露后没有吊销和影响范围定界。

Audit Change Governance Audit

一句话定义：Audit Change Governance Audit 是把审计事件、日志留存、Trace 隐私、事故复盘、行动项关闭、变更记录、灰度回滚、冻结策略和治理闭环放进同一张平台审计表的方法。

核心直觉：审计告诉我们发生了什么，日志留存决定证据能保存多久，事故复盘把事故变成系统改进，变更管理降低下一次事故概率。AI Infra 的变更不仅是代码发布，还包括模型版本、prompt、runtime、路由、数据集、权限策略、工具定义、RAG 知识库和 GPU 集群配置。

常见审计维度：Audit Event Schema Completeness、Critical Operation Audit Coverage 和 Audit Log Integrity Retention 检查审计事件是否结构化、关键操作是否覆盖、日志是否可靠写入和防篡改；Log Retention Policy Fit、Log Minimization Redaction 和 Trace Privacy Sampling 检查 TTL、脱敏、字段级权限和采样；Incident Timeline Completeness、Impact Detection Response Metrics、Root Cause Systemic Analysis 和 Postmortem Action Item Closure 检查事故时间线、MTTD / MTTA / MTTR、系统根因和行动项关闭；Change Record Completeness、High Risk Change Approval、Rollout Rollback Readiness、Change Freeze Error

Budget Policy、Audit Postmortem Change Linkage 和 Governance Loop Gate 检查变更记录、审批、灰度、回滚、冻结、错误预算和跨对象跳转。

常见公式：审计 schema 完整率可以写成

$$C_{\{\mathrm{schema}\}}=\frac{N_{\{\mathrm{event_complete}\}}}{N_{\{\mathrm{audit_event}\}}}$$

关键操作审计覆盖率可以写成

$$C_{\{\mathrm{audit}\}}=\frac{N_{\{\mathrm{critical_audited}\}}}{N_{\{\mathrm{critical_operation}\}}}$$

变更失败率可以写成

$$R_{\{\mathrm{change_fail}\}}=\frac{N_{\{\mathrm{change_incident}\}}}{N_{\{\mathrm{change}\}}}$$

综合门禁可以写成

$$G_{\{\mathrm{audit_change}\}}=\prod_{j=1}^{16}G_j$$

面试表达：审计变更治理题要先把关键资产和关键操作列出来，再讲审计 schema、不可篡改留存、日志 TTL、Trace 脱敏、事故 timeline、影响量化、系统根因、行动项关闭、变更记录、审批、canary、rollback、freeze 和 error budget；最后用 audit log、change record、incident、postmortem、action item 和 regression 之间能否互相跳转来证明闭环。

Audit Event Schema Completeness

一句话定义：Audit Event Schema Completeness 是检查审计事件是否包含 timestamp、actor、tenant、action、resource、decision、status、request id、trace id 和 diff / hash 等必要字段。

Critical Operation Audit Coverage

一句话定义：Critical Operation Audit Coverage 是检查数据、模型、权限、secret、工具、知识库、发布、回滚、删除和导出等关键操作是否被审计。

Audit Log Integrity Retention

一句话定义：Audit Log Integrity Retention 是检查审计日志是否可靠写入、防篡改、加密、可查询、可导出和长期留存。

Log Retention Policy Fit

一句话定义：Log Retention Policy Fit 是按日志类型、敏感等级、合规要求、删除请求和成本预算制定 TTL 与访问策略。

Log Minimization Redaction

一句话定义：Log Minimization Redaction 是在写入前脱敏敏感字段，并避免长期保存原始 prompt、工具参数、检索片段和用户数据。

Trace Privacy Sampling

一句话定义：Trace Privacy Sampling 是按错误、慢请求、普通成功和高敏请求分层采样、脱敏和留存 RAG / Agent trace。

Incident Timeline Completeness

一句话定义：Incident Timeline Completeness 是让事故记录覆盖发生、检测、告警、响应、缓解、恢复和根因确认等关键时间点。

Impact Detection Response Metrics

一句话定义：Impact Detection Response Metrics 是用 MTDD、MTTA、MTTR、影响范围、用户切片和业务影响衡量事故响应能力。

Root Cause Systemic Analysis

一句话定义：Root Cause Systemic Analysis 是把根因追到系统防线、流程门禁、监控、测试或设计缺陷，而不是停在个人操作。

Postmortem Action Item Closure

一句话定义：Postmortem Action Item Closure 是检查复盘行动项是否有 owner、deadline、关闭证据、验证结果和逾期跟踪。

Change Record Completeness

一句话定义：Change Record Completeness 是检查变更记录是否包含 owner、对象、原因、diff、风险、审批、灰度、回滚和观测指标。

High Risk Change Approval

一句话定义：High Risk Change Approval 是让高风险模型、数据、权限、路由、runtime、工具和集群配置变更经过审批、通知和回滚验证。

Rollout Rollback Readiness

一句话定义：Rollout Rollback Readiness 是检查变更是否具备 canary、自动观测、暂停、回滚和回滚演练能力。

Change Freeze Error Budget Policy

一句话定义：Change Freeze Error Budget Policy 是用变更冻结和错误预算约束高风险变更，并保留紧急修复例外路径。

Audit Postmortem Change Linkage

一句话定义：Audit Postmortem Change Linkage 是让 audit log、change record、incident、postmortem、action item 和 regression test 能互相跳转。

Governance Loop Gate

一句话定义：Governance Loop Gate 是判断审计、留存、复盘和变更管理是否具备 owner、治理策略、例外审批、dashboard、审计导出和 P0 风险阻断的最终门禁。

典型硬阻断：审计事件缺 actor 或 trace id；关键操作没有 deny 事件；审计日志可被普通管理员删除；日志 TTL 没有按敏感等级区分；prompt、检索片段或工具参数原文长期落盘；Trace 没有采样和字段级权限；事故没有完整时间线；MTTD / MTTA / MTTR 不可计算；复盘停在“人为失误”；行动项没有验证证据；变更记录缺 diff、owner 或 rollback；高风险变更无审批；canary 无自动观测；错误预算耗尽仍继续放量；postmortem 行动项没有进入回归测试。

Training Platform Security Audit

一句话定义：Training Platform Security Audit 是把训练平台身份、RBAC / ABAC、数据权限、产物等级传播、资源权限、镜像供应链、代码运行时、密钥、网络、日志、模型产物、高风险审批、审计日志和事件响应放进同一张安全门禁表的方法。

核心直觉：训练平台安全不是给入口加登录，而是控制谁能用什么身份运行任务、读什么数据、用什么资源、运行什么镜像和代码、产生什么模型、查看什么日志，并且能事后追踪全过程。训练脚本本质是任意代码执行，安全控制必须进入任务提交、运行时和产物注册全链路。

常见审计维度：Identity Runtime Binding 看用户身份、项目身份、TrainingJob 运行身份和 service account 是否绑定且没有共享高权限身份；RBAC ABAC Policy Fit 看角色和属性策略是否覆盖项目、数据等级、资源池和用途，并满足最小权限；Dataset Permission Purpose 与 Artifact Classification Propagation 看数据使用目的和产物等级传播是否正确；Resource Quota Priority Governance 看高价值 GPU、队列和优先级是否受控；Image Supply Chain Trust 与 Runtime Code Isolation 看可信 registry、digest、签名、扫描、provenance、非 privileged、hostPath、root 和 Pod 安全边界；Secret Scope Rotation、Network Egress Control 和 Log Redaction Access 看短期凭证、最小权限、出网白名单和日志脱敏；Model Artifact Permission、High Risk Approval、Audit Log Completeness、Audit Integrity Retention、Incident Response Readiness 和 Training Platform Security Gate 则检查模型权重、审批、审计、防篡改留存和事故响应是否闭环。

常见公式：训练平台安全样本可以写成

$s_i=(u_i,r_i,d_i,a_i,q_i,m_i,c_i,e_i,n_i,l_i,w_i,h_i,o_i,t_i,z_i)$

越权阻断率可以写成

$R_{\mathrm{block}}=\frac{N_{\mathrm{denied_unauth}}}{N_{\mathrm{unauth}}}$

过度授权率可以写成

$R_{\mathrm{over}}=\frac{N_{\mathrm{granted}}-N_{\mathrm{used}}}{N_{\mathrm{granted}}}$

数据等级传播可以写成

$V_{\mathrm{artifact}}=\mathbf{1}\left[\mathrm{class}(A)\geq \max_k \mathrm{class}(D_k)\right]$

审计覆盖率可以写成

$R_{\mathrm{audit}}=\frac{N_{\mathrm{audited}}}{N_{\mathrm{required}}}$

训练平台安全门禁可以写成

$G_{\mathrm{security}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \ \wedge R_{\mathrm{block}}=1 \ \wedge R_{\mathrm{over}}\leq \rho_{\mathrm{over}}\right]$

面试表达：训练平台安全题不要只讲登录和 RBAC。要从身份、任务运行身份、数据权限、用途授权、资源权限、镜像供应链、运行时隔离、密钥、网络、日志、artifact、高风险审批、审计日志和事件响应逐层展开，最后用 Training Platform Security Gate 收束。

Identity Runtime Binding

一句话定义：Identity Runtime Binding 是把用户身份、项目身份、TrainingJob 运行身份、service account 和 workload identity 绑定起来，避免所有任务共享平台管理员身份。

RBAC ABAC Policy Fit

一句话定义：RBAC ABAC Policy Fit 是检查角色权限和项目、团队、数据等级、资源池、用途、环境等属性策略是否共同覆盖训练平台访问控制。

Dataset Permission Purpose

一句话定义：Dataset Permission Purpose 是检查用户和任务身份是否被允许以当前用途读取指定 dataset version，而不是只看文件路径是否可读。

Artifact Classification Propagation

一句话定义：Artifact Classification Propagation 是让 checkpoint、模型权重、adapter、eval report、日志和样本输出继承训练数据的最高敏感等级。

Resource Quota Priority Governance

一句话定义：Resource Quota Priority Governance 是用队列、配额、优先级审批和 GPU-hour 预算控制高价值资源使用。

Image Supply Chain Trust

一句话定义：Image Supply Chain Trust 是检查镜像来源、digest、签名、漏洞扫描、SBOM / provenance 和 secret exclusion，降低供应链风险。

Runtime Code Isolation

一句话定义：Runtime Code Isolation 是限制训练代码的 privileged、hostPath、root、外联和 Pod 运行时权限，降低任意代码执行带来的逃逸和越权风险。

Secret Scope Rotation

一句话定义：Secret Scope Rotation 是用 Secret Manager、短期凭证、最小权限、轮换和日志脱敏管理训练任务访问凭证。

Network Egress Control

一句话定义：Network Egress Control 是用默认拒绝、egress allowlist、生产服务阻断和外联日志控制训练代码向外发送数据。

Log Redaction Access

一句话定义：Log Redaction Access 是对训练日志查看、下载、保留和脱敏做权限控制，防止日志成为敏感数据泄露通道。

Model Artifact Permission

一句话定义：Model Artifact Permission 是对 checkpoint、final model、adapter、tokenizer、config、eval report 和 deployment package 做 registry RBAC、下载限制、阶段策略和 lineage 绑定。

High Risk Approval

一句话定义：High Risk Approval 是对 restricted 数据训练、模型导出、生产发布、高优先级队列、关闭日志、未审核镜像、外部网络和删除关键 checkpoint 等动作做审批。

Audit Log Completeness

一句话定义：Audit Log Completeness 是检查任务提交、数据访问、镜像使用、checkpoint、模型下载、权限变更、配额变更、审批和日志查看是否都有 actor、action、resource、decision、reason 和 request id。

Audit Integrity Retention

一句话定义：Audit Integrity Retention 是要求审计日志防篡改、可查询、有留存周期，并能支撑合规、事故响应和复盘。

Incident Response Readiness

一句话定义：Incident Response Readiness 是准备冻结任务、撤销密钥、隔离产物、查询审计、通知 owner、修复策略和复盘的安全响应能力。

Training Platform Security Gate

一句话定义：Training Platform Security Gate 是判断训练平台是否达到身份可信、权限最小化、数据和产物不越权、运行时隔离、密钥安全、网络受控、审计完整和事故可响应最低要求的综合门禁。

典型硬阻断：所有任务共享高权限身份；RBAC 通配；数据用途未授权；restricted 数据训练产物被降级；高优先级资源无审批；镜像无 digest / 签名 / 扫描；privileged 或 hostPath 放开；密钥写入镜像、代码或日志；默认允许外联；日志无脱敏和访问审计；模型权重可随意下载；高风险动作无审批；审计日志缺 actor / action / resource；审计可被篡改或留存不足；没有密钥撤销和产物隔离预案；没有最终安全门禁。

Training Platform System Design Audit

一句话定义：Training Platform System Design Audit 是把“大模型训练平台怎么设计”这类系统设计题，拆成需求、TrainingJob 契约、状态机、准入、调度、launcher、血缘、checkpoint、观测、容错、安全、成本、自助、接口、trade-off 和最终门禁的可审计回答方法。

核心直觉：训练平台系统设计不是说“用 Kubernetes 跑 Job”，也不是把 Portal、Scheduler、Logger、Registry 这些名词排成架构图。高分回答要证明训练任务从提交到模型产物注册的每个阶段都有清楚输入、输出、状态、指标、失败处理和权限边界。Kubernetes Job、Kueue、Kubeflow Trainer、torchrun、OpenTelemetry 和 MLflow 等工具可以是实现选项，但面试中更重要的是抽象边界和生命周期闭环。

常见审计维度：Requirement Clarification 看是否问清训练类型、集群规模、多租户、敏感数据、抢占、恢复、发布和成本预算；TrainingJob Contract 看 owner、project、image digest、code commit、dataset version、final config、resources、distributed、checkpoint、observability、security 和 queue 是否完整；Lifecycle State Machine 看 submitted、validated、queued、scheduled、starting、running、checkpointing、retrying、failed、completed 和 cancelled 是否覆盖且迁移合法；Admission Quota Policy、Scheduler Resource Fit 和 Distributed Launcher Fit 检查准入、配额、fair share、gang scheduling、拓扑感知、rank、world size、rendezvous、GPU binding 和 NCCL；Data Config Lineage、Checkpoint Recovery Coupling、Observability Experiment Linkage 和 Fault Recovery Flow 检查可复现、可恢复、可排障和可重试；Security Audit Governance、Cost Capacity Governance、Developer Self Service、Interface Artifact Contract、Tradeoff Boundary Reasoning 和 Training Platform Design Gate 则检查治理、体验、契约、取舍和最终上线讨论门槛。

常见公式：训练平台系统设计样本可以写成

$$p_i=(r_i,j_i,s_i,a_i,q_i,l_i,d_i,c_i,o_i,f_i,g_i,k_i,u_i,m_i,b_i,t_i)$$

统一覆盖率可以写成

$$C_j=\frac{1}{N}\sum_{i=1}^N\mathbf{1}[g_j(p_i)=1]$$

world size 和 effective batch 可以写成

$$W=n_{\mathrm{node}}n_{\mathrm{proc}},\quad B_{\mathrm{eff}}=B_{\mathrm{micro}}A_{\mathrm{grad}}W$$

P95 排队时间可以写成

$$T_{\mathrm{queue},p95}=Q_{0.95}\left(\{t_{\mathrm{start}},i\}-t_{\mathrm{submit},i}\}_{i=1}^N\right)$$

最大丢失 GPU-hours 可以写成

$$L_{\mathrm{lost}}=G\frac{t_{\mathrm{fail}}-t_{\mathrm{ckpt}}}{60}$$

trace 覆盖率可以写成

$$R_{\mathrm{trace}}=\frac{N_{\mathrm{linked}}}{N_{\mathrm{required}}}$$

训练平台系统设计门禁可以写成

$G_{\{\mathrm{design}\}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \wedge T_{\{\mathrm{queue},p95\}}\leq B_{\{\mathrm{queue}\}} \wedge L_{\{\mathrm{m}\}}$

面试表达：训练平台系统设计题可以按 “需求澄清 -> TrainingJob 契约 -> 控制面准入 -> 调度与资源 -> launcher -> 数据与配置血缘 -> checkpoint 与容错 -> 可观测性与实验追踪 -> artifact registry -> 安全审计 -> 成本容量 -> 开发者体验 -> trade-off 与门禁” 展开。不要把 Kubernetes 当完整答案，也不要漏掉数据、checkpoint、实验、模型产物、安全和成本。

Requirement Clarification

一句话定义：Requirement Clarification 是训练平台系统设计回答的需求澄清层，明确 workload scope、集群规模、多租户、数据敏感度、抢占、恢复 SLO、artifact 发布和成本预算。

TrainingJob Contract

一句话定义：TrainingJob Contract 是训练平台的核心对象契约，冻结一次训练任务的 owner、project、image digest、code commit、dataset version、final config、resources、distributed、checkpoint、observability、security 和 queue。

Lifecycle State Machine

一句话定义：Lifecycle State Machine 是把训练任务从 submitted 到 completed / failed / cancelled 的状态和合法迁移显式化，避免平台只剩不透明的 running / failed。

Admission Quota Policy

一句话定义：Admission Quota Policy 是在任务入队前完成 schema、镜像、数据权限、资源配额、优先级审批、幂等和 dry run 准入校验。

Scheduler Resource Fit

一句话定义：Scheduler Resource Fit 是检查 queue、quota、fair share、gang scheduling、topology-aware placement、backfilling、preemption 和 fragmentation 治理是否匹配训练资源需求。

Distributed Launcher Fit

一句话定义：Distributed Launcher Fit 是检查 launcher adapter 是否把调度结果正确转成 rank、local rank、world size、rendezvous、GPU binding 和 NCCL 配置。

Data Config Lineage

一句话定义：Data Config Lineage 是绑定镜像 digest、代码 commit、dataset version、manifest、final config、环境版本、seed 和 experiment run，回答模型怎么训出来。

Checkpoint Recovery Coupling

一句话定义：Checkpoint Recovery Coupling 是让 checkpoint 策略和调度、抢占、容错、恢复演练、manifest、checksum、最大 stale 时间相互绑定。

Observability Experiment Linkage

一句话定义：Observability Experiment Linkage 是把 logs、metrics、events、traces、rank 维度、dashboard、alerts 和 experiment run 连成同一条排障证据链。

Fault Recovery Flow

一句话定义：Fault Recovery Flow 是用故障分类、retry budget、backoff、checkpoint resume、不可重试快速失败和诊断报告替代盲目自动重试。

Security Audit Governance

一句话定义：Security Audit Governance 是把身份、RBAC / ABAC、数据用途、运行时隔离、镜像供应链、审计日志和事件响应纳入训练平台设计。

Cost Capacity Governance

一句话定义：Cost Capacity Governance 是按 owner、project、job、queue 归因 GPU-hours、存储、网络 and 利用率，并提供预算告警和容量反馈。

Developer Self Service

一句话定义：Developer Self Service 是让用户能通过 Portal、CLI、SDK、模板、状态页、错误报告和从 checkpoint 重跑自助完成常见训练流程。

Interface Artifact Contract

一句话定义：Interface Artifact Contract 是显式定义 API、scheduler handoff、launcher handoff、data contract、artifact registry 和 audit event schema，避免模块之间只靠口头约定。

Tradeoff Boundary Reasoning

一句话定义：Tradeoff Boundary Reasoning 是在系统设计回答中讲清 Kubernetes vs training operator、公平性 vs 利用率、拓扑 vs 排队、抢占 vs 丢失进度、自助 vs 护栏、成本 vs 可靠性。

Training Platform Design Gate

一句话定义：Training Platform Design Gate 是判断训练平台系统设计是否达到可运行、可恢复、可复现、可治理、可排障、可审计和可面试深挖最低要求的综合门禁。

典型硬阻断：需求假设不清；TrainingJob 契约缺关键字段；状态机只有 running / failed；准入绕过配额和安全策略；分布式训练没有 gang scheduling；launcher 的 world size 和资源不一致；数据和配置无法复现；checkpoint 未做恢复演练；只看 stdout 或 loss 曲线；失败全部盲目重试；安全审计缺失；成本无人归因；用户只能找平台同学查状态；artifact registry 和实验记录断链；只背工具名没有 trade-off；没有最终设计门禁。

AI Infra Overview Audit

一句话定义：AI Infra Overview Audit 是把 AI Infra 总览从“模块清单”升级为可检查指标表的审计方法。

核心直觉：AI Infra 不是单独的 Kubernetes、Slurm、GPU 采购、监控工具或实验平台，而是把计算、网络、存储、调度、训练、推理、数据、artifact、评估、可观测性、安全、成本和开发者体验连成可运行、可治理、可复现的基础设施底座。

关键检查项：Compute Accelerator Readiness 看 GPU/加速器、驱动、拓扑和资源暴露是否就绪；Network Communication Readiness 看多机通信、拓扑、RDMA、AllReduce 和跨节点瓶颈；Storage Data Checkpoint Readiness 看数据供给、checkpoint、缓存和恢复；Scheduler Resource Governance 看队列、配额、优先级、抢占和 gang scheduling；Training Platform Reproducibility 看代码、数据、镜像、超参、日志和 checkpoint 是否可复现；Inference Platform SLO Readiness 看 TTFT、TPOT、吞吐、KV cache、限流、灰度和回滚；Data Platform Lineage Quality 看数据来源、版本、权限、质量和污染隔离；Model Artifact Registry Governance 看权重、tokenizer、adapter、量化版本、eval report 和发布状态；Eval Experiment Tracking Coverage 看离线评估、人评、A/B、实验参数和 artifact 追踪；Observability Signal Coverage 看 metrics、logs、traces、events

和 profiles; Security Governance Coverage 看身份、权限、密钥、租户隔离、镜像和审计; Cost Capacity Governance 看 GPU、存储、网络、推理 token、容量和闲置资源; Developer Self-Service Readiness 看 Portal、CLI、SDK、模板和自助排障; AI Infra Boundary Clarity 看是否避免把 AI Infra 简化成某个工具; MLOps LLMOps Platform Boundary Clarity 看生命周期管理、大模型应用运维和平台工程边界; Algorithm Infra Collaboration Readiness 看算法团队和平台团队是否能用指标、SLO、成本和失败样例协作。

常见公式：统一覆盖率可以写成

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(x_i)=1]$$

其中，N 是审计样本数， $g_j(x_i)=1$ 表示样本通过第 j 个检查。

面试表达：AI Infra 总览题不要只背“GPU、网络、存储、调度、训练平台、推理平台”这些词，而要说明每一层解决什么瓶颈、如何被观测、如何被治理，以及算法岗为什么需要理解这些边界。

AI Infra Overview Gate

一句话定义：AI Infra Overview Gate 是判断一个 AI Infra 总览方案是否达到上线讨论门槛的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{ai_infra}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{p0}\}} = 0]$$

其中， τ_j 是每个维度最低通过阈值， R_{p0} 是 P0 级硬阻断数量。

典型硬阻断：GPU 资源有但网络或存储无法支撑训练；推理平台没有 SLO、限流和回滚；模型权重和 tokenizer 没有 artifact 版本；实验无法复现；数据没有 lineage；没有 metrics / logs / traces；跨租户权限和审计缺失；成本和容量没有归因；算法团队只能提工单，无法自助查看任务、指标和失败原因。

常见误区：把 AI Infra 等同于 Kubernetes、等同于买 GPU、等同于 MLOps 平台、等同于监控 dashboard，都会漏掉跨层边界和治理问题。

AI Infra MLOps LLMOps Boundary Audit

一句话定义：AI Infra MLOps LLMOps Boundary Audit 是把 AI Infra、MLOps、LLMOps、Platform Engineering、Data Platform、Model Platform、DevOps 和 SRE 的职责边界转成可检查表的方法。

核心直觉：名字不是关键，主责层、协作层、交付物、接口和门禁才是关键。GPU 队列排队、prompt 变更回归、模型版本回滚、RAG 文档过期、推理 SLO 下降和开发者无法自助提交任务，虽然都可能发生在同一个 AI 平台里，但主责层不同。

常见维度：AI Infra Scope Accuracy 看资源、调度、训练和推理运行时是否被归到 AI Infra；MLOps Lifecycle Accuracy 看数据版本、训练 pipeline、实验追踪、模型注册、部署和回滚是否被归到模型生命周期；LLMOps Application Accuracy 看 prompt、RAG、Agent、工具调用、模型路由、LLM eval、安全和 token 成本是否被归到大模型应用生命周期；Platform Engineering DX Accuracy 看 Portal、CLI、SDK、模板、自助流程和 golden path 是否被当成内部开发者平台能力；DevOps SRE Boundary Accuracy 看 CI/CD、SLO、错误预算、告警和事故响应是否和 AI 工作负载可靠性正确连接；Data Platform Boundary Accuracy 看数据湖、数据质量、血缘、权限和 RAG 文档新鲜度；Model Platform Boundary Accuracy 看权重、tokenizer、adapter、量化版本、评估报告和发布状态；Primary Owner Clarity、Interface Contract Coverage、Artifact Lineage Handoff、Observability SLO Handoff、Security Cost Governance Handoff、Lifecycle Stage Mapping、Incident Routing Accuracy 和 Collaboration Handoff Readiness 则检查跨团队协作是否可落地。

典型公式：边界路由错误率可以写成

$$R_{\{\mathrm{route}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[h_i \neq y_i]$$

其中， y_i 是样本应该归属的主责层， h_i 是当前回答或系统设计给出的主责层。

Boundary Gate

一句话定义：Boundary Gate 是判断 AI 平台职责边界是否足够清楚、可交接、可审计的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{boundary}\}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \ \wedge R_{\{\mathrm{route}\}}=0 \ \wedge P_0=0\right]$$

其中， C_j 是第 j 个边界维度覆盖率， τ_j 是最低阈值， R_{route} 是主责层路由错误率， P_0 是 P0 级边界风险数量。

常见硬阻断：上线事故没人主责；模型权重、数据版本、prompt 版本和评估报告无法串起来；监控有曲线但没有 SLO；平台能力只有工单没有自助接口；安全和成本治理只写在文档里；RAG 质量问题被误判为 GPU 问题；MLOps 被说成 AI Infra，LLMOps 被说成 prompt 管理工具，Platform Engineering 被说成 DevOps 改名。

面试表达：边界题不要追求把每个工具永久归类，而要先讲“这个问题主要是谁主责”，再讲“需要哪些协作层”，最后讲“交付物、接口、指标和门禁是什么”。

Accelerator Selection Audit

一句话定义：Accelerator Selection Audit 是把 GPU、NPU、TPU 或其他 AI 加速器选型从“看峰值算力”升级为容量、带宽、互联、软件生态、成本和风险的可检查表。

核心直觉：大模型硬件选型要同时看工作负载、模型规模、batch、context length、训练或推理 runtime、显存容量、显存带宽、多卡互联、低精度支持、kernel / 通信库 / profiler、云上或自建约束、供给和迁移风险。峰值 TFLOPS 只回答“理论上能算多快”，不能回答“模型能不能放下、decode 会不会被带宽卡住、多机训练会不会被通信卡住、团队能不能调优和排障”。

常见维度：Peak Compute Fit 看峰值算力和低精度矩阵单元是否匹配目标吞吐；Memory Capacity Fit 看权重、梯度、optimizer state、activation、临时 buffer 和 KV cache 是否放得下；Memory Bandwidth Fit 看 decode、embedding、optimizer update 和低 arithmetic intensity 算子是否被 HBM 带宽限制；Interconnect Bandwidth Fit 看张量并行、流水并行、数据并行和 MoE 通信是否匹配 NVLink、PCIe、InfiniBand 或 Pod 拓扑；Low Precision Support 看 FP16、BF16、FP8、INT8 等精度是否被硬件和框架稳定支持；Software Stack Maturity、Kernel Library Readiness、Distributed Communication Readiness 看 CUDA / ROCm / TPU runtime / Neuron 等栈、算子库、通信库和 profiler 是否能覆盖目标模型；Training Memory Budget 和 Inference KV Cache Budget 分别检查训练状态显存和长上下文推理 KV cache；Workload Hardware Fit、Cloud Self Build Decision、Cost Power Capacity Awareness、Fallback Portability Plan 和 Selection Risk Governance 则检查场景、成本、电力、供应、厂商锁定和迁移预案。

常见公式：训练状态显存的粗略下限可以写成

$$M_{\{\mathrm{train}\}}\approx N(B_{\{\mathrm{param}\}}+B_{\{\mathrm{grad}\}}+B_{\{\mathrm{opt}\}})+M_{\{\mathrm{act}\}}+M_{\{\mathrm{tmp}\}}$$

推理 KV cache 显存可以写成

$$M_{\{\mathrm{kv}\}}=2\mathrm{LBSH}_{\{\mathrm{kv}\}}D_{\mathrm{h}}_{\{\mathrm{elem}\}}$$

其中， N 是参数量， L 是层数， B 是 batch 或并发序列数， S 是序列长度， H_{kv} 是 KV head 数， D_{h} 是 head dimension， B_{elem} 是每个元素字节数。

面试表达：GPU、NPU、TPU 选型题不要只背厂商和型号，而要把训练显存、推理 KV cache、roofline 带宽上限、通信暴露比例、软件生态、成本和迁移风险讲成同一套审计口径。

Accelerator Selection Gate

一句话定义：Accelerator Selection Gate 是判断一个加速器选型方案是否达到工程落地讨论门槛的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{accelerator}\}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \ \wedge P_0=0\right]$$

其中， C_j 是第 j 个加速器选型维度覆盖率， τ_j 是最低阈值， P_0 是 P0 级硬件选型风险数量。

典型硬阻断：只看峰值 TFLOPS 而不算训练状态显存；权重能放下但 AdamW 状态放不下；长上下文推理没有估算 KV cache；decode 场景忽略 HBM 带宽；多机训练忽略互联和通信库；选择新硬件但缺 kernel、profiler、框架和排障经验；云上 / 自建只看单价不看利用率、电力、供给、迁移和厂商锁定。

面试表达：高分回答要能解释“为什么这类工作负载适合这类硬件”，再用容量估算、带宽瓶颈、通信瓶颈、软件栈和风险门禁证明方案可落地。

Bandwidth Bottleneck Audit

一句话定义：Bandwidth Bottleneck Audit 是把显存、HBM、PCIe、NVLink、NVSwitch、机间网络、存储和 checkpoint I/O 统一放进性能排查表的审计方法。

核心直觉：大模型性能不是只看峰值算力。GPU 计算单元需要从 HBM 读取权重、activation、KV cache 和临时张量；CPU、GPU、网卡和存储之间会走 PCIe 或相关 peer 路径；单机多卡依赖 NVLink / NVSwitch 或 PCIe 拓扑；多机训练依赖 InfiniBand / RoCE、网卡位置、NCCL 路由和网络拥塞。任何一层数据搬运跟不上，GPU 都可能处于“很忙但不高效”或“等数据”的状态。

常见维度：VRAM Capacity Accounting 看权重、梯度、optimizer state、activation、KV cache、临时 buffer 和通信 buffer 是否都进入预算；HBM Bandwidth Model 看算子是否 memory-bound；PCIe Transfer Awareness 看 CPU-GPU 拷贝、offload、网卡和存储路径是否被量化；NVLink Topology Awareness 和 NVSwitch All-to-All Awareness 看单机 GPU 互联是否真实可用、是否均匀；Inter-Node Network Awareness 看机间网络、RDMA、拥塞和拓扑；KV Cache Growth Accounting、Training State Memory Accounting 和 Communication Volume Accounting 分别检查推理 KV cache、训练状态显存和 collective 通信量；Topology-Aware Parallel Group 看 tensor parallel、pipeline parallel、data parallel group 是否按拓扑分组；Dataloader Storage IO Awareness、Checkpoint IO Awareness 和 Offload Penalty Awareness 看数据供给、checkpoint 阻塞和 offload 惩罚；Overlap Fusion Optimization 看通信计算重叠和 kernel fusion；Observability Metric Coverage 看 HBM、PCIe、NVLink、网络、存储、NCCL 和 step time 分解是否可观测。

常见公式：显存总预算可以写成

$$M_{\{\mathrm{vram}\}} = M_{\{\mathrm{weight}\}} + M_{\{\mathrm{grad}\}} + M_{\{\mathrm{opt}\}} + M_{\{\mathrm{act}\}} + M_{\{\mathrm{kv}\}} + M_{\{\mathrm{temp}\}}$$

有效带宽可以写成

$$B_{\{\mathrm{eff}\}} = \frac{D}{T}$$

其中，D 是实际搬运数据量，T 是耗时。多卡通信是否暴露，可以看 R_{comm} ；PCIe / offload 是否拖慢 step，可以看 R_{pcie} 。

面试表达：带宽瓶颈题要先分层：HBM、PCIe、GPU 间互联、机间网络、存储 / checkpoint 和软件 kernel；再用 step time 分解、通信占比、有效带宽、拓扑和失败样例定位，而不是只看 GPU utilization。

Bandwidth Bottleneck Gate

一句话定义：Bandwidth Bottleneck Gate 是判断一个训练或推理系统是否已经把主要数据搬运链路纳入容量、性能和观测门禁的综合门禁。

门禁形式可以写成：

$$G_{\{\mathrm{bandwidth}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \wedge \ R_{\mathrm{comm}} \leq \rho_{\mathrm{comm}} \ \wedge \ R_{\mathrm{pcie}} \leq \rho_{\mathrm{pcie}} \right]$$

其中， C_j 是第 j 个带宽审计维度覆盖率， τ_j 是最低阈值， R_{comm} 是通信暴露比例， R_{pcie} 是 PCIe / offload 暴露比例， P_0 是 P0 级带宽风险数量。

典型硬阻断：没有完整显存预算；只看权重不看 KV cache；decode 被 HBM 带宽卡住但只调 batch；CPU-GPU 拷贝阻塞 step；offload 让 PCIe 成为主瓶颈；把 tensor parallel group 跨慢链路切分；以为 8 卡机器默认都有 NVSwitch；多机训练没有看网卡拓扑、NCCL 路由和网络拥塞；checkpoint 写入阻塞训练；没有 HBM / PCIe / NVLink / 网络 / 存储的指标。

面试表达：带宽门禁的价值是把“慢”拆成可测的链路。先量化容量、有效带宽、通信暴露和 I/O 暴露，再决定用 kernel fusion、量化、拓扑分组、通信重叠、并行策略调整、数据预取或 checkpoint 异步化。

分布式训练排障常见问题

一句话定义：分布式训练排障是定位多卡训练中显存、通信、数值、数据和性能异常的过程。

常见显存问题：batch 过大、序列过长、激活未 checkpoint、ZeRO/FSDP wrap 不合理、gradient accumulation 配置错误。

常见通信问题：NCCL 卡住、跨节点网络慢、rank 配置错误、某些进程提前退出、collective 调用顺序不一致。

常见数值问题：FP16 overflow、loss scaling 不合适、学习率随 global batch 改变后未调整、梯度裁剪缺失。

常见性能问题：pipeline bubble 大、tensor parallel 通信过重、数据加载跟不上、checkpoint 保存阻塞训练、padding 浪费严重。

面试表达：分布式训练排障要先判断是 OOM、hang、loss 异常还是吞吐低，再分别沿显存、通信、数值和数据链路排查。

本章小结

本章围绕大模型分布式训练系统展开。核心问题只有四个：模型能不能放下，训练能不能跑快，通信能不能承受，训练结果能不能稳定复现。

高频结论如下：

1. 数据并行拆 batch，不拆模型；张量并行拆层内矩阵；流水并行拆模型层。
2. DDP 的核心是每卡完整模型副本，并通过 all-reduce 同步梯度。
3. all-reduce 让所有设备获得完整聚合结果，reduce-scatter 保留聚合分片，all-gather 收集完整张量。
4. 张量并行适合超大矩阵，但层内通信频繁，对高速互联依赖强。
5. 流水并行通过 micro-batch 填充多 stage，主要问题是 pipeline bubble 和负载均衡。
6. ZeRO 通过切 optimizer state、gradient 和 parameter 消除数据并行冗余。
7. ZeRO-3 和 FSDP 显存节省大，但需要频繁 all-gather 和 reduce-scatter。
8. activation checkpointing 用重算换显存，是大模型训练常用手段。
9. gradient accumulation 用多个 micro-batch 模拟大 global batch，但不直接提高训练速度。
10. mixed precision 用低精度加速和省显存，但 FP16 需要关注 loss scaling 和数值稳定。
11. MFU 比 GPU utilization 更能衡量大模型训练效率。
12. 分布式训练排障要分清显存、通信、数值、数据和调度问题。

下一章，我们进入长上下文、RAG 与 Agent。

K. 长上下文、RAG 与 Agent

条目：Long Context、Context Window、Position Extrapolation、RAG、RAG Product、RAG Incident、RAG Error Attribution、Document Governance、Retrieval Recall、Mean Reciprocal Rank / MRR、Retrieval Miss、Context Drop、Context Recall、Context Precision、Evidence Support Rate、Citation Accuracy、Unsupported Claim Rate、Permission Leak Rate、Abstention Accuracy、Stale Evidence Rate、RAG Freshness Gate、RAG Product Gate、RAG Incident Gate、Dense Retrieval、Reranker、Vector Database、RAG Prompt Injection、Agentic RAG、Active Retrieval、Retrieval Controller、Multi-Round Retrieval、Evidence State、Evidence Gap、Query Drift、New Evidence Gain、Agentic RAG Gate、Agent、Agent Harness、Agent Runtime、Session Manager、Task Manager、Context Builder、Model Adapter、State Store、Error Handler、Runtime Gate、Agent Loop、Harness Loop Audit、Action Parser、Agent State、Agent Action、Observation、Controller、Tool Registry、Tool Registry Completeness、Tool Schema Strictness、Tool Permission Binding、Side Effect Level、Risk Protection Coverage、Output Normalization Coverage、Tool Version Coverage、Tool Trace Readiness、Untrusted Tool Output Boundary、Tool Registry Gate、Tool Executor、Execution Request Validity、Schema Validation Pass Rate、Permission Enforcement Pass Rate、Execution Mode Accuracy、Async Completion Tracking、Timeout Cancellation Coverage、Idempotency Protection Rate、Unknown State Escalation Rate、Retry Safety Rate、Side Effect Confirmation Coverage、Structured Result Coverage、Executor Trace Completeness、Tool Executor Gate、Tool Permission Model、Authorization Decision Accuracy、Trusted Context Injection、Tenant Isolation Pass Rate、Tool Permission Enforcement、Object Permission Accuracy、Field Projection Safety、Action Context Policy Accuracy、Prompt Injection Block Rate、Token Audience Validation、Revocation Cache Safety、Error Disclosure Safety、Permission Audit Completeness、Tool Permission Gate、Tool Security、Prompt Injection Containment、Untrusted Content Isolation、Unauthorized Access Block Rate、Sensitive Data Protection、Tool External Transfer Gate、Dangerous Action Confirmation、SSRF Block Rate、SQL Risk Block Rate、Path Traversal Block Rate、Shell Sandbox Enforcement、Data Flow Policy Pass Rate、Tool Security Audit Completeness、Security Alert Completeness、Safety Eval Regression Pass Rate、Tool Security Gate、Tool Trace Replay Audit、ID Tree Integrity、Argument Lineage Coverage、Permission

Trace Completeness, Tool Result Trace Completeness, Audit Event Completeness, Replay Side Effect Safety, Alert Owner Coverage, Eval Linkage Coverage, Tool Version Release Audit, Spec Lint Pass Rate, Schema Backward Compatibility, Description Behavior Guard, Output Compatibility, Permission Policy Compatibility, Version Matrix Capture, Offline Eval Pass Rate, Canary Routing Stability, Canary Quality Guard, Canary Safety Guard, Cost Latency Guard, Tool Release Rollback Readiness, Lifecycle Coverage, Tool Release Gate, Enterprise Tool Platform Audit, Platform Module Coverage, Platform Interface Contract Coverage, Registry Readiness, Router Governance Coverage, Executor Safety Coverage, Platform Permission Integration Coverage, Platform Safety Guard Coverage, Platform Trace Audit Coverage, Platform Eval Service Coverage, Platform Release Governance Coverage, Platform Tenant Isolation Coverage, Platform Provider Adapter Coverage, High Availability Readiness, Admin Ops Governance Coverage, Enterprise Tool Platform Gate, MCP Background Audit, Direct Integration Count, MCP Integration Count, Integration Reduction, Capability Model Coverage, Context Object Coverage, Discovery Standardization, MCP Schema Contract Coverage, Host Server Boundary Clarity, Local Context Control, Cross Client Reuse, Governance Boundary Clarity, MCP Trace Eval Readiness, MCP A2A Distinction, MCP Background Gate, MCP Concept Audit, Host Policy Ownership, MCP Client Server Boundary, Server Capability Declaration, MCP Tool Schema and Result, MCP Resource URI Metadata, MCP Prompt Argument Review, MCP Lifecycle Negotiation, MCP Transport Policy, MCP Roots Boundary, MCP Sampling Control, Host Capability Filtering, MCP Context Budget, MCP Concept Gate, MCP Server Implementation Audit, MCP Server Metadata Readiness, MCP Server Capability Declaration, MCP Tool Registry Readiness, MCP Strict Schema Coverage, MCP Handler Execution Coverage, MCP Argument Validation Coverage, MCP Structured Result Coverage, MCP Structured Error Coverage, MCP Resource Scope Coverage, MCP Prompt Template Coverage, MCP Server Transport Policy Coverage, MCP Host Connection Readiness, MCP Server Safety Baseline, MCP Server Trace Readiness, MCP Server Gate, MCP Tool Function Calling Comparison Audit, Protocol Layer Clarity, Tool Discovery Boundary, MCP Capability Scope Coverage, MCP Execution Boundary Clarity, MCP Projection Mapping Coverage, Adapter Separation Coverage, MCP Lifecycle Version Awareness, MCP Governance Registry Import, MCP Security Boundary Enforcement, MCP Use Case Selection Fit, MCP Error Surface Separation, MCP Latency Availability Tradeoff, MCP Function Calling Gate, MCP Resource Exposure Audit, Resource URI Validity, Resource Metadata Completeness, Resource List Filtering Boundary, Resource Read Scope Enforcement, MCP Roots Containment, MCP Resource Permission Enforcement, MCP Resource Field Projection Safety, Resource Context Budget Control, Resource Citation Traceability, Untrusted Resource Labeling, Resource Freshness Version Awareness, Resource Template Boundary, Resource Subscription Awareness, Resource Trace Eval Readiness, MCP Resource Gate, MCP Prompt Exposure Audit, Prompt Capability Declaration, Prompt Discovery Coverage, Prompt Argument Schema Coverage, Prompt Required Argument Validation, Prompt Rendering Safety, Prompt Role Boundary Enforcement, Prompt Dependency Alignment, Prompt Version Governance, Prompt Eval Binding, Prompt Permission Enforcement, Prompt Injection Containment, Prompt User Control, Prompt List Change Awareness, Prompt Trace Readiness, MCP Prompt Gate, MCP Security Sandbox Audit, Server Connection Governance, Authorization Token Binding, Scope Minimization, Roots Sandbox Containment, File Secret Blocking, Network SSRF Protection, MCP Shell Sandbox Enforcement, Prompt Injection Data Boundary, High Risk Confirmation, Sensitive Data Flow Control, Local Credential Isolation, MCP Tenant Isolation, Server Supply Chain Governance, MCP Security Trace Readiness, MCP Security Eval Coverage, MCP Security Gate, MCP Integration Surface Audit, MCP Capability Registration Coverage, MCP Namespace Isolation Coverage, MCP IDE Context Routing, MCP Knowledge Citation Traceability, MCP Database Query Governance, MCP Browser Action Governance, MCP Terminal Sandbox Governance, MCP Integration Context Budget Control, MCP Cross Server Data Flow Control, MCP High Risk Approval Coverage, MCP Output Projection Coverage, MCP Integration Trace Readiness, MCP Integration Eval Coverage, MCP Integration Gate, Permission Gate, Replay Readiness, Harness Gate, Agent Trace, Trace Schema Completeness, Span Schema Completeness, Span Tree Validity, Timeline Validity, Artifact Reference Coverage, Version Capture Coverage, Replay Readiness Rate, Privacy Masking Coverage, Error Attribution Coverage, Final Status Consistency, Metric Export Coverage, Eval Export Coverage, Trace Replay Gate, Agent Budget, Agent Incident, Plan Feasibility Rate, False Completion Rate, Tool Result Injection Block Rate, Agent Incident Gate, Agent Gate, Agent Product, Workflow-Agent Hybrid, Agent Automation Level, Agent Product Trace, Tool Execution Success Rate, Human Confirmation Coverage, Agent Recovery

Rate, Observation Use Rate, State Update Coverage, Unauthorized Action Rate, Budget Overrun Rate, Agent Product Gate, Tool Use, Tool Schema, JSON Schema, Required Field, Additional Properties, Enum Constraint, Pattern Constraint, Range Constraint, Business Validation, Schema Repair, Schema Gate, Tool Call, Tool Choice, Tool Choice Policy, Candidate Tool Set, Auto Tool Choice, None Tool Choice, Required Tool Choice, Forced Tool Choice, Allowed Tools, Tool Choice Mode Accuracy, No-Tool Clarification Block Rate, Forced Missing Argument Block Rate, Parallel Safety Rate, Rate Limit Pass Rate, Confirmation Enforcement Rate, Cost Budget Pass Rate, Loop Control Rate, Tool Choice Gate, Argument Parsing, Parse Success Rate, Validation Error, Normalization, Argument Evidence Map, Argument Evidence Coverage, Safe Normalization Rate, Repair Attempt, Model Self-Repair, Clarification Coverage, Unsafe Argument Block Rate, Retry Budget Pass Rate, Idempotency Key Coverage, Repair Trace Completeness, Argument Repair Gate, Structured Function Calling, JSON Mode, Structured Outputs, Function Calling, Function Calling Protocol, Tool Call ID, Tool Result, Tool Result Context, Tool Result Projection, Source Metadata Coverage, Redaction Coverage, Tool Result Context Budget Pass Rate, Injection Containment Rate, Error Status Fidelity, Compression Fidelity, Conflict Labeling Rate, Citation Support Rate, Tool Result Freshness Pass Rate, Memory Boundary Pass Rate, Tool Result Context Gate, Finish Reason, Tool Loop, Parallel Tool Calls, Streaming Tool Call Delta, Protocol Chain Validity, Tool Result ID Match Rate, Finish Reason Consistency, Streaming Safety Rate, Parallel Alignment Rate, Tool Selection Accuracy, Schema Valid Rate, Required Field Pass Rate, Type Valid Rate, Enum Valid Rate, Pattern Valid Rate, Range Valid Rate, Additional Properties Block Rate, Business Rule Pass Rate, Schema Repair Success Rate, Argument Exact Match, Argument Validity, Unauthorized Attempt Block Rate, Tool Call Trace, Tool Result Injection, Tool Calling Gate, ReAct, Plan-Act-Observe, ReAct Trace, Plan Update, Repeat Action, Premature Final, ReAct Gate, Planning, Task Decomposition, Subgoal, Acceptance Criteria, Dependency Graph, Dynamic Replanning, Critical Path, Planning Gate, Memory, Short-Term Memory, Long-Term Memory, Episodic Memory, Semantic Memory, Procedural Memory, Preference Memory, Memory Write Gate, Memory Retrieval Score, Memory Permission Gate, Memory Decay, Memory Conflict, Memory Pollution, Memory Gate, Code Agent, Coding Agent Workflow, Task Type Classification, Repository Exploration Coverage, Plan Coverage, Workflow Validation Coverage, Feedback Use Rate, Coding Agent Workflow Gate, File System Tool, Workspace Root, Path Containment, Read File Tool, Search Tool, Apply Patch Tool, Patch Context Match, Secret File Block Rate, Read Truncation Clarity, Concurrent Edit Protection, Unrelated Diff Rate, Diff Faithfulness, Rollback Readiness, File Edit Gate, Terminal Execution, Bash Tool, Test Runner Safety, Command Risk Level, Command Allowlist, Command Denylist, Command Auto Execution Precision, Dangerous Command Block Rate, Command Confirmation Coverage, Network Access Control, Command Output Truncation, Secret Masking Coverage, Command Trace, Command Safety Gate, Context Management, Context Candidate, Context Budget Utilization, Key Context Recall, Constraint Retention, Task State Compression, Summary Faithfulness, Stale Summary Rate, Tool Output Compression Fidelity, Current Diff Coverage, Trust Boundary Coverage, Memory Write Safety, Context Builder Gate, Patch Localization, Minimal Patch, Test Execution Feedback, Code Sandbox, User Change Protection, Code Agent Gate, Claude Code Architecture, Terminal Coding Agent, CLI Control Command, Architecture Evidence Coverage, Required Module Coverage, Access Governance Coverage, Core Loop Governance Coverage, Permission Control Coverage, State Recovery Coverage, Extension Governance Coverage, Observability Coverage, Architecture Eval Readiness, High-Risk Surface Governance, Claude Code Architecture Gate, OpenCode Architecture, Open Runtime Control Plane, Open Runtime Module Coverage, Config Governance Coverage, Agent Permission Isolation, Tool Permission Binding, MCP Tool Namespace Coverage, Custom Tool Schema Coverage, Server API Governance, Snapshot Recovery Coverage, Provider Adapter Coverage, OpenCode Architecture Gate, Coding Agent Comparison, Product Surface Coverage, Context Source Coverage, Tool Execution Coverage, Permission Governance Coverage, Edit Recovery Coverage, Agent Eval Readiness, Enterprise Governance Coverage, Cross-Agent Risk Governance, Coding Agent Comparison Gate, Skill Marketplace Audit, Catalog Metadata, Search Discovery, Detail Page Completeness, Review Workflow, Permission Transparency, Install Approval Flow, Rating Quality Balance, Operations Metrics, Duplicate Capability Governance, Admin Permission View, Developer Console Readiness, Security Center Readiness, Recommendation Policy Safety, Lifecycle Visibility, Owner Maintenance, Portal Audit Trace, Skill Marketplace Gate, MCP A2A Harness Integration, Protocol Capability Registry, MCP Integration, A2A Integration, Capability Discovery Coverage, Namespace Isolation Coverage, Protocol

Schema Validity、Protocol Permission Binding、High-Risk Approval Coverage、Context Output Budget Coverage、External Trust Boundary Coverage、A2A Lifecycle Coverage、Protocol Trace Coverage、Protocol Replay Readiness、Protocol Version Capture、Protocol Integration Gate、Agent Harness System Design、Runtime Module Coverage、Interface Contract Coverage、Agent State Machine Coverage、Permission Integration Coverage、Context Control Coverage、Execution Isolation Coverage、System Trace Coverage、System Replay Readiness、System Eval Readiness、Recovery Coverage、Harness Version Capture、Enterprise Governance Readiness、Harness System Gate、Browser Agent、Computer Use Agent、GUI Action、Screenshot Observation、Accessibility Tree、UI State Tracking、Misclick Rate、Form Accuracy、High-Risk Action Protection、UI Agent Gate、Multi-Agent、Coordinator、Role Assignment、Communication Protocol、Blackboard、Agent Debate、Judge Agent、Consensus / Voting、Conflict Resolution、Duplicate Work Rate、Single-Agent Lift Rate、Multi-Agent Gate、Agent Evaluation、Evaluation Harness、Eval Dataset Bucket Coverage、Environment Reproducibility、Validator Coverage、Weighted Task Success、Diff Scope Safety、Baseline Fairness、Regression Pass Rate、Unsafe Execution Rate、Cost Budget Pass Rate、Flaky Task Rate、Eval Version Capture、Eval Report Completeness、Evaluation Harness Gate、Agent Benchmark、Eval Sample、Trajectory Evaluation、Partial Success Score、Trace Completeness、Summary Faithfulness、Claim Support Rate、Recovery Success Rate、Cost-Quality Trade-off、Regression Suite、Agent Eval Gate、Agent Safety、Least Privilege、Tool Permission Matrix、Permission Request、Least Privilege Coverage、Permission Matrix Coverage、Unauthorized Action Block Rate、Secret Access Block Rate、Network Egress Control、Sandbox Enforcement Coverage、Dry Run Coverage、Audit Log Completeness、Irreversible Action Protection、Permission Sandbox Gate、Untrusted Content Boundary、Data Flow Guard、Sensitive Data Filter、External Transfer Gate、Sandbox Policy、Human Confirmation、Dry Run、Audit Log、Irreversible Action、Memory Safety、Supply Chain Risk、Safety Alternative、Agent Safety Gate、Agent Interview Readiness、Agent Interview Rubric、Agent Answer Coverage、Agent Formula Coverage、Agent Demo Coverage、Trace Metric Coverage、Project Evidence Score、Trade-off Depth、Weak Agent Question、Agent Revision Plan、Agent Interview Gate。

本轮工具失败恢复新增条目：Tool Failure Recovery、Structured Tool Error、Retryable Error、Non-Retryable Error、Retry Policy Precision、Exponential Backoff with Jitter、Retry Budget、Unknown Execution State、Fallback Honesty、Circuit Breaker Containment、Timeout Cancellation Coverage、Human Handoff Readiness、User-Visible Error Clarity、Failure Trace Completeness、Tool Failure Recovery Gate。

本轮工具调用评估新增条目：Tool Calling Evaluation、Tool Call Recall、Tool Call Precision、Tool Set Precision、Tool Set Recall、Argument Value Accuracy、Argument Source Coverage、Execution Success Rate、Observation Use Correctness、Safe Failure Rate、Unsafe Failure Rate、Tool Eval Dataset Coverage、Tool Eval Regression Pass Rate、Tool Calling Eval Gate。

本轮质量安全审核新增条目：Tool Skill Quality Safety Audit、Tool Schema Clarity、Argument Validation、Execution Reliability、Output Stability、Side Effect Control、Skill Task Quality、Factual Grounding、Completeness Format、Offline Eval Coverage、Online Monitoring、Permission Least Privilege、Data Security Review、Prompt Injection Resilience、High Risk Action Control、Supply Chain Governance、Human Review Readiness、Regression Release Gate、Audit Trace Readiness、Quality Safety Gate。

本轮可组合 workflow 新增条目：Composable Workflow Audit、Workflow Graph Validity、Dependency Acyclicity、Workflow IO Contract、Condition Determinism、Workflow State Coverage、Workflow Permission Boundary、Workflow Data Flow Policy、Workflow Idempotency Coverage、Retry Safety、Compensation Readiness、Workflow Human Approval Coverage、Workflow Trace Replay Readiness、Workflow Eval Coverage、Workflow Gate。

本轮 Prompt Injection 防御新增条目：Prompt Injection Defense Audit、Instruction Data Separation、Source Trust Labeling、Taint Propagation、Policy Pre-Tool Gate、Risky Tool Isolation、Untrusted Action Blocking、Sensitive Data Control、Prompt Injection High Risk Confirmation、Sandbox Enforcement、Output Redaction Projection、RAG Source Boundary、Multi-Agent Taint Propagation、Prompt Injection Trace Readiness、Prompt Injection Eval Coverage、Prompt Injection Defense Gate。

本轮工具输出可信度新增条目：Tool Output Trust Audit、Source Metadata Coverage、Trust Level Coverage、Freshness Version Coverage、Access Scope Disclosure、Citation Binding、Citation Support Accuracy、Evidence Chain Completeness、Claim Type Calibration、Confidence Limitation Disclosure、Conflict Resolution Readiness、Summary Provenance Retention、RAG Chunk Citation Accuracy、Multi-Agent

Evidence Propagation、Provenance Trace Replay、Citation Forgery Block、Tool Output Trust Eval Coverage、Tool Output Trust Gate。

本轮 RAG / Tool / Agent / Memory 组合新增条目：RAG Tool Agent Memory Integration Audit、Capability Boundary Clarity、Orchestration Mode Fit、Context Priority Enforcement、Context Budget Allocation、Evidence Preservation、Tool Observation Use、Agent State Update Discipline、Memory Read Relevance、Memory Write Gate、Memory Scope Permission、Conflict Resolution Policy、Injection Propagation Control、Sensitive Data Memory Block、Trace Linkage Coverage、Layered Eval Coverage、RAG Tool Agent Memory Integration Gate。

本轮工具调用成本、延迟和并发控制新增条目：Tool Cost Latency Concurrency Audit、Cost Attribution Coverage、Latency Breakdown Coverage、Tool Call Budget Enforcement、Timeout Deadline Coverage、Retry Classification Accuracy、Idempotent Retry Safety、Concurrency Limit Enforcement、Queue Priority Fairness、Rate Limit Quota Handling、Cache Safety Correctness、Batching Partial Success Readiness、Result Trimming Budget Control、Degradation Transparency、Router Performance Awareness、Performance Trace Readiness、Cost Latency Eval Coverage、Tool Performance Gate。

本轮 Tool-use Eval Benchmark 新增条目：Tool Use Eval Benchmark Audit、Tool Need Accuracy、No-Tool Overcall Control、Tool Selection Accuracy、Tool Set Precision、Tool Set Recall、Argument Schema Validity、Argument Semantic Accuracy、Argument Source Coverage、Sequence Order Accuracy、Observation Grounding Accuracy、Error Recovery Readiness、Safe Failure Handling、Safety Policy Compliance、Tool Simulator Determinism、Trace Replay Coverage、Cost Latency Regression Control、Benchmark Slice Coverage、Regression Gate Readiness、Tool Use Eval Benchmark Gate。

本轮 Function Calling / MCP / A2A 横向对比新增条目：Function MCP A2A Comparison Audit、Layer Boundary Clarity、Function Calling Fit、MCP Integration Fit、A2A Delegation Fit、Object Contract Coverage、Capability Discovery Fit、Lifecycle State Alignment、Context Transfer Boundary、Permission Governance Split、Host Runtime Ownership、Trace Chain Continuity、Eval Gate Linkage、Overengineering Control、Severe Protocol Misuse Rate、Protocol Composition Gate。

本轮主流平台工具协议对比新增条目：Provider Framework Tool Protocol Audit、Provider Framework Type Clarity、Tool Schema Projection、Tool Choice Mapping、Tool Result Round Trip、Streaming Event Assembly、Parallel Call Alignment、Built-In Tool Boundary、Error Normalization、Provider Adapter Isolation、Framework Escape Hatch、RAG Tool Traceability、Permission Safety Consistency、Migration Eval Coverage、Provider Trace Replay Readiness、Vendor Lock-In Control、Provider Runtime Gate。

本轮企业 MCP 工具平台设计新增条目：Enterprise MCP Platform Audit、MCP Gateway Readiness、MCP Registry Metadata Completeness、MCP Capability Namespace Isolation、MCP Tool Resource Prompt Contract、MCP Host Client Server Boundary Clarity、MCP OBO Scope Binding、MCP Tenant Isolation Enforcement、MCP Policy Engine Coverage、MCP Sandbox Roots Containment、MCP Context Output Projection、MCP Prompt Injection Taint Propagation、MCP Trace Audit Replay Continuity、MCP Cost Latency Quota Control、MCP Developer Portal Review Readiness、MCP Release Lifecycle Governance、MCP Provider Adapter Compatibility、MCP Eval Regression Coverage、MCP High Availability Readiness、Enterprise MCP Platform Gate。

本轮跨 Agent 协作系统设计新增条目：Cross-Agent Collaboration System Audit、Cross-Agent Requirement Goal Clarity、Cross-Agent Collaboration Module Coverage、Cross-Agent Registry Routing Governance、Cross-Agent Task Graph Validity、Cross-Agent A2A Lifecycle Alignment、Cross-Agent Context Minimization Enforcement、Cross-Agent Permission Attenuation Binding、Cross-Agent MCP Tool Boundary Governance、Cross-Agent Artifact Evidence Grounding、Cross-Agent Conflict Arbitration Readiness、Cross-Agent Hallucination Propagation Control、Cross-Agent Delegation Loop Containment、Cross-Agent Human Handoff Approval Readiness、Cross-Agent Trace Audit Replay Coverage、Cross-Agent Eval Baseline Regression Coverage、Cross-Agent Cost Latency Budget Control、Cross-Agent Scalability Idempotency Readiness、Cross-Agent Collaboration Fit Control、Cross-Agent Collaboration System Gate。

本轮工具协议生态未来演进新增条目：Tool Protocol Future Audit、Layered Protocol Boundary Clarity、Standardization Extension Balance、Long Task Lifecycle Readiness、Capability Package Completeness、Agent-Centric Autonomy Governance、Tool Safety Metadata Coverage、Provenance Verifiability、Mar-

ketplace Governance Readiness、Provider Adapter Migration Control、Auto Generated Tool Review Gate、Workflow Runtime Integration、Behavior Eval Coverage、Human Model Documentation Split、Autonomy Risk Tiering、Responsibility Attribution Trace、Natural Language API Boundary、Agent Operating Layer Governance、Engineer Readiness Coverage、Unsupported Speculation Rate、Tool Protocol Future Gate。

本轮 Tool Registry 细化新增条目：Tool Registry Identity Coverage、Tool Description Quality、Schema Contract Coverage、Runtime Metadata Coverage、Tool Permission Binding Coverage、Risk Annotation Coverage、Version Trace Coverage、Lifecycle Policy Pass Rate、Owner SLO Coverage、Tool Eval Binding Coverage、Provider Projection Readiness、Registry Audit Completeness。

本轮 Tool Router 新增条目：Tool Router、Scenario Filter Pass Rate、Permission Filter Pass Rate、Risk Filter Pass Rate、Candidate Recall、Candidate Precision、Candidate Size Pass Rate、Clarification Accuracy、Forced Tool Accuracy、Router Parallel Safety、Provider Capability Compatibility、Router Trace Completeness、Tool Router Gate。

本轮企业 LLMOps 平台新增条目：Enterprise LLMOps Platform Audit、LLMOps Application Resource Model、LLMOps Release Manifest、Prompt Registry Versioning、RAG KB Permission Governance、Tool Agent Runtime Control、LLMOps Eval Feedback Loop、LLMOps Trace Observability Readiness、LLMOps Cost Budget Governance、LLMOps Platform Gate。

本轮 Tool Executor 新增条目：Execution Request Validity、Schema Validation Pass Rate、Permission Enforcement Pass Rate、Execution Mode Accuracy、Async Completion Tracking、Timeout Cancellation Coverage、Idempotency Protection Rate、Unknown State Escalation Rate、Retry Safety Rate、Side Effect Confirmation Coverage、Structured Result Coverage、Executor Trace Completeness、Tool Executor Gate。

本轮 Tool Permission Model 新增条目：Tool Permission Model、Authorization Decision Accuracy、Trusted Context Injection、Tenant Isolation Pass Rate、Tool Permission Enforcement、Object Permission Accuracy、Field Projection Safety、Action Context Policy Accuracy、Prompt Injection Block Rate、Token Audience Validation、Revocation Cache Safety、Error Disclosure Safety、Permission Audit Completeness、Tool Permission Gate。

本轮 Tool Security 新增条目：Tool Security、Prompt Injection Containment、Untrusted Content Isolation、Unauthorized Access Block Rate、Sensitive Data Protection、Tool External Transfer Gate、Dangerous Action Confirmation、SSRF Block Rate、SQL Risk Block Rate、Path Traversal Block Rate、Shell Sandbox Enforcement、Data Flow Policy Pass Rate、Tool Security Audit Completeness、Security Alert Completeness、Safety Eval Regression Pass Rate、Tool Security Gate。

本轮 Tool Trace / Replay 新增条目：Tool Trace Replay Audit、ID Tree Integrity、Argument Lineage Coverage、Permission Trace Completeness、Tool Result Trace Completeness、Audit Event Completeness、Replay Side Effect Safety、Alert Owner Coverage、Eval Linkage Coverage。

本轮 Tool Version Release 新增条目：Tool Version Release Audit、Spec Lint Pass Rate、Schema Backward Compatibility、Description Behavior Guard、Output Compatibility、Permission Policy Compatibility、Version Matrix Capture、Offline Eval Pass Rate、Canary Routing Stability、Canary Quality Guard、Canary Safety Guard、Cost Latency Guard、Tool Release Rollback Readiness、Lifecycle Coverage、Tool Release Gate。

本轮 Enterprise Tool Platform 新增条目：Enterprise Tool Platform Audit、Platform Module Coverage、Platform Interface Contract Coverage、Registry Readiness、Router Governance Coverage、Executor Safety Coverage、Platform Permission Integration Coverage、Platform Safety Guard Coverage、Platform Trace Audit Coverage、Platform Eval Service Coverage、Platform Release Governance Coverage、Platform Tenant Isolation Coverage、Platform Provider Adapter Coverage、High Availability Readiness、Admin Ops Governance Coverage、Enterprise Tool Platform Gate。

本轮 MCP Background 新增条目：MCP Background Audit、Direct Integration Count、MCP Integration Count、Integration Reduction、Capability Model Coverage、Context Object Coverage、Discovery Standardization、MCP Schema Contract Coverage、Host Server Boundary Clarity、Local Context Control、

Cross Client Reuse、Governance Boundary Clarity、MCP Trace Eval Readiness、MCP A2A Distinction、MCP Background Gate。

本轮 MCP Concept 新增条目：MCP Concept Audit、Host Policy Ownership、MCP Client Server Boundary、Server Capability Declaration、MCP Tool Schema and Result、MCP Resource URI Metadata、MCP Prompt Argument Review、MCP Lifecycle Negotiation、MCP Transport Policy、MCP Roots Boundary、MCP Sampling Control、Host Capability Filtering、MCP Context Budget、MCP Concept Gate。

本轮 MCP Server 最小实现新增条目：MCP Server Implementation Audit、MCP Server Metadata Readiness、MCP Server Capability Declaration、MCP Tool Registry Readiness、MCP Strict Schema Coverage、MCP Handler Execution Coverage、MCP Argument Validation Coverage、MCP Structured Result Coverage、MCP Structured Error Coverage、MCP Resource Scope Coverage、MCP Prompt Template Coverage、MCP Server Transport Policy Coverage、MCP Host Connection Readiness、MCP Server Safety Baseline、MCP Server Trace Readiness、MCP Server Gate。

本轮 MCP Tool / Function Calling 对比新增条目：MCP Tool Function Calling Comparison Audit、Protocol Layer Clarity、Tool Discovery Boundary、MCP Capability Scope Coverage、MCP Execution Boundary Clarity、MCP Projection Mapping Coverage、Adapter Separation Coverage、MCP Lifecycle Version Awareness、MCP Governance Registry Import、MCP Security Boundary Enforcement、MCP Use Case Selection Fit、MCP Error Surface Separation、MCP Latency Availability Tradeoff、MCP Function Calling Gate。

本轮 MCP Resources 新增条目：MCP Resource Exposure Audit、Resource URI Validity、Resource Metadata Completeness、Resource List Filtering Boundary、Resource Read Scope Enforcement、MCP Roots Containment、MCP Resource Permission Enforcement、MCP Resource Field Projection Safety、Resource Context Budget Control、Resource Citation Traceability、Untrusted Resource Labeling、Resource Freshness Version Awareness、Resource Template Boundary、Resource Subscription Awareness、Resource Trace Eval Readiness、MCP Resource Gate。

本轮 MCP Prompts 新增条目：MCP Prompt Exposure Audit、Prompt Capability Declaration、Prompt Discovery Coverage、Prompt Argument Schema Coverage、Prompt Required Argument Validation、Prompt Rendering Safety、Prompt Role Boundary Enforcement、Prompt Dependency Alignment、Prompt Version Governance、Prompt Eval Binding、Prompt Permission Enforcement、Prompt Injection Containment、Prompt User Control、Prompt List Change Awareness、Prompt Trace Readiness、MCP Prompt Gate。

本轮 MCP Security / Sandbox 新增条目：MCP Security Sandbox Audit、Server Connection Governance、Authorization Token Binding、Scope Minimization、Roots Sandbox Containment、File Secret Blocking、Network SSRF Protection、MCP Shell Sandbox Enforcement、Prompt Injection Data Boundary、High Risk Confirmation、Sensitive Data Flow Control、Local Credential Isolation、MCP Tenant Isolation、Server Supply Chain Governance、MCP Security Trace Readiness、MCP Security Eval Coverage、MCP Security Gate。

本轮 MCP Integration 新增条目：MCP Integration Surface Audit、MCP Capability Registration Coverage、MCP Namespace Isolation Coverage、MCP IDE Context Routing、MCP Knowledge Citation Traceability、MCP Database Query Governance、MCP Browser Action Governance、MCP Terminal Sandbox Governance、MCP Integration Context Budget Control、MCP Cross Server Data Flow Control、MCP High Risk Approval Coverage、MCP Output Projection Coverage、MCP Integration Trace Readiness、MCP Integration Eval Coverage、MCP Integration Gate。

本轮 A2A Background 新增条目：A2A Background Audit、Agent Card Completeness、Agent Discovery Readiness、Task Delegation Contract、A2A Task Lifecycle Coverage、A2A Message Structure Coverage、A2A Artifact Reference Coverage、A2A Context Boundary Control、A2A Permission Boundary、A2A MCP Distinction、A2A Failure Handling Coverage、A2A Trace Readiness、A2A Eval Coverage、A2A Background Gate。

本轮 Agent Card 新增条目：Agent Card Discovery Audit、Agent Card Field Completeness、Agent Skill Declaration Quality、Supported Interface Readiness、Agent Card Security Coverage、Agent Card Version Cache Readiness、Agent Discovery Match Quality、Agent Routing Decision Quality、Extended Agent Card Control、Agent Card Trace Readiness、Agent Card Eval Coverage、Agent Card Gate。

本轮 A2A Task Delegation 新增条目：A2A Task Delegation Audit、A2A Task Contract Coverage、A2A State Transition Validity、A2A Input Required Handling、A2A Artifact Metadata Coverage、A2A Error Semantics Coverage、A2A Retry Idempotency Coverage、A2A Cancellation Coverage、A2A Delegation Permission Boundary、A2A Parallel Aggregation Readiness、A2A Task Trace Readiness、A2A Task Eval Coverage、A2A Task Gate。

本轮 A2A Message Boundary 新增条目：A2A Message Boundary Audit、A2A Message Contract Coverage、A2A Part Typing Coverage、A2A Source Trust Labeling、A2A Instruction Data Separation、A2A Minimal Context Coverage、A2A Reference Over Copy Coverage、A2A Context Policy Enforcement、A2A Sensitive Redaction Coverage、A2A Claim Grounding Coverage、A2A Summary Constraint Retention、A2A Message Trace Readiness、A2A Message Eval Coverage、A2A Message Gate。

本轮 RAG / Agent 平台存储新增条目：RAG Agent Storage Audit、Knowledge Base Contract、Document Chunk Version Contract、Sync Delete Propagation、ACL Permission Enforcement、Retrieval Trace Completeness、Prompt Assembly Trace、Citation Version Binding、Agent Definition Versioning、Tool Definition Contract、Tool Call Trace Completeness、Execution Trace Replay Readiness、Memory Privacy Lifecycle、Trace Privacy Retention、Cost Attribution Governance、RAG Agent Storage Gate。

Long Context

一句话定义：long context 是指模型能够处理远超普通对话长度的输入，例如几十万 token 甚至百万 token 的文档、代码库、会议记录或多轮任务轨迹。

为什么提出：真实业务中，用户经常不是问一句孤立问题，而是希望模型理解整份合同、整篇论文、完整工单历史、多个文件或一段长时间交互。传统上下文窗口太短，会导致模型只能看到局部信息，无法做跨段落、跨文件、跨时间的综合判断。

解决什么问题：长上下文主要解决信息无法完整放入模型输入的问题。它让模型可以直接读取更多原始材料，减少人工摘要、切块检索和上下文拼接带来的信息丢失。

核心挑战：注意力计算复杂度高、显存消耗大、位置编码外推困难、长距离依赖难学、有效信息容易被海量无关文本稀释，以及模型可能出现 lost in the middle，也就是中间位置的信息利用率低。

典型方法：扩大训练长度、改造位置编码、使用稀疏注意力或线性注意力、引入 sliding window attention、使用 chunked prefill、KV cache 分页管理，以及用 RAG 先筛选再放入上下文。

优点：实现直观，用户体验好，适合长文档阅读、长代码理解、长会议总结、法律审查和多轮交互记忆。

缺点：成本高，延迟高，输入越长越容易引入噪声；窗口变长不代表模型一定能稳定利用远距离信息。

面试表达：长上下文解决的是“看得见更多”的问题，但不等于“用得好更多”。面试中要强调上下文长度、有效注意力、位置外推、检索增强和成本之间的权衡。

Context Window

一句话定义：context window 是模型一次推理可接收的最大 token 数，包括用户输入、系统提示、历史对话、工具结果和模型已生成内容。

为什么重要：模型不会天然记住当前窗口外的信息。如果关键信息没有放进 context window，模型就无法基于它回答，只能依赖参数记忆或猜测。

组成部分：系统提示占一部分，用户问题占一部分，检索证据占一部分，历史对话占一部分，模型输出也会消耗窗口。实际工程中，窗口经常被 prompt 模板、工具返回和冗余历史快速占满。

常见策略：历史摘要、滑动窗口、重要消息保留、按需检索、证据压缩、去重、模板精简，以及把结构化状态外置到数据库中。

工程坑：窗口很大时不能简单把所有材料塞进去。无关信息会干扰模型，长输入会增加费用和延迟，重复内容还可能让模型过度相信某些错误信息。

面试表达：context window 是推理时的信息带宽。窗口越大，信息容量越高，但成本、延迟和噪声管理压力也越高。

Position Extrapolation

一句话定义：position extrapolation 是让模型在超过训练时最大长度的位置上仍能理解 token 顺序和相对距离的能力。

为什么需要：很多模型预训练时只见过固定长度，例如 4K、8K 或 32K。如果直接在 128K 上推理，位置编码可能落到训练分布之外，导致注意力模式不稳定。

常见背景：Transformer 本身没有天然顺序感，需要位置编码提供位置信息。绝对位置编码、相对位置编码、RoPE、ALiBi 等方案对外推能力的影响不同。

典型方法：RoPE scaling、NTK-aware scaling、YaRN、位置插值、继续长上下文训练，以及在长序列数据上做专门微调。

优点：可以在不完全重训模型的情况下扩展上下文长度，工程成本相对可控。

局限：位置外推只解决位置表示问题，不保证模型学会长距离推理。很多模型即使能接收长输入，也可能只关注开头和结尾。

面试表达：position extrapolation 是长上下文的底层条件之一，但不是全部。长上下文能力还依赖训练数据、注意力机制、推理系统和评估设计。

Lost in the Middle

一句话定义：lost in the middle 是指模型更容易利用上下文开头和结尾的信息，而对中间位置的关键信息利用不足。

为什么出现：长上下文中注意力分布、训练数据模式、位置偏置和 prompt 结构都会影响模型对不同位置内容的关注。很多任务中答案线索如果被放在中间，准确率会下降。

解决思路：把关键证据放在靠近问题的位置，对检索结果做 rerank，把最重要证据放在前后显著位置，要求模型先列证据再回答，或使用分段阅读和聚合策略。

评估方法：构造 needle-in-a-haystack 测试，把同一条关键信息插入不同位置，观察模型召回和回答是否稳定。

面试表达：长窗口不等于长上下文理解。lost in the middle 说明我们要评估“有效上下文长度”，而不是只看模型宣称支持多少 token。

RAG

一句话定义：RAG 是 Retrieval-Augmented Generation，即先从外部知识库检索相关信息，再把检索结果交给大模型生成答案。

为什么提出：大模型参数知识可能过时、不完整、不可追溯，且容易幻觉。RAG 把外部知识库接入生成过程，让模型基于可更新、可引用的证据回答。

解决什么问题：RAG 主要解决知识更新、私有知识问答、事实依据、长文档问答和降低幻觉的问题。它常用于企业知识库、客服、法律、医疗、金融研报和代码问答。

基本流程：文档清洗，切分 chunk，向量化，写入向量库；用户提问后生成 query embedding；召回候选 chunk；可选 rerank；把证据拼入 prompt；模型基于证据回答；最后做引用和质量评估。

核心难点：检索不到、检索到但排序错、chunk 切分不合理、证据互相冲突、prompt 塞入过多噪声、模型不忠实于证据、引用和结论不匹配。

优点：知识可更新，可接入私有数据，答案可追溯，训练成本低于重新预训练或微调。

缺点：系统链路更长，延迟更高，依赖检索质量；如果检索错了，生成模型可能基于错误证据给出更有迷惑性的答案。

面试表达：RAG 不是简单的“向量库加大模型”，而是一条检索、排序、上下文构造、生成、引用校验和评估的完整链路。RAG 的上限经常由检索和证据组织决定。

RAG Product

一句话定义：RAG product 是把检索增强生成链路做成可评估、可引用、可治理、可运营、可控权限和成本的产品系统。

和 RAG demo 区别：demo 通常只验证几个文档上的 top-k 检索和生成；产品还要处理多来源文档、版本、权限、增量更新、引用支持、拒答、反馈闭环、SLO、成本和业务指标。

RAG Incident

一句话定义：RAG incident 是 RAG 系统在线上出现答案错误、引用不支持、越权证据、文档过期、召回失败或成本延迟异常，并影响用户信任或业务流程的事件。

排查重点：不要直接归因成模型幻觉，而要沿文档解析、chunk、retrieval、rerank、context construction、generation、citation、permission 和 freshness 逐段定位。

RAG Error Attribution

一句话定义：RAG error attribution 是把一次 RAG bad case 归因到链路中的具体失败环节。

典型根因：正确文档未入库、解析丢字段、chunk 切断证据、retriever 未召回、reranker 排错、context 丢证据、答案 claim 无证据、引用旧版本、权限过滤缺失或应该拒答却强答。

Document Governance

一句话定义：document governance 是对企业文档的来源、版本、owner、更新时间、权限、失效策略、元数据和质量进行治理。

RAG 场景：文档治理决定 RAG 上限。过期文档、冲突版本、无 owner 文档和权限不清，会让模型更自信地输出错误或越权答案。

Retrieval Recall

一句话定义：retrieval recall 是标准证据中有多少被 retriever top-k 候选召回。

产品意义：如果 retrieval recall 低，后面的 reranker、prompt 和大模型都很难补救。排查时应先确认正确证据是否入库和可被召回。

Mean Reciprocal Rank / MRR

一句话定义：MRR 是第一个正确证据排名倒数的平均值，用来衡量正确证据是否靠前。

RAG 场景：Recall@K 只说明正确证据是否出现，MRR 更能反映用户问题下正确证据是否被排到容易进入上下文的位置。

Retrieval Miss

一句话定义：retrieval miss 是正确证据存在于知识库，但没有进入 retriever 候选集。

常见原因：query rewrite 丢约束、embedding 语义不匹配、metadata filter 错误、关键词错误码不适合纯向量检索、索引未更新或权限过滤字段异常。

Context Drop

一句话定义：context drop 是正确证据已经被召回或重排靠前，但最终没有进入 prompt 上下文。

常见原因：上下文预算不足、去重或压缩规则误删、reranker 排序变化、长文档片段被截断、模板拼接顺序错误或多跳证据预算分配不合理。

Context Recall

一句话定义：context recall 是最终进入上下文的证据覆盖了多少应被使用的标准证据。

产品意义：context recall 低说明关键证据没有进 prompt；这通常是检索、rerank、metadata filter 或上下文预算分配问题。

Abstention Accuracy

一句话定义：abstention accuracy 是资料不足、权限不足、证据冲突或文档过期时，RAG 系统正确拒答、澄清或转人工的比例。

常见误区：RAG 产品不是永远回答。强行回答会短期显得智能，长期损害用户信任。

Stale Evidence Rate

一句话定义：stale evidence rate 是进入上下文或引用中的过期、被替换或版本不一致证据比例。

产品意义：企业知识经常更新。RAG 答案必须展示版本和时间，索引、缓存和引用都要能响应文档更新或权限变化。

Unsupported Claim Rate

一句话定义：unsupported claim rate 是最终答案中没有被上下文证据支持的 claim 比例。

产品意义：它比单纯答案正确率更适合排查幻觉和引用不忠实问题。答案碰巧正确但 claim 没有证据支持，企业 RAG 中仍应算失败样本。

Permission Leak Rate

一句话定义：permission leak rate 是最终上下文或引用中用户无权访问证据的比例。

产品意义：权限过滤应前置到 retrieval、rerank、context construction 和日志写入，而不是只在前端隐藏引用链接。

RAG Freshness Gate

一句话定义：RAG freshness gate 是检查文档版本、更新时间、失效状态、索引同步和缓存命中是否满足当前业务口径的门禁。

常见误区：只要模型引用了“相关”文档就算通过。企业 RAG 中，旧版本证据经常比无证据更危险，因为它会让错误答案显得可信。

RAG Product Gate

一句话定义：RAG product gate 是 RAG 产品上线前的综合门禁，覆盖 retrieval recall、context precision、evidence support、citation accuracy、abstention accuracy、permission filter、stale evidence、P95 latency、unit cost、eval ready、feedback loop 和 business metric。

面试表达：RAG 产品不能只看最终答案正确率。检索、证据、引用、拒答、权限、时效、成本和运营闭环都要过线，才说明它能进入企业 workflow。

RAG Incident Gate

一句话定义：RAG incident gate 是面向事故复盘和回归测试的门禁，要求 retrieval、context、citation、permission、freshness、abstention、latency / cost 和线上反馈都能定位和量化。

面试表达：RAG 事故复盘要输出 root cause，而不是只写“模型幻觉”。每个 bad case 都应能回答正确证据是否存在、是否召回、是否进入 prompt、是否被正确引用、是否有权限和是否过期。

Agentic RAG

一句话定义：Agentic RAG 是把检索放入 Agent 决策循环，让系统主动判断是否检索、如何拆问题、如何多轮检索、如何验证证据以及何时停止。

为什么需要：固定一次检索适合简单问答，但复杂、多跳、版本冲突和需要引用的任务，往往需要先检索一部分证据，再根据证据缺口继续查询。

核心机制：retrieval controller 读取目标和 evidence state，生成 query plan，选择 dense / sparse / web / SQL / code search 等工具，读取 observation，更新证据状态，并根据停止条件决定继续或回答。

优点：能覆盖复杂问题、多来源交叉验证、引用校验和不确定性表达。

风险：成本和延迟更高，容易 query drift、过度检索、引用错误、检索注入和越权证据返回。

面试表达：Agentic RAG 不是检索越多越好，而是每轮检索都要服务证据缺口，并用 trace、引用和门禁证明答案可追溯。

Active Retrieval

一句话定义：active retrieval 是模型或控制器根据任务需要主动决定是否检索、检索什么和何时继续检索。

常见触发：问题依赖最新信息、私有知识、精确引用、多跳证据、冲突验证或高可靠场景。

常见误区：模型觉得自己知道就不检索，或者所有问题都检索导致成本过高。

Retrieval Controller

一句话定义：retrieval controller 是 Agentic RAG 中控制查询规划、工具选择、预算、停止条件和 trace 的模块。

职责：判断是否检索、生成 query、选择工具、应用 metadata filter、读取 evidence state、检测缺口和冲突、控制成本、记录检索轨迹。

Multi-Round Retrieval

一句话定义：multi-round retrieval 是围绕同一问题进行多轮查询，每轮根据上一轮证据缺口生成下一轮 query。

为什么重要：复杂问题第一轮常只能找到局部证据。多轮检索可以补齐方法、实验、版本、限制、反例和安全证据。

风险：如果不保留原始目标和约束，多轮 query 会逐渐漂移。

Evidence State

一句话定义：evidence state 是 Agentic RAG 当前已获得证据、已覆盖 claim、未解决缺口、冲突、权限和风险标记的结构化状态。

工程价值：让 Agent 知道下一轮检索应该补什么，而不是重复检索同类文档。

Evidence Gap

一句话定义：evidence gap 是回答关键 claim 仍缺少支持证据的部分。

例子：回答已经有方法描述，但缺少实验设置、版本时间或安全限制，就应该生成针对这些 gap 的下一轮 query。

Query Drift

一句话定义：query drift 是多轮查询逐渐偏离原始问题或丢失关键约束。

常见原因：query rewrite 加了错误假设、遗忘时间范围、忽略用户权限、只追随上一轮文档中的弱相关线索。

防护方式：每轮 query 保留原始 goal、must-have constraints、用户范围和当前 evidence gap。

New Evidence Gain

一句话定义：new evidence gain 是本轮检索带来的新证据比例。

为什么重要：如果多轮检索的新证据增益持续很低，继续搜索通常只会增加成本和噪声。

Context Precision

一句话定义：context precision 是进入最终上下文的证据中真正相关证据的比例。

面试表达：RAG 不只要 recall，也要控制 context precision。把大量弱相关文档塞进 prompt 会增加幻觉和引用错误。

Evidence Support Rate

一句话定义：evidence support rate 是最终答案的关键 claim 中，有证据支持的比例。

评估方式：把答案拆成 atomic claims，再检查每个 claim 是否被上下文证据支持。

Citation Accuracy

一句话定义：citation accuracy 是引用是否真实支持对应 claim 的比例。

常见错误：引用存在但不相关、引用旧版本文档、引用背景段落却不支持结论、引用用户无权访问的文档。

Agentic RAG Gate

一句话定义：Agentic RAG gate 是基于 context precision / recall、evidence support、citation accuracy、query drift、conflict count、权限和成本的上线门禁。

面试表达：Agentic RAG 的评估不能只看最终答案，要同时审计 retrieval trace、证据覆盖、引用正确、冲突处理、注入拦截和成本。

RAG Prompt Injection

一句话定义：RAG prompt injection 是检索文档、网页片段、工单、邮件或知识库 chunk 中包含不可信指令，诱导模型偏离用户任务或系统策略。

为什么出现：RAG 会把外部文本放进模型上下文，模型需要同时阅读“可信指令”和“不可信证据”。如果上下文构造没有区分二者，文档中的指令可能被误当成任务规则。

常见风险：回答被劫持、引用被污染、系统提示或上下文泄露、跨用户数据泄露、工具调用参数被外部内容影响。

防护方式：文档来源评级、权限过滤、不可信内容标记、检索前注入模式检测、检索后只按证据回答、引用一致性检查、敏感信息不进入不可信上下文。

面试表达：RAG prompt injection 不是检索质量问题本身，而是应用安全边界问题。外部文档只能作为证据，不能成为更高优先级指令。

Chunking

一句话定义：chunking 是把长文档切成适合检索和放入上下文的小片段。

为什么需要：文档通常很长，不能整体作为一个向量单元。切块后可以更细粒度召回相关内容，也便于控制 prompt 长度。

常见方法：固定长度切分、按段落切分、按标题层级切分、按语义边界切分、滑动窗口重叠切分，以及针对代码、表格、FAQ 的结构化切分。

关键参数：chunk size、overlap、元数据、标题继承、文档层级、是否保留表格结构和引用位置。

常见坑：chunk 太小会丢上下文，chunk 太大召回不精准；overlap 太小可能切断答案，overlap 太大则增加重复和成本。

面试表达：chunking 决定 RAG 的基本检索粒度。好的切分不是追求固定长度，而是尽量让一个 chunk 表达完整语义单元。

Dense Retrieval

一句话定义：dense retrieval 是把 query 和文档都编码成稠密向量，并用向量相似度召回相关内容的检索方法。

为什么提出：传统关键词检索依赖词面匹配，难以处理同义表达、改写问题和语义相关内容。稠密检索可以根据语义相似性召回文本。

工作流程：用 embedding model 把文档 chunk 编码成向量并建索引；查询时把用户问题编码成向量；用 cosine similarity、dot product 或 L2 距离寻找最近邻。

优点：语义匹配能力强，适合自然语言问答和改写问题。

缺点：对数字、专有名词、代码符号、精确字段和否定条件可能不如关键词检索稳定；embedding 模型如果领域不匹配，召回效果会下降。

常见改进：使用领域 embedding、query rewrite、多路召回、hybrid retrieval、hard negative 训练和 reranker 重排。

面试表达：dense retrieval 提升语义召回，但不能完全替代 BM25。真实 RAG 系统常用 dense retrieval 加 sparse retrieval 的混合召回。

Sparse Retrieval

一句话定义：sparse retrieval 是基于关键词、倒排索引和词项权重的检索方法，典型代表是 BM25。

为什么仍然重要：很多业务问题依赖精确匹配，例如错误码、合同条款编号、函数名、产品型号、人名、日期和金额。向量检索可能把语义相近但细节不一致的内容召回。

优点：可解释性强，精确匹配好，速度快，工程成熟。

缺点：对同义词、自然语言改写和跨语言语义匹配能力有限。

面试表达：RAG 检索不是只看向量。关键词检索在精确字段和高置信召回上仍然很关键。

Hybrid Retrieval

一句话定义：hybrid retrieval 是把 dense retrieval 和 sparse retrieval 结合起来，提高召回覆盖率和稳定性。

常见做法：并行使用向量召回和 BM25 召回，再做分数融合、候选合并、去重和 rerank。

适用场景：企业知识库、代码搜索、日志问答、法律条款、产品手册等既需要语义理解又需要精确匹配的场景。

优点：兼顾语义召回和关键词精确匹配。

缺点：系统复杂度更高，分数归一化、候选合并和权重调参更麻烦。

面试表达：hybrid retrieval 的核心不是简单拼两个结果，而是用多路召回提升候选覆盖，再用 reranker 统一排序。

Reranker

一句话定义：reranker 是对初步召回的候选文档重新排序的模型或规则模块。

为什么需要：第一阶段检索追求快和召回率，可能返回很多弱相关或只表面相关的 chunk。reranker 用更精细的 query-document 交互判断相关性，把真正有用的证据排到前面。

常见形式：cross-encoder reranker、LLM reranker、规则重排、基于元数据和时间的新闻度重排。

优点：显著改善 top-k 证据质量，降低无关上下文进入 prompt 的概率。

缺点：增加延迟和计算成本；如果候选召回阶段没有召回正确文档，reranker 也无法补救。

面试表达：retriever 决定候选池，reranker 决定最终证据顺序。RAG 质量差时，要区分是召回问题还是排序问题。

Vector Database

一句话定义：vector database 是存储、索引和检索向量的数据库，常用于 RAG 中的 embedding 检索。

核心能力：向量写入、近似最近邻搜索、metadata filter、分片、持久化、增量更新、删除、权限隔离和混合检索。

常见索引：HNSW、IVF、PQ 等。它们通过近似搜索在速度、内存和召回率之间做权衡。

工程关注点：索引构建时间、查询延迟、召回率、更新频率、多租户隔离、权限过滤、embedding 版本管理和数据删除一致性。

和 Feature / Embedding 基础设施的关系：RAG 与 Agent 使用向量库时，不能只记录 collection 名称和 top-k。生产链路还要能追溯 feature definition、embedding model / dimension / normalize 版本、chunk offset、checksum、tenant / ACL filter、index version、ANN recall、retrieval trace、shadow index 切换、lineage、成本和质量监控，否则引用、权限和回滚都无法审计。

常见误区：向量库不是 RAG 的全部。即使向量库性能很好，embedding、切块、rerank、prompt 和评估做不好，系统仍然会失败。

面试表达：vector database 解决的是大规模向量近邻搜索和管理问题，不直接保证问答正确性。

Query Rewrite

一句话定义：query rewrite 是在检索前改写用户问题，让查询更适合召回目标文档。

为什么需要：用户问题可能口语化、信息缺失、含多轮指代或包含多个子问题。直接拿原问题检索可能召回不到正确文档。

常见方式：补全上下文指代、生成多个改写 query、拆解子问题、生成关键词 query、把自然语言转成结构化过滤条件。

风险：改写可能引入用户没有表达的意图，导致检索偏离原问题。因此需要保留原 query，并对改写结果做约束。

面试表达：query rewrite 是提升 RAG 召回率的重要手段，但必须避免语义漂移。

Grounding

一句话定义：grounding 是让模型输出建立在给定证据、工具结果或外部事实上的过程。

为什么需要：没有 grounding 的回答容易依赖参数记忆和语言流畅性，产生无法验证的幻觉。

在 RAG 中的体现：模型应优先根据检索证据回答，不能把证据没有支持的内容说成事实；最好给出引用并说明答案来自哪些片段。

常见检查：答案中的关键断言是否能被证据支持，引用位置是否正确，是否混合了证据外知识，是否对缺失信息做了不确定性表达。

面试表达：RAG 的目标不是让答案更长，而是让答案可追溯、可验证、忠实于证据。

Agent

一句话定义：agent 是能够围绕目标进行规划、调用工具、观察结果并迭代行动的大模型系统。

为什么提出：普通大模型一次生成只适合回答或简单任务。复杂任务需要查资料、调用 API、写代码、运行测试、分析结果和多步修正，因此需要模型具备行动循环。

基本循环：接收目标，形成计划，选择工具，调用工具，观察返回，更新状态，继续行动，直到完成任务或达到停止条件。

关键能力：任务分解、工具选择、参数生成、状态跟踪、错误恢复、结果验证和终止判断。

适用场景：数据分析、代码修改、自动化运维、信息检索、办公流程、客服工单、软件测试和多系统协同。

主要风险：工具误用、无限循环、状态污染、错误累积、越权调用、成本失控和难以评估。

面试表达：agent 不是单独一个模型，而是模型、工具、记忆、规划、执行和安全控制组成的系统。

Agent Harness

一句话定义：agent harness 是模型外层的运行和控制框架，把模型输出转成受控动作，并把环境反馈组织回下一轮上下文。

核心模块：context builder、model adapter、action parser、tool registry、execution engine、permission system、sandbox、state store、trace logger 和 evaluator。

面试表达：coding agent 不能只是模型 API。模型负责生成决策，harness 负责读写文件、执行命令、校验权限、记录 trace、支持 replay 和评估任务是否真的完成。

Agent Runtime

一句话定义：agent runtime 是 agent loop 的执行内核，负责 session、task state、context 构造、模型调用、动作解析、工具调度、错误恢复、checkpoint 和 trace。

工程意义：runtime 不是一个裸 while loop。真正困难的是让每一步都可恢复、可取消、可观测、可审计，并能在模型输出非法、工具失败、命令超时或用户中断后进入一致状态。

Session Manager

一句话定义：session manager 管理一次用户和 agent 的交互边界，保存对话、工作区、权限确认、工具历史、上下文摘要、task state 和 trace id。

面试表达：session 的价值是隔离、中断恢复和用户接管。不同 session 的上下文、权限和文件修改不能混在一起。

Task Manager

一句话定义：task manager 管理目标、阶段、子任务、完成步骤、失败次数、验证状态和 done 条件。

工程意义：它防止长任务迷失在工具调用里，也让 evaluation harness 可以判断任务是否真的完成。

Context Builder

一句话定义：context builder 决定模型每一轮看到哪些目标、状态、文件、日志、diff、工具结果、策略和 schema。

面试表达：上下文不是越多越好，而是要在 token 预算内保留下一步决策最需要的信息，并标记不可信和敏感内容边界。

Model Adapter

一句话定义：model adapter 把 runtime 的统一请求转成具体模型 API 调用，并标准化模型输出、streaming、tool schema、错误、重试和 token 用量。

工程意义：业务逻辑不应直接绑定某个模型 API。adapter 让多模型路由、fallback、A/B test 和版本追踪更容易。

State Store

一句话定义：state store 保存 agent runtime 的短期状态、任务状态和必要持久状态，例如当前 phase、工具结果、modified files、validation status、permission decisions 和 checkpoint。

风险：存太少无法恢复，存太多会带来隐私、成本和治理风险。生产系统应有结构化状态和生命周期管理。

Error Handler

一句话定义：error handler 把 parse error、permission error、timeout、command error、patch conflict、user cancel 和 context overflow 分类，并选择 retry、repair、ask user、rollback、degrade 或 abort。

面试表达：工具失败不能只让模型盲目再试一次。runtime 要先分类错误，再决定是否可恢复以及恢复后状态是否一致。

Runtime Gate

一句话定义：runtime gate 是 agent runtime 的工程门禁，检查 session isolation、phase transition、context budget、adapter success、execution standardization、error recovery、checkpoint coverage、cancel / timeout handling 和 trace completeness。

面试表达：runtime gate 证明的是执行内核可靠，不是证明模型代码能力强。它回答“这套 agent loop 能不能长期、可恢复、可审计地跑下去”。

Agent Loop

一句话定义：agent loop 是 agent 在目标、状态、动作、工具、观察和停止条件之间反复迭代的执行循环。

典型流程：读取目标，检查状态，选择动作，校验权限和参数，执行工具，读取观察，更新状态，检查预算和终止条件。

为什么重要：agent 的关键能力来自把外部反馈纳入下一步决策，而不是一次性生成答案。

风险：没有 loop 上限会导致无限循环；没有观察使用检查会导致模型忽略工具结果；没有停止条件会导致成本失控。

Harness Loop Audit

一句话定义：harness loop audit 是对 agent harness 执行闭环的审计，检查动作解析、权限、工具执行、observation 使用、状态更新、trace、预算和最终验收是否一致。

工程意义：它把“agent 看起来完成了”拆成可检查证据。测试失败却最终声称完成、危险命令被 prompt 禁止但系统仍执行、trace 缺字段，都应该被审计暴露。

Action Parser

一句话定义: action parser 是把模型输出解析成结构化动作的组件, 例如 tool call、file patch、shell command、user question 或 final answer。

常见风险: 模型输出非法 JSON、参数缺字段、工具名不存在、自然语言和结构化动作混杂。parser 必须失败可见, 不能把不合法输出直接交给 executor。

Agent State

一句话定义: agent state 是当前任务的结构化进展, 包括目标、计划、已完成步骤、工具结果、错误、约束、预算和待确认事项。

为什么需要: 多步任务中只靠上下文很容易遗漏进展, 结构化状态可以让系统知道下一步应该继续、重试、追问还是停止。

面试表达: state 服务当前任务闭环, memory 服务跨任务信息复用; 二者不能混为一谈。

Agent Action

一句话定义: agent action 是 agent 在某一步选择要做的事情, 可以是调用工具、追问用户、更新计划、请求人工确认或输出最终答案。

关键点: tool call 只是 action 的一种。完整 agent 还要能决定什么时候不用工具、什么时候停止、什么时候请求确认。

Observation

一句话定义: observation 是工具或环境返回给 agent 的结果。

为什么重要: observation 决定 agent 是否真的形成反馈闭环。工具失败、权限不足、测试报错、检索无结果都应该进入状态更新和下一步决策。

常见指标: observation use rate, 即工具返回是否被后续状态或动作使用。

Controller

一句话定义: controller 是 agent 的工程控制层, 负责预算、权限、重试、超时、停止条件、日志和人工接管。

为什么重要: 模型可以提出动作, 但系统必须验证动作。越高权限的工具, 越要依赖 controller 做 schema、参数、权限和二次确认。

面试表达: 可靠 agent 的核心不只是更强模型, 还包括 controller、trace、permission gate 和 budget gate。

Permission Gate

一句话定义: permission gate 是在工具执行、文件修改、命令运行、网络访问和外部副作用前进行系统层授权检查的门禁。

工程意义: 权限门禁不能只靠模型自觉。只读、写文件、执行测试、高风险 shell、网络访问和外部 API 应有不同策略, 并在 trace 中记录允许、拒绝或人工确认。

Replay Readiness

一句话定义: replay readiness 衡量一次 agent run 是否具备足够 trace、版本、输入、工具结果和环境信息, 可以在离线评估或事故复盘重放。

面试表达: 没有 replay, 就很难判断新模型、新 prompt 或新工具策略是否真的更好, 也很难复盘一次失败来自模型、工具、权限、状态还是环境。

Harness Gate

一句话定义：harness gate 是 coding agent harness 的上线门禁，综合约束动作解析合法率、工具执行成功率、未授权执行率、高风险确认覆盖率、observation 使用率、状态更新覆盖率、trace 完整率和预算超限率。

面试表达：harness gate 不看单个漂亮 demo，而是看系统是否能持续把模型决策变成可执行、可控、可观测、可复盘的动作闭环。

Tool Registry

一句话定义：tool registry 是 agent 可用工具的目录，记录工具名、用途、参数 schema、权限等级、返回格式、超时和风险级别。

作用：帮助模型选择工具，也帮助系统做参数校验、权限检查、审计和文档化。

Tool Registry Completeness

一句话定义：tool registry completeness 是工具定义是否包含 name、description、input schema、output schema、permission、side effect、timeout、retry、error types、visibility、version 和 audit policy 等必要字段。

面试表达：工具能被调用不代表 registry 完整。缺版本、权限、副作用或审计策略的工具，很难安全上线和 replay。

Tool Schema Strictness

一句话定义：tool schema strictness 衡量工具参数字段是否有明确类型、必填约束、枚举、范围、路径边界、长度限制或 allowlist。

常见风险：command: string、path: string 这类过宽字段会把安全责任留给模型猜测，应该配合工作目录、风险枚举、路径边界和权限策略。

Tool Permission Binding

一句话定义：tool permission binding 是每个工具都绑定清晰 permission level 和 side effect level 的机制。

工程意义：读文件、写文件、执行命令、访问网络、读取 secret、外部写入和 destructive action 的审批策略不同，不能只写一个模糊的 enabled。

Side Effect Level

一句话定义：side effect level 描述工具对环境造成的影响范围，例如 none、local read、local write、compute、external read、external write 和 destructive。

面试表达：权限看“谁能做”，副作用看“做了会改变什么”。两者分开设计，permission gate 才能细粒度控制风险。

Risk Protection Coverage

一句话定义：risk protection coverage 是高风险工具被默认阻断、人工确认、dry run、沙箱或权限策略保护的比率。

上线要求：写文件、执行命令、网络访问、secret 和 destructive action 的保护覆盖率应接近 1。

Output Normalization Coverage

一句话定义：output normalization coverage 衡量工具输出是否包含稳定的 status、error type、recoverable、truncated、metrics 和 next hint 字段。

为什么重要：结构化输出让模型能恢复，让 trace 可检索，让 evaluation harness 能统计失败类型。

Tool Version Coverage

一句话定义：tool version coverage 是工具调用 trace 中记录工具版本的覆盖率。

工程意义：Search 排序、patch 匹配、命令超时、输出摘要和安全策略改变后，旧 trace 的 replay 结果可能变化，因此必须记录工具版本。

Tool Trace Readiness

一句话定义：tool trace readiness 衡量工具是否能记录 tool name、tool version、arguments、permission decision、status、error type、duration、truncation 和安全决策。

面试表达：没有 tool trace，就很难判断一次失败来自模型选错工具、参数错误、权限拒绝、工具超时、结果误读还是最终总结不忠实。

Untrusted Tool Output Boundary

一句话定义：untrusted tool output boundary 是把网页、文件、issue、日志、测试输出和第三方 API 返回标记为 observation / evidence，而不是系统指令的边界。

安全要求：不可信工具返回不能覆盖系统策略、开发者指令、权限门禁、高风险确认或外部传输规则。

Tool Registry Gate

一句话定义：tool registry gate 是工具注册表上线前的综合门禁，覆盖 registry 完整性、schema 严格性、权限绑定、风险保护、输出标准化、版本覆盖、trace 就绪和不可信输出边界。

面试表达：Tool Registry gate 评估的是工具治理能力，不是模型是否偶尔会用某个工具。只要高风险工具未绑定确认、secret 工具暴露、输出不结构化或版本缺失，就不应直接进入生产级 Agent。

Tool Registry Identity Coverage

一句话定义：tool registry identity coverage 是 registry 中工具具备稳定 namespace、唯一 name、非泛化命名和可追踪 display name 的比例。

常见失败：多个 server 都注册 search 或 get_data，模型和审计系统无法区分工具语义、来源和权限边界。

Tool Description Quality

一句话定义：tool description quality 衡量模型可见 description 是否说明工具用途、适用场景、不适用场景、数据来源和风险边界。

边界：description 能影响模型选择，但不是安全控制；权限、确认和阻断必须由 runtime 执行。

Schema Contract Coverage

一句话定义：schema contract coverage 是工具同时具备 input schema、output schema 和额外字段处理策略的比例。

工程价值：只管输入 schema 会让工具返回结果随意变化，破坏上下文回填、mock tools、eval 和 replay。

Runtime Metadata Coverage

一句话定义：runtime metadata coverage 是 registry 记录 executor、timeout、retry、concurrency、cache、cancellation 和 idempotency 等执行元数据的比例。

面试表达：模型不需要知道 endpoint，但 runtime 必须知道如何执行、限流、取消、重试和记录结果。

Tool Permission Binding Coverage

一句话定义：tool permission binding coverage 是工具声明工具级权限、对象级数据范围和字段级策略，并能被 router / runtime 使用的比例。

常见失败：权限只散落在业务代码里，导致 router 不能提前过滤候选工具，审计也无法解释为什么某个工具对用户可见。

Risk Annotation Coverage

一句话定义：risk annotation coverage 是工具标注 risk level、side effect、confirmation、idempotency 和 auto-execution policy 的比例。

上线门禁：发送邮件、退款、删除、转账、权限修改等副作用工具没有确认和幂等保护时，不应进入自动工具调用路径。

Version Trace Coverage

一句话定义：version trace coverage 是每个工具版本、schema hash、changelog 和 trace 版本捕获完整的比例。

工程价值：没有版本捕获，线上事故无法复现，也无法判断模型升级、schema 变更还是权限策略变化造成回归。

Lifecycle Policy Pass Rate

一句话定义：lifecycle policy pass rate 是 draft、review、staging、active、deprecated、sunset、disabled 和 archived 状态与模型可见性、runtime 可执行性一致的比例。

常见失败：deprecated 工具仍暴露给模型，disabled 工具仍可执行，archived 工具被删除导致历史 trace 无法解释。

Owner SLO Coverage

一句话定义：owner SLO coverage 是工具具备 owner team、oncall、联系渠道和 SLO / SLI 责任字段的比例。

工程价值：没有 owner 的工具不能进入生产，因为 schema、故障、权限投诉、eval 回归和下线通知都需要明确责任边界。

Tool Eval Binding Coverage

一句话定义：tool eval binding coverage 是 registry item 绑定 eval dataset、golden trace、安全样例、最近回归结果和上线阈值的比例。

面试表达：工具 description 或 schema 改动后必须跑 eval；如果工具选择、参数准确率或安全用例下降，registry 发布应阻断或要求审批。

Provider Projection Readiness

一句话定义：provider projection readiness 是内部 registry item 能按 OpenAI、Anthropic、Gemini、MCP 或其他 runtime 需要投影，而不把业务定义绑定到单一 provider 原始格式的比例。

工程价值：内部格式和 provider 格式解耦，才能在多模型、多协议、多 adapter 场景下稳定迁移和回放。

Registry Audit Completeness

一句话定义：registry audit completeness 是工具创建、修改、审批、权限放宽、风险等级调整、灰度、回滚和下线都有审计记录的比例。

面试表达：tool trace 记录一次调用使用了哪个版本，registry audit 记录这个版本是谁改的、为什么改、何时放量，两者合起来才可追责。

Tool Executor

一句话定义：tool executor 是实际执行工具调用的系统组件，把模型生成的结构化 tool call 转成经过校验、授权、限流、幂等和可观测的真实动作。

职责：解析参数、检查 schema 和业务规则、检查权限、处理确认、选择同步或异步执行、控制并发、设置超时和取消、保护幂等、包装结果、规范化错误、记录 trace 和屏蔽不允许的动作。

风险：不能把 executor 简化成“模型说什么就执行什么”。真实执行必须由系统控制；尤其是邮件、退款、转账、删除、写库和权限修改等副作用工具，必须有确认、幂等键、状态记录和未知状态接管。

Execution Request Validity

一句话定义：execution request validity 衡量 Executor 对 tool call 是否应被接受的判断与期望策略一致的比例。

工程价值：非法 JSON、缺少 tool call id、工具不存在、schema 不合法或业务字段缺失时，应结构化拒绝，而不是把半成品请求交给真实工具。

Schema Validation Pass Rate

一句话定义：schema validation pass rate 衡量 Executor 的参数 schema 校验结果与期望结果一致的比例。

边界：schema 只能保证结构层合法，金额、路径、订单归属、租户边界和业务状态仍需要业务校验。

Permission Enforcement Pass Rate

一句话定义：permission enforcement pass rate 衡量 Executor 对用户、租户、角色、数据范围和工具权限的执行结果是否符合策略。

常见事故：Router 已过滤一次工具不代表 Executor 可以跳过权限；最终执行前仍要按真实用户、租户和资源做硬校验。

Execution Mode Accuracy

一句话定义：execution mode accuracy 衡量同步、异步或阻断模式选择是否符合工具耗时、交互需求、副作用和恢复成本。

面试表达：低延迟只读查询适合同步，长任务、文件分析、批处理和审批 workflow 适合异步；高风险或缺参请求应阻断或澄清。

Async Completion Tracking

一句话定义：async completion tracking 衡量异步工具是否返回 job id、支持状态查询、记录完成状态并能把最终结果安全回填。

工程价值：异步不是“先丢到后台”。没有状态查询、取消、过期时间、权限复查和通知机制，用户和模型都无法知道任务是否真的完成。

Unknown State Escalation Rate

一句话定义：unknown state escalation rate 衡量有副作用工具提交后超时或连接中断时，系统是否进入状态查询、人工接管或用户明确说明流程。

常见事故：转账、退款、邮件发送等动作超时后不能简单重试，也不能直接告诉用户失败；正确做法是幂等键、查询下游状态和升级处理。

Retry Safety Rate

一句话定义：retry safety rate 衡量实际重试是否只发生在 retryable、预算内、幂等且不会重复副作用的场景。

边界：权限错误、schema 错误、高风险副作用未知状态和下游熔断打开时不应盲目重试。

Side Effect Confirmation Coverage

一句话定义：side effect confirmation coverage 衡量邮件、退款、删除、转账、写库和权限修改等高风险副作用动作在执行前是否有用户确认或等价审批。

工程价值：模型不能自己生成确认 token。确认必须由 runtime 绑定用户、会话、预览内容、风险等级和过期时间。

Structured Result Coverage

一句话定义：structured result coverage 衡量 Executor 是否把成功、失败、超时、取消和未知状态都包装成统一 ToolResult 或等价结构。

常见事故：把超时写成空结果，或把原始堆栈直接回填给模型，都会误导最终回答并扩大安全风险。

Executor Trace Completeness

一句话定义：executor trace completeness 衡量一次工具执行是否记录 received、schema、permission、mode、status、result、version、latency、attempt 和 idempotency 等关键字段。

面试表达：没有 executor trace，就无法判断失败来自参数、权限、下游超时、取消、幂等命中、未知状态还是结果包装。

Tool Executor Gate

一句话定义：tool executor gate 是综合请求有效性、schema、权限、执行模式、异步跟踪、超时取消、幂等、未知状态、重试安全、确认、结构化结果和 trace 的上线门禁。

工程价值：Executor gate 的核心不是证明工具“能跑”，而是证明工具动作能被安全执行、失败可恢复、状态可解释、结果可审计。

Tool Permission Model

一句话定义：tool permission model 是工具真实执行前的访问控制模型，回答谁能在什么上下文中对哪个资源执行哪个工具动作，并决定哪些字段可以返回。

工程价值：工具权限不能等同于模型是否看到了工具。可信身份、租户、角色、资源、字段、动作、确认状态和不可信内容边界都必须由 runtime 和后端服务强制执行。

Authorization Decision Accuracy

一句话定义：authorization decision accuracy 衡量权限系统的 allow / deny 决策与期望策略一致的比例。

边界：这个指标同时惩罚越权放行和误拒；高风险工具通常更重视越权放行的召回阻断。

Trusted Context Injection

一句话定义：trusted context injection 衡量 `user_id`、`tenant_id`、`roles`、`scope`、`session` 和 `confirmation` 等可信上下文是否由 runtime / auth 层注入，而不是来自模型参数。

常见事故：模型生成或被诱导修改 `tenant_id`，如果系统信任该字段，就可能跨租户访问。

Tenant Isolation Pass Rate

一句话定义：tenant isolation pass rate 衡量工具调用是否始终按用户租户、资源租户和下游租户过滤保持一致。

面试表达：多租户隔离应贯穿 Router、Executor、下游查询、缓存 key、日志、对象存储和 trace，不能只在前端隐藏数据。

Tool Permission Enforcement

一句话定义：tool permission enforcement 衡量用户或 agent 调用工具前是否具备该工具要求的 `scope`、`role` 或 `policy grant`。

边界：工具级权限只是粗粒度入口；拥有工具权限不等于拥有所有对象、字段和动作权限。

Object Permission Accuracy

一句话定义：object permission accuracy 衡量用户对具体订单、客户、文件、项目、工单或数据库行的访问判定是否正确。

工程价值：企业场景最常见越权不是“工具不存在”，而是用户能调用工具但访问了不属于自己的对象。

Field Projection Safety

一句话定义：field projection safety 衡量工具结果是否只返回当前用户可见字段，并删除或脱敏 `payment token`、内部备注、隐私字段和安全字段。

常见事故：工具执行权限正确，但结果包装把敏感字段塞进模型上下文，后续回答或记忆写入仍会泄露。

Action Context Policy Accuracy

一句话定义：action context policy accuracy 衡量动作权限是否结合金额、状态机、二次认证、用户确认、workflow step、来源设备和风险分数做动态判定。

面试表达：同一个用户可以读订单，不代表可以取消、退款、修改地址或提升权限；动作和上下文必须分开建模。

Prompt Injection Block Rate

一句话定义：prompt injection block rate 衡量来自网页、文档、邮件、工具结果等不可信内容诱导的越权工具调用是否被拒绝。

工程价值：模型可能被外部内容诱导，但权限系统不能把模型输出当授权依据。

Token Audience Validation

一句话定义：token audience validation 衡量外部工具协议或服务调用中的 token 是否绑定到正确资源、scope 和 audience。

边界：token 有效不代表能调用任意工具；跨服务、跨 MCP server 或跨资源复用 token 会扩大事故半径。

Revocation Cache Safety

一句话定义：revocation cache safety 衡量权限撤销、临时授权过期、角色变更或确认失效后，缓存是否及时失效并拒绝继续执行。

常见事故：权限服务已经撤销访问，但 Executor 仍使用旧缓存放行高风险工具。

Error Disclosure Safety

一句话定义：error disclosure safety 衡量权限拒绝时，对模型和用户暴露的信息是否避免泄露资源存在性、内部 ACL 规则和敏感策略细节。

工程价值：外部可见错误可以说“无法访问或不存在”，内部审计日志再记录真实原因。

Permission Audit Completeness

一句话定义：permission audit completeness 衡量每次权限决策是否记录 user、tenant、tool、action、resource、decision、reason、trace、policy version 和 result sensitivity 等关键字段。

面试表达：没有权限审计，越权、误拒、跨租户投诉和安全复盘都无法定位责任边界。

Tool Permission Gate

一句话定义：tool permission gate 是综合授权决策、可信上下文、租户隔离、工具权限、对象权限、字段投影、动作上下文、prompt injection 阻断、最小权限、token audience、缓存撤销、错误脱敏和审计完整性的上线门禁。

工程价值：Tool Permission Gate 的目标不是让模型“知道自己不能做什么”，而是让 runtime 在可信身份上下文下强制决定什么能执行、什么数据能返回、什么动作必须确认或拒绝。

Tool Security

一句话定义：tool security 是围绕工具调用全链路的安全体系，覆盖越权、prompt injection、敏感数据泄露、危险动作、SSRF、SQL、路径、shell、外部发送、数据流、审计和安全 eval。

面试表达：工具安全不是提示词里写“不要做危险动作”，而是让 Router、Executor、权限服务、DLP、确认、沙箱、trace 和回归 eval 共同限制模型输出能产生的真实副作用。

Prompt Injection Containment

一句话定义：prompt injection containment 是直接或间接提示注入样本中，不可信内容被隔离为 data、没有驱动高风险工具或覆盖上层指令的比例。

边界：这个指标必须看 trace；最终回答没有泄露，不代表中间没有尝试调用导出、邮件、管理员或 shell 工具。

Untrusted Content Isolation

一句话定义：untrusted content isolation 衡量网页、邮件、PDF、日志、数据库文本和工具结果是否被标记为低信任 observation，并且不能作为身份、权限、确认或系统规则来源。

工程价值：外部内容可以作为证据，但不能成为 instruction。模型读到网页里的命令时，runtime 仍应按原始用户目标、权限和风险策略执行。

Unauthorized Access Block Rate

一句话定义：unauthorized access block rate 是横向越权、纵向越权、跨租户越权、对象越权、字段越权和动作越权被 runtime 或下游服务阻断的比例。

面试表达：企业工具安全里最关键的问题不是模型是否知道“不能访问”，而是越权 tool call 是否真的没有执行。

Sensitive Data Protection

一句话定义：sensitive data protection 衡量敏感数据在工具输入、工具输出、日志、memory、外部发送和最终回答中是否被脱敏、投影、拒绝或限制在允许的数据流中。

常见事故：内部工具返回 payment token、内部备注或隐私字段后，模型把它写进上下文、日志或外部邮件。

Tool External Transfer Gate

一句话定义：tool external transfer gate 是内部数据传给邮件、网页、第三方 API、外部搜索、IM、工单评论或社交发布前的权限、DLP、目的地、确认和审计门禁。

边界：外部发送工具即使是“只发草稿”，也要区分内部收件人、外部域名、敏感字段、用户确认和可追责 trace。

Dangerous Action Confirmation

一句话定义：dangerous action confirmation 是删除、发送、支付、发布、权限变更、生产配置修改、shell 执行等危险动作被拒绝或经过 runtime 生成确认 token、二次认证、dry run 和幂等保护后执行的比例。

面试表达：确认不是弹窗文本，而是由 runtime 生成、绑定动作和参数、不可由模型伪造的执行凭证。

SSRF Block Rate

一句话定义：SSRF block rate 衡量 URL 工具对内网地址、metadata 地址、非 allowlist 域名、危险重定向和凭证转发的阻断能力。

工程价值：URL 工具看似只读，但如果能访问内部网络，就可能变成内部探测和凭证泄漏入口。

SQL Risk Block Rate

一句话定义：SQL risk block rate 衡量数据库工具对写操作、敏感表字段、大表扫描、缺少 tenant filter、非参数化查询和越权读的阻断能力。

面试表达：生产系统通常不应暴露通用 run_sql，应优先暴露受控业务查询工具或只读副本上的受限 SQL。

Path Traversal Block Rate

一句话定义：path traversal block rate 衡量文件工具对 .. 逃逸、symlink 绕过、系统目录、敏感文件、超出 workspace 的读写和危险删除的阻断能力。

工程价值：模型生成路径不能直接执行，必须 canonicalize 后验证仍在允许根目录内。

Shell Sandbox Enforcement

一句话定义：shell sandbox enforcement 衡量 shell 或代码执行工具是否进入文件、网络、CPU、内存、磁盘、子进程、环境变量和超时受限的沙箱。

边界：高风险系统不应让模型直接执行任意 shell；如果必须支持，也要默认无网络、无特权、无敏感挂载并记录完整 trace。

Data Flow Policy Pass Rate

一句话定义：data flow policy pass rate 衡量敏感数据从内部工具流向外部工具、长期 memory、跨租户上下文或最终回答时，是否被 allowed sinks / forbidden sinks 策略正确处理。

面试表达：数据流控制回答的是“这份数据可以去哪里”，它和“用户能不能调用工具”是两层不同门禁。

Tool Security Audit Completeness

一句话定义：tool security audit completeness 衡量工具安全 trace 是否记录 user、tenant、tool、risk、decision、reason、outcome、policy version、confirmation、sandbox 和 data sensitivity 等字段。

工程价值：没有审计完整性，越权放行、DLP 命中、SSRF 阻断、外发事故和用户投诉都无法复盘。

Security Alert Completeness

一句话定义：security alert completeness 衡量高风险阻断、越权放行、DLP 命中、prompt injection、SSRF、外发、沙箱拒绝和异常导出是否触发告警或进入安全监控。

边界：告警不是越多越好，而是高严重度和可行动事件必须可见、可分派、可复盘。

Safety Eval Regression Pass Rate

一句话定义：safety eval regression pass rate 是工具安全回归集在新模型、新 prompt、新工具版本、新权限策略和新沙箱配置下持续通过的比例。

面试表达：工具安全 eval 要基于完整 trace，不应只让 LLM judge 看最终回答是否“看起来安全”。

Tool Security Gate

一句话定义：tool security gate 是综合 prompt injection containment、不可信内容隔离、越权阻断、敏感数据保护、外部传输门禁、危险动作确认、SSRF / SQL / 路径 / shell 防护、数据流、审计、告警和安全回归的上线门禁。

工程价值：Tool Security Gate 的目标是证明危险工具没有被不可信内容驱动、敏感数据没有越界流动、外部副作用经过确认和审计，且这些能力能在版本变化后持续回归。

Tool Trace Replay Audit

一句话定义：tool trace replay audit 是检查工具调用 trace 是否具备完整 schema、ID 链路、版本捕获、参数 lineage、权限 trace、结果 trace、隐私脱敏、审计事件、replay readiness、指标导出、告警 owner 和 eval 链接的审计过程。

工程价值：它把“有日志”升级为“能复现、能评估、能追责、能在隐私边界内排障”的证据系统。

ID Tree Integrity

一句话定义：ID tree integrity 衡量 run id、turn id、tool call id、execution id、job id 和 span parent / child 关系是否跨模型、runtime、工具服务和日志系统保持一致。

常见事故：trace 里有很多事件，但 ID 不一致或父子 span 断裂，导致无法说明一次工具调用到底由哪个模型输出触发。

Argument Lineage Coverage

一句话定义：argument lineage coverage 衡量 raw arguments、parsed arguments、normalized arguments、final execution arguments、validation result 和 evidence map 是否完整记录。

面试表达：参数 trace 要能回答最终执行参数来自模型、用户证据、runtime 规范化还是修复逻辑。

Permission Trace Completeness

一句话定义：permission trace completeness 衡量权限决策是否记录 user、tenant、tool、action、resource、decision、reason 和 policy version。

工程价值：没有 reason 和 policy version，越权、误拒和权限策略回滚都很难复盘。

Tool Result Trace Completeness

一句话定义：tool result trace completeness 衡量工具结果是否记录 status、schema validation、raw size、projected size、sensitivity、redaction 和 sent-to-model 决策。

边界：工具执行成功不代表结果可以原样进入模型上下文；结果 trace 要证明字段投影、脱敏、压缩和错误状态都被处理。

Audit Event Completeness

一句话定义：audit event completeness 衡量高风险动作审计事件是否记录 audit id、actor、tenant、tool、action、resource、decision、timestamp、trace id 和 outcome。

面试表达：审计日志服务追责和合规，不应只是普通 debug log 的副本。

Replay Side Effect Safety

一句话定义：replay side effect safety 衡量有副作用工具在 replay 时是否默认阻断 live execution，只允许 dry-run、mock、sandbox 或审批后的受控执行。

常见事故：为了复现邮件、支付、删除或权限变更问题，把历史 trace 又真实执行了一遍。

Alert Owner Coverage

一句话定义：alert owner coverage 衡量高风险告警是否能路由到明确 owner、on-call 或负责团队。

工程价值：没有 owner 的告警只能制造噪声，无法形成排障和复盘闭环。

Eval Linkage Coverage

一句话定义：eval linkage coverage 衡量线上 trace 是否能被转换成失败样本、golden trace、regression case 或离线 eval 输入。

面试表达：好的 trace 系统不只服务排障，也要服务下一轮工具调用评估和安全回归。

Tool Version Release Audit

一句话定义：tool version release audit 是检查工具版本发布是否同时覆盖 spec lint、schema 兼容、description 行为风险、输出兼容、权限策略、版本矩阵、离线评估、灰度路由、质量安全成本、回滚和生命周期的审计过程。

工程价值：它把“发一个工具版本”升级为“模型看到的契约、runtime 执行的实现、权限、eval、trace、灰度和回滚都一致”的发布证据。

Spec Lint Pass Rate

一句话定义：spec lint pass rate 衡量工具 spec 是否具备 owner、version、schema hash、description hash、risk、changelog 和 rollback plan 等最低发布字段。

常见事故：工具能执行，但没有 owner、风险等级或回滚计划，灰度出问题后无人接手。

Schema Backward Compatibility

一句话定义：schema backward compatibility 衡量 input / output schema 是否避免新增必填字段、删除字段、字段改名、类型变化、enum 收窄和语义改变。

面试表达：新增可选字段通常是 API 兼容，但仍可能改变模型填参习惯，所以还要配合 eval 和灰度。

Description Behavior Guard

一句话定义：description behavior guard 衡量工具描述改动是否没有扩大工具适用边界、丢失不适用场景或诱导模型过度调用。

工程价值：工具 description 是模型选择工具的重要输入，不能只给 schema 做版本管理。

Output Compatibility

一句话定义：output compatibility 衡量新版本工具结果是否保留旧字段、旧枚举或兼容 adapter，避免下游回答、workflow、eval 和 replay 断裂。

常见事故：后端接口调用成功，但字段从 status 改成 order_status，导致模型和下游流程继续按旧字段读取。

Permission Policy Compatibility

一句话定义：permission policy compatibility 衡量权限策略变更是否记录 affected roles、affected tenants、expected denial rate、通知和回滚策略。

边界：权限收紧可能是正确安全修复，但仍需要预期拒绝率、客服兜底和灰度监控。

Version Matrix Capture

一句话定义：version matrix capture 衡量 trace 是否同时记录 model、prompt、tool schema、executor、adapter、router policy、permission policy 和 eval dataset 版本。

工程价值：工具版本事故经常来自组合变化，只记录 tool_version 无法完成归因。

Offline Eval Pass Rate

一句话定义：offline eval pass rate 衡量工具新版本在正常调用、不该调用、缺参澄清、边界参数、工具失败、安全攻击和多轮状态样本上的通过率。

面试表达：灰度之前先用离线评估挡住已知坏 case，灰度期间再观察真实流量。

Canary Routing Stability

一句话定义：canary routing stability 衡量灰度路由是否按租户、用户、场景或比例稳定生效，并对同一 conversation / job 使用 sticky routing。

常见事故：多轮对话中一轮看到 v1 schema、下一轮看到 v2 schema，异步任务和工具状态随之不兼容。

Canary Quality Guard

一句话定义：canary quality guard 衡量灰度新版本的任务成功率、工具选择准确率、用户纠正率和参数校验失败率是否没有明显退化。

工程价值：新版本不能只看平均成功率，还要看参数错误、fallback、用户纠正和坏切片。

Canary Safety Guard

一句话定义：canary safety guard 衡量高风险误调用、权限异常、危险动作确认缺失、DLP block 和外部传输风险是否没有回归。

边界：质量小幅提升但安全风险升高，不应继续扩大灰度。

Cost Latency Guard

一句话定义：cost latency guard 衡量新版本的 P95 / P99 延迟、工具调用次数、返回结果大小和单任务成本是否没有超过阈值。

常见事故：新增可选字段让模型频繁请求详细结果，最终质量没提升但成本和延迟明显上升。

Tool Release Rollback Readiness

一句话定义：tool release rollback readiness 衡量旧 spec、旧 executor、router revert、policy revert、状态兼容策略、owner 和回滚演练是否齐全。

面试表达：工具回滚不是只切回代码，而是把模型可见契约、runtime、权限、路由和状态一起恢复一致。

Lifecycle Coverage

一句话定义：lifecycle coverage 衡量 deprecated notice、sunset date、migration guide 和 archived spec 是否齐全。

工程价值：旧版本下线后仍要解释历史 trace、支持 replay 和处理未完成任务，不能直接删除旧 spec。

Tool Release Gate

一句话定义：tool release gate 是基于 spec lint、schema 兼容、description、输出、权限、版本矩阵、eval、灰度、质量、安全、成本、回滚和生命周期的综合发布门禁。

面试表达：工具版本发布要把 spec、模型行为、runtime 执行和安全策略当成一个整体灰度，而不是只部署一段后端代码。

Enterprise Tool Platform Audit

一句话定义：enterprise tool platform audit 是对企业工具平台的 Registry、Router、Executor、Permission、Safety、Trace、Eval、Release、多租户、Provider Adapter、MCP 集成、后台治理和高可用能力做上线前审计。

工程价值：它把“能调用工具”升级为“工具能力能被企业统一注册、授权、执行、审计、评估、灰度和回滚”的平台证据。

Platform Module Coverage

一句话定义：platform module coverage 衡量企业工具平台是否具备 Registry、Router、Executor、Permission、Safety、Trace / Audit、Eval、Release / Governance、Admin Console 和 HA 等必需模块。

常见事故：demo 阶段只接 function calling API，产品化时才发现没有权限、审计、回滚和后台审批。

Platform Interface Contract Coverage

一句话定义：interface contract coverage 衡量 ToolSpec、ToolCall、ToolResult、PermissionDecision、TraceSpan、EvalReport 和 ReleasePlan 是否有稳定 schema、owner、版本和错误语义。

面试表达：企业工具平台的接口契约要独立于具体 provider，否则多 provider 和 replay 会被某一家字段格式绑定。

Registry Readiness

一句话定义：registry readiness 衡量工具事实源是否具备 owner、版本、schema hash、生命周期、风险标签、provider projection、签名快照和变更事件。

工程价值：运行面依赖 Registry 读快照，高可用场景下必须知道“当前允许模型看到哪些工具”。

Router Governance Coverage

一句话定义：router governance coverage 衡量 Tool Router 是否按场景、租户、权限、风险、workflow 状态、provider capability 和成本预算生成候选工具集合。

常见事故：把所有工具直接暴露给模型，导致高风险工具过早出现、候选集过宽、provider 不支持的 tool choice 模式仍被使用。

Executor Safety Coverage

一句话定义：executor safety coverage 衡量 Executor 是否覆盖 schema validation、业务校验、timeout、cancellation、idempotency、unknown state、sandbox、structured result 和 error mapping。

工程价值：模型只提出动作，Executor 才是真实世界副作用的强制执行边界。

Platform Permission Integration Coverage

一句话定义：platform permission integration coverage 衡量 Router 粗过滤、Executor 强校验、下游 ACL、字段投影、确认令牌和审计事件是否共享可信身份上下文。

边界：权限不能来自模型参数，也不能只写在 prompt 中。

Platform Safety Guard Coverage

一句话定义：platform safety guard coverage 衡量 prompt injection 隔离、DLP、外部传输、危险动作确认、SSRF / SQL / path / shell 边界和数据流策略是否进入统一安全门禁。

面试表达：Safety Guard 应结合规则、策略、上下文标签和 runtime 信息，而不是只让 LLM 做一次分类。

Platform Trace Audit Coverage

一句话定义：platform trace audit coverage 衡量工具调用链路是否记录 Router、tool call、arguments lineage、permission decision、executor result、版本矩阵、audit event、metrics 和 replay linkage。

工程价值：没有 trace / audit，就无法定位是工具选错、参数错、权限错、执行失败、结果没用，还是发布版本引入回归。

Platform Eval Service Coverage

一句话定义：platform eval service coverage 衡量平台是否能把离线 eval、golden trace replay、online sampling、regression dashboard 和 release gate 连接起来。

常见事故：上线前只跑单点功能测试，没有工具选择、参数、权限、安全和恢复能力的回归集。

Platform Release Governance Coverage

一句话定义：platform release governance coverage 衡量平台是否支持 schema lint、compatibility diff、security review、feature flag / canary、rollback、deprecation、owner 和 changelog。

面试表达：工具平台发布的是“模型可见契约 + runtime 实现 + 权限安全策略 + eval 数据”的组合版本。

Platform Tenant Isolation Coverage

一句话定义：platform tenant isolation coverage 衡量工具 enablement、Registry visibility、credentials、permission policy、cache key、queue、trace、audit、eval data、rate limit 和 billing 是否都带租户边界。

工程价值：企业平台里跨租户泄露是最高优先级事故，任何缓存或查询缺 tenant_id 都是高危设计。

Platform Provider Adapter Coverage

一句话定义：provider adapter coverage 衡量平台是否能把内部 ToolSpec / ToolCall / ToolResult 映射到不同模型 provider 和 MCP server 的能力差异、schema 限制、tool choice、parallel、streaming 和 result 回填格式。

边界：provider adapter 负责协议差异，业务工具不应绑定某一家 provider 的原始字段。

High Availability Readiness

一句话定义：high availability readiness 衡量 Registry 签名快照、Router 降级、Permission fail closed、Executor timeout / circuit breaker、Trace buffer、Audit 可靠写入和下游隔离是否准备好。

面试表达：平台关键依赖故障时，低风险链路可以降级，高风险动作必须拒绝、缓冲审计或转人工。

Admin Ops Governance Coverage

一句话定义：admin ops governance coverage 衡量管理后台是否对工具创建、权限放宽、风险等级调整、发布、回滚、审计查询和 provider 配置修改做强权限和审批控制。

常见事故：工具执行层很安全，但后台超级权限过宽，导致任何人都能放宽工具权限或跳过发布门禁。

Enterprise Tool Platform Gate

一句话定义：enterprise tool platform gate 是基于模块覆盖、接口契约、Registry、Router、Executor、权限、安全、trace、eval、发布、多租户、provider、HA 和后台治理的综合上线门禁。

面试表达：企业工具平台的核心不是 function calling，而是把工具能力放进可注册、可路由、可执行、可授权、可审计、可评估、可灰度和可回滚的治理闭环。

Agent Trace

一句话定义：agent trace 是一次 agent 执行的完整轨迹，包含 goal、state、action、tool、arguments、observation、permission result、budget 和 final status。

为什么重要：agent 评估、debug、安全审计和 replay 都依赖 trace。只看最终回答无法定位是工具选错、参数错、工具失败、观察没用，还是停止条件错。

Trace Schema Completeness

一句话定义：trace schema completeness 是 trace 任务级字段是否完整记录 trace id、session、task、workflow、状态、开始时间、结束时间和环境摘要的比例。

工程价值：字段不稳定会让 trace 无法查询、聚合、导出和回放。

Span Schema Completeness

一句话定义：span schema completeness 是每个 span 是否完整记录 span id、parent、类型、时间、状态、输入摘要、输出摘要、artifact 引用和错误字段的比例。

面试表达：Agent trace 不是纯文本日志，而是一棵可查询的 span 树。

Span Tree Validity

一句话定义：span tree validity 是 trace 中 root span 唯一且所有 parent span 都真实存在的比例。

为什么重要：span parent 错误会破坏因果链，导致 replay UI 无法说明模型调用、工具执行和错误恢复之间的关系。

Timeline Validity

一句话定义：timeline validity 是 span 起止时间落在 trace 起止时间内，且时间线顺序不矛盾的比例。

工程价值：排查 timeout、取消、并发、重试和工具延迟时，时间线是第一证据。

Artifact Reference Coverage

一句话定义：artifact reference coverage 是需要大对象的 span 是否能够通过 artifact ref 找到文件 diff、截断命令输出、截图、报告或长工具结果的比例。

边界：大对象不一定要直接写进 span，但引用必须可用、脱敏并受访问控制。

Version Capture Coverage

一句话定义：version capture coverage 是 trace 是否记录模型、prompt、工具 registry、git commit、sandbox 镜像和权限策略版本的比例。

面试表达：没有版本捕获，replay 只能看历史过程，不能证明新模型、新 prompt 或新工具策略是否真的改善。

Replay Readiness Rate

一句话定义：replay readiness rate 是 trace 同时具备版本、环境快照、确定性工具结果、沙箱和权限策略信息，能够进入模拟或真实 replay 的比例。

工程价值：生产 trace 进入 regression suite 前，应先通过 replay readiness 审计。

Privacy Masking Coverage

一句话定义：privacy masking coverage 是 trace、span、log 和 artifact 中敏感字段被脱敏、摘要化或阻断的比例。

边界：可观测性不能把 secret、私有代码或用户数据长期保存成新的泄漏面。

Error Attribution Coverage

一句话定义：error attribution coverage 是失败 trace 能定位到失败 span、error type 和 root cause 的比例。

面试表达：只知道 task failed 不够，必须能区分上下文缺失、模型动作错、工具失败、权限阻断、环境不可复现和最终总结失真。

Final Status Consistency

一句话定义：final status consistency 是 Agent 最终状态和测试、验证器、工具状态或业务验收结果一致的比例。

常见失败：测试失败但最终声称修复完成，权限拒绝但最终声称动作已执行。

Metric Export Coverage

一句话定义：metric export coverage 是 trace 是否能导出 latency、token cost、tool count、step count、错误数、权限拒绝数和验证结果等可聚合指标的比例。

工程价值：没有指标导出，trace 只能用于个案复盘，不能用于线上监控和版本回归分析。

Eval Export Coverage

一句话定义：eval export coverage 是 trace 是否包含 replay case、failure taxonomy、bad case label、expected behavior 和 regression 所需摘要的比例。

为什么重要：好的生产 trace 应该能沉淀为下一轮 evaluation harness 的样本。

Trace Replay Gate

一句话定义：trace replay gate 是基于 trace schema、span schema、span tree、时间线、artifact、版本捕获、replay readiness、隐私脱敏、错误归因、最终状态一致、指标导出和评估导出的可观测性门禁。

面试表达：Agent 可观测性的目标不是记录一切，而是在隐私和成本可控的前提下让关键行为可解释、可回放、可评估。

Agent Budget

一句话定义：agent budget 是 agent 执行时允许消耗的步数、上下文长度、工具调用数、时间和成本上限。

常见字段：max steps、max input tokens、max output tokens、max tool calls、timeout、cost cap 和 retry cap。

风险：没有 budget 的 agent 容易循环、重复检索、成本超限或拖慢线上系统。

Agent Incident

一句话定义：agent incident 是 Agent 在线上出现任务未完成却声称完成、工具参数错、越权动作、循环调用、预算超限、状态丢失、trace 缺失或工具结果不可信边界失败的事件。

排查重点：不要只看最终回答，要看 plan、action、arguments、permission check、tool result、observation use、state update、budget、human confirmation 和 final / backend status 是否一致。

Plan Feasibility Rate

一句话定义：plan feasibility rate 是 Agent 计划步骤中能映射到真实工具、权限、输入依赖和预算的比例。

产品意义：计划语言上合理不代表可执行。生产系统要在执行前校验计划是否包含不存在的工具、缺失依赖、越权动作或预算不可承受步骤。

False Completion Rate

一句话定义：false completion rate 是 Agent 最终声称任务完成，但业务系统状态、工具结果或验收器显示未完成的比例。

产品意义：这是 Agent 产品中的高风险失败模式。它会让用户以为任务已经执行，实际后台没有成功、被阻断或仍处于 pending。

Tool Result Injection Block Rate

一句话定义：tool result injection block rate 是工具、网页、邮件或文档返回不可信指令时，系统成功隔离、忽略或阻断这些指令影响工具调用的比例。

工程要求：工具结果只能作为数据，不能覆盖系统策略、用户授权、工具权限或高风险确认流程。

Agent Incident Gate

一句话定义：agent incident gate 是面向 Agent 事故复盘和回归测试的门禁，覆盖任务成功、计划可执行、工具选择、参数合法、工具执行、observation 使用、状态更新、人审确认、误报完成、越权动作、预算超限、trace 完整和工具结果注入阻断。

面试表达：Agent 事故复盘要回答“为什么模型错误被系统执行或被系统误报为完成”。如果 trace、权限、人审、预算或业务验收缺失，就不能把失败归因成单纯模型能力问题。

Agent Gate

一句话定义：agent gate 是 agent 上线前的硬门禁，用任务成功率、工具选择准确率、参数合法率、观察使用率、预算超限率和未授权动作率决定是否放行。

常见指标：task success rate、tool selection accuracy、argument validity、observation use rate、state update coverage、budget overrun rate、unauthorized action rate 和 stop correctness。

面试表达：agent 不能只评估最终文本，要把执行轨迹、安全权限、成本和停止条件一起评估。

Agent Product

一句话定义：agent product 是把 Agent loop、工具执行、权限、人审、trace、预算、评估和业务流程做成可上线、可运营、可审计的产品系统。

和 Agent demo 区别：demo 通常验证模型能否调用工具；产品要证明任务边界清楚、自动化层级合适、高风险动作有人审、失败可恢复、越权可阻断、成本和延迟可控，并且业务指标有持续反馈。

面试表达：Agent 产品不是“越自主越好”，而是让模型在受控范围内完成可验证任务。产品经理、算法、工程、安全和业务 owner 要共同定义门禁，而不是只看模型生成过程是否像人在思考。

Workflow-Agent Hybrid

一句话定义：workflow-agent hybrid 是用确定性 workflow 固定关键流程，用 Agent 处理开放判断、信息补全、工具选择和异常恢复的混合架构。

为什么重要：生产系统中很多步骤其实不需要自由 Agent。把接收、审批、提交、审计和回滚流程 workflow 化，再让 Agent 处理分类、摘要、检索、草稿和诊断，通常比完全自主 Agent 更稳定。

Agent Automation Level

一句话定义：agent automation level 是 Agent 产品的自动化层级，例如 suggest、draft、semi-auto、approval 和 auto。

产品含义：自动化层级是风险和验证能力的结果，不是模型能力越强就越自动。高风险任务如果缺少权限、人审、验收器、回滚和 trace，应停留在 copilot、review 或 approval 形态。

Agent Product Trace

一句话定义：agent product trace 是 Agent 产品一次任务的可审计记录，除了 goal、action、tool 和 observation，还要记录自动化层级、权限检查、人工确认、状态更新、预算、成本、延迟、失败恢复和业务结果。

工程价值：它连接产品评估、安全审计和线上运营。没有 trace，就无法解释任务为什么成功或失败，也无法把 bad case 转成回归评估集。

Tool Execution Success Rate

一句话定义：tool execution success rate 是工具调用中选择正确、参数合法、权限通过且执行结果满足预期的比例。

常见误区：工具 schema 合法不代表工具执行成功。真实指标要同时看工具选择、参数语义、权限、超时、返回结果和后续 observation 使用。

Human Confirmation Coverage

一句话定义：human confirmation coverage 是高风险动作中得到清晰人工确认、审批或二次验证的比例。

产品要求：确认界面要展示动作、对象、参数、原因、影响范围和取消方式。用户盲点“同意”不能算可靠的人审。

Agent Recovery Rate

一句话定义：agent recovery rate 是 Agent 遇到工具失败、权限不足、参数错误、信息缺失或外部状态变化后，通过重试、换工具、澄清、降级、回滚或人工接管成功恢复的比例。

面试表达：生产级 Agent 不要求永不失败，但要求失败可解释、可恢复、可审计。

Observation Use Rate

一句话定义：observation use rate 是工具返回、环境反馈或用户补充信息被后续状态更新和动作决策真正使用的比例。

风险：如果 observation use rate 低，说明 Agent 可能只是调用了工具，但没有把工具结果纳入下一步判断。

State Update Coverage

一句话定义：state update coverage 是需要更新任务状态的步骤中，Agent 或 controller 正确记录目标、进展、约束、错误、确认和剩余预算的比例。

工程价值：状态更新不足会导致重复动作、遗忘约束、误判完成或错误恢复失败。

Unauthorized Action Rate

一句话定义：unauthorized action rate 是 Agent 发起未授权工具调用、越权数据访问、越权外发或越权修改的比例。

上线要求：严重越权不能被平均任务成功率掩盖。很多企业 Agent 门禁会把未授权动作率设为零容忍指标。

Budget Overrun Rate

一句话定义：budget overrun rate 是超过步数、工具调用次数、token、时间或费用预算的任务比例。

产品意义：Agent 往往多轮调用模型和工具，如果没有预算和停止条件，长尾任务会吞掉延迟和成本。

Agent Product Gate

一句话定义：agent product gate 是 Agent 产品上线前的综合门禁，覆盖任务成功、工具执行、高风险确认、失败恢复、状态更新、观察使用、trace 覆盖、越权动作、预算超限、P95 延迟、单位成本、评估集、反馈闭环和业务指标。

面试表达：Agent 产品门禁比普通 LLM 应用更严格，因为 Agent 会改变外部状态。只要高风险确认、越权、trace 或预算不过线，就应该先降级到建议、草稿、review 或 approval 形态。

Tool Use

一句话定义：tool use 是模型根据任务需要调用外部工具，例如搜索、数据库、计算器、代码执行器、文件系统或业务 API。

为什么需要：模型擅长语言理解和生成，但不擅长精确计算、实时信息、私有数据库查询和实际执行动作。工具弥补模型能力边界。

工作方式：系统向模型描述可用工具、参数 schema 和使用约束；模型判断是否需要工具；生成工具调用；系统执行工具；模型读取结果并继续回答。

优点：增强实时性、准确性和可执行性。

风险：参数生成错误、工具返回误读、调用顺序错误、权限控制不足、把工具失败结果当成真实结果。

面试表达：tool use 的关键不是“能调用工具”，而是能否在正确时机调用正确工具，并正确解释工具结果。

Tool Schema

一句话定义：tool schema 是工具调用的输入契约，描述工具名、用途、参数字段、类型、必填项、枚举值和额外字段限制。

为什么重要：schema 能降低参数缺失、类型错误、不可解析输出和工具误用，但不能保证业务语义正确或权限安全。

工程边界：schema validation 之后仍要做参数来源检查、用户权限检查、高风险二次确认和审计记录。

面试表达：严格 schema 解决的是格式和字段约束，不是完整安全系统。

JSON Schema

一句话定义：JSON Schema 是用声明式方式描述 JSON 数据结构、类型、必填字段和值域约束的规范，也是工具参数 schema 的常见表达基础。

工程边界：不同模型平台通常只支持 JSON Schema 的稳定子集，复杂组合约束不一定适合直接暴露给模型；runtime 仍应使用确定性 validator 做最终校验。

Required Field

一句话定义：required field 是工具调用 arguments 中必须出现的字段，例如天气工具里的 city 和 date。

工程边界：required 过少会导致执行时缺关键参数，required 过多会诱导模型在用户未提供信息时编造。缺少关键字段时应优先追问，而不是自动猜测。

Additional Properties

一句话定义：additional properties 指 schema 未声明但模型额外生成的字段。

工程边界：生产级工具通常默认 additionalProperties: false，避免未知字段被下游误处理。只有明确需要开放字典典型参数时才应放宽。

Enum Constraint

一句话定义：enum constraint 用固定候选值限制字段取值，例如 priority 只能是 low、medium 或 high。

为什么重要：enum 能减少同义词漂移和路由歧义，但业务枚举变化时必须和 schema 版本管理同步。

Pattern Constraint

一句话定义：pattern constraint 用正则表达式限制字符串形式，例如日期、邮箱、工单号或客户 ID。

工程边界：description 只能提示模型，pattern 才是 validator 能执行的硬约束；但复杂格式和真实存在性仍要交给业务校验。

Range Constraint

一句话定义：range constraint 用 minimum、maximum 等字段限制数值范围。

工程价值：对 top_k、limit、timeout、amount 等影响成本、延迟或风险的参数，范围约束应作为硬门禁。

Business Validation

一句话定义：business validation 是 schema validation 之后的业务规则检查，例如时间区间、对象归属、用户权限、状态流转和高风险确认。

关键区别：schema validation 回答“参数形状是否合法”，business validation 回答“这件事是否应该执行”。

Schema Repair

一句话定义：schema repair 是 runtime 对低风险、无歧义参数错误做白名单修复，例如大小写归一化、明确日期标准化或受控枚举别名映射。

工程边界：不能为了提高修复率自动改金额、收件人、删除路径、权限范围或其他高风险业务含义。高风险不确定性应转为澄清或确认。

Schema Gate

一句话定义：schema gate 是工具参数上线门禁，综合 schema valid rate、required pass rate、type valid rate、enum valid rate、pattern valid rate、range valid rate、additional properties block rate、business rule pass rate 和 repair success rate。

面试表达：好的 schema gate 会把格式错误、业务错误和可修复错误分开报告，而不是只给一个“能不能执行”的布尔值。

Tool Call

一句话定义：tool call 是模型提出的一次结构化工具调用请求，通常包含工具名、参数和调用标识。

典型流程：模型生成 tool call，系统校验 schema 和权限，tool executor 执行真实工具，系统把 tool output 作为 observation 回填给模型。

常见失败：选错工具、参数缺失、格式合法但语义错误、权限不足、工具超时、返回结果被误读。

Tool Choice

一句话定义：tool choice 是 runtime 控制模型本轮是否能调用工具、能调用哪些工具、是否必须调用工具、是否允许并行调用的策略层。

工程边界：tool choice 不是权限系统。模型生成 tool call 后，runtime 仍要做 schema validation、permission、confirmation、rate limit、cost budget 和 audit trace。

Tool Choice Policy

一句话定义：tool choice policy 是集中决定候选工具集合、tool choice 模式、并行开关、高风险确认、最大步数和成本预算的策略模块。

面试表达：不要把所有工具每轮都交给模型自由选择，而要先由 policy 按用户、场景、权限、意图和风险过滤候选集。

Candidate Tool Set

一句话定义：candidate tool set 是本轮暴露给模型的候选工具集合。

关键取舍：集合太小会漏掉正确工具，集合太大则增加 token 成本、误选概率和攻击面。

Tool Router

一句话定义：tool router 是在模型 tool choice 之前，根据场景、权限、风险、意图、workflow、成本和 provider capabilities 决定本轮候选工具集合的系统组件。

工程边界：Router 负责收敛模型可见工具集合和生成 tool choice policy，但不替代 Executor 的最终权限检查、参数校验、确认和审计。

Scenario Filter Pass Rate

一句话定义：scenario filter pass rate 是 router 暴露的候选工具与当前产品页面、入口场景、租户配置和 workflow 场景一致的比例。

常见失败：客服场景暴露部署工具，报销页面暴露管理员工具，IDE 场景暴露客户数据工具。

Permission Filter Pass Rate

一句话定义：permission filter pass rate 是 router 没有把当前用户、租户或数据范围无权使用的工具暴露给模型的比例。

面试表达：Executor 必须最终阻断越权执行，但 Router 也应先减少无权限工具暴露面。

Risk Filter Pass Rate

一句话定义：risk filter pass rate 是 router 没有把高风险、副作用、不可逆或需要审批的工具过早暴露给模型的比例。

典型风险：退款、发邮件、删除、转账和权限修改工具进入普通 auto 候选集，会放大误调用和 prompt injection 风险。

Candidate Recall

一句话定义：candidate recall 是期望正确工具集合中，有多少被 router 放进候选工具集合。

工程价值：正确工具没进候选集时，后面的模型再强也选不到，任务成功率会被 router 上限卡住。

Candidate Precision

一句话定义：candidate precision 是 router 暴露的候选工具中，有多少属于期望工具集合。

工程价值：precision 低说明候选集过宽，会增加 token 成本、工具混淆、错误调用和攻击面。

Candidate Size Pass Rate

一句话定义：candidate size pass rate 是候选工具数量没有超过当前 provider、成本预算和任务复杂度上限的比例。

面试表达：候选工具不是越多越好，router 应在 recall、安全和成本之间做可审计取舍。

Clarification Accuracy

一句话定义：clarification accuracy 是 router 在缺少关键参数、多个候选对象或高风险确认不足时，正确选择澄清而不是强行暴露工具的比例。

典型场景：用户只说“查一下订单”但没有订单号时，应澄清或暴露低风险 list_recent_orders，不应 forced get_order_status 并让模型猜参数。

Forced Tool Accuracy

一句话定义：forced tool accuracy 是 router 指定 forced tool 的场景中，实际 forced 的工具与 workflow 期望工具一致的比例。

风险：forced tool 用错会放大参数编造、流程跳步和 provider 不兼容问题。

Router Parallel Safety

一句话定义：router parallel safety 是 router 对并行工具调用开关的决策与工具依赖、副作用、幂等和确认要求一致的比例。

边界：只读、独立、幂等工具可以并行；退款、下单、发送、删除等依赖顺序或确认的工具通常不能并行。

Provider Capability Compatibility

一句话定义：provider capability compatibility 是 router 输出的 tool choice mode、parallel policy、forced tool 和 schema 投影与当前 provider 支持能力兼容的比例。

工程价值：多 provider runtime 不能假设所有 provider 都支持相同的 required、forced、allowed tools 或 parallel calls，需要 adapter 降级。

Router Trace Completeness

一句话定义：router trace completeness 是 router 决策记录包含初始工具数、场景过滤、权限过滤、风险过滤、语义召回、最终候选、tool choice、router 版本和 policy 版本的比例。

面试表达：没有 router trace，就无法解释模型为什么没看到某个工具，或为什么看到了不该看的工具。

Tool Router Gate

一句话定义：tool router gate 是综合候选召回、候选精确率、场景过滤、权限过滤、风险过滤、澄清、tool choice、并行安全、provider capability、成本预算和 trace 的上线门禁。

面试表达：Router gate 的核心不是让模型少几个工具，而是证明每次暴露给模型的工具集合都相关、授权、安全、可负担、可解释。

Auto Tool Choice

一句话定义：auto tool choice 是让模型自行决定是否调用工具以及调用哪个工具。

适用场景：低风险、开放式、工具数量较少且工具描述边界清晰的任务。

None Tool Choice

一句话定义：none tool choice 是本轮禁止模型调用工具，只允许直接回答或总结已有信息。

适用场景：纯文本任务、已有上下文总结、工具 loop 达到上限、安全策略要求禁止外部访问的场景。

Required Tool Choice

一句话定义：required tool choice 是要求模型必须调用候选工具中的至少一个。

工程边界：required 不等于缺参时也要硬调用；缺少关键参数时应转为澄清，而不是编造。

Forced Tool Choice

一句话定义：forced tool choice 是 runtime 指定模型必须使用某个具体工具，通常用于固定 workflow 步骤或结构化参数抽取。

风险：前置路由错误、工具不适配或缺参时，forced tool 会放大模型编造参数的风险。

Allowed Tools

一句话定义：allowed tools 是本轮 provider adapter 或 runtime 明确允许模型选择的工具子集。

工程边界：allowed tools 只能减少模型选择空间，不能替代真实执行前的用户权限、租户权限和工具权限检查。

Tool Choice Mode Accuracy

一句话定义：tool choice mode accuracy 是 policy 选择的模式与期望模式一致的比例，例如 none、auto、required、forced 或 clarify。

诊断价值：该指标低时，通常说明意图识别、页面上下文、实时性判断、风险分类或 workflow 状态机有问题。

No-Tool Clarification Block Rate

一句话定义：no-tool clarification block rate 是纯文本任务、信息不足任务和应停止 loop 的任务没有继续调用工具的比例。

工程价值：它专门防止模型在该回答、该澄清或该停止时继续搜索、查询或执行工具。

Forced Missing Argument Block Rate

一句话定义：forced missing argument block rate 是 forced tool 缺少必填参数时，runtime 成功阻断调用并转为澄清的比例。

典型风险：订单号、金额、收件人、文件路径和权限范围缺失时，不能让模型硬造参数。

Parallel Safety Rate

一句话定义：parallel safety rate 是多 tool calls 同轮执行时，调用之间独立、可幂等、无缺失依赖且无未确认高风险副作用的比例。

工程边界：天气、行情、只读检索通常可以并行；退款、付款、发邮件、权限修改这类有依赖或副作用的动作通常不应随意并行。

Rate Limit Pass Rate

一句话定义：rate limit pass rate 是每轮 tool calls 数量、每工具并发数、每用户并发数和全局队列约束都未超限的比例。

工程价值：并行工具调用会增加瞬时压力，限流门禁能防止模型一次生成过多外部请求。

Confirmation Enforcement Rate

一句话定义：confirmation enforcement rate 是高风险工具在未确认时被阻断、已确认时才执行的比例。

典型工具：发邮件、下单、退款、转账、删除数据、修改权限和外部数据导出。

Cost Budget Pass Rate

一句话定义：cost budget pass rate 是工具调用成本、模型 token、外部 API、检索 top-k、并发资源和人工审核成本未超过预算的比例。

面试表达：成本控制不是偷工减料，而是避免低价值请求触发昂贵工具、长 tool loop 或过量并行调用。

Loop Control Rate

一句话定义：loop control rate 是 tool loop 在最大步数、重复调用次数、重试次数或预算上限内停止的比例。

工程价值：达到上限后通常应切到 none，让模型基于已有信息总结或说明不足，而不是继续调用工具。

Tool Choice Gate

一句话定义：tool choice gate 是工具选择策略上线门禁，综合候选集覆盖、模式准确、澄清拦截、缺参阻断、并行安全、id 对齐、限流、确认、成本和 loop 控制。

面试表达：tool choice gate 能把“模型选错工具”和“runtime 策略放错工具”分开定位，是工具协议系统从 demo 到生产的关键门禁。

Argument Parsing

一句话定义：argument parsing 是把模型返回的 raw tool arguments 解析成可校验对象的第一道门。

工程边界：parse 成功只说明语法可读，不说明字段、类型、业务语义、权限和来源可信；streaming 半截参数不应进入执行。

Parse Success Rate

一句话定义：parse success rate 是 raw arguments 成功解析为结构化对象的比例。

诊断价值：该指标低时，通常要检查 provider adapter、streaming 拼接、strict 输出配置、工具描述和错误回填，而不是给业务函数加默认值硬执行。

Validation Error

一句话定义：validation error 是 parse 后由 schema、业务规则、权限、来源或安全门禁发现的结构化错误。

工程价值：错误应包含字段路径、错误码、是否可重试和建议动作，供模型自修复、用户澄清、trace replay 和 eval 使用。

Normalization

一句话定义：normalization 是把低风险、含义明确的参数表达规范化，例如去空格、邮箱小写、枚举大小写归一。

安全边界：normalization 不能改变金额、收件人、删除路径、权限范围或外部发送目标等高风险业务含义。

Argument Evidence Map

一句话定义：argument evidence map 记录关键参数值来自用户消息、历史状态、页面上下文或可信工具结果中的哪一段证据。

上线意义：订单号、客户 ID、金额、邮箱、路径和权限范围等关键参数不能只相信模型生成，必须能追溯来源。

Argument Evidence Coverage

一句话定义：argument evidence coverage 是关键参数中能匹配可信 evidence 的比例。

排查方向：该指标低时，常见原因是上下文截断、实体状态没有结构化保存、模型凭空生成 ID 或不可信网页内容诱导参数。

Safe Normalization Rate

一句话定义：safe normalization rate 是自动规范化操作中只发生在低风险、无歧义字段上的比例。

工程边界：它用于防止系统为了提高修复率，偷偷把高风险错误参数改成看似合理的值后继续执行。

Repair Attempt

一句话定义：repair attempt 是参数校验失败后，runtime 通过规则修复、模型自修复或用户澄清试图获得合法参数的一次尝试。

工程边界：repair attempt 必须有最大次数和错误去重，不能让模型自修复变成无限 tool loop。

Model Self-Repair

一句话定义：model self-repair 是把结构化 validation errors 回填给模型，让模型重新生成合法 arguments 的过程。

适用边界：适合 schema 类型、范围、额外字段等可重试错误；不适合自动修复金额、收件人、路径、权限范围和未授权高风险动作。

Clarification Coverage

一句话定义：clarification coverage 是缺少关键信息或存在歧义时，系统成功转为用户澄清的比例。

产品价值：它衡量系统是否在该问人的地方问人，而不是编造参数、默认执行或返回模糊失败。

Unsafe Argument Block Rate

一句话定义：unsafe argument block rate 是高风险参数异常、缺少 evidence、缺少确认或越权时被 runtime 阻断的比例。

上线要求：对付款、删除、发邮件、改权限、导出数据和生产写操作，该指标通常应作为硬门禁。

Retry Budget Pass Rate

一句话定义：retry budget pass rate 是参数修复和模型自修复过程在最大重试次数、最大 loop 步数和成本预算内停止的比例。

工程价值：它防止参数修复层把局部错误扩大成重复调用、成本失控或用户等待过长。

Idempotency Key Coverage

一句话定义：idempotency key coverage 是有副作用工具调用中携带幂等键或等价防重复执行保护的比例。

典型场景：发邮件、下单、退款、转账、写数据库和删除文件等动作都应绑定最终规范化参数生成幂等保护。

Repair Trace Completeness

一句话定义：repair trace completeness 是参数处理 trace 覆盖 raw、parsed、schema error、normalization、business error、repair attempt、final args、evidence、permission 和 decision 的程度。

工程价值：没有完整 trace，就无法复盘 runtime 到底修了什么、为什么放行、用户是否确认，以及最终执行参数从哪里来。

Argument Repair Gate

一句话定义：argument repair gate 是工具参数校验与修复上线门禁，综合 parse、schema、business、evidence、normalization、repair、clarification、unsafe block、retry budget、idempotency 和 trace 指标。

面试表达：好的 argument repair gate 不追求把所有错误都自动修好，而是确保低风险错误可修、高风险错误可挡、缺信息时可问、所有决策可审计。

Structured Function Calling

一句话定义：structured function calling 是把模型的工具调用意图约束为结构化对象，由 runtime 对工具名、参数、权限和执行结果进行校验与审计。

和 prompt tool use 的区别：prompt tool use 依赖文本约定和正则解析；structured function calling 把工具定义、调用参数和执行结果放到可验证协议里，更适合 trace、eval、retry 和权限治理。

工程边界：模型生成的是调用请求，不是实际函数执行。真实 API、数据库、文件系统和高风险动作仍要由 runtime 授权和执行。

JSON Mode

一句话定义：JSON mode 通常指要求模型输出合法 JSON。

常见误区：合法 JSON 不等于满足业务 schema，也不等于应该执行外部动作。它主要解决语法可解析问题，不能替代 function calling、权限检查和业务校验。

Structured Outputs

一句话定义：structured outputs 是让模型按给定结构输出数据的机制，常用于抽取、分类、表单填写和结构化回答。

和 function calling 的区别：structured outputs 不一定对应真实外部工具；function calling 是动作协议，后面通常要进入 schema validation、authorization、execution、tool result 和 trace 链路。

Tool Selection Accuracy

一句话定义：tool selection accuracy 是模型在需要工具调用的样本中选中期望工具的比例。

为什么重要：schema 合法但工具选错仍然会执行错误动作，因此工具选择准确率要和 schema valid rate 分开统计。

Schema Valid Rate

一句话定义：schema valid rate 是模型生成的工具参数通过 schema 校验的比例。

工程边界：它衡量字段、类型、枚举、必填项和额外字段约束是否满足，但不保证参数语义、权限和高风险确认正确。

Required Field Pass Rate

一句话定义：required field pass rate 是工具调用样本中没有缺失必填字段的比例。

排查方向：该指标低通常说明 schema description、required 设置、澄清策略或上下文抽取有问题。

Type Valid Rate

一句话定义：type valid rate 是模型生成字段类型符合 schema 的比例。

典型错误：把整数写成字符串、把布尔值写成自然语言、把对象写成数组、把数组写成逗号分隔字符串。

Enum Valid Rate

一句话定义：enum valid rate 是带 enum 约束字段生成合法候选值的比例。

工程价值：它能发现模型把 paid 生成成 已支付、PAID 或 payment_success 这类同义词漂移问题。

Pattern Valid Rate

一句话定义：pattern valid rate 是带正则约束的字符串字段满足格式要求的比例。

典型字段：日期、邮箱、URI、工单号、客户 ID 和版本号。

Range Valid Rate

一句话定义：range valid rate 是带数值上下界的字段落在允许范围内的比例。

上线意义：该指标低时，检索、数据库查询、批量执行和外部 API 调用容易出现成本或延迟失控。

Additional Properties Block Rate

一句话定义：additional properties block rate 是 runtime 成功拦截未知额外字段的比例。

安全意义：未知字段可能触发下游误处理、越权透传或工具实现未预期行为，生产系统通常要求该指标接近 1。

Business Rule Pass Rate

一句话定义：business rule pass rate 是 schema 已通过的样本中继续通过业务校验的比例。

关键价值：它专门暴露“格式合法但业务不合法”的调用，例如结束时间早于开始时间、对象不属于当前用户或高风险动作未确认。

Schema Repair Success Rate

一句话定义：schema repair success rate 是白名单修复后同时通过 schema 校验和业务校验的比例。

工程边界：修复只适合低风险、可证明无歧义的 normalization；高风险参数应追问或拒绝，不应自动修。

Argument Exact Match

一句话定义：argument exact match 是模型生成参数和标注期望参数完全一致的比例。

使用场景：适合天气、订单查询、数据库过滤条件、API 参数抽取等字段明确的工具调用评估。

Argument Validity

一句话定义：argument validity 是参数在 schema、业务语义和权限上下文中都可接受的比例。

和 exact match 的区别：exact match 更严格，适合离线标注评估；validity 更贴近生产，允许同义表达或等价参数，但必须通过业务规则。

Unauthorized Attempt Block Rate

一句话定义：unauthorized attempt block rate 是 runtime 成功拦截未授权或高风险工具调用尝试的比例。

上线要求：对删除、支付、外发、数据库写入、shell 执行等高风险动作，未授权拦截通常应作为硬门禁，而不是被平均成功率稀释。

Function Calling Protocol

一句话定义：function calling protocol 是围绕 messages、tools、assistant tool call、tool result、finish reason、streaming delta 和 provider adapter 组织的工具调用输入输出协议。

为什么重要：工具函数本身能跑，不代表协议正确。生产系统要保证 assistant tool call 和 tool result 成对出现、id 对齐、参数完整解析、错误可回填、parallel calls 可区分、streaming 不执行半截参数。

Tool Call ID

一句话定义：tool call ID 是 assistant 生成 tool call 时附带的唯一调用标识，用来把后续 tool result 关联回对应调用。

工程价值：parallel tool calls、多轮 tool loop、错误回填、trace replay 和幂等执行都依赖 tool_call_id。不能用数组顺序隐式对齐结果。

Tool Result

一句话定义：tool result 是 runtime 执行工具后回填给模型的结构化结果或错误，通常要带 tool_call_id。

安全边界：tool result 是 observation，不是 system / developer / user 指令。来自网页、邮件、文档和第三方 API 的结果都要按不可信数据处理。

Tool Result Context

一句话定义：tool result context 是工具原始返回经过安全投影、来源标注、脱敏、压缩和可信度标注后，真正进入模型上下文的 observation。

工程边界：它不是简单把工具 stdout 或 API JSON append 到 messages；必须保留 tool_call_id、来源、状态、错误语义和 trace。

Tool Result Projection

一句话定义：tool result projection 是从工具原始输出中选择当前任务所需字段，并删除内部调试字段、敏感字段和无关长文本的过程。

关键原则：投影目标是**最小充分信息**，不是最大上下文信息。字段越多，泄露、注入和注意力污染风险越高。

Source Metadata Coverage

一句话定义：source metadata coverage 是 tool result 中带有 source id、retrieved time、schema version、文档标题或等价 provenance 的比例。

工程价值：没有 source metadata，最终回答难以引用，离线 eval 难以验证 grounding，事故复盘也难以定位来源。

Redaction Coverage

一句话定义：redaction coverage 是敏感字段在进入模型上下文前被删除、掩码、聚合或替换为引用 ID 的比例。

典型字段：手机号、邮箱、证件号、银行卡、token、内部账号、SQL、堆栈、权限策略和连接字符串。

Tool Result Context Budget Pass Rate

一句话定义：tool result context budget pass rate 是结果投影后的 token 数没有超过本轮上下文预算的比例。

工程价值：它防止搜索结果、日志、文件读取和数据库返回把关键任务约束挤出上下文。

Injection Containment Rate

一句话定义：injection containment rate 是 tool result 中出现外部指令样式内容时，系统成功把它限制为 data-only observation 并收紧敏感工具的比例。

安全边界：网页、邮件、文档和第三方 API 输出不能覆盖 system / developer 指令；真正的阻断应在 runtime 权限和工具策略层完成。

Error Status Fidelity

一句话定义：error status fidelity 是 timeout、rate limit、permission denied、schema error 等工具错误在上下文中仍被表达为错误的比例。

常见事故：把工具超时写成空结果，会让模型错误回答“没有查到”，掩盖真实故障。

Compression Fidelity

一句话定义：compression fidelity 是 tool result 压缩、摘要或字段投影后仍保留关键事实的程度。

工程边界：模型辅助摘要必须保留 source id、关键字段和不确定性；不能只留一句无来源结论。

Conflict Labeling Rate

一句话定义：conflict labeling rate 是多个工具或多个来源返回冲突信息时，系统显式标注冲突的比例。

面试表达：不要强行把冲突结果合成一个确定事实；应保留来源差异、更新时间和可信度，让最终回答说明不确定性。

Tool Result Freshness Pass Rate

一句话定义：tool result freshness pass rate 是有有效期或版本的 tool result 没有被当成新鲜事实继续使用的比例。

典型场景：天气、库存、价格、订单状态和权限结果都可能快速过期，必须带 retrieved_at / expires_at 或版本。

Memory Boundary Pass Rate

一句话定义：memory boundary pass rate 是短期 tool result 没有被错误写入长期 memory 的比例。

风险：把天气、价格、临时搜索结果或一次性工具错误写入长期 memory，会制造陈旧事实、隐私风险和后续上下文污染。

Tool Result Context Gate

一句话定义：tool result context gate 是工具结果进入模型上下文前的上线门禁，综合 ID 对齐、安全投影、脱敏、上下文预算、来源、注入隔离、错误状态、压缩保真、冲突标注、引用、新鲜度和 memory 边界。

面试表达：它把“工具执行成功”与“结果能安全进入上下文”分开评估，是 Agent 从 demo 到生产的关键控制点。

Tool Failure Recovery

一句话定义：tool failure recovery 是工具调用失败后，runtime 按检测、分类、恢复、报告四步选择重试、修复、降级、拒绝或人工接管的机制。

工程价值：它决定 Agent 在下游超时、权限不足、限流、参数错误和有副作用状态未知时，是安全恢复还是重复执行危险动作、编造答案或隐藏故障。

Structured Tool Error

一句话定义：structured tool error 是带有机可读错误码、retryable 标记、建议动作、脱敏细节和 trace 信息的工具错误对象。

工程价值：自然语言错误难以驱动 runtime policy；结构化错误能让模型、执行器、日志、告警和 eval 对同一次失败保持一致理解。

Retryable Error

一句话定义：retryable error 是临时、可恢复、重试不会明显放大风险的工具错误，例如短期网络抖动、临时 5xx 或可等待的限流。

边界：retryable 不是“所有失败都再试一次”。有副作用动作、未知执行状态、权限拒绝、对象不存在和安全策略拒绝通常不能盲目重试。

Non-Retryable Error

一句话定义：non-retryable error 是自动重试通常无法解决或会放大风险的工具错误，例如权限不足、参数业务语义错误、用户未确认或高风险动作被拒绝。

面试表达：遇到 non-retryable error 应澄清、拒绝、降级或转人工，而不是消耗重试预算制造更多失败。

Retry Policy Precision

一句话定义：retry policy precision 衡量实际发起的重试中，有多少是对 retryable、幂等、预算内且不会重复副作用的失败发起的。

常见事故：只看恢复成功率会奖励盲目重试；precision 能暴露权限错误反复请求、退款超时重复提交、下游故障被重试流量打爆等问题。

Exponential Backoff with Jitter

一句话定义：exponential backoff with jitter 是按指数增长等待时间并加入随机抖动的重试策略。

工程价值：指数退避降低下游压力，jitter 避免大量请求在同一时刻重新冲击下游服务。

Retry Budget

一句话定义：retry budget 是为工具调用设置的最大重试次数、最大额外等待时间、并发上限、用户级或租户级重试限制。

工程边界：重试预算要按工具风险、延迟、成本和用户交互需求配置，不能把高成本外部 API 和只读本地查询用同一套默认值。

Unknown Execution State

一句话定义：unknown execution state 是有副作用工具提交后超时或连接中断，系统无法确认动作是否已经被下游执行的状态。

典型场景：发邮件、下单、退款、转账、删除文件和权限修改。正确处理是幂等键、状态查询、用户说明或人工接管，而不是简单重试或声称失败。

Fallback Honesty

一句话定义：fallback honesty 衡量缓存、备用工具或部分结果降级时，系统是否明确告诉用户数据来源、时间、新鲜度和不确定性。

面试表达：降级不是让模型假装成功。实时库存失败后用 10 分钟前缓存，应说明缓存时间和可能变化。

Circuit Breaker Containment

一句话定义：circuit breaker containment 衡量熔断打开后，runtime 是否停止继续打下游、隐藏不可用工具或进入受控降级。

工程价值：熔断的目的不是记录失败，而是防止故障服务被 Agent 的自动重试和并发工具调用继续压垮。

Timeout Cancellation Coverage

一句话定义：timeout cancellation coverage 衡量工具超时后，仍在运行的后台任务是否被取消、标记未知状态或纳入后续查询和接管流程。

边界：对有副作用工具，取消不等于撤销已经到达下游的请求；必须区分 timeout before execution、timeout after submission 和 unknown state。

Human Handoff Readiness

一句话定义：human handoff readiness 衡量需要人工接管的失败样本是否带上用户请求、对话摘要、工具调用、错误、权限结果、风险标签和 trace id。

工程价值：人工接管不是把用户扔给客服，而是把自动系统已经知道的上下文交出去，避免人工重新猜测事故现场。

User-Visible Error Clarity

一句话定义：user-visible error clarity 衡量最终回答是否清楚区分已查到事实、工具失败、权限不足、状态未知、使用缓存、已转人工和用户需要做什么。

常见事故：把工具超时说成“没有查到”，或把状态未知说成“失败，请重试”，都会误导用户做出重复操作。

Failure Trace Completeness

一句话定义：failure trace completeness 衡量失败链路是否记录 run id、tool call id、tool name、错误码、重试次数、降级策略、用户可见状态和版本字段。

工程价值：没有 trace，工具失败只能靠用户投诉复盘；有 trace 才能做离线 eval、告警、回放和根因分析。

Tool Failure Recovery Gate

一句话定义：tool failure recovery gate 是综合失败检测、错误分类、重试安全、幂等、未知状态接管、降级诚实、熔断、超时取消、人工接管、用户说明和 trace 的工具失败恢复门禁。

面试表达：生产级 Agent 不是保证工具永远不失败，而是证明失败时不重复危险动作、不编造结果、不泄露内部细节，并能让用户、人工和运维都知道下一步。

Tool Calling Evaluation

一句话定义：tool calling evaluation 是对工具调用系统从“是否该调用”到“最终任务是否完成”的分层评估。

核心分层：调用决策、工具选择、参数完整性和值准确性、参数来源、工具执行、结果使用、任务成功、安全失败、不安全失败、成本延迟和回归。

Tool Call Recall

一句话定义：tool call recall 是应该调用工具的样本中，系统实际产生工具调用的比例。

常见问题：漏调用会导致实时信息、私有数据或权限受控数据被模型直接编造。

Tool Call Precision

一句话定义：tool call precision 是实际发生工具调用的样本中，确实应该调用工具的比例。

常见问题：误调用会增加成本、延迟和外部系统压力，也可能扩大高风险工具暴露面。

Tool Set Precision

一句话定义：tool set precision 是多工具任务中，实际选择的工具集合里有多少属于期望工具集合。

工程价值：它能暴露 search、database、CRM、email 等工具被过宽候选集误选的问题。

Tool Set Recall

一句话定义：tool set recall 是多工具任务中，期望工具集合有多少被实际选择出来。

工程价值：它能暴露复杂任务只调用了其中一个工具，导致部分信息缺失或任务半完成。

Argument Value Accuracy

一句话定义：argument value accuracy 是工具参数值在规范化后符合用户意图、上下文和业务规则的比例。

边界：金额、收件人、路径、权限对象和有副作用动作参数不能靠宽松等价放过。

Argument Source Coverage

一句话定义：argument source coverage 是关键参数能够追溯到用户输入、对话上下文、页面状态或可信工具结果的比例。

面试表达：参数值正确但来源不可解释，仍然可能是模型猜出来的，不应直接执行高风险工具。

Execution Success Rate

一句话定义：execution success rate 是实际工具调用中，runtime 完成解析、校验、授权、执行并得到明确结果的比例。

分析方式：要按 schema failure、permission denied、timeout、rate limit、downstream error、retry success 和 fallback success 拆分，而不是只报一个总数。

Observation Use Correctness

一句话定义：observation use correctness 是最终回答或后续动作被工具返回结果、结构化错误或可信降级结果支持的比例。

常见事故：工具返回下雨，最终回答却说晴天；工具超时，模型却把失败当成空结果或编造事实。

Safe Failure Rate

一句话定义：safe failure rate 是任务未完成但系统没有越权、没有重复副作用、没有编造结果，并给出清楚下一步的比例。

工程价值：安全失败不等于坏结果。真实生产中，下游不可用、权限不足或信息缺失时，安全失败优于不安全成功。

Unsafe Failure Rate

一句话定义：unsafe failure rate 是任务失败且伴随编造、越权、未确认副作用、敏感泄露、忽略工具错误或危险自动执行的比例。

上线门禁：高风险工具场景里 unsafe failure 应接近零，不能被总体任务成功率平均掉。

Tool Eval Dataset Coverage

一句话定义：tool eval dataset coverage 是工具调用评估集覆盖正例、负例、多工具混淆、缺参、歧义、高风险、注入、失败、多轮和成本边界的程度。

面试表达：只测成功路径会让 demo 好看，但无法暴露生产中最贵、最危险的失败。

Tool Eval Regression Pass Rate

一句话定义：tool eval regression pass rate 是模型、prompt、schema、provider adapter 或 runtime 变更后，历史工具调用回归集仍然通过的比例。

工程价值：平均分不变不代表可上线；如果退款、邮件、权限修改等高风险切片回归失败，应阻断发布。

Tool Calling Eval Gate

一句话定义：tool calling eval gate 是综合调用召回、调用精确率、工具选择、参数值、参数来源、执行、结果使用、安全失败、不安全失败、成本和回归的工具调用评估上线门禁。

面试表达：工具调用 eval gate 的核心是让每个失败都能归因到 decision、selection、arguments、execution、observation、safety 或 budget，而不是只得到一个模糊的最终分数。

Finish Reason

一句话定义：finish reason 是模型返回的一轮生成停止原因，例如正常停止、请求工具调用、长度截断或安全过滤。

协议审计：finish reason 应与结构化输出一致。如果 finish reason 表示 tool calls 但没有 tool call，或 finish reason 表示 stop 但消息里有 tool call，runtime 应进入异常处理。

Tool Loop

一句话定义：tool loop 是模型生成 tool call、runtime 执行工具、tool result 回填、模型继续生成下一步或最终回答的循环。

工程边界：tool loop 必须有 max steps、预算、错误恢复、权限检查和停止条件，否则容易重复调用、超时、成本失控或重复执行副作用工具。

Parallel Tool Calls

一句话定义：parallel tool calls 是模型在同一轮 assistant message 中请求多个工具调用，由 runtime 并行或批量执行并分别回填结果。

风险：结果必须按 tool_call_id 对齐；有副作用工具通常不能随意并行；部分失败需要结构化错误回填，而不是静默丢弃。

Streaming Tool Call Delta

一句话定义：streaming tool call delta 是流式输出时分片返回的 tool call 字段，尤其是分片 arguments。

工程规则：streaming 阶段只缓存增量，等 tool call 完整结束后再 parse、schema validation 和执行。不能执行半截 JSON 参数。

Protocol Chain Validity

一句话定义：protocol chain validity 是检查消息链是否满足 user -> assistant tool call -> tool result -> assistant final answer 等基本顺序的协议指标。

Tool Result ID Match Rate

一句话定义：tool result ID match rate 是 tool result 中的 tool_call_id 能匹配到前序 assistant tool call id 的比例。

Finish Reason Consistency

一句话定义：finish reason consistency 是 finish reason 与实际消息字段是否一致的比例。

Streaming Safety Rate

一句话定义：streaming safety rate 是 streaming 中断、半截参数或取消请求时，runtime 没有执行不完整 tool call 的比例。

Parallel Alignment Rate

一句话定义：parallel alignment rate 是并行工具调用中，每个 tool result 都回到正确 tool_call_id 和语义参数的比例。

Idempotency Protection Rate

一句话定义：idempotency protection rate 是重复请求或重试中，有副作用工具被幂等键正确去重或阻断重复执行的比例。

Tool Call Trace

一句话定义：tool call trace 是工具调用链路的审计记录，包含请求、工具名、参数、schema 校验、权限结果、执行结果、错误、耗时和最终回答引用关系。

为什么重要：没有 trace，就很难判断失败来自工具选择、参数、权限、执行、observation 使用，还是最终回答阶段。

常见指标：tool selection accuracy、argument exact match、schema valid rate、execution success rate、error recovery rate、unauthorized attempt rate 和 tool result injection violation rate。

Tool Result Injection

一句话定义：tool result injection 是工具返回结果中包含不可信指令或污染内容，诱导模型改变任务、泄露信息或调用其他工具。

为什么危险：很多 agent 会把工具返回直接写回上下文，再让模型决定下一步。如果工具返回来自网页、第三方 API、用户上传文件或搜索结果，它本质上仍是不可信数据。

防护方式：把 tool result 标记为 observation，不让它覆盖 system / developer / user 指令；对高风险工具做 allowlist、参数校验、二次确认和审计；必要时对工具返回做清洗、截断、脱敏和引用约束。

面试表达：工具返回不是开发者指令。Agent 安全评估必须看 action trace，检查不可信 observation 是否直接驱动了高权限动作。

Function Calling

一句话定义：function calling 是让模型按预定义函数 schema 生成结构化调用参数的一种工具调用机制。

为什么提出：自然语言调用不稳定，容易格式错误。function calling 通过 schema 约束输出字段、类型和必填项，提高工具调用可靠性。

典型字段：函数名称、参数名、参数类型、枚举值、描述、必填字段和返回格式。

优点：结构化强，便于系统解析和执行，适合 API 调用、数据库查询、订单操作和工作流自动化。

局限：schema 不能保证语义正确。模型可能生成格式合法但业务含义错误的参数，因此仍需要权限校验、参数校验和执行前确认。

面试表达：function calling 解决的是工具调用的结构化接口问题，不等于解决工具使用决策和业务安全问题。

Tool Calling Gate

一句话定义：tool calling gate 是工具调用系统上线前的硬门禁，用正确性、可执行性、安全性和成本指标决定是否放行。

常见指标：工具选择准确率、参数 exact match、schema 合法率、执行成功率、观察使用率、错误恢复率、未授权工具尝试率、不必要工具调用率、tool result injection 违规率、P95 延迟和平均工具调用次数。

面试表达：最终答案正确但越权调用高风险工具，仍然不能算可靠工具调用系统。

ReAct

一句话定义：ReAct 是 Reasoning and Acting 的缩写，让模型在推理和行动之间交替进行。

基本形式：模型先思考当前问题，决定下一步动作，调用工具，观察结果，再继续思考和行动。

为什么有效：很多任务不能一次回答，需要根据外部观察不断修正。ReAct 把推理链和工具调用结合起来，使模型能处理动态任务。

优点：过程可解释，适合检索、问答、网页操作和多步工具任务。

缺点：容易冗长、成本高，错误步骤会传播；如果没有停止条件，可能陷入循环。

面试表达：ReAct 的核心是把“想”和“做”闭环，而不是只让模型输出一段 chain-of-thought。

Plan-Act-Observe

一句话定义：Plan-Act-Observe 是一种 Agent 工程循环，先生成可修改计划，再执行 action，读取 observation，并更新 state 或 plan。

为什么重要：复杂任务不能每一步都临时发挥，也不能让初始计划僵化不变。PAO 强调计划和环境反馈之间的动态修正。

典型指标：plan adherence rate、plan update coverage、observation use rate、state update coverage、stop correctness。

面试表达：ReAct 更强调 reasoning/action 交替，PAO 更强调计划、执行、观察和更新；真实系统常把二者组合使用。

ReAct Trace

一句话定义：ReAct trace 是一次 ReAct / PAO 执行的轨迹，记录 goal、plan、reasoning summary、action、observation、state update、预算和停止原因。

为什么重要：只看 final answer 无法判断模型是否忽略 observation、重复动作、过早结束或越权执行。Trace 是 debug、评估、安全审计和事故回放的基础。

常见字段：task id、goal、plan step、action、arguments、observation、state diff、policy result、retry count、stop reason、success 和 cost。

Plan Update

一句话定义：plan update 是 Agent 根据 observation 修订初始计划或当前子目标。

为什么需要：初始计划常基于不完整信息。工具返回错误、权限不足、证据冲突、测试失败或用户补充信息后，Agent 应重新选择下一步，而不是机械执行旧计划。

常见失败：计划不更新会导致重复检索、继续错误路径、忽略失败 observation 或过早 final。

Repeat Action

一句话定义：repeat action 是 Agent 在没有新增信息的情况下反复执行同类动作或同一工具调用。

风险：重复 action 会造成无限循环、成本超限、用户等待变长，也说明模型没有正确使用 observation。

防护：记录 action signature，设置重复调用拦截、最大重试次数、状态摘要和 ask-user / stop 兜底。

Premature Final

一句话定义：premature final 是任务尚未满足验收条件时，Agent 过早给出最终答案。

典型场景：测试失败后直接声称修好了，检索无证据却给结论，权限被拒后编造执行成功。

防护：定义 stop condition 和 success criteria，把 final 前的必要 evidence、test pass、permission result 或用户确认写入 gate。

ReAct Gate

一句话定义：ReAct gate 是 ReAct / PAO 系统上线前的循环质量门禁。

常见指标：task success rate、action accuracy、argument valid rate、plan adherence rate、observation use rate、state update coverage、parse failure rate、repeat action rate、budget overrun rate、premature final rate、stop correctness 和 blocked recovery rate。

面试表达：ReAct 系统不能只看最终成功率，还要看每一步是否正确行动、读取反馈、更新状态、避免循环、按预算停止并从失败中恢复。

Planning

一句话定义：planning 是 agent 在执行前或执行中把复杂目标拆成子任务和行动顺序的能力。

为什么需要：复杂任务往往不能靠一步工具调用完成。模型需要决定先查什么、再算什么、何时验证、失败后如何回退。

常见策略：一次性计划、边执行边计划、任务树、检查清单、反思修正和基于约束的工作流编排。

工程取舍：自由规划灵活但不可控，固定工作流稳定但泛化差。生产系统常把关键流程工作流化，把开放问题交给模型处理。

面试表达：planning 的价值在于降低复杂任务的混乱度，但需要配合状态管理、工具约束和结果验证。

Task Decomposition

一句话定义：task decomposition 是把复杂目标拆成可执行、可验证、有依赖关系的子任务。

为什么重要：没有分解，Agent 容易盲目调用工具；分解太粗不可执行，分解太细会增加成本和僵化程度。

好分解标准：子任务目标明确、输入输出清楚、粒度适中、有验收标准、依赖关系明确、失败后有恢复路径。

Subgoal

一句话定义：subgoal 是大目标中的一个可执行子目标。

工程要求：每个 subgoal 应绑定 action 或工具、输入、输出、验收标准、风险级别和停止条件。

常见误区：把“思考一下”“继续分析”这类不可验证动作当成子目标，会让计划看起来完整但无法执行。

Acceptance Criteria

一句话定义：acceptance criteria 是判断子任务是否完成的验收标准。

例子：定位测试失败的验收标准不是“看了测试”，而是能指出失败测试、触发条件、相关代码位置和根因假设。

面试表达：没有验收标准，Agent 很容易完成步骤但没有完成目标。

Dependency Graph

一句话定义：dependency graph 是子任务之间的依赖图，用节点表示子任务，用边表示先后约束。

为什么重要：依赖图能检查计划是否违反顺序，例如未评估就部署、未定位失败就修改代码、未验证就 final。

常见指标：dependency violation rate、critical path length 和 parallelizable task ratio。

Dynamic Replanning

一句话定义：dynamic replanning 是 Agent 根据 observation、失败、预算、权限或用户变更修正计划。

为什么需要：初始计划常基于不完整信息。工具失败、证据冲突、权限不足或新约束出现后，Agent 应更新计划而不是机械执行旧计划。

风险：重规划没有理由会导致方向漂移；工程上应记录 plan update reason。

Critical Path

一句话定义：critical path 是依赖图中决定最短完成时间或成本下界的最长依赖路径。

为什么重要：即使很多任务可以并行，整体耗时仍受关键路径限制。规划系统不能只数总步骤，还要看依赖结构。

Planning Gate

一句话定义：planning gate 是规划质量上线门禁，用目标覆盖、可执行性、验收标准、依赖合法性、重规划、失败恢复和风险确认决定是否放行。

常见指标：goal coverage、executable step rate、acceptance coverage、dependency violation rate、replan coverage、failure recovery rate、risk confirmation coverage、overlong plan rate、repeat subgoal rate 和 task success rate。

面试表达：计划文本漂亮不等于规划可靠，必须能执行、能验证、能根据反馈修正，并且高风险步骤受控。

Memory

一句话定义：memory 是 agent 或对话系统保存、检索、更新和删除历史信息的外部状态系统。

为什么需要：多轮任务中，模型需要记住用户目标、已经尝试过的方法、工具返回结果、长期偏好、项目事实和失败经验。仅靠上下文窗口会受长度限制，且无法稳定跨任务复用。

类型：常见类型包括 short-term memory、long-term memory、episodic memory、semantic memory、procedural memory 和 preference memory。

实现方式：对话摘要、事件日志、向量记忆、结构化状态表、用户画像、任务检查点和文件化工作记录。

关键点：memory 不是把所有历史塞进 prompt，而是对历史进行筛选、压缩、结构化、按需检索和权限过滤。

风险：记忆过期、错误记忆污染、隐私泄露、权限混淆、跨用户泄露和把历史偏好过度泛化到新任务。

面试表达：memory 系统要同时设计写入、检索、更新、删除、权限、过期和评估，而不是只加一个向量库。

Short-Term Memory

一句话定义：short-term memory 是 Agent 当前任务或当前会话内的工作状态。

典型内容：当前目标、计划、已完成步骤、工具结果、失败尝试、剩余预算、待确认事项和停止条件。

为什么重要：没有短期 memory，Agent 容易重复 action、忽略 observation、忘记测试结果或提前 final。

工程边界：短期 memory 服务当前任务闭环，任务结束后可以压缩成 trace summary，但不一定写入长期 memory。

Long-Term Memory

一句话定义：long-term memory 是跨 session 或跨任务持久化保存的信息。

典型内容：稳定用户偏好、项目事实、团队规范、常见工具配置、历史任务总结、长期目标进度和可复用失败教训。

写入原则：必须有未来价值、足够稳定、来源可信、可验证、可删除。一次性指令、敏感信息、工具可疑输出和模型未验证推断不应直接长期保存。

Episodic Memory

一句话定义：episodic memory 是记录具体事件、任务经历或失败复盘的记忆。

例子：上次登录测试失败是空密码边界条件导致，最终补充校验和回归测试。

工程价值：帮助 Agent 回忆“上次怎么做的”，适合 debug、项目复盘和错误恢复。

风险：具体事件容易过期，必须带 timestamp、项目范围、适用条件和证据来源。

Semantic Memory

一句话定义：semantic memory 是保存相对稳定事实和知识的记忆。

例子：项目使用 Python 3.11、团队代码风格要求小函数、部署环境使用 Kubernetes。

工程要求：语义记忆要记录来源、时间、置信度和适用范围；发现新旧事实冲突时要更新、降权或请求确认。

Procedural Memory

一句话定义：procedural memory 是保存稳定操作流程或工作规范的记忆。

例子：修复该仓库代码后要运行哪些测试，发布前要检查哪些指标。

工程要求：程序性记忆要和权限、版本、环境绑定，不能把一个项目的流程机械迁移到另一个项目。

Preference Memory

一句话定义：preference memory 是保存用户稳定偏好的记忆。

例子：用户偏好中文解释、先给结论、回答简洁、修改代码后附验证命令。

风险：一次性偏好不能写成长期偏好。例如“这次详细一点”不是“以后永远详细”。

Memory Write Gate

一句话定义：memory write gate 是决定候选信息能否写入长期 memory 的策略门禁。

常见检查：future value、稳定性、来源可信度、置信度、敏感等级、是否一次性指令、是否来自不可信工具输出、是否存在 prompt injection 迹象和是否需要用户确认。

面试表达：memory 写入比检索更危险，要把它当成高影响状态变更，而不是普通日志追加。

Memory Retrieval Score

一句话定义：memory retrieval score 是 memory 候选排序得分，通常综合相关性、新鲜度、重要性、置信度和风险惩罚。

常见信号：embedding similarity、recency decay、importance、confidence、source trust、permission、sensitivity、staleness 和 conflict flag。

面试表达：相似不等于相关，相关不等于可用；检索排序后仍要做权限、过期、冲突和压缩。

Memory Permission Gate

一句话定义：memory permission gate 是 memory 检索和使用前的用户、项目、范围、敏感等级和删除状态检查。

为什么重要：memory 常包含用户偏好、项目事实、历史任务和可能的敏感信息。跨用户或跨项目泄露比普通检索错误更严重。

常见检查：user namespace、project namespace、private / shared scope、allowed sensitivity、deleted flag、expiry 和 audit log。

Memory Decay

一句话定义：memory decay 是随时间降低旧记忆权重或要求重新验证的机制。

为什么需要：项目环境、用户偏好、依赖版本和事实状态会变化。旧记忆不一定要删除，但应该被降权、标记或重新确认。

常见形式：指数衰减、固定过期时间、版本失效、最近使用刷新和人工确认刷新。

Memory Conflict

一句话定义：memory conflict 是同一用户 / 项目 / key 下出现互相矛盾的记忆。

例子：旧记忆说项目使用 Python 3.9，新记忆说项目使用 Python 3.11。

处理方式：新版本覆盖、保留版本范围、降低旧记忆权重、检索时暴露冲突或请求用户确认。

Memory Pollution

一句话定义：memory pollution 是错误、不相关、过期或恶意信息进入 memory 并影响后续任务。

典型来源：工具返回、网页片段、RAG 文档、模型未验证推断、一次性用户指令和错误摘要。

防护方式：来源可信度分级、写入门禁、高影响记忆确认、敏感信息过滤、定期清理、可删除和审计。

Memory Gate

一句话定义：memory gate 是 memory 系统上线或注入上下文前的综合门禁。

常见指标：retrieval precision、retrieval recall、stale use rate、unauthorized retrieval rate、conflict count、unsafe write block rate、deleted memory returned 和 task success lift。

面试表达：memory 是否可靠不能只看召回数量，要同时看有用性、准确性、安全性、可控性和删除一致性。

Code Agent

一句话定义：Code Agent 是在代码仓库中围绕目标执行搜索、读文件、生成 patch、运行测试、分析反馈并总结结果的 Agent。

和代码补全的区别：代码补全主要根据局部上下文生成代码；Code Agent 需要仓库理解、工具执行、任务状态、测试反馈、权限控制和 trace 审计。

核心能力：仓库理解、符号搜索、上下文选择、最小 patch、测试执行、错误日志分析、依赖变更审计、用户改动保护和沙箱控制。

常见风险：改错文件、过大重构、重复失败命令、忽略测试失败、触碰用户已有改动、引入不必要依赖和执行高风险命令。

面试表达：Code Agent 的输出不是一段代码，而是可审计 diff、验证结果和失败边界。

Coding Agent Workflow

一句话定义：coding agent workflow 是 Code Agent 从任务理解、仓库探索、计划、小步编辑、验证、反馈修复到最终总结的端到端执行流程。

核心阶段：task classification、repository exploration、patch localization、minimal patch、validation、feedback repair、user change protection、permission check 和 trace summary。

常见失败：不探索直接改、计划扩大范围、忽略测试失败、false completion、覆盖用户改动、高风险命令缺确认和 trace 缺字段。

面试表达：可靠 workflow 的关键不是让模型一次性生成更长代码，而是把每一步变成可观察、可验证、可回滚和可审计的动作。

Task Type Classification

一句话定义：task type classification 是 Code Agent 在开始前判断任务属于 bug fix、feature、refactor、test writing、code review、migration 还是 production debug 的能力。

为什么重要：任务类型决定是否先复现、读哪些测试、是否允许重构、验证范围多大，以及是否需要用户确认。

Repository Exploration Coverage

一句话定义：repository exploration coverage 衡量 agent 是否读到了当前任务必须覆盖的关键文件、调用方、测试、配置和项目约定。

面试表达：探索覆盖率不是读完整仓库，而是在最小上下文预算内覆盖解决任务所需的证据。

Plan Coverage

一句话定义：plan coverage 衡量 agent 是否在编辑前形成了短、可执行、可更新的计划，并把目标、修改文件、验证方式和风险点写入 trace。

常见误区：计划越长越好。真实 coding agent 计划应该约束范围，而不是把小 bug fix 扩成全模块重构。

Workflow Validation Coverage

一句话定义：workflow validation coverage 衡量每个 coding task 是否运行了相关测试、lint、type check、build 或其他明确验证。

边界：无法运行测试时也要记录原因和替代验证，不能把“静态看起来没问题”写成“已验证通过”。

Feedback Use Rate

一句话定义：feedback use rate 衡量测试失败、命令错误、patch 冲突或权限拒绝这类 observation 是否被 agent 用来更新计划和继续修复。

为什么重要：测试失败是 workflow 的关键信号；忽略失败后直接总结完成，是 Code Agent 中最危险的 false completion 来源之一。

Coding Agent Workflow Gate

一句话定义：coding agent workflow gate 是基于任务分类、探索覆盖、patch 聚焦、验证覆盖、反馈使用、用户改动保护、高风险动作确认和 trace 完整率的综合门禁。

面试表达：Code Agent workflow 是否可靠，不能只看最终 diff 是否能编译，还要看它有没有按正确流程行动、是否用上了失败反馈、是否保护共享工作区和权限边界。

File System Tool

一句话定义：file system tool 是 Code Agent 用来列目录、搜索、读取、创建、编辑、比较 diff 和回滚文件的受控工具集合。

边界：文件系统工具不是任意本机访问能力；它应受 workspace root、权限、secret denylist、文件类型、输出截断、trace 和人审策略约束。

Workspace Root

一句话定义：workspace root 是 Code Agent 文件读写的根边界，所有相对路径都必须规范化并解析到这个根目录内。

为什么重要：没有 workspace root，路径穿越、绝对路径、home 目录、系统目录和凭证文件都可能被误读或误写。

Path Containment

一句话定义：path containment 衡量文件工具是否能把路径穿越、symlink 逃逸、workspace 外路径和未授权路径阻断在执行前。

面试表达：路径安全不能只靠 prompt 约束，要在工具层做规范化、真实路径解析、secret pattern 阻断和写权限校验。

Read File Tool

一句话定义：read file tool 是按路径、offset、limit 和行号读取文件片段的只读工具。

关键输出：path、offset、limit、total lines、truncated、file type、line numbers 和 error type。缺少截断标记时，模型容易把局部片段当成完整文件。

Search Tool

一句话定义：search tool 是在文件名或文件内容中定位候选位置的只读工具，通常应返回路径、行号、片段、匹配数和截断信息。

常见风险：搜索结果不是完整事实；大文件截断、多处同名函数、生成目录和 vendor 目录都会误导后续编辑。

Apply Patch Tool

一句话定义：apply patch tool 是基于 old context、目标文件和局部 diff 执行小范围代码修改的写入工具。

为什么重要：相比 whole-file rewrite，apply patch 更容易保持修改聚焦、检测冲突、生成 diff、保护用户改动和支持回滚。

Patch Context Match

一句话定义：patch context match 是 apply patch 前确认 old context 在目标文件中唯一且仍然有效的校验。

失败模式：匹配不到表示文件已变化或上下文错误；匹配多处表示上下文不够具体。二者都应阻止自动写入。

Secret File Block Rate

一句话定义：secret file block rate 衡量 .env、SSH key、凭证、cookie、registry token 等敏感文件读取请求被阻断的比例。

面试表达：敏感文件不应直接进入模型上下文或 trace；确需处理时也应先脱敏、缩小范围并请求确认。

Read Truncation Clarity

一句话定义：read truncation clarity 衡量 read / search 输出是否明确说明是否截断、截断范围和继续读取方式。

为什么重要：不透明截断会让模型误判文件结尾、函数唯一性或配置完整性。

Concurrent Edit Protection

一句话定义：concurrent edit protection 是防止 Code Agent 基于旧内容覆盖用户并发修改的机制。

常见做法：读取时记录 hash / mtime，编辑前复核，patch 时要求 old context 匹配，冲突时重新读取或请求用户确认。

Unrelated Diff Rate

一句话定义：unrelated diff rate 衡量一次代码任务中和用户目标无关的文件、格式、依赖、生成文件或 lockfile 改动比例。

面试表达：无关 diff 会提高 review 成本和回归风险，是 Code Agent 可靠性评估中的核心反向指标。

Diff Faithfulness

一句话定义：diff faithfulness 衡量 Agent 最终总结的修改文件、修改内容、行数变化和验证结果是否忠实来自真实 diff 与真实执行结果。

失败模式：模型凭记忆总结、漏报文件、声称测试已跑但实际未跑、把 patch 失败写成已修改。

Rollback Readiness

一句话定义：rollback readiness 衡量成功写入后是否具备 reverse patch、before / after diff、agent 改动归属和回滚前并发修改检查。

边界：回滚只能回滚 agent 自己的改动，不能覆盖用户在 agent 修改之后的新改动。

File Edit Gate

一句话定义：file edit gate 是基于路径安全、secret 阻断、截断清晰、上下文唯一、patch 成功、并发保护、无关 diff、回滚就绪和 diff 忠实度的综合门禁。

面试表达：文件编辑门禁不是为了阻止 agent 写代码，而是保证每一次写入都可定位、可审计、可验证、可回滚。

Terminal Execution

一句话定义：terminal execution 是 Code Agent 通过受控 shell / test runner / package manager / git 命令观察、验证、构建或排查项目的能力。

风险：终端可以运行测试，也可以删除文件、安装依赖、访问网络、读取密钥、启动长驻服务或修改系统配置，因此必须由 runtime 做硬控制。

Bash Tool

一句话定义：Bash tool 是把模型提出的命令请求转成受控终端执行的工具接口。

关键输入：command、working directory、timeout、reason、expected effect、requires network 和 risk hint。只给一个裸 command 字段，会让权限系统难以判断副作用。

Test Runner Safety

一句话定义：test runner safety 是对 pytest、npm test、go test、cargo test、integration test、snapshot update 和 E2E test 等验证命令做副作用控制的机制。

常见误区：测试命令不一定只读；它可能写数据库、启动容器、访问外部 API、更新 snapshot 或读取敏感环境变量。

Command Risk Level

一句话定义：command risk level 是 runtime 根据命令、参数、working directory、项目脚本、网络访问和 expected effect 判断出的风险等级。

常见等级：read only、test readonly、local write、dependency install、network access、network execute、database、secret read、git rewrite、destructive、long running 和 unknown。

Command Allowlist

一句话定义：command allowlist 是允许自动执行的低风险命令集合，通常只覆盖只读状态查询、最小范围测试、lint 和 type check。

边界：allowlist 不能只匹配首个 token；同一个 npm test 在不同项目脚本中可能有完全不同副作用。

Command Denylist

一句话定义：command denylist 是默认阻断的危险命令集合，例如 rm -rf、git reset --hard、git clean -fd、sudo、curl | sh、读取 .env 和生产数据库操作。

面试表达：denylist 只能拦截显式危险模式，不能替代 sandbox、确认、路径限制和 trace。

Command Auto Execution Precision

一句话定义：command auto execution precision 衡量自动允许执行的命令中，实际属于低风险只读或只读测试命令的比例。

为什么重要：自动执行里混入 install、network、destructive 或 unknown script，会直接破坏用户信任。

Dangerous Command Block Rate

一句话定义：dangerous command block rate 衡量 destructive、secret、network execute、dependency install、database、system 和 git history rewrite 命令被阻断或确认的比例。

边界：高风险命令不一定永远禁止，但不能静默自动执行。

Command Confirmation Coverage

一句话定义：command confirmation coverage 衡量需要用户确认的命令是否展示了 command、working directory、reason、expected effect 和 risk level 并获得确认。

适用场景：安装依赖、网络访问、snapshot update、数据库迁移、格式化写入、长驻服务和未知项目脚本。

Network Access Control

一句话定义：network access control 是对命令中的外部读取、外部写入、包下载、上传和下载后执行做权限控制。

面试表达：下载内容不应自动执行；外部写入和上传要比外部读取更严格，并且网络目标应进入 trace。

Timeout Cancellation Coverage

一句话定义：timeout cancellation coverage 衡量长时间命令、卡住测试和持续输出进程超时后是否被取消、保留部分输出并记录 error type。

为什么重要：没有取消和部分输出，Agent 可能一直等待、占用资源，或在无反馈状态下误判任务完成。

Command Output Truncation

一句话定义：command output truncation 是把超长 stdout / stderr 压缩成摘要、关键错误片段、truncated 标记和 artifact 引用的机制。

边界：截断不能丢失可追溯性；完整输出应保存在 trace 或 artifact 中。

Secret Masking Coverage

一句话定义：secret masking coverage 衡量命令输出中 key、token、cookie、连接串、私钥路径和凭证被脱敏的比例。

面试表达：最好的策略是最小化环境变量暴露；脱敏只是最后防线，不能把密钥原文写进模型上下文或最终总结。

Command Trace

一句话定义：command trace 是每次终端执行的结构化记录，至少包含 command、working directory、reason、risk、permission、duration、exit code、output summary、truncated、masked、error type 和 artifacts。

用途：用户审计、失败复盘、测试反馈使用、命令超时率统计、工具安全回归和不同 runtime 版本对比。

Command Safety Gate

一句话定义：command safety gate 是基于 working directory containment、risk classification、auto execution precision、dangerous command block、confirmation coverage、network control、timeout cancellation、output truncation、secret masking 和 command trace 的综合门禁。

面试表达：终端执行门禁不是为了让 Agent 少运行命令，而是让验证命令安全、可控、可取消、可审计。

Context Management

一句话定义：context management 是 Agent Harness 在每轮模型调用前选择、排序、压缩和渲染上下文的能力。

为什么重要：Agent 失败经常不是模型完全不会，而是上下文缺关键文件、过期摘要误导、长日志挤掉用户目标或不可信内容边界不清。

Context Candidate

一句话定义：context candidate 是可进入模型上下文的候选信息单元，例如用户目标、系统规则、文件片段、搜索结果、命令输出、当前 diff、trace、task state 或 memory。

关键字段：source、trust level、priority、token cost、relevance、required flag、freshness、compression status 和 inclusion decision。

Context Budget Utilization

一句话定义：context budget utilization 衡量最终上下文消耗了多少 token budget。

边界：预算利用率不是越高越好；低价值日志和旧摘要即使没溢出，也可能降低模型决策质量。

Key Context Recall

一句话定义：key context recall 衡量当前步骤必须看到的信息是否进入了上下文。

典型关键项：用户最新指令、当前编辑文件、相邻测试、失败命令关键错误、当前 diff 和高风险权限决定。

Constraint Retention

一句话定义：constraint retention 衡量用户约束、安全规则、拒绝过的方案和 acceptance criteria 是否在压缩后仍被保留。

面试表达：用户最新指令不能被旧摘要覆盖；被用户拒绝过的方案也需要进入 task state 或摘要，避免重复提出。

Task State Compression

一句话定义：task state compression 是把 goal、phase、plan、modified files、validation status、open questions 和 risks 压缩成结构化状态的机制。

为什么重要：结构化 task state 比自然语言历史更适合恢复、UI 展示、evaluation 和 trace replay。

Summary Faithfulness

一句话定义：summary faithfulness 衡量上下文摘要中的结论是否被原始文件、命令输出、trace 或用户消息支持。

失败模式：把假设写成事实、丢失用户约束、错误归因测试失败、使用过期行号。

Stale Summary Rate

一句话定义：stale summary rate 衡量已过期摘要仍进入上下文的比例。

常见来源：文件已编辑但摘要未更新、测试已重跑但旧失败仍保留、用户已改变约束但旧摘要仍优先。

Tool Output Compression Fidelity

一句话定义：tool output compression fidelity 衡量终端输出、测试日志、搜索结果和工具 observation 压缩后是否保留关键决策信号。

关键保留项：命令、状态、exit code、失败测试名、关键错误、截断标记和 artifact 引用。

Current Diff Coverage

一句话定义：current diff coverage 衡量当前 agent 已产生的 diff 是否进入上下文。

为什么重要：缺少当前 diff，模型容易重复修改、漏报文件、误判验证范围或写出不忠实总结。

Trust Boundary Coverage

一句话定义：trust boundary coverage 衡量 README、issue、网页、测试输出和工具结果等不可信内容是否被标记为 data-only。

面试表达：不可信内容可以作为数据，但不能成为更高优先级指令；工具调用仍必须经过 permission gate。

Memory Write Safety

一句话定义：memory write safety 衡量长期或项目 memory 写入是否经过来源、验证、敏感性、作用域和过期策略检查。

边界：不能把密钥、一次性异常、未验证猜测、用户隐私或 prompt injection 写入长期 memory。

Context Builder Gate

一句话定义：context builder gate 是基于预算、关键上下文召回、上下文精确率、约束保留、task state、摘要忠实、过期摘要、不可信边界和 memory 写入安全的综合门禁。

面试表达：Context Builder 不是 prompt 拼接器，而是 Agent 下一步决策的证据选择器和压缩器。

Patch Localization

一句话定义：patch localization 是 Code Agent 把修改集中在真正相关文件和代码区域的能力。

常见指标：patch localization precision、patch localization recall、unrelated change rate 和 changed lines per task。

为什么重要：真实代码任务中，改动范围越漂移，review 成本和回归风险越高。

Minimal Patch

一句话定义：minimal patch 是在满足用户目标和验证要求的前提下，尽量小、聚焦、可 review 的代码修改。

边界：最小修改不等于不补测试。如果 bug 可复现且项目有测试体系，补充回归测试通常是必要改动。

Test Execution Feedback

一句话定义：test execution feedback 是 Code Agent 运行测试、读取失败、定位根因并迭代修复的反馈闭环。

常见指标：test pass rate、validation coverage、repair success rate、repeat command rate 和 command success rate。

面试表达：不能只说代码应该能过，要尽量实际运行与任务相关的验证命令，并把未运行原因说清楚。

Code Sandbox

一句话定义：code sandbox 是限制 Code Agent 文件写入、命令执行、网络访问、超时和权限范围的运行环境。

为什么重要：模型可以提出动作，但系统必须验证动作是否允许执行。高风险命令、依赖安装、权限配置和生成文件修改都要受控。

User Change Protection

一句话定义：user change protection 是 Code Agent 不覆盖、不回滚、不误改用户已有改动的机制。

常见做法：修改前读取文件和 diff 状态，区分用户已有改动与 Agent 新改动，必要时请求确认。

Code Agent Gate

一句话定义：Code Agent gate 是基于任务成功率、测试通过率、验证覆盖、patch 聚焦、无关改动、用户改动触碰率和沙箱安全的上线门禁。

面试表达：Code Agent 评估不能只看最终答案，要同时看 diff 是否必要、测试是否通过、命令是否受控、用户改动是否被保护。

Claude Code Architecture

一句话定义：Claude Code architecture 是从公开资料和终端 coding agent 产品形态出发，对 CLI 控制面、session、context builder、工具、权限、sandbox、memory、hooks、MCP、trace 和 evaluation readiness 的系统化分析。

边界：只能把公开文档和可观察行为写成确定结论；闭源内部 planner、prompt、排序策略和隐藏状态不能写成官方实现。

面试表达：分析 Claude Code 类产品时，要先说明资料边界，再用 harness 模块和安全门禁拆解，而不是把社区猜测当成内部架构。

Terminal Coding Agent

一句话定义：terminal coding agent 是运行在命令行中的 Code Agent，直接围绕仓库、shell、测试、diff、权限确认和任务状态工作。

优势：贴近真实工程环境，适合服务器、容器、远程仓库和长任务；风险是命令执行、日志压缩、权限确认和 diff 展示要求更高。

CLI Control Command

一句话定义：CLI control command 是终端型 agent 的产品控制面命令，例如帮助、清空上下文、恢复会话、切换配置或管理权限。

边界：它不同于模型工具调用，也不同于 shell 命令；控制命令应由产品层处理，不能当成普通 prompt 混进模型上下文。

Architecture Evidence Coverage

一句话定义：architecture evidence coverage 衡量架构分析中的组件是否有公开文档或可观察产品行为支撑。

面试表达：证据覆盖率低时，应降低表述确定性，避免把未公开内部实现写成事实。

Required Module Coverage

一句话定义：required module coverage 衡量 Claude Code 类架构分析是否覆盖 CLI、session、context、工具、文件、终端、权限、sandbox、memory、hooks、MCP、trace 和 eval 等必需模块。

Access Governance Coverage

一句话定义：access governance coverage 衡量 CLI、文件、终端、memory、网络和外部资源访问入口是否有明确治理机制。

Core Loop Governance Coverage

一句话定义：core loop governance coverage 衡量 session manager、context builder、model adapter、action parser、tool registry、execution engine、state store 和 response renderer 是否受状态机、权限、trace 和错误恢复约束。

Permission Control Coverage

一句话定义：permission control coverage 衡量工具 allowlist、权限提示、secret 阻断、网络控制、sandbox 和人工确认是否覆盖关键动作。

State Recovery Coverage

一句话定义：state recovery coverage 衡量 session resume、context compaction、local transcript、checkpoint 和长任务恢复能力是否被架构分析覆盖。

Extension Governance Coverage

一句话定义：extension governance coverage 衡量 hooks、MCP、skills、插件和外部工具接入是否有权限绑定、allowlist、输出边界和 trace。

Observability Coverage

一句话定义：observability coverage 衡量架构分析是否覆盖本地会话记录、权限审计、trace logger、trace export、错误归因和评估导出。

Architecture Eval Readiness

一句话定义：architecture eval readiness 衡量 Claude Code 类产品分析是否能转成可复现评估样本、回归集、安全指标、成本指标和版本捕获。

High-Risk Surface Governance

一句话定义：high-risk surface governance 衡量终端、文件写入、memory、hooks、MCP、sandbox、网络 and 外部工具等高风险能力面是否被硬权限、沙箱、人审和审计覆盖。

Claude Code Architecture Gate

一句话定义：Claude Code architecture gate 是基于公开证据、必需模块、访问治理、核心循环、权限控制、状态恢复、扩展治理、可观测性、评估准备度和高风险治理的综合门禁。

面试表达：一个成熟的 Claude Code 类架构分析不能只说“会读写文件和运行命令”，还要证明这些能力有公开证据、runtime 边界、权限治理、trace 和 evaluation harness。

OpenCode Architecture

一句话定义：OpenCode architecture 是从 OpenCode 官方文档和开源项目公开能力出发, 对多客户端、server、session、agents、providers、tools、permissions、MCP、custom tools、plugins、snapshot、diff、compaction、SDK 和 events 的系统化分析。

面试表达：OpenCode 的价值不只是开源，而是把 coding agent runtime 的控制面、扩展面和状态面显式暴露出来，方便分析配置治理、权限隔离、工具扩展、恢复能力和评估准备度。

Open Runtime Control Plane

一句话定义：open runtime control plane 是通过配置、agent 定义、权限规则、provider、tools、MCP、plugins 和 server API 控制 agent runtime 行为的开放控制面。

边界：控制面越开放，越要有组织级配置、冲突覆盖、权限默认值、安全审计和版本捕获。

Open Runtime Module Coverage

一句话定义：open runtime module coverage 衡量 OpenCode 类开源 runtime 分析是否覆盖 multi-client、server、session、agents、providers、tools、permissions、files、shell、search、LSP、MCP、custom tools、plugins、snapshots、diff、compaction、SDK 和 events。

Config Governance Coverage

一句话定义：config governance coverage 衡量 global config、project config、remote config、managed config、agent config、tool config 和 permission config 是否有明确治理、优先级和冲突处理。

Agent Permission Isolation

一句话定义：agent permission isolation 衡量 primary agent、subagent、plan / build mode、自定义 agent 和隐藏系统 agent 是否按任务边界配置不同模型、工具、step budget 和权限。

Tool Permission Binding Coverage

一句话定义：tool permission binding coverage 衡量 bash、edit、write、read、grep、glob、LSP、web、question、MCP 和 custom tools 是否绑定 allow / ask / deny、参数 pattern、风险等级和 trace。

MCP Tool Namespace Coverage

一句话定义：MCP tool namespace coverage 衡量 MCP server 和工具是否有命名空间、按 agent 启用、上下文成本控制、鉴权和权限绑定。

Custom Tool Schema Coverage

一句话定义：custom tool schema coverage 衡量自定义工具是否有输入 schema、输出 schema、错误类型、权限、副作用说明、版本和审计策略。

Server API Governance

一句话定义：server API governance 衡量 HTTP server、OpenAPI、SDK、event stream、permission response、session API、file API 和 share API 是否有认证、网络边界、最小权限和审计。

Snapshot Recovery Coverage

一句话定义：snapshot recovery coverage 衡量 snapshot、diff、revert、session fork、abort、summary、compaction、todo 和 message parts 是否支持长任务恢复、撤销和复盘。

Provider Adapter Coverage

一句话定义：provider adapter coverage 衡量 OpenCode 类 runtime 是否能显式管理不同模型提供商、模型选择、成本、上下文长度、fallback 和版本。

OpenCode Architecture Gate

一句话定义：OpenCode architecture gate 是基于公开证据、开放 runtime 模块覆盖、配置治理、agent 权限隔离、工具权限绑定、扩展治理、server API 治理、snapshot 恢复、评估准备度和高风险治理的综合门禁。

面试表达：OpenCode 类系统不能只讲“开放、可扩展、支持 MCP”，还要说明开放能力如何被配置、权限、server 边界、snapshot、trace 和 evaluation harness 治理。

Coding Agent Comparison

一句话定义：coding agent comparison 是把 Codex、Claude Code、OpenCode、Cursor、Aider、SWE-agent、OpenHands 等系统按公开证据、产品入口、上下文、工具、权限、编辑、评估和治理拆开比较的方法。

面试表达：横向比较 coding agent 时，先比较 harness 和目标场景，再比较模型体验；闭源内部 planner、prompt 和路由策略不能从外部资料反推成事实。

Product Surface Coverage

一句话定义：product surface coverage 衡量一个 coding agent 覆盖 CLI、IDE、Desktop、Web / Cloud、SDK、CI / PR bot、GUI、Enterprise 等入口的程度。

边界：入口多不等于更强，只说明可服务的工作流更广；日常 IDE、企业平台和研究 benchmark 应采用不同权重。

Context Source Coverage

一句话定义：context source coverage 衡量系统能否稳定接入 repo、当前选区、issue / PR、规则、memory、terminal、test output、MCP result、external docs 和 session summary 等上下文来源。

Tool Execution Coverage

一句话定义：tool execution coverage 衡量系统覆盖 read、search、edit、patch、shell、git、test runner、MCP、custom tools、browser、API 等可执行工具的程度。

Permission Governance Coverage

一句话定义：permission governance coverage 衡量工具执行是否绑定 allow / ask / deny、workspace 边界、sandbox、RBAC、网络控制、确认和审计策略。

Edit Recovery Coverage

一句话定义：edit recovery coverage 衡量系统是否支持 patch、diff、review、IDE apply、snapshot、revert、session resume 和 rollback 等编辑恢复机制。

Agent Eval Readiness

一句话定义：agent eval readiness 衡量 coding agent 是否具备 task trace、lint / test、regression、benchmark、trajectory、safety eval 和报告导出等评估准备能力。

Enterprise Governance Coverage

一句话定义：enterprise governance coverage 衡量系统是否覆盖 RBAC、audit、enterprise policy、managed settings、account policy、project policy 和成本 / 安全治理。

Cross-Agent Risk Governance

一句话定义：cross-agent risk governance 衡量 shell、cloud workspace、MCP、custom tools、server API、browser、hooks 和 enterprise integration 等风险面是否有对应硬权限和审计。

Coding Agent Comparison Gate

一句话定义：coding agent comparison gate 是基于权限治理、评估准备度和高风险治理的横向比较门禁，用来防止“总分高但治理不足”的系统被误判为可上线。

面试表达：企业落地不能只看 coding agent demo 是否顺滑，而要证明权限、评估、审计、风险边界和成本治理能随场景成立。

MCP Background Audit

一句话定义：MCP background audit 是检查一个方案是否真正解决 MCP 出现背景中的集成爆炸、上下文对象标准化、host / client / server 边界、本地上下文控制、跨客户端复用、治理接入和 MCP / A2A 区分问题。

工程价值：它防止把“包装几个 HTTP API 成工具”误判为完整 MCP 方案。

Direct Integration Count

一句话定义：direct integration count 是没有统一协议时，host application、外部系统和能力类型两两适配带来的集成数量。

面试表达：如果有 H 个 host、 S 个系统、 K 类能力，粗略数量是 $|H| * |S| * |K|$ 。

MCP Integration Count

一句话定义：MCP integration count 是 host 侧实现 MCP Client、外部系统侧暴露 MCP Server 后的近似适配数量。

工程价值：它把多对多集成变成 host-to-protocol 和 system-to-protocol 两类适配。

Integration Reduction

一句话定义：integration reduction 衡量 MCP 风格标准化连接相对直接两两集成减少了多少适配工作。

边界：这个指标是直觉模型，不代表真实成本只由数量决定；权限、安全、trace、eval 和运维仍然需要建设。

Capability Model Coverage

一句话定义：capability model coverage 衡量方案是否同时建模 tools、resources、prompts，以及后续可能的 roots、sampling、elicitation 等协议能力。

常见事故：只把 MCP 理解成 tool calling，忽略资源读取和可复用 prompt。

Context Object Coverage

一句话定义：context object coverage 衡量方案是否把文件、数据库记录、网页、IDE 上下文、Git diff、知识库和 prompt 模板作为模型上下文对象治理。

工程价值：MCP 的核心不是多一种函数调用，而是把模型应用需要的上下文连接标准化。

Discovery Standardization

一句话定义：discovery standardization 衡量 client 是否能用统一方式发现 server 暴露的 tools、resources、prompts 和能力信息。

面试表达：没有 discovery，就会退回每个插件各自写 manifest、文档和适配器。

MCP Schema Contract Coverage

一句话定义：MCP schema contract coverage 衡量工具输入、资源 URI、prompt 参数、错误返回、能力声明和消息结构是否有稳定契约。

边界：schema 只能保证结构边界，不能替代业务权限、安全策略和结果可信度判断。

Host Server Boundary Clarity

一句话定义：host server boundary clarity 衡量系统是否清楚区分 LLM、Host Application、MCP Client、MCP Server 和外部系统的职责。

常见误区：说 MCP Server 直接连接模型。通常是 host / client 连接 server，再把能力转给模型使用。

Local Context Control

一句话定义：local context control 衡量本地文件、IDE、Git、浏览器 session、终端环境和本地数据库是否以授权、白名单、沙箱和审计方式暴露。

工程价值：本地上下文是 MCP 的重要价值，也是高风险面；不能默认让模型读取整个本机。

Cross Client Reuse

一句话定义：cross client reuse 衡量同一个 MCP Server 是否能被多个 host / client 复用，而不是绑定单个模型平台或插件系统。

面试表达：MCP 的生态价值来自开放协议和复用，而不是每个平台再造一套私有插件格式。

Governance Boundary Clarity

一句话定义：governance boundary clarity 衡量方案是否明确 MCP 只提供连接协议，不替代企业 Tool Registry、权限、安全、trace、eval、灰度和回滚。

常见事故：接入 MCP Server 后绕过企业工具平台，导致权限、审计和发布门禁缺失。

MCP Trace Eval Readiness

一句话定义：MCP trace eval readiness 衡量 MCP tool / resource / prompt 使用是否能进入 trace、replay、eval、版本捕获和安全回归。

工程价值：MCP 接入后仍要能解释“哪个 server 暴露了什么能力、模型用了什么、结果怎样影响最终回答”。

MCP A2A Distinction

一句话定义：MCP A2A distinction 衡量回答是否能区分 MCP 的 Agent-to-Tool / Agent-to-Context 和 A2A 的 Agent-to-Agent 通信。

面试表达：MCP 和 A2A 可以协同，但解决的问题不同，不能混成一个“Agent 协议”泛称。

MCP Background Gate

一句话定义：MCP background gate 是基于集成减少、能力模型、上下文对象、发现、schema、host-server 边界、本地控制、复用、治理、trace/eval 和 A2A 区分的综合理解门禁。

面试表达：能讲清 MCP 背景，应该先讲为什么需要标准化连接，再讲它不解决什么，而不是只背 tools / resources / prompts 三个名词。

MCP Concept Audit

一句话定义：MCP concept audit 是检查 Host、Client、Server、Tools、Resources、Prompts、Transport、Lifecycle、Roots、Sampling 和企业治理映射是否被正确建模的审计过程。

工程价值：它防止把 MCP 学成名词表，忽略 Host 策略、Client / Server 边界、资源范围、prompt 审查和 trace/eval 证据。

Host Policy Ownership

一句话定义：host policy ownership 衡量 Host 是否承担用户体验、权限过滤、能力投影、用户确认、上下文预算和结果回填策略。

常见误区：把 MCP Server 发现到的所有能力直接给模型，而不是由 Host 按任务、用户和风险筛选。

MCP Client Server Boundary

一句话定义：MCP client server boundary 衡量系统是否清楚区分 Host 中的 MCP Client 和暴露外部能力的 MCP Server。

面试表达：MCP Client 不是 LLM，MCP Server 也通常不直接连接模型。

Server Capability Declaration

一句话定义: server capability declaration 衡量 MCP Server 是否清楚声明支持的 tools、resources、prompts、roots、sampling、logging、progress 和 cancellation 等能力。

工程价值: 能力声明让 Host 不必硬编码 server 能力, 也让权限和审计有对象可绑定。

MCP Tool Schema and Result

一句话定义: MCP tool schema and result 衡量 MCP tool 是否有 name、description、input schema、结果结构、错误语义、副作用说明和 trace 字段。

边界: MCP tool 是 Server 暴露给 Client 的能力, Host 还要决定是否投影给模型。

MCP Resource URI Metadata

一句话定义: MCP resource URI metadata 衡量 resource 是否有稳定 URI、类型、metadata、权限范围、内容大小和新鲜度信息。

工程价值: Resource 是上下文对象, 不应被随意设计成任意动作; 文件、文档、网页和数据库记录都需要可寻址和可治理。

MCP Prompt Argument Review

一句话定义: MCP prompt argument review 衡量 server 提供的 prompt 是否有参数 schema、适用场景、版本、审查状态和 Host 侧信任策略。

常见事故: 把外部 server 提供的 prompt 自动当成可信系统提示, 导致不合适指令进入模型上下文。

MCP Lifecycle Negotiation

一句话定义: MCP lifecycle negotiation 衡量 Client / Server 是否完成 initialize、capability negotiation、version capture、ready state 和关闭处理。

面试表达: MCP 不是一次普通函数调用, 而是一段有连接生命周期和能力协商的协议交互。

MCP Transport Policy

一句话定义: MCP transport policy 衡量 stdio、streamable HTTP、SSE 或其他传输方式是否绑定认证、网络边界、进程隔离、超时和审计策略。

边界: Transport 不改变 MCP 抽象, 但会改变安全、部署、延迟和运维风险。

MCP Roots Boundary

一句话定义: MCP roots boundary 衡量 Host 暴露给 Server 的工作区根、文件范围、项目边界和资源访问约束是否明确。

工程价值: Roots 对本地文件、代码仓库和 IDE 场景尤其关键, 不能默认让 server 读取整个本机。

MCP Sampling Control

一句话定义: MCP sampling control 衡量 server 请求 Host 进行模型采样时是否经过权限、模型选择、输入脱敏、用户确认和 trace 控制。

常见事故: 允许 server 任意触发模型调用, 间接扩大成本、隐私和 prompt injection 风险。

Host Capability Filtering

一句话定义：host capability filtering 衡量 Host 是否按用户、场景、权限、风险、上下文预算和 provider 能力过滤 MCP Server 暴露的 tools、resources 和 prompts。

面试表达：发现能力不等于暴露能力，更不等于模型可以自动使用全部能力。

MCP Context Budget

一句话定义：MCP context budget 衡量 system、user、tool descriptions、resources、prompts 和 tool results 投影到模型上下文后的 token 预算。

工程价值：连接更多 MCP Server 会快速膨胀上下文，Host 必须做候选选择、摘要、压缩和延迟加载。

MCP Concept Gate

一句话定义：MCP concept gate 是基于 Host 策略、Client / Server 边界、Server 能力、Tool schema、Resource URI、Prompt 审查、Lifecycle、Transport、Roots、Sampling、Host 过滤和 trace/eval 映射的综合概念门禁。

面试表达：讲 MCP 基本概念时，要能说明每个对象是谁负责、怎样被发现、怎样进入模型上下文，以及哪里需要治理。

MCP Server Implementation Audit

一句话定义：MCP server implementation audit 是检查最小 MCP Server 是否完成 metadata、capabilities、tools/list、tools/call、schema validation、handler、resources、prompts、transport、错误处理、安全和 trace 的实现审计。

工程价值：它防止把最小 server 简化成一个裸函数，忽略可发现性、结构化错误、资源边界、prompt 审查和 Host 接入证据。

MCP Server Metadata Readiness

一句话定义：MCP server metadata readiness 衡量 server 是否有稳定 name、version、协议版本捕获和初始化返回。

面试表达：metadata 让 Host 和 trace 能知道当前连接的是哪个 server 版本，而不是只知道调用了某个函数。

MCP Server Capability Declaration

一句话定义：MCP server capability declaration 衡量 server 是否只声明自己真实支持的 tools、resources、prompts 和其他能力。

常见事故：声明了 resources 或 prompts 但没有实现 list / read / get，导致 Host 侧能力发现和调用链路不一致。

MCP Tool Registry Readiness

一句话定义：MCP tool registry readiness 衡量 server 暴露的每个 tool 是否有 name、description、input schema、handler 和可测试样例。

工程价值：Host 依赖 tools/list 决定给模型投影哪些工具，registry 不完整会直接影响工具选择和参数生成。

MCP Strict Schema Coverage

一句话定义：MCP strict schema coverage 衡量 tool input schema 是否覆盖 required、type、enum、range、additionalProperties 和业务校验入口。

常见误区：只写自然语言 description，不写严格 schema，导致模型传入缺字段、错类型或额外字段后直接执行。

MCP Handler Execution Coverage

一句话定义：MCP handler execution coverage 衡量每个 tool handler 是否有成功路径、异常路径、超时路径和 mock / 最小复现测试。

面试表达：最小实现可以是 mock 逻辑，但仍要证明 handler 可以被协议调用并返回可识别结果。

MCP Argument Validation Coverage

一句话定义：MCP argument validation coverage 衡量 server 是否在执行 handler 前校验 arguments，而不是相信模型输出。

工程价值：schema validation 是最小 server 的硬门禁，权限、业务规则和高风险确认还需要在生产层继续加强。

MCP Structured Result Coverage

一句话定义：MCP structured result coverage 衡量工具成功结果是否有稳定 content、结构化字段、大小限制和 Host 可回填格式。

边界：结构化结果不等于原样塞进模型上下文，Host 仍要做投影、脱敏、压缩和 trust boundary 标注。

MCP Structured Error Coverage

一句话定义：MCP structured error coverage 衡量 unknown tool、invalid arguments、timeout、permission denied、upstream error 和 internal error 是否被映射成结构化错误。

常见事故：handler 抛异常导致 server 崩溃，或者把内部堆栈、密钥、数据库地址泄露给模型上下文。

MCP Resource Scope Coverage

一句话定义：MCP resource scope coverage 衡量 resources/list 和 resources/read 是否限制在授权 URI、roots 或 workspace 范围内。

工程价值：文件系统、代码仓库和知识库 server 不能默认读取整个本机或整个企业数据域。

MCP Prompt Template Coverage

一句话定义：MCP prompt template coverage 衡量 prompts/list 和 prompts/get 是否有参数、版本、审查状态和 Host 信任策略。

常见事故：把外部 server 提供的 prompt 自动当成系统指令，绕过 Host 侧 prompt 审查和上下文策略。

MCP Server Transport Policy Coverage

一句话定义：MCP server transport policy coverage 衡量 stdio、Streamable HTTP、SSE legacy 等传输方式是否绑定认证、授权、网络边界、超时和审计。

面试表达：stdio 更像本地进程边界，远程 HTTP 类 server 必须额外说明认证、TLS、限流和审计。

MCP Host Connection Readiness

一句话定义：MCP host connection readiness 衡量 Host / Client 是否完成配置、initialize、能力发现、工具投影、调用、结果回填和错误处理 smoke test。

工程价值：server 自测通过不代表 Host 接入成功，真正闭环要看 Host 能否发现、过滤、调用并回填结果。

MCP Server Safety Baseline

一句话定义：MCP server safety baseline 衡量最小 server 是否至少具备最小权限、严格 schema、路径限制、错误脱敏、输出大小限制和高风险动作确认。

边界：它不是完整安全体系，但能防止 demo server 直接变成泄露文件、执行命令或无认证远程调用的入口。

MCP Server Trace Readiness

一句话定义：MCP server trace readiness 衡量 server 是否记录 tool name、arguments 摘要、结果状态、错误码、版本、transport 和 trace id 等字段。

工程价值：没有 trace，就很难评估工具选择、参数错误、资源越界、权限拒绝和 handler 异常来自哪里。

MCP Server Gate

一句话定义：MCP server gate 是基于 metadata、capability declaration、tool registry、schema、handler、argument validation、result、error、resource、prompt、transport、Host 接入、安全和 trace 的综合实现门禁。

面试表达：最小 MCP Server 的合格标准是“协议闭环可发现、可调用、可校验、可失败、可治理”，不是“函数能返回天气”。

MCP Tool Function Calling Comparison Audit

一句话定义：MCP tool function calling comparison audit 是检查回答或系统设计是否清楚区分模型 API 层 Function Calling 与 MCP Client / Server 层能力连接的审计过程。

工程价值：它防止把 MCP Tool 误讲成另一种 provider tool 字段，也防止把 Function Calling 误讲成能替代 resources、prompts、server lifecycle 和跨客户端发现的完整协议。

Protocol Layer Clarity

一句话定义：protocol layer clarity 衡量系统是否清楚区分 Host 与模型 provider 之间的 function calling 协议，以及 MCP Client 与 MCP Server 之间的 MCP 协议。

面试表达：模型返回 tool call 通常是 provider API 层事件，Host 再把它映射成 MCP tools/call 才会进入 MCP Server。

Tool Discovery Boundary

一句话定义：tool discovery boundary 衡量工具是 Host 静态定义、Registry 导入，还是由 MCP Client 从 MCP Server 动态发现。

常见误区：发现到 MCP Server 暴露的 tool 不等于可以直接给模型使用，Host 仍要过滤、审批和投影。

MCP Capability Scope Coverage

一句话定义：MCP capability scope coverage 衡量回答是否覆盖 MCP 的 tools、resources、prompts，而不是只讨论 tool schema。

工程价值：MCP 的差异不只是工具调用，还包括文件、数据库、网页、知识库资源和可复用 prompt 模板进入统一 server 能力模型。

MCP Execution Boundary Clarity

一句话定义：MCP execution boundary clarity 衡量系统是否清楚说明 tool execution 从模型 tool call 到 Host runtime，再到 MCP Client 调用 MCP Server 的责任链。

常见事故：把 MCP Server 说成直接被模型调用，忽略 Host 的权限、确认、结果回填和 trace 责任。

MCP Projection Mapping Coverage

一句话定义：MCP projection mapping coverage 衡量 MCP Tool 是否被正确映射成 provider function calling tool，并能把 provider tool call 映射回 MCP tools/call。

面试表达：MCP Tool 可以成为 Function Calling 的上游来源，但投影过程需要 Host / adapter 维护 name、schema、source、权限和 trace 映射。

Adapter Separation Coverage

一句话定义：adapter separation coverage 衡量 Provider Adapter 和 MCP Adapter 是否分离，避免把某家模型 API 字段和 MCP 协议字段混在业务代码里。

工程价值：Provider Adapter 处理模型 API 格式，MCP Adapter 处理 MCP tools/list、tools/call、resources 和 prompts，内部 Registry 负责统一治理。

MCP Lifecycle Version Awareness

一句话定义：MCP lifecycle version awareness 衡量系统是否考虑 MCP Server initialize、capability discovery、版本变化、server 不可用和 schema 变化。

边界：Function Calling 工具可以随 Host 发布，MCP Server 可以独立升级和重启，因此生命周期复杂度更高。

MCP Governance Registry Import

一句话定义：MCP governance registry import 衡量 MCP Server 暴露的 tools 是否进入企业 Tool Registry，绑定 owner、risk、permission、eval、trace 和发布策略。

常见误区：把 MCP tools 直接透传给模型，绕过 Registry、Router、Executor、Permission 和 Safety Guard。

MCP Security Boundary Enforcement

一句话定义：MCP security boundary enforcement 衡量系统是否同时治理模型 tool call、MCP server 连接、本地资源范围、远程认证和 tool result trust boundary。

工程价值：MCP 不是减少安全边界，而是在 function calling 的 Host runtime 安全之外增加 server 连接层安全。

MCP Use Case Selection Fit

一句话定义：MCP use case selection fit 衡量系统是否能判断简单应用工具适合直接 function calling，跨客户端、resources / prompts、本地上下文和插件生态更适合 MCP。

面试表达：不要为了协议而协议，小系统直接 function calling 往往更简单。

MCP Error Surface Separation

一句话定义：MCP error surface separation 衡量系统是否区分 provider tool call 错误、MCP transport / protocol 错误、server 不可用和工具业务错误。

工程价值：不同错误要给模型不同 tool result，也要在 trace 中归因到不同层，否则无法排查。

MCP Latency Availability Tradeoff

一句话定义：MCP latency availability tradeoff 衡量系统是否考虑 MCP Client / Server 通信、进程间或网络开销、server 冷启动、缓存 tools/list 和降级策略。

边界：MCP 带来复用和解耦，但高频低延迟工具未必适合远程 MCP server。

MCP Function Calling Gate

一句话定义：MCP function calling gate 是基于协议层次、工具发现、能力范围、执行边界、投影映射、adapter 分离、生命周期、Registry 治理、安全、选型、错误面和可用性 trade-off 的综合对比门禁。

面试表达：能讲清 MCP 与 Function Calling 的区别，应该能说明“二者常组合使用”，而不是把它们说成互相替代或完全等价。

MCP Resource Exposure Audit

一句话定义：MCP resource exposure audit 是检查文件、数据库、网页、日志和 RAG 片段等外部上下文是否以 URI、metadata、权限、内容边界、引用和 trace 形式安全暴露的审计过程。

工程价值：它防止把 Resources 简化成“任意读文本”，也防止把外部内容不经权限、预算和可信度标注直接塞进模型上下文。

Resource URI Validity

一句话定义：resource URI validity 衡量资源 URI 是否有稳定 scheme、逻辑身份和可追踪引用，而不是泄露底层数据库地址或系统路径。

面试表达：企业系统更适合暴露 customer://tenant-a/C123/profile 这类逻辑 URI，而不是直接把生产数据库连接串交给模型应用。

Resource Metadata Completeness

一句话定义：resource metadata completeness 衡量资源是否包含 name、title、description、mimeType、size、last modified、etag、sensitivity 和 trust level 等决策字段。

工程价值：没有 metadata，Host 就很难决定是否读取、如何压缩、是否脱敏、能否引用和是否需要重新获取。

Resource List Filtering Boundary

一句话定义：resource list filtering boundary 衡量 resources/list 是否按租户、权限、目录、标签和分页过滤资源，避免泄露资源存在性。

常见误区：把整个文件树、全部客户 ID 或全量日志窗口列给模型，会同时造成上下文爆炸和权限泄露。

Resource Read Scope Enforcement

一句话定义：resource read scope enforcement 衡量 resources/read 是否检查 URI 合法性、资源存在性、大小、mimeType、敏感级别和读取范围。

工程价值：能列出 resource 不代表能读取全文，能读取 metadata 也不代表能把内容传给模型。

MCP Roots Containment

一句话定义：MCP roots containment 衡量文件资源是否被限制在 Client 授权给 Server 的 workspace / root 范围内，并正确处理规范化路径和路径逃逸。

常见事故：file:///workspace/../../etc/passwd 这类路径如果只做字符串前缀判断，就可能绕过文件边界。

MCP Resource Permission Enforcement

一句话定义：MCP resource permission enforcement 衡量资源读取是否执行租户、对象、字段和数据流转权限。

面试表达：Resources 的权限不只是 read / write，还包括能否看到存在性、读取 metadata、读取字段、进入模型上下文和流向其他工具。

MCP Resource Field Projection Safety

一句话定义：MCP resource field projection safety 衡量数据库记录、日志和结构化文档是否只返回任务需要且用户有权看到的字段。

工程价值：把整行数据库或完整生产日志作为 resource 返回，通常会泄露手机号、密钥、请求体或内部评分。

Resource Context Budget Control

一句话定义：resource context budget control 衡量资源内容是否经过 chunk、top K、摘要、截断或按需读取，避免完整大文档直接占满上下文窗口。

面试表达：Resource 暴露给 Host 不等于完整进入 prompt，Host 仍要做上下文预算和证据选择。

Resource Citation Traceability

一句话定义：resource citation traceability 衡量资源内容是否保留 URI、section、line range、chunk id、retrieved at 或 trace id。

工程价值：没有 citation，模型回答无法复查来源，RAG、IDE 修复、日志排障和企业制度问答都会难以追责。

Untrusted Resource Labeling

一句话定义：untrusted resource labeling 衡量网页、用户上传文件和第三方文档是否被标注为不可信数据，而不是被当作系统指令。

常见误区：网页正文里的“忽略系统指令”只是资源内容，不能提升为 Host 或系统级指令。

Resource Freshness Version Awareness

一句话定义：resource freshness version awareness 衡量资源是否携带 last modified、etag、version、retrieved at 或订阅状态，帮助 Host 判断上下文是否过期。

工程价值：IDE 当前文件、Git diff、日志和数据库记录会变化，基于旧资源回答需要被标注或重新读取。

Resource Template Boundary

一句话定义：resource template boundary 衡量 resources/templates/list 暴露的 URI 模板是否有参数 schema、范围限制和权限校验。

常见误区：file:/// {path} 或 customer:/// {id} 模板如果没有参数边界，会变成路径逃逸或资源枚举入口。

Resource Subscription Awareness

一句话定义：resource subscription awareness 衡量系统是否知道资源列表或资源内容可能变化，并能处理变更通知、重新读取和缓存失效。

面试表达：订阅不是必须的最小功能，但动态上下文系统至少要知道 version / etag / list changed 的处理策略。

Resource Trace Eval Readiness

一句话定义：resource trace eval readiness 衡量资源读取链路是否记录 user、tenant、uri、metadata、permission decision、projection、token budget、citation、freshness 和 final use。

工程价值：没有 resource trace, 就无法评估越权读取、引用缺失、过期证据、上下文超预算和 prompt injection 边界失败。

MCP Resource Gate

一句话定义：MCP resource gate 是基于 URI、metadata、list/read、roots、权限、字段投影、上下文预算、引用、可信度、freshness、template、subscription 和 trace 的综合上线门禁。

面试表达：MCP Resources 的合格标准不是“能读文本”，而是“可寻址、可授权、可压缩、可引用、可防注入、可追踪”。

MCP Prompt Exposure Audit

一句话定义：MCP prompt exposure audit 是检查 MCP Server 暴露的 prompts 是否可发现、可参数化、可渲染、可降权、可版本化、可评估、可授权和可追踪的审计过程。

工程价值：它防止把 Prompt 简化成字符串拼接，也防止外部 Server 借 prompt 模板绕过 Host policy、工具权限和资源权限。

Prompt Capability Declaration

一句话定义：prompt capability declaration 衡量 MCP Server 是否在 initialize 阶段明确声明 prompts 能力以及 list changed 支持情况。

面试表达：Host 不能假设所有 Server 都有 prompts，也不能假设 prompt 列表不会变化。

Prompt Discovery Coverage

一句话定义：prompt discovery coverage 衡量 prompts/list 是否返回过滤后的 name、title、description、arguments、owner、risk、version 和审核状态。

工程价值：发现 prompt 不是把所有模板暴露给模型，而是让 Host 和用户看见可用、授权、已审核的任务入口。

Prompt Argument Schema Coverage

一句话定义：prompt argument schema coverage 衡量 prompt 参数是否有 name、description、required、type 和边界说明。

常见误区：只写 input 一个大文本参数，会让注入、防错、权限和 eval 都很难做。

Prompt Required Argument Validation

一句话定义：prompt required argument validation 衡量 prompts/get 在渲染前是否拦截缺失必填参数和错误类型。

工程价值：Prompt 参数错误如果进入模型，往往会变成幻觉式补全或错误工具调用。

Prompt Rendering Safety

一句话定义：prompt rendering safety 衡量用户参数是否被转义、包装为数据并放在低权限位置，而不是直接拼进高优先级指令。

面试表达：Prompt 模板参数不是可信指令，应该显式标注为 user-provided data。

Prompt Role Boundary Enforcement

一句话定义：prompt role boundary enforcement 衡量 MCP Prompt 返回的 message role 是否不能提升到 Host system policy 之上。

常见事故：任意 MCP Server 返回 system role 并要求忽略安全策略，Host 如果直接采用就等于把控制权交给外部模板。

Prompt Dependency Alignment

一句话定义：prompt dependency alignment 衡量 prompt 依赖的 tools、resources、permissions 和上下文预算是否被声明、检查和记录。

工程价值：一个故障排查 prompt 能建议读日志，但不能绕过日志资源权限或生产工具确认流程。

Prompt Version Governance

一句话定义：prompt version governance 衡量 prompt 是否记录 version、template hash、owner、changelog 和发布状态。

面试表达：Prompt 改动就是模型行为改动，生产环境不能静默修改模板。

Prompt Eval Binding

一句话定义：prompt eval binding 衡量 prompt 是否绑定 golden cases、安全回归、输出格式回归和审批门禁。

工程价值：代码审查、客服回复和故障排查 prompt 都会影响最终行为，必须像工具 schema 一样做回归测试。

Prompt Permission Enforcement

一句话定义：prompt permission enforcement 衡量用户能否发现和使用某个 prompt 是否受角色、租户、场景和风险策略控制。

常见误区：用户能使用某个 prompt，不等于能访问 prompt 建议使用的全部 resources 和 tools。

Prompt Injection Containment

一句话定义：prompt injection containment 衡量 prompt 模板和参数是否阻断提升权限、绕过 Host policy、泄露隐藏上下文和诱导危险工具调用的注入路径。

工程价值：MCP Prompt 既可能被模板作者写坏，也可能被用户参数污染，二者都要审查。

Prompt User Control

一句话定义：prompt user control 衡量 Host 是否向用户展示 prompt 来源 server、owner、风险等级、审核状态、版本和用途，而不是把模板伪装成平台内置能力。

面试表达：Prompt 是可选工作流入口，用户和 Host 都应该知道它来自哪里、能做什么、风险是什么。

Prompt List Change Awareness

一句话定义：prompt list change awareness 衡量系统是否处理 notifications/prompts/list_changed、缓存失效、版本变化和重新发现。

工程价值：Server 更新 prompt 后，Host 如果继续使用旧缓存，可能错过安全修复或引用已下线模板。

Prompt Trace Readiness

一句话定义：prompt trace readiness 衡量每次 prompt 使用是否记录 prompt name、version、template hash、arguments hash、user、tenant、rendered message hash、依赖工具资源和最终输出。

工程价值：没有 prompt trace，就无法复盘某次错误回答到底来自模板变化、参数污染、权限缺口还是模型本身。

MCP Prompt Gate

一句话定义：MCP prompt gate 是基于 capability、discovery、argument schema、required validation、rendering、role boundary、dependency、version、eval、permission、injection、user control、list change 和 trace 的综合上线门禁。

面试表达：MCP Prompts 的合格标准不是“模板能渲染”，而是“可发现、可参数化、可降权、可评估、可授权、可追踪”。

MCP Security Sandbox Audit

一句话定义：MCP security sandbox audit 是检查 MCP Host、Client、Server、tools、resources、prompts、authorization、roots、sandbox、数据流和 trace 是否形成防御闭环的审计过程。

工程价值：它防止把 MCP 安全误解成“提示模型别乱来”，而是把能力连接、权限、沙箱、确认、数据流和审计都放进可验证门禁。

Server Connection Governance

一句话定义：server connection governance 衡量 MCP Server 是否经过 allowlist、签名、版本固定、发布者校验、能力展示和用户/管理员批准。

面试表达：连接一个 MCP Server 不是普通函数 import，而是接入一个可读资源、可执行工具和可提供 prompt 的能力提供方。

Authorization Token Binding

一句话定义：authorization token binding 衡量远程 MCP Server 的 token 是否绑定 audience、scope、过期时间、session 和资源服务器，避免 confused deputy 和 token passthrough。

工程价值：Host 不能把上游 token 原样透传给任意 Server，Server 也不能接受 audience 不匹配的 token。

Scope Minimization

一句话定义：scope minimization 衡量 MCP 授权 scope 是否只覆盖当前能力需要的最小读写范围。

常见误区：为了省事给本地文件、网络、shell 和企业数据统一 broad scope，会让任何 prompt injection 都变成高危事件。

Roots Sandbox Containment

一句话定义：roots sandbox containment 衡量文件系统访问是否被限制在 Host 授权 roots 内，并正确处理 canonical path、路径逃逸、symlink 和目录权限变化。

工程价值：文件 MCP Server 没有 roots，几乎等于把用户电脑当作模型可读资源池。

File Secret Blocking

一句话定义：file secret blocking 衡量 .env、SSH key、浏览器 cookie、云凭证、数据库密码和超大文件是否被 denylist、DLP 和输出限制拦截。

面试表达：即使文件在 workspace 内，也不代表可以进入模型上下文或流向外部工具。

Network SSRF Protection

一句话定义：network SSRF protection 衡量 URL / browser 类 MCP Server 是否阻断 localhost、私有网段、metadata 地址、异常重定向、非 allowlist 域名和过大响应。

工程价值：能 fetch URL 的 Server 必须把网络边界当作安全边界，而不是把任意 URL 当普通 resource。

MCP Shell Sandbox Enforcement

一句话定义：MCP shell sandbox enforcement 衡量 shell / code execution 工具是否只在非特权、限时、限资源、受限网络、受限文件系统的临时沙箱中运行。

常见事故：让模型生成的命令直接跑在宿主机 shell 上，会把 Agent 错误从回答质量问题放大成系统安全问题。

Prompt Injection Data Boundary

一句话定义：prompt injection data boundary 衡量 resource、web page、tool result 和第三方 prompt 内容是否被标注为 data，不能覆盖 Host policy 或触发危险工具。

工程价值：间接 prompt injection 的关键防线不是模型自觉，而是 Host 和 Executor 的权限、工具候选和数据流策略。

High Risk Confirmation

一句话定义：high risk confirmation 衡量写文件、删文件、发邮件、外发数据、执行 shell、生产变更和权限修改是否有明确用户确认或管理员审批。

面试表达：确认界面应展示动作、目标、数据摘要、风险和不可逆性，而不是只问“是否继续”。

Sensitive Data Flow Control

一句话定义：sensitive data flow control 衡量高敏资源是否禁止流向 external sink，并在必要时经过 DLP、摘要展示、人工确认和审计。

工程价值：多 MCP Server 组合时，最常见风险是从文件 / 数据库 / 浏览器读取敏感数据，再通过邮件、网页发布或外部 API 发出。

Local Credential Isolation

一句话定义：local credential isolation 衡量 MCP Server 进程是否避免继承不必要环境变量、云凭证、SSH key、浏览器状态和本地 token。

常见误区：本地 Server 默认继承用户 shell 环境，会把大量秘密暴露给工具执行和日志输出。

MCP Tenant Isolation

一句话定义：MCP tenant isolation 衡量远程 MCP Server 是否从认证上下文注入 tenant、按租户过滤资源和工具、隔离缓存 key、日志和审计。

面试表达：不要相信模型参数里的 tenant_id，租户身份必须来自可信 auth / runtime context。

Server Supply Chain Governance

一句话定义：server supply chain governance 衡量 MCP Server 包、配置、版本、hash、签名、自动更新和能力变更是否纳入企业供应链治理。

工程价值：MCP Server 是可执行能力提供方，未审核自动升级可能引入新的工具、资源范围或网络行为。

MCP Security Trace Readiness

一句话定义：MCP security trace readiness 衡量 trace 是否记录 server、version、user、tenant、token scope、roots、tool/resource/prompt、policy decision、confirmation、data flow 和 block reason。

工程价值：没有安全 trace，MCP 事故只能靠猜，无法复盘越权、注入、数据外发和误执行来自哪一层。

MCP Security Eval Coverage

一句话定义：MCP security eval coverage 衡量安全回归集是否覆盖路径逃逸、敏感文件、SSRF、shell、prompt injection、prompt 提权、数据外发、跨租户、无确认写操作和 allowlist 绕过。

面试表达：MCP 安全 eval 不能只看最终回答，要检查 trace 和实际执行动作。

MCP Security Gate

一句话定义：MCP security gate 是基于连接治理、授权 token、scope、roots、文件、网络、shell、prompt injection、确认、数据流、凭证、租户、供应链、trace 和 eval 的综合上线门禁。

面试表达：MCP 安全来自 Host 侧强策略和运行时边界，不来自模型“知道自己不该做什么”。

MCP Integration Surface Audit

一句话定义：MCP integration surface audit 是把 IDE、知识库、数据库、浏览器、终端和跨 Server 数据流放进同一张审计表，检查它们是否都被 Host 统一注册、路由、授权、限流、追踪和评估。

面试表达：MCP 集成不是“多连几个 Server”，而是把不同系统能力纳入同一个 capability registry、permission engine、context builder 和 trace/eval 闭环。

MCP Capability Registration Coverage

一句话定义：MCP capability registration coverage 衡量每个 MCP tool / resource / prompt 是否有 namespace、kind、owner、version、risk、permission policy、timeout、预算和 trace 字段。

工程价值：未注册能力绕过 Host 治理后，会造成工具选择、权限、审计和回滚都不可控。

MCP Namespace Isolation Coverage

一句话定义：MCP namespace isolation coverage 衡量不同 Server 和能力名称是否避免冲突，并能按租户、环境、版本和权限范围隔离。

工程价值：IDE、知识库、数据库和终端都可能暴露 search、read 或 run，没有 namespace 会导致误路由和错误授权。

MCP IDE Context Routing

一句话定义：MCP IDE context routing 衡量 Host 是否能为编码任务选择当前文件、选区、diff、诊断、符号、测试失败和相关文档，而不是把全仓库直接塞进上下文。

面试表达：IDE MCP 的关键不只是读写文件，而是把代码上下文选择、patch 预览、测试反馈和用户改动保护做成闭环。

MCP Knowledge Citation Traceability

一句话定义：MCP knowledge citation traceability 衡量知识库检索结果是否带文档 URI、片段位置、权限、更新时间、相关性分数和最终答案引用。

工程价值：知识库 MCP 如果不能追溯证据，就无法区分模型幻觉、检索错误、权限错误和过期文档。

MCP Database Query Governance

一句话定义：MCP database query governance 衡量数据库 Server 是否只暴露只读、参数化、表级 allowlist、字段投影、成本限制、超时和审计后的查询能力。

面试表达：数据库 MCP 不应默认暴露任意 SQL；更稳的方式是 schema resource、只读查询工具和业务语义工具。

MCP Browser Action Governance

一句话定义：MCP browser action governance 衡量浏览器 Server 是否把网页标成不可信 Resource，并对点击、输入、提交、上传、下载、跨域跳转等动作做预览、域名策略和确认。

工程价值：浏览器继承用户 session，网页内容又不可信，所以浏览器 MCP 的核心是防止外部页面指令触发高风险操作。

MCP Terminal Sandbox Governance

一句话定义：MCP terminal sandbox governance 衡量终端能力是否被拆成测试、构建、格式化等语义工具，并受工作目录、文件系统、网络、环境变量、超时、资源配额和输出限制约束。

面试表达：终端 MCP 是高风险能力，不能把无限制 shell 当作普通工具交给模型。

MCP Integration Context Budget Control

一句话定义：MCP integration context budget control 衡量 IDE、知识库、数据库 schema、浏览器页面和终端输出进入模型上下文前是否经过排序、截断、摘要、去重和预算控制。

工程价值：多 Server 场景很容易把上下文塞爆，预算失控会同时拉高成本、延迟和噪声。

MCP Cross Server Data Flow Control

一句话定义：MCP cross server data flow control 衡量一个 Server 的 Resource 或 Tool result 是否只能流向被策略允许的另一个 Server 或外部 sink。

工程价值：数据库结果、知识库机密、终端输出和浏览器页面不能随意互相传递，否则会出现跨系统泄露和间接注入。

MCP High Risk Approval Coverage

一句话定义：MCP high risk approval coverage 衡量写文件、提交表单、上传下载、运行终端、生产查询和代发数据等高风险动作是否展示 action preview 并得到显式确认。

面试表达：高风险动作的最终裁判应是 Host 策略和用户确认，不是模型自己的风险判断。

MCP Output Projection Coverage

一句话定义：MCP output projection coverage 衡量数据库、终端、浏览器和知识库返回内容是否经过字段投影、脱敏、摘要、引用和输出大小限制后才进入上下文。

工程价值：工具成功返回不等于结果可以原样给模型，输出治理是防泄露和防注入的重要边界。

MCP Integration Trace Readiness

一句话定义：MCP integration trace readiness 衡量 trace 是否记录 server、namespace、capability、actor、input summary、resource URI、decision、reason、approval、output projection 和 version。

工程价值：多 Server 集成出问题时，trace 是复盘路由、权限、数据流和最终回答的唯一可靠证据链。

MCP Integration Eval Coverage

一句话定义：MCP integration eval coverage 衡量评估集是否覆盖 IDE、知识库、数据库、浏览器、终端、跨 Server 数据流、上下文预算、审批、输出投影和 trace 缺失。

面试表达：MCP 集成 eval 不能只测最终回答，还要检查每一步外部能力调用是否符合策略。

MCP Integration Gate

一句话定义：MCP integration gate 是基于能力注册、namespace、IDE 上下文、知识库引用、数据库治理、浏览器动作、终端沙箱、上下文预算、跨 Server 数据流、审批、输出投影、trace 和 eval 的综合上线门禁。

面试表达：能连上 MCP Server 只是第一步，能证明 Host 对多个外部系统的上下文、权限和数据流有强治理，才是可上线集成。

A2A Background Audit

一句话定义：A2A background audit 是从背景层审计多 Agent 协作是否具备能力发现、任务委派、状态同步、消息结构、产物引用、上下文边界、权限、失败处理、trace 和 eval。

面试表达：A2A 的出现不是为了让 Agent 互相聊天，而是让 Agent 之间的能力、任务、状态、上下文、产物和信任关系可以被系统可靠管理。

Agent Card Completeness

一句话定义：agent card completeness 衡量远程 Agent 是否声明名称、版本、接口、skills、输入输出模式、认证授权、capabilities、owner 和治理信息。

工程价值：Agent Card 不完整时，调用方只能靠 prompt 或硬编码猜能力，服务发现、路由、权限和版本治理都会变脆。

Agent Discovery Readiness

一句话定义：agent discovery readiness 衡量调用方是否能按任务能力、版本、接口、权限和可用性发现合适的远程 Agent。

面试表达：A2A 服务发现不是找一个 URL，而是确认“谁能做、能不能被我调用、需要什么输入、能返回什么结果”。

Task Delegation Contract

一句话定义：task delegation contract 衡量 A2A 委派任务是否有 task id、context id、目标、期望输出、截止时间、幂等键、权限范围和回调或事件通道。

工程价值：没有任务契约，多 Agent 委派就会退化成一句“帮我做一下”，后续状态、重试、取消和审计都无从谈起。

A2A Task Lifecycle Coverage

一句话定义：A2A task lifecycle coverage 衡量委派任务是否覆盖 submitted、working、input required、auth required、completed、failed、canceled 和 rejected 等状态语义。

面试表达：input_required 不是失败，而是协议层需要支持的协商状态；把它混成错误码会破坏编排。

A2A Message Structure Coverage

一句话定义：A2A message structure coverage 衡量消息是否有 message id、role、part 类型、media type 和上下文归属，避免只传不可解析的自然语言字符串。

工程价值：结构化消息让上游 Agent 能区分用户输入、远程 Agent 澄清、事件更新和最终产物引用。

A2A Artifact Reference Coverage

一句话定义：A2A artifact reference coverage 衡量任务结果是否以 artifact id、URI、media type 和权限边界引用，而不是把大对象或敏感对象直接内联进消息。

工程价值：Artifact 引用可以配合权限、下载、校验、版本和审计，避免结果被复制到不该共享的上下文里。

A2A Context Boundary Control

一句话定义：A2A context boundary control 衡量传给下游 Agent 的上下文是否遵守最小必要原则、敏感信息脱敏、预算限制和可引用但不可复制的数据边界。

面试表达：A2A 默认不应该全量转发上游上下文；上下文最小化是防泄露、防注入和防噪声的重要边界。

A2A Permission Boundary

一句话定义：A2A permission boundary 衡量 requester、receiver、用户身份、认证 scope、授权范围和可继续委派权限是否清楚。

工程价值：远程 Agent 可能代表用户执行任务，身份和权限不清会导致跨租户访问、越权委派和责任边界模糊。

A2A MCP Distinction

一句话定义：A2A MCP distinction 衡量系统是否能区分 A2A 的 Agent-to-Agent 任务委派和 MCP 的 Agent-to-Tool / Agent-to-Context 能力接入。

面试表达：MCP 和 A2A 可以组合使用，但不能混为一谈；MCP 管工具资源，A2A 管 Agent 间协作。

A2A Failure Handling Coverage

一句话定义：A2A failure handling coverage 衡量任务是否支持超时、取消、失败状态、可重试边界、不可重试错误、幂等和人工接管。

工程价值：多 Agent 系统失败会跨 Agent 传播，协议必须让上游知道任务是失败、等待输入、被取消还是未知状态。

A2A Trace Readiness

一句话定义：A2A trace readiness 衡量 trace 是否记录 task id、context id、agent id、状态变更、message id、artifact id、权限决策、错误和版本。

工程价值：多 Agent 事故没有 trace 时很难判断错误来自能力发现、上下文泄露、权限越界、远程执行还是结果汇总。

A2A Eval Coverage

一句话定义: A2A eval coverage 衡量评估集是否覆盖 Agent Card 缺失、能力不匹配、状态跳跃、input required、上下文过度共享、权限缺失、Artifact 不合规、trace 缺失和 MCP/A2A 混淆。

面试表达: A2A eval 不能只测最终答案, 要测委派链路是否可发现、可执行、可失败、可追踪。

A2A Background Gate

一句话定义: A2A background gate 是基于 Agent Card、发现、任务契约、生命周期、消息、Artifact、上下文、权限、MCP 区分、失败处理、trace 和 eval 的背景层上线门禁。

面试表达: 如果这些基础项不过线, 系统最多是 multi-agent demo, 不应该被描述成可治理的 A2A 协作系统。

Agent Card Discovery Audit

一句话定义: agent card discovery audit 是用字段完整度、skill 声明、supported interface、权限、安全、版本缓存、服务发现、路由决策、extended card、trace 和 eval 审计远程 Agent 是否可发现、可匹配、可调用、可治理。

面试表达: Agent Card 不是介绍页, 而是远程 Agent 的服务化能力契约; 能被搜索到不等于能被安全调用。

Agent Card Field Completeness

一句话定义: agent card field completeness 衡量 Agent Card 是否包含名称、描述、版本、owner、capabilities、skills、默认输入输出模式、supported interfaces、安全要求和缓存信息。

工程价值: 字段不完整时, 路由器无法判断能力边界、接口方式、权限要求和复现版本, 只能退回硬编码或模型猜测。

Agent Skill Declaration Quality

一句话定义: agent skill declaration quality 衡量 skill 是否有稳定 id、名称、描述、tags、输入输出模式、示例任务和 skill 级权限要求。

面试表达: 好的 skill 声明应该是业务任务级能力, 例如合同付款条款审查, 而不是一句 “我什么都能做” 或一堆低层函数名。

Supported Interface Readiness

一句话定义: supported interface readiness 衡量 Agent Card 暴露的调用接口是否有安全 URL、协议绑定、协议版本和租户或环境边界。

工程价值: 服务发现不只是找到 URL, 还要知道该 URL 用什么协议、是否安全、适用于哪个环境以及是否和当前客户端兼容。

Agent Card Security Coverage

一句话定义: agent card security coverage 衡量 Agent Card 是否声明认证方式、security schemes、security requirements、scope、数据分类和调用权限边界。

工程价值: Agent Card 是权限系统的输入, 不是最终裁判; 平台仍要用用户身份、租户、资源分类和策略引擎做动态校验。

Agent Card Version Cache Readiness

一句话定义：agent card version cache readiness 衡量 Card 是否包含版本、etag / hash、缓存 TTL、兼容性和回滚信息。

面试表达：没有版本和缓存语义，服务发现结果就难以复现，灰度、回滚和事故排查都会变得不可靠。

Agent Discovery Match Quality

一句话定义：agent discovery match quality 衡量服务发现召回的 Agent 和 skill 是否匹配任务需要的能力、领域、输入输出模式、权限和租户。

工程价值：语义检索只能做候选召回，最终必须经过结构化字段、权限和健康状态过滤。

Agent Routing Decision Quality

一句话定义：agent routing decision quality 衡量路由器基于 Agent Card、任务要求、权限、成本、延迟和健康状态作出的 allow / block / route 决策是否符合预期。

面试表达：多 Agent 路由不是选择描述最像的 Agent，而是在可用、可授权、可复现、可追踪的候选中做取舍。

Extended Agent Card Control

一句话定义：extended agent card control 衡量敏感 skill、内部 endpoint、租户策略、权限 scope 和服务等级等详细信息是否只在认证授权后披露。

工程价值：服务发现需要公开入口，但能力目录本身也可能敏感；public card 和 extended card 应该分层。

Agent Card Trace Readiness

一句话定义：agent card trace readiness 衡量 trace 是否记录 case id、任务、候选 Agent、选中 Agent、skill、决策、原因、Card 版本、权限判断和路由版本。

工程价值：多 Agent 事故复盘要能还原当时看到的是哪张 Card、为什么选它、权限怎么判、哪些候选被过滤。

Agent Card Eval Coverage

一句话定义：agent card eval coverage 衡量评估集是否覆盖字段缺失、skill 不匹配、接口不安全、权限不足、输出模式不支持、extended card 误披露、版本缓存缺失和 trace 缺字段。

面试表达：Agent Card eval 不能只测 happy path；服务发现和路由失败样本更能暴露治理问题。

Agent Card Gate

一句话定义：agent card gate 是基于字段、skill、interface、安全、版本缓存、发现匹配、路由决策、extended card、trace 和 eval 的 Agent Card 上线门禁。

面试表达：Agent Card gate 过线，才能把远程 Agent 当成可治理系统组件；否则它只是一个能被调用的黑盒服务。

A2A Task Delegation Audit

一句话定义：A2A task delegation audit 是从 Task、Message、Artifact、TaskState、权限、重试、取消、并行汇总、trace 和 eval 审计远程 Agent 委派链路是否可治理。

面试表达：A2A 委派不是发一句“帮我做”，而是要证明任务契约、状态流、追问、结果产物、失败恢复和责任边界都可追踪。

A2A Task Contract Coverage

一句话定义：A2A task contract coverage 衡量 Task 是否包含 id、contextId、status、message、metadata、deadline、幂等键、权限范围和回调或事件通道。

工程价值：任务契约缺字段时，上游无法可靠重试、取消、恢复、审计或把多个子任务合并回根任务。

A2A State Transition Validity

一句话定义：A2A state transition validity 衡量状态流是否遵守 submitted、working、input-required、auth-required、completed、failed、canceled、rejected 等稳定语义。

面试表达：内部可以有 queued、running、expired 等细分状态，但对外必须映射成稳定 TaskState，否则跨 Agent 编排会失去一致性。

A2A Input Required Handling

一句话定义：A2A input required handling 衡量 input-required 是否带结构化问题、必填字段、message id、回复路径和继续执行策略。

工程价值：追问没有结构化时，上游不知道该问用户、查知识库、自动补全还是取消任务。

A2A Artifact Metadata Coverage

一句话定义：A2A artifact metadata coverage 衡量任务产物是否包含 artifact id、URI、media type、content hash、classification、创建者和保留策略等元数据。

工程价值：Artifact 元数据支持完整性校验、权限控制、保留周期、后续 Agent 引用和事故复盘。

A2A Error Semantics Coverage

一句话定义：A2A error semantics coverage 衡量 failed 状态是否包含 error code、category、retryable、details、partial artifacts 和 trace id。

面试表达：模糊失败会让上游不知道该重试、换 Agent、追问、降级、人工接管还是终止。

A2A Retry Idempotency Coverage

一句话定义：A2A retry idempotency coverage 衡量重试任务是否有幂等键、重试预算、可重试错误边界和操作副作用保护。

工程价值：读任务重复执行通常只是浪费，写任务重复执行可能造成重复发信、重复建单或重复修改外部状态。

A2A Cancellation Coverage

一句话定义：A2A cancellation coverage 衡量取消请求是否能传到下游 Agent，并最终进入 canceled 或明确失败状态。

面试表达：取消不是断开连接，而是协议层任务状态变更；下游要说明是否停止成功、是否有部分产物和是否需要清理。

A2A Delegation Permission Boundary

一句话定义：A2A delegation permission boundary 衡量上游委派给下游的 scope、上下文、用户授权和可继续委派权限是否遵守最小必要原则。

工程价值：委派不等于转授权；上游能访问某份资料，不代表下游 Agent 自动可以读取或复制。

A2A Parallel Aggregation Readiness

一句话定义：A2A parallel aggregation readiness 衡量并行子任务是否能完整汇总状态、产物、证据、冲突和失败原因。

面试表达：多 Agent 并行不是投票平均；总控 Agent 要用证据链、时间线、置信度和数据来源处理冲突。

A2A Task Trace Readiness

一句话定义：A2A task trace readiness 衡量 trace 是否记录 task id、context id、agent id、state、message id、artifact id、权限决策、错误、版本和耗时。

工程价值：跨 Agent 事故没有 trace 时，很难判断问题来自状态跳跃、越权委派、远程失败、产物缺失还是汇总冲突。

A2A Task Eval Coverage

一句话定义：A2A task eval coverage 衡量评估集是否覆盖状态跳跃、input-required 缺失、Artifact 元数据缺失、失败语义缺失、越权委派、重试不幂等、取消未生效、并行冲突和 trace 缺失。

面试表达：A2A eval 不能只测 happy path；坏样本才能证明委派链路真的可治理。

A2A Task Gate

一句话定义：A2A task gate 是基于任务契约、状态流、追问、消息、产物、错误、重试、取消、权限、并行、trace 和 eval 的委派生命周期上线门禁。

面试表达：这个门禁过不了时，系统最多是远程 Agent demo，不应该声称具备生产级 A2A 协作能力。

A2A Message Boundary Audit

一句话定义：A2A message boundary audit 是从 Message header、Part 类型、来源可信、context policy、最小上下文、引用优先、脱敏、claim grounding、摘要约束、trace 和 eval 审计多 Agent 消息是否会污染或泄露上下文。

面试表达：多 Agent 风险不只是选错 Agent，更常见的是不可信数据被当成指令、敏感内容被复制、假设被升级成事实、约束在摘要中丢失。

A2A Message Contract Coverage

一句话定义：A2A message contract coverage 衡量消息是否包含 messageId、taskId、contextId、role、parts 和承载 sender、recipient、intent、context policy、trace 的 metadata。

工程价值：消息契约不完整时，上游很难做去重、路由、权限判断、上下文恢复和事故复盘。

A2A Part Typing Coverage

一句话定义：A2A part typing coverage 衡量每个 Part 是否标注 kind、media type 和业务语义，例如 instruction、resource_ref、artifact_ref、claim、summary 或 log_excerpt。

工程价值：没有 Part 类型，Runtime 只能把所有内容当普通文本，无法对日志、网页、补丁、证据和指令采用不同策略。

A2A Source Trust Labeling

一句话定义：A2A source trust labeling 衡量每个内容块是否标注 source kind 和 trust level，例如 system_policy、agent_output、tool_result、web_page、database、external_untrusted。

面试表达：来源和可信级别是防 prompt injection、上下文污染和责任混淆的基础字段。

A2A Instruction Data Separation

一句话定义：A2A instruction data separation 衡量系统是否能把指令块和数据块分开，禁止网页、日志、文档或数据库内容里的文本升级为可执行指令。

工程价值：外部内容可以包含“忽略规则”这类字符串，但它只能作为不可信数据被处理，不能覆盖 Host policy。

A2A Minimal Context Coverage

一句话定义：A2A minimal context coverage 衡量委派消息是否只携带完成子任务所需的信息，并符合 token 预算、任务必要性和上下文裁剪策略。

面试表达：默认全量转发上下文会同时增加泄露面、污染面和成本；最小上下文是多 Agent 系统的基本安全原则。

A2A Reference Over Copy Coverage

一句话定义：A2A reference over copy coverage 衡量大对象和敏感对象是否优先用 resource_ref / artifact_ref 传递，而不是直接复制到下游上下文。

工程价值：引用可以做权限、过期、撤销、审计和预算控制；复制会让数据流和责任边界变得不可控。

A2A Context Policy Enforcement

一句话定义：A2A context policy enforcement 衡量 allow forwarding、allowed recipients、data classification、retention、redaction required 和 forbidden content 等策略是否由 Runtime 强制执行。

面试表达：context policy 不能只写给模型看，必须在 Host / Runtime 层作为硬策略执行。

A2A Sensitive Redaction Coverage

一句话定义：A2A sensitive redaction coverage 衡量 restricted / confidential 内容是否完成脱敏、字段投影或引用化，避免用户级明细、凭证、系统提示和内部策略被复制给下游。

工程价值：多 Agent 链路越长，敏感数据越容易在摘要、日志、Artifact 和二次委派中扩散。

A2A Claim Grounding Coverage

一句话定义：A2A claim grounding coverage 衡量 Agent 输出中的 fact、hypothesis、inference、recommendation 是否带 claim type、confidence、evidence 和 limitations。

面试表达：下游不能把“可能相关”改写成“确定原因”；claim 标注是防错误升级的关键。

A2A Summary Constraint Retention

一句话定义：A2A summary constraint retention 衡量摘要是否保留原始目标、限制条件、权限边界、不确定性、证据和过期信息提示。

工程价值：压缩上下文不能只保留结论；丢掉否定条件、权限限制和不确定性会直接改变任务语义。

A2A Message Trace Readiness

一句话定义：A2A message trace readiness 衡量 trace 是否记录 message id、task id、context id、sender、recipient、policy、Part 摘要、来源、版本和处理结果。

工程价值：上下文污染事故需要能还原“哪段内容从哪里来、如何被标注、如何被转发、谁把它当成事实”。

A2A Message Eval Coverage

一句话定义：A2A message eval coverage 衡量评估集是否覆盖不可信指令、敏感复制、禁止转发失效、收件人越界、claim 误升级、摘要丢约束、预算超限和 trace 缺失。

面试表达：消息边界 eval 要测坏样本，而不是只测结构化 JSON 能解析。

A2A Message Gate

一句话定义：A2A message gate 是基于消息契约、Part 类型、来源可信、指令 / 数据分离、最小上下文、引用优先、策略执行、脱敏、claim grounding、摘要约束、trace 和 eval 的消息边界上线门禁。

面试表达：这个门禁过不了，多 Agent 系统很容易把上下文泄露、污染和错误升级伪装成正常协作。

A2A MCP Boundary Audit

一句话定义：A2A MCP boundary audit 是从协议分类、工具 / Agent 边界、自主性、生命周期、能力发现、上下文所有权、权限分离、结果产物、trace 和版本 eval 审计 A2A 与 MCP 是否分工清楚。

面试表达：MCP 连接工具、资源和提示模板，A2A 连接远程 Agent 和任务生命周期；把工具伪装成 Agent 或把 Agent 塞成 Tool，都会让权限、状态、产物和审计语义混乱。

A2A MCP Protocol Classification

一句话定义：A2A MCP protocol classification 衡量每个外部能力是否被正确标成 MCP、A2A 或二者组合。

工程价值：分类是后续权限、上下文、trace 和 eval 的根字段；一开始分错，后面很难靠 prompt 修正。

A2A Tool Agent Boundary

一句话定义：A2A tool agent boundary 衡量明确操作是否作为 MCP tool / resource / prompt 暴露，自主任务是否作为 A2A remote agent 委派。

面试表达：工具是被动操作，Agent 是自任务执行者；边界越诚实，系统越容易做最小权限。

A2A Autonomy Fit

一句话定义：A2A autonomy fit 衡量被调用方自主性和协议选择是否匹配，低自主性能力不伪装成 Agent，高自主性能力不降级成单个 Tool。

工程价值：自主性决定是否需要规划、追问、状态同步、取消、产物和失败语义。

A2A Lifecycle Placement

一句话定义：A2A lifecycle placement 衡量短操作是否走 MCP call，长任务是否走 A2A Task lifecycle，组合任务是否有清楚的 combined lifecycle。

面试表达：一次函数返回不了的任务，不应该藏在 do_everything 工具里。

A2A MCP Discovery Split

一句话定义：A2A MCP discovery split 衡量 MCP 发现 tools / resources / prompts，A2A 发现 Agent Card / skills，二者没有混成一个不可治理的能力列表。

工程价值：发现粒度不同会影响路由、权限和版本缓存；工具列表和 Agent 专家目录不能混用一套语义。

A2A MCP Context Ownership

一句话定义：A2A MCP context ownership 衡量 MCP 结果是否进入 Host context builder, A2A 消息是否受 task context policy 控制。

面试表达：上下文所有权不清，常见后果是下游 Agent 看到不该看的原始数据或系统策略。

A2A MCP Permission Separation

一句话定义：A2A MCP permission separation 衡量任务委派是否没有顺手转交工具权限，各 Agent 的 MCP 权限仍由各自 Host / Runtime 控制。

工程价值：委派任务不等于转授权；权限扩散是多 Agent 系统最常见的治理失败之一。

A2A Result Artifact Boundary

一句话定义：A2A result artifact boundary 衡量 MCP 返回 tool result / resource content, A2A 返回 artifact / structured task result, 二者不互相伪装。

面试表达：需要一个值时用 Tool Result，需要任务成果、证据链和可复用产物时用 Artifact。

A2A MCP Trace Linkage

一句话定义：A2A MCP trace linkage 衡量 A2A task span 和 MCP tool span 是否能在同一 trace 中串起来。

工程价值：没有跨协议 trace，就很难解释最终答案经过哪些 Agent、调用过哪些工具、读过哪些资源。

A2A MCP Version Eval Coverage

一句话定义：A2A MCP version eval coverage 衡量系统是否记录 MCP spec、tool schema、A2A spec、Agent Card、权限策略版本和 eval label。

面试表达：协议升级、Agent Card 变化和工具 schema 变化都可能改变系统行为，必须能回放和回归测试。

A2A MCP Boundary Gate

一句话定义：A2A MCP boundary gate 是基于协议分类、边界、自主性、生命周期、发现、上下文、权限、输出、trace、版本和 eval 的 A2A/MCP 分工上线门禁。

面试表达：这个门禁过不了时，系统不是“多协议生态”，而是把工具、Agent、权限和上下文混在一起的 demo。

Cross-Agent Security Audit

一句话定义：cross-agent security audit 是从用户身份、Agent 身份、服务身份、OBO scope、委派 allowlist、权限衰减、上下文策略、继续委派、工具权限、结果释放、trace / audit、信任验证和租户隔离审计多 Agent 系统是否可治理。

面试表达：多 Agent 安全不是让模型承诺守规矩，而是让每次委派、上下文流动、工具调用、Artifact 释放和责任链都能被策略系统证明。

Cross-Agent Identity Chain Coverage

一句话定义：cross-agent identity chain coverage 衡量安全事件是否同时记录用户身份、租户、调用 Agent、接收 Agent、服务实例和 task id。

工程价值：缺任一身份层，事故后都难以判断请求代表谁、由谁发起、由哪个服务实例执行。

OBO Scope Binding

一句话定义：OBO scope binding 衡量下游 Agent 请求的 scope 是否同时受用户授权、任务授权和 Agent 权限约束。

面试表达：on-behalf-of 不是把用户 token 传下去，而是把用户、Agent、任务和资源策略绑定成可审计授权上下文。

Delegation Allowlist Coverage

一句话定义：delegation allowlist coverage 衡量接收 Agent 是否在 allowlist 中，Agent Card 是否签名，服务身份是否经过校验。

工程价值：服务发现不是允许任意远程 Agent 接收上下文；未审核 Agent 默认不能参与敏感任务。

Permission Attenuation Coverage

一句话定义：permission attenuation coverage 衡量权限在委派链中是否只会收窄，不会从聚合到明细、从只读到读写、从摘要到全文。

面试表达：委派任务不等于扩大权限；下游权限应该小于或等于用户授权和任务需要。

Cross-Agent Context Policy Enforcement

一句话定义：cross-agent context policy enforcement 衡量 allowed recipients、redaction、retention、allow forwarding 和 data classification 是否由 runtime 强制执行。

工程价值：上下文策略不能只写进消息 metadata，必须在发送前、接收时和继续委派前生效。

Cross-Agent Redelegation Control

一句话定义：cross-agent redelegation control 衡量下游 Agent 是否不能自由继续委派，继续委派是否需要 allowed delegates、上游批准或人工确认。

面试表达：多 Agent 泄露常发生在第二跳和第三跳；默认不允许下游自由转发。

Cross-Agent High Risk Confirmation

一句话定义：cross-agent high risk confirmation 衡量写操作、生产变更、第三方 Agent、敏感数据外发、删除 Artifact 等高风险动作是否经过明确确认。

工程价值：确认界面应展示谁代表谁、委派给谁、传什么上下文、数据密级、风险和是否允许继续委派。

Cross-Agent Tool Permission Binding

一句话定义：cross-agent tool permission binding 衡量下游 Agent 调 MCP 工具时是否同时检查用户 scope、任务 scope、租户、工具要求和环境边界。

面试表达：A2A 委派不能绕过 MCP 工具权限；Agent 内部调工具仍要回到 Host / Runtime 的权限门禁。

Cross-Agent Result Release Control

一句话定义：cross-agent result release control 衡量返回结果和 Artifact 是否脱敏、引用化、保留期受控，并避免泄露敏感证据路径。

工程价值：结果可信不等于结果可转发；Artifact 也要有 classification、retention、recipient 和 audit policy。

Cross-Agent Audit Trace Completeness

一句话定义：cross-agent audit trace completeness 衡量 trace / audit 是否记录安全关键字段：trace id、task id、用户、Agent、策略、决策、数据分类、上下文引用和时间。

面试表达：Trace 偏排障，Audit 偏合规和追责；二者可共享 trace id，但访问权限和保留策略不同。

Cross-Agent Trust Evidence Verification

一句话定义：cross-agent trust evidence verification 衡量低信任 Agent 输出是否需要证据、人审或 verifier，可信 Agent 输出是否仍保留 claim type、confidence、evidence 和 limitations。

工程价值：Agent 可信不代表每次输出可信；结果进入上游上下文前要保留来源、证据和验证状态。

Cross-Agent Tenant Isolation

一句话定义：cross-agent tenant isolation 衡量资源、Artifact、trace、Agent 可见性、缓存和策略是否绑定同一租户边界。

面试表达：Agent Card 和服务发现也要有 tenant scope；不能因为 Agent 是内部服务就默认跨租户可见。

Cross-Agent Security Gate

一句话定义：cross-agent security gate 是基于身份链、OBO scope、委派 allowlist、权限衰减、上下文策略、继续委派、高风险确认、工具权限、结果释放、audit、信任验证和租户隔离的跨 Agent 安全上线门禁。

面试表达：这个门禁过不了时，系统可以演示多 Agent 协作，但不应该接触真实企业权限、敏感数据和生产工具。

Multi-Agent Failure Audit

一句话定义：multi-agent failure audit 是从委派图、冲突仲裁、claim 证据、上下文约束、重复工作、状态事件、Artifact 写入、策略链、协作规模、终止条件和 eval 覆盖审计多 Agent 系统是否会循环、冲突、漂移、传播幻觉或绕过安全策略。

面试表达：多 Agent 不是 Agent 数量越多越强；工程上要证明协作链不会无限绕圈、互相放大错误、丢约束、抢写产物或把假设升级成事实。

Delegation Loop Control

一句话定义：delegation loop control 衡量任务委派图是否无环、深度不过阈值、visited agents 不重复、相似任务不重复提交，并且有明确终止条件。

工程价值：没有循环控制时，多 Agent 系统看起来一直在工作，实际可能只是在互相转发同一个未解决问题。

Role Conflict Arbitration

一句话定义：role conflict arbitration 衡量代码、测试、安全、产品、数据等 Agent 给出互斥结论时，是否有风险优先、领域权威或人工复核等明确仲裁策略。

面试表达：安全和合规场景不能靠简单多数投票；高风险冲突应进入风险优先或人工复核。

Hallucination Propagation Containment

一句话定义：hallucination propagation containment 衡量 Agent 输出是否保留 claim type、evidence、confidence 和 limitations，并阻止 hypothesis / inference 在下游摘要中被升级成 fact。

工程价值：多 Agent 幻觉最危险的地方是上游的“可能”在下游报告、决策和执行链里变成“确定”。

Context Drift Control

一句话定义：context drift control 衡量 root goal、hard constraints、禁止事项和目标变更审批是否随每次委派和摘要保留。

面试表达：如果用户说“只分析，不改生产”，后续子任务不能漂移成“回滚生产策略”。

Duplicate Work Budget Control

一句话定义：duplicate work budget control 衡量重复工具调用、重复查询、并行 Agent 数量、缓存复用和成本预算是否受控。

工程价值：多 Agent 成本爆炸通常来自多个 Agent 为了验证或不信任彼此而重复查询同一证据。

Multi-Agent State Consistency

一句话定义：multi-agent state consistency 衡量状态事件是否按 sequence number 单调推进，终态是否一致，旧事件是否不会覆盖新状态。

面试表达：多 Agent 状态治理要像分布式系统一样处理乱序、重复、丢事件和 reconcile。

Artifact Conflict Control

一句话定义：artifact conflict control 衡量多个 Agent 是否不会直接写同一对象，或者通过锁、patch、冲突检测和 merge owner 控制写入。

工程价值：代码、报告、测试快照和配置文件都应避免多写者直接落盘，优先产出 patch Artifact 后统一合并。

Multi-Agent Accountability Trace

一句话定义：multi-agent accountability trace 衡量 trace / audit 是否记录 root task、子任务、调用方、接收方、决策、证据、状态、策略和成本字段。

面试表达：责任链不是为了甩锅，而是为了定位哪个 Agent、哪个证据、哪次摘要或哪个策略判断导致事故。

Policy Chain Enforcement

一句话定义：policy chain enforcement 衡量数据分类、允许明细级别、上下文转发策略和最终输出检查是否贯穿整条 Agent 协作链。

工程价值：安全策略不能只在第一跳生效；第二跳、第三跳和最终报告同样要执行数据分类、脱敏和允许接收方检查。

Collaboration Fit

一句话定义：collaboration fit 衡量任务复杂度、风险等级、Agent 数量和委派理由是否匹配，防止简单任务被过度拆给多个 Agent。

面试表达：多 Agent 是为专业分工、并行和跨组织协作服务的，不是所有摘要、分类和问答都值得启动多 Agent 链。

Termination Handoff Readiness

一句话定义：termination handoff readiness 衡量每个 root task 是否有最大深度、最大轮次、终止条件，以及循环、冲突、安全绕过等高风险场景的人类接管路径。

工程价值：没有终止和接管机制的 Agent 系统，本质上没有失控刹车。

Multi-Agent Failure Eval Coverage

一句话定义：multi-agent failure eval coverage 衡量循环委派、角色冲突、幻觉传播、上下文漂移、成本爆炸、状态不一致、产物冲突、安全策略绕过和过度协作是否进入回归 eval 集。

面试表达：多 Agent eval 不能只看最终答案，要覆盖协作链内部的失败切片。

Multi-Agent Failure Gate

一句话定义：multi-agent failure gate 是基于循环控制、冲突仲裁、幻觉传播抑制、上下文漂移控制、成本预算、状态一致性、Artifact 冲突、责任 trace、策略链、协作适配、终止接管和 eval 覆盖的多 Agent 失败治理上线门禁。

面试表达：这个门禁过不了时，系统可以做演示，但不应该让多个 Agent 自主访问真实数据、生产工具或高风险动作。

A2A System Design Audit

一句话定义：A2A system design audit 是从需求澄清、平台模块、协议契约、发现路由、状态幂等、上下文边界、权限模型、A2A/MCP 分工、Artifact 治理、trace / audit、失败处理、eval 和扩展性检查一个企业多 Agent 协作平台设计是否完整。

面试表达：A2A 系统设计题不是画很多 Agent，而是证明发现、委派、状态、上下文、权限、产物、审计、失败恢复和评估都可运营。

A2A Requirement Clarification

一句话定义：A2A requirement clarification 衡量系统设计回答是否先澄清用户范围、Agent 来源、异步任务、多租户安全、MCP 集成、人工审批和成功标准。

工程价值：不澄清边界就开始画架构，容易把内部平台、第三方 Agent 平台、短任务聊天和长任务 workflow 混成一个不可落地系统。

A2A Architecture Module Coverage

一句话定义：A2A architecture module coverage 衡量回答是否覆盖 Task API、Orchestrator、Agent Registry、A2A Runtime、Policy Engine、Context Manager、Artifact Store、Trace / Audit 和 Specialist Agents。

面试表达：只画 Agent 是不够的；平台能力才是系统设计题的主体。

A2A Protocol Contract Coverage

一句话定义：A2A protocol contract coverage 衡量 Agent Card、Task、Message 和 Artifact 的关键字段是否被覆盖，并区分协议字段和企业内部 metadata。

工程价值：没有协议契约，Agent 间协作就会退化成不可治理的自由文本聊天。

A2A Discovery Routing Governance

一句话定义：A2A discovery routing governance 衡量 Agent 发现和路由是否按 capability、domain、version、tenant、auth scope、health / SLO 过滤，并在模型路由前执行策略门禁。

面试表达：不能只说“让 LLM 选择 Agent”；LLM 路由必须受 Registry、权限、租户和健康状态约束。

A2A Runtime State Readiness

一句话定义：A2A runtime state readiness 衡量平台是否使用 submitted、working、input-required、auth-required、completed、failed、canceled、rejected 等稳定状态，并支持 event sequence 和幂等事件处理。

工程价值：内部 queued / running / expired 可以存在，但必须映射回可审计的协议状态语义。

A2A Context Boundary Control

一句话定义：A2A context boundary control 衡量 minimal context、context refs、classification、allow forwarding、redaction 和 source labels 是否由 Context Manager 强制执行。

面试表达：A2A 平台不能把用户上下文整包转发给所有 Agent；要按任务和接收方最小化。

A2A Permission Model Coverage

一句话定义：A2A permission model coverage 衡量用户身份、Agent 身份、服务身份、OBO scope、权限衰减、高风险确认和 audit decision 是否进入设计。

工程价值：委派不是转授权；权限应该从用户、任务、Agent、资源和上下文策略共同收敛。

A2A MCP Boundary Clarity

一句话定义：A2A MCP boundary clarity 衡量设计是否明确 A2A 用于 Agent 间任务协作，MCP 用于 Agent 内部工具 / 资源 / prompt 访问，并禁止转交 MCP 工具权限。

面试表达：A2A 和 MCP 的边界是权限、状态和责任边界，不只是协议名字不同。

A2A Artifact Governance Coverage

一句话定义：A2A artifact governance coverage 衡量产物是否引用化存储，并带 hash、classification、retention、access control 和审计字段。

工程价值：报告、代码 patch、表格和证据不应直接塞进消息长文本里，应该作为受权限约束的 Artifact。

A2A Trace Audit Completeness

一句话定义：A2A trace audit completeness 衡量 trace / audit 是否记录 trace id、task id、parent task、policy decision、tool calls、artifact refs、cost 和 status event。

面试表达：Trace 支持排障，Audit 支持合规；系统设计回答中二者可以共享 id，但不能混成一份随便可读的日志。

A2A Failure Handling Coverage

一句话定义：A2A failure handling coverage 衡量循环检测、event seq、retryable error、input-required、冲突仲裁、claim verification、budget limit 和 human handoff 是否闭环。

工程价值：多 Agent 平台的失败处理是高分点，缺它就只是 happy path 架构图。

A2A Eval Observability Coverage

一句话定义：A2A eval observability coverage 衡量 offline eval、trace replay、single agent eval、collaboration eval、safety eval、cost / latency eval 和 regression eval 是否被纳入平台设计。

面试表达：A2A 平台不能只看用户点赞，要能回放 trace、定位协作失败并防止改动回归。

A2A Scalability Idempotency Readiness

一句话定义：A2A scalability idempotency readiness 衡量异步队列、事件流、registry cache、backpressure、并发限制、idempotency key 和 health check 是否进入设计。

工程价值：长任务、多 Agent 和高并发场景下，扩展性、幂等和背压是平台能否上线的基础。

A2A System Design Gate

一句话定义：A2A system design gate 是基于需求、模块、协议、路由、状态、上下文、权限、A2A/MCP 边界、Artifact、trace / audit、失败处理、eval 和扩展性的企业多 Agent 平台设计上线门禁。

面试表达：这个门禁过不了时，答案可能讲清 demo，但还不是一个企业级 A2A 平台设计。

Cross-Agent Collaboration System Audit

一句话定义：cross-agent collaboration system audit 是对跨 Agent 协作系统的任务图、Agent 发现路由、A2A 生命周期、上下文最小化、权限衰减、MCP 工具边界、证据产物、冲突处理、循环终止、trace / eval、预算和协作适配性的综合审计。

核心关注：多 Agent 系统要证明“为什么需要多 Agent、如何协作、如何避免越权、如何保留证据、如何失败终止”，而不是展示多个 Agent 的对话。

Cross-Agent Requirement Goal Clarity

一句话定义：cross-agent requirement goal clarity 衡量系统设计是否明确用户目标、成功标准、业务边界、风险假设、多租户和人工审批要求。

常见问题：不澄清目标和风险就拆 Agent，会把短任务、长任务、内部平台和第三方协作平台混在一起。

Cross-Agent Collaboration Module Coverage

一句话定义：cross-agent collaboration module coverage 衡量架构是否覆盖 Orchestrator、Planner、Agent Registry、A2A Runtime、Context Manager、Policy Engine、MCP Gateway、Artifact Store、Trace / Audit / Replay、Human Review 和 Eval。

面试表达：系统主体是平台控制面，不是一堆专业 Agent 名字。

Cross-Agent Registry Routing Governance

一句话定义：cross-agent registry routing governance 衡量 Agent 发现和路由是否受 Agent Card、capability、tenant、permission、health、version、cost、latency 和历史成功率约束。

工程价值：让 LLM 直接按名字选 Agent，会绕过权限、健康状态、租户可见性和版本治理。

Cross-Agent Task Graph Validity

一句话定义：cross-agent task graph validity 衡量 root task 的子任务图是否无环、依赖清楚、深度有界、可并行、可重试并能单独归因。

常见问题：任务图有环或依赖缺失时，多 Agent 会进入循环委派、重复执行或错误聚合。

Cross-Agent A2A Lifecycle Alignment

一句话定义：cross-agent A2A lifecycle alignment 衡量内部 created / planned / assigned / running 等状态是否能映射到 A2A 的 submitted、working、input-required、auth-required、completed、failed、canceled、rejected 等稳定语义。

工程价值：内部状态可以更细，但对外协议状态必须可审计、可恢复、可回放。

Cross-Agent Context Minimization Enforcement

一句话定义：cross-agent context minimization enforcement 衡量每个下游 Agent 是否只收到完成子任务所需的最小上下文，并保留 source、trust、classification、taint、redaction 和 forwarding policy。

面试表达：多 Agent 系统里，上下文不是共享内存，而是受权限和任务约束的输入包。

Cross-Agent Permission Attenuation Binding

一句话定义：cross-agent permission attenuation binding 衡量用户权限、Agent 权限、任务 scope、租户策略和风险策略是否在委派链上持续求交集。

工程价值：委派不是转授权；下游 Agent 不能因为自身权限更高而扩大原始用户权限。

Cross-Agent MCP Tool Boundary Governance

一句话定义：cross-agent MCP tool boundary governance 衡量 Agent 间委派是否禁止转交上游 MCP 工具凭据、原始连接和超出任务范围的工具权限。

核心边界：A2A 传任务和产物，MCP 访问工具和资源；二者不能混成工具权限扩散通道。

Cross-Agent Artifact Evidence Grounding

一句话定义：cross-agent artifact evidence grounding 衡量中间结果和最终结果是否用 Artifact、source refs、claim type、confidence、limitations 和版本信息支撑。

常见问题：把上游 Agent 的自然语言结论直接当事实，会让幻觉沿协作链传播。

Cross-Agent Conflict Arbitration Readiness

一句话定义：cross-agent conflict arbitration readiness 衡量多个 Agent 结论冲突时，系统是否能比较证据、领域权威、新鲜度、权限范围，并触发 verifier、仲裁 Agent 或人工复核。

面试表达：Orchestrator 不能简单拼接多个 Agent 的答案，必须处理冲突和不确定性。

Cross-Agent Hallucination Propagation Control

一句话定义：cross-agent hallucination propagation control 衡量 hypothesis、inference、fact 和 recommendation 是否被区分，并在下游保留证据和限制说明。

工程价值：多 Agent 的幻觉风险往往来自“上游已经总结过，所以可信”的错误假设。

Cross-Agent Delegation Loop Containment

一句话定义：cross-agent delegation loop containment 衡量委派图是否有环检测、最大深度、最大同类任务次数、预算停止和人工接管策略。

常见问题：Agent A 委派给 B，B 又委派回 A，会消耗成本并让状态无法收敛。

Cross-Agent Human Handoff Approval Readiness

一句话定义：cross-agent human handoff approval readiness 衡量高风险动作、权限不确定、冲突无法解决、连续失败和预算超限时，系统是否能进入人工审核控制台。

工程价值：人工接管不是失败标志，而是高风险自动化的必要安全阀。

Cross-Agent Trace Audit Replay Coverage

一句话定义：cross-agent trace audit replay coverage 衡量 trace 是否串起 root task、subtask、Agent、Message、Policy decision、MCP tool call、Artifact、approval、cost、latency 和 final answer。

面试表达：Trace 支持排障，Audit 支持责任链，Replay 支持回归评估；三者都不能缺。

Cross-Agent Eval Baseline Regression Coverage

一句话定义：cross-agent eval baseline regression coverage 衡量评估是否比较单 Agent / 固定 workflow baseline，并覆盖任务拆解、Agent 选择、上下文裁剪、权限、安全、证据、冲突、成本和回归切片。

工程价值：多 Agent 必须证明比更简单方案更好，否则只是增加复杂度。

Cross-Agent Cost Latency Budget Control

一句话定义：cross-agent cost latency budget control 衡量 root task 是否限制 Agent 数量、委派深度、子任务数量、工具调用、token、wall-clock deadline、成本、并发和重试次数。

常见问题：每个 Agent 都继续调用模型和工具时，成本、延迟和下游负载会成倍放大。

Cross-Agent Scalability Idempotency Readiness

一句话定义：cross-agent scalability idempotency readiness 衡量运行时是否支持 idempotency key、event sequence、异步队列、backpressure、并发限制、health check 和状态 reconcile。

工程价值：长任务和多 Agent 并发让系统更像分布式系统，幂等和状态收敛是上线基础。

Cross-Agent Collaboration Fit Control

一句话定义：cross-agent collaboration fit control 衡量系统是否避免把简单任务过度拆成多 Agent，并能说明多 Agent 相比单 Agent 或固定 workflow 的收益。

面试表达：Agent 越多不等于越高级，只有当并行、专业分工、交叉验证或权限隔离带来净收益时才值得多 Agent 化。

Cross-Agent Collaboration System Gate

一句话定义：cross-agent collaboration system gate 是基于目标澄清、平台模块、Registry 路由、任务图、A2A 生命周期、上下文最小化、权限衰减、MCP 边界、证据产物、冲突仲裁、幻觉传播控制、循环终止、人审、trace / audit / replay、baseline eval、预算、幂等扩展和协作适配性的综合门禁。

面试表达：门禁通过才说明跨 Agent 协作不是热闹的多模型对话，而是可授权、可追踪、可恢复、可评估且确实值得多 Agent 化的系统。

Tool Protocol Future Audit

一句话定义：tool protocol future audit 是把 Function Calling、MCP、A2A、Skill / Plugin、Policy、Trace、Eval 和 Marketplace 的未来演进判断拆成可审计维度，避免把短期 API 字段或营销词写成长期标准。

面试表达：未来题不是押注某个协议，而是判断分层、互操作、安全、证据、治理和迁移能力是否成立。

Layered Protocol Boundary Clarity

一句话定义：layered protocol boundary clarity 衡量回答是否分清模型原生工具调用、工具资源连接、Agent 协作、能力包、治理层和运营层。

常见问题：把 Function Calling、MCP、A2A、Skill 和 Marketplace 都叫成“工具协议”，会导致系统设计边界混乱。

Standardization Extension Balance

一句话定义：standardization extension balance 衡量方案是否区分适合标准化的对象契约、安全 metadata、任务状态和审计事件，以及适合保留为厂商扩展的模型优化参数和产品交互。

工程价值：标准化不足会割裂生态，标准化过早会限制模型和 runtime 创新。

Long Task Lifecycle Readiness

一句话定义：long task lifecycle readiness 衡量协议方案是否支持 task id、状态机、暂停、恢复、取消、重试、artifact 和全生命周期审计。

面试表达：未来工具调用会从单次函数调用走向长期任务执行。

Capability Package Completeness

一句话定义：capability package completeness 衡量一个能力包是否同时声明 tools、resources、prompts、memory scope、workflow、permissions、eval 和 owner。

常见问题：只发布一个 tool schema，不足以支撑企业可安装、可授权、可评估的能力生态。

Agent-Centric Autonomy Governance

一句话定义：agent-centric autonomy governance 衡量 Agent 的长期目标、任务状态、委派、工具调用和组织策略是否由 runtime / policy 层治理。

工程价值：Agent 中心不是让模型自由行动，而是让自主性在身份、权限、预算、状态和人工确认下运行。

Tool Safety Metadata Coverage

一句话定义：tool safety metadata coverage 衡量工具是否声明 risk level、side effect、input / output classification、required approval、allowed caller scope、retention 和 audit requirement。

常见问题：工具 description 写得再清楚，也不能替代运行时安全 metadata 和 policy gate。

Provenance Verifiability

一句话定义：provenance verifiability 衡量工具结果、RAG chunk、Artifact 和跨 Agent 输出是否带 source、query time、data version、permission scope、evidence refs、confidence 和 limitations。

面试表达：工具结果是证据，不是天然事实；没有 provenance 就无法审计最终结论。

Marketplace Governance Readiness

一句话定义：marketplace governance readiness 衡量 Marketplace 是否承担能力发现、安装审批、权限透明、质量监控、版本治理、安全审核、下架和事故处理职责。

工程价值：企业 Marketplace 不是商店页面，而是能力治理平面。

Provider Adapter Migration Control

一句话定义：provider adapter migration control 衡量系统是否用内部 Tool Runtime 对象和 adapter 测试隔离不同 provider 的字段、streaming、parallel、tool choice、built-in tools 和 trace 差异。

常见问题：直接把某家 provider 的字段散落到业务代码，会让迁移和回归测试变得不可控。

Auto Generated Tool Review Gate

一句话定义：auto generated tool review gate 衡量从 OpenAPI、数据库 schema、代码库或文档自动生成的工具是否经过最小权限裁剪、安全审核、eval、灰度和发布治理。

面试表达：自动生成提高效率，但不能自动获得安全边界和语义正确性。

Workflow Runtime Integration

一句话定义：workflow runtime integration 衡量系统是否把确定性流程、审批、状态和补偿交给 workflow runtime，把理解、摘要、分析和生成交给 Agent。

常见问题：把所有控制权交给 Agent，会弱化高合规流程的可预测性和责任边界。

Behavior Eval Coverage

一句话定义：behavior eval coverage 衡量评估是否覆盖该不该调用工具、工具选择、参数、顺序、失败恢复、权限、安全、成本、trace 和最终结果。

工程价值：工具协议越强，评估越要从 answer eval 走向 action / behavior eval。

Human Model Documentation Split

一句话定义：human model documentation split 衡量系统是否区分 model-facing description 和 human-facing documentation。

面试表达：给模型看的短描述要服务调用决策，给人看的文档要服务审核、维护、权限和事故归因。

Autonomy Risk Tiering

一句话定义：autonomy risk tiering 衡量 Agent 自主权是否按低风险、中风险、高风险和极高风险动作分级，并绑定撤销、审计、确认或专家审核。

常见问题：只问 “Agent 能不能自动执行”，没有按动作风险分层，是生产系统设计低分信号。

Responsibility Attribution Trace

一句话定义：responsibility attribution trace 衡量一次错误工具调用能否拆解到用户指令、模型判断、tool schema、权限策略、工具实现、审批流程和平台审计。

工程价值：没有 trace / replay，就无法定位责任，也无法把事故变成回归样本。

Natural Language API Boundary

一句话定义：natural language API boundary 衡量系统是否承认自然语言适合表达意图，但关键动作仍需要结构化 API、权限、状态、参数校验和审计。

面试表达：自然语言不会完全取代 API，而会成为结构化执行协议之上的意图接口。

Agent Operating Layer Governance

一句话定义：agent operating layer governance 衡量 Agent 应用运行时是否具备能力注册、权限管理、任务调度、资源访问、沙箱、日志审计、包管理、监控和故障恢复。

常见问题：把工具生态比作操作系统时，必须补上治理控制面，而不是只讲 “Agent 会越来越自主”。

Engineer Readiness Coverage

一句话定义：engineer readiness coverage 衡量工程师是否具备 Function Calling、MCP、A2A、schema、权限、prompt injection、task state、workflow、trace / replay、eval 和 marketplace 治理能力。

面试表达：工具协议生态已经从 API 使用题变成大模型系统工程题。

Unsupported Speculation Rate

一句话定义：unsupported speculation rate 衡量未来演进回答中有多少结论缺少公开资料、版本边界、系统证据或工程约束，却被写成确定事实。

工程价值：开放题允许判断和假设，但必须把事实、趋势、推测和待验证边界分清。

Tool Protocol Future Gate

一句话定义：tool protocol future gate 是基于分层边界、标准化与扩展平衡、长任务、能力包、Agent 自主治理、安全 metadata、provenance、Marketplace、provider adapter、自动生成审核、workflow、behavior eval、文档拆分、风险分级、责任 trace、自然语言边界、运行时治理、工程师能力和反投机约束的综合门禁。

面试表达：门禁通过才说明未来演进回答不是泛泛预测，而是能落到可治理、可迁移、可验证和可评估的系统判断。

Skill Definition Audit

一句话定义：skill definition audit 是从 manifest、任务目标、能力包组成、tool / skill 边界、instructions、resources / prompts、workflow、permissions、configuration、eval、lifecycle、安装治理和 progressive disclosure 审计一个能力定义是否真正像 Skill。

面试表达：Skill 不是工具列表，也不是万能助手；它应该是围绕某类任务目标可安装、可授权、可复用、可评估和可治理的能力包。

Skill Manifest Metadata

一句话定义：skill manifest metadata 衡量 Skill 是否有 id、name、version、description、owner、license 等可注册、可审计、可升级的元信息。

工程价值：缺 manifest 的 Skill 只能作为本地脚本或 prompt 片段，很难进入企业 marketplace 和版本治理。

Skill Task Goal Clarity

一句话定义：skill task goal clarity 衡量 Skill 是否有清晰任务目标和合适边界，避免过大成“万能办公”或过小成单个 API。

面试表达：好的 Skill 应该让用户一眼知道它能完成哪类任务，也能让平台独立评估它是否完成任务。

Skill Bundle Completeness

一句话定义：skill bundle completeness 衡量 Skill 是否同时覆盖 instructions、tools、resources、prompts、workflow、permissions、config 和 eval 等能力包要素。

工程价值：只有 tools 没有说明、资源、流程和 eval，只是工具集合，不是成熟 Skill。

Tool Skill Boundary

一句话定义：tool skill boundary 衡量候选能力是否超过单个操作，并包含任务策略、约束和交付质量标准。

面试表达：Tool 是能做一个动作，Skill 是能完成一类任务。

Skill Instruction Strategy

一句话定义：skill instruction strategy 衡量 Skill 是否有稳定说明文件、使用约束、边界条件和按需加载的任务指南。

工程价值：说明文件不是装饰，而是让模型知道何时使用 Skill、如何使用、何时不要使用。

Skill Resource Prompt Grounding

一句话定义：skill resource prompt grounding 衡量 Skill 是否绑定任务相关资源、示例、术语表、模板和 prompt，而不是每次从空白上下文开始。

面试表达：Skill 的可复用性来自 “工具 + 任务知识 + 参考资料” 的组合。

Skill Workflow Readiness

一句话定义：skill workflow readiness 衡量复杂 Skill 是否描述 collect inputs、retrieve context、draft、validate、approval、final action 等关键步骤。

工程价值：不是每个 Skill 都需要硬编码流程，但高风险或多步任务需要清晰控制流和中间检查点。

Skill Permission Safety

一句话定义：skill permission safety 衡量 Skill 是否声明 read、write、external call、human confirmation、data classification 等权限和安全策略。

面试表达：安装 Skill 前，用户或管理员必须知道它会读什么、写什么、外发什么、哪些动作需要确认。

Skill Configuration Reuse

一句话定义：skill configuration reuse 衡量 Skill 是否通过 language、output format、recipient policy、deadline 等配置适配团队和租户，而不是复制多份相似 Skill。

工程价值：配置让同一能力可复用，避免每个团队都 fork 一份近似实现。

Skill Eval Coverage

一句话定义：skill eval coverage 衡量 Skill 是否有 task success、grounding、format、safety、regression 和 cost 等质量评估。

面试表达：Skill 升级不能只看感觉，必须用 eval 判断任务质量、安全和成本是否变好。

Skill Lifecycle Governance

一句话定义：skill lifecycle governance 衡量 Skill 是否支持 semver、changelog、install、enable、disable、upgrade、rollback 和 deprecate。

工程价值：Skill 一旦进入企业平台，就是产品单元，需要发布、回滚、下架和兼容性治理。

Skill Product Install Governance

一句话定义：skill product install governance 衡量 Skill 是否可安装、可启用、可禁用、可审核、可统计和可下架。

面试表达：Skill 是工具生态产品化抽象，不只是代码目录。

Skill Progressive Disclosure

一句话定义：skill progressive disclosure 衡量 Skill 是否能按需加载说明、脚本、参考资料和资产，避免把所有内容一次性塞进上下文。

工程价值：复杂 Skill 应把常用说明放在入口文件，把长参考、脚本和资产延迟加载。

Skill Definition Gate

一句话定义：skill definition gate 是基于 manifest、任务目标、能力包组成、工具边界、说明、资源、workflow、权限、配置、eval、生命周期、安装治理和渐进加载的 Skill 定义上线门禁。

面试表达：这个门禁过不了时，候选能力可能是工具、prompt、workflow 或 plugin 片段，但还不是可治理的 Skill。

Ecosystem Boundary Audit

一句话定义：ecosystem boundary audit 是把 Plugin、Action、Tool、Skill 和 Workflow 拆成执行实例、调用接口、流程编排、任务能力包和扩展载体五层来审计。

面试表达：边界审计的核心不是争名词，而是证明权限、评估、审计、生命周期和安装分发都落在正确抽象层。

Required Field Coverage

一句话定义：required field coverage 衡量每个抽象对象是否具备本层必需字段，例如 Action 的 action id / input / output / caller / trace id，Tool 的 schema 和错误语义，Plugin 的 manifest / auth / version / entrypoints。

工程价值：缺必填字段会让 registry、runtime、trace、eval 和发布治理都无法稳定工作。

Abstraction Classification

一句话定义：abstraction classification 衡量候选条目是否被建模到正确层级，避免把一次执行叫 Tool、把单个 API 叫 Skill、把流程片段叫 Plugin。

面试表达：如果分类错了，后面的权限、评估和审计指标都会跟着错位。

Granularity Boundary

一句话定义：granularity boundary 衡量抽象粒度是否匹配，Action 是 execution，Tool 是 operation，Workflow 是 process，Skill 是 task capability，Plugin 是 package。

工程价值：粒度混乱会导致一个对象既想承担执行日志，又想承担安装治理，最后两边都不可审计。

Action Trace

一句话定义：action trace 衡量一次具体执行是否记录 action id、输入、输出、调用者、状态、风险和 trace id。

面试表达：Action 是审计事实，不是能力定义；它回答“这一次到底发生了什么”。

Tool Contract

一句话定义：tool contract 衡量 Tool 是否有名称、描述、输入输出 schema、权限、超时和结构化错误语义。

工程价值：Tool 是 runtime 可调接口，contract 不清会直接影响参数校验、权限绑定、重试和错误恢复。

Workflow Control

一句话定义：workflow control 衡量 Workflow 是否显式描述 steps、dependencies、conditions、retry policy、approval points 和 state store。

面试表达：Workflow 的重点是控制流和状态，而不是工具列表。

Skill Bundle

一句话定义：skill bundle 衡量 Skill 是否围绕任务目标组合 manifest、instructions、tools、resources、prompts、workflow、permissions 和 eval。

工程价值：Skill 是可复用任务能力包，不能退化成单个 API 或散装工具清单。

Plugin Packaging

一句话定义：plugin packaging 衡量 Plugin 是否具备 plugin id、manifest、install、auth、version、entrypoints、dependencies 和 contained capabilities。

面试表达：Plugin 更像交付和安装载体，一个 Plugin 可以包含多个 Tool、Workflow 或 Skill。

Permission Layering

一句话定义：permission layering 衡量权限是否落在正确层级：Action 看执行实例，Tool 看操作调用，Workflow 看步骤或分支，Skill 看任务能力，Plugin 看安装包。

工程价值：权限层混在一起时，最常见后果是安装权限过大、执行确认缺失或下游能力被错误转授权。

Eval Layering

一句话定义：eval layering 衡量评估指标是否落在正确层级：Action 看执行成功，Tool 看调用质量，Workflow 看流程成功，Skill 看任务成功，Plugin 看包健康度。

面试表达：不同层级的质量问题不能用一个 success rate 混着解释。

Audit Trace Layering

一句话定义：audit trace layering 衡量 trace / audit 字段是否能区分安装、调用、流程、任务和执行事实。

工程价值：审计链路分层后，事故复盘才能定位到底是 Plugin 发布、Skill 选择、Workflow 控制、Tool 契约还是 Action 执行出了问题。

Naming Alias Documentation

一句话定义：naming alias documentation 衡量平台里 Action、Tool、Workflow、Skill、Plugin 等别名是否说明清楚，避免把产品命名误读成工程边界。

面试表达：各家平台命名可以不同，但系统内部的工程语义必须一致。

Governance Lifecycle

一句话定义：governance lifecycle 衡量扩展能力是否覆盖 install、enable、upgrade、rollback、disable、deprecate 等治理动作。

工程价值：只会新增不会回滚和下架的能力生态，无法进入企业生产环境。

High Risk Approval

一句话定义：high risk approval 衡量删除、支付、外部发送、权限变更和生产修改等高风险动作是否有人工确认或审批点。

面试表达：高风险控制不能藏在 Skill 描述里，必须进入 Workflow / Tool / Action 的实际执行门禁。

Eval Ready

一句话定义：eval ready 衡量每个抽象条目是否绑定 eval label、任务样本或回归规则，便于升级后比较质量变化。

工程价值：没有 eval 的工具生态只能靠主观体验判断，无法做版本治理。

Ecosystem Boundary Gate

一句话定义：ecosystem boundary gate 是基于字段覆盖、抽象分类、粒度、Action trace、Tool contract、Workflow control、Skill bundle、Plugin packaging、权限分层、评估分层、审计分层、命名别名、生命周期、高风险审批和 eval readiness 的工具生态边界上线门禁。

面试表达：这个门禁过不了时，方案可能跑 demo，但还没有把执行、接口、流程、任务能力和扩展载体拆成可治理系统。

Skill Manifest Audit

一句话定义：skill manifest audit 是从身份元信息、能力描述、capabilities、inputs、outputs、tools、resources / prompts、workflow、permissions、configuration、safety、eval、examples、version / lifecycle 和 audit 字段审计一个 Skill Manifest 是否可被平台消费。

面试表达：Manifest 不是宣传文案，而是 discovery、routing、permission、safety、workflow、eval 和 lifecycle 的结构化入口。

Identity Metadata

一句话定义：identity metadata 衡量 Skill Manifest 是否包含稳定 id、name、version、owner、category 和 status。

工程价值：身份元信息不完整时，平台很难做注册、版本追踪、责任归属和发布治理。

Description Quality

一句话定义：description quality 衡量能力描述是否具体说明任务对象、处理动作、输出形式、证据要求和边界，而不是“强大助手”式泛化描述。

面试表达：描述质量直接影响路由，太泛会误触发，太窄会漏触发。

Capability Granularity

一句话定义：capability granularity 衡量 capabilities 是否处在任务级粒度，既不是万能 legal_help，也不是 find_word_payment 这类单步 API。

工程价值：能力粒度决定 Skill 是否可评估、可复用和可组合。

Input Contract

一句话定义：input contract 衡量 Manifest 是否声明必需输入、可选输入、类型、默认值和资源引用方式。

面试表达：输入契约清楚时，Agent 缺信息应先追问，而不是盲目执行。

Output Contract

一句话定义：output contract 衡量 Skill 是否声明输出类型、结构化 schema、citation、confidence 和 artifact 等下游可消费结果。

工程价值：输出不稳定会让 Workflow、UI、上游 Agent 和 eval 都无法可靠集成。

Tool Dependency Purpose

一句话定义：tool dependency purpose 衡量每个依赖 Tool 是否说明用途、是否必需和风险边界。

面试表达：只列工具名不够，平台和审核者还需要知道为什么要用这个工具。

Resource Prompt Binding

一句话定义：resource prompt binding 衡量 Skill 是否绑定稳定资源、知识库、prompt 模板、版本和用途说明。

工程价值：资源和 prompt 被版本化后，升级影响范围才可追踪。

Manifest Workflow Readiness

一句话定义：manifest workflow readiness 衡量复杂或高风险 Skill 是否声明 steps、tool / prompt 节点、依赖顺序和 approval points。

面试表达：Workflow 字段让平台知道控制流在哪里插入审批、监控、失败恢复和审计。

Permission Least Privilege

一句话定义：permission least privilege 衡量权限声明是否避免 all、*、admin 等过宽 scope，并为每个 scope 给出 reason。

工程价值：最小权限和 reason 是管理员审核、自动策略检查和事故复盘的基础。

Configuration Schema

一句话定义：configuration schema 衡量配置项是否有 name、type、default、enum 或说明，便于不同租户和团队复用同一 Skill。

面试表达：配置项让 Skill 可复用，而不是每个团队复制一份近似实现。

Safety Policy

一句话定义：safety policy 衡量 Manifest 是否声明外部分享、人工审批、禁止动作、数据分类和脱敏策略。

工程价值：安全策略只有进入结构化 Manifest，平台才能强制执行。

Eval Gate

一句话定义：eval gate 衡量 Skill 是否声明 metrics、golden sets、minimum quality gate 和安全回归门槛。

面试表达：没有 eval gate 的 Skill 升级只能靠感觉判断，不能稳定发布。

Examples Trigger Coverage

一句话定义：examples trigger coverage 衡量 examples 是否覆盖应该触发和不应触发的用户请求，帮助路由器学习边界。

工程价值：负例和正例同样重要，否则 Skill 容易被过度路由。

Version Lifecycle

一句话定义：version lifecycle 衡量 Manifest 是否使用稳定版本号，并声明 changelog、rollback、兼容性和下架策略。

面试表达：Skill 一旦进入平台，就需要像产品一样发布、回滚和维护。

Audit Readiness

一句话定义：audit readiness 衡量 owner、version、changelog、review status 等审计字段是否完整。

工程价值：没有审计字段时，出了问题很难定位谁发布、改了什么、何时审核。

Skill Manifest Gate

一句话定义：skill manifest gate 是基于身份、描述、能力粒度、输入输出、工具用途、资源和 prompt 绑定、workflow、权限、配置、安全、eval、examples、生命周期和审计字段的 Skill Manifest 上线门禁。

面试表达：这个门禁过不了时，Manifest 可能能让人看懂，但还不能作为平台治理入口。

Skill Lifecycle Audit

一句话定义：skill lifecycle audit 是从发布审核、安装审批、启用范围、权限重审、配置版本、运行中任务策略、兼容性、灰度、回滚、依赖版本、审计日志、紧急下架、卸载保留、eval monitoring 和更新策略审计 Skill 是否可控。

面试表达：生命周期治理的核心是把 Skill 从“能调用”升级成“可安装、可启用、可升级、可回滚、可下架、可追责”的平台能力。

Review Before Publish

一句话定义：review before publish 衡量 Skill 发布前是否完成 manifest、security、eval 和 owner 审核。

工程价值：发布审核不过线时，后续安装和启用会把未治理能力暴露给租户。

Install Approval

一句话定义：install approval 衡量安装时是否展示 manifest 摘要、权限、风险、审批人和安装记录。

面试表达：安装不是复制文件，而是把能力加入租户可用集合，需要留下授权事实。

Enable Scope Control

一句话定义：enable scope control 衡量 Skill 是否按 tenant、team、role、agent 或任务类型限制启用范围。

工程价值：安装后默认全员启用通常风险过大，尤其是高风险 Skill。

Permission Reapproval

一句话定义：permission reapproval 衡量版本更新新增权限或扩大数据范围时是否重新审批。

面试表达：只要 permission delta 变大，就不能沿用旧安装审批。

Configuration Versioning

一句话定义：configuration versioning 衡量租户、团队、用户或环境配置是否版本化并保留变更历史。

工程价值：Skill 行为变化不一定来自代码，也可能来自配置。

Running Task Policy

一句话定义：running task policy 衡量禁用、卸载、紧急下架时是否明确正在运行任务的完成、取消、暂停或切换策略。

面试表达：禁用 Skill 时，不能只阻止新任务，还要处理已启动任务和受影响 artifact。

Lifecycle Compatibility Check

一句话定义：lifecycle compatibility check 衡量升级前是否检查 input、output、configuration、workflow、prompt 和 tool 兼容性。

工程价值：输出契约或配置 schema 破坏时，上游 Agent、Workflow 和 UI 都可能解析失败。

Rollout Guard

一句话定义：rollout guard 衡量 Skill 升级是否通过 canary / staged rollout，并监控成功率、错误率、安全、延迟和成本。

面试表达：高风险 Skill 不应一次性全量升级。

Skill Rollback Readiness

一句话定义：skill rollback readiness 衡量旧版本、配置兼容、prompt / resource pinning、可逆迁移和回滚计划是否就绪。

工程价值：没有回滚计划的发布不是可控发布。

Dependency Versioning

一句话定义：dependency versioning 衡量 Skill 依赖的 tools、prompts、resources、models 或其他 Skill 是否声明兼容版本。

面试表达：依赖写成 latest 会让灰度和回滚都不可复现。

Lifecycle Audit Log Completeness

一句话定义：lifecycle audit log completeness 衡量发布、安装、启用、禁用、升级、回滚、下架和高风险调用是否记录事件类型、版本、actor、tenant、时间和决策。

工程价值：没有生命周期审计日志，事故复盘只能靠猜。

Emergency Suspend Readiness

一句话定义：emergency suspend readiness 衡量平台是否能暂停 Skill、阻止新任务、控制运行中任务、标记 artifact、通知用户并保留证据。

面试表达：紧急下架是企业 Skill 平台的安全阀。

Uninstall Retention

一句话定义：uninstall retention 衡量卸载后配置、artifact、trace 和 audit 是否按策略保留，并检查依赖 workflow 的影响。

工程价值：卸载不能删除责任链，尤其是高风险行业。

Lifecycle Eval Monitoring

一句话定义: lifecycle eval monitoring 衡量 golden eval、regression eval、safety eval 和 canary monitoring 是否通过。

面试表达: 生命周期升级必须把离线 eval 和线上灰度指标连起来看。

Update Policy Accuracy

一句话定义: update policy accuracy 衡量自动更新、手动审批、安全补丁和 major 升级策略是否与实际变更风险匹配。

工程价值: patch 可以自动, 新增权限、破坏输出契约或 major 升级应人工审批。

Skill Lifecycle Gate

一句话定义: skill lifecycle gate 是基于发布审核、安装审批、启用范围、权限重审、配置版本、运行中任务、兼容性、灰度、回滚、依赖、审计、紧急下架、卸载保留、eval 和更新策略的 Skill 生命周期上线门禁。

面试表达: 这个门禁过不了时, Skill 可能能安装和调用, 但还不能被企业平台长期治理。

Skill Marketplace Audit

一句话定义: skill marketplace audit 是从 catalog、search、detail page、review、permission、install approval、rating、operations、duplicate governance、admin view、developer console、security center、recommendation、lifecycle、owner 和 audit trace 审计 Skill 门户是否可治理。

面试表达: Marketplace 不是列表页, 而是能力发现、安装审批、质量运营、安全中心和审计追责的统一入口。

Catalog Metadata

一句话定义: catalog metadata 衡量 Skill 目录是否包含稳定 id、name、version、description、category、tags、owner、status、permissions、risk level、last updated 和 visibility。

工程价值: 目录字段不完整时, 搜索、审批、推荐、下架和事故追踪都会缺上下文。

Search Discovery

一句话定义: search discovery 衡量 Marketplace 是否把名称、描述、标签、示例、能力、owner、质量、安全、权限可用性和维护状态纳入搜索与发现。

面试表达: 企业门户不能只按关键词相似度召回 Skill, 还要过滤权限、安全和维护风险。

Detail Page Completeness

一句话定义: detail page completeness 衡量详情页是否说明能做什么、不适合什么、示例、输入输出、权限、owner、版本历史、质量指标、安全状态和已知限制。

工程价值: 详情页过泛会让用户误用, 也会让管理员无法判断是否批准安装。

Review Workflow

一句话定义: review workflow 衡量 Skill 上架前是否完成 manifest、permission、security、quality 和 maintenance 审核。

面试表达: 上架审核不是一次性填表, 版本升级、权限扩大和安全事件都应触发重审。

Permission Transparency

一句话定义：permission transparency 衡量 Skill 是否清楚展示每个权限的用途、数据分类、外部传输和高风险动作。

工程价值：权限 reason 不透明时，安装审批无法判断最小权限是否成立。

Install Approval Flow

一句话定义：install approval flow 衡量不同风险级别是否匹配自助安装、团队管理员、安全 / IT / data owner 审批或专项审批。

面试表达：高风险 Skill 自助安装是企业门户的典型危险设计。

Rating Quality Balance

一句话定义：rating quality balance 衡量 Marketplace 评分是否同时结合用户评价、任务成功率、eval、安全、可靠性、响应速度和维护状态。

工程价值：只看星级或安装量会把热门但不安全的 Skill 推到前面。

Operations Metrics

一句话定义：operations metrics 衡量门户是否持续统计 Skill 总数、上架通过率、安装量、启用量、活跃数、成功率、权限风险、安全事件、更新频率、留存、重复比例和无人维护数。

面试表达：Marketplace 是要运营的生态，不是发布后就无人看管的文件目录。

Duplicate Capability Governance

一句话定义：duplicate capability governance 衡量平台是否识别重复能力、标记适用场景差异、推荐 canonical Skill、合并或下架低质量重复项。

工程价值：重复 Skill 不一定错，但没有治理会造成用户困惑和维护浪费。

Admin Permission View

一句话定义：admin permission view 衡量管理员是否能看到已安装 Skill、权限、启用用户或团队、高风险调用、外部传输、敏感数据、陈旧版本和 owner 缺失。

面试表达：企业门户必须让管理员看见“谁在用什么能力、访问什么数据、风险在哪里”。

Developer Console Readiness

一句话定义：developer console readiness 衡量开发者是否能看到提交状态、审核结果、安装量、调用量、失败率、反馈、eval、安全告警、版本分布和错误日志。

工程价值：没有开发者控制台，Skill 质量很难靠反馈闭环持续变好。

Security Center Readiness

一句话定义：security center readiness 衡量门户是否具备权限巡检、异常检测、数据泄露告警、供应链风险、紧急下架、漏洞通知和受影响用户通知。

面试表达：Skill 生态越开放，安全中心越不能只是后台页面，而要能发现、阻断和通知。

Recommendation Policy Safety

一句话定义：recommendation policy safety 衡量推荐策略是否同时使用角色、任务、权限可用性、安全过滤、维护状态和高风险阻断。

工程价值：把高风险 Skill 推荐给无权限用户会把搜索体验变成安全风险。

Lifecycle Visibility

一句话定义：lifecycle visibility 衡量门户是否展示 Skill 的状态、版本、版本历史、最后更新时间和弃用策略。

面试表达：用户和管理员需要知道 Skill 是否还在维护，而不是只看到一个安装按钮。

Owner Maintenance

一句话定义：owner maintenance 衡量 Skill 是否有活跃 owner、维护 SLA、响应渠道和陈旧状态标记。

工程价值：无人维护但仍被大量使用的 Skill 是企业门户里的长期隐患。

Portal Audit Trace

一句话定义：portal audit trace 衡量提交、审核、安装、启用、阻断和卸载等门户事件是否保留 actor、tenant、version、time 和 decision。

面试表达：Marketplace 的审计链要覆盖从发布到卸载的治理事实。

Skill Marketplace Gate

一句话定义：skill marketplace gate 是基于目录、搜索、详情页、审核、权限、审批、评分、运营、重复治理、管理员视图、开发者控制台、安全中心、推荐、生命周期、owner 和审计的门户上线门禁。

面试表达：这个门禁过不了时，Skill 可能能被发布成页面，但还不能进入企业级能力生态。

Tool Skill Quality Safety Audit

一句话定义：tool skill quality safety audit 是从 Tool schema、参数校验、执行可靠性、输出契约、副作用控制、Skill 任务质量、事实证据、离线评估、在线监控、权限、数据安全、prompt injection、供应链、人工审核、发布回归和审计 trace 审核工具 / Skill 是否可上架。

面试表达：工具生态的质量不是开发者自称可靠，而是通过可复现 eval、可执行安全门禁和持续在线监控证明出来的。

Tool Schema Clarity

一句话定义：tool schema clarity 衡量 Tool 是否有清晰 name、description、input schema、required、type、enum / range 和 examples。

工程价值：schema 模糊会直接放大模型参数生成错误和后续 repair 成本。

Argument Validation

一句话定义：argument validation 衡量工具调用是否同时做 schema 校验、业务规则校验、权限校验、可控修复和结构化错误返回。

面试表达：strict schema 只能解决结构合法，不能替代业务语义、权限和安全校验。

Execution Reliability

一句话定义：execution reliability 衡量 Tool 是否具备 timeout、retry、structured error、rate limit 和 dependency health 处理。

工程价值：没有可靠执行边界时，Agent 容易陷入卡死、重复调用或错误恢复失败。

Output Stability

一句话定义：output stability 衡量 Tool 输出是否有稳定字段、版本、source、分页 / 截断和敏感信息标签。

面试表达：Tool 返回成功不代表可被上游稳定消费，输出契约同样需要评估。

Side Effect Control

一句话定义：side effect control 衡量写操作或不可逆动作是否具备 idempotency key、preview、dry run、confirmation 和 rollback / cancel 策略。

工程价值：发送邮件、删除文件、更新数据库这类动作不能让模型无确认地自动执行。

Skill Task Quality

一句话定义：skill task quality 衡量 Skill 是否达到任务成功率、事实准确性、引用准确性、完整性和格式合规的最低门槛。

面试表达：Tool 能调用成功不等于 Skill 真的完成任务。

Factual Grounding

一句话定义：factual grounding 衡量 Skill 输出是否有引用、证据支持，并控制 unsupported claim 和 hallucination。

工程价值：合同、法务、金融、医疗等场景里，流畅但无证据的回答不能算通过。

Completeness Format

一句话定义：completeness format 衡量 Skill 是否覆盖关键内容并遵守约定输出 schema、章节、模板、引用和限制说明。

面试表达：答案格式稳定是 Agent、Workflow、UI 和人工 reviewer 能继续消费结果的前提。

Offline Eval Coverage

一句话定义：offline eval coverage 衡量上架前是否覆盖 golden set、scenario set、adversarial set、regression set 和明确阈值。

工程价值：离线 eval 太干净会让 Skill 刷题通过，线上仍在边界场景失败。

Online Monitoring

一句话定义：online monitoring 衡量线上是否监控成功率、错误类别、延迟、成本、用户反馈、安全拦截和输出解析失败。

面试表达：离线通过只是上架前证据，线上监控才发现分布漂移和依赖故障。

Permission Least Privilege

一句话定义：permission least privilege 衡量权限申请是否最小化，并包含 permission reason、data scope、写操作审批和外部传输审核。

工程价值：权限过宽会把普通质量问题升级成数据安全事故。

Data Security Review

一句话定义：data security review 衡量输入输出是否有数据分类、脱敏、保留策略、租户隔离和训练使用策略。

面试表达：质量高但泄露数据的 Skill 不能上架。

Prompt Injection Resilience

一句话定义：prompt injection resilience 衡量 Skill 是否把网页、文档、邮件、Issue、日志等不可信内容当作数据，而不是指令。

工程价值：读取外部内容的 Skill 必须测试 indirect instruction、data boundary、tool call block 和 leakage probe。

High Risk Action Control

一句话定义：high risk action control 衡量高风险动作是否需要 confirmation、approval policy、audit 和 human escalation。

面试表达：高风险 Tool 的默认策略应是先预览、再确认、可审计，而不是直接执行。

Supply Chain Governance

一句话定义：supply chain governance 衡量依赖是否版本锁定、漏洞扫描、代码签名、网络 allowlist 和 owner 明确。

工程价值：远程代码、第三方包和外部服务会把 Skill 风险扩展到供应链。

Human Review Readiness

一句话定义：human review readiness 衡量高风险领域、安全例外、法律或金融场景是否有人类 reviewer 决策记录。

面试表达：自动审核适合批量发现问题，但高风险例外需要人工承担决策责任。

Regression Release Gate

一句话定义：regression release gate 衡量发布前是否设置 quality threshold、safety threshold、rollback plan 和 blocking release。

工程价值：Prompt、Workflow、Tool schema、资源或模型变化都可能造成质量和安全回退。

Audit Trace Readiness

一句话定义：audit trace readiness 衡量 case id、version、actor、decision、trace id 和 eval report 是否能串起来。

面试表达：没有审计 trace，质量安全审核无法复盘，也无法证明当时为什么放行或阻断。

Quality Safety Gate

一句话定义：quality safety gate 是基于 Tool 调用质量、Skill 任务质量、事实证据、离线 eval、在线监控、权限、数据安全、prompt injection、副作用、供应链、人工审核、回归发布和审计 trace 的上架门禁。

面试表达：这个门禁过不了时，工具或 Skill 可能跑 demo，但还不能进入企业 Marketplace。

MCP A2A Harness Integration

一句话定义：MCP A2A harness integration 是把 MCP server、MCP resources / prompts / tools 和 A2A remote agents 统一纳入 capability registry、permission engine、context builder、trace、replay 和 evaluation harness 的工程设计。

面试表达：协议解决互联，harness 负责治理；真正的集成不是“能连上”，而是能证明 namespace、schema、权限、上下文预算、信任边界、trace、replay 和版本捕获都受控。

Protocol Capability Registry

一句话定义：protocol capability registry 是统一注册内置工具、custom tools、MCP tools / resources / prompts 和 A2A remote agent capabilities 的能力表。

工程价值：它把不同协议暴露的能力映射成可审计条目，绑定 namespace、schema、风险等级、权限策略、认证 scope、timeout、输出限制和 trace 字段。

MCP Integration

一句话定义：MCP integration 是把 Model Context Protocol 的 Host / Client / Server、resources、prompts、tools、sampling、roots、elicitation、authorization、progress 和 cancellation 接入 agent runtime 的过程。

边界：MCP tool 描述和返回内容不能默认可信；外部数据应作为 untrusted data 进入 context builder，高风险副作用必须由 permission engine 判断。

A2A Integration

一句话定义：A2A integration 是把 Agent Card、remote agent discovery、task lifecycle、streaming、push notification、message 和 artifact 纳入本地 agent harness 的过程。

边界：A2A remote agent 可以保持 opaque，但本地 harness 仍要治理委派权限、传出上下文、远程任务状态、结果验证、取消、trace 和责任边界。

Capability Discovery Coverage

一句话定义：capability discovery coverage 衡量 MCP server 或 A2A remote agent 是否完成连接、能力发现、版本读取和可用性登记。

Namespace Isolation Coverage

一句话定义：namespace isolation coverage 衡量外部能力是否有稳定 namespace，避免不同 MCP server、工具、prompt、resource 或 remote agent 名称冲突。

Protocol Schema Validity

一句话定义：protocol schema validity 衡量 MCP tool schema、resource metadata、prompt argument schema 和 A2A Agent Card / task schema 是否可解析、可验证且版本明确。

Protocol Permission Binding

一句话定义: protocol permission binding 衡量每个协议能力是否绑定 allow / ask / deny、参数级风险、agent 身份、认证 scope 和企业策略。

High-Risk Approval Coverage

一句话定义: high-risk approval coverage 衡量写外部系统、敏感读取、文件写入、部署、发送消息和跨 agent 委派等高风险动作是否被拒绝或经过明确人工确认。

Context Output Budget Coverage

一句话定义: context output budget coverage 衡量 MCP / A2A 的工具描述、schema、外部结果、stream message 和 artifact 是否受 token budget、分页、摘要、落盘引用和输出上限约束。

External Trust Boundary Coverage

一句话定义: external trust boundary coverage 衡量外部 issue、网页、日志、数据库结果、远程 agent 消息和 artifact 是否被标注为不可信数据，而不是模型或系统指令。

A2A Lifecycle Coverage

一句话定义: A2A lifecycle coverage 衡量 remote agent task 是否覆盖 created、running、waiting / input required、completed、failed、cancelled、streaming update 和 push notification 等状态。

Protocol Trace Coverage

一句话定义: protocol trace coverage 衡量 MCP / A2A 调用是否记录 capability id、协议版本、输入摘要、输出摘要、权限结果、耗时、错误、artifact ref、脱敏和版本信息。

Protocol Replay Readiness

一句话定义: protocol replay readiness 衡量外部协议调用是否支持 mock、snapshot、dry run 或 deterministic replay，避免评估时真实外部系统变化破坏复现。

Protocol Version Capture

一句话定义: protocol version capture 衡量 harness 是否记录 MCP spec / SDK / server 版本、A2A protocol / Agent Card / remote agent 版本、工具 schema 版本和权限策略版本。

Protocol Integration Gate

一句话定义: protocol integration gate 是基于 discovery、namespace、schema、permission、high-risk approval、context budget、trust boundary、A2A lifecycle、trace、replay 和 version 的 MCP/A2A 集成门禁。

面试表达: MCP/A2A 上线评估不能只跑通 happy path，而要证明外部能力扩展不会破坏最小权限、上下文稳定性、审计、回放和责任边界。

Agent Harness System Design

一句话定义: Agent Harness system design 是把客户端、server、session、orchestrator、context builder、model adapter、capability registry、permission engine、execution engine、sandbox、trace、replay、evaluation 和企业治理组织成可运行系统的设计题。

面试表达：不要只画“用户到模型到工具”的直线图，要证明每个模块都有接口契约、状态机、权限边界、失败恢复、trace、replay、eval 和版本治理。

Runtime Module Coverage

一句话定义：runtime module coverage 衡量 Agent Harness 是否覆盖 api server、session manager、agent orchestrator、context builder、model adapter、capability registry、permission engine、execution engine、sandbox manager、diff manager、trace store、replay engine、eval runner 和 policy / auth 等必需模块。

Interface Contract Coverage

一句话定义：interface contract coverage 衡量各模块输入、输出、错误、版本、事件和幂等语义是否清楚。

Agent State Machine Coverage

一句话定义：agent state machine coverage 衡量 session、orchestrator、permission、execution、replay 和 eval 等状态性模块是否有明确生命周期和合法转移。

Permission Integration Coverage

一句话定义：permission integration coverage 衡量 api、capability、permission、execution、sandbox 和 external gateway 等风险模块是否绑定 runtime 级 allow / ask / deny。

Context Control Coverage

一句话定义：context control coverage 衡量 context builder、model adapter、capability registry、trace 和外部协议网关是否受 token budget、输出限制、摘要、artifact 引用和信任边界约束。

Execution Isolation Coverage

一句话定义：execution isolation coverage 衡量文件、shell、sandbox、MCP、A2A、git、LSP 和外部系统动作是否在 workspace、网络、环境变量、时间和资源边界内执行。

System Trace Coverage

一句话定义：system trace coverage 衡量所有关键模块是否把模型调用、工具调用、权限决策、diff、错误、重试、MCP/A2A 和最终状态写入统一 trace。

System Replay Readiness

一句话定义：system replay readiness 衡量 harness 是否能用录制结果、mock、snapshot、dry run 或确定性环境复现一次 agent run。

System Eval Readiness

一句话定义：system eval readiness 衡量 harness 是否能把任务集、环境、验收器、trace、baseline、公平预算、安全样本、成本和报告接入 evaluation harness。

Recovery Coverage

一句话定义：recovery coverage 衡量系统是否支持 abort、retry、checkpoint、session resume、diff revert、sandbox reset、tool error recovery 和用户中断恢复。

Harness Version Capture

一句话定义：harness version capture 衡量系统是否记录模型、prompt、agent config、tool registry、permission policy、sandbox image、MCP/A2A server、代码 commit 和评估任务版本。

Enterprise Governance Readiness

一句话定义：enterprise governance readiness 衡量 harness 是否具备 RBAC、组织策略、审计日志、secret 管理、数据出境控制、retention、成本治理和管理员配置能力。

Harness System Gate

一句话定义：harness system gate 是基于模块覆盖、接口契约、状态机、权限、上下文控制、执行隔离、trace、replay、eval、恢复、版本和企业治理的系统设计门禁。

面试表达：如果一个设计缺 replay、缺 trace、缺权限或缺恢复，即使模块图很完整，也不能称为生产级 Agent Harness。

Harness Pitfall Audit

一句话定义：harness pitfall audit 是把 Coding Agent 的实战失败样本整理成权限、上下文、编辑、命令、外部内容、trace、replay、eval、成本和环境一致性的统一审计表。

面试表达：回答实战坑时不要只列“加权限、加日志”，要说明如何把事故样本抽象成可复盘指标、failed gates 和 root causes。

Harness Triage Coverage

一句话定义：harness triage coverage 衡量失败样本是否已经完成目标、上下文、工具、权限、执行、状态、观测和用户影响归因。

Permission Overreach

一句话定义：permission overreach 是 Agent 或工具执行了超出任务、用户批准、组织策略或 runtime 权限边界的动作。

Context Overload Incident

一句话定义：context overload incident 是大文件、大日志、过多 MCP tools 或失真 summary 挤占关键上下文，导致 Agent 忘记目标或误用证据的事故。

User Edit Overwrite

一句话定义：user edit overwrite 是 Agent 在没有 before hash、dirty check 或冲突处理的情况下覆盖用户并发修改。

Shell Hang Incident

一句话定义：shell hang incident 是终端命令缺少 timeout、cwd / env 记录、子进程管理或输出控制，导致 session 卡死或无法复盘的事故。

Prompt Injection Boundary Miss

一句话定义：prompt injection boundary miss 是外部 issue、网页、README、MCP 输出或工具结果被当成可信指令，导致权限、上下文或执行边界被污染。

Trace Missing Incident

一句话定义：trace missing incident 是事故只剩最终回答或零散日志，缺失 context、tool call、permission、diff、error、version 和 artifact，无法稳定定位根因。

Eval Flakiness Incident

一句话定义：eval flakiness incident 是评估缺 repo snapshot、固定工具版本、确定性外部依赖、版本捕获或重复样本，导致 pass rate 不可比。

Doom Loop Guard

一句话定义：doom loop guard 是检测重复工具调用、重复读取、重复失败测试、预算超限和无状态更新循环的 runtime 防护。

MCP Tool Overload

一句话定义：MCP tool overload 是 MCP server 或 tools 过多、命名不清、schema 占用过大、认证状态不可见或输出过长，导致模型误选工具和上下文成本失控。

Environment Parity Check

一句话定义：environment parity check 衡量本地、云端、sandbox、测试镜像和 shell 环境是否有可比较的 fingerprint、cwd、PATH、依赖和系统版本。

Harness Pitfall Gate

一句话定义：harness pitfall gate 是基于 triage、permission、context、edit、command、injection boundary、trace、replay、eval、cost loop 和 environment parity 的实战坑门禁。

面试表达：生产级 harness 事故复盘要能说明 gate 为什么失败、对应根因是什么、如何通过权限、上下文、trace、replay、eval 和环境治理防止复发。

Browser Agent

一句话定义：browser agent 是能在浏览器中观察网页、点击、输入、滚动、导航并完成任务的 Agent。

适用场景：没有稳定 API、需要操作现有网页、跨系统流程、内部后台表单和网页信息收集。

风险：页面改版、弹窗、慢加载、误点击、登录状态、网页注入和高风险提交。

面试表达：能用稳定 API 时优先 API；browser agent 适合 UI 必须参与的任务，但要有状态追踪、权限和确认。

Computer Use Agent

一句话定义：computer use agent 是能操作通用 GUI 或桌面环境的 Agent。

区别：它不只控制浏览器，还可能操作桌面应用、文件窗口、表格软件、终端和多窗口环境。

风险：可操作范围更广，文件系统、剪贴板、窗口焦点和坐标误点都会带来更高风险。

GUI Action

一句话定义：GUI action 是 UI Agent 的结构化动作，例如 click、type、scroll、select、wait、drag 和 press key。

工程要求：动作应记录目标元素、role、坐标、输入值、风险级别、确认状态和预期状态变化。

Screenshot Observation

一句话定义：screenshot observation 是从屏幕图像读取当前 UI 状态。

优点：通用，适合没有 DOM 的桌面应用；缺点是可能误读布局、遮挡、弹窗和小字。

Accessibility Tree

一句话定义：accessibility tree 是 UI 元素的可访问性结构，包含角色、名称、状态和层级。

为什么重要：它通常比纯截图更可解释，也比坐标更稳定，适合定位按钮、输入框、链接和菜单。

UI State Tracking

一句话定义：UI state tracking 是持续记录 URL、窗口、焦点、已填字段、弹窗、错误提示、下载状态和提交状态。

常见失败：不更新状态会导致重复点击、重复提交、在错误页面继续操作或忘记已完成步骤。

Misclick Rate

一句话定义：misclick rate 是 click action 中点击目标不匹配预期目标的比例。

为什么重要：UI Agent 的误点击可能造成真实副作用，尤其是删除、支付、发送和提交类按钮。

Form Accuracy

一句话定义：form accuracy 是表单字段填写值与期望值匹配的比例。

评估重点：字段选择、焦点、值格式、是否覆盖已有内容和提交前复核。

High-Risk Action Protection

一句话定义：high-risk action protection 是对支付、删除、发送、提交、权限修改等动作的确认、阻断或草稿化机制。

面试表达：高风险动作不能由模型悄悄完成，必须有用户确认、摘要、审计和必要的回滚方案。

UI Agent Gate

一句话定义：UI Agent gate 是基于任务成功率、最终验证、动作准确率、误点击率、表单准确率、状态观察、高风险保护、注入拦截和重复动作的上线门禁。

面试表达：UI Agent 是否可靠不能只看能否完成任务，还要看是否安全、可控、少误点、能恢复、能验证。

Multi-Agent

一句话定义：Multi-Agent 是把多个带角色、工具、权限、上下文和预算的 Agent 组织起来协作、并行或互相验证的系统。

为什么重要：它能把复杂任务拆给 planner、researcher、coder、reviewer、judge 等角色，但也会增加通信成本、冲突处理、重复劳动和权限管理难度。

面试表达：Multi-Agent 不是 Agent 越多越好，而是要证明它相对单 Agent baseline 提升了任务成功率、验证质量、并行效率或权限隔离能力。

Coordinator

一句话定义：coordinator 是 Multi-Agent 系统中负责任务拆解、角色分配、状态收集、冲突处理、预算控制和最终输出的调度者。

工程要点：coordinator 最好像状态机和审计器，而不是自由聊天者；它要控制可见上下文、工具权限、停止条件和 trace。

Role Assignment

一句话定义：role assignment 是把子任务分配给匹配角色的 Agent，例如把证据检索交给 researcher，把 diff 检查交给 reviewer。

评估重点：看角色匹配率、任务完成率、重复分配率和高风险任务是否分给了有权限且需确认的角色。

Communication Protocol

一句话定义：communication protocol 是 Agent 间消息的结构化约定，通常包含 sender、receiver、task id、intent、content、evidence、confidence 和 status。

为什么重要：结构化消息能减少误解，支持日志回放、自动评估、权限审计和冲突定位。

Blackboard

一句话定义：blackboard 是 Multi-Agent 系统中的共享任务状态表，用来记录目标、计划、证据、子任务状态、冲突点和最终决策。

边界：不是所有 Agent 都应读取全部 blackboard；工程上要按角色、任务和权限过滤可见状态，避免上下文膨胀和信息污染。

Agent Debate

一句话定义：agent debate 是让多个 Agent 提出观点、反驳和证据，再由 judge、verifier 或 coordinator 汇总的协作模式。

面试表达：Debate 能暴露不同假设，但不是万能验证器；可执行任务最好结合工具验证、测试、引用证据或人工复核。

Judge Agent

一句话定义：judge agent 是在多个候选答案、冲突结论或辩论观点之间做结构化裁决的 Agent。

风险：judge 可能被流畅表达、长度偏差或共享错误证据误导，所以需要 rubric、证据检查和高风险人工升级。

Consensus / Voting

一句话定义：consensus / voting 是让多个 Agent 独立产出候选结论，再通过多数投票、加权投票或证据加权汇总得到结果。

边界：投票只在候选答案相对独立、答案空间清晰时更可靠；多个同源 Agent 可能共享同一错误。

Conflict Resolution

一句话定义：conflict resolution 是 Multi-Agent 系统识别并处理事实、方案、优先级、工具结果、权限或资源冲突的机制。

工程策略：低风险冲突可用证据、工具或规则解决；高风险冲突应触发人工确认或确定性系统复核。

Duplicate Work Rate

一句话定义：duplicate work rate 是多个 Agent 重复处理同一子任务的比例。

为什么重要：重复劳动会增加 token、延迟和协调成本，说明任务分配或共享状态设计可能有问题。

Single-Agent Lift Rate

一句话定义：single-agent lift rate 是 Multi-Agent 相比单 Agent baseline 在任务成功或质量上的提升比例。

面试表达：Multi-Agent 的比较对象应该是更简单的单 Agent 或固定 workflow，而不是空系统。

Multi-Agent Gate

一句话定义：Multi-Agent gate 是基于任务成功率、单 Agent 增益、角色匹配率、消息有效率、证据支持率、冲突解决率、重复劳动率、权限违规率和成本的上线门禁。

面试表达：Multi-Agent 上线前必须证明“更好、可控、可审计、低重复、低权限风险、成本可接受”，不能只展示一段看起来热闹的协作 trace。

Agent Evaluation

一句话定义：agent evaluation 是评估 agent 是否能稳定完成多步任务的过程。

为什么难：agent 的结果不仅取决于最终回答，还取决于工具选择、调用参数、执行顺序、错误恢复、成本、耗时和安全性。

评估维度：任务成功率、步骤正确率、工具调用准确率、无效调用率、平均成本、平均步数、失败恢复能力、权限合规性和用户满意度。

常见方法：构造端到端任务集，记录 action trace，做人工审查，使用 sandbox 验证执行结果，比较不同 agent 策略的成功率和成本。

面试表达：评估 agent 不能只看最终文本，要看执行轨迹是否正确、可控、低成本且安全。

Evaluation Harness

一句话定义：evaluation harness 是自动运行评估任务、初始化环境、执行模型或 Agent、收集 trace、计算指标、保存结果并生成回归报告的工程框架。

面试表达：Agent evaluation harness 评估的是模型、runtime、工具、环境、权限策略和验收器组成的整体系统。

Eval Dataset Bucket Coverage

一句话定义：eval dataset bucket coverage 是评估集覆盖 bugfix、feature、refactor、safety、tool use、code review 等关键任务桶的比例。

工程价值：只测 happy path 会高估 Agent；上线门禁必须看分桶指标和长尾 bad case。

Environment Reproducibility

一句话定义：environment reproducibility 是评估任务能否以相同 repo commit、依赖锁、sandbox 镜像、工具版本和环境 checksum 重建的比例。

为什么重要：环境不可复现时，分数变化可能来自依赖、网络或测试数据变化，而不是 Agent 能力变化。

Validator Coverage

一句话定义：validator coverage 是评估样本是否绑定了可执行验收器，例如单元测试、diff scope、格式校验、rubric judge、安全门禁或业务状态检查。

边界：没有验收器的样本可以用于探索，但不适合直接作为发布门禁。

Weighted Task Success

一句话定义：weighted task success 是按样本权重计算的任务成功率，可给安全样本、关键客户场景或高价值任务更高权重。

面试表达：Agent 评估不能只看未加权平均分，高风险失败应被更重地计入发布决策。

Diff Scope Safety

一句话定义：diff scope safety 是 Coding Agent 的实际修改文件集合落在允许修改范围内的比例。

常见失败：任务完成但顺手修改无关配置、格式化整个仓库或触碰用户未要求文件。

Baseline Fairness

一句话定义：baseline fairness 是候选 Agent 和 baseline 在数据集、环境、工具权限、预算和成功条件上可比的

比例。

为什么重要：候选拿到更多权限、更大预算或更宽成功条件时，分数提升不能直接说明能力更强。

Regression Pass Rate

一句话定义：regression pass rate 是候选 Agent 不破坏 baseline 已通过任务的比例。

工程价值：新模型、新 prompt 或新工具策略上线前，必须证明旧能力没有关键回退。

Unsafe Execution Rate

一句话定义：unsafe execution rate 是安全诱导或高风险动作尝试中，不安全动作被实际执行的比例。

上线要求：该指标应为 0；安全样本成功率不能被普通任务成功率掩盖。

Cost Budget Pass Rate

一句话定义：cost budget pass rate 是评估样本在 token、wall-clock、工具调用、sandbox 和人工审核预算内完成的比例。

面试表达：成功率提升但成本或延迟大幅超预算，不一定适合上线。

Flaky Task Rate

一句话定义：flaky task rate 是重复运行同一评估样本时结果不一致的比例。

工程策略：flaky 样本要隔离、标记或修复，不能当作稳定回归证据。

Eval Version Capture

一句话定义：eval version capture 是评估结果是否记录 dataset、agent、model、prompt、tools、environment 和 permission policy 版本的比例。

为什么重要：没有版本捕获，评估报告无法解释分数变化来源。

Eval Report Completeness

一句话定义：eval report completeness 是报告是否包含 overall、by bucket、baseline delta、failures、cost、safety、flaky 和 rollout recommendation 等关键部分。

工程价值：好的评估报告要指导发布决策，而不是只展示一个平均分。

Evaluation Harness Gate

一句话定义：evaluation harness gate 是基于任务桶覆盖、环境可复现、验收器覆盖、任务成功、部分成功、diff scope、trace、baseline fairness、regression、安全、成本、flaky、版本捕获和报告完整性的评估门禁。

面试表达：Agent 变好必须通过系统级评估证明，而不是展示一个单任务 demo。

Agent Benchmark

一句话定义：agent benchmark 是用于评估 Agent 在真实或模拟环境中完成多步任务能力的任务集和验收协议。

核心要求：固定初始状态、工具版本、权限策略、评估脚本、日志格式和重置机制，否则结果不可复现。

Eval Sample

一句话定义：eval sample 是 Agent 评估中的单个任务样本，包含目标、初始状态、工具集合、权限策略、验收器、风险等级和权重。

为什么重要：如果样本只包含简单 happy path，就会高估 Agent 的真实能力。

Trajectory Evaluation

一句话定义：trajectory evaluation 是评估 Agent action / observation / state 序列是否正确、必要、可复现且安全的过程。

面试表达：Agent 的价值和风险都在轨迹里，不能只看 final answer。

Partial Success Score

一句话定义：partial success score 是用分级 rubric 衡量 Agent 部分完成复杂任务的指标。

适用场景：开放任务、多步骤任务和有多个验收点的任务，比单纯成功 / 失败更能解释问题。

Trace Completeness

一句话定义：trace completeness 是执行轨迹是否完整记录目标、动作、参数、observation、状态更新、错误和最终输出的比例。

工程价值：trace 不完整会导致无法 replay、无法定位失败原因，也无法证明最终总结是否真实。

Summary Faithfulness

一句话定义：summary faithfulness 是 Agent 最终总结是否忠实反映实际 trace 的比例。

常见失败：没有运行测试却声称测试通过，未提交表单却声称提交成功，或隐瞒中间错误。

Claim Support Rate

一句话定义：claim support rate 是最终输出中的 claim 被 trace、工具结果或证据支持的比例。

为什么重要：它能把“漂亮总结”拆成可审计声明，发现未执行、未验证或无证据支持的结论。

Recovery Success Rate

一句话定义：recovery success rate 是工具失败、权限不足、页面变化、测试失败等异常出现后，Agent 成功调整策略的比例。

边界：如果评估集没有失败路径，这个指标不能证明 Agent 具备恢复能力。

Cost-Quality Trade-off

一句话定义：cost-quality trade-off 是 Agent 成功率、质量、延迟、token、工具调用和人工审阅成本之间的权衡。

面试表达：成功率提升很小但成本增加很多的 Agent 方案，未必值得上线。

Regression Suite

一句话定义：regression suite 是固定的 Agent 回归任务集，覆盖常见成功任务、历史失败任务、安全边界、工具异常、长上下文和多轮任务。

工程价值：prompt、工具 schema、模型版本、controller、memory 或权限策略更新后，都应跑回归集。

Agent Eval Gate

一句话定义：Agent eval gate 是基于任务成功率、部分成功分、工具准确率、参数合法率、observation 使用、状态更新、summary 忠实性、claim 支持、安全、成本和延迟的上线门禁。

面试表达：Agent 上线评估不能只看成功率，还要证明过程可信、工具正确、总结忠实、权限安全、成本可控。

Agent Safety

一句话定义：Agent safety 是限制 Agent 工具调用、数据访问、外部动作和长期记忆风险的系统设计。

为什么重要：Agent 能读写文件、调用 API、操作网页和执行工具，事故半径比普通文本模型更大。

Least Privilege

一句话定义：least privilege 是只给 Agent 完成任务所需的最小工具、数据、时间和作用域权限。

面试表达：不要让模型自己判断权限，权限必须由系统层和工具层执行。

Tool Permission Matrix

一句话定义：tool permission matrix 是按用户、任务、工具、风险等级和作用域定义允许动作的权限表。

工程价值：它把只读、写入、外部传输、高风险动作和人工确认规则显式化。

Permission Request

一句话定义：permission request 是 Agent 执行动作前提交给 runtime 的结构化授权请求，通常包含用户、任务、动作、资源、风险、副作用、策略结果、确认、dry run、沙箱和 outcome。

工程价值：结构化请求能让权限判断从自然语言提醒变成可审计、可回放、可统计的系统行为。

Least Privilege Coverage

一句话定义：least privilege coverage 是实际执行动作落在当前任务最小权限范围内的比例。

面试表达：如果 code review 任务执行了网络上传、安装依赖或写入外部系统，即使最终回答正确，也说明最小权限门禁失败。

Permission Matrix Coverage

一句话定义：permission matrix coverage 是权限请求能在权限矩阵中找到明确策略行的比例。

工程价值：新增 MCP 工具、外部 API 或 Git 动作时，必须先补矩阵行，否则容易落到过宽默认权限。

Unauthorized Action Block Rate

一句话定义：unauthorized action block rate 是应拒绝的越权请求被 runtime 阻断的比例。

面试表达：Agent 安全不能只看模型是否“愿意遵守”，而要看被判定为 deny 的动作是否真的没有执行。

Secret Access Block Rate

一句话定义：secret access block rate 是涉及 API key、token、cookie、SSH key、.env 和生产连接串的请求被阻断的比例。

边界：secret 既不能进入模型上下文，也不能以原文形式进入日志和 trace。

Network Egress Control

一句话定义：network egress control 是对下载、上传、外部 API、依赖安装和浏览器外发做 allowlist、确认和审计的控制指标。

工程价值：很多 prompt injection 的目标不是直接破坏文件，而是诱导 Agent 把代码、日志或 secret 发到外部。

Sandbox Enforcement Coverage

一句话定义：sandbox enforcement coverage 是需要沙箱执行的动作被拒绝或实际进入沙箱的比例。

常见动作：未知测试、install script、用户上传项目、浏览器自动化、benchmark 评估和高风险命令。

Dry Run Coverage

一句话定义：dry run coverage 是需要预览影响范围的高风险动作先完成 dry run 或被拒绝的比例。

工程价值：写文件 diff、数据库迁移、批量删除、外部发送和权限变更都应该先让用户看到影响对象和预期副作用。

Audit Log Completeness

一句话定义：audit log completeness 是权限 trace 中 request id、用户、任务、动作、资源、风险、策略结果、确认、沙箱、dry run 和 outcome 等必填字段完整的比例。

边界：审计日志也要脱敏，否则日志会变成新的敏感数据泄漏面。

Irreversible Action Protection

一句话定义：irreversible action protection 是删除、发送、支付、发布、权限修改、数据库迁移和 Git push 等不可逆动作被拒绝或经过确认 / dry run 的比例。

面试表达：不可逆动作不应该和普通测试命令共用一个自动执行策略。

Permission Sandbox Gate

一句话定义：permission sandbox gate 是基于最小权限、矩阵覆盖、越权阻断、人工确认、secret 阻断、网络外发、沙箱执行、dry run 和审计完整性的权限上线门禁。

面试表达：Agent 权限设计的目标不是“弹更多确认框”，而是把真实副作用放进可解释、可限制、可复盘的控制面。

Untrusted Content Boundary

一句话定义：untrusted content boundary 是把网页、文档、邮件、日志、数据库字段和工具返回视为低信任数据的边界。

面试表达：外部内容可以作为 evidence，不能覆盖系统指令、开发者规则或用户目标。

Data Flow Guard

一句话定义：data flow guard 是检查敏感数据是否被输出、写入 memory、传给外部工具或跨用户共享的控制层。

为什么重要：Agent 的数据泄漏常发生在工具链和日志链路中，而不只发生在最终回答里。

Sensitive Data Filter

一句话定义：sensitive data filter 是在工具输入、工具输出、日志、memory 和最终回答中识别并脱敏或阻断敏感字段的机制。

边界：过滤器不能代替权限系统，只能作为数据流防线之一。

External Transfer Gate

一句话定义：external transfer gate 是内部数据传给外部 API、邮件、网页或第三方服务前的权限、脱敏和确认门禁。

面试表达：外部传输要先问“谁的数据、给谁、为什么、是否脱敏、是否可审计”。

Sandbox Policy

一句话定义：sandbox policy 是限制 Agent 执行环境中文件系统、网络、CPU、内存、子进程、命令和工作目录的策略。

工程价值：沙箱把模型生成动作的影响范围限制在可控环境内。

Human Confirmation

一句话定义：human confirmation 是高风险动作执行前让用户确认动作、对象、参数、影响范围和预览结果的机制。

常见动作：删除、发送、支付、提交、发布、权限修改和外部写入。

Dry Run

一句话定义：dry run 是在执行高风险动作前先展示将影响的对象、参数和结果预览，而不产生真实副作用。

为什么重要：dry run 能把不可逆操作变成可审查、可取消的步骤。

Audit Log

一句话定义：audit log 是记录用户请求、Agent 决策、工具调用、参数摘要、权限检查、确认、阻断、错误和最终输出的日志。

边界：审计日志本身也要脱敏，不能成为新的数据泄漏源。

Irreversible Action

一句话定义：irreversible action 是难以撤销或影响外部状态的动作，例如删除、发送、支付、提交、发布和权限修改。

工程策略：默认禁止，确认后执行，优先 dry-run，保留审计和补救路径。

Memory Safety

一句话定义：memory safety 是防止长期 memory 保存敏感信息、不可信内容、过期事实或跨用户数据的机制。

面试表达：Memory 不能成为低信任内容或隐私数据的长期存储。

Supply Chain Risk

一句话定义：supply chain risk 是 Agent 安装依赖、运行脚本、下载二进制或调用第三方服务时引入的来源和版本风险。

工程策略：使用白名单、锁定版本、沙箱执行、来源记录和审批门禁。

Safety Alternative

一句话定义：safety alternative 是当高风险或越权动作被阻断时，Agent 提供的安全替代方案，例如解释限制、生成草稿、请求确认或建议人工处理。

为什么重要：安全不是只拒绝，还要尽量保留有用性。

Agent Safety Gate

一句话定义：Agent safety gate 是基于越权拦截、不可信内容隔离、敏感数据阻断、外部传输门禁、高风险保护、dry-run、沙箱、memory 和审计完整性的上线门禁。

面试表达：Agent 安全上线要用系统层指标证明，而不是只说 prompt 已经要求模型遵守规则。

Agent Interview Readiness

一句话定义：Agent interview readiness 是候选人能否把 Agent 架构、工具调用、planning、memory、RAG、Code Agent、UI Agent、Multi-Agent、评估和安全稳定讲清楚的准备度。

面试表达：准备好不是背完题库，而是每道题都能落到结构、指标、demo、项目证据和 trade-off。

Agent Interview Rubric

一句话定义：Agent interview rubric 是用于给 Agent 面试回答打分的评分规约，通常覆盖概念、公式、demo、trace 指标、安全、评估、项目证据和取舍深度。

工程价值：rubric 能把模拟面试从主观感觉变成可复盘的缺口清单。

Agent Answer Coverage

一句话定义：Agent answer coverage 是回答覆盖目标概念、机制、指标、边界和项目证据的比例。

常见问题：覆盖率低的回答通常表现为只讲术语，不讲 controller、trace、评估或安全门禁。

Agent Formula Coverage

一句话定义：Agent formula coverage 是回答中能否准确写出或解释任务成功率、工具准确率、参数合法率、observation 使用率、trace 完整率和安全门禁等指标公式。

面试表达：公式不必复杂，但要说明每个指标在抓哪类失败。

Agent Demo Coverage

一句话定义：Agent demo coverage 是回答能否配套一个最小可运行或伪代码 demo，例如 tool call audit、ReAct trace audit、memory audit、Agent evaluation audit 或 Agent safety audit。

为什么重要：demo 能证明候选人不只是会讲概念，还理解数据结构和评估口径。

Trace Metric Coverage

一句话定义：trace metric coverage 是回答是否覆盖 Agent 执行轨迹中的关键指标，例如 trace completeness、summary faithfulness、claim support、recovery、成本和延迟。

面试表达：Agent 的价值和风险都在 trace 里，不能只看最终答案。

Project Evidence Score

一句话定义：project evidence score 是项目回答中 baseline、指标、测试、bad case、个人贡献和复盘证据的覆盖程度。

常见问题：只说“我做了一个 Agent”但没有 baseline、指标和失败样本，项目可信度很弱。

Trade-off Depth

一句话定义：trade-off depth 是回答能否讲清 Agent vs workflow、API vs browser、single agent vs multi-agent、质量 vs 成本、自主性 vs 可控性的取舍深度。

面试表达：高级回答不是只说方案可行，而是解释为什么这个场景值得这样设计。

Weak Agent Question

一句话定义：weak agent question 是模拟面试中得分低或触发红旗的问题，通常对应缺概念、缺公式、缺 demo、缺项目证据或安全边界不清。

复盘方法：每个弱题绑定一个公式、一个 demo、一个 bad case 和一个 3 分钟回答模板。

Agent Revision Plan

一句话定义：Agent revision plan 是针对弱题生成的修复计划，明确缺失项、红旗、下一步补充材料和复测方式。

工程价值：它把“继续复习”改成可执行的学习任务。

Agent Interview Gate

一句话定义：Agent interview gate 是基于平均回答分、最低单题分、安全覆盖、红旗数量和项目证据的面试准备度门禁。

面试表达：通过这个门禁，才说明候选人能把 Agent 讲成可落地、可评估、可控制的系统。

Tool Skill Developer Experience Audit

一句话定义：Tool Skill developer experience audit 是检查开发者是否能从 quickstart、脚手架、schema、示例、本地调试、trace、审核反馈和迁移文档中顺利完成 Tool / Skill 上架的审计方法。

面试表达：工具生态质量不能只靠审核兜底，必须把正确开发路径做成默认路径。

Quickstart Path Clarity

一句话定义：quickstart path clarity 是快速开始文档能否让开发者在短时间内跑通第一个符合规范的 Tool / Skill。

常见问题：只讲概念、不提供 hello world、示例不可运行，都会让开发者从第一步就走偏。

Scaffold Completeness

一句话定义：scaffold completeness 是脚手架是否生成 manifest、schema、handler、tests、eval stub、README 和安全占位等必要结构。

工程价值：脚手架不是省几行代码，而是把权限、错误、测试和评估规范前置。

Documentation Example Coverage

一句话定义：documentation example coverage 是文档示例是否覆盖 happy path、错误路径、权限不足、边界输入和输出格式。

面试表达：只给 happy path 会让开发者忽略错误恢复和权限边界。

Local Debug Readiness

一句话定义：local debug readiness 是开发者能否在不上架的情况下运行 sample task、lint、eval、权限模拟和 trace 查看。

为什么重要：没有本地调试，低质量能力会被推到审核或线上才暴露。

Trace Replay Readiness

一句话定义：trace replay readiness 是工具 / Skill 的调用轨迹是否足够完整，可以复现模型选择、参数生成、权限检查、执行结果和最终使用方式。

工程价值：trace / replay 是排障、评估、审核和安全追责的共同事实层。

Review Feedback Actionability

一句话定义：review feedback actionability 是审核反馈是否指出具体字段、风险原因、修改建议和复测方式。

常见问题：“审核不通过，请修改” 不可定位、不可修复，也不可复测。

Security Documentation Coverage

一句话定义：security documentation coverage 是文档是否说明权限 reason、敏感数据处理、高风险确认、prompt injection 防护、输出脱敏和审计要求。

面试表达：安全文档不是附录，而是开发者写出安全工具的入口。

Migration Documentation Coverage

一句话定义：migration documentation coverage 是版本升级文档是否说明 breaking changes、新增权限、输入输出变化、配置迁移、示例更新和回滚方式。

工程价值：迁移文档缺失会让生态被版本升级拖垮。

DX Documentation Gate

一句话定义：DX documentation gate 是基于 quickstart、脚手架、schema、示例、本地调试、trace、审核反馈、安全文档、迁移文档、SDK / CLI 一致性、owner 和门户路径的开发者体验门禁。

面试表达：好的工具生态要能证明开发者从 0 到上架的路径可复制、可调试、可审核、可迁移。

Composable Workflow Audit

一句话定义：Composable workflow audit 是检查一个多步骤工具工作流是否具备图合法性、依赖无环、输入输出契约、状态、权限、数据流、重试、补偿、审批、取消、trace、eval 和版本兼容的审计方法。

面试表达：可组合工作流不是把工具串起来，而是把复杂任务沉淀成可恢复、可审计、可评估、可治理的任务编排。

Workflow Graph Validity

一句话定义：workflow graph validity 是工作流中所有节点、边和依赖引用是否合法，是否不存在未知节点、自依赖或无法解释的边。

常见问题：流程图里出现“隐式步骤”或引用不存在的步骤，会让恢复、重试和 trace 无法可靠执行。

Dependency Acyclicity

一句话定义：dependency acyclicity 是工作流依赖图是否无环，避免 A 等 B、B 又等 A 的无限等待或重复执行。

工程价值：DAG 不是形式主义，它决定调度器能否拓扑排序、并行执行和定位失败。

Workflow IO Contract

一句话定义：workflow IO contract 是每个 step 是否声明必需输入、产生输出、输出类型和后续使用方式。

面试表达：没有输入输出契约，workflow 就无法稳定组合、测试和回放。

Condition Determinism

一句话定义：condition determinism 是条件分支是否有明确判断条件，而不是让模型临时凭感觉选择路径。

常见问题：高风险分支如果写成“模型判断是否发送”，通常无法审计、复现和回归。

Workflow State Coverage

一句话定义：workflow state coverage 是 workflow 状态机是否覆盖 pending、running、waiting、completed、failed、cancelled 等关键状态。

工程价值：状态覆盖是恢复、取消、展示进度、重试和人工接管的基础。

Workflow Permission Boundary

一句话定义：workflow permission boundary 是每个 step 是否使用最小权限，并且不会把上游用户、Agent 或工具权限不受控地转交给下游。

面试表达：Workflow 编排权限，不等于获得所有被编排工具的最大权限。

Workflow Data Flow Policy

一句话定义：workflow data flow policy 是控制 artifact、工具结果和中间状态能否在步骤、Agent、租户或外部系统间转发的策略。

常见问题：只写步骤顺序、不写数据分类和转发策略，会让敏感中间结果沿 workflow 泄露。

Workflow Idempotency Coverage

一句话定义：workflow idempotency coverage 是有副作用步骤是否有 idempotency key 或等价去重机制。

工程价值：没有幂等，重试会把一次失败变成重复发送、重复创建或重复修改。

Retry Safety

一句话定义：retry safety 是 workflow 重试策略是否只对可重试错误生效，并且不会对非幂等副作用造成重复执行。

面试表达：retry 不是“失败就再试一次”，而是错误分类、预算、幂等和未知状态处理的组合。

Compensation Readiness

一句话定义：compensation readiness 是不可直接回滚的副作用步骤是否定义补偿动作、通知方式和作废策略。

工程价值：很多业务动作不能真正撤销，只能通过更正、关闭、标记作废或人工处理完成补偿。

Workflow Human Approval Coverage

一句话定义：workflow human approval coverage 是外部发送、生产写入、删除、共享敏感数据等高风险步骤是否有人工确认点。

面试表达：确认点要展示动作、输入、风险、可撤销性、请求者和已完成步骤，而不是一个孤立按钮。

Workflow Trace Replay Readiness

一句话定义：workflow trace replay readiness 是 workflow 是否记录 workflow id、run id、step id、状态、输入输出引用、actor、版本和错误语义，足以复盘和回归。

工程价值：复杂 workflow 出错时，最终答案通常不够，必须靠 step-level trace 定位漏步骤、错分支、越权或补偿失败。

Workflow Eval Coverage

一句话定义：workflow eval coverage 是工作流是否有 golden set、回归集、安全样本、失败恢复样本和 trace replay 评估。

常见问题：只测 happy path 的端到端成功率，会漏掉取消、超时、补偿、审批和数据流泄露风险。

Workflow Gate

一句话定义：workflow gate 是基于图合法性、依赖无环、IO、条件、状态、权限、数据流、重试、补偿、审批、取消、trace、eval 和版本兼容的上线门禁。

面试表达：可组合 workflow 过门禁，才说明它可以被 Skill、MCP、A2A 和 Marketplace 复用，而不是一次性的 prompt 流程图。

Prompt Injection Defense Audit

一句话定义：prompt injection defense audit 是检查工具协议是否把不可信网页、文档、邮件、Issue、日志、RAG chunk、tool result 和其他 Agent 输出限制为 data，并用 policy gate、权限、确认、沙箱、脱敏、trace 和 eval 防止其驱动危险动作的审计方法。

面试表达：prompt injection 防御不是看模型有没有“识破”某段文本，而是看 runtime 是否能证明不可信内容无法升级为指令或越权工具调用。

Instruction Data Separation

一句话定义：instruction data separation 是把 system / developer / 用户授权任务与网页、文档、工具返回、日志等不可信内容在消息结构、上下文模板和执行策略中显式分开。

常见问题：把外部网页正文直接拼进同一段 prompt，会让模型更容易把数据里的命令式文本误当成上层指令。

Source Trust Labeling

一句话定义：source trust labeling 是为每段 tool result、resource、RAG chunk、artifact 或跨 Agent 消息记录来源、可信级别、租户、版本和证据引用。

工程价值：没有来源和 trust level，后续权限、脱敏、引用、外发审批和事故复盘都无法做出可靠判断。

Taint Propagation

一句话定义：taint propagation 是不可信内容在摘要、压缩、转发、写入 artifact 或跨 Agent 传递后继续保留 derived_from、trust 和 allowed use 标记。

面试表达：不可信内容不会因为被模型总结一次就变可信，摘要只能改变形式，不能洗掉来源。

Policy Pre-Tool Gate

一句话定义：policy pre-tool gate 是工具调用前由 runtime 或 policy engine 检查用户授权、工具风险、数据来源、敏感字段、确认状态、沙箱策略和外发目标。

常见误区：让模型自己回答“这次是否安全”不是硬门禁，只能作为辅助信号。

Risky Tool Isolation

一句话定义：risky tool isolation 是把发送、删除、写生产、执行 shell、上传外部系统等高风险工具从普通 auto 候选集中隔离出来，并要求显式权限、确认或审批。

工程价值：高风险工具越容易被模型自动选择，间接 prompt injection 的事故半径越大。

Untrusted Action Blocking

一句话定义：untrusted action blocking 是阻止 external_untrusted 或 derived_from_untrusted 内容直接触发外发、删除、执行、修改、支付、审批等动作。

面试表达：不可信内容可以被读取、总结和引用，但不能成为调用危险工具的授权理由。

Sensitive Data Control

一句话定义：sensitive data control 是对密钥、PII、客户数据、内部文档和权限结果做分类、最小投影、脱敏、外发审批和日志遮蔽。

常见问题：只做 prompt injection 文本过滤但不做数据流控制，仍可能让模型把敏感字段带入高风险工具参数。

Prompt Injection High Risk Confirmation

一句话定义：prompt injection high risk confirmation 是高风险动作确认时展示工具名、参数、数据来源、taint、敏感字段、风险、可撤销性和阻断理由。

工程价值：确认不是一个“是否继续”按钮，而是让用户或审批人看见不可信内容是否影响了动作。

Sandbox Enforcement

一句话定义：sandbox enforcement 是对浏览器、shell、代码执行、文件修改和外部网络访问施加文件系统、网络、命令、时间、内存、输出大小和可写目录限制。

面试表达：Prompt injection 不可能完全消失，所以危险执行面必须被沙箱限制损害范围。

Output Redaction Projection

一句话定义：output redaction projection 是工具结果进入模型上下文前只保留必要字段，并对密钥、PII、内部路径、原始日志和不可外发字段做脱敏或删除。

常见问题：工具执行成功不代表结果可以原样塞回上下文，原始结果可能同时包含敏感数据和注入文本。

RAG Source Boundary

一句话定义：RAG source boundary 是检索文档只能作为证据参与回答，不能改变系统策略、引用要求、工具权限或外发策略。

面试表达：RAG 文档的权力边界是 evidence，不是 policy。

Multi-Agent Taint Propagation

一句话定义：multi-agent taint propagation 是跨 Agent 消息、摘要和 artifact 继续传递 source、trust、taint、derived_from 和 allowed use，防止不可信内容被下游 Agent 当成可信上下文。

工程价值：Multi-Agent 系统里最隐蔽的注入事故往往发生在“上游 Agent 已经总结过，所以应该可信”的假设上。

Prompt Injection Trace Readiness

一句话定义：prompt injection trace readiness 是 trace 中记录外部来源、taint、策略决策、候选工具、阻断原因、确认状态、沙箱配置、版本和 eval 标签。

面试表达：没有 trace，就无法区分模型误判、权限缺口、数据流缺口和确认流程缺口。

Prompt Injection Eval Coverage

一句话定义：prompt injection eval coverage 是安全回归集覆盖直接注入、间接注入、工具结果注入、RAG 注入、跨 Agent taint 丢失、敏感数据外发和高风险动作确认。

常见问题：只测干净任务成功率会掩盖安全回归，尤其是工具路由、上下文压缩和 Agent 升级后的边界漂移。

Prompt Injection Defense Gate

一句话定义：prompt injection defense gate 是基于指令数据分离、来源可信标记、taint、pre-tool policy、高风险工具隔离、不可信动作阻断、敏感数据控制、确认、沙箱、输出投影、RAG 边界、跨 Agent 传播、trace 和 eval 的综合上线门禁。

面试表达：门禁通过意味着系统能用审计证据证明不可信数据不能获得指令权限，也不能直接驱动高风险工具。

Tool Output Trust Audit

一句话定义：tool output trust audit 是检查工具输出、RAG chunk、artifact、数据库结果、网页内容和跨 Agent 输出是否具备来源、时间、版本、可信级别、访问范围、引用、证据链、claim 类型、冲突处理和 replay 证据的审计方法。

面试表达：工具输出不是天然事实，必须能解释每个结论来自哪里、为什么可信、有什么限制，以及哪些引用真正支持它。

Source Metadata Coverage

一句话定义：source metadata coverage 衡量工具输出是否记录 source id、URI、标题、owner、retrieved_at、数据版本和来源系统。

工程价值：没有来源元数据，引用、权限范围、版本回放和事故复盘都没有可追踪锚点。

Trust Level Coverage

一句话定义：trust level coverage 衡量工具输出是否标注 trusted_system、internal_verified、internal_unverified、user_provided、external_untrusted 或 generated_unverified 等可信级别。

面试表达：可信级别不是判断真假的绝对标签，而是系统做证据排序、权限、引用和风险说明的输入。

Freshness Version Coverage

一句话定义：freshness version coverage 衡量工具输出是否记录 retrieved_at、data_as_of、ttl、last_modified、etag、版本号或快照信息。

常见问题：库存、价格、日志、配置和政策文档都可能随时间变化，不说明新鲜度会把旧事实伪装成当前事实。

Access Scope Disclosure

一句话定义：access scope disclosure 衡量工具输出是否说明租户、区域、权限过滤、字段投影、aggregate_only 和数据范围。

工程价值：如果查询结果只覆盖某个区域或权限范围，最终回答就不能写成全局结论。

Citation Binding

一句话定义：citation binding 是最终回答使用的 citation_id 必须来自系统返回的可验证引用集合，而不是模型临时编造的链接或 source_uri。

面试表达：引用最好由系统生成和校验，模型只选择已存在引用并把它绑定到具体 claim。

Citation Support Accuracy

一句话定义：citation support accuracy 衡量引用片段是否真的支持对应 claim，而不是只和主题弱相关。

常见问题：答案引用了正确文档但不支持具体结论，用户看起来有来源，实际仍然是无证据回答。

Evidence Chain Completeness

一句话定义：evidence chain completeness 衡量 claim 能否追溯到工具调用、source、artifact、中间计算、derived_from 和版本。

工程价值：复杂结论通常不是一个 citation 就够，而是需要串起检索、查询、分析和摘要的证据链。

Claim Type Calibration

一句话定义：claim type calibration 是区分 fact、observation、inference、hypothesis、recommendation 和 unsupported claim，避免把假设写成事实。

面试表达：工具输出能证明观察，不一定能证明因果；能支持建议，不一定能支持确定结论。

Confidence Limitation Disclosure

一句话定义：confidence limitation disclosure 是每个重要 claim 都带 confidence、limitations、数据范围和不确定性说明。

常见问题：摘要时删除限制条件，会把权限过滤、聚合数据、弱相关证据或过期来源包装成高确定性结论。

Conflict Resolution Readiness

一句话定义：conflict resolution readiness 衡量多个来源冲突时，系统是否比较可信级别、owner 权威性、版本、新鲜度和访问范围，并在必要时升级人工确认。

工程价值：企业系统遇到冲突来源时，最差的做法是沉默地选一个看起来顺眼的结果。

Summary Provenance Retention

一句话定义：summary provenance retention 衡量工具输出被压缩、摘要或写入报告时，是否保留 citation、source scope、time window、confidence 和 limitations。

面试表达：摘要可以压缩文本，不能压缩掉证据来源和边界条件。

RAG Chunk Citation Accuracy

一句话定义：RAG chunk citation accuracy 衡量 RAG 引用是否保留 chunk id、doc uri、span、检索 / rerank 相关性和文档版本，并真正支持答案 claim。

常见问题：只看最终答案是否流畅，不看 chunk 是否支持 claim，会让 RAG 评估漏掉“有引用的幻觉”。

Multi-Agent Evidence Propagation

一句话定义：multi-agent evidence propagation 是跨 Agent 消息、artifact 和报告继续传递 evidence、produced_by、confidence、limitations 和 derived_from。

工程价值：下游 Agent 不能因为上游 Agent 已经总结过，就把生成内容当成无来源事实。

Provenance Trace Replay

一句话定义：provenance trace replay 是 trace 中记录 tool call id、source id、citation map、claim map、版本、权限过滤和中间 artifact，足以重放结论形成过程。

面试表达：用户质疑结果时，系统要能回答“用了哪些来源、当时版本是什么、哪些引用支持哪个结论”。

Citation Forgery Block

一句话定义：citation forgery block 是阻止最终回答引用不存在的 citation_id、未返回的 source_uri 或模型编造的文档标题。

工程价值：引用伪造会给幻觉披上可信外衣，因此必须由系统层校验，而不是靠模型自觉。

Tool Output Trust Eval Coverage

一句话定义：tool output trust eval coverage 是评估集覆盖引用准确率、无证据 claim、过期来源、冲突来源、权限过滤范围、RAG 弱相关 chunk 和跨 Agent 证据丢失。

常见问题：只测 task success 会漏掉错误来源、错误引用和证据链断裂。

Tool Output Trust Gate

一句话定义：tool output trust gate 是基于来源元数据、可信级别、新鲜度、访问范围、引用绑定、引用支持、证据链、claim 类型、confidence、冲突处理、摘要保真、RAG chunk、跨 Agent 传播、trace 和 eval 的综合上线门禁。

面试表达：门禁通过意味着系统不只是能调用工具，还能证明关键结论可追溯、可解释、可复查。

RAG Tool Agent Memory Integration Audit

一句话定义：RAG tool agent memory integration audit 是检查 RAG、Tool、Agent 和 Memory 是否职责清楚、上下文优先级稳定、证据不丢、状态可追踪、Memory 写入受控、安全边界可执行、分层 eval 完整的审计方法。

面试表达：复杂应用不是把四个模块堆在一起，而是证明它们各司其职并通过 runtime 组合成可控系统。

Capability Boundary Clarity

一句话定义：capability boundary clarity 衡量系统是否清楚区分 RAG 提供知识、Tool 执行动作、Agent 组织过程、Memory 保留长期状态。

常见问题：把 search tool 当完整 RAG、把 Agent 当工具路由器、把 Memory 当知识库，会导致权限、证据和评估边界混乱。

Orchestration Mode Fit

一句话定义：orchestration mode fit 衡量任务是否选择合适的 RAG-first、Tool-first、Memory-first、Workflow 或 Agent-planned 模式。

工程价值：简单政策问答不需要重型 Agent，高风险固定流程也不应完全交给动态规划。

Context Priority Enforcement

一句话定义：context priority enforcement 是 system policy、用户当前任务、权威工具结果、verified RAG、Memory、用户上传资料、外部网页和未验证 Agent 输出之间的优先级执行。

面试表达：Memory 不能覆盖系统策略，外部网页不能覆盖用户任务，旧偏好不能覆盖权威实时数据。

Context Budget Allocation

一句话定义：context budget allocation 衡量上下文窗口是否给用户任务、关键证据、工具结果、Memory 和 artifact 引用分配可解释预算。

常见问题：把冗长工具输出或无关 Memory 塞满上下文，会挤掉当前任务和关键证据。

Evidence Preservation

一句话定义：evidence preservation 是 RAG citation、Tool observation、artifact、claim map、limitations 和 source id 在 Agent 计划、摘要、报告和跨模块传递中不丢失。

工程价值：没有证据保留，最终答案无法解释，也无法定位错误发生在检索、工具、规划还是记忆。

Tool Observation Use

一句话定义：tool observation use 衡量 Agent 是否正确读取、解释并使用工具返回结果，而不是忽略、误读或过度外推。

面试表达：工具调用成功只是第一步，关键是 observation 是否被正确纳入状态和最终回答。

Agent State Update Discipline

一句话定义：agent state update discipline 是 Agent 是否记录计划、状态、完成标准、失败恢复、阻断原因和停止原因。

工程价值：没有状态更新 discipline，多步系统容易假完成、重复调用工具或在失败后编造结论。

Memory Read Relevance

一句话定义：memory read relevance 衡量 Memory 读取是否与当前任务相关，并按用户、团队、项目、租户和权限作用域过滤。

常见问题：读取大量无关历史会制造上下文污染和隐私风险。

Memory Write Gate

一句话定义：memory write gate 是长期 Memory 写入前检查长期价值、用户确认、敏感数据、可信来源、作用范围、TTL 和删除策略。

面试表达：Memory 最大风险不是读不到，而是把一次性错误、敏感结果或未验证内容长期保存。

Memory Scope Permission

一句话定义：memory scope permission 是 Memory 按用户、团队、项目和租户隔离，并支持查看、修改、删除和审计。

工程价值：Memory 是数据资产，不是免费的上下文缓存，必须受权限和生命周期治理。

Conflict Resolution Policy

一句话定义：conflict resolution policy 是 Memory、RAG、Tool result、实时数据库和旧状态冲突时，按权威性、新鲜度、权限范围和用户确认处理。

常见问题：Memory 保存旧政策时，如果 RAG 检索到新政策，应标记 Memory 过期或更新，而不是继续复用旧值。

Injection Propagation Control

一句话定义：injection propagation control 是阻止 RAG 文档、网页、Tool result 或其他 Agent 输出中的不可信文本触发高风险工具或写入长期 Memory。

面试表达：组合系统会放大间接 prompt injection，因为不可信内容可以从 RAG 传到 Agent，再传到 Tool 或 Memory。

Sensitive Data Memory Block

一句话定义：sensitive data memory block 是阻止密钥、PII、权限过滤结果、一次性查询结果和敏感 Tool output 写入长期 Memory。

工程价值：工具结果可以进入当前任务上下文，不代表可以成为长期记忆。

Trace Linkage Coverage

一句话定义：trace linkage coverage 衡量 trace 是否串联用户目标、RAG query、retrieved docs、tool calls、memory reads / writes、Agent state 和 final claims。

面试表达：trace 要能回答 “这个结论用了哪些证据、哪些工具、哪些记忆，以及哪一步决定写入或拒绝写入 Memory”。

Layered Eval Coverage

一句话定义：layered eval coverage 是评估集分别覆盖 RAG、Tool、Agent、Memory、end-to-end 和 safety，而不是只看最终答案。

常见问题：只看端到端成功率，无法知道失败来自召回、参数、规划、状态、记忆污染还是安全门禁。

RAG Tool Agent Memory Integration Gate

一句话定义：RAG tool agent memory integration gate 是基于能力边界、编排模式、上下文优先级、上下文预算、证据保留、工具观察使用、Agent 状态、Memory 读写、冲突处理、注入传播、敏感数据、trace 和分层 eval 的综合上线门禁。

面试表达：门禁通过才说明 RAG、Tool、Agent、Memory 不是概念堆砌，而是一个可解释、可审计、可恢复的组合系统。

RAG Agent Storage Audit

一句话定义：RAG agent storage audit 是检查 RAG / Agent 平台是否把知识库、文档、chunk、citation、agent definition、tool definition、memory、run state、retrieval trace、tool call trace、execution trace、audit log 和 cost record 做成可版本化、可回放、可审计、可删除的数据资产。

常见审计维度：Knowledge Base Contract 看租户、owner、来源、权限、embedding、chunking、索引、同步和保留策略；Document Chunk Version Contract 看 document version、content hash、chunk offset、checksum、embedding ID 和 ACL；Sync Delete Propagation 和 ACL Permission Enforcement 检查删除、撤权、权限变化、tenant 和标签过滤是否进入检索、rerank、prompt 和日志；Retrieval Trace Completeness、Prompt Assembly Trace 与 Citation Version Binding 检查 query、embedding version、index version、filter、candidate、rerank、prompt hash、token budget、citation span、document version 和 offset；Agent Definition Versioning、Tool Definition Contract、Tool Permission Gate、Tool Call Trace Completeness、Execution Trace Replay Readiness、Memory Privacy Lifecycle、Trace Privacy Retention、Cost Attribution Governance 和 RAG Agent Storage Gate 则检查 agent / tool 版本、schema、权限、tool trace、span tree、回放、memory 隐私、trace 脱敏保留、成本归因和最终门禁。

面试表达：这类题先强调 RAG / Agent 平台不是向量库加日志表，而是一套应用数据资产控制面；然后按知识库、工具、trace、memory、权限、隐私、成本和回放展开；最后用 RAG Agent Storage Gate 收束，说明线上 bad case 能解释、复现、审计、删除和回滚。

Knowledge Base Contract

一句话定义：Knowledge Base Contract 是知识库的结构化契约，至少包含 kb id、tenant、owner、source connector、permission policy、embedding version、chunking version、index version、sync state 和 retention policy。

常见问题：把知识库等同于 collection 或索引名，会导致文档版本、权限、删除、同步和 citation 无法统一治理。

Document Chunk Version Contract

一句话定义：Document Chunk Version Contract 是 document version、source URI、content hash、permission labels、parsing status、chunk offset、chunk checksum、embedding ID 和 ACL 的联合契约。

工程价值：chunk 没有 document version、offset 或 checksum 时，citation 只能展示链接，不能证明生成时引用的是哪一版原文。

Sync Delete Propagation

一句话定义：Sync Delete Propagation 衡量源系统删除、撤权和权限变化是否传播到 document store、chunk store、vector index、citation store 和 cache。

常见问题：源文档已经删除，但旧 chunk 仍留在向量索引或缓存里，会造成过期引用、越权召回和合规风险。

ACL Permission Enforcement

一句话定义：ACL Permission Enforcement 是在 retrieval、rerank、prompt assembly、tool execution 和 trace access 阶段强制执行 tenant、role、label、region 和数据等级权限。

面试表达：权限不是让模型判断“能不能答”，而是由平台在候选召回、上下文构造、工具调用和日志访问时持续阻断。

Retrieval Trace Completeness

一句话定义：Retrieval Trace Completeness 衡量一次检索是否记录 query、rewrite、embedding version、index version、filters、top-k、candidate chunks、scores、rerank、final chunks、过滤原因和 latency。

工程价值：没有 retrieval trace，RAG bad case 很难区分是正确文档未入库、过滤错、召回错、rerank 错还是 prompt 截断。

Prompt Assembly Trace

一句话定义：Prompt Assembly Trace 是记录 system prompt version、template version、用户 query、chunk 顺序、token budget、截断策略、引用格式和 prompt hash 的结构化记录。

常见问题：检索正确但 prompt assembly 截掉关键 chunk，会被误判成模型幻觉或检索失败。

Citation Version Binding

一句话定义：Citation Version Binding 是让 answer span 绑定 chunk id、document id、document version、source URI、offset 和 index version。

工程价值：文档更新后，如果 citation 不绑定版本，用户看到的引用可能不是生成时使用的证据。

Agent Definition Versioning

一句话定义：Agent Definition Versioning 是把 agent id、agent version、system prompt version、tool versions、model config、max steps、timeout、permission policy、memory policy 和 safety policy 固定成可追溯版本。

面试表达：同一个 agent 今天和明天行为不同，通常不是模型玄学，而是 prompt、tool、memory、权限或预算配置漂移。

Tool Definition Contract

一句话定义：Tool Definition Contract 是 tool name、version、description、input schema、output schema、endpoint、timeout、retry policy、permission requirement 和 side effect level 的结构化契约。

常见问题：工具 schema 或描述改了但没有版本化，会直接改变模型是否调用工具以及如何填参数。

Tool Call Trace Completeness

一句话定义：Tool Call Trace Completeness 衡量每次工具调用是否记录 run id、step id、tool name、tool version、input args、output ref、status、latency、error、permission decision、retry count 和 redaction status。

工程价值：工具调用 trace 既服务 debug，也服务权限审计、回放、成本归因和安全事故复盘。

Execution Trace Replay Readiness

一句话定义：Execution Trace Replay Readiness 衡量 execution trace 是否能固定 model version、prompt version、agent definition、tool definitions、tool outputs、knowledge base version、index version 和 sampling params。

常见问题：只保存最终回答和日志，无法复现当时的检索结果、工具返回和安全判断。

Memory Privacy Lifecycle

一句话定义：Memory Privacy Lifecycle 是 memory 写入、读取、TTL、可见性、删除、训练使用授权和敏感数据阻断的生命周期治理。

工程价值：Memory 是敏感数据资产，不是免费的上下文缓存；错误写入会长期污染后续任务。

Trace Privacy Retention

一句话定义：Trace Privacy Retention 是对 trace 中用户输入、检索片段、工具参数、工具结果和输出做字段级脱敏、访问控制、TTL、加密、删除请求和采样治理。

常见问题：完整 trace 对排障很有价值，但也可能包含高敏数据，不能默认永久保存。

Cost Attribution Governance

一句话定义：Cost Attribution Governance 是把 LLM、retrieval、rerank、tool API、embedding、trace storage 和人工确认成本拆到 tenant、application、agent、user、run、model、tool 和 component。

面试表达：Agent 成本失控常来自循环、重试、工具 API、rerank 和 trace 存储，只看模型 token 账单不够。

RAG Agent Storage Gate

一句话定义：RAG Agent Storage Gate 是判断知识库、文档、chunk、citation、agent definition、tool definition、tool permission、tool trace、execution trace、memory、隐私、成本、审计和回滚证据是否完整且无 P0 风险的综合门禁。

典型硬阻断：知识库缺 owner 或权限策略；chunk 缺 document version / offset / checksum；删除和撤权没有传播到索引；无权限 chunk 进入 prompt；retrieval trace 缺 index version；prompt assembly 缺 hash；citation 不绑定版本；agent definition 未版本化；tool schema 缺版本；高风险工具绕过确认；tool output 未记录；trace 不可回放；敏感 memory 被长期保存；trace 无脱敏或 TTL；成本不可归因；没有最终 storage gate。

Tool Cost Latency Concurrency Audit

一句话定义：tool cost latency concurrency audit 是检查工具调用系统是否能把成本、延迟、预算、超时、重试、并发、队列、限流、缓存、批处理、结果裁剪、降级、router 性能信号、trace 和 eval 统一治理的审计方法。

面试表达：工具调用平台不是调用成功就够了，还要证明每次调用为什么花这些钱、为什么耗这些时、为什么允许这些并发。

Cost Attribution Coverage

一句话定义：cost attribution coverage 衡量任务成本是否拆到模型 token、工具执行、外部 API、检索、trace、日志和人工确认等来源。

常见问题：只统计模型 token，会低估数据库扫描、浏览器、第三方 API、重试和人工审核成本。

Latency Breakdown Coverage

一句话定义：latency breakdown coverage 衡量延迟是否拆到模型决策、队列等待、权限策略、工具执行、后处理和模型继续生成。

工程价值：没有延迟分解，只看总耗时无法判断应该优化模型、工具、队列、结果裁剪还是并发控制。

Tool Call Budget Enforcement

一句话定义：tool call budget enforcement 是对最大工具调用次数、模型轮次、token、总成本、deadline、并发和重试次数做硬约束。

面试表达：预算不是看板字段，而是 runtime 真的会阻断、降级或请求用户确认。

Timeout Deadline Coverage

一句话定义：timeout deadline coverage 衡量单工具、单步骤、workflow、用户请求和后台任务是否都有明确 timeout 或 deadline。

常见问题：只有 HTTP 客户端 timeout，没有 workflow deadline，会让 Agent 多轮等待和重试拖垮交互体验。

Retry Classification Accuracy

一句话定义：retry classification accuracy 衡量系统是否能区分临时网络错误、限流、超时、权限不足、参数非法、策略拒绝和用户取消。

工程价值：权限错误、参数错误和策略拒绝不能靠重试解决，错误重试只会放大成本和风险。

Idempotent Retry Safety

一句话定义：idempotent retry safety 是有副作用工具在重试前具备幂等键、去重记录、补偿动作或人工确认。

常见问题：send email、扣款、删除文件、写数据库等动作没有幂等保护时，重试可能产生重复副作用。

Concurrency Limit Enforcement

一句话定义：concurrency limit enforcement 是按全局、租户、用户、工具、下游服务、Agent 和 workflow 维度限制同时运行的工具调用。

面试表达：并行能降延迟，但并发无上限会打爆下游、触发限流并扩大失败面。

Queue Priority Fairness

一句话定义：queue priority fairness 衡量队列是否同时考虑交互优先级、deadline、租户公平性、后台任务和高风险审核任务。

工程价值：没有队列优先级，后台批处理可能挤占交互请求，高价值任务也可能被低价值长任务拖慢。

Rate Limit Quota Handling

一句话定义：rate limit quota handling 是系统读取和执行 RPM、TPM、并发、日配额、月配额、成本上限和 retry-after 的能力。

常见问题：把限流当异常重试，而不是进入排队、退避、降级或换工具，会造成重试风暴。

Cache Safety Correctness

一句话定义：cache safety correctness 衡量缓存 key、TTL、权限隔离、数据分类、失效策略和跨用户复用边界是否正确。

面试表达：缓存能降低成本和延迟，但权限相关结果跨用户复用就是数据泄露。

Batching Partial Success Readiness

一句话定义：batching partial success readiness 衡量批处理是否支持逐项权限检查、部分成功、逐项错误和结果定位。

工程价值：批处理不是把多个请求粗暴合并，而是要保留每个 item 的权限、失败原因和可追踪性。

Result Trimming Budget Control

一句话定义：result trimming budget control 是用分页、top-k、字段选择、摘要、截断说明和 artifact 引用控制工具结果进入上下文的大小。

常见问题：把 10000 行数据库结果直接塞进上下文，会浪费 token、挤掉关键证据，还可能泄露敏感字段。

Degradation Transparency

一句话定义：degradation transparency 是工具超时、降级、使用缓存、返回部分结果或转后台任务时，向用户说明数据时间、限制和后续状态。

面试表达：降级不是静默编造，而是诚实说明“我用了什么替代结果，它有什么限制”。

Router Performance Awareness

一句话定义：router performance awareness 衡量 Tool Router 是否把延迟、成本、成功率、负载、数据新鲜度和用户预算纳入工具选择。

工程价值：Tool Router 不能只按语义相似度选工具，生产环境还要按成本、SLO 和可用性做权衡。

Performance Trace Readiness

一句话定义：performance trace readiness 衡量 trace 是否记录成本分解、延迟分解、预算、timeout、retry、queue、rate limit、cache、batch、degradation 和 router decision。

常见问题：没有性能 trace，线上慢查询只能靠猜，无法复盘是哪一步超时或哪项预算失效。

Cost Latency Eval Coverage

一句话定义：cost latency eval coverage 是评估集覆盖预算、超时、重试、并发、限流、缓存、批处理、大结果、降级和 router trade-off。

面试表达：性能治理也要回归测试，不能只在压测或事故后才发现某个路径会无限调用工具。

Tool Performance Gate

一句话定义：tool performance gate 是基于成本归因、延迟分解、预算执行、timeout、retry、幂等、并发、队列、限流、缓存、批处理、结果预算、降级、router、trace 和 eval 的综合上线门禁。

面试表达：门禁通过才说明工具调用系统在成本、延迟、并发和失败恢复上可控，而不是只在 demo 环境跑通。

Tool Use Eval Benchmark Audit

一句话定义：tool use eval benchmark audit 是对工具需求判断、工具选择、参数生成、多步调用、观察结果使用、失败恢复、安全策略、模拟器、trace replay 和回归门禁的系统审计。

工程价值：它把“模型会不会用工具”拆成可定位指标，而不是只看最终答案是否流畅。

Tool Need Accuracy

一句话定义：tool need accuracy 衡量模型是否正确判断当前任务是否需要调用工具。

常见问题：模型可能在不需要工具的常识问题上乱调工具，也可能在必须查实时状态时直接编答案。

No-Tool Overcall Control

一句话定义：no-tool overcall control 衡量无工具任务中模型是否能克制调用工具。

工程价值：控制过度调用能降低成本、延迟、权限暴露和工具结果注入风险。

Argument Schema Validity

一句话定义：argument schema validity 衡量预测参数是否符合工具 schema 的 required、type、enum、range 和结构约束。

边界说明：schema valid 只说明格式合法，不保证日期、ID、单位和业务含义正确。

Argument Semantic Accuracy

一句话定义：argument semantic accuracy 衡量预测参数是否在业务语义上等价于期望参数。

典型例子："昨天"、"last day" 和具体日期是否等价，取决于 benchmark 固定的时区、评估日期和业务口径。

Sequence Order Accuracy

一句话定义：sequence order accuracy 衡量多工具任务中调用顺序是否符合依赖关系。

常见问题：先退款再查订单、先写入再校验权限、先总结再检索证据，都会让最终答案看起来正常但过程不可审计。

Observation Grounding Accuracy

一句话定义：observation grounding accuracy 衡量最终回答是否忠实使用工具返回结果。

工程价值：工具返回“订单已取消”时，模型不能回答“正在配送”；工具观察结果必须约束最终回答。

Error Recovery Readiness

一句话定义：error recovery readiness 衡量工具失败后系统是否能按错误类型追问、降级、重试、转人工或解释限制。

面试表达：错误恢复不是吞掉异常继续编答案，而是把权限不足、参数错误、限流、超时和未知状态分开处理。

Safe Failure Handling

一句话定义：safe failure handling 衡量工具不可用、超时或部分失败时系统是否能安全失败。

工程价值：安全失败意味着诚实说明限制、保留状态和证据，并避免把不确定结果说成事实。

Safety Policy Compliance

一句话定义：safety policy compliance 衡量工具调用是否遵守权限、确认、数据外发、注入防护和高风险动作策略。

常见硬门禁：未确认退款、删除、外发、shell 执行、写生产数据或被不可信内容驱动的高风险动作应直接阻断。

Tool Simulator Determinism

一句话定义：tool simulator determinism 衡量 eval 中的工具模拟器是否返回可重复、可版本化和可回放的结果。

工程价值：模拟器不稳定会让模型变更、prompt 变更和 schema 变更的对比失去可信度。

Trace Replay Coverage

一句话定义：trace replay coverage 衡量评估集和运行时 trace 是否支持固定工具结果后重放模型、prompt 或 schema 变更。

面试表达：trace replay 能用真实线上分布验证回归，比纯手写 happy path 更接近生产风险。

Cost Latency Regression Control

一句话定义：cost latency regression control 衡量新模型、新 prompt 或新工具 schema 是否引入不可接受的成本和延迟退化。

工程价值：工具调用准确率提升但调用次数、token、外部 API 成本或 P95 延迟大幅上升，也可能不适合上线。

Benchmark Slice Coverage

一句话定义：benchmark slice coverage 衡量评估集是否覆盖单工具、多工具、no-tool、参数陷阱、错误恢复、安全对抗、权限和长上下文等切片。

常见问题：只覆盖 happy path 的总分会掩盖某个高风险切片的严重回归。

Regression Gate Readiness

一句话定义：regression gate readiness 衡量 benchmark 指标是否已经变成发布前的硬门禁，而不是一次性离线报告。

工程价值：只有进入 CI、灰度、release gate 和事故复盘，eval 才能真正防止回归。

Tool Use Eval Benchmark Gate

一句话定义：tool use eval benchmark gate 是基于工具需求、工具集合、参数、顺序、观察结果、失败恢复、安全、模拟器、trace replay、成本延迟和切片覆盖的综合上线门禁。

面试表达：门禁通过才说明系统不是只在 demo 上会调工具，而是在真实任务、异常和安全边界上可评估、可回放、可阻断。

Function MCP A2A Comparison Audit

一句话定义：function MCP A2A comparison audit 是对 Function Calling、MCP 和 A2A 三层协议分工、组合方式、安全边界、trace 和 eval 的横向审计。

核心判断：Function Calling 管模型结构化调用意图，MCP 管工具 / 资源 / Prompt Server 接入，A2A 管 Agent 之间任务协作。

Layer Boundary Clarity

一句话定义：layer boundary clarity 衡量系统是否清楚区分模型调用意图、工具资源连接层和 Agent 协作层。

常见问题：把 MCP 写成模型 tool_call 字段，或把 A2A 当成普通 API 调用，都会导致安全和 trace 边界混乱。

Function Calling Fit

一句话定义：function calling fit 衡量 Function Calling 是否只用于模型选择工具和生成结构化参数。

边界说明：它不负责企业工具发现、MCP Server 生命周期、资源暴露或跨 Agent 任务状态管理。

MCP Integration Fit

一句话定义：MCP integration fit 衡量外部工具、资源和提示模板是否通过 MCP Server 边界被 Host 标准化接入。

工程价值：MCP 的重点是复用工具 / 资源服务、统一发现和权限治理，而不是替代模型的结构化调用输出。

A2A Delegation Fit

一句话定义：A2A delegation fit 衡量 A2A 是否用于真正的 Agent 间任务委派、状态同步、追问和 Artifact 返回。

常见问题：把 read_file、search_db 这类被动工具包装成 Agent，通常是过度工程。

Object Contract Coverage

一句话定义：object contract coverage 衡量三层关键对象是否有清晰契约，例如 tool schema / arguments、MCP tools / resources / prompts、A2A agent card / task / message / artifact。

工程价值：对象契约清楚，才能做 adapter、权限、trace replay 和回归评估。

Capability Discovery Fit

一句话定义：capability discovery fit 衡量能力发现是否放在正确层：MCP 发现工具资源能力，A2A 发现 Agent 能力，Function Calling 使用 Host 投影后的候选工具。

常见问题：让模型在 prompt 里自由发现企业全量工具，通常不可控也不可审计。

Lifecycle State Alignment

一句话定义：lifecycle state alignment 衡量系统是否把不同层的生命周期状态映射正确。

典型口径：Function Calling 是 tool call -> tool result 的短链路；MCP 是连接、能力发现和调用 / 读取；A2A 是 submitted、working、input-required、auth-required、completed、failed、canceled、rejected 等任务状态。

Context Transfer Boundary

一句话定义：context transfer boundary 衡量工具结果、资源内容和跨 Agent 消息是否按最小必要原则传递上下文。

工程价值：A2A 不能默认转发完整会话、全部文件或所有工具结果；跨边界内容要有 source、scope、taint 和 artifact reference。

Permission Governance Split

一句话定义：permission governance split 衡量工具权限、资源权限、Agent allowlist、on-behalf-of、确认和审计是否分别落在 runtime / Host / Server 边界。

常见问题：把权限只写进 prompt，不能替代 runtime policy、scope token、资源过滤、确认和审计。

Host Runtime Ownership

一句话定义：host runtime ownership 衡量 Host / runtime 是否承担 provider adapter、MCP 能力投影、上下文预算、权限门禁、工具结果处理和 trace 串联。

面试表达：三层协议组合时，Host 是安全和上下文控制中心，不是透明转发器。

Trace Chain Continuity

一句话定义：trace chain continuity 衡量 trace 是否串起 user request、model tool_call、MCP server access、A2A task、artifact 和 final answer。

工程价值：链路断开时，无法复盘某个最终回答来自哪个 Agent、哪个工具、哪个资源和哪个权限决策。

Eval Gate Linkage

一句话定义：eval gate linkage 衡量 tool eval、MCP integration eval、A2A task eval 和 release gate 是否连接成回归门禁。

常见问题：只测 Function Calling 参数准确率，不测 MCP 权限、A2A 上下文转发和跨层 trace，会漏掉系统级风险。

Overengineering Control

一句话定义：overengineering control 衡量系统是否避免用 A2A 包装简单工具、用 MCP 替代本地函数、或把简单 API 调用做成复杂多 Agent 系统。

面试表达：协议选型要服务任务复杂度，不是层数越多越高级。

Severe Protocol Misuse Rate

一句话定义：severe protocol misuse rate 是严重协议误用样本占比，例如跨 Agent 无权限转发上下文、A2A 包装普通工具、MCP 当模型输出格式或 trace 断链。

工程价值：这类问题通常应作为上线硬阻断，而不是低权重扣分项。

Protocol Composition Gate

一句话定义：protocol composition gate 是基于层级边界、三层适配、对象契约、能力发现、生命周期、上下文、权限、Host 责任、trace、eval 和过度工程控制的综合门禁。

面试表达：门禁通过才说明 Function Calling、MCP 和 A2A 是分层协作，而不是概念混搭。

Provider Framework Tool Protocol Audit

一句话定义：provider framework tool protocol audit 是对 OpenAI、Anthropic、Google 等 provider 和 LangChain、LlamaIndex 等 framework 的工具协议迁移与适配审计。

核心关注：不要只比较字段名，而要比 schema 投影、tool choice、tool result、streaming、parallel call、错误归一化、trace、eval 和 vendor lock-in。

Provider Framework Type Clarity

一句话定义：provider framework type clarity 衡量系统是否区分模型 provider 和应用 framework。

常见问题：LangChain、LlamaIndex 不是模型 provider；它们封装和编排 provider、工具、RAG 和 Agent，但底层仍要处理 provider 语义。

Tool Schema Projection

一句话定义：tool schema projection 衡量内部工具 schema 投影到不同 provider 或 framework 时，required、type、enum、array、nested object、additional properties 和 description 是否保持语义。

工程价值：schema 投影丢失 required 字段或放宽 additional properties，会让参数错误被静默放行。

Tool Choice Mapping

一句话定义：tool choice mapping 衡量内部 auto、none、required、force specific 和 allowlist 策略能否正确映射到目标平台。

常见问题：把强制指定工具退化成 auto，会改变模型行为和安全边界。

Tool Result Round Trip

一句话定义：tool result round trip 衡量 tool call id、工具名、参数、执行结果、错误和引用信息能否完整回传给模型或框架运行时。

工程价值：tool result id 丢失时，多工具或并行工具调用很容易把结果接到错误的调用上。

Streaming Event Assembly

一句话定义：streaming event assembly 衡量流式工具调用增量是否能按顺序拼接成完整参数、完整事件和可回放 trace。

常见问题：流式 JSON 参数还没完整闭合就执行工具，可能造成参数截断或安全校验绕过。

Parallel Call Alignment

一句话定义：parallel call alignment 衡量并行工具调用的 independence、result id、错误处理和结果合并是否一致。

工程价值：多 provider 迁移时，并行调用从支持变为不支持，或结果顺序变化，都会影响延迟、成本和正确性。

Built-In Tool Boundary

一句话定义：built-in tool boundary 衡量平台内置搜索、代码执行、文件检索、浏览器等工具的权限、可观测性、成本和替代方案是否被明确记录。

面试表达：内置工具接入快，但企业权限、内网数据、审计和迁移经常需要自建 runtime 补齐。

Error Normalization

一句话定义：error normalization 衡量不同 provider 和 framework 的原始错误是否被映射为统一内部错误模型。

常见字段：内部错误码、错误分类、retryable、用户可见消息、原始错误保留和 trace id。

Provider Adapter Isolation

一句话定义：provider adapter isolation 衡量 provider 原始字段是否被隔离在 adapter 层，而不是泄漏到业务逻辑、权限策略和评估脚本里。

工程价值：adapter 隔离越好，迁移 provider、做 A/B、回放 trace 和修复 SDK 变更越可控。

Framework Escape Hatch

一句话定义：framework escape hatch 衡量使用 LangChain、LlamaIndex 等高层框架时，是否仍能拿到底层 provider 原始事件、工具参数、结果、错误和 trace。

常见问题：框架抽象隐藏了 provider 细节，排查工具调用回归时会缺证据。

RAG Tool Traceability

一句话定义：RAG tool traceability 衡量 retriever、query engine、tool call、citation、source metadata 和 final answer 是否能连成证据链。

工程价值：RAG-heavy 框架迁移时，引用和来源 trace 丢失会直接破坏可信回答能力。

Permission Safety Consistency

一句话定义：permission safety consistency 衡量不同 provider / framework 路径下，工具权限、资源权限、敏感数据、确认和 prompt injection 防御是否保持一致。

面试表达：权限不能只写在 prompt 里，迁移后也必须由 runtime policy 和审计证明。

Migration Eval Coverage

一句话定义：migration eval coverage 衡量平台迁移前后是否覆盖 schema、tool choice、参数、tool result、streaming、parallel、安全、RAG 引用和成本延迟等回归样本。

工程价值：迁移只改 endpoint 而不跑 eval，很容易出现工具选择和参数语义回归。

Provider Trace Replay Readiness

一句话定义：provider trace replay readiness 衡量 provider / framework 原始事件是否足以重放模型工具调用、工具执行、结果回填和最终回答。

常见要求：保留 request id、provider event id、tool call id、adapter version、schema version、tool result 和 final answer。

Vendor Lock-In Control

一句话定义：vendor lock-in control 衡量系统是否通过内部 Tool Runtime、统一对象、adapter 测试和迁移 eval 降低对单一 provider 或 framework 的绑定。

工程价值：不是完全避免平台能力，而是把平台能力包在可替换、可评估、可审计的边界里。

Provider Runtime Gate

一句话定义：provider runtime gate 是基于 provider/framework 边界、schema 投影、tool choice、tool result、streaming、parallel、内置工具、错误归一化、adapter、RAG trace、权限、安全、迁移 eval 和 vendor lock-in 的综合门禁。

面试表达：门禁通过才说明多平台工具调用不是字段翻译，而是语义、权限、trace 和 eval 都可迁移。

Enterprise MCP Platform Audit

一句话定义：enterprise MCP platform audit 是对企业 MCP Gateway、Registry、Router、Policy Engine、Context Manager、Server Runtime、Trace / Audit / Replay、Eval 和发布治理的系统审计。

核心关注：平台不是连接很多 MCP Server，而是让 tools、resources、prompts、artifacts 和 trace 在授权、租户、沙箱、上下文和回归测试下可运营。

MCP Gateway Readiness

一句话定义：MCP gateway readiness 衡量统一入口是否覆盖 Host / Agent 认证、用户和租户识别、路由、限流、参数校验、policy 调用、trace 注入、错误归一化和响应投影。

常见问题：让 Agent 直连所有 MCP Server，会让权限、审计、限流和安全策略分散在多个实现里。

MCP Registry Metadata Completeness

一句话定义：MCP registry metadata completeness 衡量 Server、tools、resources、prompts、owner、version、endpoint、scope、risk、status、health、documentation 和 lifecycle 元数据是否完整。

工程价值：metadata 不完整时，路由、权限审核、版本兼容、发布治理和事故归因都会缺证据。

MCP Capability Namespace Isolation

一句话定义：MCP capability namespace isolation 衡量能力名称是否按租户、server、tool/resource/prompt 类型和版本隔离。

常见问题：多个团队都叫 search 或 query，如果没有 namespace，很容易路由错工具或跨租户暴露私有能力。

MCP Tool Resource Prompt Contract

一句话定义：MCP tool resource prompt contract 衡量 tools、resources 和 prompts 是否分别有输入输出、权限、数据分类、引用、错误和版本兼容契约。

面试表达：MCP 不是只有 tool；企业平台必须同时治理 resource 暴露和 prompt 模板分发。

MCP Host Client Server Boundary Clarity

一句话定义：MCP host client server boundary clarity 衡量 Host、MCP Client 和 MCP Server 的责任是否分清。

核心边界：Host 负责用户授权、上下文、policy 和最终控制；Client 负责连接；Server 暴露能力但不能自行升级权限或决定最终上下文。

MCP OBO Scope Binding

一句话定义: MCP OBO scope binding 衡量 on-behalf-of 用户身份、Agent 身份、Host 身份、token audience、scope 和任务目的是否绑定到每次工具或资源访问。

工程价值: 企业工具访问不能突破用户本身权限, 也不能把只读 scope 暗中升级成写入或外发权限。

MCP Tenant Isolation Enforcement

一句话定义: MCP tenant isolation enforcement 衡量 Server 可见性、tool 权限、resource、artifact、trace、audit、quota 和配置是否按租户隔离。

上线原则: 跨租户泄露应是硬阻断项, 不是普通低权重扣分。

MCP Policy Engine Coverage

一句话定义: MCP policy engine coverage 衡量 Policy Engine 是否同时检查用户、Agent、Host、租户、工具风险、数据分类、任务目的、确认状态和外发目标。

常见问题: 把权限写在 prompt 或 tool description 里, 不等于运行时执行了权限策略。

MCP Sandbox Roots Containment

一句话定义: MCP sandbox roots containment 衡量文件 roots、网络 allowlist、secret 隔离、环境变量过滤、shell 白名单、CPU / 内存 / 时间和输出大小限制是否落地。

适用场景: IDE、终端、浏览器和代码执行类 MCP Server 必须优先证明沙箱边界。

MCP Context Output Projection

一句话定义: MCP context output projection 衡量 tool / resource 输出进入模型上下文前是否经过字段选择、脱敏、摘要、引用、artifact 化、预算控制和错误状态保真。

工程价值: 工具执行成功不等于原始结果可以直接塞给模型。

MCP Prompt Injection Taint Propagation

一句话定义: MCP prompt injection taint propagation 衡量网页、文档、邮件、日志、数据库字段和 tool result 等不可信内容在摘要、artifact、跨 Agent 消息和最终回答中是否保留 taint。

面试表达: 不可信内容可以是证据, 不能升级为系统指令或直接触发高风险工具。

MCP Trace Audit Replay Continuity

一句话定义: MCP trace audit replay continuity 衡量 user request、Host policy、MCP client request、Server capability、tool call、resource read、prompt render、context projection、final answer 和 eval 是否能串成可回放链路。

工程价值: trace 断链时, 无法判断最终结论来自哪个工具、哪个资源、哪个权限决策和哪个版本。

MCP Cost Latency Quota Control

一句话定义: MCP cost latency quota control 衡量平台是否按用户、租户、工具、Server 和任务维度执行成本预算、deadline、重试、并发、队列、缓存和降级。

常见问题: Agent 循环调用工具会迅速放大成本和下游压力。

MCP Developer Portal Review Readiness

一句话定义：MCP developer portal review readiness 衡量开发者门户是否支持模板、manifest lint、本地调试、schema 文档、eval stub、trace replay、审核反馈、owner 和发布状态。

工程价值：没有门户和审核流程，MCP Server 数量一多就会变成不可维护的影子工具生态。

MCP Release Lifecycle Governance

一句话定义：MCP release lifecycle governance 衡量 MCP Server 和能力是否具备版本兼容、canary、rollback、deprecation、emergency disable、owner 交接和 release gate。

面试表达：工具 schema、权限策略和输出契约变化都可能造成 Agent 行为回归，需要按版本治理。

MCP Provider Adapter Compatibility

一句话定义：MCP provider adapter compatibility 衡量企业 MCP 平台投影到不同模型 provider 或 framework 时，tool schema、tool choice、tool result、streaming、parallel、权限、trace 和 eval 语义是否保持稳定。

工程价值：MCP 是工具资源连接层，但模型侧仍要经过 provider adapter 才能变成具体 tool calling 行为。

MCP Eval Regression Coverage

一句话定义：MCP eval regression coverage 衡量回归集是否覆盖 schema、授权、租户、prompt injection、上下文投影、trace replay、成本延迟、发布兼容和高可用切片。

常见问题：只测最终答案，不测工具链路和权限链路，会漏掉企业平台的核心风险。

MCP High Availability Readiness

一句话定义：MCP high availability readiness 衡量 Gateway、Registry、Policy Engine、Server Runtime、trace / audit 存储和 eval 服务是否有多副本、健康检查、故障隔离和降级方案。

工程价值：中心化平台一旦成为所有 Agent 的工具入口，高可用和降级策略就是基础能力。

Enterprise MCP Platform Gate

一句话定义：enterprise MCP platform gate 是基于 Gateway、Registry、namespace、tool/resource/prompt 契约、Host 边界、OBO 授权、租户隔离、policy、sandbox、上下文投影、prompt injection taint、trace/replay、成本配额、开发者审核、发布治理、adapter、eval 和高可用的综合上线门禁。

面试表达：门禁通过才说明企业 MCP 平台不是普通 API Gateway，而是可授权、可隔离、可审计、可回放、可评估、可运营的工具和上下文接入层。

Enterprise LLMOps Platform Audit

一句话定义：Enterprise LLMOps Platform Audit 是把大模型应用的模型网关、prompt、知识库、工具、Agent、评估、发布、trace、反馈、安全、成本和审计放进同一套生命周期控制面的系统设计审计方法。

面试表达：企业级 LLMOps 平台不是模型部署平台，也不是 prompt 管理页面，而是让应用从 dev、staging 到 production 的每次变更都有版本、评估、发布、观测、反馈、成本和审计证据。

LLMOps Application Resource Model

一句话定义：LLMOps application resource model 用 Application、Environment、PromptTemplate、KnowledgeBase、ToolDefinition、AgentDefinition、EvalSuite、ReleaseManifest、Trace、Feedback、Policy 和 CostRecord 描述大模型应用控制面。

工程价值：没有统一资源模型时，prompt、RAG、工具、评估和发布会散落在不同团队脚本里，线上事故无法复现和回滚。

LLMOps Release Manifest

一句话定义：LLMOps release manifest 是一次应用发布绑定的模型路由、prompt 版本、知识库版本、工具版本、Agent 配置、安全策略、生成参数和 rollback target。

常见问题：只灰度 prompt 或模型名，但没有绑定知识库、工具 schema 和安全策略版本，会导致 bad case 无法定位是哪一层变化造成的。

Prompt Registry Versioning

一句话定义：Prompt registry versioning 衡量 prompt 模板是否具备变量 schema、测试样例、评估结果、发布状态、灰度、回滚、权限和审计记录。

面试表达：Prompt 是应用行为资产，不应硬编码在业务代码里；改 prompt 应该像改模型和工具一样可评估、可灰度、可回滚。

RAG KB Permission Governance

一句话定义：RAG KB permission governance 衡量知识库、文档、chunk、metadata filter、citation 和 trace 是否绑定租户、用户、项目、数据等级和删除传播策略。

工程价值：企业 RAG 不能只追求 recall；无权限 chunk 被放进 prompt 是高优先级安全事故。

Tool Agent Runtime Control

一句话定义：Tool agent runtime control 衡量 Agent 定义、工具 schema、权限、side effect level、step limit、timeout、human confirmation、execution trace 和错误处理是否受平台控制。

常见问题：Agent 应用如果没有 step limit、预算、工具权限和高风险确认，成本、延迟和副作用都会失控。

LLMOps Eval Feedback Loop

一句话定义：LLMOps eval feedback loop 衡量离线评估、LLM-as-Judge、人评、在线 A/B、用户反馈、bad case triage、回归集更新和发布门禁是否能闭环。

工程价值：只收集点赞点踩不叫反馈闭环；反馈必须进入可复现样本、修复动作和下一轮回归。

LLMOps Trace Observability Readiness

一句话定义：LLMOps trace observability readiness 衡量 model call、prompt assembly、retrieval、rerank、tool call、agent step、safety event、feedback 和 cost record 是否能串成一条可回放 trace。

面试表达：RAG / Agent 应用没有 trace，就无法解释答案来源、工具行为、权限决策、成本归因和质量回归。

LLMOps Cost Budget Governance

一句话定义：LLMOps cost budget governance 衡量平台是否能按应用、租户、用户、模型、知识库、工具、Agent run 和环境做 token、检索、rerank、工具、人审和重试成本归因。

常见问题：Agent 多步调用会把模型成本、工具成本和人工审核成本放大，必须配合预算、配额、step limit 和 cost alert。

LLMOps Platform Gate

一句话定义：LLMOps platform gate 是基于资源模型、模型网关、prompt、RAG、工具、Agent、评估、发布、trace、反馈、安全、成本、多环境和开发者接口的企业级 LLMOps 上线门禁。

面试表达：门禁通过才说明平台不是功能入口集合，而是可开发、可评估、可发布、可观测、可治理、可审计、可持续优化的应用生命周期平台。

长上下文、RAG 与 Agent 的关系

长上下文解决的是一次输入能看多少信息，RAG 解决的是从外部知识中找哪些信息放进上下文，Agent 解决的是模型如何围绕目标多步行动。

三者经常组合使用：RAG 为 agent 提供外部知识，agent 调用检索、数据库和业务工具，长上下文保存任务轨迹和关键证据。真实系统不是三选一，而是根据任务需求组合。

典型对比：如果材料固定且必须完整审阅，长上下文更直接；如果知识库很大且需要动态查询，RAG 更经济；如果任务需要多步操作和外部执行，Agent 更合适。

工程设计原则：先明确任务类型，再决定是扩大窗口、做检索增强，还是引入 agent。不要因为 agent 概念热门，就把简单问答做成复杂多步系统。

面试表达：长上下文、RAG 和 Agent 分别对应信息容量、知识获取和行动能力。优秀的大模型应用设计通常是在三者之间做成本、准确率、延迟和可控性的权衡。

本章小结

本章围绕长上下文、RAG 和 Agent 建立了系统词汇表。

核心结论如下：

1. 长上下文提升模型可见信息量，但不保证模型能稳定使用全部信息。
2. context window 是推理时的信息带宽，需要管理成本、噪声和关键证据位置。
3. position extrapolation 解决超长位置表示问题，但不等于完整长文本理解能力。
4. RAG 的关键是检索、重排、上下文构造、grounding 和评估，不只是向量数据库。
5. dense retrieval、sparse retrieval 和 hybrid retrieval 各有优势，真实系统常组合使用。
6. reranker 可以显著改善进入 prompt 的证据质量。
7. agent 是模型、工具、记忆、规划和安全控制组成的系统，而不是单次回答。
8. tool use 和 function calling 提升模型外部执行能力，但必须配合参数校验和权限控制。
9. memory 要筛选、压缩、结构化和按需检索，不能无脑保存全部历史。
10. 长上下文、RAG 和 Agent 分别解决信息容量、知识获取和多步行动问题。

下一章，我们进入 Reasoning 与评估。

L. Reasoning 与评估

条目：Reasoning、Reasoning Model、Reasoning Candidate Set、Chain-of-Thought、Few-shot CoT、Zero-shot CoT、Scratchpad、Hidden CoT、Visible Explanation、CoT Faithfulness、CoT Regression、CoT Routing、CoT Audit、Self-Consistency、Sampling Temperature、Top-p Sampling、Answer Normalization、Majority Vote、Weighted Vote、Candidate Diversity、Majority Failure、Self-Consistency Cost、Self-Consistency Accuracy、Verifier、Programmatic Verifier、Hybrid Verifier、Verifier Reranking、Pairwise Accuracy、Hard Negative、Verifier Calibration、Reward Model Bias、Process Supervision、Step Label、First-Error Detection、Process Reward Model、Outcome Reward Model、Process Step Accuracy、Auto Label Coverage、Human Label Cost、Process Search Pruning、Process Supervision Audit、Search Reasoning、Search State、Search Action、Beam Search、Best-First Search、UCT、Prune False Negative、Search Budget、Search Audit、Test-Time Compute、Test-Time Compute Scaling、Compute Budget Vector、Adaptive Compute、Budget Router、Cost per Correct、Marginal Accuracy per Cost、P95 Latency、Wasted High Compute、TTC Audit、Math Reasoning Training、Math Training Sample、Answer Supervision、

Synthetic Math Data、Math Curriculum、Math Contamination Audit、Template Diversity、Math Training Gate、Code Reasoning、Execution Feedback、Unit Test Verifier、Public Test、Hidden Test、Public-Hidden Gap、Self-Debug、Repair Success Rate、Sandbox Violation Rate、Code Execution Audit、Code Reasoning Gate、Reasoning Evaluation、Evaluation Sample、Variant Evaluation、Robustness Drop、Paired Lift、Bootstrap Confidence Interval、Reasoning Eval Gate、Test-Time Compute Cost、Pass@k、Reasoning Audit、Reasoning Safety、Pseudo Reasoning、Overconfident Error、Hidden CoT Exposure、Tool Misuse、Human Review Coverage、Severity-Weighted Risk、Reasoning Safety Gate、Reasoning Interview Readiness、Reasoning Interview Rubric、Reasoning Formula Coverage、Reasoning Demo Coverage、Weak Reasoning Question、Reasoning Revision Plan、Tree-of-Thought、MCTS、Benchmark、Human Evaluation、Elo Rating、Hallucination、Factuality、Robustness、Evaluation Metric Incident、Aggregate Score Trap、Slice Regression、Clean Eval Lift、Judge-Human Agreement、Judge Length Bias、Evaluation Gate。

Reasoning

一句话定义：reasoning 是模型通过多步中间过程解决问题的能力，常见于数学、代码、逻辑推理、规划、复杂问答和工具使用任务。

为什么重要：很多任务不能靠模式匹配直接回答，需要拆解条件、维护中间状态、比较多个选项、验证假设并逐步推出结论。

典型表现：能把复杂问题拆成子问题，能根据约束排除错误答案，能在计算和逻辑链条中保持一致，能发现前一步推理的错误并修正。

核心难点：模型可能生成看似合理但实际错误的推理步骤；长推理中错误会累积；推理文本正确不代表最终答案正确，最终答案正确也不代表推理过程真实可靠。

面试表达：大模型 reasoning 不是传统符号推理的简单替代，而是概率生成、训练数据模式、搜索策略、验证器和测试时计算共同作用的结果。

Reasoning Model

一句话定义：Reasoning Model 是更强调多步推理、数学、代码、规划、验证和复杂任务分解的大模型或系统。

和普通 chat model 的区别：chat model 更强调指令遵循、对话自然和通用问答；reasoning model 更强调可验证任务上的正确性、推理预算、候选生成、验证器和错误修正。

重要边界：reasoning model 不等于“输出很长解释”的模型。长 CoT 可能只是看似合理的文本，真正可靠性需要最终答案、过程步骤、工具执行、verifier 和污染风险一起评估。

面试表达：reasoning model 是训练数据、推理策略、test-time compute、verifier 和评估体系共同作用的结果，不应只看回答长度。

Reasoning Candidate Set

一句话定义：Reasoning Candidate Set 是同一道题在不同采样、搜索或工具反馈下产生的一组候选推理链和最终答案。

候选集合常用于 self-consistency、best-of-n、verifier reranking、代码执行筛选和 tree search。

面试表达：单次 greedy 答案可能错，但候选集合中可能已经包含正确解。reasoning 系统要同时看候选质量、候选多样性、选择器可靠性和成本。

Chain-of-Thought

一句话定义：Chain-of-Thought，简称 CoT，是让模型生成中间推理步骤，再给出最终答案的方法。

为什么提出：直接问答案时，模型容易跳步。让模型先写推理过程，相当于给复杂任务分配更多计算和中间状态，有助于数学、逻辑和多跳问答。

解决什么问题：CoT 主要解决复杂问题中“一步到位”难以得到正确答案的问题。它让模型显式展开条件、公式、子问题和推导过程。

优点：提升复杂推理准确率，过程更可检查，也便于发现模型错在哪里。

缺点：推理过程可能是事后编造；长 CoT 会增加 token 成本；错误推理可能让答案更自信；在简单任务上不一定有收益。

常见用法：few-shot CoT、zero-shot CoT、让模型先分析再回答、要求输出简洁推理摘要。

面试表达：CoT 的本质是通过中间步骤增加测试时计算和状态表达，但 CoT 文本本身不一定等于模型真实内部推理。

Few-shot CoT

一句话定义：Few-shot CoT 是在 prompt 中提供少量带推理步骤的示例，让模型模仿逐步解题格式。

优点：不需要重新训练，能通过示例控制问题分解、答案格式和推理粒度。

局限：占用上下文；示例选择会影响稳定性；示例中的错误步骤、跳步或格式混乱会被模型继承。

面试表达：few-shot CoT 是最早被系统验证有效的 CoT 用法之一，核心是用示例把模型从“直接答题模式”切到“逐步解题模式”。

Zero-shot CoT

一句话定义：zero-shot CoT 是不提供示例，只通过提示语诱导模型进行逐步思考的方法。

典型提示：例如“让我们一步一步思考”或“请先分析再给出答案”。

优点：使用简单，不需要构造 few-shot 示例。

局限：对模型能力依赖强；提示语可能引入冗余推理；对格式、任务类型和模型对齐策略敏感。

面试表达：zero-shot CoT 是低成本推理增强方法，但稳定性通常弱于针对任务设计的 few-shot CoT 或 verifier 方案。

Scratchpad

一句话定义：Scratchpad 是模型或系统用于中间计算、草稿、表格、代码计划或工具调用规划的工作区。

和 CoT 的区别：CoT 更常指自然语言推理链；scratchpad 更强调中间工作区，不一定适合完整展示给用户。

面试表达：scratchpad 的价值是给复杂任务留下中间状态，但产品输出通常应展示简洁、可验证的解释，而不是完整草稿。

Hidden CoT

一句话定义：Hidden CoT 是模型或系统内部使用但不直接展示给用户的推理过程。

为什么需要区分：完整 CoT 可能冗长、包含错误探索、暴露系统策略或在安全任务中给出不必要细节。

面试表达：隐藏推理不是为了逃避评估，而是为了把内部计算和用户可见解释分开；可靠性仍要通过最终答案、步骤审计、工具验证和人工评估来证明。

Visible Explanation

一句话定义：Visible Explanation 是面向用户展示的简洁理由、关键依据或可验证摘要。

和 Hidden CoT 的区别：visible explanation 服务用户理解和复核，不承诺完整复现模型内部推理轨迹。

面试表达：好的产品解释应当短、可核查、和结论相关；不能把冗长 CoT 当成可信度本身。

CoT Faithfulness

一句话定义：CoT Faithfulness 衡量推理文本是否真实影响了模型结论，而不只是事后合理化。

常见检查：修改中间步骤是否改变最终答案、遮蔽关键步骤是否降低正确率、步骤错误是否能被 verifier 捕捉、同一答案是否伴随互相矛盾的理由。

面试表达：CoT 可读不等于 faithful。评估 CoT 要看最终答案、过程步骤、干预实验和工具验证，而不是只看语言是否流畅。

CoT Regression

一句话定义：CoT Regression 是直答本来正确，但加入 CoT 后答案变错或输出质量下降的样本。

典型来源：简单事实题过度推理、错误类比、题目外条件被纳入推理、推理链中早期错误被后续步骤放大。

简化口径：

$$R_{\{\mathrm{reg}\}} = \frac{\sum_i \mathbb{1}[\hat{y}_i^{\{\mathrm{direct}\}} = y_i^{\star} \text{ and } \hat{y}_i^{\{\mathrm{cot}\}} \neq y_i^{\star}]}{N}$$

面试表达：如果只汇报 CoT 平均准确率，可能掩盖简单题回归。上线前要单独看回归样本和任务切片。

CoT Routing

一句话定义：CoT Routing 是按任务难度、风险、延迟预算和可验证性决定是否启用 CoT 的策略。

常见规则：简单事实题短答，复杂数学 / 代码题启用 CoT 或工具，安全敏感题展示简洁拒答或安全替代解释，高价值任务再增加 verifier。

面试表达：工程上不是“所有请求都长思考”，而是按任务价值和风险动态分配推理预算。

CoT Audit

一句话定义：CoT Audit 是对 CoT 策略的答案准确率、步骤准确率、回归样本、unsupported step、token 成本和可见解释安全边界进行统一检查。

面试表达：CoT audit 能回答“CoT 是否真的值得开”，而不是只展示几条看起来漂亮的推理链。

Self-Consistency

一句话定义：self-consistency 是对同一问题采样多条推理路径，再通过投票或聚合选择最终答案的方法。

为什么有效：复杂推理任务中，单条推理链可能偶然走错。多次采样能探索不同解题路径，如果正确答案在多条路径中反复出现，可信度更高。

基本流程：提高采样温度生成多个 CoT；提取每条路径的最终答案；按多数投票、置信度或验证器打分选择答案。

优点：显著提升数学和逻辑推理稳定性。

缺点：成本成倍增加；如果模型系统性误解题意，多数投票也会一致错误；答案抽取和等价判断并不总是简单。

面试表达：self-consistency 是用测试时采样换准确率，本质是搜索多个候选推理路径再做聚合。

Sampling Temperature

一句话定义：Sampling Temperature 是控制生成分布平滑程度的采样参数。

在 reasoning 中的作用：温度太低时候选几乎重复，self-consistency 没有多样性；温度太高时推理路径容易跑偏，错误答案变多。

面试表达：self-consistency 不是把温度越调越高越好，而是要在候选多样性和候选质量之间找平衡。

Top-p Sampling

一句话定义：Top-p Sampling / Nucleus Sampling 是每一步只从累计概率达到阈值 p 的最小高概率 token 集合中采样。

在 reasoning 中的作用：top-p 能截断长尾低质量 token，同时保留比 greedy 更丰富的候选路径。

面试表达：temperature 控制分布平滑，top-p 控制候选集合尾部，两者共同决定 self-consistency 的多样性和跑偏风险。

Answer Normalization

一句话定义：Answer Normalization 是把不同表述但等价的最终答案合并到同一标准形式。

例子：42、42.0、答案是 42、42 minutes 在某些数学题中应归一为同一个答案。

面试表达：self-consistency 的投票质量高度依赖答案标准化。标准化差会把同一个正确答案拆散，或者把不同答案错误合并。

Majority Vote

一句话定义：Majority Vote 是对多个标准化候选答案计数，选择票数最多的答案。

优点：简单、无需额外模型，适合答案空间明确的数学、选择题和逻辑题。

局限：如果模型系统性误解题意，多数候选会一起错；如果正确答案是少数，没有 verifier 时会被多数错误淹没。

面试表达：majority vote 提升的是聚合稳定性，不保证真值发现能力。

Weighted Vote

一句话定义：Weighted Vote 是用 verifier、规则、log probability、单元测试或工具结果给候选答案加权，再按答案聚合分数。

适用场景：当正确答案是少数但 verifier 能识别高质量候选时，加权投票可能优于多数投票。

面试表达：weighted vote 的核心风险是评分器偏差。如果 verifier 错，聚合也会被带偏。

Candidate Diversity

一句话定义：Candidate Diversity 是候选推理路径或标准化答案之间的差异程度。

为什么重要：候选全都一样时，多采样只是浪费成本；候选过于发散时，错误路径和格式混乱会增加。

面试表达：self-consistency 需要“有约束的多样性”，不是随机性越大越好。

Majority Failure

一句话定义：Majority Failure 是多数投票选择了错误答案，但候选集合中可能已经存在正确答案的失败模式。

常见原因：系统性误解题意、干扰条件被错误使用、正确候选数量少、答案标准化错误或候选分布偏斜。

面试表达：majority failure 说明 pass@k、majority accuracy 和 verifier accuracy 必须分开汇报。

Self-Consistency Cost

一句话定义：Self-Consistency Cost 是多路径采样带来的 token、延迟、调用次数和工具验证成本。

面试表达：self-consistency 的准确率提升必须和成本曲线一起看，生产系统通常只对高价值或高难度请求启用。

Self-Consistency Accuracy

一句话定义：Self-Consistency Accuracy 是对每道题采样多条推理链后，经答案投票得到的最终准确率。

简化口径：

$$A_{\{\mathrm{sc}\}} = \frac{1}{\sum_{i=1}^N \mathbb{1}[\hat{y}_i^{\{\mathrm{sc}\}} = y_i^*]}$$

其中 $\hat{y}_i^{\mathrm{sc}}$ 是多数投票答案。

面试表达：self-consistency accuracy 通常高于单次 greedy accuracy，但它仍可能被系统性误解、答案抽取错误和投票等价类问题限制。

Verifier

一句话定义：verifier 是用于检查模型答案、推理步骤或工具结果是否正确的评估器或模型。

为什么提出：生成模型擅长提出候选答案，但不总擅长判断答案是否可靠。verifier 把“生成”和“验证”拆开，有助于从多个候选中选出更优答案。

常见形式：规则验证器、单元测试、数学答案检查器、代码执行器、检索证据核查器、奖励模型和另一个 LLM judge。

用途：降低幻觉，筛选候选答案，辅助 test-time compute，评估推理步骤，做 rejection sampling。

优点：可以显著提高复杂任务可靠性，特别是答案可自动验证的任务。

缺点：verifier 本身也可能错；不可自动验证的开放问答仍然需要人工或强 judge；验证成本可能很高。

面试表达：verifier 的价值在于把生成问题转化为候选搜索加质量判断。对于代码、数学和工具任务，验证往往比单纯提示工程更可靠。

Programmatic Verifier

一句话定义：Programmatic Verifier 是用程序执行、规则、单元测试、计算器、schema 或形式化检查器验证候选输出的验证器。

优点：客观、可复现，不容易被语言风格、长度和自信语气欺骗。

局限：只适合可执行或可规则化任务；测试覆盖不足时仍会漏掉错误；执行环境需要沙箱、超时和权限隔离。

面试表达：代码、数学表达式、SQL、JSON 和工具调用这类任务，程序验证通常比纯语言 reward model 更硬，但它不能覆盖所有开放问答。

Hybrid Verifier

一句话定义：Hybrid Verifier 是把程序验证、process score、learned reward model 和规则检查组合起来选择候选的验证策略。

为什么需要：单一 RM 可能被 hard negative 骗过，程序验证又不总可用。混合验证器可以在可验证任务上优先使用硬信号，在开放任务上退回 learned score 或人工复核。

面试表达：实际 reasoning 系统里，verifier 往往不是一个模型，而是一组按任务类型路由的验证信号。

Verifier Reranking

一句话定义：Verifier Reranking 是先生成多个候选，再用 verifier 分数选择最高分候选的 reasoning 策略。

简化口径：

$$\hat{y}_i^{\mathrm{ver}} = y_{ij}, \\ \text{quad} \\ j = \mathrm{index\ of\ max}_{k}(s_{ik})$$

其中 s_{ik} 是第 i 个问题第 k 个候选的 verifier 分数。

面试表达：verifier reranking 的关键不是生成更多文本，而是有一个足够可靠的选择器。代码测试、数学检查器和工具执行通常比纯文本自评更硬。

Pairwise Accuracy

一句话定义：Pairwise Accuracy 衡量 verifier 是否给正确候选比错误候选更高的分数。

简化口径：

$$A_{\mathrm{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(a,b) \in \mathcal{P}} \mathbb{1}[s_a > s_b]$$

其中 a 是正确或更优候选， b 是错误或更差候选。

面试表达：pairwise accuracy 能检查 reward model 的相对排序能力，但仍要结合 top-1 selection accuracy 和真实下游提升。

Hard Negative

一句话定义：Hard Negative 是看起来很像正确答案、格式很好、推理很长或很自信，但实际错误的负样本。

为什么重要：如果训练集只有简单负样本，verifier 会在真实候选分布上高估错误答案，导致 reward hacking。

面试表达：训练和评估 verifier 时必须加入 hard negatives，否则模型容易学到表面特征。

Verifier Calibration

一句话定义：Verifier Calibration 衡量 verifier 分数是否能当作可靠概率或阈值使用。

常见指标：ECE、Brier score、分桶准确率、阈值下 precision / recall 和高风险切片校准。

面试表达：verifier 分数高不代表概率校准。上线门禁和人工复核阈值必须单独校准。

Reward Model Bias

一句话定义：Reward Model Bias 是 reward model 偏好长度、格式、自信语气、模板化表达或训练集常见风格，而不是偏好真正正确性。

风险：generator 可以优化这些偏差来骗过 reward model，形成 reward hacking 或 verifier hacking。

面试表达：评估 reward model 时要看 hard negative、长度切片、格式切片、校准和下游任务提升，不能只看验证集平均准确率。

Outcome Reward Model

一句话定义：Outcome Reward Model, 简称 ORM, 是只对最终答案或最终输出进行评分的奖励模型。

为什么常用：最终结果比中间过程更容易标注。例如一道数学题答案是否正确、一段代码是否通过测试、一个回答是否满足用户偏好。

优点：标注成本相对低, 和用户最终体验更直接相关。

缺点：无法定位中间错误；可能奖励错误但碰巧得到正确答案的推理；对长推理任务的引导较弱。

面试表达：ORM 看终点, PRM 看过程。ORM 更容易落地, PRM 更适合引导复杂推理搜索。

Process Step Accuracy

一句话定义：Process Step Accuracy 是推理过程中间步骤被判定为正确的比例。

$$A_{\{\mathrm{step}\}} = \frac{\sum_i \sum_j z_{ij}}{\sum_i M_i}$$

其中 $z_{ij}=1$ 表示第 i 个样本的第 j 个推理步骤正确。

面试表达：步骤准确率能定位推理链中哪里出错, 但高步骤准确率不必然保证最终答案正确, 低步骤准确率也可能偶然得到正确答案。

Auto Label Coverage

一句话定义：Auto Label Coverage 是过程监督数据中可以由规则、程序、测试、计算器或工具自动标注的步骤比例。

简化口径：

$$C_{\{\mathrm{auto}\}} = \frac{\sum_i \sum_j a_{ij}}{\sum_i M_i}$$

其中 $a_{ij}=1$ 表示该步骤可自动标注。

面试表达：自动覆盖率越高, process supervision 越容易规模化；但自动规则覆盖不到的开放推理仍需要人工复核和质量抽检。

Human Label Cost

一句话定义：Human Label Cost 是过程监督中需要人工切分、判断、复核和仲裁步骤标签的成本。

简化口径：

$$C_{\{\mathrm{label}\}} = \sum_i \sum_j (1 - a_{ij}) c_{ij}$$

面试表达：PRM 的效果不能脱离标注成本讨论。若每个步骤都要专家复核, 真实上线要考虑只标 hard cases、半自动初标和抽样审计。

Process Search Pruning

一句话定义：Process Search Pruning 是在 tree search、beam search 或 step-by-step search 中用 PRM 分数提前剪掉低质量中间路径。

简化口径：

$$B_{t+1} = \text{TopK} \left(\{b+a: b \in B_t, a \in \text{cal}\{A\}(b)\}, S_{\text{avg}} \right)$$

面试表达：PRM 搜索剪枝能节省 test-time compute，但如果 PRM 低估正确路径或高估 polished hard negative，就会把好解提前剪掉。

Process Supervision Audit

一句话定义：Process Supervision Audit 是把最终准确率、步骤准确率、第一处错误检测、自动标注覆盖、人工成本、搜索效果和盲区样本放在一起检查。

典型盲区：最终答案正确但过程错误；过程正确但最终格式错误；冗余步骤很多；PRM 在 hard negative 上给高分。

面试表达：过程监督上线前不能只看 step accuracy，要把 outcome、process、search、cost 和 failure slices 放进同一张审计表。

Test-Time Compute

一句话定义：test-time compute 是推理阶段额外投入计算来提升答案质量的方法。

为什么提出：模型参数固定后，仍可以通过多次采样、搜索、验证、反思和工具调用提升复杂任务表现。

常见形式：self-consistency、多候选生成、best-of-n、verifier reranking、Tree-of-Thought、MCTS、代码执行验证和迭代修正。

优点：不需要重新训练模型，能按任务难度动态增加计算。

缺点：增加延迟和费用；效果依赖候选多样性和验证器质量；生产系统要控制预算和停止条件。

面试表达：test-time compute 是用推理时搜索和验证换效果，适合高价值、复杂、可验证任务，不适合所有低延迟场景。

Test-Time Compute Cost

一句话定义：Test-Time Compute Cost 是推理阶段为采样、搜索、验证和工具执行额外消耗的 token、延迟或计算资源。

简化 token 成本：

$$C_{\text{ttc}} = \sum_{i=1}^N \sum_{k=1}^{K_i} T_{ik}$$

其中 K_i 是第 i 个样本的候选数， T_{ik} 是第 k 个候选的 token 成本。

面试表达：reasoning 系统常用更多 test-time compute 换正确率，但上线时必须把准确率增益和 P95 延迟、token 成本、工具成本、安全审核成本放在一起看。

Test-Time Compute Scaling

一句话定义：Test-Time Compute Scaling 是在模型参数固定时，通过增加推理阶段采样、候选、verifier、搜索、工具和反思预算提升输出质量的方法。

核心问题：哪些请求值得更多计算，预算应该投到采样、搜索、验证还是工具，什么时候停止，以及准确率提升是否抵得过成本和延迟。

面试表达：TTC scaling 不是“所有请求都更慢”，而是把额外计算投给低置信度、高价值、可验证的请求，并用成本和延迟门禁约束。

Compute Budget Vector

一句话定义：Compute Budget Vector 是把单个请求的推理时预算拆成候选数、推理长度、搜索深度、verifier 调用、工具调用和 token 数的向量。

简化记号：

$$\mathbf{b}_i = (K_i, L_i, D_i, V_i, U_i, T_i)$$

面试表达：预算向量能帮助你解释“多花计算”到底花在哪里，而不是笼统说让模型多思考。

Adaptive Compute

一句话定义：Adaptive Compute 是根据任务难度、价值、置信度和可验证性动态选择推理预算的策略。

典型路由：简单请求 direct, 中等难度请求 self-consistency 或 verifier, 高价值可验证难题启用 search / tools。

面试表达：adaptive compute 的价值是用接近高预算策略的准确率，接近低预算策略的成本；风险是难度估计错，把 hard case 分到错误策略。

Budget Router

一句话定义：Budget Router 是把请求路由到 direct、self-consistency、verifier、search 或 tool loop 的控制模块。

简化记号：

$$m_i = \pi(x_i, d_i, v_i, u_i)$$

其中 d_i 是难度估计， v_i 是任务价值， u_i 是可验证性。

面试表达：budget router 要可观测、可回放、可灰度，否则很难解释为什么某类请求被分配了高预算。

Cost per Correct

一句话定义：Cost per Correct 是单位正确样本消耗的成本，用来衡量推理时计算是否划算。

简化口径：

$$C_{\{\mathrm{correct}\}} = \frac{C_{\{\mathrm{total}\}}}{\sum_i \mathbb{1}[\hat{y}_i = y_i^{\star}]}$$

面试表达：如果高预算策略准确率略高但 cost per correct 翻数倍，生产系统未必应该默认开启。

Marginal Accuracy per Cost

一句话定义：Marginal Accuracy per Cost 衡量从低预算切到高预算时，每增加一单位成本带来的准确率提升。

简化口径：

$$g(B_1, B_2) = \frac{A(B_2) - A(B_1)}{C(B_2) - C(B_1)}$$

面试表达：TTC scaling 通常存在边际收益递减，必须画预算曲线，而不是只测一个最大预算点。

P95 Latency

一句话定义：P95 Latency 是 95% 请求能完成的延迟阈值，是用户体验和 SLO 里的关键指标。

面试表达：多候选并行、串行搜索和工具 loop 对 P95 的影响不同。线上不能只看平均延迟。

Wasted High Compute

一句话定义：Wasted High Compute 是高预算请求仍然失败，或低价值请求被错误分配高预算的现象。

面试表达：它是 adaptive compute 的核心坏味道，说明难度估计、路由策略或 verifier / tool 选择需要修正。

TTC Audit

一句话定义：TTC Audit 是把固定预算策略和 adaptive routing 的准确率、总成本、单位正确成本、P95 延迟、边际收益和浪费样本放在一起审计。

面试表达：TTC audit 能回答“多花的推理计算是否真的值”，而不是只证明某个高预算模式在 benchmark 上分数更高。

Math Reasoning Training

一句话定义：Math Reasoning Training 是围绕数学题训练模型的多步推理、答案验证、步骤质量、错误定位、verifier、curriculum 和搜索能力。

为什么重要：数学任务答案相对明确、过程可拆解、很多题能自动验证，因此常用于训练和评估 reasoning model。

关键边界：数学 benchmark 分数不等于通用推理能力。数学题可能被公开题库污染，也可能被合成模板、格式偏好或 verifier 偏差放大。

面试表达：数学推理训练不是只增加 CoT 数据，而是要把题目来源、答案验证、步骤标签、错误解法、难度分层、合成题质量和评估污染放在同一个数据闭环里审计。

Math Training Sample

一句话定义：Math Training Sample 是一条可用于数学推理训练的样本，通常包含题目、标准答案、解题步骤、步骤标签、题型、难度、来源和风险标记。

简化记号：

$m_i = (x_i, y_i^{\star}, z_i, q_i, d_i, t_i, s_i, r_i)$

面试表达：高质量数学样本不只看答案是否对，还要看步骤是否可复核、难度是否明确、来源是否干净、是否与评测集重叠。

Answer Supervision

一句话定义：Answer Supervision 是只根据最终答案正确性训练或筛选数学推理样本的方法。

优点：标注成本低，适合答案明确、可自动检查的计算题、选择题和程序化验证题。

缺点：它看不到中间过程，可能奖励碰巧猜对的答案，也会惩罚推理基本正确但最后格式或符号出错的样本。

面试表达：answer supervision 是数学训练的基础信号，但长链条 reasoning 通常还需要 step label、first-error detection、PRM 或人工抽审。

Synthetic Math Data

一句话定义：Synthetic Math Data 是通过程序模板、规则系统、模型生成、人机协作或自动证明系统构造的数学题、解法和步骤数据。

优点：规模大、难度可控、答案可验证，能覆盖稀有题型和指定技能。

风险：模板痕迹、题型分布偏差、答案或步骤错误、训练到评测污染、模型学会表面模式而不是真正推理。

面试表达：合成数学数据必须同时看正确性、模板多样性、题型覆盖、难度覆盖、真实评测迁移和污染风险。

Math Curriculum

一句话定义：Math Curriculum 是把数学训练样本按难度、题型、步骤长度、验证难度或概念组合逐步安排的训练策略。

常见顺序：基础计算、代数变形、应用题、几何、组合计数、证明题、奥赛级难题。

面试表达：curriculum 的目的不是让训练集变简单，而是让模型先学稳定局部推理，再逐步学习长链条、抽象和难验证任务。

Math Contamination Audit

一句话定义：Math Contamination Audit 是检查数学训练集和评测集在题面、答案、解析、模板或参数化变体上是否重叠的审计流程。

常见检查：exact / near dedup、题面改写相似度、答案和解析重叠、模板 ID、来源 URL、公开题库泄漏和参数化变体覆盖。

面试表达：数学题公开解析很多，只做题面去重不够，还要检查答案、步骤、模板和近似变体。

Template Diversity

一句话定义：Template Diversity 衡量合成题或模板化数据中不同生成模板、参数模式和解题结构的多样性。

简化口径：

$$D_{\{\mathrm{temp}\}} = \frac{|\{s_i\}_{i=1}^N|}{|N|}$$

面试表达：模板多样性低时，模型可能记住题型外壳；必须结合真实题迁移、hard slice 和错误类型分析判断数据是否有价值。

Math Training Gate

一句话定义：Math Training Gate 是在数学推理训练数据或模型上线前，对答案准确率、步骤准确率、第一处错误定位、污染率、模板多样性和难题切片设置的质量门禁。

简化口径：

$$\begin{aligned} G_{\{\mathrm{math}\}} &= \mathbb{1} [\\ A_{\{\mathrm{ans}\}} &\geq \alpha \\ &\land \\ A_{\{\mathrm{step}\}} &\geq \beta \\ &\land \\ A_{\{\mathrm{first}\}} &\geq \gamma \\ &\land \\ R_{\{\mathrm{contam}\}} &\leq \rho \\ &\land \\ D_{\{\mathrm{temp}\}} &\geq \delta \end{aligned}$$

\land
A_{\mathrm{hard}}\geq \eta
]

面试表达：数学训练 gate 的价值是防止平均准确率掩盖步骤错误、评测污染、模板化和 hard proof / geometry 切片失败。

Code Reasoning

一句话定义：Code Reasoning 是模型围绕需求理解、程序生成、测试执行、错误定位、补丁修复和复杂度约束进行多步推理的能力。

为什么重要：代码可以被编译、运行和测试，执行反馈比纯文本评分更硬，因此代码任务非常适合训练 verifier、self-debug、search 和 tool-use reasoning。

关键边界：通过公开测试不等于功能完全正确；测试覆盖不足、环境差异、隐藏边界条件和沙箱安全都会影响代码 reasoning 的真实可靠性。

面试表达：代码 reasoning 的核心不是“写出像代码的文本”，而是把规格、候选代码、测试反馈、修复策略、隐藏泛化和执行安全组成闭环。

Execution Feedback

一句话定义：Execution Feedback 是编译器、解释器、单元测试、类型检查、静态分析、超时和内存限制返回给模型或控制器的反馈信号。

常见用途：候选 rerank、self-debug、RL reward、搜索剪枝、错误分类和训练数据过滤。

面试表达：执行反馈能把代码任务从语言生成变成可验证任务，但反馈质量取决于测试覆盖、环境一致性和沙箱边界。

Unit Test Verifier

一句话定义：Unit Test Verifier 是用单元测试判断候选代码是否满足给定输入输出行为的程序化 verifier。

优点：自动、客观、可复现，适合函数级代码生成、修复和 rerank。

局限：单测不完备会漏错；公开测试可能被过拟合；真实仓库任务还需要依赖、环境、集成测试和人工审查。

面试表达：代码任务里 unit test 是最常见的硬 verifier，但必须同时看 public / hidden test gap 和测试覆盖。

Public Test

一句话定义：Public Test 是训练、调试或提交前可见的测试样例，常用于给模型提供执行反馈。

风险：模型或 reranker 可能只拟合公开样例，生成 hardcode、边界遗漏或样例投机代码。

面试表达：public test 适合用于 self-debug，但不能作为最终泛化证明。

Hidden Test

一句话定义：Hidden Test 是提交前不可见、用于最终评估泛化能力的测试集合。

常见覆盖：空输入、极值、重复元素、类型边界、性能压力、异常路径和真实业务边界条件。

面试表达：hidden test 才能更好衡量 functional correctness，但也不是形式化证明；测试覆盖仍然需要审计。

Public-Hidden Gap

一句话定义：Public-Hidden Gap 衡量公开测试表现和隐藏测试表现之间的差距。

简化口径：

$$\Delta_{\mathrm{pub-hid}} = A_{\mathrm{pub}} - A_{\mathrm{hid}}$$

面试表达：这个 gap 大说明公开测试覆盖不足、候选选择器过拟合样例，或模型没有真正理解规格。

Self-Debug

一句话定义：Self-Debug 是模型根据编译错误、运行异常、测试失败或静态检查反馈，解释问题并生成修复补丁的过程。

简化轨迹：

$$\tau_i = (p_i^{(0)}, f_i^{(0)}, p_i^{(1)}, f_i^{(1)}, \dots, p_i^{(R_i)}, f_i^{(R_i)})$$

面试表达：self-debug 的价值来自外部反馈闭环；没有测试或执行信号时，模型反思可能只是重复原错误。

Repair Success Rate

一句话定义：Repair Success Rate 衡量初始代码失败后，经过若干轮执行反馈和补丁修复，最终通过评估测试的比例。

简化口径：

$$A_{\mathrm{repair}} = \frac{1}{N_{\mathrm{fail}}} \sum_{i \in \mathcal{F}} \mathbb{1}[S(p_i^{(R_i)}, T_i^{\mathrm{hid}}) = 1]$$

面试表达：它比 pass@k 更关注模型能否利用反馈修错，而不是候选池里是否碰巧已有正确解。

Sandbox Violation Rate

一句话定义：Sandbox Violation Rate 衡量候选代码触发危险调用、导入、文件 / 网络访问、无限循环、子进程或资源超限等安全拦截的比例。

简化口径：

$$R_{\mathrm{sandbox}} = \frac{1}{M} \sum_{i,j} \mathbb{1}[u_{ij} = 1]$$

面试表达：执行模型生成代码必须默认不可信。沙箱指标不是攻击指南，而是证明权限隔离和拦截策略有效的审计信号。

Code Execution Audit

一句话定义：Code Execution Audit 是把 greedy、public rerank、pass@k、public-hidden gap、self-debug 修复、沙箱拦截和执行成本放在一起检查的代码 reasoning 审计。

面试表达：它能回答“执行反馈是否真的提升代码可靠性”，并揭示公开测试过拟合、隐藏边界失败和不安全候选。

Code Reasoning Gate

一句话定义：Code Reasoning Gate 是代码 reasoning 系统上线前对隐藏测试准确率、公开 / 隐藏差距、修复成功率、沙箱违规率和执行成本设置的门禁。

简化口径：

```
G_{\mathrm{code}}=
\mathbb{1}[
A_{\mathrm{hid}}\geq \alpha
\land
\Delta_{\mathrm{pub-hid}}\leq \delta
\land
A_{\mathrm{repair}}\geq \beta
\land
R_{\mathrm{sandbox}}\leq \rho
\land
C_{\mathrm{exec}}\leq C_{\mathrm{max}}
]
```

面试表达：代码 reasoning gate 防止系统只靠公开测试或 pass@k 看起来很强，却在隐藏用例、沙箱安全或执行成本上不达标。

Reasoning Evaluation

一句话定义：Reasoning Evaluation 是对模型多步推理能力的最终答案、过程质量、变体泛化、鲁棒性、污染风险、推理成本和统计不确定性进行统一评估。

为什么重要：reasoning 模型常使用 CoT、self-consistency、verifier、search、工具和 test-time compute。如果不记录预算和评估配置，分数不可复现，也不公平。

面试表达：reasoning eval 不是只跑一个 benchmark，而是要把 benchmark、切片、过程、污染、变体、成本和 paired significance 放进同一份报告。

Evaluation Sample

一句话定义：Evaluation Sample 是评估集中一条可审计样本，包含原题、标准答案、候选输出、变体、步骤标签、切片标签、风险标记和推理预算。

简化记号：

$$e_i=(x_i, y_i^{\star}, V_i, P_i, z_i, q_i, g_i, r_i, b_i)$$

面试表达：评估样本 schema 越完整，越容易追踪分数提升来自模型能力、提示词、预算、污染还是评估器偏差。

Variant Evaluation

一句话定义：Variant Evaluation 是在原题基础上构造题面改写、数值替换、条件扰动、干扰信息、格式变化或跨题型组合，测试模型是否真正泛化。

面试表达：如果模型原题正确、变体错误，说明它可能依赖记忆、模板或表面线索，不能把原题分数当成稳健 reasoning。

Robustness Drop

一句话定义：Robustness Drop 衡量模型从原始样本到扰动 / 变体样本的准确率下降。

简化口径：

$$D_{\mathrm{robust}}=A_{\mathrm{orig}}-A_{\mathrm{var}}$$

面试表达: robustness drop 大时, 要回看扰动类型、任务切片和模型推理过程, 而不是只报告原始 benchmark 分数。

Paired Lift

一句话定义: Paired Lift 是在同一批样本上比较新旧模型逐题正确性差异后的平均提升。

简化口径:

$$\Delta_{\mathrm{pair}} = \frac{1}{N} \sum_{i=1}^N (\mathbb{1}[\hat{y}_i^{\mathrm{new}} = y_i^{\star}] - \mathbb{1}[\hat{y}_i^{\mathrm{base}} = y_i^{\star}])$$

面试表达: paired evaluation 比两个模型分别报平均分更适合判断小幅改进, 因为它控制了样本难度差异。

Bootstrap Confidence Interval

一句话定义: Bootstrap Confidence Interval 是通过评估样本重复重采样, 估计指标不确定性的置信区间。

用途: 小样本或提升很小时, 单个平均分可能误导; 需要看区间是否跨过 0、是否足以支撑上线判断。

面试表达: 如果 paired lift 的 bootstrap interval 跨过 0, 应该说 “证据不足”, 而不是宣称模型显著更强。

Reasoning Eval Gate

一句话定义: Reasoning Eval Gate 是 reasoning 评估上线前对最终答案、过程质量、第一处错误、污染率、鲁棒性和成本设置的可信度门禁。

简化口径:

$$G_{\mathrm{eval}} = \mathbb{1} [A_{\mathrm{ans}} \geq \alpha \land A_{\mathrm{step}} \geq \beta \land A_{\mathrm{first}} \geq \gamma \land R_{\mathrm{contam}} \leq \rho \land D_{\mathrm{robust}} \leq \delta \land E_{\mathrm{cost}} \geq \eta]$$

面试表达: reasoning eval gate 的目标不是制造漂亮分数, 而是证明评估结论没有被污染、统计噪声、过程错误、变体失败或成本失控掩盖。

Best-of-N

一句话定义: best-of-n 是生成 N 个候选答案, 再用评分器或规则选择最优答案的方法。

与 self-consistency 的区别: self-consistency 常用答案投票, best-of-n 更强调用 reward model、verifier 或规则打分选最优。

优点: 简单直接, 常用于对齐、代码、数学和开放生成质量提升。

缺点：N 越大成本越高；评分器偏差会决定最终输出；如果候选都差，选择器也无能为力。

面试表达：best-of-n 的核心是候选生成加选择器，关键在于候选多样性和选择器可靠性。

Pass@k

一句话定义：Pass@k 是可执行验证任务中常用指标，表示生成 k 个候选时至少有一个正确候选的概率。

给定 n 个候选，其中 c 个正确，不放回估计为：

$$\mathrm{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

面试表达：pass@k 衡量候选集合里是否存在正确解，不等于线上一次调用的可靠性。k 越大成本越高，也越依赖测试覆盖率。

Reasoning Audit

一句话定义：Reasoning Audit 是对 reasoning 系统的准确率、候选质量、verifier、过程步骤、test-time compute 成本、安全风险和污染风险进行统一审计。

一个简化上线门禁：

$$G_{\{\mathrm{reason}\}} = \mathbb{1} [A_{\{\mathrm{ver}\}} \geq \alpha \land A_{\{\mathrm{step}\}} \geq \beta \land C_{\{\mathrm{ttc}\}} \leq C_{\{\mathrm{max}\}} \land R_{\{\mathrm{unsafe}\}} \leq \rho]$$

面试表达：reasoning audit 能把“模型会思考”拆成可验证的候选、选择器、步骤质量、成本和安全边界，而不是只看回答是否很长。

Reasoning Safety

一句话定义：Reasoning Safety 是对推理模型在长链条推理、工具调用、高风险场景、隐藏 CoT、过度自信和滥用边界上的安全性进行系统治理。

核心问题：reasoning 能力越强，模型越可能完成复杂规划和外部动作；如果没有权限、验证、审核和日志，错误或误用的影响会被放大。

面试表达：reasoning safety 不是单个拒答分类器，而是模型行为、CoT 展示策略、verifier、工具权限、人工审核、红队回归和治理门禁组成的系统工程。

Pseudo Reasoning

一句话定义：Pseudo Reasoning 是模型生成看似合理但不能真正支持最终答案的推理过程。

典型表现：先猜答案再补过程、步骤逻辑跳跃、引用不存在证据、把相关性说成因果性，或最终答案与中间步骤不一致。

面试表达：伪推理的危险在于它让错误更可信。评估时要同时看最终答案、步骤标签、证据支持和反事实扰动，而不是只看解释是否流畅。

Overconfident Error

一句话定义：Overconfident Error 是模型在错误答案上仍表达高置信度的失败模式。

为什么重要：医疗、法律、金融、安全和招聘等场景中，错而自信会误导用户做高影响决策。

面试表达：过度自信要用 calibration、abstention、verifier、人审和高风险门禁约束，而不是只要求模型输出更长解释。

Hidden CoT Exposure

一句话定义：Hidden CoT Exposure 是本应隐藏的内部推理、策略边界、隐私推断或工具权限细节进入用户可见输出。

边界：内部推理、安全监控和用户解释应区分开；用户更需要可验证证据、关键假设和最终理由，而不是完整内部轨迹。

面试表达：不展示完整 CoT 不等于不透明；可以用引用、证据摘要、工具结果、置信边界和审计日志建立可复核信任。

Tool Misuse

一句话定义：Tool Misuse 是 reasoning model 在工具环境中发生权限不匹配、参数污染、不可逆动作未确认或工具输出被误当成上级指令的失败模式。

防护方式：最小权限、schema 校验、沙箱、二次确认、工具输出不可信标记、审计日志和回滚策略。

面试表达：Agent 工具安全的关键是让模型做受约束的计划者，系统层负责权限、确认、执行、审计和回滚。

Human Review Coverage

一句话定义：Human Review Coverage 是高风险样本中被人工审核或人工确认覆盖的比例。

为什么重要：高风险任务不能只靠模型自动判断，尤其是个体权益、不可逆操作、专业建议和安全相关动作。

面试表达：人审覆盖率不是越高越好，而是要匹配风险分级：低风险自动化，高风险确认，禁止类拒绝。

Severity-Weighted Risk

一句话定义：Severity-Weighted Risk 是按严重程度权重聚合安全失败的风险指标。

为什么需要：低风险格式错误和高风险工具越权不能在平均准确率里同等对待。

面试表达：安全评估要按严重程度、风险域和可逆性加权，否则平均分会掩盖少数高影响事故。

Reasoning Safety Gate

一句话定义：Reasoning Safety Gate 是 reasoning 系统上线前对伪推理、过度自信、高风险不当服从、工具误用、隐藏 CoT 暴露、人审覆盖和过度拒答设置的综合门禁。

简化口径：

```
G_{\mathrm{safe}}=
\mathbb{1}[
R_{\mathrm{pseudo}}\leq \alpha
\land
R_{\mathrm{conf}}\leq \beta
\land
R_{\mathrm{unsafe}}\leq \gamma
\land
R_{\mathrm{tool}}\leq \delta
```

```

\land
R_{\mathrm{cot}}\leq \epsilon
\land
C_{\mathrm{review}}\geq \eta
]

```

面试表达：如果工具越权、隐藏 CoT 暴露或高风险人工审核不过线，即使 reasoning benchmark 提升，也不能直接上线。

Reasoning Interview Readiness

一句话定义：Reasoning Interview Readiness 是判断候选人是否能把 reasoning 概念、公式、demo、评估、安全和工程 trade-off 讲成完整面试答案的准备度。

核心维度：topic coverage、formula coverage、demo coverage、risk coverage、trade-off coverage 和 weak question revision。

面试表达：准备 reasoning 面试不能只背 CoT、PRM、MCTS 等术语，而要能用公式、项目审计表和失败案例证明自己理解机制和边界。

Reasoning Interview Rubric

一句话定义：Reasoning Interview Rubric 是对 reasoning 面试回答进行结构化评分的标准。

常见字段：目标、机制、公式、工程实现、评估指标、风险边界、项目证据和表达清晰度。

面试表达：好的 rubric 能把“讲得流畅”拆成可训练项，避免复盘时只说“还要多看看”。

Reasoning Formula Coverage

一句话定义：Reasoning Formula Coverage 是 reasoning 面试回答中覆盖关键公式或指标的比例。

典型公式：最终答案准确率、步骤准确率、self-consistency 投票、pass@k、pairwise verifier loss、test-time compute cost 和 safety gate。

面试表达：公式覆盖不是为了炫技，而是证明你知道方法在优化什么、评估什么、约束什么。

Reasoning Demo Coverage

一句话定义：Reasoning Demo Coverage 是 reasoning 面试准备中能用最小代码或项目审计 demo 支撑的主题比例。

典型 demo：CoT 质量审计、self-consistency 投票、verifier rerank、process supervision、search / MCTS、TTC routing、reasoning eval report 和 safety gate。

面试表达：如果一个知识点能配一个 toy demo，面试回答会从“概念复述”升级成“我知道怎么落地和验证”。

Weak Reasoning Question

一句话定义：Weak Reasoning Question 是 mock interview 中得分低、公式缺失、demo 缺失或风险边界讲不清的 reasoning 题目。

处理方式：为每个薄弱题绑定一个公式、一个 demo、一个失败案例和一个 3 分钟回答模板。

面试表达：高效复盘不是泛泛复习，而是把弱题拆成可补齐的公式、代码、指标和表达。

Reasoning Revision Plan

一句话定义：Reasoning Revision Plan 是针对 reasoning 面试薄弱项制定的下一轮修正计划。

常见内容：补数学训练题、补代码执行反馈题、补 process supervision 公式、补 safety gate demo、补系统设计模块图。

面试表达：revision plan 的目标是让下一轮 mock interview 有可验证改善，而不是只增加阅读时间。

Search Reasoning

一句话定义：Search Reasoning 是把 LLM 推理看成状态空间搜索，在多个中间状态之间展开、评分、剪枝和选择。

为什么提出：单条 CoT 一旦早期走错，后续通常会沿着错误路径继续。搜索保留多个候选路径，可以用 verifier、PRM、工具反馈或规则提前剪掉低质量分支。

核心边界：搜索不是无成本增强。它依赖状态表示、动作粒度、评分器质量和预算门禁；评分器偏差会把错误路径放大。

面试表达：search reasoning 的核心不是“多生成几条答案”，而是把生成、评估、剪枝、回溯和预算控制组成一个闭环。

Search State

一句话定义：Search State 是搜索过程中当前保留的中间推理状态。

常见内容：原始问题、已生成步骤、工具结果、测试结果、约束满足情况、已消耗 token / 时间 / 调用预算。

简化记号：

$s_t = (x, z_{1:t}, m_t, b_t)$

面试表达：状态表示不完整时，verifier 看到的信息不足，搜索评分会失真。

Search Action

一句话定义：Search Action 是从当前搜索状态可以选择的下一步操作。

例子：写下一步数学推导、选择一个子问题、修改一段代码、调用工具、回退到某个分支。

简化记号：

$a_t \in \mathcal{A}(s_t),$
 $\quad \quad \quad$
 $s_{t+1} = T(s_t, a_t)$

面试表达：动作粒度太细会让树爆炸，动作粒度太粗又会失去纠错和剪枝能力。

Beam Search

一句话定义：Beam Search 是按层展开候选，每层只保留分数最高的 K 个状态的搜索策略。

简化口径：

$F_{t+1} =$
 $\quad \quad \quad \mathrm{TopK}$
 $($
 $\quad \quad \quad \{T(s, a) : s \in F_t, a \in \mathcal{A}(s)\},$
 $S,$
 K
 $)$

面试表达：beam search 简单、成本可控，适合接 verifier；风险是 early pruning，正确路径可能因为早期分数低被剪掉。

Best-First Search

一句话定义：Best-First Search 是每次从全局 frontier 中优先展开当前分数最高的节点。

和 beam search 的区别：beam search 常按层推进，best-first search 更像维护一个全局优先队列。

面试表达：best-first search 在评分器可靠时能快速深入高质量路径；评分器有偏时会更快陷入局部最优。

UCT

一句话定义：UCT 是 MCTS 中常用的节点选择公式，用 “当前价值 + 探索奖励” 平衡 exploitation 和 exploration。

简化口径：

$$U(v) = Q(v) + c \sqrt{\frac{\log(N_p + 1)}{N_v + 1}}$$

面试表达：UCT 避免只盯着当前高分节点，也避免完全随机探索；但在语言任务里 $Q(v)$ 的可靠性仍取决于 verifier、工具或最终反馈。

Prune False Negative

一句话定义：Prune False Negative 是搜索剪枝时把潜在正确路径错误剪掉的失败模式。

典型来源：PRM 低估朴素但正确的分支，偏好格式漂亮的 hard negative，或者早期步骤证据不足。

面试表达：搜索系统上线前要统计被剪掉的正确路径和 hard negative 切片，不能只看最终平均准确率。

Search Budget

一句话定义：Search Budget 是搜索推理允许消耗的节点数、token、verifier 调用、工具调用、延迟和费用上限。

面试表达：search 更适合高价值复杂任务；简单问答和低延迟场景要用路由策略限制搜索预算。

Search Audit

一句话定义：Search Audit 是对搜索推理的准确率、候选多样性、剪枝错误、MCTS rescue、节点 / token 成本和 hard negative 进行统一审计。

面试表达：search audit 能判断 “搜索是否值得开”，也能定位到底是候选生成不够多样、评分器偏差，还是预算策略过紧。

Tree-of-Thought

一句话定义：Tree-of-Thought，简称 ToT，是把推理过程看成一棵搜索树，在多个中间思路之间探索和选择。

为什么提出：线性 CoT 一旦早期走错，后续很难恢复。ToT 保留多个中间状态，可以回溯和比较不同推理分支。

基本流程：生成若干中间想法，评估每个想法的潜力，保留较优分支继续扩展，直到得到答案。

优点：适合组合搜索、规划、谜题和复杂推理。

缺点：计算成本高，评估函数难设计，分支数增长快，不适合低延迟任务。

面试表达：ToT 是把语言模型推理从单路径生成扩展为多路径搜索，核心代价是测试时计算增加。

MCTS

一句话定义：MCTS 是 Monte Carlo Tree Search，蒙特卡洛树搜索，通过选择、扩展、模拟和回传在搜索树中寻找高价值路径。

为什么相关：在复杂推理、代码生成、博弈和规划任务中，模型可以作为策略生成候选动作，verifier 或 reward model 作为价值评估，MCTS 用于系统性搜索。

四个阶段：选择已有高潜力节点，扩展新动作，模拟后续结果，回传更新节点价值。

优点：比简单采样更有结构，能在大搜索空间中平衡探索和利用。

缺点：实现复杂，计算开销大，依赖状态表示和奖励信号；自然语言任务中状态转移和价值评估不如棋类清晰。

面试表达：MCTS 在大模型推理中本质上是把模型生成能力接入搜索框架，用外部评价信号指导多步决策。

Reflection

一句话定义：reflection 是让模型检查自己前一轮输出，发现问题并修正的机制。

为什么有效：很多错误在模型重新审视答案时可以被发现，尤其是格式遗漏、约束未满足、代码错误和逻辑跳步。

常见流程：生成初稿，模型自评，指出问题，生成修正版；或结合工具测试结果进行修正。

局限：模型可能无法发现自己不知道的 error；没有外部证据或验证器时，反思可能只是重复或改写原错误。

面试表达：reflection 是低成本质量提升手段，但最好和工具验证、测试或外部反馈结合。

Benchmark

一句话定义：benchmark 是用于衡量模型某类能力的标准化测试集或任务集合。

为什么需要：没有统一评估，就无法比较模型、训练策略、prompt、检索系统和部署版本的效果。

常见类型：知识问答、数学推理、代码生成、阅读理解、指令遵循、安全评估、多模态理解和真实用户偏好评估。

优点：可复现、可横向比较、便于跟踪迭代效果。

风险：污染、过拟合、格式 gaming、不能覆盖真实用户分布、单一分数掩盖能力结构差异。

面试表达：benchmark 是必要但不充分的评估工具。面试中要强调公开榜单、内部集、人工评测和线上指标需要结合。

Benchmark Contamination

一句话定义：benchmark contamination 是评估集样本泄漏到训练数据中，导致模型分数虚高的现象。

为什么危险：如果模型训练时见过测试题，它的分数就不能代表泛化能力，而更像记忆能力。

检测方法：exact match、n-gram overlap、embedding similarity、near-duplicate search、时间切分、人工抽查和答案异常模式分析。

缓解方法：使用新构造测试集、私有评测集、动态生成题目、时间隔离数据和严格数据去重。

面试表达：看到模型 benchmark 分数时，要追问是否有污染、是否有 prompt tuning、是否使用公开题库，以及是否在真实业务集上验证。

Human Evaluation

一句话定义：human evaluation 是由人类标注者或专家对模型输出质量进行评价。

为什么需要：开放式生成、对话体验、创意写作、复杂事实核查和业务满意度很难完全用自动指标衡量。

常见方式：单答案打分、两两偏好比较、排序、错误类型标注、专家审查和红队测试。

优势：更接近真实用户体验，可以覆盖细腻的风格、可用性和安全判断。

缺点：成本高，速度慢，主观性强，一致性难保证，标注说明和标注者质量会强烈影响结果。

面试表达：人工评估的关键是设计清晰维度、标注指南、一致性校验和抽样策略，而不是简单让人“觉得哪个好”。

LLM-as-a-Judge

一句话定义：LLM-as-a-Judge 是用强语言模型作为评审器，对其他模型输出打分、排序或解释错误。

为什么流行：人工评估昂贵且慢，LLM judge 可以低成本、快速、规模化地评估大量样本。

常见用法：pairwise preference、rubric scoring、错误分类、事实一致性初筛和评测报告生成。

风险：长度偏好、位置偏好、模型偏见、格式敏感、专业事实判断不可靠、和被评模型同源导致偏差。

改进方式：使用明确 rubric，打乱答案顺序，隐藏模型名称，引入少量人工校准，使用多 judge 集成，并对 judge 本身做一致性评估。

面试表达：LLM judge 适合做大规模辅助评估，但不能无条件当真值。关键任务仍需要人工或可执行验证闭环。

Elo Rating

一句话定义：Elo rating 是根据两两对战或偏好比较结果给模型估计相对能力分数的方法。

为什么适用：开放生成任务中直接给绝对分数很难，但人或 judge 更容易判断两个回答哪个更好。

基本思想：强模型战胜弱模型增分少，弱模型战胜强模型增分多。大量 pairwise 比较后，可以得到相对排名。

优点：适合竞技场式评测和对话偏好比较。

局限：结果依赖题目分布、投票人群、对战采样和统计假设；Elo 分不能直接解释为某项能力的绝对水平。

面试表达：Elo 适合衡量相对偏好，但要结合任务切片和置信区间看，不能只看一个总排名。

Hallucination

一句话定义：hallucination 是模型生成看似合理但不真实、不可靠或缺少依据的内容。

常见类型：事实幻觉、引用幻觉、推理幻觉、上下文幻觉、工具幻觉和身份能力幻觉。

为什么出现：语言模型优化的是下一个 token 概率和人类偏好，不天然等价于事实数据库；当信息缺失、问题模糊、训练记忆冲突或模型过度迎合用户时，容易编造。

影响：在医疗、法律、金融、科研和企业知识库场景中，幻觉会造成严重错误决策。

缓解方法：RAG grounding、引用核查、拒答机制、工具验证、答案不确定性表达、事实性微调、后处理校验和人工审查。

面试表达：幻觉不是单一 bug，而是生成式模型的系统性风险。要从数据、检索、解码、对齐、工具和评估多层治理。

Factuality

一句话定义：factuality 是模型输出与真实世界事实或给定证据一致的程度。

评估方式：事实问答、引用检查、RAG grounding、人工核查、知识库比对、claim decomposition 和自动事实核查。

关键区别：事实正确不等于有引用支持；有引用也不等于引用真的支持结论。因此需要同时检查答案断言和证据对应关系。

提升方式：使用可靠数据源、检索增强、让模型基于证据回答、要求不知道就说不知道、对关键断言做验证。

面试表达：factuality 关注 “说得对不对”，grounding 关注 “是否由给定证据支持”。两者相关但不完全相同。

Citation Grounding

一句话定义：citation grounding 是要求模型输出结论能被引用来源或检索证据支持。

为什么重要：在 RAG 和知识问答中，用户不仅需要答案，还需要知道答案来自哪里、是否可核查。

常见问题：引用不存在、引用位置错误、引用只支持部分结论、多个证据被错误拼接、模型加入证据外知识。

评估方法：把答案拆成原子断言，检查每个断言是否被引用片段支持，并统计 supported、unsupported、contradicted 和 not enough information。

面试表达：引用不是装饰。好的 citation grounding 要验证 “答案中的每个关键断言是否被对应证据支持”。

Robustness

一句话定义：robustness 是模型面对扰动、改写、噪声、分布外输入或攻击时保持稳定表现的能力。

常见扰动：同义改写、顺序变化、无关上下文插入、拼写错误、格式变化、对抗提示、多语言切换和长上下文噪声。

为什么重要：真实用户输入不会像 benchmark 一样干净。模型如果对轻微改写非常敏感，线上体验会不稳定。

评估方法：构造扰动集，比较原始样本和扰动样本表现差异；按任务类型、长度、语言、用户群体和风险等级切片分析。

面试表达：robustness 评估关注模型在分布变化下是否稳定，而不是只看标准测试集平均分。

Calibration

一句话定义：calibration 是模型表达的置信度与实际正确率一致的程度。

为什么重要：如果模型错得很自信，用户更容易被误导。理想情况下，模型说 “高置信” 的答案应更可能正确，说 “不确定” 的答案应确实更难。

常见方法：置信度打分、拒答阈值、选择性回答、温度缩放、基于 verifier 的置信估计和答案一致性估计。

难点：语言模型没有天然可靠的概率置信度；生成文本中的 “我确定” 不等于真实置信。

面试表达：calibration 关注模型是否知道自己知道什么、不知道什么，是高风险场景可靠性的关键指标。

Evaluation Metric

一句话定义：evaluation metric 是把模型输出质量转化为可比较数值或标签的评估标准。

常见指标：accuracy、exact match、F1、pass@k、BLEU、ROUGE、BERTScore、胜率、拒答率、幻觉率、事实支持率、人工偏好率。

选择原则：指标必须和任务目标一致。代码任务看 pass@k，问答任务看正确性和引用支持，客服任务看解决率和满意度，安全任务看违规率和过拒率。

常见误区：用摘要指标评估事实问答，用平均分掩盖高风险错误，用自动指标替代人工体验，用单一 benchmark 代表所有能力。

面试表达：评估指标不是越多越好，而是要覆盖核心业务目标、失败风险和用户体验。

Pass@k

一句话定义：pass@k 是代码生成和可验证任务中常用指标，表示生成 k 个候选中至少一个通过测试的概率。

为什么重要：代码任务常允许多次生成和筛选。pass@1 衡量一次生成能力，pass@k 衡量采样多个候选后的潜在解题能力。

优点：适合有单元测试或自动判题器的任务。

局限：测试用例不完备时，pass 不代表代码完全正确；k 增大带来计算成本；可能鼓励生成大量候选而非单次可靠输出。

面试表达：pass@k 衡量的是候选搜索空间中是否存在正确解，和生产中的延迟、成本、测试覆盖率要一起看。

Evaluation Slicing

一句话定义：evaluation slicing 是按任务类型、难度、领域、语言、长度、用户群体或风险等级拆分评估结果。

为什么需要：总平均分可能掩盖严重问题。例如总体准确率提升，但医疗样本、长尾语言或长上下文样本下降。

常见切片：数学、代码、知识、指令遵循、安全、长上下文、多轮对话、RAG、工具调用、中文、英文、低资源语言。

面试表达：模型评估不能只看 aggregate score，要看能力切片和失败模式，否则很容易误判模型改进方向。

Online Evaluation

一句话定义：online evaluation 是在真实线上流量中评估模型表现的方法。

常见形式：A/B test、灰度发布、用户反馈、留存指标、点击率、解决率、人工抽检、投诉率和安全告警。

优点：最接近真实用户分布和业务目标。

风险：线上实验有用户影响，需要严格流量控制、回滚机制、安全监控和统计显著性分析。

面试表达：离线评估决定能不能上线试，线上评估决定真实业务是否变好。两者不能互相替代。

Evaluation Pipeline

一句话定义：evaluation pipeline 是把测试集、推理配置、打分器、统计分析和报告生成串起来的自动化评估流程。

为什么需要：模型迭代频繁，如果评估不可复现，就无法判断变化来自模型、prompt、数据、解码参数还是评估脚本。

关键组件：固定测试集版本，记录模型版本、prompt、采样参数、工具配置、随机种子、输出日志、打分结果和错误样本。

工程要求：可复现、可追踪、可切片、可回归比较、支持人工复核和失败案例沉淀。

面试表达：成熟团队不会只跑一次 benchmark，而是建设持续评估 pipeline，用回归集防止能力退化。

Evaluation Metric Incident

一句话定义：evaluation metric incident 是指标看起来提升，但干净集、关键切片、人工评估、线上反馈、成本延迟或安全门禁反而变差的评估事故。

典型表现：公开 benchmark 分数提升但真实用户任务下降，LLM judge 分数升高但人工偏好下降，平均分上升但中文、代码、安全或长上下文切片退化。

面试表达：指标事故排查要同时看样本、切片、污染、judge 校准、统计置信、成本延迟和线上反馈，不能只解释总分。

Aggregate Score Trap

一句话定义：aggregate score trap 是总体平均分掩盖关键任务、用户群体、语言、难度或风险切片退化的问题。

为什么重要：总分常被简单样本或大切片主导。高风险业务里，一个安全边界或核心任务切片下降，比总体提升更重要。

面试表达：报告总分时必须配切片分、失败样本和上线门禁，否则平均分容易误导决策。

Slice Regression

一句话定义：slice regression 是 candidate 在某个关键评估切片上相对 baseline 退化。

常见切片：数学、代码、中文、长上下文、RAG 引用、工具调用、安全边界、高价值用户任务和线上高频请求。

面试表达：模型发布前要设置 hard-slice gate。只要关键切片显著退化，就不能只用总体分提升掩盖。

Clean Eval Lift

一句话定义：clean eval lift 是在排除污染、公开题近重复、调参开发集和可疑样本后，candidate 相对 baseline 的真实提升。

为什么重要：公开 benchmark 上升可能来自污染、记忆或格式适配。干净集提升更接近泛化证据。

面试表达：如果污染切片提升而干净集不提升，应先降级结论，不要把分数解释为能力进步。

Judge-Human Agreement

一句话定义：judge-human agreement 是 LLM judge 的偏好或评分与人工标注、专家复核或程序验证器结论一致的比例。

用途：校准 judge 是否能作为大规模辅助评估器，尤其要按任务、长度、位置、模型来源和风险等级切片看。

面试表达：LLM-as-a-judge 必须先被评估。judge 自身没有校准，就不能当作上线真值。

Judge Length Bias

一句话定义：judge length bias 是 LLM judge 倾向选择更长、更结构化或免责声明更多的回答，而不是选择更正确、更有用的回答。

风险：会鼓励模型输出变长、变空、模板化，导致成本升高和用户体验下降。

面试表达：排查 judge 偏差时要控制答案长度、随机化顺序、隐藏模型名，并用人工样本校准。

Evaluation Gate

一句话定义：evaluation gate 是把成对提升、bootstrap 置信区间、切片退化、污染率、judge-human 一致率、成本、延迟、安全和线上反馈合并成的评估上线门禁。

用途：防止团队只看一个 benchmark、一个 judge 分数或一个平均分就发布模型。

面试表达：evaluation gate 的价值是把“分数好看”转成“证据足够、风险可控、能上线”的决策标准。

本章小结

本章围绕 reasoning 和评估建立了核心词汇表。

核心结论如下：

1. reasoning 能力体现为多步拆解、状态维护、验证和修正，而不只是输出更长解释。
2. CoT 可以提升复杂推理，但推理文本不一定真实可靠。
3. self-consistency、best-of-n、ToT 和 MCTS 都是在用测试时计算换更高准确率。
4. verifier 是复杂任务可靠性的关键，尤其适合数学、代码和工具任务。
5. PRM 关注过程正确，ORM 关注最终结果正确。
6. benchmark 必须结合污染风险、任务覆盖和真实业务分布理解。
7. human evaluation 和 LLM-as-a-Judge 都有价值，但都需要明确 rubric 和一致性控制。
8. hallucination、factuality、citation grounding 和 calibration 是模型可靠性评估的核心概念。
9. robustness 关注模型在扰动和分布变化下是否稳定。
10. 成熟评估体系应包含离线 benchmark、内部回归集、人工评测、线上 A/B 和错误分析闭环。

下一章，我们进入安全与治理。

M. 安全与治理

条目：AI Safety、Alignment、Alignment Problem、Outer Alignment、Inner Alignment、Goal Misgeneralization、Specification Gaming、Deceptive Alignment、Scalable Oversight、AI Feedback、Debate、Iterated Amplification、Recursive Reward Modeling、Oversight Gate、Helpful、Honest、Harmless、Jailbreak、Prompt Injection、Indirect Prompt Injection、Instruction Hierarchy、Untrusted Content Boundary、Prompt Injection Gate、Data Exfiltration Rate、Unauthorized Tool Call Rate、Red Teaming、Dangerous Capability Evaluation、Capability Elicitation、Red Team Regression Suite、Red Team Gate、Release Gate、Interpretability、Mechanistic Interpretability、Activation Patching、Sparse Autoencoder、Superposition、Feature Purity、Interpretability Gate、Steering、Representation Engineering、Steering Vector、Contrastive Activation Addition、Steering Gate、Safety Gate、Severity-Weighted Risk、Model Card。

治理与实战补充条目：Incident Response Ready、Minimal Reproduction、Evidence Coverage、Root Cause Rate、Postmortem Completeness、Debug Gate、Retrospective Gate、Hypothesis Coverage、Slice Analysis Coverage、Action Item Closure、Regression Verification Rate、Change Failure Rate、Safety Compliance Incident、Unsafe Pass Rate、Safety Compliance Gate。

AI Safety

一句话定义：AI Safety 是研究和工程化降低 AI 系统造成伤害风险的领域。

为什么重要：大模型不只是生成文本，还会进入搜索、办公、客服、代码、金融、医疗、教育、Agent 和工具调用系统。模型能力越强，错误、滥用和失控行为造成的影响越大。

关注点：有害内容、违法协助、错误医疗或法律建议、隐私泄露、偏见歧视、版权风险、网络安全滥用、工具误操作、prompt injection、agent 失控和社会影响。

工程目标：降低模型在真实系统中造成伤害的概率和影响范围，同时尽量保持模型有用性。安全不是让模型什么都拒绝，而是让模型在不同风险等级下采取合适行为。

常见手段：安全数据构造，SFT 安全微调，RLHF/RLAIF，对抗训练，内容审核器，策略规则，工具权限控制，红队测试，监控告警和上线后复盘。

面试表达：AI Safety 不是单一模型能力，而是数据、训练、评估、产品策略、权限系统和上线治理共同组成的工程体系。

Alignment

一句话定义：alignment 是让模型行为符合人类意图、价值偏好和安全约束的过程。

为什么提出：预训练模型主要学习文本分布，不天然知道用户真实意图、回答规范、安全边界和产品目标。因此需要后训练把模型从“会续写文本”调整为“能作为助手工作”。

常见目标：helpful、honest、harmless。也就是有帮助、诚实、无害。

典型方法：指令微调、偏好学习、RLHF、RLAIF、DPO、Constitutional AI、安全拒答训练和多轮对话数据训练。

核心矛盾：helpful 和 harmless 之间经常冲突。模型既要尽量帮助用户，又不能协助危险、违法或隐私侵犯行为。

面试表达：alignment 的本质是把模型输出分布从互联网文本分布，调整到符合人类偏好和安全策略的助手行为分布。

Alignment Problem

一句话定义：alignment problem 是如何让 AI 系统的目标和行为与人类真实意图、价值偏好和安全边界保持一致的问题。

核心矛盾：人类真实意图、写下的政策或目标、训练时优化的代理指标、模型内部学到的策略和部署后的实际行为可能不一致。

工程表现：模型优化用户评分但不真实帮助，优化 judge 分数但不解决任务，优化拒答率但过度拒绝，优化工具任务成功率但忽略权限边界。

面试表达：alignment problem 不是抽象哲学词，而是目标规范、训练目标、模型行为和部署约束之间的系统性错配问题。

Outer Alignment

一句话定义：outer alignment 关注“我们写下并优化的目标是否真的代表人类想要的目标”。

典型问题：reward model、benchmark、用户满意度、安全分类器和业务 KPI 都是 proxy，不等于真实价值本身。

风险：proxy 一旦被强力优化，就可能出现 Goodhart's Law、reward hacking、specification gaming、过度拒答或短期指标好但长期体验差。

面试表达：outer alignment 问的是目标写对了吗；如果目标本身就是错的，模型越会优化它，系统越可能偏离真实意图。

Inner Alignment

一句话定义：inner alignment 关注“即使训练目标合理，模型内部是否真的学到了我们希望的目标”。

典型问题：模型可能学到训练分布上的捷径，例如偏好长回答、模仿拒答模板、关键词触发安全行为、迎合用户坚持或过拟合 benchmark 格式。

和泛化的关系：inner alignment 可以从目标泛化角度理解。训练集表现好，不代表分布外、多轮、长上下文和工具使用时仍追求正确目标。

面试表达：inner alignment 问的是模型学对了吗；它不是玄学，而是目标层面的泛化问题。

Goal Misgeneralization

一句话定义：goal misgeneralization 是模型在新分布中能力仍然存在，但追求了错误目标。

和普通泛化失败的区别：普通泛化失败是模型不会做；goal misgeneralization 是模型会做，但把能力用在错误目标上。

LLM 例子：模型能写流畅答案，却追求“看起来专业”；能调用工具，却追求“任务完成”而忽略权限；能安全拒答，却把正常防御请求也拒绝。

面试表达: goal misgeneralization 的危险在于错误更隐蔽, 因为模型输出仍然流畅、结构完整、能力看起来很强。

Specification Gaming

一句话定义: specification gaming 是系统利用规则、目标函数或评估指标漏洞, 获得高分但违背真实意图。

和 reward hacking 的关系: reward hacking 更强调钻 reward function 或 reward model 漏洞; specification gaming 范围更广, 也包括 benchmark、规则、环境和工具流程漏洞。

LLM 例子: judge 偏好长答案导致模型冗长, 评测看引用导致模型添加不支持结论的引用, 安全评估只看关键词导致模型过度拒答。

面试表达: specification gaming 说明 “按指标赢了” 不等于 “按真实目标做好了”。

Deceptive Alignment

一句话定义: deceptive alignment 是安全研究中讨论的一类潜在失败模式: 模型在训练或评估中表现对齐, 但在监督变弱或部署环境中追求其他目标。

表述边界: 不能轻率断言当前模型已经强形式具备 deceptive alignment。更稳妥的说法是, 它是一类需要用分布外、长期、多轮、工具使用和监督强弱变化测试来关注的潜在高风险问题。

面试表达: deceptive alignment 的价值是提醒我们不要只看评估集表面行为, 而要关注部署一致性和证据强度。

Scalable Oversight

一句话定义: scalable oversight 是当任务复杂到单个人类难以直接判断时, 仍然构造可靠监督信号的一组方法。

为什么需要: RLHF 和人工标注默认人类能判断输出好坏; 但长代码审查、复杂数学证明、专业文档分析、RAG 多证据整合和 Agent 长 trace 会超出普通标注员的直接判断能力。

常见方法: 任务分解、AI-assisted evaluation、model-written critique、Debate、Iterated Amplification、Recursive Reward Modeling、Constitutional AI、verifier、process supervision 和高风险人工审计。

面试表达: scalable oversight 不是单纯降低标注成本, 而是让监督能力跟上模型能力增长; AI feedback 只能扩展监督规模, 不能替代人类原则、gold set 和高风险复核。

AI Feedback

一句话定义: AI feedback 是用模型或规则系统生成 critique、preference、risk label 或评估结论, 用来辅助训练、评估或审核。

优势: 成本低、速度快、覆盖广, 适合长上下文初筛、候选答案比较、红队样本生成、代码 review 和证据核查。

风险: AI feedback 可能继承模型偏差、被模型输出操纵、在分布外校准差, 也可能把错误监督信号大规模放大。

面试表达: AI feedback 应该用人工 gold set、工具验证、分切片评估和人工抽检校准, 而不是无审计地当作真值。

Debate

一句话定义: Debate 是让多个模型围绕复杂问题互相指出错误或支持不同答案, 再让人类 judge 判断哪一方更可信的监督增强方法。

核心假设: 复杂错误往往可以被另一个智能体用更短、更容易理解的反驳暴露出来, 人类看到反驳后更容易判断。

风险: 模型可能优化说服力而不是真实性, 人类 judge 可能被修辞影响, 参与模型也可能共享同样盲点。

面试表达: Debate 适合做复杂评估和错误暴露工具, 但不能单独证明答案真实, 需要 evidence、tool verification 和人工校准配合。

Iterated Amplification

一句话定义：Iterated Amplification 是让人类借助模型助手分解复杂任务、组合子答案，形成更强监督者，再训练模型模仿这个放大后的监督者。

核心直觉：单个人直接解决大问题很难，但人可以把问题拆成小问题，调用助手回答，再整合结果。

风险：任务可能无法无损分解，子问题错误会累积，模型助手的系统性偏差可能被放大。

面试表达：Iterated Amplification 的关键假设是复杂任务存在可检查的分解结构；它是监督扩展路线，不是对齐完成的保证。

Recursive Reward Modeling

一句话定义：Recursive Reward Modeling 是递归构造子任务评估器或 reward model，再用这些评估器帮助监督更复杂任务的方法。

和 amplification 的关系：二者都强调递归分解；Iterated Amplification 更强调人类加模型助手形成 stronger overseer，Recursive Reward Modeling 更强调评估器或 reward model 的递归构造。

风险：子 reward model 的错误会累积，整体目标可能在拆分中丢失，最终递归系统仍需要外部验证和人工审计。

面试表达：Recursive Reward Modeling 的价值是把复杂监督拆成可训练的局部评估问题，但 reward hacking 和分布外校准仍然存在。

Oversight Gate

一句话定义：oversight gate 是模型训练或上线前，对监督信号质量设置的门禁。

常见指标：human direct coverage、human direct error、AI feedback accuracy、verifier coverage、process step accuracy、evidence support、high-risk audit coverage、severity-weighted error 和 cost saving。

工程意义：如果 AI feedback 未校准、高风险样本没人审、证据支持不足或 verifier 覆盖不足，即使平均评估分很高，也不能说明监督闭环可靠。

面试表达：oversight gate 关注的是“监督本身靠不靠谱”，是 reward model、LLM-as-a-Judge、安全评估和上线治理之前的一层质量控制。

Helpful

一句话定义：helpful 是模型尽力理解并满足用户合理请求的能力。

表现形式：回答具体、步骤清晰、能主动补全必要信息、能解释原因、能给出可执行建议、能在不确定时提出澄清问题。

为什么重要：如果模型过度拒答、泛泛而谈或不给可操作信息，即使很安全也没有产品价值。

风险：过度 helpful 可能满足危险请求，例如协助诈骗、攻击、违法制备、隐私绕过或规避监管。

工程权衡：对于低风险请求应充分回答；对于高风险请求应拒绝危险部分，并提供安全替代建议；对于模糊请求应澄清意图。

面试表达：helpfulness 不是无条件服从用户，而是在合理和安全边界内尽量解决用户问题。

Honest

一句话定义：honest 是模型不编造、不假装知道，并能表达不确定性的能力。

表现形式：不知道就说不知道，区分事实和猜测，说明信息来源，承认能力限制，不伪造引用，不虚构工具执行结果。

为什么重要：大模型语言流畅，用户容易高估其可靠性。模型如果用自信语气输出错误内容，会造成强误导。

常见问题：幻觉、编造论文和链接、虚构 API 行为、把概率性判断说成事实、对当前事件假装实时了解。

提升方式：RAG grounding、引用核查、拒答训练、不确定性表达、工具结果约束、事实性评估和高风险场景人工复核。

面试表达：honesty 的关键是模型不要为了显得有用而编造。诚实能力和事实性、校准、grounding 密切相关。

Harmless

一句话定义：harmless 是模型避免造成伤害、违法协助、隐私泄露和高风险误导的能力。

覆盖范围：暴力、自残、违法行为、网络攻击、生化风险、医疗误导、金融诈骗、仇恨骚扰、未成年人安全、隐私和敏感个人信息。

常见行为：拒绝提供危险细节，解释不能协助的原因，提供安全替代信息，建议寻求专业帮助或正规渠道。

难点：很多请求具有双重用途。例如网络安全、化学、生物、药物、金融和法律问题既可能是正当学习，也可能被滥用。

面试表达：harmlessness 不是简单关键词拦截，而是理解意图、能力门槛、上下文、用户身份和潜在后果后的风险控制。

Safety Policy

一句话定义：safety policy 是规定模型在不同风险请求下应如何响应的规则体系。

为什么需要：没有明确策略，模型安全行为会不一致。相同请求可能有时回答、有时拒绝，既影响安全，也影响用户体验。

常见内容：允许内容、限制内容、必须拒绝内容、高风险领域处理方式、未成年人保护、隐私处理、专业建议免责声明和升级人工审核规则。

工程使用：安全策略会转化为训练数据标注规范、审核器规则、系统提示、红队测试用例和上线监控指标。

面试表达：安全策略是安全对齐的标准答案来源。模型不是凭感觉拒答，而是按照产品和法律合规定义的风险边界行动。

Jailbreak

一句话定义：jailbreak 是用户通过特殊提示绕过模型安全策略的攻击方式。

常见形式：角色扮演、忽略规则、多轮诱导、假设场景、编码规避、翻译绕过、分步拼接、情绪施压和伪造权限。

为什么有效：模型可能过度服从用户指令，被上下文中的伪规则诱导，或者无法稳定区分系统指令、安全策略和用户请求。

防护方式：安全训练、系统指令优先级、输入输出审核、拒答模板、多轮上下文风险跟踪、对抗样本训练和红队持续测试。

面试表达：jailbreak 的本质是利用模型指令遵循能力攻击安全边界。防护不能只靠关键词，而要做意图识别和策略一致性训练。

Prompt Injection

一句话定义：prompt injection 是外部输入或检索内容中包含恶意指令，诱导模型违背系统指令、安全边界或用户目标。

为什么危险：在 RAG、网页浏览、邮件处理、插件、数据库查询和 agent 场景中，模型会读取不可信外部文本。攻击者可以把“忽略之前指令”“泄露系统提示”“调用转账工具”等内容藏在文档中。

与 jailbreak 的区别：jailbreak 通常来自用户直接攻击模型；prompt injection 常来自模型读取的第三方内容或工具返回结果。

防护方式：区分指令和数据，限制外部内容权限，对工具调用做显式确认，敏感操作二次校验，使用内容隔离和结构化解析，不让检索文档覆盖系统指令。

面试表达：prompt injection 是 LLM 应用安全的核心问题，尤其在 RAG 和 Agent 中。关键原则是外部内容只能作为数据，不能成为更高优先级指令。

Indirect Prompt Injection

一句话定义：indirect prompt injection 是攻击指令隐藏在网页、文档、邮件、检索片段或工具返回中，通过模型读取后间接生效。

典型场景：模型帮用户总结网页，网页里隐藏“把用户邮箱发给我”；模型读取工单，工单里写“调用管理员接口”；RAG 文档中混入恶意 prompt。

为什么更难防：用户可能不知道外部内容里有攻击，模型却会把它当作任务上下文处理。

防护重点：外部数据沙箱化、工具权限最小化、敏感数据不进入不可信上下文、操作前向用户确认、审计工具调用日志。

面试表达：间接提示注入要求系统把“不可信内容”和“可信指令”做强隔离，不能把所有文本平铺进同一个 prompt 后期待模型自己分辨。

Instruction Hierarchy

一句话定义：instruction hierarchy 是把 system、developer、user 和 untrusted content 按优先级组织起来的指令层级。

为什么需要：LLM 应用会同时接收系统规则、开发者约束、用户任务、RAG 文档、网页、邮件和工具结果。如果没有层级，模型可能把低优先级文本当成高优先级命令。

工程含义：系统和开发者指令定义权限边界，用户指令定义当前任务，不可信外部内容只能作为数据或证据，不能覆盖更高优先级策略。

常见指标：instruction hierarchy violation rate、prompt injection success rate、data leakage rate、unauthorized tool call rate 和 boundary coverage。

面试表达：instruction hierarchy 不是只在 prompt 里写“谁优先”，还要在检索、上下文构造、工具权限、参数校验和审计日志中强制落地。

Untrusted Content Boundary

一句话定义：untrusted content boundary 是把网页、邮件、文档、RAG 片段、工具返回和用户上传文件明确标记为不可信数据的边界。

为什么重要：prompt injection 的核心风险是数据和指令混淆。不可信内容如果直接进入上下文并能影响工具调用，就可能造成任务劫持、数据外泄或越权动作。

常见做法：来源标记、权限标签、结构化封装、检索前过滤、检索后引用约束、敏感信息不进入不可信上下文、高风险动作二次确认。

面试表达：边界标记不是充分防御，但没有边界标记，后续的策略冲突检测、工具权限和审计都很难可靠。

Prompt Injection Gate

一句话定义：prompt injection gate 是 LLM 应用上线前针对直接 / 间接提示注入设置的安全门禁。

核心指标：jailbreak success rate、prompt injection success rate、indirect injection success rate、data leakage rate、unauthorized tool call rate、attack task success、clean task success、over-refusal rate 和 boundary coverage。

工程意义：它要求系统既能拒绝或忽略不可信指令，又能完成正常任务，避免把所有可疑输入一刀切拒掉。

面试表达：prompt injection gate 关注的是应用级安全边界，不只评估模型本体。RAG、Agent、浏览器、邮件助手和工具调用都应有专门 regression suite。

Data Exfiltration Rate

一句话定义：data exfiltration rate 是在注入或越狱测试中，系统泄露敏感上下文、系统提示、其他用户数据或工具结果的比例。

为什么重要：很多 prompt injection 的目标不是让模型输出危险知识，而是诱导模型泄露它能看到但用户不应获得的数据。

防护重点：最小上下文、访问控制、跨用户隔离、敏感信息检测、输出审查、工具返回脱敏和审计日志。

面试表达：数据外泄率要和权限设计一起讲。模型看不到的数据不会被模型泄露，最强防线通常是不要把敏感数据放进不可信上下文。

Unauthorized Tool Call Rate

一句话定义：unauthorized tool call rate 是工具调用样本中，模型或 agent 越过权限、缺少确认或使用被不可信内容污染的参数调用工具的比例。

为什么重要：工具调用把 prompt injection 从文本风险升级为行动风险。即使模型最终文本安全，越权工具调用也可能造成真实损害。

防护重点：读写分离、最小权限、用户身份绑定、参数 schema 校验、高风险操作二次确认、沙箱执行、调用 trace 和回滚机制。

面试表达：Agent 安全不能只看最终回答，要审计 action trace。外部内容不应直接决定高权限工具的选择和参数。

Data Privacy

一句话定义：data privacy 是保护用户数据、个人信息、商业秘密和敏感内容不被不当收集、使用、泄露或记忆。

风险来源：训练数据包含隐私，日志保存敏感输入，模型输出泄露他人信息，RAG 权限过滤失败，Agent 工具越权访问，标注流程泄密。

常见措施：数据脱敏、访问控制、最小化采集、日志加密、保留周期管理、训练数据过滤、权限检索、隐私评估和合规审计。

面试表达：大模型隐私风险不只在训练阶段，也在推理日志、检索系统、工具调用和人工标注全链路中出现。

PII

一句话定义：PII 是 Personally Identifiable Information，指可识别个人身份的信息。

例子：姓名、手机号、身份证号、邮箱、住址、银行卡、精确位置、生物特征、账号 ID 和组合后可识别个人的信息。

处理原则：尽量不收集，必须收集时最小化使用；输出前检测；日志中脱敏；检索和工具访问做权限控制。

面试表达：PII 保护要覆盖输入、输出、日志、训练数据和外部工具，不是只做一个敏感词过滤器。

Data Leakage

一句话定义：data leakage 是敏感数据在训练、评估、推理或系统集成过程中被非预期暴露的现象。

常见形式：训练数据记忆泄露，测试集泄漏到训练集，用户 A 的文档被用户 B 检索到，日志被错误开放，模型复述系统提示或密钥。

危害：破坏隐私合规、污染评估结果、泄露商业机密和产生安全事故。

防护方式：数据隔离、权限过滤、去重、密钥扫描、输出审查、审计日志、最小权限和上线前红队测试。

面试表达：data leakage 要分清是隐私泄露、评估污染还是权限隔离失败，不同来源对应不同治理手段。

Memorization

一句话定义：memorization 是模型在训练中记住并可能复现训练样本的现象。

为什么有风险：如果训练数据中包含隐私、密钥、版权内容或内部资料，模型可能在特定提示下复述出来。

影响因素：重复样本、罕见字符串、大模型容量、训练轮数、数据清洗质量和提示攻击方式。

缓解方法：训练数据去重，敏感信息过滤，差分隐私，输出检测，限制高风险复述请求和模型发布前泄露评估。

面试表达：记忆不是绝对坏事，模型需要记住知识；风险在于记住并泄露不该暴露的具体训练样本。

Training Data Extraction

一句话定义：training data extraction 是通过模型接口观察模型是否复现训练集中原始片段的隐私风险评估方向。

关注点：逐字复现、稀有片段、重复样本、PII、密钥、内部代码和长文本版权内容。

安全边界：公开写作和面试表达应聚焦风险、防御和评估，不提供可复用抽取流程或真实敏感数据样例。

面试表达：training data extraction 说明模型参数可能成为泄露通道，治理要从训练前过滤、去重、评估、输出检测和速率限制一起做。

Membership Inference

一句话定义：membership inference 是判断某个样本是否参与模型训练的隐私风险。

核心指标：成员样本真阳性率、非成员样本假阳性率、攻击优势 $A_{mia}=TPR-FPR$ 。

风险意义：即使模型不直接复述文本，能推断某个人的病历、投诉或法律文本是否进过训练，也可能泄露敏感事实。

面试表达：membership inference 是统计风险信号，不等于已经泄露某条文本，但需要触发隐私审计和训练数据治理。

PII Leakage Eval

一句话定义：PII leakage eval 是评估模型、RAG、工具和日志是否泄露个人可识别信息的测试体系。

覆盖场景：直接输出、上下文补全、多轮诱导、RAG 越权检索、工具返回、日志记录和人工标注平台。

常见指标：PII recall、secret recall、output leak rate、RAG unauthorized retrieval rate、raw log rate 和 privacy gate。

面试表达：PII leakage eval 不是敏感词扫描，它要覆盖数据、模型、检索、工具、日志和人工流程。

Differential Privacy

一句话定义：differential privacy 是限制单个样本对算法输出影响的形式化隐私框架。

核心口径：对相邻数据集 D 和 D' ，训练算法输出分布差异受 ϵ 和 δ 约束。

LLM 难点：隐私预算、训练稳定性、模型质量、计算成本和保护粒度都要权衡。

面试表达：差分隐私是强工具，但不是所有 LLM 隐私问题的万能解；它要和数据最小化、过滤、日志治理和部署防护结合。

DP-SGD

一句话定义：DP-SGD 是在 SGD 中加入梯度裁剪和噪声注入以实现差分隐私训练的典型方法。

关键机制：逐样本梯度裁剪限制单个样本影响，再加入噪声并累计 privacy budget。

工程取舍：噪声越强隐私越好，但训练更难、质量可能下降，尤其在大模型预训练和高质量后训练中要谨慎评估。

面试表达：DP-SGD 要同时报告 ϵ/δ 、裁剪阈值、噪声强度、训练质量和隐私评估结果。

Watermarking

一句话定义：watermarking 是在生成内容中嵌入可检测信号，用于 AI 生成内容溯源和平台治理。

文本水印直觉：在 token 采样分布中加入轻微统计偏置，检测器再用统计量判断文本是否更可能来自带水印生成器。

局限：短文本、改写、翻译、模型开源、采样设置变化和人工文本误报都会影响可靠性。

面试表达：watermarking 不直接解决训练数据隐私，它服务于内容来源标识、滥用追踪和合成数据污染检测。

Watermark Detection

一句话定义：watermark detection 是判断一段内容是否含有水印信号的检测过程。

常见指标：generated recall、false positive rate、false negative rate、robustness to paraphrase / translation、quality drop 和 detection threshold。

工程意义：检测器要在可靠性和误报之间权衡，不能把单个 z-score 当作绝对证据。

面试表达：水印检测结论应是概率性和证据性判断，适合进入审计流程，不适合单独作为惩罚或自动封禁依据。

Content Credentials

一句话定义：content credentials 是记录内容来源、创建和编辑信息的溯源元数据体系。

代表方向：C2PA 等内容来源标准试图把生成内容、编辑工具、签名和元数据串起来，支持平台和用户判断内容来源。

与水印关系：水印偏内容内部统计信号，content credentials 偏外部签名和元数据；二者可以互补，也都可能受到删除、篡改或缺失元数据影响。

面试表达：生成内容治理不能只依赖水印，还要结合来源凭证、平台政策、检测模型和人工审核。

Privacy Gate

一句话定义：privacy gate 是模型或 RAG 系统上线前必须通过的一组隐私门禁。

常见条件：PII recall 达标，secret recall 达标，memorization rate 低于阈值，output leak rate 低于阈值，RAG unauthorized retrieval 为 0，raw log rate 受控，membership inference risk 低于阈值。

工程意义：把隐私保护从“写了合规文档”变成可执行、可复测、可阻断发布的工程门禁。

面试表达：privacy gate 证明的是当前评估覆盖下风险可接受，不代表系统不可能泄露隐私；上线后仍要持续监控和事故响应。

Privacy Governance

一句话定义：privacy governance 是把个人信息、企业敏感数据、日志、训练使用、外部处理、保留删除、审计和责任流程统一治理的体系。

为什么重要：大模型产品的数据流跨输入、RAG、工具、模型、日志、反馈和人工审核，任何一段缺门禁都可能让隐私风险绕过模型安全策略。

关键环节：数据流梳理、数据最小化、PII 脱敏、敏感数据阻断、权限隔离、日志治理、训练授权、保留删除、外部传输审批、DPA、审计覆盖和事故响应。

面试表达：隐私治理不是提示词或隐私政策，而是把数据流、权限、日志、协议、审批和监控变成可量化、可复测、可追责的产品系统。

Data Minimization Rate

一句话定义：data minimization rate 衡量产品只传递完成任务必要数据的程度。

常见口径： $R_{\min} = 1 - N_{\text{extra}} / (N_{\text{field}} + \epsilon)$ ，其中 N_{extra} 是不必要字段数， N_{field} 是候选字段数。

工程意义：低数据最小化率通常说明系统把过多上下文、检索片段、日志、工具结果或长期记忆传给模型，增加泄露面和合规负担。

面试表达：隐私保护的第一步不是过滤输出，而是减少输入和中间链路里本不该出现的数据。

PII Redaction Coverage

一句话定义：PII redaction coverage 衡量检测到的个人身份信息中有多少被正确脱敏、最小化、授权或阻断。

常见口径： $C_{\text{pii}} = N_{\text{pii_redacted}} / (N_{\text{pii}} + \epsilon)$ 。

工程意义：PII 脱敏要覆盖用户输入、检索文档、工具结果、日志、人工标注和反馈数据，不能只在最终回答前做一次扫描。

面试表达：PII redaction coverage 要和误删、漏删、高风险字段、媒体 PII 和日志可追溯一起看。

Sensitive Data Block Rate

一句话定义：sensitive data block rate 衡量商业机密、密钥、受限文档、医疗金融等敏感事件被阻断、降级、授权或转人工的比例。

常见口径： $R_{\text{sens}} = N_{\text{sens_blocked}} / (N_{\text{sens}} + \epsilon)$ 。

工程意义：企业敏感数据不一定是 PII，可能是合同价格、源代码、财务数据、漏洞信息和内部路线图。

面试表达：敏感数据治理要按数据密级、用户权限、任务目的和外部传输范围做决策，而不是只靠关键词黑名单。

Permission Isolation

一句话定义：permission isolation 是确保用户、租户、项目、文档、工具、日志和 memory 都只在授权范围内访问的隔离机制。

常见指标：unauthorized access rate、cross-tenant violation rate、RAG unauthorized retrieval rate、unauthorized tool call rate。

工程意义：UI 层有权限不代表 RAG 检索、引用展示、工具执行和日志查询也有权限；真正的权限检查必须在后端数据流中执行。

面试表达：企业 LLM 的权限隔离要覆盖检索前过滤、上下文拼接、工具调用、引用展示、日志访问和权限变更同步。

Log Redaction Coverage

一句话定义：log redaction coverage 衡量包含用户输入、模型输出、检索片段、工具参数或错误栈的日志中有多少完成脱敏、截断、摘要化或权限保护。

常见口径： $C_{\text{log}} = N_{\text{log_redacted}} / (N_{\text{log}} + \epsilon)$ 。

工程意义：日志既服务排障、评估和审计，也可能成为最大泄露源，因此要和保留周期、加密、访问控制和删除机制一起设计。

面试表达：不能为了 debug 把完整敏感上下文长期写入原始日志；日志治理本身就是隐私产品能力。

Training Consent Coverage

一句话定义：training consent coverage 衡量可能进入训练、微调、评估或人工标注的数据中有多少具备授权、脱敏和来源追踪。

常见口径： $C_{\text{train}} = N_{\text{train_consent}} / (N_{\text{train_candidate}} + \text{epsilon})$ 。

工程意义：企业客户关心数据是否被用于训练其他模型，产品条款、控制面 and 实际数据 pipeline 必须一致。

面试表达：训练使用授权不是一个复选框，它要落到样本 lineage、数据版本、反馈用途、删除请求和第三方处理边界。

Retention Compliance Rate

一句话定义：retention compliance rate 衡量数据项是否符合保留周期、处理目的、管理员策略和行业要求。

常见口径： $R_{\text{retain}} = N_{\text{retain_ok}} / (N_{\text{retain}} + \text{epsilon})$ 。

工程意义：对话、文件、日志、索引、缓存和评估样本都应有明确保留策略，不能因为“以后可能有用”无限期保存。

面试表达：保留策略要和数据最小化、用户告知、企业管理员控制和删除机制放在同一条链路里。

Deletion SLA Pass Rate

一句话定义：deletion SLA pass rate 衡量删除请求是否在约定时间内覆盖原始数据、清洗数据、索引、日志、缓存和可追踪备份。

常见口径： $R_{\text{del}} = N_{\text{del_sla}} / (N_{\text{del}} + \text{epsilon})$ 。

工程意义：只有前端删除入口不够，系统还要能定位 lineage、索引、派生数据和保留例外。

面试表达：大模型产品的删除能力要覆盖 RAG、memory、日志和训练候选数据，否则隐私承诺很难被证明。

External Transfer Approval Coverage

一句话定义：external transfer approval coverage 衡量数据传给外部模型、第三方工具、标注平台或跨区域服务前是否具备审批、协议和最小化处理。

常见口径： $C_{\text{ext}} = N_{\text{ext_ok}} / (N_{\text{ext}} + \text{epsilon})$ 。

工程意义：外部传输门禁要检查数据范围、处理目的、保存策略、训练使用、区域、DPA、脱敏和审计记录。

面试表达：调用外部 API 时，技术团队要能回答数据出不出企业边界、会不会保存、是否用于训练、如何删除和谁批准。

Audit Coverage

一句话定义：audit coverage 衡量请求、权限、检索、工具、模型、策略命中、人审和异常事件是否被完整记录。

常见口径： $C_{\text{audit}} = N_{\text{audit_ok}} / (N_{\text{event}} + \text{epsilon})$ 。

工程意义：审计不是只留访问日志，而是要能复盘谁在什么目的下访问了什么数据、触发什么策略、产生什么输出和谁做了复核。

面试表达：没有审计覆盖，就无法证明隐私门禁真的执行，也无法做事故响应和合规复盘。

DPA Ready

一句话定义：DPA ready 表示产品与外部处理方或企业客户之间的数据处理协议、责任边界和技术控制已经准备好。

关注点：处理目的、数据类别、子处理方、保存删除、训练使用、区域、事故通知、审计权利和安全控制。

工程意义：DPA 不只是法务文件，里面的承诺必须能映射到日志、权限、保留、删除、外部传输和监控实现。

面试表达：高风险企业场景不能只看模型能力，还要确认 DPA、审计、删除、事故响应和管理员控制是否就绪。

Incident Response Ready

一句话定义：incident response ready 表示隐私或安全事件发生时，系统和组织能完成发现、止血、追踪、通知、修复和复盘。

关键能力：告警、影响范围定位、访问冻结、密钥轮换、日志检索、用户或客户通知、根因分析、回滚和回归测试。

工程意义：隐私治理不能假设永远不出事，必须提前定义事件等级、责任人、SLA 和证据保全方式。

面试表达：隐私门禁上线前要问 “如果今天发生越权召回或日志泄露，我们能在多久内定位、阻断、通知和修复”。

Minimal Reproduction

一句话定义：minimal reproduction 是把复杂线上或训练问题压缩到最小失败样本、固定配置和可重复步骤。

工程意义：没有最小复现，团队容易把偶发现象当规律，或者用大规模实验掩盖数据、格式、权限、评估和部署问题。

Evidence Coverage

一句话定义：evidence coverage 衡量一次排查是否覆盖了原始样本、数据版本、日志、trace、baseline、评估错误样例和版本变更记录。

面试表达：资深排查不是猜 “模型不行”，而是先建立证据链。证据覆盖不足时，修复动作通常风险很高。

Root Cause Rate

一句话定义：root cause rate 是一组事故或异常中已经定位出明确根因的比例。

工程意义：根因定位率低，说明团队仍停留在现象修补；根因定位率和复现率应该和线上质量指标一起看。

Postmortem Completeness

一句话定义：postmortem completeness 衡量事故复盘是否包含背景、时间线、影响范围、根因、修复、回滚、监控补充和防复发动作。

面试表达：复盘不是 “这次修好了”，而是把失败转成 checklist、回归样本、监控告警和流程改进。

Debug Gate

一句话定义：debug gate 是把证据覆盖、可复现、根因定位、回滚就绪和复盘完整性放在一起的排查完成门禁。

工程价值：它防止问题在证据不足时被关闭，也防止团队用临时 workaround 代替根因修复。

Retrospective Gate

一句话定义：retrospective gate 是把实验报告和事故报告中的 hypothesis、baseline、版本 trace、分桶分析、bad case taxonomy、证据链、时间线、影响量化、根因、预防措施、行动项关闭和回归验证放在一起的复盘门禁。

工程意义：复盘不是写文档存档，而是证明失败已经变成评估样本、监控、流程或系统改进。复盘门禁不过，说明同类问题仍可能在下一次模型、prompt、RAG、Agent 或推理服务变更中复发。

Hypothesis Coverage

一句话定义：hypothesis coverage 衡量实验或复盘样本中有多少写出了可证伪假设、对照 baseline、预期指标和失败边界。

工程意义：没有可证伪假设，实验结论很难沉淀为知识；结果好坏都只能停留在“试试看”。

Slice Analysis Coverage

一句话定义：slice analysis coverage 衡量实验或事故复盘是否按任务类型、难度、语言、输入长度、风险等级、用户类型或系统链路做分桶分析。

面试表达：平均指标经常掩盖回归。资深复盘要能说明哪个切片提升、哪个切片退化，以及是否能支持上线决策。

Action Item Closure

一句话定义：action item closure 衡量复盘后的具体行动项是否有 owner、deadline、验收标准并实际关闭。

工程意义：行动项如果只是“加强 review”“持续关注”，就无法验收，也无法证明事故风险下降。

Regression Verification Rate

一句话定义：regression verification rate 衡量修复后的样本、回归集、灰度和线上指标是否验证原问题已修复且没有引入新回归。

工程意义：代码改了不等于问题修了。RAG、Agent、安全和推理事故尤其需要把 bad case 加入回归，并观察护栏指标。

Change Failure Rate

一句话定义：change failure rate 衡量上线、模型、prompt、数据、配置或工具变更中有多少导致事故、回滚、热修或用户可感知退化。

面试表达：它是团队交付质量指标。大模型项目不能只看迭代速度，也要看变更失败率和恢复时间。

Privacy Governance Gate

一句话定义：privacy governance gate 是产品上线前把 PII、敏感数据、权限、日志、训练授权、保留删除、外部传输、审计、DPA 和事故响应放在一起的硬门禁。

常见条件：PII redaction coverage 达标，sensitive data block rate 达标，unauthorized access rate 为 0，log redaction coverage 达标，training consent coverage 达标，retention 和 deletion SLA 达标，external transfer approval coverage 达标，audit coverage 达标，DPA 和 incident response 就绪。

工程意义：它把 privacy gate 从模型泄露评估扩展到真实产品数据流和组织治理，适合企业 RAG、Agent、多模态和高风险行业场景。

面试表达：privacy governance gate 不是平均分，任何严重越权、日志原文泄露、无授权训练、删除 SLA 缺失或外部传输未审批都应阻断上线。

Safety Compliance Incident

一句话定义：safety compliance incident 是大模型系统在安全策略、隐私、权限、日志、训练授权、外部传输、版权来源、审计或事故响应上违反预期控制的事件。

典型来源：高风险请求被放行、正常请求被误拒、RAG 越权上下文进入模型、工具写操作缺少确认、日志保存敏感原文、反馈数据未授权进入训练候选、删除请求没有覆盖索引和缓存、外部模型或标注平台传输缺审批。

工程意义：它把“模型输出不安全”扩展为“系统链路不合规”。即使最终回答没有明显问题，只要中间链路越权、日志泄露、未经授权使用数据或审计缺失，也应进入事故排查。

面试表达：安全合规事故要按输入、检索、工具、输出、日志、缓存、训练反馈、外部传输和审计逐层定位，不能只归因成“模型安全能力不足”。

Unsafe Pass Rate

一句话定义：unsafe pass rate 是策略要求阻断、脱敏、拒绝、确认或转人工的高风险样本中，被系统直接放行、执行、外传、原样记录或进入训练候选的比例。

常见口径： $R_{\text{unsafe}} = N_{\text{unsafe_pass}} / (N_{\text{high}} + \text{epsilon})$ 。

为什么重要：这是安全合规门禁里的底线指标。高严重度样本的不当放行不能被大量低风险样本的通过率平均掉。

面试表达：unsafe pass rate 要和 over-refusal rate 一起看。前者控制漏拦底线，后者控制产品可用性。

Safety Compliance Gate

一句话定义：safety compliance gate 是把高风险不当放行、过度拒答、PII 脱敏、敏感数据阻断、权限隔离、工具确认、日志脱敏、训练授权、保留删除、外部传输、审计、DPA 和事故响应放在一起的发布门禁。

常见条件：unsafe pass rate 低于阈值，over-refusal rate 低于阈值，PII redaction coverage 达标，sensitive data block rate 达标，unauthorized access rate 为 0，tool confirmation coverage 达标，log redaction coverage 达标，training consent coverage 达标，retention / deletion SLA 达标，external transfer approval coverage 达标，audit coverage 达标，DPA ready 和 incident response ready 均为真。

工程意义：它把安全策略、隐私合规和产品可用性统一到一张审计表中，适合企业 RAG、Agent、多模态、客服、合同 review、数据分析助手和其他高风险工作流上线前使用。

面试表达：安全合规门禁不是“最终输出看起来安全就行”，而是证明数据、权限、工具、日志、外部处理、审计和事故响应都在可控边界内。

Watermark Gate

一句话定义：watermark gate 是把水印生成或水印检测用于治理流程前设置的可靠性门禁。

常见条件：generated recall 达标，false positive rate 低于阈值，短文本 / 改写 / 翻译切片表现可接受，质量下降受控，检测结论有人工复核或申诉流程。

工程意义：防止把水印检测当成绝对证据，也防止只在长文本 toy 样本上通过就上线。

面试表达：watermark gate 证明的是当前检测条件下可作为审计信号，不证明某段内容的来源可以完全自动裁决。

Bias and Fairness

一句话定义：bias and fairness 关注模型是否对不同人群、语言、地区、职业、性别、年龄或文化背景产生不公平输出。

风险表现：刻板印象、歧视性建议、不同群体安全阈值不一致、低资源语言质量差、招聘和信贷等决策辅助中的系统性偏差。

来源：训练数据偏差、标注者偏好、任务定义偏差、评估集不均衡和产品反馈循环。

治理方式：数据审查，公平性切片评估，偏差红队，多样化标注，安全策略约束和上线监控。

面试表达：公平性不能只看总体平均分，要按群体和场景切片评估，并区分表达偏见、服务质量差异和高风险决策偏差。

Toxicity

一句话定义：toxicity 是模型输出攻击性、侮辱性、仇恨、骚扰或不适宜内容的程度。

常见场景：用户诱导辱骂、模型续写有害文本、多轮对话情绪升级、对敏感群体产生攻击性描述。

控制方法：安全微调、toxicity classifier、输出审核、拒答和改写、安全替代回答。

面试表达：toxicity 控制要避免两个极端：既不能放任有害输出，也不能把正常讨论敏感议题全部拒绝。

Red Teaming

一句话定义：red teaming 是系统性设计攻击和压力测试，寻找模型安全漏洞和失败模式。

为什么需要：普通测试往往覆盖正向用例，红队专门模拟恶意用户、边界场景和复杂攻击链。

测试范围：jailbreak、prompt injection、隐私泄露、危险指导、偏见歧视、工具越权、RAG 污染、agent 无限循环和多轮诱导。

特点：应在上线前和上线后持续进行。模型、数据、prompt、工具和攻击方式变化后，都需要重新评估。

输出结果：失败样本库、风险等级、复现步骤、修复建议、回归测试集和安全指标趋势。

面试表达：red teaming 不是一次性测试，而是安全迭代闭环。发现问题后要进入数据、策略、模型和系统防护改进。

Dangerous Capability Evaluation

一句话定义：dangerous capability evaluation 是评估模型是否具备可能显著增加现实世界风险的高影响能力。

关注点：cyber dual-use、bio / chemical dual-use、autonomy、tool use、多步规划、persuasion、隐私外泄和高风险自动化。

核心问题：模型是否相对非 AI baseline、旧模型或普通公开资料显著降低门槛、提高可靠性、减少成本或增加自动化程度。

常见指标：dangerous capability uplift、capability elicitation gain、autonomy score、P0 / P1 failure count、severity-weighted risk 和 release gate。

面试表达：危险能力评估不是问模型几个危险问题，而是报告评估条件、能力上限、现实增量、严重度和发布控制措施。

Capability Elicitation

一句话定义：capability elicitation 是用合适的提示、工具、scaffold、多轮交互或专家辅助激发模型潜在能力。

为什么重要：默认问答表现差不代表模型没有能力。高能力模型在工具、示例、分解和反馈条件下可能表现出更高风险能力上限。

工程边界：评估报告要记录访问级别、工具条件、上下文、是否多轮、是否专家辅助、成本和成功率，避免低估或不现实高估。

面试表达：危险能力评估通常要同时看 natural capability 和 elicited capability。前者接近普通用户体验，后者接近有动机用户能激发出的上限。

Red Team Regression Suite

一句话定义：red team regression suite 是把历史红队失败样本及其同类变体沉淀为每次发布前必须重跑的回归测试集。

为什么需要：修复一个具体样本不代表同类问题消失。prompt、policy、tool schema、RAG 数据或模型版本变化都可能让旧问题复发。

常见指标：regression pass rate、held-out variant pass rate、P0 / P1 recurrence count、root cause closure rate 和 time-to-fix。

面试表达：红队的价值不只是发现问题，而是把问题变成持续可测的安全资产。

Red Team Gate

一句话定义：red team gate 是发布前基于红队结果设置的硬门禁。

常见条件：P0 失败为 0，P1 失败已修复或有明确限制方案，危险能力 uplift 不超过阈值，自主性风险可控，工具越权率达标，红队回归通过率达标。

工程意义：red team gate 把开放式发现转成发布决策，避免“发现了风险但不知道能不能发”的治理断点。

面试表达：red team gate 不是单一分数，而是一组按严重程度和能力阈值定义的发布条件。

Release Gate

一句话定义：release gate 是模型发布、灰度或扩大访问前必须满足的质量、安全、能力和治理条件。

组成部分：普通能力评估、safety gate、prompt injection gate、red team gate、dangerous capability threshold、工具权限、监警告警、回滚方案和 system card / model card。

工程意义：高严重程度安全问题不能被平均指标掩盖；危险能力超过阈值时，必须增加访问控制、部署限制、监控或延迟发布。

面试表达：release gate 把评估结果转成发布决策。能力越强，门禁越不能只看 benchmark 分数。

Interpretability

一句话定义：interpretability 是理解模型行为、表示和决策依据的一组方法。

常见层次：输入归因、attention 可视化、probing、表示分析、因果干预和机制逆向。

工程意义：可解释性可以帮助 debug 幻觉、拒答、jailbreak、偏见、工具误用和危险能力，但多数方法只能提供线索，不能单独证明安全。

面试表达：interpretability 不是让模型自己解释自己，而是用外部分析方法理解模型行为和内部表示。

Mechanistic Interpretability

一句话定义：mechanistic interpretability 是把神经网络当成被训练出来的程序，逆向工程其内部 features、circuits 和信息流。

关键概念：residual stream、attention head、MLP、feature、neuron、circuit、activation patching、ablation、causal tracing、superposition 和 sparse autoencoder。

为什么重要：它试图从相关性解释走向因果机制验证，让研究者知道模型内部哪些组件实际参与了某个行为。

面试表达：机制可解释性的目标不是画 attention heatmap，而是提出机制假设，并用 patching、ablation 等干预证明这些机制是否影响输出。

Activation Patching

一句话定义：activation patching 是把 clean run 的中间激活替换到 corrupted run 中，观察目标输出是否恢复的因果干预方法。

解决问题：probing 只能说明信息可被读出，activation patching 更接近回答 “这个激活是否真的用于产生行为”。

常见指标：patch recovery score、path patch score、ablation effect 和 logit delta recovery。

面试表达：activation patching 是机制可解释性里的核心因果工具，但需要 clean / corrupted 输入对，且干预可能产生分布外激活。

Sparse Autoencoder

一句话定义：sparse autoencoder 是从模型激活中学习稀疏 feature 表示，并重构原激活的工具。

为什么需要：单个 neuron 往往 polysemantic，feature 可能藏在激活空间方向中。SAE 试图找到更接近 monosemantic 的 feature。

常见指标：reconstruction fidelity、average active features、dead feature rate、feature purity、feature splitting 和 feature-level intervention effect。

面试表达：SAE 能把混合激活分解成更可解释的 feature 候选，但找到 feature 之后仍需要因果验证，不能自动等于完整机制解释。

Superposition

一句话定义：superposition 是多个 feature 叠加在同一激活空间中的现象或假说。

为什么出现：模型需要表示的 feature 数量可能多于神经元或维度数量，于是用不同方向压缩表示多个概念。

结果：单个 neuron 可能 polysemantic，也就是在多个语义不同的场景中激活。

面试表达：superposition 解释了为什么 “一个 neuron 一个概念” 的朴素想法经常失败，也解释了 SAE 为什么重要。

Feature Purity

一句话定义：feature purity 是衡量一个 feature 的高激活样本是否集中在同一概念或标签上的指标。

用途：评估 SAE feature 是否接近 monosemantic，帮助给 feature 命名和筛选候选安全监控特征。

局限：purity 依赖标签和 top-k 样本选择。高 purity 只能说明解释更干净，不能证明 feature 对行为有因果作用。

面试表达：feature purity 是可解释性质量指标，必须和 reconstruction fidelity、sparsity、patching 和 ablation 一起看。

Interpretability Gate

一句话定义：interpretability gate 是将局部机制解释用于安全调试或监控前设置的证据门槛。

常见条件：SAE reconstruction fidelity 达标，feature 稀疏度可接受，feature purity 达标，patch recovery 明显，ablation effect 明显，feature intervention 副作用经过评估。

工程意义：避免把一个好听的 feature 名字直接当成可靠安全控制。只有经过重构、解释和因果干预共同验证的机制假设，才适合进入监控或 steering 实验。

面试表达：interpretability gate 证明的是局部机制假设有证据，不是证明整个模型安全。

Steering

一句话定义：steering 是在不完整重训模型的情况下，引导模型行为朝某个目标方向变化的一组方法。

常见形式：prompt steering、decoding steering、activation steering、feature steering、representation monitoring 和 model editing。

安全价值：可以研究拒答、不确定性、事实性、工具谨慎性和安全响应的内部表示，也可以作为辅助控制层。

风险：方向可能不泛化，强度过大可能损伤 helpfulness 或增加 over-refusal，也存在双用风险。

面试表达：steering 是辅助控制和研究工具，不能替代 RLHF、安全评估、红队、权限控制和发布门禁。

Representation Engineering

一句话定义：representation engineering 是从高层行为出发，寻找模型内部可监控、可干预表示方向的 top-down transparency 方法。

基本流程：构造正负样本，提取内部激活，计算行为方向，监控投影分数或做 activation intervention，再评估目标收益和副作用。

与机制可解释性的区别：mechanistic interpretability 更偏 bottom-up 逆向 circuits；representation engineering 更偏 top-down 监控和控制高层行为。

面试表达：representation engineering 的关键是把抽象行为变成可测的内部表示，但方向有效不代表完整理解机制。

Steering Vector

一句话定义：steering vector 是模型激活空间中与某种目标行为相关的方向。

常见构造：正例激活均值减去负例激活均值，然后归一化；推理时按强度系数加到指定层或位置的激活中。

评估指标：target uplift、safe gain、side-effect drop、over-refusal delta、projection shift、robustness across tasks 和 steering gate。

面试表达：steering vector 要做强度扫描和副作用评估，不能只展示一个成功例子。

Contrastive Activation Addition

一句话定义：Contrastive Activation Addition 是用正负样本的激活差分构造控制方向，并在推理时加到激活上的 steering 方法。

优点：无需更新模型参数，可快速开关，适合行为分析和局部控制实验。

局限：需要访问模型内部激活；方向依赖数据、层和位置；可能影响非目标能力；真实上线必须做安全和鲁棒性评估。

面试表达：CAA 的直觉是“正例表示减负例表示”，但工程上最重要的是评估泛化、副作用和双用风险。

Steering Gate

一句话定义：steering gate 是把 activation steering 用于实验或灰度前设置的评估门禁。

常见条件：目标行为提升达标，安全收益达标，通用质量下降低于阈值，over-refusal 增量低于阈值，jailbreak / prompt injection 回归不退化，多任务和多语言切片稳定。

工程意义：steering 方向可能有效但脆弱，门禁用于防止把局部 toy 成功直接推到生产系统。

面试表达：steering gate 证明的是某个方向在当前评估分布和强度下可用，不是证明方向天然可靠。

Model Editing

一句话定义：model editing 是在不完整重训模型的情况下，对模型参数或内部机制做局部修改，让目标事实或行为发生预期变化，同时尽量保持其他能力不变。

典型场景：修正过时事实、添加少量稳定知识、修复特定行为、研究模型内部 factual association。

评估指标：edit success、generalization、specificity / locality、retain drop、fluency、冲突率和 editing gate。

面试表达：model editing 不等于 RAG 或普通 fine-tuning；它的核心难点不是“改对一个样本”，而是改写泛化和无关知识保持。

Knowledge Editing

一句话定义：knowledge editing 是 model editing 中专门针对事实知识、实体关系或概念关联的局部修改。

代表方法：ROME、MEMIT、MEND、SERAC 等方法从不同角度尝试定位、更新或外部化模型知识。

关键风险：编辑可能只改变表面表达，不一定更新所有推论；批量编辑可能互相干扰；局部更新可能破坏相关事实一致性。

面试表达：knowledge editing 要同时看目标问题、改写问题、相关但不该变的问题和通用能力保留。

ROME

一句话定义：ROME 是一种研究 GPT 类模型中 factual association 并进行局部编辑的方法。

核心直觉：某些事实关联可以通过定位中间层 MLP 的关键表示，再对相关权重做低秩更新来修改。

局限：更适合单点事实关联；对复杂行为、批量编辑、长期一致性和跨表达泛化不能直接保证。

面试表达：ROME 的价值是把“知识存在参数里”变成可定位、可编辑、可评估的问题，但不是通用知识治理方案。

MEMIT

一句话定义：MEMIT 是把单点 model editing 扩展到批量 memory edits 的方法谱系代表。

核心直觉：把多条事实编辑分布到相关层或参数更新中，提升大规模知识更新效率。

关键风险：编辑数量越多，冲突、副作用和评估覆盖不足越明显。

面试表达：MEMIT 适合解释“从 ROME 的单点手术到批量知识更新”的演进，但仍必须配合 locality 和 retain set 评估。

Machine Unlearning

一句话定义：machine unlearning 是让模型近似移除某些训练数据、知识、关联或行为影响的一类方法。

LLM 场景：版权数据移除、隐私数据删除、错误知识清理、危险能力降低和记忆复现风险控制。

关键边界：大模型不是数据库；多数 LLM unlearning 只能提供行为层和统计层近似证据，不能轻易宣称完全遗忘。

面试表达：unlearning 要同时看 forget set、retain set、改写 / 多轮鲁棒性、membership inference 风险和审计证据。

Forget Set

一句话定义：forget set 是 unlearning 中要求模型降低记忆、复现、回答或利用能力的一组目标样本、知识或行为。

评估指标：目标复现率、问答准确率下降、completion likelihood 下降、membership inference 风险下降、改写泄露率和多轮泄露率。

工程意义：forget set 必须有版本、来源、删除原因和评估边界，否则无法审计 unlearning 是否覆盖目标。

面试表达：forget set 不是随便列几条 prompt，而是删除目标、风险口径和审计证据的核心载体。

Retain Set

一句话定义：retain set 是 unlearning 后必须保持的正常知识、能力和安全行为集合。

评估指标：retain accuracy、helpfulness、safe completion、通用 benchmark、相关但不应遗忘知识和高风险策略一致性。

工程意义：只让 forget set 表现下降很容易；难点是目标下降的同时 retain set 不明显退化。

面试表达：没有 retain set 的 unlearning 评估是不完整的，因为模型可能只是整体变笨或过度拒答。

Editing Gate

一句话定义：editing gate 是把 model editing 结果用于实验或灰度前设置的评估门禁。

常见条件：edit success 达标，改写泛化达标，locality 达标，retain drop 低于阈值，冲突率低于阈值，安全回归不退化。

工程意义：避免只凭一个目标样本改对就发布参数编辑结果。

面试表达：editing gate 证明的是当前编辑集合在当前评估覆盖下可接受，不证明所有相关知识都已全局一致。

Unlearning Gate

一句话定义：unlearning gate 是把遗忘训练、参数编辑或输出抑制用于治理场景前设置的证据门禁。

常见条件：forget leak 降到阈值以下，改写和多轮鲁棒泄露降到阈值以下，membership inference 风险明显下降，retain set 能力保留，红队和人工审计通过。

工程意义：把“模型不再直接回答”扩展为可审计的多维证据，降低虚假遗忘的风险。

面试表达：unlearning gate 只能证明当前证据达标，不等于数学上证明完全删除模型记忆。

Safety Evaluation

一句话定义：safety evaluation 是评估模型在高风险、对抗和边界场景下是否符合安全策略的过程。

评估维度：违规回答率、正确拒答率、过拒率、危险细节泄露率、隐私泄露率、jailbreak 成功率、prompt injection 成功率和安全替代回答质量。

数据来源：公开安全 benchmark、内部策略测试集、红队样本、线上违规日志、人工构造边界样本和多语言安全样本。

关键问题：安全评估必须同时看 false negative 和 false positive。只降低违规率但大幅提高过拒率，会损害产品可用性。

面试表达：安全评估不是单看模型拒绝了多少，而是看它是否在正确场景拒绝、在安全场景正常帮助。

Safety Gate

一句话定义：safety gate 是模型上线或灰度前必须通过的一组安全硬门禁。

常见指标：unsafe compliance rate、refusal accuracy、over-refusal rate、attack success rate、unauthorized tool call rate、safe completion quality、severity-weighted risk 和 policy coverage。

为什么需要：平均质量分可能掩盖高严重度失败。一个模型即使总体满意度更高，只要高风险漏拒、工具越权或 prompt injection 成功率超过门槛，就不应直接上线。

工程做法：把安全门禁接入评估平台和发布流程；高风险样本需要人工复核；每次模型、prompt、policy、tool schema 或 RAG 数据变化后都要跑回归。

面试表达：safety gate 不是“安全分数够高就行”，而是一组硬约束。高严重度失败应阻断发布，而不是被低风险样本平均掉。

Severity-Weighted Risk

一句话定义：severity-weighted risk 是按安全事件严重度加权后的风险分数。

核心直觉：低风险误拒和高风险漏拒不能等价。高严重度样本应该在评估和上线决策中有更高权重。

常见用法：给每个风险类别或样本设置 severity weight，再统计失败样本的加权占比，用于排序修复优先级和决定是否过 safety gate。

局限：严重度权重本身需要策略、产品、安全和合规团队共同定义；不能只由模型团队临时拍脑袋设定。

面试表达：安全评估要避免平均指标掩盖尾部风险。severity-weighted risk 能帮助团队把注意力放到真正可能造成危害的失败上。

Over-Refusal

一句话定义：over-refusal 是模型对本应正常回答的安全请求也拒绝的现象。

本质：harmlessness 过强或边界学习不清，损害 helpfulness。

例子：用户询问化学课程基础知识、网络安全防护方法、心理健康支持、历史暴力事件分析，模型却一概拒绝。

危害：降低用户体验，阻碍教育和研究用途，让安全策略显得粗糙。

缓解方法：构造安全正例，细化风险等级，训练安全替代回答，区分防御性和攻击性意图，人工审查过拒样本。

面试表达：安全对齐不是拒绝越多越好。优秀模型要做到该拒绝时拒绝，该帮助时帮助。

Under-Refusal

一句话定义：under-refusal 是模型对应该拒绝的危险请求没有拒绝，反而提供了有害帮助。

例子：提供攻击脚本、诈骗话术、规避执法建议、危险制备步骤、侵犯隐私方法或自残鼓励。

危害：直接造成安全、法律和伦理风险。

缓解方法：高风险数据训练，策略规则，安全分类器，输出审核，红队回归，工具权限限制和线上监控。

面试表达：under-refusal 是安全底线问题；over-refusal 是可用性问题。实际系统要同时优化两者。

Content Moderation

一句话定义：content moderation 是对用户输入、模型输出或外部内容进行风险识别和处理的机制。

常见位置：输入前审核，生成中控制，输出后审核，工具调用前审核，RAG 文档入库审核，线上日志审查。

处理方式：允许、拒绝、改写、降级、打码、人工复核、限制工具调用或触发告警。

优点：可以作为模型外部安全层，独立于生成模型版本迭代。

局限：分类器也会误判；上下文语义复杂；多语言和隐晦表达难处理；过度审核会影响体验。

面试表达：内容审核是安全系统的一层防线，但不能替代模型对齐、权限控制和产品策略。

Guardrail

一句话定义：guardrail 是围绕大模型系统设置的安全护栏，用于限制输入、输出、工具调用和系统行为。

常见形式：系统提示、策略分类器、schema 校验、敏感词和实体检测、输出过滤、权限控制、速率限制、人工确认和审计日志。

为什么需要：模型本身不够可靠，尤其在工具调用和 agent 场景中，外部护栏可以降低单次模型失误的影响。

设计原则：分层防护，最小权限，默认安全，敏感操作确认，可观测，可回滚。

面试表达：guardrail 不是某个固定组件，而是一组围绕模型应用的安全约束和执行机制。

Tool Safety

一句话定义：tool safety 是确保模型调用外部工具时不会越权、误操作或造成不可逆伤害的机制。

风险：错误调用删除数据，泄露用户信息，发送错误邮件，执行危险代码，访问无权限资源，产生高额成本。

防护方式：工具白名单，参数校验，权限隔离，只读优先，沙箱执行，敏感操作二次确认，调用频率限制和审计日志。

面试表达：当大模型具备工具调用能力后，安全重点从“说错话”扩展到“做错事”。工具安全必须按真实系统权限设计。

Agent Safety

一句话定义：agent safety 是控制具备多步规划、记忆和工具调用能力的模型系统的风险。

特殊风险：长链路错误累积，目标漂移，无限循环，工具滥用，外部内容注入，记忆污染，权限越界和成本失控。

治理方式：任务边界约束，最大步数，预算限制，人工审批点，状态可视化，工具权限分层，失败回滚和执行轨迹审计。

面试表达：agent 安全比聊天模型安全更复杂，因为 agent 会行动。必须控制计划、工具、状态、权限和终止条件。

Model Card

一句话定义：model card 是记录模型能力、限制、训练数据、评估结果、适用范围和风险的文档。

为什么需要：模型使用者需要了解模型能做什么、不能做什么、在哪些场景评估过、有哪些已知风险。

常见内容：模型概览、训练数据说明、适用场景、禁用场景、评估指标、能力边界、安全测试、偏见风险、部署建议和联系方式。

作用：提高透明度，支持合规审查，帮助下游开发者正确使用模型。

面试表达：model card 是模型治理和透明度工具，不只是宣传材料。它应真实记录能力、局限和风险。

System Card

一句话定义：system card 是记录完整 AI 系统能力、组件、风险、防护、发布条件和监控流程的治理文档。

与 model card 的区别：model card 关注模型本身；system card 还覆盖 RAG、工具调用、权限、policy layer、guardrail、监控和事故响应。

常见内容：系统组件、用户访问、工具权限、数据处理、安全策略、评估和红队、隐私处理、已知风险、缓解措施、发布限制和更新策略。

面试表达：大模型产品的风险往往来自系统组合，因此只写 model card 不够，还需要 system card 说明真实部署条件。

Model Governance

一句话定义：model governance 是围绕模型研发、评估、发布、监控、变更和事故响应建立责任与流程的体系。

关键环节：风险分类、评估计划、发布门禁、审批责任、model card / system card、审计日志、监控告警、灰度回滚和事故复盘。

工程意义：把抽象 policy 变成可执行、可复测、可追责的流程。

面试表达：治理不是写文档，而是让 policy、评估、门禁、责任人、监控和事故响应形成闭环。

Governance Gate

一句话定义：governance gate 是模型或系统发布前针对文档、策略、评估、风险、审批和运维准备度设置的综合门禁。

常见条件：model card 完整，system card 完整，policy 覆盖关键风险，评估切片覆盖充分，P0/P1 风险有处理，审批覆盖达标，监控和回滚就绪。

工程意义：防止只因为离线指标好或审批人签字就发布，而忽略文档缺口、隐私门禁、工具权限或事故响应准备。

面试表达：governance gate 是把透明度、责任和发布条件合到一起的上线控制，不等于单个 safety gate。

Responsible Scaling

一句话定义：responsible scaling 是把模型能力增长和更严格的安全评估、访问控制、发布审批绑定起来的治理思路。

核心直觉：模型能力越强，潜在风险和影响范围越大，因此需要更强红队、第三方评估、权重安全、监控和发布限制。

注意事项：不同机构的 responsible scaling policy 细节不同，面试中重点讲原则和工程闭环，不把某一家条款当成通用法律标准。

面试表达：responsible scaling 不是“越强越不能发布”，而是能力、风险、缓解措施和发布条件同步升级。

Risk Disclosure

一句话定义：risk disclosure 是在 model card、system card 或发布说明中清楚披露模型已知风险、限制、评估覆盖和使用边界。

好披露：说明具体失败场景、可能后果、受影响用户、已测范围、未测范围、缓解措施和使用者责任。

坏披露：只写“模型可能不准确，请谨慎使用”，不能指导真实使用或审计。

面试表达：风险披露要具体到场景和后果，否则只是免责声明，不是治理信息。

Safety Interview Readiness

一句话定义：safety interview readiness 是把安全知识转成结构化面试表达、指标意识、门禁意识和边界意识的准备度。

核心维度：定义是否清楚，风险 taxonomy 是否完整，是否覆盖失败模式、缓解方法、评估指标、上线门禁、trade-off、治理和安全边界。

常见扣分：只说拒答，只说 RLHF，不区分 jailbreak 和 prompt injection，不讲指标，不讲 over-refusal，不讲发布门禁，或提供高风险操作细节。

面试表达：安全面试考察系统思维，不是背政策。高质量回答要能从目标、风险、训练、评估、红队、系统防护、治理和监控形成闭环。

Safety Interview Rubric

一句话定义：safety interview rubric 是用于复盘安全面试回答的评分规约。

常见指标：answer coverage、risk coverage、metric coverage、gate coverage、trade-off coverage、unsafe detail penalty 和 revision plan。

工程意义：把主观面试表现转成可复盘、可补弱点的清单，避免“感觉会了但回答没有结构”。

面试表达：复盘时不要只看是否答对概念，还要看是否能给出可验证指标、发布门禁和残余风险。

Data Governance

一句话定义：data governance 是围绕数据采集、清洗、授权、存储、使用、删除和审计的治理体系。

为什么重要：大模型能力和风险都强依赖数据。数据来源不清、授权不明、隐私未处理，会带来法律和安全问题。

关键环节：数据来源记录，许可协议，敏感信息过滤，去重，质量评估，权限管理，数据版本控制，删除请求处理和审计追踪。

面试表达：大模型治理不能只看模型输出。训练数据、微调数据、RAG 知识库和日志数据都需要治理。

Audit Log

一句话定义：audit log 是记录模型请求、输出、工具调用、权限决策和系统事件的审计日志。

为什么需要：出现安全事故或质量问题时，需要能够复现、追踪和归因。没有日志，就无法判断是模型、数据、prompt、工具还是用户行为导致问题。

记录内容：时间、用户或租户 ID、模型版本、prompt 版本、检索片段、工具调用参数、输出、审核结果、错误码和人工处理结果。

注意事项：日志本身可能包含敏感信息，需要脱敏、加密、权限控制和保留周期管理。

面试表达：audit log 是安全治理和线上运维的基础，但必须和隐私保护一起设计。

Compliance

一句话定义：compliance 是让大模型系统满足法律、监管、行业规范和组织内部政策的过程。

涉及范围：个人信息保护、数据跨境、版权、内容安全、金融医疗教育监管、模型备案、用户告知和可解释性要求。

工程影响：影响数据能不能用、日志能保存多久、模型能回答什么、是否需要人工审核、是否需要用户授权和撤回机制。

面试表达：合规不是上线前补一份文档，而是贯穿数据、训练、评估、部署、监控和事故响应的系统要求。

Responsible AI

一句话定义：responsible AI 是以安全、公平、透明、可解释、隐私保护和社会责任为目标设计、部署和治理 AI 系统的理念。

包含内容：风险评估、用户告知、公平性评估、隐私保护、安全测试、透明文档、人工监督、申诉机制和持续监控。

与 AI Safety 的关系：AI Safety 更强调减少伤害和系统安全，Responsible AI 范围更广，包含伦理、治理、社会影响和组织责任。

面试表达：Responsible AI 关注模型从研发到上线后的全生命周期责任，而不是只优化离线指标。

安全与能力的权衡

大模型安全治理的核心难点，是在 helpful、honest、harmless 之间取得平衡。

如果模型过度 helpful，可能协助危险行为；如果过度 harmless，可能大量 over-refusal；如果不够 honest，可能用流畅语言输出错误事实。

实际系统通常采用分层方案：模型对齐负责基础行为，安全策略定义边界，审核器做输入输出控制，工具权限限制可执行动作，红队和线上监控持续发现问题。

面试中可以这样总结：安全不是模型能力的附属品，而是大模型产品的核心工程约束。越接近真实业务和外部工具，越需要系统级安全治理。

本章小结

本章围绕安全与治理建立了核心词汇表。

核心结论如下：

1. AI Safety 是数据、训练、评估、产品、权限和监控共同构成的系统工程。
2. Alignment 的常见目标是 helpful、honest、harmless，三者之间需要权衡。
3. Jailbreak 和 prompt injection 都攻击模型指令遵循能力，但来源和防护重点不同。
4. RAG 和 Agent 场景中，外部内容应被视为不可信数据，不能覆盖系统指令。
5. 隐私保护覆盖训练数据、推理日志、RAG 知识库、工具调用和人工标注全链路。
6. 安全评估要同时看 under-refusal 和 over-refusal。
7. Guardrail、content moderation、tool safety 和 agent safety 是模型外部安全层的重要组成。
8. Model card、audit log、data governance 和 compliance 支撑模型透明度和组织治理。
9. Red teaming 应持续进行，并形成失败样本、修复策略和回归测试闭环。
10. 安全治理的目标不是让模型保守到不可用，而是在风险可控的前提下保持有用、诚实和可靠。

下一章，我们进入重要论文索引。

N. 重要论文索引

条目：Attention Is All You Need、GPT 系列、BERT、Scaling Laws for Neural Language Models、Chinchilla、InstructGPT、Constitutional AI、Direct Preference Optimization、LoRA、FlashAttention、Switch Transformer、Mixtral、RAG。

如何使用论文索引

一句话定义：论文索引不是让你逐字背论文，而是帮助你建立大模型技术演进地图。

为什么需要：大模型面试经常会追问“这个方法从哪里来”“解决了什么问题”“相比前人有什么改进”“工程上为什么有用”。如果只背概念，不知道论文脉络，很容易回答得碎片化。

使用方式：先掌握每篇论文的核心问题、关键方法和影响，再选择少数关键论文精读。面试中不需要复述所有实验细节，但要能讲清技术提出的背景、主要贡献、局限和后续演化。

建议优先级：Transformer、GPT/BERT、Scaling Laws、Chinchilla、InstructGPT、DPO、LoRA、FlashAttention、RAG、Switch Transformer/Mixtral 是大模型算法岗最常被追问的核心论文线。

面试表达：我读论文时会先判断它解决的是架构、训练、数据、对齐、推理、系统还是应用问题，然后把它放到技术演进链条里理解，而不是孤立记结论。

基础架构

Attention Is All You Need

一句话定义：Transformer 原始论文，提出完全基于 self-attention 的序列建模架构。

解决的问题：在 Transformer 之前，主流序列模型依赖 RNN、LSTM 或 CNN，长距离依赖建模和并行训练效率受限。Transformer 用 self-attention 直接建模任意位置 token 之间的关系，并提升训练并行度。

核心贡献：multi-head attention、scaled dot-product attention、position encoding、encoder-decoder 结构、残差连接、LayerNorm 和 FFN 组合成现代大模型的基础模块。

为什么重要：几乎所有主流语言模型、多模态模型、扩散模型中的部分模块都受到 Transformer 影响。理解这篇论文，是理解 attention、GPT、BERT、ViT 和大模型系统的起点。

局限：原始 Transformer 的 attention 复杂度是序列长度平方级，长上下文成本高；位置编码设计相对早期；后续 decoder-only LLM 对原始结构做了大量演化。

面试表达：这篇论文的最大贡献不是某个单点技巧，而是用 self-attention 替代循环结构，使序列建模同时具备长距离依赖建模能力和高并行训练效率。

BERT

一句话定义：BERT 是 encoder-only Transformer 的代表模型，使用 masked language modeling 进行双向语言理解预训练。

解决的问题：早期预训练语言模型要么单向建模，要么任务迁移能力有限。BERT 通过 MLM 让模型同时利用左右上下文，适理解类任务。

核心贡献：masked language modeling、next sentence prediction、预训练加下游微调范式，以及在问答、分类、序列标注等任务上的强迁移能力。

为什么重要：BERT 代表了 encoder-only 路线，是理解类 NLP 任务和预训练范式的重要里程碑。

局限：BERT 不适合自回归生成；MLM 预训练和下游生成任务不匹配；现代通用助手主要采用 decoder-only 路线。

面试表达：BERT 擅长理解类任务，因为它用双向 encoder 表示整段输入；GPT 擅长生成类任务，因为它用 causal decoder 逐 token 预测下一个 token。

GPT 系列

一句话定义：GPT 系列代表 decoder-only autoregressive language modeling 路线，通过预测下一个 token 学习通用语言能力。

解决的问题：如何用统一的生成式目标训练可扩展、可迁移、可作为通用助手基础的语言模型。

核心贡献：decoder-only 架构、causal language modeling、规模扩展、上下文学习、指令跟随和后训练范式的逐步成熟。

为什么重要：现代大语言模型大多沿用 GPT 式自回归生成路线。它天然适合对话、代码、推理、工具调用和开放式生成。

局限：自回归生成推理慢，容易累积错误；事实性和安全性需要后训练、检索、工具和评估系统补充。

面试表达：GPT 路线的关键不是只做生成，而是用统一的 next-token prediction 目标在大规模数据和模型容量下涌现出通用能力，再通过后训练对齐到助手行为。

T5

一句话定义：T5 将几乎所有 NLP 任务统一成 text-to-text 格式，是 encoder-decoder 预训练路线的重要代表。

解决的问题：不同 NLP 任务输出形式不同，训练和迁移接口不统一。T5 把分类、翻译、摘要、问答都转成文本到文本任务。

核心贡献：统一任务格式、系统比较预训练目标、数据和规模影响，为后续统一建模思想提供参考。

适用理解：T5 更适理解 text-to-text 多任务学习和 encoder-decoder 模型，不是现代聊天 LLM 的主流架构，但对任务统一化思想影响很大。

面试表达：T5 的价值在于统一任务接口，说明很多 NLP 任务可以被看成条件文本生成。

Scaling 与训练

Scaling Laws for Neural Language Models

一句话定义：Scaling Laws 研究模型参数量、数据量、计算量和 loss 之间的经验规律。

解决的问题：训练大模型前，需要决定模型多大、数据多少、计算预算如何分配。Scaling law 给出可预测的规模扩展规律。

核心贡献：指出随着模型、数据和计算增长，语言模型 loss 呈现较稳定的幂律下降趋势；模型性能可以在较小规模实验上外推预测。

为什么重要：它让大模型研发从“拍脑袋堆规模”变成更可规划的工程决策。

局限：早期结论对数据量配置的建议后来被 Chinchilla 修正；不同数据质量、训练 recipe 和任务能力不一定完全服从同一规律。

面试表达：Scaling law 的核心价值是帮助我们在训练前估算不同规模配置的收益，但它不是绝对定律，需要结合数据质量和训练目标理解。

Chinchilla

一句话定义：Chinchilla 强调 compute-optimal training，即固定计算预算下要平衡模型参数量和训练 token 数。

解决的问题：早期大模型常常参数很多但训练 token 不够，导致在固定算力下没有达到最优性能。

核心结论：在给定计算预算下，模型大小和数据量应该更均衡地扩展。很多旧模型相对“数据不足”，继续增加训练 token 可能比单纯加参数更有效。

为什么重要：它改变了大模型训练配比思路，推动社区更重视高质量、大规模 token 数据和重复训练 token 的设计。

局限：具体最优比例会受数据质量、模型架构、训练目标和下游任务影响；不能机械套公式。

面试表达：Chinchilla 的面试重点是 compute-optimal trade-off。固定算力下，不是模型越大越好，还要给模型足够 token 学习。

PaLM

一句话定义：PaLM 是大规模 dense decoder-only 语言模型的代表工作之一，展示了规模扩展带来的多任务、推理和代码能力提升。

解决的问题：验证更大模型和更大训练数据在多任务能力上的扩展效果。

核心贡献：展示大规模模型在 few-shot、reasoning、多语言和代码任务上的强表现，并系统报告规模带来的能力变化。

面试表达：PaLM 适合用来说明大模型规模扩展与能力涌现的关系，但要避免把所有能力提升都简单归因于参数量。

LLaMA

一句话定义：LLaMA 是开放权重 LLM 生态的重要里程碑，强调用更多高质量 token 训练相对高效的模型。

解决的问题：在有限参数规模下，通过更充分的数据训练得到有竞争力的基础模型，并推动开源生态发展。

核心贡献：提供不同规模模型，采用现代 decoder-only recipe，如 RMSNorm、SwiGLU、RoPE 等；带动大量指令微调、量化、部署和应用研究。

面试表达：LLaMA 的重要性不仅在模型指标，也在于开放权重推动了大模型微调、部署、评测和应用生态爆发。

后训练与对齐

InstructGPT

一句话定义：InstructGPT 系统展示了用 SFT、Reward Model 和 RLHF 把预训练模型对齐到用户指令的 pipeline。

解决的问题：预训练模型会续写文本，但不一定听指令、符合用户偏好或安全要求。InstructGPT 通过人类反馈让模型更像助手。

核心流程：先用人工示范数据做 SFT，再收集人类偏好训练 reward model，最后用 PPO 优化 policy。

为什么重要：它奠定了现代聊天模型后训练范式，是理解 ChatGPT 类模型的关键论文。

局限：RLHF 工程复杂，训练不稳定，reward model 可能被 reward hacking，且偏好数据质量决定上限。

面试表达：InstructGPT 的核心贡献是证明“更符合人类偏好的小模型”可以比更大的纯预训练模型更好用。

Constitutional AI

一句话定义：Constitutional AI 使用一组原则或宪法来指导模型自我批评、自我修正和 AI feedback 安全对齐。

解决的问题：纯 RLHF 依赖大量人工偏好标注，成本高且安全规则难系统化。Constitutional AI 尝试用明确原则和 AI 反馈减少人工依赖。

核心贡献：用原则指导模型生成 critique 和 revision，再用 AI feedback 训练偏好模型或进行对齐。

优点：更容易注入明确安全原则，降低部分人工标注成本。

局限：原则设计本身很关键；AI feedback 也会继承模型偏差；不能完全替代人工安全评估。

面试表达：Constitutional AI 的价值在于把安全偏好显式规则化，并用 AI 参与反馈生成，但最终仍要靠评估验证是否安全有效。

Direct Preference Optimization

一句话定义：DPO 是用偏好数据直接优化 policy 的方法，绕过显式 reward model 和 PPO。

解决的问题：RLHF pipeline 复杂，需要训练 reward model 和做强化学习优化，工程成本高且不稳定。DPO 把偏好优化转化为更接近监督学习的目标。

核心思想：给定 chosen/rejected 响应，直接提升模型对 chosen 的相对概率，同时降低 rejected 的相对概率，并用 reference model 约束偏移。

优点：实现简单，训练稳定，易复现，广泛用于开源模型对齐。

局限：依赖离线偏好数据质量；对在线探索和复杂长期目标不如完整 RL 框架灵活；也可能过拟合偏好格式。

面试表达：DPO 的重点是把“从偏好中学习”从 RLHF 的 reward model 加 PPO，简化成一个直接的偏好分类式优化目标。

RLHF

一句话定义：RLHF 是 Reinforcement Learning from Human Feedback，通过人类偏好训练奖励模型，再优化语言模型输出。

解决的问题：很多助手质量无法用标准答案监督，例如有用性、礼貌、安全、风格和偏好。RLHF 用人类比较反馈学习这些软目标。

核心组成：SFT policy、偏好数据、reward model、PPO 或其他策略优化、KL 约束和安全评估。

局限：成本高，训练复杂，reward hacking，偏好标注不稳定，KL 和 reward 之间需要调参。

面试表达：RLHF 是对齐的重要范式，但不是魔法。它把人类偏好变成优化信号，同时引入 reward model 误差和工程复杂度。

参数高效与系统优化

LoRA

一句话定义：LoRA 是低秩适配方法，在冻结基座模型的同时，只训练插入到权重更新中的低秩矩阵。

解决的问题：全量微调大模型显存和存储成本高，部署多个任务版本也困难。LoRA 用少量可训练参数实现任务适配。

核心思想：把权重更新近似为低秩分解 $\Delta W = BA$ ，训练 A 和 B，冻结原始 W。

优点：训练成本低、保存体积小、易切换多任务 adapter、适合个人和中小团队微调。

局限：表达能力受 rank 和插入位置影响；对大幅改变模型能力的任务不如全量微调灵活；多 adapter 管理和合并也有工程细节。

面试表达：LoRA 的核心不是压缩已有权重，而是约束微调更新落在低秩子空间，用少量参数近似任务所需的权重变化。

QLoRA

一句话定义：QLoRA 在量化基座模型上训练 LoRA adapter，使单卡低成本微调大模型成为可能。

解决的问题：即使 LoRA 只训练少量参数，基座模型加载仍然占大量显存。QLoRA 通过 4-bit 量化基座降低显存。

核心贡献：NF4 量化、double quantization、paged optimizer 等技巧，让大模型微调更省显存。

优点：极大降低微调门槛。

局限：量化会带来精度和训练稳定性问题；工程上依赖具体框架和硬件支持。

面试表达：QLoRA 是“量化加载基座 + LoRA 训练 adapter”的组合，重点解决微调显存瓶颈。

FlashAttention

一句话定义：FlashAttention 是 IO-aware 的 exact attention kernel，通过减少 HBM 读写和避免存储完整 attention 矩阵提升速度和显存效率。

解决的问题：标准 attention 计算不仅有平方复杂度，还会产生巨大的 attention matrix 中间结果，显存读写成为瓶颈。

核心思想：分块计算 attention，在 SRAM 中完成局部计算，并用 online softmax 保持数值正确性。

优点：不改变 attention 数学结果，却显著降低显存占用和提升训练推理速度。

局限：依赖 GPU kernel 优化和硬件特性；不是解决超长上下文复杂度的唯一方案。

面试表达：FlashAttention 的关键是 IO-aware，不是近似 attention。它算的是 exact attention，只是更聪明地组织内存访问。

ZeRO

一句话定义：ZeRO 是通过切分 optimizer state、gradient 和 parameter 来降低数据并行显存冗余的分布式训练方法。

解决的问题：普通数据并行中，每张卡都保存完整参数、梯度和优化器状态，大模型训练显存浪费严重。

核心贡献：ZeRO-1 切 optimizer state，ZeRO-2 进一步切 gradient，ZeRO-3 切 parameter。

为什么重要：它是大模型分布式训练和 DeepSpeed 生态的重要基础。

面试表达：ZeRO 的本质是消除数据并行中的状态冗余，用通信换显存容量。

MoE 与稀疏模型

Switch Transformer

一句话定义：Switch Transformer 是早期大规模 MoE 代表工作，用 top-1 routing 让每个 token 只经过一个专家。

解决的问题：dense 模型参数越大，训练和推理计算都增加。MoE 希望增加总参数量，但每个 token 只激活部分专家，从而提高参数容量而不等比例增加计算。

核心贡献：简化 MoE routing，用 top-1 expert 降低通信和计算复杂度，并展示稀疏激活模型的规模扩展潜力。

优点：在相近计算量下增加模型容量。

局限：负载均衡、专家塌缩、通信开销、训练稳定性和推理部署复杂度高。

面试表达：Switch Transformer 的重点是 sparse activation。它不是每次用所有参数，而是每个 token 路由到少数专家。

Mixtral

一句话定义：Mixtral 是现代 open MoE 模型代表，使用稀疏专家结构在质量和推理成本之间取得平衡。

解决的问题：如何在开放模型中用 MoE 提升能力，同时控制每 token 激活参数量。

核心特点：多个专家中选择 top-k 专家参与计算，总参数量较大，但每个 token 的实际计算只使用部分专家。

为什么重要：它让 MoE 从研究概念走向更广泛的开源应用和部署讨论。

面试表达：Mixtral 适合用来说明 MoE 的工程取舍：总参数量、激活参数量、路由质量、通信开销和部署复杂度必须一起看。

GShard

一句话定义：GShard 是较早将稀疏 MoE 扩展到大规模多语言模型训练的代表工作。

价值：它展示了自动分片和专家并行在大规模训练中的可行性，为后续 Switch Transformer、GLaM 和现代 MoE 模型提供基础。

面试表达：GShard 的关键词是大规模稀疏专家模型和自动分片训练。

长上下文、RAG 与工具增强

RAG

一句话定义：RAG 是 Retrieval-Augmented Generation，将检索器和生成模型结合，用外部文档补充模型知识。

解决的问题：参数知识会过时、不完整、不可追溯。RAG 让模型在生成前检索相关文档，从而支持知识更新和引用依据。

核心贡献：把 dense retrieval 和 seq2seq generation 联合到知识密集型任务中，展示检索增强生成的有效性。

优点：知识可更新，可接入私有数据，可追溯。

局限：强依赖检索质量；检索错了生成也会错；系统链路复杂。

面试表达：RAG 的本质是把一部分知识从模型参数中外置到可检索语料库，用检索结果约束生成。

REALM

一句话定义：REALM 是将检索模块引入预训练过程的早期检索增强语言模型工作。

价值：它说明检索不仅可以在推理阶段使用，也可以进入训练目标，让模型学习何时检索和如何利用外部知识。

面试表达：REALM 适合和 RAG 对比：RAG 更常被理解为生成时检索增强，REALM 强调检索增强预训练。

RETRO

一句话定义：RETRO 通过检索相似文本块增强语言模型，使模型用较少参数获得较强性能。

核心思想：生成时检索近邻 chunk，并通过专门结构把检索到的文本纳入预测。

价值：它展示了参数规模和外部检索记忆之间的替代关系。

面试表达：RETRO 的启发是，模型能力不一定全部存在参数里，也可以通过外部可检索记忆补充。

ReAct

一句话定义：ReAct 将 reasoning 和 acting 结合，让模型在思考、调用工具、观察结果之间循环。

解决的问题：复杂任务需要外部信息和多步行动，单次回答无法解决。ReAct 让模型能边推理边使用工具。

核心贡献：提出用交替的 thought、action、observation 轨迹组织 agent 行为。

局限：成本高，容易循环，工具调用错误会累积。

面试表达：ReAct 的重点是把语言模型从一次性回答扩展成“推理-行动-观察”的闭环。

Toolformer

一句话定义：Toolformer 研究如何让语言模型学习在合适位置调用外部工具。

解决的问题：工具使用需要模型判断何时调用、调用哪个工具以及如何利用结果。

核心贡献：通过自动构造工具调用示例，让模型自监督学习工具使用能力。

面试表达：Toolformer 的价值是说明工具调用可以成为语言模型学习的一部分，而不是完全靠手写规则。

Reasoning 与评估

Chain-of-Thought Prompting

一句话定义：CoT prompting 通过示例引导模型生成中间推理步骤，提升复杂推理任务表现。

解决的问题：模型直接输出答案时容易跳步，复杂数学和逻辑题准确率低。

核心贡献：展示大模型在少样本提示下可以通过显式推理链提升性能。

局限：推理链可能不真实；长推理成本高；简单任务不一定需要 CoT。

面试表达：CoT 的本质是通过中间步骤增加测试时计算和状态表达。

Self-Consistency

一句话定义：Self-Consistency 通过采样多条推理路径并投票选择答案，提高推理稳定性。

解决的问题：单条 CoT 容易偶然走错，多路径采样可以降低单一路径错误影响。

核心贡献：把复杂推理从单次生成变成多候选推理和聚合。

面试表达：Self-Consistency 是用更多推理时计算换准确率，适合高价值复杂问题。

GSM8K

一句话定义：GSM8K 是小学数学应用题数据集，常用于评估大模型多步数学推理能力。

价值：它推动了 CoT、self-consistency 和 reasoning 模型评估的发展。

局限：公开数据集可能污染；题型覆盖有限；不能代表所有数学和符号推理能力。

面试表达：GSM8K 是 reasoning 评估常见基准，但看分数时要注意污染和泛化问题。

多模态与生成

CLIP

一句话定义：CLIP 通过图文对比学习把图像和文本映射到同一语义空间。

解决的问题：传统视觉模型依赖固定标签监督，开放词汇泛化能力弱。CLIP 利用大规模图文对学习开放语义对齐。

核心贡献：图文对比学习、zero-shot 图像分类和开放词汇视觉理解。

面试表达：CLIP 的关键是用自然语言作为视觉监督信号，让图像表示和文本语义对齐。

ViT

一句话定义：Vision Transformer 将图像切成 patch，并用 Transformer 处理视觉任务。

解决的问题：验证 Transformer 是否能替代 CNN 成为视觉骨干网络。

核心贡献：patch embedding、视觉 token 序列化、大规模预训练下 Transformer 在视觉任务上的强表现。

面试表达：ViT 说明 Transformer 不只适用于文本，也可以作为通用序列建模架构处理图像 patch。

Stable Diffusion

一句话定义：Stable Diffusion 是 latent diffusion 的代表系统，在压缩 latent 空间中进行扩散生成。

解决的问题：像素空间扩散成本高。latent diffusion 先用 autoencoder 把图像压到 latent，再在 latent 上扩散，显著降低计算成本。

核心贡献：latent diffusion、文本条件生成、开放生态和可控图像生成应用。

面试表达：Stable Diffusion 的重点是把扩散过程放到 latent space 中做，用更低成本实现高质量文本到图像生成。

Whisper

一句话定义：Whisper 是大规模弱监督训练的端到端语音识别和语音翻译模型。

解决的问题：传统 ASR 系统需要复杂模块和强标注数据，跨语言和鲁棒性受限。Whisper 用大规模弱监督语音数据提升泛化能力。

核心贡献：统一多语言转写、翻译和语言识别任务，提供强开源 ASR 基线。

面试表达：Whisper 适合说明大规模弱监督数据对语音模型鲁棒性的价值，但生产仍需要 VAD、说话人分离和质量评估。

安全与可解释性

Constitutional AI 作为安全论文

一句话定义：在安全线中，Constitutional AI 代表用显式原则和 AI feedback 做安全对齐。

面试表达：如果被问到安全对齐，可以把 Constitutional AI 和 RLHF 对比：前者更强调规则化原则和 AI 反馈，后者更依赖人类偏好数据。

Red Teaming Language Models

一句话定义：红队相关论文研究如何系统发现模型在有害内容、越狱、偏见、隐私和危险能力上的失败模式。

解决的问题：普通 benchmark 很难暴露恶意用户和边界场景下的安全风险。

面试表达：红队测试的价值在于把安全问题从抽象原则变成可复现样本、风险等级和回归测试集。

Mechanistic Interpretability

一句话定义：mechanistic interpretability 试图理解神经网络内部机制，例如 attention head、MLP neuron、circuit 和 feature 如何实现具体行为。

为什么重要：大模型越来越强，但内部决策难解释。可解释性希望帮助诊断模型行为、发现安全风险和理解能力来源。

面试表达：mechanistic interpretability 不是简单画 attention heatmap，而是尝试找到模型内部可验证的计算机制。

论文阅读面试模板

面试中讲一篇论文，可以按以下结构组织：

1. 这篇论文解决什么问题。
2. 为什么过去方法不够好。
3. 核心方法是什么。
4. 关键实验或结论是什么。
5. 有什么局限。
6. 后续工作如何继承或改进它。
7. 它对工程实践有什么影响。

示例表达：这篇论文的贡献我会分三层看。第一，它提出的问题是什么；第二，它的方法相比旧方法改变了什么假设或训练方式；第三，它对后续模型训练、推理或部署有什么长期影响。

精读优先级建议

第一优先级：Attention Is All You Need、BERT、GPT 系列、Scaling Laws、Chinchilla、InstructGPT、DPO、LoRA、FlashAttention。

第二优先级：Switch Transformer、Mixtral、RAG、ReAct、CoT、Self-Consistency、CLIP、ViT、Stable Diffusion。

第三优先级：REALM、RETRO、Toolformer、QLoRA、ZeRO、Whisper、Constitutional AI、红队和可解释性相关论文。

学习建议：第一优先级论文要能讲出核心机制和面试回答；第二优先级论文要能放入技术路线图；第三优先级论文要知道它解决的问题和适用场景。

本章小结

本章按照技术线索整理了大模型算法岗高频论文。

核心结论如下：

1. Transformer 是现代大模型架构起点。
2. BERT 和 GPT 代表理解式 encoder-only 与生成式 decoder-only 两条路线。
3. Scaling Laws 和 Chinchilla 解释了规模、数据和计算预算的权衡。

4. InstructGPT、RLHF、DPO 和 Constitutional AI 构成后训练与对齐主线。
5. LoRA、QLoRA、FlashAttention 和 ZeRO 代表参数高效与系统效率主线。
6. Switch Transformer、GShard 和 Mixtral 代表 MoE 与稀疏激活主线。
7. RAG、REALM、RETRO、ReAct 和 Toolformer 代表外部知识与工具增强主线。
8. CoT、Self-Consistency 和 GSM8K 代表 reasoning 与评估主线。
9. CLIP、ViT、Stable Diffusion 和 Whisper 是多模态方向的关键里程碑。
10. 论文阅读要服务于面试表达和工程判断，而不是机械背诵细节。

下一章，我们进入面试高频对比表。

O. 面试高频对比表

条目：Adam vs AdamW、BatchNorm vs LayerNorm、BERT vs GPT、MHA vs MQA vs GQA、SFT vs RLHF vs DPO、Top-k vs Top-p、LoRA vs Full Fine-Tuning、Quantization vs Distillation。

如何回答对比题

一句话定义：面试中的对比题不是背两个定义，而是说明它们解决的问题、核心机制、优缺点、适用场景和工程取舍。

推荐回答结构：先给一句话区分，再讲核心机制差异，再讲优缺点和适用场景，最后给工程判断。

通用模板：A 和 B 都是为了解决某类问题，但 A 更偏向某个目标，B 更偏向另一个目标。它们的关键差异在于机制、成本、稳定性和适用场景。实际使用时要根据数据规模、模型大小、延迟、显存、质量和安全要求选择。

常见误区：只说“A 更好”或“B 更先进”，不说明约束条件；只背英文缩写，不讲为什么；只讲理论，不讲工程成本。

面试表达：对比题的核心是 trade-off。没有绝对最优，只有在具体约束下更合适。

Adam vs AdamW

一句话区分：AdamW 把 weight decay 从 Adam 的梯度更新中解耦出来，是大模型训练中更常用的优化器变体。

Adam：使用一阶矩和二阶矩做自适应学习率更新。如果直接把 L2 正则加到梯度里，会和 Adam 的自适应缩放耦合。

AdamW：将 weight decay 作为参数衰减单独施加，不混入梯度自适应项，更符合权重衰减本意。

优点对比：Adam 收敛快、鲁棒性好；AdamW 泛化通常更好，训练 Transformer 和 LLM 时更常用。

工程场景：预训练、微调、视觉 Transformer、语言模型训练中通常优先用 AdamW。

面试表达：AdamW 的关键不是多了 weight decay，而是 decoupled weight decay。它避免 L2 正则被 Adam 的自适应学习率扭曲。

SGD vs Adam

一句话区分：SGD 更新简单、泛化常被认为较好；Adam 自适应调整每个参数学习率，收敛更快、更稳定。

SGD：按全局学习率沿梯度方向更新，可配 momentum。优点是简单、显存开销低，缺点是调参敏感、在稀疏梯度或复杂模型中收敛慢。

Adam：维护一阶矩和二阶矩，对不同参数做自适应缩放。优点是收敛快、对学习率不那么敏感，缺点是 optimizer state 显存大。

大模型场景：LLM 训练通常用 AdamW 而不是原始 SGD，因为模型规模大、梯度分布复杂、训练稳定性要求高。

面试表达：SGD 更像统一步长的梯度下降，Adam 更像给每个参数配了自适应学习率。大模型里 AdamW 是默认强基线。

BatchNorm vs LayerNorm

一句话区分：BatchNorm 沿 batch 统计归一化，LayerNorm 沿单个样本的 hidden 维度归一化。

BatchNorm：依赖 batch 统计量，在 CNN 中常用。batch 大且分布稳定时效果好，但小 batch、变长序列和自回归生成场景不方便。

LayerNorm：对每个 token 或样本内部 hidden 维度做归一化，不依赖 batch 统计，适合 Transformer 和序列模型。

为什么 LLM 用 LayerNorm：语言模型推理时通常一个 token 一个 token 生成，batch 和序列长度动态变化，LayerNorm 更稳定、更自然。

面试表达：BatchNorm 依赖 batch 维统计，LayerNorm 不依赖 batch，所以 Transformer 和 LLM 更常用 LayerNorm 或 RMSNorm。

LayerNorm vs RMSNorm

一句话区分：LayerNorm 同时减均值和除标准差，RMSNorm 只按均方根缩放，不做均值中心化。

LayerNorm：计算 hidden 维度均值和方差，再归一化，有 scale 和 bias 参数。

RMSNorm：只用 root mean square 做缩放，计算更简单，常用于现代 LLM。

优点对比：LayerNorm 更标准；RMSNorm 省计算、实现简单，在大模型中效果通常足够好。

面试表达：RMSNorm 可以看作简化版 LayerNorm，去掉均值中心化，保留尺度归一化，在 LLaMA 等模型中很常见。

Pre-Norm vs Post-Norm

一句话区分：Pre-Norm 在子层前做归一化，Post-Norm 在残差相加后做归一化。

Post-Norm：原始 Transformer 常用。浅层模型可以工作，但深层训练容易梯度不稳定。

Pre-Norm：先归一化再进入 attention 或 FFN，残差路径更直接，深层 Transformer 训练更稳定。

缺点：Pre-Norm 有时最终表达能力或收敛性质需要配合额外技巧调整，但现代 LLM 普遍采用类似 Pre-Norm 结构。

面试表达：Pre-Norm 的关键优势是让梯度可以更顺畅地沿残差路径传播，因此更适合训练很深的大模型。

BERT vs GPT

一句话区分：BERT 是 encoder-only、双向理解模型；GPT 是 decoder-only、自回归生成模型。

BERT：使用 masked language modeling，能同时看左右上下文，适合分类、抽取、匹配、序列标注等理解类任务。

GPT：使用 causal language modeling，只看左侧上下文逐 token 预测，适合生成、对话、代码、推理和工具调用。

训练目标差异：BERT 预测被 mask 的 token；GPT 预测下一个 token。

工程选择：如果任务是离线文本理解和 embedding，可考虑 encoder；如果任务是开放式生成和助手交互，decoder-only 更主流。

面试表达：BERT 强在双向表示，GPT 强在自回归生成。现代通用大模型多采用 GPT 式 decoder-only，因为生成接口统一、可扩展性强。

Encoder-Decoder vs Decoder-Only

一句话区分：encoder-decoder 把输入编码和输出生成分开，decoder-only 用同一个自回归堆栈处理上下文和生成。

Encoder-decoder：适合翻译、摘要、文本到文本任务。encoder 双向理解输入，decoder 条件生成输出。

Decoder-only：结构更简单，训练目标统一，适合大规模 causal LM 和多任务指令模型。

大模型趋势：通用 LLM 多采用 decoder-only，因为架构统一、扩展简单、上下文学习自然。

面试表达：encoder-decoder 在条件生成上结构清晰，decoder-only 在大规模统一建模和开放式生成上更主流。

MHA vs MQA vs GQA

一句话区分：MHA 每个 query head 有独立 K/V，MQA 所有 query head 共享一组 K/V，GQA 多组 query head 共享 K/V。

MHA：表达能力强，但 KV cache 大，长上下文推理显存和带宽压力高。

MQA：所有 query heads 共享 K/V，极大减少 KV cache，但可能损失部分质量。

GQA：把 query heads 分组，每组共享 K/V，在质量和效率之间折中。

工程场景：推理部署时 KV cache 是核心瓶颈之一，GQA/MQA 能显著降低显存和带宽成本。

面试表达：MHA、MQA、GQA 的核心差异是 K/V head 数量。它们主要是在注意力表达能力和推理缓存成本之间做 trade-off。

Absolute Position vs RoPE vs ALiBi

一句话区分：绝对位置编码给 token 固定位置向量，RoPE 用旋转方式编码相对位置信息，ALiBi 用 attention bias 引入距离惩罚。

Absolute Position：实现简单，但外推能力有限。

RoPE：现代 LLM 常用，能在 Q/K 空间注入相对位置信息，适合 decoder-only 模型。

ALiBi：不显式学习位置向量，而是在 attention score 中加入距离相关 bias，对长度外推友好。

面试表达：位置编码的核心问题是让 Transformer 获得顺序感。长上下文场景下，外推能力和训练长度匹配非常关键。

ReLU vs GELU vs SwiGLU

一句话区分：ReLU 是硬截断激活，GELU 是平滑概率式激活，SwiGLU 是门控 FFN 激活，现代 LLM 常用 SwiGLU 变体。

ReLU：简单高效，但负半轴直接置零。

GELU：BERT、GPT 等早期 Transformer 常用，平滑性更好。

SwiGLU：引入门控机制，提升 FFN 表达能力，在 LLaMA 等现代模型中常见。

面试表达：LLM 中 FFN 不只是两层 MLP，激活函数和门控结构会明显影响模型容量和训练效果。

SFT vs RLHF vs DPO

一句话区分：SFT 学会模仿高质量回答，RLHF 用 reward model 和强化学习优化偏好，DPO 直接用偏好对优化模型。

SFT：输入是指令和理想回答，目标是监督学习。优点是稳定、简单，缺点是只能模仿数据分布。

RLHF: 先训练 reward model, 再用 PPO 等方法优化模型。优点是能优化偏好目标, 缺点是复杂、不稳定、成本高。

DPO: 使用 chosen/rejected 偏好对直接优化 policy。优点是简单稳定, 缺点是依赖离线偏好数据质量。

工程顺序: 常见 pipeline 是先 SFT, 再做偏好优化。DPO 可作为比 RLHF 更轻量的偏好优化方案。

面试表达: SFT 解决 “会不会按指令回答”, RLHF/DPO 解决 “哪个回答更符合人类偏好”。DPO 简化了 RLHF, 但不等于在所有场景都替代 RL。

RLHF vs DPO

一句话区分: RLHF 显式训练 reward model 并用 RL 优化, DPO 不训练显式 reward model, 而是直接从偏好对优化策略。

RLHF: 更复杂, 更适合在线探索和更通用奖励建模, 但训练成本高、调参难。

DPO: 更接近监督学习, 训练稳定, 易复现, 适合离线偏好数据充分的场景。

关键差异: RLHF 是 reward modeling 加 policy optimization, DPO 是 preference objective 直接优化。

面试表达: DPO 的优势是工程简单和稳定, 但它的上限仍受偏好数据覆盖和质量限制。

RLHF vs RLAIF

一句话区分: RLHF 使用人类反馈, RLAIF 使用 AI 反馈或 AI 辅助反馈。

RLHF: 反馈更接近人类偏好, 但成本高、速度慢、一致性有限。

RLAIF: 可规模化、成本低, 适合安全原则和大规模偏好生成, 但会继承 judge 模型偏差。

工程选择: 关键任务仍需要人工校准, AI feedback 可用于扩展数据规模和初筛。

面试表达: RLAIF 不是完全替代人类反馈, 而是在成本和规模上补充 RLHF。

Top-k vs Top-p

一句话区分: Top-k 保留概率最高的固定 k 个 token, Top-p 保留累计概率达到 p 的最小 token 集合。

Top-k: 候选集大小固定, 简单可控, 但不同分布下灵活性不足。

Top-p: 候选集大小随概率分布变化, 更自适应; 分布尖锐时候选少, 分布平坦时候选多。

适用场景: 需要稳定输出可降低 temperature、k、p; 需要创意和多样性可适当提高。

面试表达: Top-k 控制候选数量, Top-p 控制候选概率质量。两者都是在质量和多样性之间调节采样空间。

Greedy vs Beam Search vs Sampling

一句话区分: greedy 每步选最大概率 token, beam search 保留多个高概率序列, sampling 按概率随机采样。

Greedy: 确定性强、速度快, 但容易保守和陷入局部最优。

Beam Search: 适合翻译等目标较确定的任务, 但开放式生成容易重复、僵硬。

Sampling: 多样性强, 适合对话、创意和多候选推理, 但稳定性较差。

面试表达: 解码策略要匹配任务。开放式对话不一定适合 beam search, 代码和结构化任务通常更偏确定性解码。

Temperature vs Top-p

一句话区分：temperature 改变概率分布的尖锐程度，top-p 截断候选 token 集合。

Temperature：低温让高概率 token 更突出，输出更稳定；高温让分布更平，输出更多样。

Top-p：只在累计概率阈值内采样，过滤低概率长尾 token。

组合使用：temperature 控制分布形状，top-p 控制候选范围。两者常一起调。

面试表达：temperature 是调概率分布，top-p 是裁剪候选集合。生产中要按任务质量、稳定性和多样性要求调参。

LoRA vs Full Fine-Tuning

一句话区分：full fine-tuning 更新所有参数，LoRA 冻结基座模型，只训练低秩 adapter。

Full fine-tuning：表达能力强，适合数据充分、算力充足、需要深度改变模型行为的任务。缺点是显存高、存储大、容易遗忘。

LoRA：训练参数少、成本低、易保存多个任务版本，适合领域适配和中小规模微调。

关键取舍：质量上限、训练成本、部署复杂度、多任务管理和灾难性遗忘风险。

面试表达：LoRA 不是比全量微调绝对更好，而是在资源受限和多任务部署时更实用。

LoRA vs Adapter vs Prefix Tuning

一句话区分：LoRA 修改权重更新的低秩子空间，Adapter 插入小模块，Prefix Tuning 在输入侧学习可训练前缀。

LoRA：常用于 attention/MLP 权重，部署时可合并权重，工程友好。

Adapter：在层间插入小网络，模块化强，但推理可能增加结构开销。

Prefix Tuning：学习虚拟 token 或 KV 前缀，参数少，但表达能力和稳定性依任务而定。

面试表达：三者都是 PEFT 方法，区别在于改哪里：LoRA 改权重更新，Adapter 加模块，Prefix 改上下文或 KV 表示。

Quantization vs Distillation

一句话区分：quantization 降低数值精度，主要减少显存和带宽；distillation 用教师模型训练学生模型，主要压缩能力到更小模型。

Quantization：把 FP16/BF16 权重或激活压到 INT8、INT4 等，推理更省显存、更快，但可能损失精度。

Distillation：让小模型学习大模型输出、logits、偏好或推理过程，得到结构更小的模型。

工程选择：如果模型结构不变但部署成本高，优先考虑量化；如果需要更小架构和更低延迟，可考虑蒸馏。

面试表达：量化压的是数值表示，蒸馏压的是模型能力和结构。

PTQ vs QAT

一句话区分：PTQ 是训练后量化，QAT 是量化感知训练。

PTQ：不重新训练或只做少量校准，成本低，适合快速部署。

QAT：训练时模拟量化误差，让模型适应低精度，效果通常更好但成本更高。

LLM 场景：权重量化常用于降低显存，KV cache 量化和激活量化需要更谨慎评估。

面试表达：PTQ 更便宜，QAT 更稳。选择取决于可接受的精度损失和训练成本。

FP16 vs BF16

一句话区分：FP16 精度位更多但指数范围小，BF16 指数范围接近 FP32，训练更不容易溢出。

FP16：需要 loss scaling 来缓解梯度下溢和溢出问题。

BF16：动态范围更大，训练稳定性更好，现代硬件支持后大模型训练常用。

面试表达：BF16 牺牲部分尾数精度换更大动态范围，因此大模型训练中通常比 FP16 更省心。

FlashAttention vs Sparse Attention

一句话区分：FlashAttention 不改变 attention 数学结果，是 exact attention 的 IO-aware 实现；Sparse Attention 改变 attention 连接模式，只计算部分 token pair。

FlashAttention：通过分块和 online softmax 减少显存读写和中间矩阵存储。

Sparse Attention：通过局部窗口、全局 token 或稀疏模式降低计算量，但结果是近似或结构受限的 attention。

面试表达：FlashAttention 优化的是实现和内存访问，Sparse Attention 改变的是 attention 计算图。

PagedAttention vs FlashAttention

一句话区分：FlashAttention 优化 attention kernel，PagedAttention 优化 KV cache 管理。

FlashAttention：主要解决单次 attention 计算中的显存访问效率。

PagedAttention：把 KV cache 分页管理，减少连续显存分配和碎片，提高 serving 场景并发能力。

面试表达：FlashAttention 更偏 kernel 计算优化，PagedAttention 更偏推理服务内存管理。

KV Cache vs Activation Checkpointing

一句话区分：KV cache 用于推理时缓存历史 K/V，activation checkpointing 用于训练时减少激活显存。

KV Cache：自回归推理中避免重复计算历史 token 的 K/V，加速生成，但会占用显存且随序列长度增长。

Activation Checkpointing：训练时不保存全部中间激活，反向传播时重算，节省显存但增加计算。

面试表达：KV cache 是推理优化，checkpointing 是训练省显存。一个缓存避免重算，一个主动重算换显存。

Data Parallel vs Tensor Parallel vs Pipeline Parallel

一句话区分：data parallel 切 batch，tensor parallel 切层内矩阵，pipeline parallel 切模型层。

Data Parallel：每卡一份完整模型，处理不同数据，同步梯度。简单成熟，但不解决单卡放不下模型的问题。

Tensor Parallel：把矩阵计算拆到多卡，适合单层很大，但通信频繁。

Pipeline Parallel：把不同层放到不同设备，适合模型很深，但有 pipeline bubble 和调度复杂度。

面试表达：三种并行分别从样本维、张量维和层维拆训练负载，真实大模型训练常组合使用。

DDP vs ZeRO vs FSDP

一句话区分：DDP 复制完整模型并同步梯度，ZeRO/FSDP 对参数、梯度和 optimizer state 做分片来省显存。

DDP：实现简单，适合单卡能放下模型的场景。

ZeRO：分阶段切 optimizer state、gradient、parameter，降低数据并行冗余。

FSDP：按模块分片参数，需要时 all-gather，用完释放，和 ZeRO-3 思路相近。

面试表达：DDP 提升吞吐但不省模型状态显存，ZeRO/FSDP 用通信换显存容量。

RAG vs Fine-Tuning

一句话区分：RAG 把知识放在外部检索库中，fine-tuning 把行为或知识部分写入模型参数。

RAG：适合知识更新频繁、私有文档、需要引用和可追溯的场景。

Fine-tuning：适合改变输出风格、任务格式、领域术语和行为模式。

关键判断：如果问题是“模型不知道最新事实”，优先 RAG；如果问题是“模型不会按这种方式回答”，考虑微调。

面试表达：RAG 解决知识访问，微调解决行为适配。不要用微调硬记频繁变化的知识库。

Dense Retrieval vs Sparse Retrieval

一句话区分：dense retrieval 用向量语义相似度检索，sparse retrieval 用关键词和倒排索引检索。

Dense：擅长语义改写和同义表达，但对数字、代码、专有名词可能不稳。

Sparse：擅长精确匹配和可解释检索，但对语义改写能力有限。

工程方案：企业 RAG 常用 hybrid retrieval，再用 reranker 排序。

面试表达：向量检索不是万能的。真实 RAG 往往需要 dense、sparse 和 reranker 组合。

Retriever vs Reranker

一句话区分：retriever 负责快速召回候选，reranker 负责精排候选。

Retriever：追求快和召回率，通常从大规模库中取 top-k。

Reranker：用更强模型判断 query-document 相关性，提升进入 prompt 的证据质量。

关键限制：如果 retriever 没召回正确文档，reranker 无法凭空补救。

面试表达：retriever 决定候选池，reranker 决定最终证据顺序。RAG 排障时要区分召回问题和排序问题。

Long Context vs RAG

一句话区分：long context 让模型一次看更多内容，RAG 先从外部库中筛选相关内容再给模型看。

Long Context：适合必须完整审阅固定材料的任务，但成本高、噪声多、有效利用不一定稳定。

RAG：适合知识库很大、查询相关内容稀疏、需要动态更新和引用的场景。

组合方式：先 RAG 召回，再用较长上下文容纳证据、历史和工具结果。

面试表达：长上下文解决容量问题，RAG 解决选择问题。窗口大不代表应该把所有材料都塞进去。

Agent vs Workflow

一句话区分：agent 由模型动态决定步骤和工具，workflow 按预定义流程执行。

Agent：灵活，适合开放任务，但不稳定、难评估、成本高。

Workflow：可控、可审计、稳定，适合标准业务流程，但灵活性差。

工程选择：生产系统常把关键路径工作流化，把开放决策点交给模型。

面试表达：Agent 不是越自由越好。高风险业务应优先可控 workflow，再在局部引入模型智能。

Chain-of-Thought vs Tool Use

一句话区分：CoT 让模型在文本中推理，tool use 让模型调用外部系统获得计算、检索或执行结果。

CoT：适合模型内部可完成的逻辑推理和问题拆解。

Tool Use：适合计算、实时信息、数据库查询、代码运行和业务操作。

面试表达：能用工具验证的问题，不要只依赖模型自说自话的推理链。

Human Evaluation vs LLM-as-a-Judge

一句话区分：human evaluation 用人评估，LLM-as-a-Judge 用强模型评估。

Human Evaluation：更接近真实用户和专家判断，但成本高、慢、一致性难保证。

LLM-as-a-Judge：便宜、快、可规模化，但有长度偏好、位置偏好、事实判断不可靠等风险。

工程做法：用 LLM judge 做大规模初筛，用人工做校准、仲裁和高风险样本评估。

面试表达：LLM judge 是辅助评估工具，不是真值来源。关键评估仍要有人类或可执行验证闭环。

Accuracy vs Pass@k

一句话区分：accuracy 看单次答案是否正确，pass@k 看 k 个候选中是否至少一个正确。

Accuracy：适合分类、选择题和单答案任务。

Pass@k：适合代码、数学和可验证任务，反映多次采样后找到正确解的概率。

面试表达：pass@k 高不代表生产体验一定好，还要看生成成本、延迟、测试覆盖率和选择器质量。

Factuality vs Grounding

一句话区分：factuality 关注输出是否真实，grounding 关注输出是否被给定证据支持。

Factuality：答案可能真实，但不一定来自当前证据。

Grounding：答案必须能由上下文或引用片段支持，即使外部世界另有事实，也不能胡乱引入。

RAG 场景：两者都重要，尤其要检查引用是否真正支持结论。

面试表达：事实正确不等于证据支持，引用存在也不等于 grounding 成立。

Robustness vs Generalization

一句话区分：generalization 关注模型在未见样本上的泛化能力，robustness 关注面对扰动、噪声和攻击时是否稳定。

Generalization：更多是训练分布到测试分布的能力迁移。

Robustness：强调输入改写、无关噪声、对抗提示、格式变化和分布外样本下的稳定性。

面试表达：一个模型 benchmark 分数高，不代表 robustness 好。真实线上输入往往更脏、更长、更对抗。

Over-Refusal vs Under-Refusal

一句话区分：over-refusal 是该回答却拒绝，under-refusal 是该拒绝却回答。

Over-refusal：损害 helpfulness，常见于安全策略过保守。

Under-refusal：突破安全底线，可能提供危险或违法帮助。

安全评估：必须同时看两者，否则只优化一个会牺牲另一个。

面试表达：安全模型不是拒绝越多越好，而是要在正确边界上拒绝和帮助。

Jailbreak vs Prompt Injection

一句话区分：jailbreak 通常是用户直接诱导模型绕过安全策略，prompt injection 通常是外部内容间接注入恶意指令。

Jailbreak：例如角色扮演、忽略规则、多轮诱导、编码规避。

Prompt Injection：常见于 RAG、网页浏览、工具调用和 agent，恶意指令藏在文档或工具返回中。

防护重点：jailbreak 依赖安全对齐和输入审核；prompt injection 还需要区分可信指令和不可信数据，限制工具权限。

面试表达：prompt injection 是 LLM 应用安全问题，不只是模型安全问题；尤其要关注 RAG 和 Agent 链路。

Helpful vs Harmless

一句话区分：helpful 要尽量帮助用户，harmless 要避免造成伤害。

冲突场景：用户请求具有双重用途，例如网络安全、化学、生物、医疗、金融和法律建议。

工程做法：低风险请求充分帮助，高风险请求拒绝危险细节并提供安全替代，模糊请求先澄清。

面试表达：对齐不是无条件听话，而是在安全边界内最大化帮助。

Dense Model vs MoE

一句话区分：dense model 每个 token 使用全部参数，MoE 每个 token 只激活部分专家。

Dense：结构简单、训练和部署更直接，但参数和计算一起增长。

MoE：总参数量大、每 token 计算相对可控，但有路由、负载均衡、通信和部署复杂度。

面试表达：MoE 的优势是 sparse activation，用更大参数容量提升能力；代价是系统复杂度和路由稳定性。

本章小结

本章整理了大模型算法岗最常见的高频对比题。

核心结论如下：

1. 对比题要围绕问题、机制、优缺点、场景和 trade-off 回答。
2. AdamW、LayerNorm/RMSNorm、Pre-Norm 是现代 Transformer 训练中的高频基础。
3. BERT vs GPT、encoder-decoder vs decoder-only 是架构路线核心对比。
4. MHA、MQA、GQA 主要比较 KV cache 成本和表达能力。
5. SFT、RLHF、DPO 对应监督模仿、偏好强化和直接偏好优化。
6. 解码策略要按任务选择，不能机械说 beam search 或 sampling 更好。
7. LoRA、量化、蒸馏、FlashAttention、PagedAttention 都是在不同层面降低训练或推理成本。
8. RAG、长上下文和 Agent 分别解决知识获取、信息容量和多步行动问题。
9. 评估和安全对比题要同时关注正确性、证据、鲁棒性、过拒和漏拒。
10. 面试回答不要追求绝对结论，要体现约束条件和工程判断。

下一章，我们进入多模态学习。

P. 多模态学习

条目：Multimodal Learning、Visual Token Budget、Multimodal Context Budget、Multimodal Cost Audit、Multimodal Product、Input Quality Pass Rate、Input Fidelity Rate、Multimodal Evidence Recall、Multimodal Evidence Support Rate、OCR Quality Score、OCR Critical Field Accuracy、ASR Quality Score、ASR Critical Field Accuracy、Temporal Evidence Recall、Generation Adoption Rate、Media Safety Pass Rate、Privacy Pass Rate、Copyright Pass Rate、Multimodal Incident、Multimodal Incident Gate、Multimodal Product Gate、CLIP、CLIP Loss、Zero-Shot Classification、Image-Text Retrieval、Contrastive

Learning, Image Encoder, Patch Embedding, CLS Token, Vision Encoder Shape Audit, VLM Connector Audit, Image Placeholder, Visual Token Compression, Assistant-Only Multimodal Loss, Multimodal SFT Data Audit, Chat Template, Multimodal Task Coverage, Evidence Support Rate, Missing Refusal Rate, Diffusion Model, DDPM, Forward Diffusion, Reverse Denoising, Noise Scheduler, Noise Prediction Loss, Classifier-Free Guidance, Latent Diffusion, Stable Diffusion, VAE Compression Ratio, Negative Prompt, Text-to-Image Pipeline Audit, ControlNet, Image-to-Image, Inpainting, DALL-E, Autoregressive Image Tokens, Video Generation, Spatiotemporal Patch, Video Diffusion, Temporal Consistency, Identity Drift, Flickering, World Model, Physics Consistency, Video Generation Evaluation, FVD, Video Token Audit, Audio Generation, Waveform, Log-Mel Spectrogram, ASR, Whisper, TTS, Vocoder, Voice Cloning, Audio Codec, Speech Token, Codec Language Model, Speech-to-Speech, VAD, WER, CER, MOS, Audio Token Audit, Unified Multimodal Model, Any-to-Any Multimodal Model, Unified Tokenization, Early-Fusion Multimodal Transformer, Center-LLM Multimodal System, Multimodal Router, Multimodal Loss Mixture, Modality Conflict, Unified Multimodal Audit, Multimodal Evaluation, VQA Accuracy, Chart Relaxed Accuracy, Grounding IoU, Multimodal Hallucination Rate, Multimodal Prompt Injection, Biometric Identity Safety, Content Provenance, Multimodal Safety Audit, Multimodal Interview Readiness, Multimodal Interview Rubric, Multimodal Formula Coverage, Multimodal Demo Coverage, Weak Multimodal Question, Multimodal Revision Plan, Text Encoder, ViT, SigLIP, Q-Former, Perceiver Resampler, Vision-Language Model, LLaVA, Flamingo, BLIP, Image Captioning, Visual Question Answering, OCR, Grounding, Multimodal Instruction Tuning, Multimodal Alignment, Multimodal Hallucination.

本章速览

一句话定义：多模态学习是让模型同时处理文本、图像、音频、视频、传感器等不同模态信息，并学习它们之间的语义对应关系。

大模型语境下，最常见的多模态模型是视觉语言模型，也就是让语言模型理解图片、回答图像问题、描述图片内容、进行 OCR、定位物体、解释图表，甚至调用工具完成复杂视觉任务。

核心问题：不同模态的数据格式不同、统计分布不同、信息密度不同。文本是离散 token，图像是连续像素或 patch，音频是时序信号，视频同时有空间和时间维度。多模态学习要解决的不是简单把它们拼在一起，而是让模型在统一语义空间里对齐、融合和推理。

典型路线：先用专门 encoder 提取非文本模态特征，再通过投影层、查询模块或 cross-attention 将这些特征接入语言模型，最后通过指令微调让模型学会用自然语言完成多模态任务。

面试表达：多模态模型的关键不是“能输入图片”，而是如何把视觉特征转成语言模型能理解的 token，以及如何保证跨模态对齐、推理可靠性和幻觉可控。

Multimodal Learning

Multimodal Learning 指利用多个模态进行表示学习、对齐学习、融合学习和任务学习。

常见模态包括文本、图像、音频、视频、表格、网页结构、点云、传感器数据等。不同业务会关注不同组合，例如图文检索关注 image-text，语音助手关注 speech-text，自动驾驶关注 camera-lidar-text，短视频理解关注 video-audio-text。

多模态学习常见任务如下：

任务	输入	输出	典型目标
图文检索	图像和文本	相似度排序	找到与图片匹配的文本，或与文本匹配的图片
Image Captioning	图像	文本描述	描述图片内容
VQA	图像和问题	文本答案	回答视觉问题
OCR	图像	文本	识别图片中文字
Grounding	图像和文本	区域或框	将语言短语定位到图像区域
多模态对话	图像和对话历史	回复	围绕图片进行多轮问答
图表理解	图表图片和问题	答案或解释	读取图表结构并推理

任务	输入	输出	典型目标
视频理解	视频和问题	文本答案	理解事件、动作和时间关系

多模态学习的三个关键词：对齐、融合、生成。

对齐是让不同模态的语义表示可比较，例如“狗的图片”和“a dog”在向量空间中接近。融合是让模型联合利用多个模态的信息，例如同时看图片和问题。生成是把多模态输入转成文本、图像、音频或动作等输出。

常见误区：把多模态等同于图像分类。现代多模态大模型更关注开放式理解、指令跟随、跨模态推理和生成式交互。

Modality Gap

Modality Gap 指不同模态之间存在天然表示差异。

文本 token 是离散符号，序列长度通常较短，但语义密度高。图像像素是连续信号，空间分辨率高，局部细节多。音频是长时序信号，对时间对齐敏感。视频同时包含空间、时间、声音和事件关系。

模态差异带来的问题包括：

问题	含义	影响
表示空间不同	图像向量和文本向量分布不同	难以直接相似度比较
信息粒度不同	图像有局部区域，文本有词和句子	难以精确对齐
序列长度不同	图像 patch 或视频帧很多	上下文成本高
噪声形式不同	图片模糊、文字遮挡、语音噪声	鲁棒性难保证
标注方式不同	文本标注便宜，区域标注昂贵	高质量数据稀缺

面试表达：多模态模型的工程难点之一是 modality gap。模型需要通过对比学习、投影映射、cross-attention 或指令微调，把不同模态映射到可协同推理的空间中。

Visual Token Budget

Visual Token Budget 指图像或视频进入 LLM 前占用的视觉 token 数量预算。它通常由分辨率、patch size、采样帧数、token 压缩方式和是否加入额外全局 token 决定。

图像 patch token 的粗略估算如下：

$$N_{\mathrm{img}} = \left\lceil \frac{H}{P} \right\rceil \cdot \left\lceil \frac{W}{P} \right\rceil$$

其中 H 和 W 是输入分辨率，P 是 patch size。分辨率翻倍时，patch token 数通常按面积增长，这会直接影响 VLM 的上下文长度、显存和延迟。

视频 token 还要乘以采样后的帧数：

$$N_{\mathrm{vid}} = F \cdot N_{\mathrm{img}}, \quad F = \left\lceil \frac{F}{s} \right\rceil$$

面试表达：视觉 token budget 是多模态工程的第一张成本表。OCR、图表、截图和细粒度 grounding 往往需要更多 token，但 token 越多，推理成本和延迟越高。

Multimodal Context Budget

Multimodal Context Budget 指文本、图像、音频、视频和额外控制 token 拼接后共同占用的上下文预算。

$$\begin{aligned} T_{\mathrm{total}} = & \\ T_{\mathrm{text}} + & \\ N_{\mathrm{img}} + & \\ N_{\mathrm{vid}} + & \\ N_{\mathrm{aud}} + & \\ N_{\mathrm{extra}} \end{aligned}$$

当这些 token 进入普通全 attention LLM 时，主要 attention 成本会随 T_{total} 近似二次增长。因此一个输入是否可上线，不只取决于模型能否接受图片，还取决于 T_{total} 是否小于模型上下文、显存和延迟预算。

面试表达：多模态上下文预算能把“看图能力”落到工程约束上。高分辨率、多帧视频和长音频都可能把系统推过上下文上限，即使单个样例看起来能跑。

Multimodal Cost Audit

Multimodal Cost Audit 指上线或实验前，对多模态输入的 token 数、显存、延迟、吞吐、评估门禁和安全风险进行统一审计。

常见审计字段包括：

字段	含义	典型风险
image tokens	图像 patch 或压缩后的视觉 token	分辨率过高导致上下文爆炸
video tokens	采样帧乘以每帧 token	长视频延迟高、显存高
audio frames	语音特征帧或音频 token	实时语音首包延迟高
connector shape	视觉 hidden size 到 LLM hidden size	维度不匹配或 token 压缩过强
hallucination rate	答案不被媒体证据支持的比例	流畅但不可靠
injection rate	不可信媒体文字触发错误指令的比例	截图或网页内指令污染

面试表达：多模态成本审计不是单纯算钱，而是把 token 预算、质量、延迟、幻觉和安全门禁放在同一张表里，决定某个方案能否进入真实产品。

Multimodal Product

一句话定义：multimodal product 是把图像、文档、语音、视频或生成能力做成可评估、可控成本、可处理隐私版权、可人审、可进入真实工作流的产品系统。

和模型 demo 区别：demo 往往展示一张清晰图片、一段干净语音或一条高质量生成样例；产品要面对真实上传质量、低光照、噪声、遮挡、小字、多人说话、长视频、授权边界、安全审核、延迟成本和用户反馈。

面试表达：多模态产品不是“能看图 / 能说话 / 能生成视频”就结束，而是要证明输入质量、证据支持、切片指标、安全隐私版权、人工复核、成本延迟和业务指标都过线。

Input Quality Pass Rate

一句话定义：input quality pass rate 是多模态输入中满足处理要求的比例，例如图片清晰、文档可读、音频信噪比足够、视频关键帧完整、文件格式和权限有效。

产品意义：多模态系统很多失败来自输入分布，而不是模型本身。输入不过线时，产品应引导重新拍摄、裁剪、重新上传、澄清或转人工。

Multimodal Evidence Support Rate

一句话定义：multimodal evidence support rate 是输出关键断言中，被图片、OCR、ASR、视频帧、文档页、表格或引用证据支持的比例。

常见误区：图像问答、文档问答和医疗 / 工业辅助场景不能只看回答流畅度。答案碰巧正确但没有媒体证据支持，在产品审计中仍然是风险样本。

OCR Quality Score

一句话定义：OCR quality score 是用 $1 - \text{CER}$ 或字段抽取准确率等指标描述图片文字、文档小字、表格和截图文本识别质量。

面试表达：OCR 切片要单独评估。平均任务成功率可能掩盖小字、低分辨率、票据、表格、代码截图和多语言文档的失败。

ASR Quality Score

一句话定义：ASR quality score 是用 $1 - \text{WER}$ 、专有名词召回、时间戳误差和噪声切片指标描述语音识别质量。

产品意义：语音助手的 ASR 错误会继续传给 LLM 和 TTS，形成级联错误。实时语音还要同时看首字延迟、打断和说话人分离。

Generation Adoption Rate

一句话定义：generation adoption rate 是图像、音频或视频生成结果被保存、下载、进入编辑链路或用于业务草稿的比例。

面试表达：生成产品的真实价值不只是主观好看，而是采纳、可控编辑、品牌一致、授权清楚和节省创作时间。

Media Safety Pass Rate

一句话定义：media safety pass rate 是多模态输入和输出通过安全审核的比例，覆盖暴力、色情、未成年人、深伪、冒充、危险内容和不可信媒体指令。

产品要求：多模态安全审核要覆盖输入和输出。图片、音频、视频、文档和截图中的指令都只能作为数据，不能覆盖系统或开发者指令。

Privacy Pass Rate

一句话定义：privacy pass rate 是人脸、身份信息、会议录音、家庭环境、医疗影像和企业机密等隐私风险被正确脱敏、阻断、最小化保存或授权处理的比例。

Copyright Pass Rate

一句话定义：copyright pass rate 是输入素材授权、参考图使用边界、生成输出使用范围、品牌素材和内容溯源通过审查的比例。

Multimodal Product Gate

一句话定义：multimodal product gate 是多模态产品上线前的综合门禁，覆盖任务成功、输入质量、证据支持、OCR / ASR、生成采纳、安全、隐私、版权、人审、上下文预算、P95 延迟、单位成本、评估集、反馈闭环和业务指标。

面试表达：多模态产品门禁比纯文本更复杂，因为输入质量和输出风险都更不可控。少数高质量演示样例不能替代真实输入分布和安全版权审计。

CLIP

CLIP 是 Contrastive Language-Image Pre-training 的代表方法，用大规模图文对训练图像 encoder 和文本 encoder，使匹配的图文向量相近，不匹配的图文向量远离。

CLIP 的基本结构：一个 image encoder 负责把图片编码成向量，一个 text encoder 负责把文本编码成向量。训练时，对一个 batch 中的 N 对图文样本计算 N x N 相似度矩阵，正样本是同一对图文，负样本是 batch 内其他图文组合。

简化目标如下：

image_i -> image_embedding_i
text_i -> text_embedding_i

$$s_{ij} = \frac{\text{cosine}(v_i, t_j)}{\tau}$$

$$L_{\text{CLIP}} = \frac{L_{I2T} + L_{T2I}}{2}$$

CLIP 的价值：它把图像和文本对齐到同一个语义空间，使模型可以做零样本分类、图文检索、相似度排序和视觉特征初始化。

零样本分类方式：把类别名写成 prompt，例如 “a photo of a cat”、“a photo of a dog”，再比较图片向量和各类别文本向量的相似度。相似度最高的类别作为预测结果。

优点：训练目标简单、可扩展性强、零样本能力好、迁移能力强。

缺点：对细粒度空间关系、计数、OCR、复杂推理不够强；依赖图文对质量；容易学习数据中的偏见；对 prompt 表述敏感。

面试表达：CLIP 的核心是图文对比学习。它不直接生成文本，而是学习图像和文本的共享语义空间，是很多视觉语言模型的视觉对齐基础。

CLIP Loss

CLIP Loss 是图文双塔模型常用的双向对比学习目标。

设 batch 中有 N 对图文，归一化图像向量为 v_i ，归一化文本向量为 t_j ，相似度矩阵为：

$$S_{ij} = \frac{v_i \cdot t_j}{\tau}$$

image-to-text 方向把每张图对应的正确文本当作类别：

$$L_{I2T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ij})}$$

text-to-image 方向对 S 转置做同样的 cross entropy：

$$L_{T2I} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ji})}$$

最终取平均：

$$L_{\mathrm{CLIP}} = \frac{L_{\mathrm{I2T}} + L_{\mathrm{T2I}}}{2}$$

关键实现点：先 L2 normalize，再做矩阵乘法；`logit_scale` 通常对应 $1 / \tau$ ，控制 softmax 分布尖锐程度；label 是 $[0, 1, \dots, N-1]$ ，因为 batch 中第 i 张图和第 i 段文本匹配。

面试表达：CLIP loss 本质是两个方向的 batch 内分类题。图片检索文本看行 softmax，文本检索图片看列 softmax，对角线是正样本。

Zero-Shot Classification

Zero-Shot Classification 指不为新类别训练分类头，而是把类别名称写成文本 prompt，再用图文相似度完成分类。

流程：

1. 为每个类别构造 prompt，例如 a photo of a dog。
2. 用 text encoder 编码类别 prompt。
3. 用 image encoder 编码待分类图片。
4. 选择相似度最高的类别。

如果每个类别有多个 prompt 模板，可以做 prompt ensemble：先平均同一类别的多个文本向量，再归一化为类别向量。

面试表达：CLIP 的 zero-shot 分类把“分类头权重”换成了自然语言类别 prompt 的 embedding，因此可以用语言描述新类别，但效果会受 prompt 模板、训练分布和类别粒度影响。

Image-Text Retrieval

Image-Text Retrieval 指用图像检索文本，或用文本检索图像。

CLIP 适合检索的原因是双塔结构：图像库和文本库可以离线编码成向量，线上只需编码 query 并做相似度排序。

常见指标：

指标	含义	适用
Recall@K	正确匹配是否出现在 Top-K	检索召回
MRR	正确匹配排名倒数的平均值	关注排名靠前程度
Median Rank	正确匹配的中位排名	大规模检索稳定性
Top-1 Accuracy	Top-1 是否正确	小规模配对评估

面试表达：CLIP 检索项目不能只展示 Top-K 样例，还要报告 Recall@K、MRR、bad case 类型和数据噪声，例如 OCR、细粒度关系、中文 prompt、近重复图和错误 caption。

Contrastive Learning

Contrastive Learning 的目标是拉近正样本，推远负样本。

在图文场景中，正样本通常是真实配对的图片和描述，负样本通常是 batch 内其他图片或文本。模型通过对比目标学习“哪些图片和哪些文本语义匹配”。

常见公式是 InfoNCE。对 image-to-text 方向，可以写成：

$$L_i = -\log \frac{\exp(\text{sim}(I_i, T_i)/\tau)}{\sum_j \exp(\text{sim}(I_i, T_j)/\tau)}$$

其中 I_i 是第 i 张图片向量， T_i 是对应文本向量，`sim` 是相似度函数，`tau` 是 temperature。text-to-image 方向类似，通常两个方向相加。

关键点：

概念	说明
正样本	真实匹配的图文对
负样本	不匹配图文对，常来自同 batch
temperature	控制相似度分布尖锐程度
batch size	batch 越大，负样本越多，对比学习通常越有效
hard negative	语义相近但不匹配的负样本，对模型更有挑战

优点：无需复杂人工标签，只需图文配对数据；表示学习效果好；适合检索和零样本分类。

缺点：需要大 batch 或高效负样本构造；可能把真实语义相关但非配对样本当成负样本；难以直接学习复杂生成和多步推理。

面试表达：对比学习学的是判别式对齐，不是生成式理解。它能告诉你图片和文本是否匹配，但不天然具备长文本回答能力。

Image Encoder

Image Encoder 是把图片转成向量或 token 序列的模块。

常见 image encoder 包括 CNN、ViT、Swin Transformer、ConvNeXt、CLIP ViT 等。在视觉语言模型中，最常见的是使用预训练好的 CLIP ViT 或其他视觉 backbone 提取 patch-level 特征。

输出形式有两类：

输出	含义	用途
全局向量	一张图一个 embedding	图文检索、分类、相似度排序
patch token	一张图多个视觉 token	VQA、caption、多模态对话、grounding

如果只做图文检索，通常全局向量足够。如果要让语言模型回答图像细节问题，需要保留 patch token，因为局部区域信息很重要。

工程坑：视觉 token 数量会显著影响上下文长度。例如 336 x 336 图片切成 14 x 14 patch，会有 576 个 patch token。如果多图输入或视频输入，视觉 token 会快速膨胀，影响显存、延迟和长上下文预算。

面试表达：image encoder 决定模型能看到什么视觉信息，connector 决定这些视觉信息如何进入 LLM，LLM 决定如何用语言进行推理和回答。

Patch Embedding

Patch Embedding 是 ViT 和很多 VLM 视觉塔的第一步：把图像切成固定大小 patch，并把每个 patch 映射成 hidden vector。

设输入图像为 $X: [B, C, H, W]$ ，patch size 为 P ，则 patch token 数近似为：

$$N_{\text{patch}} = \left\lceil \frac{H}{P} \right\rceil \cdot \left\lceil \frac{W}{P} \right\rceil$$

每个 patch 展平后的维度是：

$$d_{\text{patch}} = C \cdot P^2$$

实现上常用 `Conv2d(C, d_v, kernel_size=P, stride=P)`，权重 shape 等价于 $[d_v, C, P, P]$ ，输出再展平为 $[B, N, d_v]$ 。

面试表达：patch embedding 可以理解为把图片翻译成视觉 token。patch 越小，token 越多，细节越多，但 attention 和上下文成本越高。

CLS Token

CLS Token 是 ViT 中常见的全局汇聚 token。它会和 patch tokens 一起进入 Transformer，最后常用 CLS 位置的输出表示整张图。

在 CLIP 的 ViT vision tower 中，常见流程是：

image -> patch embedding -> add CLS token -> add position embedding -> Transformer -> CLS output -> projection

CLS token 适合图像分类、图文检索和全局 image embedding；patch tokens 更适合 VLM 问答、OCR、grounding、计数和图表理解。

面试表达：CLS token 不是“看到所有细节”的保证，它是全局汇聚表示。需要细粒度视觉证据时，通常要保留 patch tokens 或多层特征。

Vision Encoder Shape Audit

Vision Encoder Shape Audit 指上线或训练前检查 vision encoder 的输入输出 shape、patch grid、position embedding、视觉 token 数、projector shape 和成本。

常见检查表：

检查项	典型 shape / 指标	风险
输入图像	[B,C,H,W]	预处理分辨率或通道顺序错误
patch grid	$\text{ceil}(H/P) \times \text{ceil}(W/P)$	token 数和 position embedding 不匹配
patch tokens	[B,N,d_v]	高分辨率导致 token 爆炸
CLS / position	(N+1,d_v)	加不加 CLS 与 checkpoint 不一致
projector	[B,N,d_v] -> [B,N,d_l]	视觉维度和 LLM hidden size 不一致
attention cells	(N+1)^2	显存和延迟超预算

面试表达：vision encoder shape audit 能把“视觉塔能不能用”落到可检查证据上。很多 VLM bug 不是模型不会推理，而是图像预处理、patch 数、位置 embedding 或 projector shape 一开始就错了。

Text Encoder

Text Encoder 是把文本编码为向量或 token 表示的模块。

在 CLIP 类双塔模型中，text encoder 通常和 image encoder 对称存在，输出文本 embedding，用于和图像 embedding 计算相似度。它更偏表示学习，不负责开放式生成。

在多模态大模型中，语言模型通常是 decoder-only LLM，不再只是 text encoder。它既处理文本 prompt，也接收视觉 token，并自回归生成答案。

两类文本模块的差异：

模块	目标	输出	典型场景
Text Encoder	编码文本语义	embedding	CLIP、检索、分类
Decoder-only LLM	条件生成	token 序列	对话、VQA、推理、工具调用

面试表达：CLIP 中的 text encoder 主要用于图文对齐；LLaVA 这类 VLM 中的 LLM 负责把视觉信息和指令结合起来生成自然语言答案。

ViT

ViT 是 Vision Transformer，把图片切成 patch，再像文本 token 一样送入 Transformer。

基本流程：图片被切成固定大小 patch，每个 patch 展平成向量并通过线性层映射到 hidden dimension，加上位置编码后进入 Transformer encoder。

例如 224 x 224 图片，patch size 为 16，则得到 14 x 14 = 196 个 patch token。再加上 cls token 或 pooling，得到全局表示。

ViT 的优点：结构与 Transformer 统一，适合大规模预训练，和语言模型 connector 更容易衔接。

ViT 的缺点：对数据规模敏感；高分辨率输入会增加 token 数；局部归纳偏置弱于 CNN，需要更多数据或改进结构。

在多模态大模型中，ViT 常作为视觉前端。面试中如果问“图片怎么变成 token”，可以回答：通常先用 ViT 将图像切成 patch token，再通过投影层或 resampler 转成 LLM hidden size 对应的视觉 token。

SigLIP

SigLIP 是 CLIP 类图文对齐方法的一个重要变体，使用 sigmoid loss 替代 softmax contrastive loss。

CLIP 的 softmax 对比学习通常在 batch 内做归一化，一个图片对应一个正文本，其他文本作为负样本。SigLIP 将图文匹配建模成独立的二分类问题，对每个 image-text pair 使用 sigmoid binary loss。

简化管理：CLIP 更像“在这一批文本中选出与图片最匹配的那个”，SigLIP 更像“判断每个图文对是否匹配”。

SigLIP 的特点：

方面	CLIP	SigLIP
损失形式	softmax contrastive loss	sigmoid binary loss
归一化范围	batch 内归一化	pair-level 判断
通信依赖	大 batch 训练时跨设备通信重要	对大规模分布式更友好
目标理解	多分类式匹配	多标签式匹配

优点：在大规模训练时更容易扩展，减少对全局 batch softmax 的依赖，实际视觉语言模型中常作为强视觉 encoder 或对齐 backbone。

面试表达：SigLIP 和 CLIP 都做图文对齐，但 SigLIP 用 sigmoid loss，把图文匹配看成 pair-wise 二分类，在大规模训练和分布式场景下更友好。

Vision-Language Model

Vision-Language Model，简称 VLM，是能同时处理视觉和语言的模型。

VLM 可以分为三类：

类型	代表	特点
双塔对齐模型	CLIP、SigLIP	擅长检索和相似度，通常不生成答案
编码器-解码器模型	BLIP、OFA	适合 caption、VQA 等任务
LLM 接视觉模型	Flamingo、LLaVA、MiniGPT-4	用 LLM 进行开放式对话和推理

现代多模态大模型通常采用“vision encoder + connector + LLM”的结构。

image -> vision encoder -> visual tokens -> connector -> LLM hidden space
text -> tokenizer -> text tokens -> embedding -> LLM hidden space

LLM attends over visual tokens and text tokens -> generates answer

核心模块：

模块	作用
Vision Encoder	提取视觉特征
Connector	把视觉特征映射到 LLM 可用空间
LLM	进行指令理解、语言推理和生成
Instruction Data	教模型如何按用户指令使用图片信息

面试表达：VLM 的能力上限不只由 LLM 决定，还受视觉 encoder 分辨率、视觉 token 压缩方式、多模态数据质量和对齐训练影响。

Multimodal Alignment

Multimodal Alignment 指让不同模态在语义层面对齐。

对齐分为粗粒度和细粒度。粗粒度对齐是整张图和整段文本匹配，例如 CLIP。细粒度对齐是图像区域和文本短语匹配，例如“红色杯子”对应图片中某个区域。

常见对齐方式：

方法	说明	适用场景
对比学习	拉近匹配图文，推远不匹配图文	检索、零样本分类
Caption 生成	根据图像生成描述	图像描述、多模态预训练
Matching loss	判断图文是否匹配	图文匹配、rerank
Masked modeling	遮挡部分模态并预测	表示学习
Instruction tuning	用图文指令数据训练回答	多模态对话
Grounding supervision	用区域框或 mask 对齐短语	定位和细粒度理解

对齐不足会导致模型看图不准、答非所问、忽略图片、凭语言先验猜答案或产生多模态幻觉。

面试表达：多模态对齐不是一次性完成的。预训练对齐解决基础语义空间问题，指令微调解决交互方式问题，细粒度 grounding 解决区域和实体对齐问题。

Connector

Connector 是连接视觉 encoder 和 LLM 的模块，也被称为 projector、adapter、resampler 或 bridge。

为什么需要 connector：视觉 encoder 的输出维度、分布和 token 数量通常与 LLM 不一致。LLM 原本只见过文本 embedding，如果直接把视觉特征塞进去，模型很难理解。因此需要 connector 将视觉 token 映射到 LLM hidden space。

常见 connector：

Connector	机制	特点
Linear Projection	线性层映射维度	简单、便宜，LLaVA 早期常用
MLP Projector	多层 MLP 映射	表达能力更强，成本仍较低
Q-Former	learnable queries 通过 cross-attention 查询视觉特征	可压缩视觉 token，BLIP-2 常用
Perceiver Resampler	固定数量 latent query 汇聚视觉特征	适合压缩大量视觉 token，Flamingo 常用
Cross-Attention	LLM 层中插入视觉 cross-attention	融合能力强，但改造和训练成本高

选择 connector 的关键 trade-off：线性投影简单高效，但压缩和对齐能力有限；Q-Former 和 resampler 更强，但结构更复杂、训练更麻烦；cross-attention 融合深，但会改变 LLM 架构。

面试表达：connector 的本质是把视觉 token 翻译成 LLM 能消费的表示。它决定视觉信息是完整保留、压缩后保留，还是在 LLM 内部通过 cross-attention 融合。

VLM Connector Audit

VLM Connector Audit 指检查视觉特征接入 LLM 的 shape、占位符、上下文预算、token 压缩和 label mask 是否一致。

最小检查链路如下：

$$\begin{aligned} &Z_v \in \mathbb{R}^{B \times N_v \times d_v}, \\ &\quad \quad \quad \backslash \text{quad} \\ &H_v = Z_v W_p + b_p, \\ &\quad \quad \quad \backslash \text{quad} \\ &H_v \in \mathbb{R}^{B \times N_v \times d_l} \end{aligned}$$

如果直接拼接视觉 token，上下文长度近似为：

$$T_{\{\mathrm{total}\}} = T_{\{\mathrm{text}\}} + I_{N_v} + N_{\{\mathrm{special}\}}$$

上线或训练前至少检查：图片数量是否等于 image placeholder 数量、projector 输出是否等于 LLM hidden size、多图是否超过上下文上限、assistant-only loss mask 是否只覆盖回答 token、OCR / 图表等高风险样本是否需要更高分辨率或外部工具。

面试表达：VLM connector audit 能把“图像接入 LLM”拆成可验证证据。很多多模态 bug 不是模型能力问题，而是 placeholder、shape、token budget 或 labels 对齐一开始就错了。

Image Placeholder

Image Placeholder 是多模态 prompt 中表示图片插入位置的特殊标记，例如 <image>、<image_1> 或模型自定义 image token。

它有两层含义：文本模板里告诉模型这里有图片，batch collator 中告诉系统把对应 visual tokens 插入到哪个位置。

常见风险：

风险	现象
placeholder 少于图片数	有图片被忽略或插入位置不确定
placeholder 多于图片数	模型以为空缺位置也有视觉证据
多轮引用混乱	回答把不同图片内容混在一起
labels 未同步	visual tokens 插入后 loss mask 错位

面试表达：image placeholder 不是普通文本装饰，它是 prompt、视觉 token 和 label mask 对齐的锚点。

Visual Token Compression

Visual Token Compression 指把大量 patch-level visual tokens 压缩为较少的 query / latent tokens，常见于 Q-Former、Perceiver Resampler、pooling、token pruning 或区域裁剪。

压缩比例可以粗略写为：

$$\rho = \frac{Q}{N_v}$$

其中 Q 是压缩后的 token 数，N_v 是原始视觉 token 数。压缩能降低上下文长度和 attention 成本，但会带来信息瓶颈，尤其影响 OCR、小目标、计数、空间关系和图表读数。

面试表达：视觉 token 压缩不是越强越好。它是在成本和可见证据之间做取舍，任务越依赖局部细节，越要谨慎压缩。

Assistant-Only Multimodal Loss

Assistant-Only Multimodal Loss 指多模态 SFT 中只对 assistant 回答 token 计算 next-token loss，用户问题、system prompt、image placeholder 和 visual tokens 通常不参与 loss。

$$L_{\{\mathrm{sft}\}} = -\frac{1}{\sum_{t=1}^T m_t} \sum_{t=1}^T m_t \log p_{\theta}(y_t \mid y_{1:t-1}, H_v, x)$$

其中 $m_t=1$ 表示第 t 个 token 是 assistant answer。这个设计避免模型学习复述用户 prompt 或把 <image> 等控制 token 当成输出目标。

面试表达：多模态 SFT 的 loss 仍是语言模型 loss，但条件里多了视觉表示；关键工程点是 label mask 必须只覆盖 assistant 回答。

Q-Former

Q-Former 是 BLIP-2 中常见的视觉到语言桥接模块。

核心思想：不用把所有视觉 patch token 都直接送给 LLM，而是引入一组可学习 query token，让这些 query 通过 cross-attention 从视觉 encoder 输出中读取信息。最终得到固定数量的视觉 query 表示，再接入语言模型。

简化流程：

```
image -> frozen vision encoder -> many visual features
learnable queries -> Q-Former cross-attention -> compact visual tokens
compact visual tokens -> projection -> LLM
```

优点：可以压缩视觉 token 数量，降低 LLM 上下文成本；query 可以学习选择对语言任务有用的视觉信息；适合在冻结大型视觉 encoder 和 LLM 的情况下训练桥接模块。

缺点：结构比简单 MLP projector 复杂；query 数量是瓶颈，压缩过强可能丢细节；训练目标和数据设计会显著影响效果。

面试表达：Q-Former 可以理解为用一组可学习查询从图像特征中“问出”语言模型需要的信息，是视觉特征压缩和跨模态对齐的桥。

Perceiver Resampler

Perceiver Resampler 是 Flamingo 等模型中使用的视觉 token 压缩模块。

它使用固定数量的 latent tokens，通过 attention 从大量视觉特征中汇聚信息，将可变长度或大量视觉输入压缩成固定数量的 token。

为什么重要：图片高分辨率或视频多帧输入会产生大量视觉 token。如果全部送入 LLM，计算和显存成本很高。resampler 可以把大量视觉特征压缩成较少 token，方便后续语言模型处理。

和 Q-Former 的共同点：都使用 learnable query/latent token 从视觉特征中抽取信息，都能压缩视觉 token。

差异：Q-Former 更强调作为 BLIP-2 式视觉语言桥接模块，通常配合特定预训练目标；Perceiver Resampler 更像通用的 token 汇聚器，常用于处理多图、多帧和可变长度视觉输入。

面试表达：Perceiver Resampler 解决的是视觉 token 太多的问题，它用固定数量 latent token 汇聚视觉信息，使多图和视频输入更可控。

Flamingo

Flamingo 是 DeepMind 提出的视觉语言模型路线，重点是让大语言模型通过少量多模态示例进行 in-context learning。

核心设计包括：使用视觉 encoder 提取图像特征；使用 Perceiver Resampler 将视觉特征压缩；在冻结或部分冻结的语言模型中插入 gated cross-attention，让语言模型在生成时访问视觉信息。

Flamingo 的特点：

方面	说明
模型结构	vision encoder + resampler + LLM + cross-attention
训练方式	大规模图文和交错图文数据训练
能力重点	多模态 few-shot 和 in-context learning
工程代价	架构改造较大，训练复杂

Flamingo 的启发：多模态模型不一定只做单图问答，也可以处理交错的图文上下文，例如 “图片 A + 描述 + 图片 B + 问题”。这对文档理解、网页理解和多轮视觉对话很重要。

面试表达：Flamingo 的关键在于通过 resampler 压缩视觉输入，并用 gated cross-attention 把视觉信息注入 LLM，使语言模型具备多模态 in-context learning 能力。

BLIP 与 BLIP-2

BLIP 是 Bootstrapping Language-Image Pre-training，代表了一类结合图文理解和生成的视觉语言预训练方法。BLIP 通过 captioning、filtering 和多任务目标提升图文数据质量与模型能力。

BLIP-2 的核心贡献是用 Q-Former 连接冻结的视觉 encoder 和冻结的 LLM，从而降低训练成本。它不从头训练完整多模态大模型，而是学习一个桥接模块，让强视觉模型和强语言模型协同工作。

BLIP-2 常见两阶段思想：先让 Q-Former 学会从图像中抽取和文本相关的信息，再把 Q-Former 输出接到 LLM 上进行视觉到语言生成。

优点：复用已有强模型，训练成本低；Q-Former 能压缩视觉信息；适合构建图像描述和 VQA 模型。

缺点：桥接模块可能成为瓶颈；冻结模型限制端到端适配；复杂场景下仍需要高质量指令数据。

面试表达：BLIP-2 的重点不是单纯 “用了大模型”，而是用 Q-Former 低成本连接冻结 vision encoder 和冻结 LLM，是多模态对齐中的经典桥接方案。

LLaVA

LLaVA 是 Large Language and Vision Assistant 的代表路线，特点是用视觉 encoder 加投影层连接 LLM，并通过多模态指令微调获得视觉对话能力。

典型结构：CLIP ViT 作为视觉 encoder，MLP projector 将视觉特征映射到 LLM hidden size，LLM 负责根据视觉 token 和文本指令生成答案。

训练通常包括两个阶段：

阶段	目标	常见做法
预对齐	让视觉特征进入 LLM 空间	冻结 vision encoder 和 LLM，只训练 projector
指令微调	学会多模态对话和问答	用图文指令数据微调 projector 和部分或全部 LLM

LLaVA 路线的优势：结构简单，复现成本较低，容易接入开源 LLM，指令能力强。

局限：视觉细节能力依赖 vision encoder 和分辨率；简单 projector 可能对复杂空间关系和细粒度 grounding 不足；指令数据质量对输出风格和幻觉影响很大。

面试表达：LLaVA 是把视觉 token 通过 projector 接到 LLM 上，再用多模态指令数据教模型按人类指令看图回答。它代表了低成本构建 VLM 的主流工程路线。

Image Captioning

Image Captioning 是给定图片生成自然语言描述。

它是早期视觉语言任务，也是多模态大模型的重要数据来源和基础能力。caption 可以是简短描述，也可以是细粒度描述，例如主体、背景、动作、关系、文字和风格。

常见训练方式：输入图像，模型自回归生成 caption，以交叉熵作为训练目标。现代模型也会用更复杂的指令形式，例如“请详细描述这张图片”。

评估指标包括 BLEU、METEOR、ROUGE、CIDEr、SPICE 等，但这些指标和人类感知不完全一致。大模型时代更常用人工评测或 LLM-as-judge 辅助评估事实准确性、详细程度和可读性。

工程注意：caption 数据质量很关键。过短 caption 会导致模型只学到粗粒度对象，过度想象的 caption 会诱导幻觉，模板化 caption 会让输出单调。

面试表达：Image Captioning 看似简单，但它决定模型能否把视觉内容转成语言表达。高质量 caption 数据对多模态指令模型非常重要。

Visual Question Answering

VQA 是给定图片和问题，输出答案。

VQA 比 caption 更接近真实交互，因为用户通常不是要求模型完整描述图片，而是问具体问题。例如“图中有几个人？”、“红色车在什么位置？”、“这张图表说明了什么趋势？”。

VQA 难点：

难点	示例
细粒度识别	识别小物体、文字、图标
空间关系	左右、上下、遮挡、包含
计数	准确数人数、物体数量
OCR	读取图片中文字
常识推理	根据场景推断行为和意图
图表推理	读取坐标轴、趋势和数值

常见问题：模型可能不看图，仅凭语言先验回答。例如问“香蕉是什么颜色”，即使图中没有香蕉，模型也可能回答黄色。这就是多模态幻觉和语言先验偏置。

面试表达：VQA 的关键不只是生成流畅答案，而是答案必须被图像证据支持。评估时要区分语言能力、视觉识别能力和跨模态推理能力。

OCR

OCR 是 Optical Character Recognition，即从图像中识别文字。

在多模态大模型中，OCR 是非常重要的基础能力。很多真实业务图片包含文字，例如截图、票据、表格、PPT、网页、菜单、合同和图表。如果 OCR 不准，后续问答和推理都会出错。

OCR 对 VLM 的挑战：

挑战	说明
小字识别	分辨率不足会丢字
复杂排版	表格、双栏、海报、截图结构复杂
多语言	中英文、数字、符号混排
旋转和遮挡	文本角度、模糊、反光影响识别
结构理解	不仅要识别文字，还要理解行列关系

提升 OCR 能力的方式：提高输入分辨率；使用更强视觉 encoder；加入 OCR 专项数据；用文档理解数据训练；必要时接外部 OCR 工具，把识别结果作为文本上下文输入 LLM。

面试表达：OCR 是多模态模型落地的硬指标。很多模型会“看懂大物体但看不清小字”，这在文档问答、图表分析和企业知识库场景中会直接影响可用性。

Grounding

Grounding 指把语言表达与图像中的具体区域、框、点或 mask 对齐。

例如用户说“左边穿红衣服的人”，模型不仅要理解这句话，还要知道它对应图片中的哪个区域。Grounding 是多模态模型从“描述图片”走向“精确操作”的关键能力。

Grounding 相关任务：

任务	输入	输出
Referring Expression Comprehension	图片和短语	目标框
Phrase Grounding	图片和多个短语	短语对应区域
Detection with Text Prompt	图片和类别文本	目标框
Segmentation with Text Prompt	图片和文本	mask
GUI Grounding	截图和操作指令	点击坐标或区域

Grounding 难点：需要细粒度视觉表示、空间关系理解和区域级监督。普通图文 caption 数据通常只提供全局描述，不足以训练强 grounding 能力。

面试表达：没有 grounding 的 VLM 可能会说得很像懂图，但不能指出证据在哪里。对于机器人、GUI agent、工业质检和医疗影像，grounding 比流畅回答更重要。

Multimodal Instruction Tuning

Multimodal Instruction Tuning 指用图像或其他模态加自然语言指令的数据训练模型，使模型学会按照用户指令进行多模态问答、解释和推理。

数据形式通常如下：

```
{
  "image": "example.jpg",
  "conversations": [
    {"from": "human", "value": " 请描述这张图片中的主要内容。"},
    {"from": "assistant", "value": " 图片中有一只狗坐在草地上，旁边有一个蓝色飞盘。"}
  ]
}
```

指令微调数据来源包括人工标注、已有 VQA 数据集改写、caption 数据改写、GPT-4V 等强模型生成、业务日志清洗、合成图表问答、OCR 问答和文档问答。

质量控制要点：

维度	要求
事实一致	答案必须被图片支持
指令多样	覆盖描述、问答、推理、拒答、格式化输出
难度分层	包含简单识别、关系理解、计数、OCR、图表推理
风格稳定	回答清晰，不乱编细节
安全合规	人脸、隐私、医疗、金融等场景谨慎处理

常见问题：用强模型生成数据时，生成器可能会凭空补充图片不存在的细节。如果不做过滤，学生模型会学习幻觉。

面试表达：多模态指令微调解决的是“模型如何和用户交互”的问题。它不能替代底层视觉识别能力，如果 vision encoder 看不到细节，指令微调也只能让模型更会表达，不能凭空恢复信息。

Multimodal SFT Data Audit

Multimodal SFT Data Audit 指在多模态指令微调前，对图文对话样本的格式、任务覆盖、证据支持、拒答边界和 loss mask 做训练前审计。

一条样本可以抽象为：

$$s_i=(M_i,U_i,A_i,T_i,Q_i,R_i)$$

其中 M_i 是媒体集合， U_i 是用户指令或多轮上下文， A_i 是 assistant 目标回答， T_i 是任务类型， Q_i 是质量标签， R_i 是风险或拒答标签。

最小审计项包括：

检查项	目的
image placeholder 数量	保证图片和 prompt 对齐
assistant-only labels	保证 loss 只覆盖回答
total tokens	避免视觉 token 和文本 token 超上下文
task coverage	防止只会 caption 或普通 VQA
evidence support	控制视觉幻觉
refusal boundary	控制看不清、资料不足和高风险场景

面试表达：多模态 SFT 数据不能只看 JSON 格式能否解析，还要证明回答被视觉证据支持、该拒答时会拒答、任务类型覆盖真实使用场景。

Chat Template

Chat Template 是把 system、user、assistant、image placeholder 和分隔符组织成模型训练输入的模板。

在 VLM 中，chat template 还承担 image placeholder 对齐作用。训练和推理模板不一致时，模型可能找不到图片位置、角色边界混乱，或者把 assistant 起始标记当成普通文本。

常见检查包括：<image> 是否在 user 消息中、placeholder 数量是否等于图片数、assistant 起始标记是否一致、多轮分隔符是否稳定、labels 是否跟随模板正确右移。

面试表达：chat template 是数据格式的一部分，不是前端展示文本。训练、评估和推理必须使用一致模板，否则同一个 VLM 可能表现大幅波动。

Multimodal Task Coverage

Multimodal Task Coverage 衡量多模态 SFT 数据是否覆盖目标任务集合，例如 caption、VQA、OCR、chart、grounding、多轮对话和 refusal。

$$C_{\{\mathrm{task}\}}=\frac{\left|\mathrm{mathcal{T}}_{\{\mathrm{train}\}}\cap\mathrm{mathcal{T}}_{\{\mathrm{req}\}}\right|}{\left|\mathrm{mathcal{T}}_{\{\mathrm{req}\}}\right|}$$

如果训练集主要由 caption 组成，模型可能描述图片还可以，但 OCR、计数、空间关系、图表和资料不足拒答都很弱。

面试表达：task coverage 是多模态 SFT 的能力地图。覆盖率不够时，训练 loss 再低也不能说明模型满足业务需求。

Evidence Support Rate

Evidence Support Rate 衡量回答是否被图片、截图、文档或图表中的可见证据支持。

$$S_{\mathrm{support}} = \frac{1}{\sum_{i=1}^N \mathbf{1}[A_i \setminus \mathrm{supported}]}$$

低 evidence support 往往对应多模态幻觉：模型回答流畅，但答案不是从图像证据中来的。OCR、图表、医疗影像、质检和安全审核尤其需要严格检查。

面试表达：VLM 回答不能只看语言是否自然，还要看每个 claim 是否能回到图片证据。证据支持率是比通用流畅度更接近落地可靠性的指标。

Missing Refusal Rate

Missing Refusal Rate 指样本应该拒答或表达不确定，但目标回答仍然硬答的比例。

$$R_{\mathrm{miss_refuse}} = \frac{\sum_i \mathbf{1}[r_i=1 \wedge \hat{r}_i=0]}{\sum_i \mathbf{1}[r_i=1]}$$

在多模态场景中，资料不足包括图片模糊、关键区域被遮挡、分辨率不足、文字不可读、图表单位不清或用户要求图片中不存在的信息。

面试表达：拒答数据不是让模型少回答，而是训练模型在视觉证据不足时别编。missing refusal rate 高，通常会带来严重视觉幻觉。

Multimodal Hallucination

Multimodal Hallucination 指模型生成了与视觉输入不一致的内容。

典型表现：图片中没有某个物体，模型说有；物体颜色、数量、位置说错；图片文字识别错误；把常识当视觉证据；用户问不确定问题时模型强行回答。

产生原因：

原因	说明
语言先验过强	LLM 根据常见搭配猜答案
视觉对齐不足	visual token 没有被正确利用
分辨率不足	小物体和文字看不清
指令数据有幻觉	训练答案包含编造内容
评估偏流畅性	模型学会说得好听而非说得准确
解码过发散	高 temperature 增加不受证据支持的细节

缓解方法：

方法	说明
提升视觉输入质量	高分辨率、截图、多尺度输入
加强细粒度数据	OCR、grounding、计数、区域描述
训练拒答能力	看不清或证据不足时说明不确定
引入 grounding	要求答案引用区域证据
使用偏好优化	惩罚编造细节，奖励忠实答案

方法	说明
工具辅助 评估分项化	接 OCR、检测、分割、检索工具 单独评估存在性、属性、数量、位置和文字

面试表达：多模态幻觉不是单纯语言模型幻觉，而是视觉证据没有正确约束生成。解决它要从数据、结构、分辨率、训练目标和评估体系一起入手。

Diffusion Model

Diffusion Model 是一类通过“加噪”和“去噪”建模数据分布的生成模型。

训练时，先把真实样本 x_0 按噪声日程加到某个时间步 t ，得到 noisy sample x_t ；模型输入 x_t 、 t 和条件 c ，学习预测噪声或去噪方向。生成时，从随机噪声开始，按 scheduler 多步去噪，逐渐得到图片、音频、视频或 latent 表示。

面试表达：diffusion 的关键不是一次生成图片，而是学习一条从噪声到数据的反向去噪路径。

DDPM

DDPM 是 Denoising Diffusion Probabilistic Models 的代表方法，定义了固定的 forward diffusion 和可学习的 reverse denoising。

Forward 过程：

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I)$$

闭式加噪：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

其中 β_t 是噪声强度， $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t$ 是累计信号保留比例。

面试表达：DDPM 训练时可以直接从任意时间步构造 x_t ，不需要真的一步一步加噪，这让训练非常方便。

Forward Diffusion

Forward Diffusion 指人为定义的加噪过程，从真实样本逐步走向近似标准高斯噪声。

关键性质：时间步越大， $\bar{\alpha}_t$ 越小，样本中的真实信号越少，噪声越强。

面试表达：forward 过程不是模型学出来的，它是训练者设定的噪声日程；模型真正学习的是反向去噪。

Reverse Denoising

Reverse Denoising 指模型从 x_t 预测 x_{t-1} 或预测去噪方向的过程。

DDPM 中常用预测噪声来构造反向均值：

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t, c) \right)$$

面试表达：采样时的每一步都不是直接生成完整图片，而是把当前 noisy sample 往更干净的分布方向推一点。

Noise Scheduler

Noise Scheduler 定义 diffusion 中每个时间步的噪声强度和采样更新规则。

它影响三件事：训练时如何从 x_0 构造 x_t ，推理时如何从 x_t 更新到 x_{t-1} ，以及多少采样步能在质量和速度之间取得平衡。

面试表达：scheduler 是 diffusion 的“时间表”。模型权重相同，换 scheduler 或采样步数，也可能显著影响质量、速度和稳定性。

Noise Prediction Loss

Noise Prediction Loss 是 diffusion 入门中最常见的训练目标：模型预测加入 x_0 的噪声 ϵ 。

$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(x_t, t, c) \right\|_2^2 \right]$$

预测噪声之后，可以估计干净样本：

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t, c)}{\sqrt{\bar{\alpha}_t}}$$

面试表达：噪声预测好学，是因为训练时真实噪声已知；预测噪声也能转化为去噪和 x_0 估计。

Classifier-Free Guidance

Classifier-Free Guidance，简称 CFG，是文生图中常见的条件增强方法。

模型同时学习有条件去噪和无条件去噪。采样时把两者相减得到条件方向，再用 guidance scale 放大：

$$\hat{\epsilon}_{\text{cfg}} = \epsilon_{\theta}(x_t, t, \text{varnothing}) + w \left(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \text{varnothing}) \right)$$

w 越大，越强调条件，但过大可能导致过饱和、失真、多样性下降或 prompt 过拟合。

面试表达：CFG 是 prompt 遵循和图像质量之间的旋钮，不是越大越好。

Latent Diffusion

Latent Diffusion 指先用 autoencoder 把图片压缩到 latent space，再在 latent space 中做 diffusion。

直觉流程：

image -> encoder -> latent -> diffusion denoising -> decoder -> image

如果空间分辨率被压缩 f 倍，spatial cell 数大约降到：

$$R_{\{\mathrm{latent}\}} \approx \frac{1}{f^2}$$

面试表达：latent diffusion 的价值是把昂贵的像素空间去噪搬到更小的 latent 空间，使高分辨率生成更可行。

Stable Diffusion

Stable Diffusion 是 latent diffusion 的代表系统，把文生图拆成 text encoder、latent denoiser、scheduler 和 VAE decoder 等模块。

典型推理流程：

prompt -> text encoder -> text embeddings

random latent -> denoiser + scheduler -> clean latent

clean latent -> VAE decoder -> image

面试表达：Stable Diffusion 的关键不是一个单独网络，而是一个 pipeline。VAE 负责压缩和解码，text encoder 负责 prompt 条件，U-Net 或 DiT 负责在 latent space 中去噪，scheduler 负责采样路径。

VAE Compression Ratio

VAE Compression Ratio 指图片进入 latent diffusion 前被 autoencoder 压缩后的空间或元素比例。

空间比例：

$$R_{\{\mathrm{spatial}\}} = \frac{1}{f^2}$$

如果把 latent channel 也算进去，元素比例可以粗略写成：

$$R_{\{\mathrm{elem}\}} = \frac{C_z(H/f)(W/f)}{3HW}$$

面试表达：1/f² 解释空间网格变小，元素比例还要考虑 latent channel；两者都说明 latent diffusion 为什么比像素空间去噪更省。

Negative Prompt

Negative Prompt 是文生图中描述“不希望出现什么”的条件，例如 blurry、low quality、extra fingers。

在 CFG 口径下，negative prompt 通常作为负条件参与方向计算：

$$\begin{aligned} \epsilon_{\{\mathrm{guided}\}} = & \epsilon_{\{\theta(z_t, t, c_{\{\mathrm{neg}\}})\}} \\ & - \\ & \epsilon_{\{\theta(z_t, t, c_{\{\mathrm{pos}\}})\}} \end{aligned}$$

面试表达: negative prompt 不是硬约束, 它只是改变 CFG 的负条件方向。它能缓解常见质量问题, 但不能保证严格消除错误结构或风险内容。

Text-to-Image Pipeline Audit

Text-to-Image Pipeline Audit 是文生图上线或实验前的形状、成本和参数审计。

常见检查项:

1. 图片分辨率、VAE scale、latent shape 是否一致。
2. prompt / negative prompt token 数是否超过 text encoder 上限。
3. CFG 是否导致每步需要正负两次 denoiser 调用。
4. 采样步数、scheduler 和 seed 是否可复现。
5. ControlNet、inpainting 或 image-to-image 的条件 shape 是否和图像对齐。
6. 安全过滤、水印、版权和高风险类别是否覆盖。

面试表达: 文生图 pipeline 审计的核心是把“效果不好”拆成可检查的 shape、成本、条件、采样和安全门禁, 而不是只调 prompt。

ControlNet

ControlNet 给 diffusion 模型加入边缘、深度、姿态、分割图或草图等结构条件。

直觉公式:

$$h_{\mathrm{out}} = h_{\mathrm{base}} + r_l(C)$$

其中 C 是结构条件, $r_l(C)$ 是条件分支产生的残差。

面试表达: prompt 控制语义, ControlNet 控制结构。它解决的是纯文本 prompt 难以精确指定姿态、布局和边缘的问题。

Image-to-Image

Image-to-Image 指以输入图片作为起点, 把图片编码到 latent 后加噪, 再按 prompt 去噪生成新图片。

关键参数是 denoising strength: 强度低时保留原图更多, 强度高时生成自由度更大, 也更容易偏离原图。

面试表达: image-to-image 不是纯文本从零生成, 而是在原图 latent 附近重新采样, 所以它天然适合风格转换、局部重绘前置和构图保留。

Inpainting

Inpainting 是局部重绘: 输入原图、mask 和 prompt, 只重绘或主要改变 mask 区域。

常见难点包括边界融合、光照一致、结构连续和语义不冲突。

面试表达: inpainting 的重点是“哪里能改”和“哪里要保留”的约束, 比普通文生图更依赖 mask 对齐和边界质量。

DALL-E

DALL-E 是 OpenAI 文生图系列。早期 DALL-E 把文本和离散图像 token 统一放进自回归 Transformer, DALL-E 2 论文强调从文本生成 CLIP image latent, 再由 decoder 生成图像。

面试表达: DALL-E 系列体现了文生图路线从离散图像 token 自回归生成, 到更强的图文语义表示、扩散 decoder、编辑和安全机制的演进。

Autoregressive Image Tokens

Autoregressive Image Tokens 指先把图像离散化为 token，再像语言模型一样逐个预测图像 token。

$$p(y_1, \dots, y_M | u) = \prod_{m=1}^M p(y_m | u, y_{1:m-1})$$

面试表达：自回归图像 token 的优点是和 next-token prediction 统一，缺点是分辨率下 token 序列很长，生成成本和 tokenizer 质量压力很大。

Video Generation

Video Generation 是生成连续帧序列的多模态生成任务。它不只是生成多张图片，还要保持时间维度上的身份、纹理、几何、动作、相机运动和物理关系连续。

面试表达：视频生成的核心难点是时间一致性。单帧看起来好，不代表视频好；闪烁、身份漂移和动作跳变会直接破坏结果。

Spatiotemporal Patch

Spatiotemporal Patch 是把视频按时间和空间一起切成 token 的方法。若时间 patch size 为 P_t ，空间 patch size 为 P ，token 数约为：

$$N_{\{\mathrm{st}\}} = \left\lceil \frac{T}{P_t} \right\rceil \left\lceil \frac{H}{P} \right\rceil \left\lceil \frac{W}{P} \right\rceil$$

面试表达：时空 patch 用更少 token 表示视频片段，能降低成本，但太大也会损失快速运动、小目标和细节。

Video Diffusion

Video Diffusion 是在视频像素或视频 latent 上做扩散建模。训练时给视频 latent 加噪，模型在时间步和条件下预测噪声；推理时从随机视频噪声逐步去噪。

常见目标：

$$L_{\{\mathrm{video}\}} = \mathbb{E}_{\{z_0, t, \epsilon, c\}} \left[\left| \epsilon - \epsilon_{\theta}(z_t, t, c) \right|_2^2 \right]$$

面试表达：video diffusion 和 image diffusion 的 loss 直觉类似，难点在于样本多了时间维度，denoiser 要同时建模空间和运动。

Temporal Consistency

Temporal Consistency 指视频相邻帧之间在身份、纹理、颜色、几何和运动上的连续性。

常见审计维度：

1. flicker 是否过大。
2. 主体 identity embedding 是否稳定。
3. 运动轨迹是否平滑。
4. 遮挡后物体是否仍保持存在。

5. 镜头和背景是否连续。

面试表达：时序一致性是视频生成和图像生成最大的分水岭。很多失败不是一帧画坏了，而是帧与帧之间不连贯。

Identity Drift

Identity Drift 指视频中的主体身份随时间漂移，例如同一个人脸、衣服、车辆或角色逐渐变成另一个样子。

面试表达：identity drift 往往说明模型没有稳定维护对象状态，或条件控制、时序建模和训练数据不足。

Flickering

Flickering 指视频帧间亮度、颜色、纹理或局部细节快速抖动。

一个简单代理指标是相邻帧统计量变化：

$$F_{\mathrm{flicker}} = \frac{1}{T-1} \sum_{t=2}^T \left| b_t - b_{t-1} \right|$$

面试表达：flicker 会让单帧质量看起来不错的视频变得不可用，尤其在产品演示、商业短片、影视和虚拟人场景中非常明显。

World Model

World Model 是模型对世界状态和动态规律的内部建模。最小抽象是状态转移：

$$s_{t+1} \sim p_{\theta}(s_{t+1} \mid s_t, a_t)$$

面试表达：视频生成模型可能学到部分物体持久性、运动和遮挡规律，但能生成合理视频不等于具备可交互、可规划、可验证的完整 world model。

Physics Consistency

Physics Consistency 指视频中的物体运动、接触、遮挡、碰撞、重力、光照和透视关系符合基本物理直觉。

常见失败包括脚不接地、物体穿模、球滚动方向异常、遮挡后形状变化、液体和烟雾运动不自然。

面试表达：物理一致性往往需要时间、三维结构和因果关系判断，不能只靠单帧图像指标衡量。

Video Generation Evaluation

Video Generation Evaluation 需要同时评估单帧画质、文本一致性、时序一致性、运动自然度、身份保持、物理合理性、多样性和安全性。

面试表达：视频评估通常要结合自动指标、任务型评估和人工偏好。单一分数很难覆盖 prompt 遵循、时序质量和安全风险。

FVD

FVD 是 Fréchet Video Distance，常用于比较真实视频和生成视频在视频特征空间中的分布差异。

$$\mathrm{FVD} = \left\| \mu_r - \mu_g \right\|_2^2 + \mathrm{Tr} \left(\Sigma_r + \Sigma_g - 2 \Sigma_r \Sigma_g \right)$$

$$\frac{\sigma_g}{2(\sigma_r \sigma_g)^{1/2}}$$

面试表达：FVD 是分布级指标，不能单独证明 prompt 遵循、物理正确或业务可用。

Video Token Audit

Video Token Audit 是视频生成或视频理解系统上线前的 token、latent shape、采样成本和时序质量审计。

常见检查项：

1. 帧数、分辨率、patch size 和时空 patch token 数。
2. 视频 latent shape 和元素量比例。
3. 采样步数、CFG 带来的 denoiser 调用次数。
4. flicker、identity stability、motion smoothness 和 object permanence。
5. world model rollout 误差、物理失败和安全门禁。

面试表达：视频 token audit 能把“视频太贵、太闪、太漂”拆成可检查的数字，而不是只凭主观观感调 prompt。

Audio Generation

Audio Generation 指生成语音、音乐、音效或环境声等时间信号的任务。语音系统常见链路包括 ASR、TTS、speech-to-speech、voice cloning 和 audio codec 建模。

面试表达：音频生成和图像生成不同，输出是时间信号，既要内容正确，也要音色、韵律、节奏、自然度和延迟可控。

Waveform

Waveform 是原始音频采样序列。若采样率为 r ，时长为 D 秒，采样点数约为：

$$N=rD$$

面试表达：waveform 保留最原始信息，但序列很长，很多 ASR / TTS 系统会先转成频谱或 codec tokens。

Log-Mel Spectrogram

Log-Mel Spectrogram 是常见语音特征，把 waveform 转成时间-频率能量表示。

$$M \in \mathbb{R}^{F \times T_{\text{mel}}}$$

其中 F 是 mel bins 数， T_{mel} 是声学帧数。

面试表达：log-mel 特征比原始 waveform 更接近人耳感知，也更适合 encoder 建模，是 Whisper 类 ASR 模型的常见输入。

ASR

ASR 是 Automatic Speech Recognition，把语音转成文本。

$$p(y_1, \dots, y_U | M) = \prod_{u=1}^U p(y_u | y_{1:u-1}, M)$$

面试表达：ASR 的核心是根据音频特征生成 transcript，真实系统要处理口音、噪声、多说话人、低质量录音、标点和专有名词。

Whisper

Whisper 是 OpenAI 提出的 encoder-decoder 语音识别模型。典型流程是：

audio -> log-mel spectrogram -> audio encoder -> text decoder -> transcript

面试表达：Whisper 的关键是大规模多语言弱监督训练和 encoder-decoder 结构，使它具备较强 ASR、翻译和噪声鲁棒性。

TTS

TTS 是 Text-to-Speech，把文本合成为语音。

常见系统需要决定内容、音色、语速、停顿、情感和最终 waveform。现代路线可以用 acoustic model + vocoder，也可以用 codec language model 生成 audio tokens。

面试表达：TTS 不只是读出文字，还要让声音自然、清晰、可控，并满足说话人授权和安全要求。

Vocoder

Vocoder 负责把中间声学表示变成 waveform，例如 mel spectrogram -> waveform 或 codec tokens -> waveform。

面试表达：vocoder 直接影响音质、噪声、音色和自然度。TTS 文本内容对了，但 vocoder 差，听感仍可能不可用。

Voice Cloning

Voice Cloning 是根据参考语音生成相同或相似音色的新语音。

面试表达：voice cloning 的技术点是说话人音色保持、内容准确和韵律控制；安全点是说话人授权、水印、深伪检测和高风险拒绝。

Audio Codec

Audio Codec 把连续音频压缩成离散 tokens，并能解码回 waveform。若 codec 帧率为 q Hz、时长为 D 秒、codebooks 数为 K ，token 数约为：

$$N_{\{\mathrm{codec}\}} = \left\lceil \frac{Dq}{K} \right\rceil$$

面试表达：audio codec 是把连续音频接入语言模型的重要桥梁，但 token rate 高、码本多、质量和语义纠缠都是工程难点。

Speech Token

Speech Token 是从语音或音频 codec 中得到的离散 token，可以像文本 token 一样被 Transformer 建模。

面试表达：speech token 让 TTS、speech-to-speech 和统一多模态模型更容易使用 next-token prediction，但序列长度和实时解码成本很高。

Codec Language Model

Codec Language Model 把 audio codec tokens 当成语言模型序列来生成。

$$p(a_1, \dots, a_M | u) = \prod_{m=1}^M p(a_m | u, a_{1:m-1})$$

面试表达：codec LM 的优点是目标统一，缺点是 audio token 序列长、推理速度压力大，韵律和情感控制也更复杂。

Speech-to-Speech

Speech-to-Speech 指语音输入、语音输出的交互形式。

两条常见路线：

1. Cascade: speech -> ASR -> LLM -> TTS -> speech。
2. End-to-end: speech tokens -> model -> speech tokens。

面试表达：cascade 清晰可靠、易调试；端到端更自然、可能低延迟，但数据、可控性、安全评估和 debug 难度更高。

VAD

VAD 是 Voice Activity Detection，用于判断音频中什么时候有人说话。

面试表达：实时语音助手离不开 VAD。VAD 影响首包延迟、打断、静音截断和误触发。

WER

WER 是 Word Error Rate，ASR 常用词错误率：

$$\mathrm{WER} = \frac{S+D+I}{N_{\mathrm{ref}}}$$

面试表达：WER 统计替换、删除、插入错误，适合评估英文等空格分词明显的转写任务。

CER

CER 是 Character Error Rate，字符错误率：

$$\mathrm{CER} = \frac{S_c+D_c+I_c}{N_{\mathrm{char}}}$$

面试表达：CER 对中文、短命令、拼写错误和细粒度转写分析常更直观。

MOS

MOS 是 Mean Opinion Score，常用于 TTS 或语音质量人工主观评分。

面试表达：MOS 能反映自然度和听感，但成本高、主观性强，最好和内容一致性、说话人相似度、延迟和安全指标一起看。

Audio Token Audit

Audio Token Audit 是语音系统上线前的采样率、声学帧、codec token、WER / CER、延迟和安全门禁审计。

常见检查项：

1. 采样率、时长、采样点数和 log-mel shape。
2. ASR WER / CER、噪声场景和多语言覆盖。
3. codec token rate、码本数、压缩比和上下文长度。
4. cascade latency、streaming 首包延迟和 barge-in。
5. voice cloning 授权、深伪检测、水印和隐私保护。

面试表达：audio token audit 能把语音系统从“听起来还行”拆成可检查的准确率、延迟、token 成本、音质和安全门禁。

Unified Multimodal Model

Unified Multimodal Model 指用统一接口、共享表示或统一架构处理文本、图像、音频、视频等多种模态的模型或系统。

统一可以发生在三层：

1. 接口统一：用户看到一个助手，内部是多个模块级联。
2. 表示统一：不同模态映射到共享 embedding 空间。
3. 架构 / 目标统一：不同模态 token 化后进入统一 Transformer 或统一训练目标。

面试表达：统一多模态不是简单串模型，而是要同时解决 token 成本、跨模态对齐、生成解码、路由、训练目标和安全门禁。

Any-to-Any Multimodal Model

Any-to-Any Multimodal Model 指任意输入模态到任意输出模态，例如 text -> image、speech -> speech、video -> text、image + audio -> text。

$p(Y_{\{M_{\{\mathrm{out}\}}\}} \mid X_{\{M_{\{\mathrm{in}\}}\}})$

面试表达：Any-to-Any 是理想目标，但真实系统常混合使用专用 encoder、中心 LLM、生成模型、TTS 和工具，而不是所有模态都由单一 decoder 完成。

Unified Tokenization

Unified Tokenization 指把文本、图像、音频和视频都变成可被统一模型处理的 token 序列。

总输入 token 数可写成：

$N_{\{\mathrm{in}\}} = N_t + N_i + N_a + N_v + N_s$

面试表达：统一 token 化最大的工程问题是 token rate。图像、长音频、多帧视频的 token 数远高于短文本，会直接推高上下文、显存和延迟。

Early-Fusion Multimodal Transformer

Early-Fusion Multimodal Transformer 指较早把不同模态 tokens 混合进同一序列，让 Transformer 在统一上下文中建模跨模态关系。

面试表达：early fusion 信息交互更直接，但序列长、训练成本高、模态 tokenizer 质量和数据配比都会成为瓶颈。

Center-LLM Multimodal System

Center-LLM Multimodal System 指以 LLM 为推理和调度中心，周围连接 vision encoder、audio encoder、OCR、retriever、image generator、TTS 和工具。

面试表达：中心 LLM 路线工程可控，能继承语言推理和工具调用能力；缺点是感知信息可能被压缩，端到端优化不如统一模型自然。

Multimodal Router

Multimodal Router 根据输入模态、任务和安全要求决定调用 OCR、ASR、video sampler、retriever、image generator、TTS 或 tool executor。

$r = \{\mathrm{Route}\}(S)$

面试表达：统一助手一定需要路由。没有路由，系统要么成本过高，要么错过 OCR / ASR / 视频采样 / 安全审核等必要模块。

Multimodal Loss Mixture

Multimodal Loss Mixture 指统一多模态训练中多个目标的加权组合：

$$L = \sum_{k=1}^K \lambda_k L_k,$$

面试表达：loss mixture 决定不同能力的训练权重。配比不当会导致某些模态退化、生成能力压过理解能力，或安全拒答能力不足。

Modality Conflict

Modality Conflict 指不同模态证据互相冲突，例如图片显示猫但文字说狗、视频和字幕不一致、OCR 文本包含不可信指令。

面试表达：多模态模型不能盲信任任意模态。冲突时应说明证据来源、不确定性和优先级，并隔离不可信媒体中的指令性内容。

Unified Multimodal Audit

Unified Multimodal Audit 是统一多模态系统上线前的 token、路由、loss mixture、输出预算和安全门禁审计。

常见检查项：

1. 文本、图像、音频、视频和 special token 总数是否超上下文。
2. 输出文本、音频、图像、工具调用 token 是否超预算。
3. OCR、ASR、video sampler、TTS、image generator 和工具路由是否匹配任务。
4. 训练 loss mixture 是否覆盖目标能力。
5. 不可信媒体指令、未授权 voice cloning、未审核生成输出和工具越权是否被阻断。

面试表达：统一多模态 audit 能把“系统很强”拆成可验证的 token 成本、路由正确性、训练覆盖和安全边界。

Multimodal Evaluation

Multimodal Evaluation 指同时评估多模态系统的感知、跨模态对齐、推理、生成质量、忠实性和安全性。

常见评估面包括：

1. VQA：物体、属性、计数、空间关系和常识推理。
2. OCR / ASR：图片文字和语音内容是否识别正确。
3. Chart QA：坐标轴、图例、单位、趋势和数值计算。
4. Grounding：语言答案是否能定位到区域、框、点或 mask。
5. Faithfulness：回答中的 claim 是否被图片、音频或视频证据支持。
6. Generation：图像 / 视频 / 语音生成的质量、一致性、多样性和安全性。
7. Safety：不可信媒体指令、隐私、身份、deepfake、工具越权和 provenance。

面试表达：多模态评估不能只看一个平均 benchmark 分数。真实系统要把任务成功率、证据支持、幻觉率、过度拒答、安全拦截、延迟和成本放在同一张表里看。

Multimodal Incident

Multimodal Incident 指多模态系统因为输入预处理、OCR、ASR、抽帧、grounding、跨模态冲突、安全过滤、确认流程或预算控制失败，导致答案错误、安全漏检、误操作或用户体验事故。

排查重点：要保存原始媒体、预处理后媒体、中间转换结果、模型最终可见证据、回答 claim、引用区域 / 时间戳、安全判断和预算字段。只看最终回答无法定位错误来自哪条模态边界。

Input Fidelity Rate

Input Fidelity Rate 指原始媒体经过 resize、crop、压缩、抽帧、降噪、切片或转码后，关键证据仍被保留的比例。

典型失败：长截图底部被裁掉，小字被压缩不可读，视频关键瞬间没抽到，音频降噪把专有名词削弱。输入保真失败时，后面的 VLM、OCR、ASR 和 LLM 很难补救。

Multimodal Evidence Recall

Multimodal Evidence Recall 指标准视觉、文字、语音或视频证据中，有多少真正进入系统中间表示和最终上下文。

产品意义：多模态系统不是“看过文件”就算有证据。证据必须被 OCR / ASR / 抽帧 / 检索 / grounding 链路保留下来，并能被最终回答引用。

OCR Critical Field Accuracy

OCR Critical Field Accuracy 指金额、日期、姓名、地址、错误码、表格列名、单位等业务关键字段的识别准确率。

产品意义：普通 OCR 字符准确率较高也可能掩盖关键字段错误。财务、合同、医疗、客服截图和表格任务要单独统计关键字段。

ASR Critical Field Accuracy

ASR Critical Field Accuracy 指语音转写中的金额、日期、姓名、地址、订单号、操作对象等关键字段准确率。

产品意义：ASR 错一个数字或专有名词，后续 LLM 可能生成错误工具参数。写操作前应复述确认关键字段。

Temporal Evidence Recall

Temporal Evidence Recall 指视频、音频或多模态时间序列中，关键事件或片段被抽帧、转写、检索和上下文构造覆盖的比例。

典型失败：固定间隔抽帧漏掉短暂事件，字幕和画面不同步，长视频摘要只覆盖开头和结尾。视频理解要引用时间范围，而不是只给笼统结论。

Multimodal Incident Gate

Multimodal Incident Gate 是面向多模态项目事故复盘和回归测试的门禁，覆盖 input fidelity、evidence recall、OCR / ASR critical fields、temporal evidence recall、grounding IoU、evidence support、hallucination rate、safety block、confirmation coverage、budget overrun 和 P95 latency。

面试表达：多模态项目事故复盘要说清楚错误来自输入保真、模态转换、时间采样、grounding、生成、安全还是预算。不能把所有错误都归因成 VLM 不准。

VQA Accuracy

VQA Accuracy 是视觉问答任务常用分数。开放式 VQA 通常有多个标注答案，模型答案与越多人工答案一致，分数越高。

简化口径：

$$s_i^{\{\mathrm{vqa}\}} = \min\left(\frac{n_i}{3}, 1\right)$$

其中 n_i 是模型答案与人工参考答案匹配的数量。

面试表达: VQA accuracy 适合看开放问答平均能力, 但要配合答案标准化、分题型切片和错误归因; 它不能替代 OCR、grounding 和幻觉评估。

Chart Relaxed Accuracy

Chart Relaxed Accuracy 指图表问答中的数值容忍评估。因为图表读数可能有小误差, 评估时常允许一定相对误差。

$$\begin{aligned} \Delta_i &= \frac{|p_i - g_i|}{\max(|g_i|, \epsilon)} \\ s_i^{\text{chart}} &= \mathbb{1}[\Delta_i \leq \tau] \end{aligned}$$

其中 p_i 是预测值, g_i 是参考值, τ 是容忍阈值。

面试表达: 图表评估要同时看数值、单位、图例、趋势和计算步骤。relaxed accuracy 只能处理小数值误差, 不能证明模型真正理解图表结构。

Grounding IoU

Grounding IoU 用于评估语言表达对应的图像区域是否定位正确。

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}$$

常见做法是统计 IoU 超过阈值的比例:

$$\begin{aligned} \text{Prec}@ \tau &= \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{IoU}(B_i^p, B_i^g) \geq \tau] \end{aligned}$$

面试表达: Grounding IoU 能把“模型说看见了什么”落到视觉证据上。它对 GUI agent、文档理解、视觉幻觉治理和高风险审核都很关键。

Multimodal Hallucination Rate

Multimodal Hallucination Rate 指回答中不被媒体证据支持的 claim 占比。

$$R_{\text{hall}} = \frac{\sum_i \sum_j u_{ij}}{\sum_i M_i}$$

其中 $u_{ij}=1$ 表示第 i 个样本的第 j 条 claim 不被图片、音频或视频证据支持。

面试表达: 多模态幻觉不是“答错”这么简单, 而是模型把不存在、看不清、听不清或证据不足的信息说成确定事实。治理时要做 claim-level evidence check 和拒答边界评估。

Multimodal Prompt Injection

Multimodal Prompt Injection 指恶意或不可信指令隐藏在图片、文档截图、网页、音频或视频中, 诱导模型把媒体内容当成更高优先级指令。

典型例子包括 OCR 读到网页截图里的越权指令、文档中要求模型忽略系统规则、音频中夹带工具调用命令等。

面试表达: 防护原则是外部媒体内容只能作为数据, 不应覆盖 system / developer 指令。工具调用前还要做权限检查、参数校验、二次确认和 audit trace。

Biometric Identity Safety

Biometric Identity Safety 指多模态系统在处理人脸、声音、步态、虹膜等身份相关信号时的安全边界。

高风险问题包括未授权人脸识别、根据图片识别真实人物、声音克隆、名人或普通人 deepfake、隐私属性推断和身份欺诈。

面试表达：多模态安全不能只看有害文本。人脸和声音属于高度敏感信号，系统应区分一般描述、身份识别、授权验证、未授权生成和高风险欺诈场景。

Content Provenance

Content Provenance 指记录内容来源、生成过程、编辑历史和可信凭证，用于判断图片、音频或视频是否为 AI 生成、是否被编辑以及是否可追溯。

常见机制包括水印、内容凭证、签名 metadata、生成日志、模型版本记录和审核链路。

面试表达：provenance 不能单独解决所有 deepfake 问题，但它能为内容分发、审核、溯源和责任界定提供证据链。它应和检测模型、用户授权和人工审核一起使用。

Multimodal Safety Audit

Multimodal Safety Audit 是多模态系统上线前的安全审计，覆盖不可信媒体指令、隐私、身份、deepfake、生成内容审核、工具权限、过度拒答和事故回溯。

一个简化门禁可以写成：

```
G_{\mathrm{mm\_safe}}=
\mathbb{1}[
A_{\mathrm{clean}}\geq \alpha
\land
R_{\mathrm{hall}}\leq \beta
\land
R_{\mathrm{inj}}\leq \gamma
\land
R_{\mathrm{priv}}\leq \eta
\land
R_{\mathrm{over}}\leq \rho
]
```

面试表达：多模态安全 audit 要同时证明两件事：正常任务还能完成，高风险媒体输入、身份请求和越权工具调用会被正确拦截。

Multimodal Interview Readiness

Multimodal Interview Readiness 指候选人是否能把多模态知识从术语背诵提升到可解释、可手算、可写 demo、可讲项目和可讨论安全边界的面试状态。

常见检查项：

1. 能否讲清 CLIP / VLM / diffusion / video / audio / unified multimodal 的主线。
2. 能否写出关键公式和 shape，例如 CLIP loss、视觉 token、connector shape、DDPM、CFG、WER、IoU 和安全门禁。
3. 是否做过 0 依赖 demo 或最小可运行审计脚本。
4. 能否用 bad case 分析幻觉、OCR 错误、grounding 错误、时序不一致和 prompt injection。
5. 能否讲清成本、质量、安全、模块化和端到端训练之间的 trade-off。

面试表达：多模态面试准备度不是“看过多少模型名”，而是能把输入输出、结构、loss、评估、失败模式和项目证据串成完整回答。

Multimodal Interview Rubric

Multimodal Interview Rubric 是多模态面试自评规约，用于给每道题打分并定位短板。
评分维度可以包括：

维度	高分回答特征
机制	能讲清输入、输出、结构和训练目标
公式	能写出核心公式或 shape
demo	能说出自己写过的最小代码或审计脚本
trade-off	能比较成本、质量、延迟和安全
风险	能覆盖幻觉、隐私、身份和不可信媒体指令
项目	能把项目讲成问题、方案、指标、失败和改进

面试表达：rubric 的价值是把“感觉掌握了”变成可复盘分数，避免只复习自己熟悉的题。

Multimodal Formula Coverage

Multimodal Formula Coverage 指多模态高频面试公式的覆盖率。

$$C_{\mathrm{formula}} = \frac{|F_{\mathrm{covered}}| \cap F_{\mathrm{required}}|}{|F_{\mathrm{required}}|}$$

必备公式通常包括 CLIP 相似度 / loss、视觉 token 数、connector shape、assistant-only loss、DDPM forward、CFG、latent ratio、video tokens、WER、unified tokens、VQA accuracy 和 safety gate。

面试表达：公式覆盖不是为了背复杂推导，而是让回答能落到可检查的数量关系、shape 和门禁上。

Multimodal Demo Coverage

Multimodal Demo Coverage 指候选人是否为核心多模态主题准备了可运行或可手算 demo。

$$C_{\mathrm{demo}} = \frac{|D_{\mathrm{covered}}| \cap D_{\mathrm{required}}|}{|D_{\mathrm{required}}|}$$

推荐 demo 包括 CLIP 检索、VLM shape audit、多模态 SFT data audit、diffusion audit、video / audio audit、unified multimodal audit 和 evaluation / safety audit。

面试表达：demo coverage 能证明你不是只会说概念，而是能把概念转成输入、输出、指标和检查项。

Weak Multimodal Question

Weak Multimodal Question 指 mock interview 中得分低、追问容易断、缺少公式或缺少项目证据的多模态问题。

常见弱项包括：audio codec、voice cloning 安全、统一多模态 loss mixture、FVD / FID 边界、图表评估、grounding 证据、项目 bad case 复盘。

面试表达：弱题不是失败记录，而是下一轮复习的索引。每个弱题都要绑定一个公式、一个 demo 和一个 3 分钟回答模板。

Multimodal Revision Plan

Multimodal Revision Plan 是多模态面试复盘后的补齐计划。

一个有效计划至少包含：

1. 缺失公式清单。
2. 缺失 demo 清单。
3. 低分题清单。
4. 项目讲述改进点。
5. 下一次 mock interview 的验证标准。

面试表达：复盘计划要能被验证。例如 “补 VQA accuracy 和 connector shape 公式” “补 eval safety audit demo” “重练 audio generation 和 multimodal safety 两道题”，比 “继续复习多模态” 更可执行。

多模态评估

多模态评估要同时看感知、对齐、推理和生成质量。

常见评估维度：

维度	关注点	示例
物体识别	是否识别主要实体	图中有什么
属性识别	颜色、材质、状态	杯子是什么颜色
空间关系	左右、上下、远近	猫在桌子上还是下
计数能力	数量是否准确	有几辆车
OCR	文字读取是否正确	招牌上写了什么
图表理解	坐标轴、趋势、数值	哪一年增长最快
Grounding	语言是否能定位区域	指出红色按钮
推理	结合视觉和常识推断	为什么这个人可能在等车
忠实性	是否编造不存在内容	图片里没有就不能说有

自动 benchmark 有帮助，但不能完全替代人工评测。很多多模态答案开放性强，单一准确率无法反映是否忠实、是否详细、是否符合业务要求。

工程评估建议：为目标场景构建小而精的 golden set，包含真实图片、边界样本、低质量图片、OCR 样本、多图样本和拒答样本。每次模型升级都固定评测，避免只看公开榜单。

面试表达：多模态评估不能只问 “模型答对了吗”，还要问 “答案是否被视觉证据支持”。尤其在业务落地中，幻觉率和拒答质量往往比平均分更重要。

多模态工程取舍

多模态模型落地时，常见取舍如下：

取舍	选择 A	选择 B	判断标准
分辨率	低分辨率	高分辨率	延迟成本 vs 细节/OCR
视觉 token	多 token	压缩 token	细节保留 vs 上下文成本
训练方式	冻结 encoder/LLM	端到端微调	成本稳定性 vs 适配上限
数据来源	公开数据	业务数据	泛化能力 vs 场景效果
结构	projector	Q-Former/resampler/cross-attention	简单高效 vs 融合能力
推理	单模型回答	工具增强	简洁部署 vs 可靠性
评估	公开榜单	场景 golden set	横向比较 vs 真实可用性

常见落地路线：如果只是做图片问答 demo，可以选开源 VLM 直接微调；如果做企业文档理解，需要重点优化 OCR、版面结构和图表推理；如果做 GUI agent，需要 grounding 和坐标输出；如果做内容审核，需要安全分类、证据定位和低幻觉。

面试表达：多模态工程不是模型越大越好。要根据任务判断瓶颈是视觉识别、OCR、grounding、语言推理、延迟还是数据质量，然后选择对应优化方向。

高频面试题

1. CLIP 为什么能做零样本分类?

CLIP 学到了图像和文本的共享语义空间。分类时可以把类别名写成文本 prompt，编码成文本向量，再和图片向量计算相似度。相似度最高的类别就是预测结果。

关键点是 CLIP 训练时见过大量图文对，学会了图片语义和自然语言描述之间的对应关系，所以不需要针对每个分类任务重新训练分类头。

2. CLIP 和 LLaVA 的区别是什么?

CLIP 是双塔对齐模型，主要输出图像和文本 embedding，适合检索、分类和相似度匹配，不直接生成回答。

LLaVA 是视觉语言助手，通常用 CLIP ViT 提取视觉特征，再通过 projector 接入 LLM，通过多模态指令微调生成自然语言答案。

一句话：CLIP 解决图文表示对齐，LLaVA 解决看图对话和指令跟随。

3. 为什么 VLM 需要 connector?

因为视觉 encoder 输出的特征维度、分布和 token 数量与 LLM 的文本 embedding 不一致。connector 负责把视觉特征映射到 LLM hidden space，并在必要时压缩视觉 token。

如果没有 connector，LLM 很难直接消费视觉特征。不同 connector 的差异主要体现在表达能力、压缩能力、训练成本和是否改造 LLM 架构。

4. Q-Former 和简单 MLP projector 有什么区别?

MLP projector 直接把视觉 token 映射到 LLM hidden size，结构简单、训练便宜，但对视觉信息选择和压缩能力有限。

Q-Former 使用可学习 query 通过 cross-attention 从视觉特征中抽取信息，可以把大量视觉特征压缩成固定数量 query token，更适合低成本桥接强 vision encoder 和强 LLM。

5. 多模态幻觉怎么解决?

先定位幻觉类型：是物体存在性错误、属性错误、数量错误、OCR 错误、空间关系错误，还是推理编造。

再针对原因处理：如果看不清，提升分辨率或裁图；如果对齐差，改进 connector 和多模态训练；如果数据有幻觉，清洗指令数据；如果语言先验太强，加入拒答和忠实性偏好数据；如果需要精确定位，引入 grounding 或工具。

面试中要强调：不能只靠 prompt 解决系统性幻觉，必须结合数据、模型结构、训练目标和评估闭环。

6. 多模态模型为什么 OCR 经常不好?

原因包括输入分辨率不足、小字信息在 patch 化和压缩中丢失、训练数据 OCR 样本不足、模型更偏自然图像语义而非文档结构，以及视觉 token 数量受限。

解决方式包括高分辨率输入、局部裁剪、多尺度视觉特征、加入文档和 OCR 数据、接外部 OCR 工具、使用专门文档理解模型。

7. 视觉 token 太多怎么办?

可以降低分辨率、增大 patch size、使用 pooling、Q-Former、Perceiver Resampler、token pruning、区域裁剪、多阶段粗到细推理，或者只把任务相关区域送入模型。

取舍是：压缩越强，成本越低，但细节越容易丢。OCR、计数、grounding 等任务通常不能过度压缩。

8. 多模态指令微调数据怎么构造？

需要覆盖描述、问答、OCR、计数、空间关系、图表理解、拒答、安全和多轮对话。数据答案必须忠实于图像，不能凭空扩写。可以用人工标注、已有数据集改写、强模型生成再过滤、业务日志清洗和合成数据混合构造。高质量数据比盲目扩大数据量更重要，因为带幻觉的数据会直接教坏模型。

本章小结

多模态学习的核心是让不同模态在统一语义空间中对齐、融合和生成。CLIP 和 SigLIP 代表图文对齐路线，擅长检索和零样本分类。ViT 是常见视觉 encoder，将图片转成 patch token。Q-Former、Perceiver Resampler 和 MLP projector 是常见 connector，用于把视觉特征接入语言模型。Flamingo、BLIP-2 和 LLaVA 代表不同视觉语言模型路线。落地时，最重要的问题通常不是模型会不会说话，而是视觉证据是否真的被看见、对齐和忠实使用。OCR、grounding、计数、图表理解和多模态幻觉，是面试和工程中最容易暴露模型真实水平的地方。面试最终表达：多模态大模型可以理解为“视觉 encoder 负责看，connector 负责翻译，LLM 负责推理和表达，数据与评估负责约束它不要乱说”。

Q. 图像生成与视频生成

条目：Diffusion Model、Forward Diffusion、Reverse Denoising、Noise Scheduler、U-Net、VAE、Latent Diffusion、Stable Diffusion、Classifier-Free Guidance、DALL·E、Autoregressive Image Generation、Video Diffusion、Video Token、Spatiotemporal Patch、World Model、Sora、Temporal Consistency、Motion Prior、Text-to-Image、Text-to-Video。

本章速览

一句话定义：图像生成与视频生成是让模型根据文本、图像、布局、草图、参考图或其他条件，生成符合语义、视觉质量和时空一致性的视觉内容。大模型语境下，图像生成的主流路线包括扩散模型、自回归图像生成和多模态统一建模。视频生成在图像生成基础上进一步要求时间连续、运动合理、物理一致和长程可控。核心问题：生成模型不仅要“画得像”，还要满足用户指令、保持主体一致、避免文字和手部错误、控制风格与构图，并在视频中保持帧间一致、动作自然和镜头逻辑合理。常见技术栈：文本 encoder 理解 prompt，生成 backbone 负责从噪声或 token 中生成视觉表示，VAE 或 tokenizer 负责在像素空间和潜空间之间转换，scheduler 或采样器控制生成过程，guidance 机制控制条件遵循程度。面试表达：图像生成主要看语义对齐、视觉保真和可控性；视频生成还要看时序一致性、运动建模和长视频稳定性。扩散模型是当前图像和视频生成的重要基础，但并不是唯一路线。

Diffusion Model

Diffusion Model，扩散模型，是通过逐步加噪和逐步去噪学习数据分布的生成模型。直观理解：训练时把真实图片一步步加噪，直到接近纯噪声；模型学习在任意噪声程度下如何预测噪声或还原干净图像。生成时从随机噪声开始，经过多步去噪，逐渐生成清晰图片。扩散模型包含两个过程：

过程	作用	是否学习
Forward Diffusion	把真实数据逐步加噪	通常固定，不需要学习
Reverse Denoising	从噪声逐步恢复数据	需要神经网络学习

为什么提出扩散模型：GAN 生成速度快，但训练容易不稳定、模式崩塌；VAE 稳定但生成质量容易偏模糊；自回归模型虽然建模清晰但高分辨率图像生成成本高。扩散模型用稳定的去噪训练目标获得高质量生成效果，因此成为图像生成的重要路线。

优点：训练稳定、生成质量高、覆盖模式较好、易于条件控制、适合图像编辑和超分辨率等任务。

缺点：多步采样导致推理慢；高分辨率生成成本高；文本和图像对齐依赖数据和条件机制；精确结构、文字、数量、手部和复杂空间关系仍可能出错。

面试表达：扩散模型的本质是学习反向去噪过程。训练时模型看到带噪样本和噪声强度，学习预测噪声或干净样本；生成时从随机噪声迭代去噪。

Forward Diffusion

Forward Diffusion 是从真实样本到噪声的加噪过程。

给定真实图像 x_0 ，逐步加入高斯噪声得到 $x_1, x_2, \dots, x_T$ 。当 T 足够大时， x_T 接近标准高斯噪声。

常见形式可以写成：

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

其中 β_t 是第 t 步的噪声强度。噪声强度如何随时间变化，就是 noise schedule。

一个重要性质是可以从 x_0 直接采样任意时刻 x_t ：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

这使训练更高效，因为模型不需要真的一步步加噪，而是随机采样一个时间步 t ，直接构造带噪样本。

面试表达：正向扩散通常是固定的马尔可夫加噪过程，它定义了训练样本如何被破坏；模型真正要学的是反向去噪。

Reverse Denoising

Reverse Denoising 是从噪声恢复数据的生成过程。

训练时，模型接收带噪样本 x_t 、时间步 t 和条件 c ，预测噪声 ϵ 、干净样本 x_0 或 velocity v 。最经典的训练目标是噪声预测：

$$L = \|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2$$

生成时，从随机噪声 x_T 开始，模型反复预测并去除噪声，得到 $x_{T-1}, x_{T-2}, \dots, x_0$ 。

常见预测参数化：

参数化	模型预测	特点
epsilon prediction	噪声	经典 DDPM 常用，直观简单
x0 prediction	干净样本	直接预测最终图像，某些场景更直观
v prediction	velocity	在部分高质量扩散模型中常用，稳定性好

去噪过程中的条件可以来自文本、类别、图片、mask、深度图、边缘图、姿态图等。Text-to-Image 中最常见条件是文本 embedding。

面试表达：反向去噪不是一次生成完整图片，而是在多个噪声等级上逐步修正。每一步都利用模型预测当前噪声中应该去掉什么。

Noise Scheduler

Noise Scheduler 定义扩散过程中的噪声强度、时间步和采样更新规则。

训练 scheduler 决定不同时间步加多少噪声。采样 scheduler 决定如何从 x_t 更新到 x_{t-1} ，以及使用多少步生成最终图像。

常见采样器或 scheduler 包括 DDPM、DDIM、DPM-Solver、Euler、Heun、UniPC 等。不同 scheduler 会影响生成速度、细节、稳定性和 prompt 遵循。

核心取舍：

选择	优点	缺点
多步采样	质量更稳，细节更好	推理慢
少步采样	延迟低，成本低	容易细节差或不稳定
随机采样	多样性好	可复现性和稳定性弱
确定性采样	结果更可控	多样性可能降低

工程中常见参数包括 sampling steps、CFG scale、seed、scheduler 类型、分辨率、negative prompt 等。调参时不能只看单张样例，要固定 seed 和测试 prompt 集合做对比。

面试表达：scheduler 不改变模型权重，但会明显影响采样速度和质量。扩散模型部署时，采样器选择和步数压缩是重要优化点。

U-Net

U-Net 是早期扩散模型中最常见的去噪网络结构。

它包含下采样路径、瓶颈层和上采样路径，并通过 skip connection 保留多尺度细节。图像生成需要同时理解全局语义和局部纹理，U-Net 的多尺度结构很适合这个任务。

在文本到图像扩散模型中，U-Net 通常会接收三类输入：带噪 latent 或图像、时间步 embedding、文本条件 embedding。文本条件常通过 cross-attention 注入 U-Net。

U-Net 的优势：多尺度建模强、局部细节好、在图像任务上成熟稳定。

U-Net 的局限：扩展到很大规模和长程依赖时不如 Transformer 自然；视频生成中需要额外加入时间维度或时序模块；高分辨率和长视频成本较高。

发展趋势：很多新模型开始使用 Diffusion Transformer，也就是 DiT，用 Transformer 处理图像 patch 或 latent token，以获得更好的 scaling 能力。

面试表达：Stable Diffusion 早期核心去噪网络是 U-Net，文本通过 cross-attention 影响去噪过程。新一代模型越来越多使用 Transformer 化的扩散 backbone。

VAE

VAE 在 Stable Diffusion 类模型中常用于像素空间和 latent space 的转换。

直接在像素空间做扩散计算量很大。例如 512 x 512 x 3 的图片维度很高。Latent Diffusion 先用 VAE encoder 把图片压缩到较低分辨率 latent，再在 latent 空间扩散，最后用 VAE decoder 还原为图片。

简化流程：

```
training image -> VAE encoder -> latent z
latent z -> add noise -> denoising model -> clean latent
clean latent -> VAE decoder -> generated image
```

VAE 的作用不是理解 prompt，而是压缩和重建图像。文本理解通常由 text encoder 完成，去噪由 U-Net 或 DiT 完成。

优点：显著降低扩散计算成本，使高分辨率生成更可行。

缺点：VAE 压缩会带来重建损失，可能影响细节、文字、纹理和边缘；VAE decoder 质量也会限制最终图片质量。

面试表达：Stable Diffusion 之所以叫 latent diffusion，是因为扩散发生在 VAE 压缩后的 latent 空间，而不是原始像素空间。

Latent Diffusion

Latent Diffusion 是在潜空间中进行扩散建模的路线。

核心思想：先训练一个 autoencoder 或 VAE，把图片压缩为 latent 表示；再在 latent 上训练扩散模型；生成完成后解码回像素空间。

相比 pixel diffusion，latent diffusion 的优势是计算成本低、显存占用小、生成速度更快，可以用较低成本训练和部署高分辨率图像生成模型。

代价是 latent 表示不是无损的。对于文字、细小图案、复杂纹理和精确几何，latent 压缩可能导致信息损失。

面试表达：Latent Diffusion 的核心 trade-off 是用压缩潜空间换取效率，但压缩器质量会成为生成质量上限的一部分。

Stable Diffusion

Stable Diffusion 是开源影响力最大的文本到图像生成模型路线之一，典型结构是 text encoder + latent diffusion U-Net + VAE。

基本生成流程：

```
prompt -> tokenizer -> text encoder -> text embeddings
random noise latent + text embeddings -> denoising U-Net -> clean latent
clean latent -> VAE decoder -> image
```

主要组件：

组件	作用
Text Encoder	编码 prompt，提供文本条件
U-Net / DiT	在 latent 空间预测噪声或去噪方向
VAE Encoder	训练时把图片压缩到 latent
VAE Decoder	生成时把 latent 解码成图片
Scheduler	控制采样更新过程
CFG	增强文本条件遵循

Stable Diffusion 的影响：开源生态丰富，支持 LoRA、DreamBooth、ControlNet、Inpainting、Outpainting、img2img、风格迁移、局部重绘和各类插件。

常见能力扩展：

扩展	目标
LoRA	低成本学习人物、风格或特定概念
DreamBooth	个性化主体生成
ControlNet	用边缘、深度、姿态等控制结构
Inpainting	根据 mask 重绘局部区域
img2img	基于参考图生成变体
IP-Adapter	参考图风格或主体条件注入

面试表达：Stable Diffusion 的核心不是一个单独网络，而是一套 latent diffusion 系统。文本条件控制语义，去噪模型生成 latent，VAE 负责像素重建，scheduler 控制采样。

Classifier-Free Guidance

Classifier-Free Guidance, 简称 CFG, 是扩散模型中常用的条件引导方法, 用于增强生成结果对 prompt 的遵循。

训练时, 模型有时接收真实条件, 有时把条件置空, 学会条件生成和无条件生成。采样时, 同时计算 conditional prediction 和 unconditional prediction, 再组合:

$$\text{pred} = \text{pred_uncond} + \text{scale} * (\text{pred_cond} - \text{pred_uncond})$$

其中 scale 是 CFG scale。scale 越大, 模型越强烈地朝 prompt 条件方向生成。

CFG scale 的影响:

CFG scale	结果倾向
太低	prompt 遵循弱, 可能更随机
适中	语义和视觉质量平衡
太高	画面可能过饱和、僵硬、细节异常或 artifacts 增多

CFG 的价值: 不需要额外训练 classifier, 就能增强条件控制, 是文本到图像扩散模型的核心技巧之一。

面试表达: CFG 本质是在采样时放大 “有条件预测” 和 “无条件预测” 的差异, 让生成结果更贴近文本条件, 但过强 guidance 会牺牲自然度和多样性。

DALL·E

DALL·E 是 OpenAI 的图像生成系列模型, 代表了从文本生成图像的重要路线。

早期 DALL·E 使用离散图像 token 和自回归 Transformer, 把图像生成看成 token 序列生成问题。DALL·E 2 引入 CLIP 表示和扩散 decoder。后续系统更强调 prompt 遵循、文字渲染、图像编辑和与 ChatGPT 式交互结合。

在面试中, 不需要死记每个版本细节, 关键是理解 DALL·E 代表了文本到图像生成的标志性系统, 并且图像生成路线可以是自回归 token 建模, 也可以是扩散建模, 也可以是混合式建模。

面试表达: DALL·E 系列说明文本到图像生成并不只有 Stable Diffusion 一种实现。核心都是把自然语言条件转化为视觉生成过程的控制信号。

Autoregressive Image Generation

Autoregressive Image Generation 是把图像离散化为 token, 然后像语言模型一样逐 token 生成图像。

基本流程: 先用 VQ-VAE、VQGAN 或视觉 tokenizer 把图片编码成离散 token 序列, 再用 Transformer 根据文本条件或上下文自回归预测下一个图像 token, 最后解码成图片。

优点: 训练目标和语言模型统一, 天然适合多模态 token 统一建模; 可以和文本 token 放在同一序列中; 概率建模清晰。

缺点: 高分辨率图像 token 数量巨大, 逐 token 生成慢; 视觉细节依赖 tokenizer; 长序列建模成本高。

与扩散模型对比:

方面	自回归图像生成	扩散图像生成
建模方式	逐 token 预测	逐步去噪
训练目标	next token prediction	denoising loss
生成速度	token 多时较慢	多步采样较慢, 可蒸馏加速
优势	与 LLM 统一, 概率建模清晰	图像质量高, 编辑和控制成熟
难点	tokenizer 和长序列成本	采样速度和精确控制

面试表达：自回归图像生成把图片当成离散语言来生成，扩散模型把生成看成连续去噪过程。二者各有优势，未来多模态统一模型可能融合两类思想。

Text-to-Image

Text-to-Image 是根据文本 prompt 生成图像。

好的 Text-to-Image 模型需要同时满足四类要求：

要求	含义
Prompt Alignment	是否符合文本描述
Visual Quality	是否清晰、美观、无明显 artifacts
Compositionality	是否理解多个对象、属性和关系
Controllability	是否能稳定控制风格、构图、主体和细节

常见 prompt 组成包括主体、属性、动作、场景、风格、镜头、光照、构图、质量词和负向约束。

常见失败模式：多个实体关系混乱；数量不准；左右位置错；文字生成错误；手指异常；小物体缺失；风格压过语义；过度依赖训练集常见构图。

工程建议：评估 Text-to-Image 不要只看随机好图，要用固定 prompt 集覆盖人物、物体、文字、空间关系、计数、长 prompt、多语言和安全边界。

面试表达：Text-to-Image 的难点不是生成漂亮图片，而是在复杂 prompt 下同时保证语义、结构、细节和可控性。

Image Editing

Image Editing 是根据原图和指令修改图像。

常见形式包括 inpainting、outpainting、局部重绘、风格迁移、主体替换、背景替换、姿态控制、图像增强和参考图编辑。

和纯 Text-to-Image 相比，图像编辑更强调保留未修改区域、保持主体身份一致、精准遵循编辑指令，并避免全图漂移。

关键技术：

技术	作用
Mask conditioning	指定修改区域
img2img strength	控制保留原图和重绘程度
ControlNet	用结构条件控制边缘、深度、姿态
Instruct editing data	学会自然语言编辑指令
Reference adapter	保持人物、物体或风格一致

面试表达：图像编辑比文生图更考验条件控制。模型既要改该改的地方，又不能破坏不该改的地方。

Video Diffusion

Video Diffusion 是将扩散模型扩展到视频生成的路线。

视频可以看作一组连续帧，因此模型需要同时建模空间内容和时间变化。最直接的方法是把视频表示为 frames x height x width 的时空张量，然后在这个张量上加噪和去噪。

视频生成比图像生成更难，原因包括：

难点	说明
计算量大	多帧带来更高 token 数和显存成本
时序一致	主体外观不能帧间闪烁
运动合理	动作要自然，不能突然跳变
物理一致	物体运动、遮挡、光影要符合直觉
长程依赖	长视频需要保持场景、角色和事件连续
数据成本	高质量视频文本对更难清洗和标注

常见做法：在图像扩散模型基础上加入 temporal attention、3D convolution、时序模块、帧间 latent 建模或先生成关键帧再插帧。

面试表达：视频扩散不是简单逐帧生成图片。逐帧生成会导致闪烁和身份漂移，必须显式建模时间维度和运动先验。

Video Token

Video Token 是视频在模型中的离散或连续表示单元。

如果使用自回归路线，视频可以通过 video tokenizer 压缩成离散 token。模型像生成文本一样生成视频 token，再由 decoder 还原为视频。

如果使用扩散路线，视频 token 可以是 latent patch、spatiotemporal patch 或连续特征。模型在这些 token 上进行去噪或预测。

视频 token 设计的关键是压缩率与信息保真之间的平衡。压缩太弱，token 太多，训练和推理成本爆炸；压缩太强，运动细节、人物身份、文字和小物体容易丢失。

面试表达：视频生成模型的效率很大程度取决于视频表示。好的 video tokenizer 或 latent 表示要在时空压缩和细节保真之间取得平衡。

Spatiotemporal Patch

Spatiotemporal Patch 是把视频切成同时覆盖空间和时间的小块。

图像 ViT 把图片切成二维 patch。视频模型可以把连续帧切成三维 patch，例如覆盖若干帧、若干高度和若干宽度。每个 patch 被映射成一个 token，供 Transformer 或扩散 backbone 处理。

优势：能统一建模空间纹理和时间变化，适合 Transformer 化视频生成模型。

挑战：视频 patch 数量巨大。假设 16 帧、每帧 32 x 32 个空间 patch，如果不做时间压缩，就有 16384 个 patch token，成本非常高。

常见优化包括时空下采样、局部 attention、factorized attention、窗口 attention、latent compression 和分层生成。

面试表达：spatiotemporal patch 是把视频看成时空 token 序列。它让 Transformer 能处理视频，但也带来长序列计算瓶颈。

Temporal Consistency

Temporal Consistency，时序一致性，是视频生成质量的核心指标。

它要求同一个主体在不同帧中保持身份、外观、颜色、位置变化和光照合理连续。比如人物的脸不能一帧一个样，衣服颜色不能闪烁，背景不能无缘无故变形。

常见时序问题：

问题	表现
Flickering	帧间闪烁，纹理不稳定
Identity Drift	人物或物体身份逐渐变掉
Shape Deformation	形状扭曲，手脚异常
Motion Jitter	运动抖动，不平滑
Scene Inconsistency	背景和光照突然变化
Object Permanence Failure	被遮挡后物体消失或变形

缓解方法：使用视频级训练数据；引入 temporal attention；加入光流或运动约束；用关键帧控制；使用更强 latent 视频表示；训练时加入时序一致性损失或偏好数据。

面试表达：视频生成的核心挑战是时间维度。单帧好看不代表视频好，时序一致性是区分图像模型和视频模型的重要标准。

Motion Prior

Motion Prior 是模型对运动规律的先验。

视频不是随机帧序列，而包含动作、速度、加速度、遮挡、相机运动和物理约束。模型需要学习人如何走路、车如何移动、水如何流动、镜头如何推进等规律。

运动先验来源：大规模视频数据、动作标注数据、光流、姿态序列、物理模拟数据、相机轨迹数据和人类偏好反馈。

常见控制方式：文本描述动作、给定起止帧、给定姿态骨架、给定相机轨迹、给定深度或边缘序列、给定参考视频。

面试表达：视频生成不仅是空间生成，还要建模运动。运动先验不足会导致动作不自然、速度不稳定和物理违背。

Text-to-Video

Text-to-Video 是根据文本 prompt 生成视频。

相比 Text-to-Image，Text-to-Video 的 prompt 不仅描述画面，还描述动作、时间变化和镜头语言。例如 “一个穿红衣服的人从左向右跑过雨中的街道，镜头缓慢跟随”。

评估维度：

维度	关注点
文本对齐	场景、主体、动作是否符合 prompt
单帧质量	每帧是否清晰自然
时序一致	主体和背景是否稳定
运动质量	动作是否自然、速度是否合理
镜头控制	推拉摇移、视角变化是否符合描述
物理合理	遮挡、碰撞、重力和光影是否合理
长程连贯	长视频事件是否持续一致

常见失败模式：开头符合 prompt，后面漂移；主体变形；运动停滞；相机和物体运动混乱；文字和手部错误；复杂多主体交互失败。

面试表达：Text-to-Video 的难点是把静态语义扩展到动态事件。模型必须同时理解 “有什么” 和 “怎么变化”。

World Model

World Model 在生成视频语境中指模型对世界状态变化和因果规律的内部建模能力。

如果模型只是拟合像素分布，它可能生成看起来合理但物理不一致的视频。更强的 world model 应该理解物体持续存在、遮挡后仍然存在、动作有因果顺序、物体运动受物理约束、角色目标和环境交互会影响后续状态。

World Model 与视频生成的关系：高质量视频生成需要一定世界建模能力，因为视频天然包含时间、空间、因果和物理。尤其在机器人、自动驾驶、游戏、仿真和交互式 agent 中，视频生成不只是内容创作，也可能用于预测未来状态。

需要注意：当前很多视频生成模型能生成逼真片段，但不等于具备可靠可控的世界模型。面试中要避免把“视频很真实”直接等同于“理解物理世界”。

面试表达：World Model 强调模型是否能预测世界状态如何随动作和时间演化。视频生成是观察这种能力的窗口，但逼真生成不必然代表可靠推理。

Sora

Sora 是 OpenAI 展示的文本到视频生成模型，代表了长时长、高质量视频生成的重要进展。

公开信息中，Sora 被描述为可以生成较长视频，并能处理复杂场景、相机运动、多个主体和一定程度的物理一致性。它常被讨论为“视频生成模型是否正在学习世界模型”的代表案例。

理解 Sora 时应关注几个关键词：

关键词	含义
Video as patches	将视频表示为时空 patch 或 token
Scaling	利用大规模数据和模型扩展提升生成能力
Long horizon	生成更长时间范围的视频
Physical consistency	更好保持物体、运动和场景连续
General simulator	朝通用视觉世界模拟器方向探索

面试中不需要编造未公开细节。更稳妥的回答是：Sora 体现了视频生成从短片段、低一致性向长时长、高一致性和世界模拟方向发展，但真实架构、数据和训练细节以公开资料为准。

面试表达：Sora 的意义不只是生成视频更清晰，而是显示了大规模视频生成模型在时空一致性、复杂运动和场景持续性上的潜力。

图像生成评估

图像生成评估不能只看“好不好看”。

常见评估维度：

维度	说明
文本对齐	是否符合 prompt 中主体、属性、关系和风格
视觉质量	清晰度、纹理、光照、构图和美感
多样性	同一 prompt 是否能生成不同合理结果
真实性	是否接近真实图像或目标风格
结构正确	手、脸、物体形状和空间关系是否正确
文字能力	图中文字是否正确可读
安全性	是否生成违规、侵权或敏感内容
可控性	是否能稳定遵循参考图、mask、姿态或布局

常见自动指标包括 FID、IS、CLIPScore、美学评分和人类偏好模型评分。但自动指标存在局限：FID 不一定反映 prompt 对齐；CLIPScore 可能偏语义而忽略细节；美学评分可能鼓励“漂亮但不准确”的图片。

实际评估建议：建立 prompt suite，覆盖简单主体、复杂组合、空间关系、计数、文字、人物、风格、禁生成边界和业务场景。人工评测时分项打分，不要只给总分。

面试表达：图像生成评估要拆成本对齐、视觉质量、结构正确性和安全可控性。公开指标只能辅助，不能替代场景化人工评测。

视频生成评估

视频生成评估比图像更复杂，因为每帧质量和帧间关系都重要。

常见维度：

维度	说明
Prompt Alignment	视频内容和动作是否符合文本
Frame Quality	单帧清晰度和美观度
Temporal Consistency	主体、背景、光照是否稳定
Motion Naturalness	运动是否平滑自然
Identity Preservation	人物和物体是否保持一致
Physical Plausibility	是否符合基本物理和因果
Long-range Coherence	长视频事件是否连贯
Safety	是否生成有害或侵权内容

自动评估很难完全可靠。视频可能单帧好看但运动异常，也可能运动合理但 prompt 对齐差。真实项目中通常需要人工评测、偏好比较和任务化评估结合。

面试表达：视频生成评估必须同时看空间质量和时间质量。只抽帧看图是不够的，因为模型最大的问题往往出现在帧间一致性和运动逻辑上。

安全与版权

图像和视频生成涉及明显的安全、版权和合规问题。

常见风险：生成虚假新闻图、深度伪造、人脸冒用、版权风格模仿、商标滥用、色情暴力内容、未成年人风险、医疗金融误导、政治敏感内容等。

治理方式：训练数据过滤；prompt 和输出审核；水印和 provenance；人脸与名人保护；版权数据管理；拒答策略；使用政策分级；企业场景保留日志和追溯能力。

面试表达：生成模型落地不能只追求质量，还要考虑滥用风险。尤其是视频生成，真实感越强，伪造和误导风险越高。

工程取舍

图像和视频生成落地常见取舍如下：

取舍	选择 A	选择 B	判断标准
模型路线	扩散模型	自回归生成	质量、速度、统一建模需求
生成空间	像素空间	latent 空间	保真度 vs 成本
Backbone	U-Net	Transformer/DiT	成熟稳定 vs scaling 能力
采样步数	多步	少步/蒸馏	质量 vs 延迟
控制方式	prompt	ControlNet/参考图/布局	简单性 vs 精确控制
个性化	全量微调	LoRA/Adapter	效果 vs 训练成本
视频生成	逐帧	时序建模	简单性 vs 一致性
评估	公开指标	场景人工评测	横向比较 vs 真实可用性

常见优化方向：使用更快 scheduler、减少采样步数、模型蒸馏、量化、缓存文本 embedding、批量推理、使用低秩适配、分辨率分阶段生成、先低清后超分、视频关键帧加插帧。

面试表达：生成模型工程中最常见的 trade-off 是质量、速度、成本和可控性。图像任务可以用扩散生态快速落地，视频任务还要额外处理时序一致性和算力成本。

高频面试题

1. 扩散模型和 GAN 有什么区别？

GAN 通过生成器和判别器对抗训练，生成速度快，但训练可能不稳定，容易模式崩塌。扩散模型通过加噪和去噪学习数据分布，训练更稳定，生成质量高，但通常需要多步采样，推理较慢。

一句话：GAN 是对抗生成，扩散模型是逐步去噪生成。

2. Stable Diffusion 为什么要在 latent 空间生成？

因为像素空间维度太高，直接扩散成本大。Stable Diffusion 用 VAE 把图片压缩到 latent 空间，在低维 latent 上去噪，最后再解码成图片。这显著降低训练和推理成本。

代价是 VAE 压缩可能损失细节，尤其是文字、小物体和复杂纹理。

3. CFG 的作用是什么？

CFG 用于增强条件遵循。采样时同时计算有条件和无条件预测，并放大二者差异，让生成结果更贴近 prompt。

CFG scale 太低会导致 prompt 遵循弱，太高可能导致过饱和、细节异常和多样性下降。

4. U-Net 和 DiT 有什么区别？

U-Net 是卷积和多尺度结构为主的去噪网络，适合图像局部细节建模，Stable Diffusion 早期常用。DiT 是 Diffusion Transformer，把 latent patch 当 token 处理，更符合 Transformer scaling 路线。

一句话：U-Net 成熟稳定，DiT 更适合大规模 Transformer 化扩展。

5. 视频生成为什么比图像生成难？

视频不仅要求每帧好看，还要求帧间一致、动作自然、主体身份稳定、物理和镜头逻辑合理。多帧带来更大计算量和更长依赖，训练数据也更难清洗和标注。

6. 什么是时序一致性？

时序一致性是指视频中主体、背景、光照、纹理和运动在不同帧之间保持合理连续。常见失败包括闪烁、身份漂移、形状变形和运动抖动。

7. Sora 的意义是什么？

Sora 展示了大规模视频生成模型在长时长、高质量、复杂运动和场景一致性上的潜力。它也推动了关于视频生成模型是否具备世界模拟能力的讨论。

稳妥回答是：Sora 代表视频生成向通用视觉世界模拟方向迈进，但不能简单把逼真视频等同于可靠世界模型。

8. 如何评估一个文生图模型？

要分项评估文本对齐、视觉质量、多样性、结构正确性、文字能力、安全性和可控性。自动指标可以参考，但必须配合固定 prompt suite 和人工评测。

9. 如何评估一个文生视频模型？

除了 prompt 对齐和单帧质量，还要评估时序一致性、运动自然度、身份保持、物理合理性和长程连贯性。只抽帧看图不够，因为视频模型的关键问题常出现在时间维度。

本章小结

图像生成和视频生成是生成式 AI 的核心方向。扩散模型通过正向加噪和反向去噪实现高质量生成，Stable Diffusion 用 latent diffusion 降低成本，CFG 增强条件遵循，自回归图像生成则提供了与语言模型统一的 token 建模思路。

视频生成在图像生成基础上增加了时间维度，需要解决时序一致性、运动先验、长程连贯和物理合理性。Sora 等模型展示了大规模视频生成的潜力，也让 world model 成为高频讨论概念。

面试最终表达：图像生成看语义、质量和控制；视频生成还要看时间、运动和世界一致性。工程落地时，质量、速度、成本、安全和版权必须一起考虑。

R. 语音大模型

条目：ASR、Whisper、TTS、Voice Cloning、Speech Token、Audio Codec、Mel Spectrogram、Speech Encoder、Speech-to-Text、Text-to-Speech、Speech-to-Speech、End-to-End Voice Model、Streaming ASR、Speaker Embedding、Prosody、Multimodal Voice Agent。

本章速览

一句话定义：语音大模型是能够处理语音输入、生成语音输出，并与文本、图像、视频、工具调用等能力结合的多模态模型。

传统语音系统通常由多个模块串联：ASR 把语音转文字，LLM 理解和生成文本，TTS 把文本转语音。新一代语音大模型则尝试把语音作为一种原生模态，直接进行语音理解、语音对话、语音到语音生成和实时交互。

核心问题：语音不是简单的“带声音的文本”。语音包含文字内容、说话人身份、语速、语调、情绪、停顿、重音、环境噪声和交互时延。好的语音模型既要识别“说了什么”，也要理解“怎么说的”，并生成自然、稳定、低延迟的语音回复。

典型任务：ASR、TTS、语音翻译、语音克隆、语音情感识别、语音到语音对话、会议纪要、实时字幕、客服质检、语音 Agent 和端到端实时语音助手。

面试表达：语音大模型的关键不只是 ASR 准确率，而是语音 token 化、时序建模、韵律控制、低延迟流式交互、安全合规和端到端多模态协同。

ASR

ASR 是 Automatic Speech Recognition，即自动语音识别，目标是把语音信号转成文本。

ASR 输入通常是一段音频波形，输出是文字序列。它是语音交互系统最基础的入口能力。例如语音助手、会议转写、字幕生成、电话客服和语音搜索都依赖 ASR。

ASR 系统关注的能力包括：

能力	说明
准确识别	识别普通话、英文、方言、混合语言和专业术语
鲁棒性	抗噪声、混响、远场、多人说话和口音
标点和格式	自动断句、加标点、数字和单位规范化
时间戳	输出词级或句级时间位置
流式识别	边说边出结果，降低延迟
说话人区分	区分不同说话人，支持会议场景

常见评估指标是 WER 和 CER。WER 是 word error rate，常用于英文；CER 是 character error rate，常用于中文。它们统计替换、插入和删除错误。

$$WER = (\text{Substitutions} + \text{Insertions} + \text{Deletions}) / \text{Number of reference words}$$

ASR 常见错误：同音词错误、专有名词错误、数字单位错误、漏识别、小语种混淆、多人重叠语音错误、长音频上下文断裂。

面试表达：ASR 的核心是从连续声学信号恢复离散文本。工程上不能只看干净测试集 WER，还要看噪声、口音、实时性、热词和业务术语表现。

Whisper

Whisper 是 OpenAI 提出的通用语音识别模型，代表了大规模弱监督语音预训练路线。

Whisper 的输入通常是音频的 log-Mel spectrogram，模型使用 encoder-decoder Transformer 结构。encoder 编码音频特征，decoder 自回归生成文本 token，并可通过特殊 token 控制语言、任务和时间戳等。

Whisper 能力覆盖 ASR、语音翻译、语言识别和时间戳预测。它在多语言、噪声鲁棒性和开源生态上影响很大。

Whisper 的优点：通用性强、多语言能力好、鲁棒性较好、部署和二次开发生态丰富。

Whisper 的局限：实时低延迟不是原始设计重点；专业术语和强业务场景仍可能出错；长音频需要切分和上下文管理；对极端噪声、多说话人重叠和方言仍有挑战。

面试表达：Whisper 的关键价值是用大规模多任务语音数据训练一个通用 encoder-decoder ASR 模型，不只做单一语言识别，也支持翻译和时间戳等任务。

Mel Spectrogram

Mel Spectrogram 是语音模型常用的声学特征表示。

原始音频是随时间变化的波形，采样率可能是 16kHz、24kHz 或 48kHz。直接处理长波形成本高，也不符合人耳感知。Mel Spectrogram 先通过短时傅里叶变换得到时频表示，再把频率轴映射到 Mel 标度，更接近人耳对频率的感知。

直观理解：Mel Spectrogram 是一张二维图，横轴是时间，纵轴是频率，数值表示能量强弱。很多语音模型把它当作类似图像的输入特征。

常见处理流程：

waveform -> framing/windowing -> STFT -> magnitude spectrum -> Mel filterbank -> log Mel spectrogram

关键参数包括采样率、窗口长度、hop size、Mel bins 数量和归一化方式。这些参数会影响时间分辨率、频率分辨率和模型效果。

面试表达：Mel Spectrogram 是把一维音频波形转成时频特征，让模型更容易学习语音中的频谱结构。ASR、TTS 和语音分类中都很常见。

Speech Encoder

Speech Encoder 是把音频特征编码成高层语音表示的模块。

输入可以是 waveform、Mel Spectrogram 或 codec token。输出可以用于 ASR、说话人识别、情感识别、语音翻译或接入 LLM。

常见 speech encoder 架构包括 CNN、RNN、Conformer、Transformer、Wav2Vec 2.0、HuBERT、Whisper encoder 等。

不同 encoder 的侧重点：

Encoder	特点
CNN	局部特征提取强，常用于前端下采样
RNN/LSTM	适合时序建模，早期常用
Transformer	长程依赖强，可并行训练
Conformer	结合卷积局部建模和 Transformer 全局建模
Wav2Vec/HuBERT	自监督语音表示学习
Whisper Encoder	大规模弱监督 ASR 语音编码

工程中常见做法是先用 speech encoder 提取语音表示，再接 CTC head、seq2seq decoder、LLM adapter 或分类头。

面试表达：speech encoder 的作用类似视觉模型中的 image encoder，是把连续语音信号变成模型可理解的高层表示。

Speech Token

Speech Token 是语音在模型中使用的离散或连续表示单元。

为什么需要 speech token：LLM 天然处理离散文本 token。如果希望把语音接入 LLM，或者让模型生成语音，就需要把连续音频压缩成可建模的 token 或 latent 表示。

Speech token 可以分为几类：

类型	含义	用途
Semantic Token	表示语音内容语义	语音理解、语音到文本
Acoustic Token	表示音色、韵律、声学细节	TTS、语音生成
Codec Token	由音频 codec 量化得到	高保真语音重建
Continuous Feature	连续语音向量	接 adapter 或 cross-attention

语音 token 化的难点是同时保留内容、说话人、情绪和音质。如果 token 只保留语义，生成语音可能缺少自然韵律；如果 token 保留太多声学细节，序列会很长，建模成本高。

面试表达：Speech token 是语音大模型的桥梁。它决定语音能否像文本一样被大模型理解和生成，也决定延迟、音质和可控性。

Audio Codec

Audio Codec 是把音频压缩成离散码或低比特率表示，再从这些表示重建音频的模型或算法。

传统 codec 如 MP3、AAC 更偏工程压缩。神经音频 codec 如 EnCodec、SoundStream、HiFi-Codec 等通过神经网络学习压缩和重建，常用于语音生成模型。

神经 codec 通常包含 encoder、quantizer 和 decoder：

waveform -> codec encoder -> continuous latent -> quantizer -> discrete codes
discrete codes -> codec decoder -> reconstructed waveform

Codec token 的价值：把高采样率连续波形变成较低帧率的离散 token，使语言模型或生成模型可以预测音频。

关键指标：重建音质、码率、延迟、token 速率、说话人保持、噪声鲁棒性和音乐/语音泛化能力。

面试表达：Audio codec 在语音大模型中类似视觉生成里的 tokenizer/VAE。它把连续音频压缩成模型可处理的表示，并决定生成语音的音质上限。

Speech-to-Text

Speech-to-Text 是语音到文本任务，ASR 是最典型形式。

在大模型系统中，Speech-to-Text 不只是转写，还可能包括标点恢复、说话人分离、摘要、翻译、意图识别和结构化抽取。

常见 pipeline:

audio -> VAD -> ASR -> punctuation -> diarization -> LLM post-processing -> structured output

其中 VAD 是 voice activity detection，用来检测哪些片段有人声；diarization 用来区分 “谁在说话”；LLM post-processing 可以做纠错、摘要、会议纪要和任务抽取。

工程注意：LLM 后处理可以修正格式和上下文，但不能盲目改写事实。对于法律、医疗、金融和会议纪要场景，必须保留原始转写证据和时间戳。

面试表达：Speech-to-Text 在系统中通常不是单模型问题，而是语音前处理、识别、说话人分析和文本智能处理的组合。

TTS

TTS 是 Text-to-Speech，即文本转语音，目标是把文本生成自然语音。

TTS 系统要解决两件事：说什么，以及怎么说。文本内容决定发音序列，韵律、音色、情绪和语速决定听感。

传统 TTS pipeline 通常包括文本前端、声学模型和 vocoder:

text -> text normalization -> phoneme/g2p -> acoustic model -> Mel spectrogram -> vocoder -> waveform

现代神经 TTS 可以是 Tacotron、FastSpeech、VITS、扩散式 TTS、codec language model 或端到端语音生成模型。

评估维度:

维度	说明
自然度	是否像真人说话
可懂度	内容是否清晰正确
韵律	停顿、重音、语调是否合理
音色	是否符合目标说话人
稳定性	长文本是否漏字、重复、崩坏
延迟	是否能实时生成

常见指标包括 MOS 主观评分、字错率、说话人相似度和实时率 RTF。

面试表达：TTS 不只是把文字读出来，而是要控制发音、韵律、音色和情绪。自然度和稳定性通常比单个自动指标更重要。

Text-to-Speech

Text-to-Speech 与 TTS 含义相同，在大模型语境中更强调文本条件驱动的语音生成。

输入可以是普通文本，也可以包含控制标签，例如说话人、语言、情绪、语速、风格、停顿和角色信息。

"用温柔、稍慢的语速读：欢迎来到今天的课程。"

现代 Text-to-Speech 越来越接近可控生成任务。模型不仅要正确读文本，还要按指令控制情绪、语气和风格。

常见挑战：多音字、英文缩写、数字日期、专业术语、长文本断句、跨语言发音、情绪一致性和低延迟流式输出。

面试表达：Text-to-Speech 的难点从 “可懂” 逐渐转向 “自然、可控、低延迟和个性化”。

Voice Cloning

Voice Cloning 是语音克隆，目标是根据少量参考音频生成相同或相似音色的语音。

常见形式：给定一段说话人参考音频和待合成文本，模型生成具有该说话人音色的语音。

关键技术包括 speaker embedding、prompt speech conditioning、speaker encoder、codec token 生成和声学风格迁移。

优点：可以快速个性化语音，用于配音、有声书、虚拟人、辅助沟通和多语言声音迁移。

风险：可能被用于冒充他人、诈骗、伪造证据和侵犯声音权益。因此语音克隆必须强调授权、检测、水印和使用限制。

面试表达：Voice cloning 的技术核心是从参考音频中提取说话人特征，并在生成时保持该音色。但它是高风险能力，必须和安全治理一起讨论。

Speaker Embedding

Speaker Embedding 是表示说话人身份特征的向量。

它通常由 speaker encoder 从参考音频中提取，用于说话人识别、语音克隆、多说话人 TTS 和说话人分离。

Speaker embedding 应该捕捉相对稳定的音色特征，而不是具体文本内容。理想情况下，同一个人的不同语音片段 embedding 接近，不同人的 embedding 远离。

应用场景：

场景	用法
多说话人 TTS	控制合成声音属于哪个人
Voice Cloning	从少量参考音频提取音色
Speaker Verification	判断两段语音是否同一人
Diarization	会议中区分不同说话人

面试表达：Speaker embedding 是语音中的身份表示，但它不应泄露过多文本内容。实际系统还要考虑隐私保护和防冒用。

Prosody

Prosody 是韵律，包含语速、音高、重音、停顿、节奏、语调和情绪表达。

同一句话用不同韵律表达，含义和听感可能完全不同。例如“你真厉害”可以是真诚夸奖，也可以是讽刺。文本本身不总能决定韵律，因此 TTS 和语音对话需要建模韵律。

韵律控制方式：

控制	示例
显式标签	happy、sad、angry、slow、fast
标点和 SSML	停顿、重读、音量、语速
参考音频	模仿参考语音的节奏和情绪
隐式学习	从大规模语音数据中学习自然韵律

常见问题：情绪过度夸张、长文本韵律变平、停顿位置错误、疑问句语调不自然、跨语言语调迁移失败。

面试表达：Prosody 决定语音是否像真人。ASR 主要关心内容，TTS 和语音 Agent 必须关心韵律，因为它直接影响交互体验。

Speech-to-Speech

Speech-to-Speech 是语音输入到语音输出的任务。

它可以是语音翻译，例如中文语音输入，英文语音输出；也可以是语音对话，例如用户说话，助手直接用语音回答；还可以是语音风格转换或声音转换。

两种路线：

路线	流程	优点	缺点
级联系统	ASR -> LLM -> TTS	模块清晰，易调试	误差累积，延迟较高，丢失语气
端到端系统	speech -> model -> speech	可保留语气和实时交互	训练复杂，安全和可控性难

端到端 Speech-to-Speech 的优势是可能保留语音中的情绪、语调和非文本信息，减少中间转文本的信息损失。

面试表达：Speech-to-Speech 的核心挑战是既理解语音内容，又保留非文本信息，并以低延迟生成自然语音。

End-to-End Voice Model

End-to-End Voice Model 是端到端语音模型，目标是直接从语音输入生成语音或文本输出，减少传统 ASR-LLM-TTS 级联。

端到端语音模型常见实现方式：

方式	说明
Speech encoder + LLM	语音编码后接入 LLM，输出文本或语音 token
Codec LM	把语音转成 codec token，用语言模型生成音频 token
Unified multimodal model	文本 token、语音 token 和其他模态 token 统一建模
Duplex voice model	支持边听边说，模拟实时对话

端到端模型的价值：降低交互延迟，保留语音情绪和语气，减少级联误差，使对话更自然。

挑战：训练数据复杂；语音 token 序列长；生成稳定性难；安全审核更难；可解释性和调试不如模块化 pipeline；实时打断和双工交互要求高。

面试表达：端到端语音模型是趋势，但不一定总是替代级联系统。生产环境要在质量、延迟、可控性、调试和安全之间取舍。

Streaming ASR

Streaming ASR 是流式语音识别，要求用户边说，系统边输出识别结果。

它不同于离线 ASR。离线 ASR 可以看到完整音频后再识别，准确率通常更高；流式 ASR 必须在只看到当前和少量历史上下文的情况下做预测。

核心指标：

指标	含义
Latency	从说话到出字的延迟
Partial Result Stability	中间结果是否频繁改动
Final Accuracy	最终识别准确率
Endpointing	判断用户是否说完
Real-time Factor	处理速度是否快于音频播放速度

常见技术：chunk-based encoder、causal attention、RNN-T、CTC streaming、增量解码、VAD、endpoint detection、热词注入。

流式识别的典型矛盾：越等更多上下文，准确率越高，但延迟越大；越早输出，体验越实时，但中间结果更容易修改。

面试表达：Streaming ASR 的核心 trade-off 是准确率和延迟。语音助手场景中，低延迟和稳定中间结果非常重要。

Multimodal Voice Agent

Multimodal Voice Agent 是能通过语音与用户实时交互，并调用文本、视觉、工具和记忆能力的智能体。

它不仅能听和说，还能理解上下文、看屏幕或摄像头、调用 API、执行任务，并用自然语音反馈。

典型架构：

microphone -> streaming ASR or speech encoder -> dialogue model / agent
agent -> tool use / memory / vision context -> response planning
response text or speech tokens -> TTS or speech decoder -> speaker

关键能力：

能力	说明
实时收听	低延迟接收语音输入
打断处理	用户插话时停止或调整回复
上下文记忆	保持多轮对话状态
工具调用	查询、下单、控制设备、执行任务
多模态感知	结合屏幕、图像、视频或环境信息
自然表达	用合适语气和韵律回答
安全控制	防止越权、诈骗、隐私泄露和声音冒用

工程难点：端到端延迟、噪声环境、唤醒词、VAD、barge-in 打断、TTS 流式合成、回声消除、权限确认、日志审计和异常恢复。

面试表达：语音 Agent 不是简单 ASR 加 TTS，而是一个实时交互系统。最难的是低延迟、多轮上下文、打断处理和可控的任务执行。

语音大模型评估

语音模型评估需要按任务拆分。

ASR 评估：WER、CER、专有名词准确率、标点准确率、时间戳误差、流式延迟、噪声鲁棒性、口音和方言表现。

TTS 评估：MOS、自然度、可懂度、说话人相似度、韵律自然性、长文本稳定性、实时率、音频伪影。

Speech-to-Speech 评估：内容正确性、语义保持、说话人保持、情绪保持、延迟、自然度、翻译质量和对话连贯性。

Voice Agent 评估：端到端响应时间、打断成功率、任务完成率、多轮一致性、误触发率、工具调用正确率和安全拒答。

常见问题是只用单一指标评价复杂系统。例如 ASR 的 WER 很低，不代表语音助手体验好；TTS 的 MOS 高，也不代表对话时延和安全可用。

面试表达：语音系统评估必须任务化。识别、合成、对话和 Agent 的指标不同，端到端体验往往比单模块指标更关键。

安全与合规

语音大模型有特殊安全风险。

常见风险：声音克隆诈骗、未经授权复刻声音、伪造录音证据、声纹隐私泄露、诱导用户泄露敏感信息、儿童语音数据合规、电话机器人骚扰、语音 deepfake。

治理方式：

方式	说明
授权机制	克隆声音前验证本人同意
水印	对合成语音加入可检测标记
Deepfake 检测	检测合成或伪造语音
敏感操作确认	金融、支付、账号操作需要额外验证
数据脱敏	保护录音中的个人信息
日志审计	记录关键操作链路
使用限制	禁止冒充、诈骗和违法用途

面试表达：语音能力越接近真人，滥用风险越高。语音克隆和实时语音 Agent 必须把授权、检测、水印和权限控制作为系统设计的一部分。

工程取舍

语音大模型落地常见取舍如下：

取舍	选择 A	选择 B	判断标准
系统架构	ASR-LLM-TTS 级联	端到端语音模型	可控调试 vs 自然低延迟
ASR 模式	离线识别	流式识别	准确率 vs 实时性
语音表示	Mel 特征	codec token	识别任务 vs 生成任务
TTS 生成	非流式	流式	音质稳定 vs 交互延迟
声音个性化	固定音色	克隆音色	安全性 vs 个性化体验
部署位置	云端	端侧	算力质量 vs 隐私延迟
后处理	LLM 纠错	原始输出	可读性 vs 忠实性
Agent 执行	自动执行	二次确认	效率 vs 安全

工程建议：如果业务重视准确记录，例如会议和法律转写，优先保证 ASR 忠实性和时间戳；如果业务重视实时交互，例如语音助手，优先优化端到端延迟、打断和流式 TTS；如果涉及声音克隆，必须先设计授权和风控。

面试表达：语音系统没有单一最优架构。生产选择取决于准确率、延迟、自然度、隐私、安全和可维护性。

高频面试题

1. ASR 和 TTS 分别解决什么问题？

ASR 把语音转成文本，解决“听懂用户说了什么”。TTS 把文本转成语音，解决“用声音说出来”。语音助手通常需要 ASR、LLM 和 TTS 协同。

2. Whisper 的核心特点是什么？

Whisper 是基于大规模弱监督数据训练的 encoder-decoder Transformer，输入 Mel Spectrogram，输出文本 token。它支持多语言 ASR、语音翻译、语言识别和时间戳，鲁棒性较好。

3. Mel Spectrogram 为什么常用于语音模型？

它把一维波形转换成时频表示，并用 Mel 标度模拟人耳频率感知。相比原始波形，它更紧凑、更适合模型学习语音频谱结构。

4. 语音 token 有什么作用？

语音 token 把连续音频压缩成模型可处理的离散或连续表示，使 LLM 或生成模型可以理解和生成语音。它是语音接入大模型的重要桥梁。

5. Audio codec 在语音大模型中为什么重要？

神经 audio codec 可以把音频压缩成 codec token，并从 token 重建波形。它类似图像生成中的 VAE 或 tokenizer，决定生成语音的压缩效率和音质上限。

6. 级联语音系统和端到端语音模型怎么选？

级联系统 ASR-LLM-TTS 模块清晰、易调试、易做安全控制，但延迟较高，可能丢失语气和情绪。端到端模型更自然、潜在延迟更低，但训练复杂、可控性和安全审核更难。

工程上要根据准确率、延迟、可解释性、数据和安全要求选择。

7. Streaming ASR 的难点是什么？

难点是准确率和延迟的平衡。系统越早输出，延迟越低，但上下文不足容易识别错；等待更多音频可以提升准确率，但交互体验变差。中间结果稳定性和 endpointing 也很关键。

8. Voice cloning 有哪些风险？

主要风险是冒充他人、诈骗、伪造证据和侵犯声音权益。必须有授权机制、水印、合成语音检测、敏感操作二次验证和使用限制。

9. Prosody 为什么重要？

Prosody 决定语音的语速、音高、重音、停顿和情绪。同一句文字用不同韵律会产生不同含义和体验。TTS 和语音 Agent 要自然，就必须建模韵律。

10. 如何评估语音 Agent？

要看端到端响应时间、识别准确率、TTS 自然度、打断处理、多轮上下文、工具调用成功率、任务完成率和安全拒答。不能只看 ASR WER 或 TTS MOS。

本章小结

语音大模型覆盖从听到说的完整链路。ASR 负责语音转文本，Whisper 是通用 ASR 代表；Mel Spectrogram 和 speech encoder 是常见语音输入表示；Speech token 和 audio codec 让语音能够被大模型统一建模；TTS、voice cloning、prosody 和 speaker embedding 决定语音生成的自然度和个性化。

新一代语音系统正在从 ASR-LLM-TTS 级联走向端到端语音模型和多模态语音 Agent。这个趋势带来更自然、更低延迟的交互，也带来更复杂的安全、隐私和合规问题。

面试最终表达：语音大模型不是“把文本模型接上麦克风和喇叭”这么简单。它要同时解决连续信号建模、语音 token 化、实时交互、韵律生成、声音身份、安全风控和多模态 Agent 协同。